

```

1  ; *****
2  ; IBMDOS7.S (PCDOS 7.1 Kernel) - RETRO DOS v5.0 by ERDOGAN TAN - 01/01/2024
3  ; -----
4  ; Last Update: 04/02/2026 - Retro DOS v5.0 (Modified PCDOS 7.1)
5  ; -----
6  ; Beginning: 22/04/2019 (Retro DOS 4.0), 03/11/2022 (Retro DOS 4.2)
7  ; -----
8  ; Assembler: NASM version 2.15
9  ; -----
10 ; ((nasm ibmdos7.s -l ibmdos7.txt -o IBMDOS.COM -Z error.txt))
11 ; -o IBMDOS7.BIN
12 ; *****
13 ; main file: 'retrodos5.s'
14 ; incbin 'IBMDOS7.BIN'
15 ; =====
16 ; Modified from 'msdos6.s' (modified MSDOS 6.21 kernel src as Retro DOS v4.2)
17 ; 29/09/2023 /// Retro DOS v4.2 (2023) -> Modified MSDOS 6.22 IO.SYS+MSDOS.SYS
18 ; =====
19 ;
20 ; 30/12/2022 - Retro DOS v4.2 Kernel ('msdos6.s')
21 ; Modified from 'msdos5.s' (29/12/2022, Retro DOS v4.1 Kernel) file
22 ; as below:
23 ; 1) MS-DOS version has been changed to 6.22 (It was 5.0)
24 ; 2) Retro DOS version has been changed to 4.2 (It was 4.1)
25 ; (The content has not been changed except kernel version because the kernel
26 ; code is already compatible with MSDOS 6.x and it is optimized before.)
27 ; (But IO.SYS part of the kernel is not same with Retro DOS v4.1 code.)
28 ;
29 ; -----
30 ;
31 ; 03/11/2022 - Erdogan Tan (Istanbul)
32 ;
33 ; Note: This code is a part of Retro DOS 4.0 kernel source code
34 ; (as included binary, 'MSDOS5.BIN')
35 ; Equivalent of MSDOS 5.0 MSDOS.SYS kernel file
36 ;
37 ; ((MSDOS 6.0 kernel source code has been modified by using disassembled
38 ; MSDOS 5.0 MSDOS.SYS)) -- Disassembler: HEX-RAYS IDA Pro --
39 ; ((Disassembly -Reverse engineering- reference: MSDOS 6.0 kernel src))
40 ;
41 ; ----- Retro DOS v2 (v3) boot sector loads RETRODOS.SYS (MSDOS.SYS)
42 ; at 1000h:0000h and loader (initialization) part of RETRODOS kernel
43 ; moves IO.SYS (DOSBIOSCODE & DOSBIOSDATA, 'IOSYS5.BIN') to 70h:0000h.
44 ; Then SYSINIT code to the next segment (4D6h for current version)..
45 ; SYSINIT code relocates itself and DOSBIOSCODE and MSDOS.SYS
46 ; (MSDOS5.BIN) according to request/setting in 'config.sys' file.
47 ;
48 ; -----
49 ;
50 ; 01/01/2024 - Retro DOS v5.0 Kernel ('ibmdos7.s')
51 ; Modified from 'msdos6.s' (29/09/2023, Retro DOS v4.2 kernel: MSDOS.SYS) file
52 ; as below:
53 ;
54 ; 1) Retro DOS v5.0 IBMDOS.COM is based on disassembled source code
55 ; of PCDOS 7.1 IBMDOS.COM (2003) and it is derived using Retro DOS v4.2
56 ; MSDOS.SYS source code. Retro DOS v5.0 (IBMDOS.COM) kernel source code
57 ; is modified (and optimized) and so, it is not same with the original
58 ; PCDOS 7.1 IBMDOS.COM.
59 ;
60 ; 2) Retro DOS v4.2 MSDOS.SYS is based on disassembled source code
61 ; of MSDOS 6.21 MSDOS.SYS, derived using MSDOS 6.0 source code.
62 ; (And then it has been verified and updated by comparing it with
63 ; the disassembled source code of MSDOS 6.22 kernel file MSDOS.SYS.)
64 ;
65 ; 3) Labels, names, comments, explanations and structure definitions
66 ; about procedures and code details are almost entirely taken from
67 ; the original MSDOS 6.0 source code, except for the details that
68 ; Erdogan Tan personally experienced. Some of them are incompatible
69 ; with PCDOS 7.1 code. But they have not been deleted to preserve
70 ; the originality of the descriptions.)
71 ;
72 ; ('msdos6.s' has been converted to 'ibmdos7.s' and 'retrodos42.s' has been
73 ; converted to 'retrodos5.s'. 'ibmdos7.s' is IBMDOS.COM source code file
74 ; while 'retrodos5.s' is source code of Retro DOS v5 kernel file 'PCDOS.SYS'.
75 ; 'retrodos5.s' includes 'ibmdos7.bin' or IBMDOS.COM as binary file.)
76 ;
77 ; -----
78 ;
79 ; =====
80 ; Most of comments in this file are from the original MSDOS 6.0 source code
81 ; -----
82 ;
83 ; MSDOS 6.0 kernel source files:
84 ; MSDATA.ASM,
85 ; (MSHEAD.ASM, MSCONST.ASM, CONST2.ASM, MS_DATA.ASM,
86 ; DOSTAB.ASM, LMSTUB.ASM, WPATCH.INC, MPATCH.ASM)
87 ; Mstable.ASM, MScode.ASM, MSDOSME.ASM (DOSMES.INC), TIME.ASM,
88 ; GETSET.ASM, PARSE.ASM, MISC.ASM, MISC2.ASM, CRIT.ASM, CPMIO.ASM,
89 ; CPMIO2.ASM, FCBIO.ASM, FCBIO2.ASM, SEARCH.ASM, PATH.ASM, IOCTL.ASM,
90 ; DELETE.ASM, RENAME.ASM, FINFO.ASM, DUP.ASM, CREATE.ASM, OPEN.ASM,
91 ; DINFO.ASM, ISEARCH.ASM, BUF.ASM, ABORT.ASM, CLOSE.ASM, DIRCALL.ASM,
92 ; DISK.ASM, DISK2.ASM, DISK3.ASM, DIR.ASM, DIR2.ASM, DEV.ASM,
93 ; MKNODE.ASM, ROM.ASM, FCB.ASM, MSCTRLC.ASM, FAT.ASM, MSPROC.ASM
94 ; ALLOC.ASM, SRVCALL.ASM, UTIL.ASM, MACRO.ASM, MACRO2.ASM, HANDLE.ASM
95 ; FILE.ASM, LOCK.ASM, ROMFIND.ASM, SHARE.ASM, MSINIT.ASM, ORIGIN.ASM
96 ;
97 ; MSDOS 2.0 kernel source files:
98 ; MSDOS.ASM (STDSW.ASM + MSHEAD.ASM + MSDATA.ASM)
99 ; MScode.ASM
100 ; DOSMES.ASM ... STDIO.ASM, TIME.ASM, XENIX.ASM, XENIX2.ASM
101 ;
102 ; =====
103 ; DOSLINK
104 ; =====
105 ; msdos mscode dosmes misc getset dircall alloc dev dir +
106 ; disk fat rom stdbuf stdcall stdctrlc stdfcb stdproc +
107 ; stdio time xenix xenix2
108 ;
109 ; =====
110 ; This MSDOS source code is verified & modified by using IDA Pro Disassembler
111 ; output in TASM syntax (July 2018 -> NASM syntax) [ IBMDOS.COM, 17/03/1987 ]
112 ; =====
113 ; #####
114 ; # This file is generated by The Interactive Disassembler (IDA) #
115 ; # Copyright (c) 2010 by Hex-Rays SA, <support@hex-rays.com> #
116 ; # Licensed to: Freeware version #
117 ; #####
118 ;
119 ;
120 ; Input MD5 : 75959BC417C19135B982F7959EE9C92A
121 ;
122 ; -----
123 ; File Name : C:\Documents and Settings\Erdogan Tan\Desktop\MSDOS621.BIN
124 ; Format : Binary file

```

```

125 ;=====
126 ; MSDOS621.BIN = MSDOS.SYS, 13/02/1994, 38138 bytes (MSDOS 6.21 kernel) 2019
127 ;-----
128 ; MSDOS5.BIN = MSDOS.SYS, 11/11/1991, 37394 bytes (MSDOS 5.0 kernel) 2022
129 ;-----
130 ;
131 ; MSDOS.ASM
132 ;=====
133 ;
134 ;TITLE    Standard MSDOS
135 ;NAME     MSDOS_2
136 ;
137 ; Number of disk I/O buffers
138 ;
139 ; INCLUDE STDSW.ASM
140 ; INCLUDE MSHEAD.ASM
141 ; INCLUDE MSDATA.ASM
142 ;
143 ; END
144 ;=====
145 ; STDSW.ASM
146 ;=====
147 ;
148 TRUE EQU 0FFFFH
149 FALSE EQU ~TRUE ; NOT TRUE
150 ;
151 ; Use the switches below to produce the standard Microsoft version or the IBM
152 ; version of the operating system
153 ;MSVER EQU false
154 ;IBM EQU true
155 ;WANG EQU FALSE
156 ;ALTVECT EQU FALSE
157 ;
158 ; Set this switch to cause DOS to move itself to the end of memory
159 ;HIGHMEM EQU FALSE
160 ;
161 ; IF IBM
162 ESCCH EQU 0 ;character to begin escape seq.
163 CANCEL EQU 27 ;Cancel with escape
164 TOGLINS EQU TRUE ;One key toggles insert mode
165 TOGLPRN EQU TRUE ;One key toggles printer echo
166 ZEROEXT EQU TRUE
167 ; ELSE
168 ; IF WANG
169 ;ESCCH EQU 1FH ;Are we assembling for WANG?
170 ; ;Yes. Use 1FH for escape character
171 ; ELSE
172 ;ESCCH EQU 1BH
173 ; ENDIF
174 ;CANCEL EQU "X"-@" ;Cancel with Ctrl-X
175 ;TOGLINS EQU WANG ;Separate keys for insert mode on
176 ; ;and off if not WANG
177 ;TOGLPRN EQU FALSE ;Separate keys for printer echo on
178 ; ;and off
179 ;ZEROEXT EQU TRUE
180 ; ENDIF
181 ;
182 ;=====
183 ; MSHEAD.ASM
184 ;=====
185 ;
186 ;-----
187 ; TITLE MSHEAD.ASM -- MS-DOS DEFINITIONS
188 ;-----
189 ;
190 ; MS-DOS High-performance operating system for the 8086 version 1.28
191 ; by Microsoft MSDOS development group:
192 ; Tim Paterson (Ret.)
193 ; Aaron Reynolds
194 ; Nancy Panners (Parenting)
195 ; Mark Zbikowski
196 ; Chris Peters (BIOS) (ret.)
197 ;
198 ; ***** Revision History *****
199 ; >> EVERY change must noted below!! <<
200 ;
201 ; 0.34 12/29/80 General release, updating all past customers
202 ; 0.42 02/25/81 32-byte directory entries added
203 ; 0.56 03/23/81 Variable record and sector sizes
204 ; 0.60 03/27/81 Ctrl-C exit changes, including register save on user stack
205 ; 0.74 04/15/81 Recognize I/O devices with file names
206 ; 0.75 04/17/81 Improve and correct buffer handling
207 ; 0.76 04/23/81 Correct directory size when not 2^N entries
208 ; 0.80 04/27/81 Add console input without echo, Functions 7 & 8
209 ; 1.00 04/28/81 Renumbr for general release
210 ; 1.01 05/12/81 Fix bug in 'STORE'
211 ; 1.10 07/21/81 Fatal error trapping, NUL device, hidden files, date & time,
212 ; RENAME fix, general cleanup
213 ; 1.11 09/03/81 Don't set CURRENT BLOCK to 0 on open; fix SET FILE SIZE
214 ; 1.12 10/09/81 Zero high half of CURRENT BLOCK after all (CP/M programs don't)
215 ; 1.13 10/29/81 Fix classic "no write-through" error in buffer handling
216 ; 1.20 12/31/81 Add time to FCB; separate FAT from DPT; Kill SMALLDIR; Add
217 ; FLUSH and MAPDEV calls; allow disk mapping in DSKCHG; Lots
218 ; of smaller improvements
219 ; 1.21 01/06/82 HIGHMEM switch to run DOS in high memory
220 ; 1.22 01/12/82 Add VERIFY system call to enable/disable verify after write
221 ; 1.23 02/11/82 Add defaulting to parser; use variable escape character Don't
222 ; zero extent field in IBM version (back to 1.01!)
223 ; 1.24 03/01/82 Restore fcn. 27 to 1.0 level; add fcn. 28
224 ; 1.25 03/03/82 Put marker (00) at end of directory to speed searches
225 ; 1.26 03/03/82 Directory buffers searched as a circular queue, current buffer
226 ; is searched first when possible to minimize I/O
227 ; 03/03/82 STORE routine optimized to tack on partial sector tail as
228 ; full sector write when file is growing
229 ; 03/09/82 Multiple I/O buffers
230 ; 03/29/82 Two bugs: Delete all case resets search to start at beginning
231 ; of directory (infinite loop possible otherwise), DSKRESET
232 ; must invalidate all buffers (disk and directory).
233 ; 1.27 03/31/82 Installable device drivers
234 ; Function call 47 - Get pointer to device table list
235 ; Function call 48 - Assign CON AUX LIST
236 ; 04/01/82 Spooler interrupt (INT 28) added.
237 ; 1.28 04/15/82 DOS restructured to use ASSUMES and PROC labels around system
238 ; call entries. Most CS relative references changed to SS
239 ; relative with an eye toward putting a portion of the DOS in
240 ; ROM. DOS source also broken into header, data and code pieces
241 ; 04/15/82 GETDMA and GETVECT calls added as 24 and 32. These calls
242 ; return the current values.
243 ; 04/15/82 INDOS flag implemented for interrupt processing along with
244 ; call to return flag location (call 29)
245 ; 04/15/82 Volume ID attribute added
246 ; 04/17/82 Changed ABORT return to user to a long ret from a long jump to
247 ; avoid a CS relative reference.
248 ; 04/17/82 Put call to STATCHK in dispatcher to catch ^C more often

```

```

249 ; 04/20/82 Added INT int_upooler into loop ^S wait
250 ; 04/22/82 Dynamic disk I/O buffer allocation and call to manage them
251 ; call 49.
252 ; 04/23/82 Added GETDSKPTDL as call 50, similar to GETFATPT(DL), returns
253 ; address of DPB
254 ; 04/29/82 Mod to WRTDEV to look for ^C or ^S at console input when
255 ; writting to console device via file I/O. Added a console
256 ; output attribute to devices.
257 ; 04/30/82 Call to en/dis able ^C check in dispatcher call 51
258 ; 04/30/82 Code to allow assignment of func 1-12 to disk files as well
259 ; as devices.... pipes, redirection now possible
260 ; 04/30/82 Expanded GETLIST call to 2.0 standard
261 ; 05/04/82 Change to INT int_fatal_abort callout int HARDERR. DOS SS
262 ; (data segment) stashed in ES, INT int_fatal_abort routines must
263 ; preserve ES. This mod so HARDERR can be ROMed.
264 ; 1.29 06/01/82 Installable block and character devices as per 2.0 spec
265 ; 06/04/82 Fixed Bug in CLOSE regarding call to CHKFATWRT. It got left
266 ; out back about 1.27 or so (oops). ARR
267 ; 1.30 06/07/82 Directory sector buffering added to main DOS buffer queue
268 ; 1.40 06/15/82 Tree structured directories. XENIX Path Parser MKDIR CHDIR
269 ; RMDIR Xenix calls
270 ; 1.41 06/13/82 Made GETBUFFR call PLACEBUF
271 ; 1.50 06/17/82 FATS cached in buffer pool, get FAT pointer calls disappear
272 ; Frees up lots of memory.
273 ; 1.51 06/24/82 BREAKDOWN modified to do EXACT one sector read/write through
274 ; system buffers
275 ; 1.52 06/30/82 OPEN, CLOSE, READ, WRITE, DUP, DUP2, LSEEK implemented
276 ; 1.53 07/01/82 OPEN CLOSE mod for Xenix calls, saves and gets remote dir
277 ; 1.54 07/11/82 Function calls 1-12 make use of new 2.0 PDB. Init code
278 ; changed to set file handle environment.
279 ; 2.00 08/01/82 Number for IBM release
280 ; 01/19/83 No environ bug in EXEC
281 ; 01/19/83 MS-DOS OEM INT 21 extensions (SET_OEM_HANDLER)
282 ; 01/19/83 Performance bug fix in cooked write to NUL
283 ; 01/27/83 Growcnt fixed for 32-bits
284 ; 01/27/83 Find-first problem after create
285 ; 2.01 02/17/83 International DOS
286 ; 2.11 08/12/83 Dos split into several more modules for assembly on
287 ; an IBM PC
288 ; 08/07/2018 - Retro DOS v3.0 by Erdogan Tan
289 ; (MSHEAD.ASM, MSDOS 6.0, 1991) - mshead.asm 1.1 85/04/10 -
290 ; 2.10 03/09/83 Start of NETWORK support
291 ; New Buffer structure
292 ; New Sytem file table structure
293 ; FCB moved to internal representation
294 ; DOS re-organized
295 ; 2.11 04/21/83 Continuation of 2.10, preliminary Network
296 ; device interface.
297 ; 2.11 08/12/83 Dos split into several more modules for assembly on
298 ; an IBM PC
299 ; 2.50 09/12/83 More network stuff
300 ;
301 ; *****
302 ;
303 ; -----
304 ; EQUATES
305 ;
306 ; Interrupt Entry Points:
307 ;
308 ; INTBASE: ABORT
309 ; INTBASE+4: COMMAND
310 ; INTBASE+8: BASE EXIT ADDRESS
311 ; INTBASE+C: CONTROL-C ABORT
312 ; INTBASE+10H: FATAL ERROR ABORT
313 ; INTBASE+14H: BIOS DISK READ
314 ; INTBASE+18H: BIOS DISK WRITE
315 ; INTBASE+1CH: END BUT STAY RESIDENT (NOT SET BY DOS)
316 ; INTBASE+20H: SPOOLER INTERRUPT
317 ; INTBASE+40H: Long jump to CALL entry point
318 ;
319 ENTRYPOINTSEG EQU 0Ch
320 MAXDIF EQU 0FFFh
321 SAVEXIT EQU 10
322 ; 06/05/2019
323 WRAPOFFSET EQU 0FEF0h ; (MISC.ASM, MSDOS 6.0, 1991)
324 ;
325 ; INCLUDE DOSSYM.ASM
326 ; INCLUDE DEVSYM.ASM
327 ;
328 ; SUBTTL ^C, terminate/abort/exit and Hard error actions
329 ; PAGE
330 ; There are three kinds of context resets that can occur during normal DOS
331 ; functioning: ^C trap, terminate/abort/exit, and Hard-disk error. These must
332 ; be handles in a clean fashion that allows nested executions along with the
333 ; ability to trap one's own errors.
334 ;
335 ; ^C trap - A process may elect to catch his own ^Cs. This is achieved by
336 ; using the $GET_INTERRUPT_VECTOR and $SET_INTERRUPT_VECTOR as
337 ; follows:
338 ;
339 ; $GET_INTERRUPT_VECTOR for INT int_ctrl_c
340 ; Save it in static memory.
341 ; $SET_INTERRUPT_VECTOR for INT int_ctrl_c
342 ;
343 ; The interrupt service routine must preserve all registers and
344 ; return carry set iff the operation is to be aborted (via abort
345 ; system call), otherwise, carry is reset and the operation is
346 ; restarted. ANY DEVIATION FROM THIS WILL LEAD TO UNRELIABLE
347 ; RESULTS.
348 ;
349 ; To restore original ^C processing (done on terminate/abort/exit),
350 ; restore INT int_ctrl_c from the saved vector.
351 ;
352 ; Hard-disk error -- The interrupt service routine for INT int_fatal_abort must
353 ; also preserve registers and return one of three values in AL: 0 and
354 ; 1 imply retry and ignore (???) and 2 indicates an abort. The user
355 ; himself is not to issue the abort, rather, the dos will do it for
356 ; him by simulating a normal abort/exit system call. ANY DEVIATION
357 ; FROM THIS WILL LEAD TO UNRELIABLE RESULTS.
358 ;
359 ; terminate/abort/exit -- The user may not, under any circumstances trap an
360 ; abort call. This is reserved for knowledgeable system programs.
361 ; ANY DEVIATION FROM THIS WILL LEAD TO UNRELIABLE RESULTS.
362 ;
363 ;SUBTTL SEGMENT DECLARATIONS
364 ;
365 ; The following are all of the segments used. They are declared in the order
366 ; that they should be placed in the executable
367 ;
368 ;
369 ; segment ordering for MSDOS
370 ;
371 ;
372 ;START SEGMENT BYTE PUBLIC 'START'

```

```

374 ;START                                ENDS
375 ;CONSTANTS                          SEGMENT BYTE PUBLIC 'CONST'
376 ;CONSTANTS                          ENDS
377
378 ;DATA                                SEGMENT WORD PUBLIC 'DATA'
379 ;DATA                                ENDS
380
381 ;CODE                                SEGMENT BYTE PUBLIC 'CODE'
382 ;CODE                                ENDS
383
384 ;LAST                                SEGMENT BYTE PUBLIC 'LAST'
385 ;LAST                                ENDS
386
387 ;DOSGROUP      GROUP    CODE,CONSTANTS,DATA,LAST
388
389 ; The following segment is defined such that the data/const classes appear
390 ; before the code class for ROMification
391
392 ;START                                SEGMENT BYTE PUBLIC 'START'
393 ;                                     ASSUME CS:DOSGROUP,DS:NOHING,ES:NOHING,SS:NOHING
394 ;                                     JMP     DOSINIT
395 ;START                                ENDS
396
397 ;=====
398 ; BPB.INC, MSDOS 6.0, 1991
399 ;=====
400 ; 09/07/2018 - Retro DOS v3.0
401
402 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
403 ;             C A V E A T   P R O G R A M M E R                               ;
404 ;                                                                                   ;
405 ;
406 ;** BIOS PARAMETER BLOCK DEFINITION
407 ;
408 ; The BPB contains information about the disk structure. It dates
409 ; back to the earliest FAT systems and so FAT information is
410 ; intermingled with physical driver information.
411 ;
412 ; A boot sector contains a BPB for its device; for other disks
413 ; the driver creates a BPB. DOS keeps copies of some of this
414 ; information in the DPB.
415 ;
416 ; The BDS structure contains a BPB within it.
417
418 ; 01/01/2024
419 %if 0
420
421 struct A_BPB
422 .BPB_BYTESPERSECTOR:        resw    1
423 .BPB_SECTORSERCLUSTER:      resb    1
424 .BPB_RESERVEDSECTORS:       resw    1
425 .BPB_NUMBEROFFATS:          resb    1
426 .BPB_ROOTENTRIES:           resw    1
427 .BPB_TOTALSECTORS:          resw    1
428 .BPB_MEDIADSCRIPTOR:        resb    1
429 .BPB_SECTORSERFAT:           resw    1
430 .BPB_SECTORSERTRACK:         resw    1
431 .BPB_HEADS:                 resw    1
432 .BPB_HIDDENSECTORS:          resw    1
433                             resw    1
434 .BPB_BIGTOTALSECTORS:        resw    1
435                             resw    1
436                             resb    6 ; NOTE: many times these
437 ;                                     ; 6 bytes are omitted
438 ;                                     ; when BPB manipulations
439 ;                                     ; are performed!
440 .size:
441 endstruct
442
443 %else
444
445 ; 14/04/2024
446 ; 01/01/2024 - Retro DOS v5.0
447
448 struct A_BPB
449 00000000 ???? .BYTESPERSECTOR:    resw 1
450 00000002 ?? .SECTORSERCLUSTER:    resb 1
451 00000003 ???? .RESERVEDSECTORS:    resw 1
452 00000005 ?? .NUMBEROFFATS:          resb 1
453 00000006 ???? .ROOTENTRIES:           resw 1
454 00000008 ???? .TOTALSECTORS:          resw 1
455 0000000A ?? .MEDIADSCRIPTOR:        resb 1
456 0000000B ???? .SECTORSERFAT:           resw 1
457 0000000D ???? .SECTORSERTRACK:         resw 1
458 0000000F ???? .HEADS:                 resw 1
459 00000011 ?????????? .HIDDENSECTORS:          resd 1
460 00000015 ?????????? .BIGTOTALSECTORS:        resd 1
461 ;..... FAT32 ..... + 28
462 00000019 ?????????? .FATSIZE32:          resd 1
463 0000001D ???? .EXTFLAGS:            resw 1
464 0000001F ???? .FSVER:                  resw 1
465 00000021 ?????????? .ROOTDIRCLUSTER:        resd 1
466 00000025 ???? .FSINFOSECTOR:          resw 1 ; (offset from FAT32 bs)
467 00000027 ???? .BACKUPBOOTSECTOR:        resw 1 ; (offset from FAT32 bs)
468 00000029 <res Ch> .RESERVEDBYTES:          resb 12 ; (zero bytes)
469 ; 14/04/2024
470 00000035 ?????????????? resb 6 ; A_BPB.size must be 59
471 .size:
472 endstruct
473
474 %endif
475
476 ;
477 ;             C A V E A T   P R O G R A M M E R                               ;
478 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
479 ;
480 ;=====
481 ; BUFFER.INC, MSDOS 6.0, 1991
482 ;=====
483 ; 04/05/2019 - Retro DOS v4.0
484 ; 03/01/2024 - Retro DOS v5.0
485
486 ; <Disk I/O Buffer Header>
487
488 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
489 ;             C A V E A T   P R O G R A M M E R                               ;
490 ;                                                                                   ;
491 ;
492 ; Field definition for I/O buffer information
493
494 ; 03/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
495
496 struct BUFFINFO

```


[illegible]


```
745 addr_int_abort EQU 4 * int_abort
746 addr_int_command EQU 4 * int_command
747 addr_int_terminate EQU 4 * int_terminate
748 addr_int_ctrl_c EQU 4 * int_ctrl_c
749 addr_int_fatal_abort EQU 4 * int_fatal_abort
750 addr_int_disk_read EQU 4 * int_disk_read
751 addr_int_disk_write EQU 4 * int_disk_write
752 addr_int_keep_process EQU 4 * int_keep_process
753 ;-----+-----;
754 ; C A V E A T P R O G R A M M E R ;
755 ;-----+-----;
756 ;
757 addr_int_spooler EQU 4 * int_spooler
758 addr_int_fastcon EQU 4 * int_fastcon
759 addr_int_ibm EQU 4 * int_IBM
760 ;
761 ; C A V E A T P R O G R A M M E R ;
762 ;-----+-----;
763 ;=====
764 ; DIRENT.INC, MSDOS 6.0, 1991
765 ;=====
766 ; 04/05/2019 - Retro DOS v4.0
767 ;
768 ; BREAK <Directory entry>
769 ;
770 ;
771 ;
772 |-----|
773 | (12 BYTE) filename/ext | 0 0
774 |-----|
775 | (BYTE) attributes | 11 B
776 |-----|
777 | (10 BYTE) reserved | 12 C
778 |-----|
779 | (WORD) time of last write | 22 16
780 |-----|
781 | (WORD) date of last write | 24 18
782 |-----|
783 | (WORD) First cluster | 26 1A
784 |-----|
785 | (DWORD) file size | 28 1C
786 |-----|
787 ;
788 ; First byte of filename = E5 -> free directory entry
789 ; = 00 -> end of allocated directory
790 ; Time: Bits 0-4=seconds/2, bits 5-10=minute, 11-15=hour
791 ; Date: Bits 0-4=day, bits 5-8=month, bits 9-15=year-1980
792 ;
793 ; 01/01/2024
794 %if 0
795 ;
796 struct dir_entry
797 .dir_name: resb 11 ; file name
798 .dir_attr: resb 1 ; attribute bits
799 .dir_codepg: resw 1 ; code page DOS 4.00
800 .dir_extcluster: resw 1 ; extended attribute starting cluster
801 .dir_attr2: resb 1 ; reserved
802 .dir_pad: resb 5 ; reserved for expansion
803 .dir_time: resw 1 ; time of last write
804 .dir_date: resw 1 ; date of last write
805 .dir_first: resw 1 ; first allocation unit of file
806 .dir_size_l: resw 1 ; low 16 bits of file size
807 .dir_size_h: resw 1 ; high 16 bits of file size
808 .size:
809 endstruct
810 ;
811 %else
812 ; 01/01/2024 - Retro DOS v5.0 (*)
813 ;
814 struct dir_entry
815 .dir_name: resb 11 ; file name (short file name)
816 .dir_attr: resb 1 ; file attributes (*)
817 .dir_nt_res: resb 1 ; reserved for use by windows NT. 0
818 .dir_pad: ;resb 7 ; (creation time, last access date
819 ; for use by windows)
820 .dir_crtime_tenth:
821 resb 1
822 .dir_crtime: resw 1
823 .dir_crdate: resw 1
824 .dir_lstaccdate:
825 resw 1
826 .dir_fclus_hi: resw 1 ; FAT32 fs ; high word of first cluster number
827 .dir_time: resw 1 ; time of last write
828 .dir_date: resw 1 ; date of last write
829 .dir_fclus:
830 .dir_first: resw 1 ; first cluster (alloc. unit) of file
831 .dir_file_size:
832 .dir_size_l: resw 1 ; low 16 bits of file size
833 .dir_size_h: resw 1 ; high 16 bits of file size
834 .size:
835 endstruct
836 ;
837 %endif
838 ;
839 attr_read_only EQU 1h
840 attr_hidden EQU 2h
841 attr_system EQU 4h
842 attr_volume_id EQU 8h
843 attr_directory EQU 10h
844 attr_archive EQU 20h
845 attr_device EQU 40h ; This is a VERY special bit.
846 ; NO directory entry on a disk EVER
847 ; has this bit set. It is set non-zero
848 ; when a device is found by GETPATH
849 ;
850 attr_all EQU attr_hidden+attr_system+attr_directory
851 ; OR of hard attributes for FINDENTRY
852 attr_ignore EQU attr_read_only+attr_archive
853 ; ignore this(ese) attribute(s)
854 ; during search first/next
855 attr_changeable EQU attr_read_only+attr_hidden+attr_system+attr_archive
856 ; changeable via CHMOD
857 DIRFREE equ 0E5h ; stored in dir_name[0] to indicate free slot
858 ; -----
859 ; 01/01/2024 - Retro DOS v5.0
860 ; ref: FAT32 File System Specification v1.03 (Microsoft, December 6, 2000) (*)
861 ;
862 attr_long_name equ attr_read_only|attr_hidden|attr_system|attr_volume_id
863 attr_longname_mask equ attr_long_name|attr_directory|attr_archive
```

```
;
;      C   A   V   E   A   T       P   R   O   G   R   A   M   M   E   R
;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
=====
; SF.INC, MSDOS 6.0, 1991
=====
; 25/04/2019 - Retro DOS v4.0
; 07/07/2018 - Retro DOS v3.0
;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
;
;      C   A   V   E   A   T       P   R   O   G   R   A   M   M   E   R
```



```

993 ;
994 ; The system file table entries are allocated in contiguous groups.
995 ; There may be more than one such groups; the SF "superstructure"
996 ; tracks the groups.
997 ; -----
998
999 struct SFT
1000 .SFLink: resd 1
1001 .SFCount: resw 1 ; number of entries
1002 .SFTable: resw 1 ; beginning of array of the following
1003 .size:
1004 endstruc
1005
1006 ; -----
1007 ; ** System file table entry
1008 ;
1009 ; These are the structures which are at SFTABLE in the SF structure.
1010 ; -----
1011
1012 ; 01/01/2024 - Retro DOS v5.0
1013 ; 25/04/2019 - Retro DOS v4.0
1014
1015 struct SF_ENTRY
1016 .sf_ref_count: resw 1 ; 0 ; number of processes sharing entry
1017 ; if FCB then ref count
1018 .sf_mode: resw 1 ; 2 ; mode of access or high bit on if FCB
1019 .sf_attr: resb 1 ; 4 ; attribute of file
1020 .sf_flags: resw 1 ; 5 ; Bits 8-15
1021 ; Bit 15 = 1 if remote file
1022 ; = 0 if local file or device
1023 ; Bit 14 = 1 if date/time is not to be
1024 ; set from clock at CLOSE. Set by
1025 ; FILETIMES and FCB_CLOSE. Reset by
1026 ; other reseters of the dirty bit
1027 ; (WRITE)
1028 ; Bit 13 = Pipe bit (reserved)
1029 ;
1030 ; Bits 0-7 (old FCB_devid bits)
1031 ; If remote file or local file, bit
1032 ; 6=0 if dirty Device ID number, bits
1033 ; 0-5 if local file.
1034 ; bit 7=0 for local file, bit 7
1035 ; =1 for local I/O device
1036 ; If local I/O device, bit 6=0 if EOF (input)
1037 ; Bit 5=1 if Raw mode
1038 ; Bit 0=1 if console input device
1039 ; Bit 1=1 if console output device
1040 ; Bit 2=1 if null device
1041 ; Bit 3=1 if clock device
1042 .sf_devptr: resd 1 ; 7 ; Points to DPB if local file, points
1043 ; to device header if local device,
1044 ; points to net device header if
1045 ; remote
1046 .sf_firclus: resw 1 ; 11 ; First cluster of file (bit 15 = 0)
1047 .sf_time: resw 1 ; 13 ; Time associated with file
1048 .sf_date: resw 1 ; 15 ; Date associated with file
1049 .sf_size: resd 1 ; 17 ; Size associated with file
1050 .sf_position: resd 1 ; 21 ; Read/Write pointer or LRU count for FCBS
1051
1052 ; Starting here, the next 7 bytes may be used by the file system to store
1053 ; an ID
1054
1055 ; 09/07/2018 - Retro DOS v3.0
1056
1057 ; MSDOS 3.3 SF.INC, 1987
1058 .sf_cluspos: resw 1 ; Position of last cluster accessed
1059 .sf_lstclus: resw 1 ; Last cluster accessed
1060 .sf_dirsec: resw 1 ; Sector number of directory sector
1061 ; for this file
1062 .sf_dirpos: resb 1 ; Offset of this entry in the above
1063
1064 ; MSDOS 6.0, SF.INC, 1991
1065 .sf_cluspos: resw 1 ; 25 ; Position of last cluster accessed
1066 .sf_dirsec: resd 1 ; 27 ; Sector number of directory sector
1067 ; for this file
1068 .sf_dirpos: resb 1 ; 31 ; Offset of this entry in the above
1069
1070 ; End of 7 bytes of file-system specific info.
1071
1072 .sf_name: resb 11 ; 32 ; 11 character name that is in the
1073 ; directory entry. This is used by
1074 ; close to detect file deleted and
1075 ; disk changed errors.
1076 .sf_fclus32: ; 22/03/2024
1077 ; SHARING INFO
1078 .sf_chain: resd 1 ; 43 ; link to next SF
1079 .sf_UID: resw 1 ; 47
1080 .sf_PID: resw 1 ; 49 ; owner process identifier (PSP segment)
1081 .sf_MFT: resw 1 ; 51
1082
1083 ; MSDOS 6.0, SF.INC, 1991
1084 .sf_lstclus: resw 1 ; 53 ; AN009; Last cluster accessed
1085 .sf_IFS_HDR: resd 1 ; 55 ; pointer to IFS drive or 0:0 for files
1086
1087 .size:
1088 endstruc
1089
1090 ; 20/07/2018
1091 ; MSDOS 3.3, SF.INC, 1987
1092 %define sf_netid SF_ENTRY.sf_cluspos ; byte
1093 %define sf_OpenAge SF_ENTRY.sf_position+2 ; word
1094 %define sf_LRU SF_ENTRY.sf_position ; word
1095 ; MSDOS 6.0, SF.INC, 1991
1096 %define sf_fsda SF_ENTRY.sf_cluspos ; byte ;DOS 4.00
1097 %define sf_serial_ID SF_ENTRY.sf_firclus ; word ;DOS 4.00
1098
1099 ; 19/07/2018
1100 ; MSDOS 3.3, SF.INC, 1987
1101
1102 sf_default_number EQU 5
1103
1104 ; Note that we need to mark an SFT as being busy for OPEN/CREATE. This is
1105 ; because an INT 24 may prevent us from 'freeing' it. We mark this as such
1106 ; by placing a -1 in the ref_count field.
1107
1108 sf_busy EQU -1
1109
1110 ; mode mask for FCB detection
1111 sf_isFCB EQU 1000000000000000B
1112
1113 ; Flag word masks
1114 sf_isnet EQU 1000000000000000B
1115 sf_close_nodate EQU 0100000000000000B
1116 sf_pipe EQU 0010000000000000B

```

```

1117 sf_no_inherit EQU 0001000000000000B
1118 sf_net_spool EQU 0000100000000000B
1119
1120 ; 25/04/2019
1121 sf_entry_size equ SF_ENTRY.size ; 59 (MSDOS 6.0)
1122
1123 ; -----
1124 ; Local file/device flag masks
1125 ; -----
1126
1127 devid_file_clean EQU 40h ; true if file and not written
1128 devid_file_mask_drive EQU 3Fh ; mask for drive number
1129
1130 devid_device EQU 80h ; true if a device
1131 devid_device_EOF EQU 40h ; true if end of file reached
1132 devid_device_raw EQU 20h ; true if in raw mode
1133 devid_device_special EQU 10h ; true if special device
1134 devid_device_clock EQU 08h ; true if clock device
1135 devid_device_null EQU 04h ; true if null device
1136 devid_device_con_out EQU 02h ; true if console output
1137 devid_device_con_in EQU 01h ; true if console input
1138
1139 ; -----
1140 ; structure of devid field as returned by IOCTL is:
1141 ;
1142 ; BIT 7 6 5 4 3 2 1 0
1143 ; |---|---|---|---|---|---|---|---|
1144 ; | I | E | R | S | I | I | I | I |
1145 ; | S | O | A | P | S | S | S | S |
1146 ; | D | F | W | E | C | N | C | C |
1147 ; | E | | | C | L | U | O | I |
1148 ; | V | | | L | K | L | T | N |
1149 ; |---|---|---|---|---|---|---|---|
1150 ; ISDEV = 1 if this channel is a device
1151 ; = 0 if this channel is a disk file
1152 ;
1153 ; If ISDEV = 1
1154 ;
1155 ; EOF = 0 if End Of File on input
1156 ; RAW = 1 if this device is in Raw mode
1157 ; = 0 if this device is cooked
1158 ; ISCLK = 1 if this device is the clock device
1159 ; ISNUL = 1 if this device is the null device
1160 ; ISCOT = 1 if this device is the console output
1161 ; ISCIN = 1 if this device is the console input
1162 ;
1163 ; If ISDEV = 0
1164 ; EOF = 0 if channel has been written
1165 ; Bits 0-5 are the block device number for
1166 ; the channel (0 = A, 1 = B, ...)
1167 ; -----
1168
1169 devid_ISDEV EQU 80h
1170 devid_EOF EQU 40h
1171 devid_RAW EQU 20h
1172 devid_SPECIAL EQU 10h
1173 devid_ISCLK EQU 08h
1174 devid_ISNUL EQU 04h
1175 devid_ISCOT EQU 02h
1176 devid_ISCIN EQU 01h
1177
1178 devid_block_dev EQU 1Fh ; mask for block device number
1179
1180 ; =====
1181 ; PDB.INC, MSDOS 6.0, 1991
1182 ; =====
1183 ; 04/05/2019 - Retro DOS v4.0
1184 ; 08/07/2018 - Retro DOS v3.0
1185
1186 ; -----
1187 ; BREAK <Process data block>
1188 ; -----
1189 ; ** Process data block (otherwise known as program header)
1190 ;
1191 ;
1192 ; These offset are documented in the MSDOS Encyclopedia, so nothing
1193 ; can be rearranged here, ever. Reserved areas are probably safe
1194 ; for use.
1195 ; -----
1196
1197 FILPERPROC EQU 20
1198
1199 struc PDB ; Process_data_block
1200 .EXIT_CALL: resw 1 ; INT int_abort system terminate
1201 .BLOCK_LEN: resw 1 ; size of execution block
1202 .CPM_CALL: resb 1 ;
1203 .EXIT: resd 1 ; pointer to exit routine
1204 .CTRL_C: resd 1 ; pointer to ^C routine
1205 .FATAL_ABORT: resd 1 ; pointer to fatal error
1206 .PARENT_PID: resw 1 ; PID of parent (terminate PID)
1207 .JFN_TABLE: resb FILPERPROC ; indices into system table
1208 .ENVIRON: resw 1 ; segment address of environment
1209 .USER_STACK: resd 1 ; stack of self during system calls
1210 .JFN_Length: resw 1 ; number of handles allowed
1211 .JFN_Pointer: resd 1 ; pointer to JFN table
1212 .Next_PDB: resd 1 ; pointer to nested PDB's
1213 .InterCon: resb 1 ; MSDOS 6.0 ; *** jh-3/28/90 ***
1214 .Append: resb 1 ; MSDOS 6.0 ; *** Not sure if still used ***
1215 .Novell_Used: resb 2 ; MSDOS 6.0 ; Novell shell (redir) uses these
1216 .Version: resw 1 ; MSDOS 6.0 ; DOS version reported to this app
1217 .PAD1: resb 14 ; 0Eh
1218 .CALL_SYSTEM: resb 5 ; portable method of system call
1219 .PAD2: resb 7 ; reserved so FCB 1 can be used as
1220 ; an extended FCB
1221 ;endstruc ; MSDOS 3.3
1222 ; MSDOS 6.0
1223
1224 .FCB1: resb 16 ; 10h ; default FCB 1
1225 .FCB2: resb 16 ; 10h ; default FCB 2
1226 .PAD3: resb 4 ; not sure if this is used by PDB_FCB2
1227 .TAIL: resb 128 ; command tail and default DTA
1228 endstruc
1229
1230 ; =====
1231 ; EXE.INC, MSDOS 6.0, 1991
1232 ; =====
1233 ; 04/05/2019 - Retro DOS v4.0
1234
1235 ; ** EXE.INC - Definitions for the EXEC command and EXE files
1236 ; -----
1237 ; The following get used as arguments to the EXEC system call. They indicate
1238 ; whether or not the program is executed or whether or not a program header
1239 ; gets created.
1240

```

```

1241 exec_func_no_execute EQU 1; no execute bit
1242 exec_func_overlay EQU 2; overlay bit
1243
1244 struct EXEC0
1245 00000000 ???? .ENVIRON: resw 1 ; seg addr of environment
1246 00000002 ???????? .COM_LINE: resd 1 ; pointer to asciz command line
1247 00000006 ???????? .5C_FCB: resd 1 ; default fcb at 5C
1248 0000000A ???????? .6C_FCB: resd 1 ; default fcb at 6C
1249 .size:
1250 endstruc
1251
1252 struct EXEC1
1253 00000000 ???? .ENVIRON: resw 1 ; seg addr of environment
1254 00000002 ???????? .COM_LINE: resd 1 ; pointer to asciz command line
1255 00000006 ???????? .5C_FCB: resd 1 ; default fcb at 5C
1256 0000000A ???????? .6C_FCB: resd 1 ; default fcb at 6C
1257 0000000E ???? .SP: resw 1 ; stack pointer of program
1258 00000010 ???? .SS: resw 1 ; stack seg register of program
1259 00000012 ???? .IP: resw 1 ; entry point IP
1260 00000014 ???? .CS: resw 1 ; entry point CS
1261 .size:
1262 endstruc
1263
1264 struct EXEC3
1265 00000000 ???? .load_addr: resw 1 ; seg address of load point
1266 00000002 ???? .reloc_fac: resw 1 ; relocation factor
1267 endstruc
1268
1269 ;** Exit codes (in upper byte) for terminating programs
1270
1271 EXIT_TERMINATE EQU 0
1272 EXIT_ABORT EQU 0
1273 EXIT_CTRL_C EQU 1
1274 EXIT_HARD_ERROR EQU 2
1275 EXIT_KEEP_PROCESS EQU 3
1276
1277 ;** EXE File Header Description
1278
1279 struct EXE
1280 00000000 ???? .signature: resw 1 ; must contain 4D5A (yay zibo!)
1281 00000002 ???? .len_mod_512: resw 1 ; low 9 bits of length
1282 00000004 ???? .pages: resw 1 ; number of 512b pages in file
1283 00000006 ???? .rle_count: resw 1 ; count of reloc entries
1284 00000008 ???? .par_dir: resw 1 ; number of paragraphs before image
1285 0000000A ???? .min_BSS: resw 1 ; minimum number of para of BSS
1286 0000000C ???? .max_BSS: resw 1 ; max number of para of BSS
1287 0000000E ???? .SS: resw 1 ; stack of image
1288 00000010 ???? .SP: resw 1 ; SP of image
1289 00000012 ???? .chksum: resw 1 ; checksum of file (ignored)
1290 00000014 ???? .IP: resw 1 ; IP of entry
1291 00000016 ???? .CS: resw 1 ; CS of entry
1292 00000018 ???? .rle_table: resw 1 ; byte offset of reloc table
1293 0000001A ???? .iov: resw 1 ; overlay number (0 for root)
1294 0000001C ???????? .sym_tab: resd 1 ; offset of symbol table in file
1295 .size:
1296 endstruc
1297
1298 exe_valid_signature EQU 5A4Dh
1299 exe_valid_old_signature EQU 4D5Ah
1300
1301 ;** EXE file symbol info definitions
1302
1303 struct symbol_entry
1304 00000000 ???????? .value: resd 1
1305 00000004 ???? .type: resw 1
1306 00000006 ?? .len: resb 1
1307 00000007 <res FFh> .name: resb 255
1308 endstruc
1309
1310 ;** Data structure passed for ExecReady call
1311
1312 struct ERStruc
1313 00000000 ???? .ER_Reserved: resw 1 ; reserved, should be zero
1314 00000002 ???? .ER_Flags: resw 1
1315 00000004 ???????? .ER_ProgName: resd 1 ; ptr to ASCII str of prog name
1316 00000008 ???? .ER_PSP: resw 1 ; PSP of the program
1317 0000000A ???????? .ER_StartAddr: resd 1 ; Start CS:IP of the program
1318 0000000E ???????? .ER_ProgSize: resd 1 ; Program size including PSP
1319 .size:
1320 endstruc
1321
1322 ;** bit fields in ER_Flags
1323
1324 ER_EXE equ 0001h
1325 ER_OVERLAY equ 0002h
1326
1327
1328 ;=====
1329 ; ARENA.INC, MSDOS 6.0, 1991
1330 ;=====
1331 ; 24/04/2019 - Retro DOS v4.0
1332 ; 04/08/2018 - Retro DOS v3.0
1333
1334 ;BREAK <Memory arena structure>
1335
1336 ;** Arena Header
1337
1338 struct ARENA
1339 00000000 ?? .SIGNATURE: resb 1 ; 4D for valid item, 5A for last item
1340 00000001 ???? .OWNER: resw 1 ; owner of arena item
1341 00000003 ???? .SIZE: resw 1 ; size in paragraphs of item
1342 00000005 ?????? .RESERVED: resb 3 ; reserved
1343 00000008 ?????????????? .NAME: resb 8 ; owner file name
1344 .headersize:
1345 endstruc
1346
1347 ; 20/05/2019 - Retro DOS v4.0
1348 ARENAHEADERSIZE equ ARENA.headersize
1349
1350 ; CAUTION: The routines in ALLOC.ASM rely on the fact that arena_signature
1351 ; and arena_owner_system are all equal to zero and are contained in DI.
1352 ; change them and change ALLOC.ASM.
1353
1354 arena_owner_system EQU 0 ; free block indication
1355
1356 arena_signature_normal EQU 4Dh ; valid signature, not end of arena
1357 arena_signature_end EQU 5Ah ; valid signature, last block in arena
1358
1359 FIRST_FIT EQU 0000000B
1360 BEST_FIT EQU 00000001B
1361 LAST_FIT EQU 00000010B
1362
1363 ; MSDOS 6.0
1364 LOW_FIRST EQU 0000000B ; M001

```

```

1365 HIGH_FIRST EQU 10000000B ; M001
1366 HIGH_ONLY EQU 01000000B ; M001
1367
1368 LINKSTATE EQU 00000001B ; M002
1369
1370 HF_MASK EQU ~HIGH_FIRST ; M001
1371 HO_MASK EQU ~HIGH_ONLY ; M001
1372
1373 STRAT_MASK EQU HF_MASK & HO_MASK ; M001;
1374 ; M026: used to mask of bits
1375 ; M026: 6 & 7 of AllocMethod
1376
1377 ;=====
1378 ; MI.INC, MSDOS 6.0, 1991
1379 ;=====
1380 ; 07/07/2018 - Retro DOS v3.0
1381
1382 ;BREAK <Machine instruction, flag definitions and character types>
1383
1384 mi_INT EQU 0CDh
1385 mi_long_jump EQU 0EAh
1386 mi_Long_CALL EQU 09Ah
1387 mi_Long_RET EQU 0CBh
1388 mi_Near_RET EQU 0C3h
1389
1390 ;
1391 f_Overflow EQU xxxxoditszxaxpxc
1392 f_Direction EQU 0000100000000000B
1393 f_Interrupt EQU 0000010000000000B
1394 f_Trace EQU 0000000100000000B
1395 f_Sign EQU 0000000010000000B
1396 f_Zero EQU 0000000001000000B
1397 f_Aux EQU 0000000000010000B
1398 f_Parity EQU 000000000000100B
1399 f_Carry EQU 000000000000001B
1400
1401 ;=====
1402 ; FILEMODE.INC, MSDOS 6.0, 1991
1403 ;=====
1404 ; 13/07/2018 - Retro DOS v3.0
1405 ; 29/04/2019 - Retro DOS v4.0
1406
1407 ;** Standard I/O file handles
1408
1409 stdin EQU 0
1410 stdout EQU 1
1411 stderr EQU 2
1412 stdaux EQU 3
1413 stdprn EQU 4
1414
1415 ;** File Modes
1416 ; <Xenix subfunction assignments> ; MSDOS 3.3 FILEMODE.INC
1417
1418 open_for_read EQU 0
1419 open_for_write EQU 1
1420 open_for_both EQU 2
1421
1422 ; MSDOS 6.0
1423 OPEN_FOR_BOTH equ 2
1424 EXEC_OPEN equ 3 ; access code of 3 indicates that open was
1425 ; made from exec
1426
1427 access_mask EQU 0Fh ; 09/08/2018
1428
1429 SHARING_MASK equ 0F0h
1430 SHARING_COMPAT equ 000h
1431 SHARING_DENY_BOTH equ 010h
1432 SHARING_DENY_WRITE equ 020h
1433 SHARING_DENY_READ equ 030h
1434 SHARING_DENY_NONE equ 040h
1435 SHARING_NET_FCB equ 070h
1436 SHARING_NO_INHERIT equ 080h
1437
1438 ; 29/04/2019
1439
1440 ;** Extended Open Definitions
1441
1442 RESERVED_BITS_MASK equ 0FE00h ; reserved bits for extended open flags
1443 EXISTS_MASK equ 0Fh ; "file exists" action field
1444 NOT_EXISTS_MASK equ 0F0h
1445
1446 ;* SF_MODE values
1447
1448 AUTO_COMMIT_WRITE equ 4000h
1449 INT_24_ERROR equ 2000h
1450
1451 ;* Flags in EXTOPEN_ON
1452
1453 EXT_OPEN_ON equ 01h
1454 EXT_FILE_NOT_EXISTS equ 04h
1455 EXT_OPEN_I24_OFF equ 02h
1456
1457 ;* Flags in EXTOPEN_FLAG
1458
1459 ACTION_OPENED equ 01h
1460 ACTION_CREATED_OPENED equ 02h
1461 ACTION_REPLACED_OPENED equ 03h
1462 EXT_EXISTS_OPEN equ 01h
1463 EXT_EXISTS_FAIL equ 00h
1464 EXT_NEXISTS_CREATE equ 10h
1465
1466 ;** Extended Open Structure
1467
1468 struc EXT_OPEN_PARM
1469 00000000 ???????? .SET_LIST: resd 1
1470 00000004 ????. .NUM_OF_PARM: resw 1
1471 endstruc
1472
1473 ;=====
1474 ; SYSCALL.INC, MSDOS 6.0, 1991
1475 ;=====
1476 ; 29/04/2019 - Retro DOS v4.0
1477 ; 09/07/2018 - Retro DOS v3.0 (SYSCALL.INC, MSDOS 3.3, 1987)
1478
1479 ; <system call definitions>
1480
1481 ABORT EQU 0 ; 0 0
1482 STD_CON_INPUT EQU 1 ; 1 1
1483 STD_CON_OUTPUT EQU 2 ; 2 2
1484 STD_AUX_INPUT EQU 3 ; 3 3
1485 STD_AUX_OUTPUT EQU 4 ; 4 4
1486 STD_PRINTER_OUTPUT EQU 5 ; 5 5
1487 RAW_CON_IO EQU 6 ; 6 6
1488 RAW_CON_INPUT EQU 7 ; 7 7

```

1489	STD_CON_INPUT_NO_ECHO	EQU 8	; 8	8	
1490	STD_CON_STRING_OUTPUT	EQU 9	; 9	9	
1491	STD_CON_STRING_INPUT	EQU 10	; 10	A	
1492	STD_CON_INPUT_STATUS	EQU 11	; 11	B	
1493	STD_CON_INPUT_FLUSH	EQU 12	; 12	C	
1494	DISK_RESET	EQU 13	; 13	D	
1495	SET_DEFAULT_DRIVE	EQU 14	; 14	E	
1496	FCB_OPEN	EQU 15	; 15	F	
1497	FCB_CLOSE	EQU 16	; 16	10	
1498	DIR_SEARCH_FIRST	EQU 17	; 17	11	
1499	DIR_SEARCH_NEXT	EQU 18	; 18	12	
1500	FCB_DELETE	EQU 19	; 19	13	
1501	FCB_SEQ_READ	EQU 20	; 20	14	
1502	FCB_SEQ_WRITE	EQU 21	; 21	15	
1503	FCB_CREATE	EQU 22	; 22	16	
1504	FCB_RENAME	EQU 23	; 23	17	
1505	GET_DEFAULT_DRIVE	EQU 25	; 25	19	
1506	SET_DMA	EQU 26	; 26	1A	
1507					
1508					
1509					
1510	GET_DEFAULT_DPB	EQU 31	; 31	1F	
1511					
1512					
1513					
1514	FCB_RANDOM_READ	EQU 33	; 33	21	
1515	FCB_RANDOM_WRITE	EQU 34	; 34	22	
1516	GET_FCB_FILE_LENGTH	EQU 35	; 35	23	
1517	GET_FCB_POSITION	EQU 36	; 36	24	
1518	SET_INTERRUPT_VECTOR	EQU 37	; 37	25	
1519	CREATE_PROCESS_DATA_BLOCK	EQU 38	; 38	26	
1520	FCB_RANDOM_READ_BLOCK	EQU 39	; 39	27	
1521	FCB_RANDOM_WRITE_BLOCK	EQU 40	; 40	28	
1522	PARSE_FILE_DESCRIPTOR	EQU 41	; 41	29	
1523	GET_DATE	EQU 42	; 42	2A	
1524	SET_DATE	EQU 43	; 43	2B	
1525	GET_TIME	EQU 44	; 44	2C	
1526	SET_TIME	EQU 45	; 45	2D	
1527	SET_VERIFY_ON_WRITE	EQU 46	; 46	2E	
1528	; Extended functionality group				
1529	GET_DMA	EQU 47	; 47	2F	
1530	GET_VERSION	EQU 48	; 48	30	
1531	KEEP_PROCESS	EQU 49	; 49	31	
1532					
1533					
1534					
1535	GET_DPB	EQU 50	; 50	32	
1536					
1537					
1538					
1539	SET_CTRL_C_TRAPPING	EQU 51	; 51	33	
1540	GET_INDOS_FLAG	EQU 52	; 52	34	
1541	GET_INTERRUPT_VECTOR	EQU 53	; 53	35	
1542	GET_DRIVE_FREESPACE	EQU 54	; 54	36	
1543	CHAR_OPER	EQU 55	; 55	37	
1544	INTERNATIONAL	EQU 56	; 56	38	
1545	; XENIX CALLS				
1546	; Directory Group				
1547	MKDIR	EQU 57	; 57	39	
1548	RMDIR	EQU 58	; 58	3A	
1549	CHDIR	EQU 59	; 59	3B	
1550	; File Group				
1551	CREAT	EQU 60	; 60	3C	
1552	OPEN	EQU 61	; 61	3D	
1553	CLOSE	EQU 62	; 62	3E	
1554	READ	EQU 63	; 63	3F	
1555	WRITE	EQU 64	; 64	40	
1556	UNLINK	EQU 65	; 65	41	
1557	LSEEK	EQU 66	; 66	42	
1558	CHMOD	EQU 67	; 67	43	
1559	IOCTL	EQU 68	; 68	44	
1560	XDUP	EQU 69	; 69	45	
1561	XDUP2	EQU 70	; 70	46	
1562	CURRENT_DIR	EQU 71	; 71	47	
1563	; Memory Group				
1564	ALLOC	EQU 72	; 72	48	
1565	DEALLOC	EQU 73	; 73	49	
1566	SETBLOCK	EQU 74	; 74	4A	
1567	; Process Group				
1568	EXEC	EQU 75	; 75	4B	
1569	EXIT	EQU 76	; 76	4C	
1570	_WAIT	EQU 77	; 77	4D	
1571	FIND_FIRST	EQU 78	; 78	4E	
1572	; Special Group				
1573	FIND_NEXT	EQU 79	; 79	4F	
1574	; SPECIAL SYSTEM GROUP				
1575					
1576					
1577					
1578	SET_CURRENT_PDB	EQU 80	; 80	50	
1579	GET_CURRENT_PDB	EQU 81	; 81	51	

```

1613 XNAMETRANS EQU 96 ; 96 60
1614 PATHPARSE EQU 97 ; 97 61
1615 GETCURRENTPSP EQU 98 ; 98 62
1616 HONGEUL EQU 99 ; 99 63
1617 ;-----+-----+-----+-----+-----+-----+-----+-----+-----;
1618 ; C A V E A T P R O G R A M M E R ;
1619 ; ;
1620 SET_PRINTER_FLAG EQU 100 ; 100 64
1621 ; ;
1622 ; C A V E A T P R O G R A M M E R ;
1623 ;-----+-----+-----+-----+-----+-----+-----+-----+-----;
1624 GETEXTCNTRY EQU 101 ; 101 65
1625 GETSETCDPG EQU 102 ; 102 66
1626 EXTHANDLE EQU 103 ; 103 67
1627 COMMIT EQU 104 ; 104 68
1628
1629 ; 29/04/2019 - Retro DOS v4.0
1630 ; (MSDOS 6.0, SYSCALL.INC, 1987)
1631
1632 GetSetMediaID EQU 105 ; 105 69
1633 IFS_IOCTL EQU 107 ; 107 6B
1634 ExtOpen EQU 108 ; 108 6C
1635
1636 ;-----+-----+-----+-----+-----+-----+-----+-----+-----;
1637 ; C A V E A T P R O G R A M M E R ;
1638 ; ;
1639 ;ifdef ROMEXEC
1640 ;ROM_FIND_FIRST EQU 109 ; 109 6D
1641 ;ROM_FIND_NEXT EQU 110 ; 110 6E
1642 ;ROM_EXCLUDE EQU 111 ; 111 6F ; M035
1643 ;endif
1644 ; ;
1645 ; C A V E A T P R O G R A M M E R ;
1646 ;-----+-----+-----+-----+-----+-----+-----+-----+-----;
1647
1648 SET_OEM_HANDLER EQU 248 ; 248 F8
1649 ;OEM_C1 EQU 249 ; 249 F9
1650 ;OEM_C2 EQU 250 ; 250 FA
1651 ;OEM_C3 EQU 251 ; 251 FB
1652 ;OEM_C4 EQU 252 ; 252 FC
1653 ;OEM_C5 EQU 253 ; 253 FD
1654 ;OEM_C6 EQU 254 ; 254 FE
1655 ;OEM_C7 EQU 255 ; 255 FF
1656
1657 ; 01/01/2024 - Retro DOS v5.0 - PCDOS 7.1 IBMDOS.COM
1658 ; (PCDOS 7.1 extension to MSDOS 6.22 system calls)
1659
1660 ExtCountryInfo equ 112 ; 70h
1661 LONGNAME equ 113 ; 71h
1662 LFNFINDCLOSE equ 114 ; 72h
1663 FAT32EXT equ 115 ; 73h
1664
1665 ;=====
1666 ; VERSIONA.INC (MSDOS 6.0, 1991)
1667 ;=====
1668 ; 24/04/2019 - Retro DOS 4.0
1669
1670 ;MAJOR_VERSION EQU 6
1671 ;;MINOR_VERSION EQU 00
1672 ;MINOR_VERSION EQU 21 ; MSDOS 6.21
1673
1674 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
1675 ;MAJOR_VERSION EQU 5
1676 ;MINOR_VERSION EQU 0
1677
1678 ; 30/12/2022 - Retro DOS v4.2
1679 ;MAJOR_VERSION EQU 6
1680 ;MINOR_VERSION EQU 22
1681
1682 ; 01/01/2024 - Retro DOS v5.0
1683 MAJOR_VERSION EQU 7
1684 MINOR_VERSION EQU 10 ; PCDOS 7.1 (7.10)
1685
1686 ;=====
1687 ; INTNAT.INC, MSDOS 3.3, 1987
1688 ;=====
1689 ; 09/07/2018 - Retro DOS 3.0
1690
1691 ; Current structure of the data returned by the international call
1692
1693 struc INTERNAT_BLOCK ; (-*-) Same with MSDOS 2.11 & MSDOS 6.0
1694 .Date_tim_format:
1695 RESW 1 ; 0-USA, 1-EUR, 2-JAP
1696
1697 .Currency_sym:
1698 RESB 5 ; Currency Symbol 5 bytes
1699
1700 .Thous_sep:
1701 RESB 2 ; Thousands separator 2 bytes
1702
1703 .Decimal_sep:
1704 RESB 2 ; Decimal separator 2 bytes
1705
1706 .Date_sep:
1707 RESB 2 ; Date separator 2 bytes
1708
1709 .Time_sep:
1710 RESB 2 ; Time separator 2 bytes
1711
1712 .Bit_field:
1713 RESB 1 ; Bit values
1714 ; ; Bit 0 = 0 if currency symbol first
1715 ; ; ; Bit 1 = 1 if currency symbol last
1716 ; ; ; Bit 1 = 0 if No space after currency symbol
1717 ; ; ; = 1 if space after currency symbol
1718
1719 .Currency_cents:
1720 RESB 1 ; Number of places after currency dec point
1721
1722 .Time_24:
1723 RESB 1 ; 1 if 24 hour time, 0 if 12 hour time
1724
1725 .Map_call:
1726 RESW 1 ; Address of case mapping call (DWORD)
1727 RESW 1 ; THIS IS TWO WORDS SO IT CAN BE INITIALIZED
1728 ; in pieces.
1729
1730 .Data_sep:
1731 RESB 2 ; Data list separator character
1732
1733 .size:
1734 endstruc
1735
1736 ; Max size of the block returned by the INTERNATIONAL call
1737
1738 internat_block_max EQU 32
1739
1740 ;=====
1741 ; SYSVAR.INC (MSDOS 6.0, 1991)
1742 ;=====
1743 ; 08/07/2018 - Retro DOS v3.0
1744
1745 ; 01/01/2024 - Retro DOS v5.0
1746
1747 ;SysInitVars STRUC

```



```

1737 struct SYSI
1738 .DPB:      resd 1      ; 0      ; DPB chain
1739 .SFT:      resd 1      ; 4      ; SFT chain
1740 .CLOCK:    resd 1      ; 8      ; CLOCK device
1741 .CON:      resd 1      ; 12     ; CON device
1742 .MAXSEC:   resw 1 ; 16 ; maximum sector size
1743 .BUF:      resd 1      ; 18     ; points to Hashinitvar
1744 .CDS:      resd 1      ; 22     ; CDS list
1745 .FCB:      resd 1      ; 26     ; FCB chain
1746 .Keep:     resw 1      ; 30     ; keep count
1747 .NUMIO:    resb 1      ; 32     ; Number of block devices
1748 .NCDS:     resb 1      ; 33     ; number of CDS's
1749 .DEV:      resd 1      ; 34     ; device list
1750 ; 09/07/2018
1751 ; Above parameters are described in MSDOS 3.3 SYSVAR.INC (85/04/10)
1752 ; Following parameters are used with MSDOS 6.0 (Retro DOS v4.0)
1753 .ATTR:     resw 1      ; 38     ; null device attribute word
1754 .STRAT:     resw 1      ; 40     ; null device strategy entry point
1755 .INTER:     resw 1      ; 42     ; null device interrupt entry point
1756 .NAME:      resb 8      ; 44     ; null device name
1757 .SPLICE:    resb 1 ; 52 ; TRUE -> splicees being done
1758 .IBMDOS_SIZE: resw 1 ; 53 ; DOS size in paragraphs
1759 .IFS_DOSCALL@: resd 1 ; 55 ; IFS DOS service routine entry
1760 .IFS:       resd 1      ; 59     ; IFS header chain
1761 .BUFFERS:   resw 2 ; 63 ; BUFFERS= values (m,n)
1762 .BOOT_DRIVE: resb 1      ; 67     ; boot drive A=1 B=2,..
1763 .DWMOVE:    resb 1 ; 68 ; 1 if 386 machine
1764 .EXT_MEM:   resw 1 ; 69 ; Extended memory size in KB.
1765 endstruc
1766 ;SysInitVars ENDS
1767
1768 ;This is added for more information exchange between DOS, BIOS.
1769 ;DOS will give the pointer to SysInitTable in ES:DI. - J.K. 5/29/86
1770
1771 ;SysInitVars_Ext struc
1772 struc SYSI_EXT
1773 .SysInitVars:      resd 1      ; Points to the above structure.
1774 .Country_Tab:      resd 1      ; DOS_Country_cdpq_info
1775 endstruc
1776 ;SysInitVars_Ext ends
1777
1778 ;=====
1779 ; IOCTL.INC - MSDOS 6.0 - 1991
1780 ;=====
1781 ; 09/07/2018 - Retro DOS v3.0
1782
1783 ;*** J.K.
1784 ;General Guide -
1785 ;Category Code:
1786 ; 0... .... DOS Defined
1787 ; 1... .... User defined
1788 ; .xxx xxxx Code
1789
1790 ;Function Code:
1791 ; 0... .... Return error if unsupported
1792 ; 1... .... Ignore if unsupported
1793 ; .0.. .... Intercepted by DOS
1794 ; .1.. .... Passed to driver
1795 ; ..0. .... Sends data/commands to device
1796 ; ..1. .... Queries data/info from device
1797 ; ...x .... Subfunction
1798 ;
1799 ; Note that "Sends/queries" data bit is intended only to regularize the
1800 ; function set. It plays no critical role; some functions may contain both
1801 ; command and query elements. The convention is that such commands are
1802 ; defined as "sends data".
1803
1804 ;*****,*
1805 ; BLOCK DRIVERS ;*
1806 ;*****,*
1807
1808 ; IOCTL SUB-FUNCTIONS
1809 ; (MSDOS 3.3 + MSDOS 6.0)
1810 IOCTL_GET_DEVICE_INFO EQU 0
1811 IOCTL_SET_DEVICE_INFO EQU 1
1812 IOCTL_READ_HANDLE EQU 2
1813 IOCTL_WRITE_HANDLE EQU 3
1814 IOCTL_READ_DRIVE EQU 4
1815 IOCTL_WRITE_DRIVE EQU 5
1816 IOCTL_GET_INPUT_STATUS EQU 6
1817 IOCTL_GET_OUTPUT_STATUS EQU 7
1818 IOCTL_CHANGEABLE? EQU 8
1819 IOCTL_DeviceLocOrRem? EQU 9
1820 IOCTL_HandleLocOrRem? EQU 0Ah ;10
1821 IOCTL_SHARING_RETRY EQU 0Bh ;11
1822 GENERIC_IOCTL_HANDLE EQU 0Ch ;12
1823 GENERIC_IOCTL EQU 0Dh ;13
1824 ; (MSDOS 6.0 + MSDOS 3.3)
1825 IOCTL_GET_DRIVE_MAP EQU 0Eh ;14
1826 IOCTL_SET_DRIVE_MAP EQU 0Fh ;15
1827 ; (MSDOS 6.0)
1828 IOCTL_QUERY_HANDLE EQU 10h ;16
1829 IOCTL_QUERY_BLOCK EQU 11h ;17
1830
1831 ; GENERIC IOCTL CATEGORY CODES
1832 IOC_OTHER EQU 0 ; other device control J.K. 4/29/86
1833 IOC_SE EQU 1 ; SERIAL DEVICE CONTROL
1834 IOC_TC EQU 2 ; TERMINAL CONTROL
1835 IOC_SC EQU 3 ; SCREEN CONTROL
1836 IOC_KC EQU 4 ; KEYBOARD CONTROL
1837 IOC_PC EQU 5 ; PRINTER CONTROL
1838 IOC_DC EQU 8 ; DISK CONTROL (SAME AS RAWIO)
1839
1840 ; GENERIC IOCTL SUB-FUNCTIONS
1841 RAWIO EQU 8
1842
1843 ; RAWIO SUB-FUNCTIONS
1844 ; (MSDOS 3.3 + MSDOS 6.0)
1845 GET_DEVICE_PARAMETERS EQU 60H
1846 SET_DEVICE_PARAMETERS EQU 40H
1847 READ_TRACK EQU 61H
1848 WRITE_TRACK EQU 41H
1849 VERIFY_TRACK EQU 62H
1850 FORMAT_TRACK EQU 42H
1851 ; (MSDOS 6.0)
1852 GET_MEDIA_ID EQU 66h ;AN000;AN003;changed from 63h
1853 SET_MEDIA_ID EQU 46h ;AN000;AN003;changed from 43h
1854 GET_ACCESS_FLAG EQU 67h ;AN002;AN003;Unpublished function.Changed from 64h
1855 SET_ACCESS_FLAG EQU 47h ;AN002;AN003;unpublished function.Changed from 44h
1856 SENSE_MEDIA_TYPE EQU 68H ;Added for 5.00
1857
1858 ; SPECIAL FUNCTION FOR GET DEVICE PARAMETERS
1859 BUILD_DEVICE_BPB EQU 00000001B
1860

```

```

1861 ; SPECIAL FUNCTIONS FOR SET DEVICE PARAMETERS
1862 INSTALL_FAKE_BPB EQU 000000001B
1863 ONLY_SET_TRACKLAYOUT EQU 000000010B
1864 TRACKLAYOUT_IS_GOOD EQU 000000100B
1865
1866 ; SPECIAL FUNCTION FOR FORMAT TRACK
1867 ; (MSDOS 3.3 + MSDOS 6.0)
1868 STATUS_FOR_FORMAT EQU 000000001B
1869 ; (MSDOS 6.0)
1870 DO_FAST_FORMAT EQU 000000010B ;AN001;
1871
1872 ; CODES RETURNED FROM FORMAT STATUS CALL
1873 FORMAT_NO_ROM_SUPPORT EQU 000000001B
1874 FORMAT_COMB_NOT_SUPPORTED EQU 000000010B
1875
1876 ; DEVICETYPE VALUES
1877 ; (MSDOS 3.3 + MSDOS 6.0)
1878 MAX_SECTORS_IN_TRACK EQU 63 ; MAXIMUM SECTORS ON A DISK.(was 40 in DOS 3.2)
1879 DEV_5INCH EQU 0
1880 DEV_5INCH96TPI EQU 1
1881 DEV_3INCH720KB EQU 2
1882 DEV_8INCHSS EQU 3
1883 DEV_8INCHDS EQU 4
1884 DEV_HARDDISK EQU 5
1885 DEV_OTHER EQU 7
1886 ; (MSDOS 6.0)
1887 ;DEV_3INCH1440KB EQU 7
1888 DEV_3INCH2880KB EQU 9
1889 ; Retro DOS v2.0 - 26/03/2018
1890 ;;DEV_TAPE EQU 6
1891 ;;DEV_ERIMO EQU 8
1892 ;DEV_3INCH2880KB EQU 9
1893 DEV_3INCH1440KB EQU 10
1894
1895 ; (MSDOS 3.3)
1896 ;MAX_DEV_TYPE EQU 7
1897
1898 ; (MSDOS 6.0)
1899 MAX_DEV_TYPE EQU 10 ; MAXIMUM DEVICE TYPE THAT WE
1900 ; CURRENTLY SUPPORT.
1901
1902 00000000 ???? struc A_SECTORTABLE
1903 00000002 ???? .ST_SECTORNUMBER: resw 1
1904 .ST_SECTORSIZE: resw 1
1905 .size:
1906 endstruc
1907
1908 ;=====
1909 ; DEVSYS.INC
1910 ;=====
1911 ; 07/07/2018 - Retro DOS v3.0
1912 ; 30/04/2019 - Retro DOS v4.0 (DEVSYS.INC, MSDOS 6.0, 1991)
1913 ; 21/02/2024 - Retro DOS v5.0
1914
1915 ;** DevSym.inc - Device Symbols
1916
1917 ; The device table list has the form:
1918 struc SYSDEV
1919 00000000 ???????? .NEXT: resd 1 ;Pointer to next device header
1920 00000004 ???? .ATT: resw 1 ;Attributes of the device
1921 00000006 ???? .STRAT: resw 1 ;Strategy entry point
1922 00000008 ???? .INT: resw 1 ;Interrupt entry point
1923 0000000A ???????????????? .NAME: resb 8 ;Name of device (only first byte used for block)
1924 .size:
1925 endstruc
1926
1927 ;
1928 ; ATTRIBUTE BIT MASKS
1929 ;
1930 ; CHARACTER DEVICES:
1931 ;
1932 ; BIT 15 -> MUST BE 1
1933 ; 14 -> 1 IF THE DEVICE UNDERSTANDS IOCTL CONTROL STRINGS
1934 ; 13 -> 1 IF THE DEVICE SUPPORTS OUTPUT-UNTIL-BUSY
1935 ; 12 -> UNUSED
1936 ; 11 -> 1 IF THE DEVICE UNDERSTANDS OPEN/CLOSE
1937 ; 10 -> MUST BE 0
1938 ; 9 -> MUST BE 0
1939 ; 8 -> UNUSED
1940 ; 7 -> UNUSED
1941 ; 6 -> UNUSED
1942 ; 5 -> UNUSED
1943 ; 4 -> 1 IF DEVICE IS RECIPIENT OF INT 29H
1944 ; 3 -> 1 IF DEVICE IS CLOCK DEVICE
1945 ; 2 -> 1 IF DEVICE IS NULL DEVICE
1946 ; 1 -> 1 IF DEVICE IS CONSOLE OUTPUT
1947 ; 0 -> 1 IF DEVICE IS CONSOLE INPUT
1948
1949 ; BLOCK DEVICES:
1950 ;
1951 ; BIT 15 -> MUST BE 0
1952 ; 14 -> 1 IF THE DEVICE UNDERSTANDS IOCTL CONTROL STRINGS
1953 ; 13 -> 1 IF THE DEVICE DETERMINES MEDIA BY EXAMINING THE FAT ID BYTE.
1954 ; THIS REQUIRES THE FIRST SECTOR OF THE FAT TO *ALWAYS* RESIDE IN
1955 ; THE SAME PLACE.
1956 ; 12 -> UNUSED
1957 ; 11 -> 1 IF THE DEVICE UNDERSTANDS OPEN/CLOSE/REMOVABLE MEDIA
1958 ; 10 -> MUST BE 0
1959 ; 9 -> MUST BE 0
1960 ; 8 -> UNUSED
1961 ; 7 -> UNUSED
1962 ; 6 -> IF DEVICE HAS SUPPORT FOR GETMAP/SETMAP OF LOGICAL DRIVES.
1963 ; IF THE DEVICE UNDERSTANDS GENERIC IOCTL FUNCTION CALLS.
1964 ; 5 -> UNUSED
1965 ; 4 -> UNUSED
1966 ; 3 -> UNUSED
1967 ; 2 -> UNUSED
1968 ; 1 -> UNUSED
1969 ; 0 -> UNUSED
1970
1971 ;Attribute bit masks
1972 DEVTYP EQU 8000H ;Bit 15 - 1 if char, 0 if block
1973 DEVIOCTL EQU 4000H ;Bit 14 - CONTROL mode bit
1974 ISFATBYDEV EQU 2000H ;Bit 13 - Device uses FAT ID bytes, comp media.
1975
1976 ; 09/07/2018 - Retro DOS (DEVSYS.INC, MSDOS 3.3, 1987)
1977
1978 OUTTILBUSY EQU 2000H ; OUTPUT UNTIL BUSY IS ENABLED
1979 ISNET EQU 1000H ; BIT 12 - 1 IF A NET DEVICE, 0 IF
1980 ; NOT. CURRENTLY BLOCK ONLY.
1981 DEVOPCL EQU 0800H ; BIT 11 - 1 IF THIS DEVICE HAS
1982 ; OPEN,CLOSE AND REMOVABLE MEDIA
1983 ; ENTRY POINTS, 0 IF NOT
1984

```



```

1985 EXTENTBIT EQU 0400H ; BIT 10 - CURRENTLY 0 ON ALL DEVS
1986 ; THIS BIT IS RESERVED FOR FUTURE USE
1987 ; TO EXTEND THE DEVICE HEADER BEYOND
1988 ; ITS CURRENT FORM.
1989
1990 ; NOTE BIT 9 IS CURRENTLY USED ON IBM SYSTEMS TO INDICATE "DRIVE IS SHARED".
1991 ; SEE IOCTL FUNCTION 9. THIS USE IS NOT DOCUMENTED, IT IS USED BY SOME
1992 ; OF THE UTILITIES WHICH ARE SUPPOSED TO FAIL ON SHARED DRIVES ON SERVER
1993 ; MACHINES (FORMAT,CHKDSK,RECOVER,...).
1994
1995 IOQUERY EQU 0080H ;Bit 7 - Supports generic IOCTL query
1996
1997 DEV320 EQU 0040H ;BIT 6 - FOR BLOCK DEVICES, THIS
1998 ;DEVICE SUPPORTS SET/GET MAP OF
1999 ;LOGICAL DRIVES, AND SUPPORTS
2000 ;GENERIC IOCTL CALLS.
2001 ;FOR CHARACTER DEVICES, THIS
2002 ;DEVICE SUPPORTS GENERIC IOCTL.
2003 ;THIS IS A DOS 3.2 DEVICE DRIVER.
2004
2005 ISSPEC EQU 0010H ;Bit 4 - This device is special ; 15/03/2018
2006 ;ISIBM EQU 0010H ;Bit 4 - This device is special
2007 ISCLOCK EQU 0008H ;Bit 3 - This device is the clock device.
2008 ISNULL EQU 0004H ;Bit 2 - This device is the null device.
2009 ISCOU EQU 0002H ;Bit 1 - This device is the console output.
2010 ISCI EQU 0001H ;Bit 0 - This device is the console input.
2011
2012 EXTDRV EQU 0002h ;BIT 1 - BLOCK DEVICE EXTENDED DRIVER
2013 ; (MSDOS 6.0, DEVSYM.INC, 1991) ; 30/04/2019
2014
2015 ;Static Request Header
2016 struc SRHEAD
2017 .REQLEN: resb 1 ;Length in bytes of request block
2018 .REQUNIT: resb 1 ;Device unit number
2019 .REQFUNC: resb 1 ;Type of request
2020 .REQSTAT: resw 1 ;Status word
2021 .RESERVED: resb 8 ;Reserved for queue links
2022 .size:
2023 endstruc
2024
2025 ;Status word masks
2026 STERR EQU 8000H ;Bit 15 - Error
2027 STBUI EQU 0200H ;Bit 9 - Busy
2028 STDON EQU 0100H ;Bit 8 - Done
2029 STECODE EQU 00FFH ;Error code
2030 WRECODE EQU 0
2031
2032 ;Function codes
2033 ;DINITHL EQU 26 ;Size of init header
2034 ; 11/04/2024 - Retro DOS 5.0
2035 DINITHL EQU 25 ; PCDOS 7.1 ;Size of init header
2036 DMEDHL EQU 15 ;Size of media check header
2037 DBPBHL EQU 22 ;Size of Get BPB header
2038 DRDWRHL EQU 22 ;Size of RD/WR header
2039 DRDNDHL EQU 14 ;Size of non destructive read header
2040 DSTATHL EQU 13 ;Size of status header
2041 ; 21/02/2024
2042 ;DFLSHL EQU 15 ;Size of flush header
2043 DFLSHL EQU 13 ; PCDOS 7.1 IBMDOS.COM ; 21/02/2024
2044
2045 DEVINIT EQU 0 ;Initialization
2046 DEVMDCH EQU 1 ;Media check
2047 DEVBPB EQU 2 ;Get BPB
2048 DEVRDIOCTL EQU 3 ;IOCTL read
2049 DEVRD EQU 4 ;Read
2050 DEVRDND EQU 5 ;Non destructive read no wait (character devs)
2051 DEVIST EQU 6 ;Input status
2052 DEVIFL EQU 7 ;Input flush
2053 DEVRT EQU 8 ;write
2054 DEVRTV EQU 9 ;write with verify
2055 DEVOST EQU 10 ;Output status
2056 DEVOFL EQU 11 ;Output flush
2057 DEVRIOCTL EQU 12 ;IOCTL write
2058
2059 ; 09/07/2018 - Retro DOS v3.0 (DEVSYM.INC, MSDOS 3.3, 1987)
2060 DEVOPN EQU 13 ;DEVICE OPEN
2061 DEVCLS EQU 14 ;DEVICE CLOSE
2062 DOPCLHL EQU 13 ;SIZE OF OPEN/CLOSE HEADER
2063 DEVRMD EQU 15 ;REMOVABLE MEDIA
2064 ; 07/08/2018 - Retro DOS v3.0
2065 REMHL EQU 13 ;SIZE OF REMOVABLE MEDIA HEADER
2066 GENIOCTL EQU 19
2067
2068 ; THE NEXT THREE ARE USED IN DOS 4.0
2069 ; 20
2070 ; 21
2071 ; 22
2072
2073 DEVGETOWN EQU 23 ;GET DEVICE OWNER
2074 DEVSETOWN EQU 24 ;SET DEVICE OWNER
2075 ; 18/05/2019 - Retro DOS v4.0
2076 IOCTL_QUERY EQU 25 ;Query generic ioctl support
2077
2078 OWNHL EQU 13 ;SIZE OF DEVICE OWNER HEADER
2079
2080 DEVOUT EQU 16 ; OUTPUT UNTIL BUSY.
2081 DEVOUTL EQU DEVRT ; LENGTH OF OUTPUT UNTIL BUSY
2082
2083 ; ADDED FOR DOS 5.00
2084
2085 ; GENERIC IOCTL REQUEST STRUCTURE
2086 ; SEE THE DOS 4.0 DEVICE DRIVER SPEC FOR FURTHER ELABORATION.
2087
2088 struc IOCTL_REQ
2089 .SRHEAD: resb SRHEAD.size ; GENERIC IOCTL ADDITION.
2090 .MAJORFUNCTION: resb 1 ;FUNCTION CODE
2091 .MINORFUNCTION: resb 1 ;FUNCTION CATEGORY
2092 .REG_SI: resw 1
2093 .REG_DI: resw 1
2094 .GENERICIOCTL_PACKET: resd 1 ; POINTER TO DATA BUFFER
2095 .size: ; 07/08/2018
2096 endstruc
2097
2098 ; DEFINITIONS FOR IOCTL_REQ.MINORFUNCTION
2099 GEN_IOCTL_WRT_TRK EQU 40H
2100 GEN_IOCTL_RD_TRK EQU 60H
2101 GEN_IOCTL_FN_TST EQU 20H ; USED TO DIFF. BET READS AND WRTS
2102
2103 ;; 32-bit absolute read/write input list structure
2104
2105 struc ABS_32RW
2106 .SECTOR_RBA: resd 1 ; relative block address
2107 .ABS_RW_COUNT: resw 1 ; number of sectors to be transferred
2108

```

```

2109 00000006 ???????? .BUFFER_ADDR:      resd 1      ; data address
2110 .size:
2111 endstruc
2112
2113 ;; media ID info
2114
2115 struc MEDIA_ID_INFO
2116 00000000000000000000 .MEDIA_level:      resw 1      ; info level
2117 000000002 ?????????? .MEDIA_Serial:      resd 1      ; serial #
2118 000000006 <res Bh> .MEDIA_Label:      resb 11     ; volume label
2119 000000011 ?????????? .MEDIA_System:      resb 8      ; system type
2120 .size:
2121 endstruc
2122
2123 ; equates for DOS34_FLAG
2124 ; (BUGBUG: why are bits 0,1,3 and 4 not defined.)
2125
2126 FROM_DISK_RESET      EQU 000000000100b ;from disk reset
2127 Force_I24_Fail      EQU 000000100000b ;form IFS CALL BACK
2128 Disable_EOF_I24     EQU 000001000000b ;disable EOF int24 for input status
2129 DBCS_VOLID          EQU 000010000000b ;indicate from volume id
2130 DBCS_VOLID2         EQU 000100000000b ;indicate 8th char is DBCS
2131 CTRL_BREAK_FLAG     EQU 001000000000b ;indicate control break is input
2132 SEARCH_FASTOPEN     EQU 010000000000b ;set fastopen flag for search
2133 EXEC_AWARE_REDIRE   EQU 100000000000b ;M018: this bit is set by a redir
2134 ;M018: that knows how to handle
2135 ;M018: open for exec
2136
2137 NO_FROM_DISK_RESET   EQU ~FROM_DISK_RESET ;not from disk reset
2138 NO_Force_I24_Fail    EQU ~Force_I24_Fail  ;not form IFS CALL BACK
2139 NO_Disable_EOF_I24   EQU ~Disable_EOF_I24
2140
2141 ;=====
2142 ; ERROR.INC (MSDOS 6.0, 1991)
2143 ;=====
2144 ; 16/07/2018 - Retro DOS v3.0
2145
2146 ** ERROR.INC - DOS Error Codes
2147
2148 ; The newer (DOS 2.0 and above) "XENIX-style" calls
2149 ; return error codes through AX. If an error occurred then
2150 ; the carry bit will be set and the error code is in AX. If no error
2151 ; occurred then the carry bit is reset and AX contains returned info.
2152
2153 ; Since the set of error codes is being extended as we extend the operating
2154 ; system, we have provided a means for applications to ask the system for a
2155 ; recommended course of action when they receive an error.
2156
2157 ; The GetExtendedError system call returns a universal error, an error
2158 ; location and a recommended course of action. The universal error code is
2159 ; a symptom of the error REGARDLESS of the context in which GetExtendedError
2160 ; is issued.
2161
2162 ; 2.0 error codes
2163
2164 error_invalid_function      EQU 1
2165 error_file_not_found        EQU 2
2166 error_path_not_found        EQU 3
2167 error_too_many_open_files   EQU 4
2168 error_access_denied         EQU 5
2169 error_invalid_handle        EQU 6
2170 error_arena_trashed         EQU 7
2171 error_not_enough_memory     EQU 8
2172 error_invalid_block         EQU 9
2173 error_bad_environment        EQU 10
2174 error_bad_format            EQU 11
2175 error_invalid_access        EQU 12
2176 error_invalid_data          EQU 13
2177 ;**** reserved              EQU 14 ; ****
2178 error_invalid_drive          EQU 15
2179 error_current_directory     EQU 16
2180 error_not_same_device        EQU 17
2181 error_no_more_files          EQU 18
2182
2183 ; These are the universal int 24 mappings for the old INT 24 set of errors
2184
2185 error_write_protect          EQU 19
2186 error_bad_unit               EQU 20
2187 error_not_ready              EQU 21
2188 error_bad_command            EQU 22
2189 error_CRC                    EQU 23
2190 error_bad_length             EQU 24
2191 error_seek                   EQU 25
2192 error_not_DOS_disk           EQU 26
2193 error_sector_not_found       EQU 27
2194 error_out_of_paper           EQU 28
2195 error_write_fault            EQU 29
2196 error_read_fault             EQU 30
2197 error_gen_failure            EQU 31
2198
2199 ; the new 3.0 error codes reported through INT 24
2200
2201 error_sharing_violation      EQU 32
2202 error_lock_violation         EQU 33
2203 error_wrong_disk             EQU 34
2204 error_FCB_unavailable        EQU 35
2205 error_sharing_buffer_exceeded EQU 36
2206 error_Code_Page_Mismatched   EQU 37 ; DOS 4.00 ;AN000;
2207 error_handle_EOF             EQU 38 ; DOS 4.00 ;AN000;
2208 error_handle_Disk_Full       EQU 39 ; DOS 4.00 ;AN000;
2209
2210 ; New OEM network-related errors are 50-79
2211
2212 error_not_supported           EQU 50
2213
2214 error_net_access_denied       EQU 65 ;M028
2215
2216 ; End of INT 24 reportable errors
2217
2218 error_file_exists             EQU 80
2219 error_DUP_FCB                 EQU 81 ; ****
2220 error_cannot_make             EQU 82
2221 error_FAIL_I24                EQU 83
2222
2223 ; New 3.0 network related error codes
2224
2225 error_out_of_structures       EQU 84
2226 error_already_assigned        EQU 85
2227 error_invalid_password        EQU 86
2228 error_invalid_parameter       EQU 87
2229 error_NET_write_fault         EQU 88
2230 error_sys_comp_not_loaded     EQU 90 ; DOS 4.00 ;AN000;
2231
2232 ; BREAK <Interrupt 24 error codes>

```

```

2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356

; ** Int24 Error Codes
error_I24_write_protect EQU 0
error_I24_bad_unit EQU 1
error_I24_not_ready EQU 2
error_I24_bad_command EQU 3
error_I24_CRC EQU 4
error_I24_bad_length EQU 5
error_I24_Seek EQU 6
error_I24_not_DOS_disk EQU 7
error_I24_sector_not_found EQU 8
error_I24_out_of_paper EQU 9
error_I24_write_fault EQU 0Ah
error_I24_read_fault EQU 0Bh
error_I24_gen_failure EQU 0Ch
; NOTE: Code 0DH is used by MT-DOS.
error_I24_wrong_disk EQU 0Fh

; THE FOLLOWING ARE MASKS FOR THE AH REGISTER ON Int 24
;
; NOTE: ABORT is ALWAYS allowed
Allowed_FAIL EQU 00001000B
Allowed_RETRY EQU 00010000B
Allowed_IGNORE EQU 00100000B

I24_operation EQU 00000001B ;Z if READ,NZ if Write
I24_area EQU 00000110B ; 00 if DOS
; 01 if FAT
; 10 if root DIR
; 11 if DATA
I24_class EQU 10000000B ;Z if DISK, NZ if FAT or char

; BREAK <GetExtendedError CLASSEs ACTIONs LOCUSs>

; ** The GetExtendedError call takes an error code and returns CLASS,
; ACTION and LOCUS codes to help programs determine the proper action
; to take for error codes that they don't explicitly understand.

; Values for error CLASS
errCLASS_OutRes EQU 1 ; Out of Resource
errCLASS_TempSit EQU 2 ; Temporary Situation
errCLASS_Auth EQU 3 ; Permission problem
errCLASS_Intrn EQU 4 ; Internal System Error
errCLASS_HrdFail EQU 5 ; Hardware Failure
errCLASS_SysFail EQU 6 ; System Failure
errCLASS_Apperr EQU 7 ; Application Error
errCLASS_NotFnd EQU 8 ; Not Found
errCLASS_BadFmt EQU 9 ; Bad Format
errCLASS_Locked EQU 10 ; Locked
errCLASS_Media EQU 11 ; Media Failure
errCLASS_Already EQU 12 ; Collision with Existing Item
errCLASS_Unk EQU 13 ; Unknown/other

; Values for error ACTION
errACT_Retry EQU 1 ; Retry
errACT_DlyRet EQU 2 ; Delay Retry, retry after pause
errACT_User EQU 3 ; Ask user to regive info
errACT_Abort EQU 4 ; abort with clean up
errACT_Panic EQU 5 ; abort immediately
errACT_Ignore EQU 6 ; ignore
errACT_IntRet EQU 7 ; Retry after User Intervention

; Values for error LOCUS
errLOC_Unk EQU 1 ; No appropriate value
errLOC_Disk EQU 2 ; Random Access Mass Storage
errLOC_Net EQU 3 ; Network
errLOC_SerDev EQU 4 ; Serial Device
errLOC_Mem EQU 5 ; Memory

;=====
; INT2A.INC (MSDOS 6.0, 1991)
;=====
; 04/05/2019 - Retro DOS v4.0

; ** Int 2A functions
; -----
; Int 2A is an interface to the network code; it's also overloaded
; as a critical section handler since critical sections
; were originally created to support the net.
; -----

; -----
; ** This table was created by examining the source and may not be
; complete or completely accurate - JGL
;
; M010 MD 8/31/90 - Added definition for AH = 5
;
; (ah) = 0 installation check
; (returns ah !=0 if installed)
; (ah) = 1 cooked net bios call
; (ah) = 3 query drive shared
; (ds:si) = "n:" asciz string
; (ah) = 4 net bios
; (al) = 0 cooked net bios call
; (al) = 1 raw net bios call
; (al) = 2 ???
;
; (ah) = 5 Get Net Adaptor Resources. CX returns the number of
; NCBS available/outstanding. DX returns the number of
; sessions. Supposedly, this is documented in an old
; IBM PC-LAN reference. Lotus Notes uses it. DOS LAN
; Manager 2.0 Enhanced responds to it. But it should
; not be used, as it is a hack, only to get Lotus
; Notes running.
;
; (ah) = 80h enter critical section
; (ah) = 81h leave critical section
; (ah) = 82h free all critical sections (Leave-all)
; (ah) = 84h entering idle loop (don't understand how this works)
; -----

; ** Critical section definitions
; -----
; Although DOS is not designed to be reentrant there are some hacks
; which various programs use to make it so, in a limited fashion.
; Both WIN386 and some servers block copy a section of the DOS data
; area so that DOS can be reentered on behalf of another thread/program.
; DOS's global data structures, such as the memory arena, are not
; in this area, so critical section indicators are used to protect

```

```

2357 ; those areas. DOS flags a critical section by issuing an INT_IBM
2358 ; (int 2Ah) at each critical section entry and exit. Some clients
2359 ; (such as WIN386) just don't "context switch" the DOS when one
2360 ; of these is in effect, others, such as the IBM server, go ahead
2361 ; and reenter the DOS and if they get an int 2A to reenter the same
2362 ; critical section they then switch away from that second thread and
2363 ; let the first one finish and exit the section.
2364 ; -----
2365 ;
2366 ; These below are subject to leave-all sections
2367 critDisk EQU 1 ; Disk I/O critical section
2368 critShare EQU 1 ; Sharer I/O critical section
2369 critMem EQU 1 ; memory maintenance critical section
2370 critSFT EQU 1 ; sft table allocation
2371 critDevice EQU 2 ; Device I/O critical section
2372 critNet EQU 5 ; network critical section
2373 critIFS EQU 6 ; ifsfunc critical section
2374 ; These below are not subject to leave-all sections
2375 critASSIGN EQU 8 ; Assign has munged a system call
2376 ;
2377 ;=====
2378 ; MULT.INC (MSDOS 6.0, 1991)
2379 ;=====
2380 ; 04/05/2019 - Retro DOS v4.0
2381 ;
2382 ;Break <Multiplex channels>
2383 ;
2384 ; -----
2385 ; The current set of defined multiplex channels is (* means documented):
2386 ;
2387 ; Channel(h) Issuer Receiver Function
2388 ; 00 server PSPRINT print job control
2389 ; *01 print/apps PRINT Queueing of files
2390 ; 02 BIOS REDIR signal open/close of printers
2391 ;
2392 ; 05 command REDIR obtain text of net int 24 message
2393 ; *06 server/assign ASSIGN Install check
2394 ;
2395 ; 08 external driver IBMBIO interface to internal routines
2396 ;
2397 ; 10 sharer/server Sharer install check
2398 ; 11 DOS/server Redir install check/redirection funcs
2399 ; 12 sharer/redir DOS dos functions and structure maint
2400 ; 13 MSNET MSNET movement of NCBS
2401 ; 13 external driver IBMBIO Reset_Int_13, allows installation
2402 ; of alternative INT_13 drivers after
2403 ; boot_up
2404 ; 14 (IBM) DOS NLSFUNC down load NLS country info,DOS 3.3
2405 ; 14 (MS) APPS POPUP MSDOS 4 popup screen functions
2406 ; 15 APPS MSCDEX CD-ROM extensions interface
2407 ; 16 WIN386 WIN386 windows communications
2408 ; 17 Clipboard WINDOWS Clipboard interface
2409 ; *18 Applications MS-Manger Toggle interface to manager
2410 ; 19 Shell
2411 ; 1A Ansi.sys
2412 ; 1B Fastopen,Vdisk IBMBIO EMS INT 67H stub handler
2413 ;
2414 ; 40h OS/2
2415 ; 41h Lanman
2416 ; 42h Lanman
2417 ; 43h Himem
2418 ; AL = 20h reserved for Mach 20 Himem support
2419 ; AL = 30h reserved for Himem external A20 code
2420 ;
2421 ; 44h Dosextdender
2422 ; 45h Windows profiler
2423 ; 46h windows/286 DOS extender
2424 ; 47h Basic Compiler Vn. 7.0
2425 ; 48h Doskey
2426 ; 49h DOS 5.x install
2427 ; 4Ah Multi Purpose
2428 ; multMULTSWPDSK 0 - Swap Disk in drive A (BIOS)
2429 ; multMULTGETHMAPTR 1 - Get available HMA & ptr
2430 ; multMULTALLOCHMA 2 - Allocate HMA (bx == no of bytes)
2431 ; multMULTTASKSHELL 5 - Shell/switcher API
2432 ; multMULTRPLTOM 6 - Top Of Memory for RPL support
2433 ;
2434 ; multSmartdrv 10h
2435 ; multMagicdrv 11h
2436 ; 4Bh Task Switcher API
2437 ;
2438 ; 4Ch APPS APM Advanced power management
2439 ; 4Dh Kana Kanji Converter, MSKK
2440 ;
2441 ; 51h ODI real mode support driver (for Chicago)
2442 ;
2443 ; 53h POWER.EXE - used for broadcasting APM events ; M036
2444 ; 54h POWER.EXE - used for POWER API ; M036
2445 ;
2446 ; 55h COMMAND.COM
2447 ; multCOMFIRST 0 - API to determine whether 1st
2448 ; instance of command.com
2449 ; multCOMFIRSTROM 1 - API to determine whether 1st
2450 ; instance of ROM COMMAND
2451 ;
2452 ; 56h Sewell Development
2453 ; INTERLNK
2454 ;
2455 ; 57h Iomega Corp.
2456 ;
2457 ; ABh Unspecified IBM use
2458 ; ACh Graphics
2459 ; ADh NLS (toronto)
2460 ; AEh
2461 ; AFh Mode
2462 ; B0h GRAFTABL GRAFTABL
2463 ;
2464 ; D7h Banyan VINES
2465 ; -----
2466 ;MUX 00-3F reserved for IBM
2467 ;MUX 80-BF reserved for IBM
2468 ;
2469 ;MUX 40-7F reserved for Microsoft
2470 ;
2471 ;MUX C0-FF users
2472 multSHARE EQU 10h ; sharer
2473 ; 1 MFT_enter
2474 ; 2 MFTclose
2475 ; 3 MFTclU
2476 ; 4 MFTcloseP
2477 ; 5 MFTclon
2478 ; 6 set_block
2479 ; 7 clr_block
2480 ; 8 chk_block

```

```

2481 ; 9 MFT_get
2482 ; 10 ShSave
2483 ; 11 ShChk
2484 ; 12 ShCol
2485 ; 13 ShCloseFile
2486
2487 MultNET EQU 11h ; Network support
2488 MultIFS EQU 11h ; Network support
2489 ; 1 IFS_RMDIR
2490 ; 2 IFS_SEQ_RMDIR
2491 ; 3 IFS_MKDIR
2492 ; 4 IFS_SEQ_MKDIR
2493 ; 5 IFS_CHDIR
2494 ; 6 IFS_CLOSE
2495 ; 7 IFS_COMMIT
2496 ; 8 IFS_READ
2497 ; 9 IFS_WRITE
2498 ; 10 IFS_LOCK
2499 ; 11 IFS_UNLOCK
2500 ; 12 IFS_DISK_INFO
2501 ; 13 IFS_SET_FILE_ATTRIBUTE
2502 ; 14 IFS_SEQ_SET_FILE_ATTRIBUTE
2503 ; 15 IFS_GET_FILE_INFO
2504 ; 16 IFS_SEQ_GET_FILE_INFO
2505 ; 17 IFS_RENAME
2506 ; 18 IFS_SEQ_RENAME
2507 ; 19 IFS_DELETE
2508 ; 20 IFS_SEQ_DELETE
2509 ; 21 IFS_OPEN
2510 ; 22 IFS_SEQ_OPEN
2511 ; 23 IFS_CREATE
2512 ; 24 IFS_SEQ_CREATE
2513 ; 25 IFS_SEQ_SEARCH_FIRST
2514 ; 26 IFS_SEQ_SEARCH_NEXT
2515 ; 27 IFS_SEARCH_FIRST
2516 ; 28 IFS_SEARCH_NEXT
2517 ; 29 IFS_ABORT
2518 ; 30 IFS_ASSOPER
2519 ; 31 Printer_SET_STRING
2520 ; 32 IFSFlushBuf
2521 ; 33 IFSBufWrite
2522 ; 34 IFSResetEnvironment
2523 ; 35 IFSspoolCheck
2524 ; 36 IFSspoolClose
2525 ; 37 IFSDeviceOper
2526 ; 38 IFSspoolEchoCheck
2527 ; 39 - - - Unused - - -
2528 ; 40 - - - Unused - - -
2529 ; 41 - - - Unused - - -
2530 ; 42 SERVER_DOSCALL_CLOSEFILES_FOR_UID
2531 ; 43 DEVICE_IOCTL
2532 ; 44 IFS_UPDATE_CB
2533 ; 45 IFS_FILE_XATTRIBUTES
2534 ; 46 IFS_XOPEN
2535 ; 47 IFS_DEPENDENT_IOCTL
2536
2537 MultDOS EQU 12h ; DOS call back
2538 ; 1 DOS_CLOSE
2539 ; 2 RECSET
2540 ; 3 Get DOSGROUP
2541 ; 4 PATHCHRCMP
2542 ; 5 OUT
2543 ; 6 NET_I24_ENTRY
2544 ; 7 PLACEBUF
2545 ; 8 FREE_SFT
2546 ; 9 BUFWRITE
2547 ; 10 SHARE_VIOLATION
2548 ; 11 SHARE_ERROR
2549 ; 12 SET_SFT_MODE
2550 ; 13 DATE16
2551 ; 14 SETVISIT
2552 ; 15 SCANPLACE
2553 ; 16 SKIPVISIT
2554 ; 17 StrCpy
2555 ; 18 StrLen
2556 ; 19 UCase
2557 ; 20 POINTCOMP
2558 ; 21 CHECKFLUSH
2559 ; 22 SFFromSFN
2560 ; 23 GetCDSFromDrv
2561 ; 24 Get_User_Stack
2562 ; 25 GetThisDrv
2563 ; 26 DriveFromText
2564 ; 27 SETYEAR
2565 ; 28 DSUM
2566 ; 29 DSLIDE
2567 ; 30 StrCmp
2568 ; 31 initcds
2569 ; 32 pjfnfromhandle
2570 ; 33 $NameTrans
2571 ; 34 CAL_LK
2572 ; 35 DEVNAME
2573 ; 36 Idle
2574 ; 37 DStrLen
2575 ; 38 NLS_OPEN DOS 3.3
2576 ; 39 $CLOSE DOS 3.3
2577 ; 40 NLS_LSEEK DOS 3.3
2578 ; 41 $READ DOS 3.3
2579 ; 42 FastInit DOS 4.0
2580 ; 43 NLS_IOCTL DOS 3.3
2581 ; 44 GetDevList DOS 3.3
2582 ; 45 NLS_GETEXT DOS 3.3
2583 ; 46 MSG_RETRIEVAL DOS 4.0
2584 ; 47 FAKE_VERSION DOS 4.0
2585
2586 NLSFUNC EQU 14h ; NLSFUNC CALL , DOS 3.3
2587 ; 0 NLSInstall
2588 ; 1 ChgCodePage
2589 ; 2 GetExtInfo
2590 ; 3 SetCodePage
2591 ; 4 GetCntry
2592
2593 multANSI EQU 1Ah ; ANSI multiplex number
2594 ; 0 INSTALL_CHECK ; install check for ANSI
2595 ; 1 IOCTL_2F ; 2F interface to IOCTL
2596 ; 2 DA_INFO_2F ; J.K. Information passing to ANSI.
2597
2598 multMULT EQU 4Ah
2599 multMAGIC EQU 256*multMULT + 11h
2600 multMULTRPLTOM EQU 06h
2601
2602 ; 0 swap disk function for single floppy drive m/cs
2603 ; BIOS broadcasts with cx==0, and apps who handle
2604 ; swap disk messaging set cx == -1. BIOS sets dl == requested

```

```

2605 ; drive
2606 ;
2607 ; 1 Get available HMA & pointer to it. Returns in BX & ES:DI
2608 ; 2 Allocate HMA. BX == number of bytes in HMA to be allocated
2609 ; returns pointer in ES:DI
2610 ;
2611 ; 3-4 currently used by nobody
2612 ; 5 Switcher API
2613 ; 6 Top of Memory for RPL.
2614 ; BIOS issues INT 2f AX=4a06 & DX = Top of Mem and any RPL
2615 ; code present in TOM should respond with a new TOM in DX
2616 ; to protect itself from MSLOAD & SYSINIT tromping over it.
2617 ; SYSINIT builds an arena with owner type 8 & name 'RPL' to
2618 ; protect the RPL code from COMMAND.COM transient protion.
2619 ; It is the responsibility of RPL program to release the mem.
2620 ; 7 Reserved for PROTMAN support.
2621 ; 10 smartdrv 4.0
2622 ; 11 dblspace api
2623 ; 12 MRCI api
2624 ; 13 dblspace/mrci stealth packet api
2625
2626 MultAPM EQU 4ch ; Obselete ???
2627 ; 00h APM_VER_CHK
2628 ; 01h APM_SUS_SYS_REQ
2629 ; FFh APM_SUS_RES_BATT_NOTIFY
2630
2631 MultPWR_BRDCST EQU 53h ; Used by POWER.EXE to broadcast ; M036
2632 ; APM events ; M036
2633 MultPWR_API EQU 54h ; Used for accessing POWER.EXE's API ; M036
2634
2635 ;FASTOPEN is not chained through INT 2F ; DOS 3.3 F.C.
2636 ; it calls Multdos 42 to set up an entry routine address
2637 ; 0 Install status (reserved)
2638 ; 1 Lookup
2639 ; 2 Insert
2640 ; 3 Delete
2641 ; 4 Purge (reserved)
2642
2643 ;=====
2644 ; FIND.INC (MSDOS 6.0, 1991)
2645 ;=====
2646 ; 17/05/2019 - Retro DOS v4.0
2647 ; 09/07/2018 - Retro DOS v3.0 (MSDOS 3.3, 1987)
2648
2649 ;Break <find first/next buffer>
2650
2651 struc find_buf
2652 .drive: resb 1 ; drive of search
2653 .name: resb 11 ; formatted name
2654 .sattr: resb 1 ; attribute of search
2655 .LastEnt: resw 1 ; LastEnt
2656 .DirStart: resw 1 ; DirStart
2657 .NETID: resb 4 ; MSDOS 6.0 ; Reserved for NET
2658 .attr: resb 1 ; attribute found
2659 .time: resw 1 ; time
2660 .date: resw 1 ; date
2661 .size_l: resw 1 ; low(size)
2662 .size_h: resw 1 ; high(size)
2663 .pname: resb 13 ; packed name
2664 .size:
2665 endstruc
2666
2667 ;=====
2668 ; DOSCNTRY.INC (MSDOS 6.0, 1991)
2669 ;=====
2670 ; 29/04/2019 - Retro DOS v4.0
2671 ; 09/07/2018 - Retro DOS v3.0 (MSDOS 3.3, 1987)
2672
2673 ;Equates for COUNTRY INFORMATION.
2674 SetCountryInfo EQU 1 ;country info
2675 SetUcase EQU 2 ;uppercase table
2676 SetLcase EQU 3 ;lowercase table (Reserved)
2677 SetUcaseFile EQU 4 ;uppercase file spec table
2678 SetFileList EQU 5 ;valid file character list
2679 SetCollate EQU 6 ;collating sequence
2680 SetDBCS EQU 7 ;double byte character set
2681 SetALL EQU -1 ;all the entries
2682
2683 ;DOS country and code page information table structure.
2684 ;Internally, IBMDOS gives a pointer to this table.
2685 ;IBMBIO, MODE and NLSFUNC modules communicate with IBMDOS through
2686 ;this structure.
2687
2688 struc DOS_CCDPG ; DOS_country_cdpd_info
2689 .ccInfo_reserved: resb 8 ;reserved for internal use
2690 .ccPath_CountrySys: resb 64 ;path and filename for country info
2691 .ccSysCodePage: resw 1 ;system code page id
2692 .ccNumber_of_entries: resw 1 ; (default value = 6)
2693 .ccSetUcase: resb 1 ; (default value = SetUcase)
2694 .ccUcase_ptr: resd 1 ;pointer to Ucase table
2695
2696 .ccSetUcaseFile: resb 1 ; (default value = SetUcaseFile)
2697 .ccFileUcase_ptr: resd 1 ;pointer to File Ucase table
2698
2699 .ccSetFileList: resb 1 ; (default value = SetFileList)
2700 .ccFileChar_ptr: resd 1 ;pointer to File char list table
2701
2702 .ccSetCollate: resb 1 ; (default value = SetCollate)
2703 .ccCollate_ptr: resd 1 ;pointer to collate table
2704
2705 ; MSDOS 6.0
2706 .ccSetDBCS: resb 1 ; (default value = SetDBCS)
2707 .ccDBCS_ptr: resd 1 ; pointer to DBCS table
2708
2709 .ccSetCountryInfo: resb 1 ; (default value = SetCountryInfo)
2710 .ccCountryInfoLen: resw 1 ;length of country info
2711 .ccDosCountry: resw 1 ;system country code id
2712 .ccDosCodePage: resw 1 ;system code page id
2713 .ccDFormat: resw 1 ;date format
2714 .ccCurSymbol: resb 5 ;5 byte of (currency symbol+0)
2715 .cc1000Sep: resb 2 ;2 byte of (1000 sep. + 0)
2716 .ccDecSep: resb 2 ;2 byte of (Decimal sep. + 0)
2717 .ccDateSep: resb 2 ;2 byte of (date sep. + 0)
2718 .ccTimeSep: resb 2 ;2 byte of (time sep. + 0)
2719 .ccCFormat: resb 1 ;currency format flags
2720 .ccSigDigits: resb 1 ;# of digits in currency
2721 .ccTFormat: resb 1 ;time format
2722 .ccMono_ptr: resd 1 ;monocase routine entry point
2723 .ccListSep: resb 2 ;data list separator
2724 .ccReserved_area: resw 5 ;reserved
2725 .size:
2726 endstruc
2727
2728 ;Ucase table

```


[illegible]

```

2853 ; | Current Extent(word) |
2854 ; +-----+
2855 ; | Record size (word) |
2856 ; +-----+
2857 ; | File Size (2 words) |
2858 ; +-----+
2859 ; | Date of write |
2860 ; +-----+
2861 ; | Time of write |
2862 ; +-----+
2863 ; +-----+
2864 ; C A V E A T P R O G R A M M E R ;
2865 ; +-----+
2866 ; | 8 bytes reserved |
2867 ; +-----+
2868 ;
2869 ; C A V E A T P R O G R A M M E R ;
2870 ; +-----+
2871 ; | next record number |
2872 ; +-----+
2873 ; | random record number |
2874 ; +-----+
2875 ;
2876 ;
2877
2878 struct SYS_FCB
2879 .drive: resb 1
2880 .name: resb 8
2881 .ext: resb 3
2882 .EXTENT: resw 1
2883 .RECSIZ: resw 1 ; Size of record (user settable)
2884 .FILSIZ: resw 1 ; Size of file in bytes; used with the
2885 ; following word
2886 .DRVBP: resw 1 ; BP for SEARCH FIRST and SEARCH NEXT
2887 .FDATE: resw 1 ; Date of last writing
2888 .FTIME: resw 1 ; Time of last writing
2889 ; +-----+
2890 ; C A V E A T P R O G R A M M E R ;
2891 ; +-----+
2892 .reserved: resb 8 ; RESERVED
2893 ;
2894 ; C A V E A T P R O G R A M M E R ;
2895 ; +-----+
2896 .NR: resb 1 ; Next record
2897 .RR: resb 4 ; Random record
2898 .size:
2899 endstruc
2900
2901 FILDIRENT EQU SYS_FCB.FILSIZ ; Used only by SEARCH FIRST and SEARCH
2902 ; NEXT
2903
2904 ; 20/07/2018
2905 %define fcb_sfn SYS_FCB.reserved ; byte
2906
2907 ; Note that fcb_net_handle, fcb_nsl_drive, fcb_nsl_drive and fcb_l_drive
2908 ; all must point to the same byte. Otherwise, the FCBRegen will fail.
2909 ; NOTE about this byte (fcb_nsl_drive)
2910 ; The high two bits of this byte are used as follows to indicate the FCB type
2911 ; 00 means a local file or device with sharing loaded
2912 ; 10 means a remote (network) file
2913 ; 01 means a local file with no sharing loaded
2914 ; 11 means a local device with no sharing loaded
2915
2916 ; 20/07/2018
2917
2918 ; Network FCB
2919
2920
2921 %define fcb_net_drive SYS_FCB.reserved+1 ; byte
2922 %define fcb_net_handle SYS_FCB.reserved+2 ; word
2923 %define fcb_netID SYS_FCB.reserved+4 ; dword
2924
2925 ;
2926 ; No sharing local file FCB
2927 ;
2928
2929 %define fcb_nsl_drive SYS_FCB.reserved+1 ; byte
2930 %define fcb_nsl_bits SYS_FCB.reserved+2 ; byte
2931 %define fcb_nsl_firclus SYS_FCB.reserved+3 ; word
2932 %define fcb_nsl_dirsec SYS_FCB.reserved+5 ; word
2933 %define fcb_nsl_dirpos SYS_FCB.reserved+7 ; byte
2934
2935 ;
2936 ; No sharing local device FCB
2937 ;
2938
2939 %define fcb_nsl_drive SYS_FCB.reserved+1 ; byte
2940 %define fcb_nsl_drvptr SYS_FCB.reserved+2 ; dword
2941
2942 ;
2943 ; Sharing local FCB
2944 ;
2945
2946 %define fcb_l_drive SYS_FCB.reserved+1 ; byte
2947 %define fcb_l_firclus SYS_FCB.reserved+2 ; word
2948 %define fcb_l_mfs SYS_FCB.reserved+4 ; word
2949 %define fcb_l_attr SYS_FCB.reserved+6 ; byte
2950
2951 ;
2952 ; Bogusness: the four cases are:
2953 ;
2954 ; local file 00
2955 ; local device 40
2956 ; local sharing C0
2957 ; network 80
2958 ;
2959 ; Since sharing and network collide, we cannot use a test instruction for
2960 ; deciding whether a network or a share check is involved
2961 ;
2962 FCBDEVICE EQU 040h
2963 FCBNETWORK EQU 080h
2964 FCBSHARE EQU 0C0h
2965
2966 ; FCBSPCIAL must be able to mask off both net and share
2967 FCBSPCIAL EQU 080h
2968 FCBMASK EQU 0C0h
2969
2970 ;=====
2971 ; FASTOPEN.INC, MSDOS 6.0, 1991
2972 ;=====
2973 ; 11/07/2018 - Retro DOS v3.0
2974 ; 25/04/2019 - Retro DOS v4.0
2975 ; 21/02/2024 - Retro DOS v5.0
2976

```



```

2977 struct FEI ; FASTOPEN_EXTENDED_INFO
2978 .dirpos: resb 1
2979 .dirsec: resd 1 ; MSDOS 6.0
2980 ;.dirsec: resw 1 ; MSDOS 3.3
2981 .clusnum: resw 1
2982 resw 1 ; PCDOS 7.1 ; 21/02/2024
2983 .lastent: resw 1 ; for search first ; MSDOS 6.0
2984 .firststart: resw 1 ; for search first ; MSDOS 6.0
2985 .size:
2986 endstruc
2987
2988 ; 23/07/2018
2989 ;FASTOPEN NAME CACHING Subfunctions
2990 FONC_Look_up equ 1
2991 FONC_insert equ 2
2992 FONC_delete equ 3
2993 FONC_update equ 4
2994 FONC_purge equ 5 ;reserved for the future use.
2995 FONC_Rename equ 6 ;AN001
2996
2997 ; 27/07/2018
2998 ;FastOpen Data Structure
2999 struct fastopen_entry ;Fastopen Entry pointer in DOS
3000 .entry_size: resw 1 ; = 4 ; size of the following
3001 .name_caching: resd 1
3002 ; MSDOS 6.0
3003 ;.fatchain_caching: resd 1;reserved for future use
3004 .size:
3005 endstruc
3006
3007 ; 27/07/2018
3008 ;Equates used in DOS.
3009 FastOpen_Set equ 00000001b
3010 FastOpen_Reset equ 11111110b
3011 Lookup_Success equ 00000010b
3012 Lookup_Reset equ 11111101b
3013 Special_Fill_Set equ 00000100b
3014 Special_Fill_Reset equ 11111011b
3015 No_Lookup equ 00001000b
3016 Set_For_Search equ 00010000b ;DCR 167
3017
3018 ; 09/08/2018
3019 ; (FASTXXX.INC, MSDOS 6.0, 1991)
3020 ; Fastxxx equates
3021 FastOpen_ID equ 1
3022 FastSeek_ID equ 2
3023 Fast_yes equ 10000000b ; 80h ; fastxxx flag
3024
3025 ;Structure definitions
3026 ;
3027 struct Fasttable_Entry ; Fastxxx Entry pointer in DOS
3028 .Fast_Entry_Num: resw 1 ; number of entries
3029 .FastOpen_Seek: resd 1 ; fastopen & fastseek entry address
3030 endstruc
3031
3032 ;=====
3033 ; LOCK.INC, MSDOS 6.0, 1991
3034 ;=====
3035 ; 14/07/2018 - Retro DOS v3.0
3036
3037 ;** LOCK.INC - Definitions for Record Locking
3038
3039 ;** LOCK functions
3040
3041 LOCK_ALL equ 0
3042 UNLOCK_ALL equ 1
3043 LOCK_MUL_RANGE equ 2
3044 UNLOCK_MUL_RANGE equ 3
3045 LOCK_READ equ 4
3046 WRITE_UNLOCK equ 5
3047 LOCK_ADD equ 6
3048
3049 ;** Structure for Lock buffer
3050
3051 struct LockBuf
3052 .Lock_position: resd 1 ; file position for LOCK
3053 .Lock_length: resd 1 ; number of bytes to LOCK
3054 endstruc
3055
3056 ;=====
3057 ; DPL.ASM, MSDOS 6.0, 1991
3058 ;=====
3059 ; 04/08/2018 - Retro DOS v3.0
3060
3061 ; (SRVCALL.ASM)
3062
3063 struct DPL
3064 .AX: resw 1 ; AX register
3065 .BX: resw 1 ; BX register
3066 .CX: resw 1 ; CX register
3067 .DX: resw 1 ; DX register
3068 .SI: resw 1 ; SI register
3069 .DI: resw 1 ; DI register
3070 .DS: resw 1 ; DS register
3071 .ES: resw 1 ; ES register
3072 .rsrvd: resw 1 ; Reserved
3073 .UID: resw 1 ; User (Machine) ID (0 = local machine)
3074 .PID: resw 1 ; Process ID (0 = local user PID)
3075 .size:
3076 endstruc
3077
3078 ;-----
3079 ; DOSDATA
3080 ;-----
3081 ;=====
3082 ; 24/04/2019 - Retro DOS v4.0
3083
3084 DosDataSg equ 3 ; DOS Data Segment address (dw in 'retrodos4.s')
3085 ; ((just after resident IO.SYS code&data))
3086
3087 ;=====
3088 ; WIN386.INC, MSDOS 6.0, 1991
3089 ;=====
3090 ; 24/04/2019 - Retro DOS 4.0
3091
3092 ;
3093 ; Symbols and structures relating to WIN386 support.
3094 ;
3095 ; Used by files in both the DOS and the BIOS.
3096 ;
3097 ; Created: 7-13-89 by MRW
3098 ;
3099 ;
3100 ; WIN386 broadcast int 2fh multiplex number and subfunction numbers

```

```

3101      Multwin386      equ      16h      ; Int 2f multiplex number
3102
3103      win386_Init      equ      05h      ; win386 initialization
3104      win386_Exit      equ      06h      ; win386 exit
3105      win386_Devcall   equ      07h      ; win386 device call out
3106      win386_InitDone  equ      08h      ; win386 initialization is complete
3107
3108      ; When win386_Devcall is broadcast, BX is the Device ID. DOS must
3109      ; answer call outs from the DOSMGR
3110
3111      win386_DOSMGR     equ      15H
3112
3113      ; The following structures are used to communicate instance data to
3114      ; win386 from the DOS and the BIOS. See win386 API documentation
3115      ; (chapter 3, "Call Out Interfaces") for further description.
3116
3117      struc win386_SIS   ; Startup Info Structure
3118      .Version:         resb      2      ; db 3, 0
3119      .Next_Dev_Ptr:     resd      1      ; pointer to next SIS in list
3120      .Virt_Dev_File_Ptr: resd      1
3121      .Reference_Data:   resd      1
3122      .Instance_Data_Ptr: resd      1      ; pointer to instance data array
3123      endstruc
3124
3125      size_of_win386_SIS equ 18 ; 24/04/2019 - Retro DOS v4.0
3126
3127      struc win386_IIS   ; Instance Item Structure
3128      .Ptr:              resd      1      ; pointer to an instance item
3129      .Size:             resw      1      ; size of an instance item
3130      endstruc
3131
3132      size_of_win386_IIS equ 6 ; 24/04/2019 - Retro DOS v4.0
3133
3134      ;win386 DOSMGR function return values to indicate operation done
3135
3136      WIN_OP_DONE        equ      0B97Ch ;
3137      DOSMGR_OP_DONE     equ      0A2ABh ;
3138
3139      ;M021
3140      ; WINoldap callout multiplex number
3141
3142      WINOLDAP           equ      46h      ;
3143
3144      ;=====
3145      ;-----
3146      ; DOSCODE
3147      ;-----
3148      ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3149
3150      ;=====
3151      ; MSHEAD.ASM (MSDOS 6.0, 1991)
3152      ;=====
3153      ; 16/07/2018 - Retro DOS 3.0
3154      ;-----
3155      ; 24/04/2019 - Retro DOS 4.0
3156
3157      ; MSDOS 6.0
3158      ;-----
3159      ; FILE : ORIGIN.INC
3160      ;-----
3161      ; This is included in origin.asm and mshead.asm. Contains the equate that
3162      ; is used for ORGing the DOS code.
3163      ;
3164      ; Brief Description of the necessacity of this ORG:
3165      ; -----
3166      ;
3167      ; A special problem exists when running out of the HMA. The HMA starts at
3168      ; address FFFF:10. There is no place in the HMA with an offset of zero.
3169      ; This means programs running out off the HMA must use non-zero offset base
3170      ; addresses. It also means that if we're running multiple programs from the
3171      ; HMA, the base offset of each segment must atleast be as big as all of the
3172      ; HMA segments that precede it.
3173      ;
3174      ; One solution to this problem to ORG each module at 64K minus its size.
3175      ; For instance a code segment 1234h bytes in length would org'd at edcbh.
3176      ; This gives max. flexibility regarding it's location in the HMA. By
3177      ; selecting segment values between f124h and ffffh it could be located
3178      ; anywhere in the HMA. The problem with this is that programs with such
3179      ; high ORGs would not be able to run in low RAM.
3180      ;
3181      ; A compromise solution is to set the ORG address somewhere between 0010h
3182      ; and ffffh - their size. In the particular case of the BIOS and the DOS
3183      ; the following solution has been implemented:
3184      ;
3185      ; The Bios Code segment will have a very small offset and run at the very
3186      ; front of the HMA, after the VDISK header. THE Dos Code segment will have
3187      ; a base offset of (700+<min. size off RAM based BIOS>+<min. size of the DOS
3188      ; DATA segment when DOS is running low>). This will reflect the lowest
3189      ; possible physical address at which DOS code will run, while still providing
3190      ; max. possible flexibility in HMA positioning. This offset MUST NOT be
3191      ; smaller then that 20+size of Bios Code segment when running high. This is
3192      ; mostly true.
3193      ;
3194      ; Also this ORG'd value must be communicated to the BIOS. This is done by
3195      ; putting this value after the first jmp instruction in the DOS code in
3196      ; mshead.asm.
3197      ;
3198      ; In order for the stripz utility to know how many zeroes to be stripped
3199      ; out, this value is placed at the beginning of the binary in origin.asm.
3200      ;
3201      ; Revision History:
3202      ;
3203      ; Currently this is being done manually. Therefore any change in the DOS DATA
3204      ; Size or the BIOS size should be reflected here. --- Feb 90
3205      ;
3206      ; BDSIZE.INC contains the equates for BIODATASIZE, BIOCODESIZ and DOSDATASIZ.
3207      ; A utility called getsize will obtain the corresponding values from msdos
3208      ; and msbio.map and update the values in BDSIZ.INC if they are different.
3209      ; DOS should now be built using the batch file makedos.bat which invokes this
3210      ; utility. The FORMAT of BDSIZE.INC should not be changed as getsize is
3211      ; dependant on that. --- Apr 3 '90
3212      ;
3213      ; For ROMDOS, however, there is no need to org the doscode to any location
3214      ; other than zero. Therefore the stripz utility will not need to be used,
3215      ; so the offset will not need to be included at the beginning of the code
3216      ; segment. Also, the BIOS can just assume that the resident code begins
3217      ; at offset zero within the segment.
3218      ;
3219      ;
3220      ;-----
3221      ;
3222      BIODATASTART      EQU      00700h
3223      ;include bdsiz.inc ; this sets the values:
3224

```

```

3225 ; BIODATASIZ
3226 ; BIOCODESIZ
3227 ; DOSDATASIZ
3228
3229 ; 05/12/2022
3230 ;BIODATASIZ EQU 00910H ; 0900h for MSDOS 6.21 IO.SYS
3231 ; ; 0900h for MSDOS 5.0 IO.SYS
3232 ;BIOCODESIZ EQU 01A70H ; 1A70h for MSDOS 6.21 IO.SYS
3233 ; ; 1A60h for MSDOS 5.0 IO.SYS
3234 ;DOSDATASIZ EQU 01370H ; 1370h for MSDOS 6.21 IO.SYS
3235 ; ; 1370h for MSDOS 5.0 IO.SYS
3236
3237 ;ifndef ROMDOS
3238 ;BYTSTART EQU BIODATASTART+BIODATASIZ+BIOCODESIZ+DOSDATASIZ
3239 ;PARASTART EQU (BYTSTART + 0FH) AND (NOT 0FH)
3240 ;
3241 ;else
3242 ;
3243 ;BYTSTART EQU 0
3244 ;PARASTART EQU 0
3245 ;
3246 ;endif ; ROMDOS
3247
3248 ; 24/04/2019 - Retro DOS v4.0 - Modification
3249 ; -----
3250 ;MSDAT001E equ 136Ah ; 4970 ; for MSDOS 6.21
3251 ;MSDAT001E equ 1370h ; 4976 ; for Retro DOS v4.0 modif. 25/05/2019
3252 ;DOSDATASIZE equ MSDAT001E
3253 ; 05/12/2022
3254 ;DOSDATASIZE equ $ ; 29/04/2019 ; -only- for RETRO DOS v4.0 :
3255 ;_PARASTART_ equ DOSDATASIZE ; segment value will point to start of
3256 ; ; of DOSDATA (in low memory) while
3257 ; ; dos/kernel code starts just after
3258 ; ; this data block ((org = DOSDATASIZE))
3259 ; ; (in low memory or in HMA)
3260 ; -----
3261
3262 ; 04/11/2022
3263 ; -----
3264 ; NOTE:
3265 ; Microsoft dos programmers were calling 'IO.SYS' as dos 'BIOS'
3266 ; (Also, they were calling 'ROMBIOS' as 'ROM' only!)
3267 ; -----
3268
3269 ; -----
3270 ; 06/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3271 ; -----
3272 ; -----
3273 ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
3274 ; -----
3275 ; -----
3276 ; -----
3277 ; Start of (PCDOS 7.1) IBMDOS.COM
3278 ; -----
3279
3280 ;segment .code vstart=3DD0h ; 06/12/2022
3281 ; 29/09/2023
3282 ;segment .code vstart=3DE0h ; 19/09/2023 - Retro DOS v4.2 (Modified MSDOS 6.22)
3283 ; -----
3284 ; 01/01/2024
3285 segment .code vstart=3F10h ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1)
3286
3287 ; =====
3288
3289 ;[ORG 3DE0h]
3290
3291 ;[ORG _PARASTART_] ; [org 136Ah]
3292
3293 ;[ORG 1370h] ; 25/05/2019 - Retro DOS v4.0
3294
3295 ; 01/01/2024 - Retro DOS v5.0
3296 ;[ORG 3F10h]
3297
3298 ; 05/12/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3299 ;PARASTART equ 3DD0h ; BIOSDATASTART+BIOSDATASIZE
3300 ; ; +BIOSCODESIZE+DOSDATASIZE (rounded up)
3301
3302 ; 29/09/2023
3303 ; 19/09/2023 - Retro DOS v4.2 (Modified MSDOS 6.22 MSDOS.SYS)
3304 ;PARASTART equ 3DE0h ; (MSDOS 6.22 MSDOS.SYS)
3305
3306 ; 01/01/2024 - Retro DOS v4.2 (Modified PCDOS 7.1 IBMDOS.COM)
3307 PARASTART equ 3F10h ; (PCDOS 7.1 IBMDOS.COM)
3308
3309 [ORG PARASTART]
3310
3311 _$STARTCODE:
3312
3313 ;PARASTART:
3314 JMP DOSINIT
3315
3316 ;dw PARASTART ; PARASTART = 3DE0h for MSDOS 6.0, 6.22
3317 ; 04/11/2022 ; PARASTART = 3DD0h for MSDOS 5.0
3318 dw _$STARTCODE ; PARASTART = 3F10h for PCDOS 7.1 ; 01/01/2024
3319
3320 BioDataSeg:
3321 dw 0070h ; Bios data segment fixed at 70h
3322
3323 ; DosDSeg is a data word in the DOSCODE segment that is loaded with
3324 ; the segment address of DOSDATA. This is purely an optimization, that
3325 ; allows getting the DOS data segment without going through the
3326 ; BIOS data segment. It is used by the "getdseg" macro.
3327
3328 DosDSeg:
3329 dw 0
3330
3331 ; =====
3332 ; MSTABLE.ASM (MSDOS 6.0, 1991)
3333 ; =====
3334 ; 16/07/2018 - Retro DOS 3.0
3335 ; 29/04/2019 - Retro DOS 4.0
3336
3337 ; (MSDOS version)
3338 ; DOSCODE:3DE9h (MSDOS 6.21, MSDOS.SYS)
3339 ;db 6
3340 ;db 20
3341 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3342 ; DOSCODE:3DD9h (MSDOS 5.0, MSDOS.SYS)
3343 ;db 5
3344 ;db 0
3345
3346 ; Offset 0c78h in IBMDOS.COM (MSDOS 3.3, 1987)
3347 MSVERS: ; MS-DOS version in hex for $GET_VERSION
3348 DB MAJOR_VERSION ; DOS_MAJOR_VERSION

```

```

3349 0000000A 0A      MSMINOR: DB MINOR_VERSION ; DOS_MINOR_VERSION
3350
3351 ;;hkn YRTAB & MONTAB moved to DOSDATA in ms_data.asm
3352 ; I_am YRTAB,8,<200,166,200,165,200,165,200,165> ; [SYSTEM]
3353 ; I_am MONTAB,12,<31,28,31,30,31,30,31,31,30,31,30,31> ; [SYSTEM]
3354
3355 ; DOSTAB.ASM (MSDOS 6.0, 1991)
3356 ; YRTAB & MONTAB moved from TABLE segment in ms_table.asm
3357
3358 ; I_am YRTAB,8,<200,166,200,165,200,165,200,165>
3359 ; I_am MONTAB,12,<31,28,31,30,31,30,31,31,30,31,30,31>
3360
3361 ; This is the error code mapping table for INT 21 errors. This table defines
3362 ; those error codes which are "allowed" for each system call. If the error
3363 ; code ABOUT to be returned is not "allowed" for the call, the correct action
3364 ; is to return the "real" error via Extended error, and one of the allowed
3365 ; errors on the actual call.
3366
3367 ; The table is organized as follows:
3368
3369 ; Each entry in the table is of variable size, but the first
3370 ; two bytes are always:
3371
3372 ; call#,Cnt of bytes following this byte
3373
3374 ; EXAMPLE:
3375 ; Call 61 (OPEN)
3376
3377 ; DB 61,5,12,3,2,4,5
3378
3379 ; 61 is the AH INT 21 call value for OPEN.
3380 ; 5 indicates that there are 5 bytes after this byte (12,3,2,4,5).
3381 ; Next five bytes are those error codes which are "allowed" on OPEN.
3382 ; The order of these values is not important EXCEPT FOR THE LAST ONE (in
3383 ; this case 5). The last value will be the one returned on the call if
3384 ; the "real" error is not one of the allowed ones.
3385
3386 ; There are a number of calls (for instance all of the FCB calls) for which
3387 ; there is NO entry. This means that NO error codes are returned on this
3388 ; call, so set up an Extended error and leave the current error code alone.
3389
3390 ; The table is terminated by a call value of 0FFh
3391
3392 ;PUBLIC I21_MAP_E_TAB
3393 ; 10/08/2018
3394
3395 ; 29/04/2019
3396 ; DOSCODE:3DE9h (MSDOS 6.21, MSDOS.SYS)
3397 ; 04/11/2022
3398 ; DOSCODE:3DDBh (MSDOS 5.0 MSDOS.SYS)
3399 ; 01/01/2024
3400 ; DOSCODE:3F1Bh (PCDOS 7.1 IBMDOS.COM)
3401
3402 I21_MAP_E_TAB: ; LABEL BYTE
3403 0000000B 38020102 DB INTERNATIONAL,2,error_invalid_function,error_file_not_found
3404 0000000F 3903030205 DB MKDIR,3,error_path_not_found,error_file_not_found,error_access_denied
3405 00000014 3A041003 DB RMDIR,4,error_current_directory,error_path_not_found
3406 00000018 0205 DB error_file_not_found,error_access_denied
3407 0000001A 38020203 DB CHDIR,2,error_file_not_found,error_path_not_found
3408 0000001E 3C040302 DB CREAT,4,error_path_not_found,error_file_not_found
3409 00000022 04 DB error_too_many_open_files
3410 00000023 05 DB error_access_denied
3411 ; MSDOS 6.0
3412 00000024 3D0603020C DB OPEN,6,error_path_not_found,error_file_not_found,error_invalid_access
3413 00000029 04 DB error_too_many_open_files
3414 0000002A 1A05 DB error_not_DOS_disk,error_access_denied
3415 ; MSDOS 3.3
3416 ;DB OPEN,5,error_path_not_found,error_file_not_found,error_invalid_access
3417 ;DB error_too_many_open_files,error_access_denied
3418 0000002C 3E0106 DB CLOSE,1,error_invalid_handle
3419 0000002F 3F020605 DB READ,2,error_invalid_handle,error_access_denied
3420 00000033 40020605 DB WRITE,2,error_invalid_handle,error_access_denied
3421 00000037 4103030205 DB UNLINK,3,error_path_not_found,error_file_not_found,error_access_denied
3422 0000003C 42020601 DB LSEEK,2,error_invalid_handle,error_invalid_function
3423 00000040 4304030201 DB CHMOD,4,error_path_not_found,error_file_not_found,error_invalid_function
3424 00000045 05 DB error_access_denied
3425 00000046 44050F0D01 DB IOCTL,5,error_invalid_drive,error_invalid_data,error_invalid_function
3426 0000004B 0605 DB error_invalid_handle,error_access_denied
3427 0000004D 45020604 DB XDUP,2,error_invalid_handle,error_too_many_open_files
3428 00000051 46020604 DB XDUP2,2,error_invalid_handle,error_too_many_open_files
3429 ; MSDOS 6.0
3430 00000055 47021A0F DB CURRENT_DIR,2,error_not_DOS_disk,error_invalid_drive
3431 ; MSDOS 3.3
3432 ;DB CURRENT_DIR,1,error_invalid_drive
3433 00000059 48020708 DB ALLOC,2,error_arena_trashed,error_not_enough_memory
3434 0000005D 49020709 DB DEALLOC,2,error_arena_trashed,error_invalid_block
3435 00000061 4A03070908 DB SETBLOCK,3,error_arena_trashed,error_invalid_block,error_not_enough_memory
3436 00000066 4B08030102 DB EXEC,8,error_path_not_found,error_invalid_function,error_file_not_found
3437 0000006B 040B0A DB error_too_many_open_files,error_bad_format,error_bad_environment
3438 0000006E 0805 DB error_not_enough_memory,error_access_denied
3439 00000070 4E03030212 DB FIND_FIRST,3,error_path_not_found,error_file_not_found,error_no_more_files
3440 00000075 4F0112 DB FIND_NEXT,1,error_no_more_files
3441 ; MSDOS 6.0
3442 00000078 5605110302 DB RENAME,5,error_not_same_device,error_path_not_found,error_file_not_found
3443 0000007D 1005 DB error_current_directory,error_access_denied
3444 ; MSDOS 3.3
3445 ;DB RENAME,4,error_not_same_device,error_path_not_found,error_file_not_found
3446 ;DB error_access_denied
3447 ; MSDOS 6.0
3448 0000007F 57040608 DB FILE_TIMES,4,error_invalid_handle,error_not_enough_memory
3449 00000083 0B01 DB error_invalid_data,error_invalid_function
3450 ; MSDOS 3.3
3451 ;DB FILE_TIMES,2,error_invalid_handle,error_invalid_function
3452 00000085 580101 DB ALLOCOPER,1,error_invalid_function
3453 00000088 5A040302 DB CREATETEMPFILE,4,error_path_not_found,error_file_not_found
3454 0000008C 0405 DB error_too_many_open_files,error_access_denied
3455 0000008E 5B055003 DB CREATENEF,5,error_file_exists,error_path_not_found
3456 00000092 020405 DB error_file_not_found,error_too_many_open_files,error_access_denied
3457 00000095 5C040601 DB LOCKOPER,4,error_invalid_handle,error_invalid_function
3458 00000099 2421 DB error_sharing_buffer_exceeded,error_lock_violation
3459 0000009B 65020102 DB GETTEXTCNTRY,2,error_invalid_function,error_file_not_found ;DOS 3.3
3460 0000009F 66020102 DB GETSETCDPG,2,error_invalid_function,error_file_not_found ;DOS 3.3
3461 000000A3 680106 DB COMMIT,1,error_invalid_handle ;DOS 3.3
3462 000000A6 67030408 DB EXTHANDLE,3,error_too_many_open_files,error_not_enough_memory
3463 000000AA 01 DB error_invalid_function
3464 ; MSDOS 6.0
3465 000000AB 6C0A DB ExtOpen,10
3466 000000AD 03020C DB error_path_not_found,error_file_not_found,error_invalid_access
3467 000000B0 045008 DB error_too_many_open_files,error_file_exists,error_not_enough_memory
3468 000000B3 1A0D DB error_not_DOS_disk,error_invalid_data
3469 000000B5 0105 DB error_invalid_function,error_access_denied
3470 000000B7 69040F0D DB GetSetMediaID,4,error_invalid_drive,error_invalid_data
3471 000000BB 0105 DB error_invalid_function,error_access_denied
3472 ; 01/01/2024

```

[illegible]

			; short_addr _\$CREATE_PROCESS_DATA_BLOCK ; 38 26
3595	00000138 [4F16]		
3596			
3597		C A V E A T P R O G R A M M E R	
3598		-+-----+	
3599	0000013A [6722]	short_addr _\$FCB_RANDOM_READ_BLOCK ; 39 27	
3600	0000013C [6322]	short_addr _\$FCB_RANDOM_WRITE_BLOCK ; 40 28	
3601	0000013E [D212]	short_addr _\$PARSE_FILE_DESCRIPTOR ; 41 29	
3602	00000140 [D80A]	short_addr _\$GET_DATE ; 42 2A	
3603	00000142 [F50A]	short_addr _\$SET_DATE ; 43 2B	
3604	00000144 [140B]	short_addr _\$GET_TIME ; 44 2C	
3605	00000146 [250B]	short_addr _\$SET_TIME ; 45 2D	
3606	00000148 [C30C]	short_addr _\$SET_VERIFY_ON_WRITE ; 46 2E	
3607			
3608		; Extended functionality group	
3609	0000014A [340F]	short_addr _\$GET_DMA ; 47 2F	
3610	0000014C [9C0C]	short_addr _\$GET_VERSION ; 48 30	
3611	0000014E [1072]	short_addr _\$KEEP_PROCESS ; 49 31	
3612		-+-----+	
3613		C A V E A T P R O G R A M M E R	
3614		-+-----+	
3615	00000150 [4413]	short_addr _\$GET_DPB ; 50 32	
3616			
3617		C A V E A T P R O G R A M M E R	
3618		-+-----+	
3619	00000152 [3D02]	short_addr _\$SET_CTRL_C_TRAPPING ; 51 33	
3620	00000154 [2C13]	short_addr _\$GET_INDOS_FLAG ; 52 34	
3621	00000156 [690F]	short_addr _\$GET_INTERRUPT_VECTOR ; 53 35	
3622	00000158 [030F]	short_addr _\$GET_DRIVE_FREESPACE ; 54 36	
3623	0000015A [A50F]	short_addr _\$CHAR_OPER ; 55 37	
3624	0000015C [CA0C]	short_addr _\$INTERNATIONAL ; 56 38	
3625			
3626		; XENIX CALLS	
3627	0000015E [1B28]	; Directory Group	
3628	00000160 [3C27]	short_addr _\$MKDIR ; 57 39	
3629	00000162 [8B27]	short_addr _\$RMDIR ; 58 3A	
3630		short_addr _\$CHDIR ; 59 3B	
3631	00000164 [9C80]	; File Group	
3632	00000166 [C47F]	short_addr _\$CREAT ; 60 3C	
3633	00000168 [6877]	short_addr _\$OPEN ; 61 3D	
3634	0000016A [6F78]	short_addr _\$CLOSE ; 62 3E	
3635	0000016C [CC78]	short_addr _\$READ ; 63 3F	
3636	0000016E [OE81]	short_addr _\$WRITE ; 64 40	
3637	00000170 [D178]	short_addr _\$UNLINK ; 65 41	
3638	00000172 [A980]	short_addr _\$LSEEK ; 66 42	
3639	00000174 [8328]	short_addr _\$CHMOD ; 67 43	
3640	00000176 [3579]	short_addr _\$TOCTL ; 68 44	
3641	00000178 [5379]	short_addr _\$DUP ; 69 45	
3642	0000017A [D826]	short_addr _\$DUP2 ; 70 46	
3643		short_addr _\$CURRENT_DIR ; 71 47	
3644	0000017C [0973]	; Memory Group	
3645	0000017E [7E74]	short_addr _\$ALLOC ; 72 48	
3646	00000180 [5A74]	short_addr _\$DEALLOC ; 73 49	
3647		short_addr _\$SETBLOCK ; 74 4A	
3648	00000182 [E36B]	; Process Group	
3649	00000184 [4872]	short_addr _\$EXEC ; 75 4B	
3650	00000186 [D96B]	short_addr _\$EXIT ; 76 4C	
3651	00000188 [2226]	short_addr _\$WAIT ; 77 4D	
3652		short_addr _\$FIND_FIRST ; 78 4E	
3653	0000018A [7626]	; Special Group	
3654		short_addr _\$FIND_NEXT ; 79 4F	
3655		; SPECIAL SYSTEM GROUP	
3656		-+-----+	
3657		C A V E A T P R O G R A M M E R	
3658	0000018C [A202]	short_addr _\$SET_CURRENT_PDB ; 80 50	
3659	0000018E [AE02]	short_addr _\$GET_CURRENT_PDB ; 81 51	
3660	00000190 [3813]	short_addr _\$GET_IN_VARS ; 82 52	
3661	00000192 [6514]	short_addr _\$SETDPB ; 83 53	
3662			
3663		C A V E A T P R O G R A M M E R	
3664		-+-----+	
3665	00000194 [BE0C]	short_addr _\$GET_VERIFY_ON_WRITE ; 84 54	
3666		-+-----+	
3667		C A V E A T P R O G R A M M E R	
3668		-+-----+	
3669	00000196 [3E16]	short_addr _\$DUP_PDB ; 85 55	
3670			
3671		C A V E A T P R O G R A M M E R	
3672		-+-----+	
3673	00000198 [3481]	short_addr _\$RENAME ; 86 56	
3674	0000019A [6179]	short_addr _\$FILE_TIMES ; 87 57	
3675	0000019C [B374]	short_addr _\$ALLOCPER ; 88 58	
3676			
3677		; 08/07/2018 - Retro DOS v3.0	
3678			
3679		; MSDOS 6.0 - MSTABLE.ASM, 1991	
3680			
3681		; Network extention system calls	
3682	0000019E [B90F]	short_addr _\$GetExtendedError ; 89 59	
3683	000001A0 [C081]	short_addr _\$CreateTempFile ; 90 5A	
3684	000001A2 [A881]	short_addr _\$CreateNewFile ; 91 5B	
3685	000001A4 [8383]	short_addr _\$LockOper ; 92 5C	
3686	000001A6 [7E75]	short_addr _\$ServerCall ; 93 5D	

[illegible]

```

3842 ; -----
3843 ; BREAK <Copyright notice and version>
3844 ; -----
3845
3846 ;CODSTRT EQU $
3847
3848 ; 08/07/2018 - Retro DOS v3.0 by Erdogan Tan
3849 ; (MSTABLE.ASM, MSDOS 6.0, 1991)
3850
3851 ; NOTE WARNING: This declaration of HEADER must be THE LAST thing in this
3852 ; module. The reason is so that the data alignments are the same in
3853 ; IBM-DOS and MS-DOS up through header.
3854
3855 ;PUBLIC HEADER
3856
3857 HEADER: ; LABEL BYTE
3858 ;IF DEBUG
3859 ;DB 13,10,"Debugging DOS version "
3860 ;DB MAJOR_VERSION + "0"
3861 ;DB " "
3862 ;DB (MINOR_VERSION / 10) + "0"
3863 ;DB (MINOR_VERSION MOD 10) + "0"
3864 ;ENDIF
3865
3866 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3867 ; (MSDOS 5.0 MSDOS.SYS compatibility)
3868 %if 0
3869 ;IF NOT IBM
3870 DB 13,10,"MS-DOS version "
3871 DB MAJOR_VERSION + "0"
3872 DB " "
3873 DB (MINOR_VERSION / 10) + "0"
3874 DB (MINOR_VERSION MOD 10) + "0"
3875 DB (MINOR_VERSION % 10) + "0"
3876
3877 ;IF HIGHMEM
3878 DB "H"
3879 ;ENDIF
3880
3881 ;DB 13,10,"Copyright 1981,82,83,84,88 Microsoft Corp.",13,10,"$"
3882 ; 30/04/2019 - Retro DOS v4.0
3883 DB 13,10,"Copyright 1981-1993 Microsoft Corp.",13,10,"$"
3884
3885 ;ENDIF
3886
3887 %endif
3888
3889 ;IF DEBUG
3890 ; DB 13,10,"$"
3891 ;ENDIF
3892
3893 ;include copyrigh.inc
3894
3895 ; DOSCODE:3FDDh (MSDOS 6.21, MSDOS.SYS)
3896
3897 ;DB "MS DOS Version 6 (C)Copyright 1981-1993 Microsoft Corp "
3898 ;DB "Licensed Material - Property of Microsoft "
3899 ;DB "All rights reserved "
3900
3901 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3902 ; DOSCODE:3FCDh (MSDOS 5.0, MSDOS.SYS)
3903
3904 ; 28/12/2022 - Retro DOS v4.1
3905 %if 0
3906 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
3907 ms_copyright:
3908 db 'MS DOS Version 5.00 (C)Copyright 1981-1991 Microsoft Corp '
3909 db 'Licensed Material - Property of Microsoft '
3910 db 'All rights reserved '
3911
3912 %endif
3913
3914 ;; 28/12/2022 - Retro DOS v4.1
3915 ;ms_copyright:
3916 ;db 13,10,"MS DOS Version 5.0"
3917 ;db 13,10,"Copyright 1981-1991 Microsoft Corp.",13,10,"$",0
3918
3919 ; ; 21/09/2023 - Retro DOS v4.2 MSDOS.SYS
3920 ; ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:3FDDh (File offset: 509))
3921 ;ms_copyright:
3922 ; db 'MS DOS Version 6 (C)Copyright 1981-1994 Microsoft Corp '
3923 ; db 'Licensed Material - Property of Microsoft All rights reserved '
3924
3925 ; 01/01/2024 - Retro DOS v5.0
3926
3927 ; 20/09/2023 - Retro DOS v4.2
3928 ;ms_copyright:
3929 ;db 13,10,"MS DOS Version 6.22"
3930 ;db 13,10,"Copyright 1981-1994 Microsoft Corp.",13,10,"$",0
3931
3932 ;=====
3933 ; MSCODE.ASM
3934 ;=====
3935
3936 ; Retro DOS v2.0 (NASM 2.11) source code modifications by Erdogan Tan
3937 ; 03/03/2018
3938
3939 ;
3940 ; MSCODE.ASM -- MSDOS code
3941 ;
3942
3943 ;INCLUDE DOSSEG.ASM
3944 ;INCLUDE STDSW.ASM
3945
3946 ;CODE SEGMENT BYTE PUBLIC 'CODE'
3947 ;ASSUME CS:DOSGROUP,DS:NOTHING,ES:NOTHING,SS:NOTHING
3948
3949 ;.xcref
3950 ;INCLUDE DOSSYM.ASM
3951 ;INCLUDE DEVSYM.ASM
3952 ;.cref
3953 ;.list
3954
3955 ;IFNDEF KANJI
3956 ;KANJI EQU 0 ; FALSE
3957 ;ENDIF
3958
3959 ;IFNDEF IBM
3960 ;IBM EQU 0
3961 ;ENDIF
3962
3963 ;IFNDEF HIGHMEM
3964 ;HIGHMEM EQU 0
3965 ;ENDIF

```



```

3966
3967 ;i_need USER_SP,WORD
3968 ;i_need USER_SS,WORD
3969 ;i_need SAVEDS,WORD
3970 ;i_need SAVEBX,WORD
3971 ;i_need INDOS,BYTE
3972 ;i_need NSP,WORD
3973 ;i_need NSS,WORD
3974 ;i_need CURRENTPDB,WORD
3975 ;i_need AUXSTACK,BYTE
3976 ;i_need CONSWAP,BYTE
3977 ;i_need IDLEINT,BYTE
3978 ;i_need NOSETDIR,BYTE
3979 ;i_need ERRORMODE,BYTE
3980 ;i_need IOSTACK,BYTE
3981 ;i_need WPERR,BYTE
3982 ;i_need DSKSTACK,BYTE
3983 ;i_need CNTCFLAG,BYTE
3984 ;i_need LEAVEADDR,WORD
3985 ;i_need NULLDEVPT,DWORD
3986
3987 ;IF NOT IBM
3988 ;i_need OEM_HANDLER,DWORD
3989 ;ENDIF
3990
3991 ;EXTRN DSKSTATCHK:NEAR,GETBP:NEAR,DSKREAD:NEAR,DSKWRITE:NEAR
3992
3993 ;=====
3994 ; MSDISP.ASM, MSDOS 6.0, 1991
3995 ;=====
3996 ; 11/07/2018 - Retro DOS v3.0
3997 ; 01/05/2019 - Retro DOS v4.0
3998
3999 ; DosCode SEGMENT
4000
4001 ; =====
4002
4003 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4004 ; DOSCODE:4045h (MSDOS 5.0, MSDOS.SYS)
4005
4006 ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
4007 ; DOSCODE:4123h (PCDOS 7.1, IBMDOS.COM)
4008
4009 _$SET_CTRL_C_TRAPPING:
4010 ; 01/05/2019 - Retro DOS v4.0
4011
4012 0000023D 3C07 cmp al,7 ; 01/01/2024 - Retro DOS v5.0
4013 ;cmp AL,6 ; Is this a valid subfunction?
4014 0000023F 7603 jbe short scct_1 ; If yes continue processing
4015
4016 00000241 B0FF mov AL,0FFh ; Else set AL to -1 and
4017 00000243 CF iret
4018 scct_1:
4019 00000244 1E push DS
4020
4021 ;getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4022 00000245 2E8E1E[0700] mov ds,[cs:DosDSeg]
4023
4024 0000024A 50 push AX ; DL only register that can change
4025 0000024B 56 push SI
4026
4027 0000024C BE[3703] mov SI,CNTCFLAG ; DS:SI --> Ctrl C Status byte
4028 0000024F 30E4 xor AH,AH ; Clear high byte of AX
4029 00000251 09C0 or AX,AX ; Check for subfunction 0
4030 00000253 7504 jnz short scct_2 ; If not 0 jmp to next check
4031
4032 00000255 8A14 mov DL,[SI] ; Else move current ctrl C status
4033 00000257 EB32 jmp SHORT scct_9s ; into DL and jmp to exit
4034 scct_2:
4035 00000259 48 dec AX ; Now dec AX and see if it was 1
4036 0000025A 7507 jnz short scct_3 ; If not 0 it wasn't 1 so do next chk
4037
4038 0000025C 80E201 and DL,1 ; Else mask off bit 0 of DL and
4039 0000025F 8814 mov [SI],DL ; save it as new Ctrl C status
4040 00000261 EB28 jmp SHORT scct_9s ; jmp to exit
4041 scct_3:
4042 00000263 48 dec AX ; Dec AX again to see if it was 2
4043 00000264 7507 jnz short scct_4 ; If not 0 wasn't 2 so go to next chk
4044
4045 00000266 80E201 and DL,1 ; Else mask off bit 0 of DL and
4046 00000269 8614 xchg [SI],DL ; Exchange DL with old status byte
4047 0000026B EB1E jmp SHORT scct_9s ; Jump to exit (returning old status)
4048 scct_4:
4049 0000026D 3C03 cmp al,3 ; 01/01/2024
4050 ;cmp AX,3 ; Test for 5 after it was dec twice
4051 0000026F 7506 jne short scct_5 ; If not equal then not get boot drv
4052 00000271 8A16[6900] mov DL,[BOOTDRIVE] ; Else return boot drive in DL
4053 00000275 EB14 jmp SHORT scct_9s ; Jump to exit (returning boot drive)
4054 scct_5:
4055 ; 01/01/2024 - Retro DOS v5.0
4056 00000277 7212 jb short scct_9s ; PCDOS 7.1
4057 ;
4058 00000279 3C04 cmp al,4 ; 01/01/2024
4059 ;cmp AX,4 ; Test for 6 after it was dec twice
4060 ;jne short scct_9s ; If not equal then not get version
4061 0000027B 7512 jne short scct_6 ; 01/01/2024 ; PCDOS 7.1
4062
4063 ;mov BX,(Minor_Version SHL 8) + Major_Version
4064
4065 ;;mov bx,1406h ; 6.20 ; DOSCODE:4092h (MSDOS 6.21, MSDOS.SYS)
4066 ;;mov bx,1606h ; 6.22 ; DOSCODE:4092h (MSDOS 6.22, MSDOS.SYS)
4067
4068 ;mov bx,0A07h ; 7.10 ; DOSCODE:4163h (PCDOS 7.1, IMBDOS.COM)
4069 0000027D BB070A mov bx,(MINOR_VERSION<<8)+MAJOR_VERSION ; true dos version
4070
4071 ;mov dl,0 ; revision 0
4072 ;mov DL,DOSREVN ; 0
4073
4074 ;xor dh,dh ; assume vanilla DOS
4075 ; 01/01/2024
4076 00000280 BA0000 mov dx,0
4077 00000283 3836[660D] cmp byte [DosHashMA],dh ; 0
4078 ;cmp byte [DosHashMA],0 ; is DOS in HMA? (M021)
4079 ;;je short @F
4080 ;je short scct_6
4081 00000287 7402 je short scct_9s ; 01/01/2024
4082 ; 01/01/2024
4083 00000289 B610 mov dh,10h ; version flags bit 4
4084 ;or DH,DOSINHMA ; 10h ; 'DOS in HMA' status
4085 ;@@:
4086 ;scct_6:
4087 ;ifdef ROMDOS
4088 ; or DH,DOSINROM ; 08h
4089 ;endif ; ROMDOS

```

```

4090
4091 ; 01/01/2024 (PCDOS 7.1)
4092 ; jmp short short scct_9s
4093
4094 ; 01/01/2024
4095 ; scct_6:
4096 ; ....
4097
4098 scct_9s:
4099 pop SI
4100 pop AX
4101 pop DS
4102 scct_9f:
4103 iret
4104
4105 ; 01/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
4106 scct_6:
4107 and byte [DOS_FLAG],0DFh ; clear bit 5 of DOS flag
4108 cmp dl,1
4109 jne short scct_9s
4110 or byte [DOS_FLAG],20h ; set bit 5 of DOS flag
4111 jmp short scct_9s
4112
4113 SetCtrlShortEntry: ; This allows a conditional entry
4114 ; from main dispatch code
4115 jmp SHORT _$SET_CTRL_C_TRAPPING
4116
4117 ; =====
4118 ;
4119 ; The following two routines are dispatched to directly with ints disabled
4120 ; immediately after the int 21h entry. no DIS state is set.
4121 ;
4122 ; $Set_current_PDB takes BX and sets it to be the current process
4123 ; *** THIS FUNCTION CALL IS SUBJECT TO CHANGE!!! ***
4124 ;
4125 ; =====
4126
4127 _$SET_CURRENT_PDB:
4128 push DS
4129 ; getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4130 mov ds,[cs:DosDSeg]
4131 mov [CurrentPDB],BX ; Set new PSP segment from caller's BX
4132 pop DS
4133 iret
4134
4135 ; =====
4136 ;
4137 ; $get_current_PDB returns in BX the current process
4138 ; *** THIS FUNCTION CALL IS SUBJECT TO CHANGE!!! ***
4139 ;
4140 ; =====
4141
4142 _$GET_CURRENT_PDB:
4143 push DS
4144 ; getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4145 mov ds,[cs:DosDSeg]
4146 mov BX,[CurrentPDB] ; Return current PSP segment in BX
4147 pop DS
4148 iret
4149
4150 ; =====
4151 ;
4152 ; Sets the Printer Flag to whatever is in AL.
4153 ; NOTE: THIS PROCEDURE IS SUBJECT TO CHANGE!!!
4154 ;
4155 ; =====
4156
4157 _$SET_PRINTER_FLAG:
4158 push ds
4159 ; getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4160 mov ds,[cs:DosDSeg]
4161 mov [PRINTER_FLAG],AL ; set printer flag from caller's AL
4162 pop ds
4163 iret
4164
4165 ; 01/05/2019 - Retro DOS v4.0
4166 ; 08/07/2018 - Retro DOS v3.0
4167 ; (MSDISP.ASM, MSDOS 6.0, 1991)
4168
4169 ; -----
4170 ; BREAK <System call entry points and dispatcher>
4171 ; -----
4172
4173 ; DOSCODE:40CCh (MSDOS 6.21, MSDOS.SYS)
4174
4175 ; =====
4176 ;
4177 ; The Quit entry point is where all INT 20h's come from. These are old- style
4178 ; exit system calls. The CS of the caller indicates which Process is dying.
4179 ; The error code is presumed to be 0. We simulate an ABORT system call.
4180 ;
4181 ; =====
4182
4183 SYSTEM_CALL: ; PROC NEAR
4184
4185 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4186 ; DOSCODE:40BFh (MSDOS 5.0, MSDOS.SYS)
4187
4188 ; entry QUIT
4189 QUIT: ; INT 20H entry point
4190 ; MOV AH,0
4191 xor ah,ah ; 08/07/2018
4192 jmp SHORT SAVREGS
4193
4194 ; -----
4195 ;
4196 ; The system call in AH is out of the range that we know how
4197 ; to handle. We arbitrarily set the contents of AL to 0 and
4198 ; IRET. Note that we CANNOT set the carry flag to indicate an
4199 ; error as this may break some programs compatability.
4200
4201 BADCALL:
4202 ; MOV AL,0
4203 xor al,al ; 08/07/2018
4204 IRET: ; 06/05/2019
4205 _IRET:
4206 iret
4207
4208 ; -----
4209 ;
4210 ; 01/05/2019 - Retro DOS v4.0
4211 ; DOSCODE:40D3h (MSDOS 6.21 MSDOS.SYS)
4212 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4213 ; DOSCODE:40C6h (MSDOS 5.0 MSDOS.SYS)

```

```

4214
4215 ; An alternative method of entering the system is to perform a
4216 ; CALL 5 in the program segment prefix with the contents of CL
4217 ; indicating what system call the user would like. A subset of
4218 ; the possible system calls is allowed here only the
4219 ; CPM-compatible calls may get dispatched.
4220
4221 ; System call entry point and dispatcher
4222 CALL_ENTRY:
4223 000002CC 1E      push    DS
4224 ;getdseg <DS>      ; DS -> DosData, ASSUME DS:DosSeg
4225 000002CD 2E8E1E[0700] mov     ds,[cs:DosDSeg]
4226 000002D2 8F06[EC05] pop     word [SAVEDS]      ; Save original DS
4227
4228 000002D6 58      POP     AX      ; IP from the long call at 5
4229 000002D7 58      POP     AX      ; Segment from the long call at 5
4230 000002D8 8F06[8405] POP     WORD [USER_SP]    ; IP from the CALL 5
4231
4232 ; Re-order the stack to simulate an interrupt 21.
4233
4234 000002DC 9C      PUSHF      ; Start re-ordering the stack
4235 000002DD FA      CLI
4236 000002DE 50      PUSH     AX      ; Save segment
4237 000002DF FF36[8405] PUSH     WORD [USER_SP]    ; Stack now ordered as if INT had been used
4238 ; 04/11/2022
4239 ; DOSCODE:40EAh (MSDOS 6.21 MSDOS.SYS)
4240 ; DOSCODE:40DDh (MSDOS 5.0 MSDOS.SYS)
4241 000002E3 FF36[EC05] push     word [SAVEDS]
4242 000002E7 1F      pop     ds
4243 ;
4244 ; cmp     cl,36
4245 000002E8 80F924      CMP     CL,MAXCALL      ; This entry point doesn't get as many calls
4246 000002EB 77DC      JA      SHORT BADCALL
4247 000002ED 88CC      MOV     AH,CL
4248 ; 08/07/2018
4249 000002EF EB0E      jmp     short SAVREGS
4250
4251 ; -----
4252
4253 ; 01/05/2019 - Retro DOS v4.0
4254 ; 01/01/2024 - Retro DOS v5.0
4255
4256 ; This is the normal INT 21 entry point. We first perform a
4257 ; quick test to see if we need to perform expensive DOS-entry
4258 ; functions. Certain system calls are done without interrupts
4259 ; being enabled.
4260
4261 ;entry COMMAND      ; Interrupt call entry point (int 21h)
4262
4263 ; DOSCODE:40F8h (MSDOS 6.21, MSDOS.SYS)
4264 ; 04/11/2022
4265 ; DOSCODE:40EBh (MSDOS 5.0, MSDOS.SYS)
4266 ; 01/01/2024
4267 ; DOSCODE:41D7h (PCDOS 7.1, IBMDOS.COM)
4268
4269 COMMAND:
4270 ; 22/12/2022
4271 000002F1 FA      cli
4272
4273 ; 01/05/2019 - Retro DOS v4.0
4274 ; 08/07/2018 - Retro DOS v3.0
4275
4276 ; 22/12/2022
4277 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4278 ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
4279
4280 ; IF NOT IBM
4281 000002F2 80FCF8      CMP     AH,SET_OEM_HANDLER
4282 000002F5 7203      JB      SHORT NOTOEM
4283 000002F7 E98701      JMP     _$SET_OEM_HANDLER
4284
4285 NOTOEM:
4286 ;ENDIF
4287
4288 ; DOSCODE:40F8h (MSDOS 6.21, MSDOS.SYS)
4289 ; DOSCODE:40EBh (MSDOS 5.0, MSDOS.SYS)
4290 ; 01/01/2024 - Retro DOS v5.0
4291 ; DOSCODE:41D7h (PCDOS 7.1, IBMDOS.COM)
4292
4293 ; 22/12/2022
4294 ; cli      ; 08/07/2018
4295
4296 ; 01/01/2024
4297 _COMMAND: ; MSDOS 3.3 (IBMDOS)
4298 ; cmp     ah,6Ch ; MSDOS 6.21
4299 ; cmp     ah,73h ; PCDOS 7.1 ; Max int 21h function call number
4300 ; 04/11/2022
4301 000002FA 80FC73      CMP     AH,MAXCOM ; 6Ch for MSDOS 6.0 (6.21,6.22) & MSDOS 5.0
4302 ; 73h for PCDOS 7.1
4303 ; JBE     SHORT SAVREGS
4304 000002FD 77CA      JA      SHORT BADCALL ; 08/07/2018
4305
4306 ; 31/05/2019
4307
4308 ; The following set of calls are issued by the server at
4309 ; *arbitrary* times and, therefore, must be executed on
4310 ; the user's entry stack and executed with interrupts off.
4311
4312 SAVREGS:
4313 ; 01/05/2019 - Retro DOS v4.0
4314 ; 10/08/2018
4315 ; 08/07/2018 - Retro DOS v3.0
4316 000002FF 80FC33      cmp     ah,33h      ; Check Minimum special case #
4317 ; je      _$SET_CTRL_C_TRAPPING
4318 ; je      short SetCtrlShortEntry ; If equal jmp directly to function
4319 00000302 7218      jb      short SaveAllRegs ; Not special case so continue
4320 ; 04/11/2022
4321 je      short SetCtrlShortEntry ; If equal jmp directly to function
4322 cmp     ah,64h      ; Check Max case number
4323 ja      short SaveAllRegs ; Not special case so continue
4324 0000030B 74AD      je      short _$SET_PRINTER_FLAG ; If equal jmp directly to function
4325 0000030D 80FC51      cmp     ah,51h      ; Is this a Get PSP call (51h)?
4326 00000310 749C      je      short _$GET_CURRENT_PDB ; Yes, jmp directly to function
4327 ; 25/06/2024
4328 cmp     ah,50h      ; Is this a Set PSP call (50h) ?
4329 00000315 748B      je      short _$SET_CURRENT_PDB ; Yes, jmp directly to function
4330 00000317 80FC62      cmp     ah,62h      ; Is this a Get PSP call (62h)?
4331 0000031A 7492      je      short _$GET_CURRENT_PDB ; Yes, jmp directly to function
4332
4333 SaveAllRegs:
4334 ; 01/05/2019 - Retro DOS v4.0
4335 ; 01/01/2024 - Retro DOS v5.0
4336
4337 0000031C 06      push     ES

```

```

4338 0000031D 1E      push    DS
4339 0000031E 55      push    BP
4340 0000031F 57      push    DI
4341 00000320 56      push    SI
4342 00000321 52      push    DX
4343 00000322 51      push    CX
4344 00000323 53      push    BX
4345 00000324 50      push    AX
4346
4347 00000325 8CD8      mov     AX,DS
4348      ;getdseg <DS>          ; DS -> DosData, ASSUME DS:DosSeg
4349 00000327 2E8E1E[0700]  mov     ds,[cs:DosDSeg]
4350 0000032C A3[EC05]      mov     [SAVEDS],AX      ; save caller's DS
4351 0000032F 891E[EA05]      mov     [SAVEBX],BX
4352
4353      ;INC     BYTE [INDOS]          ; Flag that we're in the DOS
4354
4355      ; 08/07/2018 - Retro DOS v3.0
4356      ;xor     ax,ax
4357      ;mov     [USER_ID],ax
4358      ;mov     ax,[CurrentPDB]
4359      ;mov     [PROC_ID],ax
4360
4361      ; 01/05/2019
4362
4363      ; Note: Nsp and Nss have to be unconditionally initialized here
4364      ; even if IndOS is zero. Programs like CROSSTALK 3.7 depend on
4365      ; this!!!
4366
4367 00000333 A1[8405]      MOV     AX,[USER_SP]
4368 00000336 A3[F205]      MOV     [NSP],AX
4369 00000339 A1[8605]      MOV     AX,[USER_SS]
4370 0000033C A3[F005]      MOV     [NSS],AX
4371
4372 0000033F 31C0      xor     AX,AX ; 0
4373 00000341 A2[7205]      mov     [FSHARING],AL      ; allow redirection
4374
4375 00000344 F606[5B0F]01  test    byte [IsWin386],1    ; WIN386 patch. Do not update USER_ID
4376 00000349 7503      jnz     short set_indos_flag ; if win386 present
4377 0000034B A3[3E03]      mov     [USER_ID],AX
4378      set_indos_flag:
4379 0000034E FE06[2103]  INC     BYTE [INDOS]          ; Flag that we're in the DOS
4380
4381      ; 01/01/2024 (PCDOS 7.1 IBMDOS.COM)
4382 00000352 FE06[B812]  inc     byte [INDOS_FLAG]      ; duplicated INDOS flag (what for ?)
4383
4384 00000356 8926[8405]      MOV     [USER_SP],SP
4385 0000035A 8C16[8605]      MOV     [USER_SS],SS
4386
4387 0000035E A1[3003]      mov     AX,[CurrentPDB]
4388 00000361 A3[3C03]      mov     [PROC_ID],AX
4389 00000364 8ED8      mov     DS,AX
4390 00000366 58      pop     AX
4391 00000367 50      push    AX
4392
4393      ; save user stack in his area for later returns (possibly from EXEC)
4394
4395 00000368 89262E00      MOV     [PDB.USER_STACK],SP      ; mov [2Eh], sp
4396 0000036C 8C163000      MOV     [PDB.USER_STACK+2],SS    ; mov [30h], ss
4397
4398      ; 18/07/2018
4399      ;mov     byte [CS:FSHARING], 0
4400
4401      ;MOV     BX,CS          ; no holes here.
4402      ;MOV     SS,BX
4403
4404      ;getdseg <ss>          ; ss -> dosdat, already flag is CLI
4405 00000370 2E8E16[0700]  mov     ss,[cs:DosDSeg]
4406      ;entry REDISP
4407
4408 00000375 BC[A007]      REDISP: MOV     SP,AUXSTACK      ; Enough stack for interrupts
4409 00000378 FB      STI          ; stack is in our space now...
4410
4411 00000379 8CD3      mov     bx,ss
4412 0000037B 8EDB      mov     ds,bx
4413
4414 0000037D 93      xchg     ax,bx
4415
4416 0000037E 31C0      xor     ax,ax ; 0
4417
4418      ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4419      ; MSDOS 5.0 MSDOS.SYS - DOSCODE:416Eh (from org 3DD0h)
4420      ; MSDOS 6.21 MSDOS.SYS - DOSCODE:417Bh (from org 3DE0h)
4421
4422      ; (Note: ss: segment prefix was not needed here! ds=ss ! -04/11/2022-)
4423
4424      ;mov     [ss:EXTOPEN_ON],al ; 0; Clear extended open flag
4425      ;and     word [ss:DOS34_FLAG],EXEC_AWARE_REDIR
4426      ;and     word [ss:DOS34_FLAG],800h ; clear all bits except bit 11
4427      ;mov     [ss:CONSWAP],al ; 0 ; random clean up of possibly mis-set flags
4428      ;mov     [ss:NoSetDir],al ; 0 ; set directories on search
4429      ;mov     [ss:FAILERR],al ; 0 ; FAIL not in progress
4430      ;inc     ax
4431      ;inc     AL          ; AL = 1
4432      ;mov     [ss:IDLEINT],al ; presume that we can issue INT 28h
4433
4434      ; 15/12/2022
4435 00000380 A2[F605]      mov     [EXTOPEN_ON],al ; 0 ; Clear extended open flag
4436      ;and     word [DOS34_FLAG],EXEC_AWARE_REDIR
4437 00000383 8126[1106]0008  and     word [DOS34_FLAG],800h ; clear all bits except bit 11
4438 00000389 A2[5703]      mov     [CONSWAP],al ; 0 ; random clean up of possibly mis-set flags
4439      ;mov     byte [IDLEINT],1
4440 0000038C A2[4C03]      mov     [NoSetDir],al ; 0 ; set directories on search
4441 0000038F A2[4A03]      mov     [FAILERR],al ; 0 ; FAIL not in progress
4442 00000392 40      inc     ax
4443      ;inc     al          ; AL = 1
4444 00000393 A2[5803]      mov     [IDLEINT],al ; presume that we can issue INT 28h
4445
4446 00000396 93      XCHG     AX,BX          ; Restore AX and BX = 1
4447
4448 00000397 88E3      MOV     BL,AH
4449 00000399 D1E3      SHL     BX,1          ; 2 bytes per call in table
4450
4451 0000039B FC      CLD
4452      ; Since the DOS maintains mucho state information across system
4453      ; calls, we must be very careful about which stack we use.
4454      ; First, all abort operations must be on the disk stack. This
4455      ; is due to the fact that we may be hitting the disk (close
4456      ; operations, flushing) and may need to report an INT 24.
4457
4458 0000039C 08E4      OR      AH,AH
4459 0000039E 7416      JZ      SHORT DSKROUT      ; ABORT
4460
4461      ;CMP     AH,12

```

```

4462             ;JBE      SHORT IOROUT          ; Character I/O
4463             ;CMP      AH,GET_CURRENT_PDB    ; INT 24h needs GET,SET PDB
4464             ;JZ SHORT IOROUT
4465             ;CMP      AH,SET_CURRENT_PDB
4466             ;JNZ      SHORT DSKROUT
4467
4468             ; Second, PRINT and PSPRINT and the server issue
4469             ; GetExtendedError calls at INT 28 and INT 24 time.
4470             ; This call MUST, therefore, use the AUXSTACK.
4471
4472             ; 10/08/2018
4473 000003A0 80FC59 cmp     ah,GETEXTENDEDERROR ; 59h
4474 000003A3 7439 je      short DISPCALL
4475
4476             ; 01/05/2019
4477
4478             ; Old 1-12 system calls may be either on the IOSTACK (normal
4479             ; operation) or on the AUXSTACK (at INT 24 time).
4480
4481 000003A5 80FC0C cmp     ah,12 ; STD_CON_INPUT_FLUSH ; 0Ch
4482 000003A8 770C ja      short DSKROUT
4483
4484 IOROUT:
4485             ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
4486             ; (ss: prefix was not needed here! ds=ss)
4487             ; cmp     byte [ss:ERRORMODE],0 ; Are we in an INT 24h?
4488             ; 15/12/2022
4489 000003AA 803E[2003]00 cmp     BYTE [ERRORMODE],0 ; Are we in an INT 24h?
4490 000003AF 752D jnz     short DISPCALL ; Stay on AUXSTACK if INT 24h
4491 000003B1 BC[A00A] mov     sp,IOSTACK
4492 000003B4 EB28 jmp     short DISPCALL
4493
4494             ; We are on a system call that is classified as "the rest".
4495             ; We place ourselves onto the DSKSTACK and away we go.
4496             ; We know at this point:
4497             ; * An INT 24 cannot be in progress. Therefore we reset
4498             ;   ErrorMode and WpErr
4499             ; * That there can be no critical sections in effect.
4500             ;   We signal the server to remove all the resources.
4501
4502 DSKROUT:
4503             ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
4504             ; 15/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4505             ; 08/07/2018 - Retro DOS v3.0
4506 000003B6 A3[3A03] mov     [USER_IN_AX],ax ; Remember what user is doing
4507             ; 01/01/2024
4508             ; mov     byte [EXTERR_LOCUS],1 ; errLOC_Unk (Default)
4509             ; MOV     BYTE [WPERR],-1 ; error mode, so good place to
4510             ; ; make sure flags are reset
4511 000003B9 C706[2203]FF01 mov     word [WPERR],1FFh
4512
4513 000003BF C606[2003]00 MOV     BYTE [ERRORMODE],0 ; Cannot make non 1-12 calls in
4514
4515             ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
4516             ; (ss: prefix was not needed here! ds=ss)
4517
4518             ; mov     [ss:USER_IN_AX],ax ; Remember what user is doing
4519             ; mov     byte [ss:EXTERR_LOCUS],1 ; errLOC_Unk (Default)
4520             ; mov     byte [ss:ERRORMODE],0 ; Cannot make non 1-12 calls in
4521             ; mov     byte [ss:WPERR],-1 ; error mode, so good place to
4522             ; ; make sure flags are reset
4523
4524 000003C4 50 push     ax
4525 000003C5 B482 mov     ah,82h ; Release all resource information
4526 000003C7 CD2A int     2Ah ; Microsoft Networks
4527             ; END DOS CRITICAL SECTIONS 0 THROUGH 7
4528 000003C9 58 pop     ax
4529
4530             ; Since we are going to be running on the DSKStack and since
4531             ; INT 28 people will use the DSKStack, we must turn OFF the
4532             ; generation of INT 28's.
4533
4534             ; 15/12/2022
4535             ; mov     byte [ss:IDLEINT],0
4536             ;
4537             ; mov     sp,DSKSTACK
4538             ; test    byte [ss:CNTCFLAG],-1 ; 0FFh
4539             ; jz short DISPCALL
4540
4541 000003CA C606[5803]00 mov     byte [IDLEINT],0
4542
4543 000003CF BC[2009] MOV     SP,DSKSTACK
4544 000003D2 F606[3703]FF TEST     BYTE [CNTCFLAG],-1
4545 000003D7 7405 JZ      SHORT DISPCALL
4546
4547 000003D9 50 PUSH     AX
4548             ; invoke DSKSTATCHK
4549 000003DA E8225A CALL     DSKSTATCHK
4550 000003DD 58 POP     AX
4551
4552 DISPCALL:
4553             ; 01/05/2019 - Retro DOS v4.0
4554             ; mov     bx,[CS:BX+DISPATCH]
4555
4556             ; 15/12/2022
4557 000003E3 871E[EA05] xchg     bx,[SAVEBX]
4558 000003E7 8E1E[EC05] MOV     DS,[SAVEDS]
4559
4560             ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
4561             ; (ss: prefix was not needed here! ds=ss)
4562             ; xchg     bx,[ss:SAVEBX]
4563             ; mov     ds,[ss:SAVEDS]
4564 000003EB 36FF16[EA05] call     word [ss:SAVEBX] ; near call
4565
4566             ; The EXEXA200FF bit of DOS_FLAG will now be unconditionally cleared
4567             ; here. Please see under M003, M009 and M068 tags in dossym.inc
4568             ; for explanation. Also NOTE that a call to ExecReady (ax=4b05) will
4569             ; return to LeaveDOS and hence will not clear this bit. This is
4570             ; because this bit is used to indicate to the next int 21 call that
4571             ; the previous int 21 was an exec.
4572             ;
4573             ; So do not add any code between the call above and the label
4574             ; LeaveDOS if it needs to be executed even for ax=4b05
4575
4576             ; and byte [ss:DOS_FLAG],~EXECA200FF
4577             ; and byte [ss:DOS_FLAG],0FBh ; clear bit 2
4578
4579             ; 01/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
4580 000003F0 368026[8600]DB and     byte [ss:DOS_FLAG],0DBh ; clear bit 2 and bit 5
4581
4582             ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4583             ; DOSCODE:41F7h
4584
4585             ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)

```

```

4586 ; DOSCODE:42D4h
4587
4588 ;entry LEAVE
4589 ;;;_LEAVE: ; Exit from a system call
4590 LeaveDOS: ; 18/07/2018
4591 ;ASSUME SS:NOTHING ; User routines may misbehave
4592 CLI
4593
4594 ; 01/05/2019
4595 ;getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4596 mov ds,[cs:DosDSeg]
4597 cmp byte [A20OFF_COUNT],0 ; M068: Q: is count 0
4598 jne short disa20 ; M068: N: dec count and turn a20 off
4599
4600 LeaveA20on:
4601 DEC BYTE [INDOS]
4602
4603 ; 01/01/2024 (PCDOS 7.1 IBMDOS.COM)
4604 dec byte [INDOS_FLAG] ; duplicated INDOS flag (what for ?)
4605
4606 ; 04/11/2022
4607 mov ss,[USER_SS]
4608 MOV SP,[USER_SP]
4609 ;MOV SS,[USER_SS]
4610 MOV BP,SP
4611 ;MOV [BP.user_AX],AL
4612 ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
4613 ;mov [bp+0],al ; MSDOS 5.0 MSDOS.SYS - DOSCODE:4212h
4614 ;MOV [BP+user_env.user_AX],AL ; user_env.user_AX = 0
4615
4616 ; 15/12/2022
4617 MOV [BP],AL ; mov [bp+0],al
4618
4619 ;MOV AX,[NSP]
4620 ;MOV [USER_SP],AX
4621 ;MOV AX,[NSS]
4622 ;MOV [USER_SS],AX
4623
4624 ; 01/01/2024
4625 les ax,[NSS]
4626 mov [USER_SS],ax
4627 mov [USER_SP],es
4628 pop AX
4629 pop BX
4630 pop CX
4631 pop DX
4632 pop SI
4633 pop DI
4634 pop BP
4635 pop DS
4636 pop ES
4637
4638 IRET
4639
4640 disa20: ; M068 - Start
4641 mov bx,[A20OFF_PSP] ; bx = PSP for which a20 to be off'd
4642 cmp bx,[CurrentPDB] ; Q: do the PSP's match
4643 jne short LeaveA20on ; N: don't clear bit and don't turn
4644 ; a20 off
4645 ; Y: turn a20 off and dec a20off_count
4646 dec byte [A20OFF_COUNT] ; M068 - End
4647 ; Start - M004
4648 push ds ; segment of stub
4649 mov bx,disa20_iret ; offset in stub
4650 push bx
4651 retf ; go to stub
4652 ; End - M004
4653
4654 ;SYSTEM_CALL ENDP
4655
4656 ; DOSCODE:424Ch (MSDOS 6.21, MSDOS.SYS)
4657 ; 04/11/2022
4658 ; DOSCODE:423Fh (MSDOS 5.0, MSDOS.SYS)
4659
4660 ; =====
4661 ; Restore_world restores all registers ('cept SS:SP, CS:IP, flags) from
4662 ; the stack prior to giving the user control
4663 ; =====
4664
4665 ; 01/05/2019 - Retro DOS v4.0
4666
4667 ;procedure restore_world,NEAR
4668 restore_world:
4669 ;getdseg <es> ; es -> dosdata
4670 mov es,[cs:DosDSeg]
4671
4672 POP WORD [ES:RESTORE_TMP]
4673
4674 POP AX
4675 POP BX
4676 POP CX
4677 POP DX
4678 POP SI
4679 POP DI
4680 POP BP
4681 POP DS
4682
4683 jmp word [ES:RESTORE_TMP]
4684
4685 ;restore_world ENDP
4686
4687 ; 01/05/2019 - Retro DOS v4.0 (MSDOS 6.0, MSDISP.ASM, 1991)
4688
4689 ; DOSCODE:4263h (MSDOS 6.21, MSDOS.SYS)
4690 ; 04/11/2022
4691 ; DOSCODE:4256h (MSDOS 5.0, MSDOS.SYS)
4692
4693 ; =====
4694 ; Save_world saves complete registers on the stack
4695 ; =====
4696
4697 ;procedure save_world,NEAR
4698 save_world:
4699 ;getdseg <es> ; es -> dosdata
4700 mov es,[cs:DosDSeg]
4701
4702 POP WORD [ES:RESTORE_TMP]
4703
4704 ; 12/05/2019
4705 PUSH DS
4706
4707
4708
4709

```



```

4710 00000463 55          PUSH    BP
4711 00000464 57          PUSH    DI
4712 00000465 56          PUSH    SI
4713 00000466 52          PUSH    DX
4714 00000467 51          PUSH    CX
4715 00000468 53          PUSH    BX
4716 00000469 50          PUSH    AX
4717
4718 0000046A 26FF36[EE05]      push    word [ES:RESTORE_TMP]
4719
4720 0000046F 55          push    BP
4721 00000470 89E5        mov     BP,SP
4722 00000472 8E4614      mov     ES,[BP+20]      ; es was pushed before call
4723 00000475 5D          pop     BP
4724
4725 00000476 C3          retn
4726
4727 ;save_world ENDP
4728
4729 ; 01/05/2019
4730
4731 ; DOSCODE:4282h (MSDOS 6.21, MSDOS.SYS)
4732 ; 04/11/2022
4733 ; DOSCODE:4275h (MSDOS 5.0, MSDOS.SYS)
4734
4735 ; =====
4736 ;
4737 ; Get_User_Stack returns the user's stack (and hence registers) in DS:SI
4738 ;
4739 ; =====
4740
4741 ;procedure get_user_stack,NEAR
4742 Get_User_Stack:
4743 ;getdseg <DS>          ; DS -> DosData, ASSUME DS:DosSeg
4744 mov     ds,[cs:DosDSeg]
4745 lds     si,[USER_SP]
4746 retn
4747
4748 ;get_user_stack ENDP
4749
4750 ; 22/12/2022
4751 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0, MSDOS.SYS)
4752 ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1, IBMDOS.COM)
4753 ;%if 0
4754
4755 ; -----
4756 ;
4757 ; Set_OEM_Handler -- Set OEM sys call address and handle OEM calls
4758 ; Inputs:
4759 ;   User registers, User Stack, INTS disabled
4760 ;   If CALL F8, DS:DX is new handler address
4761 ; Function:
4762 ;   Process OEM INT 21 extensions
4763 ; Outputs:
4764 ;   Jumps to OEM_HANDLER if appropriate
4765 ;
4766 ; -----
4767
4768 ;IF NOT IBM
4769
4770 _$SET_OEM_HANDLER:
4771 ; 01/05/2019 - Retro DOS v4.0
4772
4773 ;(cmp  ah,SET OEM HANDLER ; 0F8h)
4774 ;(jb  short NOTOOEM)
4775
4776 00000481 06          push    es ; *
4777 ;getdseg <es>          ; es -> dosdata
4778 00000482 2E8E06[0700]  mov     es,[cs:DosDSeg]
4779
4780 00000487 750C        jne     short check_trueversion_request ; check Retro DOS true version
4781 ; (message) request
4782 ; AH = 0F8h = SET OEM HANDLER
4783
4784 00000489 268916[1400]  MOV     [es:OEM_HANDLER],DX ; Set Handler
4785 0000048E 268C1E[1600]  MOV     [es:OEM_HANDLER+2],DS
4786
4787 00000493 07          pop     es ; *
4788
4789 00000494 CF          IRET          ; Quick return, Have altered no registers
4790
4791 check_trueversion_request:
4792 ; 18/07/2019 - Retro DOS v3.0
4793
4794 ; Retro DOS v2.0 - 20/04/2018
4795 00000495 83F8FF      CMP     AX,0FFFFh
4796 ; 18/07/2018
4797 00000498 7520        jne     short DO_OEM_FUNC ; 01/05/2019
4798
4799 ; 01/05/2019
4800 0000049A 07          pop     es ; *
4801
4802 0000049B B40E        mov     ah,0Eh
4803
4804 ; Retro DOS v4.0 feature only!
4805 0000049D 81FBA101    cmp     bx,417 ; Signature to bypass
4806 ; Retro DOS true version message
4807 000004A1 7414        je      short true_version_iret
4808
4809 000004A3 56          push    si
4810 000004A4 53          push    bx
4811
4812 000004A5 BE[C200]  mov     si,RETRODOSMSG
4813 wrdosmsg:
4814 ;movb  ah,0Eh
4815 000004A8 BB0700  mov     bx,7
4816 wrdosmsg_nxt:
4817 000004AB 2EAC        cs lodsb
4818 000004AD 3C24        cmp     al,'$'
4819 000004AF 7404        je      short wrdosmsg_ok
4820 000004B1 CD10        int     10h
4821 000004B3 EBF6        jmp     short wrdosmsg_nxt
4822
4823 wrdosmsg_ok:
4824 000004B5 5B          pop     bx
4825 000004B6 5E          pop     si
4826
4827 true_version_iret:
4828 ; ah = 0Eh
4829 ;mov  al,40h ; Retro DOS v4.0
4830 ;
4831 ;mov  al,41h ; Retro DOS v4.1
4832 ; 30/12/2022
4833 ;mov  al,42h ; Retro DOS v4.2

```

```

4834             ; 01/01/2024
4835 000004B7 B050      mov     al,50h ; Retro DOS v5.0
4836 000004B9 CF       ired
4837
4838             ; If above F8 try to jump to handler
4839
4840 DO_OEM_FUNC:
4841             ; 01/05/2019
4842 000004BA 26833E[1400]FF      cmp     word [es:OEM_HANDLER],-1
4843 000004C0 7504             jne     short OEM_JMP
4844 000004C2 07             pop     es ; *
4845 000004C3 E903FE             jmp     BADCALL ; Handler not initialized
4846
4847 OEM_JMP:
4848 000004C6 06             push    es
4849 000004C7 1F             pop     ds ; DOSDATA segment !
4850 000004C8 07             pop     es ; *
4851
4852             ; 22/12/2022
4853 000004C9 FB             sti     ; (enable interrupts before jumping to private handler)
4854 000004CA FF2E[1400]      jmp     far [OEM_HANDLER]
4855
4856             ;
4857             ; -----
4858             ;
4859             ;%endif
4860
4861             ;=====
4862             ; MCODE.ASM, MSDOS 6.0, 1991
4863             ;=====
4864             ; 17/07/2018 - Retro DOS v3.0
4865
4866             ;
4867             ; TITLE MISC DOS ROUTINES - Int 25 and 26 handlers and other
4868             ; NAME IBMCODE
4869
4870             ;BREAK <NullDev -- Driver for null device>
4871
4872             ; ROMDOS note:
4873             ; NUL device driver used to be here, but it was removed and placed in
4874             ; DOSDATA, because the entry points have to be in the segment as the
4875             ; header, which is also in DOSDATA.
4876
4877             ;BREAK <AbsDRD, AbsDWRT -- INT int_disk_read, int_disk_write handlers>
4878
4879             ; -----
4880             ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0, MSDOS.SYS)
4881             ; -----
4882             ; DOSCODE:428Ch (MSDOS 6.21 MSDOS.SYS)
4883             ; DOSCODE:427Fh (MSDOS 5.0 MSDOS.SYS)
4884
4885             ; 01/01/2024 - Retro DOS v5.0
4886             ; DOSCODE:435Fh (PCDOS 7.1 IBMDOS.COM)
4887
4888             ;Public MSC001S, MSC001E
4889             ;MSC001S label byte
4890             ;IF IBM
4891             ; Codes returned by BIOS
4892 ERRIN:
4893 000004CE 02             db      2 ; NO RESPONSE
4894 000004CF 06             db      6 ; SEEK FAILURE
4895 000004D0 0C             db      12 ; GENERAL ERROR
4896 000004D1 04             db      4 ; BAD CRC
4897 000004D2 08             db      8 ; SECTOR NOT FOUND
4898 000004D3 00             db      0 ; WRITE ATTEMPT ON WRITE-PROTECT DISK
4899
4900 ERRROUT:
4901             ; DISK ERRORS RETURNED FROM INT 25 and 26
4902 000004D4 80             db      80h ; NO RESPONSE
4903 000004D5 40             db      40h ; Seek failure
4904 000004D6 02             db      2 ; Address Mark not found
4905 000004D7 10             db      10h ; BAD CRC
4906 000004D8 04             db      4 ; SECTOR NOT FOUND
4907 000004D9 03             db      3 ; WRITE ATTEMPT TO WRITE-PROTECT DISK
4908
4909 NUMERR EQU $-ERRROUT
4910             ;ENDIF
4911             ;MSC001E label byte
4912             ; -----
4913             ;=====
4914             ; MCODE.ASM - MSDOS 6.0 - 1991
4915             ;=====
4916             ; 18/07/2018 - Retro DOS v3.0
4917             ; 15/05/2019 - Retro DOS v4.0
4918
4919             ; 02/01/2024 - Retro DOS v5.0
4920             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:436Bh
4921
4922             ;BREAK <AbsDRD, AbsDWRT -- INT int_disk_read, int_disk_write handlers>
4923
4924             ; AbsSetup - setup for abs disk functions
4925             ; -----
4926
4927 AbsSetup:
4928             ; 02/01/2024 (PCDOS 7.1, Retro DOS 5.0)
4929             ;;;
4930             ;
4931 000004DA 1E             push    ds ; *
4932 000004DB 16             push    ss
4933 000004DC 1F             pop     ds
4934             ;
4935 000004DD 8826[DB0A]      mov     [absdrw_extd],ah
4936             ;mov     [ss:absdrw_extd],ah ; Extended ABS Disk Read/Write flag
4937             ; (AH=1 for INT 21h ax=7305h function)
4938 000004E1 08E4             or      ah,ah
4939 000004E3 7508             jnz     short AbsSetup1 ; INT 21h AX=7305h
4940
4941             ; INT 25h
4942 000004E5 FE06[B812]      inc     byte [INDOS_FLAG]
4943             ;inc     byte [ss:INDOS_FLAG] ; windows DOSBOX's INDOS flag ?
4944             ;;;
4945 000004E9 FE06[2103]      inc     byte [INDOS]
4946             ;inc     byte [ss:INDOS] ; SS override
4947 AbsSetup1:
4948 STI
4949 CLD
4950             ; 02/01/2024
4951             ;PUSH DS
4952             ;push    ss
4953             ;pop     ds
4954 000004EF E83C01      call    GETBP
4955             ; 02/01/2024
4956 000004F2 1F             pop     ds ; *
4957 000004F3 7229             jc      short errdriv ; PM. error drive ;AN000;

```



```

4958
4959
4960 ; 02/01/2024
4961 ;mov word [es:bp+1Fh]
4962 ;MOV WORD [ES:BP+DPB.FREE_CNT],-1 ; do not trust user at all.
4963 ;errdriv:
4964 ;POP DS
4965 ;jnc short AbsSetup2
4966 ;AbsSetup_retn:
4967 ;retn
4968
4969 AbsSetup2:
4970 ; 15/05/2019 - Retro DOS v4.0
4971 ; MSDOS 6.0
4972 ; SS override
4973 MOV word [SS:HIGH_SECTOR],0 ;>32mb from API ;AN000;
4974 CALL RW32_CONVERT ;>32mb convert 32bit format to 16bit ;AN000;
4975 ;jc short AbsSetup_retn
4976 ;call SET_RQ_SC_PARAMS ;LB. set up SC parms ;AN000;
4977 ; 02/01/2024 - Retro DOS v5.0
4978 ;jc short errdriv
4979 ;call null_sub ; retn ; PCDOS 7.1 IBMDOS.COM
4980
4981 ; MSDOS 3.3 (& MSDOS 6.0)
4982 PUSH DS
4983 PUSH SI
4984 PUSH AX
4985
4986 push ss
4987 pop ds
4988 MOV SI,OPENBUF
4989 MOV [SI],AL
4990 ADD BYTE [SI],"A"
4991 MOV WORD [SI+1],003AH ; ":" ,0
4992 MOV AX,0300H
4993 CLC
4994 INT int_IBM ; int 2Ah ; will set carry if shared
4995
4996 ; 04/11/2022
4997 ; (INT 2Ah - AX = 0300h)
4998 ; Microsoft Networks - CHECK DIRECT I/O
4999 ; DS:SI -> ASCIIIZ disk device name (may be full path or
5000 ; only drive specifier--must include the colon)
5001 ; Return: CF clear if absolute disk access allowed
5002
5003 POP AX
5004 POP SI
5005 POP DS
5006 jnc short AbsSetup_retn
5007
5008 ; 02/01/2024
5009 errdriv:
5010 ;mov word [ss:EXTERR],32h
5011 MOV word [ss:EXTERR],error_not_supported
5012 ; 02/01/2024 - Retro DOS v5.0
5013 mov word [ss:AbsDskErr],207h ; PCDOS 7.1 IBMDOS.COM
5014
5015 ; 02/01/2024
5016 AbsSetup_retn:
5017 retn
5018
5019 ;-----
5020 ;
5021 ; Procedure Name : ABSDRD
5022 ;
5023 ; Interrupt 25 handler. Performs absolute disk read.
5024 ; Inputs: AL - 0-based drive number
5025 ; DS:BX point to destination buffer
5026 ; CX number of logical sectors to read
5027 ; DX starting logical sector number (0-based)
5028 ; Outputs: Original flags still on stack
5029 ; Carry set
5030 ; AH error from BIOS
5031 ; AL same as low byte of DI from INT 24
5032 ;-----
5033 ;
5034 ;procedure ABSDRD,FAR
5035 ABSDRD:
5036 ; 15/05/2019 - Retro DOS v4.0
5037 ; MSDOS 6.21 (DOSCODE:42E5h)
5038 ; 04/11/2022
5039 ; MSDOS 5.0 (DOSCODE:42D8h)
5040
5041 ; 02/01/2024 - Retro DOS v5.0
5042 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:43C4h
5043
5044 ;;;
5045 xor ah,ah ; ah=0 ; Interrupt 25h handler (ah=0)
5046 xor si,si ; si=0 ; clear read/write mode flags
5047 ; (used with INT 21h ax=7305h)
5048 FAT32_ABSDRD: ; ah=1 si=0 cf=0 (jump from '_$FAT32EXT')
5049 ; 25/06/2024
5050 ;cli ; *
5051 ;clc ; not necessary
5052 ; INT 21h
5053 ; AX = 7305h
5054 ; FAT32 - EXTENDED ABSOLUTE DISK READ/WRITE
5055 ; CX = FFFFh
5056 ; DL = drive number (01h=A:, etc.)
5057 ; SI = read/write mode flags
5058 ; DS:BX -> disk I/O packet
5059
5060 ; Extended Absolute Disk Read/write mode flags:
5061 ;
5062 ; Bit(s) Description
5063 ; 0 direction (0=read, 1=write)
5064 ; 12-1 reserved (0)
5065 ; 14-13 write type (should be 00 on reads).
5066 ; 00 unknown data.
5067 ; 01 FAT data.
5068 ; 10 directory data.
5069 ; 11 file data
5070 ; 15 reserved (0)
5071
5072 ; Format of disk read/write packet:
5073 ;
5074 ; Offset Size Description
5075 ; 00h DWORD sector number
5076 ; 04h WORD number of sectors to r/w
5077 ; 06h DWORD transfer address
5078
5079 ; ref: Ralf Brown's Interrupt List
5080
5081 ;;;
absdrd_1:

```

```

5082             ; MSDOS 6.0
5083             ; 25/06/2024 - Retro DOS v5.0
5084 00000531 FA  CLI ; *
5085
5086             ; set up ds to point to DOSDATA
5087
5088 00000532 50   push    ax                ; preserve AX value
5089 00000533 8CD8 mov     ax,ds                ; store DS value in AX
5090             ; getdseg <ds>
5091 00000535 2E8E1E[0700] mov     ds,[cs:DosDSeg]
5092 0000053A A3[630D] mov     [TEMPSEG],ax        ; store DS value in TEMPSEG
5093 0000053D 58   pop     ax                ; restore AX value
5094
5095             ; M072:
5096             ; We shall save es on the user stack here. We need to use ES in
5097             ; order to access the DOSDATA variables AbsRdwr_SS/SP at exit
5098             ; time in order to restore the user stack.
5099
5100 0000053E 06   push    es ; ****                ; M072
5101
5102             ; 25/06/2024
5103             ; 02/01/2024 - RetroDOS v5.0 (PCDOS 7.1 IBMDOS.COM)
5104             ;;;
5105 0000053F 7305 jnc     short absdrd_2        ; (not jumped from ABSDRWT) absolute disk read
5106
5107             ;(jumped from ABSDRWT)
5108 00000541 08E4 or      ah,ah
5109 00000543 F9   stc                ; absolute disk write
5110 00000544 EB02 jmp     short absdrd_3
5111 absdrd_2:
5112 00000546 08E4 or      ah,ah
5113 absdrd_3:
5114 00000548 7510 jnz     short absdrd_4        ; EXTENDED ABSOLUTE DISK READ/WRITE
5115             ;;;
5116
5117 0000054A 8C16[1B06] MOV     [AbsRdwr_SS],SS        ; M013
5118 0000054E 8926[1D06] MOV     [AbsRdwr_SP],SP        ; M013
5119
5120             ; set up ss to point to DOSDATA
5121             ;
5122             ; NOTE! Due to an obscure bug in the 80286, you cannot use the ROMDOS
5123             ; version of the getdseg macro with the SS register! An interrupt will
5124             ; sneak through.
5125
5126             ;ifndef ROMDOS
5127             ;getdseg <ss>                ; cli in entry of routine
5128
5129 00000552 2E8E16[0700] mov     ss,[cs:DosDSeg]
5130
5131             ;else
5132             ; mov     ds, cs:[BioDataSeg]
5133             ; assume  ds:bdata
5134             ;
5135             ; mov     ss, ds:[DosDataSg]
5136             ; assume  ss:DOSDATA
5137             ;
5138             ;endif ; ROMDOS
5139
5140 00000557 BC[2009] MOV     SP,DSKSTACK        ; 02/01/2024
5141             ;"@#IBM:12.01.2003.build_1.32#@ IBMDOS.COM(USA)"
5142             ; (PCDOS 7.1 IBMDOS.COM)
5143 0000055A 8E1E[630D] absdrd_4: mov     ds,[TEMPSEG]        ; restore DS value
5144
5145 0000055E 06   push    es ; *** (MSDOS 6.21)
5146 0000055F E8F6FE call   save_world        ; save all regs
5147
5148 00000562 06   PUSH    ES ; **
5149
5150             ; 02/01/2024 - RetroDOS v5.0
5151             ;;;
5152 00000563 7303 jnc     short absdrd_5        ; absolute disk read
5153 00000565 E9A400 jmp     absdrwt_3        ; (jumping back to) absolute disk write
5154 absdrd_5:
5155             ;;;
5156
5157 00000568 E86FFF CALL    AbsSetup
5158 0000056B 723D JC      short ILEAVE
5159
5160             ; Here is a gross temporary fix to get around a serious design flaw in
5161             ; the secondary cache. The secondary cache does not check for media
5162             ; changed (it should). Hence, you can change disks, do an absolute
5163             ; read, and get data from the previous disk. To get around this,
5164             ; we just won't use the secondary cache for absolute disk reads.
5165             ; -mw 8/5/88
5166
5167             ;EnterCrit critDisk
5168 0000056D E87213 call   ECritDisk
5169 00000570 36C606[7511]FF MOV     byte [ss:CurSC_DRIVE],-1 ; invalidate SC ;AN000;
5170             ;LeaveCrit critDisk
5171 00000576 E89613 call   LCritDisk
5172
5173             ;invoke DSKREAD
5174 00000579 E8C33A CALL    DSKREAD
5175 0000057C 7513 jnz     short ERR_LEAVE        ;Jump if read unsuccessful.
5176
5177 0000057E 89F9 mov     cx,di
5178 00000580 368C1E[0E06] mov     [ss:TEMP_VAR2],ds
5179 00000585 36891E[0C06] mov     [ss:TEMP_VAR],bx
5180
5181             ; CX = # of contiguous sectors read. (These constitute a block of
5182             ; sectors, also termed an "Extent".)
5183             ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
5184             ; [TEMP_VAR2]:[TEMP_VAR] = Transfer address (destination data address).
5185             ; ES:BP -> Drive Parameter Block (DPB).
5186
5187             ;
5188             ; The Buffer Queue must now be scanned: the contents of any dirty
5189             ; buffers must be "read" into the transfer memory block, so that the
5190             ; transfer memory reflects the most recent data.
5191
5192 0000058A E8443D ;invokeDskRdBufScan ;This trashes DS, but don't care.
5193 0000058D EB1B call   DskRdBufScan
5194             jmp     short ILEAVE
5195
5196 0000058F 7419 TLEAVE: JZ      short ILEAVE
5197
5198             ERR_LEAVE:
5199             ; 15/07/2018 - Retro DOS v3.0
5200             ;IF IBM
5201             ; Translate the error code to ancient 1.1 codes
5202 00000591 06   PUSH    ES ; *
5203 00000592 0E   PUSH    CS
5204 00000593 07   POP     ES
5205 00000594 30E4 XOR     AH,AH                ; Null error code

```

```

5206             ;mov     cx,6
5207 00000596 B90600      MOV CX,NUMERR          ; Number of possible error conditions
5208 00000599 BF[CE04]    MOV DI,ERRIN         ; Point to error conditions
5209 0000059C F2AE        REPNE     SCASB
5210 0000059E 7504        JNZ SHORT LEAVECODE      ; Not found
5211             ;mov     ah,[ES:DI+5]
5212 000005A0 268A6505     MOV AH,[ES:DI+NUMERR-1] ; Get translation
5213 LEAVECODE:
5214 000005A4 07          POP ES ; *
5215             ; 15/05/2019 - Retro DOS v4.0
5216 000005A5 36A3[BFD0]   mov     [ss:AbsDskErr],ax
5217             ;ENDIF
5218
5219 000005A9 F9          STC
5220 ILEAVE:
5221             ; 15/05/2019
5222 000005AA 07          POP ES ; **
5223 000005AB E893FE       call    restore_world
5224 000005AE 07          pop     es ; *** (MSDOS 6.21)
5225
5226             ; 02/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
5227             ;;;
5228 000005AF 9C          pushf
5229 000005B0 36803E[DB0A]00 cmp     byte [ss:absdrw_extd],0
5230             ; FAT32- EXTENDED ABSOLUTE DISK READ/WRITE flag
5231 000005B6 751F        jnz     short ILEAVE_EXTD      ; INT 21h AX=7305h
5232             ; INT 25h
5233 000005B8 9D          popf
5234             ;;;
5235
5236 000005B9 FA          CLI
5237 000005BA 36A1[BFD0]   mov     ax,[ss:AbsDskErr]      ; restore error
5238 000005BE 36FE0E[2103] DEC     BYTE [SS:INDOS]
5239             ;
5240             ; 02/01/2024 (PCDOS 7.1 IBMDOS.COM)
5241 000005C3 36FE0E[B812] dec     byte [ss:INDOS_FLAG] ; Windows DOSBOX's INDOS flag ?
5242             ;
5243             push     ss                ; M072 - Start
5244 000005C9 07          pop     es         ; es - dosdata
5245 000005CA 268E16[1B06] mov     ss,[es:AbsRdwr_SS]      ; M013
5246 000005CF 268B26[1D06] mov     sp,[es:AbsRdwr_SP]      ; M013
5247 000005D4 07          pop     es         ; Note es was saved on user
5248             ; stack at entry
5249             ; M072 - End
5250 000005D5 FB          STI
5251 000005D6 CB          RETF ; ! FAR return !
5252
5253             ; 02/01/2024 - Retro DOS v5.0
5254             ; (PCDOS 7.1 IBMDOS.COM)
5255             ;;;
5256 ILEAVE_EXTD:         ; return from INT 21h AX=7305h
5257 000005D7 9D          popf
5258 000005D8 36A1[BFD0]   mov     ax,[ss:AbsDskErr]      ; restore error
5259 000005DC 07          pop     es         ; ****
5260 000005DD FB          sti
5261 000005DE C3          retn
5262             ;;;
5263
5264 ;ABSDRD      ENDP
5265
5266 -----
5267 ;
5268 ; Procedure Name : ABSDWRT
5269 ;
5270 ; Interrupt 26 handler. Performs absolute disk write.
5271 ; Inputs:  AL - 0-based drive number
5272 ;          DS:BX point to source buffer
5273 ;          CX number of logical sectors to write
5274 ;          DX starting logical sector number (0-based)
5275 ; Outputs: Original flags still on stack
5276 ;          Carry set
5277 ;          AH error from BIOS
5278 ;          AL same as low byte of DI from INT 24
5279 ;
5280 -----
5281 ;procedure   ABSDWRT,FAR
5282 ABSDWRT:
5283             ; 15/05/2019 - Retro DOS v4.0
5284             ; MSDOS 6.21 (DOSCODE:436Ch)
5285             ; 04/11/2022
5286             ; MSDOS 5.0 (DOSCODE:435Fh)
5287
5288             ; 03/01/2024 - Retro DOS v5.0
5289             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:4477h
5290
5291             ;;;
5292 000005DF 30E4          xor     ah,ah      ; ah=0          ; Interrupt 26h handler (ah=0)
5293 000005E1 BE0100       mov     si,1        ; set direction flag/bit to write
5294             ; (used with INT 21h ax=7305h)
5295 FAT32_ABSDWRT:        ; ah=1 si=1 cf=0 (jump from '$_FAT32EXT')
5296
5297             ; INT 21h
5298             ; AX = 7305h
5299             ; FAT32 - EXTENDED ABSOLUTE DISK READ/WRITE
5300             ; CX = FFFFh
5301             ; DL = drive number (01h=A:, etc.)
5302             ; SI = read/write mode flags
5303             ; DS:BX -> disk I/O packet
5304
5305             ; Extended Absolute Disk Read/Write mode flags:
5306             ;
5307             ; Bit(s) Description
5308             ; 0      direction (0=read, 1=write)
5309             ; 12-1   reserved (0)
5310             ; 14-13  write type (should be 00 on reads).
5311             ;          00 unknown data.
5312             ;          01 FAT data.
5313             ;          10 directory data.
5314             ;          11 file data
5315             ; 15     reserved (0)
5316
5317             ; Format of disk read/write packet:
5318             ;
5319             ; Offset Size Description
5320             ; 00h    DWORD sector number
5321             ; 04h    WORD  number of sectors to r/w
5322             ; 06h    DWORD transfer address
5323
5324             ; ref: Ralf Brown's Interrupt List
5325 000005E4 3C02          cmp     al,2
5326 000005E6 7220          jb      short absdrwt_2      ; floppy disk
5327             ; hard disk
5328 000005E8 53          push     bx
5329 000005E9 1E          push     ds

```

```

5330 000005EA 2E8E1E[0700]      mov     ds,[cs:DosDSeg]
5331 000005EF 30FF              xor     bh,bh
5332 000005F1 88C3              mov     bl,al
5333                                ; NOTE: PC DOS 7.1 kernel does not set
5334                                ; DOS_FLAG bit 6 or drive_flags bit 7
5335                                ; (It appears that these bits are set
5336                                ; by windows or a system utility or
5337                                ; driver that knows the addresses of
5338                                ; these FLAGS in the DOSDATA segment.)
5339                                ; Erdogan Tan - 03/01/2024
5340
5341 000005F3 F687[0813]80      test    byte [drive_flags+bx],80h
5342                                ; test bit 7 (locked bit) ; 29/01/2024
5343 000005F8 7505              jnz     short absdwr1_1
5344                                ; locked (logical drive) -allowed to abs write-
5345 000005FA F606[8600]40      test    byte [DOS_FLAG],40h
5346                                ; NOTE: lock/unlock are MSDOS/PCDOS 7 extd functions
5347                                ; test bit 6 (large disk support -windows- bit?)
5348                                ; (windows OS running bit ?) ; 23/02/2024
5349                                ; NOTE: Retro DOS v5 kernel must set this bit.
5350
5351 000005FF 1F              absdwr1_1:
5352 00000600 5B              pop     ds
5353 00000601 7505              pop     bx
5354 00000603 F9              jnz     short absdwr1_2
5355 00000604 E817FF          stc
5356 00000607 CB              call    errdriv
5357                                ; error
5358                                retf
5359
5360                                ;absdwr1_2:
5361                                ;;;
5362                                ; 03/01/2024
5363                                %if 0
5364                                CLI
5365                                ;
5366                                ; set up ds to point to DOSDATA
5367
5368                                push    ax
5369                                mov     ax,ds
5370                                ;getdseg <ds>
5371                                mov     ds,[cs:DosDSeg]
5372                                mov     [TEMPSEG],ax
5373                                pop     ax
5374
5375                                ; M072:
5376                                ; we shall save es on the user stack here. We need to use ES in
5377                                ; order to access the DOSDATA variables AbsRdWr_SS/SP at exit
5378                                ; time in order to restore the user stack.
5379
5380                                push    es ; ****
5381                                ; M072
5382
5383                                MOV     [AbsRdWr_SS],SS
5384                                MOV     [AbsRdWr_SP],SP
5385                                ; M013
5386                                ; M013
5387
5388                                ; set up ss to point to DOSDATA
5389                                ;
5390                                ; NOTE! Due to an obscure bug in the 80286, you cannot use the
5391                                ; ROMDOS version of the getdseg macro with the SS register!
5392                                ; An interrupt will sneak through.
5393
5394                                ;ifndef ROMDOS
5395                                ;getdseg <ss>
5396                                ; cli in entry of routine
5397                                mov     ss,[cs:DosDSeg]
5398                                ;else
5399                                ; mov     ds, cs:[BioDataSeg]
5400                                ; assume ds:bdata
5401                                ;
5402                                ; mov     ss, ds:[DosDataSg]
5403                                ; assume ss:DOSDATA
5404                                ;
5405                                ;endif ; ROMDOS
5406
5407                                MOV     SP,DSKSTACK
5408                                ; we are now switched to DOS's disk stack
5409
5410                                mov     ds,[TEMPSEG]
5411                                ; restore user's ds
5412
5413                                push    es ; *** (MSDOS 6.21)
5414                                call    save_world
5415                                ; save all regs
5416
5417                                PUSH    ES ; **
5418                                %endif
5419                                ; 03/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
5420                                ;;;
5421                                absdwr1_2:
5422                                ; 25/06/2024
5423                                ; cli
5424                                stc
5425                                ; writable disk
5426                                ; ('jumped from ABSDWR1' sign for common r/w code)
5427                                jmp     absdwr1_1
5428                                ; jump to ABSDRD (common r/w) code
5429                                ;;;
5430
5431                                absdwr1_3:
5432                                CALL    AbsSetup
5433                                JC      short ILEAVE
5434
5435                                ; 03/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
5436                                call    chk_set_first_access
5437
5438                                ;EnterCrit critDisk
5439                                call    ECritDisk
5440                                MOV     byte [ss:CurSC_DRIVE],-1 ; invalidate SC ;AN000;
5441                                CALL    Fastxxx_Purge
5442                                ; purge fatopen ;AN000;
5443                                ;LeaveCrit critDisk
5444                                call    LCritDisk
5445
5446                                ;M039
5447                                ;
5448                                ; DS:BX = transfer address (source data address).
5449                                ; CX = # of contiguous sectors to write. (These constitute a block of
5450                                ; sectors, also termed an "Extent".)
5451                                ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
5452                                ; ES:BP -> Drive Parameter Block (DPB).
5453                                ; [CURSC_DRIVE] = -1 (invalid drive).
5454                                ;
5455                                ; Free any buffered sectors which are in Extent; they are being over-
5456                                ; written. Note that all the above registers are preserved for
5457                                ; DSKWRITE.
5458
5459                                push    ds
5460                                ;invoke DskwrtBufPurge
5461                                call    DskwrtBufPurge
5462                                ; This trashes DS.
5463                                pop     ds
5464
5465                                ;M039
5466                                ;invoke DSKWRITE
5467                                call    DSKWRITE

```

```

5454 0000062B E961FF      JMP     TLEAVE
5455
5456 ;ABSDWRT ENDP
5457
5458 ;-----
5459 ;
5460 ; Procedure Name : GETBP
5461 ;
5462 ; Inputs:
5463 ;   AL = Logical unit number (A = 0)
5464 ; Function:
5465 ;   Find Drive Parameter Block
5466 ; Outputs:
5467 ;   ES:BP points to DPB
5468 ;   [THISDPB] = ES:BP
5469 ;   Carry set if unit number bad or unit is a NET device.
5470 ;   Later case sets extended error error_I24_not_supported
5471 ; No other registers altered
5472 ;-----
5473 ;
5474 ;
5475 ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
5476 ; 04/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
5477 ; PCDOS 76.1 IBMDOS.COM - DOSCODE:44C7h
5478
5479 GETBP:
5480 ; 15/05/2019 - Retro DOS v4.0
5481 ; 11/07/2018 - Retro DOS v3.0
5482 PUSH    AX
5483 ADD     AL,1          ; No increment; need carry flag
5484 JC      SHORT SKIPGET
5485 CALL    GETTHISDRV
5486 ; MSDOS 6.0
5487 JNC     SHORT SKIPGET      ;PM. good drive          ;AN000;
5488
5489 ; 23/03/2024 - Retro DOS v5.0
5490 ;XOR    AH,AH          ;DCR. ax= error code ;AN000;
5491 ;CMP    AX,error_not_DOS_disk ;DCR. is unknown media ? ;AN000;
5492 ;JZ     SHORT SKIPGET  ;DCR. yes, let it go ;AN000;
5493 ;STC    ;DCR. ;AN000;
5494 mov     ah,0
5495 MOV     [EXTERR],AX      ;PM. invalid drive or Non DOS drive ;AN000;
5496 MOV     WORD [AbsDskErr],201h
5497 SKIPGET:
5498 POP     AX
5499 JC      SHORT GETBP_RET_N ; 15/12/2022
5500 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
5501 ;jnc    short getbp_t
5502 ;retn
5503
5504 getbp_t:
5505 LES     BP,[THISCDS]
5506 ; 15/12/2022
5507 test    byte [es:bp+curdir.flags+1],curdir_isnet>>8
5508 ; 07/12/2022
5509 ;TEST   WORD [ES:BP+43H],8000H
5510 ;TEST   WORD [ES:BP+curdir.flags],curdir_isnet ; clears carry
5511 JZ      SHORT GETBP_CDS
5512 GETBP_err: ; 04/01/2024
5513 MOV     WORD [EXTERR],error_not_supported ; 32h
5514 STC
5515 GETBP_RET_N:
5516 RETN
5517
5518 GETBP_CDS:
5519 ;LES    BP,[ES:BP+45H]
5520 LES     BP,[ES:BP+curdir.devptr]
5521 ; 04/01/2024 (PCDOS 7.1 IBMDOS.COM)
5522 ;;;
5523 push    ax
5524 ; 25/06/2024
5525 mov     ax,es
5526 or      ax,bp
5527 pop     ax
5528 jz      short GETBP_err ; zero address, error
5529 ;;;
5530 ;-----
5531 GOTDPB:
5532 ; Load THISDPB from ES:BP
5533 MOV     [THISDPB],BP
5534 MOV     [THISDPB+2],ES
5535 RETN
5536
5537 ;BREAK <SYS_RET_OK SYS_RET_ERR CAL_LK ETAB_LK set system call returns>
5538
5539 ;-----
5540 ;
5541 ; Procedure Name : SYS_RETURN
5542 ;
5543 ; These are the general system call exit mechanisms. All internal system
5544 ; calls will transfer (jump) to one of these at the end. Their sole purpose
5545 ; is to set the user's flags and set his AX register for return.
5546 ;-----
5547 ;
5548 ;procedure    SYS_RETURN,NEAR
5549 SYS_RETURN:
5550 ;entry        SYS_RET_OK
5551 SYS_RET_OK:
5552 call    Get_User_Stack
5553 ; turn off user's carry flag
5554 SYS_RET_OK_c1c: ; 25/06/2019
5555 ;;and        word [SI+16h],0FFFEh
5556 ;and        word [SI+user_env.user_F],~f_Carry
5557 ; 25/06/2019
5558 and      byte [SI+user_env.user_F],~f_Carry ; 0FEh
5559 ; 04/01/2024
5560 ;JMP      SHORT DO_RET
5561 DO_RET:
5562 ;MOV      [SI+user_env.user_AX],AX ; Really only sets AH
5563 MOV      [SI],AX
5564 RETN
5565
5566 ;entry        SYS_RET_ERR
5567 SYS_RET_ERR:
5568 XOR      AH,AH          ; hack to allow for smaller error rets
5569 call    ETAB_LK          ; Make sure code is OK, EXTERR gets set
5570 CALL     ErrorMap
5571
5572 ;entry        From_GetSet
5573 From_GetSet:
5574 call    Get_User_Stack
5575 ; signal carry to user
5576 ;;or        word [SI+16h],1
5577 ;OR        word [SI+user_env.user_F],f_Carry

```

```

5578             ; 25/06/2019
5579 00000683 804C1601 or byte [SI+user_env.user_F],f_Carry
5580 00000687 F9 STC ; also, signal internal error
5581             ; 04/01/2024
5582 00000688 EBEB jmp short DO_RET
5583 ;DO_RET:
5584             ;;MOV [SI+user_env.user_AX],AX ; Really only sets AH
5585             ;MOV [SI],AX
5586             ;RETN
5587
5588             ;entry FCB_RET_OK
5589 FCB_RET_OK:
5590             ;entry NO_OP ; obsolete system calls dispatch to here
5591 NO_OP:
5592 0000068A 30C0 XOR AL,AL
5593 0000068C C3 retn
5594
5595             ;entry FCB_RET_ERR
5596 FCB_RET_ERR:
5597 0000068D 30E4 XOR AH,AH
5598 0000068F 36A3[2403] mov [ss:EXTERR],AX
5599 00000693 E80300 CALL ErrorMap
5600 00000696 B0FF MOV AL,-1
5601 00000698 C3 retn
5602
5603             ;entry ErrorMap
5604 ErrorMap:
5605 00000699 56 PUSH SI
5606             ; ERR_TABLE_21 is now in DOSDATA
5607 0000069A BE[DB0D] MOV SI,ERR_TABLE_21
5608             ; SS override for FAILERR and EXTERR
5609 0000069D 36803E[4A03]00 CMP byte [ss:FAILERR],0 ; Check for SPECIAL case.
5610 000006A3 7407 JZ short EXTENDED_NORMAL ; All is OK.
5611             ; Oops, this is the REAL reason
5612             ;mov word [ss:EXTERR],53h
5613 000006A5 36C706[2403]5300 MOV word [ss:EXTERR],error_FAIL_I24
5614 EXTENDED_NORMAL:
5615 000006AC E80200 call CAL_LK ; Set CLASS,ACTION,LOCUS for EXTERR
5616 000006AF 5E POP SI
5617 000006B0 C3 retn
5618
5619             ;EndProc SYS_RETURN
5620
5621 ;-----
5622 ;
5623 ; Procedure Name : CAL_LK
5624 ;
5625 ; Inputs:
5626 ; SI is OFFSET in DOSDATA of CLASS,ACTION,LOCUS Table to use
5627 ; (DS NEED not be DOSDATA)
5628 ; [EXTERR] is set with error
5629 ; Function:
5630 ; Look up and set CLASS ACTION and LOCUS values for GetExtendedError
5631 ; Outputs:
5632 ; [EXTERR_CLASS] set
5633 ; [EXTERR_ACTION] set
5634 ; [EXTERR_LOCUS] set (EXCEPT on certain errors as determined by table)
5635 ; Destroys SI, FLAGS
5636 ;-----
5637 ;
5638 ;
5639 ;procedure CAL_LK,NEAR
5640 CAL_LK:
5641 000006B1 1E PUSH DS
5642 000006B2 50 PUSH AX
5643 000006B3 53 PUSH BX
5644
5645 ;M048 Context DS ; DS:SI -> Table
5646 ;
5647 ; Since this function can be called thru int 2f we shall not assume that SS
5648 ; is DOSDATA
5649
5650 ;getdseg <ds> ; M048: DS:SI -> Table
5651 ; 15/05/2019 - Retro DOS v4.0
5652 000006B4 2E8E1E[0700] mov ds,[cs:DosDSeg]
5653
5654 ; 18/07/2018
5655 ;push ss
5656 ;pop ds
5657
5658 000006B9 8B1E[2403] MOV BX,[EXTERR] ; Get error in BL
5659 TABLK1:
5660 000006BD AC LODSB
5661
5662 000006BE 3CFF CMP AL,0FFH
5663 000006C0 7409 JZ short GOT_VALS ; End of table
5664 000006C2 38D8 CMP AL,BL
5665 000006C4 7405 JZ short GOT_VALS ; Got entry
5666 000006C6 83C603 ADD SI,3 ; Next table entry
5667 ; 15/08/2018
5668 000006C9 EBF2 JMP short TABLK1
5669
5670 GOT_VALS:
5671 000006CB AD LODSW ; AL is CLASS, AH is ACTION
5672
5673 000006CC 80FCFF CMP AH,0FFH
5674 000006CF 7404 JZ short NO_SET_ACT
5675 000006D1 8826[2603] MOV [EXTERR_ACTION],AH ; Set ACTION
5676 NO_SET_ACT:
5677 000006D5 3CFF CMP AL,0FFH
5678 000006D7 7403 JZ short NO_SET_CLS
5679 000006D9 A2[2703] MOV [EXTERR_CLASS],AL ; Set CLASS
5680 NO_SET_CLS:
5681 000006DC AC LODSB ; Get LOCUS
5682
5683 000006DD 3CFF CMP AL,0FFH
5684 000006DF 7403 JZ short NO_SET_LOC
5685 000006E1 A2[2303] MOV [EXTERR_LOCUS],AL
5686 NO_SET_LOC:
5687 000006E4 5B POP BX
5688 000006E5 58 POP AX
5689 000006E6 1F POP DS
5690 000006E7 C3 retn
5691
5692 ;EndProc CAL_LK
5693
5694 ;-----
5695 ;
5696 ; Procedure Name : ETAB_LK
5697 ;
5698 ; Inputs:
5699 ; AX is error code
5700 ; [USER_IN_AX] has AH value of system call involved
5701 ; Function:

```



```

5702 ; Make sure error code is appropriate to this call.
5703 ; Outputs:
5704 ; AX MAY be mapped error code
5705 ; [EXTERR] = Input AX
5706 ; Destroys ONLY AX and FLAGS
5707 ;
5708 ;-----
5709
5710 ;procedure ETAB_LK,NEAR
5711
5712 ETAB_LK: ; 10/08/2018 - Retro DOS v3.0
5713 000006E8 1E    PUSH    DS
5714 000006E9 56    PUSH    SI
5715 000006EA 51    PUSH    CX
5716 000006EB 53    PUSH    BX
5717
5718 ;Context DS ; SS is DOSDATA
5719
5720 000006EC 16    push    ss
5721 000006ED 1F    pop     ds
5722
5723 000006EE A3[2403] MOV     [EXTERR],AX ; Set EXTERR with "real" error
5724
5725 ; I21_MAP_E_TAB is now in DOSCODE
5726 000006F1 BE[0B00] MOV     SI,I21_MAP_E_TAB
5727 000006F4 88C7 MOV     BH,AL ; Real code to BH
5728 000006F6 8A1E[3B03] MOV     BL,[USER_IN_AX+1] ; Sys call to BL
5729
5730 TABLK2: ; 15/05/2019 - Retro DOS v4.0
5731 000006FA 2E    CS
5732 000006FB AD    lodsw ; MSDOS 6.0 (MSDOS 6.21 - MSDOS.SYS, DOSCODE:447Dh)
5733
5734 ; 18/07/2018 - Retro DOS v3.0
5735 ;lodsw ; IBMDOS.COM (MSDOS 3.3) - Offset 16F7h
5736
5737 000006FC 3CFF CMP     AL,0FFH ; End of table?
5738 000006FE 740C JZ      short NOT_IN_TABLE ; Yes
5739 00000700 38D8 CMP     AL,BL ; Found call?
5740 00000702 740C JZ      short GOT_CALL ; Yes
5741 00000704 86E0 XCHG    AH,AL ; Count to AL
5742 00000706 30E4 XOR     AH,AH ; Make word for add
5743 00000708 01C6 ADD     SI,AX ; Next table entry
5744 0000070A EBEE JMP     short TABLK2
5745
5746 NOT_IN_TABLE:
5747 0000070C 88F8 MOV     AL,BH ; Restore original code
5748 0000070E EB0C JMP     SHORT NO_MAP
5749
5750 GOT_CALL:
5751 00000710 88E1 MOV     CL,AH
5752 00000712 30ED XOR     CH,CH ; Count of valid err codes to CX
5753
5754 CHECK_CODE: ; 15/05/2019 - Retro DOS v4.0
5755 00000714 2E    CS
5756 00000715 AC    lodsb ; MSDOS 6.0 (MSDOS 6.21 - MSDOS.SYS, DOSCODE:4497h)
5757
5758 ; 18/07/2018
5759 ;lodsb ; IBMDOS.COM (MSDOS 3.3) - Offset 1710h
5760
5761 00000716 38F8 CMP     AL,BH ; Code OK?
5762 00000718 7402 JZ      short NO_MAP ; Yes
5763 0000071A E2F8 LOOP   CHECK_CODE
5764
5765 NO_MAP:
5766 0000071C 30E4 XOR     AH,AH ; AX is now valid code
5767 0000071E 5B    POP     BX
5768 0000071F 59    POP     CX
5769 00000720 5E    POP     SI
5770 00000721 1F    POP     DS
5771 00000722 C3    retn
5772
5773 ;EndProc ETAB_LK
5774
5775 ; 18/07/2018 - Retro DOS v3.0
5776 ;-----
5777 ; BREAK <DOS 2F Handler and default NET 2F handler>
5778
5779 ;IF installed ; (*)
5780
5781 ;-----
5782 ;
5783 ; Procedure Name : SetBad
5784 ;
5785 ; SetBad sets up info for bad functions
5786 ;
5787 ;-----
5788
5789 SetBad:
5790 00000723 B80100 ;mov ax,1
5791 MOV     AX,error_invalid_function ; ALL NET REQUESTS get inv func
5792
5793 ; MSDOS 3.3
5794 ;mov byte [cs:EXTERR_LOCUS],1
5795 ;MOV byte [CS:EXTERR_LOCUS],errLOC_Unk
5796
5797 ; set up ds to point to DOSDATA
5798
5799 ; 15/05/2019 - Retro DOS v4.0
5800 00000726 1E    push    ds
5801
5802 ;getdseg <ds>
5803 00000727 2E8E1E[0700] mov     ds,[cs:DosDSeg]
5804
5805 0000072C C606[2303]01 MOV     byte [EXTERR_LOCUS],errLOC_Unk ; 1
5806
5807 00000731 1F    pop     ds ;hkn; restore ds
5808
5809 STC
5810 00000733 C3    retn
5811
5812 ;-----
5813 ;
5814 ; Procedure Name : BadCall
5815 ;
5816 ; BadCall is the initial routine for bad function calls
5817 ;
5818 ;-----
5819
5820 BadCall:
5821 00000734 E8ECFF call    SetBad
5822 00000737 CB    retf
5823
5824 ;-----
5825 ;

```

```

5826 ; OKCall always sets carry to off.
5827 ;
5828 ;-----
5829
5830 OKCall:
5831 00000738 F8 CLC
5832 00000739 CB retf
5833
5834 ;-----
5835
5836 ; Procedure Name : INT2F
5837 ;
5838 ; INT 2F handler works as follows:
5839 ; PUSH AX
5840 ; MOV AX,multiplex:function
5841 ; INT 2F
5842 ; POP ...
5843 ; The handler itself needs to make the AX available for the various routines.
5844 ;-----
5845
5846 ; 15/05/2019 - Retro DOS v4.0
5847
5848 ;KERNEL_SEGMENT equ 70h
5849 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
5850 DOSBIODATASEG equ 70h
5851
5852 ; retrodos4.s - offset in BIOSDATA
5853 bios_i2f equ 5
5854
5855 ;PUBLIC Int2F
5856 ;INT2F PROC FAR
5857
5858 ; 15/05/2019
5859 ; DOSCODE:44BDh (MSDOS 6.21, MSDOS.SYS)
5860
5861 ; 04/11/2022
5862 ; DOSCODE:44B0h (MSDOS 5.0, MSDOS.SYS)
5863
5864 ; 15/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
5865 ; 18/07/2018 - Retro DOS v3.0
5866 ; 05/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
5867 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:45DAh
5868
5869 INT2F:
5870 ; Offset 172Fh in IBMDOS.COM (MSDOS 3.3), 1987
5871 INT2FNT:
5872 ; ASSUME CS:DOSCODE,DS:NOTHING,ES:NOTHING,SS:NOTHING
5873 0000073A FB STI
5874 ; cmp ah,11h
5875 0000073B 80FC11 CMP AH,MultNET
5876 0000073E 750A JNZ short INT2FSHR
5877
5878 00000740 08C0 TestInstall: OR AL,AL
5879 00000742 7403 JZ short Leave2F
5880
5881 00000744 E8DCFF BadFunc: CALL SetBad
5882
5883 ; entry Leave2F
5884
5885 00000747 CA0200 Leave2F: RETF 2 ; long return + clear flags off stack
5886
5887
5888 INT2FSHR:
5889 ; cmp ah,10h
5890 0000074A 80FC10 CMP AH,MultSHARE ; is this a share request
5891 0000074D 74F1 JZ short TestInstall ; yes, check for installation
5892
5893 INT2FNLS:
5894 ; cmp ah,14h
5895 0000074F 80FC14 CMP AH,NLSFUNC ; is this a DOS 3.3 NLSFUNC request
5896 00000752 74EC JZ short TestInstall ; yes check for installation
5897
5898 INT2FDOS:
5899 ; ASSUME CS:DOSCODE,DS:NOTHING,ES:NOTHING,SS:NOTHING
5900
5901 ; 18/07/2018
5902 ; MSDOS 3.3
5903 ; cmp ah,12h
5904 ; CMP AH,MultDOS
5905 ; jz short DispatchDOS
5906 ; iret
5907
5908 ; 15/05/2019
5909 ; MSDOS 6.0
5910 ; cmp ah,12h ; 07/12/2022
5911 00000754 80FC12 CMP AH,MultDOS
5912 00000757 7503 JNZ short check_win ;check if win386 broadcast
5913 00000759 E93F02 jmp DispatchDOS
5914
5915 ; .... win386 ....
5916
5917 check_win:
5918 ; cmp ah,16h
5919 0000075C 80FC16 cmp ah,Multwin386 ; Is this a broadcast from win386?
5920 0000075F 7408 je short Win386_Msg
5921
5922 ; M044
5923 ; Check if the callout is from winoldap indicating swapping out or in
5924 ; of windows. If so, do special action of going and saving last para
5925 ; of the windows memory arena which winoldap does not save due to a
5926 ; bug
5927
5928 00000761 80FC46 cmp ah,WINOLDAP ; 46h ; from winoldap?
5929 ; jne short next_i2f ; no, chain on
5930 ; 15/12/2022
5931 ; jmp winold_swap ; yes, do desired action
5932 ; je short winold_swap
5933 ; jmp next_i2f
5934
5935 ; 15/12/2022
5936 ; next_i2f:
5937 ; ; jmp bios_i2f
5938 ; ; jmp far ptr 70h:5 ; MSDOS 6.21 (MSDOS.SYS, DOSCODE:44F1h)
5939 ; ; jmp KERNEL_SEGMENT:bios_i2f
5940 ; ; 04/11/2022
5941 ; jmp DOSBIODATASEG:bios_i2f
5942
5943 ; IRET ; This assume that we are at the head
5944 ; of the list
5945
5946 ;INT2F ENDP
5947
5948 ; 15/05/2019 - Retro DOS v4.0
5949
5950 ; We have received a message from win386. There are three possible
5951 ; messages we could get from win386:
5952 ;
5953 ; Init - for this, we set the Iswin386 flag and return a pointer

```



```

5950 ; to the win386 startup info structure.
5951 ; Exit - for this, we clear the Iswin386 flag.
5952 ; DOSMGR query - for this, we need to indicate that instance data
5953 ; has already been handled. this is indicated by setting
5954 ; CX to a non-zero value.
5955
5956 win386_Msg:
5957 00000769 1E push ds
5958
5959 ;getdseg <DS> ; ds is DOSDATA
5960 0000076A 2E8E1E[0700] mov ds,[cs:DosDSeg]
5961
5962 ; For WIN386 2.xx instance data
5963
5964 0000076F 3C03 cmp al,3 ; win386 2.xx instance data call?
5965 00000771 7503 jne short win386_Msg_exit
5966 00000773 E94801 jmp Oldwin386Init ; yes, return instance data
5967
5968 win386_Msg_exit:
5969 00000776 3C06 cmp al,win386_Exit ; 6 ; is it an exit call?
5970 00000778 7503 jne short win386_Msg_devcall
5971 0000077A E94A01 jmp win386_Leaving
5972
5973 win386_Msg_devcall:
5974 0000077D 3C07 cmp al,win386_Devcall ; 7 ; is it call from DOSMGR?
5975 0000077F 7503 jne short win386_Msg_init
5976 00000781 E97E01 jmp win386_Query
5977
5978 win386_Msg_init:
5979 00000784 3C05 cmp al,win386_Init ; 5 ; is it an init call?
5980 00000786 7403 je short win386_Starting
5981 00000788 E90001 jmp win_nexti2f ; no, return
5982
5983 win386_Starting:
5984 ; 05/01/2024 - Retro DOS v5.0
5985 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:4630h
5986 ;;;
5987 0000078B 50 push ax
5988 0000078C 51 push cx
5989 0000078D 57 push di
5990 0000078E 06 push es
5991 0000078F 1E push ds
5992 00000790 07 pop es
5993 00000791 BF[FB01] mov di,INBUF
5994 00000794 B90500 mov cx,5
5995 00000797 FC cld
5996
5997 win386_s_floop:
5998 00000798 B8434F mov ax,'CO'; 4F43h
5999 0000079B AB stosw
6000 0000079C B84E20 mov ax,'N '; 204Eh
6001 0000079F AB stosw
6002 000007A0 83C737 add di,55 ; (write 5 times 'CON ' and skip 55 bytes between them)
6003 ; what for ?
6004 loop win386_s_floop
6005 pop es
6006 pop di
6007 pop cx
6008 pop ax
6009 ;;;
6010 ; 17/12/2022
6011 test dl,1
6012 ;test dx,1 ; is this really win386?
6013 jz short win386_vchk ; YES! go and handle it
6014 jmp win_nexti2f ; NO! It's win 286 dos extender! M002
6015
6016 win386_vchk:
6017 ; M018 -- start of block changes
6018 ; The VxD needs to be loaded only for win 3.0. If version is greater
6019 ; than 030Ah, we skip the VxD presence check
6020
6021 ;M067 -- Begin changes
6022 ; If win 3.0 is run, the VxD ptr has been initialized. If win 3.1 is now
6023 ; run, it tries to unnecessarily load the VxD even though it is not needed.
6024 ; So, we null out the VxD ptr before the check.
6025
6026 ;mov word [win386_Info+6],0
6027 mov word [win386_Info+win386_SIS.Virt_Dev_File_Ptr],0
6028 ;mov word [win386_Info+8],0
6029 mov word [win386_Info+win386_SIS.Virt_Dev_File_Ptr+2],0
6030
6031 ;M067 -- End changes
6032
6033 ;ifdef JAPAN
6034 ; cmp di,0300h ; version >= 300 i.e 3.10 ;M037
6035 ;else
6036 ; cmp di,030Ah ; version >= 30a i.e 3.10 ;M037
6037 ; 05/01/2024 - PC DOS 7.1 - Retro DOS v5.0
6038 cmp di,0400h ; version >= 400 ; 05/01/2024
6039 ;endif
6040 ;jae novxd31 ; yes, VxD not needed ;M037
6041 jnb short win386_vxd
6042 jmp novxd31
6043
6044 ; 15/12/2022
6045 winold_swap:
6046 push ds
6047 push es
6048 push si
6049 push di
6050 push cx
6051
6052 ;getdseg <ds> ;ds = DOSDATA
6053 mov ds,[cs:DosDSeg]
6054
6055 cmp al,1 ;swap windows out call
6056 jne short swapin ;no, check if Swap in call
6057 call getwinlast
6058 push ds
6059 pop es
6060 mov ds,si ;ds = memory arena of windows
6061 xor si,si
6062 ;mov di,6 ; 05/01/2024
6063 mov di,winoldPatch1 ; 6
6064 mov cx,8
6065 cld
6066 ;push cx
6067 rep movsb ;save first 8 bytes
6068 ;pop cx
6069 ; 25/06/2024
6070 mov cl,8
6071 ;mov di,1176h ; 05/01/2024
6072 mov di,winoldPatch2 ; 1176h
6073 rep movsb ;save next 8 bytes
6074 jmp short winold_done
6075
6076 swapin:
6077 cmp al,2 ;swap windows in call?
6078 jne short winold_done ;no, something else, pass it on

```

```

6074 000007F3 E86901      call    getwinlast
6075 000007F6 8EC6        mov     es,si
6076 000007F8 31FF        xor     di,di
6077 000007FA BE[0600]     mov     si,winoldPatch1
6078 000007FD B90800     mov     cx,8
6079 00000800 FC          cld
6080                      ;push    cx
6081 00000801 F3A4        rep     movsb          ;restore first 8 bytes
6082                      ;pop     cx
6083                      ; 25/06/2024
6084 00000803 B108        mov     cl,8
6085 00000805 BE[7611]     mov     si,winoldPatch2
6086 00000808 F3A4        rep     movsb          ;restore next 8 bytes
6087 winold_done:
6088 0000080A 59          pop     cx
6089 0000080B 5F          pop     di
6090 0000080C 5E          pop     si
6091 0000080D 07          pop     es
6092 0000080E 1F          pop     ds
6093 0000080F EB7B        jmp     short next_i2f    ;chain on
6094                      ; 15/12/2022
6095                      ;jmp     next_i2f
6096
6097 win386_vxd:
6098 00000811 50          push    ax
6099 00000812 53          push    bx
6100 00000813 51          push    cx
6101 00000814 52          push    dx
6102 00000815 56          push    si
6103 00000816 57          push    di          ; save regs !!dont change order!!
6104
6105 00000817 8B1E[8C00]   mov     bx,[UMB_HEAD]    ; M062 - Start
6106 0000081B 83FBFF     cmp     bx,0FFFFh        ; Q: have umbs been initialized
6107 0000081E 741F        je      short Vxd31      ; N: continue
6108                      ; Y: save arena associated with
6109                      ;     umb_head
6110
6111 00000820 C606[DA0D]01  mov     byte [UmbSaveFlag],1 ; indicate that we're saving
6112                      ; umb_arena
6113 00000825 1E          push    ds
6114 00000826 06          push    es
6115
6116                      ;mov     ax,ds
6117                      ;mov     es,ax          ; es -> dosdata
6118                      ; 05/01/2024
6119 00000827 1E          push    ds
6120 00000828 07          pop     es
6121
6122 00000829 8EDB        mov     ds,bx
6123 0000082B 31F6        xor     si,si          ; ds:si -> umb_head
6124
6125                      ; 05/01/2024 PCDOS 7.1
6126                      ;;;
6127                      ;cld    ; not necessary (XOR already clears CF)
6128
6129 restore_umbhead:      ; !! PCDOS 7.1 bug !!
6130                      ; jump from 'win386_Leaving' here was/is wrong
6131                      ; (DI and SI would be reversed for 'win386_Leaving')
6132                      ; Erdogan Tan - 05/01/2024
6133                      ;;;
6134
6135 0000082D FC          cld
6136
6137 0000082E BF[2F11]   mov     di,UmbSave1
6138 00000831 B90B00     mov     cx,11
6139 00000834 F3A4        rep     movsb
6140
6141 00000836 BF[D50D]   mov     di,UmbSave2
6142                      ;mov     cx,5
6143                      ; 18/12/2022
6144 00000839 B105        mov     cl,5
6145 0000083B F3A4        rep     movsb
6146
6147                      ; 05/01/2024 PCDOS 7.1
6148                      ;;;
6149                      ;jnb     short restore_umbhead_c ; (not jumped from 'win386_Leaving')
6150                      ;jmp     restore_umbhead_ok ; (jumped from 'win386_Leaving' just after 'stc')
6151 ;restore_umbhead_c:
6152                      ;;;
6153
6154 0000083D 07          pop     es
6155 0000083E 1F          pop     ds          ; M062 - End
6156
6157 vxd31:
6158                      ;test    byte [DOS_FLAG],2
6159 0000083F F606[8600]02  test    byte [DOS_FLAG],SUPPRESS_WINA20 ; M066
6160 00000844 7408        jz      short Dont_Supress ; M066
6161 00000846 5F          pop     di          ; M066
6162 00000847 5E          pop     si          ; M066
6163 00000848 5A          pop     dx          ; M066
6164 00000849 59          pop     cx          ; M066
6165 0000084A 5B          pop     bx          ; M066
6166 0000084B 58          pop     ax          ; M066
6167 0000084C EB55        jmp     short noVxd31    ; M066
6168
6169                      ; We check here if the vxd is available in the root of the boot drive.
6170                      ; We do an extended open to suppress any error messages
6171
6172 Dont_Supress:
6173 0000084E A0[6900]   mov     al,[BOOTDRIVE]
6174 00000851 0440        add     al,'A' - 1      ; get drive letter
6175 00000853 A2[F812]   mov     [VxDpath],al    ; path is root of bootdrive
6176                      ;mov     ah,ExtOpen ;6ch ; extended open
6177                      ;mov     al,0 ; no extended attributes
6178                      ; 18/12/2022
6179 00000856 B8006C   mov     ax,ExtOpen<<8 ; 6C00h
6180 00000859 BB8020   mov     bx,2080h        ; read access, compatibility mode
6181                      ; no inherit, suppress crit err
6182 0000085C B90700   mov     cx,7            ; hidden,system,read-only attr
6183                      ;05/01/2024
6184                      ;inc     dx ; dx bit 0 = 1 ; fail if file does not exist
6185 0000085F BA0100   mov     dx,1            ; fail if file does not exist
6186
6187 00000862 BE[F812]   mov     si,VxDpath ; "c:\\wina20.386"
6188                      ; path of vxd file
6189 00000865 BFFFFF   mov     di,0FFFFh        ; no extended attributes
6190
6191 00000868 CD21        int     21h            ; do extended open
6192
6193 0000086A 5F          pop     di
6194 0000086B 5E          pop     si
6195 0000086C 5A          pop     dx
6196 0000086D 59          pop     cx
6197

```

```

6198 0000086E 7321      jnc     short VxDthere      ; we found the VxD, go ahead
6199
6200                  ; We could not find the VxD. Cannot let windows load. Return cx != 0
6201                  ; to indicate error to windows after displaying message to user that
6202                  ; VxD needs to be present to run windows in enhanced mode.
6203
6204 00000870 52          push    dx
6205 00000871 1E          push    ds
6206 00000872 56          push    si
6207 00000873 BE[300A]      mov     si,NovxDErrMsg
6208 00000876 0E          push    cs
6209 00000877 1F          pop     ds
6210 00000878 B96300      mov     cx,VxDMesLen ; 99      ;
6211 0000087B B402          mov     ah,2              ; write char to console
6212 0000087D FC          cld
6213
VxDlp:
6214 0000087E AC          lodsb
6215 0000087F 86D0      xchg    dl,al              ; get char in dl
6216 00000881 CD21      int     21h
6217 00000883 E2F9      loop   VxDlp
6218
6219 00000885 5E          pop     si
6220 00000886 1F          pop     ds
6221 00000887 5A          pop     dx
6222 00000888 5B          pop     bx
6223 00000889 58          pop     ax              ;all registers restored
6224 0000088A 41          inc     cx              ;cx != 0 to indicate error
6225                  ; 15/12/2022
6226                  ; jmp     win_nexti2f      ;chain on
6227                  ; jmp     short win_nexti2f
6228
        ; 15/12/2022
6229
win_nexti2f:
6230
6231 0000088B 1F          pop     ds
6232                  ; jmp     short next_i2f      ; go to BIOS i2f handler
6233                  ; 15/12/2022
6234
next_i2f:
6235                  ; jmp     bios_i2f
6236                  ; jmp     far ptr 70h:5 ; MSDOS 6.21 (MSDOS.SYS, DOSCODE:44F1h)
6237                  ; jmp     KERNEL_SEGMENT:bios_i2f
6238                  ; 04/11/2022
6239 0000088C EA05007000      jmp     DOSBIODATASEG:bios_i2f
6240
6241
VxDthere:
6242 00000891 89C3      mov     bx,ax
6243 00000893 B43E      mov     ah,CLOSE ; 3Eh
6244 00000895 CD21      int     21h              ;close the file
6245
        ; Update the VxD ptr in the instance data structure with path to VxD
6246
        ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
6247
6248
6249      ; mov     bx,win386_Info
6250      ; mov     word [bx+win386_SIS.Virt_Dev_File_Ptr],VxDpath
6251      ; mov     word [bx+win386_SIS.Virt_Dev_File_Ptr+2],ds
6252      ; 15/12/2022
6253      mov     word [win386_Info+win386_SIS.Virt_Dev_File_Ptr],VxDpath
6254 0000089D 8C1E[E90E]      mov     word [win386_Info+win386_SIS.Virt_Dev_File_Ptr+2],ds
6255
6256 000008A1 5B          pop     bx
6257 000008A2 58          pop     ax
6258
novxD31:
6259                  ; M018; End of block changes
6260
6261 000008A3 800E[5B0F]01      or      byte [IsWin386],1      ; Indicate WIN386 present
6262 000008A8 800E[650D]01      or      byte [redir_patch],1    ; Enable critical sections; M002
6263
6264                  ; M002;
6265                  ; Save the previous es:bx (instance data ptr) into our instance table
6266
6267 000008AD 52          push    dx              ; M002
6268 000008AE 89DA      mov     dx,bx              ; M002
6269                  ; point ES:BX to win386_Info ; M002
6270
6271 000008B0 BB[E10E]      mov     bx,win386_Info
6272 000008B3 895702      mov     [bx+2],dx          ; M002
6273 000008B6 8C4704      mov     [bx+4],es          ; M002
6274 000008B9 5A          pop     dx              ; M002
6275 000008BA 1E          push    ds              ; M002
6276 000008BB 07          pop     es              ; M002
6277                  ; jmp     win_nexti2f      ; M002
6278                  ; 15/12/2022
6279      jmp     short win_nexti2f
6280
        ; 15/12/2022
6281
        ; Code to return win386 2.xx instance table
6282
oldwin386Init:
6283 000008BE 58          pop     ax              ; discard ds pushed on stack
6284 000008BF BE[FC10]      mov     si,OldInstanceJunk
6285                  ; ds:si = instance table
6286                  ; indicate instance data present
6287      mov     ax,5248h ; 'HR'
6288      ; jmp     next_i2f
6289      ; 15/12/2022
6290      jmp     short next_i2f
6291
win386_Leaving:
6292      ; 15/12/2022
6293 000008C7 F6C201      test    dl,1
6294                  ; test    dx,1              ; is this really win386?
6295                  ; jz      short win386_Leaving_c
6296      jmp     win_nexti2f      ; NO! It's win 286 dos extender! M002
6297      ; 15/12/2022
6298 000008CA 75BF      jnz     short win_nexti2f
6299
win386_Leaving_c:
6300
6301                  ; M062 - Start
6302 000008CC 803E[DA0D]01      cmp     byte [UmbSaveFlag],1    ; Q: was umb_arena saved at win start
6303                  ; up.
6304      jne     short noumb
6305 000008D3 C606[DA0D]00      mov     byte [UmbSaveFlag],0    ; N: not saved
6306                  ; Y: clear UmbSaveFlag and restore
6307                  ; previously saved umb_head
6308
        ; 05/01/2024 - PC DOS 7.1 (has BUG here!)
6309      ; PC DOS 7.1 IBMDOS.COM - DOSCODE:472Bh
6310
        ;
6311      ; push    ax ; (not necessary)
6312      ; push    es
6313      ; push    cx
6314      ; push    si
6315      ; push    di
6316      ; mov     es,[UMB_HEAD]
6317      ; xor     di,di
6318      ; stc
6319      ; jmp     restore_umbhead      ; !! PC DOS 7.1 bug !! IBMDOS.COM - DOSCODE:4737h
6320      ;
6321      ; (jumped code does not restore umbhead,
        ; MSDOS 6.22 "win386_Leaving" code was/is correct,

```

```

6322 ; ; modified code -in PC DOS 7.1 IBMDOS.COM- is wrong)
6323 ; ; ;
6324 ; ; ;
6325 ; ; ;
6326 ; ; ;
6327 ; 05/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1)
6328 ;push ax ; (not necessary)
6329 000008D8 06 push es
6330 000008D9 51 push cx
6331 000008DA 56 push si
6332 000008DB 57 push di
6333 ;
6334 ;mov ax,[UMB_HEAD]
6335 ;mov es,ax
6336 ; 05/01/2024
6337 000008DC 8E06[8C00] mov es,[UMB_HEAD]
6338 000008E0 31FF xor di,di ; es:di -> umb_head
6339 ;
6340 000008E2 FC cld
6341 ;
6342 000008E3 BE[2F11] mov si,UmbSave1
6343 000008E6 B90B00 mov cx,11
6344 000008E9 F3A4 rep movsb
6345 000008EB BE[D50D] mov si,UmbSave2
6346 ;mov cx,5
6347 ; 18/12/2022
6348 000008EE B105 mov cl,5
6349 000008F0 F3A4 rep movsb
6350 ;
6351 restore_umbhead_ok: ; 05/01/2024 - PC DOS 7.1 IBMDOS.COM - DOSCODE:473Ah
6352 000008F2 5F pop di
6353 000008F3 5E pop si
6354 000008F4 59 pop cx
6355 000008F5 07 pop es
6356 ; 05/01/2024
6357 ;pop ax
6358 ;
6359 ; ; M062 - End
6360 ; ; win386 is gone
6361 000008F6 8026[5B0F]FE and byte [IsWin386],0
6362 000008FB 8026[650D]00 and byte [IsWin386],0FEh ; ~1
6363 00000900 EB89 and byte [redir_patch],0 ; Disable critical sections ; M002
6364 jmp short win_nexti2f
6365 ;
6366 ; ; 15/12/2022
6367 ; ; Code to return Win386 2.xx instance table
6368 ;Oldwin386Init:
6369 ; pop ax ; discard ds pushed on stack
6370 ; mov si,OldInstanceJunk
6371 ; ; ds:si = instance table
6372 ; mov ax,5248h ; 'RH' ; indicate instance data present
6373 ; ; jmp next_i2f
6374 ; ; 15/12/2022
6375 ; jmp short _next_i2f
6376 ;
6377 win386_Query:
6378 cmp bx,win386_DOSMGR ; 15h; is this from DOSMGR?
6379 jne short win_nexti2f ; no, ignore it & chain to next
6380 or cx,cx ; is it an instance query?
6381 jnz short dosmgr_func ; no, some DOSMGR query
6382 inc cx ; indicate that data is instanced
6383 ;
6384 ; M001; we were previously returning a null ptr in es:bx. This will not work.
6385 ; M001; WIN386 needs a ptr to a table in es:bx with the following offsets:
6386 ; M001;
6387 ; M001; OFFSETS STRUC
6388 ; M001; Major_version db ?
6389 ; M001; Minor_version db ?
6390 ; M001; SavedS dw ?
6391 ; M001; SaveBX dw ?
6392 ; M001; Indos dw ?
6393 ; M001; User_id dw ?
6394 ; M001; CritPatch dw ?
6395 ; M001; OFFSETS ENDS
6396 ; M001;
6397 ; M001; User_Id is the only variable really important for proper functioning
6398 ; M001; of win386. The other variables are used at init time to patch stuff
6399 ; M001; out. In DOS 5.0, we do the patching ourselves. But we still need to
6400 ; M001; pass this table because win386 depends on this table to get the
6401 ; M001; User_Id offset.
6402 ; M001;
6403 mov bx,win386_DOSVars ; M001
6404 push ds ; M001
6405 pop es ; es:bx points at offset table ; M001
6406 jmp short PopIret ; M001
6407 ;
6408 ; ; 15/12/2022
6409 ; ; Code to return win386 2.xx instance table
6410 ;Oldwin386Init:
6411 ; pop ax ; discard ds pushed on stack
6412 ; mov si,OldInstanceJunk
6413 ; ; ds:si = instance table
6414 ; mov ax,5248h ; 'RH' ; indicate instance data present
6415 ; ; jmp next_i2f
6416 ; ; 15/12/2022
6417 ; jmp short _next_i2f
6418 ;
6419 dosmgr_func:
6420 dec cx
6421 jz short win386_patch ; call to patch DOS
6422 dec cx
6423 jz short PopIret ; remove DOS patches, ignore
6424 dec cx
6425 jz short win386_size ; get size of DOS data structures
6426 dec cx
6427 jz short win386_inst ; instance more data
6428 ;dec cx
6429 ;jnz short PopIret ; no functions above this
6430 ; 05/01/2024 (PC DOS 7.1 IBMDOS.COM DOSCODE:4771h)
6431 loop PopIret
6432 ;
6433 ; Get DOS device driver size -- es:di points at device driver header
6434 ; In DOS 4.x, the para before the device header contains an arena
6435 ; header for the driver.
6436 mov ax,es ; ax = device header segment
6437 ;
6438 ; We check to see if we have a memory arena for this device driver.
6439 ; The way to do this would be to look at the previous para to see if
6440 ; it has a 'D' marking it as an arena and also see if the owner-field
6441 ; in the arena is the same as the device header segment. These two
6442 ; checks together should take care of all cases
6443 ;
6444 dec ax ; get arena header
6445 push es

```

```

6446 00000925 8EC0      mov     es,ax          ; arena header for device driver
6447
6448 00000927 26803D44    cmp     byte [es:di],'D'    ; is it a device arena?
6449 0000092B 7517      jnz     short cantsize     ; no, cant size this driver
6450 0000092D 40        inc     ax                ; get back device header segment
6451 0000092E 26394501    cmp     [es:di+1],ax        ; owner field pointing at driver?
6452 00000932 7510      jnz     short cantsize     ; no, not a proper arena
6453
6454 00000934 268B4503    mov     ax,[es:di+3]        ; get arena size in paras
6455 00000938 07        pop     es
6456
6457      ; We have to multiply by 16 to get the number of bytes in (bx:cx)
6458      ; Speed is not critical and so we choose the shortest method
6459      ; -- use "mul"
6460
6461 00000939 BB1000      mov     bx,16
6462 0000093C F7E3      mul     bx
6463 0000093E 89C1      mov     cx,ax
6464 00000940 89D3      mov     bx,dx
6465 00000942 EB09      jmp     short win386_done   ; return with device driver size
6466
cantsize:
6467      pop     es
6468 00000945 31C0      xor     ax,ax
6469
win386_inst:
6470      xor     dx,dx          ; 05/01/2024
6471 00000949 EB08      jmp     short PopIret       ; ask DOSMGR to use its methods
6472      ; return
6473
win386_patch:
6474      ; dx contains bits marking the patches to be applied. We return
6475      ; the field with all bits set to indicate that all patches have been
6476      ; done
6477
6478 0000094B 89D3      mov     bx,dx              ; move patch bitfield to bx
6479      jmp     short win386_done ; done, return
6480      ; 15/12/2022
6481      ; 15/12/2022
6482
win386_done:
6483      mov     ax,WIN_OP_DONE    ; 0B97Ch
6484 00000950 BAAB2      mov     dx,DOSMGR_OP_DONE    ; 0A2ABh
6485
PopIret:
6486      pop     ds
6487 00000954 CF        iret
6488
6489
win386_size:
6490      ; Return the size of DOS data structures -- currently only CDS size
6491
6492      ; 17/12/2022
6493 00000955 F6C201    test    dl,1
6494      ;test    dx,1          ; check for CDS size bit
6495 00000958 74F9      jz      short PopIret       ; no, unknown structure -- return
6496
6497      mov     cx,curdirLen      ; 88
6498 0000095D EBEE      jmp     short win386_done   ; return with the size
6499
6500      ; 05/01/2024
6501      %if 0
6502
win386_inst:
6503      ; WIN386 check to see if DOS has identified the CDS,SFT and device
6504      ; chain as instance data. Currently, we let the WIN386 DOSMGR handle
6505      ; this by returning a status of not previously instanced. The basic
6506      ; structure of these things have not changed and so the current
6507      ; DOSMGR code should be able to work it out
6508
6509      xor     dx,dx            ; make sure dx has a not done value
6510      jmp     short PopIret     ; skip done indication
6511      %endif
6512
6513      ; 15/12/2022
6514
win386_done:
6515      mov     ax,WIN_OP_DONE    ; 0B97Ch
6516      mov     dx,DOSMGR_OP_DONE ; 0A2ABh
6517
PopIret:
6518      pop     ds
6519      iret                    ; return back up the chain
6520
6521      ; 15/12/2022
6522
win_nexti2f:
6523      pop     ds
6524      jmp     next_i2f         ; go to BIOS i2f handler
6525
6526
;End WIN386 support
6527
6528      ; 15/05/2019
6529
6530      ;M044; Start of changes
6531      ; winoldap has a bug in that its calculations for the windows memory image
6532      ; to save is off by 1 para. This para can happen to be a windows arena if the
6533      ; DOS top of memory happens to be at an odd boundary (as is the case when
6534      ; UMBS are present). This is because Windows builds its arenas only at even
6535      ; para boundaries. This arena now gets trashed when windows is swapped back
6536      ; in leading to a crash. winoldap issues callouts when it swaps windows out
6537      ; and back in. We sit on these callouts. On the windows swapout, we save the
6538      ; last para of the windows memory block and then restore this para on the
6539      ; windows swapin callout.
6540
6541
getwinlast:
6542      ; 07/12/2022
6543 0000095F 8B36[3003]    mov     si,[CurrentPDB]
6544 00000963 4E        dec     si
6545 00000964 8EC6      mov     es,si
6546 00000966 2603360300    add     si,[es:3]
6547 0000096B C3        retn
6548
6549      ; 15/12/2022
6550      %if 0
6551
winold_swap:
6552      push    ds
6553      push    es
6554      push    si
6555      push    di
6556      push    cx
6557
6558      ;getdseg <ds>          ;ds = DOSDATA
6559      mov     ds,[cs:DosDseg]
6560
6561      cmp     al,1            ;swap windows out call
6562      jne     short swapin    ;no, check if Swap in call
6563      call    getwinlast
6564      push    ds
6565      pop     es
6566      mov     ds,si            ;ds = memory arena of Windows
6567      xor     si,si
6568      mov     di,winoldPatch1
6569      mov     cx,8

```

```

6570      cld
6571      push    cx
6572      rep     movsb          ;save first 8 bytes
6573      pop     cx
6574      mov     di,winoldPatch2
6575      rep     movsb          ;save next 8 bytes
6576      jmp     short winold_done
6577  swapin:
6578      cmp     al,2           ;swap windows in call?
6579      jne     short winold_done ;no, something else, pass it on
6580      call    getwinlast
6581      mov     es,si
6582      xor     di,di
6583      mov     si,winoldPatch1
6584      mov     cx,8
6585      cld
6586      push    cx
6587      rep     movsb          ;restore first 8 bytes
6588      pop     cx
6589      mov     si,winoldPatch2
6590      rep     movsb          ;restore next 8 bytes
6591  winold_done:
6592      pop     cx
6593      pop     di
6594      pop     si
6595      pop     es
6596      pop     ds
6597      jmp     next_i2f       ;chain on
6598
6599  %endif
6600
6601  ;M044; End of changes
6602
6603  ; -----
6604  ; 06/01/2024 - Retro DOS v5.0 (Modified PCDS 7.1 IBMDOS.COM/MSDOS.SYS)
6605  ; PCDS 7.1 IBMDOS.COM - DOSCODE:4811h
6606
6607      ; INT 2Fh AX=1231h
6608      ; Windows95 - SET/CLEAR "REPORT WINDOWS TO DOS PROGRAMS" FLAG
6609      ; (Ref: Ralf Brown's Interrupt List)
6610  int_2Fh_1231h:
6611      0000096C 1E      push    ds
6612      0000096D 2E8E1E[0700]  mov     ds,[cs:DosDSeg]
6613      00000972 31C0      xor     ax,ax ; 0
6614      00000974 08D2      or      dl,dl
6615      00000976 7507      jnz     short not_1231_d1_0
6616      00000978 C606[5C0F]01  mov     byte [IsWin386+1],1 ; set byte after "ISWIN386" to 01h
6617      0000097D EB1A      jmp     short int_2f_1231h_retn
6618
6619      ;nop
6620
6621  not_1231_d1_0:
6622      0000097F 80FA01      cmp     dl,1
6623      00000982 7507      jne     short not_1231_d1_1 ; clear "ISWIN386" bit 1
6624      00000984 800E[5B0F]02  or      byte [IsWin386],2 ; set "ISWIN386" bit 1
6625      00000989 EB0E      jmp     short int_2f_1231h_retn
6626
6627      ;nop
6628
6629  not_1231_d1_1:
6630      0000098B 80FA02      cmp     dl,2
6631      0000098E 7507      jne     short not_1231_d1_2
6632      00000990 8026[5B0F]FD  and     byte [IsWin386],0FDh ; clear bit 1
6633      00000995 EB02      jmp     short int_2f_1231h_retn
6634  not_1231_d1_2:
6635      00000997 40          inc     ax ; return error, ax = 1
6636      00000998 F9          stc
6637  int_2f_1231h_retn:
6638      00000999 1F          pop     ds
6639      0000099A C3          retn
6640
6641  ; -----
6642
6643  ; 15/05/2019
6644
6645  ; 06/01/2024 - Retro DOS v5.0
6646  ; PCDS 7.1 IBMDOS.COM - DOSCODE:4842h
6647
6648  DispatchDOS:
6649      0000099B 2EFF36[D401]  PUSH    word [CS:F00] ; push return address
6650      000009A0 2EFF36[D601]  PUSH    word [CS:DTab] ; push table address
6651      000009A5 50          PUSH    AX ; push index
6652      000009A6 55          PUSH    BP
6653      000009A7 89E5      MOV     BP,SP
6654      ; stack looks like:
6655      ; 0 BP
6656      ; 2 DISPATCH
6657      ; 4 TABLE
6658      ; 6 RETURN
6659      ; 8 LONG-RETURN
6660      ; C FLAGS
6661      ; E AX
6662
6663      MOV     AX,[BP+0Eh] ; get AX value
6664      POP     BP
6665      call    TableDispatch
6666      JMP     BadFunc ; return indicates invalid function
6667
6668  INT2F_etcetera:
6669      ;entry DosGetGroup
6670  DosGetGroup:
6671      ; MSDOS 3.3
6672      ;push cs
6673      ;pop ds
6674      ;retn
6675
6676      ; MSDOS 6.0
6677  ;SR; Cannot use CS now
6678      ;
6679      ; PUSH CS
6680      ; POP DS
6681
6682      ; 04/11/2022
6683      ; (MSDOS 5.0 MSDOS.SYS - DOSCODE:46FBh)
6684
6685      ;getdseg <ds>
6686      000009B3 2E8E1E[0700]  mov     ds,[cs:DosDSeg]
6687      000009B8 C3          retn
6688
6689      ;entry DOSInstall
6690  DOSInstall:
6691      000009B9 B0FF      MOV     AL,0FFh
6692      000009BB C3          retn
6693

```



```

6694 ;ENDIF ; (*)
6695
6696
6697 ; 15/05/2019 - Retro DOS v4.0
6698 ; 06/01/2024 - Retro DOS v5.0
6699
6700 ;-----
6701 ;
6702 ; Procedure Name : RW32_CONVERT
6703 ;
6704 ; Input: same as ABSDRD and ABSDWRT
6705 ; ES:BP -> DPB
6706 ; Functions: convert 32bit absolute RW input parms to 16bit input parms
6707 ; Output: carry set when CX=-1 and drive is less then 32mb
6708 ; carry clear, parms ok
6709 ;
6710 ;-----
6711
6712 ; 06/01/2024
6713 RW32_CONVERT:
6714 000009BC 83F9FF CMP CX,-1 ;>32mb new format ? ;AN000;
6715 ;inc CX ; * ; 01 -> 0
6716 000009BF 7423 JZ short new32format ;>32mb yes ;AN000;
6717 ;dec CX
6718
6719 ;cmp word [es:bp+0Fh],0
6720 000009C1 26837E0F00 cmp word [es:bp+DPB.FAT_SIZE],0 ; PCDOS 7.1
6721 000009C6 741A JZ short rw32_conv_err ; FAT32 fs
6722
6723 000009C8 50 PUSH AX ;>32mb save ax ;AN000;
6724 000009C9 52 PUSH DX ;>32mb save dx ;AN000;
6725 ;mov ax,[es:bp+0Dh]
6726 000009CA 268B460D MOV AX,[ES:BP+DPB.MAX_CLUSTER] ;>32mb get max cluster # ;AN000;
6727 ;mov dl,[es:bp+4]
6728 000009CE 268A5604 MOV DL,[ES:BP+DPB.CLUSTER_MASK] ;>32mb ;AN000;
6729 000009D2 80FAFE CMP DL,0FEh ; 254 ;>32mb removable ? ;AN000;
6730 000009D5 7407 JZ short letold ;>32mb yes ;AN000;
6731 ;INC DL ;>32mb ;AN000;
6732 ; 17/12/2022
6733 000009D7 42 inc dx
6734 000009D8 30F6 XOR DH,DH ;>32mb dx = sector/cluster ;AN000;
6735 000009DA F7E2 MUL DX ;>32mb dx:ax= max sector # ;AN000;
6736 000009DC 09D2 OR DX,DX ; (clears CF) ;>32mb > 32mb ? ;AN000;
6737 letold:
6738 000009DE 5A POP DX ;>32mb restore dx ;AN000;
6739 000009DF 58 POP AX ;>32mb restore ax ;AN000;
6740 000009E0 7418 JZ short old_style ; cf=0 ;>32mb no ;AN000;
6741
6742 ; 06/01/2024
6743 ;push ds
6744 ;;getdseg <ds>
6745 ;mov ds,[cs:DosDSeg]
6746 ;mov word [AbsDskErr],207h ;>32mb bad address mark
6747 ;pop ds
6748 rw32_conv_err:
6749 000009E2 F9 STC ;>32mb ;AN000;
6750 000009E3 C3 retn ;>32mb ;AN000;
6751
6752 new32format:
6753 ;mov dx,[bx+2]
6754 000009E4 8B5702 MOV DX,[BX+ABS_32RW.SECTOR_RBA+2] ;>32mb ;AN000;
6755
6756 000009E7 1E push ds ; set up ds to DOSDATA
6757 ;getdseg <ds>
6758 000009E8 2E8E1E[0700] mov ds,[cs:DosDSeg]
6759 000009ED 8916[0706] MOV [HIGH_SECTOR],DX ;>32mb ;AN000;
6760 000009F1 1F pop ds
6761
6762 000009F2 8B17 mov dx,[bx]
6763 ;MOV DX,[BX+ABS_32RW.SECTOR_RBA] ;>32mb ;AN000;
6764 ;mov cx,[bx+4]
6765 000009F4 8B4F04 MOV CX,[BX+ABS_32RW.ABS_RW_COUNT] ;>32mb ;AN000;
6766 ;lds bx,[bx+6]
6767 000009F7 C55F06 LDS BX,[BX+ABS_32RW.BUFFER_ADDR] ;>32mb ;AN000;
6768 old_style:
6769 ; 06/01/2024
6770 ; cf=0
6771 ;CLC
6772 000009FA C3 retn ;>32mb ;AN000;
6773
6774 ;-----
6775 ;
6776 ; Procedure Name : Fastxxx_Purge
6777 ;
6778 ; Input: None
6779 ; Functions: Purge Fastopen/ Cache Buffers
6780 ; Output: None
6781 ;
6782 ;-----
6783
6784 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
6785
6786 Fastxxx_Purge:
6787 000009FB 50 PUSH AX ; save regs. ;AN000;
6788 000009FC 56 PUSH SI ;AN000;
6789 000009FD 52 PUSH DX ;AN000;
6790
6791 000009FE 1E push ds ; set up ds to DOSDATA
6792 ;getdseg <ds>
6793 000009FF 2E8E1E[0700] mov ds,[cs:DosDSeg]
6794
6795 00000A04 F606[4611]80 TEST byte [FastOpenFlg],Fast_yes ; 80h
6796 ; fastopen installed ? ;AN000;
6797 00000A09 1F pop ds
6798 00000A0A 740B JZ short nofast ; no ;AN000;
6799 00000A0C B401 MOV AH,FastOpen_ID ; 1 ;AN000;
6800
6801 00000A0E B005 dofast:
6802 MOV AL,FONC_purge ;5 ; purge ;AN000;
6803 ;mov dl,[es:bp+0]
6804 ; 05/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
6805 ;MOV DL,[ES:BP+DPB.DRIVE] ; set up drive number ;AN000;
6806 ; 15/12/2022
6807 00000A10 268A5600 mov dl,[es:bp]
6808 00000A14 E85823 ;invoke Fast_Dispatch ; call fastopen/seek ;AN000;
6809 call Fast_Dispatch
6810 00000A17 5A POP DX ;AN000;
6811 00000A18 5E POP SI ; restore regs ;AN000;
6812 00000A19 58 POP AX ;AN000;
6813 00000A1A C3 retn ; exit
6814
6815 ;=====
6816 ; DOSMES.INC (MSDOS 6.0, 1991)
6817 ;=====

```

```

6818 ; 29/04/2019 - Retro DOS v4.0
6819
6820 ;include dossym.inc
6821 ;include dosmac.inc
6822 ;include doscntry.inc
6823
6824 ; DOSCODE Segment
6825
6826 ; 17/07/2018 - Retro DOS v3.0 [ DOSMES.INC (MSDOS 3.3, 1987) ]
6827 ; -----
6828 ;include divmes.inc
6829
6830 ; DOSCODE:48C3h (PCDOS 7.1, IBMDOS.COM) - 06/04/2024 -
6831 ; -----
6832 ; DOSCODE:4778h (MSDOS 6.21, MSDOS.SYS)
6833 ; -----
6834 ; DOSCODE:476Bh (MSDOS 5.0, MSDOS.SYS) - 05/11/2022 -
6835
6836 ; THIS IS THE ONLY DOS "MESSAGE". IT DOES NOT NEED A TERMINATOR.
6837 ;PUBLIC DIVMES
6838
6839 00000A1B 0D0A44697669646520- DIVMES: DB 13,10,"Divide overflow",13,10
6839 00000A24 6F766572666C6F770D-
6839 00000A2D 0A
6840
6841 ;PUBLIC DivMesLen
6842 DivMesLen:
6843 00000A2E 1300 DW $-DIVMES ; 19 ; Length of the above message in bytes
6844
6845 ; DOSCODE:478Dh (MSDOS 6.21, MSDOS.SYS)
6846 ; -----
6847 ; DOSCODE:4780h (MSDOS 5.0, MSDOS.SYS) - 05/11/2022 -
6848
6849 ; (MSDOS 6.0)
6850 ; VxD not found error message
6851
6852 NovVxDErrMsg:
6853 00000A30 596F75206D75737420- db 'You must have the file WINA20.386 in the root of your boot drive'
6853 00000A39 686176652074686520-
6853 00000A42 66696C652057494E41-
6853 00000A4B 32302E33383620696E-
6853 00000A54 2074686520726F6F74-
6853 00000A5D 206F6620796F757220-
6853 00000A66 626F6F742064726976-
6853 00000A6F 65
6854 00000A70 0D0A746F2072756E20- db 0Dh,0Ah,'to run windows in Enhanced Mode',0Dh,0Ah
6854 00000A79 57696E646F77732069-
6854 00000A82 6E20456E68616E6365-
6854 00000A8B 64204D6F64650D0A
6855
6856 VxDMesLen equ $ - NovVxDErrMsg ; 99
6857
6858 ; 13/05/2019 - Retro DOS v4.0
6859 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
6860
6861 ;include yesno.asm (MNSDOS 6.0)
6862 ; -----
6863 ; DOSCODE:47F0h (MSDOS 6.21, MSDOS.SYS)
6864 ; DOSCODE:47E3h (MSDOS 5.0, MSDOS.SYS) - 05/11/2022 -
6865
6866 ; This is for country Yes and No
6867
6868 ; 06/01/2024 (Retro DOS 5.0 - PCDOS 7.1 IBMDOS.COM)
6869 ;NLS_YES: db 'Y'
6870 ;NLS_NO: db 'N'
6871 ;NLS_yes2: db 'y'
6872 ;NLS_no2: db 'n'
6873
6874 ; -----
6875
6876 ; DOSCODE:493Bh (PCDOS 7.1, IBMDOS.COM) - 06/04/2024 -
6877 ; -----
6878 ; DOSCODE:47F4h (MSDOS 6.21, MSDOS.SYS)
6879 ; DOSCODE:47E7h (MSDOS 5.0, MSDOS.SYS) - 05/11/2022 -
6880
6881 ;SUBTTL EDIT FUNCTION ASSIGNMENTS AND HEADERS
6882
6883 ; The following two tables implement the current buffered input editing
6884 ; routines. The tables are pairwise associated in reverse order for ease
6885 ; in indexing. That is; The first entry in ESCTAB corresponds to the last
6886 ; entry in ESCFUNC, and the last entry in ESCTAB to the first entry in ESCFUNC.
6887
6888 ;PUBLIC CANCHAR
6889 CANCHAR:
6890 00000A93 1B DB CANCEL ; 1Bh ;Cancel line character
6891
6892 ;PUBLIC ESCCHAR
6893 ESCCHAR:
6894 00000A94 00 DB ESCCH ; 0 ;Lead-in character for escape sequences
6895
6896 ;IF NOT Rainbow
6897
6898 ESCTAB: ; LABEL BYTE
6899
6900 ;IF IBM
6901 00000A95 40 DB 64 ; Ctrl-Z - F6
6902 00000A96 4D DB 77 ; Copy one char - -->
6903 00000A97 3B DB 59 ; Copy one char - F1
6904 00000A98 53 DB 83 ; Skip one char - DEL
6905 00000A99 3C DB 60 ; Copy to char - F2
6906 00000A9A 3E DB 62 ; Skip to char - F4
6907 00000A9B 3D DB 61 ; Copy line - F3
6908 00000A9C 3D DB 61 ; Kill line (no change to template) - Not used
6909 00000A9D 3F DB 63 ; Reedit line (new template) - F5
6910 00000A9E 4B DB 75 ; Backspace - <--
6911 00000A9F 52 DB 82 ; Enter insert mode - INS (toggle)
6912 00000AA0 52 DB 82 ; Exit insert mode - INS (toggle)
6913 00000AA1 41 DB 65 ; Escape character - F7
6914 00000AA2 41 DB 65 ; End of table
6915 ;ENDIF
6916
6917 ESCEND: ; LABEL BYTE
6918
6919 ESCTABLEN EQU ESCEND-ESCTAB
6920
6921 ESCFUNC: ; LABEL WORD
6922
6923 00000AA3 [DF19] short_addr GETCH ; Ignore the escape sequence
6924 00000AA5 [5C1A] short_addr TWOESC
6925 00000AA7 [511B] short_addr EXITINS
6926 00000AA9 [511B] short_addr ENTERINS
6927 00000AAB [571A] short_addr BACKSP
6928 00000AAD [3D1B] short_addr REEDIT
6929 00000AAF [441A] short_addr KILNEW

```



```

6930 00000AB1 [D31A]      short_addr COPYLIN
6931 00000AB3 [051B]      short_addr SKIPSTR
6932 00000AB5 [D91A]      short_addr COPYSTR
6933 00000AB7 [FC1A]      short_addr SKIPONE
6934 00000AB9 [DE1A]      short_addr COPYONE
6935 00000ABB [DE1A]      short_addr COPYONE
6936 00000ABD [581B]      short_addr CTRLZ
6937
6938      ;ENDIF
6939
6940      ; DOSMES.INC (MSDOS 6.0, 1991)
6941      ;
6942      ; DOSMES.ASM (MSDOS 2.11, 1983)
6943
6944      ; OEMFunction key is expected to process a single function
6945      ; key input from a device and dispatch to the proper
6946      ; routines leaving all registers UNTOUCHED.
6947
6948      ; Inputs:  CS, SS are DOSGROUP
6949      ; Outputs: None. This function is expected to JMP to onw of
6950      ; the following labels:
6951      ;
6952      ;         GetCh      - ignore the sequence
6953      ;         TwoEsc     - insert an ESCChar in the buffer
6954      ;         ExitIns    - toggle insert mode
6955      ;         EnterIns   - toggle insert mode
6956      ;         BackSp     - move backwards one space
6957      ;         ReEdit     - reedit the line with a new template
6958      ;         KilNew     - discard the current line and start from scratch
6959      ;         CopyLin    - copy the rest of the template into the line
6960      ;         SkipStr    - read the next character and skip to it in the template
6961      ;         CopyStr    - read next char and copy from template to line until char
6962      ;         SkipOne    - advance position in template one character
6963      ;         CopyOne    - copy next character in template into line
6964      ;         CtrlZ      - place a ^Z into the template
6965      ; Registers that are allowed to be modified by this function are:
6966      ;         AX, CX, BP
6967
6968      ; 13/05/2019 - Retro DOS v4.0
6969      ;
6970      ; DOSCODE:4820h (MSDOS 6.21, MSDOS.SYS)
6971
6972      ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
6973      ;
6974      ; DOSCODE:4813h (MSDOS 5.0, MSDOS.SYS)
6975
6976      ; 06/01/2024 - Retro DOS v5.0
6977      ;
6978      ; DOSCODE:4967h (PCDOS 7.1, IBMDOS.COM)
6979
6980      OEMFunctionKey:
6981      CALL    _$STD_CON_INPUT_NO_ECHO      ; Get the second byte of the sequence
6982      MOV     CL,ESCTABLEN ; 14           ; length of table for scan
6983      PUSH    DI                           ; save DI (cannot change it!)
6984      MOV     DI,ESCTAB                    ; offset of second byte table
6985      push    es
6986      push    cs
6987      pop     es
6988      REPNE   SCASB                        ; Look it up in the table
6989      pop     es
6990      POP     DI                           ; restore DI
6991      SHL     CX,1                         ; convert byte offset to word
6992      MOV     BP,CX                        ; move to indexable register
6993      ;JMP     word [BP+ESCFUNC]            ; Go to the right routine
6994      JMP     word [CS:BP+ESCFUNC]
6995
6996      ;DOSCODE ENDS
6997
6998      ;=====
6999      ; TIME.ASM (MSDOS 6.0, 1991)
7000      ;=====
7001      ; Retro DOS v3.0 - 18/07/2018
7002
7003      ; SYSCALL.ASM (MSDOS 2.11, 1983)
7004      ;
7005      ; Retro DOS v2.0 - 13/03/2018
7006
7007      ;** TIME.ASM - System Calls and low level routines for DATE and TIME
7008
7009      ;BREAK <DATE AND TIME - SYSTEM CALLS 42,43,44,45>
7010
7011      ;** $GET_DATE - Get Current Date
7012      ;-----
7013      ; ENTRY none
7014      ; EXIT (cx:dx) = current date
7015      ; USES all
7016
7017      ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
7018      ; 06/01/2024 - Retro DOS v5.0 (Modified MSDOS 7.1 IBMDOS.COM)
7019
7020      _$GET_DATE: ;System call 42
7021
7022      PUSH    SS
7023      POP     DS
7024      CALL    READTIME                    ;Check for rollover to next day
7025      MOV     AX,[YEAR]
7026
7027      ; WARNING!!!! DAY and MONTH must be adjacently allocated!
7028
7029      MOV     BX,[DAY]                    ; fetch both day and month
7030      CALL    Get_user_stack ;Get pointer to user registers
7031      ;MOV     [SI+6],BX                    ;DH=month, DL=day
7032      MOV     [SI+user_env.user_DX],BX
7033      ADD     AX,1980                      ;Put bias back
7034      ;MOV     [SI+4],AX                    ;CX=year
7035      MOV     [SI+user_env.user_CX],AX
7036      MOV     AL,[SS:WEEKDAY]              ;hkn; SS override
7037      RET20: ; 05/11/2022
7038      RET24: ; 18/12/2022
7039      RETN
7040
7041      ;** $SET_DATE - Set Current Date
7042      ;-----
7043      ; ENTRY (cx:dx) = current date
7044      ; EXIT (al) = -1 iff bad date
7045      ; (al) = 0 if ok
7046      ; USES all
7047
7048      _$SET_DATE: ;System call 43
7049
7050      MOV     AL,-1                        ;Be ready to flag error
7051      SUB     CX,1980                      ;Fix bias in year
7052      JC      SHORT RET24                  ;Error if not big enough
7053      ; 05/11/2022

```

```

7054 00000AFB 72F7      jc      short RET20
7055 00000AFD 83F977    cmp     cx,119      ;Year must be less than 2100
7056 00000B00 77F2      ja      short RET24
7057 00000B02 08F6      or      dh,dh
7058                      ;JZ      short RET24
7059                      ; 05/11/2022
7060 00000B04 74EE      jz      short RET20
7061 00000B06 08D2      or      dl,dl
7062                      ;JZ      short RET24      ;Error if either month or day is 0
7063                      ; 05/11/2022
7064 00000B08 74EA      jz      short RET20
7065 00000B0A 80FE0C    cmp     dh,12      ;Check against max. month
7066 00000B0D 77E5      ja      short RET24
7067 00000B0F 16          push    ss
7068 00000B10 1F          pop     ds
7069                      ;CALL   DODATE
7070                      ; 18/12/2022
7071 00000B11 E90501    jmp     DODATE
7072                      ;RET24:
7073                      ;RETN
7074
7075                      ;** $GET_TIME - Get Current Time
7076                      ;-----
7077                      ; ENTRY   none
7078                      ; EXIT    (cx:dx) = current time
7079                      ; USES    all
7080
7081                      _$GET_TIME:      ;System call 44
7082
7083                      push    ss
7084                      pop     ds
7085                      call    READTIME
7086                      call    Get_User_Stack ;Get pointer to user registers
7087                      ;MOV     [SI+6],DX
7088                      mov     [SI+user_env.user_DX],DX
7089                      ;MOV     [SI+4],CX
7090                      mov     [SI+user_env.user_CX],CX
7091                      set_time_ok:      ; 06/01/2024
7092                      xor     al,al
7093                      RET26:
7094                      RETN
7095
7096                      ;** $SET_TIME - Set Current Time
7097                      ;-----
7098                      ; ENTRY   (cx:dx) = time
7099                      ; EXIT    (al) = 0 if Ok
7100                      ;        (al) = -1 if invalid
7101                      ; USES    ALL
7102
7103                      _$SET_TIME:      ;System call 45
7104
7105                      mov     al,-1      ;Flag in case of error
7106                      cmp     ch,24      ;Check hours
7107                      jae     short RET26
7108                      cmp     cl,60      ;Check minutes
7109                      jae     short RET26
7110                      cmp     dh,60      ;Check seconds
7111                      jae     short RET26
7112                      cmp     dl,100     ;Check 1/100's
7113                      jae     short RET26
7114                      push    cx
7115                      push    dx
7116                      push    ss
7117                      pop     ds
7118
7119                      ; 07/02/2024
7120                      %if 0
7121                      mov     bx,TIMEBUF
7122                      mov     cx,6
7123                      ; 06/02/2024 ; *
7124                      ;xor     dx,dx
7125                      ;mov     ax,dx
7126                      ;xor     ax,ax
7127                      ;cld      ; 06/01/2024
7128                      push    bx
7129                      ;CALL   SETREAD
7130                      ; 06/02/2024 ; *
7131                      call    SETREAD_X
7132                      %else
7133                      call    SETREAD_XT
7134                      %endif
7135
7136                      push    ds
7137                      lds     si,[BCLOCK]
7138                      call    DEVIOCALL2      ;Get correct day count
7139                      pop     ds
7140                      pop     bx
7141                      call    SETWRITE
7142                      pop     word [TIMEBUF+4]
7143                      pop     word [TIMEBUF+2]
7144                      lds     si,[BCLOCK]
7145                      call    DEVIOCALL2      ;Set the time
7146                      ; 06/01/2024
7147                      ;xor     al,al
7148                      ;RETN
7149                      jmp     short set_time_ok
7150
7151                      ; 11/07/2018 - Retro DOS v3.0
7152                      ; Retro DOS v2.0 - 14/03/2018
7153
7154                      FOURYEARS EQU 3*365 + 366 ; = 1461
7155
7156                      ;SUBTTL DATE16, READTIME, DODATE -- GUTS OF TIME AND DATE
7157                      ;-----
7158                      ; Date16 returns the current date in AX, current time in DX
7159                      ; AX - YYYYYYMMMMDDDDD years months days
7160                      ; DX - HHHHHMMMMSSSSSS hours minutes seconds/2
7161
7162                      DATE16:
7163
7164                      ;M048      Context DS
7165                      ;
7166                      ; Since this function can be called thru int 2f we shall not assume that SS
7167                      ; is DOSDATA
7168
7169                      ;push    ss
7170                      ;pop     ds
7171
7172                      ;getdseg <ds>      ; M048
7173
7174                      ; 13/05/2019 - Retro DOS v4.0
7175                      mov     ds,[cs:DosDSEG]
7176
7177                      push    cx

```

```

7178 00000B66 06          PUSH    ES
7179 00000B67 E82000      CALL    READTIME
7180 00000B6A 07          POP     ES
7181 00000B6B D0E1      SHL     CL,1          ;Minutes to left part of byte
7182 00000B6D D0E1      SHL     CL,1
7183 00000B6F D1E1      SHL     CX,1          ;Push hours and minutes to left end
7184 00000B71 D1E1      SHL     CX,1
7185 00000B73 D1E1      SHL     CX,1
7186 00000B75 D0EE      SHR     DH,1          ;Count every two seconds
7187 00000B77 08F1      OR      CL,DH          ;Combine seconds with hours and minutes
7188 00000B79 89CA      MOV     DX,CX
7189
7190          ; WARNING! MONTH and YEAR must be adjacently allocated
7191
7192 00000B7B A1[5103]     MOV     AX,[MONTH]    ;Fetch month and year
7193 00000B7E B104      MOV     CL,4
7194 00000B80 D2E0      SHL     AL,CL          ;Push month to left to make room for day
7195 00000B82 D1E0      SHL     AX,1
7196 00000B84 59          POP     CX
7197 00000B85 0A06[5003] OR      AL,[DAY]
7198 RET21:
7199          RETN
7200
7201          ;-----
7202
7203 READTIME:
7204
7205          ;Gets time in CX:DX. Figures new date if it has changed.
7206          ;Uses AX, CX, DX.
7207
7208 00000B8A C706[BD0D]0000 MOV     word [DATE_FLAG],0 ; reset date flag for CPMIO
7209 00000B90 56          PUSH    SI
7210 00000B91 53          PUSH    BX
7211
7212 00000B92 BB[B603]     MOV     BX,TIMEBUF
7213          ; 07/02/2024
7214 %if 0
7215          MOV     CX,6
7216          ; 06/02/2024
7217          ;;XOR    DX,DX
7218          ;;MOV    AX,DX
7219          ; 06/01/2024
7220          ;xor     ax,ax
7221          ;cld
7222          ;CALL    SETREAD
7223          ; 06/02/2024
7224          call    SETREAD_X
7225 %else
7226 00000B95 E87447      call    SETREAD_XTC
7227 %endif
7228 00000B98 1E          PUSH    DS
7229 00000B99 C536[2E00]   LDS     SI,[BCLOCK]
7230 00000B9D E8F146      CALL    DEVIOCALL2    ;Get correct date and time
7231 00000BA0 1F          POP     DS
7232 00000BA1 5B          POP     BX
7233 00000BA2 5E          POP     SI
7234 00000BA3 A1[B603]     MOV     AX,[TIMEBUF]
7235 00000BA6 8B0E[B803]   MOV     CX,[TIMEBUF+2]
7236 00000BAA 8B16[BA03]   MOV     DX,[TIMEBUF+4]
7237 00000BAE 3B06[5403]   CMP     AX,[DAYCNT]    ;See if day count is the same
7238          ;JZ      SHORT RET22
7239 00000BB2 74D5      JZ      SHORT RET21 ; 18/07/2018
7240          ;cmp     ax,43830
7241 00000BB4 3D36AB      CMP     AX,FOURYEARS*30 ;Number of days in 120 years
7242 00000BB7 733D      JAE     SHORT RET22    ;Ignore if too large
7243 00000BB9 A3[5403]     MOV     [DAYCNT],AX
7244 00000BBC 56          PUSH    SI
7245 00000BBD 51          PUSH    CX
7246 00000BBE 52          PUSH    DX          ;Save time
7247 00000BBF 31D2      XOR     DX,DX
7248          ;mov     CX,1461
7249 00000BC1 B9B505      MOV     CX,FOURYEARS    ;Number of days in 4 years
7250 00000BC4 F7F1      DIV     CX          ;Compute number of 4-year units
7251 00000BC6 D1E0      SHL     AX,1
7252 00000BC8 D1E0      SHL     AX,1
7253 00000BCA D1E0      SHL     AX,1          ;Multiply by 8 (no. of half-years)
7254 00000BCC 89C1      MOV     CX,AX          ;<240 implies AH=0
7255
7256 00000BCE BE[140D]     MOV     SI,YRTAB        ;Table of days in each year
7257
7258          ;CALL    DSLIDE          ;Find out which of four years we're in
7259          ; 26/06/2024
7260 00000BD1 E82500      call    DSLIDE1 ; ah = 0
7261 00000BD4 D1E9      SHR     CX,1          ;Convert half-years to whole years
7262 00000BD6 7304      JNC     SHORT SK      ;Extra half-year?
7263 00000BD8 81C2C800   ADD     DX,200
7264 SK:
7265 00000BDC E82400      CALL    SETYEAR
7266 00000BDF B101      MOV     CL,1          ;At least at first month in year
7267
7268 00000BE1 BE[1C0D]     MOV     SI,MONTAB       ;Table of days in each month
7269
7270          ;CALL    DSLIDE          ;Find out which month we're in
7271          ; 26/06/2024
7272 00000BE4 E81200      call    DSLIDE1 ; ah = 0
7273 00000BE7 880E[5103]   MOV     [MONTH],CL
7274 00000BEB 42          INC     DX
7275 00000BEC 8816[5003]   MOV     [DAY],DL
7276 00000BF0 E88C00      CALL    WKDAY          ;Set day of week
7277 00000BF3 5A          POP     DX
7278 00000BF4 59          POP     CX
7279 00000BF5 5E          POP     SI
7280 RET22:
7281 00000BF6 C3          RETN
7282
7283          ;-----
7284
7285 DSLIDE:
7286          ; 26/06/2024
7287 00000BF7 B400      MOV     AH,0
7288          ; 06/01/2024
7289          ; (AH=0)
7290 DSLIDE1:
7291 00000BF9 AC          LODSB          ;Get count of days
7292 00000BFA 39C2      CMP     DX,AX          ;See if it will fit
7293          ;JB      SHORT RET23      ;If not, done
7294 00000BFC 72F8      jnb     short RET22 ; 13/05/2019 - Retro DOS v4.0
7295 00000BFE 29C2      SUB     DX,AX
7296 00000C00 41          INC     CX          ;Count one more month/year
7297 00000C01 EBF6      JMP     SHORT DSLIDE1
7298
7299          ;-----
7300 SETYEAR:
7301

```

```

7302
7303 ;Set year with value in CX. Adjust length of February for this year.
7304
7305 ; NOTE: This can also be called thru int 2f. If this is called then it will
7306 ; set DS to DOSDATA. Since the only guy calling this should be the DOS
7307 ; redir, DS will be DOSDATA anyway. It is going to be in-efficient to
7308 ; preserve DS as CHKYR is also called as a routine.
7309
7310 ; MSDOS 6.0 (18/07/2018) ; *
7311
7312 ;GETDSEG DS
7313
7314 ;PUSH CS ; *
7315 ;POP DS ; *
7316
7317 ; 13/05/2019 - Retro DOS v4.0
7318 00000C03 2E8E1E[0700] mov ds,[cs:DosDseg]
7319
7320 ; Offset 18CEh in IBMDOS.COM (MSDOS 3.3), 1987
7321 ; 05/11/2022
7322 ; DOSCODE:4970h in MSDOS.SYS (MSDOS 5.0), 1991
7323
7324 00000C08 880E[5203] MOV [YEAR],CL
7325 CHKYR:
7326 00000C0C F6C103 TEST CL,3 ;Check for leap year
7327 00000C0F B01C MOV AL,28
7328 00000C11 7502 JNZ SHORT SAVFEB ;28 days if no leap year
7329 00000C13 FEC0 INC AL ;Add leap day
7330 SAVFEB:
7331 00000C15 A2[1D0D] mov [february],al
7332 ;MOV [MONTAB+1],AL ;Store for February
7333 RET23:
7334 00000C18 C3 RETN
7335
7336 ;-----
7337
7338 DODATE:
7339 00000C19 EF0FF CALL CHKYR ;Set Feb. up for new year
7340 00000C1C 88F0 MOV AL,DH
7341
7342 00000C1E BB[1B0D] MOV BX,MONTAB-1 ;DOSDATA:0D1Bh for MSDOS 6.21
7343 ; 06/01/2024
7344 ;DOSDATA:0D1Bh for PCDOS 7.1
7345
7346 00000C21 D7 XLAT ;Look up days in month
7347 00000C22 38D0 CMP AL,DL
7348 00000C24 B0FF MOV AL,-1 ;Restore error flag, just in case
7349 ;JB SHORT RET25 ;Error if too many days
7350 00000C26 72F0 jb short RET23 ; 18/07/2018
7351 00000C28 E8D8FF CALL SETYEAR
7352
7353 ;
7354 ; WARNING! DAY and MONTH must be adjacently allocated
7355 ;
7356 00000C2B 8916[5003] MOV [DAY],DX ;Set both day and month
7357 00000C2F D1E9 SHR CX,1
7358 00000C31 D1E9 SHR CX,1
7359 00000C33 88B505 ;mov ax,1461
7360 00000C36 89D3 MOV AX,FOURYEARS
7361 00000C38 F7E1 MOV BX,DX
7362 00000C3A 8A0E[5203] MUL CX
7363 00000C3E 80E103 MOV CL,[YEAR]
7364 AND CL,3
7365 00000C41 BE[140D] MOV SI,YRTAB
7366
7367 00000C44 89C2 MOV DX,AX
7368 00000C46 D1E1 SHL CX,1 ;Two entries per year, so double count
7369 00000C48 E84700 CALL DSUM ;Add up the days in each year
7370 00000C4B 88F9 MOV CL,BH ;Month of year
7371
7372 00000C4D BE[1C0D] MOV SI,MONTAB
7373
7374 00000C50 49 DEC CX ;Account for months starting with one
7375 00000C51 E83E00 CALL DSUM ;Add up days in each month
7376 00000C54 88D9 MOV CL,BL ;Day of month
7377 00000C56 49 DEC CX ;Account for days starting with one
7378 00000C57 01CA ADD DX,CX ;Add in to day total
7379 00000C59 92 XCHG AX,DX ;Get day count in AX
7380 00000C5A A3[5403] MOV [DAYCNT],AX
7381 00000C5D 56 PUSH SI
7382 00000C5E 53 PUSH BX
7383 00000C5F 50 PUSH AX
7384
7385 ; 07/02/2024
7386 %if 0
7387 MOV BX,TIMEBUF
7388 MOV CX,6
7389 ; 06/02/2024 ; *
7390 ;;XOR DX,DX
7391 ;;MOV AX,DX
7392 ;; 06/01/2024
7393 ;xor ax,ax
7394 ;cwd
7395 PUSH BX
7396 ;CALL SETREAD
7397 ; 06/02/2024 ; *
7398 call SETREAD_X
7399 %else
7400 00000C60 E8A546 call SETREAD_XT
7401 %endif
7402
7403 00000C63 1E PUSH DS
7404 00000C64 C536[2E00] LDS SI,[BCLOCK]
7405 00000C68 E82646 CALL DEVIOCALL2 ;Get correct date and time
7406 00000C6B 1F POP DS
7407 00000C6C 5B POP BX
7408 00000C6D E8D546 CALL SETWRITE
7409 00000C70 8F06[B603] POP WORD [TIMEBUF]
7410 00000C74 1E PUSH DS
7411 00000C75 C536[2E00] LDS SI,[BCLOCK]
7412 00000C79 E81546 CALL DEVIOCALL2 ;Set the date
7413 00000C7C 1F POP DS
7414 00000C7D 5B POP BX
7415 00000C7E 5E POP SI
7416
7417 00000C7F A1[5403] WKDAY: MOV AX,[DAYCNT]
7418 00000C82 31D2 XOR DX,DX
7419 00000C84 B90700 MOV CX,7
7420 00000C87 40 INC AX
7421 00000C88 40 INC AX ;First day was Tuesday
7422 00000C89 F7F1 DIV CX ;Compute day of week
7423 00000C8B 8816[5603] MOV [WEEKDAY],DL
7424 00000C8F 30C0 XOR AL,AL ;Flag OK
7425 RET25:

```

```

7426 00000C91 C3      RETN
7427
7428
7429
7430
7431
7432
7433
7434
7435
7436
7437
7438
7439 00000C92 B400
7440 00000C94 E305
7441
7442
7443 00000C96 AC
7444 00000C97 01C2
7445 00000C99 E2FB
7446
7447 00000C9B C3
7448
7449
7450
7451
7452
7453
7454
7455
7456
7457
7458
7459
7460
7461
7462
7463
7464
7465
7466
7467
7468
7469
7470
7471
7472
7473
7474
7475
7476
7477
7478
7479
7480
7481
7482
7483
7484
7485
7486
7487
7488
7489
7490
7491
7492
7493
7494
7495
7496
7497
7498
7499
7500
7501
7502
7503
7504
7505
7506
7507
7508
7509
7510
7511
7512
7513
7514
7515
7516
7517 00000C9C 36C50E[B203]
7518 00000CA1 8CDB
7519
7520
7521
7522
7523
7524 00000CA3 16
7525 00000CA4 1F
7526
7527
7528
7529
7530
7531
7532
7533
7534
7535
7536 00000CA5 3C01
7537 00000CA7 7502
7538
7539
7540
7541
7542
7543 00000CA9 30FF
7544
7545
7546
7547
7548
7549

;-----
; ** DSUM - Compute the sum of a string of bytes
;
; ENTRY (cx) = byte count
;        (ds:si) = byte address
;        (dx) = sum register, initialized by caller
; EXIT (dx) updated
; USES ax, cx, dx, si, flags
;
DSUM:
    MOV     AH,0
    JCXZ    DSUM9 ; 13/05/2019 - Retro DOS v4.0
    ;JCXZ    RET25 ; 18/07/2018
DSUM1:
    LODSB
    ADD     DX,AX
    LOOP    DSUM1
DSUM9:
    RETN

;=====
; GETSET.ASM (MSDOS 6.0, 1991)
;=====
; 29/04/2019 - Retro DOS v4.0
; 18/07/2018 - Retro DOS v3.0 (GETSET.ASM, MSDOS 6.0, 1991)
;
; 12/03/2018 - Retro DOS v2.0
;
;TITLE     GETSET - GETting and SETting MS-DOS system calls
;NAME      GETSET
;
;CODE      SEGMENT BYTE PUBLIC 'CODE'
;          ASSUME SS:DOSGROUP,CS:DOSGROUP
;
;USERNUM:
;    DW     0 ; 24 bit user number
;    DB     0
;    IF     IBM
;OEMNUM: DB 0 ; 8 bit OEM number
;    ELSE
;OEMNUM: DB 0FFH ; 8 bit OEM number
;    ENDIF
;
;MSVERS:   ; MS-DOS version in hex for $GET_VERSION
; 08/07/2018 - Retro DOS v3.0
;MSMAJOR: DB MAJOR_VERSION ; DOS_MAJOR_VERSION
;MSMINOR: DB MINOR_VERSION ; DOS_MINOR_VERSION
;
;BREAK <$Get_Version -- Return MSDOS version number>
;-----
; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:4A0Fh (MSDOS 5.0 MSDOS.SYS)
;
; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; DOSCODE:4B5Fh (PCDOS 7.1 IBMDOS.COM)
;
_$GET_VERSION:
; Inputs:
; None
; Function:
; Return MS-DOS version number
; Outputs:
; OEM number in BH
; User number in BL:CX (24 bits)
; Version number as AL:AH in binary
; NOTE: On pre 1.28 DOSs AL will be zero
;
; MSDOS 6.0
;
; Fake_Count is used to lie about the version numbers to support
; old binarys. See ms_table.asm for more info.
;
; if input al = 00
; (bh) = OEM number
; else if input al = 01
; (bh) = version flags
;
; bits 0-2 = DOS internal revision
; bits 3-7 = DOS type flags
; bit 3 = DOS is in ROM
; bit 4 = DOS in in HMA
; bits 5-7 = reserved
; M007 change - only bit 3 is now valid. other bits
; are 0 when AL = 1
;
; 06/01/2024 (PCDOS 7.1 IBMDOS.COM)
    lds     cx, [ss:USERNUM]
    mov     bx, ds
;
; MSDOS 3.3 (IBMDOS.COM, offset 196Dh)
;-----
; MSDOS 6.21 (MSDOS.SYS, DOSCODE:4A1Ch)
;
    PUSH    SS
    POP     DS
;
; 06/01/2024
;MOV     BX,[USERNUM+2]
;MOV     CX,[USERNUM]
;
; 13/05/2019 - Retro DOS v4.0
;
; If AL == 1, ROMDOS will return BH = dos internal version # &
; DOS flags
    cmp     AL,1
    jne     short Norm_Vers
;
;ifdef ROMDOS
;    mov     BH,DOSINROM ; Just set the bit for ROM version
;                    (DOSINROM = 8)
;else
;    xor     bh,bh ; otherwise return 0
;endif
;M007 end
;
Norm_Vers:
    MOV     AX,[MSVERS] ; MSDOS 3.3
;
; MSDOS 6.0 ; MSVERS is a label in TABLE segment

```

```

7550             ; 26/06/2024 - Retro DOS v5.0
7551             ; (PCDOS 7.1 IBMDOS.COM)
7552             ; 13/05/2019 - Retro DOS v4.0
7553             ;push ds             ; Get the version number from the
7554 00000CAB 8E1E[3003] mov ds,[CurrentPDB] ; current app's PSP segment
7555             ;mov ax,[40h]
7556 00000CAF A14000 mov ax,[PDB.Version] ; AX = DOS version number
7557             ; 07/12/2022
7558             ;pop ds
7559 00000CB2 E8C2F7 call Get_User_Stack
7560             ; Put values for return registers
7561             ; in the proper place on the user's
7562             ; stack addressed by DS:SI
7563             ; 06/01/2024 (PCDOS 7.1 IBMDOS.COM)
7564 gdrvfspc_ret:
7565             ;MOV [SI+user_env.user_AX],AX
7566 00000CB5 8904 MOV [SI],AX
7567             ;MOV [SI+4],CX
7568 00000CB7 894C04 mov [SI+user_env.user_CX],CX
7569 set_user_bx:
7570             ;MOV [SI+2],BX
7571 00000CBA 895C02 mov [SI+user_env.user_BX],BX
7572             RETN
7573 00000CBD C3
7574
7575             ; 18/07/2018 - Retro DOS v3.0
7576
7577 ;BREAK <$Get/Set_Verify_on_Write - return/set verify-after-write flag>
7578 ;-----
7579
7580 ;** $Get_Verify_On_Write - Get Status of Verify on write flag
7581 ;
7582 ; ENTRY none
7583 ; EXIT (al) = value of VERIFY flag
7584 ; USES all
7585
7586
7587 _$GET_VERIFY_ON_WRITE:
7588
7589 ;hkn; SS override
7590 00000CBE 36A0[FF02] MOV AL,[SS:VERFLG] ; Retro DOS v2.0 - 12/03/2018
7591 00000CC2 C3 retn
7592
7593 ;** $Set_Verify_On_Write - Set Status of Verify on write flag
7594 ;
7595 ; ENTRY (al) = value of VERIFY flag
7596 ; EXIT none
7597 ; USES all
7598
7599 _$SET_VERIFY_ON_WRITE:
7600
7601 00000CC3 2401 AND AL,1
7602 ;hkn; SS override
7603 00000CC5 36A2[FF02] MOV [SS:VERFLG],AL ; Retro DOS v2.0 - 12/03/2018
7604 RET27: ; 18/07/2018
7605 00000CC9 C3 retn
7606
7607             ; 19/07/2018 - Retro DOS v3.0
7608
7609 ;BREAK <$International - return country-dependent information>
7610 ;-----
7611 ;
7612 ; Procedure Name : $INTERNATIONAL
7613 ;
7614 ; Inputs:
7615 ; MOV AH,International
7616 ; MOV AL,country (al = 0 => current country)
7617 ; [MOV BX,country]
7618 ; LDS DX,block
7619 ; INT 21
7620 ; Function:
7621 ; give users an idea of what country the application is running
7622 ; Outputs:
7623 ; IF DX != -1 on input (get country)
7624 ; AL = 0 means return current country table.
7625 ; 0<AL<0FFH means return country table for country AL
7626 ; AL = 0FF means return country table for country BX
7627 ; No Carry:
7628 ; Register BX will contain the 16-bit country code.
7629 ; Register AL will contain the low 8 bits of the country code.
7630 ; The block pointed to by DS:DX is filled in with the information
7631 ; for the particular country.
7632 ; BYTE Size of this table excluding this byte and the next
7633 ; BYTE Country code represented by this table
7634 ; A sequence of n bytes, where n is the number specified
7635 ; by the first byte above and is not > internat_block_max,
7636 ; in the correct order for being returned by the
7637 ; INTERNATIONAL call as follows:
7638 ; WORD Date format 0=mdy, 1=dmy, 2=ymd
7639 ; 5 BYTE Currency symbol null terminated
7640 ; 2 BYTE thousands separator null terminated
7641 ; 2 BYTE Decimal point null terminated
7642 ; 2 BYTE Date separator null terminated
7643 ; 2 BYTE Time separator null terminated
7644 ; 1 BYTE Bit field. Currency format.
7645 ; Bit 0. =0 $ before # =1 $ after #
7646 ; Bit 1. no. of spaces between # and $ (0 or 1)
7647 ; 1 BYTE No. of significant decimal digits in currency
7648 ; 1 BYTE Bit field. Time format.
7649 ; Bit 0. =0 12 hour clock =1 24 hour
7650 ; DWORD Call address of case conversion routine
7651 ; 2 BYTE Data list separator null terminated.
7652 ; Carry:
7653 ; Register AX has the error code.
7654 ; IF DX = -1 on input (set current country)
7655 ; AL = 0 is an error
7656 ; 0<AL<0FFH means set current country to country AL
7657 ; AL = 0FF means set current country to country BX
7658 ; No Carry:
7659 ; Current country SET
7660 ; Register AL will contain the low 8 bits of the country code.
7661 ; Carry:
7662 ; Register AX has the error code.
7663 ;-----
7664
7665 ;procedure $INTERNATIONAL,NEAR ; DOS 3.3
7666
7667             ; 13/05/2019 - Retro DOS v4.0
7668             ; DOSCODE:4A4Dh (MSDOS 6.21, MSDOS.SYS)
7669
7670             ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
7671             ; DOSCODE:4A40h (MSDOS 5.0, MSDOS.SYS)
7672
7673             ; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 PCDOS.COM)

```



```

7674 ; DOSCODE:4B8Dh (PCDOS 7.1, IBMDOS.COM)
7675
7676 _$INTERNATIONAL: ; IBMDOS.COM (MSDOS 3.3), offset 1992h
7677
7678 00000CCA 3CFF      CMP     AL,0FFh
7679 00000CCC 7404      JZ      short BX_HAS_CODE ; -1 means country code is in BX
7680 00000CCE 88C3      MOV     BL,AL ; Put AL country code in BX
7681 00000CD0 30FF      XOR     BH,BH
7682 BX_HAS_CODE:
7683 00000CD2 1E      PUSH    DS
7684 00000CD3 07      POP     ES
7685 00000CD4 52      PUSH    DX
7686 00000CD5 5F      POP     DI ; User buffer to ES:DI
7687
7688 ;hkn; SS is DOSDATA
7689 ; context DS
7690
7691 00000CD6 16      push    ss
7692 00000CD7 1F      pop     ds
7693
7694 00000CD8 83FFFF      CMP     DI,-1
7695 00000CDB 745D      JZ      short international_set
7696 00000CDD 09DB      OR      BX,BX
7697 00000CDF 7505      JNZ     short international_find
7698
7699 ;hkn; country_cdpq is in DOSDATA segment.
7700 00000CE1 BE[2A12] MOV     SI,COUNTRY_CDPG
7701
7702 00000CE4 EB39      JMP     SHORT international_copy
7703
7704 international_find:
7705 ;MOV     BP,0 ; flag it for GetCntry only
7706 ; 06/01/2024
7707 00000CE6 31ED      xor     bp,bp ; 0
7708 00000CE8 E80A00 CALL    international_get
7709 00000CEB 7255      JC      short errtn
7710 ;CMP     BX,0 ; nlsfunc finished it ?
7711 ; 06/01/2024
7712 00000CED 09DB      or      bx,bx
7713 00000CEF 752E      JNZ     SHORT international_copy ; no, copy by myself
7714 00000CF1 89D3      MOV     BX,DX ; put country back
7715 00000CF3 EB3A      JMP     SHORT international_ok3
7716
7717 international_get:
7718 00000CF5 BE[2A12] MOV     SI,COUNTRY_CDPG
7719
7720 ;hkn; country_cdpq is in DOSDATA segment.
7721 ;hkn; use ss override to access COUNTRY_CDPG fields
7722
7723 ; MSDOS 3.3
7724 ;;cmp     bx,[SI+63h]
7725 ;CMP     BX,[SI+DOS_CCDPG.ccDosCountry]
7726 ;jz      short RET27
7727
7728 ; 13/05/2019 - Retro DOS v4.0
7729
7730 ; MSDOS 6.0
7731 ;cmp     bx,[ss:si+68h]
7732 00000CF8 363B5C68 CMP     BX,[ss:SI+DOS_CCDPG.ccDosCountry] ; = current country id
7733 00000CFC 74CB      jz      short RET27 ; return if equal
7734
7735 00000CFE 89DA      MOV     DX,BX
7736 00000D00 31DB      XOR     BX,BX ; bx = 0, default code page
7737 ;CallInstall NLSInstall,NLSFUNC,0 ; check if NLSFUNC in memory
7738 00000D02 B80014 mov     ax,1400h
7739 00000D05 CD2F int     2Fh ; - Multiplex - NLSFUNC.COM - INSTALLATION CHECK
7740 ; Return: AL = 00h not installed, OK to install
7741 ; 01h not installed, not OK
7742 ; FFh installed
7743 00000D07 3CFF      CMP     AL,0FFh
7744 00000D09 7510      JNZ     short interr ; not in memory
7745
7746 ; 06/01/2024
7747 00000D0B B80314 mov     ax,1403h ; set country info
7748
7749 ;cmp     bp,0
7750 00000D0E 09ED      or      bp,bp ; GetCntry ?
7751 00000D10 7501      JNZ     short stcdpg
7752
7753 ;CallInstall GetCntry,NLSFUNC,4 ; get country info
7754 ;mov     ax,1404h
7755 00000D12 40      inc     ax ; AX = 1404h ; get country info
7756
7757 ; 06/01/2024
7758 ;int     2Fh ; - Multiplex - NLSFUNC.COM - GET COUNTRY INFO
7759 ; ; BX = code page, DX = country code,
7760 ; ; DS:SI -> internal code page structure
7761 ; ; ES:DI -> user buffer
7762 ; ; Return: AL = status
7763 ;
7764 ;JMP     short chkok
7765
7766 ;nop
7767
7768 stcdpg:
7769 ;CallInstall SetCodePage,NLSFUNC,3 ; set country info
7770 ; 06/01/2024
7771 ;mov     ax,1403h
7772 gscdpg:
7773 00000D13 CD2F int     2Fh ; - Multiplex - NLSFUNC.COM - SET COUNTRY INFO
7774 ; ; DS:SI -> internal code page structure
7775 ; ; BX = code page, DX = country code
7776 ; ; Return: AL = status
7777
7778 00000D15 08C0      or      al,al ; success ?
7779 ;retz ; yes
7780 00000D17 74B0      jz      short RET27
7781
7782 setcarry:
7783 00000D19 F9      STC ; set carry
7784 00000D1A C3      retn
7785
7786 00000D1B B0FF      MOV     AL,0FFh ; flag nlsfunc error
7787 00000D1D EBFA      JMP     short setcarry
7788
7789 international_copy:
7790
7791 ;hkn; country_cdpq is in DOSDATA segment.
7792 ;hkn; use ss override to access COUNTRY_CDPG fields
7793
7794 ; MSDOS 3.3
7795 ;;mov     bx,[SI+63h]
7796 ;mov     BX,[SI+DOS_CCDPG.ccDosCountry]
7797 ;mov     SI,COUNTRY_CDPG+DOS_CCDPG.ccDFormat ; 08/09/2018

```

```

7798
7799 ; 13/05/2019 - Retro DOS v4.0
7800
7801 ; MSDOS 6.0
7802 ;mov bx,[ss:si+68h]
7803 00000D1F 368B5C68 MOV BX,[ss:SI+DOS_CCDPG.ccDosCountry] ; = current country id
7804 00000D23 BE[9612] MOV SI,COUNTRY_CDPG+DOS_CCDPG.ccDFormat ; COUNTRY_CDPG + 108
7805
7806 ;mov cx,24
7807 00000D26 B91800 MOV CX,OLD_COUNTRY_SIZE
7808
7809 ; MSDOS 6.0
7810
7811 ;hkn; must set up DS to SS so that international info can be copied
7812
7813 00000D29 1E push ds
7814
7815 00000D2A 16 push ss ; cs -> ss
7816 00000D2B 1F pop ds
7817
7818 00000D2C F3A4 REP MOVSB ; copy country info
7819
7820 ; MSDOS 6.0
7821
7822 00000D2E 1F pop ds ;hkn; restore ds
7823
7824 international_ok3:
7825 00000D2F E845F7 call Get_User_Stack
7826 ;ASSUME DS:NOTHING
7827 ;;MOV [SI+2],BX
7828 ;MOV [SI+user_env.user_BX],BX
7829 ; 26/06/2024 (PCDOS 7.1 IBMDOS.COM)
7830 00000D32 E885FF call set_user_bx
7831
7832 00000D35 89D8 MOV AX,BX ; Return country code in AX too.
7833 ;SYS_RET_OK_jmp:
7834 ; 05/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
7835 ; 09/01/2024
7836 ;nono: ; 15/12/2022
7837 SYS_RET_OK_jmp:
7838 00000D37 E934F9 jmp SYS_RET_OK
7839
7840 international_set:
7841
7842 ;hkn; ASSUME DS:DOSGROUP
7843 ;ASSUME DS:DOSDATA
7844
7845 00000D3A BD0100 MOV BP,1 ; flag it for SetCodePage only
7846 00000D3D E8B5FF CALL international_get
7847 00000D40 73F3 JNC short international_ok
7848
7849 00000D42 3CFF errtn: CMP AL,0FFh
7850 00000D44 7403 JZ short errtn2
7851
7852 00000D46 E92FF9 errtn1: jmp SYS_RET_ERR ; return what we got from NLSFUNC
7853
7854 errtn2: ;error error_invalid_function; NLSFUNC not existent
7855
7856 ;mov al,1
7857 00000D49 B001 mov al,error_invalid_function
7858 00000D4B EBF9 jmp short errtn1 ; 13/05/2019 - Retro DOS v4.0
7859
7860 ;errtn3:
7861 ; jmp SYS_RET_ERR
7862
7863 ;EndProc $INTERNATIONAL
7864
7865 ; -----
7866
7867 ; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 PCDOS.COM)
7868 ; DOSCODE:4C10h (PCDOS 7.1, IBMDOS.COM)
7869
7870 _$ExtCountryInfo:
7871 ; INT 21h, AH = 70h
7872 ; GET/SET INTERNATIONALIZATION INFORMATION
7873 ; ****
7874 ; AL = subfunction
7875 ; 00h SET general internationalization info
7876 ; CX = buffer size (up to 38 bytes)
7877 ; DS:SI -> buffer containing internationalization info
7878 ; first three bytes are skipped, the rest is copied to
7879 ; somewhere in the DOS data segment
7880 ; 01h SET extended internationalization info
7881 ; CX = number of bytes to set (up to 58 bytes)
7882 ; DS:SI -> buffer containing internationalization info
7883 ; 02h GET extended internationalization info
7884 ; CX = buffer size in bytes (up to 58 bytes used)
7885 ; ES:DI -> buffer
7886 ; ****
7887 ; (Ref: Ralf Brown's Interrupt List) - had some mistakes -
7888 00000D4D 3C02 cmp al,2
7889 00000D4F 77F8 ja short errtn2
7890 00000D51 06 push es
7891 00000D52 16 push ss
7892 00000D53 750F jnz short ext_cntry_inf_1
7893 00000D55 07 pop es ; AX = GET 35 bytes info (from offset 3 to 37)
7894 ; (38 bytes buffer is used)
7895 00000D56 BF[9212] mov di,_COUNTRY_ID
7896 00000D59 268B45FE mov ax,[es:di-2] ; NEW_COUNTRY_SIZE = 38
7897 00000D5D BB0300 mov bx,3 ; skip the 1st 3 bytes of the buffer
7898 00000D60 01DE add si,bx
7899 00000D62 EB13 jmp short ext_cntry_inf_4
7900
7901 ext_cntry_inf_1:
7902 00000D64 FEC8 dec al
7903 00000D66 7506 jnz short ext_cntry_inf_2 ; AX = 2
7904 ; AX = 1 (set)
7905 00000D68 07 pop es
7906 00000D69 BF[BA12] mov di,_ENU ; "ENU"
7907 00000D6C EB04 jmp short ext_cntry_inf_3
7908
7909 ext_cntry_inf_2:
7910 00000D6E 1F pop ds ; AX = 2 (get)
7911 00000D6F BE[BA12] mov si,_ENU ; "ENU"
7912
7913 ext_cntry_inf_3: ; CODE XREF: DOSCODE:4C2FAj
7914 00000D72 31DB xor bx,bx ; 0
7915 00000D74 B83A00 mov ax,58 ; 3Ah
7916
7917 ext_cntry_inf_4:
7918 00000D77 39C1 cmp cx,ax ; > 38 ? (58)
7919 ;jb short ext_cntry_inf_5 ; no
7920 00000D79 7602 jna short ext_cntry_inf_5 ; 06/01/2024
7921 00000D7B 89C1 mov cx,ax ; yes, decrease size to 38 (58)

```



```

7922
7923
7924 00000D7D 89C8
7925 00000D7F 29D9
7926 00000D81 F3A4
7927 00000D83 07
7928 00000D84 E91D03
7929
7930
7931
7932
7933
7934
7935
7936
7937
7938
7939
7940
7941
7942
7943
7944
7945
7946
7947
7948
7949
7950
7951
7952
7953
7954
7955
7956
7957
7958
7959
7960
7961
7962
7963
7964
7965
7966
7967
7968
7969
7970
7971
7972
7973
7974
7975
7976
7977
7978 00000D87 3C20
7979 00000D89 726A
7980
7981 00000D8B A880
7982 00000D8D 7505
7983
7984
7985 00000D8F BB[FC0A]
7986 00000D92 EB05
7987
7988
7989
7990
7991
7992 00000D94 247F
7993
7994
7995 00000D96 BB[7E0B]
7996
7997 00000D99 3C20
7998 00000D9B 750D
7999 00000D9D 88D0
8000 00000D9F E81650
8001 00000DA2 E8D2F6
8002
8003 00000DA5 884406
8004 00000DA8 EB24
8005
8006 00000DAA 3C23
8007 00000DAC 7523
8008
8009 00000DAE 31C0
8010
8011
8012
8013
8014
8015
8016
8017
8018
8019 00000DB0 363A16[3613]
8020 00000DB5 7416
8021
8022 00000DB7 363A16[3813]
8023 00000DBC 740F
8024
8025 00000DBE 363A16[3713]
8026 00000DC3 7409
8027
8028 00000DC5 363A16[3913]
8029 00000DCA 7402
8030
8031 00000DCC 40
8032
8033 00000DCD 40
8034
8035
8036
8037
8038
8039
8040 00000DCE E99DF8
8041
8042
8043 00000DD1 89D6
8044 00000DD3 3C21
8045 00000DD5 750D

ext_cntry_inf_5:
    mov     ax,cx          ; buffer (filled) size
    sub     cx,bx          ; copy byte count
    rep movsb
    pop     es
    jmp     ret_ax_to_user_cx ; ax -> user's cx

; -----
; 19/07/2018
;BREAK <$GetExtCntry - return extended country-dependent information>
; -----
;
; Procedure Name : $GetExtCntry
;
; Inputs:
;   if AL >= 20H
;   AL= 20H   capitalize single char, DL= char
;   21H   capitalize string, CX= string length
;   22H   capitalize ASCII string
;   23H   YES/NO check, DL=1st char DH= 2nd char (DBCS)
;   80H bit 0 = use normal upper case table
;   1 = use file upper case table
;   DS:DX points to string
;
; else
;
;   MOV     AH,$GetExtCntry ; DOS 3.3
;   MOV     AL,INFO_ID      ( info type,-1 selects all )
;   MOV     BX,CODE_PAGE    ( -1 = active code page )
;   MOV     DX,COUNTRY_ID   ( -1 = active country )
;   MOV     CX,SIZE         ( amount of data to return )
;   LES     DI,COUNTRY_INFO ( buffer for returned data )
;   INT     21
;
; Function:
;   give users extended country dependent information
;   or capitalize chars
;
; Outputs:
;   No Carry:
;     extended country info is succesfully returned
;
;   Carry:
;     Register AX has the error code.
;     AX=0, NO   for YES/NO CHECK
;     1, YES
; -----
;
; procedure    $GetExtCntry,NEAR    ; DOS 3.3
;
;   ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
;   ; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
;
;   ; MSDOS 6.0
; _$GetExtCntry:
;   CMP     AL,CAP_ONE_CHAR      ; < 20H ?
;   JB      short notcap
;
; capcap:
;   TEST    AL,UPPER_TABLE ; 80h ; which upper case table
;   JNZ     short fileupper      ; file upper case
;
; ;hkn; UCASE_TAB in DOSDATA
;   MOV     BX,UCASE_TAB+2        ; get normal upper case
;   JMP     SHORT capit
;
; fileupper:
;   ; 06/01/2024 (PCDOS 7.1 IBMDOS.COM - DOSCODE:4C57h)
;   ; ((Note: This must be a bugfix, because bit 7 of AX is 1 here!))
;   ; AL >= 80h
;   and     al,7Fh
;
; ;hkn; FILE_UCASE_TAB in DOSDATA
;   MOV     BX,FILE_UCASE_TAB+2 ; get file upper case
;
; capit:
;   CMP     AL,CAP_ONE_CHAR ; 20h ; cap one char ?
;   JNZ     short chkyes        ; no
;   MOV     AL,DL                ; set up AL
;   call    GETLET3              ; upper case it
;   call    Get_user_stack       ; get user stack
;   ;mov     [si+6],al
;   MOV     [SI+user_env.user_DX],AL ; user's DL=AL
;   JMP     SHORT nono           ; done
;
; chkyes:
;   CMP     AL,CHECK_YES_NO      ; 23h ; check YES or NO ?
;   JNZ     short capstring      ; no
;
;   XOR     AX,AX                ; presume NO
;
; ;hkn; NLS_YES, NLS_NO, NLS_yes2, NLS_no2 is defined in msdos.cl3 which is
; ;hkn; included in yesno.asm in the DOSCODE segment.
;
;   ; 29/01/2026 - Retro DOS v5.0 BugFix
;   ; cs: -> ss:
;   ; 06/08/2018 - Retro DOS v3.0
;   ; 13/05/2019 - Retro DOS v4.0
;   ;cmp     dl,'y'
;   CMP     DL,[ss:NLS_YES]      ; is 'Y' ?
;   JZ      short yesyes         ; yes
;   ;cmp     dl,'y'
;   CMP     DL,[ss:NLS_yes2]     ; is 'y' ?
;   JZ      short yesyes         ; yes
;   ;cmp     dl,'n'
;   CMP     DL,[ss:NLS_NO]       ; is 'N'?
;   JZ      short nono           ; no
;   ;cmp     dl,'n'
;   CMP     DL,[ss:NLS_no2]      ; is 'n' ?
;   JZ      short nono           ; no
;
; ;dbcs_char:
;   INC     AX                    ; not YES or NO
;
; yesyes:
;   INC     AX                    ; return 1
;   ; 15/12/2022
;   ; 09/01/2024 - Retro DOS v5.0
;
; nono:
;   jmp     short SYS_RET_OK_jump ;
;   ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;   ; 09/01/2024
;   jmp     SYS_RET_OK           ; done
;
; capstring:
;   MOV     SI,DX                ; si=dx
;   CMP     AL,CAP_STRING ; 21h ; cap string ?
;   JNZ     short capasci       ; no

```

```

8046             ;OR      CX,CX             ; check count 0
8047             ;JZ      short nono         ; yes finished
8048             ; 06/01/2024
8049 0000DD7 E3F5      jcxz      nono
8050
8051 concap:          ;
8052             LODSB                     ; get char
8053             call     GETLET3           ; upper case it
8054             MOV      byte [SI-1],AL    ; store back
8055 ;next99:          ;
8056             LOOP     concap            ; continue
8057             JMP      short nono        ; done
8058 capascii:         ;
8059             CMP      AL,CAP_ASCIIIZ ; 22h ; cap ASCIIIZ string ?
8060             JNZ      short capinval    ; no
8061 concap2:          ;
8062             LODSB                     ; get char
8063             or       al,al             ; end of string ?
8064             JZ       short nono        ; yes
8065             call     GETLET3           ; upper case it
8066             MOV      [SI-1],AL        ; store back
8067             JMP      short concap2     ; continue
8068
8069             ; MSDOS 3.3 (& MSDOS 6.0)
8070
8071 ; offset 1A19h in IBMDOS.COM (MSDOS 3.3), 1987
8072 ; _$GetExtCntry:
8073
8074 notcap:           ;
8075             CMP      CX,5              ; minimum size is 5
8076             jnb      short sizeerror   ;
8077             ; 09/01/2024
8078             jnb      short capinval     ; (size error)
8079
8080 GEC_CONT:         ;
8081 ;hkn; SS is DOSDATA
8082 ;context DS
8083
8084             push     ss
8085             ;pop     es ; ! (Retro DOS v3.0 BUG) !
8086             pop      ds ; 13/05/2019 - Retro DOS v4.0
8087
8088 ;hkn; COUNTRY_CDPG is in DOSDATA
8089             MOV      SI,COUNTRY_CDPG
8090
8091 ; -----
8092             ; 06/01/2024 - Retro DOS v5.0
8093             ; PC DOS 7.1 IBMDOS.COM - DOSCODE:4CC6h
8094             ;;;
8095             or       al,al
8096             jnz      short GETCNTRY
8097             ; AL = 0 (INT 21h, AX=6500h)
8098             ; Set extended country-dependent information
8099             ; (SET GENERAL INTERNATIONALIZATION INFO)
8100
8101             sub      cx,7              ; minimum 8 bytes
8102             jbe      short capinval    ; error_invalid_function
8103             mov      bx,[si+48h]       ; [SI+DOS_CCDPG.ccSysCodePage]
8104             lea      si,[si+66h]       ; SI+DOS_CCDPG.ccCountryInfoLen
8105             mov      ax,[si]
8106             sub      ax,4
8107             cmp      cx,ax
8108             jbe      short set_intern_inf
8109             mov      cx,ax
8110 set_intern_inf:   ;
8111             mov      ax,cx
8112             add      ax,4
8113             mov      [es:di+1], ax    ; info length/size (will be written)
8114             add      si,6              ; DOS_CCDPG.ccDFormat
8115             add      di,7              ; points to date format
8116             call     XCHGP             ; ds:si = user's buffer + 6
8117             ; es:di = country info buffer + 7
8118             jmp      short OK_RETN
8119             ;;;
8120 ; -----
8121             ; 06/01/2024
8122 GETCNTRY:         ;
8123             CMP      DX,-1 ; COUNTRY_ID ; active country ?
8124             JNZ      short GETCDPG     ; no
8125
8126 ;hkn; use DS override to access country_cdp fields
8127             ;mov     dx,[si+63h] ; MSDOS 3.3
8128             ;mov     dx,[si+68h] ; MSDOS 6.0
8129             MOV      DX,[SI+DOS_CCDPG.ccDosCountry]
8130             ; get active country id;smr;use DS
8131
8132 GETCDPG:         ;
8133             CMP      BX,-1 ; CODE_PAGE ; active code page?
8134             JNZ      short CHKAGAIN     ; no, check again
8135
8136 ;hkn; use DS override to access country_cdp fields
8137             ;mov     bx,[si+65h] ; MSDOS 3.3
8138             ;mov     bx,[si+6Ah] ; MSDOS 6.0
8139             MOV      BX,[SI+DOS_CCDPG.ccDosCodePage]
8140             ; get active code page id;smr;Use DS
8141
8142 CHKAGAIN:        ;
8143             cmp      dx,[si+68h] ; MSDOS 6.0
8144             CMP      DX,[SI+DOS_CCDPG.ccDosCountry]
8145             ; same as active country id?;smr;use DS
8146             JNZ      short CHKNLS      ; no
8147             ;mov     bx,[si+6Ah] ; MSDOS 6.0
8148             CMP      BX,[SI+DOS_CCDPG.ccDosCodePage]
8149             ; same as active code pg id?;smr;use DS
8150             JNZ      short CHKNLS      ; no
8151
8152 CHKTYPE:         ;
8153             ;mov     bx,[si+48h]
8154             MOV      BX,[SI+DOS_CCDPG.ccSysCodePage]
8155             ; bx = sys code page id;smr;use DS
8156             ; save CX
8157             PUSH     CX
8158             ;mov     cx,[si+4Ah]
8159             MOV      CX,[SI+DOS_CCDPG.ccNumber_of_entries] ;smr;use DS
8160             ;mov     si,COUNTRY_CDPG+76
8161             MOV      SI,COUNTRY_CDPG+76
8162             ;smr;CDPG in DOSDATA
8163
8164 NXTENTRY:        ;
8165             CMP      AL,[SI]            ; compare info type;smr;use DS
8166             JZ       short FOUNDIT
8167             ADD      SI,5              ; next entry
8168             LOOP     NXTENTRY
8169             POP      CX
8170
8171 capinval:        ;
8172             ;error   error_invalid_function; info type not found
8173             ;mov     al,1
8174             mov     al,error_invalid_function
8175
8176 ;SYS_RET_ERR_jmp:
8177             jmp      SYS_RET_ERR

```

```

8170             ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8171 SYS_RET_ERR_jmp:
8172     00000E5D E918F8     jmp     SYS_RET_ERR
8173
8174 FOUNDIT:
8175     MOVSB             ; move info id byte
8176     POP     CX         ; restore char count
8177     ;cmp     al,1
8178     CMP     AL,SetCountryInfo ; select country info type ?
8179     JZ      short setsize
8180     MOV     CX,4        ; 4 bytes will be moved
8181     MOV     AX,5        ; 5 bytes will be returned in CX
8182
8183 OK_RETN:
8184     REP     MOVSB       ; copy info
8185     MOV     CX,AX       ; CX = actual length returned
8186     MOV     AX,BX       ; return sys code page in ax
8187
8188 GETDONE:
8189     call    Get_User_Stack ; return actual length to user's CX
8190     ;mov     [si+4],cx
8191     MOV     [SI+user_env.user_CX],CX
8192     ;jmp     SYS_RET_OK
8193     ; 15/12/2022
8194     ; 25/06/2019
8195     jmp     SYS_RET_OK_c1c
8196     ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8197     ; 15/12/2022
8198 ;nono_jmp:
8199     ;jmp     short nono
8200 setsize:
8201     SUB     CX,3        ; size after length field
8202     CMP     [SI],CX     ; less than table size ;smr;use ds
8203     JAE     short setsize2 ; no
8204     MOV     CX,[SI]     ; truncate to table size ;smr;use ds
8205
8206 setsize2:
8207     MOV     [ES:DI],CX  ; copy actual length to user's buffer
8208     ;ADD     DI,2       ; update index
8209     ;ADD     SI,2
8210     ; 06/01/2024
8211     inc     di
8212     inc     di
8213     inc     si
8214     inc     si
8215     MOV     AX,CX
8216     ADD     AX,3        ; AX has the actual length
8217     JMP     short OK_RETN ; go move it
8218
8219 CHKNLS:
8220     XOR     AH,AH
8221     ;PUSH    AX         ; save info type
8222     ;POP     BP         ; bp = info type
8223     ; 06/01/2024
8224     mov     bp,ax
8225
8226 ;CallInstall NLSInstall,NLSFUNC,0 ; check if NLSFUNC in memory
8227 mov     ax,1400h
8228 int     2Fh           ; - Multiplex - NLSFUNC.COM - INSTALLATION CHECK
8229             ; Return: AL = 00h not installed, OK to install
8230             ; 01h not installed, not OK
8231             ; FFh installed
8232
8233 CMP     AL,0FFh
8234 JZ      short NLSNXT   ; in memory
8235
8236 sizeerror:
8237     ; 09/01/2024 - Retro DOS v5.0
8238     ;
8239     ;error error_invalid_function
8240     ;mov     al,1
8241     ;mov     al,error_invalid_function
8242     ;jmp     SYS_RET_ERR
8243     ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8244 ;sys_ret_err_jmp2:
8245     jmp     short SYS_RET_ERR_jmp
8246     ; 09/01/2024
8247     jmp     short capinval
8248
8249 NLSNXT:
8250     ;CallInstall GetExtInfo,NLSFUNC,2 ;get extended info
8251     mov     ax,1402h
8252     int     2Fh         ; - Multiplex - NLSFUNC.COM - GET COUNTRY INFO
8253             ; BP = subfunction, BX = code page
8254             ; DX = country code, DS:SI -> internal code page structure
8255             ; ES:DI -> user buffer, CX = size of user buffer
8256             ; Return: AL = status
8257             ; 00h successful
8258             ; else DOS error code
8259
8260 CMP     AL,0           ; success ?
8261 JNZ     short NLSERROR
8262 ;mov     ax,[si+48h] ; 13/05/2019
8263 MOV     AX,[SI+DOS_CCDPG.ccSysCodePage]
8264             ; ax = sys code page id;smr;use ds;
8265             ;BUGBUG;check whether DS is OK after the above calls
8266     JMP     short GETDONE
8267
8268 seterr:
8269     ; 15/12/2022
8270 NLSERROR:
8271     ;jmp     SYS_RET_ERR ; return what is got from NLSFUNC
8272     ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8273     ;jmp     short sys_ret_err_jmp2
8274     ; 15/12/2022
8275     jmp     short SYS_RET_ERR_jmp
8276
8277 ;EndProc $GetExtCntry
8278
8279 ; 13/05/2019 - Retro DOS v4.0
8280 ; DOSCODE:4BD6h (MSDOS 6.21, MSDOS.SYS)
8281
8282 ;BREAK <$GetSetCdPg - get or set global code page>
8283 ;-----
8284 ;** $GetSetCdPg - Get or Set Global Code Page
8285 ;
8286 ; System call format:
8287 ;
8288 ; MOV     AH,GetSetCdPg ; DOS 3.3
8289 ; MOV     AL,n         ; n = 1 : get code page, n = 2 : set code page
8290 ; MOV     BX,CODE_PAGE (set code page only)
8291 ; INT     21
8292 ;
8293 ; ENTRY (al) = n
8294 ;       (bx) = code page
8295 ; EXIT  'c' clear
8296 ;       global code page is set (set global code page)
8297 ;       (BX) = active code page id (get global code page)
8298 ;       (DX) = system code page id (get global code page)
8299 ;       'c' set

```

```

8294 ; (AX) = error code
8295 ;
8296 ;procedure $GetSetCdPg,NEAR ; DOS 3.3
8297 ;
8298 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
8299 ; DOSCODE:4BC9h
8300 ;
8301 ; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
8302 ; DOSCODE:4D73h
8303
8304 _$GetSetCdPg:
8305 ;hkn; SS is DOSDATA
8306 ;context DS
8307
8308 push ss
8309 00000EB1 16 pop ds
8310 00000EB2 1F
8311
8312 ;hkn; COUNTRY_CDPG is in DOSDATA
8313 00000EB3 BE[2A12] MOV SI,COUNTRY_CDPG ; (DOSDATA:122Ah for MSDOS 6.21)
8314
8315 00000EB6 3C01 CMP AL,1 ; get global code page
8316 00000EB8 7512 JNZ short setglpg ; set global code page
8317
8318 ;;mov bx,[si+65h] ; MSDOS 3.3
8319 ;;mov bx,[si+6Ah] ; MSDOS 6.0
8320 00000EBA 8B5C6A MOV BX,[SI+DOS_CCDPG.ccdosCodePage] ; get active code page id;smr;use ds
8321
8322 ;mov dx,[si+48h]
8323 00000EBD 8B5448 MOV DX,[SI+DOS_CCDPG.ccSysCodePage] ; get sys code page id;smr;use ds
8324
8325 00000EC0 E8B4F5 call Get_User_Stack
8326 ;ASSUME DS:NOTHING
8327 ;;mov [si+2],bx
8328 ;MOV [SI+user_env.user_BX],BX ; update returned bx
8329 ; 06/01/2024 (PCDOS 7.1 IBMDOS.COM)
8330 00000EC3 E8F4FD call set_user_bx ; MOV [SI+user_env.user_BX],BX
8331 ;mov [si+6],dx
8332 00000EC6 895406 MOV [SI+user_env.user_DX],DX ; update returned dx
8333
8334 OK_RETURN:
8335 ; 15/12/2022
8336 00000EC9 E9A2F7 ;transfer SYS_RET_OK
8337 jmp SYS_RET_OK
8338 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8339 ;jmp short nono_jmp
8340
8341 ;hkn; ASSUME DS:DOSGROUP
8342 ;ASSUME DS:DOSDATA
8343
8344 00000ECC 3C02 setglpg:
8345 00000ECE 752F CMP AL,2
8346 JNZ short nomem
8347
8348 ;;mov dx,[si+63h] ; MSDOS 3.3
8349 00000ED0 8B5468 ;mov dx,[si+68h] ; MSDOS 6.0
8350 MOV DX,[SI+DOS_CCDPG.ccdosCountry] ;smr;use ds
8351
8352 ;CallInstall NLSInstall,NLSFUNC,0 ; check if NLSFUNC in memory
8353 00000ED3 B80014 mov ax,1400h
8354 00000ED6 CD2F int 2Fh ; - Multiplex - NLSFUNC.COM - INSTALLATION CHECK
8355 ; Return: AL = 00h not installed, OK to install
8356 ; 01h not installed, not OK
8357 ; FFh installed
8358 00000ED8 3CFF CMP AL,0FFh
8359 00000EDA 7523 JNZ short nomem ; not in memory
8360
8361 ;CallInstall SetCodePage,NLSFUNC,1 ;set the code page
8362 00000EDC B80114 mov ax,1401h
8363 00000EDF CD2F int 2Fh ; - Multiplex - NLSFUNC.COM - CHANGE CODE PAGE
8364 ; DS:SI -> internal code page structure
8365 ; BX = new code page, DX = country code???
8366 ; Return: AL = status
8367 ; 00h successful
8368 ; else DOS error code
8369 00000EE1 08C0 ;cmp al,0
8370 00000EE3 74E4 or al,al ; success ?
8371 JZ short OK_RETURN ; yes
8372
8373 00000EE5 3C41 CMP AL,65 ; set device code page failed
8374 00000EE7 75C6 JNZ short seterr
8375 ;MOV AX,65
8376 ; 06/01/2024
8377 00000EE9 98 cbw
8378 00000EEA A3[2403] MOV [EXTERR],AX
8379 ;mov byte [EXTERR_ACTION],6
8380 ;mov byte [EXTERR_CLASS],5
8381 ;mov byte [EXTERR_LOCUS],4
8382 00000EED C606[2603]06 MOV byte [EXTERR_ACTION],errACT_Ignore
8383 00000EF2 C606[2703]05 MOV byte [EXTERR_CLASS],errCLASS_HrdFail
8384 00000EF7 C606[2303]04 MOV byte [EXTERR_LOCUS],errLOC_SerDev
8385 ;transfer From_GetSet
8386 jmp From_GetSet
8387
8388 ; 15/12/2022
8389 ;seterr:
8390 ;;;transfer SYS_RET_ERR
8391 ;;;jmp SYS_RET_ERR
8392 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8393 ;jmp short NLSERROR
8394
8395 nomem:
8396 ;error error_invalid_function; function not defined
8397 ;mov al,1
8398 00000EFF B001 mov al,error_invalid_function
8399 00000F01 EBAC jmp short seterr
8400
8401 ;EndProc $GetSetCdPg
8402
8403 ; 13/05/2019 - Retro DOS v4.0
8404 ; DOSCODE:4C2Bh (MSDOS 6.21, MSDOS.SYS)
8405
8406 ;BREAK <$Get_Drive_Freespace -- Return bytes of free disk space on a drive>
8407 ;-----
8408 ;** $Get_Drive_Freespace - Return amount of drive free space
8409 ;
8410 ; $Get_Drive_Freespace returns the # of free allocation units on a
8411 ; drive.
8412 ;
8413 ; This call returns the same info in the same registers (except for the
8414 ; FAT pointer) as the old FAT pointer calls
8415 ;
8416 ; ENTRY DL = Drive number
8417 ; EXIT AX = Sectors per allocation unit
8418 ; = -1 if bad drive specified

```

```

8418 ; On User Stack
8419 ; BX = Number of free allocation units
8420 ; DX = Total Number of allocation units on disk
8421 ; CX = Sector size
8422
8423 ;procedure $GET_DRIVE_FREESPACE,NEAR
8424
8425 ; 09/01/2024
8426 ; 06/01/2024 - Retro DOS v5.0
8427 ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:4DC4h)
8428
8429 _$GET_DRIVE_FREESPACE:
8430
8431 ;hkn; SS is DOSDATA
8432 ;context DS
8433 0000F03 16 push ss
8434 0000F04 1F pop ds
8435
8436 0000F05 88D0 MOV AL,DL
8437 ;invoke GetThisDrv ; Get drive
8438 0000F07 E87E6C call GETTHISDRV
8439 SET_AX_RET:
8440 0000F0A 7220 JC short BADFDRV
8441
8442 ;invoke DISK_INFO
8443 0000F0C E85024 call DISK_INFO
8444 ; 09/01/2024
8445 ;XCHG DX,BX
8446 ;JC short SET_AX_RET ; User FAILED to I 24
8447 ; 26/06/2024
8448 ; 06/01/2024
8449 ;JC short BADFDRV
8450 ; 26/06/2024
8451 0000F0F 7304 jnc short gdrvfspc_1
8452 0000F11 87D3 xchg dx,bx
8453 0000F13 EB17 jmp short BADFDRV
8454
8455 ; 09/01/2024 - Retro DOS v5.0 (PCDOS 7.1)
8456 gdrvfspc_1:
8457 0000F15 30E4 XOR AH,AH ; Chuck Fat ID byte
8458 ;;;
8459 0000F17 57 push di
8460 0000F18 E80E09 call TestNet
8461 0000F1B 5F pop di
8462 0000F1C 7203 JC short gdrvfspc_2
8463 0000F1E E84225 call modify_cluster_count
8464 ; if hw of total clusters (di) > 0
8465 ; sectors per cluster and cluster counts
8466 ; will be modified (shifted)
8467 ; (but sectors per clust * clust count will be same)
8468 ; /// disk size -calculation- limit = 2 GB ///
8469
8470 0000F21 87D3 gdrvfspc_2:
8471 xchg dx,bx ; bx = free clusters (after xchg)
8472 ;;;
8473 0000F23 E851F5 DoSt:
8474 call Get_User_Stack
8475 ;ASSUME DS:NOTHING
8476 ;mov [si+6],dx
8477 ;mov [si+4],cx
8478 ;mov [si+2],bx
8479 ; 09/01/2024
8480 MOV [SI+user_env.user_DX],DX ; total clusters
8481 ;MOV [SI+user_env.user_CX],CX
8482 ;MOV [SI+user_env.user_BX],BX
8483 ;MOV [SI+user_env.user_AX],AX
8484 ;mov [si],ax
8485 ;return
8486 ;retn
8487 0000F29 E989FD ; 09/01/2024
8488 jmp gdrvfspc_ret ; ax = sectors per cluster (modified)
8489
8490 BADFDRV:
8491 ; MSDOS 3.3
8492 ;mov al,0Fh
8493 ;mov al,error_invalid_drive; Assume error
8494
8495 ; 13/05/2019 - Retro DOS v4.0
8496
8497 ; MSDOS 6.0 & MSDOS 3.3
8498 0000F2C E85EF7 ;invoke FCB_RET_ERR
8499 call FCB_RET_ERR
8500 0000F2F B8FFFF MOV AX,-1
8501 0000F32 EBEF JMP short DoSt
8502
8503 ;EndProc $GET_DRIVE_FREESPACE
8504
8505 ; BREAK <$Get_DMA, $Set_DMA -- Get/Set current DMA address>
8506 ;-----
8507 ** $Get_DMA - Get Disk Transfer Address
8508 ;
8509 ; ENTRY none
8510 ; EXIT ES:BX is current transfer address
8511 ; USES all
8512
8513 ; 09/01/2024
8514 _$GET_DMA:
8515 0000F34 368B1E[2C03] MOV BX,[SS:DMAADD]
8516 0000F39 368B0E[2E03] MOV CX,[SS:DMAADD+2]
8517 0000F3E E836F5 call Get_User_Stack
8518 ;mov [si+2],bx
8519 ;mov [si+10h],cx
8520 ; 09/01/2024
8521 ;MOV [SI+user_env.user_BX],BX
8522 0000F41 894C10 MOV [SI+user_env.user_ES],CX
8523 ;retn
8524 ; 09/01/2024
8525 0000F44 E973FD jmp set_user_bx
8526
8527 ** $Set_DMA - Set Disk Transfer Address
8528 ;-----
8529 ; ENTRY DS:DX is current transfer address
8530 ; EXIT none
8531 ; USES all
8532
8533 _$SET_DMA:
8534 0000F47 368916[2C03] MOV [SS:DMAADD],DX
8535 0000F4C 368C1E[2E03] MOV [SS:DMAADD+2],DS
8536 0000F51 C3 retn
8537
8538 ; BREAK <$Get_Default_Drive, $Set_Default_Drive -- Set/Get default drive>
8539 ;-----
8540 ** $Get_Default_Drive - Get Current Default Drive
8541

```

```

8542 ;-----
8543 ; ENTRY none
8544 ; EXIT (AL) = drive number
8545 ; USES all
8546
8547 _$GET_DEFAULT_DRIVE:
8548 00000F52 36A0[3603] MOV AL,[SS:CURDRV]
8549 00000F56 C3 retn
8550
8551 ;** $Set_Default_Drive - Specify new Default Drive
8552 ;-----
8553 ; ENTRY (DL) = Drive number for new default drive
8554 ; EXIT (AL) = Number of drives, NO ERROR RETURN IF DRIVE NUMBER BAD
8555
8556 _$SET_DEFAULT_DRIVE:
8557 00000F57 88D0 MOV AL,DL
8558 00000F59 FEC0 INC AL ; A=1, B=2...
8559 00000F5B E80E6C call GetVisDrv ; see if visible drive
8560 00000F5E 7204 JC short SETRET ; errors do not set
8561 00000F60 36A2[3603] MOV [SS:CURDRV],AL ; no, set
8562
8563 SETRET:
8564 00000F64 36A0[4700] MOV AL,[SS:CDSCOUNT] ; let user see what the count really is
8565 00000F68 C3 retn
8566
8567 ;BREAK <$Get/Set_Interrupt_Vector - Get/Set interrupt vectors>
8568 ;-----
8569
8570 ;** $Get_Interrupt_Vector - Get Interrupt Vector
8571 ;-----
8572 ; $Get_Interrupt_Vector is the official way for user pgms to get the
8573 ; contents of an interrupt vector.
8574 ;
8575 ; ENTRY (AL) = interrupt number
8576 ; EXIT (ES:BX) = current interrupt vector
8577
8578 _$GET_INTERRUPT_VECTOR:
8579 00000F69 E82E00 CALL RECSET
8580 00000F6C 26C41F LES BX,[ES:BX]
8581 00000F6F E805F5 call Get_User_Stack
8582
8583 set_user_es_bx:
8584 ; 09/01/2024 (PCDOS 7.1 IBMDOS.COM)
8585 ; mov [si+2],bx
8586 ; mov [si+10h],es
8587 00000F72 8C4410 MOV [SI+user_env.user_BX],BX
8588 ; mov [SI+user_env.user_ES],ES
8589 00000F75 E942FD ; retn
8590 jmp set_user_bx
8591
8592 ;** $Set_Interrupt_Vector - Set Interrupt Vector
8593 ;-----
8594 ; $Set_Interrupt_Vector is the official way for user pgms to set the
8595 ; contents of an interrupt vector.
8596 ;
8597 ; M004, M068: Also set A20OFF_COUNT to 1 if EXECA200FF bit has been set
8598 ; and if A200FF_COUNT is non-zero. See under tag M003 in inc\dosym.inc
8599 ; for explanation.
8600 ;
8601 ; ENTRY (AL) = interrupt number
8602 ; (ds:dx) = desired new vector value
8603 ; EXIT none
8604 ; USES all
8605
8606 ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
8607 ; 13/05/2019 - Retro DOS v4.0
8608
8609 _$SET_INTERRUPT_VECTOR:
8610 00000F78 E81F00 CALL RECSET
8611 00000F7B FA CLI ; Watch out!!!! Folks sometimes use
8612 00000F7C 268917 MOV [ES:BX],DX ; this for hardware ints (like timer).
8613 00000F7F 268C5F02 MOV [ES:BX+2],DS
8614 00000F83 FB STI
8615 ; M004, M068 - Start
8616 00000F84 36F606[8600]04 test byte [ss:DOS_FLAG],EXECA200FF ; 4
8617 ; Q: was the previous call an int 21h
8618 ; exec call
8619 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8620 ; jnz short siv_1 ; Y: go set count
8621 ; retn ; N: return
8622 ; 15/12/2022
8623 00000F8A 740D jz short siv_2
8624
8625 siv_1:
8626 00000F8C 36803E[8500]00 cmp byte [ss:A200FF_COUNT],0 ; Q: is count 0
8627 00000F92 7505 jnz short siv_2 ; N: done
8628 ; 20/09/2023
8629 00000F94 36FE06[8500] inc byte [ss:A200FF_COUNT]
8630 ; mov byte [ss:A200FF_COUNT],1 ; Y: set it to 1 to indicate to dos
8631 ; dispatcher to turn A20 Off before
8632 ; returning to user.
8633
8634 siv_2:
8635 ; 07/12/2022
8636 00000F99 C3 retn ; M004, M068 - End
8637
8638 RECSET:
8639 00000F9A 31DB XOR BX,BX
8640 00000F9C 8EC3 MOV ES,BX
8641 00000F9E 88C3 MOV BL,AL
8642 00000FA0 D1E3 SHL BX,1
8643 00000FA2 D1E3 SHL BX,1
8644 00000FA4 C3 retn
8645
8646 ; BREAK <$Char_Oper - hack on paths, switches so that xenix can look like PCDOS>
8647 ;-----
8648
8649 ;** $Char_Oper - Manipulate Switch Character
8650 ;
8651 ; This function was put in to facilitate XENIX path/switch compatibility
8652 ;
8653 ; ENTRY AL = function:
8654 ; 0 - read switch char
8655 ; 1 - set switch char (char in DL)
8656 ; 2 - read device availability
8657 ; Always returns available
8658 ; 3 - set device availability
8659 ; No longer supported (NOP)
8660 ; EXIT (al) = 0xff iff error
8661 ; (al) != 0xff if ok
8662 ; (dl) = character/flag, if "read switch char" subfunction
8663 ; USES AL, DL
8664 ;
8665 ; NOTE This already obsolete function has been deactivated in DOS 5.0
; The character / is always returned for subfunction 0,
; subfunction 2 always returns -1, all other subfunctions are ignored.

```



```

8666
8667 ; 13/05/2019 - Retro DOS v4.0
8668 ; DOSCODE:4CC9h (MSDOS 6.21, MSDOS.SYS)
8669
8670 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
8671 ; DOSCODE:4CBCh (MSDOS 5.0, MSDOS.SYS)
8672
8673 _$CHAR_OPER:
8674 ; MSDOS 6.0
8675 00000FA5 08C0 or al,al ; get switch?
8676 00000FA7 B22F mov dl, '/' ; assume yes
8677 00000FA9 7407 jz short chop_1 ; jump if yes
8678 00000FAB 3C02 cmp al,2 ; check device availability?
8679 00000FAD B2FF mov dl,-1 ; assume yes
8680 00000FAF 7401 jz short chop_1 ; jump if yes
8681 00000FB1 C3 retn ; otherwise just quit
8682
8683 ; subfunctions requiring return of value to user come here. DL holds
8684 ; value to return
8685
8686 chop_1:
8687 00000FB2 E8C2F4 call Get_User_Stack
8688 00000FB5 895406 mov [SI+user_env.user_DX],dx ; store value for user
8689 00000FB8 C3 retn
8690
8691 ; MSDOS 3.3
8692 ; Offset 1B87h in IBMDOS.COM (MSDOS 3.3), 1987
8693 ; push ss
8694 ; pop ds
8695 ; cmp al,1
8696 ; jb short chop_1
8697 ; jz short chop_2
8698 ; cmp al,3
8699 ; jb short chop_3
8700 ; jz short chop_5
8701 ; mov al,0FFh
8702 ; retn
8703 chop_1:
8704 ; mov dl,[chSwitch]
8705 ; jmp short chop_4
8706 chop_2:
8707 ; mov [chSwitch],dl
8708 ; retn
8709 chop_3:
8710 ; mov dl, FFh
8711 chop_4:
8712 ; call Get_User_Stack
8713 ; mov [si+6],dx
8714 chop_5:
8715 ; retn
8716
8717 ;** $GetExtendedError - Return Extended error code
8718 -----
8719 ; This function reads up the extended error info from the static
8720 ; variables where it was stored.
8721 ;
8722 ; ENTRY none
8723 ; EXIT AX = Extended error code (0 means no extended error)
8724 ; BL = recommended action
8725 ; BH = class of error
8726 ; CH = locus of error
8727 ; ES:DI = may be pointer
8728 ; USES ALL
8729
8730 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
8731
8732 _$GetExtendedError:
8733 00000FB9 16 push ss
8734 00000FBA 1F pop ds
8735 00000FBB A1[2403] MOV AX,[EXTErr]
8736 00000FBE C43E[2803] LES DI,[EXTErrPT]
8737 00000FC2 8B1E[2603] MOV BX,[EXTErr_ACTION] ; BL = Action, BH = Class
8738 00000FC6 8A2E[2303] MOV CH,[EXTErr_LOCUS]
8739 00000FCA E8AAF4 call Get_User_Stack
8740 ; mov [si+0Ah],di
8741 00000FCD 897C0A MOV [SI+user_env.user_DI],DI
8742
8743 ; 09/01/2024 (PCDOS 7.1 IBMDOS.COM)
8744 ; mov [si+10h],es
8745 ; MOV [SI+user_env.user_ES],ES
8746 ; mov [si+2],bx
8747 ; MOV [SI+user_env.user_BX],BX
8748 00000FD0 E89FFF call set_user_es_bx
8749
8750 ; mov [si+4],cx
8751 00000FD3 894C04 MOV [SI+user_env.user_CX],CX
8752 jmp_SYS_RET_OK:
8753 ; 15/12/2022
8754 ; jmp SYS_RET_OK
8755 ; 25/06/2019
8756 00000FD6 E998F6 jmp SYS_RET_OK_c1c ; 15/12/2022
8757 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8758 ; jmp_SYS_RET_OK:
8759 ; jmp SYS_RET_OK
8760
8761 ; -----
8762 ; 09/01/2024
8763 %if 0
8764 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8765 ; DOSCODE:4CF3h
8766 ; patch_or_unknown:
8767 ; get_code_page:
8768 push si
8769 mov si, COUNTRY_CDPG
8770 mov ax, [si+DOS_CDPG.ccDosCodePage]
8771 mov ax, [ss:si+6Ah]
8772 pop si
8773 retn
8774 %endif
8775 ; -----
8776
8777 ; 29/04/2019 - Retro DOS v4.0
8778
8779 ;BREAK <ECS_call - Extended Code System support function>
8780 -----
8781 ; Inputs:
8782 ; AL = 0 get lead byte table
8783 ; on return DS:SI has the table location
8784 ;
8785 ; AL = 1 set / reset interim console flag
8786 ; DL = flag (00H or 01H)
8787 ; no return
8788 ;
8789 ; AL = 2 get interim console flag

```

```

8790 ; on return DL = current flag value
8791 ;
8792 ; AL = OTHER then error, and returns with:
8793 ; AX = error_invalid_function
8794 ;
8795 ; NOTE: THIS CALL DOES GUARANTEE THAT REGISTER OTHER THAN
8796 ; SS:SP WILL BE PRESERVED!
8797 ;-----
8798
8799 _$ECS_Call:
8800 0000FD9 08C0 or al,al ; AL = 0 (get table)?
8801 ;jnz short _okok
8802 ; 15/12/2022
8803 0000FDB 7403 jz short get_lbt
8804 ;_okok:
8805 0000FDD E98EF6 jmp SYS_RET_OK
8806 get_lbt:
8807 0000FE0 E894F4 call Get_User_Stack ; *
8808
8809 ;hkn; dbcs_table moved low to dosdata
8810 ;mov word [si+8],DBCS_TAB+2
8811 0000FE3 C74408[020D] mov word [SI+user_env.user_SI],DBCS_TAB+2
8812
8813 0000FE8 06 push es
8814 ;getdseg <es> ; es = DOSDATA
8815 0000FE9 2E8E06[0700] mov es,[cs:DosDSeg]
8816 ;mov [si+14],es
8817 0000FEE 8C440E mov [SI+user_env.user_DS],es
8818 0000FF1 07 pop es
8819
8820 ; 15/12/2022
8821 0000FF2 EBE2 jmp short jmp_SYS_RET_OK ; jmp SYS_RET_OK_clic ; *
8822 ;_okok:
8823 ; 15/12/2022
8824 ;;transfer SYS_RET_OK
8825 ;jmp short jmp_SYS_RET_OK
8826 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8827 ;jmp SYS_RET_OK
8828 ;jmp short jmp_SYS_RET_OK
8829
8830 ;-----
8831 ; 10/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM - DOSCODE:4EB4h)
8832 ;-----
8833
8834 ; INT 21h, AH=71h - LONG FILENAME FUNCTIONS
8835 ; INT 21h, AH=72h - LFN FindClose
8836 _$LONGNAME:
8837 0000FF4 30C0 xor al,al ; longname functions are not supported
8838 lfn_error:
8839 0000FF6 E87EF4 call Get_User_Stack
8840 0000FF9 834C1601 or word [si+16h],1 ; [SI+user_env.user_F],f_Carry
8841 0000FFD F9 stc
8842 0000FFE 8904 mov [si],ax ; [SI+user_env.user_ax]
8843 0001000 C3 retn
8844 ;-----
8845
8846 ; FAT32 - EXTENDED FUNCTIONS
8847 ; INT 21h, AH=73h
8848 _$FAT32EXT:
8849 00001001 3C05 cmp al,5 ; INT 21h AX = 7305h
8850 00001003 7609 jbe short valid_fat32_ext_function
8851 00001005 B001 mov al,1 ; error_invalid_function
8852 fat32_ext_func_err:
8853 00001007 E96EF6 jmp SYS_RET_ERR
8854
8855 function_5_invalid_cx:
8856 0000100A B057 mov al,57h ; error_invalid_parameter
8857 fat32_ext_func_err_j:
8858 0000100C EBF9 jmp short fat32_ext_func_err
8859
8860 valid_fat32_ext_function:
8861 0000100E 7524 jnz short not_function_5
8862 00001010 83F9FF cmp cx,0FFFFh ; Function 5 - FAT32 - EXTENDED ABSOLUTE DISK READ/WRITE
8863 00001013 75F5 jne short function_5_invalid_cx
8864 00001015 F7C6FE9F test si,9FEEh ; read/write mode flags
8865 00001019 75EF jnz short function_5_invalid_cx
8866 0000101B 88D0 mov al,dl ; drive number, 1 = A
8867 0000101D FEC8 dec al
8868 0000101F B401 mov ah,1
8869 00001021 F7C60100 test si,1
8870 00001025 7405 jz short function_5_read
8871 00001027 E8BAF5 call FAT32_ABSDWRT ; INT 21h AX = 7305h (SI bit 0 = 1)
8872 0000102A EB03 jmp short fat32_absdrw_ret
8873
8874 function_5_read:
8875 0000102C E802F5 call FAT32_ABSDRD ; INT 21h AX = 7305h (SI bit 0 = 0)
8876 fat32_absdrw_ret:
8877 0000102F 72C5 jc short lfn_error
8878 00001031 E93AF6 jmp SYS_RET_OK
8879
8880 not_function_5:
8881 00001034 3C03 cmp al,3 ; Function 3 - FAT32 - GET EXTENDED FREE SPACE ON DRIVE
8882 00001036 7475 je short function_73_3
8883 00001038 3C02 cmp al,2
8884 0000103A 7203 jb short chk_drive_lock_flush
8885 0000103C E90503 jmp _$GET_DPB ; Function 2 - FAT32 - "Get_ExtDPB" - GET EXTENDED DPB
8886 ; Function 4 - FAT32 - Set DPB TO USE FOR FORMATTING
8887
8888 chk_drive_lock_flush:
8889 0000103F 80FA1A cmp dl,26 ; MSDOS 7 - DRIVE LOCKING AND FLUSHING
8890 00001042 7604 jbe short drv_lock_flush_1
8891 00001044 B00F mov al,0Fh ; invalid drive number
8892 drv_lock_flush_err:
8893 ;jmp short fat32_ext_func_err_j ; ax = error code
8894 ; 10/01/2024
8895 00001046 EBBF jmp short fat32_ext_func_err
8896
8897 drv_lock_flush_1:
8898 00001048 FECA dec dl
8899 0000104A 7905 jns short drv_lock_flush_2
8900 0000104C 368A16[3603] mov dl,[ss:CURDRV] ; 0 = default/current drive)
8901 drv_lock_flush_2:
8902 00001051 B600 mov dh,0
8903 00001053 89D3 mov bx,dx
8904 00001055 80F901 cmp cl,1 ; which flag to get or set
8905 00001058 7604 jbe short drv_lock_flush_3
8906 0000105A B001 mov al,1 ; error_invalid_function
8907 ;jmp short drv_lock_flush_err
8908 ; 10/01/2024
8909 0000105C EBA9 jmp short fat32_ext_func_err
8910
8911 drv_lock_flush_3:
8912 0000105E 368AA7[0813] mov ah,[ss:drive_flags+bx]
8913 00001063 08C9 or cl,cl

```



```

8914 00001065 7422      jz      short get_set_indctd_flag ; use bit 1 and bit 2
8915 00001067 08C0      or       al,al          ; get drive's dirty-buffers flag
8916 00001069 7419      jz      short get_dirty_buf_flag ; use bit 3
8917 0000106B 80E4F7    and     ah,0F7h        ; clear bit 3
8918 0000106E 80E508    and     ch,8          ; isolate bit 3 of the new flag value
8919 00001071 08EC      or       ah,ch        ; set AH bit 3 according to CH bit 3
8920 00001073 3688A7[0813] mov     [ss:drive_flags+bx],ah ; set or reset dirty buffer flag
8921 00001078 F6C408    test    ah,8          ;
8922 0000107B 7505      jnz     short set_dirty_flag_ok      ; bit 3 is set/1
8923 0000107D B0FF      mov     al,0FFh
8924 0000107F E8115A    call    FLUSHBUF
8925                          set_dirty_flag_ok:
8926 00001082 EB26      jmp     short jmp_to_SYS_RET_OK
8927
8928                          get_dirty_buf_flag:
8929 00001084 80E408    and     ah,8          ; isolate dirty buffers flag
8930 00001087 EB19      jmp     short mov_flag_cl_to_al      ; AH = new flag and 08h (bit 3 used)
8931
8932                          get_set_indctd_flag:
8933 00001089 08C0      or       al,al
8934 0000108B 7412      jz      short get_indicated_flag
8935 0000108D 80E4F9    and     ah,0F9h        ; clear bit 1 and bit 2
8936 00001090 F6C502    test    ch,2          ; new value for indicated flag
8937 00001093 7403      jz      short reset_indctd_flags ; bit 1 is zero
8938 00001095 80CC06    or       ah,6          ; set bit 1 and bit 2
8939                          reset_indctd_flags:
8940 00001098 3688A7[0813] mov     [ss:drive_flags+bx],ah
8941 0000109D EB0B      jmp     short jmp_to_SYS_RET_OK
8942
8943                          get_indicated_flag:
8944 0000109F 80E406    and     ah,6          ; AH = new flag and 06h (bits 1 and 2 used)
8945                          mov_flag_cl_to_al:
8946 000010A2 88C8      mov     al,cl          ; CODE XREF: DOSCODE:4F47^j
8947                          ret_ax_to_user_cx: ; ax -> user's cx
8948                          call    Get_User_Stack
8949 000010A7 894404    mov     [si+4],ax      ; [SI+user_env.user_cx] ; requested flag
8950                          jmp_to_SYS_RET_OK:
8951 000010AA E9C1F5    jmp     SYS_RET_OK
8952
8953                          function_73_3:
8954 000010AD 89D6      mov     si,dx          ; FAT32 - GET EXTENDED FREE SPACE ON DRIVE
8955                          ; AX = 7303h
8956                          ; DS:DX -> ASCIZ string for drive ("C:\" or "\\SERVER\Share")
8957                          ; ES:DI -> buffer for extended free space structure
8958                          ; CX = length of buffer for extended free space
8959 000010AF E8D26E    call    DriveFromText
8960 000010B2 92        xchg    ax,dx
8961 000010B3 AD        lodsw
8962 000010B4 FECA      dec     dl
8963 000010B6 80FA1A    cmp     dl,26
8964 000010B9 7323      jnb     short func_73_3_err2
8965 000010BB 08E4      or      ah,ah
8966 000010BD 751F      jnz     short func_73_3_err2
8967 000010BF E8244D    call    PATHCHRCMP
8968 000010C2 751A      jnz     short func_73_3_err2
8969 000010C4 FEC2      inc     dl
8970 000010C6 83F92C    cmp     cx,44          ; buffer (Structure) size must be 44
8971 000010C9 721C      jb      short func_73_3_err4
8972 000010CB 26837D0200 cmp     word [es:di+2],0 ; buffer structure version (must be 0)
8973 000010D0 7511      jnz     short func_73_3_err3
8974 000010D2 16        push    ss
8975 000010D3 1F        pop     ds
8976 000010D4 88D0      mov     al,dl
8977 000010D6 E8AF6A    call    GETTHISDRV
8978 000010D9 7310      jnc     short fill_efs_struct_b
8979                          func_73_3_err1:
8980 000010DB E8AFF5    call    FCB_RET_ERR
8981                          func_73_3_err2:
8982 000010DE B00F      mov     al,0Fh        ; error_invalid_drive
8983                          jmp_to_SYS_RET_ERR:
8984 000010E0 E995F5    jmp     SYS_RET_ERR
8985
8986                          func_73_3_err3:
8987 000010E3 B057      mov     al,57h        ; error_invalid_parameter
8988                          jmp_to_jmp_SYS_RET_ERR:
8989 000010E5 EBF9      jmp     short jmp_to_SYS_RET_ERR
8990
8991                          func_73_3_err4:
8992 000010E7 B018      mov     al,18h        ; error_bad_length
8993 000010E9 EBFA      jmp     short jmp_to_jmp_SYS_RET_ERR
8994
8995                          fill_efs_struct_b:
8996 000010EB E87122    call    DISK_INFO
8997 000010EE 72EB      jc      short func_73_3_err1
8998 000010F0 30E4      xor     ah,ah
8999 000010F2 56        push    si             ; si:dx = free cluster count
9000 000010F3 57        push    di             ; di:bx = number of clusters
9001 000010F4 E880F3    call    Get_User_Stack
9002 000010F7 8E4410    mov     es,[si+10h]    ; user's buffer segment (in ES)
9003 000010FA 8B7C0A    mov     di,[si+0Ah]    ; user's buffer offset/address (in DI)
9004                          ; 10/01/2024
9005 000010FD 06        push    es
9006 000010FE 1F        pop     ds ; (*)
9007                          ;
9008                          ;mov     [es:di+10h],bx ; total number of clusters on the drive
9009                          ;mov     [es:di+20h],bx ; total allocation units, without adjustment for compression
9010 000010FF 895D10    mov     [di+10h],bx
9011 00001102 895D20    mov     [di+20h],bx
9012 00001105 5B        pop     bx
9013                          ;mov     [es:di+12h],bx ; total number of clusters on the drive, hw
9014                          ;mov     [es:di+22h],bx ; total allocation units, hw
9015                          ;mov     [es:di+0Ch],dx ; number of available clusters
9016                          ;mov     [es:di+1Ch],dx ; number of available allocation units, without adjustment
9017 00001106 895D12    mov     [di+12h],bx
9018 00001109 895D22    mov     [di+22h],bx
9019 0000110C 89550C    mov     [di+0Ch],dx
9020 0000110F 89551C    mov     [di+1Ch],dx
9021 00001112 5A        pop     dx
9022                          ;mov     [es:di+0Eh],dx ; number of available clusters, hw
9023                          ;mov     [es:di+1Eh],dx ; number of available allocation units, hw
9024                          ;mov     [es:di+8],cx ; bytes per sector
9025                          ;mov     [es:di+4],ax ; sectors per cluster (with adjustment for compression)
9026 00001113 89550E    mov     [di+0Eh],dx
9027 00001116 89551E    mov     [di+1Eh],dx
9028 00001119 894D08    mov     [di+8],cx
9029 0000111C 894504    mov     [di+4],ax
9030 0000111F 89C1      mov     cx,ax
9031 00001121 F7E3      mul     bx             ; 32 bit multiplication
9032 00001123 720B      jc      short dsk_cap_calc_overf ; disk capacity calculation overflow error
9033 00001125 89C3      mov     bx,ax
9034                          ;mov     ax,[es:di+20h] ; total allocation units, 1w
9035 00001127 8B4520    mov     ax,[di+20h]
9036 0000112A F7E1      mul     cx
9037 0000112C 01DA      add     dx,bx

```

```

9038 0000112E 7305      jnb     short dsk_cap_calc_ok
9039      dsk_cap_calc_overf:
9040      mov     ax,0FFFFh      ; set to 0FFFFFFFFh
9041      mov     dx,ax
9042      dsk_cap_calc_ok:
9043      ; 10/01/2024
9044      ; ds = es (*)
9045      ;mov     [es:di+1Ah],dx
9046      ;mov     [es:di+18h],ax      ; total number of physical sectors on the drive,
9047      ; without adjustment for compression
9048      mov     [di+1Ah],dx
9049      mov     [di+18h],ax
9050      mov     ax,cx      ; 32 bit multiplication
9051      ;mul     word [es:di+0Eh]      ; number of available clusters, hw
9052      mul     word [di+0Eh]
9053      jc      short dsk_free_calc_overf
9054      mov     bx,ax
9055      ;mov     ax,[es:di+0Ch]      ; number of available clusters, lw
9056      mov     ax,[di+0Ch]
9057      mul     cx
9058      add     dx,bx
9059      jnc     short dsk_free_calc_ok
9060      dsk_free_calc_overf:
9061      mov     ax,0FFFFh
9062      mov     dx,ax
9063      dsk_free_calc_ok:
9064      ; 10/01/2024
9065      ;mov     [es:di+16h],dx      ; hw
9066      ;mov     [es:di+14h],ax      ; number of physical sectors available on the drive,
9067      ; without adjustment for compression
9068      mov     [di+16h],dx
9069      mov     [di+14h],ax
9070      xor     ax,ax      ; 0
9071      ;mov     [es:di+0Ah],ax      ; number of bytes per sector, high word = 0
9072      ;mov     [es:di+6],ax      ; number of sectors per cluster, high word = 0
9073      mov     [di+0Ah],ax
9074      mov     [di+6],ax
9075
9076      ;mov     [es:di+24h],ax      ; reserved, 8 bytes zero
9077      ;mov     [es:di+26h],ax
9078      ;mov     [es:di+28h],ax
9079      ;mov     [es:di+2Ah],ax
9080      add     di,24h ; 36 ; 10/01/2024
9081      stosw
9082      stosw
9083      stosw
9084      stosw
9085      mov     ax,44
9086      sub     di,ax ; 10/01/2024
9087      ;mov     [es:di],ax      ; size of returned structure = 44
9088      ; 10/01/2024
9089      ;mov     [di],ax
9090      stosw
9091      jmp     SYS_RET_OK
9092
9093      ; -----
9094
9095      ; 12/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM/MSDOS.SYS)
9096      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:504Ah
9097
9098      Set_DPBforFormat:  ; INT 21h, AX = 7304h (continues from _$GET_DPB)
9099      ; 12/01/2024      ; (Ref: Ralf Brown's Interrupt List)
9100      mov     ax,18h
9101      cmp     [si+4],ax
9102      ;cmp     word ptr [si+4],24      ; [SI+user_env.user_CX]
9103      ; size of buffer (must be at least 18h)
9104      ;jnb     short setdpbf_2
9105      ;mov     al,18h      ; error_bad_length
9106      ;setdpbf_1:
9107      ;jmp     SYS_RET_ERR
9108      jb      short setdpbf_1 ; al = 18h
9109      setdpbf_2:
9110      push     es      ; ES:BP = Drive parameter block
9111      push     bp
9112      mov     es,[si+10h]      ; [SI+user_env.user_ES]
9113      mov     di,[si+0Ah]      ; [SI+user_env.user_DI]
9114      pop     si
9115      pop     ds
9116
9117      mov     ax,[es:di+4]      ; (call) function number
9118      cmp     word [es:di+2],0      ; structure version (must be 0)
9119      jnz     short setdpbf_3
9120      cmp     word [es:di+6],0      ; (must be 0)
9121      jnz     short setdpbf_3
9122      cmp     ax,4      ; (max) 5 functions (0 to 4)
9123      jbe     short setdpbf_4
9124      setdpbf_3:
9125      mov     al,57h      ; error_invalid_parameter
9126      ;jmp     short setdpbf_1
9127      setdpbf_1:
9128      jmp     SYS_RET_ERR
9129
9130      setdpbf_4:
9131      mov     word [es:di],18h      ; (call) size
9132      or      al,al
9133      jz      short setdpbf_5      ; invalidate DPB counts
9134      jmp     setdpbf_18
9135
9136      setdpbf_5:
9137      xor     dx,dx      ; 0
9138      mov     bx,[si+0Dh]      ; DPB.MAX_CLUSTER
9139      cmp     [si+0Fh],dx ; 0
9140      ;cmp     word [si+0Fh],0      ; DPB.FAT_SIZE
9141      ;jnz     short setdpbf_6      ; not FAT32
9142      mov     dx,[si+2Fh]
9143      mov     bx,[si+2Dh]      ; DPB.LAST_CLUSTER
9144      setdpbf_6:
9145      mov     ax,[es:di+0Ah]
9146      mov     cx,[es:di+8]      ; new DPB free count
9147      ; (00000000h=no change, FFFFFFFFh=unknown)
9148      or      ax,ax
9149      jnz     short setdpbf_7
9150      jcxz     setdpbf_11
9151      setdpbf_7:
9152      cmp     ax,0FFFFh
9153      jne     short setdpbf_8
9154      cmp     cx,0FFFFh
9155      je      short setdpbf_10      ; (set as UNKNOWN/INITIAL)
9156      setdpbf_8:
9157      cmp     ax,dx      ; must be < DPB.LAST_CLUSTER
9158      jne     short setdpbf_9
9159      cmp     cx,bx
9160      setdpbf_9:
9161      jnb     short setdpbf_3

```

```

9162
9163 000011DA 804C1801
9164 000011DE 894C1F
9165 000011E1 837C0F00
9166 000011E5 7503
9167 000011E7 894421
9168
9169 000011EA 268B450E
9170 000011EE 268B4D0C
9171
9172 000011F2 09C0
9173 000011F4 7502
9174 000011F6 E32E
9175
9176 000011F8 83F8FF
9177 000011FB 7505
9178 000011FD 83F9FF
9179 00001200 7411
9180
9181 00001202 21C0
9182
9183 00001204 7505
9184 00001206 83F902
9185
9186 00001209 728E
9187
9188 0000120B 39D0
9189 0000120D 7502
9190 0000120F 39D9
9191
9192 00001211 7786
9193
9194 00001213 804C1801
9195 00001217 894C1D
9196 0000121A 837C0F00
9197 0000121E 7506
9198 00001220 89443B
9199 00001223 894C39
9200
9201 00001226 B80473
9202 00001229 E942F4
9203
9204
9205
9206 0000122C 48
9207 0000122D 7514
9208 0000122F 1E
9209 00001230 56
9210 00001231 26C57508
9211 00001235 5D
9212 00001236 07
9213 00001237 B95845
9214 0000123A BA5241
9215 0000123D E82502
9216 00001240 E92BF4
9217
9218
9219
9220 00001243 48
9221 00001244 7507
9222
9223
9224 00001246 804C1880
9225
9226 0000124A E921F4
9227
9228
9229 0000124D 837C0F00
9230 00001251 7405
9231 00001253 B00F
9232
9233
9234 00001255 E920F4
9235
9236
9237
9238 00001258 48
9239 00001259 7454
9240 0000125B 8B4435
9241
9242 0000125E 2689450C
9243 00001262 8B4437
9244 00001265 2689450E
9245 00001269 268B4D0A
9246 0000126D 268B4508
9247 00001271 83F8FF
9248 00001274 7505
9249 00001276 83F9FF
9250 00001279 74CF
9251
9252 0000127B 21C9
9253
9254
9255 0000127D 7509
9256 0000127F 83F802
9257
9258 00001282 7304
9259
9260
9261
9262 00001284 B057
9263 00001286 EB0D
9264
9265
9266 00001288 3B4C2F
9267 0000128B 7503
9268 0000128D 3B442D
9269
9270 00001290 77F2
9271 00001292 894C37
9272 00001295 894435
9273 00001298 804C1802
9274 0000129C 1E
9275 0000129D 56
9276 0000129E 5D
9277 0000129F 07
9278 000012A0 E83F06
9279 000012A3 E8C221
9280 000012A6 E86606
9281 000012A9 739F
9282 000012AB B01F
9283
9284 000012AD EBA6
9285

setdpbf_10:
    or     byte [si+18h],1          ; DPB.FIRST_ACCESS (bit 0 = 1)
    mov     [si+1Fh],cx             ; DPB.FREE_CNT
    cmp     word [si+0Fh],0         ; DP.FAT_SIZE
    jnz     short setdpbf_11       ; FAT12 or FAT16
    mov     [si+21h],ax            ; DPB.FREE_CNT_HW
setdpbf_11:
    mov     ax,[es:di+0Eh]
    mov     cx,[es:di+0Ch]         ; new DPB next-free
                                   ; (00000000h=no change, FFFFFFFFh=unknown)
    or     ax,ax
    jnz     short setdpbf_12
    jcxz    setdpbf_17
setdpbf_12:
    cmp     ax,0FFFFh
    jne     short setdpbf_13
    cmp     cx,0FFFFh
    je      short setdpbf_16       ; (set as UNKNOWN/INITIAL)
setdpbf_13:
    and     ax,ax
    ;cmp    ax,0                   ; must be >= 2
    jnz     short setdpbf_14
    cmp     cx,2
;setdpbf_14:
    jb      short setdpbf_3
setdpbf_14:
    cmp     ax,dx                   ; must be < DPB.LAST_CLUSTER
    jne     short setdpbf_15
    cmp     cx,bx
setdpbf_15:
    ja      short setdpbf_3
setdpbf_16:
    or     byte [si+18h],1          ; DPB.FIRST_ACCESS (bit 0 = 1)
    mov     [si+1Dh],cx            ; DPB.NEXT_FREE
    cmp     word [si+0Fh],0         ; DPB.FAT_SIZE
    jnz     short setdpbf_17       ; FAT16 or FAT12
    mov     [si+3Bh],ax
    mov     [si+39h],cx            ; DPB.FAT32_NXTFREE
setdpbf_17:
    mov     ax,7304h               ; done (successful)
    jmp     SYS_RET_OK
setdpbf_18:
    ;dec    al
    dec     ax
    jnz     short setdpbf_19
    push    ds                     ; rebuild DPB from BPB
    push    si
    lds     si,[es:di+8]           ; BIOS Parameter Block
    pop     bp
    pop     es
    mov     cx,4558h               ; 'XE' (NASM syntax)
    mov     dx,4152h               ; 'RA' (NASM syntax)
    call    _$SETDPB
    jmp     SYS_RET_OK
setdpbf_19:
    ;dec    al
    dec     ax
    jnz     short setdpbf_20
    ; force media change
    ; (next access to drive rebuild DPB)
    or     byte [si+18h],80h       ; DPB.FIRST_ACCESS (bit 7 = 1)
; 12/01/2024
setdpbf_29:
    jmp     SYS_RET_OK
setdpbf_20:
    cmp     word [si+0Fh],0         ; DPB.FAT_SIZE
    jz      short setdpbf_22       ; FAT32
    mov     al,0Fh                 ; error_invalid_drive
    ; (function 3 or 4 are only for drives with FAT32 fs)
setdpbf_21:
    jmp     SYS_RET_ERR
setdpbf_22:
    ;dec    al
    dec     ax
    jz      short setdpbf_30       ; get/set active FAT number and mirroring
    mov     ax,[si+35h]            ; DPB.ROOT_CLUSTER
    ; get/set root directory cluster number
    mov     [es:di+0Ch],ax         ; (ret) previous root directory cluster number
    mov     ax,[si+37h]
    mov     [es:di+0Eh],ax
    mov     cx,[es:di+0Ah]
    mov     ax,[es:di+8]           ; (call) new root directory cluster number
    cmp     ax,0FFFFh             ; -1 --> return only previous root dir cluster number
    jne     short setdpbf_23
    cmp     cx,0FFFFh
    je      short setdpbf_29
setdpbf_23:
    and     cx,cx
    ;cmp    cx,0                   ; cluster number must be >= 2
    ;jnz    short setdpbf_24
    jnz     short setdpbf_26
    cmp     ax,2
setdpbf_24:
    jnb     short setdpbf_26
setdpbf_25:
    ;jmp     setdpbf_3              ; error (invalid parameter)
    ; 12/01/2024
    mov     al,57h                 ; error_invalid_parameter
    jmp     short setdpbf_21
setdpbf_26:
    cmp     cx,[si+2Fh]             ; must be <= DPB.LAST_CLUSTER
    jne     short setdpbf_27
    cmp     ax,[si+2Dh]
setdpbf_27:
    ja      short setdpbf_25       ; error
    mov     [si+37h],cx            ; DPB.ROOT_CLUSTER
    mov     [si+35h],ax
    or     byte [si+18h],2         ; DPB.FIRST_ACCESS (bit 1 = 1)
    push    ds
    push    si
    pop     bp
    pop     es
    call    ECritDisk
    call    update_fat32_fsinfo
    call    LCritDisk
    jnc     short setdpbf_29
    mov     al,1Fh                 ; error_gen_failure
setdpbf_28:
    jmp     short setdpbf_21

```

```

9286         ; 12/01/2024
9287 ;setdpbf_29:
9288         ; jmp     SYS_RET_OK
9289
9290 setdpbf_30:
9291         ; 12/01/2024
9292         ; ax = 0
9293 000012AF 8164238F00 and     word [si+23h],8Fh    ; DPB.EXT_FLAGS (clear bit 4-6)
9294 000012B4 8B4C23     mov     cx,[si+23h]
9295 000012B7 26894D0C     mov     [es:di+0Ch],cx    ; (ret) previous active FAT/mirroring state
9296 000012BB 2689450E     mov     [es:di+0Eh],ax ; 0
9297         ; mov     word [es:di+0Eh],0    ; put zero to Set_DPBforFormat struct offset 14
9298 000012BF 268B5508     mov     dx,[es:di+8]    ; (call) new active FAT/mirroring state,
9299         ; or FFFFFFFFh to get
9300 000012C3 83FAFF     cmp     dx,0FFFFh
9301         ; jnz     short setdpbf_31
9302         ; jmp     SYS_RET_OK
9303         ; 12/01/2024
9304 000012C6 7482     je      short setdpbf_29
9305
9306 setdpbf_31:
9307 000012C8 F7C270FF     test    dx,0FF70h
9308         ; jz      short setdpbf_33    ; bit 4-6 of DPB.EXT_FLAGS must be 0
9309         ; mov     al,57h              ; error_invalid_parameter
9310         ; 12/01/2024
9311 000012CC 75B6     jnz     short setdpbf_25
9312 setdpbf_32:
9313         ; jmp     short setdpbf_28
9314         ; 12/01/2024
9315         ; jmp     short setdpbf_21
9316
9317 setdpbf_33:
9318 000012CE B001     mov     al,1    ; error_invalid_function
9319         ;                               ; (modification is not allowed)
9320         ; jmp     short setdpbf_32
9321         ; 12/01/2024
9322 000012D0 EB83     jmp     short setdpbf_21
9323
9324 ; -----
9325 ;=====
9326 ; PARSE.ASM, MSDOS 6.0, 1991
9327 ;=====
9328 ; 19/07/2018 - Retro DOS v3.0
9329 ; 15/05/2019 - Retro DOS v4.0
9330
9331 ; System calls for parsing command lines
9332 ;
9333 ; $PARSE_FILE_DESCRIPTOR
9334 ;
9335 ; Modification history:
9336 ;
9337 ; Created: ARR 30 March 1983
9338 ; EE PathParse 10 Sept 1983
9339 ;
9340 ;
9341 ;BREAK <$Parse_File_Descriptor -- Parse an arbitrary string into an FCB>
9342 ;-----
9343 ; Inputs:
9344 ; DS:SI Points to a command line
9345 ; ES:DI Points to an empty FCB
9346 ; Bit 0 of AL = 1 At most one leading separator scanned off
9347 ;               = 0 Parse stops if separator encountered
9348 ; Bit 1 of AL = 1 If drive field blank in command line - leave FCB
9349 ;               = 0 " " - put 0 in FCB
9350 ; Bit 2 of AL = 1 If filename field blank - leave FCB
9351 ;               = 0 " " - put blanks in FCB
9352 ; Bit 3 of AL = 1 If extension field blank - leave FCB
9353 ;               = 0 " " - put blanks in FCB
9354 ; Function:
9355 ; Parse command line into FCB
9356 ; Returns:
9357 ; AL = 1 if '*' or '?' in filename or extension, 0 otherwise
9358 ; DS:SI points to first character after filename
9359 ;-----
9360
9361 ; 13/01/2024
9362 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:51BFh
9363 ; MSDOS 6.22 MSDOS.SYS - DOSCODE:4D22h
9364
9365 _$PARSE_FILE_DESCRIPTOR:
9366     call    MAKEFCB
9367 000012D2 E88B49     push    SI
9368 000012D5 56         push    SI
9369 000012D6 E89EF1     call    Get_User_Stack
9370         ; pop     word [si+8]
9371 000012D9 8F4408     pop     word [SI+user_env.user_SI]
9372 000012DC C3         retn
9373
9374 ; -----
9375 ; 13/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM - DOSCODE:51CAh)
9376 ; -----
9377
9378 set_exerr_locus_unk:
9379     push    ax
9380 000012DE B001     mov     al,1    ; errLOC_Unk
9381
9382 set_exerr_locus:
9383     mov     [ss:EXERR_LOCUS],al
9384     pop     ax
9385     retn
9386
9387 set_exerr_locus_disk:
9388     push    ax
9389 000012E6 50         mov     al,2    ; errLOC_Disk
9390     jmp     short set_exerr_locus
9391
9392 set_exerr_locus_ser:
9393     push    ax
9394 000012EC B004     mov     al,4    ; errLOC_SerDev
9395     jmp     short set_exerr_locus
9396
9397 set_exerr_locus_mem:
9398     push    ax
9399 000012F0 50         mov     al,5    ; errLOC_Mem
9400     jmp     short set_exerr_locus
9401
9402 ; -----
9403 ;=====
9404 ; MISC.ASM, MSDOS 6.0, 1991
9405 ;=====
9406 ; 19/07/2018 - Retro DOS v3.0
9407 ; 29/04/2019 - Retro DOS v4.0

```

```

9410 ;ENTRYPOINTSEG EQU 0CH
9411 ;MAXDIF EQU 0FFFh
9412 ;SAVEXIT EQU 10
9413 ;WRAPOFFSET EQU 0FEF0h
9414
9415 ;
9416 ;-----
9417 ;
9418 ;** $SLEAZEFUNC - Get a Pointer to the Media Byte
9419 ;
9420 ; Return Stuff sort of like old get fat call
9421 ;
9422 ; ENTRY none
9423 ; EXIT DS:BX = Points to FAT ID byte (IBM only)
9424 ; GOD help anyone who tries to do ANYTHING except
9425 ; READ this ONE byte.
9426 ; DX = Total Number of allocation units on disk
9427 ; CX = Sector size
9428 ; AL = Sectors per allocation unit
9429 ; = -1 if bad drive specified
9430 ;
9431 ; USES all
9432 ;
9433 ;** $SLEAZEFUNC DL - Get a Pointer to the Media Byte
9434 ;
9435 ; Identical to $SLEAZEFUNC except (dl) = drive
9436 ;
9437 ; ENTRY (dl) = drive (0=default, 1=A, 2=B, etc.)
9438 ; EXIT DS:BX = Points to FAT ID byte (IBM only)
9439 ; GOD help anyone who tries to do ANYTHING except
9440 ; READ this ONE byte.
9441 ; DX = Total Number of allocation units on disk
9442 ; CX = Sector size
9443 ; AL = Sectors per allocation unit
9444 ; = -1 if bad drive specified
9445 ;
9446 ; USES all
9447 ;
9448 ;-----
9449 ; 10/01/2024 - Retro DOS v5.0
9450 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:51E2h
9451
9452 _$SLEAZEFUNC:
9453 ; 15/05/2019 - Retro DOS v4.0
9454 MOV DL,0
9455 _$SLEAZEFUNC DL:
9456 push ss
9457 pop ds
9458 MOV AL,DL
9459 call GETTHISDRV ; Get CDS structure
9460 SET_AL_RET:
9461 ; MSDOS 3.3
9462 ; ;mov al, 0Fh
9463 ; MOV AL,error_invalid_drive; Assume error ;AC000;
9464 ;
9465 ; MSDOS 6.0 & MSDOS 3.3
9466 JC short BADSLDRIVE
9467
9468 call DISK_INFO
9469 ; JC short SET_AL_RET ; User FAILED to I 24
9470 JC short BADSLDRIVE
9471 MOV [FATBYTE],AH ; FAT (MEDIA) ID byte
9472
9473 ; 10/01/2024 - Retro DOS v5.0 (PC DOS 7.1)
9474 ; ;
9475 xor ah, ah ; AH = 0
9476 ; AL = sectors per cluster
9477 ; di:bx = number of clusters
9478 push di
9479 call TestNet
9480 pop di
9481 JC short sleazefunc1
9482 push cx ; bytes per sector
9483 call modify_cluster_count
9484 pop cx ; ax = sectors per cluster (modified)
9485 ; dx = number of clusters (modified)
9486 ; if bytes per cluster > 16384
9487 ; and hw of cluster count > 0
9488 ; bx = 0FFFEh (invalidated parm sign)
9489
9490 sleazefunc1:
9491 ; ;
9492 ; NOTE THAT A FIXED MEMORY CELL IS USED --> THIS CALL IS NOT
9493 ; RE-ENTRANT. USERS BETTER GET THE ID BYTE BEFORE THEY MAKE THE
9494 ; CALL AGAIN
9495 ;
9496 ; MOV DI,FATBYTE
9497 ; 10/01/2024
9498 ; XOR AH,AH ; AL has sectors/cluster
9499 call Get_User_Stack
9500 ; mov [si+4],cx
9501 ; mov [si+6],bx
9502 ; mov [si+2],di
9503 MOV [SI+user_env.user_CX],CX
9504 MOV [SI+user_env.user_DX],BX
9505 MOV [SI+user_env.user_BX],DI
9506 ; 10/01/2024
9507 MOV word [SI+user_env.user_BX],FATBYTE
9508
9509 ; mov [si+0Eh],ss
9510 MOV [SI+user_env.user_DS],SS ; stash correct pointer
9511
9512 retn
9513
9514 BADSLDRIVE:
9515 jmp FCB_RET_ERR
9516
9517 ;-----
9518 ;
9519 ;** $Get_INDOS_Flag - Return location of DOS Critical Section Flag
9520 ;
9521 ; Returns location of DOS status for interrupt routines
9522 ;
9523 ; ENTRY none
9524 ; EXIT (es:bx) = flag location
9525 ; USES all
9526 ;
9527 ;-----
9528 ;
9529 _$GET_INDOS_FLAG:
9530 CALL Get_User_Stack
9531 ; MOV WORD [SI+2],INDOS
9532 MOV word [SI+user_env.user_BX],INDOS
9533

```

```

9534 getin_segm: ; 13/01/2024
9535 ;MOV [SI+10H],SS
9536 00001334 8C5410 MOV [SI+user_env.user_ES],SS
9537 00001337 C3 RETN
9538 ;
9539 ;-----
9540 ;
9541 ;** $Get_IN_Vars - Return Pointer to DOS Variables
9542 ;
9543 ; Return a pointer to interesting DOS variables This call is version
9544 ; dependent and is subject to change without notice in future versions.
9545 ; Use at risk.
9546 ;
9547 ; ENTRY none
9548 ; EXIT (es:bx) = address of SYSINITVAR
9549 ; uses ALL
9550 ;
9551 ;-----
9552 ;
9553 ; 13/01/2024 - Retro DOS v5.0
9554 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:5226h
9555 ; MSDOS 6.22 MSDOS.SYS - DOSCODE:4D65h
9556 ; MSDOS 5.0 MSDOS.SYS - DOSCODE:4D58h
9557 ;
9558 ;
9559 ;
9560 _$GET_IN_VARS:
9561 00001338 E83CF1 CALL Get_User_Stack
9562 ;MOV WORD [SI+2],SYSINITVAR
9563 ;MOV word [SI+user_env.user_BX],SYSINITVAR
9564 0000133B C74402[2600] MOV word [SI+user_env.user_BX],SYSINITVAR
9565 ; 13/01/2024
9566 ;MOV [SI+10H],SS
9567 ;MOV [SI+user_env.user_ES],SS
9568 ;RETN
9569 00001340 EBF2 jmp short getin_segm
9570 ;
9571 ;-----
9572 ;
9573 ;** $Get_Default_DPB - Return a pointer to the Default DPB
9574 ;
9575 ; Return pointer to drive parameter table for default drive
9576 ;
9577 ; ENTRY none
9578 ; EXIT (ds:bx) = DPB address
9579 ; USES all
9580 ;
9581 ;** $Get_DPB - Return a pointer to a specified DPB
9582 ;
9583 ; Return pointer to a specified drive parameter table
9584 ;
9585 ; ENTRY (dl) = drive # (0 = default, 1=A, 2=B, etc.)
9586 ; EXIT (al) = 0 iff ok
9587 ; (ds:bx) = DPB address
9588 ; (al) = -1 if bad drive
9589 ; USES all
9590 ;
9591 ;-----
9592 ;
9593 ; 10/01/2024
9594 %if 0
9595 ;
9596 ; 15/05/2019 - Retro DOS v4.0
9597 ;
9598 ;
9599 _$GET_DEFAULT_DPB:
9600 MOV DL,0
9601 _$GET_DPB:
9602 push ss
9603 pop ds
9604 ;
9605 MOV AL,DL
9606 call GETTHISDRV ; Get CDS structure
9607 JC short ISNODRV ; no valid drive
9608 LES DI,[THISCDS] ; check for net CDS
9609 ;;test word [es:di+43h],8000h
9610 ;TEST word [ES:DI+curdir.flags],curdir_isnet
9611 ;test byte [es:di+44h],80h
9612 test byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
9613 JNZ short ISNODRV ; No DPB to point at on NET stuff
9614 call ECritDisk
9615 call FATREAD_CDS ; Force Media Check and return DPB
9616 call LCritDisk
9617 JC short ISNODRV ; User FAILED to I 24, only error we
9618 ; have.
9619 call Get_User_Stack
9620 ;mov [si+2],bp
9621 MOV [SI+user_env.user_BX],BP
9622 ;mov [si+0Eh],es
9623 MOV [SI+user_env.user_DS],ES
9624 XOR AL,AL
9625 retn
9626 ISNODRV:
9627 MOV AL,-1
9628 retn
9629 ;
9630 %else
9631 ;
9632 ; 13/01/2024
9633 ; 10/01/2024 - Retro DOS v5.0
9634 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:5230h
9635 ;
9636 ;
9637 _$GET_DEFAULT_DPB:
9638 MOV DL,0
9639 _$GET_DPB:
9640 push ss
9641 pop ds
9642 MOV AL,DL
9643 call GETTHISDRV ; Get CDS structure
9644 mov al,0Fh ; error_invalid_drive
9645 jnc short getdpb_3
9646 ;
9647 ; no valid drive
9648 getdpb_1:
9649 ; 13/01/2024
9650 0000134F E825F1 call Get_User_Stack
9651 00001352 813C0273 cmp word [si],7302h ; INT 21h, AX = 7302h ? Get_ExtDPB
9652 ; GET EXTENDED DPB
9653 00001356 7406 je short getdpb_2
9654 00001358 813C0473 cmp word [si],7304h ; INT 21h, AX = 7304h ? Set_DPBforFormat
9655 ; Set DPB TO USE FOR FORMATTING
9656 0000135C 7503 jne short ISNODRV
9657 getdpb_2:

```



```

9658 0000135E E917F3      jmp     SYS_RET_ERR
9659
9660
9661 00001361 B0FF      ISNODRV: mov     al,0FFh ; -1          ; invalid (or network) drive
9662 00001363 C3          retn
9663
9664
9665 00001364 C43E[A205]   getdpb_3: les     di,[THISCDS] ; check for net CDS
9666                ; ;test word [es:di+43h],8000h
9667                ; TEST word [ES:DI+curdir.flags],curdir_isnet
9668                ; ;test byte [es:di+44h],80h
9669 00001368 26F6454480   test    byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
9670                ; JNZ short ISNODRV ; No DPB to point at on NET stuff
9671                ; 10/01/2024
9672 0000136D 75E0      jnz     short getdpb_1
9673 0000136F E87005     call    ECritDisk
9674 00001372 E8E151     call    FATREAD_CDS ; Force Media Check and return DPB
9675 00001375 E89705     call    LCritDisk
9676                ; JC short ISNODRV ; User FAILED to I 24,
9677 00001378 B053      mov     al,53h ; error_FAIL_I24 ; only error we have.
9678 0000137A 72D3      jc      short getdpb_1
9679 0000137C E8F8F0     call    Get_User_Stack
9680                ; 13/01/2024
9681 0000137F 813C0473   cmp     word [si],7304h ; INT 21h, AX = 7304h ?
9682 00001383 7503      jnz     short getdpb_4
9683 00001385 E9E8FD     jmp     Set_DPBforFormat
9684
9685                ; 13/01/2024
9686
9687 00001388 813C0273   getdpb_4: cmp     word [si],7302h ; INT 21h, AX = 7302h ?
9688 0000138C 7410      je      short getdpb_5
9689 0000138E 26837E0F00   cmp     word [es:bp+0Fh],0
9690 00001393 74BA      jz      short getdpb_1
9691 00001395 896C02     mov     [si+2],bp
9692 00001398 8C440E     mov     [si+0Eh],es
9693 0000139B 30C0      xor     al,al ; status = 0 = successful
9694 0000139D C3          retn
9695
9696
9697 0000139E B018      getdpb_5: mov     al,18h ; error_bad_length
9698 000013A0 837C043F   cmp     word [si+4],63 ; [SI+user_env.user_CX]
9699                ; length of buffer (must be 63 bytes)
9700                ; error
9701                ;
9702 000013A6 06          push    es ; ES:BP = Drive parameter block
9703 000013A7 55          push    bp
9704 000013A8 8E4410     mov     es,[si+16] ; [SI+user_env.user_ES]
9705 000013AB 8B7C0A     mov     di,[si+10] ; [SI+user_env.user_DI]
9706 000013AE 8B5C08     mov     bx,[si+8] ; [SI+user_env.user_SI] (!)
9707 000013B1 5E          pop     si
9708 000013B2 1F          pop     ds
9709
9710 000013B3 B83D00     mov     ax,61 ; length of following data (003Dh)
9711 000013B6 AB          stosw
9712 000013B7 89C1      mov     cx,ax
9713 000013B9 57          push    di
9714 000013BA F3A4      rep movsb
9715 000013BC 5F          pop     di
9716
9717                ; 13/01/2024
9718 000013BD 06          push    es
9719 000013BE 1F          pop     ds
9720 000013BF 31C0      xor     ax,ax ; 0
9721
9722 000013C1 81FBA6F1   cmp     bx,0F1A6h ; (!) signature (undocumented),
9723                ; must be 0F1A6h to get device driver
9724                ; address and next-DPB pointer
9725                ; (Ref: Ralf Brown's Interrupt List)
9726 000013C5 740E      je      short getdpb_6
9727
9728                ; xor ax,ax ; 0
9729 000013C7 48          dec     ax ; -1
9730                ; mov [es:di+19h],ax ; pointer to next DPB (invalidated)
9731                ; mov [es:di+1Bh],ax
9732                ; mov [es:di+13h],ax ; pointer to driver address (invalidated)
9733                ; mov [es:di+15h],ax
9734                ; 13/01/2024
9735 000013C8 894519     mov     [di+19h],ax ; -1 ; pointer to next DPB (invalidated)
9736 000013CB 89451B     mov     [di+1Bh],ax
9737 000013CE 894513     mov     [di+13h],ax ; pointer to driver address (invalidated)
9738 000013D1 894515     mov     [di+15h],ax
9739
9740                ; 13/01/2024
9741 000013D4 40          inc     ax ; 0
9742
9743 getdpb_6:
9744                ; 13/01/2024
9745                ; ax = 0
9746 000013D5 39450F     cmp     [di+0Fh],ax ; 0
9747                ; cmp [es:di+0Fh],ax ; 0 ; FAT (16 bit) size ?
9748                ; ; cmp word [es:di+0Fh],0
9749                ; jz short getdpb_8 ; FAT32
9750                ; ; FAT16 or FAT12
9751                ; mov ax,[es:di+0Dh] ; DPB.MAX_CLUSTER
9752                ; mov [es:di+2Dh],ax ; DPB.LAST_CLUSTER
9753                ; mov ax,[es:di+0Fh] ; DPB.FAT_SIZE
9754                ; mov [es:di+31h],ax ; DPB.FAT32_SIZE
9755                ; mov ax,[es:di+1Dh] ; DPB.NEXT_FREE
9756                ; mov [es:di+39h],ax ; DPB.FAT32_NXTFREE
9757                ; mov ax,[es:di+0Bh] ; DPB.FIRST_SECTOR
9758                ; mov [es:di+29h],ax ; DPB.FCLUS_FSECTOR
9759                ; 13/01/2024
9760 000013DA 8B450D     mov     ax,[di+0Dh] ; DPB.MAX_CLUSTER
9761 000013DD 89452D     mov     [di+2Dh],ax ; DPB.LAST_CLUSTER
9762 000013E0 8B450F     mov     ax,[di+0Fh] ; DPB.FAT_SIZE
9763 000013E3 894531     mov     [di+31h],ax ; DPB.FAT32_SIZE
9764 000013E6 8B451D     mov     ax,[di+1Dh] ; DPB.NEXT_FREE
9765 000013E9 894539     mov     [di+39h],ax ; DPB.FAT32_NXTFREE
9766 000013EC 8B450B     mov     ax,[di+0Bh] ; DPB.FIRST_SECTOR
9767 000013EF 894529     mov     [di+29h],ax ; DPB.FCLUS_FSECTOR
9768
9769 000013F2 31C0      xor     ax,ax ; 0
9770                ; mov [es:di+2Fh],ax ; DPB.LAST_CLUSTER high word
9771                ; mov [es:di+33h],ax ; DPB.FAT32_SIZE high word
9772                ; mov [es:di+3Bh],ax ; DPB.FAT32_NXTFREE high word
9773                ; mov [es:di+2Bh],ax ; DPB.FCLUS_FSECTOR high word
9774                ; mov [es:di+35h],ax ; DPB.ROOT_CLUSTER
9775                ; mov [es:di+37h],ax ; DPB.ROOT_CLUSTER high word
9776                ; mov [es:di+23h],ax ; DPB.EXT_FLAGS
9777                ; 13/01/2024
9778 000013F4 89452F     mov     [di+2Fh],ax ; 0 ; DPB.LAST_CLUSTER high word
9779 000013F7 894533     mov     [di+33h],ax ; DPB.FAT32_SIZE high word
9780 000013FA 89453B     mov     [di+3Bh],ax ; DPB.FAT32_NXTFREE high word
9781 000013FD 89452B     mov     [di+2Bh],ax ; DPB.FCLUS_FSECTOR high word
9782 00001400 894535     mov     [di+35h],ax ; DPB.ROOT_CLUSTER

```

```

9782 00001403 894537      mov     [di+37h],ax      ; DPB.ROOT_CLUSTER high word
9783 00001406 894523      mov     [di+23h],ax      ; DPB.EXT_FLAGS
9784 00001409 48          dec     ax                ; -1
9785                      ;mov     [es:di+25h],ax      ; DPB.FSINFO_SECTOR (invalidated)
9786                      ;mov     [es:di+27h],ax      ; DPB.BKBOOT_SECTOR (invalidated)
9787                      ; 13/01/2024
9788 0000140A 894525      mov     [di+25h],ax      ; DPB.FSINFO_SECTOR (invalidated)
9789 0000140D 894527      mov     [di+27h],ax      ; DPB.BKBOOT_SECTOR (invalidated)
9790 00001410 3B451F      cmp     ax,[di+1Fh]
9791                      ;cmp     ax,[es:di+1Fh]      ; DPB.FREE_COUNT (= -1 ?)
9792 00001413 7401      je      short getdpb_7
9793 00001415 40          inc     ax                ; -1 -> 0
9794
9795 00001416 894521      mov     [di+21h],ax      ; DPB.PB.FREE_COUNT high word
9796                      ;mov     [es:di+21h],ax
9797
9798 ;getdpb_8:
9799                      ; 26/06/2024
9799 00001419 31C0      xor     ax,ax              ; status = 0 = successful
9800
9801 0000141B E950F2      getdpb_8:  ; 03/07/2024
9802                      jmp     SYS_RET_OK
9803
9804 %endif
9805
9806 ;-----
9807
9808 ** $Disk_Reset - Flush out Dirty Buffers
9809
9810 ; $DiskReset flushes and invalidates all buffers. BUGBUG - do
9811 ; we really invalidate? Should we? This screws non-removable
9812 ; caching. Maybe CHKDSK relies upon it, though....
9813
9814 ; ENTRY none
9815 ; EXIT none
9816 ; USES all
9817
9818 ;-----
9819
9820 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
9821 ; DOSCODE:4D94h
9822 ; 13/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
9823 ; DOSCODE:5322h
9824
9825 _$DISK_RESET:
9826 0000141E B0FF      mov     al,0FFh ; -1
9827 00001420 16          push    ss
9828 00001421 1F          pop     ds
9829                      ; 06/11/2022
9830                      ;MOV     AL,-1
9831 00001422 E8BD04      call    ECritDisk
9832                      ; MSDOS 6.0
9833                      ;;or     word [DOS34_FLAG],4
9834                      ;or     word [DOS34_FLAG],FROM_DISK_RESET ;AN000;
9835 00001425 800E[1106]04 or     byte [DOS34_FLAG],FROM_DISK_RESET ; 4 ; 15/05/2019
9836 0000142A E86656      call    FLUSHBUF
9837                      ; MSDOS 6.0
9838                      ;and     word [DOS34_FLAG],0FFFBh
9839                      ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
9840                      ;and     word [DOS34_FLAG],NO_FROM_DISK_RESET ;AN000;
9841                      ; 15/12/2022
9842 0000142D 8026[1106]FB and     byte [DOS34_FLAG],NO_FROM_DISK_RESET ; 0FBh ; 15/05/2019
9843
9844                      ; 13/01/2024 - Retro DOS v5.0
9845                      ; (PCDOS 7.1 IBMDOS.COM)
9846                      ;;;
9847 00001432 C42E[2600]    les     bp,[DPBHEAD]
9848
9849 00001436 83FDFF      drst_1:  cmp     bp,0FFFFh      ; -1 ?
9850 00001439 7409      je      short drst_2      ; yes, it is the last DPB
9851 0000143B E82A20      call    update_fat32_fsinfo ; update FSINFO (sector) parameters
9852 0000143E 26C46E19    les     bp,[es:bp+19h] ; DPB.NEXT_DPB
9853 00001442 EBF2      jmp     short drst_1
9854
9855 drst_2:
9856      ;;;
9857 00001444 C706[8310]0000    mov     word [SC_STATUS],0      ; Throw out secondary cache ; M041
9858
9859 ; we will "ignore" any errors on the flush, and go ahead and invalidate. This
9860 ; call doesn't return any errors and it is supposed to FORCE a known state, so
9861 ; let's do it.
9862
9863 ; Invalidate 'last-buffer' used
9864
9865 0000144A BBFFFF      MOV     BX,-1 ; 0FFFFh
9866 0000144D 891E[2000]    MOV     [LastBuffer+2],BX
9867 00001451 891E[1E00]    MOV     [LastBuffer],BX
9868
9869 ; MSDOS 3.3
9870 ; IBMDOS.COM, Offset 1C66h
9871 ;;;
9872 ;lds     si,[BUFFHEAD]
9873 ;mov     ax,20FFh      ; .buf_ID, AL = FFh (Free buffer)
9874                      ; .buf_flags, AH = 0, reset/clear
9875
9876 ;DRST_1:
9877 ;;mov     [si+4],ax
9878 ;mov     [si+BUFFINFO.buf_ID],ax
9879 ;lds     si,[SI]
9880 ;cmp     si,bx ; -1
9881 ;je      short DRST_2
9882 ;;mov     [si+4],ax
9883 ;mov     [si+BUFFINFO.buf_ID],ax
9884 ;lds     si,[SI]
9885 ;cmp     si,bx
9886 ;jne     short DRST_1
9887 ;;;
9888 ;DRST_2:
9889 00001455 E8B704      call    LCritDisk
9890 00001458 B8FFFF      MOV     AX,-1
9891                      ; 07/12/2022
9892                      ;mov     ax,0FFFFh
9893 0000145B 50          ;CallInstall NetFlushBuf,MultNET,32,AX,AX
9894 0000145C B82011      push    ax ; * MSDOS 6.0 ; 15/05/2019
9895 0000145F CD2F      mov     ax,1120h
9896                      int     2Fh      ; Multiplex - NETWORK REDIRECTOR - FLUSH ALL DISK BUFFERS
9897                      ; DS = DOS CS
9898                      ; Return: CF clear (successful)
9899 00001461 58          pop     ax ; * MSDOS 6.0 ; 15/05/2019
9900
9901      retn
9902
9903 ; 19/07/2018 - Retro DOS v3.0
9904
9905 ;
9906 ; BREAK <$SetDPB - Create a valid DPB from a user-specified BPB>

```



```

9906 ;
9907 ;-----
9908 ;
9909 ** $SetDPB - Create a DPB
9910 ;
9911 ; SetDPB Creates a valid DPB from a user-specified BPB
9912 ;
9913 ; ENTRY ES:BP Points to DPB
9914 ; DS:SI Points to BPB
9915 ; EXIT DPB setup
9916 ; USES ALL but BP, DS, ES
9917 ;-----
9918 ;
9919 ;
9920 ; 10/05/2019 - Retro DOS v4.0
9921 ;
9922 ; DOSCODE:4DD6h (MSDOS 6.21, MSDOS.SYS)
9923 ;
9924 ; MSDOS 6.0
9925 00001463 0300 word3: dw 3 ; M008 -- word value for divides
9926 ;
9927 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0)
9928 ; DOSCODE:4DC9h (MSDOS 5.0, MSDOS.SYS)
9929 ;
9930 ; 13/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1)
9931 ; DOSCODE:5369h (PCDOS 7.1, IBMDOS.COM)
9932 ;
9933 ; 13/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1)
9934 ; Windows ME IO.SYS (Extracted) - BIOSCODE:4EF8h
9935 ;
9936 ;procedure $SETDPB,NEAR
9937 ;
9938 ; 12/04/2024
9939 _$SETDPB:
9940 MOV DI,BP
9941 00001465 89EF ;ADD DI,2 ; skip over dpb_drive and dpb_UNIT
9942 ; 13/01/2024
9943 inc di
9944 00001467 47 inc di
9945 00001468 47 inc di
9946 00001469 AD LODSW
9947 0000146A AB STOSW ; dpb_sector_size
9948 ;
9949 ; 13/01/2024
9950 ; Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
9951 ;;;
9952 0000146B 81F95845 cmp cx,4558h ; 'XE' (NASM syntax)
9953 ; CX = signature 4558h ('EX') for FAT32 extended BPB/DPB
9954 0000146F 7506 jne short not_fat32_extension
9955 00001471 81FA5241 cmp dx,4152h ; 'RA' (NASM syntax)
9956 ; DX = signature 4152h ('AR') for FAT32 extended BPB/DPB
9957 00001475 7402 je short chk_fat32_conditions
9958 not_fat32_extension:
9959 00001477 31C9 xor cx,cx ; 0 ; (Do not use FAT32 extensions -32 bit parameters-)
9960 chk_fat32_conditions:
9961 00001479 51 push cx ; (*)
9962 ;;;
9963 ;
9964 ; 13/01/2024
9965 ; MSDOS 6.0
9966 ;cmp byte [si+3],0
9967 ;CMP BYTE [SI+A_BPB.BPB_NUMBEROFFATS-2],0
9968 0000147A 807C0300 cmp byte [SI+A_BPB.NUMBEROFFATS-2],0 ; FAT file system drive ;AN000;
9969 ;JNZ short yesfat ; yes ;AN000;
9970 0000147E 740C jz short nofat ; 13/01/2024 (PCDOS 7.1)
9971 ;
9972 ; 13/01/2024 - Retro DOS v5.0
9973 ;;;
9974 ;cmp word [si+9],0 ; .BPB_SECTORSPEFAT ; BPB_FATsz16
9975 00001480 837C0900 cmp word [si+A_BPB.SECTORSPEFAT-2],0
9976 00001484 7516 jnz short yesfat
9977 ;cmp word [si+29],0 ; .BPB_FAT32VERSION ; BPB_FSVer
9978 00001486 837C1D00 cmp word [si+A_BPB.FSVER-2],0
9979 0000148A 7410 jz short yesfat
9980 nofat:
9981 ;;;
9982 ;
9983 ; 13/01/2024
9984 ;;mov byte [es:di+4],0
9985 ;MOV BYTE [ES:DI+DPB.FAT_COUNT-4],0
9986 ;JMP short setend ; NO ;AN000;
9987 ;
9988 ; 13/01/2024 (WINME IO.SYS)
9989 ;mov byte [es:di+4],0 ; DPB.FAT_COUNT
9990 ;xor eax,eax
9991 ;jmp setend
9992 ;
9993 ; 13/01/2024 (PCDOS7.1 IBMDOS.COM)
9994 0000148C 31C0 xor ax,ax ; 0
9995 0000148E 26884504 mov [es:di+DPB.FAT_COUNT-4],al ; DPB.FAT_COUNT = 0
9996 ;
9997 ; 13/01/2024 (not necessary)
9998 ;add di,15 ; DPB.DRIVER_ADDR
9999 ;
10000 00001492 83C60B add si,11 ; .BPB_SECTORSPEFAT
10001 ;
10002 ;mov [es:bp+15],ax ; DPB.FAT_SIZE = 0
10003 00001495 2689460F mov [es:bp+DPB.FAT_SIZE],ax
10004 ;
10005 00001499 E9B500 jmp setend
10006 ;
10007 yesfat: ; 10/08/2018
10008 0000149C 89C2 MOV DX,AX
10009 0000149E AC LODSB
10010 ;;;
10011 ; 13/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
10012 0000149F 08C0 or al,al
10013 000014A1 74E9 jz short nofat
10014 ;;;
10015 ;DEC AL
10016 ; 17/12/2022
10017 000014A3 48 dec ax
10018 000014A4 AA STOSB ; dpb_cluster_mask
10019 ;INC AL
10020 000014A5 40 inc ax
10021 000014A6 30E4 XOR AH,AH
10022 LOG2LOOP:
10023 000014A8 A801 test AL,1
10024 000014AA 7506 JNZ short SAVLOG
10025 000014AC FEC4 INC AH
10026 000014AE D0E8 SHR AL,1
10027 000014B0 EBF6 JMP SHORT LOG2LOOP
10028 SAVLOG:
10029 000014B2 88E0 MOV AL,AH

```

```

10030 000014B4 AA      STOSB                ; dpb_cluster_shift
10031 000014B5 88C3    MOV     BL,AL
10032 000014B7 A5      MOVSW                ; dpb_first_FAT Start of FAT (# of reserved sectors)
10033 000014B8 AC      LODSB
10034 000014B9 AA      STOSB                ; dpb_FAT_count Number of FATs
10035                ; OR     AL,AL                ; NONFAT ?                ;AN000;
10036                ; JZ     short setend        ; yes, don't do anything        ;AN000;
10037 000014BA 88C7    MOV     BH,AL
10038 000014BC AD      LODSW
10039 000014BD AB      STOSW                ; dpb_root_entries Number of directory entries
10040 000014BE B105    MOV     CL,5
10041 000014C0 D3EA    SHR     DX,CL                ; Directory entries per sector
10042 000014C2 48      DEC     AX
10043 000014C3 01D0    ADD     AX,DX                ; Cause Round Up
10044 000014C5 89D1    MOV     CX,DX
10045 000014C7 31D2    XOR     DX,DX
10046 000014C9 F7F1    DIV     CX
10047 000014CB 89C1    MOV     CX,AX                ; Number of (root) directory sectors
10048 000014CD 47      INC     DI
10049 000014CE 47      INC     DI                ; Skip dpb_first_sector
10050 000014CF A5      MOVSW                ; Total number of sectors in DSKSIZ (temp as dpb_max_cluster)
10051 000014D0 AC      LODSB
10052                ;mov     [es:bp+17h],al
10053 000014D1 26884617 MOV     [ES:BP+DPB.MEDIA],AL ; Media byte
10054 000014D5 AD      LODSW                ; Number of sectors in a FAT
10055
10056                ;;;
10057                ;MSDOS 3.3
10058                ;
10059                ;STOSB                ; DPB.FAT_SIZE
10060                ;MUL     BH
10061
10062                ;MSDOS 6.0
10063                ;
10064 000014D6 AB      STOSW                ; DPB.FAT_SIZE ;AC000;;>32mb dpb_FAT_size
10065
10066                ; 13/01/2024
10067                %if 0
10068                MOV     DL,BH                ;AN000;;>32mb
10069                XOR     DH,DH                ;AN000;;>32mb
10070                MUL     DX                ;AC000;;>32mb Space occupied by all FATs
10071                ;;;
10072
10073                ;add     ax,[es:bp+6]
10074                ADD     AX,[ES:BP+DPB.FIRST_FAT]
10075                STOSW                ; dpb_dir_sector
10076                ADD     AX,CX                ; Add number of (root) directory sectors
10077                ;mov     [es:bp+0Bh],ax
10078                MOV     [ES:BP+DPB.FIRST_SECTOR],AX
10079
10080                ; MSDOS 6.0
10081                MOV     CL,BL                ;F.C. >32mb                ;AN000;
10082                ;cmp     word [es:bp+0Bh],0
10083                ;CMP     WORD [ES:BP+DSKSIZ],0 ;F.C. >32mb                ;AN000;
10084                ;JNZ     short normal_dpb    ;F.C. >32mb                ;AN000;
10085                ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10086                ; 15/12/2022
10087                ; 28/07/2019
10088                mov     bx,[ES:BP+DSKSIZ]
10089                or      bx,bx
10090                JNZ     short normal_dpb    ;F.C. >32mb                ;AN000;
10091                ;CMP     WORD [ES:BP+DSKSIZ],0 ;F.C. >32mb                ;AN000;
10092                ;JNZ     short normal_dpb    ;F.C. >32mb                ;AN000;
10093
10094                XOR     CH,CH                ;F.C. >32mb                ;AN000;
10095                ;mov     bx,[si+8]
10096                mov     bx,[SI+A_BPB.BIGTOTALSECTORS-A_BPB.SECTORSPERTRACK] ; 01/01/2024 (temporary)
10097                ;MOV     BX,[SI+A_BPB.BPB_BIGTOTALSECTORS-A_BPB.BPB_SECTORSPERTRACK] ;AN000;
10098                ;mov     dx,[si+10]
10099                mov     dx,[SI+A_BPB.BIGTOTALSECTORS-A_BPB.SECTORSPERTRACK+2] ; 01/01/2024 (temporary)
10100                ;MOV     DX,[SI+A_BPB.BPB_BIGTOTALSECTORS-A_BPB.BPB_SECTORSPERTRACK+2] ;AN000;
10101                SUB     BX,AX                ;AN000;;F.C. >32mb
10102                SBB     DX,0                ;AN000;;F.C. >32mb
10103                OR      CX,CX                ;AN000;;F.C. >32mb
10104                JZ      short norot        ;AN000;;F.C. >32mb
10105                rott:                ;AN000;;F.C. >32mb
10106                CLC                ;AN000;;F.C. >32mb
10107                RCR     DX,1                ;AN000;;F.C. >32mb
10108                RCR     BX,1                ;AN000;;F.C. >32mb
10109                LOOP    rott                ;AN000;;F.C. >32mb
10110                norot:                ;AN000;
10111                ; 15/12/2022
10112                ;MOV     AX,BX                ;AN000;;F.C. >32mb
10113                JMP     short setend        ;AN000;;F.C. >32mb
10114
10115                normal_dpb:
10116                ;sub     ax,[es:bp+0Bh]
10117                ;SUB     AX,[ES:BP+DSKSIZ]
10118                ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10119                ; 15/12/2022
10120                ; bx = [es:bp+DSKSIZ]
10121                ;sub     ax,bx ; 28/07/2019
10122                ;SUB     AX,[ES:BP+DSKSIZ]
10123                ; 15/12/2022
10124                sub     bx,ax
10125                ;NEG     AX                ; Sectors in data area
10126                ;MOV     CL,BL                ; dpb_cluster_shift
10127                ; 15/12/2022
10128                ; CL = cluster shift
10129                ; BX = number of data sectors
10130                ;SHR     AX,CL                ; Div by sectors/cluster
10131                shr     bx,cl
10132                setend:
10133                ; M008 - CAS
10134                ;
10135                ; 15/12/2022
10136                inc     bx
10137                ;INC     AX                ; +2 (reserved), -1 (count -> max)
10138                ;
10139                ; There has been a bug in our fatsize calculation for so long
10140                ; that we can't correct it now without causing some user to
10141                ; experience data loss. There are even cases where allowing
10142                ; the number of clusters to exceed the fats is the optimal
10143                ; case -- where adding 2 more fat sectors would make the
10144                ; data field smaller so that there's nothing to use the extra
10145                ; fat sectors for.
10146                ;
10147                ; Note that this bug had very minor known symptoms. CHKDSK would
10148                ; still report that there was a cluster left when the disk was
10149                ; actually full. Very graceful failure for a corrupt system
10150                ; configuration. There may be worse cases that were never
10151                ; properly traced back to this bug. The problem cases only
10152                ; occurred when partition sizes were very near FAT sector
10153                ; rounding boundaries, which were rare cases.

```

```

10154 ; Also, it's possible that some third-party partition program might
10155 ; create a partition that had a less-than-perfect FAT calculation
10156 ; scheme. In this hypothetical case, the number of allocation
10157 ; clusters which don't actually have FAT entries to represent
10158 ; them might be larger and might create a more catastrophic
10159 ; failure. So we'll provide the safeguard of limiting the
10160 ; max_cluster to the amount that will fit in the FATs.
10161 ;
10162 ; ax = maximum legal cluster, ES:BP -> dpb
10163 ;
10164 ; make sure the number of fat sectors is actually enough to
10165 ; hold that many clusters. otherwise, back the number of
10166 ; clusters down
10167 ;
10168 ; 15/12/2022
10169 ; bx = number of clusters
10170 ;
10171 ; 19/07/2018 - Retro DOS v3.0
10172 ; MSDOS 6.0
10173 ; 15/12/2022
10174 ;mov bx,ax ; remember calculated # clusters
10175 ;
10176 ; 01/08/2018 (MSDOS 3.3)
10177 ;mov al,[ES:BP+DPB.FAT_SIZE]
10178 ;xor ah,ah
10179 ;
10180 ; 10/05/2019 - Retro DOS v4.0
10181 ;mov ax,[ES:BP+0Fh]
10182 mov ax,[ES:BP+DPB.FAT_SIZE]
10183 ;
10184 ;mul word [es:bp+2]
10185 mul word [ES:BP+DPB.SECTOR_SIZE] ; how big is the FAT?
10186 cmp bx,4096-10 ; 0FF6h ; test for 12 vs. 16 bit fat
10187 jb short setend_fat12
10188 shr dx,1
10189 ;
10190 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10191 ; 15/12/2022
10192 ;cs3 7/2/92
10193 jnz short setend_faterr ; some bonehead gave us more fatspace
10194 ; than enough for the maximum FAT,
10195 ; so go ahead and use the calculated
10196 ; number of clusters.
10197 ;cs3 7/2/92
10198 ;
10199 rcr ax,1 ; find number of entries
10200 cmp ax,4096-10+1 ; would this truncation move us
10201 ; into 12-bit fatland?
10202 ; jb short setend_faterr ; then go ahead and let the
10203 ; inconsistency pass through
10204 ; ; rather than lose data by
10205 ; ; correcting the fat type
10206 jmp short setend_fat16
10207 ;
10208 setend_fat12:
10209 add ax,ax ; (fatsiz*2)/3 = # of fat entries
10210 adc dx,dx
10211 ;
10212 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10213 ;cs3 7/2/92
10214 ; 15/12/2022
10215 cmp dx,3 ; if our fatspace is WAY more than
10216 jnb short setend_faterr ; we need, we may get an overflow
10217 ; here. check for it and use
10218 ; the calculated size in this case.
10219 ;cs3 7/2/92
10220 ;
10221 div word [cs:word3]
10222 ;
10223 setend_fat16:
10224 dec ax ; limit at 1
10225 cmp ax,bx ; is fat big enough?
10226 jbe short setend_fat ; use max value that'll fit
10227 ;
10228 setend_faterr:
10229 mov ax,bx ; use calculated value
10230 ;
10231 setend_fat:
10232 ;
10233 ; now ax = maximum legal cluster
10234 ;
10235 ; end M008
10236 ;
10237 ;mov [es:bp+0Dh], ax
10238 MOV [ES:BP+DPB.MAX_CLUSTER],AX
10239 ;
10240 ;;mov word [es:bp+1Ch],0 ; MSDOS 3.3
10241 ;mov word [es:bp+1Dh],0 ; MSDOS 6.0
10242 MOV word [ES:BP+DPB.NEXT_FREE],0
10243 ; Init so first ALLOC starts at
10244 ; beginning of FAT
10245 ;;mov word [es:bp+1Eh],-1 ; MSDOS 3.3
10246 ;mov word [es:bp+1Fh],-1 ; MSDOS 6.0
10247 MOV word [ES:BP+DPB.FREE_CNT],-1 ; current count is invalid.
10248 ;
10249 retn
10250 ;
10251 %else
10252 ; 13/01/2024 - Retro DOS v5.0
10253 ;;;
10254 xor dx,dx ; 0
10255 or ax,ax
10256 jnz short savlog1 ; 16 bit FAT size
10257 ; (*) FAT32 extensions
10258 pop dx ; (use 32 bit FAT and Root Dir size if >0)
10259 ; (*)
10260 push dx
10261 or dx,dx
10262 jz short savlog1 ; Do not use FAT32 extensions
10263 ; (do not use 32 bit FAT size field)
10264 mov ax,[si+12] ; .BPB_SECTORS PER FAT32 ; BPB_FATSz32
10265 mov dx,[si+14] ; .BPB_SECTORS PER FAT32+2
10266 savlog1:
10267 push cx ; (**) Root directory sectors
10268 mov cx,ax ; 32 bit multiply
10269 mov al,bh ; FAT count
10270 xor ah,ah
10271 mul dx
10272 xchg ax,cx
10273 mov dl,bh ; FAT count
10274 xor dh,dh
10275 mul dx
10276 add dx,cx
10277 pop cx ; (**)

```

```

10278 000014FC 39D0      cmp     ax,dx
10279 000014FE 7504      jne     short savlog2
10280
10281      ; 13/01/2024
10282      ;or     ax,ax
10283      ;jnz     short savlog2
10284
10285      ; 13/01/2024 (not necessary)
10286      ;inc     di
10287      ;inc     di
10288      ; di = DPB.DRIVER_ADDR
10289
10290      ;jmp     short setend
10291
10292      ; 13/01/2024
10293 00001500 09C0      or      ax,ax
10294 00001502 744D      jz      short setend
10295
10296 savlog2:
10297 00001504 26034606    add     ax,[es:bp+DPB.FIRST_FAT]
10298      ;add     ax,[es:bp+6] ; dx:ax = (total) FAT sectors
10299      add     dx,0
10300 0000150B AB      stosw      ; DPB.DIR_SECTOR
10301 0000150C 01C8      add     ax,cx ; + root directory size
10302 0000150E 2689460B    mov     [es:bp+DPB.FIRST_SECTOR],ax
10303      ;mov     [es:bp+11],ax ; DPB.FIRST_SECTOR ; First data sector
10304 00001512 83D200      adc     dx,0
10305 00001515 59      pop     cx ; (*)
10306 00001516 51      push    cx
10307 00001517 E308      jcxz     savlog3
10308 00001519 26894629    mov     [es:bp+DPB.FCLUS_FSECTOR],ax
10309      ;mov     [es:bp+41],ax ; DPB.BIG_FIRST_SECTOR ; FAT32 first sector field
10310 0000151D 2689562B    mov     [es:bp+DPB.FCLUS_FSECTOR+2],dx
10311      ;mov     [es:bp+43],dx
10312
10313 00001521 88D9      mov     cl,bl ; cluster shift
10314 00001523 26837E0D00    cmp     word [es:bp+DPB.MAX_CLUSTER],0
10315      ;cmp     word [es:bp+13],0 ; DPB.MAX_CLUSTER
10316      ; (contains 16 bit .BPB_TOTALSECTORS as temporary)
10317 00001528 751D      jnz     short normal_dpb
10318 0000152A 30ED      xor     ch,ch
10319 0000152C 52      push    dx ; (**)
10320 0000152D 8B5C08      mov     bx,[si+8] ; SI points to .BPB_SECTORS PER TRACK and SI+8 is
10321      ; .BPB_BIGTOTALSECTORS (32 bit total sectors)
10322 00001530 8B540A      mov     dx,[si+10]
10323 00001533 29C3      sub     bx,ax
10324 00001535 58      pop     ax ; (**)
10325 00001536 19C2      sbb     dx,ax ; dx:bx = data sectors (for cluster count calc)
10326 00001538 09C9      or      cx,cx
10327 0000153A 7407      jz      short norot
10328
10329 0000153C F8      rott:    clc
10330 0000153D D1DA      rcr     dx,1
10331 0000153F D1DB      rcr     bx,1
10332 00001541 E2F9      loop    rott
10333
10334 00001543 89D8      norot:   mov     ax,bx ; dx:ax = cluster count
10335 00001545 EB0A      jmp     short setend
10336
10337 normal_dpb:
10338 00001547 262B460D    sub     ax,[es:bp+DPB.MAX_CLUSTER]
10339      ;sub     ax,[es:bp+13] ; first sector - total sectors
10340 0000154B 31D2      xor     dx,dx
10341 0000154D F7D8      neg     ax ; data sectors = total sectors - first sector
10342 0000154F D3E8      shr     ax,cl ; cluster count
10343
10344 setend:
10345      ; 13/01/2024 (PCDOS 7.1 IBMDOS.COM)
10346 00001551 59      pop     cx ; (*) 0 = not 32 bit fat sectors
10347 00001552 51      push    cx ; (*)
10348
10349      ; si = BIOS Parameter Block + 13
10350
10351      ; 12/04/2024
10352      ; 13/01/2024 (PCDOS 7.1 IBMDOS.COM)
10353 00001553 83C001      add     ax,1
10354 00001556 83D200      adc     dx,0 ; calculated # clusters HW
10355 00001559 89C3      mov     bx,ax ; calculated # clusters LW
10356 0000155B 268B460F    mov     ax,[es:bp+DPB.FAT_SIZE]
10357      ;mov     ax,[es:bp+15] ; FAT size (16 bit)
10358 0000155F E30C      jcxz     setend1 ; Do not use 32 bit FAT sectors field
10359 00001561 31C9      xor     cx,cx
10360 00001563 09C0      or      ax,ax
10361 00001565 7506      jnz     short setend1
10362 00001567 8B440C      mov     ax,[si+12] ; .BPB_SECTORS PER FAT32 ; 32 bit FAT size field.
10363 0000156A 8B4C0E      mov     cx,[si+14] ; .BPB_SECTORS PER FAT32+2
10364
10365 0000156D 52      setend1: push    dx ; (**)
10366      ; cx:ax = FAT size (in sectors)
10367      ; dx:bx = calculated number of clusters
10367 0000156E 91      xchg     ax,cx
10368 0000156F 26F76602    mul     word [es:bp+DPB.SECTOR_SIZE]
10369      ;mul     word [es:bp+2] ; DPB.SECTOR_SIZE
10370 00001573 91      xchg     ax,cx
10371 00001574 26F76602    mul     word [es:bp+DPB.SECTOR_SIZE]
10372      ;mul     word [es:bp+2]
10373 00001578 01CA      add     dx,cx ; dx:ax = FAT size in bytes
10374 0000157A 59      pop     cx ; (**)
10375 0000157B 09C9      or      cx,cx
10376      ;jnz     short setend2 ; FAT32
10377 0000157D 750B      jnz     short setend3 ; 12/04/2024 (BugFix)
10378 0000157F 81FBF60F    cmp     bx,0FF6h
10379 00001583 721E      jb      short setend_fat12 ; FAT12
10380
10381 setend2:
10382      ; (PCDOS 7.1 IBMDOS.COM)
10383      ;or      cx,cx ; HW of calculated cluster count
10384      ;jnz     short setend3
10385 00001585 83FBF6      cmp     bx,0FFF6h
10386 00001588 720C      jb      short setend4 ; FAT16
10387
10388 setend3:
10389      shr     dx,1 ; FAT32 ; 4 byte (32 bit) cluster number
10390      ; fatsiz/4 = # of fat entries
10389 0000158C D1D8      rcr     ax,1
10390 0000158E D1EA      shr     dx,1
10391 00001590 7408      jz      short setend5 ; dx = 0
10392 00001592 D1D8      rcr     ax,1
10393 00001594 EB1B      jmp     short setend_fat16
10394
10395 setend4:
10396 00001596 D1EA      shr     dx,1 ; FAT16 ; 2 byte (16 bit) cluster number
10397      ; fatsiz/2 = # of fat entries
10398      jnz     short setend_faterr ; dx > 0
10399
10400 setend5:
10401      rcr     ax,1 ; FAT16 ; 2 byte (16 bit) cluster number
10402      ; fatsiz/2 = # of fat entries

```

```

10402 0000159C 3DF70F      cmp     ax,0FF7h          ; 4096-10+1
10403 0000159F 721E      jnb     short setend_faterr
10404 000015A1 EB0E      jmp     short setend_fat16
10405
10406
10407 000015A3 01C0      setend_fat12:
10408                      add     ax,ax          ; FAT12 ; 1.5 byte (12 bit) cluster number
10409                      ; (fatsiz*2)/3 = # of fat entries
10410                      adc     dx,dx
10411                      cmp     dx,3          ; if our fatspace is more than we need
10412                      jnb     short setend_faterr ; use calculated size
10413 000015AC 2EF736[6314]  div     word [cs:word3]
10414
10415 000015B1 83E801      setend_fat16:
10416 000015B4 83DA00      sub     ax,1
10417 000015B7 39CA      sbb     dx,0
10418 000015B9 7704      cmp     dx,cx          ; is fat big enough?
10419 000015BB 39D8      ja      short setend_faterr
10420 000015BD 7604      cmp     ax,bx
10421                      jbe     short setend_fat32 ; use max value that'll fit
10422 000015BF 89D8      setend_faterr:
10423 000015C1 89CA      mov     ax,bx          ; use calculated value
10424                      mov     dx,cx
10425 000015C3 26837E0F00  setend_fat32:
10426                      cmp     word [es:bp+DPB.FAT_SIZE],0
10427                      ;cmp word [es:bp+15],0 ; DPB.FAT_SIZE ; 16 bit FAT size
10428 000015C8 750E      jnz     short setend6
10429 000015CA 26C74611FFFF  mov     word [es:bp+DPB.DIR_SECTOR],-1
10430                      ;mov word [es:bp+17],0FFFFh; DPB.DIR_SECTOR
10431                      ; (16 bit directory sector field)
10432 000015D0 26C746D00000  mov     word [es:bp+DPB.MAX_CLUSTER],0
10433                      ;mov word [es:bp+13],0 ; DPB.MAX_CLUSTER (16 bit)
10434                      jmp     short setend7
10435
10436 000015D8 2689460D      setend6:
10437                      mov     [es:bp+DPB.MAX_CLUSTER],ax
10438                      ;mov [es:bp+13],ax ; DPB.MAX_CLUSTER = calculated last cluster number
10439 000015DC 59      setend7:
10440                      pop     cx          ; (*)
10441                      ; 1 = use FAT32 extensions
10442                      ; 0 = don't use FAT32 extensions (32 bit fields)
10443                      jcxz     setend_fat ; do not use FAT32 extensions
10444 000015DD E353      [es:bp+45],ax ; DPB.MAX_CLUSTER32 ; dx:ax = last cluster number
10445                      ;mov [es:bp+47],dx
10446 000015DF 2689462D      mov     [es:bp+DPB.LAST_CLUSTER],ax
10447 000015E3 2689562F      mov     [es:bp+DPB.LAST_CLUSTER+2],dx
10448 000015E7 B8FFFF      mov     ax,0FFFFh ; -1
10449 000015EA 8D7E21      lea     di,[bp+DPB.FREE_CNT_HW]
10450                      ;lea di,[bp+33] ; DPB.FAT32_EXT ; FAT32 extensions
10451                      ; -1 = ready
10452 000015ED AB      stosw
10453 000015EE 8D7410      lea     si,[si+16] ; FAT32 flags
10454 000015F1 A5      movsw ; DPB FAT32 flags ; [bp+23h]
10455 000015F2 83C606      add     si,6
10456 000015F5 AD      lodsw
10457 000015F6 8B54DC      mov     dx,[si-24h] ; FSINFO structure sector number
10458 000015F9 09C0      or      ax,ax ; .BPB_RESERVEDSECTORS ; Number of reserved sectors.
10459 000015FB 7404      jz      short setend8
10460 000015FD 39D0      cmp     ax,dx
10461 000015FF 7203      jb      short setend9
10462
10463 00001601 B8FFFF      setend8:
10464                      mov     ax,0FFFFh ; -1 ; invalid
10465 00001604 AB      setend9:
10466                      stosw
10467                      ; DPB FSINFO structure sector number
10468                      ; [bp+25h]
10469                      ; Sector number of the backup boot sector
10470                      lodsw
10471                      or      ax,ax
10472                      jz      short setend10
10473 0000160A 39D0      cmp     ax,dx
10474 0000160C 7203      jb      short setend11
10475
10476 0000160E B8FFFF      setend10:
10477                      mov     ax,0FFFFh ; -1 ; invalid
10478 00001611 AB      setend11:
10479                      stosw
10480                      ; DPB backup boot sector address
10481                      ; [bp+27h]
10482                      ; [bp+31h]
10483 00001612 83C708      add     di,8
10484 00001615 31D2      xor     dx,dx
10485 00001617 268B45DE      mov     ax,[es:di-34] ; [bp+0Fh] ; DPB.MAX_CLUSTER
10486 0000161B 39D0      cmp     ax,dx
10487 0000161D 7506      jnz     short setend12 ; > 0 (not FAT32)
10488 0000161F 8B44F0      mov     ax,[si-16] ; FAT32 Sectors per FAT ; .BPB_SECTORS PER FAT32
10489 00001622 8B54F2      mov     dx,[si-14]
10490
10491 00001625 AB      setend12:
10492                      stosw ; DPB FAT32 FAT size in sectors ; [bp+31h]
10493 00001626 89D0      mov     ax,dx
10494 00001628 AB      stosw
10495 00001629 83EE08      sub     si,8 ; Root directory cluster number
10496 0000162C A5      movsw ; DPB Root Dir Cluster ; [bp+35h]
10497 0000162D A5      movsw
10498 0000162E 31C0      xor     ax,ax ; DPB reserved ; [bp+39h]
10499 00001630 AB      stosw
10500 00001631 AB      stosw
10501
10502 %endif
10503
10504 setend_fat:
10505 ; 13/01/2024 - Retro DOS v5.0
10506 xor     ax,ax ; 0
10507
10508 ;mov word [es:bp+1Dh],ax ; 0
10509 mov     word [es:bp+DPB.NEXT_FREE],ax ; 0
10510 ; Init so first ALLOC starts at
10511 ; begining of FAT
10512 dec     ax ; -1
10513 ;mov word [es:bp+1Fh],ax ; -1
10514 mov     word [es:bp+DPB.FREE_CNT],ax ; -1 ; current count is invalid.
10515
10516 0000163D C3      retn
10517
10518 ;EndProc $SETDPB
10519
10520 ;BREAK <$Create_Process_Data_Block,SetMem -- Set up process data block>
10521
10522 ;
10523 ;-----
10524 ;
10525 ;** $Dup_PDB
10526 ;
10527 ; Inputs: DX is new segment address of process
10528 ; SI is end of new allocation block
10529 ;
10530 ;-----
10531 ;
10532 ; 14/01/2024 - Retro DOS 5.0
10533 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:554Fh
10534

```

```

10526 _$DUP_PDB:
10527
10528 ;hkn; CreatePDB would have a CS override. This is not valid.
10529 ;hkn; Must set up ds in order to access CreatePDB. Also SS is
10530 ;hkn; has been assumed to be NOTHING. It may not have DOSDATA.
10531
10532 ; MSDOS 3.3
10533 ;MOV byte [CS:CreatePDB],0FFh ; indicate a new process
10534 ;MOV DS,[CS:CurrentPDB]
10535
10536 ; 15/05/2019 - Retro DOS v4.0
10537 ; MSDOS 6.0
10538 0000163E 2E8E1E[0700] mov ds,[cs:DosDSeg]
10539 00001643 C606[A803]FF MOV byte [CreatePDB],0FFh
10540 00001648 8E1E[3003] MOV DS,[CurrentPDB]
10541
10542 0000164C 56 PUSH SI
10543 0000164D EB0A JMP SHORT CreateCopy
10544
10545 ;
10546 ;-----
10547 ;
10548 ; Inputs:
10549 ; DX = Segment number of new base
10550 ; Function:
10551 ; Set up program base and copy term and ^C from int area
10552 ; Returns:
10553 ; None
10554 ; Called at DOS init
10555 ;
10556 ;-----
10557 ;
10558 ;
10559 ; 15/05/2019 - Retro DOS v4.0
10560 ; DOSCODE:4EB6h (MSDOS 6.21, MSDOS.SYS)
10561
10562 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
10563 ; DOSCODE:4EA2h (MSDOS 5.0, MSDOS.SYS)
10564
10565 _$CREATE_PROCESS_DATA_BLOCK:
10566 ; Offset 1D02h in IBMDOS.COM (MSDOS 3.3), 1987
10567 0000164F E825EE CALL Get_User_Stack
10568 ;mov ds,[si+14h]
10569 00001652 8E5C14 MOV DS,[SI+user_env.user_CS]
10570 ;push word [2]
10571 00001655 FF360200 PUSH word [PDB.BLOCK_LEN] ;*
10572 CreateCopy:
10573 00001659 8EC2 MOV ES,DX
10574
10575 0000165B 31F6 XOR SI,SI ; copy entire PDB
10576 0000165D 89F7 MOV DI,SI
10577 0000165F B98000 MOV CX,128
10578 00001662 F3A5 REP MOVSW
10579
10580 ; DOS 3.3 7/9/86
10581 ;mov cx,20
10582 ;MOV CX,FILPERPROC ; copy handles in case of
10583 ; 15/12/2022
10584 00001664 B114 mov cl,FILPERPROC ; 06/07/2019
10585 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10586 ;mov cx,FILPERPROC
10587
10588 ;mov di,18h
10589 00001666 BF1800 MOV DI,PDB.JFN_TABLE ; Set Handle Count has been issued
10590 ;;PUSH DS ; * 15/05/2019
10591 ;;lds si,[34h]
10592 ;LDS SI,[PDB.JFN_Pointer]
10593 ;REP MOVSB
10594 ;;POP DS ; * 15/05/2019
10595 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10596 ; 05/12/2022
10597 ; (push ds then pop ds is not needed here!)
10598 ;push ds
10599 ;lds si,[34h]
10600 00001669 C5363400 lds si,[PDB.JFN_Pointer]
10601 0000166D F3A4 rep movsb
10602 ;pop ds
10603
10604 ; DOS 3.3 7/9/86
10605 ;hkn ;CreatePDB would have a CS override. This is not valid.
10606 ;hkn ;Must set up ds in order to access CreatePDB. Also SS is
10607 ;hkn ;has been assumed to be NOTHING. It may not have DOSDATA.
10608
10609 0000166F 2E8E1E[0700] mov ds,[cs:DosDSeg] ; 15/05/2019
10610
10611 ;;test byte [cs:CreatePDB],0FFh
10612 ;cmp byte [CS:CreatePDB],0 ; Shall we create a process?
10613 ; 17/12/2022
10614 00001674 380E[A803] cmp [CreatePDB],cl ; 0
10615 ;cmp byte [CreatePDB],0 ; 15/05/2019
10616 00001678 744A JZ short Create_PDB_cont ; nope, old style call
10617
10618 ; Here we set up for a new process...
10619
10620 ;PUSH CS ; Called at DOSINIT time, NO SS
10621 ;POP DS
10622
10623 ; MSDOS 6.0
10624 ;;getdseg <ds> ; ds -> dosdata
10625 ;mov ds,[cs:DosDSeg] ; 15/05/2019
10626 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10627 ; (nonsense! but i put this for addr compatibility as temporary)
10628 ; 15/12/2022
10629 ;mov ds,[cs:DosDSeg] ; 15/05/2019
10630
10631 0000167A 31DB XOR BX,BX ; dup all jfns
10632 ;mov cx,20
10633 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10634 ;MOV CX,FILPERPROC ; only 20 of them
10635 ; 15/12/2022
10636 0000167C B114 mov cl,FILPERPROC ; 06/07/2019
10637
10638 Create_dup_jfn:
10639 0000167E 06 PUSH ES ;** ; save new PDB
10640 0000167F E84C60 call SFFromHandle ; get sf pointer
10641 00001682 B0FF MOV AL,-1 ; unassigned JFN
10642 00001684 7224 JC short CreateStash ; file was not really open
10643 ;;test word [es:di+5],1000h
10644 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_no_inherit
10645 ; 15/05/2019
10646 ;test byte [es:di+6],10h
10647 00001686 26F6450610 test byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_no_inherit>>8)
10648 0000168B 751D JNZ short CreateStash ; if no-inherit bit is set, skip dup.
10649

```



```

10650 ; we do not inherit network file handles.
10651
10652 ;mov ah,[es:di+2]
10653 0000168D 268A6502 MOV AH,[ES:DI+SF_ENTRY.sf_mode]
10654 ;and ah,0F0h
10655 00001691 80E4F0 AND AH,SHARING_MASK
10656 ;cmp ah,70h
10657 00001694 80FC70 CMP AH,SHARING_NET_FCB
10658 00001697 7411 JZ short CreateStash
10659
10660 ; The handle we have found is duplicatable (and inheritable). Perform
10661 ; duplication operation.
10662
10663 00001699 893E[9E05] MOV [THISSFT],DI
10664 0000169D 8C06[A005] MOV [THISSFT+2],ES
10665 000016A1 E80F1A call DOS_DUP ; signal duplication
10666
10667 ; get the old sfh for copy
10668
10669 000016A4 E80A60 call pJFNFromHandle ; ES:DI is jfn
10670 000016A7 268A05 MOV AL,[ES:DI] ; get sfh
10671
10672 ; Take AL (old sfh or -1) and stash it into the new position
10673
10674 CreateStash:
10675 000016AA 07 POP ES ;**
10676 ;mov [es:bx+18h],al
10677 000016AB 26884718 MOV [ES:BX+PDB.JFN_TABLE],AL ; copy into new place!
10678 000016AF 43 INC BX ; next jfn...
10679 000016B0 E2CC LOOP Create_dup_jfn
10680
10681 000016B2 8B1E[3003] MOV BX,[CurrentPDB] ; get current process
10682 ; 06/11/2022
10683 ;mov [es:16h],bx
10684 000016B6 26891E1600 MOV [ES:PDB.PARENT_PID],BX; stash in child
10685 000016BB 8C06[3003] MOV [CurrentPDB],ES
10686 ;MOV DS,BX ; 28/07/2019
10687 ; 07/12/2022
10688 ;mov ds,[cs:DosDSeg]
10689 ; 15/12/2022
10690 ; ds = [cs:DosDSeg]
10691 000016BF C606[A803]00 mov byte [CreatePDB],0 ; reset flag
10692 ;mov ds,bx
10693 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10694 ; 15/12/2022
10695 ;mov ds,bx
10696
10697 ; end of new process create
10698
10699 Create_PDB_cont:
10700 ;MOV BYTE [CS:CreatePDB],0 ; reset flag
10701
10702 ;hkn; It comes to this point from 2 places. So, change to DOSDATA temporarily
10703
10704 ;, 28/07/2019
10705 ;;push ds
10706 ;;mov ds,[cs:DosDSeg]
10707 ;mov byte [CreatePDB],0
10708 ;;pop ds
10709
10710 ; 05/12/2022
10711 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10712 ; ; (push-pop ds is nonsense here!
10713 ; ; but i am using same code with original MSDOS.SYS
10714 ; ; for address compatibility.)
10715 ; push ds
10716 ; ds = [cs:DosDSeg] !
10717 ; mov ds,[cs:DosDSeg] ; again !
10718 ; mov byte [CreatePDB],0
10719 ; pop ds
10720
10721 000016C4 58 POP AX ;*
10722
10723 ;entry SETMEM
10724
10725 ; 17/12/2022
10726 ; cx = 0
10727
10728 ;-----
10729 ; Inputs:
10730 ; AX = Size of memory in paragraphs
10731 ; DX = Segment
10732 ; Function:
10733 ; Completely prepares a program base at the
10734 ; specified segment.
10735 ; Called at DOS init
10736 ; Outputs:
10737 ; DS = DX
10738 ; ES = DX
10739 ; [0] has INT int_abort
10740 ; [2] = First unavailable segment
10741 ; [5] to [9] form a long call to the entry point
10742 ; [10] to [13] have exit address (from int_terminate)
10743 ; [14] to [17] have ctrl-C exit address (from int_ctrl_c)
10744 ; [18] to [21] have fatal error address (from int_fatal_abort)
10745 ; DX,BP unchanged. All other registers destroyed.
10746 ;-----
10747
10748 SETMEM:
10749 ;XOR CX,CX
10750 ; 17/12/2022
10751 ; cx = 0
10752 000016C5 8ED9 MOV DS,CX
10753 000016C7 8EC2 MOV ES,DX
10754 ;mov si,88h
10755 000016C9 BE8800 MOV SI,addr_int_terminate
10756 ;mov di,10 ; 0Ah
10757 000016CC BF0A00 MOV DI,SAVEXIT
10758 ;MOV CX,6
10759 ; 15/12/2022
10760 000016CF B106 mov cl,6
10761 000016D1 F3A5 REP MOVSW
10762 000016D3 26A30200 MOV [ES:2],AX
10763 000016D7 29D0 SUB AX,DX
10764 000016D9 3DFF0F CMP AX,MAXDIF ; 0FFFh
10765 000016DC 7603 JBE short HAVDIF
10766 000016DE B8FF0F MOV AX,MAXDIF
10767
10768 000016E1 83E810 SUB AX,10h ; Allow for 100h byte "stack"
10769 000016E4 BB0C00 MOV BX,ENTRYPOINTSEG ; 0Ch; in .COM files
10770 000016E7 29C3 SUB BX,AX
10771 000016E9 B104 MOV CL,4
10772 000016EB D3E0 SHL AX,CL
10773 000016ED 8EDA MOV DS,DX

```

```

10774 ; (MSDOS 6.0 note)
10775 ;
10776 ; The address in BX:AX will be F01D:FEF0 if there is 64K or more
10777 ; memory in the system. This is equivalent to 0:c0 if A20 is OFF.
10778 ; If DOS is in HMA this equivalence is no longer valid as A20 is ON.
10779 ; But the BIOS which now resides in FFFF:30 has 5 bytes in FFFF:D0
10780 ; (F01D:FEF0) which is the same as the ones in 0:C0, thereby
10781 ; making this equivalence valid for this particular case. If however
10782 ; there is less than 64K remaining the address in BX:AX will not
10783 ; be the same as above. We will then stuff 0:c0, the call 5 address
10784 ; into the PSP.
10785 ;
10786 ; Therefore for the case where there is less than 64k remaining in
10787 ; the system old CPM Apps that look at PSP:6 to determine memory
10788 ; requirements will not work. Call 5, however will continue to work
10789 ; for all cases.
10790 ;
10791 ;
10792 ;
10793 ;mov [6],ax
10794 ;mov [8],bx
10795
10796 000016EF A30600 MOV [PDB.CPM_CALL+1],AX
10797 000016F2 891E0800 MOV [PDB.CPM_CALL+3],BX
10798
10799 ; 06/05/2019 - Retro DOS v4.0
10800 000016F6 3DF0FE cmp ax,WRAPOFFSET ; 0FEF0h ; Q: does the system have >= 64k of
10801 ; memory left
10802 000016F9 740C je short addr_ok ; Y: the above calculated address is
10803 ; OK
10804 ; N:
10805
10806 000016FB C7060600C000 MOV WORD [PDB.CPM_CALL+1],0C0h
10807 00001701 C70608000000 MOV WORD [PDB.CPM_CALL+3],0
10808
10809 addr_ok:
10810 00001707 C7060000CD20 ;mov word [0],20CDh
10811 MOV word [PDB.EXIT_CALL],(int_abort*256) + mi_INT
10812 0000170D C60605009A ;mov byte [5],9Ah
10813 MOV BYTE [PDB.CPM_CALL],mi_Long_CALL
10814 00001712 C7065000CD21 ;mov word [50h],21CDh
10815 MOV WORD [PDB.CALL_SYSTEM],(int_command*256) + mi_INT
10816 00001718 C6065200CB ;mov byte [52h],0CBh
10817 MOV BYTE [PDB.CALL_SYSTEM+2],mi_Long_RET
10818 0000171D C70634001800 ;mov word [34h],18h
10819 MOV WORD [PDB.JFN_Pointer],PDB.JFN_TABLE
10820 00001723 8C1E3600 ;mov word [36h],ds
10821 MOV WORD [PDB.JFN_Pointer+2],DS
10822 00001727 C70632001400 ;mov word [32h],20
10823 MOV WORD [PDB.JFN_Length],FILPERPROC
10824 ;
10825 ; The server runs several PDB's without creating them VIA EXEC. We need to
10826 ; enumerate all PDB's at CPS time in order to find all references to a
10827 ; particular SFT. We perform this by requiring that the server link together
10828 ; for us all sub-PDB's that he creates. The requirement for us, now, is to
10829 ; initialize this pointer.
10830 ;
10831 0000172D C7063800FFFF ;mov word [38h],-1
10832 MOV word [PDB.Next_PDB],-1
10833 00001733 C7063A00FFFF ;mov word [3Ah],-1
10834 MOV word [PDB.Next_PDB+2],-1
10835
10836 ; 06/05/2019
10837 ; Set the real version number in the PSP - 5.00
10838
10839 ;mov word [es:PDB.Version],1406h ; MSDOS 6.21 (DOSCODE:4FB6h)
10840 00001739 26C7064000070A ; 07/12/2022
10841 mov word [ES:PDB.Version],(MINOR_VERSION*256)+MAJOR_VERSION
10842 00001740 C3
10843
10844 ; 29/04/2019 - Retro DOS v4.0
10845
10846 ;BREAK <$GSetMediaID -- get set media ID>
10847
10848 ;-----
10849 ; Inputs:
10850 ; BL= drive number as defined in IOCTL
10851 ; AL= 0 get media ID
10852 ; 1 set media ID
10853 ; DS:DX= buffer containing information
10854 ; DW 0 info level (set on input)
10855 ; DD ? serial #
10856 ; DB 11 dup(?) volume id
10857 ; DB 8 dup(?) file system type
10858 ; Function:
10859 ; Get or set media ID
10860 ; Returns:
10861 ; carry clear, DS:DX is filled
10862 ; carry set, error
10863 ;-----
10864
10865 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
10866
10867 ; 14/01/2024
10868 %if 0
10869
10870 _$GSetMediaID:
10871 ; RAWIO - GET_MEDIA_ID
10872 mov cx,0866h ;AN000;MS.; assume get for IOCTL
10873 cmp al,0 ;AN001;MS.; get ?
10874 je short doioc1 ;AN000;MS.; yes
10875 ;cmp al,1 ;AN000;MS.; set ?
10876 ;jne short errorfunc ;AN000;MS.; no
10877 ; 15/12/2022
10878 dec al
10879 jnz short errorfunc ; al > 1
10880 ; RAWIO - SET_MEDIA_ID
10881 ;mov cx,0846h ;AN001;MS.;
10882 ; 15/12/2022
10883 mov cl,46h ; cx = 0846h
10884
10885 doioc1: ;AN000;
10886 mov al,0dh ;AN000;MS.; generic IOCTL
10887 ;invoke $IOCTL ;AN000;MS.; let IOCTL take care of it
10888 ;call _$IOCTL
10889 ;retn ;AN000;MS.;
10890 ; 15/12/2022
10891 jmp _$IOCTL
10892 errorfunc: ;AN000;
10893 ;error error_invalid_function;AN000;MS. ; invalid function
10894 ;mov al,1
10895 mov al,error_invalid_function
10896 jmp SYS_RET_ERR
10897
10898 %else

```



```

10898 ; 14/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
10899 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:5667h
10900
10901 _GSetMediaID:
10902 00001741 1E      push    ds
10903 00001742 56      push    si
10904 00001743 36C536[A205] lds     si,[ss:THISCDS]
10905 00001748 C57445      lds     si,[si+45h]
10906 ; [si+curdir.devptr]
10907 00001748 88740F      mov     si,[si+0Fh]
10908 0000174E B96608      mov     cx,0866h
10909 00001751 3C01      cmp     al,1
10910 00001753 7208      jnb     short doioctl1
10911 00001755 7733      ja      short errorfunc
10912 00001757 B146      mov     cl,46h ; cx = 0846h
10913 ; or si,si
10914 00001759 21F6      and     si,si ; 14/01/2024
10915 0000175B 7424      jz      short doioctl2
10916
10917 0000175D 09F6      or      si,si
10918 0000175F 7522      jnz     short doioctl1
10919 00001761 1F      pop     ds
10920 00001762 1E      push    ds
10921 00001763 89D6      mov     si,dx ; disk info
10922 ; .....
10923 ; 00h WORD 0000h (info level)
10924 ; 02h DWORD disk serial number (binary)
10925 ; 06h 11 BYTES volume label or "NO NAME" if none present
10926 ; 11h 8 BYTES (AL=00h only) filesystem type
10927 ; "FAT12"
10928 ; "FAT16"
10929 ; "FAT32" ; PCDOS 7.1
10930 ; "CDROM"
10931 ; "CD001"
10932 ; "CDAUDIO"
10933 ; .....
10934 ; (ref: Ralf Brown's Interrupt List)
10935
10936
10937 00001765 817C114641      cmp     word [si+11h],4146h ; 'FA'
10938 0000176A 7517      jne     short doioctl1
10939 0000176C 817C135433      cmp     word [si+13h],3354h ; 'T3'
10940 00001771 7510      jne     short doioctl1
10941 00001773 817C153220      cmp     word [si+15h],2032h ; '2 '
10942 00001778 7509      jne     short doioctl1
10943 0000177A 817C172020      cmp     word [si+17h],2020h ; ' '
10944 0000177F 7502      jne     short doioctl1
10945
10946 00001781 B548      mov     ch,48h ; cx = 4846h
10947
10948 00001783 5E      pop     si
10949 00001784 1F      pop     ds
10950 00001785 B00D      mov     al,0Dh ; generic IOCTL
10951 ; call _$IOCTL
10952 ; retn
10953 00001787 E9F910      jmp     _$IOCTL ; let IOCTL take care of it
10954
10955 0000178A B001      mov     al,1 ; error_invalid_function
10956 0000178C E9E9EE      jmp     SYS_RET_ERR
10957
10958 %endif
10959
10960 ; 16/05/2019 - Retro DOS v4.0
10961
10962 ; =====
10963 ; MISC2.ASM, MSDOS 6.0, 1991
10964 ; =====
10965 ; 20/07/2018 - Retro DOS v3.0
10966 ; 29/04/2019 - Retro DOS v4.0
10967
10968 ; Break <STRCMP - compare two ASCIZ strings DS:SI to ES:DI>
10969 ; -----
10970 ;
10971 ; Strcmp - compare ASCIZ DS:SI to ES:DI. Case INSENSITIVE. '/' = '\'
10972 ; Strings of different lengths don't match.
10973 ; Inputs: DS:SI - pointer to source string ES:DI - pointer to dest string
10974 ; Outputs: Z if strings same, NZ if different
10975 ; Registers modified: NONE
10976 ; -----
10977
10978 ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
10979
10980 ; 14/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
10981 ; (PCDOS 7.1 IBMDOS.COM DOSCODE:56B6h)
10982 ; (MSDOS 6.22 MSDOS.SYS DOSCODE:4FD7h)
10983
10984 0000178F 56      push    si
10985 00001790 57      push    di
10986 00001791 50      push    ax
10987
10988 00001792 AC      LODSB
10989 00001793 E8FD45      call    UCase ; convert to upper case
10990 00001796 E84D46      call    PATHCHRCMP ; convert '/' to '\' ; 07/12/2022 ('\')
10991 00001799 88C4      MOV     AH,AL
10992 0000179B 268A05      MOV     AL,[ES:DI]
10993 0000179E 47      INC     DI
10994 0000179F E8F145      call    UCase ; convert to upper case
10995 000017A2 E84146      call    PATHCHRCMP ; convert '/' to '\' ; 07/12/2022 ('\')
10996 000017A5 38C4      CMP     AH,AL
10997 000017A7 7504      JNZ     short PopRet ; strings dif
10998
10999 000017A9 08C0      OR      AL,AL
11000 000017AB 75E5      JNZ     short Cmplp ; More string
11001
11002 000017AD 58      pop     ax
11003 000017AE 5F      pop     di
11004 000017AF 5E      pop     si
11005 000017B0 C3      retn
11006
11007 ;Break <STRCPY - copy ASCIZ string from DS:SI to ES:DI>
11008 ; -----
11009 ;
11010 ; Strcpy - copy an ASCIZ string from DS:SI to ES:DI and make uppercase
11011 ; FStrcpy - copy an ASCIZ string from DS:SI to ES:DI. no modification of
11012 ; characters.
11013 ;
11014 ; Inputs: DS:SI - pointer to source string
11015 ; ES:DI - pointer to destination string
11016 ; Outputs: ES:DI point byte after nul byte at end of dest string
11017 ; DS:SI point byte after nul byte at end of source string
11018 ; Registers modified: SI,DI
11019 ; -----
11020
11021 StrCpy:

```

```

11022 000017B1 50      push    ax
11023 CPYLoop:
11024 000017B2 AC      LODSB
11025 000017B3 E8DD45   call    UCASE      ; convert to upper case
11026 000017B6 E82D46   call    PATHCHRCMP ; convert / to \ ;
11027 000017B9 AA      STOSB
11028
11029 000017BA 08C0      OR      AL,AL
11030 000017BC 75F4      JNZ     short CPYLoop
11031 000017BE 58      pop     ax
11032 000017BF C3      retn
11033
11034 ;-----
11035 ; Procedure Name : FStrCpy
11036 ;-----
11037
11038 FStrCpy:
11039 000017C0 50      push    ax
11040 FCPYLoop:
11041 000017C1 AC      LODSB
11042 000017C2 AA      STOSB
11043 000017C3 08C0      OR      AL,AL
11044 000017C5 75FA      JNZ     short FCPYLoop
11045 000017C7 58      pop     ax
11046 000017C8 C3      retn
11047
11048 ; 20/07/2018 - Retro DOS v3.0
11049
11050 ; UCASE, IBMDOS.COM (MSDOS 3.3), 1987 - Offset 1E2Fh
11051 ;-----
11052
11053 ; UCASE:
11054 ; call    _UCASE    ; Offset 5518h (GetLet, Offset 5517h)
11055 ; retn
11056
11057 ; Break <StrLen - compute length of string ES:DI>
11058 ;-----
11059 ** StrLen - Compute Length of String
11060 ;
11061 ; StrLen computes the length of a string, including the trailing 00
11062 ;
11063 ; ENTRY   (es:di) = address of string
11064 ; EXIT    (cx) = size of string
11065 ; USES    cx, flags
11066 ;-----
11067
11068 StrLen:
11069 000017C9 57      push    di
11070 000017CA 50      push    ax
11071 ; MOV     CX,-1
11072 000017CB B9FFFF   mov     cx,65535
11073 000017CE 30C0      XOR     AL,AL
11074 000017D0 F2AE      REPNE  SCASB
11075 000017D2 F7D1      NOT     CX
11076 000017D4 58      pop     ax
11077 000017D5 5F      pop     di
11078 000017D6 C3      retn
11079
11080 ;-----
11081 ** DStrLen - Compute Length of String
11082 ;
11083 ; ENTRY   (ds:si) = address of string
11084 ; EXIT    (cx) = size of string, including trailing NUL
11085 ; USES    cx, flags
11086 ;-----
11087
11088 DStrLen: ; BUGBUG - this guy is a pig, who uses him?
11089 000017D7 E80300   CALL    XCHGP
11090 000017DA E8ECFF   CALL    StrLen
11091 ; CALL    XCHGP
11092 ; retn
11093 ; 18/12/2022
11094 ; jmp     short XCHGP
11095
11096 ;-----
11097 ** XCHGP - Exchange Source and Destination Pointers
11098 ;
11099 ; XCHGP exchanges (DS:SI) and (ES:DI)
11100 ;
11101 ; ENTRY   none
11102 ; EXIT    pairs exchanged
11103 ; USES    SI, DI, DS, ES
11104 ;-----
11105
11106 XCHGP:
11107 000017DD 1E      push    ds
11108 000017DE 06      push    es
11109 000017DF 1F      pop     ds
11110 000017E0 07      pop     es
11111 000017E1 87F7     XCHG    SI,DI
11112
11113 000017E3 C3      xchgp_retn: retn
11114
11115 ; Break <Idle - wait for a specified amount of time>
11116 ;-----
11117
11118 ; Idle - when retrying an operation due to a lock/sharing violation,
11119 ; we spin until RetryLoop is exhausted.
11120 ;
11121 ; Inputs: RetryLoop is the number of times we spin
11122 ; Outputs: wait
11123 ; Registers modified: none
11124 ;-----
11125
11126 Idle:
11127 ; test    byte [SS:FSHARING],0FFh
11128 000017E4 36803E[7205]00 cmp     byte [SS:FSHARING],0 ;hkn; SS override
11129 ; retnz
11130 000017EA 75F7     jnz     short xchgp_retn
11131 ; SAVE    <CX>
11132 000017EC 51      push    cx
11133 000017ED 368B0E[1C00] mov     cx,[ss:RetryLoop] ;hkn; SS override
11134 000017F2 E308     JCXZ    Idle3
11135
11136 000017F4 51      PUSH    CX
11137 000017F5 31C9     XOR     CX,CX
11138
11139 000017F7 E2FE     LOOP   Idle2
11140 000017F9 59      POP     CX
11141 000017FA E2F8     LOOP   Idle1
11142
11143 Idle3:
11144 000017FC 59      ; RESTORE <CX>
11145 000017FD C3      pop     cx
retn

```

```

11146
11147 ;Break <TableDispatch - dispatch to a table>
11148 ;-----
11149 ;
11150 ; TableDispatch - given a table and an index, jmp to the appropriate
11151 ; routine. Preserve all input registers to the routine.
11152 ;
11153 ; Inputs: Push return address
11154 ;         Push Table address
11155 ;         Push index (byte)
11156 ; Outputs: appropriate routine gets jumped to.
11157 ;         return indicates invalid index
11158 ; Registers modified: none.
11159 ;-----
11160
11161 struct TFrame ; TableFrame
11162 .OldBP: resw 1 ; 0
11163 .OldRet: resw 1 ; 2
11164 .Index: resb 1 ; 4
11165 .Pad: resb 1 ; 5
11166 .Tab: resw 1 ; 6
11167 .NewRet: resw 1 ; 8
11168 endstruc
11169
11170 TableDispatch:
11171 PUSH BP
11172 MOV BP,SP
11173 PUSH BX ; save BX
11174 ;mov bx,[bp+6]
11175 MOV BX,[BP+TFrame.Tab] ; get pointer to table
11176 MOV BL,[CS:BX] ; maximum index
11177 ;cmp [bp+4],bl
11178 CMP [BP+TFrame.Index],BL ; table error?
11179 JAE short TableError ; yes
11180 ;mov bl,[bp+4]
11181 MOV BL,[BP+TFrame.Index] ; get desired table index
11182 XOR BH,BH ; convert to word
11183 SHL BX,1 ; convert to word pointer
11184 INC BX ; point past first length byte
11185 ; 17/08/2018
11186 ;add bx,[bp+6]
11187 ADD BX,[BP+TFrame.Tab] ; get real offset
11188 MOV BX,[CS:BX] ; get contents of table entry
11189 ;mov [bp+6],bx
11190 MOV [BP+TFrame.Tab],BX ; put table entry into return address
11191 POP BX ; restore BX
11192 POP BP ; restore BP
11193 ADD SP,4 ; clean off Index and our return addr
11194 retn ; do operation
11195
11196 TableError:
11197 POP BX ; restore BX
11198 POP BP ; restore BP
11199 RETN 6 ; clean off Index, Table and RetAddr
1200
1201 ;Break <TestNet - determine if a CDS is for the network>
1202 ;-----
1203 ;
1204 ; TestNet - examine CDS pointed to by ThisCDS and see if it indicates a
1205 ; network CDS. This will handle NULL cds also.
1206 ;
1207 ; Inputs: ThisCDS points to CDS or NULL
1208 ; Outputs: ES:DI = ThisCDS
1209 ;         carry Set => network
1210 ;         carry Clear => local
1211 ; Registers modified: none.
1212 ;-----
1213
1214 TestNet:
1215 ;LES DI,[CS:THISCDS]
1216 ; 16/05/2019 - Retro DOS v4.0
1217 mov es,[cs:DosDseg]
1218 LES DI,[ES:THISCDS]
1219 CMP DI,-1
1220 JZ short CMCRet ; UNC? carry is clear
1221 ;test word [es:di+43h],8000h
1222 ;TEST word [ES:DI+curdir.flags],curdir_isnet
1223 ;test byte [es:di+44h],80h
1224 TEST byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
1225 JNZ short CMCRet ; jump has carry clear
1226 retn ; carry is clear
1227 CMCRet:
1228 CMC
1229 retn
1230
1231 ;Break <IsSFTNet - see if an sft is for the network>
1232 ;-----
1233 ;
1234 ; IsSFTNet - examine SF pointed to by ES:DI and see if it indicates a
1235 ; network file.
1236 ;
1237 ; Inputs: ES:DI point to SFT
1238 ; Outputs: Zero set if not network sft
1239 ;         zero reset otherwise
1240 ;         Carry CLEAR!!!
1241 ; Registers modified: none.
1242 ;-----
1243
1244 IsSFTNet:
1245 ;test word [es:di+5],8000h
1246 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
1247 ; 16/05/2019
1248 ;test byte [es:di+6],80h
1249 TEST byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
1250 retn
1251
1252 ;Break <FastInit - Initialize FastTable entries >
1253 ;-----
1254 ;
1255 ; DOS 4.00 2/9/87
1256 ; FastInit - initialize the FASTXXX routine entry
1257 ; in the FastTable
1258 ;
1259 ; Inputs: BX = FASTXXX ID ( 1=fastopen )
1260 ;         DS:SI = address of FASTXXX routine entry
1261 ;         SI = -1 for query only
1262 ; Outputs: Carry flag clear, if success
1263 ;         Carry flag set, if failure
1264 ;
1265 ;-----
1266
1267 ;Procedure FastInit,NEAR
1268 ; ASSUME CS:DOSCODE,SS:NOTHING
1269

```

```

11270 ; ; MSDOS 3.3
11271 ; ; IBMDOS.COM (1987) - Offset 1EB3h
11272 ;FastInit:
11273 ; mov di,FastTable ; FastOpenTable
11274 ; mov ax,[cs:di+4] ; Entry segment
11275 ; mov bx,cs ; get DOS segment
11276 ; cmp ax,bx ; first time installed ?
11277 ; je short ok_install ; yes
11278 ; stc ; set carry
11279 ; retn ; (cf=1 means) already installed !
11280
11281 ;ok_install:
11282 ; mov bx,FastTable ; FastOpenTable
11283 ; mov cx,ds
11284 ; ; set address of FASTXXX (FASTOPEN) routine entry
11285 ; mov [cs:bx+4],cx
11286 ; mov [cs:bx+2],si
11287 ; retn
11288
11289 ; 16/05/2019 - Retro DOS v4.0
11290
11291 ; 14/01/2024 - Retro DOS v5.0
11292 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:5773h
11293
11294 FastInit:
11295 ; MSDOS 6.0
11296 ;hkn; set up es to dosdataseg.
11297 00001848 06 push es
11298 ;getdseg <es> ; es -> dosdata
11299 00001849 2E8E06[0700] mov es,[cs:DosDseg]
11300
11301 ;hkn; FastTable is in DOSDATA
11302 0000184E BF[3E11] MOV DI,FastTable+2 ;AN000;FO. points to fastxxx entry
11303 00001851 4B DEC BX ;AN000;FO.;; decrement index
11304 00001852 89DA MOV DX,BX ;AN000;FO.;; save bx
11305 00001854 D1E3 SHL BX,1 ;AN000;FO.;; times 4, each entry is DWORD
11306 00001856 D1E3 SHL BX,1 ;AN000;FO.
11307 00001858 01DF ADD DI,BX ;AN000;FO. index to the entry
11308 0000185A 268B4502 MOV AX,[ES:DI+2] ;AN000;FO. get entry segment
11309 fcheck: ;AN000;
11310 0000185E 8CC9 MOV CX,CS ;AN000;FO.;; get DOS segment
11311 00001860 39C8 CMP AX,CX ;AN000;FO.;; first time installed ?
11312 00001862 7405 JZ short ok_install ;AN000;FO.;; yes
11313 00001864 09C0 OR AX,AX ;AN000;FO.;;
11314 ;JZ short ok_install ;AN000;FO.;;
11315 ;STC ;AN000;FO.;; already installed !
11316 ;JMP SHORT FSret ;AN000;FO. set carry
11317 ; 14/01/2024
11318 00001866 F9 stc
11319 00001867 7517 jnz short FSret
11320 ok_install: ;AN000;
11321 00001869 83FEFF CMP SI,-1 ;AN000;FO.;; Query only ?
11322 0000186C 7412 JZ short FSret ;AN000;FO.;; yes
11323 0000186E 8CD9 MOV CX,DS ;AN000;FO.;; get FASTXXX entry segment
11324 00001870 26894D02 MOV [ES:DI+2],CX ;AN000;FO.;; initialize routine entry
11325 00001874 268935 MOV [ES:DI],SI ;AN000;FO.;; initialize routine offset
11326
11327 ;hkn; FastFlg moved to DOSDATA
11328 00001877 BF[4611] MOV DI,FastFlg ;AN000;FO.;; get addr of FASTXXX flags
11329 0000187A 01D7 ADD DI,DX ;AN000;FO.;; index to a FASTXXX flag
11330 ;or byte [es:di],80h
11331 0000187C 26800D80 OR byte [ES:DI],Fast_yes ;AN000;FO.;; indicate installed
11332 FSret: ;AN000;
11333 00001880 07 pop es
11334 00001881 C3 retn ;AN000;FO.
11335
11336 ;EndProc FastInit
11337
11338 ;Break <FastRet - initial routine in FastOpenTable >
11339 -----
11340 ; DOS 3.3 6/10/86
11341 ; FastRet - indicate FASTXXXX not in memory
11342 ;
11343 ; Inputs: None
11344 ; Outputs: AX = -1 and carry flag set
11345 ;
11346 ; Registers modified: none.
11347 -----
11348
11349 FastRet:
11350 ;mov ax,-1
11351 ;stc
11352 ;retf
11353 00001882 F9 STC
11354 00001883 19C0 sbb ax,ax ; (ax) = -1, 'C' set
11355 00001885 CB RETF
11356
11357 ;Break <NLS_OPEN - do $open for NLSFUNC>
11358 -----
11359 ; DOS 3.3 6/10/86
11360 ; NLS_OPEN - call $OPEN for NLSFUNC
11361 ;
11362 ; Inputs: Same input as $OPEN except CL = mode
11363 ; Outputs: same output as $OPEN
11364 ;
11365 -----
11366
11367 ;hkn; NOTE! SS MUST HAVE BEEN SET UP TO DOSDATA BY THE TIME THESE
11368 ;hkn; NLS FUNCTIONS ARE CALLED!!! THERE FORE WE WILL USE SS OVERRIDES
11369 ;hkn; IN ORDER TO ACCESS DOS DATA VARIABLES!
11370
11371 NLS_OPEN:
11372 ; MOV BL,[CPSWFLAG] ; disable code page matching logic
11373 ; MOV BYTE [CPSWFLAG],0
11374 ; PUSH BX ; save current state
11375
11376 00001886 88C8 MOV AL,CL ; set up correct interface for $OPEN
11377 00001888 E83967 call _$OPEN
11378
11379 ; POP BX ; restore current state
11380 ; MOV [CPSWFLAG],BL
11381
11382 0000188B C3 RETN
11383
11384 ;Break <NLS_LSEEK - do $LSEEK for NLSFUNC>
11385 -----
11386 ; DOS 3.3 6/10/86
11387 ; NLS_LSEEK - call $LSEEK for NLSFUNC
11388 ;
11389 ; Inputs: BP = open mode
11390 ; Outputs: same output as $LSEEK
11391 ;
11392 -----
11393

```

```

11394 ; 16/05/2019 - Retro DOS v4.0
11395
11396 NLS_LSEEK:
11397     PUSH    word [SS:USER_SP] ; save user stack
11398     PUSH    word [SS:USER_SS]
11399     CALL    Fake_User_Stack
11400     MOV     AX,BP ; set up correct interface for $LSEEK
11401     CALL    _$LSEEK
11402 NLS_SEEK_RET:
11403     ; 26/06/2024
11404     POP     word [SS:USER_SS] ; restore user stack
11405     POP     word [SS:USER_SP]
11406     RETN
11407
11408 ;Break    <Fake_User_Stack - save user stack>
11409 ;-----
11410 ;   DOS 3.3   6/10/86
11411 ;   Fake_User_Stack - save user stack pointer
11412 ;-----
11413
11414 Fake_User_Stack:
11415     MOV     AX,[SS:USER_SP_2F] ; replace with INT 2Fh stack
11416     MOV     [SS:USER_SP],AX
11417     MOV     AX,SS
11418     MOV     [SS:USER_SS],AX
11419     RETN
11420
11421 ;Break    <GetDevList - get device header list pointer>
11422 ;-----
11423 ;   DOS 3.3   7/25/86
11424 ;   GetDevList - get device header list pointer
11425 ;
11426 ;   Output: AX:BX points to the device header list
11427 ;-----
11428
11429 GetDevList:
11430     ; 16/05/2019 - Retro DOS v4.0
11431     MOV     SI,SysInitTable
11432     mov     ds,[cs:DosDSeg]
11433     LDS     SI,[SI]
11434     ;mov     ax,[si+34] ; SSYSINITVARS offset 34 = [SI+SYSI.DEV]
11435     MOV     AX,[SI+SYSI.DEV]
11436     ;mov     bx,[si+36] ; SSYSINITVARS offset 36 = [SI+SYSI.DEV+2]
11437     MOV     BX,[SI+SYSI.DEV+2]
11438     RETN
11439
11440 ;Break    <NLS_IOCTL - do $_IOCTL for NLSFUNC>
11441 ;-----
11442 ;   DOS 3.3   7/25/86
11443 ;   NLS_IOCTL - call $_IOCTL for NLSFUNC
11444 ;
11445 ;   Inputs: BP = function code 0CH
11446 ;   Outputs:      same output as generic $_IOCTL
11447 ;-----
11448
11449 NLS_IOCTL:
11450     ; 16/05/2019 - Retro DOS v4.0
11451     PUSH    word [SS:USER_SP] ; save user stack
11452     PUSH    word [SS:USER_SS]
11453     CALL    Fake_User_Stack
11454     MOV     AX,BP ; set up correct interface for $_IOCTL
11455     CALL    _$_IOCTL
11456     ;POP     word [SS:USER_SS] ; restore user stack
11457     ;POP     word [SS:USER_SP]
11458     ;RETN
11459     ; 26/06/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
11460     jmp     short NLS_SEEK_RET
11461
11462 ;Break    <NLS_GETEXT- get extended error for NLSFUNC>
11463 ;-----
11464 ;   DOS 3.3   7/25/86
11465 ;   NLS_GETEXT -
11466 ;
11467 ;   Inputs: none
11468 ;   Outputs:      AX = extended error
11469 ;-----
11470
11471 NLS_GETEXT:
11472     ; 16/05/2019 - Retro DOS v4.0
11473     MOV     AX,[SS:EXTERR] ; return extended error
11474     ; 23/09/2023
11475 MSG_RETRIEVAL:
11476     RETN
11477
11478 ; 29/04/2019 - Retro DOS v4.0
11479
11480 ;Break    <MSG_RETRIEVAL- get beginning addr of system and parser messages>
11481 ;-----
11482 ;   DOS 4.00
11483 ;
11484 ;   Inputs: DL=0 get extended error message addr
11485 ;           =1 set extended error message addr
11486 ;           =2 get parser error message addr
11487 ;           =3 set parser error message addr
11488 ;           =4 get critical error message addr
11489 ;           =5 set critical error message addr
11490 ;           =6 get file system error message addr
11491 ;           =7 set file system error message addr
11492 ;           =8 get address for code reduction
11493 ;           =9 set address for code reduction
11494 ;
11495 ;   Function: get/set message address
11496 ;   Outputs:      ES:DI points to addr when get
11497 ;-----
11498
11499 ;Procedure MSG_RETRIEVAL,NEAR
11500 ;   ASSUME CS:DOSCODE,SS:NOTHING
11501
11502 ; 23/09/2023
11503 MSG_RETRIEVAL:
11504
11505 ;; NOTE: This function lives in command.com resident code now.
11506 ;; If the int 2F ever gets this far, we'll return registers
11507 ;; unchanged, which produces the same result as before, if
11508 ;; command.com wasn't present (and therefore no messages available).
11509 ;;
11510 ;; I didn't point the entry in the 2F table to No_Op because
11511 ;; No_Op zeroes AL.
11512 ;;
11513 ;;
11514 ;;
11515 ;;hkn; set up ds to point to DOSDATA
11516 ;; push ds
11517 ;; getdseg <ds> ; ds -> dosdata

```

```

11518 ;
11519 ;
11520 ; PUSH AX ;AN000;;MS. save regs
11521 ; PUSH SI ;AN000;;MS. save regs
11522 ; MOV AX,DX ;AN000;;MS.
11523 ; MOV SI,OFFSET DOSDATA:MSG_EXTERROR ;AN000;;MS.
11524 ; test AL,1 ;AN000;;MS. get ?
11525 ; JZ toget ;AN000;;MS. yes
11526 ; DEC AL ;AN000;;MS.
11527 ; toget: ;AN000;;
11528 ; SHL AL,1 ;AN000;;MS. times 2
11529 ; XOR AH,AH ;AN000;;MS.
11530 ; ADD SI,AX ;AN000;;MS. position to the entry
11531 ; test DL,1 ;AN000;;MS. get ?
11532 ; JZ getget ;AN000;;MS. yes
11533 ; MOV WORD PTR DS:[SI],DI ;AN000;;MS. set MSG
11534 ; MOV WORD PTR DS:[SI+2],ES ;AN000;;MS. address to ES:DI
11535 ; JMP SHORT MSGret ;AN000;;MS. exit
11536 ; getget: ;AN000;;
11537 ; LES DI,DWORD PTR DS:[SI] ;AN000;;MS. get msg addr
11538 ; MSGret: ;AN000;;
11539 ; POP SI ;AN000;;MS.
11540 ; POP AX ;AN000;;MS.
11541 ;
11542 ; pop ds
11543 ;
11544 ; return ;AN000;;MS. exit
11545 ; 23/09/2023
11546 ; retn ; 29/04/2019
11547 ;
11548 ;=====
11549 ; ECritDisk, LCritDisk, ECritDevice, LCritDevice
11550 ; IBMDOS.COM (MSDOS 3.3), 1987 - Offset 1F36h
11551 ;=====
11552 ; 20/07/2018 - Retro DOS v3.0
11553 ;
11554 ; ; MSDOS 3.3
11555 ; ; 08/08/2018 - Retro DOS v3.0
11556 ; ECritMEM:
11557 ; ECritSFT:
11558 ;
11559 ; ECritDisk:
11560 ; retn
11561 ; ;push ax
11562 ;
11563 ; mov ax,8001h
11564 ; int 2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11565 ; ; AL = critical section number (00h-0Fh)
11566 ;
11567 ; pop ax
11568 ; retn
11569 ;
11570 ; ; MSDOS 3.3
11571 ; ; 08/08/2018 - Retro DOS v3.0
11572 ; LCritMEM:
11573 ; LCritSFT:
11574 ;
11575 ; LCritDisk:
11576 ; retn
11577 ; ;push ax
11578 ;
11579 ; mov ax,8101h
11580 ; int 2Ah ; Microsoft Networks - END DOS CRITICAL SECTION
11581 ; ; AL = critical section number (00h-0Fh)
11582 ;
11583 ; pop ax
11584 ; retn
11585 ; ECritDevice:
11586 ; retn
11587 ; ;push ax
11588 ;
11589 ; mov ax,8002h
11590 ; int 2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11591 ; ; AL = critical section number (00h-0Fh)
11592 ;
11593 ; pop ax
11594 ; retn
11595 ; LCritDevice:
11596 ; retn
11597 ; ;push ax
11598 ;
11599 ; mov ax,8102h
11600 ; int 2Ah ; Microsoft Networks - END DOS CRITICAL SECTION
11601 ; ; AL = critical section number (00h-0Fh)
11602 ;
11603 ; pop ax
11604 ; retn
11605 ;=====
11606 ; CRIT.ASM, MSDOS 6.0, 1991
11607 ;=====
11608 ; 12/05/2019 - Retro DOS v4.0
11609 ;
11610 ; Critical Section Routines
11611 ;
11612 ; MSDOS 6.21 - MSDOS.SYS - DOSCODE:513Ah
11613 ;
11614 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
11615 ; DOSCODE:5126h (MSDOS 5.0 MSDOS.SYS)
11616 ;
11617 ; -----
11618 ; Each handler must leave everything untouched; including flags!
11619 ;
11620 ; Sleaze for time savings: first instruction is a return. This is patched
11621 ; by the sharer to be a PUSH AX to complete the correct routines.
11622 ; -----
11623 ; (DOSMAC.INC, MSDOS 6.0, 1991)
11624 ; -----
11625 ; Some old versions of the 80286 have a bug in the chip. The popf instruction
11626 ; will enable interrupts. Therefore in a section of code with interrupts
11627 ; disabled and you need a popf instruction use the 'popff' macro instead.
11628 ; -----
11629 ;
11630 ;%macro POPFF 0
11631 ; jmp $+3
11632 ; iret
11633 ; push cs
11634 ; call $-2
11635 ;%endmacro
11636 ;
11637 ; -----
11638 ;
11639 ; 14/01/2024 - Retro DOS v5.0
11640 ;%if 0
11641 ;

```



```

11642 ;Procedure ECritDisk,NEAR
11643 ;public ECritMEM
11644 ;public ECritSFT
11645 ECritMEM:
11646 ECritSFT:
11647 ;
11648 ECritDisk:
11649
11650 ;SR; Check if critical section is to be entered
11651
11652     pushf
11653     cmp     byte [ss:redir_patch],0
11654     jz      short ECritDisk_2
11655
11656 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11657 ; ;popff ; * (macro)
11658 ; jmp      short ECritDisk_1 ; *
11659 ;
11660 ;ECritDisk_iret: ; *
11661 ; iret ; *
11662
11663 ; 16/12/2022
11664 ; 13/11/2022
11665 ; jmp      short ECritDisk_1
11666 ; 06/11/2022
11667 ;ECritDisk_iret:
11668 ; iret
11669
11670 ; 06/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11671 ECritDisk_1:
11672     push    cs ; *
11673     call    ECritDisk_iret ; *
11674
11675 ECritDisk_0:
11676     PUSH    AX
11677     ;MOV    AX,8000h+critDisk
11678     ;INT    int_IBM
11679     mov     ax,8001h
11680     int     2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11681                ; AL = critical section number (00h-0Fh)
11682     POP     AX
11683     retn
11684
11685 ; 16/12/2022
11686 ; 13/11/2022
11687 ECritDisk_iret: ; 12/05/2019 - Retro DOS v4.0
11688 LCritDisk_iret:
11689     iret
11690
11691 ECritDisk_2:
11692     ;popff ; *
11693     ;retn
11694     ; jmp    short ECritDisk_3 ; *
11695 ;ECritDisk_iret2: ; *
11696 ; iret
11697
11698 ; 16/12/2022
11699 ; 13/11/2022
11700 ; jmp    short ECritDisk_3
11701 ;ECritDisk_iret2:
11702 ; iret
11703
11704 ECritDisk_3:
11705     push    cs ; *
11706     ; 13/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11707     ; call   ECritDisk_iret2 ; *
11708     ; retn
11709     ; 16/12/2022
11710     call    ECritDisk_iret
11711     retn
11712
11713 ;EndProc ECritDisk
11714
11715 ; -----
11716
11717 ;Procedure LCritDisk,NEAR
11718 ;public LCritMEM
11719 ;public LCritSFT
11720 LCritMEM:
11721 LCritSFT:
11722 ;
11723 LCritDisk:
11724
11725 ;SR; Check if critical section is to be entered
11726
11727     pushf
11728     cmp     byte [ss:redir_patch],0
11729     jz      short LCritDisk_2
11730     ;popff ; * (macro)
11731     ; jmp    short LCritDisk_1 ; *
11732 ;
11733 ;LCritDisk_iret: ; *
11734 ; iret ; *
11735
11736 ; 16/12/2022
11737 ; 13/11/2022
11738 ; jmp    short LCritDisk_1
11739 ;LCritDisk_iret:
11740 ; iret
11741
11742 LCritDisk_1:
11743     push    cs ; *
11744     call    LCritDisk_iret ; *
11745
11746 LCritDisk_0:
11747     PUSH    AX
11748     ;MOV    AX,8100h+critDisk
11749     ;INT    int_IBM
11750     mov     ax,8101h
11751     int     2Ah ; Microsoft Networks - END DOS CRITICAL SECTION
11752                ; AL = critical section number (00h-0Fh)
11753     POP     AX
11754     retn
11755
11756 ;LCritDisk_iret: ; 12/05/2019 - Retro DOS v4.0
11757 ; iret
11758
11759 LCritDisk_2:
11760     ;popff ; *
11761     ;retn
11762     ; jmp    short LCritDisk_3 ; *
11763 ;LCritDisk_iret2: ; *
11764 ; iret
11765

```

```

11766         ; 16/12/2022
11767         ; 13/11/2022
11768         ; jmp short LCritDisk_3
11769 ;LCritDisk_iret2:
11770         ; iret
11771
11772 LCritDisk_3:
11773     push    cs ; *
11774     ; 13/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11775     ; call LCritDisk_iret2 ; *
11776     ; retn
11777     ; 16/12/2022
11778     call    LCritDisk_iret
11779     retn
11780
11781 ;EndProc LCritDisk
11782
11783 ; -----
11784
11785 ;Procedure ECritDevice,NEAR
11786
11787 ECritDevice:
11788
11789 ;SR; Check if critical section is to be entered
11790
11791     pushf
11792     cmp     byte [ss:redir_patch],0
11793     jz      short ECritDevice_2
11794     ; popff ; * (macro)
11795     ; jmp    short ECritDevice_1 ; *
11796
11797 ;ECritDevice_iret: ; *
11798 ; iret ; *
11799
11800     ; 16/12/2022
11801     ; 13/11/2022
11802     ; jmp    short ECritDevice_1
11803 ;ECritDevice_iret:
11804 ; iret
11805
11806 ECritDevice_1:
11807     push    cs ; *
11808     call    ECritDevice_iret ; *
11809
11810 ECritDevice_0:
11811     PUSH    AX
11812     ; MOV    AX,8000h+critDevice
11813     ; INT     int_IBM
11814     mov     ax,8002h
11815     int     2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11816             ; AL = critical section number (00h-0Fh)
11817     POP     AX
11818     retn
11819
11820     ; 16/12/2022
11821     ; 06/12/2022
11822 ECritDevice_iret: ; 12/05/2019 - Retro DOS v4.0
11823 LCritDevice_iret:
11824     iret
11825
11826 ECritDevice_2:
11827     ; popff ; *
11828     ; retn
11829     ; jmp    short ECritDevice_3 ; *
11830 ;ECritDevice_iret2: ; *
11831 ; iret
11832
11833     ; 16/12/2022
11834     ; 13/11/2022
11835     ; jmp    short ECritDevice_3
11836 ;ECritDevice_iret2:
11837 ; iret
11838
11839 ECritDevice_3:
11840     push    cs ; *
11841     ; 13/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11842     ; call    ECritDevice_iret2 ; *
11843     ; retn
11844     ; 16/12/2022
11845     call    ECritDevice_iret
11846     retn
11847
11848 ;EndProc ECritDevice
11849
11850 ; -----
11851
11852 ;Procedure LCritDevice,NEAR
11853
11854 LCritDevice:
11855
11856 ;SR; Check if critical section is to be entered
11857
11858     pushf
11859     cmp     byte [ss:redir_patch],0
11860     jz      short LCritDevice_2
11861     ; popff ; * (macro)
11862     ; jmp    short LCritDevice_1 ; *
11863
11864 ;LCritDevice_iret: ; *
11865 ; iret ; *
11866
11867     ; 16/12/2022
11868     ; 13/11/2022
11869     ; jmp    short LCritDevice_1
11870 ;LCritDevice_iret:
11871 ; iret
11872
11873 LCritDevice_1:
11874     push    cs ; *
11875     call    LCritDevice_iret ; *
11876
11877 LCritDevice_0:
11878     PUSH    AX
11879     ; MOV    AX,8100h+critDevice
11880     ; INT     int_IBM
11881     mov     ax,8102h
11882     int     2Ah ; Microsoft Networks - END DOS CRITICAL SECTION
11883             ; AL = critical section number (00h-0Fh)
11884     POP     AX
11885     retn
11886
11887 ;LCritDevice_iret: ; 12/05/2019 - Retro DOS v4.0
11888 ; iret
11889

```



```

11890      LCritDevice_2:
11891          ;;popff ; *
11892          ;;retn
11893      ; jmp short LCritDevice_3 ; *
11894      ;LCritDevice_iret2: ; *
11895      ; iret
11896
11897          ; 16/12/2022
11898          ; 13/11/2022
11899          ; jmp short LCritDevice_3
11900      ;LCritDevice_iret2:
11901      ;iret
11902
11903      LCritDevice_3:
11904          push cs ; *
11905          ; 13/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11906          ; call LCritDevice_iret2 ; *
11907          ;retn
11908          ; 16/12/2022
11909          call LCritDevice_iret
11910          retn
11911
11912      ;EndProc LCritDevice
11913
11914      %endif
11915
11916          ; 15/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1)
11917          ; PC DOS 7.1 IBMDOS.COM - DOSCODE:580Dh
11918
11919      ; 15/01/2024 - Retro DOS v5.0
11920      %if 1
11921          ;;
11922          ; -----
11923      ECritMEM:
11924      ECritSFT:
11925      ; -----
11926          ; PC DOS 7.1 IBMDOS.COM
11927      ECritDisk:
11928          push cx
11929          mov ch,0
11930          mov cl,[ss:redir_patch]
11931          jcxz ECritDisk_3
11932          pop cx
11933          push ax
11934          mov ax,8001h ; BEGIN DOS CRITICAL SECTION
11935                      ; AL = critical section number (01h)
11936      ECritDisk_1:
11937      ; -----
11938      LCritDisk_1:
11939      ECritDevice_1:
11940      LCritDevice_1:
11941          push cx
11942          mov ch,0
11943          mov cl,[ss:Iswin386]
11944          jcxz ECritDisk_2
11945          pop cx
11946          push ax
11947          int 2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11948                      ; AL = critical section number (00h-0Fh)
11949          pop ax
11950          retn
11951
11952      ECritDisk_2:
11953          push es
11954          ;mov cx,0
11955          ; 15/01/2024
11956          ; cx = 0
11957          mov es,cx
11958          pushf ; simulate INT 2Ah
11959          call far [es:00A8h] ; call far (INT 2Ah vector)
11960          pop es
11961          pop cx
11962          pop ax
11963          retn
11964
11965      ECritDisk_3:
11966      ; -----
11967      LCritDisk_3:
11968      ECritDevice_3:
11969      LCritDevice_3:
11970          pop cx
11971          retn
11972
11973      ; -----
11974      LCritMEM:
11975      LCritSFT:
11976      ; -----
11977          ; PC DOS 7.1 IBMDOS.COM
11978      LCritDisk:
11979          push cx
11980          mov ch,0
11981          mov cl,[ss:redir_patch]
11982          jcxz LCritDisk_3
11983          pop cx
11984          push ax
11985          mov ax,8101h ; END DOS CRITICAL SECTION
11986                      ; AL = critical section number (01h)
11987          jmp short LCritDisk_1
11988
11989      ; -----
11990      ECritDevice:
11991          push cx
11992          mov ch,0
11993          mov cl,[ss:redir_patch]
11994          jcxz ECritDevice_3
11995          pop cx
11996          push ax
11997          mov ax,8002h ; BEGIN DOS CRITICAL SECTION
11998                      ; AL = critical section number (02h)
11999          jmp short ECritDevice_1
12000
12001      ; -----
12002      LCritDevice:
12003          push cx
12004          mov ch,0
12005          mov cl,[ss:redir_patch]
12006          jcxz LCritDevice_3
12007          pop cx
12008          push ax
12009          mov ax,8102h ; END DOS CRITICAL SECTION
12010                      ; AL = critical section number (02h)
12011          jmp short LCritDevice_1
12012
12013

```

```

12014 ; -----
12015 ;;;
12016 %endif
12017
12018 ;=====
12019 ; CPMIO.ASM, MSDOS 6.0, 1991
12020 ;=====
12021 ; 20/07/2018 - Retro DOS v3.0
12022 ;=====
12023 ;
12024 ; STDIO.ASM - (MSDOS 2.0)
12025 ;=====
12026 ;
12027 ;
12028 ; Standard device IO for MSDOS (first 12 function calls)
12029 ;
12030 ;
12031 ;.xlist
12032 ;.xcref
12033 ;INCLUDE STDSW.ASM
12034 ;INCLUDE DOSSEG.ASM
12035 ;.cref
12036 ;.list
12037
12038 ;TITLE   STDIO - device IO for MSDOS
12039 ;NAME     STDIO
12040
12041 ;INCLUDE IO.ASM
12042
12043 ; -----
12044 ;
12045 ; NOTE for Retro DOS v2.0 : (ERDOGAN TAN - 13/03/2018)
12046 ; IO.ASM is missing in MSDOS 2.0 kernel source code files !!!
12047 ; INSTEAD of IO.ASM, I have disassembled IBMDOS.COM (MSDOS 2.0)
12048 ; and I have used CPMIO.ASM (MSDOS 6.0 source code)
12049 ; to restore MSDOS 2.0 device IO source code
12050 ;
12051 ; (STRIN.ASM has '$STD_CON_STRING_INPUT' code.)
12052 ;
12053 ;=====
12054 ; STDIO.ASM - (MSDOS 2.0)
12055 ;=====
12056 ;
12057 ;
12058 ; Standard device IO for MSDOS (first 12 function calls)
12059 ;
12060 ;
12061 ;.xlist
12062 ;.xcref
12063 ;INCLUDE STDSW.ASM
12064 ;INCLUDE DOSSEG.ASM
12065 ;.cref
12066 ;.list
12067
12068 ;TITLE   STDIO - device IO for MSDOS
12069 ;NAME     STDIO
12070
12071 ;INCLUDE IO.ASM
12072
12073 ; -----
12074 ;
12075 ; NOTE for Retro DOS v2.0 : (ERDOGAN TAN - 13/03/2018)
12076 ; IO.ASM is missing in MSDOS 2.0 kernel source code files !!!
12077 ; INSTEAD of IO.ASM, I have disassembled IBMDOS.COM (MSDOS 2.0)
12078 ; and I have used CPMIO.ASM (MSDOS 6.0 source code)
12079 ; to restore MSDOS 2.0 device IO source code
12080 ;
12081 ; (STRIN.ASM has '$STD_CON_STRING_INPUT' code.)
12082 ;
12083 ;=====
12084 ; IO.ASM (MSDOS 2.0) (IBMDOS.COM 2.0) - STRIN.ASM (MSDOS 2.0, 19/08/1983)
12085 ;=====
12086 ; Retro DOS v2.0 by Erdogan Tan, 13/03/2018 - 14/03/2018
12087
12088 ; (Disassembled code of IBMDOS.COM, 08/03/1983) - Dissassembler: IDA Pro Free
12089 ; (Comments are from CPMIO.ASM - 1991, MSDOS 6.0)
12090
12091 ;=====
12092 ; CPMIO.ASM (MSDOS 6.0, 1991)
12093 ;=====
12094 ; Retro DOS v4.0 by Erdogan Tan, 04/05/2019
12095
12096 ; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
12097
12098 ;** Standard device IO for MSDOS (first 12 function calls)
12099 ;
12100 ; TITLE   IBMCPMIO - device IO for MSDOS
12101 ; NAME     IBMCPMIO
12102
12103 ; Old style CP/M 1-12 system calls to talk to reserved devices
12104 ;
12105 ; $Std_Con_Input_No_Echo
12106 ; $Std_Con_String_Output
12107 ; $Std_Con_String_Input
12108 ; $RawConIO
12109 ; $RawConInput
12110 ; RAWOUT
12111 ; RAWOUT2
12112 ;
12113 ;
12114 ; The following routines form the console I/O group (funcs 1,2,6,7,8,9,10,11).
12115 ; They assume ES and DS NOTHING, while not strictly correct, this forces data
12116 ; references to be SS or CS relative which is desired.
12117 ;
12118 ; -----
12119 ;
12120 ; TITLE   CPMIO2 - device IO for MSDOS
12121 ; NAME     CPMIO2
12122 ;
12123 ;
12124 ; Microsoft Confidential
12125 ; Copyright (C) Microsoft Corporation 1991
12126 ; All Rights Reserved.
12127 ;
12128 ;
12129 ;** Old style CP/M 1-12 system calls to talk to reserved devices
12130 ;
12131 ; $Std_Con_Input
12132 ; $Std_Con_Output
12133 ; OUTT
12134 ; TAB
12135 ; BUFOUT
12136 ; $Std_Aux_Input
12137 ; $Std_Aux_Output

```

```

12138 ; $Std_Printer_Output
12139 ; $Std_Con_Input_Status
12140 ; $Std_Con_Input_Flush
12141 ;
12142 ; Revision History:
12143 ;
12144 ; AN000 version 4.00 - Jan. 1988
12145 ;
12146 ; The following routines form the console I/O group (funcs 1,2,6,7,8,9,10,11).
12147 ; They assume ES and DS NOTHING, while not strictly correct, this forces data
12148 ; references to be SS or CS relative which is desired.
12149 ;
12150 ;DOSCODE SEGMENT
12151 ; ASSUME SS:DOSDATA,CS:DOSCODE
12152 ;
12153 ;
12154 ;hkn; All the variables use SS override or DS. Therefore there is
12155 ;hkn; no need to specifically set up any seg regs unless SS assumption is
12156 ;hkn; not valid.
12157 ;
12158 ; DOSCODE:51BAh (MSDOS 6.21, MSDOS.SYS)
12159 ; 08/11/2022
12160 ; DOSCODE:51A6h (MSDOS 5.0, MSDOS.SYS)
12161 ;
12162 ;
12163 ;-----
12164 ; Procedure : $Std_Con_Input_No_Echo
12165 ;
12166 ;-----
12167 ;
12168 ;
12169 ;
12170 _$STD_CON_INPUT_NO_ECHO: ;system call 8
12171 ;
12172 ; Inputs:
12173 ; None
12174 ; Function:
12175 ; Input character from console, no echo
12176 ; Returns:
12177 ; AL = character
12178 ;
12179 00001942 1E push ds
12180 00001943 56 push si
12181 ;
12182 00001944 E86745 INTEST: call STATCHK
12183 00001947 753A jnz short GET ; 08/09/2018
12184 ;*****
12185 ;hkn; SS override
12186 00001949 36803E[A00A]00 cmp byte [SS:PRINTER_FLAG],0 ; is printer idle?
12187 0000194F 7505 jnz short no_sys_wait
12188 00001951 B405 mov ah,5 ; get input status with system wait
12189 00001953 E86E37 call IOFUNC
12190 no_sys_wait:
12191 ;*****
12192 00001956 B484 MOV AH,84h ; (Microsoft Networks - KEYBOARD BUSY LOOP)
12193 00001958 CD2A INT int_IBM ; int 2Ah
12194 ;
12195 ;;; 7/15/86 update the date in the idle loop
12196 ;;; Dec 19, 1986 D.C.L. changed following CMP to Byte Ptr from Word Ptr
12197 ;;; to shorten loop in consideration of the PC Convertible
12198 ;
12199 ;hkn; SS override
12200 0000195A 36803E[BD0D]FF CMP byte [SS:DATE_FLAG],-1; date is updated may be every
12201 00001960 751A JNZ short NoUpdate ; 65535 x ? ms if no one calls
12202 ;
12203 00001962 50 PUSH AX
12204 00001963 53 PUSH BX ; following is tricky,
12205 00001964 51 PUSH CX ; it may be called by critical handler
12206 00001965 52 PUSH DX ; at that time, DEVCALL is used by
12207 ; other's READ or WRITE
12208 00001966 1E PUSH DS ; save DS = SFT's segment
12209 ;
12210 ;hkn; READTIME must use ds = DOSDATA
12211 ;hkn; PUSH CS ; READTIME must use DS=CS
12212 ;
12213 00001967 16 PUSH SS ; 04/05/2019
12214 00001968 1F POP DS
12215 ;
12216 ;MOV AX,0 ; therefore, we save DEVCALL
12217 ; 26/06/2024
12218 00001969 31C0 xor ax,ax
12219 0000196B E89A02 CALL Save_Restore_Packet ; save DEVCALL packet
12220 ;invoke READTIME ; readtime
12221 0000196E E819F2 call READTIME
12222 00001971 B80100 MOV AX,1
12223 00001974 E89102 CALL Save_Restore_Packet ; restore DEVCALL packet
12224 ;
12225 ; ; MSDOS 3.3 (IBMDOS.COM, Offset 1F8Ch)
12226 ; ; (MSDOS 6.0 code does not contain IBM DOS FETCHI_TAG check)
12227 ; push bx
12228 ; mov bx,DATE_FLAG
12229 ; add bx,2 ; mov bx,FETCHI_FLAG
12230 ; cmp word [cs:bx],5872h
12231 ; jz short FETCHI_TAG_chk_ok
12232 ; call DOSINIT
12233 ;FETCHI_TAG_chk_ok:
12234 ; pop bx
12235 ;
12236 00001977 1F POP DS ; restore DS
12237 00001978 5A POP DX
12238 00001979 59 POP CX
12239 0000197A 58 POP BX
12240 0000197B 58 POP AX
12241 NoUpdate:
12242 ;
12243 ;hkn; SS override
12244 0000197C 36FF06[BD0D] INC word [SS:DATE_FLAG]
12245 ;
12246 ;;; 7/15/86 ;;;
12247 00001981 EBC1 JMP short INTEST
12248 GET:
12249 00001983 30E4 XOR AH,AH
12250 00001985 E83C37 call IOFUNC
12251 00001988 5E POP SI
12252 00001989 1F POP DS
12253 ;;; 7/15/86
12254 ;hkn; SS override
12255 ; MSDOS 6.0
12256 0000198A 36C606[BC0D]00 MOV BYTE [SS:SCAN_FLAG],0
12257 ;
12258 ;
12259 00001990 3C00 CMP AL,0 ; extended code ( AL )
12260 00001992 7505 JNZ short noscan
12261 ;

```

```

12262 ;hkn; SS override
12263 ;MOV BYTE [SS:SCAN_FLAG],1 ; set this flag for ALT_Q key
12264 ; 20/06/2023
12265 00001994 36FE06[BC0D] ; inc byte [SS:SCAN_FLAG]
12266 noscan:
12267 00001999 C3 retn
12268 ;
12269 ;-----
12270 ;
12271 ;** $STD_CON_STRING_OUTPUT - Console String Output
12272 ;
12273 ;
12274 ; ENTRY (DS:DX) Point to output string '$' terminated
12275 ; EXIT none
12276 ; USES ALL
12277 ;
12278 ;-----
12279 ;
12280
12281 _$STD_CON_STRING_OUTPUT: ;System call 9
12282 0000199A 89D6 mov si,dx
12283 STRING_OUT1:
12284 0000199C AC lodsb
12285 0000199D 3C24 cmp al,'$'
12286 0000199F 74F8 je short noscan
12287 NEXT_STR1:
12288 000019A1 E88D02 call OUTT
12289 000019A4 EBF6 jmp short STRING_OUT1
12290 ;
12291 ;-----
12292 ;
12293 ;** $STD_CON_STRING_INPUT - Input Line from Console
12294 ;
12295 ; $STD_CON_STRING_INPUT Fills a buffer from console input until CR
12296 ;
12297 ; ENTRY (ds:dx) = input buffer
12298 ; EXIT none
12299 ; USES ALL
12300 ;
12301 ;-----
12302 ;
12303 ; 15/01/2024 - Retro DOS v5.0
12304 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:58D4h
12305
12306 _$STD_CON_STRING_INPUT: ;System call 10
12307 ;
12308 ; 15/01/2024
12309 ;mov ax,ss
12310 ;mov es,ax
12311 000019A6 16 push ss
12312 000019A7 07 pop es
12313 ;
12314 000019A8 89D6 mov si,dx
12315 000019AA 30ED xor ch,ch
12316 000019AC AD lodsw
12317 ;
12318 ; (AL) = the buffer length
12319 ; (AH) = the template length
12320 ;
12321 000019AD 08C0 or al,al
12322 000019AF 74E8 jz short noscan ;Buffer is 0 length!!?
12323 000019B1 88E3 mov bl,ah ;Init template counter
12324 000019B3 88EF mov bh,ch ;Init template counter
12325 ;
12326 ; (BL) = the number of bytes in the template
12327 ;
12328 000019B5 38D8 cmp al,bl
12329 000019B7 7605 jbe short NOEDIT ;If length of buffer inconsistent with contents
12330 000019B9 80380D cmp byte [bx+si],c_CR ; 0dh
12331 000019BC 7402 jz short EDITON ;If CR correctly placed EDIT is OK
12332 ;
12333 ; The number of chars in the template is >= the number of chars in buffer or
12334 ; there is no CR at the end of the template. This is an inconsistent state
12335 ; of affairs. Pretend that the template was empty:
12336 ;
12337 ;
12338 NOEDIT:
12339 000019BE 88EB mov bl,ch ;Reset buffer
12340 EDITON:
12341 000019C0 88C2 mov dl,al
12342 000019C2 4A dec dx ;DL is # of bytes we can put in the buffer
12343 ;
12344 ; Top level. We begin to read a line in.
12345 ;
12346 NEWLIN:
12347 000019C3 36A0[F901] mov al,[SS:CARPOS]
12348 000019C7 36A2[FA01] mov [SS:STARTPOS],al ;Remember position in raw buffer
12349 ;
12350 000019CB 56 push si
12351 000019CC BF[FB01] mov di,INBUF ;Build the new line here
12352 000019CF 36882E[7905] mov byte [SS:INSMODE],ch ;Insert mode off
12353 000019D4 88EF mov bh,ch ;No chars from template yet
12354 000019D6 88EE mov dh,ch ;No chars to new line yet
12355 000019D8 E867FF call _$STD_CON_INPUT_NO_ECHO ;Get first char
12356 000019DB 3C0A cmp al,c_LF ; 0Ah ;Linefeed
12357 000019DD 7503 jnz short GOTCH
12358 ;
12359 ; This is the main loop of reading in a character and processing it.
12360 ;
12361 ; (BH) = the index of the next byte in the template
12362 ; (BL) = the length of the template
12363 ; (DH) = the number of bytes in the buffer
12364 ; (DL) = the length of the buffer
12365 ;
12366 GETCH:
12367 000019DF E860FF call _$STD_CON_INPUT_NO_ECHO
12368 GOTCH:
12369 ;
12370 ; Brain-damaged Tim Patterson ignored ^F in case his BIOS did not flush the
12371 ; input queue.
12372 ;
12373 000019E2 3C06 cmp al,"F"-"@" ; CMP AL, 6 ; Ignore ^F
12374 000019E4 74F9 jz short GETCH
12375 ;
12376 ; If the leading char is the function-key lead byte
12377 ;
12378 ;cmp al,[SS:ESCCHAR]
12379 ;
12380 ; 04/05/2019 - Retro DOS v4.0
12381 ;
12382 ;hkn; ESCCHAR is in TABLE seg (DOSCODE)
12383 ;
12384 000019E6 2E3A06[940A] CMP AL,[cs:ESCCHAR]
12385 000019EB 7439 jz short ESCAPE ;change reserved keyword DBM 5-7-87

```

```

12386
12387
12388 ; Rubout and ^H are both destructive backspaces.
12389 000019ED 3C7F      cmp al,c_DEL ; 7FH
12390                ;jz short BACKSPJ
12391                ; 15/01/2024
12392 000019EF 7466      je      short BACKSP
12393 000019F1 3C08      cmp     al,c_BS ; 8
12394                ;jz short BACKSPJ
12395                ; 15/01/2024
12396 000019F3 7462      je      short BACKSP
12397
12398 ; 04/05/2019 - MSDOS 6.0, also MSDOS 6.21 has bug (bullshit) here.
12399 ; Two NOPs -instead of a JMP short, as two bytes-
12400 ; after CMP and a CMP again!
12401 ;
12402 ;
12403 ; -It would be better if they use a 'JMP short' to
12404 ; DOSCODE:5279h from DOSCODE:5271h and leave NOPs
12405 ; between them. Then, they would be able use a patch
12406 ; between 5271h and 5279h when if it will be required.
12407 ; I think Tim Patterson would not do this CMP mistake!-
12408 ; (MSDOS.SYS, from DOSCODE:5271h to DOSCODE:5279h)
12409
12410 ; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
12411 ;
12412 ; (Note: nops below might be used for patching code for windows 3.1)
12413
12414 ;DOSCODE:526D      cmp     al, 8
12415 ;DOSCODE:526F      jz      short BACKSPJ
12416 ;DOSCODE:5271      cmp     al, 17h
12417 ;DOSCODE:5273      nop
12418 ;DOSCODE:5274      nop
12419 ;DOSCODE:5275      cmp     al, 15h
12420 ;DOSCODE:5277      nop
12421 ;DOSCODE:5278      nop
12422 ;DOSCODE:5279      cmp     al, 0Dh
12423 ;DOSCODE:527B      jz      short ENDLIN
12424 ;DOSCODE:527D      cmp     al, 0Ah
12425 ;DOSCODE:527F      jz      short PHYCRLF
12426
12427 ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12428 ; DOSCODE:525Dh
12429
12430 ; 16/12/2022
12431 %if 0
12432 ; MSDOS 6.0
12433 ; ^W deletes backward once and then backs up until a letter is before the
12434 ; cursor
12435
12436 CMP     AL,"w"-"@" ; 17h
12437
12438 ; The removal of the comment characters before the jump statement will
12439 ; cause ^W to backup a word.
12440
12441 ;***JZ      short wordDel
12442 NOP
12443 NOP
12444
12445 CMP     AL,"u"-"@" ; 15h
12446
12447 ; The removal of the comment characters before the jump statement will
12448 ; cause ^U to clear a line.
12449
12450 ;***JZ      short LineDel
12451 NOP
12452 NOP
12453
12454 %endif
12455
12456 ; CR terminates the line.
12457
12458 000019F5 3C0D      cmp al,c_CR ; 0Dh
12459 000019F7 7430      jz      short ENDLIN
12460
12461 ; LF goes to a new line and keeps on reading.
12462
12463 000019F9 3C0A      cmp al,c_LF ; 0Ah
12464 000019FB 7442      jz      short PHYCRLF
12465
12466 ; ^X (or ESC) deletes the line and starts over
12467
12468 ; MSDOS 3.3
12469 ;cmp     al,[ss:CANCHAR] ; 1Bh
12470 ;jz      short KILNEW
12471
12472 ; MSDOS 6.0 (& MSDOS 6.21)
12473
12474 ;hkn;      CANCHAR is in TABLE seg (DOSCODE), so CS override
12475
12476 000019FD 2E3A06[930A] cmp al,[cs:CANCHAR] ; 1Bh
12477 00001A02 7440      jz      short KILNEW
12478
12479 ;cmp     al,CANCEL ; 1Bh ; Retro DOS v3.0
12480 ;jz      short KILNEW
12481
12482 ; Otherwise, we save the input character.
12483
12484 SAVCH:
12485 00001A04 38D6      cmp     dh,dI
12486 00001A06 7317      jnb     short BUFFUL ; buffer is full.
12487 00001A08 AA      stosb
12488 00001A09 FEC6      inc     dh ; increment count in buffer.
12489 00001A0B E8B702      call    BUFOUT ; Print control chars nicely
12490
12491 00001A0E 36803E[7905]00 cmp byte [SS:INSMODE], 0
12492 00001A14 75C9      jnz     short GETCH ; insertmode => don't advance template
12493 00001A16 38DF      cmp     bh,bI
12494 00001A18 73C5      jnb     short GETCH ; no more characters in template
12495 00001A1A 46      inc     si ; skip to next char in template
12496 00001A1B FEC7      inc     bh ; remember position in template
12497 00001A1D EBC0      jmp     short GETCH
12498
12499 ; 15/01/2024
12500 ;BACKSPJ:
12501 ;jmp     short BACKSP
12502
12503 BUFFUL:
12504 00001A1F B007      mov     al, 7 ; Bell to signal full buffer
12505 00001A21 E80D02      call    OUTT
12506 00001A24 EBB9      jmp     short GETCH
12507
12508 ESCAPE:
12509 ;transfer OEMFunctionKey

```

```

12510 00001A26 E996F0      JMP     OEMFunctionKey      ; let the OEM's handle the key dispatch
12511
12512      ENDLIN:
12513 00001A29 AA          stosb                      ; Put the CR in the buffer
12514 00001A2A E80402      call    OUTT                      ; Echo it
12515 00001A2D 5F          pop di                      ; Get start of user buffer
12516 00001A2E 8875FF      mov [di-1], dh          ; Tell user how many bytes
12517 00001A31 FEC6          inc dh                      ; DH is length including CR
12518
12519      COPYNEW:
12520          ; (IBMDOS.COM, MSDOS 2.0, STRIN.ASM)
12521          ;mov bp, es
12522          ;mov bx, ds
12523          ;mov es, bx
12524          ;mov ds, bp
12525          ;mov si, INBUF
12526          ;mov cl, dh
12527          ;rep movsb
12528          ;retn
12529
12530          ; CPMIO.ASM (MSDOS 6.0)
12531          ; (IBMDOS.COM, MSDOS 3.3, Offset 2061h)
12532          ;SAVE <DS,ES>
12533 00001A33 1E          PUSH    DS
12534 00001A34 06          PUSH    ES
12535          ;RESTORE <DS,ES>          ; XCHG ES,DS
12536 00001A35 1F          POP     DS
12537 00001A36 07          POP     ES
12538
12539      ;;hkn; INBUF is in DOSDATA
12540 00001A37 BE[FB01]      MOV     SI,INBUF
12541 00001A3A 88F1          MOV     CL,DH          ; set up count
12542 00001A3C F3A4          REP     MOVSB          ; Copy final line to user buffer
12543
12544 00001A3E C3          OLDBAK_RETN: RETN
12545
12546      ; Output a CRLF to the user screen and do NOT store it into the buffer
12547
12548      PHYCRLF:
12549          CALL    CRLF
12550 00001A42 EB9B          JMP short GETCH
12551
12552          ; MSDOS 6.0 (& MSDOS 3.3, IBMDOS.COM, 1987)
12553
12554      ; DOSCODE:52CAh (MSDOS 621, MSDOS.SYS)
12555
12556          ; Note: Following routines were not used in IBMDOS.COM
12557          ; -CTRL+W, CTRL+U is not activated-
12558          ; but they were in the kernel code!?)
12559
12560          ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12561          ; DOSCODE:52B6h
12562
12563      ;;;;;;
12564
12565      ; 16/12/2022
12566      %if 0
12567      ;
12568      ; Delete the previous line
12569      ;
12570      LineDel:
12571          OR      DH,DH
12572          JZ      short GETCH          ; 06/12/2022
12573          Call    BackSpace
12574          JMP     short LineDel
12575
12576      %endif
12577
12578      ;
12579      ; delete the previous word.
12580      ;
12581      WordDel:
12582      wordLoop:
12583          ; Call    BackSpace          ; backspace the one spot
12584          ; OR      DH,DH
12585          ; JZ      short GetChj
12586          ; MOV     AL,[ES:DI-1]
12587          ; cmp     al,'0'
12588          ; jb      short GetChj
12589          ; cmp     al,'9'
12590          ; jbe     short wordLoop
12591          ; OR      AL,20h
12592          ; CMP     AL,'a'
12593          ; JB      short GetChj
12594          ; CMP     AL,'z'
12595          ; JBE     short wordLoop
12596      ;GetChj:
12597          ; JMP     GETCH
12598
12599      ; 16/12/2022
12600      %if 0
12601          ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12602          ; (worddel is not called or jumped from anywhere!)
12603      wordDel:
12604      wordLoop:
12605          Call    BackSpace          ; backspace the one spot
12606          ; OR      DH,DH
12607          ; JZ      short GetChj
12608          ; MOV     AL,[ES:DI-1]
12609          ; cmp     al,'0'
12610          ; jb      short GetChj
12611          ; cmp     al,'9'
12612          ; jbe     short wordLoop
12613          ; OR      AL,20h
12614          ; CMP     AL,'a'
12615          ; JB      short GetChj
12616          ; CMP     AL,'z'
12617          ; JBE     short wordLoop
12618      GetChj:
12619          JMP     GETCH
12620
12621      %endif
12622
12623      ;;;;;;
12624
12625      ; DOSCODE:52F3h (MSDOS 621, MSDOS.SYS)
12626
12627      ; The user wants to throw away what he's typed in and wants to start over.
12628      ; We print the backslash and then go to the next line and tab to the correct
12629      ; spot to begin the buffered input.
12630
12631      KILNEW:
12632          mov al,'\'
12633          call    OUTT          ;Print the CANCEL indicator

```



```

12634 00001A49 5E                pop si                ;Remember start of edit buffer
12635
12636 00001A4A E81001          PUTNEW:
12637 00001A4D 36A0[FA01]      call    CRLF          ;Go to next line on screen
12638 00001A51 E85102          mov     al,[SS:STARTPOS]
12639 00001A54 E96CFF          call    TAB           ;Tab over
12640                                JMP      NEWLIN        ;Start over again
12641
12642                                ; Destructively back up one character position
12643
12644                                BACKSP:
12645 00001A57 E80800          ; 09/09/2018
12646 00001A5A EB83          call    BackSpace
12647                                JMP      short GETCH   ; 15/01/2024
12648
12649                                ; 15/01/2024
12650                                ;User really wants an ESC character in his line
12651 00001A5C 2EA0[940A]      TWOESC:
12652 00001A60 EBA2          mov     al,[cs:ESCCHAR] ; 10/06/2019
12653                                jmp     short SAVCH
12654
12655 00001A62 08F6          BackSpace:
12656 00001A64 7419          or      dh,dh
12657 00001A66 E85800          jz      short OLDBAK   ;No chars in line, do nothing to line
12658 00001A69 268A05        call    BACKUP          ;Do the backup
12659 00001A6C 3C20          mov     al,[es:di]     ;Get the deleted char
12660 00001A6E 730F          cmp     al,20h         ;
12661 00001A70 3C09          jnb     short OLDBAK   ;was a normal char
12662 00001A72 741B          cmp     al,c_HT        ;
12663                                jz      short BAKTAB     ;was a tab, fix up users display
12664 00001A74 3C15          ;; 9/27/86 fix for ctrl-U backspace
12665 00001A76 7407          CMP     AL,"U"-"@" ; 15h ; ctrl-U is a section symbol not ^U
12666 00001A78 3C14          JZ      short OLDBAK   ;
12667 00001A7A 7403          CMP     AL,"T"-"@" ; 14h ; ctrl-T is a paragraphs symbol not ^T
12668                                JZ      short OLDBAK
12669 00001A7C E84500          ;; 9/27/86 fix for ctrl-U backspace
12670                                call    BACKMES        ;was a control char, zap the '^'
12671 00001A7F 36803E[7905]00 OLDBAK:
12672 00001A85 75B7          cmp     byte [SS:INSMODE], 0
12673 00001A87 08FF          jnz     short OLDBAK_RETN ;In insert mode, done
12674 00001A89 74B3          or      bh,bh
12675                                jz      short OLDBAK_RETN
12676 00001A8B FECF          dec     bh             ;Not advanced in template, stay where we are
12677 00001A8D 4E          dec     si             ;Go back in template
12678 00001A8E C3          retn
12679
12680 00001A8F 57          BAKTAB:
12681 00001A90 4F          push     di
12682 00001A91 FD          dec     di             ;Back up one char
12683 00001A92 88F1          std     di             ;Go backward
12684 00001A94 B020          mov     cl,dh          ;Number of chars currently in line
12685 00001A96 53          mov     al,20h         ;
12686 00001A97 B307          push     bx
12687 00001A99 E30E          mov     bl,7           ;Max
12688                                jcxz     FIGTAB         ;At start, do nothing
12689 00001A9B AE          FNDPOS:
12690 00001A9C 7609          scasb                    ;Look back
12691 00001A9E 26807D0109      jbe     short CHKCNT    ;
12692 00001AA3 7409          cmp     byte [es:di+1],9
12693 00001AA5 FECB          jz      short HAVTAB    ;Found a tab
12694                                dec     bl             ;Back one char if non tab control char
12695 00001AA7 E2F2          CHKCNT:
12696                                loop     FNDPOS
12697 00001AA9 362A1E[FA01]      FIGTAB:
12698                                sub     bl,[SS:STARTPOS]
12699 00001AAE 28F3          HAVTAB:
12700 00001AB0 00D9          sub     bl,dh
12701 00001AB2 80E107        add     cl,bl
12702 00001AB5 FC          and     cl,7           ;CX has correct number to erase
12703 00001AB6 5B          cld                    ;Back to normal
12704 00001AB7 5F          pop     bx
12705 00001AB8 74C5          pop     di
12706                                jz      short OLDBAK   ;Nothing to erase
12707 00001ABA E80700          TABBAK:
12708 00001ABD E2FB          call    BACKMES
12709 00001ABF EBBE          loop   TABBAK         ;Erase correct number of chars
12710                                jmp     short OLDBAK
12711
12712 00001AC1 FECE          BACKUP:
12713 00001AC3 4F          dec     dh             ;Back up in line
12714                                dec     di
12715 00001AC4 B008          BACKMES:
12716 00001AC6 E86801        mov     al,c_BS ; 8     ;Backspace
12717 00001AC9 B020          call    OUTT
12718 00001ACB E86301        mov     al,20h ; ' '   ;Erase
12719 00001ACE B008          call    OUTT
12720 00001AD0 E95E01        mov     al,c_BS ; 8     ;Backspace
12721                                jmp     OUTT           ;Done
12722                                ; 15/01/2024
12723                                ;User really wants an ESC character in his line
12724                                ;TWOESC:
12725                                ; mov     al,[cs:ESCCHAR] ; 10/06/2019
12726                                ; jmp     SAVCH
12727
12728                                ;Copy the rest of the template
12729                                COPYLIN:
12730 00001AD3 88D9          mov     cl,bl          ;Total size of template
12731 00001AD5 28F9          sub     cl,bh          ;Minus position in template, is number to move
12732 00001AD7 EB07          jmp     short COPYEACH
12733
12734                                COPYSTR:
12735 00001AD9 E83200          call    FINDOLD         ;Find the char
12736 00001ADC EB02          jmp     short COPYEACH ;Copy up to it
12737
12738                                ;Copy one char from template to line
12739                                COPYONE:
12740 00001ADE B101          mov     cl,1
12741                                ;Copy CX chars from template to line
12742                                COPYEACH:
12743 00001AE0 36C606[7905]00      mov     byte [SS:INSMODE],0 ;All copies turn off insert mode
12744 00001AE6 38D6          cmp     dh,dh
12745 00001AE8 740F          jz      short GETCH2    ;At end of line, can't do anything
12746 00001AEA 38DF          cmp     bh,bl
12747 00001AEC 740B          jz      short GETCH2    ;At end of template, can't do anything
12748 00001AEE AC          lodsb
12749 00001AEF AA          stosb
12750 00001AF0 E8D201        call    BUFOUT
12751 00001AF3 FEC7          inc     bh             ;Ahead in template
12752 00001AF5 FEC6          inc     dh             ;Ahead in line
12753 00001AF7 E2E7          loop   COPYEACH
12754
12755 00001AF9 E9E3FE          GETCH2:
12756                                jmp     GETCH
12757                                ;Skip one char in template

```

```

12758
12759 00001AFC 38DF
12760 00001AFE 74F9
12761 00001B00 FEC7
12762 00001B02 46
12763
12764
12765 00001B03 EBF4
12766
12767
12768 00001B05 E80600
12769 00001B08 01CE
12770 00001B0A 00CF
12771
12772
12773 00001B0C EBEB
12774
12775
12776
12777
12778
12779
12780
12781
12782
12783 00001B0E E831FE
12784
12785
12786
12787
12788
12789
12790
12791
12792
12793
12794 00001B11 2E3A06[940A]
12795 00001B16 7505
12796
12797 00001B18 E827FE
12798 00001B1B EB1D
12799
12800 00001B1D 88D9
12801 00001B1F 28F9
12802 00001B21 7417
12803 00001B23 49
12804 00001B24 7414
12805 00001B26 06
12806 00001B27 1E
12807 00001B28 07
12808 00001B29 57
12809 00001B2A 89F7
12810 00001B2C 47
12811 00001B2D F2AE
12812 00001B2F 5F
12813 00001B30 07
12814 00001B31 7507
12815 00001B33 F6D1
12816 00001B35 00D9
12817 00001B37 28F9
12818
12819 00001B39 C3
12820
12821
12822 00001B3A 5D
12823
12824
12825
12826 00001B3B EBBC
12827
12828
12829 00001B3D B040
12830 00001B3F E8EF00
12831 00001B42 5F
12832 00001B43 57
12833 00001B44 06
12834 00001B45 1E
12835 00001B46 E8EAFE
12836 00001B49 1F
12837 00001B4A 07
12838 00001B4B 5E
12839 00001B4C 88F3
12840 00001B4E E9F9FE
12841
12842
12843
12844 00001B51 36F616[7905]
12845
12846
12847 00001B56 EBE3
12848
12849
12850
12851 00001B58 B01A
12852 00001B5A E9A7FE
12853
12854
12855
12856 00001B5D B00D
12857 00001B5F E8CF00
12858 00001B62 B00A
12859 00001B64 E9CA00
12860
12861
12862
12863
12864
12865
12866
12867
12868
12869
12870
12871
12872
12873
12874
12875
12876
12877
12878
12879
12880
12881

```

```

SKIPONE:
    cmp     bh,b1
    jz      short GETCH2      ;At end of template
    inc     bh                ;Ahead in template
    inc     si
    ; jmp    GETCH
    ; 15/01/2024
    jmp     short GETCH2

SKIPSTR:
    call    FINDOLD           ;Find out how far to go
    add     si,cx              ;Go there
    add     bh,c1
    ; jmp    GETCH
    ; 15/01/2024
    jmp     short GETCH2

;Get the next user char, and look ahead in template for a match
;CX indicates how many chars to skip to get there on output
;NOTE: WARNING: If the operation cannot be done, the return
; address is popped off and a jump to GETCH is taken.
; Make sure nothing extra on stack when this routine
; is called!!! (no PUSHes before calling it).

FINDOLD:
    call    _$STD_CON_INPUT_NO_ECHO
    ; STRIN.ASM (MSDOS 2.11, 19/07/2018)

    ;CMP     AL,[SS:ESCCHAR]
    ;JNZ     SHORT FINDSETUP

    ; CPMIO.ASM (MSDOS 6.0, 04/05/2019 - Retro DOS v4.0)

;hkn; ESCCHAR is in TABLE seg (DOSCODE), so CS override
    CMP     AL,[CS:ESCCHAR]    ; did he type a function key?
    JNZ     SHORT FINDSETUP    ; no, set up for scan

    CALL     _$STD_CON_INPUT_NO_ECHO    ; eat next char
    JMP     SHORT NOTFND            ; go try again

FINDSETUP:
    mov     cl,b1
    sub     cl,bh                ;CX is number of chars to end of template
    jz      short NOTFND        ;At end of template
    dec     cx                  ;Cannot point past end, limit search
    jz      short NOTFND        ;If only one char in template, forget it

    push    es
    push    ds
    pop     es
    push    di
    mov     di,si               ;Template to ES:DI
    inc     di
    repne   scasb               ;Look
    pop     di
    pop     es
    jnz     short NOTFND        ;Didn't find the char
    not     cl                  ;Turn how far to go into how far we went
    add     cl,b1               ;Add size of template
    sub     cl,bh               ;Subtract current pos, result distance to skip

FINDOLD_RETN:
    retn

NOTFND:
    pop     bp                  ;Chuck return address
    ; jmp    GETCH
    ; 15/01/2024
    GETCH2_j:
    jmp     short GETCH2

REEDIT:
    mov     al,'@'              ;Output re-edit character
    call    OUTT
    pop     di
    push    di
    push    es
    push    ds
    call    COPYNEW             ;Copy current line into template
    pop     ds
    pop     es
    pop     si
    mov     bl,dh               ;Size of line is new size template
    jmp     PUTNEW              ;Start over again

EXITINS:
ENTERINS:
    not     byte [SS:INSMODE]
    ; jmp    GETCH
    ; 15/01/2024
    jmp     short GETCH2_j

;Put a real live ^Z in the buffer (embedded)
CTRLZ:
    mov     al,"z"-"@" ; 1Ah
    jmp     SAVCH

;Output a CRLF
CRLF:
    mov     al,c_CR ; 0Dh
    call    OUTT
    mov     al,c_LF ; 0Ah
    jmp     OUTT

;-----
; ** $RAW_CON_IO - Do Raw Console I/O
;
; Input or output raw character from console, no echo
;
; ENTRY DL = -1 if input
;         = output character if output
; EXIT (AL) = input character if input
; USES all
;-----
; 20/07/2018 - Retro DOS v3.0

; 04/05/2019 - Retro DOS v4.0
; DOSCODE:541Ch (MSDOS 6.21, MSDOS.SYS)

; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:5408h (MSDOS 5.0, MSDOS.SYS)

```



```

12882 _$_RAW_CON_IO: ; System call 6
12883
12884 MOV AL,DL
12885 CMP AL,-1
12886 JNZ SHORT RAWOUT ; 16/12/2022
12887 ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12888 ;jz short rc1
12889 ;jmp short RAWOUT
12890 ; 16/12/2022
12891 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12892 ;nop
12893
12894 rc1: ; Get pointer to register save area
12895 LES DI,[SS:USER_SP] ; 12/03/2018
12896 XOR BX,BX
12897 ;CALL GET_IO_FCB ; MSDOS 2.11 (Retro DOS v2.0)
12898 CALL GET_IO_SFT ; MSDOS 3.3 & MSDOS 6.0
12899 ;JC SHORT RET17
12900 jc short FINDOLD_RETN
12901 MOV AH,1
12902 CALL IOFUNC
12903 JNZ SHORT RESFLG
12904 CALL SPOOLINT
12905 ;OR BYTE [ES:DI+16H],40H
12906 OR BYTE [ES:DI+user_env.user_F],40H ; Set user's zero flag
12907 XOR AL,AL
12908 RET17:
12909 RETN
12910
12911 RESFLG:
12912 ;AND BYTE [ES:DI+16H],0FFH-40H ; 0BFh
12913 AND BYTE [ES:DI+user_env.user_F],0FFH-40H
12914 ; Reset user's zero flag
12915 ;RILP:
12916 rc10: CALL SPOOLINT
12917
12918 ;
12919 -----
12920 ;
12921 ** $Raw_CON_INPUT - Raw Console Input
12922 ;
12923 ; Input raw character from console, no echo
12924 ;
12925 ; ENTRY none
12926 ; EXIT (al) = character
12927 ; USES al
12928 ;
12929 -----
12930 ;
12931 ;rci0: invoke SPOOLINT
12932 ;
12933 ;entry $RAW_CON_INPUT
12934 ;
12935 ; 04/05/2019 - Retro DOS v4.0
12936 ;
12937 ; DOSCODE:544Bh (MSDOS 6.21, MSDOS.SYS)
12938 ;
12939 ; 15/01/2024 - Retro DOS v5.0
12940 ;
12941 ; DOSCODE:5ACBh (PCDOS 7.1, IBMDOS.COM)
12942 ;
12943 _$_RAW_CON_INPUT: ; System call 7
12944
12945 push bx
12946 XOR BX,BX
12947 ;CALL GET_IO_FCB ; MSDOS 2.11 (Retro DOS v2.0)
12948 CALL GET_IO_SFT ; MSDOS 3.3 & MSDOS 6.0
12949 pop bx
12950 JC SHORT RET17
12951 MOV AH,1
12952 CALL IOFUNC
12953 ;JZ SHORT RILP ; MSDOS 2.11
12954 ;XOR AH,AH
12955 ;CALL IOFUNC
12956 ;RETN
12957 jnz short rc15 ; MSDOS 3.3 & MSDOS 6.0
12958 MOV AH,84h
12959 INT int_IBM ; int 2Ah
12960 JMP short rc10
12961 rc15:
12962 XOR AH,AH
12963 ;CALL IOFUNC
12964 ;RETN
12965 ; 18/12/2022
12966 jmp IOFUNC
12967
12968 ; Output the character in AL to stdout
12969 ;
12970 ;entry RAWOUT
12971 RAWOUT:
12972 PUSH BX
12973 MOV BX,1
12974
12975 ;CALL GET_IO_FCB ; MSDOS 2.11 (Retro DOS v2.0)
12976 CALL GET_IO_SFT ; MSDOS 3.3 & MSDOS 6.0
12977 JC SHORT RAWRET1
12978
12979 ;
12980 ; MSDOS 2.11
12981 ;TEST BYTE [SI+18H],080H ; output to file?
12982 ;JZ SHORT RAWNORM ; if so, do normally
12983 ;PUSH DS
12984 ;PUSH SI
12985 ;LDS SI,[SI+19H] ; output to special?
12986 ;TEST BYTE [SI+4],ISSPEC
12987 ;POP SI
12988 ;
12989 ; MSDOS 3.3 & MSDOS 6.0
12990 ;mov bx,[si+5]
12991 MOV BX,[SI+SF_ENTRY.sf_flags] ;hkn; DS set up by get_io_sft
12992 ;
12993 ; If we are a network handle OR if we are not a local device then go do the
12994 ; output the hard way.
12995 ;
12996 ;
12997 ;and bx,8080h
12998 AND BX,sf_isnet+devid_device
12999 ;cmp bx,80h
13000 CMP BX,devid_device
13001 jnz short RAWNORM
13002 push ds
13003 ;lds bx,[si+7]
13004 LDS BX,[SI+SF_ENTRY.sf_devptr] ; output to special?
13005 ;test byte [bx+4],10h

```

```

13006 00001BC8 F6470410      TEST    BYTE [BX+SYSDEV.ATT],ISSPEC
13007                          ;
13008
13009 00001BCC 1F             POP     DS
13010 00001BCD 740E          JZ      SHORT RAWNORM      ; if not, do normally
13011
13012                          ; 15/01/2024
13013                          ;INT    int_fastcon  ; int 29h; quickly output the char
13014
13015                          ; 26/06/2024
13016 00001BCF 1E           push    ds ; *
13017
13018                          ; 12/04/2024 - Retro DOS v5.0
13019                          ; (PCDOS 7.1 IBMBOS.COM)
13020 00001BD0 31DB          xor     bx,bx
13021 00001BD2 8EDB          mov     ds,bx ; 0
13022
13023                          ; 15/01/2024
13024 00001BD4 9C           pushf
13025 00001BD5 FF1EA400      call    far [29h*4]      ; simulate INT 29h
13026                          ; call far [00A4h]
13027
13028 00001BD9 1F           pop     ds ; *
13029
13030                          ;JMP     SHORT RAWRET
13031 ;RAWNORM:
13032 ;      CALL    RAWOUT3
13033 RAWRET:
13034 00001BDA F8            CLC
13035 RAWRET1:
13036 00001BDB 5B           POP     BX
13037 RAWRET2:
13038 00001BDC C3           RETN
13039 RAWNORM:
13040 00001BDD E80700        CALL    RAWOUT3
13041 00001BE0 EBF8        jmp     short RAWRET
13042
13043 ;      Output the character in AL to handle in BX
13044 ;
13045 ;      entry    RAWOUT2
13046
13047 RAWOUT2:
13048 ;CALL    GET_IO_FCB      ; MSDOS 2.11 (Retro DOS v2.0)
13049 ;JC      SHORT RET18
13050 00001BE2 E8E022        CALL    GET_IO_SFT      ; MSDOS 3.3 & MSDOS 6.0
13051 00001BE5 72F5        JC      SHORT RAWRET2
13052 RAWOUT3:
13053 00001BE7 50           PUSH    AX
13054 00001BE8 EB0C        JMP     SHORT RAWOSTRT
13055 ROLP:
13056 00001BEA E89342        CALL    SPOOLINT
13057
13058 ; 01/05/2019 - Retro DOS v4.0
13059
13060 ; MSDOS 6.0
13061 ;OR      word [ss:DOS34_FLAG],CTRL_BREAK_FLAG ; 001000000000b
13062 ; 17/12/2022
13063 00001BED 36800E[1206]02 or     byte [ss:DOS34_FLAG+1],(CTRL_BREAK_FLAG>>8) ; 02h
13064 ;or      word [ss:DOS34_FLAG],200h
13065 ;AN002; set control break
13066 ;invoke DSKSTATCHK
13067 00001BF3 E80942        call    DSKSTATCHK      ;AN002; check control break
13068 RAWOSTRT:
13069 00001BF6 B403        MOV     AH,3
13070 00001BF8 E8C934        CALL    IOFUNC
13071 00001BF8 74ED        JZ      SHORT ROLP
13072
13073 ; MSDOS 6.0
13074 ;SR;
13075 ; IOFUNC now returns ax = 0ffffh if there was an I24 on a status call and
13076 ; the user failed. We do not send a char if this happens. We however return
13077 ; to the caller with carry clear because this DOS call does not return any
13078 ; status.
13079 ;
13080 ;      inc     ax          ;fail on I24 if ax = -1
13081 00001BFE 58           POP     AX
13082 00001BFF 7405        jz      short nosend      ;yes, do not send char
13083 00001C01 B402        MOV     AH,2
13084 00001C03 E8BE34        call    IOFUNC
13085 nosend:
13086 00001C06 F8           CLC
13087 00001C07 C3           retn
13088
13089 ; MSDOS 3.3 & MSDOS 2.11
13090 ;POP     AX
13091 ;MOV     AH,2
13092 ;CALL    IOFUNC
13093 ;CLC
13094 ;      ; Clear carry indicating successful
13095 ;RET18:
13096 ;RETN
13097
13098 ;;10/08/2018
13099 ; 20/07/2018 - Retro DOS v3.0
13100 ; -----
13101 ; Retro DOS v2.0 (MSDOS 2.11) - OUTMES
13102 ; -----
13103
13104 ; This routine is called at DOS init
13105
13106 ;; ;procedure OUTMES,NEAR ; String output for internal messages
13107 ;;OUTMES:
13108 ;; ;LODS    CS:BYTE PTR [SI]
13109 ;; CS      LODSB
13110 ;; CMP     AL,"$" ; 24h
13111 ;; JZ      SHORT RET18
13112 ;; CALL    OUTT
13113 ;; JMP     SHORT OUTMES
13114
13115 ; -----
13116 ; 20/07/2018 - Retro DOS v3.0
13117
13118 ; IBMDOS.COM (MSDOS 3.3 kernel) - Offset 2252h
13119
13120 ; -----
13121 ;
13122 ;
13123 ; Inputs:
13124 ; AX=0 save the DEVCALL request packet
13125 ; =1 restore the DEVCALL request packet
13126 ; Function:
13127 ; save or restore the DEVCALL packet
13128 ; Returns:
13129 ; none

```

```

13130 ;
13131 ;-----
13132 ;
13133 ;
13134 ; 04/05/2019 - Retro DOS v4.0
13135 ; DOSCODE:54B9h (MSDOS 6.21, MSDOS.SYS)
13136 ;
13137 ; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
13138 ; DOSCODE:54A5h (MSDOS 5.0, MSDOS.SYS)
13139 ;
13140 ; 12/05/2019
13141 ;
13142 ; 15/01/2024 - Retro DOS v5.0
13143 ; DOSCODE:5B42h (PCDOS 7.1, IBMDOS.COM)
13144 ;
13145 Save_Restore_Packet:
13146     PUSH    DS
13147     PUSH    ES
13148     PUSH    SI
13149     PUSH    DI
13150 ;
13151 ; 16/12/2022
13152 ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13153 ; 09/09/2018
13154     mov     di,FAKE_STACK_2F
13155     mov     si,DEVCALL
13156 ;
13157 ; 21/09/2023
13158     or      ax,ax
13159     ;CMP     AX,0 ; save packet
13160     JZ      short save_packet ; 16/12/2022
13161     ;je      short set_seg
13162 ;
13163 ; MSDOS 6.0
13164 restore_packet:
13165     ; MOV     SI,OFFSET DOSDATA:Packet_Temp ;source
13166     ; MOV     DI,OFFSET DOSDATA:DEVCALL ;destination
13167     ; MSDOS 3.3
13168     ;mov     si,FAKE_STACK_2F ; DOS_TEMP ; Packed_Temp
13169     ;mov     di,DEVCALL ; 09/09/2018
13170 ;
13171     ;JMP     short set_seg
13172 ;
13173 ; 16/12/2022
13174 ; 09/09/2018
13175     xchg    si,di ; DI = offset DEVCALL, SI = offset FAKE_STACK_2F
13176 ;
13177 ; 16/12/2022
13178 %if 0
13179     ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13180     cmp     ax,0 ; save packet
13181     jz      short save_packet
13182     mov     si,FAKE_STACK_2F ; 07/12/2022
13183     mov     di,DEVCALL
13184     jmp     short set_seg
13185 ;
13186 ; MSDOS 6.0
13187 save_packet:
13188     ; MOV     DI,OFFSET DOSDATA:Packet_Temp ;destination
13189     ; MOV     SI,OFFSET DOSDATA:DEVCALL ;source
13190     ; 09/09/2018
13191     ; MSDOS 3.3
13192     ;mov     di,FAKE_STACK_2F ; DOS_TEMP ; Packed_Temp
13193     ;mov     si,DEVCALL ; 09/09/2018
13194 ;
13195     ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13196     mov     di,FAKE_STACK_2F ; DOS_TEMP ; Packed_Temp
13197     mov     si,DEVCALL
13198 %endif
13199 ;
13200 ; 16/12/2022
13201 save_packet:
13202 ;set_seg:
13203     ; MSDOS 3.3
13204     ;mov     ax,cs
13205 ;
13206 ; MSDOS 6.0
13207     ;MOV     AX,SS ; set DS,ES to DOSDATA
13208     ;MOV     DS,AX
13209     ;MOV     ES,AX
13210     ; 15/01/2024
13211     push    ss
13212     pop     ds
13213     push    ds
13214     pop     es
13215 ;
13216     MOV     CX,11 ; 11 words to move
13217     REP     MOVSW
13218 ;
13219     POP     DI
13220     POP     SI
13221     POP     ES
13222     POP     DS
13223     retn
13224 ;
13225 ;=====
13226 ; CPMIO2.ASM, MSDOS 6.0, 1991
13227 ;=====
13228 ; 20/07/2018 - Retro DOS v3.0
13229 ; 01/05/2019 - Retro DOS v4.0
13230 ;
13231 ;hkn; All the variables use SS override or DS. Therefore there is
13232 ;hkn; no need to specifically set up any seg regs unless SS assumption is
13233 ;hkn; not valid.
13234 ;
13235 ;
13236 ;-----
13237 ;
13238 ;** $STD_CON_INPUT - System Call 1
13239 ;
13240 ; Input character from console, echo
13241 ;
13242 ; ENTRY none
13243 ; EXIT (al) = character
13244 ; USES ALL
13245 ;
13246 ;-----
13247 ;
13248 ;
13249 _$STD_CON_INPUT: ;system call 1
13250 ;
13251     CALL    _$STD_CON_INPUT_NO_ECHO
13252     PUSH    AX
13253     CALL    OUTT

```

```

13254 00001C2D 58      POP      AX
13255 CON_INPUT_RETN:
13256 00001C2E C3      RETN
13257
13258
13259
13260
13261
13262
13263
13264
13265
13266
13267
13268
13269
13270
13271
13272
13273
13274
13275
13276
13277
13278
13279
13280
13281
13282 00001C2F 88D0
13283
13284 00001C31 3C20
13285 00001C33 725C
13286 00001C35 3C7F
13287 00001C37 7405
13288
13289
13290 00001C39 36FE06[F901]
13291
13292 00001C3E 1E
13293 00001C3F 56
13294
13295
13296 00001C40 36FE06[0003]
13297
13298
13299 00001C45 368026[0003]3F
13300 00001C4B 7505
13301
13302 00001C4D 50
13303 00001C4E E85D42
13304 00001C51 58
13305
13306 00001C52 E859FF
13307
13308 00001C55 5E
13309 00001C56 1F
13310
13311
13312
13313 00001C57 36F606[FE02]FF
13314 00001C5D 74CF
13315
13316 00001C5F 53
13317 00001C60 1E
13318 00001C61 56
13319 00001C62 B80100
13320
13321
13322
13323 00001C65 E85D22
13324
13325 00001C68 7224
13326
13327
13328
13329
13330 00001C6A 8B5C05
13331
13332
13333 00001C6D F6C780
13334 00001C70 751C
13335
13336
13337 00001C72 F6C380
13338 00001C75 7417
13339
13340
13341
13342
13343
13344
13345 00001C77 B80400
13346 00001C7A E84822
13347 00001C7D 720F
13348
13349
13350
13351 00001C7F F6440608
13352
13353
13354
13355
13356 00001C83 7503
13357 00001C85 E98700
13358
13359
13360
13361
13362 00001C88 36C606[FE02]00
13363
13364
13365
13366
13367
13368
13369 00001C8E E98100
13370
13371
13372
13373
13374
13375
13376
13377 00001C91 3C0D

;-----
; ** $STD_CON_OUTPUT - System Call 2
;
; Output character to console
;
; ENTRY (dl) = character
; EXIT  none
; USES  all
;-----

; DOSCODE:54E9h (MSDOS 6.21, MSDOS.SYS)
; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:54D5h (MSDOS 5.0, MSDOS.SYS)
; 15/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; DOSCODE:5B70h (PCDOS 7.1, IBMDOS.COM)

_$STD_CON_OUTPUT: ;system call 2
    MOV     AL,DL
OUTT:
    CMP     AL,20h ; " "
    JB      SHORT CTRLOUT
    CMP     AL,c_DEL ; 7Fh
    JZ      SHORT OUTCH
OUTCHA:
    ;INC     BYTE PTR [CARPOS]
    INC     BYTE [SS:CARPOS]
OUTCH:
    PUSH     DS
    PUSH     SI
    ;INC     BYTE PTR [CHARCO] ;invoke statchk...
    ;AND     BYTE PTR [CHARCO],00111111B ;AN000; every 64th char
    INC     BYTE [SS:CHARCO]
    ;AND     BYTE [SS:CHARCO],00111111B
    ; 01/05/2019 - Retro DOS v4.0
    and     byte [SS:CHARCO],3Fh
    JNZ     SHORT OUTSKIP
    PUSH     AX
    CALL     STATCHK
    POP      AX
OUTSKIP:
    CALL     RAWOUT ;output the character
    POP     SI
    POP     DS
    ;TEST    BYTE PTR [PFLAG],-1
    ;retz
    TEST     BYTE [SS:PFLAG],0FFh
    JZ      SHORT CON_INPUT_RETN
    PUSH     BX
    PUSH     DS
    PUSH     SI
    MOV     BX,1
    ; 20/07/2018 - Retro DOS v3.0
    ; MSDOS 3.3
    ; MSDOS 6.0 (CPMIO2.ASM)
    CALL     GET_IO_SFT ;hkn; GET_IO_SFT will set up DS:SI
    ;hkn; to sft entry
    JC      SHORT TRIPOPJ
    ; 01/05/2019 - Retro DOS v4.0
    ;mov     bx,[si+5]
    MOV     BX,[SI+SF_ENTRY.sf_flags]
    ;test    bx,8000h
    ;TEST    BX,sf_isnet ; 8000h ; output to NET?
    test     bh,(sf_isnet>>8) ; 80h
    JNZ     short TRIPOPJ ; if so, no echo
    ;;test    bx,80h
    ;TEST    BX,devid_device ; output to file?
    test     bl,devid_device ; 80h
    JZ      SHORT TRIPOPJ ; if so, no echo
    ; 14/03/2018
    ;call     GET_IO_FCB ; IBMDOS.COM, MSDOS 2.11
    ;jc      short TRIPOPJ
    ; MSDOS 2.11
    ;test     byte [SI+18H], 80h
    ;jz      short TRIPOPJ
    MOV     BX,4
    CALL     GET_IO_SFT
    JC      SHORT TRIPOPJ
    ;;test    word [si+5], 800h
    ;TEST    word [SI+SF_ENTRY.sf_flags],sf_net_spool ; 800H
    ;test     byte [si+6],8 ; 08/11/2022
    test     byte [SI+SF_ENTRY.sf_flags+1],(sf_net_spool>>8) ; 8
    ; StdPrn redirected?
    ;;JZ     SHORT LISSTRT2J ; No, OK to echo
    ;jz      LISSTRT2 ; 10/08/2021
    ; 16/12/2022
    jnz     short outch1
    jmp     LISSTRT2
    ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
    ;jz      short LISSTRT2J
outch1:
    ;MOV     BYTE [PFLAG],0
    MOV     BYTE [SS:PFLAG],0 ; If a spool, NEVER echo
    ; MSDOS 2.11
    ;mov     bx,4
    ;jmp     short LISSTRT2
TRIPOPJ:
    ; 20/07/2018
    JMP     TRIPOP
    ; 16/12/2022
    ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;LISSTRT2J:
; JMP     LISSTRT2
CTRLOUT:
    CMP     AL,c_CR ; 0Dh

```

```

13378 00001C93 7420      JZ      SHORT ZERPOS
13379 00001C95 3C08      CMP     AL,C_BS ; 8
13380 00001C97 7424      JZ      SHORT BACKPOS
13381 00001C99 3C09      CMP     AL,C_HT ; 9
13382 00001C9B 75A1      JNZ     SHORT OUTCH
13383                      ;MOV     AL,[CARPOS]
13384 00001C9D 36A0[F901]  MOV     AL,[SS:CARPOS]
13385 00001CA1 0CF8      OR      AL,0F8H
13386 00001CA3 F6D8      NEG     AL
13387
13388 00001CA5 51      TAB:    PUSH    CX
13389 00001CA6 88C1      MOV     CL,AL
13390 00001CA8 B500      MOV     CH,0
13391 00001CAA E307      JCXZ    POPTAB
13392
13393 00001CAC B020      TABLP:  MOV     AL," "
13394 00001CAE E880FF  CALL    OUTT
13395 00001CB1 E2F9      LOOP    TABLP
13396
13397 00001CB3 59      POPTAB: POP     CX
13398
13399 00001CB4 C3      RETN
13400
13401
13402 ZERPOS:  ;MOV     BYTE PTR [CARPOS],0
13403 00001CB5 36C606[F901]00  MOV     BYTE [SS:CARPOS],0
13404                      ; 10/08/2018
13405 00001CBB EB81      JMP     short OUTCH ; 04/05/2019
13406
13407                      ; 18/12/2022
13408
13409 ;OUTJ:
13410 ;JMP     OUTT
13411
13412 BACKPOS:
13413 00001CBD 36FE0E[F901]  ;DEC     BYTE PTR [CARPOS]
13414 00001CC2 E979FF  DEC     BYTE [SS:CARPOS]
13415                      JMP     OUTCH
13416
13417 00001CC5 3C20      BUFOUT:  CMP     AL," "
13418 00001CC7 7315      JAE     SHORT OUTJ           ;Normal char
13419 00001CC9 3C09      CMP     AL,9
13420 00001CCB 7411      JZ      SHORT OUTJ           ;OUT knows how to expand tabs
13421                      ;DOS 3.3 7/14/86
13422 00001CCD 3C15      CMP     AL,"U"-"@" ; 15h      ; turn ^U to section symbol
13423 00001CCF 740D      JZ      short CTRLU
13424 00001CD1 3C14      CMP     AL,"T"-"@" ; 14h      ; turn ^T to paragraph symbol
13425 00001CD3 7409      JZ      short CTRLU
13426
13427 NOT_CTRLU:
13428 00001CD5 50      ;DOS 3.3 7/14/86
13429 00001CD6 B05E      PUSH    AX
13430 00001CD8 E856FF  MOV     AL,"^"
13431 00001CDB 58      CALL    OUTT           ;Print '^' before control chars
13432 00001CDC 0C40      POP     AX
13433                      OR      AL,40H           ;Turn it into Upper case mate
13434
13435 CTRLU:
13436 ;CALL    OUTT
13437 ; 18/12/2022
13438 OUTJ:
13439 jmp     OUTT
13440 ;BUFOUT_RETN:
13441 ;RETN
13442
13443 ;
13444 ;-----
13445 ;** $STD_AUX_INPUT - System Call 3
13446 ;
13447 ; $STD_AUX_INPUT returns a character from Aux Input
13448 ;
13449 ; ENTRY   none
13450 ; EXIT    (al) = character
13451 ; USES    all
13452 ;-----
13453 ;
13454 ; 08/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
13455
13456 _$STD_AUX_INPUT: ;system call 3
13457
13458 CALL     STATCHK
13459 00001CE1 E8CA41  MOV     BX,3
13460 00001CE4 BB0300  CALL    GET_IO_SFT ; 20/07/2018 - MSDOS 3.3 (MSDOS 6.0)
13461 00001CE7 E8DB21  ;CALL   GET_IO_FCB ; 14/03/2018 - MSDOS 2.11
13462                      ;retc
13463                      ; 16/12/2022
13464                      ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13465                      ;JC      SHORT BUFOUT_RETN
13466                      ;JMP     SHORT TAISTRRT
13467                      ; 07/12/2022
13468                      jnc     SHORT TAISTRRT
13469 00001CEA 7304      jmp     SHORT TAISTRRT
13470 00001CEC C3      retn
13471
13472 AUXILP:
13473 00001CED E89041  CALL    SPOOLINT
13474
13475 00001CF0 B401      TAISTRRT: MOV     AH,1
13476 00001CF2 E8CF33  CALL    IOFUNC
13477 00001CF5 74F6      JZ      SHORT AUXILP
13478 00001CF7 30E4      XOR     AH,AH
13479                      ; 16/12/2022
13480                      ;CALL    IOFUNC
13481                      ;RETN
13482                      ; 07/12/2022
13483 00001CF9 E9C833  jmp     IOFUNC
13484
13485 ;
13486 ;-----
13487 ;** $STD_AUX_OUTPUT - Output character to AUX
13488 ;
13489 ; ENTRY   (dl) = character
13490 ; EXIT    none
13491 ; USES    all
13492 ;-----
13493 ;
13494 _$STD_AUX_OUTPUT: ;system call 4
13495
13496 PUSH     BX
13497 MOV      BX,3
13498 JMP      SHORT SENDOUT
13499 00001CFC 53
13500 00001CFD BB0300  PUSH    BX,3
13501 00001D00 EB04      JMP     SHORT SENDOUT

```

```

13502
13503
13504
13505
13506
13507
13508
13509
13510
13511
13512
13513
13514
13515
13516
13517 00001D02 53
13518 00001D03 BB0400
13519
13520 00001D06 88D0
13521 00001D08 50
13522 00001D09 E8A241
13523 00001D0C 58
13524 00001D0D 1E
13525 00001D0E 56
13526
13527 00001D0F E8D0FE
13528
13529 00001D12 5E
13530 00001D13 1F
13531 00001D14 5B
13532
13533 00001D15 C3
13534
13535
13536
13537
13538
13539
13540
13541
13542
13543
13544
13545
13546
13547
13548
13549
13550 00001D16 E89541
13551 00001D19 B000
13552
13553 00001D1B 74F8
13554
13555
13556 00001D1D 48
13557
13558 00001D1E C3
13559
13560
13561
13562
13563
13564
13565
13566
13567
13568
13569
13570
13571
13572
13573
13574
13575
13576 00001D1F 50
13577 00001D20 52
13578 00001D21 31DB
13579 00001D23 E89F21
13580
13581 00001D26 7205
13582 00001D28 B404
13583 00001D2A E89733
13584
13585
13586 00001D2D 5A
13587 00001D2E 58
13588 00001D2F 88C4
13589 00001D31 3C01
13590 00001D33 7413
13591 00001D35 3C06
13592 00001D37 740F
13593 00001D39 3C07
13594 00001D3B 740B
13595 00001D3D 3C08
13596 00001D3F 7407
13597 00001D41 3C0A
13598 00001D43 7403
13599 00001D45 B000
13600 00001D47 C3
13601
13602
13603 00001D48 FA
13604
13605 00001D49 E929E6
13606
13607
13608
13609
13610
13611
13612
13613
13614
13615
13616
13617
13618
13619
13620
13621
13622
13623
13624
13625

```

```

;
;-----
;
; ** $STD_PRINTER_OUTPUT - Output character to printer
;
; ENTRY (dl) = character
; EXIT none
; USES al
;
;-----
;
;_STD_PRINTER_OUTPUT: ;System call 5
;
; PUSH BX
; MOV BX,4
SENDOUT:
; MOV AL,DL
; PUSH AX
; CALL STATCHK
; POP AX
; PUSH DS
; PUSH SI
LISSTR2:
; CALL RAWOUT2
TRIPOP:
; POP SI
; POP DS
; POP BX
SCIS_RETN: ; 20/07/2018
; RETN
;
;-----
;
; ** $STD_CON_INPUT_STATUS - System Call 11
;
; Check console input status
;
; ENTRY none
; EXIT AL = -1 character available, = 0 no character
; USES al
;
;-----
;
;_STD_CON_INPUT_STATUS: ;System call 11
;
; CALL STATCHK
; MOV AL,0 ; no xor!!
; retz
; JZ SHORT SCIS_RETN ; 15/04/2018
; OR AL,-1
; ; 15/01/2024 (PCDOS 7.1 IBMDOS.COM)
; dec ax ; al = -1
; SCIS_RETN:
; RETN
;
;-----
;
; ** $STD_CON_INPUT_FLUSH - System Call 12
;
; Flush console input buffer and perform call in AL
;
; ENTRY (AL) = DOS function to be called after flush (1,6,7,8,10)
; EXIT (al) = 0 iff (al) was not one of the supported fcns
; return arguments for the fcn supplied in (AL)
; USES al
;
;-----
;
;_STD_CON_INPUT_FLUSH: ;system call 12
;
; PUSH AX
; PUSH DX
; XOR BX,BX
; CALL GET_IO_SFT ; 20/07/2018 - MSDOS 3.3 (MSDOS 6.0)
; CALL GET_IO_FCB ; 14/03/2018 - MSDOS 2.11
; JC SHORT BADJFNCON
; MOV AH,4
; CALL IOFUNC
BADJFNCON:
; POP DX
; POP AX
; MOV AH,AL
; CMP AL,1
; JZ SHORT REDISPJ
; CMP AL,6
; JZ SHORT REDISPJ
; CMP AL,7
; JZ SHORT REDISPJ
; CMP AL,8
; JZ SHORT REDISPJ
; CMP AL,10
; JZ SHORT REDISPJ
; MOV AL,0
; RETN
REDISPJ:
; CLI
; ;transfer REDISP
; JMP REDISP
;
;=====
; FCBIO.ASM, MSDOS 6.0, 1991
;=====
; 20/07/2018 - Retro DOS v3.0
; 17/05/2019 - Retro DOS v4.0
;
; ** FCBIO.ASM - Ancient 1.0 1.1 FCB system calls
;
; $GET_FCB_POSITION
; $FCB_DELETE
; $GET_FCB_FILE_LENGTH
; $FCB_CLOSE
; $FCB_RENAME
; SaveFCBInfo
; ResetLRU
; SetOpenAge
; LRUFCB
; FCBRegen
; BlastSFT

```



```

13626 ; CheckFCB
13627 ; SFTFromFCB
13628 ; FCBHardErr
13629 ;
13630 ; Revision history:
13631 ;
13632 ;         Created: ARR 4 April 1983"
13633 ;         MZ 6 June 1983 completion of functions
13634 ;         MZ 15 Dec 1983 Brain damaged programs close FCBs multiple
13635 ;         times. Change so successive closes work by
13636 ;         always returning OK. Also, detect I/O to
13637 ;         already closed FCB and return EOF.
13638 ;         MZ 16 Jan 1984 More braindamage. Need to separate info
13639 ;         out of sft into FCB for reconnection
13640 ;
13641 ;         A000 version 4.00 Jan. 1988
13642 ;
13643 ;Break <$Get_FCB_Position - set random record fields to current pos>
13644 ;
13645 ;-----
13646 ; $Get_FCB_Position - look at an FCB, retrieve the current position from the
13647 ; extent and next record field and set the random record field to point
13648 ; to that record
13649 ;
13650 ; Inputs: DS:DX point to a possible extended FCB
13651 ; Outputs: The random record field of the FCB is set to the current record
13652 ; Registers modified: all
13653 ;
13654 ;-----
13655 ;
13656 ;
13657 ;_$GET_FCB_POSITION:
13658 ; call GetExtended ; point to FCB
13659 ; call GetExtent ; DX:AX is current record
13660 ; mov [si+21h],ax
13661 ; mov [SI+SYS_FCB.RR],AX ; drop in low order piece
13662 ; mov [si+23h],dl
13663 ; mov [SI+SYS_FCB.RR+2],DL ; drop in high order piece
13664 ; cmp word [si+0Eh],64
13665 ; cmp word [SI+SYS_FCB.RECSIZ],64
13666 ; jae short GetFCBBye
13667 ; mov [si+24h],dh
13668 ; mov [SI+SYS_FCB.RR+2+1],DH; Set 4th byte only if record size < 64
13669 ; GoodPath: ; 16/12/2022
13670 ; GetFCBBye:
13671 ; jmp FCB_RET_OK
13672 ;
13673 ;Break <$FCB_Delete - remove several files that match the input FCB>
13674 ;
13675 ;-----
13676 ; ** $FCB_Delete - Delete from FCB Template
13677 ;
13678 ; given an FCB, remove all directory entries in the current
13679 ; directory that have names that match the FCB's ? marks.
13680 ;
13681 ; ENTRY (DS:DX) = address of FCB
13682 ; EXIT entries matching the FCB are deleted
13683 ; (al) = ff iff no entries were deleted
13684 ; USES all
13685 ;
13686 ;-----
13687 ;
13688 ; ; 08/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
13689 ;
13690 ;_$FCB_DELETE: ; System call 19
13691 ; ; OpenBuf is in DOSDATA
13692 ; MOV DI,OPENBUF ; appropriate place
13693 ;
13694 ; call TransFCB ; convert FCB to path
13695 ; JC short BadPath ; signal no deletions
13696 ;
13697 ; push SS
13698 ; pop DS ; SS is DOSDATA
13699 ;
13700 ; call DOS_DELETE ; wham
13701 ; JC short BadPath
13702 ; ; 16/12/2022
13703 ; jnc short GoodPath
13704 ; GoodPath:
13705 ; ; jmp FCB_RET_OK ; do a good return
13706 ; ; ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13707 ; jmp short GetFCBBye
13708 ;
13709 ; BadPath:
13710 ; ; Error code is in AX
13711 ;
13712 ; jmp FCB_RET_ERR ; let someone else signal the error
13713 ;
13714 ;Break <$Get_FCB_File_Length - return the length of a file>
13715 ;
13716 ;-----
13717 ; $Get_FCB_File_Length - set the random record field to the length of the
13718 ; file in records (rounded up if partial).
13719 ;
13720 ; Inputs: DS:DX - point to a possible extended FCB
13721 ; Outputs: Random record field updated to reflect the number of records
13722 ; Registers modified: all
13723 ;
13724 ;-----
13725 ;
13726 ; ; 15/01/2024 - Retrodos v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
13727 ;
13728 ;_$GET_FCB_FILE_LENGTH:
13729 ;
13730 ; call GetExtended ; get real FCB pointer
13731 ; ; DX points to Input FCB
13732 ;
13733 ; MOV DI,OPENBUF ; OpenBuf is in DOSDATA
13734 ; ; appropriate buffer
13735 ;
13736 ; push ds ; save pointer to true FCB
13737 ; push si
13738 ; call TransFCB ; Trans name DS:DX, sets ATTRIB
13739 ; pop si
13740 ; pop ds
13741 ; JC short BadPath
13742 ; push ds ; save pointer
13743 ; push si
13744 ; push ss
13745 ; pop ds
13746 ; call GET_FILE_INFO ; grab the info
13747 ; pop si ; get pointer back
13748 ; pop ds
13749 ; JC short BadPath ; invalid something

```

```

13750             ; 15/01/2024
13751             ;MOV    DX,BX (*)           ; get high order size
13752             ;MOV    AX,DI (**)          ; get low order size
13753 00001D90 89D8             mov     ax,bx ; hw of file size
13754             ;
13755             ;mov     bx,[si+0Eh]
13756 00001D92 8B5C0E          MOV     BX,[SI+SYS_FCB.RECSIZ]; get his record size
13757 00001D95 09DB             OR      BX,BX           ; empty record => 0 size for file
13758 00001D97 7502             JNZ     short GetSize       ; not empty
13759             ;MOV     BX,128
13760 00001D99 B380             mov     b1,128 ; 15/01/2024
13761             GetSize:
13762             ; 15/01/2024
13763             ;MOV     DI,AX               ; save low order word
13764             ;MOV     AX,DX               ; move high order for divide
13765             ;xchg    ax,dx ; (*)
13766             ; ax = hw of file size
13767             ;
13768 00001D9B 31D2             XOR     DX,DX           ; clear out high
13769 00001D9D F7F3             DIV     BX             ; wham
13770 00001D9F 50              PUSH    AX             ; save dividend
13771 00001DA0 89F8             MOV     AX,DI ; (**)    ; get low order piece
13772 00001DA2 F7F3             DIV     BX             ; wham
13773 00001DA4 89D1             MOV     CX,DX          ; save remainder
13774 00001DA6 5A              POP     DX             ; get high order dividend
13775 00001DA7 E306             JCXZ    LengthStore    ; no roundup
13776 00001DA9 83C001          ADD     AX,1
13777 00001DAC 83D200          ADC     DX,0           ; 32-bit increment
13778             LengthStore:
13779             ;mov     [si+21h],ax
13780 00001DAF 894421          MOV     [SI+SYS_FCB.RR],AX ; store low order
13781             ;mov     [si+23h],d1
13782 00001DB2 885423          MOV     [SI+SYS_FCB.RR+2],DL ; store high order
13783 00001DB5 08F6             OR      DH,DH
13784 00001DB7 74A8             JZ      short GoodPath  ; not storing insignificant zero
13785             ;mov     [si+24h],dh
13786 00001DB9 887424          MOV     [SI+SYS_FCB.RR+3],DH ; save that high piece
13787             ; 16/12/2022
13788             GoodRet:
13789             ;jmp     FCB_RET_OK
13790 00001DBC EBA3             jmp     short GoodPath
13791             ;
13792             ;Break <$FCB_Close - close a file>
13793             ;-----
13794             ;
13795             ; $FCB_Close - given an FCB, look up the SFN and close it. Do not free it
13796             ; as the FCB may be used for further I/O
13797             ;
13798             ; Inputs: DS:DX point to FCB
13799             ; Outputs: AL = FF if file was not found on disk
13800             ; Registers modified: all
13801             ;-----
13802             ;
13803             ;
13804             ; 16/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
13805             ;
13806             _$FCB_CLOSE:           ; System call 16
13807             ;
13808 00001DBE 30C0             XOR     AL,AL           ; default search attributes
13809 00001DC0 E88604          call    GetExtended     ; DS:SI point to real FCB
13810 00001DC3 7403             JZ      short NoAttr     ; not extended
13811 00001DC5 8A44FF          MOV     AL,[SI-1]        ; get attributes
13812             NoAttr:
13813             ; SS override
13814 00001DC8 36A2[6B05]       MOV     [SS:ATTRIB],AL    ; stash away found attributes
13815 00001DCC E8CD03          call    SFTFromFCB      ;
13816 00001DCF 72EB             JC      short GoodRet    ; MZ 16 Jan Assume death
13817             ;
13818             ; If the sharer is present, then the SFT is not regenable. Thus,
13819             ; there is no need to set the SFT's attribute.
13820             ;
13821             ;;; 9/8/86 F.C. save SFT attribute and restore it back when close is
13822             ;;; done
13823             ;
13824             ;mov     al,[es:di+4]
13825 00001DD1 268A4504        MOV     AL,[ES:DI+SF_ENTRY.sf_attr]
13826 00001DD5 30E4             XOR     AH,AH
13827 00001DD7 50              PUSH    AX
13828             ;
13829             ;;; 9/8/86 F.C. save SFT attribute and restore it back when close is
13830             ;;; done
13831             ;
13832 00001DD8 E85A66          call    CheckShare
13833 00001ddb 7508             JNZ     short NoStash
13834 00001ddd 36A0[6B05]       MOV     AL,[SS:ATTRIB]
13835             ;mov     [es:di+4],al
13836 00001DE1 26884504        MOV     [ES:DI+SF_ENTRY.sf_attr],AL ; attempted attribute for close
13837             NoStash:
13838             ;
13839             ; 16/01/2024
13840             %if 0
13841             ;mov     ax,[si+14h]
13842             MOV     AX,[SI+SYS_FCB.FDATE] ; move in the time and date
13843             ;mov     [es:di+0Fh],ax
13844             MOV     [ES:DI+SF_ENTRY.sf_date],AX
13845             ;mov     ax,[si+16h]
13846             MOV     AX,[SI+SYS_FCB.FTIME]
13847             ;mov     [es:di+0Dh],ax
13848             MOV     [ES:DI+SF_ENTRY.sf_time],AX
13849             ;mov     ax,[si+10h]
13850             MOV     AX,[SI+SYS_FCB.FILSIZ]
13851             ;mov     [es:di+11h],ax
13852             MOV     [ES:DI+SF_ENTRY.sf_size],AX
13853             ;mov     ax,[si+12h]
13854             MOV     AX,[SI+SYS_FCB.FILSIZ+2]
13855             ;mov     [es:di+13h],ax
13856             MOV     [ES:DI+SF_ENTRY.sf_size+2],AX
13857             ;or      word [es:di+5],4000h
13858             ; 17/12/2022
13859             or      byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_close_nodate>>8) ; 40h
13860             ;OR      word [ES:DI+SF_ENTRY.sf_flags],sf_close_nodate
13861             %else
13862             ; 16/01/2024 (PCDOS 7.1 IBMDOS.COM)
13863 00001DE5 1E              push    ds
13864             ;lds     ax,[si+14h]
13865 00001DE6 C54414          lds     ax,[si+SYS_FCB.FDATE] ; move in the time and date
13866             ;mov     [es:di+0Fh],ax
13867 00001DE9 2689450F        mov     [es:di+SF_ENTRY.sf_date],ax
13868             ;mov     [es:di+0Dh],ds
13869 00001DED 268C5D0D        mov     [es:di+SF_ENTRY.sf_time],ds
13870 00001DF1 1F              pop     ds
13871             ;lds     ax,[si+10h]
13872 00001DF2 C54410          lds     ax,[si+SYS_FCB.FILSIZ]
13873             ;mov     [es:di+11h],ax

```



```

13874 00001DF5 26894511      mov     [es:di+SF_ENTRY.sf_size],ax
13875                        ;mov     [es:di+13h],ds
13876 00001DF9 268C5D13      mov     [es:di+SF_ENTRY.sf_size+2],ds
13877                        ; 16/01/2024
13878                        ;;or     word [es:di+5],4000h
13879                        ;or     word [es:di+SF_ENTRY.sf_flags],sf_close_nodate
13880 00001DFD 26804D0640      or      byte [es:di+SF_ENTRY.sf_flags+1],(sf_close_nodate>>8) ; 40h
13881                        %endif
13882
13883 00001E02 16                push    ss
13884 00001E03 1F                pop     ds
13885 00001E04 E81D19          call   DOS_CLOSE      ; wham
13886 00001E07 C43E[9E05]      les     di,[THISSFT]
13887
13888                        ;;; 9/8/86 F.C. restore SFT attribute
13889 00001E0B 59                POP     CX
13890                        ;mov     [es:di+4],cl
13891 00001E0C 26884D04      MOV     [ES:DI+SF_ENTRY.sf_attr],CL
13892                        ;;; 9/8/86 F.C. restore SFT attribute
13893
13894 00001E10 9C                PUSHF
13895                        ;test    word [es:di],0FFFFh
13896                        ;cmp     word [ES:DI+SF_ENTRY.sf_ref_count],0
13897                        ; zero ref count gets blasted
13898 00001E11 26833D00      cmp     word [ES:DI],0
13899 00001E15 7507                jnz     short CloseOK
13900 00001E17 50                PUSH    AX
13901 00001E18 B04D          MOV     AL,'M' ; 4Dh
13902 00001E1A E8FA02          call   BlastSFT
13903 00001E1D 58                POP     AX
13904
13905 00001E1E 9D                CloseOK: POPF
13906 00001E1F 739B          JNC     short GoodRet
13907                        ;cmp     al,6
13908 00001E21 3C06          CMP     AL,error_invalid_handle
13909 00001E23 7497          JZ      short GoodRet
13910                        ;mov     al,2
13911 00001E25 B002          MOV     AL,error_file_not_found
13912
13913                        fren90:
13914                        ; 16/12/2022
13915 00001E27 E963E8      fcb_close_err: jmp     FCB_RET_ERR
13916
13917                        ;
13918                        ;
13919                        ;-----
13920                        ;** $FCB_Rename - Rename a File
13921                        ;
13922                        ; $FCB_Rename - rename a file in place within a directory. Renames
13923                        ; multiple files copying from the meta characters.
13924                        ;
13925                        ; ENTRY DS:DX point to an FCB. The normal name field is the source
13926                        ; name of the files to be renamed. Starting at offset 11h
13927                        ; in the FCB is the destination name.
13928                        ; EXIT AL = 0 -> no error occurred and all files were renamed
13929                        ; AL = FF -> some files may have been renamed but:
13930                        ; rename to existing file or source file not found
13931                        ; USES ALL
13932                        ;
13933                        ;-----
13934                        ;
13935                        ; 08/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
13936                        ; 16/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
13937
13938                        _$FCB_RENAME: ; System call 23
13939
13940 00001E2A E81C04          call   GetExtended      ; get pointer to real FCB
13941 00001E2D 52                push    dx
13942 00001E2E 8A04          MOV     AL,[SI]         ; get drive byte
13943 00001E30 83C610          ADD     SI,10h          ; point to destination
13944
13945                        ; RenBuf is in DOSDATA
13946 00001E33 BF[3E04]      MOV     DI,RENBUF        ; point to destination buffer
13947 00001E36 FF34          push    word [SI]
13948 00001E38 1E                push    ds
13949                        ;push    di ; save source pointer for TransFCB
13950                        ; 16/01/2024 (Retro DOS v4 BugFix!)
13951 00001E39 56                push    si
13952 00001E3A 8804          MOV     [SI],AL          ; drop in real drive
13953 00001E3C 89F2          MOV     DX,SI           ; let TransFCB know where the FCB is
13954 00001E3E E8B65D          call   TransFCB        ; munch this pathname
13955 00001E41 5E                pop     si
13956 00001E42 1F                pop     ds
13957 00001E43 8F04          pop     WORD [SI]        ; get path back
13958 00001E45 5A                pop     dx               ; Original FCB pointer
13959 00001E46 72DF          JC      short fren90    ; bad path -> error
13960
13961                        ; SS override for WFP_Start & Ren_WFP
13962 00001E48 368B36[B205]      MOV     SI,[ss:WFP_START] ; get pointer
13963 00001E4D 368936[B405]      MOV     [ss:REN_WFP],SI  ; stash it
13964
13965                        ; OpenBuf is in DOSDATA
13966 00001E52 BF[BE03]      MOV     DI,OPENBUF       ; appropriate spot
13967 00001E55 E89F5D          call   TransFCB        ; wham
13968                        ; NOTE that this call is pointing
13969                        ; back to the ORIGINAL FCB so
13970                        ; ATTRIB gets set correctly
13971 00001E58 72CD          JC      short fren90    ; error
13972
13973                        ;;;
13974                        ; 16/01/2024 - Retro DOS 5.0
13975                        ; (PCDOS 7.1 IBMDOS.COM)
13976 00001E5A 36C606[5411]43      mov     byte [ss:PATHNAMELEN], 67 ; DIRSTRLEN = 67
13977                        ;mov     word [ss:PATHNAMELEN], 67 ; set pathname length to 67
13978                        ;;;
13979 00001E60 E8130F          call   DOS_RENAME
13980 00001E63 72C2          JC      short fren90
13981 00001E65 E922E8      ; 16/12/2022
13982                        jmp     FCB_RET_OK
13983
13984                        ; Error -
13985                        ;
13986                        ; (al) = error code
13987
13988                        ; 16/12/2022
13989 00001E68 72C2          ;fren90: jmp     FCB_RET_ERR
13990                        ; ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13991                        ; jmp     short fcb_close_err
13992
13993                        ;Break <Misbehavior fixers>
13994                        ;
13995                        ; FCBS suffer from several problems. First, they are maintained in the
13996                        ; user's space so he may move them at will. Second, they have a small
13997                        ; reserved area that may be used for system information. Third, there was

```

```

13998 ; never any "rules for behavior" for FCBs; there was no protocol for their
13999 ; usage.
14000 ;
14001 ; This results in the following misbehavior:
14002 ;
14003 ; infinite opens of the same file:
14004 ;
14005 ; while (TRUE) { while (TRUE) {
14006 ;     FCBOpen (FCB);     FCBOpen (FCB);
14007 ;     Read (FCB);        write (FCB);
14008 ; } }
14009 ;
14010 ; infinite opens of different files:
14011 ;
14012 ; while (TRUE) { while (TRUE) {
14013 ;     FCBOpen (FCB[i++]); FCBOpen (FCB[i++]);
14014 ;     Read (FCB);        write (FCB);
14015 ; } }
14016 ;
14017 ; multiple closes of the same file:
14018 ;
14019 ; FCBOpen (FCB);
14020 ; while (TRUE)
14021 ;     FCBClose (FCB);
14022 ;
14023 ; I/O after closing file:
14024 ;
14025 ; FCBOpen (FCB);
14026 ; while (TRUE) {
14027 ;     FCBWrite (FCB);
14028 ;     FCBClose (FCB);
14029 ; }
14030 ;
14031 ; The following is an implementation of a methodology for emulating the
14032 ; above with the exception of I/O after close. We are NOT attempting to
14033 ; resolve that particular misbehavior. We will enforce correct behaviour in
14034 ; FCBs when they refer to a network file or when there is file sharing on
14035 ; the local machine.
14036 ;
14037 ; The reserved fields of the FCB (10 bytes worth) is divided up into various
14038 ; structures depending on the file itself and the state of operations of the
14039 ; OS. The information contained in this reserved field is enough to
14040 ; regenerate the SFT for the local non-shared file. It is assumed that this
14041 ; regeneration procedure may be expensive. The SFT for the FCB is
14042 ; maintained in a LRU cache as the ONLY performance improvement.
14043 ;
14044 ; No regeneration of SFTs is attempted for network FCBs.
14045 ;
14046 ; To regenerate the SFT for a local FCB, it is necessary to determine if the
14047 ; file sharer is working. If the file sharer is present then the SFT is not
14048 ; regenerated.
14049 ;
14050 ; Finally, if there is no local sharing, the full name of the file is no
14051 ; longer available. We can make up for this by using the following
14052 ; information:
14053 ;
14054 ; The Drive number (from the DPB).
14055 ; The physical sector of the directory that contains the entry.
14056 ; The relative position of the entry in the sector.
14057 ; The first cluster field.
14058 ; The last used SFT.
14059 ; OR In the case of a device FCB
14060 ; The low 6 bits of sf_flags (indicating device type)
14061 ; The pointer to the device header
14062 ;
14063 ; We read in the particular directory sector and examine the indicated
14064 ; directory entry. If it matches, then we are kosher; otherwise, we fail.
14065 ;
14066 ; Some key items need to be remembered:
14067 ;
14068 ; Even though we are caching SFTs, they may contain useful sharing
14069 ; information. We enforce good behavior on the FCBs.
14070 ;
14071 ; Network support must not treat FCBs as impacting the ref counts on
14072 ; open VCs. The VCs may be closed only at process termination.
14073 ;
14074 ; If this is not an installed version of the DOS, file sharing will
14075 ; always be present.
14076 ;
14077 ; We MUST always initialize lstclus to = firclus when regenerating a
14078 ; file. Otherwise we start allocating clusters up the wazoo.
14079 ;
14080 ; Always initialize, during regeneration, the mode field to both isFCB
14081 ; and open_for_both. This is so the FCB code in the sharer can find the
14082 ; proper OI record.
14083 ;
14084 ; The test bits are:
14085 ;
14086 ; 00 -> local file
14087 ; 40 -> sharing local
14088 ; 80 -> network
14089 ; C0 -> local device
14090 ;
14091 ;Break <SaveFCBInfo - store pertinent information from an SFT into the FCB>
14092 ;-----
14093 ;
14094 ; SaveFCBInfo - given an FCB and its associated SFT, copy the relevant
14095 ; pieces of information into the FCB to allow for subsequent
14096 ; regeneration. Poke LRU also.
14097 ;
14098 ; Inputs: ThisSFT points to a complete SFT.
14099 ;         DS:SI point to the FCB (not an extended one)
14100 ; Outputs: The relevant reserved fields in the FCB are filled in.
14101 ;          DS:SI preserved
14102 ;          ES:DI point to sft
14103 ; Registers modified: All
14104 ;
14105 ;-----
14106 ;
14107 ;
14108 ; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
14109 ; 20/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
14110 ;
14111 SaveFCBInfo:
14112 ;
14113 LES     DI,[SS:THISSFT]          ; SS override
14114 call    ISSFTNet
14115 JZ      short SaveLocal         ; if not network then save local info
14116 ;
14117 ;----- In net support -----
14118 ;
14119 ; 17/05/2019 - Retro DOS v4.0
14120 ;
14121 ; MSDOS 3.3

```

```

14122      ;mov ax,[es:di+1Dh]
14123      ;mov ax,[es:di+SF_ENTRY.sf_dirsec]
14124      ;mov [si+1Ah],ax
14125      ;mov [si+fcbl_net_handle],ax
14126      ;push es
14127      ;push di
14128      ;les di,[es:di+19h]
14129      ;LES DI,[ES:DI+sf_netid]
14130      ;mov [si+1Ch],di
14131      ;MOV [SI+fcbl_netID],DI      ; save net ID
14132      ;mov [si+1Eh],es
14133      ;MOV [SI+fcbl_netID+2],ES
14134      ;pop di
14135      ;pop es
14136
14137      ; MSDOS 6.0
14138      ;mov ax,[es:di+0Bh]
14139 00001E72 268B450B      MOV AX,[ES:DI+sf_serial_ID] ;AN000;;IFS. save IFS ID
14140      ;mov [si+1Ch],ax
14141 00001E76 89441C      MOV [SI+fcbl_netID],ax      ;AN000;;IFS.
14142
14143      ;mov bl,80h
14144 00001E79 B380      MOV BL,FCBNETWORK
14145      ;
14146      ;----- END In net support -----
14147      ;
14148 00001E7B EB63      jmp SHORT SaveSFN
14149
14150 SaveLocal:
14151      ;IF Installed
14152 00001E7D E8B565      call CheckShare
14153      ;JZ short SaveNoShare ; no sharer
14154      ;JMP short SaveShare ; sharer present
14155      ; 16/12/2022
14156      ; 28/07/2019
14157 00001E80 7559      jnz short SaveShare
14158      ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
14159      ;JZ short SaveNoShare ; no sharer
14160      ;JMP short SaveShare ; sharer present
14161
14162 SaveNoShare:
14163      ;test word [es:di+5],80h
14164      ;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device
14165 00001E82 26F6450580 test byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; 80h
14166 00001E87 7542      JNZ short SaveNoShareDev ; Device
14167
14168      ; Save no sharing local file information
14169
14170      ;mov ax,[es:di+1Dh] ; MSDOS 3.3
14171      ;mov ax,[es:di+1Bh] ; MSDOS 6.0
14172 00001E89 268B451B      MOV AX,[ES:DI+SF_ENTRY.sf_dirsec] ; get directory sector F.C.
14173      ;mov [si+1Dh],ax
14174 00001E8D 89441D      MOV [SI+fcbl_nsl_dirsec],AX
14175
14176      ; MSDOS 6.0
14177
14178      ;SR; Store high byte of directory sector
14179      ;mov ax,[es:di+1Dh]
14180 00001E90 268B451D      mov ax,[es:di+SF_ENTRY.sf_dirsec+2] ; get high word
14181
14182      ; SR;
14183      ; We have to store the read-only and archive attributes of the file.
14184      ; We extract it from the SFT and store it in the top two bits of the
14185      ; sector number ( sector number == 22 bits only )
14186
14187      ;mov bl,[es:di+4]
14188 00001E94 268A5D04      mov bl,[es:di+SF_ENTRY.sf_attr]
14189 00001E98 88DF      mov bh,bl
14190 00001E9A D0CB      ror bl,1
14191 00001E9C D0E7      shl bh,1
14192 00001E9E 08FB      or bl,bh
14193 00001EA0 80E3C0      and bl,0C0h
14194 00001EA3 08D8      or al,bl
14195      ;mov [si+18h],al ; 08/11/2022
14196 00001EA5 884418      mov [si+fcbl_sfn],al ; sector number = 22 bits
14197
14198      ; MSDOS 6.0 (& MSDOS 3.3)
14199      ;mov al,[es:di+1Fh]
14200 00001EA8 268A451F      MOV AL,[ES:DI+SF_ENTRY.sf_dirpos] ; location in sector
14201      ;mov [si+1Fh],al
14202 00001EAC 88441F      MOV [SI+fcbl_nsl_dirpos],AL
14203
14204      ; 20/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
14205      ;;;
14206      ;mov ax,[es:di+0Bh] ; .sf_firclus:
14207      ;MOV AX,[ES:DI+SF_ENTRY.sf_firclus] ; first cluster
14208      ; 20/01/2024
14209      ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:5DF5h)
14210      ; (Windows ME IO.SYS - BIOSCODE:5D60h)
14211 00001EAF 268B452B      mov ax,[es:di+2Bh] ; .sf_chain !!! (MSDOS 6.22)
14212      ;mov ax,[es:di+SF_ENTRY.sf_chain] ; first cluster (32 bit) !?
14213      ;;;
14214      ;mov [si+1Bh],ax
14215 00001EB3 89441B      MOV [SI+fcbl_nsl_firclus],AX
14216 00001EB6 B300      MOV BL,0
14217
14218      ; Create the bits field from the dirty/device bits of the flags word
14219      ; and the mode byte
14220
14221 SetFCBBits:
14222      ;mov ax,[es:di+5]
14223 00001EB8 268B4505      MOV AX,[ES:DI+SF_ENTRY.sf_flags]
14224 00001EBC 24C0      AND AL,0C0h ; mask off drive bits
14225      ;or al,[es:di+2]
14226 00001EBE 260A4502      OR AL,[ES:DI+SF_ENTRY.sf_mode] ; stick in open mode
14227      ;mov [si+1Ah], al
14228 00001EC2 88441A      MOV [SI+fcbl_nsl_bits],AL ; save dirty info
14229
14230      ; MSDOS 6.0
14231
14232      ; SR;
14233      ; Check if we came here for local file or device. If for local file,
14234      ; skip setting of SFT index
14235
14236 00001EC5 08DB      or bl,bl
14237 00001EC7 7428      jz short SaveNoSFN ; do not save SFN if local file
14238
14239 00001EC9 EB15      JMP short SaveSFN ; go and save SFN
14240
14241      ; Save no sharing local device information
14242
14243 SaveNoShareDev:
14244      ; 20/01/2024
14245      ;mov ax,[es:di+7]

```

```

14246             ;MOV     AX,[ES:DI+SF_ENTRY.sf_devptr]
14247             ;;mov     [si+1Ah],ax
14248             ;MOV     [SI+fcblnsld_drvptr],AX
14249             ;;mov     ax,[es:di+9]
14250             ;MOV     AX,[ES:DI+SF_ENTRY.sf_devptr+2]
14251             ;MOV     [SI+fcblnsld_drvptr+2],AX
14252             ; 20/01/2024 (PCDOS 7.1 IBMDOS.COM)
14253 00001ECB 06     push     es
14254 00001ECC 26C44507 les     ax,[es:di+SF_ENTRY.sf_devptr]
14255 00001ED0 89441A  mov     [si+fcblnsld_drvptr],ax
14256 00001ED3 8C441C  mov     [si+fcblnsld_drvptr+2],es
14257 00001ED6 07     pop      es
14258
14259             ;mov     bl,40h
14260 00001ED7 B340    MOV     BL,FCBDEVICE
14261             ; 28/12/2022
14262 00001ED9 EBDD    JMP     short SetFCBBits      ; go and save SFN
14263
14264 SaveShare:
14265             ;ENDIF
14266
14267 ;----- In share support -----
14268
14269             ;call     far [ss:ShSave]
14270 00001EDB 36FF1E[B800] Call     far [ss:JShare+(10*4)] ; 10 = ShSave ; SS override
14271
14272 ;----- end in share support -----
14273
14274             ; 17/05/2019
14275
14276 SaveSFN:
14277             ;lea     ax,[di-6]
14278 00001EE0 8D45FA  LEA     AX,[DI-SFT.SFTable]
14279
14280             ; Adjust for offset to table.
14281
14282 00001EE3 362B06[4000] SUB     AX,[SS:SFTFCB]      ; SS override for SftFCB
14283
14284 00001EE8 53     push     bx                ;bx = FCB type (net/Share or local)
14285             ;;mov     bl,53 ; MSDOS 3.3
14286             ;mov     bl,59 ; MSDOS 6.0
14287 00001EE9 833B    MOV     BL,SF_ENTRY.size
14288 00001EEB F6F3    DIV     BL
14289             ;mov     [si+18h],al
14290 00001EED 884418  MOV     [SI+fcblnsfn],AL      ; last used SFN
14291 00001EF0 5B     pop      bx                ;restore bx
14292
14293 SaveNoSFN:
14294             ;mov     ax,[es:di+5]
14295 00001EF1 268B4505 MOV     AX,[ES:DI+SF_ENTRY.sf_flags]
14296 00001EF5 243F    AND     AL,3Fh              ; get real drive
14297 00001EF7 08D8    OR      AL,BL
14298             ;mov     [si+19h],al
14299 00001EF9 884419  MOV     [SI+fcblnsdrive],AL
14300
14301 00001EFC 36A1[1000] MOV     AX,[SS:FCBLRU]      ; get lru count
14302 00001F00 40     INC     AX
14303             ;mov     [es:di+15h],ax
14304 00001F01 26894515 MOV     [ES:DI+sf_LRU],AX
14305 00001F05 7506    JNZ     short SimpleStuff
14306
14307             ; lru flag overflowed. Run through all FCB sfts and adjust:
14308             ; LRU < 8000H get set to 0. Others -= 8000h. This LRU = 8000h
14309
14310             ;mov     bx,15h
14311 00001F07 8B1500  MOV     BX,SF_ENTRY.sf_position
14312 00001F0A E80500  call     ResetLRU
14313
14314             ; Set new LRU to AX
14315 SimpleStuff:
14316 00001F0D 36A3[1000] MOV     [SS:FCBLRU],AX
14317 00001F11 C3     retn
14318
14319 ;Break      <ResetLRU - reset overflowed lru counts>
14320 ;
14321 ;-----
14322 ; ResetLRU - during lru updates, we may wrap at 64K. we must walk the
14323 ; entire set of SFTs and subtract 8000h from their lru counts and truncate
14324 ; at 0.
14325 ;
14326 ; Inputs: BX is offset into SFT field where lru firld is kept
14327 ;         ES:DI point to SFT currently being updated
14328 ; Outputs: All FCB SFTs have their lru fields truncated
14329 ;         AX has 8000h
14330 ; Registers modified: none
14331 ;
14332 ;-----
14333 ;
14334 ; 17/05/2019 - Retro DOS v4.0
14335
14336 ResetLRU:
14337             ; ResetLRU is only called from fcbio.asm. so SS can be assumed to be
14338             ; DOSDATA
14339
14340             MOV     AX,8000h
14341             push    es
14342             push    di
14343             ;LES     DI,[CS:SFTFCB]      ; get pointer to head
14344 00001F17 36C43E[4000] LES     DI,[SS:SFTFCB] ; MSDOS 6.0
14345             ;mov     cx,[es:di+4]
14346 00001F1C 268B4D04 MOV     CX,[ES:DI+SFT.SFCount]
14347             ;lea     di,[di+6]
14348 00001F20 8D7D06  LEA     DI,[DI+SFT.SFTable] ; point at table
14349
14350 ovScan:
14351             SUB     [ES:DI+BX],AX      ; decrement lru count
14352             JA      short ovLoop
14353             MOV     [ES:DI+BX],AX      ; truncate at 0
14354
14355 ovLoop:
14356             ;;add     di,53 ; MSDOS 3.3
14357             ;add     di,59 ; MSDOS 6.0
14358 00001F2B 83C73B  ADD     DI,SF_ENTRY.size      ; advance to next
14359             LOOP    ovScan
14360             pop     di
14361             pop     es
14362             MOV     [ES:DI+BX],AX
14363             retn
14364
14365 ;IF 0 ; we dont need this routine any more.
14366 ;
14367 ;Break      <SetOpenAge - update the open age of a SFT>
14368 ;-----
14369 ; SetOpenAge - In order to maintain the first N open files in the FCB cache,
14370 ; we keep the 'open age' or an LRU count based on opens. we update the

```

```

14370 ; count here and fill in the appropriate field.
14371 ;
14372 ; Inputs: ES:DI point to SFT
14373 ; Outputs: ES:DI has the open age field filled in.
14374 ; If open age has wraparound, we will have subtracted 8000h
14375 ; from all open ages.
14376 ; Registers modified: AX
14377 ;
14378 ;-----
14379 ;
14380 ;SetOpenAge:
14381 ; 20/07/2018 - Retro DOS v3.0
14382 ; MSDOS 3.3 - IBMDOS.COM, Offset 2597h
14383 ; (& MSDOS 6.0, FCBI0.ASM)
14384 ;
14385 ; SetOpenAge is called from fcbio2.asm. SS can be assumed to be valid.
14386 ;
14387 ; MOV AX,[CS:OpenLRU] ; SS override
14388 ; INC AX
14389 ; mov [es:di+17h],ax
14390 ; MOV [ES:DI+sf_OpenAge],AX
14391 ; JNZ short SetDone
14392 ; mov bx,17h
14393 ; MOV BX,SF_ENTRY.sf_position+2 ; mov bx,sf_OpenAge
14394 ; call ResetLRU
14395 ;SetDone:
14396 ; MOV [CS:OpenLRU],AX
14397 ; retn
14398 ;
14399 ;ENDIF ; SetOpenAge no longer needed
14400 ;
14401 ; 21/07/2018 - Retro DOS v3.0
14402 ; LRUFCB for MSDOS 6.0 !
14403 ;
14404 ;Break <LRUFCB - perform LRU on FCB sfts>
14405 ;-----
14406 ;
14407 ; LRUFCB - find LRU fcb in cache. Set ThisSFT and return it. We preserve
14408 ; the first keepcount sfts if they are network sfts or if sharing is
14409 ; loaded. If carry is set then NO BLASTING is NECESSARY.
14410 ;
14411 ; Inputs: none
14412 ; Outputs: ES:DI point to SFT
14413 ; ThisSFT points to SFT
14414 ; SFT is zeroed
14415 ; Carry set of closes failed
14416 ; Registers modified: none
14417 ;
14418 ;-----
14419 ;
14420 ; MSDOS 6.0
14421 ; IF 0 ; rewritten this routine
14422 ;
14423 ; LRUFCB: ; MSDOS 3.3 - IBMDOS.COM (1987) - Offset 25ADh
14424 ; call save_world
14425 ;
14426 ; Find nth oldest NET/SHARE FCB. We want to find its age for the second scan
14427 ; to find the lease recently used one that is younger than the open age. We
14428 ; operate by scanning the list n times finding the least age that is greater
14429 ; or equal to the previous minimum age.
14430 ;
14431 ; BP is the count of times we need to go through this loop.
14432 ; AX is the current acceptable minimum age to consider
14433 ;
14434 ; mov bp,[CS:KEEPCOUNT] ; k = keepcount;
14435 ; XOR AX,AX ; low = 0;
14436 ;
14437 ; If we've scanned the table n times, then we are done.
14438 ;
14439 ; lru1:
14440 ; CMP bp,0 ; while (k--) {
14441 ; JZ short lru75
14442 ; DEC bp
14443 ;
14444 ; Set up for scan.
14445 ;
14446 ; AX is the minimum age for consideration
14447 ; BX is the minimum age found during the scan
14448 ; SI is the position of the entry that corresponds to BX
14449 ;
14450 ; MOV BX,-1 ; min = 0xffff;
14451 ; MOV si,BX ; pos = 0xffff;
14452 ; LES DI,[CS:SFTFCB] ; for (CX=FCBcount; CX>0; CX--)
14453 ; mov cx,[es:di+4]
14454 ; MOV CX,[ES:DI+SFT.SFCount]
14455 ; lea di,[di+6]
14456 ; LEA DI,[DI+SFT.SFTable]
14457 ;
14458 ; Innermost loop. If the current entry is free, then we are done. Or, if the
14459 ; current entry is busy (indicating a previous aborted allocation), then we
14460 ; are done. In both cases, we use the found entry.
14461 ;
14462 ; lru2:
14463 ; cmp word [es:di],0
14464 ; ;cmp word [es:di+SF_ENTRY.sf_ref_count],0
14465 ; jz short lru25
14466 ; ;cmp word [es:di],-1
14467 ; ;cmp word [es:di+SF_ENTRY.sf_ref_count],sf_busy
14468 ; cmp word [es:di],sf_busy
14469 ; jnz short lru3
14470 ;
14471 ; The entry is usable without further scan. Go and use it.
14472 ;
14473 ; lru25:
14474 ; MOV si,DI ; pos = i;
14475 ; JMP short lru11 ; goto got;
14476 ;
14477 ; See if the entry is for the network or for the sharer.
14478 ;
14479 ; If for the sharer or network then
14480 ; if the age < current minimum AND >= allowed minimum then
14481 ; this entry becomes current minimum
14482 ;
14483 ; lru3:
14484 ; test word [es:di+5],8000h
14485 ; TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
14486 ; ; if (!net[i]
14487 ; JNZ short lru35
14488 ; if installed
14489 ; call CheckShare ; && !sharing)
14490 ; JZ short lru5 ; else
14491 ; ENDIF
14492 ;
14493 ; This SFT is for the net or is for the sharer. See if it less than the

```

```

14494 ; current minimum.
14495 ;
14496 ;lru35:
14497 ;mov dx,[es:di+17h]
14498 ;MOV DX,[ES:DI+sf_OpenAge]
14499 ;CMP DX,AX ; if (age[i] >= low &&
14500 ;JB short lru5
14501 ;CMP DX,BX
14502 ;JAE short lru5 ; age[i] < min) {
14503 ;
14504 ; entry is new minimum. Remember his age.
14505 ;
14506 ; mov bx,DX ; min = age[i];
14507 ; mov si,di ; pos = i;
14508 ;
14509 ; End of loop. gp back for more
14510 ;
14511 ;lru5:
14512 ;add di,53
14513 ;add di,SF_ENTRY.size
14514 ;loop lru2 ; }
14515 ;
14516 ; The scan is complete. If we have successfully found a new minimum (pos != -1)
14517 ; set then threshold value to this new minimum + 1. Otherwise, the scan is
14518 ; complete. Go find LRU.
14519 ;
14520 ;lru6:
14521 ;cmp si,-1 ; position not -1?
14522 ;jz short lru75 ; no, done with everything
14523 ;lea ax,[bx+1] ; set new threshold age
14524 ;jmp short lru1 ; go and loop for more
14525 ;lru65:
14526 ;stc
14527 ;jmp short LRUDead ; return -1;
14528 ;
14529 ; Main loop is done. We have AX being the age+1 of the nth oldest sharer or
14530 ; network entry. We now make a second pass through to find the LRU entry
14531 ; that is local-no-share or has age >= AX
14532 ;
14533 ;lru75:
14534 ;mov bx,-1 ; min = 0xffff;
14535 ;mov si,bx ; pos = 0xffff;
14536 ;LES DI,[CS:SFTFCB] ; for (CX=FCBCount; CX>0; CX--)
14537 ;mov cx,[es:di+4]
14538 ;MOV CX,[ES:DI+SFT.SFCount]
14539 ;lea di,[di+6]
14540 ;LEA DI,[DI+SFT.SFTable]
14541 ;
14542 ; If this is is local-no-share then go check for LRU else if age >= threshold
14543 ; then check for lru.
14544 ;
14545 ;lru8:
14546 ;test word [es:di+5],8000h
14547 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
14548 ;jnz short lru85 ; is for network, go check age
14549 ;call CheckShare ; sharer here?
14550 ;jz short lru86 ; no, go check lru
14551 ;
14552 ; Network or sharer. Check age
14553 ;
14554 ;lru85:
14555 ;cmp [es:di+17h],ax
14556 ;cmp [es:di+sf_OpenAge],ax
14557 ;jb short lru9 ; age is before threshold, skip it
14558 ;
14559 ; Check LRU
14560 ;
14561 ;lru86:
14562 ;cmp [es:di+15h],bx
14563 ;cmp [es:di+sf_LRU],bx ; is LRU less than current LRU?
14564 ;jae short lru9 ; no, skip this
14565 ;mov si,di ; remember position
14566 ;mov bx,[es:di+15h]
14567 ;mov bx,[es:di+sf_LRU] ; remember new minimum LRU
14568 ;
14569 ; Done with this entry, go back for more.
14570 ;
14571 ;lru9:
14572 ;add di, 53
14573 ;add di,SF_ENTRY.size
14574 ;loop lru8
14575 ;
14576 ; Scan is complete. If we found NOTHING that satisfied us then we bomb
14577 ; out. The conditions here are:
14578 ;
14579 ; No local-no-shares AND all net/share entries are older than threshold
14580 ;
14581 ;lru10:
14582 ;cmp si,-1 ; if no one f
14583 ;jz short lru65 ; return -1;
14584 ;lru11:
14585 ;mov di,si
14586 ;MOV [CS:THISSFT],DI ; set thissft
14587 ;MOV [CS:THISSFT+2],ES
14588 ;
14589 ; If we have sharing or thissft is a net sft, then close it until ref count
14590 ; is 0.
14591 ;
14592 ;test word [es:di+5],8000h
14593 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
14594 ;JNZ short LRUClose
14595 ;IF INSTALLED
14596 ;call CheckShare
14597 ;JZ short LRUDone
14598 ;ENDIF
14599 ;
14600 ; Repeat close until ref count is 0
14601 ;
14602 ;LRUClose:
14603 ;push ss
14604 ;pop ds
14605 ;LES DI,[THISSFT]
14606 ;cmp word [es:di],0
14607 ;CMP word [ES:DI+SFT.sf_ref_count],0 ; is ref count still <> 0?
14608 ;JZ short LRUDone ; nope, all done
14609 ;call DOS_CLOSE
14610 ;jnc short LRUClose ; no error => clean up
14611 ;;cmp al,6
14612 ;cmp al,error_invalid_handle
14613 ;jz short LRUClose
14614 ;stc
14615 ;JMP short LRUDead
14616 ;LRUDone:
14617 ;XOR AL,AL

```



```

14618 ; call BlastSFT ; fill SFT with 0 (AL), 'c' cleared
14619 ;
14620 ;LRUDead:
14621 ; call restore_world
14622 ; LES DI,[CS:THISST]
14623 ; jnc short LRUFCB_retn
14624 ;LRUFCB_err:
14625 ; ; mov al, 23h
14626 ; MOV AL,error_FCB_unavailable
14627 ;LRUFCB_retn:
14628 ; retn:
14629 ;
14630 ;ENDIF ; LRUFCB has been rewritten below.
14631 ;
14632 ; 17/05/2019 - Retro DOS v4.0
14633 ; LRUFCB for MSDOS 6.0 !
14634 ;-----
14635 ;
14636 ; LruFCB -- allocate the LRU SFT from the SFT Table. The LRU scheme
14637 ; maintains separate counts for net/Share and local SFTs. We allocate a
14638 ; net/Share SFT only if we do not find a local SFT. This helps keep
14639 ; net/Share SFTs which cannot be regenerated for as long as possible. We
14640 ; optimize regeneration operations by keeping track of the current local
14641 ; SFT. This avoids scanning of the SFTs as long as we have at least one
14642 ; local SFT in the SFT Block.
14643 ;
14644 ; Inputs: al = 0 => Regenerate SFT operation
14645 ; = 1 => Allocate new SFT for Open/Create
14646 ;
14647 ; Outputs: Carry clear
14648 ; es:di = Address of allocated SFT
14649 ; ThisSFT = Address of allocated SFT
14650 ;
14651 ; carry set if closes of net/Share files failed
14652 ; al = error_FCB_unavailable
14653 ;
14654 ; Registers affected: None
14655 ;-----
14656 ;
14657 ;LruFCB PROC NEAR
14658 LRUFCB:
14659 ; 17/05/2019 - Retro DOS v4.0
14660 ; DOSCODE:5805h (MSDOS 6.21, MSDOS.SYS)
14661 ;
14662 ; 08/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
14663 ; DOSCODE:57F1h (MSDOS 5.0, MSDOS.SYS)
14664 ;
14665 ; 20/01/2024 Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
14666 ; DOSCODE:5E7Ch (PCDOS 7.1, IBMDOS.COM)
14667 ;
14668 ;
14669 00001F36 06 push es ; * (MSDOS 6.21)
14670 ;
14671 00001F37 E81EE5 call save_world
14672 ;
14673 ;getdseg <ds> ;ds = DOSDATA
14674 00001F3A 2E8E1E[0700] mov ds,[cs:DosDSeg]
14675 ;
14676 00001F3F 08C0 or al,al ;Check if regenerate allocation
14677 00001F41 7515 jnz short lru1 ;Try to find SFT to use
14678 ;
14679 ; This is a regen call. If LocalSFT contains the address of a valid
14680 ; local SFT, just return that SFT to reuse
14681 ;
14682 ; 20/01/2024
14683 ;mov di,[LocalSFT]
14684 ;or di,[LocalSFT+2] ;is address == 0?
14685 ;jz short lru1 ;invalid local SFT, find one
14686 ;
14687 ; we have found a valid local SFT. Recycle this SFT
14688 ;
14689 00001F43 C43E[760F] les di,[LocalSFT]
14690 ;
14691 ; 20/01/2024 (PCDOS 7.1 IBMDOS.COM)
14692 00001F47 8CC1 mov cx,es
14693 00001F49 09F9 or cx,di ; is address == 0?
14694 00001F4B 740B jz short lru1 ; invalid local SFT, find one
14695 ;
14696 ;
14697 00001F4D 893E[9E05] gotlocalSFT:
14698 00001F51 8C06[A005] mov [THISST],di
14699 ; mov [THISST+2],es
14700 ; 04/02/2026
14701 00001F55 E9A900 jmp LRUDone ;clear up SFT and return
14702 ;
14703 ;
14704 00001F58 C43E[4000] lru1:
14705 ;les di,[SFTFCB] ;es:di = SF Table for FCBs
14706 ;mov cx,[es:di+4]
14707 ;mov cx,[es:di+SFT.SFCount];cx = number of SFTs
14708 ;lea di,[di+6]
14709 ;lea di,[di+SFT.SFTable] ;es:di = first SFT
14710 ;
14711 ; We scan through all the SFTs scanning for a free one. It also
14712 ; remembers the LRU SFT for net/Share SFTs and local SFTs separately.
14713 ; bx = min. LRU for local SFTs
14714 ; si = pos. of local SFT with min. LRU
14715 ; dx = min. LRU for net/Share SFTs
14716 ; bp = pos. of net/Share SFT with min. LRU
14717 00001F63 BBFFFF mov bx,-1 ; init. to 0xffff ( max. LRU value )
14718 00001F66 89DE mov si,bx
14719 00001F68 89DA mov dx,bx
14720 00001F6A 89DD mov bp,bx
14721 ;
14722 ;
14723 ;findSFT:
14724 00001F6C 26830D00 ;See if this SFT is a free one. If so, return it
14725 ;or word [es:di],0
14726 00001F70 744C ;or word [es:di+SF_ENTRY.sf_ref_count],0 ;reference count = 0 ?
14727 ;jz short gotSFT ;yes, SFT is free
14728 ;cmp word [es:di],-1
14729 00001F72 26833DFF ;cmp word [es:di+SF_ENTRY.sf_ref_count],sf_busy ;Is it busy?
14730 00001F76 7446 cmp word [es:di],sf_busy ; -1
14731 ;jz short gotSFT ;no, can use it
14732 ;
14733 ; Check if this SFT is local and store its address in LocalSFT. Can be
14734 ; used for a later regen.
14735 ;
14736 ; 16/12/2022
14737 ; 08/11/2022
14738 00001F78 26F6450680 ;test byte [es:di+6],80h
14739 ;test byte [es:di+SF_ENTRY.sf_flags+1],(sf_isnet>>8) ; 80h
14740 ; 08/11/2022 Retro DOS v4.0 (MSDOS 5.0 MSDOS.SYS compatibility)
14741 ;;test word [es:di+5],8000h
14742 ;test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet ; network SFT?

```

```

14742 00001F7D 7531      jnz     short lru5      ;yes, get net/Share LRU
14743
14744
14745 00001F7F E8B364     ;IF installed
14746      call    CheckShare      ;Share present?
14747 00001F82 752C     ;ENDIF
14748      jnz     short lru5      ;yes, get net/Share LRU
14749
14750      ;Local SFT, register its address
14751
14752      ; !!HACK!!!
14753      ; There is a slightly dirty hack out here in a desperate bid to save
14754      ; code space. There is similar code duplicated at label 'gotSFT'. We
14755      ; enter from there if al = 0, update the LocalSFT variable, and since
14756      ; al = 0, we jump out of the loop to the exit point. I have commented
14757      ; out the code that previously existed at label 'gotSFT'
14758
14759 00001F84 893E[760F]   hackpoint:
14760 00001F88 8C06[780F]   mov     [LocalSFT],di
14761      mov     [LocalSFT+2],es      ;store local SFT address
14762 00001F8C 08C0      or      al,al      ;Is operation = REGEN?
14763 00001F8E 74BD      jz      short gotlocalSFT ;yes, return this SFT for reuse
14764
14765      ;Get LRU for local files
14766
14767      ;cmp     [es:di+15h],bx
14768 00001F90 26395D15     cmp     [es:di+sf_LRU],bx ;SFT.LRU < min?
14769 00001F94 7306      jae     short lru4      ;no, skip
14770
14771      ;mov     bx,[es:di+15h]
14772 00001F96 268B5D15     mov     bx,[es:di+sf_LRU] ;yes, store new minimum
14773 00001F9A 89FE      mov     si,di      ;store SFT position
14774
14775      lru4:
14776 00001F9C 83C73B     ;add     di,59
14777 00001F9F E2CB      add     di,SF_ENTRY.size ;go to next SFT
14778      loop    findSFT
14779
14780 00001FA1 49      ; 20/01/2024
14781      dec     cx ; -1
14782
14783      ; Check whether we got a net/Share or local SFT. If local SFT
14784      ; available, we will reuse it instead of net/Share LRU
14785 00001FA2 89F7      mov     di,si
14786      ;cmp     si,-1      ;local SFT available?
14787 00001FA4 39CE      cmp     si,cx ; 20/01/2024
14788 00001FA6 7516      jnz     short gotSFT      ;yes, return it
14789
14790      ;No local SFT, see if we got a net/Share SFT
14791
14792 00001FA8 89EF      mov     di,bp
14793
14794 00001FAA 39CD      cmp     bp,cx ; -1 ; 20/01/2024
14795      ;cmp     bp,-1      ;net/Share SFT available?
14796 00001FAC 752D      jnz     short gotnetSFT      ;yes, return it
14797
14798      noSFT:
14799      ; NB: This error should never occur. We always must have an LRU SFT.
14800      ; This error can occur only if the SFT has been corrupted or the LRU
14801      ; count is not maintained properly.
14802 00001FAE EB4E      jmp     short errorbadSFT ;error, no FCB available.
14803
14804      ; Handle the LRU for net/Share SFTs
14805
14806      lru5:
14807 00001FB0 26395515     ;cmp     [es:di+15h],dx
14808 00001FB4 73E6      cmp     [es:di+sf_LRU],dx ;SFT.LRU < min?
14809      jae     short lru4      ;no, skip
14810
14811 00001FB6 268B5515     ;mov     dx,[es:di+15h]
14812      mov     dx,[es:di+sf_LRU] ;yes, store new minimum
14813 00001FBA 89FD      mov     bp,di      ;store SFT position
14814 00001FBC EBDE      jmp     short lru4      ;continue with next SFT
14815
14816      gotSFT:
14817 00001FBE 08C0      or      al,al
14818 00001FC0 74C2      jz      short hackpoint      ;save es:di in LocalSFT
14819
14820      ; HACK!!!
14821      ; The code here differs from the code at 'hackpoint' only in the
14822      ; order of the check for al. If al = 0, we can jump to 'hackpoint'
14823      ; and then from there jump out to 'gotlocalSFT'. The original code
14824      ; has been commented out below and replaced by the code just above.
14825
14826      ;If regen, then this SFT can be registered as a local one ( even if free ).
14827      ;
14828      ; or      al,al      ;Regen?
14829      ; jnz     short notlocaluse ;yes, register it and return
14830      ;
14831      ;Register this SFT as a local one
14832      ;
14833      ; mov     [LocalSFT],di
14834      ; mov     [LocalSFT+2],es
14835      ; jmp     gotlocalSFT ;return to caller
14836
14837      ;notlocaluse:
14838
14839      ; The caller is probably going to use this SFT for a net/Share file.
14840      ; We will come here only on a Open/Create when the caller($FCB_OPEN)
14841      ; does not really know whether it is a local file or not. We
14842      ; invalidate LocalSFT if the SFT we are going to use was previously
14843      ; registered as a local SFT that can be recycled.
14844
14845 00001FC2 8CC0      mov     ax,es
14846 00001FC4 393E[760F]     cmp     [LocalSFT],di      ;Offset same?
14847 00001FC8 750E      jne     short notinvalid
14848 00001FCA 3906[780F]     cmp     [LocalSFT+2],ax      ;Segments same?
14849      ;je      short zerolocalSFT ;no, no need to invalidate
14850      ; 20/01/2024 (PCDOS 7.1 IBMDOS.COM)
14851 00001FCE 7508      jne     short notinvalid
14852
14853 00001FD0 31C0      zerolocalSFT:
14854 00001FD2 A3[760F]     xor     ax,ax ; 0
14855 00001FD5 A3[780F]     mov     [LocalSFT],ax
14856      mov     [LocalSFT+2],ax
14857
14858 00001FD8 E972FF     notinvalid:
14859      jmp     gotlocalSFT
14860
14861      ; The SFT we are going to use was registered in the LocalSFT variable.
14862      ; Invalidate this variable i.e LocalSFT = NULL
14863
14864      ;zerolocalSFT:
14865      ;xor     ax,ax ; 0
14866      ;mov     [LocalSFT],ax

```



```

14866         ;mov     [LocalSFT+2],ax
14867         ;
14868         ;jmp     gotlocalSFT
14869
14870 gotnetSFT:
14871         ; We have an SFT that is currently net/Share. If it is going to be
14872         ; used for a regen, we know it has to be a local SFT. Update the
14873         ; LocalSFT variable
14874
14875 00001FDB 08C0         or     al,al
14876 00001FDD 7508         jnz     short closenet
14877
14878 00001FDF 893E[760F]   mov     [LocalSFT],di
14879 00001FE3 8C06[780F]   mov     [LocalSFT+2],es        ;store local SFT address
14880 closenet:
14881 00001FE7 893E[9E05]   mov     [THISST],di        ; set thisst
14882 00001FEB 8C06[A005]   mov     [THISST+2],es
14883
14884         ; If we have sharing or thisSFT is a net sft, then close it until ref
14885         ; count is 0.
14886         ; NB: We come here only if it is a net/Share SFT that is going to be
14887         ; recycled -- no need to check for this.
14888
14889 LRUclose:
14890 00001FEF 26833D00     cmp     word [es:di],0
14891         ;cmp     word [es:di+SF_ENTRY.sf_ref_count],0 ; is ref count still <> 0?
14892 00001FF3 740C         jz      short LRUDone ; nope, all done
14893
14894 00001FF5 E82C17     call    DOS_CLOSE
14895 00001FF8 73F5         jnc     short LRUclose ; no error => clean up
14896
14897         ; Bugbug: I dont know why we are trying to close after we get an
14898         ; error closing. Seems like we could have a potential infinite loop
14899         ; here. This has to be verified.
14900
14901 00001FFA 3C06         cmp     al,error_invalid_handle ; 6
14902 00001FFC 74F1         je      short LRUclose
14903 errorbadSFT:
14904 00001FFE F9         stc
14905 00001FFF EB05         jmp     short LRUDead
14906 LRUDone:
14907 00002001 30C0         xor     al,al
14908 00002003 E81101     call    BlastSFT        ; fill SFT with 0 (AL), 'c' cleared
14909
14910 LRUDead:
14911 00002006 E838E4     call    restore_world    ; use macro
14912
14913 00002009 07         pop     es ; * (MSDOS 6.21)
14914
14915         ;getdseg <es>
14916 0000200A 2E8E06[0700]   mov     es,[cs:DosDSeg]
14917 0000200F 26C43E[9E05]   les     di,[es:THISST]        ;es:di points at allocated SFT
14918
14919         ;;retnc
14920         ;jc      short LruFCB_err
14921         ;retn
14922
14923         ; 16/12/2022
14924         ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
14925 00002014 7302         jnc     short LruFCB_retn
14926         ;jc      short LruFCB_err
14927         ;retn
14928
14929 LruFCB_err:
14930 00002016 B023         mov     al,error_FCB_unavailable ; 23h
14931 LruFCB_retn:
14932 00002018 C3         retn
14933
14934 ;LruFCB     ENDP
14935
14936 ; 17/05/2019 - Retro DOS v4.0
14937
14938 ; DOSCODE:58F3h (MSDOS 6.21, MSDOS.SYS)
14939
14940 ; 26/06/2024
14941 %if 0
14942
14943 ; -----
14944 ;**** RegenCopyName -- This function copies the filename from the FCB to
14945 ; SFT and also to DOS local buffers. There was duplicate code in FCBRegen
14946 ; to copy the name to different destinations
14947 ;
14948 ; Inputs: ds:si = source string
14949 ;         es:di = destination string
14950 ;         cx = length of string
14951 ;
14952 ; Outputs: String copied to destination
14953 ;
14954 ; Registers affected: cx,di,si
14955 ; -----
14956
14957 RegenCopyName:
14958 CopyName:
14959     lodsb                ;load character
14960     call    UCase        ; convert char to upper case
14961 StuffChar2:
14962     stosb                ;store converted character
14963     loop    CopyName     ;
14964 DoneName:
14965     retn
14966
14967 %endif
14968
14969 ; -----
14970
14971 ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
14972 ; 21/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
14973
14974 FCBRegen:
14975     ; called from SFTFromFCB. SS already DOSDATA
14976
14977     ; General data filling. Mode is sf_isFCB + open_for_both, date/time
14978     ; we do not fill, size we do no fill, position we do not fill,
14979     ; bit 14 of flags = TRUE, other bits = FALSE
14980
14981     ;mov     al,[si+19h]
14982 00002019 8A4419     mov     al,[SI+fcbl_drive]
14983
14984     ; We discriminate based on the first two bits in the reserved field.
14985
14986     ;test     al,80h
14987 0000201C A880     test     al,FCBSPECIAL    ; check for no sharing test
14988 0000201E 741C         jz      short RegenNoSharing ; yes, go regen from no sharing
14989

```

```

14990 ; The FCB is for a network or a sharing based system. At this point
14991 ; we have already closed the SFT for this guy and reconnection is
14992 ; impossible.
14993 ;
14994 ; Remember that he may have given us a FCB with bogus information in
14995 ; it. Check to see if sharing is present or if the redir is present.
14996 ; If either is around, presume that we have cycled out the FCB and
14997 ; give the hard error. Otherwise, just return with carry set.
14998
14999 00002020 E81264 call CheckShare ; test for sharer
15000 00002023 7509 JNZ short RegenFail ; yep, fail this.
15001
15002 ;mov ax,1100h
15003 00002025 B80011 MOV AX,MultNET<<8 ; install check on multnet
15004 00002028 CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - INSTALLATION CHECK
15005 ; Return: AL = 00h not installed, OK to install
15006 ; 01h not installed, not OK to install
15007 ; FFh installed
15008 0000202A 08C0 OR AL,AL ; is it there?
15009 0000202C 740C JZ short RegenDead ; no, just fail the operation
15010 RegenFail:
15011 ; 17/05/2019 - Retro DOS v4.0
15012 ;MOV AX,[CS:USER_IN_AX] ; SS override
15013 0000202E 36A1[3A03] mov ax,[SS:USER_IN_AX] ; MSDOS 6.0
15014
15015 ;cmp ah,10h
15016 00002032 80FC10 cmp AH,FCB_CLOSE
15017 00002035 7403 jz short RegenDead
15018 00002037 E89801 call FCBHardErr ; massive hard error.
15019 RegenDead:
15020 0000203A F9 STC ; carry set
15021 FCBRegen_retn:
15022 0000203B C3 retn
15023
15024 ; Local FCB without sharing. Check to see if sharing is loaded. If
15025 ; so fail the operation.
15026
15027 RegenNoSharing:
15028 0000203C E8F663 call CheckShare ; Sharing around?
15029 0000203F 75ED JNZ short RegenFail
15030
15031 ; Find an SFT for this guy.
15032
15033 ; 17/05/2019 - Retro DOS v4.0
15034
15035 ; MSDOS 3.3
15036 ;call LRUFEB
15037 ;jc short FCBRegen_retn
15038
15039 ; MSDOS 6.0
15040 00002041 50 push ax
15041 00002042 B000 mov al,0 ;indicate it is a regen operation
15042 00002044 E8EFFF call LRUFEB
15043 00002047 58 pop ax
15044 00002048 72F1 jc short FCBRegen_retn
15045
15046 ;mov word [es:di+2],8002h
15047 0000204A 26C745020280 MOV word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB+open_for_both+SHARING_COMPAT
15048 00002050 243F AND AL,3Fh ; get drive number for flags
15049 00002052 98 CBW
15050 ;or ax,4000h
15051 00002053 0D0040 OR AX,sf_close_nodate ; normal FCB operation
15052
15053 ; The bits field consists of the upper two bits (dirty and device)
15054 ; from the SFT and the low 4 bits from the open mode.
15055
15056 ;mov cl,[si+1Ah]
15057 00002056 8A4C1A MOV CL,[SI+fcbl_nsl_bits] ; stick in dirty bits.
15058 00002059 88CD MOV CH,CL
15059 0000205B 80E5C0 AND CH,0C0h ; mask off the dirty/device bits
15060 0000205E 08E8 OR AL,CH
15061 ;and cl,0Fh
15062 00002060 80E10F AND CL,access_mask ; get the mode bits
15063 ;mov [es:di+2],cl
15064 00002063 26884D02 MOV [ES:DI+SF_ENTRY.sf_mode],CL
15065 ;mov [es:di+5],ax
15066 00002067 26894505 MOV [ES:DI+SF_ENTRY.sf_flags],AX ; initial flags
15067 ;MOV AX,[CS:PROC_ID] ; SS override
15068 0000206B 36A1[3C03] mov ax,[ss:PROC_ID] ; MSDOS 6.0
15069 ;mov [es:di+31h],ax
15070 0000206F 26894531 MOV [ES:DI+SF_ENTRY.sf_PID],AX
15071 00002073 1E push ds
15072 00002074 56 push si
15073 00002075 06 push es
15074 00002076 57 push di
15075 00002077 16 push ss
15076 00002078 07 pop es
15077 00002079 BF[4B05] MOV DI,NAME1 ; NAME1 is in DOSDATA
15078
15079 0000207C B90800 MOV CX,8
15080 0000207F 46 INC SI ; Skip past drive byte to name in FCB
15081
15082 ; MSDOS 3.3
15083 ;RegenCopyName:
15084 ;lodsb
15085 ;call UCase
15086 ;stosb
15087 ;loop RegenCopyName
15088
15089 ; MSDOS 6.0
15090 00002080 E88C00 call RegenCopyName ;copy the name to NAME1
15091
15092 00002083 16 push ss ; SS is DOSDATA
15093 00002084 1F pop ds
15094
15095 ;mov byte [ATTRIB],16h
15096 00002085 C606[6B05]16 MOV byte [ATTRIB],attr_hidden+attr_system+attr_directory
15097 ; Must set this to something interesting
15098 ; to call DEVNAME.
15099 0000208A E8432D call DEVNAME ; check for device
15100 0000208D 5E pop si
15101 0000208E 07 pop es
15102 0000208F 5E pop si
15103 00002090 1F pop ds
15104 00002091 7219 JC short RegenFileNoSharing ; not found on device list => file
15105
15106 ; Device found. We can ignore disk-specific info
15107
15108 ;mov [es:di+5],bh
15109 00002093 26887D05 MOV [ES:DI+SF_ENTRY.sf_flags],BH ; device parms
15110 ;mov byte [es:di+4],0
15111 00002097 26C6450400 MOV byte [ES:DI+SF_ENTRY.sf_attr],0 ; attribute
15112 ; SS override
15113 ;LDS SI,[CS:DEVPT] ; get device driver

```

```

15114 0000209C 36C536[9A05]      lds     si,[ss:DEVPT] ; MSDOS 6.0
15115                                regen_save_dpb: ; 26/06/2024
15116                                ;mov     [es:di+7],si
15117 000020A1 26897507          MOV     [ES:DI+SF_ENTRY.sf_devptr],SI
15118                                ;mov     [es:di+9],ds
15119 000020A5 268C5D09          MOV     [ES:DI+SF_ENTRY.sf_devptr+2],DS
15120 000020A9 C3                    retn     ; carry is clear
15121
15122                                RegenDeadJ:
15123 000020AA EB8E                JMP     short RegenDead
15124
15125                                ; File found. Just copy in the remaining pieces.
15126
15127                                RegenFileNoSharing:
15128                                ;mov     ax,[es:di+5]
15129 000020AC 268B4505          MOV     AX,[ES:DI+SF_ENTRY.sf_flags]
15130 000020B0 83E03F          AND     AX,03Fh
15131 000020B3 1E                push    ds
15132 000020B4 56                push    si
15133 000020B5 E8035A          call    FIND_DPB
15134                                ;;mov   [es:di+7],si
15135                                ;MOV     [ES:DI+SF_ENTRY.sf_devptr],SI
15136                                ;;mov   [es:di+9],ds
15137                                ;MOV     [ES:DI+SF_ENTRY.sf_devptr+2],DS
15138                                ; 26/06/2024 (PCDOS 7.1 IBMDOS.COM)
15139 000020B8 E8E6FF          call    regen_save_dpb
15140 000020BB 5E                pop     si
15141 000020BC 1F                pop     ds
15142 000020BD 72EB          jc      short RegenDeadJ ; if find DPB fails, then drive
15143                                ; indicator was bogus
15144                                ;mov     ax,[si+1Dh]
15145 000020BF 8B441D          MOV     AX,[SI+fcb_nsl_dirsec]
15146                                ;;mov   [es:di+1Dh],ax ; MSDOS 3.3
15147                                ;mov     [es:di+1Bh],ax ; MSDOS 6.0
15148 000020C2 2689451B          MOV     [ES:DI+SF_ENTRY.sf_dirsec],AX
15149
15150                                ; MSDOS 6.0
15151
15152                                ; SR;
15153                                ; Extract the read-only and archive bits from the top 2 bits of the sector
15154                                ; number
15155
15156                                ;mov     al,[si+18h]
15157 000020C6 8A4418          mov     al,[si+fcb_sfn]
15158 000020C9 24C0          and     al,0C0h ;get the 2 attribute bits
15159 000020CB 88C4          mov     ah,al
15160 000020CD D0C4          rol     ah,1
15161 000020CF D0E8          shr     al,1
15162 000020D1 08E0          or      al,ah
15163 000020D3 243F          and     al,03Fh ;mask off unused bits
15164                                ;mov     [es:di+4],al
15165 000020D5 26884504          mov     [es:di+SF_ENTRY.sf_attr],al
15166
15167                                ; SR;
15168                                ; Update the higher word of the directory sector from the FCB
15169
15170                                ;;mov   al,[si+18h]
15171 000020D9 8A4418          mov     al,[si+fcb_sfn]
15172 000020DC 243F          and     al,03Fh ;mask off top 2 bits -- attr bits
15173 000020DE 28E4          sub     ah,ah
15174                                ;mov     [es:di+1Dh],ax
15175 000020E0 2689451D          mov     [es:di+SF_ENTRY.sf_dirsec+2],ax ;update high word
15176
15177                                ; 21/01/2024
15178                                ; MSDOS 6.0 (& MSDOS 3.3)
15179                                ;mov     ax,[si+1Bh]
15180 000020E4 8B441B          MOV     AX,[SI+fcb_nsl_firclus]
15181                                ;;mov   [es:di+0Bh],ax
15182                                ;MOV     [ES:DI+SF_ENTRY.sf_firclus],AX
15183                                ;;;
15184                                ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15185 000020E7 2689452B          mov     [es:di+2Bh],ax ; .sf_chain !!! (MSDOS 6.22)
15186                                ;mov     [es:di+SF_ENTRY.sf_chain],ax ; first cluster (32 bit) !?
15187                                ;;;
15188
15189                                ;;mov   [es:di+1Bh],ax ; MSDOS 3.3
15190                                ;mov     [es:di+35h],ax ; MSDOS 6.0
15191 000020EB 26894535          MOV     [ES:DI+SF_ENTRY.sf_1stclus],AX
15192
15193                                ;;;
15194                                ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15195 000020EF 31C0          xor     ax,ax ; 0
15196 000020F1 2689452D          mov     [es:di+2Dh], ax ; 0
15197                                ;mov     [es:di+SF_ENTRY.sf_chain+2],ax
15198                                ; ;.sf_chain ! (MSDOS 6.22)
15199                                ; ;high word of first cluster (32 bit) !?
15200 000020F5 26894537          mov     [es:di+37h],ax ; 0
15201                                ;mov     [es:di+SF_ENTRY.sf_1stclus+2],ax ; hw of last cluster
15202                                ;;;
15203
15204                                ;mov     al,[si+1Fh]
15205 000020F9 8A441F          MOV     AL,[SI+fcb_nsl_dirpos]
15206                                ;mov     [es:di+1Fh],al
15207 000020FC 2688451F          MOV     [ES:DI+SF_ENTRY.sf_dirpos],AL
15208                                ;INC     word [ES:DI+SF_ENTRY.sf_ref_count]
15209 00002100 26FF05          inc     word [ES:DI] ; Increment reference count.
15210                                ; Existing FCB entries would be
15211                                ; flushed unnecessarily because of
15212                                ; check in CheckFCB of the ref_count.
15213                                ; July 22/85 - BAS
15214                                ;;;
15215                                ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15216 00002103 E80E29          call    set_sftfcb_entry ; put SFT entry number in the SFTFCB table
15217                                ; ;as FCB index number
15218                                ;;;
15219
15220                                ;lea     si,[si+1]
15221 00002106 8D7401          LEA     SI,[SI+SYS_FCB.name]
15222                                ;lea     di,[di+20h]
15223 00002109 8D7D20          LEA     DI,[DI+SF_ENTRY.sf_name]
15224                                ;mov     cx,11
15225 0000210C B90B00          MOV     CX,SYS_FCB.EXTENT-SYS_FCB.name ; 12-1
15226
15227                                ; 26/06/2024
15228                                ; MSDOS 6.0
15229                                ;call    RegenCopyName ;copy name to SFT
15230                                ; 26/06/2024
15231                                ; cf = 0 (at the result of the 'test' instruction)
15232
15233                                ; MSDOS 3.3
15234                                ;RegenCopyName2:
15235                                ;lods     byte ptr [di]
15236                                ;call     UCCase
15237                                ;stos     byte ptr [di]

```

```

15238         ;loop    RegenCopyName2
15239
15240         ; 26/06/2024
15241         ; cf = 0
15242         ;clc
15243         ;retn
15244
15245 ; 26/06/2024
15246 %if 1
15247
15248 ; -----
15249 ; **** RegenCopyName -- This function copies the filename from the FCB to
15250 ; SFT and also to DOS local buffers. There was duplicate code in FCBRegen
15251 ; to copy the name to different destinations
15252 ;
15253 ; Inputs: ds:si = source string
15254 ; es:di = destination string
15255 ; cx = length of string
15256 ;
15257 ; Outputs: String copied to destination
15258 ;
15259 ; Registers affected: cx,di,si
15260 ; -----
15261
15262 RegenCopyName:
15263 CopyName:
15264     lodsb                    ;load character
15265     call    UCase ; *      ; convert char to upper case
15266
15267 StuffChar2:
15268     STOSB                    ;store converted character
15269     LOOP    CopyName        ;
15270     ; 26/06/2024
15271     ; cf= 0 ; *
15272 DoneName:
15273     retn
15274
15275 %endif
15276
15277 ; 17/05/2019 - Retro DOS v4.0
15278 ; 21/01/2024 - Retro DOS v5.0
15279
15280 ;** BlastSFT - Fill SFT with Garbage
15281 ; -----
15282 ; BlastSFT is used when an SFT is no longer needed; it's called with
15283 ; various garbage values to put into the SFT. I don't know why,
15284 ; presumably to help with debugging (jgl). We clear the few fields
15285 ; necessary to show that the SFT is free after filling it.
15286 ;
15287 ; ENTRY (es:di) = address of SFT
15288 ; (al) = fill character
15289 ; EXIT (ax) = -1
15290 ; 'c' clear
15291 ; USES AX, CX, Flags
15292
15293 BlastSFT:
15294     ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15295     ;;;
15296     call    SFT_FREE
15297     ;;;
15298     push    di
15299     ;mov     cx,53 ; MSDOS 3.3
15300     ;mov     cx,59 ; MSDOS 6.0
15301     mov     cx,SF_ENTRY.size
15302     rep     stosb
15303     pop     di
15304     sub     ax,ax ; 0 ; clear 'c'-----;
15305     mov     [es:di],ax
15306     ;mov     [es:di+SF_ENTRY.sf_ref_count],ax ; set ref count ;
15307     ;mov     [es:di+15h],ax
15308     mov     [es:di+sf_LRU],ax ; set lru ;
15309     dec     ax ; -1 ;
15310     ;mov     [es:di+17h],ax ; 0FFFFh ; -1
15311     mov     [es:di+sf_OpenAge],ax ; set open age to -1 ;
15312 BlastSFT_retn:
15313     retn ; return with 'c' clear ;
15314
15315 ;Break <CheckFCB - see if the SFT pointed to by the FCB is still OK>
15316 ; -----
15317 ;
15318 ; CheckFCB - examine an FCB and its contents to see if it needs to be
15319 ; regenerated.
15320 ;
15321 ; Inputs: DS:SI point to FCB (not extended)
15322 ; AL is SFT index
15323 ; Outputs: Carry Set - FCB needs to be regened
15324 ; Carry clear - FCB is OK. ES:DI point to SFT
15325 ; Registers modified: AX and BX
15326 ; -----
15327
15328 ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
15329 ; DOSCODE:59F0h (MSDOS 5.0, MSDOS.SYS)
15330
15331 ; 21/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
15332 ; DOSCODE:607Eh (PCDOS 7.1 IBMDOS.COM)
15333
15334 CheckFCB:
15335     ; called from $fcb_open and sftfromfcb. ss already set up to DOSDATA
15336
15337     ; MSDOS 3.3
15338
15339     ; LES DI,[CS:SFTFCB]
15340
15341     ; MSDOS 6.0
15342
15343     ; SR;
15344     ; We check if the given FCB is for a local file. If so, we return a
15345     ; bad SFT status forcing the caller to regenerate the SFT.
15346
15347     ;test     byte [si+19h],0C0h
15348     test     byte [si+fcb_l_drive],FCBNETWORK|FCBSHARE|FCBDEVICE
15349     jz       short BadSFT ;Local file, return bad SFT
15350     LES     DI,[SS:SFTFCB] ; SS override
15351
15352     ; MSDOS 6.0 (& MSDOS 3.3)
15353     ;cmp     [es:di+4],al
15354     CMP     [ES:DI+SFT.SFCount],AL
15355     JC      short BadSFT
15356     ;;mov     b1,53 ; MSDOS 3.3
15357     ;mov     b1,59 ; MSDOS 6.0
15358     MOV     BL,SF_ENTRY.size
15359     MUL     BL
15360     ;lea     di,[di+6]
15361     LEA     DI,[DI+SFT.SFTable]
15362     ADD     DI,AX

```

```

15362      ;MOV     AX,[CS:PROC_ID]      ; MSDOS 3.3
15363 0000214A 36A1[3C03]      mov     ax,[SS:PROC_ID] ; MSDOS 6.0 ; SS override
15364      ;cmp     [es:di+31h],ax
15365 0000214E 26394531      cmp     [ES:DI+SF_ENTRY.sf_PID],AX
15366 00002152 7546      jnz     short BadSFT ; must match process
15367 00002154 26833D00      cmp     word [es:di],0
15368      ;CMP     word [ES:DI+SF_ENTRY.sf_ref_count],0
15369 00002158 7440      jz     short BadSFT ; must also be in use
15370      ;mov     al,[si+19h]
15371 0000215A 8A4419      MOV     AL,[SI+fcb_l_drive]
15372      ;test    al,80h
15373 0000215D A880      test    AL,FCBSPECIAL ; a special FCB?
15374 0000215F 7427      jz     short CheckNoShare ; No. try local or device
15375
15376      ; Since we are a special FCB, try NOT to use a bogus test instruction.
15377      ; FCBSHARE is a superset of FCBNETWORK.
15378
15379 00002161 50      PUSH     AX
15380      ;and     al,0C0h
15381 00002162 24C0      AND     AL,FCBMASK
15382      ;cmp     al,0C0h
15383 00002164 3CC0      CMP     AL,FCBSHARE ; net FCB?
15384 00002166 58      POP     AX
15385 00002167 7515      JNZ     short CheckNet ; no
15386      ; yes
15387
15388      ;----- In share support -----
15389      ;
15390      ;call     far [cs:JShare+(11*4)]
15391 00002169 36FF1E[BC00]      Call     far [ss:JShare+(11*4)] ; 11 = ShChk ; SS Override
15392 0000216E 722A      JC     short BadSFT
15393
15394      ; 21/01/2024
15395 %if 0
15396      JMP     SHORT CheckD
15397
15398      ;----- End in share support -----
15399      ;
15400      ; 09/11/2022
15401      ; (There is not any procedure/sub
15402      ; which calls or jumps to CheckFirClus here)
15403      ;;;
15404 CheckFirClus:
15405      ;cmp     bx,[es:di+0Bh]
15406      ; 07/12/2022
15407      CMP     BX,[ES:DI+SF_ENTRY.sf_firclus]
15408      JNZ     short BadSFT
15409      ;;;
15410 %endif
15411
15412 CheckD:
15413 00002170 243F      AND     AL,3Fh
15414      ;mov     ah,[es:di+5]
15415 00002172 268A6505      MOV     AH,[ES:DI+SF_ENTRY.sf_flags]
15416 00002176 80E43F      AND     AH,3Fh
15417 00002179 38C4      CMP     AH,AL
15418      ; 26/06/2024
15419      ; 16/12/2022
15420      ;jz     short BlastSFT_retn ; carry is clear
15421      ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
15422 0000217B 751D      jnz     short BadSFT
15423
15424 CheckD_retn:
15425      retn
15426
15427 ; 26/06/2024
15428 ;BadSFT:
15429 ; STC
15430 ; retn
15431
15432 CheckNet:
15433      ; 17/05/2019 - Retro DOS v4.0
15434
15435 ;----- In net support -----
15436
15437      ; MSDOS 3.3
15438      ;;mov     ax,[si+1Ah]
15439      ;mov     ax,[si+fcb_net_handle]
15440      ;;cmp     ax,[es:di+1Dh]
15441      ;cmp     ax,[ES:DI+SF_ENTRY.sf_dirsec]
15442      ;jnz     short BadSFT
15443      ;;cmp     ax,[es:di+19h]
15444      ;cmp     ax,[ES:DI+sf_netid]
15445      ;jnz     short BadSFT
15446      ;;mov     ax,[si+1Eh]
15447      ;mov     ax,[si+fcb_l_attr]
15448      ;;cmp     ax,[es:di+1Bh]
15449      ;cmp     ax,[es:di+SF_ENTRY.sf_lstclus]
15450      ;jnz     short BadSFT
15451
15452      ; MSDOS 6.0
15453 0000217E 8B441C      MOV     AX,[SI+fcb_netID] ;AN000;IFS.DOS 4.00
15454      ; 09/11/2022
15455      ;cmp     ax,[es:di+0Bh]
15456 00002181 263B450B      CMP     AX,[ES:DI+sf_serial_ID] ;AN000;IFS.DOS 4.00
15457 00002185 7513      JNZ     short BadSFT
15458
15459 ;----- END In net support -----
15460
15461 CheckNet_retn:
15462 00002187 C3      retn
15463
15464 CheckNoShare:
15465
15466      ; 16/12/2022
15467      ; 09/11/2022 (following test instruction is nonsense!)
15468      ; (I am leaving it here for MSDOS 5.0 MSDOS.SYS compatibility)
15469      ; test    al,40h
15470      ; test    AL,FCBDEVICE ; Device?
15471      ; jnz     short $+2 ; 09/11/2022
15472      ; JNZ     short CheckNoShareDev ; Yes
15473
15474      ; MSDOS 3.3 - IBMDOS.COM - Offset 27EFh
15475      ;;mov     bx,[si+1Dh]
15476      ;MOV     BX,[SI+fcb_ns_dirsec]
15477      ;;cmp     bx,[es:di+1Dh]
15478      ;cmp     bx,[ES:DI+SF_ENTRY.sf_dirsec]
15479      ;jnz     short BadSFT
15480      ;;mov     bl,[si+1Fh]
15481      ;MOV     bl,[SI+fcb_ns_dirpos]
15482      ;;cmp     bl,[es:di+1Fh]
15483      ;cmp     bl,[ES:DI+SF_ENTRY.sf_dirpos]
15484      ;jnz     short BadSFT
15485      ;;mov     bl,[si+1Ah]

```

```

15486 ;MOV     bl,[SI+fcbl_nsl_bits]
15487 ;;mov    bh,[es:di+5]
15488 ;MOV     bh,[ES:DI+SF_ENTRY.sf_flags]
15489 ;xor     bh,bl
15490 ;and     bh,0C0h
15491 ;jnz     short BadSFT
15492 ;;xor    bl,[es:di+2]
15493 ;xor     bl,[ES:DI+SF_ENTRY.sf_mode]
15494 ;and     bl,0Fh
15495 ;jnz     short BadSFT
15496 ;push    di
15497 ;push    si
15498 ;;lea    di,[di+20h] ; MSDOS 3.3
15499 ;LEA     DI,[DI+SF_ENTRY.sf_name]
15500 ;;lea    si,[si+1]
15501 ;LEA     SI,[SI+SYS_FCB.name]
15502 ;;mov    cx,11
15503 ;MOV     CX,SYS_FCB.EXTENT-SYS_FCB.name ; 12-1
15504 ;repe    cmpsb
15505 ;pop     si
15506 ;pop     di
15507 ;jnz     short BadSFT
15508 ;;mov    bx,[si+18h]
15509 ;MOV     BX,[SI+fcbl_nsl_firclus]
15510 ;jmp     short CheckFirClus
15511
15512 ; MSDOS 6.0
15513
15514 ; SR;
15515 ; The code below to match a local FCB with its SFT can no longer be
15516 ; used. We just return a no-match status. This check is done right
15517 ; at the top.
15518
15519 CheckNoShareDev:
15520 ;mov     bx,[si+1Ah]
15521 00002188 8B5C1A MOV     BX,[SI+fcbl_nsl_drvptr]
15522 ;cmp     bx,[es:di+7]
15523 0000218B 263B5D07 CMP     BX,[ES:DI+SF_ENTRY.sf_devptr]
15524 0000218F 7509 JNZ     short BadSFT
15525 ;mov     bx,[si+1Ch]
15526 00002191 8B5C1C MOV     BX,[SI+fcbl_nsl_drvptr+2]
15527 ;cmp     bx,[es:di+9]
15528 00002194 263B5D09 CMP     BX,[ES:DI+SF_ENTRY.sf_devptr+2]
15529 ;JNZ     short BadSFT
15530 ;JMP     short CheckD
15531 ; 26/06/2024
15532 00002198 74D6 jz      short CheckD
15533
15534 ; 26/06/2024
15535 BadSFT:
15536 0000219A F9 STC
15537 0000219B C3 retn
15538
15539 ;Break <SFTFromFCB - take a FCB and obtain a SFT from it>
15540 ;-----
15541 ;
15542 ; SFTFromFCB - the workhorse of this compatability crap. Check to see if
15543 ; the SFT for the FCB is Good. If so, make ThisSFT point to it. If not
15544 ; good, get one from the cache and regenerate it. Overlay the LRU field
15545 ; with PID
15546 ;
15547 ; Inputs: DS:SI point to FCB
15548 ; Outputs: ThisSFT point to appropriate SFT
15549 ; Carry clear -> OK ES:DI -> SFT
15550 ; Carry set -> error in ax
15551 ; Registers modified: ES,DI, AX
15552 ;-----
15553
15554 SFTFromFCB:
15555 ; called from fcbio and $fcb_close. SS already set up to DOSDATA
15556 ; 17/05/2019 - Retro DOS v4.0
15557
15558
15559
15560 0000219C 50 push    ax
15561 0000219D 53 push    bx
15562 ;mov     al,[si+18h]
15563 0000219E 8A4418 MOV     AL,[SI+fcbl_sfn] ; set SFN for check
15564 000021A1 E88CFF call    CheckFCB
15565 000021A4 5B pop     bx
15566 000021A5 58 pop     ax
15567 ;MOV     [CS:THISSFT],DI ; SS override
15568 ;MOV     [CS:THISSFT+2],ES ; SS override
15569 000021A6 36893E[9E05] MOV     [SS:THISSFT],DI ; SS override
15570 000021AB 368C06[A005] MOV     [SS:THISSFT+2],ES
15571 000021B0 7311 JNC     short Set_SFT ; no problems, just set thissft
15572
15573 ; 09/11/2022 (MSDOS 5.0)
15574 ; 31/05/2019
15575 000021B2 06 push    es ; * (MSDOS 6.21) & (MSDOS 5.0)
15576 000021B3 E8A2E2 call    save_world
15577 000021B6 E860FE call    FCBRegen
15578 000021B9 E885E2 call    restore_world ; use macro restore world
15579 000021BC 07 pop     es ; * (MSDOS 6.21) ; 31/05/2019 ; 09/11/2022 (MSDOS 5.0)
15580
15581 ;MOV     AX,[CS:EXTERR] ; SS override
15582 000021BD 36A1[2403] MOV     AX,[SS:EXTERR] ; SS override
15583 000021C1 72C4 jc      short CheckNet_retn
15584
15585 Set_SFT:
15586 ;LES     DI,[CS:THISSFT] ; SS override for THISSFT & PROC_ID
15587 000021C3 36C43E[9E05] les     di,[ss:THISSFT]
15588 ;PUSH    word [CS:PROC_ID] ; set process id
15589 000021C8 36FF36[3C03] push    word [ss:PROC_ID]
15590 ;pop     word [es:di+31h]
15591 000021CD 268F4531 POP     word [ES:DI+SF_ENTRY.sf_PID]
15592 000021D1 C3 retn ; carry is clear
15593
15594 ;Break <FCBHardErr - generate INT 24 for hard errors on FCBS>
15595 ;-----
15596 ;
15597 ; FCBHardErr - signal to a user app that he is trying to use an
15598 ; unavailable FCB.
15599 ;
15600 ; Inputs: none.
15601 ; Outputs: none.
15602 ; Registers modified: all
15603 ;-----
15604
15605 ; 21/01/2024 - Retro DOS v5.0
15606 FCBHardErr:
15607 ; 17/05/2019 - Retro DOS v4.0
15608 mov     es,[cs:DosDSeg]
15609 000021D2 2E8E06[0700]

```



```

15610 ;
15611 ;mov ax,23h
15612 000021D7 B82300 MOV AX,error_FCB_unavailable
15613 ;;mov byte [cs:ALLOWED],8
15614 ;MOV byte [CS:ALLOWED],Allowed_FAIL
15615 000021DA 26C606[4B03]08 mov byte [es:ALLOWED],Allowed_FAIL
15616 ;
15617 ;LES BP,[CS:THISDPB]
15618 000021E0 26C42E[8A05] les bp,[es:THISDPB]
15619 ;
15620 000021E5 BF0100 MOV DI,1 ; Fake some registers
15621 000021E8 89F9 MOV CX,DI
15622 ;;;
15623 ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15624 000021EA 31D2 xor dx,dx ; 0
15625 ;cmp [es:bp+0Fh],dx
15626 000021EC 2639560F cmp [es:bp+DPB.FAT_SIZE],dx ; 0
15627 000021F0 740B jz short fcbharderr_fat32 ; FAT32
15628 000021F2 268916[0706] mov [es:HIGH_SECTOR],dx ; 0
15629 ;mov dx,[es:bp+0Bh]
15630 000021F7 268B560B mov dx,[es:bp+DPB.FIRST_SECTOR]
15631 000021FB EB0D jmp short fcbharderr_fat
15632 fcbharderr_fat32:
15633 ;mov dx,[es:bp+2Bh]
15634 000021FD 268B562B mov dx,[es:bp+DPB.FCLUS_FSECTOR+2]
15635 00002201 268916[0706] mov [es:HIGH_SECTOR],dx
15636 ;mov dx,[es:bp+29h]
15637 00002206 268B5629 mov dx,[es:bp+DPB.FCLUS_FSECTOR]
15638 fcbharderr_fat:
15639 ;;;
15640 ; 21/01/2024
15641 ;;mov dx,[es:bp+0Bh]
15642 ;MOV DX,[ES:BP+DPB.FIRST_SECTOR]
15643 ;
15644 0000220A E8423E call HARDERR
15645 0000220D F9 STC
15646 0000220E C3 retn
15647 ;
15648 ;=====
15649 ; FCBI02.ASM, MSDOS 6.0, 1991
15650 ;=====
15651 ; 21/07/2018 - Retro DOS v3.0
15652 ; 17/05/2019 - Retro DOS v4.0
15653 ;
15654 ;** FCBI02.ASM - Ancient 1.0 1.1 FCB system calls
15655 ;
15656 ; GetRR
15657 ; GetExtent
15658 ; SetExtent
15659 ; GetExtended
15660 ; GetRecSize
15661 ; FCBIO
15662 ; $FCB_OPEN
15663 ; $FCB_CREATE
15664 ; $FCB_RANDOM_WRITE_BLOCK
15665 ; $FCB_RANDOM_READ_BLOCK
15666 ; $FCB_SEQ_READ
15667 ; $FCB_SEQ_WRITE
15668 ; $FCB_RANDOM_READ
15669 ; $FCB_RANDOM_WRITE
15670 ;
15671 ; Revision history:
15672 ;
15673 ; Created: ARR 4 April 1983
15674 ; MZ 6 June 1983 completion of functions
15675 ; MZ 15 Dec 1983 Brain damaged programs close FCBs multiple
15676 ; times. Change so successive closes work by
15677 ; always returning OK.Also, detect I/O to
15678 ; already closed FCB and return EOF.
15679 ; MZ 16 Jan 1984 More braindamage. Need to separate info
15680 ; out of sft into FCB for reconnection
15681 ;
15682 ; A000 version 4.00 Jan. 1988
15683 ;
15684 ; Defintions for FCBop flags
15685 ;
15686 RANDOM equ 2 ; random operation
15687 FCBREAD equ 4 ; doing a read
15688 BLOCK equ 8 ; doing a block I/O
15689 ;
15690 ;Break <GetRR - return the random record field in DX:AX>
15691 ;-----
15692 ;
15693 ; GetRR - correctly load DX:AX with the random record field (3 or 4 bytes)
15694 ; from the FCB pointed to by DS:SI
15695 ;
15696 ; Inputs: DS:SI point to an FCB
15697 ; BX has record size
15698 ; Outputs: DX:AX contain the contents of the random record field
15699 ; Registers modified: none
15700 ;-----
15701 ;
15702 GetRR:
15703 ;mov ax,[si+21h]
15704 0000220F 8B4421 MOV AX,[SI+SYS_FCB.RR] ; get low order part
15705 ;mov dx,[si+23h]
15706 00002212 8B5423 MOV DX,[SI+SYS_FCB.RR+2] ; get high order part
15707 00002215 83FB40 CMP BX,64 ; ignore MSB of RR if recsiz > 64
15708 00002218 7202 JB short GetRRBye
15709 GetExtent_bye: ; 21/01/2024
15710 0000221A 30F6 XOR DH,DH
15711 GetRRBye:
15712 0000221C C3 retn
15713 ;
15714 ;Break <GetExtent - retrieve next location for sequential IO>
15715 ;-----
15716 ;
15717 ; GetExtent - Construct the next record to perform I/O from the EXTENT and
15718 ; NR fields in the FCB.
15719 ;
15720 ; Inputs: DS:SI - point to FCB
15721 ; Outputs: DX:AX contain the contents of the random record field
15722 ; Registers modified: none
15723 ;-----
15724 ;
15725 GetExtent:
15726 ;mov al,[si+20h]
15727 0000221D 8A4420 MOV AL,[SI+SYS_FCB.NR] ; get low order piece
15728 ;mov dx,[si+0Ch]
15729 00002220 8B540C MOV DX,[SI+SYS_FCB.EXTENT]; get high order piece
15730 00002223 D0E0 SHL AL,1
15731 00002225 D1EA SHR DX,1
15732 00002227 D0D8 RCR AL,1 ; move low order bit of DL to high order of AH
15733 00002229 88D4 MOV AH,DL

```

```

15734 0000222B 88F2      MOV     DL,DH
15735                  ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15736                  ;XOR     DH,DH
15737                  ;retn
15738 0000222D EBEB      jmp      short GetExtent_bye
15739
15740                  ;Break <SetExtent - update the extent/NR field>
15741                  ;-----
15742                  ;
15743                  ; SetExtent - change the position of an FCB by filling in the extent/NR
15744                  ; fields
15745                  ;
15746                  ; Inputs: DS:SI point to FCB
15747                  ;       DX:AX is a record location in file
15748                  ; Outputs: Extent/NR fields are filled in
15749                  ; Registers modified: CX
15750                  ;-----
15751
15752 SetExtent:
15753 0000222F 50          push    ax
15754 00002230 52          push    dx
15755 00002231 89C1       MOV     CX,AX
15756 00002233 247F       AND     AL,7FH          ; next rec field
15757                  ;mov     [si+20h],al
15758 00002235 884420     MOV     [SI+SYS_FCB.NR],AL
15759 00002238 80E180     AND     CL,80H          ; save upper bit
15760 0000223B D1E1       SHL     CX,1
15761 0000223D D1D2       RCL     DX,1          ; move high bit of CX to low bit of DX
15762 0000223F 88E8       MOV     AL,CH
15763 00002241 88D4       MOV     AH,DL
15764                  ;mov     [si+0Ch], ax
15765 00002243 89440C     MOV     [SI+SYS_FCB.EXTENT],AX; all done
15766 00002246 5A          pop     dx
15767 00002247 58          pop     ax
15768 00002248 C3          retn
15769
15770                  ;Break <GetExtended - find FCB in potential extended fcb>
15771                  ;-----
15772                  ;
15773                  ; GetExtended - Make DS:SI point to FCB from DS:DX
15774                  ;
15775                  ; Inputs: DS:DX point to a possible extended FCB
15776                  ; Outputs: DS:SI point to the FCB part
15777                  ;       zeroflag set if not extended fcb
15778                  ; Registers modified: SI
15779                  ;-----
15780
15781 GetExtended:
15782 00002249 89D6       MOV     SI,DX          ; point to Something
15783 0000224B 803CFF     CMP     BYTE [SI],-1      ; look for extension
15784 0000224E 7503       JNZ     short GetBye      ; not there
15785 00002250 83C607     ADD     SI,7          ; point to FCB
15786
15787 00002253 39D6       CMP     SI,DX          ; set condition codes
15788
15789 00002255 C3          getextd_retn: retn
15790
15791                  ;Break <GetRecSize - return in BX the FCB record size>
15792                  ;-----
15793                  ;
15794                  ; GetRecSize - return in BX the record size from the FCB at DS:SI
15795                  ;
15796                  ; Inputs: DS:SI point to a non-extended FCB
15797                  ; Outputs: BX contains the record size
15798                  ; Registers modified: None
15799                  ;-----
15800
15801                  ; 22/01/2024
15802                  ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
15803
15804 GetRecSize:
15805 00002256 8B5C0E     ;mov     bx,[si+0Eh]
15806 00002259 09DB       MOV     BX,[SI+SYS_FCB.RECSIZ]; get his record size
15807                  OR     BX,BX          ; is it nul?
15808                  ;jz      short getextd_retn
15809 0000225B 75F8       ; 22/01/2024 (BugFix)
15810                  jnz     short getextd_retn
15811 0000225D B380       ;MOV     BX,128          ; use default size
15812                  mov     b1,128      ; (PCDOS 7.1 IBMDOS.COM)
15813 0000225F 895C0E     ;mov     [si+0Eh],bx
15814 00002262 C3          MOV     [SI+SYS_FCB.RECSIZ],BX; stuff it back
15815                  retn
15816
15817                  ; 23/01/2024 - Retro DOS v5.0
15818                  ; PC DOS 7.1 IBMDOS.COM - DOSCODE:61B3h
15819
15820                  ; 22/07/2018 - Retro DOS v3.0
15821
15822                  ;BREAK <$FCB_Random_write_Block - write a block of records to a file >
15823                  ;-----
15824                  ;
15825                  ; $FCB_Random_Write_Block - retrieve a location from the FCB, seek to it
15826                  ; and write a number of blocks from it.
15827                  ;
15828                  ; Inputs: DS:DX point to an FCB
15829                  ; Outputs: AL = 0 write was successful and the FCB position is updated
15830                  ;       AL <> 0 Not enough room on disk for the output
15831                  ;-----
15832
15833 _$FCB_RANDOM_WRITE_BLOCK:
15834                  ;mov     AL,0Ah
15835 00002263 B00A       MOV     AL,RANDOM+BLOCK
15836 00002265 EB12       JMP     short FCBI0      ; 23/01/2024
15837
15838                  ;BREAK <$FCB_Random_Read_Block - read a block of records to a file >
15839                  ;-----
15840                  ;
15841                  ; $FCB_Random_Read_Block - retrieve a location from the FCB, seek to it
15842                  ; and read a number of blocks from it.
15843                  ;
15844                  ; Inputs: DS:DX point to an FCB
15845                  ; Outputs: AL = error codes defined above
15846                  ;-----
15847
15848 _$FCB_RANDOM_READ_BLOCK:
15849                  ;mov     AL,0Eh
15850 00002267 B00E       MOV     AL,RANDOM+FCBREAD+BLOCK
15851 00002269 EB0E       JMP     short FCBI0      ; 23/01/2024
15852
15853                  ;BREAK <$FCB_Seq_Read - read the next record from a file >
15854                  ;-----
15855                  ;
15856                  ; $FCB_Seq_Read - retrieve the next record from an FCB and read it into
15857

```



```

15858 ; memory
15859 ;
15860 ; Inputs: DS:DX point to an FCB
15861 ; Outputs: AL = error codes defined above
15862 ;
15863 ;-----
15864
15865 _$FCB_SEQ_READ:
15866 ;mov AL,4
15867 0000226B B004 MOV AL,FCBREAD
15868 0000226D EB0A JMP short FCBI0 ; 23/01/2024
15869
15870 ;BREAK <$FCB_Seq_write - write the next record to a file >
15871 ;-----
15872 ;
15873 ; $FCB_Seq_write - retrieve the next record from an FCB and write it to the
15874 ; file
15875 ;
15876 ; Inputs: DS:DX point to an FCB
15877 ; Outputs: AL = error codes defined above
15878 ;
15879 ;-----
15880
15881 _$FCB_SEQ_WRITE:
15882 0000226F B000 MOV AL,0
15883 00002271 EB06 JMP short FCBI0 ; 23/01/2024
15884
15885 ;BREAK <$FCB_Random_Read - Read a single record from a file >
15886 ;-----
15887 ;
15888 ; $FCB_Random_Read - retrieve a location from the FCB, seek to it and read a
15889 ; record from it.
15890 ;
15891 ; Inputs: DS:DX point to an FCB
15892 ; Outputs: AL = error codes defined above
15893 ;
15894 ;-----
15895
15896 _$FCB_RANDOM_READ:
15897 ;mov AL,6
15898 00002273 B006 MOV AL,RANDOM+FCBREAD
15899 ; 23/01/2024
15900 ;jmp FCBI0 ; single block
15901 00002275 EB02 jmp short FCBI0
15902
15903 ;BREAK <$FCB_Random_Write - write a single record to a file >
15904 ;-----
15905 ;
15906 ; $FCB_Random_Write - retrieve a location from the FCB, seek to it and write
15907 ; a record to it.
15908 ;
15909 ; Inputs: DS:DX point to an FCB
15910 ; Outputs: AL = error codes defined above
15911 ;
15912 ;-----
15913
15914 _$FCB_RANDOM_WRITE:
15915 ;mov AL,2
15916 00002277 B002 MOV AL,RANDOM
15917 ; 23/01/2024
15918 ;;jmp FCBI0
15919 ;jmp short FCBI0
15920
15921 ;BREAK <FCBI0 - do internal FCB I/O>
15922 ;-----
15923 ;
15924 ; FCBI0 - look at FCBOP and merge all FCB operations into a single routine.
15925 ;
15926 ; Inputs: FCBOP flags which operations need to be performed
15927 ; DS:DX point to FCB
15928 ; CX may have count of number of records to xfer
15929 ; Outputs: AL has error code
15930 ; Registers modified: all
15931 ;-----
15932 ;
15933 ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
15934 ; DOSCODE:5B17h (MSDOS 5.0 MSDOS.SYS)
15935 ;
15936 ; 23/01/2024
15937 ; DOSCODE:5B2Bh (MSDOS 6.22 MSDOS.SYS)
15938 ;
15939 ; 23/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
15940 ; DOSCODE:61C9h (PCDOS 7.1 IBMDOS.COM)
15941
15942 FCBI0:
15943
15944 FE0FEQU 1
15945 FTRIM EQU 2
15946
15947 %define FCBErr byte [bp-1] ; byte
15948 %define cRec word [bp-3] ; word
15949 ;%define RecPos word [bp-7] ; dword
15950 %define RecPosL word [bp-7] ; word
15951 %define RecPosH word [bp-5] ; word
15952 %define RecSize word [bp-9] ; word
15953 ;%define bPos word [bp-13] ; dword
15954 %define bPosL word [bp-13] ; word
15955 %define bPosH word [bp-11] ; word
15956 %define cByte word [bp-15] ; word
15957 %define cResult word [bp-17] ; word
15958 %define cRecRes word [bp-19] ; word
15959 %define FCBOp byte [bp-20] ; byte
15960 ; 23/01/2024
15961 %define bPos bp-13
15962
15963 ;Enter
15964
15965 00002279 55 push bp
15966 0000227A 89E5 mov bp,sp
15967 0000227C 83EC14 sub sp,20
15968 ;mov [bp-20],al
15969 0000227F 8846EC MOV FCBOp,AL
15970 ;mov byte [bp-1],0
15971 00002282 C646FF00 MOV FCBErr,0 ; FCBErr = 0;
15972 00002286 E8C0FF call GetExtended ; FCB = GetExtended ();
15973 ;test byte [bp-20],8
15974 00002289 F646EC08 TEST FCBOp,BLOCK ; if ((OP&BLOCK) == 0)
15975 0000228D 7503 JNZ short GetPos
15976 0000228F B90100 MOV CX,1 ; cRec = 1;
15977
15978 GetPos:
15979 00002292 894EFD ;mov [bp-3],cx ;*Tail coalesce
15980 00002295 E885FF MOV cRec,CX ; RecPos = GetExtent ();
15981 00002298 E8BBFF call GetExtent ; RecSize = GetRecSize ();

```

```

15982      ;mov     [bp-9],bx
15983 0000229B 895EF7      MOV     RecSize,BX
15984      ;test    byte [bp-20],2
15985 0000229E F646EC02     TEST    FCBOp,RANDOM      ; if ((OP&RANDOM) <> 0)
15986 000022A2 7403      JZ      short GetRec
15987 000022A4 E868FF     call   GetRR              ; RecPos = GetRR ();
15988
15989      GetRec:
15990      ;mov     [bp-7],ax
15991      MOV     RecPosL,AX      ;*Tail coalesce
15992      ;mov     [bp-5],dx
15993      MOV     RecPosH,DX
15994      call   SetExtent      ; SetExtent (RecPos);
15995      ;mov     ax,[bp-5]
15996      MOV     AX,RecPosH      ; bPos = RecPos * RecSize;
15997      MUL     BX
15998      MOV     DI,AX
15999      ;mov     ax,[bp-7]
16000      MOV     AX,RecPosL
16001      MUL     BX
16002      ADD     DX,DI
16003      ;mov     [bp-13],ax
16004      MOV     bPosL,AX
16005      ;mov     [bp-11],dx
16006      MOV     bPosH,DX
16007      ;mov     ax,[bp-3]
16008      MOV     AX,cRec        ; cByte = cRec * RecSize;
16009      MUL     BX
16010      ;mov     [bp-15],ax
16011      MOV     cByte,AX
16012
16013      ;hkn;      SS override
16014      ADD     AX,[SS:DMAADD]   ; if (cByte+DMA > 64K) {
16015      ADC     DX,0
16016      JZ      short DoOper
16017      ;mov     byte [bp-1],2
16018      MOV     FCBErr,FTRIM    ; FCBErr = FTRIM;
16019
16020      ;hkn;      SS override
16021      MOV     AX,[SS:DMAADD]   ; cRec = (64K-DMA)/RecSize;
16022      NEG     AX
16023      JNZ     short DoDiv
16024      DEC     AX
16025
16026      DoDiv:
16027      XOR     DX,DX
16028      DIV     BX
16029      ;mov     [bp-3],ax
16030      MOV     cRec,AX
16031      MUL     BX              ; cByte = cRec * RecSize;
16032      ;mov     [bp-15],ax
16033      MOV     cByte,AX        ; }
16034
16035      DoOper:
16036      XOR     BX,BX
16037      ;mov     [bp-17],bx
16038      MOV     cResult,BX      ; cResult = 0;
16039      ;cmp     [bp-15],bx
16040      CMP     cByte,BX        ; if (cByte <> 0 ||
16041      JNZ     short DoGetExt
16042      ;test    byte [bp-1],2
16043      TEST    FCBErr,FTRIM    ; (FCBErr&FTRIM) == 0) {
16044      JZ      short DoGetExt
16045      ;JMP     short SkipOp
16046      ; 16/12/2022
16047      jnz     short SkipOp
16048      ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
16049      ;JZ      short DoGetExt
16050      ;JMP     short SkipOp
16051
16052      DoGetExt:
16053      call   SFTFromFCB      ; if (!SFTFromFCB (SFT,FCB))
16054      JNC     short ContinueOp
16055
16056      FCBDeath:
16057      call   FCB_RET_ERR      ; signal error, map for extended
16058      ;mov     word [bp-19],0
16059      MOV     cRecRes,0        ; no bytes transferred
16060      ;mov     byte [bp-1],1
16061      MOV     FCBErr,FE0F      ; return FTRIM;
16062      JMP     FCBSave        ; bam!
16063
16064      ContinueOp:
16065      ; 23/01/2024
16066      ; (PCDOS 7.1 IBMDOS.COM)
16067      ;
16068      ;;mov     ax,[si+10h]
16069      ;MOV     AX,[SI+SYS_FCB.FILSIZ]
16070      ;;mov     [es:di+11h],ax
16071      ;MOV     [ES:DI+SF_ENTRY.sf_size],AX
16072      ;;mov     ax,[si+12h]
16073      ;MOV     AX,[SI+SYS_FCB.FILSIZ+2]
16074      ;;mov     [es:di+13h],ax
16075      ;MOV     [ES:DI+SF_ENTRY.sf_size+2],AX
16076      ;;
16077      push    ds
16078      lds     ax,[si+SYS_FCB.FILSIZ]
16079      mov     [es:di+SF_ENTRY.sf_size],ax
16080      mov     [es:di+SF_ENTRY.sf_size+2],ds
16081      lds     ax,[bPos] ; lds ax,[bp-13]
16082      mov     dx,ds
16083      pop     ds
16084      ;;
16085      ;;mov     ax,[bp-13]
16086      ;MOV     AX,bPosL
16087      ;;mov     dx,[bp-11]
16088      ;MOV     DX,bPosH
16089
16090      ;mov     [es:di+15h],ax
16091      MOV     [ES:DI+SF_ENTRY.sf_position],AX
16092      ;xchg     dx,[es:di+17h]
16093      XCHG    [ES:DI+SF_ENTRY.sf_position+2],DX
16094      PUSH    DX              ; save away Open age.
16095      ;mov     cx,[bp-15]
16096      MOV     CX,cByte        ; cResult =
16097
16098      ;hkn; DOS_Read is in DOSCODE
16099      MOV     DI,DOS_READ      ;
16100      ;test    byte [bp-20],4
16101      TEST    FCBOp,FCBREAD    ;
16102      JNZ     short DoContext  ; : DOS_write)(cRec);
16103
16104      ;hkn; DOS_Write is in DOSCODE
16105      MOV     DI,DOS_WRITE
16106
16107      DoContext:
16108      push    bp
16109      push    ds
16110      push    si
16111
16112      ;hkn; SS is DOSDATA

```

```

16106 00002340 16      push    ss
16107 00002341 1F      pop     ds
16108
16109      ;; Fix for disk full
16110 00002342 FFD7      CALL    DI      ; DOS_READ or DOS_WRITE
16111
16112 00002344 5E      pop     si
16113 00002345 1F      pop     ds
16114 00002346 5D      pop     bp
16115 00002347 72BB      JC      short FCBDeath
16116
16117 00002349 36803E[0B06]00      CMP     BYTE [SS:DISK_FULL],0 ; treat disk full as error
16118 0000234F 7406      JZ      short NODSKFULL
16119 00002351 36C606[0B06]00      MOV     BYTE [SS:DISK_FULL],0 ; clear the flag
16120
16121      ; 23/01/2024
16122      ; (PCDOS 7.1 IBMDOS.COM)
16123      ;;mov    byte [bp-1],1
16124      ;MOV     FCBErr,FE0F      ; set disk full flag
16125
16126 NODSKFULL:
16127      ;; Fix for disk full
16128      ;mov     [bp-17],cx
16129      MOV     cResult,cx
16130 0000235A E80BFB      call    SaveFCBInfo      ; SaveFCBInfo (FCB);
16131      ;pop     word [es:di+17h]
16132 0000235D 268F4517      POP     WORD [ES:DI+SF_ENTRY.sf_position+2] ; restore open age
16133      ; (sf_OpenAge = SF_ENTRY.sf_position+2)
16134
16135      ; 23/01/2024
16136      ; (PCDOS 7.1 IBMDOS.COM)
16137      ;
16138      ;;mov    ax,[es:di+11h]
16139      ;MOV     AX,[ES:DI+SF_ENTRY.sf_size]
16140      ;;mov    [si+10h],ax
16141      ;MOV     [SI+SYS_FCB.FILSIZ],AX
16142      ;;mov    ax,[es:di+13h]
16143      ;MOV     AX,[ES:DI+SF_ENTRY.sf_size+2]
16144      ;;mov    [si+12h],ax
16145      ;MOV     [SI+SYS_FCB.FILSIZ+2],AX
16146      ;;
16147 00002361 06      push    es
16148 00002362 26C44511      les     ax,[es:di+SF_ENTRY.sf_size]
16149 00002366 894410      mov     [si+SYS_FCB.FILSIZ],ax
16150 00002369 8C4412      mov     [si+SYS_FCB.FILSIZ+2],es
16151 0000236C 07      pop     es
16152      ;;
16153      ;      }
16154
16155 skipOp:
16156      ;mov     ax,[bp-17]
16157      MOV     AX,cResult      ; cRecRes = cResult / RecSize;
16158      XOR     DX,DX
16159      ;div     word [bp-9]
16160      DIV     RecSize
16161      ;mov     [bp-19],ax
16162      MOV     cRecRes,AX
16163      ;add     [bp-7],ax
16164      ADD     RecPosL,AX      ; RecPos += cRecResult;
16165      ;adc     word [bp-5],0
16166      ADC     RecPosH,0
16167
16168      ; If we have not gotten the expected number of records, we signal an EOF
16169      ; condition. On input, this is EOF. On output this is usually disk full.
16170      ; BUT... Under 2.0 and before, all device output IGNORED this condition. So
16171      ; do we.
16172
16173      ;cmp     ax,[bp-3]
16174      CMP     AX,cRec      ; if (cRecRes <> cRec)
16175      JZ      short TryBlank
16176      ;test    byte [bp-20],4
16177      TEST    FCBOp,FCBREAD      ; if (OP&FCBRead || !DEVICE)
16178      JNZ     short SetEOF
16179      ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
16180      ; MSDOS 3.3
16181      ;;test    word [es:di+5],80h
16182      ;TEST    word [ES:DI+SF_ENTRY.sf_flags],devid_device
16183      ;JNZ     short TryBlank
16184      ; MSDOS 5.0 & MSDOS 6.0
16185      ;test    byte [es:di+5],80h
16186      test    byte [ES:DI+SF_ENTRY.sf_flags],devid_device
16187      jnz     short TryBlank
16188
16189 SetEOF:
16190      ;mov     byte [bp-1],1
16191      MOV     FCBErr,FE0F      ; FCBErr = FE0F;
16192
16193 TryBlank:
16194      OR     DX,DX      ; if (cResult%RecSize <> 0) {
16195      JZ      short SetExt
16196      ;add     word [bp-7],1
16197      ADD     RecPosL,1      ; RecPos++;
16198      ;adc     word [bp-5],0
16199      ADC     RecPosH,0
16200      ;test    byte [bp-20],4
16201      TEST    FCBOp,FCBREAD      ; if(OP&FCBRead) <> 0) {
16202      JZ      short SetExt
16203      ;inc     word [bp-19]
16204      INC     cRecRes      ; cRecRes++;
16205      ;mov     byte [bp-1],3
16206      MOV     FCBErr,FTRIM+FE0F      ; FCBErr = FTRIM | FE0F;
16207      ;mov     cx,[bp-9]
16208      MOV     CX,[bp-9]
16209      ;mov     CX,RecSize
16210      SUB     CX,DX      ; Blank (RecSize-cResult%RecSize,
16211      XOR     AL,AL      ; DMA+cResult);
16212
16213 ;hkn;
16214      les     di,[ss:DMAADD]
16215      ;add     di,[bp-17]
16216      ADD     DI,cResult
16217      REP     STOSB      ; } }
16218
16219 SetExt:
16220      ;mov     dx,[bp-5]
16221      MOV     DX,RecPosH
16222      ;mov     ax,[bp-7]
16223      MOV     AX,RecPosL
16224      ;test    byte [bp-20],2
16225      TEST    FCBOp,RANDOM      ; if ((OP&Random) == 0 ||
16226      JZ      short DoSetExt
16227      ;test    byte [bp-20],8
16228      TEST    FCBOp,BLOCK      ; (OP&BLOCK) <> 0)
16229      JZ      short TrySetRR
16230
16231 DoSetExt:
16232      call    SetExtent      ; SetExtent (RecPos, FCB);
16233
16234 TrySetRR:
16235      ;test    byte [bp-20],8
16236      TEST    FCBOp,BLOCK      ; if ((op&BLOCK) <> 0)

```

```

16230 000023D8 740F      JZ      short TryReturn
16231      ;mov     [si+21h],ax
16232 000023DA 894421     MOV     [SI+SYS_FCB.RR],AX      ;      FCB->RR = RecPos;
16233      ;mov     [si+23h],dl
16234 000023DD 885423     MOV     [SI+SYS_FCB.RR+2],DL
16235      ;cmp     word [si+0Eh],64
16236 000023E0 837C0E40    CMP     word [SI+SYS_FCB.RECSIZ],64
16237 000023E4 7303      JAE     short TryReturn
16238      ;mov     [si+24h],dh
16239 000023E6 887424     MOV     [SI+SYS_FCB.RR+2+1],DH; Set 4th byte only if record size < 64
16240
16241      TryReturn:
16242      ;test     byte [bp-20],4
16243 000023E9 F646EC04    TEST     FCBop,FCBREAD      ;      if (!(FCBOP & FCBREAD)) {
16244 000023ED 750B      JNZ     short FCBSave
16245 000023EF 1E      push     ds      ;      FCB->FDate = date;
16246 000023F0 E86DE7     call     DATE16      ;      FCB->FTime = time;
16247      ;pop     ds
16248 000023F4 894414     MOV     [SI+SYS_FCB.FDATE],AX
16249      ;mov     [si+16h],dx
16250 000023F7 895416     MOV     [SI+SYS_FCB.FTIME],DX ;      }
16251
16252      FCBSave:
16253 000023FA F646EC08    ;test     byte [bp-20],8
16254 000023FE 7409      TEST     FCBop,BLOCK      ;      if ((op&BLOCK) <> 0)
16255      ;jz      short DoReturn
16256 00002400 8B4EED     MOV     CX,[bp-19]
16257 00002403 E871E0     call     CX,cRecRes      ;      user_CX = cRecRes;
16258      ;call     Get_User_Stack
16259 00002406 894C04     MOV     [SI+user_env.user_CX],CX
16260
16261      DoReturn:
16262 00002409 8A46FF     MOV     al,[bp-1]
16263      MOV     AL,FCBErr      ;      return (FCBERR);
16264      ;leave
16265      mov     sp,bp
16266 0000240F C3      pop     bp
16267      retn
16268
16269      ; 22/07/2018 - Retro DOS v3.0
16270
16271      ;Break <$FCB_Open - open an old-style FCB>
16272      ;-----
16273      ;
16274      ; $FCB_Open - CPM compatability file open. The user has formatted an FCB
16275      ; for us and asked to have the rest filled in.
16276      ;
16277      ; Inputs: DS:DX point to an unopened FCB
16278      ; Outputs: AL indicates status 0 is ok FF is error
16279      ;           FCB has the following fields filled in:
16280      ;           Time/Date Extent/NR Size
16281      ;-----
16282      ; 23/01/2024 - Retro DOS v5.0
16283      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:6362h
16284
16285      _$FCB_OPEN:      ; System call 15
16286
16287      ;mov     ax,2
16288 00002410 B80200     MOV     AX,SHARING_COMPAT+open_for_both
16289
16290      ;hkn; DOS_Open is in DOSCODE
16291 00002413 B9[FA31]     MOV     CX,DOS_OPEN
16292
16293      ; The following is common code for Creation and opening of FCBs. AX is
16294      ; either attributes (for create) or open mode (for open)... DS:DX points to
16295      ; the FCB
16296
16297      DoAccess:
16298      push     ds
16299 00002417 52      push     dx
16300 00002418 51      push     cx
16301 00002419 50      push     ax      ; save FCB pointer away
16302
16303      ;hkn; OpenBuf is in DOSDATA
16304 0000241A BF[BE03]     MOV     DI,OPENBUF
16305 0000241D E8D757     call     TransFCB      ; crunch the fcb
16306 00002420 58      pop     ax
16307 00002421 59      pop     cx
16308 00002422 5A      pop     dx
16309 00002423 1F      pop     ds      ; get fcb
16310 00002424 7303     JNC     short FindFCB      ; everything seems ok
16311
16312      FCBOpenErr:
16313      ; AL has error code
16314      jmp     FCB_RET_ERR
16315
16316      FindFCB:
16317      call     GetExtended      ; DS:SI will point to FCB
16318
16319      ; 17/05/2019 - Retro DOS v4.0
16320
16321      ; MSDOS 3.3
16322      ;call     LRUFEB
16323      ;jc      short HardMessage
16324
16325      ; MSDOS 6.0
16326 0000242C 50      push     ax
16327 0000242D B001     mov     al,1      ;indicate Open/Create operation
16328 0000242F E804FB     call     LRUFEB      ; get a sft entry (no error)
16329 00002432 58      pop     ax
16330 00002433 722A     jc      short HardMessage
16331
16332      ;mov     word [es:di+2],8000h
16333 00002435 26C745020080    mov     word [es:di+SF_ENTRY.sf_mode],sf_isFCB
16334 0000243B 1E      push     ds
16335 0000243C 56      push     si
16336 0000243D 53      push     bx      ; save fcb pointer
16337 0000243E 89CE     MOV     SI,CX
16338
16339      ;hkn; SS is DOSDATA
16340 00002440 16      push     ss
16341 00002441 1F      pop     ds      ; let DOS_Open see variables
16342 00002442 FFD6     CALL     SI ; DOS_OPEN or DOS_CREATE ; go open the file
16343 00002444 5B      pop     bx
16344 00002445 5E      pop     si
16345 00002446 1F      pop     ds      ; get fcb
16346
16347      ;hkn; SS override
16348 00002447 36C43E[9E05]    LES     DI,[SS:THISSFT]      ; get sf pointer
16349 0000244C 7318     JNC     short FCBOk      ; operation succeeded
16350
16351      failopen:
16352      PUSH     AX
16353 0000244E 50      MOV     AL,"r" ; 52h      ; clear out field (free sft)
16354 0000244F B052     call     BLASTSFT
16355 00002451 E8C3FC     call     POP     AX
16356 00002454 58      ;cmp     ax,4

```

```

16354 00002455 83F804      CMP     AX,error_too_many_open_files
16355 00002458 7405      JZ      short HardMessage
16356      ;cmp     ax,24h
16357 0000245A 83F824      CMP     AX,error_sharing_buffer_exceeded
16358 0000245D 7505      jnz     short DeadFCB
16359      HardMessage:
16360 0000245F 50      PUSH    AX
16361 00002460 E86FFD      call   FCBHardErr
16362 00002463 58      POP     AX
16363      DeadFCB:
16364      ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
16365      ; jmp     FCB_RET_ERR
16366 00002464 EBC0      jmp     short FCBOpenErr
16367      FCBOK:
16368      ; MSDOS 6.0
16369 00002466 E8D9F3      call   IsSFTNet          ;AN007;F.C. >32mb Non Fat file?
16370 00002469 7531      JNZ     short FCBOK2      ;AN007;F.C. >32mb yes
16371 0000246B E8C75F      call   CheckShare        ;AN000;F.C. >32mb share around?
16372      ;JNZ     short FCBOK2      ;AN000;F.C. >32mb yes
16373      ; 23/01/2024 - Retro DOS v5.0
16374      ; (PCDOS 7.1 IBMDOS.COM)
16375 0000246E 750A      jnz     short FCBOK1
16376
16377      ;SR;
16378      ; If we reach here, we know we have got a local SFT. Let's update the
16379      ; LocalSFT variable to reflect this.
16380
16381 00002470 36893E[760F]  mov     [ss:LocalSFT],di
16382 00002475 368C06[780F]  mov     [ss:LocalSFT+2],es; Store the SFT address
16383      ;;SR;
16384      ;; The check below is not valid anymore since we regenerate for media > 32M.
16385      ;;
16386      ;; CMP     WORD [ES:DI+SF_ENTRY.sf_dirsec+2],0
16387      ;;          ;AN000;F.C. >32mb if dirsec >32mb
16388      ;; JZ      short FCBOK2      ;AN000;F.C. >32mb then error
16389      ;; MOV     AX,error_sys_comp_not_loaded ;AN000;F.C. >32mb
16390      ;; JMP     short failopen    ;AN000;F.C. >32mb
16391
16392      ; 23/01/2024 - Retro DOS v5.0
16393      ; (PCDOS 7.1 IBMDOS.COM)
16394      ;;
16395      ;;
16396      FCBOK1:
16397      ;test     byte [es:di+5],80h
16398      test     byte [es:di+SF_ENTRY.sf_flags],devid_device
16399      jnz     short FCBOK2      ; local device
16400      ;test     byte [es:di+4],8
16401      test     byte [es:di+SF_ENTRY.sf_attr],attr_volume_id
16402      jnz     short FCBOK2      ; local file
16403      push     es
16404      push     di
16405      ;les     di,[es:di+07h]
16406      les     di,[es:di+SF_ENTRY.sf_devptr] ; local file's DPB
16407      ;cmp     word [es:di+0Fh],0
16408      cmp     word [es:di+DPB.FAT_SIZE],0 ; (16 bit FAT size)
16409      pop     di
16410      pop     es
16411      jnz     short FCBOK2      ; not FAT32 (ok)
16412      ;mov     ax,0Fh          ; error_invalid_drive (for FAT32)
16413      mov     ax,error_invalid_drive
16414      jmp     short failopen
16415      ;;;
16416
16417      FCBOK2:
16418      ; MSDOS 6.0 (& MSDOS 3.3)
16419 0000249C 26FF05      inc     word [es:di]
16420      ;INC     word [ES:DI+SF_ENTRY.sf_ref_count] ; increment reference count
16421
16422      ; 23/01/2024 - Retro DOS v5.0
16423      ; (PCDOS 7.1 IBMDOS.COM)
16424      ;;;
16425 0000249F E87225      call   set_sftfcb_entry
16426      ;;;
16427
16428 000024A2 E8C3F9      call   SaveFCBInfo
16429
16430      ; MSDOS 3.3
16431      ;call    SetOpenAge
16432      ; MSDOS 6.0 (& MSDOS 3.3)
16433      ;test     word [es:di+5],80h
16434      ;TEST     word [ES:DI+SF_ENTRY.sf_flags],devid_device
16435 000024A5 26F6450580 test     byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; 28/07/2019
16436 000024AA 7508      JNZ     short FCBNoDrive      ; do not munge drive on devices
16437
16438 000024AC 8A04      MOV     AL,[SI]              ; get drive byte
16439 000024AE E8D756      call   GETTHISDRV           ; convert
16440      ;INC     AL
16441      ; 17/12/2022
16442      inc     ax
16443 000024B2 8804      MOV     [SI],AL              ; stash in good drive letter
16444
16445      FCBNoDrive:
16446      ;mov     word [si+0Eh],128
16447 000024B4 C7440E8000 MOV     word [SI+SYS_FCB.RECSIZ],80h ; stuff in default record size
16448
16449      ; 23/01/2024 - Retro DOS v5.0
16450      ; (PCDOS 7.1 IBMDOS.COM)
16451      ;;;
16452      ;;mov     ax,[es:di+0Dh]
16453      ;MOV     AX,[ES:DI+SF_ENTRY.sf_time] ; set time
16454      ;;mov     [si+16h],ax
16455      ;MOV     [SI+SYS_FCB.FTIME],AX
16456      ;;mov     ax,[es:di+0Fh]
16457      ;MOV     AX,[ES:DI+SF_ENTRY.sf_date] ; set date
16458      ;;mov     [si+14h],ax
16459      ;MOV     [SI+SYS_FCB.FDATE],AX
16460      ;;mov     ax,[es:di+11h]
16461      ;MOV     AX,[ES:DI+SF_ENTRY.sf_size] ; set sizes
16462      ;;mov     [si+10h],ax
16463      ;MOV     [SI+SYS_FCB.FILSIZ],AX
16464      ;;mov     ax,[es:di+13h]
16465      ;MOV     AX,[ES:DI+SF_ENTRY.sf_size+2]
16466      ;;mov     [si+12h],ax
16467      ;MOV     [SI+SYS_FCB.FILSIZ+2],AX
16468      ;
16469 000024B9 06      push     es
16470      ;les     ax,[es:di+0Dh]
16471 000024BA 26C4450D      les     ax,[es:di+SF_ENTRY.sf_time]
16472      ;mov     [si+16h],ax
16473 000024BE 894416      mov     [si+SYS_FCB.FTIME],ax ; set time
16474      ;mov     [si+14h],es
16475 000024C1 8C4414      mov     [si+SYS_FCB.FDATE],es ; set date
16476 000024C4 07      pop     es
16477 000024C5 06      push     es

```

```

16478             ;les     ax,[es:di+11h]
16479 000024C6 26C44511 les     ax,[es:di+SF_ENTRY.sf_size] ; set size
16480             ;mov     [si+10h],ax
16481 000024CA 894410  mov     [si+SYS_FCB.FILSIZ],ax
16482             ;mov     [si+12h],ax
16483 000024CD 8C4412  mov     [si+SYS_FCB.FILSIZ+2],es
16484 000024D0 07      pop     es
16485             ;;;
16486
16487 000024D1 31C0      XOR     AX,AX                ; convenient zero
16488             ;mov     [si+0Ch],ax
16489 000024D3 89440C  MOV     [SI+SYS_FCB.EXTENT],AX; point to beginning of file
16490
16491 ; We must scan the set of FCB SFTs for one that appears to match the current
16492 ; one. We cheat and use CheckFCB to match the FCBs.
16493
16494 ;hkn; SS override
16495 000024D6 36C43E[4000] LES     DI,[SS:SFTFCB]      ; get the pointer to head of the list
16496             ;mov     ah,[es:di+4]
16497 000024DB 268A6504 MOV     AH,[ES:DI+SFT.SFCOUNT]; get number of SFTs to scan
16498
16499 OpenScan:
16500             ;cmp     al,[si+18h]
16501 000024DF 3A4418  CMP     AL,[SI+fcb_sfn]      ; don't compare ourselves
16502 000024E2 7407      JZ      short SkipCheck
16503 000024E4 50      push    ax                ; preserve count
16504 000024E5 E848FC  call    CheckFCB            ; do they match
16505 000024E8 58      pop     ax                ; get count back
16506 000024E9 7309      JNC     short OpenFound    ; found a match!
16507
16508 SkipCheck:
16509             INC     AL        ; advance to next FCB
16510             CMP     AL,AH      ; table full?
16511             JNZ     short OpenScan ; no, go for more
16512
16513 OpenDone:
16514             xor     al,al      ; return success
16515             retn
16516
16517 ; The SFT at ES:DI is the one that is already in use for this FCB. We set the
16518 ; FCB to use this one. We increment its ref count. We do NOT close it at all.
16519 ; Consider:
16520 ;
16521 ; open (foo) delete (foo) open (bar)
16522 ;
16523 ; This causes us to recycle (potentially) bar through the same local SFT as
16524 ; foo even though foo is no longer needed; this is due to the server closing
16525 ; foo for us when we delete it. Unfortunately, we cannot see this closure.
16526 ; If we were to CLOSE bar, the server would then close the only reference to
16527 ; bar and subsequent I/O would be lost to the redirector.
16528 ;
16529 ; This gets solved by NOT closing the sft, but zeroing the ref count
16530 ; (effectively freeing the SFT) and informing the sharer (if relevant) that
16531 ; the SFT is no longer in use. Note that the SHARER MUST keep its ref counts
16532 ; around. This will allow us to access the same file through multiple network
16533 ; connections and NOT prematurely terminate when the ref count on one
16534 ; connection goes to zero.
16535
16536 OpenFound:
16537             ;mov     [si+18h],al
16538             MOV     [SI+fcb_sfn],AL      ; assign with this
16539             inc     word [es:di]
16540             ;INC     word [ES:DI+SF_ENTRY.sf_ref_count]
16541             ; remember this new invocation
16542             ; 23/01/2024 - Retro DOS v5.0
16543             ; (PCDOS 7.1 IBMDOS.COM)
16544             ;;;
16545             call    set_sftfcb_entry
16546             ;;;
16547             ; 24/01/2024
16548             push    ss
16549             pop     ds
16550
16551             ;MOV     AX,[SS:FCBLRU]      ; update LRU counts
16552             mov     ax,[FCBLRU] ; 24/01/2024
16553             ;mov     [es:di+15h],ax
16554             MOV     [ES:DI+sf_LRU],AX
16555
16556 ; We have an FCB sft that is now of no use. We release sharing info and then
16557 ; blast it to prevent other reuse.
16558 ;
16559             ;push    ss
16560             ;pop     ds
16561
16562             LES     DI,[THISST]
16563             dec     word [es:di]
16564             ;DEC     word [ES:DI+SF_ENTRY.sf_ref_count]
16565             ; free the newly allocated SFT
16566             call    ShareEnd
16567             MOV     AL,'C' ; 43h
16568             call    BlastSFT
16569             JMP     short OpenDone
16570
16571 ;BREAK <$FCB_Create - create a new directory entry>
16572 -----
16573 ;
16574 ; $FCB_Create - CPM compatability file create. The user has formatted an
16575 ; FCB for us and asked to have the rest filled in.
16576 ;
16577 ; Inputs: DS:DX point to an unopened FCB
16578 ; Outputs: AL indicates status 0 is ok FF is error
16579 ; FCB has the following fields filled in:
16580 ; Time/Date Extent/NR Size
16581 -----
16582
16583 _$FCB_CREATE: ; System call 22
16584
16585 ;hkn; DOS_Create is in DOSCODE
16586             MOV     CX,DOS_CREATE      ; routine to call
16587             XOR     AX,AX              ; attributes to create
16588             call    GetExtended        ; get extended FCB
16589             JZ      short DoAccessJ    ; not an extended FCB
16590             MOV     AL,[SI-1]          ; get attributes
16591
16592 DoAccessJ:
16593             JMP     DoAccess            ; do dirty work
16594
16595 ;=====
16596 ; SEARCH.ASM, MSDOS 6.0, 1991
16597 ;=====
16598 ; 22/07/2018 - Retro DOS v3.0
16599 ; 17/05/2019 - Retro DOS v4.0
16600
16601 ; DOSCODE:5DDFh (MSDOS 6.21, MSDOS.SYS)
16602
16603 ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
16604 ; DOSCODE:5DCBh (MSDOS 5.0, MSDOS.SYS)

```



```

16602
16603
16604
16605
16606
16607
16608
16609
16610
16611
16612
16613
16614
16615
16616
16617
16618
16619
16620
16621
16622
16623
16624
16625
16626
16627
16628
16629
16630
16631
16632
16633
16634
16635
16636
16637
16638
16639
16640
16641
16642 00002527 368916[A605]
16643 0000252C 368C1E[A805]
16644 00002531 89D6
16645 00002533 803CFF
16646 00002536 7503
16647 00002538 83C607
16648
16649 0000253B FF34
16650
16651 0000253D 16
16652 0000253E 07
16653
16654 0000253F BF[BE03]
16655 00002542 E8B256
16656 00002545 7304
16657 00002547 5B
16658
16659
16660
16661
16662
16663 00002548 E942E1
16664
16665
16666 0000254B 16
16667 0000254C 1F
16668
16669
16670
16671 0000254D C43E[2C03]
16672 00002551 57
16673 00002552 06
16674 00002553 C706[2C03][BE04]
16675 00002559 8C1E[2E03]
16676
16677
16678
16679 0000255D E8BA0F
16680 00002560 8F06[2E03]
16681 00002564 8F06[2C03]
16682 00002568 735A
16683 0000256A 5B
16684
16685
16686
16687
16688
16689 0000256B EBD8
16690
16691
16692
16693
16694
16695
16696
16697
16698
16699
16700
16701
16702
16703
16704
16705
16706
16707
16708
16709
16710
16711 0000256D 368916[A605]
16712 00002572 368C1E[A805]
16713
16714
16715
16716 00002577 B000
16717 00002579 36A2[6D05]
16718 0000257D 36A2[6D05]
16719
16720 00002581 16
16721 00002582 07
16722
16723 00002583 BF[BE04]
16724
16725 00002586 89D6

; ** Search.asm
;-----
; Directory search system calls.
; These will be passed direct text of the pathname from the user.
; They will need to be passed through the macro expander prior to
; being sent through the low-level stuff.
; I/O specs are defined in DISPATCH. The system calls are:
;
; $Dir_Search_First      written
; $Dir_Search_Next       written
; $Find_First            written
; $Find_Next             written
; PackName               written
;
; Modification history:
;
; Created: ARR 4 April 1983
;-----
; Procedure Name : $DIR_SEARCH_FIRST
;-----
; Inputs:
; DS:DX Points to unopened FCB
; Function:
; Directory is searched for first matching entry and the directory
; entry is loaded at the disk transfer address
; Returns:
; AL = -1 if no entries matched, otherwise 0
;-----
; IBMDOS.COM (MSDOS 3.3) - Offset 2B88h
;
; 24/01/2024
; MSDOS 5.0 MSDOS.SYS - DOSCODE:5DCBh
; MSDOS 6.22 MSDOS.SYS - DOSCODE:5DDFh
; PC DOS 7.1 IBMDOS.COM - DOSCODE:647Bh
; Windows ME IO.SYS - BIOSCODE:61E5h
;-----
_$DIR_SEARCH_FIRST:
MOV     [SS:THISFCB],DX
MOV     [SS:THISFCB+2],DS
MOV     SI,DX
CMP     BYTE [SI],0FFh
JNZ     short NORMFCB4
ADD     SI,7
; Point to drive select byte
NORMFCB4:
push    word [SI]
; save original drive byte for later
push    ss
pop      es
; get es to address DOSGroup
MOV     DI,OPENBUF
call    TransFCB
; appropriate buffer
; convert the FCB, set SATTRIB EXTFCB
JNC     short SearchIt
; no error, go and look
pop     bx
; Clean stack
; Error code is in AX
; 09/11/2022
dcf_errj:
jmp     FCB_RET_ERR
; error
SearchIt:
push    ss
pop      ds
; get ready for search
; push word [DMAADD]
; push word [DMAADD+2]
; 24/01/2024
les     di,[DMAADD]
push    di
push    es
MOV     WORD [DMAADD],SEARCHBUF
MOV     WORD [DMAADD+2],DS
; MSDOS 3.3
; call DOS_SEARCH_FIRST
; MSDOS 6.0
call    GET_FAST_SEARCH
; search
pop     word [DMAADD+2]
pop     word [DMAADD]
JNC     short SearchSet
; no error, transfer info
pop     bx
; Clean stack
; Error code is in AX
; 09/11/2022
; jmp FCB_RET_ERR
; jmp short dcf_errj
;-----
; Procedure Name : $DIR_SEARCH_NEXT
;-----
; Inputs:
; DS:DX points to unopened FCB returned by $DIR_SEARCH_FIRST
; Function:
; Directory is searched for the next matching entry and the directory
; entry is loaded at the disk transfer address
; Returns:
; AL = -1 if no entries matched, otherwise 0
;-----
; 24/01/2024
; MSDOS 5.0 MSDOS.SYS - DOSCODE:5E5Fh
; MSDOS 6.22 MSDOS.SYS - DOSCODE:5E73h
; PC DOS 7.1 IBMDOS.COM - DOSCODE:6517h
; Windows ME IO.SYS - BIOSCODE:6273h
;-----
_$DIR_SEARCH_NEXT:
MOV     [SS:THISFCB],DX
MOV     [SS:THISFCB+2],DS
; 24/01/2024 (PCDOS 7.1)
; MOV byte [SS:SATTRIB],0
; MOV byte [SS:EXTFCB],0
mov     al,0
mov     [SS:SATTRIB],al
; 0
mov     [SS:SATTRIB],al
; 0
push    ss
pop      es
MOV     DI,SEARCHBUF
MOV     SI,DX

```

```

16726 00002588 803CFF      CMP     BYTE [SI],0FFh
16727 0000258B 750D      JNZ     short NORMFCB6
16728 0000258D 83C606    ADD     SI,6
16729 00002590 AC        LODSB
16730
16731 00002591 36A2[6D05] MOV     [SS:SATTRIB],AL
16732 00002595 36FE0E[6C05] DEC     byte [SS:EXTFCB]
16733
NORMFCB6:
16734 0000259A AC        LODSB                ; Get original user drive byte
16735 0000259B 50        push     ax                ; Put it on stack
16736 0000259C 8A4414    MOV     AL,[SI+20]            ; Get correct search contin drive byte
16737 0000259F AA        STOSB                ; Put in correct place
16738 000025A0 B90A00    MOV     CX,20/2
16739 000025A3 F3A5      REP     MOVSW                ; Transfer in rest of search contin info
16740
16741 000025A5 16        push     ss
16742 000025A6 1F        pop      ds
16743
16744          ;push word [DMAADD]
16745          ;push word [DMAADD+2]
16746          ; 24/01/2024
16747 000025A7 C43E[2C03] les     di,[DMAADD]
16748 000025AB 57        push     di
16749 000025AC 06        push     es
16750 000025AD C706[2C03][BE04] MOV     WORD [DMAADD],SEARCHBUF
16751 000025B3 8C1E[2E03] MOV     WORD [DMAADD+2],DS
16752 000025B7 E85F10    call    DOS_SEARCH_NEXT        ; Find it
16753 000025BA 8F06[2E03] pop     word [DMAADD+2]
16754 000025BE 8F06[2C03] pop     word [DMAADD]
16755 000025C2 7249      JC      short SearchNoMore
16756          ; 24/01/2024
16757          ;JMP SearchSet                ; Ok set return
16758
16759 ;;; ; 24/01/2024 - Retro DOS v5.0
16760
16761 ; The search was successful (or the search-next). We store the information
16762 ; into the user's FCB for continuation.
16763
16764 SearchSet:
16765 000025C4 BE[BE04]      MOV     SI,SEARCHBUF
16766 000025C7 C43E[A605]      LES     DI,[THISFCB]                ; point to the FCB
16767 000025CB F606[6C05]FF    TEST    byte [EXTFCB],0FFh
16768 000025D0 7403      JZ      short NORMFCB1
16769 000025D2 83C707    ADD     DI,7                ; Point past the extension
16770
NORMFCB1:
16771 000025D5 5B        pop      bx                ; Get original drive byte
16772 000025D6 08DB      OR      BL,BL
16773 000025D8 7506      JNZ     short SearchDrv
16774 000025DA 8A1E[3603] MOV     BL,[CURDRV]
16775 000025DE FEC3      INC     BL
16776
SearchDrv:
16777 000025E0 AC        LODSB                ; Get correct search contin drive byte
16778 000025E1 86C3      XCHG     AL,BL                ; Search byte to BL, user byte to AL
16779 000025E3 47        INC     DI
16780          ;STOSB                ; Store the correct "user" drive byte
16781          ; at the start of the search info
16782 000025E4 B90A00    MOV     CX,20/2
16783 000025E7 F3A5      REP     MOVSW                ; Rest of search cont info, SI -> entry
16784 000025E9 86C3      XCHG     AL,BL                ; User drive byte back to BL, search
16785          ; byte to AL
16786 000025EB AA        STOSB                ; Search contin drive byte at end of
16787          ; contin info
16788 000025EC C43E[2C03] LES     DI,[DMAADD]
16789 000025F0 F606[6C05]FF    TEST    byte [EXTFCB],0FFh
16790 000025F5 740C      JZ      short NORMFCB2
16791 000025F7 B0FF      MOV     AL,0FFh
16792 000025F9 AA        STOSB
16793 000025FA FEC0      INC     AL
16794          ;MOV CX,5
16795          ; 17/12/2022
16796          ;mov cl,5
16797          ;REP STOSB
16798          ; 03/07/2024
16799 000025FC AB        stosw
16800 000025FD AB        stosw
16801 000025FE AA        stosb
16802 000025FF A0[6D05] MOV     AL,[SATTRIB]
16803 00002602 AA        STOSB
16804
NORMFCB2:
16805 00002603 88D8      MOV     AL,BL                ; User Drive byte
16806 00002605 AA        STOSB
16807          ;MOV CX,16                ; 32 / 2 words of dir entry
16808          ; 17/12/2022
16809 00002606 B110      mov     cl,16
16810 00002608 F3A5      REP     MOVSW
16811 0000260A E97DE0    jmp     FCB_RET_OK
16812
16813
16814 SearchNoMore:
16815 0000260D C43E[A605] LES     DI,[THISFCB]
16816 00002611 F606[6C05]FF TEST    byte [EXTFCB],0FFh
16817 00002616 7403      JZ      short NORMFCB8
16818 00002618 83C707    ADD     DI,7                ; Point past the extension
16819
NORMFCB8:
16820 0000261B 5B        pop      bx                ; Get original drive byte
16821 0000261C 26881D    MOV     [ES:DI],BL          ; Store the correct "user" drive byte
16822          ; at the right spot
16823 ; error code is in AX
16824
16825 0000261F E96BE0    jmp     FCB_RET_ERR
16826
16827 ; 17/05/2019 - Retro DOS v4.0
16828
16829 ; DOSCODE:5EE6h (MSDOS 6.21, MSDOS.SYS)
16830
16831 ;-----
16832 ;
16833 ; Procedure Name : $FIND_FIRST
16834 ;
16835 ; Assembler usage:
16836 ;     MOV AH, FindFirst
16837 ;     LDS DX, name
16838 ;     MOV CX, attr
16839 ;     INT 21h
16840 ; ; DMA address has datablock
16841 ;
16842 ; Error Returns:
16843 ;     AX = error_path_not_found
16844 ;     = error_no_more_files
16845 ;-----
16846
16847 ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
16848 ; DOSCODE:5ED2h (MSDOS 5.0, MSDOS.SYS)
16849

```



```

16850             ; 24/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
16851             ; DOSCODE:6594h (PCDOS 7.1, IBMDOS.COM)
16852
16853 _$FIND_FIRST:
16854 00002622 89D6      MOV     SI,DX          ; get name in appropriate place
16855 00002624 36880E[6D05] MOV     [SS:SATTRIB],CL      ; Search attribute to correct loc
16856
16857 00002629 BF[BE03]   MOV     DI,OPENBUF        ; appropriate buffer
16858
16859 0000262C E82E56     call    TransPathSet      ; convert the path
16860 0000262F 7305     JNC     short Find_it      ; no error, go and look
16861
16862 FindError:
16863             ;mov     al,3
16864             ;mov     al,error_path_not_found      ; error and map into one.
16865             ; 09/11/2022
16866 00002633 E942E0     jmp     SYS_RET_ERR
16867
16868 Find_it:
16869             push    ss
16870             pop     ds
16871
16872             ;push    word [DMAADD]
16873             ;push    word [DMAADD+2]
16874             ; 24/01/2024 (PCDOS 7.1 IBMDOS.COM)
16875             les     di,[DMAADD]
16876             push    di
16877             push    es
16878             MOV     WORD [DMAADD],SEARCHBUF
16879             MOV     WORD [DMAADD+2],DS
16880             ; MSDOS 3.3
16881             ;call    DOS_SEARCH_FIRST
16882             ; MSDOS 6.0
16883             call    GET_FAST_SEARCH      ; search
16884             pop     word [DMAADD+2]
16885             pop     word [DMAADD]
16886
16887             ; 16/12/2022
16888             JNC     short FindSet        ; no error, transfer info
16889             jc      short FF_errj      ; jmp SYS_RET_ERR
16890
16891             ; jmp     SYS_RET_ERR
16892             ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
16893 ;FFF_errj:
16894             jmp     short FF_errj      ; jmp SYS_RET_ERR
16895
16896 FindSet:
16897             MOV     SI,SEARCHBUF
16898             LES     DI,[DMAADD]
16899             MOV     CX,21
16900             REP     MOVSB
16901             PUSH    SI                  ; Save pointer to start of entry
16902             ;mov     al,[si+08h]
16903             MOV     AL,[SI+dir_entry.dir_attr]
16904             STOSB
16905             ;add     si,16h ; 22
16906             ADD     SI,dir_entry.dir_time
16907             MOVSW                     ; dir_time
16908             MOVSW                     ; dir_date
16909             INC     SI
16910             INC     SI                  ; skip dir_first
16911             MOVSW                     ; dir_size (2 words)
16912             MOVSW
16913             POP     SI                  ; Point back to dir_name
16914             CALL    PackName
16915             jmp     SYS_RET_OK          ; bye with no errors
16916
16917 -----
16918 ;
16919 ; Procedure Name : $FIND_NEXT
16920 ;
16921 ; Assembler usage:
16922 ; ; dma points at area returned by find_first
16923 ; MOV AH, findnext
16924 ; INT 21h
16925 ; ; next entry is at dma
16926 ;
16927 ; Error Returns:
16928 ; AX = error_no_more_files
16929 -----
16930
16931             ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
16932
16933             ; 24/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
16934             ; DOSCODE:65ECh (PCDOS 7.1, IBMDOS.COM)
16935
16936 _$FIND_NEXT:
16937             push    ss
16938             pop     es
16939
16940             MOV     DI,SEARCHBUF
16941
16942             LDS     SI,[SS:DMAADD]
16943
16944             MOV     CX,21
16945             REP     MOVSB                ; Put the search continuation info
16946                                     ; in the right place
16947             push    ss
16948             pop     ds                  ; get ready for search
16949
16950             ; 24/01/2024 (Retro DOS v5-v4)
16951             ;push    word [DMAADD]
16952             ;push    word [DMAADD+2]
16953             les     di,[DMAADD]
16954             push    di
16955             push    es
16956             MOV     WORD [DMAADD],SEARCHBUF
16957             MOV     WORD [DMAADD+2],DS
16958             call    DOS_SEARCH_NEXT      ; Find it
16959             pop     word [DMAADD+2]
16960             pop     word [DMAADD]
16961             JNC     short FindSet        ; No error, set info
16962             jmp     SYS_RET_ERR
16963             ; 16/12/2022
16964             jmp     short FF_errj      ; jmp SYS_RET_ERR
16965             ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
16966             jmp     short FFF_errj      ; jmp SYS_RET_ERR
16967
16968 -----
16969 ;** PackName - Convert file names from FCB to ASCIZ format.
16970 ;
16971 ; PackName transfers a file name from DS:SI to ES:DI and converts it to
16972 ; the ASCIZ format.
16973 ;

```

```

16974 ; ENTRY (DS:SI) = 11 character FCB or dir entry name
16975 ; (ES:DI) = destination area (13 bytes)
16976 ; EXIT (ds:SI) and (es:DI) advanced
16977 ; USES al, CX, SI, DI, Flags (BUGBUG - not verified - jgl)
16978 ;-----
16979
16980 ; 25/01/2024 - Retro DOS v5.0
16981 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:6627h
16982
16983 PackName:
16984 ; Move over 8 characters to cover the name component, then trim it's
16985 ; trailing blanks.
16986
16987 ;MOV CX,8 ; Pack the name
16988 ;REP MOVSB ; Move all of it
16989 ; 25/01/2024
16990 mov cx,4
16991 rep movsw
16992
16993 main_kill_tail:
16994 CMP BYTE [ES:DI-1], " "
16995 JNZ short find_check_dot
16996 DEC DI ; Back up over trailing space
16997 INC CX
16998 CMP CX,8
16999 JB short main_kill_tail
17000 find_check_dot:
17001 ;CMP WORD [SI], (" " << 8) | " "
17002 cmp word [si], 2020h
17003 JNZ short got_ext ; Some chars in extension
17004 CMP BYTE [SI+2], " "
17005 JZ short find_done ; No extension
17006 got_ext:
17007 MOV AL, "." ; 2Eh
17008 STOSB
17009 ;MOV CX,3
17010 ; 18/12/2022
17011 ;mov cl,3
17012 ;REP MOVSB
17013 ;movsb
17014 ;movsb
17015 ;movsb
17016 ; 25/01/2024
17017 movsw
17018 movsb
17019 ext_kill_tail:
17020 CMP BYTE [ES:DI-1], " "
17021 JNZ short find_done
17022 DEC DI ; Back up over trailing space
17023 JMP short ext_kill_tail
17024 find_done:
17025 XOR AX, AX
17026 STOSB ; NUL terminate
17027 retn
17028 ;-----
17029
17030 ; 24/01/2024
17031 %if 0
17032 ; 17/05/2019 - Retro DOS v4.0
17033 GET_FAST_SEARCH:
17034 ; 22/07/2018
17035 ; MSDOS 6.0
17036 ; 17/12/2022
17037 OR byte [ss:DOS34_FLAG+1], (SEARCH_FASTOPEN>>8) ; 04h
17038 ;OR word [ss:DOS34_FLAG], SEARCH_FASTOPEN ; 400h
17039 ;FO.trigger fastopen ;AN000;
17040 ;call DOS_SEARCH_FIRST
17041 ;retn
17042 ; 17/12/2022
17043 jmp DOS_SEARCH_FIRST
17044 %endif
17045
17046 ;=====
17047 ; PATH.ASM, MSDOS 6.0, 1991
17048 ;=====
17049 ; 06/08/2018 - Retro DOS v3.0
17050 ; 17/05/2019 - Retro DOS v4.0
17051
17052 ; DOSCODE:5FB0h (MSDOS 6.21, MSDOS.SYS)
17053
17054 ;** Directory related system calls. These will be passed direct text of the
17055 ; pathname from the user. They will need to be passed through the macro
17056 ; expander prior to being sent through the low-level stuff. I/O specs are
17057 ; defined in DISPATCH. The system calls are:
17058 ;
17059 ; $CURRENT_DIR Written
17060 ; $RMDIR Written
17061 ; $CHDIR Written
17062 ; $MKDIR Written
17063 ;
17064 ;
17065 ; Modification history:
17066 ;
17067 ; Created: ARR 4 April 1983
17068 ; MZ 10 May 1983 CurrentDir implemented
17069 ; MZ 11 May 1983 Rmdir, Chdir, Mkdir implemented
17070 ; EE 19 Oct 1983 Rmdir no longer allows you to delete a
17071 ; current directory.
17072 ; MZ 19 Jan 1983 Brain damaged applications rely on success
17073 ;
17074 ; I_Need ThisCDS, DWORD ; pointer to Current CDS
17075 ; I_Need WFP_Start, WORD ; pointer to beginning of directory text
17076 ; I_Need Curr_Dir_End, WORD ; offset to end of directory part
17077 ; I_Need OpenBuf, 128 ; temp spot for translated name
17078 ; I_need fSplice, BYTE ; TRUE => do splice
17079 ; I_Need NoSetDir, BYTE ; TRUE => no exact match on splice
17080 ; I_Need cMeta, BYTE
17081 ; I_Need DrvErr, BYTE ;AN000;
17082
17083 ;BREAK <$CURRENT_DIR - dump the current directory into user space>
17084 ;-----
17085 ;
17086 ; Procedure Name : $CURRENT_DIR
17087 ;
17088 ; Assembler usage:
17089 ; LDS SI, area
17090 ; MOV DL, drive
17091 ; INT 21h
17092 ; DS:SI is a pointer to 64 byte area that contains drive
17093 ; current directory.
17094 ; Error returns:
17095 ; AX = error_invalid_drive
17096 ;-----
17097

```

```

17098
17099 ; 06/08/2018 - Retro DOS v3.0
17100 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 2D4Eh
17101
17102 ; 25/01/2024 - Retro DOS v5.0
17103 ; MSDOS 5.0 MSDOS.SYS - DOSCODE:5F9Ch ; Retro DOS v4.1 (& v4.0)
17104 ; MSDOS 6.22 MSDOS.SYS - DOSCODE:5FB0h ; Retro DOS v4.2
17105 ; Windows ME IO.SYS - BIOSCODE:6393h
17106 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:6664h ; Retro DOS v5.0
17107
17108 _$CURRENT_DIR:
17109 call ECritDisk
17110 mov AL,DL ; get drive number (0=def, 1=A)
17111 call GetVisDrv ; grab it
17112 jnc short CurrentValidate ; no error -> go and validate dir
17113 curdirErr:
17114 call LCritDisk
17115
17116 ; MSDOS 3.3
17117 ;mov al,0Fh
17118
17119 ; MSDOS 6.0
17120 push ds
17121 mov ds,[cs:DosDSeg]
17122 mov al,[DrvErr] ;IFS. ;AN000;
17123 pop ds
17124
17125 curdir_errj:
17126 jmp SYS_RET_ERR ;IFS. make noise ;AN000;
17127
17128 CurrentValidate:
17129 push ds ; save destination
17130 push si
17131
17132 ;LDS SI,[CS:THISCDS] ; MSDOS 3.3
17133
17134 ; MSDOS 6.0
17135 mov ds,[cs:DosDSeg]
17136 ; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17137 mov byte [NoSetDir],0 ; *
17138
17139 ; 25/01/2024
17140 ;lds si,[THISCDS]
17141
17142 ; 16/12/2022
17143 %if 0
17144 ; 09/11/2022 (following test instruction is nonsense!)
17145 ; (I am leaving it here for MSDOS 5.0 MSDOS.SYS compatibility)
17146
17147 ;test word [si+43h],8000h
17148 TEST word [SI+curdir.flags],curdir_isnet
17149 ;jnz short $+2 ; 09/11/2022
17150 jnz short DoCheck
17151 %endif
17152
17153 ; Random optimization nuked due to some utilities using GetCurrentDir to do
17154 ; media check.
17155 ; CMP word [SI+curdir.ID],0
17156 ; JZ short GetDst
17157 DoCheck:
17158 ;MOV byte [cs:NoSetDir],0 ; interested only in contents
17159
17160 ; 25/01/2024
17161 ; MSDOS 6.0
17162 ;push ds
17163 ;mov ds,[cs:DosDSeg]
17164 ;mov byte [NoSetDir],0 ; *
17165 ;pop ds
17166
17167 MOV DI,OPENBUF
17168 call ValidateCDS ; output is ES:DI -> CDS
17169
17170 push es ; swap source and destination
17171 push di
17172 pop si
17173 pop ds
17174 GetDst:
17175 pop di
17176 pop es ; get real destination
17177 jnc short CurdirErr
17178 ;ADD SI,curdir.text ; add si,0 ; 09/08/2018
17179 ;
17180 ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17181 ; DOSCODE:5FE2h (MSDOS 5.0, MSDOS.SYS)
17182 ; 16/12/2022
17183 ;add si,0 ; add si,curdir.text
17184 ;
17185 ;add si,[si+4Fh] ; 17/05/2019
17186 ADD SI,[SI+curdir.end]
17187 CMP BYTE [SI],'\'; 5Ch ; root or subdirs present?
17188 JNZ short CurrentCopy
17189 INC SI
17190 CurrentCopy:
17191 ; call FStrCpy
17192 ;; 10/29/86 E5 char
17193 PUSH AX
17194 LODSB ; get char
17195 OR AL,AL
17196 JZ short FOK
17197 CMP AL,05H
17198 JZ short FCHANGE
17199 JMP short FFF
17200 FCPYNEXT:
17201 LODSB ; get char
17202 FFF:
17203 CMP AL,\'\' ; beginning of directory
17204 JNZ short FOK ; no
17205 STOSB ; put into user's buffer
17206 LODSB ; 1st char of dir is 05?
17207 CMP AL,05H
17208 JNZ short FOK ; no
17209 FCHANGE:
17210 MOV AL,0E5H ; make it E5
17211 FOK:
17212 STOSB ; put into user's buffer
17213 OR AL,AL ; final char
17214 JNZ short FCPYNEXT ; no
17215 POP AX
17216
17217 ;; 10/29/86 E5 char
17218 xor AL,AL ; MZ 19 Jan 84
17219 call LCritDisk
17220 jmp SYS_RET_OK ; no more, bye!
17221

```

```

17222 ; 17/05/2019 - Retro DOS v4.0
17223 ;
17224 ; DOSCODE:6029h (MSDOS 6.21, MSDOS.SYS)
17225 ;
17226 ;BREAK <$Rmdir -- Remove a directory>
17227 ;-----
17228 ;
17229 ; Procedure Name : $Rmdir
17230 ;
17231 ; Inputs:
17232 ; DS:DX Points to asciz name
17233 ; Function:
17234 ; Delete directory if empty
17235 ; Returns:
17236 ; STD XENIX Return
17237 ; AX = error_path_not_found If path bad
17238 ; AX = error_access_denied If
17239 ; Directory not empty
17240 ; Path not directory
17241 ; Root directory specified
17242 ; Directory malformed (. and .. not first two entries)
17243 ; User tries to delete a current directory
17244 ; AX = error_current_directory
17245 ;-----
17246 ;
17247 ;
17248 ; 10/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
17249 ; DOSCODE:6015h (MSDOS 5.0, MSDOS.SYS)
17250 ;
17251 ; 25/01/2025 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
17252 ; DOSCODE:66C8h (PCDOS 7.1, IBMDOS.COM)
17253 ;
17254 ;_RMDIR:
17255 ; push dx ; Save ptr to name
17256 ; push ds
17257 ; mov si, dx ; Load ptr into si
17258 ; mov di, OPENBUF ; di = ptr to buf for trans name
17259 ; push di
17260 ; call TransPathNoSet ; Translate the name
17261 ; pop di ; di = ptr to buf for trans name
17262 ; jnc short rmlset ; If transpath succeeded, continue
17263 ; pop ds
17264 ; pop dx ; Restore the name
17265 ; mov al, 3
17266 ; mov al, error_path_not_found ; Otherwise, return an error
17267 ; 16/12/2022
17268 rmdir_errj: ; 10/08/2018
17269 chdir_errj:
17270 ; jmp short curdir_errj
17271 ; jmp SYS_RET_ERR
17272 ;
17273 rmlset:
17274 ; CMP byte [ss:CMETA], -1 ; if (cMeta >= 0)
17275 ; Jnz short rmerr ; return (-1);
17276 ; push ss
17277 ; pop es
17278 ; xor al, al ; al = 0 , ie drive a:
17279 ; rmlloop:
17280 ; call GetCDSFromDrv ; Get curdir for drive in al
17281 ; jc short rmcont ; If error, exit loop & cont normally
17282 ;
17283 ; 25/01/2024 - Retro DOS v5.0
17284 ; (PCDOS 7.1 IBMDOS.COM)
17285 ;
17286 ; test word [si+43h], 4000h
17287 ; test word [si+curdir.flags], curdir_inuse
17288 ; test byte [si+curdir.flags+1], (curdir_inuse>>8)
17289 ; jz short rmdir_nxt
17290 ;
17291 ; call StrCmp ; Are the 2 paths the same?
17292 ; jz short rmerr ; Yes, report error.
17293 ; rmdir_nxt: ; 25/01/2024
17294 ; inc al ; No, inc al to next drive number
17295 ; jmp short rmlloop ; Go check next drive.
17296 ;
17297 ; rmerr:
17298 ; pop ds
17299 ; pop dx ; Restore ptr the name
17300 ; mov al, 10h
17301 ; mov al, error_current_directory ; error
17302 ; 16/12/2022
17303 ; 10/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17304 ; chdir_errj:
17305 ; jmp short rmdir_errj
17306 ;
17307 ; rmcont:
17308 ; pop ds
17309 ; pop dx ; Restore ptr the name
17310 ; MOV SI, DOS_RMDIR
17311 ;
17312 ; 25/01/2024 - Retro DOS v5.0
17313 ; (PCDOS 7.1 IBMDOS.COM)
17314 ;
17315 ; call TestNet
17316 ; mov di, 67 ; DIRSTRLEN
17317 ; 26/06/2024
17318 ; mov al, 67
17319 ; jnc short rmcont2 ; local directory
17320 ; mov di, 128
17321 ; mov al, 128
17322 ; rmcont2:
17323 ; 26/06/2024
17324 ; mov [ss:PATHNAMELEN], di
17325 ; mov [ss:PATHNAMELEN], al
17326 ;
17327 ; JMP DoDirCall
17328 ;
17329 ; 17/05/2019 - Retro DOS v4.0
17330 ;
17331 ; DOSCODE:6065h (MSDOS 6.21, MSDOS.SYS)
17332 ;
17333 ;BREAK <$ChDir -- Change current directory on a drive>
17334 ;-----
17335 ;
17336 ; $ChDir - Top-level change directory system call. This call is responsible
17337 ; for setting up the CDS for the specified drive appropriately. There are
17338 ; several cases to consider:
17339 ;
17340 ; o Local, simple CDS. In this case, we take the input path and convert
17341 ; it into a WFP. We verify the existence of this directory and then
17342 ; copy the WFP into the CDS and set up the ID field to point to the
17343 ; directory cluster.
17344 ; o Net CDS. We form the path from the root (including network prefix)
17345 ; and verify its existence (via DOS_Chdir). If successful, we copy the
17346 ; WFP back into the CDS.
17347 ; o SUBST'ed CDS. This is no different than the local, simple CDS.

```

```

17346 ; o JOIN'ed CDS. This is trouble as there are two CDS's at work. If we
17347 ; call TransPath, we will get the PHYSICAL CDS that the path refers to
17348 ; and the PHYSICAL WFP that the input path refers to. This is perfectly
17349 ; good for the validation but not for currency. We call TransPathNoSet
17350 ; to process the path but to return the logical CDS and the logical
17351 ; path. We then copy the logical path into the logical CDS.
17352 ;
17353 ; Inputs:
17354 ; DS:DX Points to asciz name
17355 ; Returns:
17356 ; STD XENIX Return
17357 ; AX = chdir_path_not_found if error
17358 ;
17359 ;-----
17360 ; 25/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
17361
17362 _$CHDIR:
17363 ; 25/01/2024
17364 ;;;
17365 ; mov byte [ss:PATHNAMELEN],67 ; Retro DOS v5.0
17366 0000278B 36C606[5411]43 ; mov word [ss:PATHNAMELEN],67 ; DIRSTRLEN = 67
17367 ;;;
17368 ; MOV DI,OPENBUF ; spot for translated name
17369 00002791 BF[BE03] ; MOV SI,DX ; get source
17370 00002794 89D6 ; call TransPath ; go munge the path and get real CDS
17371 00002796 E8C054 ; JNC short ChDirCrack ; no errors, try path
17372 00002799 7304
17373 ChDirErrP:
17374 ; mov al,3
17375 0000279B B003 ; MOV AL,error_path_not_found
17376 ChDirErr:
17377 ; jmp SYS_RET_ERR ; oops!
17378 ; 10/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17379 0000279D EBAF ; jmp short chdir_errj
17380
17381 ChDirCrack:
17382 0000279F 803E[7A05]FF ; CMP byte [CMETA],-1 ; No meta chars allowed.
17383 000027A4 75F5 ; JNZ short ChDirErrP
17384
17385 ; We cannot do a ChDir (yet) on a raw CDS. This is treated as a path not
17386 ; found.
17387
17388 000027A6 C43E[A205] ; LES DI,[THISCDS]
17389 000027AA 83FFFF ; CMP DI,-1 ; if (ThisCDS == NULL)
17390 000027AD 74EC ; JZ short ChDirErrP ; error ();
17391
17392 ; Find out if the directory exists.
17393
17394 000027AF E87C12 ; call DOS_CHDIR
17395 ; Jc short ChDirErr
17396 ; 16/12/2022
17397 000027B2 729A ; jc short chdir_errj
17398
17399 ; Get back CDS to see if a join as seen. Set the currency pointer (only if
17400 ; not network). If one was seen, all we need to do is copy in the text
17401 ;
17402 ; 25/01/2024 - Retro DOS v5.0
17403 000027B4 FF36[DC0A] ; push word [DIRSTART_HW] ; *
17404
17405 000027B8 C43E[A205] ; LES DI,[THISCDS]
17406 ; test word [es:di+43h],2000h
17407 ; 17/12/2022
17408 000027BC 26F6454420 ; test byte [ES:DI+curdir.flags+1],curdir_splice>>8
17409 ; TEST word [ES:DI+curdir.flags],curdir_splice
17410
17411 ; 25/01/2024 (PCDOS 7.1)
17412 ; mov dx,[DIRSTART_HW]
17413
17414 000027C1 742B ; JZ short GotCDS
17415
17416 ; The CDS was joined. Let's go back and grab the logical CDS.
17417
17418 000027C3 06 ; push es
17419 000027C4 57 ; push di
17420 ; push dx ; 25/01/2024 (PCDOS 7.1)
17421 000027C5 51 ; push cx ; word [DIRSTART] ; save CDS and cluster...
17422 000027C6 E8AEDC ; call Get_User_Stack ; get original text
17423
17424 ; mov di,[si+6]
17425 000027C9 8B7C06 ; MOV DI,[SI+user_env.user_DX]
17426 ; mov ds,[si+0Eh]
17427 000027CC 8E5C0E ; MOV DS,[SI+user_env.user_DS]
17428
17429 000027CF BE[BE03] ; MOV SI,OPENBUF ; spot for translated name
17430 000027D2 87F7 ; XCHG SI,DI
17431 000027D4 30C0 ; XOR AL,AL ; do no splicing
17432 000027D6 57 ; push di
17433 000027D7 E88B54 ; call TransPathNoSet ; Munge path
17434 000027DA 5E ; pop si
17435
17436 ; There should NEVER be an error here.
17437
17438 ; IF FALSE
17439 ; JNC SkipErr
17440 ; fmt <>,<>,<"$p: Internal CHDIR error\n">
17441 ; SkipErr:
17442 ; ENDIF
17443 000027DB C43E[A205] ; LES DI,[THISCDS] ; get new CDS
17444 ; mov word [es:di+49h],-1
17445 000027DF 26C74549FFFF ; MOV word [ES:DI+curdir.ID],-1
17446 ; no valid cluster here...
17447 ;;; 25/01/2024 (PCDOS 7.1)
17448 ; mov word [es:di+4Bh],-1
17449 000027E5 26C7454BFFFF ; mov word [es:di+curdir.ID+2],-1
17450 ;;;
17451 000027EB 59 ; pop cx ; word [DIRSTART]
17452 ; pop dx ; 25/01/2024 (PCDOS 7.1)
17453 000027EC 5F ; pop di
17454 000027ED 07 ; pop es
17455
17456 ; ES:DI point to the physical CDS, CX is the ID (local only)
17457
17458 GotCDS:
17459 ; 25/01/2024 - Retro DOS v5.0
17460 000027EE 5A ; pop dx ; word [DIRSTART_HW] ; *
17461
17462 ; wfp_start points to the text. See if it is long enough
17463
17464 ; MSDOS 3.3
17465 ; push ss
17466 ; pop ds
17467 ; mov si,[WFP_START]
17468 ; push cx
17469 ; call DStrLen

```

```

17470             ;cmp    cx,67 ; cmp cx,DIRSTRLEN
17471             ;pop     cx
17472             ;ja      short ChDirErrP
17473
17474             ; MSDOS 6.0
17475 000027EF E85C00 CALL    Check_PathLen          ;PTM.          ;AN000;
17476 000027F2 77A7 JA      short ChDirErrP
17477             ; MSDOS 3.3 & MSDOS 6.0
17478             ;TEST    word [ES:DI+curdir.flags],curdir_isnet ; 8000h
17479             ; 17/12/2022
17480 000027F4 26F6454480 test    byte [ES:DI+curdir.flags+1],curdir_isnet>>8
17481 000027F9 7518 JNZ     short SkipRecency
17482             ; MSDOS 6.0
17483             ;test    word [es:di+43h],2000h
17484             ; 17/12/2022
17485 000027FB 26F6454420 test    byte [ES:DI+curdir.flags+1],curdir_splice>>8
17486             ;TEST    word [ES:DI+curdir.flags],curdir_splice
17487             ;PTM. for Join and Subst ;AN000;
17488 00002800 7405 JZ      short setdirclus          ;PTM.          ;AN000;
17489 00002802 B9FFFF MOV     CX,-1                    ;PTM.          ;AN000;
17490             ;;; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17491 00002805 89CA mov     dx,cx
17492 setdirclus:
17493             ;mov     [es:di+49h],cx
17494 00002807 26894D49 MOV     [ES:DI+curdir.ID],CX
17495             ;;; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17496 0000280B 2689554B mov     [es:di+curdir.ID+2],dx
17497             ;;;
17498 0000280F C43E[A205] LES     DI,[THISCDS]          ; get logical CDS
17499 skipRecency:
17500 00002813 E8AAEF call    FStrCpy
17501 00002816 30C0 XOR     AL,AL
17502 mkdir_ok:
17503 00002818 E953DE jmp     SYS_RET_OK
17504
17505 ; 17/05/2019 - Retro DOS v4.0
17506
17507 ; DOSCODE:60E1h (MSDOS 6.21, MSDOS.SYS)
17508
17509 ;BREAK <$MkDir - Make a directory entry>
17510 -----
17511 ;
17512 ; Procedure Name : $MkDir
17513 ; Inputs:
17514 ; DS:DX Points to asciz name
17515 ; Function:
17516 ; Make a new directory
17517 ; Returns:
17518 ; STD XENIX Return
17519 ; AX = mkdir_path_not_found if path bad
17520 ; AX = mkdir_access_denied If
17521 ; Directory cannot be created
17522 ; Node already exists
17523 ; Device name given
17524 ; Disk or directory(root) full
17525 -----
17526
17527 ; 10/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
17528
17529 ; 25/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
17530 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:67B0h
17531
17532 _$MKDIR:
17533 ;MOV     SI,DOS_MKDIR
17534 ;;;
17535 ; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17536 0000281B 36C606[5411]43 mov     byte [ss:PATHNAMELEN],67 ; Retro DOS v5.0
17537             ;mov     word [ss:PATHNAMELEN],67 ; DIRSTRLEN (standard length = 67)
17538 mkdir_x:
17539 00002821 BE[1939] mov     si,DOS_MKDIR
17540             ;;;
17541 doDirCall:
17542 00002824 BF[BE03] MOV     DI,OPENBUF          ; spot for translated name
17543
17544             push    si
17545 00002828 89D6 MOV     SI,DX          ; get source
17546 0000282A E82C54 call    TransPath          ; go munge the path
17547 0000282D 5E pop     si
17548 0000282E 7305 JNC     short MkdirCrack    ; no errors, try path
17549 MkdirErrP:
17550 00002830 B003 MOV     AL,error_path_not_found ; oops!
17551 MkdirErr:
17552 00002832 E943DE jmp     SYS_RET_ERR
17553 MkdirCrack:
17554 00002835 36803E[7A05]FF CMP     byte [SS:CMETA],-1
17555 0000283B 75F3 JNZ     short MkdirErrP
17556
17557             ; MSDOS 3.3
17558             ;push    ss
17559             ;pop     ds
17560             ;call    si
17561             ;jb      short MkdirErr
17562             ;;jmp    short mkdir_ok
17563             ;jmp     SYS_RET_OK
17564
17565             ; MSDOS 6.0
17566 0000283D 56 PUSH     SI          ;PTM.          ;AN000;
17567             ; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17568             ; check path len > [PATNAMELEN] ; 67 or 128
17569 0000283E E80D00 CALL    Check_PathLen          ;PTM. check path len > 67 ? ;AN000;
17570 00002841 5E POP     SI          ;PTM.          ;AN000;
17571 00002842 7604 JBE     short pathok          ;PTM.          ;AN000;
17572             ;mov     al,5
17573 00002844 B005 MOV     AL,error_access_denied;PTM. ops!
17574             ;jmp     SYS_RET_ERR          ;PTM.
17575 00002846 EBFA jmp     short MkdirErr
17576 pathok:
17577 00002848 FFD6 CALL    SI          ; go get file
17578 0000284A 72E6 JC      short MkdirErr          ; error
17579             ; 16/12/2022
17580             ; 10/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17581 0000284C EBFA jmp     short mkdir_ok          ; ok
17582             ;jmp     SYS_RET_OK
17583
17584 -----
17585 ;
17586 ; Procedure Name : Check_PathLen
17587 ;
17588 ; Inputs:
17589 ; nothing
17590 ; Function:
17591 ; check if final path length greater than 67
17592 ; Returns:
17593 ; Above flag set if > 67

```



```

17594 ;
17595 ;-----
17596 ;
17597 ; 25/01/2024 - Retro DOS v5.0
17598 ; (PCDOS 7.1 IBMDOS.COM)
17599
17600 Check_PathLen:
17601 ; 09/09/2018
17602 ;mov SI,[WFP_START]
17603 0000284E 368B36[B205] MOV SI,[SS:WFP_START] ; MSDOS 6.0
17604
17605 Check_PathLen2:
17606 ; 25/01/2024 - Retro DOS v5.0
17607 ; Note: dx is not changed in 'Check_PathLen';
17608 ; no need to push/pop (*)
17609 ;
17610 00002853 16 push ss
17611 00002854 1F pop ds
17612 00002855 51 ;mov SI,[WFP_START] ; MSDOS 3.3
17613 push CX
17614 ; (PCDOS 7.1 IBMDOS.COM)
17615 00002856 E87EEF ;push dx ; 25/01/2024 (*)
17616 CALL DStrLen
17617 00002859 3B0E[5411] ; (PCDOS 7.1 IBMDOS.COM)
17618 cmp CX,[PATHNAMELEN] ; 67 or 128
17619 ;CMP CX,DIRSTRLEN ; 67
17620 0000285D 59 ;pop dx
17621 0000285E C3 POP CX
17622 retn
17623
17624 ;=====
17625 ; IOCTL.ASM, MSDOS 6.0, 1991
17626 ;=====
17627 ; 07/08/2018 - Retro DOS v3.0
17628 ; 17/05/2019 - Retro DOS v4.0
17629
17630 ;** IOCTL system call.
17631 ;-----
17632 $IOCTL
17633 ;
17634 ; Revision history:
17635 ;
17636 ; Created: ARR 4 April 1983
17637 ;
17638 ; GenericIOCTL added: KGS 22 April 1985
17639 ;
17640 ; A000 version 4.00 Jan. 1988
17641 ;
17642 ; Used jump table to dispatch IOCTL functions. HKN 3/12/90
17643 ;
17644 ;BREAK <IOCTL - munge on a handle to do device specific stuff>
17645 ;-----
17646 ;
17647 ; Assembler usage:
17648 ; MOV BX, Handle
17649 ; MOV DX, Data
17650 ;
17651 ; (or LDS DX,BUF
17652 ; MOV CX,COUNT)
17653 ;
17654 ; MOV AH, Ioctl
17655 ; MOV AL, Request
17656 ; INT 21h
17657 ;
17658 ; AH = 0 Return a combination of low byte of sf_flags and device driver
17659 ; attribute word in DX. handle in BX:
17660 ; DH = high word of device driver attributes
17661 ; DL = low byte of sf_flags
17662 ; 1 Set the bits contained in DX to sf_flags. DH MUST be 0. Handle
17663 ; in BX.
17664 ; 2 Read CX bytes from the device control channel for handle in BX
17665 ; into DS:DX. Return number read in AX.
17666 ; 3 Write CX bytes to the device control channel for handle in BX from
17667 ; DS:DX. Return bytes written in AX.
17668 ; 4 Read CX bytes from the device control channel for drive in BX
17669 ; into DS:DX. Return number read in AX.
17670 ; 5 Write CX bytes to the device control channel for drive in BX from
17671 ; DS:DX. Return bytes written in AX.
17672 ; 6 Return input status of handle in BX. If a read will go to the
17673 ; device, AL = 0FFh, otherwise 0.
17674 ; 7 Return output status of handle in BX. If a write will go to the
17675 ; device, AL = 0FFh, otherwise 0.
17676 ; 8 Given a drive in BX, return 1 if the device contains non-
17677 ; removable media, 0 otherwise.
17678 ; 9 Return the contents of the device attribute word in DX for the
17679 ; drive in BX. 0200h is the bit for shared. 1000h is the bit for
17680 ; network. 8000h is the bit for local use.
17681 ; A Return 8000h if the handle in BX is for the network or not.
17682 ; B Change the retry delay and the retry count for the system. BX is
17683 ; the count and CX is the delay.
17684 ;
17685 ; Error returns:
17686 ; AX = error_invalid_handle
17687 ; = error_invalid_function
17688 ; = error_invalid_data
17689 ;-----
17690 ;
17691 ; This is the documentation copied from DOS 4.0 it is much better
17692 ; than the above
17693 ;
17694 ; There are several basic forms of IOCTL calls:
17695 ;
17696 ;** Get/Set device information: **
17697 ;
17698 ; ENTRY (AL) = function code
17699 ; 0 - Get device information
17700 ; 1 - Set device information
17701 ; (BX) = file handle
17702 ; (DX) = info for "Set Device Information"
17703 ; EXIT 'C' set if error
17704 ; (AX) = error code
17705 ; 'C' clear if OK
17706 ; (DX) = info for "Get Device Information"
17707 ; USES ALL
17708 ;
17709 ;** Read/Write Control Data From/To Handle **
17710 ;
17711 ; ENTRY (AL) = function code
17712 ; 2 - Read device control info
17713 ; 3 - Write device control info
17714 ; (BX) = file handle
17715 ;
17716 ;
17717 ;

```

```

17718 ; (CX) = transfer count
17719 ; (DS:DX) = address for data
17720 EXIT 'C' set if error
17721 ; (AX) = error code
17722 ; 'C' clear if OK
17723 ; (AX) = count of bytes transfered
17724 USES ALL
17725 ;
17726 ;
17727 ; ** Read/Write Control Data From/To Block Device **
17728 ;
17729 ENTRY (AL) = function code
17730 ; 4 - Read device control info
17731 ; 5 - Write device control info
17732 ; (BL) = Drive number (0=default, 1='A', 2='B', etc)
17733 ; (CX) = transfer count
17734 ; (DS:DX) = address for data
17735 EXIT 'C' set if error
17736 ; (AX) = error code
17737 ; 'C' clear if OK
17738 ; (AX) = count of bytes transfered
17739 USES ALL
17740 ;
17741 ;
17742 ; ** Get Input/Output Status **
17743 ;
17744 ENTRY (AL) = function code
17745 ; 6 - Get Input Status
17746 ; 7 - Get Output Status
17747 ; (BX) = file handle
17748 EXIT 'C' set if error
17749 ; (AX) = error code
17750 ; 'C' clear if OK
17751 ; (AL) = 00 if not ready
17752 ; (AL) = FF if ready
17753 USES ALL
17754 ;
17755 ;
17756 ; ** Get Drive Information **
17757 ;
17758 ENTRY (AL) = function code
17759 ; 8 - Check for removable media
17760 ; 9 - Get device attributes
17761 ; (BL) = Drive number (0=default, 1='A', 2='B', etc)
17762 EXIT 'C' set if error
17763 ; (AX) = error code
17764 ; 'C' clear if OK
17765 ; (AX) = 0/1 media is removable/fixed (func. 8)
17766 ; (DX) = device attribute word (func. 9)
17767 USES ALL
17768 ;
17769 ;
17770 ; ** Get Redirected bit**
17771 ;
17772 ENTRY (AL) = function code
17773 ; 0Ah - Network stuff
17774 ; (BX) = file handle
17775 EXIT 'C' set if error
17776 ; (AX) = error code
17777 ; 'C' clear if OK
17778 ; (DX) = SFT flags word, 8000h set if network file
17779 USES ALL
17780 ;
17781 ;
17782 ; ** Change sharer retry parameters **
17783 ;
17784 ENTRY (AL) = function code
17785 ; 0Bh - Set retry parameters
17786 ; (CX) = retry loop count
17787 ; (DX) = number of retries
17788 EXIT 'C' set if error
17789 ; (AX) = error code
17790 ; 'C' clear if OK
17791 USES ALL
17792 ;
17793 ;
17794 ;
17795 ;
17796 ; ** New Standard Control **
17797 ;
17798 ; ALL NEW IOCTL FACILITIES SHOULD USE THIS FORM. THE OTHER
17799 ; FORMS ARE OBSOLETE.
17800 ;
17801 ;
17802 ;
17803 ENTRY (AL) = function code
17804 ; 0Ch - Control Function subcode
17805 ; (BX) = File Handle
17806 ; (CH) = Category Indicator
17807 ; (CL) = Function within category
17808 ; (DS:DX) = address for data, if any
17809 ; (SI) = Passed to device as argument, use depends upon function
17810 ; (DI) = Passed to device as argument, use depends upon function
17811 EXIT 'C' set if error
17812 ; (AX) = error code
17813 ; 'C' clear if OK
17814 ; (SI) = Return value, meaning is function dependent
17815 ; (DI) = Return value, meaning is function dependent
17816 ; (DS:DX) = Return address, use is function dependent
17817 USES ALL
17818 ;
17819 ;
17820 ; ===== Generic IOCTL Definitions for DOS 3.2 =====
17821 ; (See inc\ioctl.inc for more info)
17822 ;
17823 ENTRY (AL) = function code
17824 ; 0Dh - Control Function subcode
17825 ; (BL) = Drive Number (0 = Default, 1= 'A')
17826 ; (CH) = Category Indicator
17827 ; (CL) = Function within category
17828 ; (DS:DX) = address for data, if any
17829 ; (SI) = Passed to device as argument, use depends upon function
17830 ; (DI) = Passed to device as argument, use depends upon function
17831 EXIT 'C' set if error
17832 ; (AX) = error code
17833 ; 'C' clear if OK
17834 ; (DS:DX) = Return address, use is function dependent
17835 USES ALL
17836 ;
17837 ;-----
17838 ;
17839 ; 17/05/2019 - Retro DOS v4.0
17840 ; DOSCODE:611Eh (MSDOS 6.21, MSDOS.SYS)
17841 ;

```



```

17842 ; 11/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
17843 ; DOSCODE:610Ah (MSDOS 5.0, MSDOS.SYS)
17844
17845 ; 14/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
17846 ; DOSCODE:67F7h (PCDOS 7.1, IBMDOS.COM)
17847
17848 IOCTLJMPTABLE: ;label word
17849 ; MSDOS 3.3 (& MSDOS 6.0)
17850 0000285F [9B28] dw ioctl_getset_data ; 0
17851 00002861 [9B28] dw ioctl_getset_data ; 1
17852 00002863 [E528] dw ioctl_control_string ; 2
17853 00002865 [E528] dw ioctl_control_string ; 3
17854 00002867 [7D2A] dw ioctl_get_dev ; 4
17855 00002869 [7D2A] dw ioctl_get_dev ; 5
17856 0000286B [FD28] dw ioctl_status ; 6
17857 0000286D [FD28] dw ioctl_status ; 7
17858 0000286F [D729] dw ioctl_rem_media ; 8
17859 00002871 [202A] dw ioctl_drive_attr ; 9
17860 00002873 [6F2A] dw ioctl_handle_redir ; A
17861 00002875 [122A] dw Set_Retry_Parameters ; B
17862 00002877 [1929] dw GENERICIOCTLHANDLE ; C
17863 00002879 [3329] dw GENERICIOCTL ; D
17864 ; MSDOS 6.0 (& MSDOS 3.3)
17865 0000287B [1E2B] dw ioctl_drive_owner ; E
17866 0000287D [1E2B] dw ioctl_drive_owner ; F
17867 ; MSDOS 6.0
17868 0000287F [1929] dw query_handle_support ; 10h
17869 00002881 [3329] dw query_device_support ; 11h
17870
17871 ; 11/11/2022
17872 _$IOCTL:
17873 00002883 8CDE MOV SI,DS ; Stash DS for calls 2,3,4 and 5
17874 00002885 16 push ss
17875 00002886 1F pop ds ;hkn; SS is DOSDATA
17876
17877 ; MSDOS 3.3
17878 ;cmp al,0Fh
17879 ; MSDOS 6.0
17880 00002887 3C11 cmp al,11h ; al must be between 0 & 11h
17881 00002889 770D ja short ioctl_bad_funj2 ; if not bad function #
17882
17883 ; 14/01/2024
17884 ; 28/05/2019
17885 ;push AX ; 14/01/2024 ; Need to save AL for generic IOCTL
17886 0000288B 89C7 mov di,ax ; di NOT a PARM
17887 0000288D 81E7FF00 and di,0FFh ; di = al
17888 00002891 D1E7 shl di,1 ; di = index into jmp table
17889 ;pop AX ; Restore AL for generic IOCTL
17890
17891 00002893 2EFA5[5F28] jmp word [CS:DI+IOCTLJMPTABLE]
17892
17893 ioctl_bad_funj2:
17894 00002898 E90401 JMP ioctl_bad_fun ; 10/08/2018
17895
17896 ;-----
17897 ;
17898 ; IOCTL: AL = 0,1
17899 ;
17900 ; ENTRY: DS = DOSDATA
17901 ;
17902 ;-----
17903
17904 ; 29/01/2024 - Retro DOS v5.0
17905 ioctl_getset_data:
17906 ; MSDOS 6.0
17907 0000289B E8304E call SFFromHandle ; ES:DI -> SFT
17908 0000289E 7305 JNC short ioctl_check_permissions ; have valid handle
17909 ioctl_bad_handle:
17910 ;mov al,6
17911 000028A0 B006 mov al,error_invalid_handle
17912
17913 000028A2 E9D3DD ioctl_error: jmp SYS_RET_ERR
17914
17915 ioctl_check_permissions:
17916 CMP AL,0
17917 ;mov al,[es:di+5]
17918 000028A7 268A4505 MOV AL,[ES:DI+SF_ENTRY.sf_flags]; Get low byte of flags
17919 000028AB 7419 JZ short ioctl_read ; read the byte
17920
17921 or dh,dh
17922 000028AF 7404 JZ short ioctl_check_device ; can I set with this data?
17923 ;mov al,0Dh
17924 000028B1 B00D mov al,error_invalid_data ; no DH <> 0
17925 ;jmp SYS_RET_ERR
17926 000028B3 EBED jmp short ioctl_error
17927
17928 ioctl_check_device:
17929 000028B5 A880 test AL,devid_device ; 80h; can I set this handle?
17930 000028B7 74DF jz short ioctl_bad_funj2
17931 000028B9 80CA80 OR DL,devid_device ; Make sure user doesn't turn off the
17932 ; device bit!! He can muck with the
17933 ; others at will.
17934 ;MOV byte [EXERR_LOCUS],errLOC_SerDev ; 4
17935 ; 29/01/2024
17936 ;;;
17937 000028BC E82CEA call set_exerr_locus_ser ; (PCDOS 7.1 IBMDOS.COM)
17938 ;;;
17939 000028BF 26885505 MOV BYTE [ES:DI+SF_ENTRY.sf_flags],DL ;AC000;MS.; Set flags
17940
17941 000028C3 E9A8DD ioctl_ok: jmp SYS_RET_OK
17942
17943 ioctl_read:
17944 ;MOV byte [EXERR_LOCUS],errLOC_Disk ; 2
17945 ; 29/01/2024
17946 ;;;
17947 000028C6 E81DEA call set_exerr_locus_disk ; (PCDOS 7.1 IBMDOS.COM)
17948 ;;;
17949 000028C9 30E4 XOR AH,AH
17950 000028CB A880 test AL,devid_device ; Should I set high byte
17951 000028CD 740B jz short ioctl_no_high ; no
17952 ;MOV byte [EXERR_LOCUS],errLOC_SerDev ; 4
17953 ; 29/01/2024
17954 ;;;
17955 000028CF E819EA call set_exerr_locus_ser ; (PCDOS 7.1 IBMDOS.COM)
17956 ;;;
17957 ;les di,[es:di+7]
17958 000028D2 26C47D07 LES DI,[ES:DI+SF_ENTRY.sf_devptr] ; Get device pointer
17959 ;mov ah,[es:di+5]
17960 000028D6 268A6505 MOV AH,[ES:DI+SYSDEV.ATT+1] ; Get high byte
17961
17962 000028DA 89C2 ioctl_no_high: MOV DX,AX
17963
17964 000028DC E898DB ioctl_set_dx: ; 16/12/2022
17965 ;call Get_User_Stack
17966 ;mov [si+6],dx

```

```

17966 000028DF 895406      MOV     [SI+user_env.user_DX],DX
17967                      ;;jmp     SYS_RET_OK
17968                      ;; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17969 ioctl_ok_j:
17970                      ;; 16/12/2022
17971 000028E2 E98CDD      jmp     SYS_RET_OK_c1c ; (after 'Get_User_Stack')
17972                      ;;jmp     short ioctl_ok
17973                      ;; 26/07/2019
17974                      ;;jmp     SYS_RET_OK_c1c
17975
17976                      ;-----
17977                      ;
17978                      ; IOCTL: AL = 2,3
17979                      ;
17980                      ; ENTRY: DS = DOSDATA
17981                      ; SI = user's DS
17982                      ;
17983                      ;-----
17984
17985                      ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
17986 ioctl_control_string:
17987 000028E5 E8E64D      call    SFFromHandle ; ES:DI -> SFT
17988 000028E8 72B6      jc     short ioctl_bad_handle; invalid handle
17989                      ; 07/12/2022
17990                      ;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device ; can I?
17991                      ;jz     short ioctl_bad_funj2 ; No it is a file
17992                      ; MSDOS 5.0 & MSDOS 6.0
17993 000028EA 26F6450580 test    byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; can I?
17994 000028EF 74A7      jz     short ioctl_bad_funj2 ; No it is a file
17995                      ;MOV     byte [EXTERR_LOCUS],errLOC_SerDev ; 4
17996                      ; 29/01/2024
17997                      ;;;
17998 000028F1 E8F7E9      call    set_exerr_locus_ser ; (PCDOS 7.1 IBMDOS.COM)
17999                      ;;;
18000 000028F4 26C47D07    les     DI,[ES:DI+SF_ENTRY.sf_devptr] ; Get device pointer
18001 000028F8 30DB      xor     BL,BL ; Unit number of char dev = 0
18002 000028FA E98801      jmp     ioctl_do_string
18003
18004                      ;-----
18005                      ;
18006                      ; IOCTL: AL = 6,7
18007                      ;
18008                      ; ENTRY: DS = DOSDATA
18009                      ;
18010                      ;-----
18011
18012 ioctl_status:
18013 000028FD B401      mov     AH,1
18014 000028FF 2C06      sub     AL,6 ; 6=0,7=1
18015 00002901 7402      jz     short ioctl_get_status
18016 00002903 B403      mov     AH,3
18017 ioctl_get_status:
18018 00002905 50      push    AX
18019 00002906 E8BC15      call    GET_IO_SFT
18020 00002909 58      pop     AX
18021                      ;JNC     short DO_IOFUNC
18022                      ;JMP     short ioctl_bad_handle; invalid SFT
18023                      ; 16/12/2022
18024 0000290A 7294      jc     short ioctl_bad_handle
18025 DO_IOFUNC:
18026 0000290C E8B527      call    IOFUNC
18027 0000290F 88C4      mov     AH,AL
18028 00002911 B0FF      mov     AL,0FFh
18029                      ;JNZ     short ioctl_status_ret
18030                      ; 29/01/2024
18031 00002913 75AE      jnz     short ioctl_ok
18032 00002915 FEC0      inc     AL
18033 ioctl_status_ret:
18034                      ;jmp     SYS_RET_OK
18035                      ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18036                      ;jmp     short ioctl_ok_j
18037                      ; 16/12/2022
18038 00002917 EBAA      jmp     short ioctl_ok
18039
18040                      ;-----
18041                      ;
18042                      ; Generic IOCTL entry point. AL = C, D, 10h, 11h
18043                      ;
18044                      ; here we invoke the Generic IOCTL using the IOCTL_Req structure.
18045                      ; SI:DX -> Users Device Parameter Table
18046                      ; IOCALL -> IOCTL_Req structure
18047                      ;
18048                      ; If on entry AL >= IOCTL_QUERY_HANDLE the function is a
18049                      ; QueryIOCtlSupport call ELSE it's a standard generic IOCtl
18050                      ; call.
18051                      ;
18052                      ; BUGBUG: Don't push anything on the stack between GENERIOCTL: and
18053                      ; the call to Check_If_Net because Check_If_Net gets our
18054                      ; return address off the stack if the drive is invalid.
18055                      ;
18056                      ;-----
18057
18058                      ; 29/01/2024 - Retro DOS v5.0
18059
18060 query_handle_support: ; Entry point for handles
18061 GENERICIOCTLHANDLE:
18062 00002919 E8B24D      call    SFFromHandle ; Get SFT for device.
18063 0000291C 727E      jc     short ioctl_bad_handlej
18064
18065                      ;test word [es:di+5],8000h
18066                      ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet ; M031;
18067                      ;test byte [es:di+6],80h
18068 0000291E 26F6450680 test    byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
18069 00002923 757A      jnz     short ioctl_bad_fun ; Cannot do this over net.
18070
18071                      ;mov     byte [EXTERR_LOCUS],errLOC_SerDev ; 4
18072                      ; 29/01/2024
18073                      ;;;
18074 00002925 E8C3E9      call    set_exerr_locus_ser ; (PCDOS 7.1 IBMDOS.COM)
18075                      ;;;
18076
18077                      ;les     di,[es:di+7]
18078 00002928 26C47D07    les     di,[es:di+SF_ENTRY.sf_devptr] ; Get pointer to device.
18079                      ;;;
18080                      ; 29/01/2024 - PCDOS 7.1 IBMDOS.COM
18081 0000292C C606[B103]FF mov     byte [IOCTL_drvnum],0FFh ; invalidate drive number
18082                      ; (for extended -lock/unlock- functions)
18083                      ;;;
18084 00002931 EB16      jmp     short Do_GenIOCTL
18085
18086                      ; 29/01/2024 - Retro DOS v5.0
18087                      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:68DAh
18088                      ; (WINME IO.SYS - BIOSCODE:6612h)
18089

```

```

18090 query_device_support:      ; Entry point for devices:
18091 GENERICIOCTL:
18092     ;mov     byte [EXERR_LOCUS],errLOC_Disk ; 2
18093     ; 29/01/2024
18094     ;;;
18095     call    set_exerr_locus_disk
18096     cmp     ch,48h          ; category (extended, disk lock/unlock)
18097     je      short GenIOCTL_chk_net      ; extended (MSDOS/PCDOS 7)
18098     ;;;
18099
18100     cmp     ch,IOC_DC ; 8      ; Only disk devices are allowed to use
18101     jne     short ioctl_bad_fun      ; no handles with Generic IOCTL.
18102
18103     ; 29/01/2024
18104     ;;;
18105     GenIOCTL_chk_net:
18106     mov     [IOCTL_drvnum],bl      ; drive number
18107     ;;;
18108
18109     CALL     Check_If_Net          ; ES:DI := Get_hdr_block of device in BL
18110     JNZ      short ioctl_bad_fun      ; There are no "net devices", and they
18111
18112     Do_GenIOCTL:
18113     ; 29/01/2024 - Retro DOS v5.0
18114     ;;;
18115     cmp     ch,48h          ; category code 48h for FAT32
18116     je      short GenIOCTL_extended ; MSDOS/PCDOS 7 functions (lock/unlock)
18117     ;cmp     ch,8
18118     cmp     ch,IOC_DC ; 8      ; disk control
18119     je      short GenIOCTL_extended
18120     GenIOCTL_normal:          ; MSDOS 5-6.22 functions
18121     ;;;
18122
18123     ;TEST    word [ES:DI+SYSDEV.ATT],DEV320
18124     ; Can device handle Generic IOCTL funcs
18125
18126     ; 09/09/2018
18127     test    byte [es:di+4],40h
18128     TEST     byte [ES:DI+SYSDEV.ATT],DEV320 ; 0040h
18129     jz       short ioctl_bad_fun
18130
18131     ; 17/05/2019 - Retro DOS v4.0
18132
18133     ; MSDOS 6.0
18134     ;mov     byte [IOCALL_REQFUNC],19 ; 13h
18135     mov     byte [IOCALL_REQFUNC],GENIOCTL ; Assume real Request
18136     ;cmp     al,10h
18137     cmp     AL,IOCTL_QUERY_HANDLE ; See if this is just a query
18138     jl       short SetIOctlBlock
18139
18140     ;TEST    word [ES:DI+SYSDEV.ATT],IOQUERY ; See if device supports a query
18141     ;test    byte [es:di+4],80h
18142     TEST     byte [ES:DI+SYSDEV.ATT],IOQUERY ; See if device supports a query
18143     jz       short ioctl_bad_fun      ; No support for query
18144     ;
18145     ;mov     byte [IOCALL_REQFUNC],19h
18146     mov     byte [IOCALL_REQFUNC],IOCTL_QUERY ; Just a query (5.00)
18147
18148     SetIOctlBlock:
18149     PUSH     ES                ; DEVIOPCALL2 expects Device header block
18150     PUSH     DI                ; in DS:SI
18151     ; Setup Generic IOCTL Request Block
18152     ;mov     byte [IOCALL_REQLEN],23
18153     mov     byte [IOCALL_REQLEN],IOCTL_REQ.size
18154     ; 07/09/2018 (MSDOS 3.3)
18155     ;mov     byte [IOCALL_REQFUNC],19
18156     ;mov     byte [IOCALL_REQFUNC],GENIOCTL ; 07/09/2018
18157     ;
18158     MOV      [IOCALL_REQUNIT],BL
18159     MOV      [IOCALL+IOCTL_REQ.MAJORFUNCTION],CH
18160     MOV      [IOCALL+IOCTL_REQ.MINORFUNCTION],CL
18161     MOV      [IOCALL+IOCTL_REQ.REG_SI],SI
18162     MOV      [IOCALL+IOCTL_REQ.REG_DI],DI
18163     MOV      [IOCALL+IOCTL_REQ.GENERICIOCTL_PACKET],DX
18164     MOV      [IOCALL+IOCTL_REQ.GENERICIOCTL_PACKET+2],SI
18165
18166     ;hkn; IOCALL is in DOSDATA
18167     MOV      BX,IOCALL
18168
18169     PUSH     SS
18170     POP      ES
18171
18172     ; DS:SI -> Device header.
18173     POP      SI
18174     POP      DS
18175     ; 10/08/2018
18176     jmp      ioctl_do_IO          ; Perform call to device driver
18177
18178     ioctl_bad_handlej:
18179     jmp      ioctl_bad_handle
18180
18181     ; 29/01/2024
18182     ioctl_bad_fun:
18183     mov     al,error_invalid_function ; 1
18184     jmp     SYS_RET_ERR
18185
18186     ; 29/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
18187     GenIOCTL_extended:
18188     cmp     cl,6Ah          ; UNLOCK LOGICAL VOLUME
18189     ;je      short GenIOCTL_chk_lock
18190     je      short GenIOCTL_lock_unlock
18191     cmp     cl,4Ah          ; LOCK LOGICAL VOLUME
18192     jne     short GenIOCTL_normal
18193     ;GenIOCTL_chk_lock:
18194     ;cmp     cl,4Ah          ; LOCK LOGICAL VOLUME
18195     ;jne     short GenIOCTL_lock_unlock
18196     cmp     bh,4            ; lock level (0-4)
18197     je      short GenIOCTL_lock_unlock
18198     or      bh,bh
18199     jnz     short ioctl_bad_fun
18200     GenIOCTL_lock_unlock:
18201     mov     bl,[IOCTL_drvnum]      ; drive number (1=A:, 2=B: ..)
18202     xor     bh,bh
18203     dec     bx
18204     cmp     bl,26            ; logical disk number limit
18205     jnb     short ioctl_bad_fun
18206     cmp     cl,6Ah          ; UNLOCK LOGICAL VOLUME
18207     jne     short GenIOCTL_lock
18208     and     byte [bx+drive_flags],7Fh ; UNLOCK
18209     jmp     short GenIOCTL_OK
18210     GenIOCTL_lock:
18211     or      byte [bx+drive_flags],80h ; LOCK
18212     GenIOCTL_OK:
18213     jmp     SYS_RET_OK

```

```

18214 ;
18215 ; IOCTL: AL = 8
18216 ;
18217 ; ENTRY: DS = DOSDATA
18218 ;
18219 ; BUGBUG: Don't push anything on the stack between ioctl_rem_media: and
18220 ; the call to Check_If_Net because Check_If_Net gets our
18221 ; return address off the stack if the drive is invalid.
18222 ;
18223 ;-----
18224 ;
18225 ; 30/01/2024
18226 ;
18227 ; ioctl_rem_media:
18228 ; MSDOS 3.3 (& MSDOS 6.0)
18228 000029D7 E83501 CALL Check_If_Net
18229 000029DA 75C3 JNZ short ioctl_bad_fun ; There are no "net devices", and they
18230 ; ; certainly don't know how to do this
18231 ; ; call.
18232 ;
18232 ;test word [es:di+4],800h
18233 ;TEST word [ES:DI+SYSDEV.ATT],DEVOPCL ; See if device can
18234 ;test byte [es:di+5],8
18235 000029DC 26F6450508 TEST byte [es:di+SYSDEV.ATT+1],(DEVOPCL>>8)
18236 000029E1 74BC JZ short ioctl_bad_fun ; NO
18237 ;
18238 ;hkn; SS override for IOCALL
18239 ; 30/01/2024
18240 ; ds = ss = DOSDATA segment ('Get_Driver_BL' in 'Check_If_Net')
18241 ;MOV byte [SS:IOCALL_REQFUNC],DEVPRMD ; 15
18242 000029E3 C606[7E03]0F mov byte [IOCALL_REQFUNC],DEVPRMD ; 15
18243 000029E8 B00D MOV AL,REHML ; 13
18244 000029EA 88DC MOV AH,BL ; Unit number
18245 ;MOV [SS:IOCALL_REQLEN],AX
18246 000029EC A3[7C03] mov [IOCALL_REQLEN],ax
18247 000029EF 31C0 XOR AX,AX
18248 ;MOV [SS:IOCALL_REQSTAT],AX
18249 000029F1 A3[7F03] mov [IOCALL_REQSTAT],ax ; 0
18250 ;
18251 000029F4 06 PUSH ES
18252 000029F5 1F POP DS
18253 000029F6 89FE MOV SI,DI ; DS:SI -> driver
18254 000029F8 16 PUSH SS
18255 000029F9 07 POP ES
18256 ;
18257 ;hkn; IOCALL is in DOSDATA (msconst.asm)
18258 ; 30/01/2024
18259 ; (ds <-> ss, ss = DOSDATA segment)
18260 000029FA BB[7C03] MOV BX,IOCALL ; ES:BX -> Call header
18261 000029FD 1E push ds
18262 000029FE 56 push si
18263 000029FF E88F28 call DEVIOCALL2
18264 00002A02 5E pop si
18265 00002A03 1F pop ds
18266 ;
18267 ;hkn; SS override
18268 00002A04 36A1[7F03] MOV AX,[SS:IOCALL_REQSTAT]; Get Status word
18269 ;AND AX,STBUI ; 200h ; Mask to busy bit
18270 ; 29/01/2024
18271 00002A08 80E402 and ah,STBUI>>8 ; 2
18272 00002A0B B109 MOV CL,9
18273 00002A0D D3E8 SHR AX,CL ; Busy bit to bit 0
18274 ;
18275 00002A0F E95CDC ioctl_da_ok_j: ; 11/11/2022
18276 jmp SYS_RET_OK
18277 ; 29/01/2024
18278 ;-----
18279 ;
18280 ; IOCTL: AL = B
18281 ;
18282 ; ENTRY: DS = DOSDATA
18283 ;
18284 ;-----
18285 ;
18286 ; Set_Retry_Parameters:
18287 ; 09/09/2018
18288 00002A12 890E[1C00] MOV [RetryLoop],CX ; 0 retry loop count allowed
18289 00002A16 09D2 OR DX,DX ; zero retries not allowed
18290 00002A18 7485 JZ short ioctl_bad_fun
18291 ; 29/01/2024
18292 ;jnz short set_new_retry_cnt
18293 ;jmp ioctl_bad_fun
18294 ;set_new_retry_cnt:
18295 00002A1A 8916[1A00] MOV [RetryCount],DX ; Set new retry count
18296 doneok:
18297 ;jmp SYS_RET_OK ; Done
18298 ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18299 ;jmp short ioctl_status_ret
18300 ; 16/12/2022
18301 ;jmp short ioctl_ok ; jmp SYS_RET_OK
18302 ; 29/01/2024
18303 00002A1E EBEF jmp short ioctl_da_ok_j
18304 ;-----
18305 ;
18306 ; IOCTL: AL = 9
18307 ;
18308 ; ENTRY: DS = DOSDATA
18309 ;
18310 ;-----
18311 ;
18312 ; 30/01/2024 - Retro DOS v5.0
18313 ;
18314 ;
18315 ; ioctl_drive_attr:
18316 ; MSDOS 3.3 (& MSDOS 6.0)
18317 00002A20 88D8 mov al,b1
18318 00002A22 E86351 call GETTHISDRV
18319 00002A25 7243 jc short ioctl_drv_err
18320 00002A27 E8BA00 call Get_Driver_BL
18321 ; MSDOS 6.0
18322 00002A2A 723E JC short ioctl_drv_err ; drive not valid
18323 ;
18324 ; 03/07/2024
18325 ; jnz = net drive (current dir is net)
18326 ; jz = local drive
18327 ;
18328 ; 03/07/2024
18329 ; 30/01/2024 - Retro DOS v5.0
18330 ; 30/01/2024 - PC DOS 7.1 IBMDOS.COM
18331 ;;;
18332 00002A2C BA4209 mov dx,942h ; 0942h -> Attribute word
18333 ; ; bit 11 - open/close/remmedia calls supported
18334 ; ; bit 8 - (new type driver)
18335 ; ; bit 6 - Generic IOCTL call supported
18336 ; ; bit 1 - driver supports 32-bit sector addressing
18337 00002A2F 7504 jnz short ioctl_drive_attr2 ; NET device

```

```

18338 ; 30/01/2024
18339 ; NOTE: 'jnz' condition is correct for windows ME
18340 ; 'Get_Driver_BL' because it tests bit 0 of [drive_flags]
18341 ; but in PC DOS 7.1 'Get_Driver_BL', this flag
18342 ; (remote or removable? disk flag) is not tested
18343 ; the last test is net device test) ; (!)
18344 ;;;;
18345 ;mov dx,[es:di+4]
18346 mov dx,[es:di+SYSDEV.ATT]
18347 00002A31 268B5504 ; get device attribute word
18348 ; 26/06/2024
18349 ioctl_drive_attr2: ; 30/01/2024
18350 MOV BL,AL ; Phys letter to BL (A=0)
18351 00002A35 88C3
18352 ;hkn; SS override
18353 ; 30/01/2024
18354 ; (MSDOS 6.22 IO.SYS - DOSCODE:62B8h)
18355 ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:69DBh)
18356 ;LES DI,[SS:THISCDS] ; NOTE: PC DOS 7.1 has bug here,
18357 ; ds must be same with ss here...
18358 ; because there is 'les di, [ds:THISCDS]' in
18359 ; Get_Driver_BL
18360 ; and a second 'test byte ptr es:[di+44h],80h'
18361 ; is not necessary; also its result (jnz)
18362 ; overwrites DS. /// Erdogan Tan - 30/01/2024
18363 ; 30/01/2024
18364 ; Retro DOS v5.0
18365 ; (windows ME IO.SYS - BIOSCODE:67ABh)
18366 00002A37 C43E[A205] les di,[THISCDS]
18367 ;test word [es:di+43h],8000h
18368 ;TEST word [ES:DI+curdir.flags],curdir_isnet
18369 ;test byte [es:di+44h],80h
18370 TEST byte [ES:DI+curdir.flags+1],(curdir_isnet>>8) ; (!)
18371 00002A3B 26F6454480 JZ short IOCTLShare
18372 00002A40 7403
18373 ;or dx,1000h ; (MSDOS 3.3)
18374 ; Net devices don't return a device attribute word.
18375 ; Bit 12 = 1, meaning net device, all others = 0.
18376 MOV DX,1000h ; MSDOS 6.0
18377
18378 IOCTLShare:
18379 ; 30/01/2024
18380 ; ds = ss = DOSDATA segment
18381 ;push ss
18382 ;pop ds
18383 MOV SI,OPENBUF
18384 ADD BL,"A" ; 41h
18385 MOV [SI],BL
18386 MOV WORD [SI+1],003AH ; ":",0
18387 MOV AX,0300h
18388 CLC
18389 ;INT int_IBM
18390 int 2Ah ; Microsoft Networks - CHECK DIRECT I/O
18391 ; DS:SI -> ASCIZ disk device name
18392 ; (may be full path or only drive
18393 ; specifier--must include the colon)
18394 ; Return: CF clear if absolute disk access allowed
18395 00002A56 CD2A JNC short IOCTLLocal ; Not shared
18396 ;OR DX,0200H ; Shared, bit 9
18397 ; 17/12/2022
18398 or dh,02h
18399 IOCTLLocal:
18400 ;test word [es:di+43h],1000h
18401 ;TEST word [ES:DI+curdir.flags],curdir_local
18402 ;test byte [es:di+44h],10h
18403 TEST byte [ES:DI+curdir.flags+1],(curdir_local>>8)
18404 ;JZ short ioctl_set_DX
18405 ; 16/12/2022
18406 jz short _ioctl_set_DX
18407 ;OR DX,8000h
18408 ; 17/12/2022
18409 or dh,80h
18410 ;ioctl_set_DX:
18411 _ioctl_set_DX:
18412 ; 16/12/2022
18413 jmp ioctl_set_dx
18414 ; 16/12/2022
18415 %if 0
18416 call Get_User_Stack
18417 MOV [SI+user_env.user_DX],DX
18418 ;;jmp SYS_RET_OK
18419 ;; 25/06/2019
18420 ;jmp SYS_RET_OK_c1c
18421 ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18422 ioctl_gd_ok_j:
18423 jmp short ioctl_da_ok_j
18424 %endif
18425
18426 ioctl_drv_err:
18427 mov al,error_invalid_drive ; 0Fh
18428 ioctl_gd_err_j: ; 11/11/2022
18429 jmp SYS_RET_ERR
18430
18431 ;-----
18432 ;
18433 ; IOCTL: AL = A
18434 ;
18435 ; ENTRY: DS = DOSDATA
18436 ;-----
18437
18438 ioctl_handle_redir:
18439 call SFFromHandle ; ES:DI -> SFT
18440 JNC short ioctl_got_sft ; have valid handle
18441 jmp ioctl_bad_handle ; 10/08/2018
18442
18443 ioctl_got_sft:
18444 ;mov dx,[es:di+5]
18445 MOV DX,[ES:DI+SF_ENTRY.sf_flags] ; Get flags
18446 ;JMP short ioctl_set_DX ; pass dx to user and return
18447 ; 16/12/2022
18448 jmp short _ioctl_set_DX
18449
18450 ; 16/12/2022
18451 ;ioctl_bad_funj:
18452 ;JMP ioctl_bad_fun
18453
18454 ;-----
18455 ;
18456 ;
18457 ;
18458 ;
18459 ;
18460 ;
18461 ;

```

```

18462 ; IOCTL: AL= 4,5
18463 ;
18464 ; ENTRY: DS = DOSDATA
18465 ; SI = user's DS
18466 ;
18467 ;
18468 ; BUGBUG: Don't push anything on the stack between ioctl_get_dev: and
18469 ; the call to Check_If_Net because Check_If_Net gets our
18470 ; return address off the stack if the drive is invalid.
18471 ;
18472 ;-----
18473 ; 30/01/2024 - Retro DOS v5.0
18474 ;
18475 ;
18476 ;
18477 00002A7D E88F00 ; 16/12/2022
18478 ; jz short ioctl_do_string
18479 ; There are no "net devices", and they
18480 ; certainly don't know how to do this
18481 ; call.
18482 00002A80 7403 ; 16/12/2022
18483 ; jz short ioctl_do_string
18484 00002A82 E91AFF ; 16/12/2022
18485 ; jmp short ioctl_bad_funj
18486 ;
18487 ;
18488 ;
18489 ;
18490 00002A85 26F6450540 ; 16/12/2022
18491 00002A8A 74F6 ; jz short ioctl_bad_funj ; NO
18492 ;
18493 00002A8C C606[7E03]03 ; assume IOCTL read
18494 ; MOV byte [IOCALL_REQFUNC],DEVRDIOCTL ; 3
18495 00002A91 A801 ;
18496 00002A93 7405 ; TEST AL,1 ; is it func. 4/5 or 2/3
18497 ; jz short ioctl_control_call ; it is read. goto ioctl_control_call
18498 ;
18499 00002A95 C606[7E03]0C ; it is an IOCTL write
18500 ; MOV byte [IOCALL_REQFUNC],DEVWRIOCTL ; 12
18501 ;
18502 ;
18503 ;
18504 ;
18505 00002A9A B014 ; 30/01/2024
18506 ; mov al,20 ; PCDOS 7.1 IBMDOS.COM
18507 00002A9C 88DC ; 26/06/2024
18508 00002A9E A3[7C03] ; MOV AH,BL ; Unit number
18509 00002AA1 31C0 ; MOV [IOCALL_REQLEN],AX
18510 00002AA3 A3[7F03] ; XOR AX,AX
18511 00002AA6 A2[8903] ; MOV [IOCALL_REQSTAT],AX ; 0
18512 00002AA9 890E[8E03] ; MOV [IOMED],AL
18513 00002AAD 8916[8A03] ; MOV [IOSCNT],CX
18514 00002AB1 8936[8C03] ; MOV [IOXAD],DX
18515 00002AB5 06 ; MOV [IOXAD+2],SI
18516 00002AB6 1F ; PUSH ES
18517 00002AB7 89FE ; POP DS
18518 00002AB9 16 ; MOV SI,DI ; DS:SI -> driver
18519 00002ABA 07 ; PUSH SS
18520 ; POP ES
18521 00002ABB BB[7C03] ; MOV BX,IOCALL ; ES:BX -> Call header
18522 ;
18523 00002ABE E8D027 ; ioctl_do_IO:
18524 ; call DEVIOCALL2
18525 ;
18526 ;
18527 ;
18528 ;
18529 00002AC1 36F606[8003]80 ; hkn; SS override for IOCALL
18530 00002AC7 7507 ; test word [SS:IOCALL_REQSTAT],8000h
18531 ; TEST word [SS:IOCALL_REQSTAT],STERR ; Error?
18532 ; test byte [SS:IOCALL_REQSTAT+1],80h
18533 00002AC9 36A1[8E03] ; TEST byte [SS:IOCALL_REQSTAT+1],(STERR>>8)
18534 ; jnz short ioctl_string_err
18535 ;
18536 ;
18537 ;
18538 ;
18539 ;
18540 00002AD0 368B3E[7F03] ; hkn; SS override
18541 ; MOV AX,[SS:IOSCNT] ; Get actual bytes transferred
18542 00002AD5 81E7FF00 ; 16/12/2022
18543 00002AD9 89F8 ; jmp SYS_RET_OK
18544 00002ADB E8AE37 ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18545 ; jmp short ioctl_gd_ok_j
18546 ;
18547 ;
18548 00002ADE 36A1[2403] ; ioctl_string_err:
18549 ; MOV DI,[SS:IOCALL_REQSTAT];Get Error
18550 ; device_err:
18551 00002AE2 EB88 ; AND DI,STECODE ; 00FFh ; mask out irrelevant bits
18552 ; MOV AX,DI
18553 ; call SET_I24_EXTENDED_ERROR
18554 ;
18555 ;
18556 ;
18557 ;
18558 ;
18559 ;
18560 ;
18561 ;
18562 ;
18563 ;
18564 ;
18565 ;
18566 ;
18567 ;
18568 00002AE4 50 ; hkn; use SS override
18569 00002AE5 88D8 ; hkn; mov ax,[CS:EXTERR]
18570 00002AE7 E89E50 ; mov ax,[SS:EXTERR]
18571 00002AEA 7221 ; jmp SYS_RET_ERR
18572 00002AEC 30DB ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18573 00002AEE C606[2303]03 ; jmp short ioctl_gd_err_j
18574 00002AF3 C43E[A205] ;
18575 ;
18576 ;
18577 ;
18578 00002AF7 26F6454480 ; 17/05/2019 - Retro DOS v4.0
18579 ;
18580 00002AFC 26C47D45 ;
18581 00002B00 750B ;
18582 ;
18583 ;
18584 ;
18585 00002B02 E8E1E7 ;

```



```

18586             ;;
18587             ;mov     bl,[es:di+1]
18588 00002B05 268A5D01 MOV     BL,[ES:DI+DPB.UNIT] ; Unit number
18589             ;les     di,[es:di+13h]
18590 00002B09 26C47D13 LES     DI,[ES:DI+DPB.DRIVER_ADDR] ; Driver addr
18591 got_dev_ptr:
18592             ; 30/01/2024
18593             ; cf=0
18594             ;CLC
18595 iocctl_bad_drv:
18596 00002B0D 58 POP     AX
18597 00002B0E C3 retn
18598
18599 -----
18600 ; Proc Name : Check_If_Net:
18601 ;
18602 ;
18603 ; Checks if the device is over the net or not. Returns result in ZERO flag.
18604 ; If no device is found, the return address is popped off the stack, and a
18605 ; jump is made to iocctl_drv_err.
18606 ;
18607 ; On Entry:
18608 ; Registers same as those for Get_Driver_BL
18609 ;
18610 ; On Exit:
18611 ; ZERO flag - set if not a net device
18612 ;             - reset if net device
18613 ; ES:DI -> the device
18614 ;
18615 ;
18616 ; BUGBUG: This function assumes the following stack setup on entry
18617 ;
18618 ;     SP+2 -> Error return address
18619 ;     SP   -> Normal return address
18620 ;
18621 -----
18622
18623             ; 30/01/2024 - Retro DOS v5.0
18624             ; MSDOS 6.22 MSDOS.SYS - DOSCODE:639Ch
18625             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:6A91h
18626             ; Windows ME IO.SYS - BIOSCODE:68E1h
18627
18628 Check_If_Net:
18629             ; MSDOS 3.3 (& MSDOS 6.0)
18630 00002B0F E8D2FF CALL    Get_Driver_BL
18631 00002B12 7201 JC      short iocctl_drv_err_pop ; invalid drive letter
18632
18633 ; 30/01/2024 ('Get_Driver_BL' returns with
18634 ;             'curdir_isnet' condition/ZF, no need to a second test)
18635 %if 0
18636             ;;
18637             ; (PCDOS 7.1 IBMDOS.COM, Windows ME IO.SYS)
18638             PUSH     ES
18639             PUSH     DI
18640             LES     DI,[THISCDS]
18641             ;test word [es:di+43h],8000h
18642             ;TEST word [ES:DI+curdir.flags],curdir_isnet
18643             ;test byte [es:di+44h],80h
18644             TEST     byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
18645             POP      DI
18646             POP      ES
18647             ;;
18648 %endif
18649 00002B14 C3 retn
18650
18651 iocctl_drv_err_pop:
18652             pop      ax ; pop off return address
18653             jmp      iocctl_drv_err
18654
18655 iocctl_bad_funj3:
18656             jmp      iocctl_bad_fun
18657
18658 iocctl_string_errj:
18659 00002B1C EBB2 jmp      short iocctl_string_err ; 25/05/2019
18660
18661 -----
18662 ; IOCTL: AL = E, F
18663 ;
18664 ; ENTRY: DS = DOSDATA
18665 ;
18666 ;
18667 ; BUGBUG: Don't push anything on the stack between iocctl_drive_owner: and
18668 ; the call to Check_If_Net because Check_If_Net gets our
18669 ; return address off the stack if the drive is invalid.
18670 ;
18671 -----
18672
18673 iocctl_drive_owner:
18674             ; MSDOS 3.3 (& MSDOS 6.0)
18675             Call     Check_If_Net
18676 00002B1E E8EEFF Call     short iocctl_bad_funj3 ; There are no "net devices", and they
18677 00002B21 75F6 JNZ     ; certainly don't know how to do this
18678             ; call.
18679             ;
18680             ;TEST word [ES:DI+SYSDEV.ATT],DEV320 ; See if device can handle this
18681             ; 09/09/2018
18682             ;test byte [es:di+4],40h
18683 00002B23 26F6450440 TEST    byte [ES:DI+SYSDEV.ATT],DEV320 ; 0040h
18684 00002B28 74EF JZ      short iocctl_bad_funj3 ; NO
18685             ;mov     byte [IOCALL_REQFUNC],23
18686 00002B2A C606[7E03]17 mov     byte [IOCALL_REQFUNC],DEVGETOWN ; default to get owner
18687 00002B2F 3C0E cmp     al,0Eh ; Get Owner ?
18688 00002B31 7405 jz      short GetOwner
18689
18690 00002B33 C606[7E03]18 SetOwner: MOV     byte [IOCALL_REQFUNC],DEVSETOWN ; 24
18691
18692 00002B38 B00D GetOwner: MOV     AL,OWNHL ; 13
18693 00002B3A 88DC MOV     AH,BL ; Unit number
18694 00002B3C A3[7C03] MOV     [IOCALL_REQLEN],AX
18695 00002B3F 31C0 XOR     AX,AX
18696 00002B41 A3[7F03] MOV     [IOCALL_REQSTAT],AX
18697 00002B44 06 PUSH    ES
18698 00002B45 1F POP     DS
18699 00002B46 89FE MOV     SI,DI ; DS:SI -> driver
18700 00002B48 16 PUSH    SS
18701 00002B49 07 POP     ES
18702 00002B4A BB[7C03] MOV     BX,IOCALL ; ES:BX -> Call header
18703 00002B4D 1E push    ds
18704 00002B4E 56 push    si
18705 00002B4F E83F27 call    DEVIOCALL2
18706 00002B52 5E pop     si
18707 00002B53 1F pop     ds
18708 ;hkn; SS override
18709             ;TEST word [SS:IOCALL_REQSTAT],STERR ;Error?

```

```

18710             ;test    byte [SS:IOCALL_REQSTAT+1],80h
18711 00002B54 36F606[8003]80 TEST    byte [SS:IOCALL_REQSTAT+1],(STERR>>8)
18712 00002B5A 75C0      jnz    short ioctl_string_errj
18713 00002B5C 36A0[7D03] MOV     AL,[SS:IOCALL_REQUNIT]; Get owner returned by device
18714                                     ; owner returned is 1-based.
18715 00002B60 E90BDB      jmp     SYS_RET_OK
18716
18717 ;=====
18718 ; DELETE.ASM, MSDOS 6.0, 1991
18719 ;=====
18720 ; 07/08/2018 - Retro DOS v3.0
18721 ; 17/05/2019 - Retro DOS v4.0
18722
18723 ; TITLE  DOS_DELETE - Internal DELETE call for MS-DOS
18724 ; NAME   DOS_DELETE
18725
18726 ;
18727 ; Microsoft Confidential
18728 ; Copyright (C) Microsoft Corporation 1991
18729 ; All Rights Reserved.
18730 ;
18731
18732 ;** DELETE.ASM - Low level routine for deleting files
18733 ;-----
18734 ;           DOS_DELETE
18735 ;           REN_DEL_Check
18736 ;           FastOpen_Delete           ; DOS 3.3
18737 ;           FastOpen_Update           ; DOS 3.3
18738
18739 ; Revision history:
18740 ;
18741 ; A000 version 4.00 Jan. 1988
18742 ; A001 Fastopen Rename fix April 1989
18743
18744 ;Installed = TRUE
18745
18746 ; i_need NoSetDir,BYTE
18747 ; i_need Creating,BYTE
18748 ; i_need DELALL,BYTE
18749 ; i_need THISDPB,DWORD
18750 ; i_need THISSFT,DWORD
18751 ; i_need THISCDS,DWORD
18752 ; i_need CURBUF,DWORD
18753 ; i_need ATTRIB,BYTE
18754 ; i_need SATTRIB,BYTE
18755 ; i_need WFP_START,WORD
18756 ; i_need REN_WFP,WORD ;BN001
18757 ; i_need NAME1,BYTE ;BN001
18758 ; i_need FoundDel,BYTE
18759 ; i_need AUXSTACK,BYTE
18760 ; i_need VOLCHNG_FLAG,BYTE
18761 ; i_need JShare,DWORD
18762 ; i_need FastOpenTable,BYTE ; DOS 3.3
18763 ; i_need FastTable,BYTE ; DOS 4.00
18764
18765 ; i_need Del_ExtCluster,WORD ; DOS 4.00
18766
18767 ; i_need SAVE_BX,WORD ; DOS 4.00
18768 ; i_need DMAADD,DWORD
18769 ; i_need RENAMEDMA,BYTE
18770
18771 ;-----
18772 ;
18773 ; Procedure Name : DOS_DELETE
18774 ;
18775 ; Inputs:
18776 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
18777 ; terminated)
18778 ; [CURR_DIR_END] Points to end of Current dir part of string
18779 ; (= -1 if current dir not involved, else
18780 ; points to first char after last "/" of current dir part)
18781 ; [THISCDS] Points to CDS being used
18782 ; (Low word = -1 if NUL CDS (Net direct request))
18783 ; [SATTRIB] Is attribute of search, determines what files can be found
18784 ;
18785 ; Function:
18786 ; Delete the specified file(s)
18787 ;
18788 ; Outputs:
18789 ; CARRY CLEAR
18790 ; OK
18791 ; CARRY SET
18792 ; AX is error code
18793 ; error_file_not_found
18794 ; Last element of path not found
18795 ; error_path_not_found
18796 ; Bad path (not in curr dir part if present)
18797 ; error_bad_curr_dir
18798 ; Bad path in current directory part of path
18799 ; error_access_denied
18800 ; Attempt to delete device or directory
18801 ; ***error_sharing_violation***
18802 ; Deny both access required, generates an INT 24.
18803 ; This error is NOT returned. The INT 24H is generated,
18804 ; and the file is ignored (not deleted). Delete will
18805 ; simply continue on looking for more files.
18806 ; Carry will NOT be set in this case.
18807 ;
18808 ; DS preserved, others destroyed
18809 ;-----
18810
18811 FILEFOUND equ 01h
18812 FILEDELETED equ 10h
18813
18814 ; 12/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
18815 ; DOSCODE:63E9h (MSDOS 5.0, MSDOS.SYS)
18816
18817 ; 30/01/2024 - Retro DOS v5.0 (Modified MSDOS 5.0 IBMDOS.COM)
18818 ; DOSCODE:6B11h (PCDOS 7.1, IBMDOS.COM)
18819
18820 DOS_DELETE:
18821 ;hkn; DOS_Delete is called from file.asm and fcbio.asm. DS has been set up
18822 ;hkn; appropriately at this point.
18823
18824 call TestNet
18825 jnc short LOCAL_DELETE
18826
18827 ;IF NOT Installed
18828 ; transfer NET_DELETE
18829 ;ELSE
18830 ;MOV AX,(MulTNET SHL 8) | 19
18831 ;INT 2FH
18832 ;return
18833
18834 mov ax,1113h

```



```

18834 00002B6B CD2F          int     2Fh      ; Multiplex - NETWORK REDIRECTOR - DELETE REMOTE FILE
18835                          ; SS = DS = DOS CS, SDA first filename pointer ->
18836                          ; fully-qualified filename in DOS CS
18837                          ; SDA CDS pointer -> current directory structure for drive with file
18838                          ; Return: CF set on error
18839 00002B6D C3             retn
18840                          ;ENDIF
18841
18842 LOCAL_DELETE:
18843 00002B6E C606[6F05]00     MOV     byte [FOUNDDEL],0      ; No files found and no files deleted
18844 00002B73 E86CED          call    ECritDisk
18845                          ;mov     word [CREATING],0E500h
18846 00002B76 C706[7E05]00E5 MOV     word [CREATING],DIRFREE*256+0 ; Assume not del *.*
18847 00002B7C 8B36[B205]     MOV     SI,[WFP_START]
18848
18849 00002B80 AC               SKPNUL:
18850 00002B81 08C0             LODSB
18851 00002B83 75FB             OR      AL,AL
18852 00002B85 83EE04           JNZ     short SKPNUL          ; go to end
18853 00002B88 813C2A2E         SUB     SI,4                  ; Back over possible *.*
18854 00002B8C 7506             CMP     word [SI],2E2Ah ; *.*
18855 00002B8E 807C022A        JNZ     short TEST_QUEST
18856 00002B92 741F             CMP     byte [SI+2],"*"
18857                          JZ      short CHECK_ATTTS
18858 00002B94 83EE09           TEST_QUEST:
18859 00002B97 87FE             SUB     SI,9                  ; Back over possible "????????.???"
18860                          XCHG    DI,SI
18861 00002B99 16               push    ss
18862                          ;pop     ds ; ! Retro DOS v3.0 BUG !
18863 00002B9A 07               pop     es ; 17/05/2019
18864
18865 00002B9B B83F3F           MOV     AX,"?" ; 3F3Fh
18866 00002B9E B90400           MOV     CX,4                  ; four sets of "?"
18867 00002BA1 F3AF             REPE    SCASW
18868 00002BA3 751C             JNZ     short NOT_ALL
18869 00002BA5 87FE             XCHG    DI,SI
18870 00002BA7 AD              LODSW
18871 00002BA8 3D2E3F           CMP     AX,3F2Eh ; ".?"
18872 00002BAB 7514             JNZ     short NOT_ALL
18873 00002BAD AD              LODSW
18874 00002BAE 3D3F3F           CMP     AX,"?"
18875 00002BB1 750E             JNZ     short NOT_ALL
18876
18877 00002BB3 A0[6D05]          CHECK_ATTTS:
18878                          MOV     AL,[SATTRIB]
18879 00002BB6 241F             ;and    al,1Fh
18880                          AND     AL,attr_hidden+attr_system+attr_directory+attr_volume_id+attr_read_only
18881                          ;cmp     al,1Fh
18882 00002BB8 3C1F             CMP     AL,attr_hidden+attr_system+attr_directory+attr_volume_id+attr_read_only
18883                          ; All must be set
18884 00002BBA 7505             JNZ     short NOT_ALL
18885
18886 ; NOTE WARNING DANGER-----
18887 ; This DELALL stuff is not safe. It allows directories to be deleted.
18888 ; It should ONLY be used by FORMAT in the ROOT directory.
18889
18890 00002BBC C606[7F05]00     MOV     byte [DELALL],0      ; DEL *.* - flag deleting all
18891
18892 00002BC1 C606[4C03]01     NOT_ALL:
18893 00002BC6 E84C1F           MOV     byte [NoSetDir],1
18894 00002BC9 7312             call    GetPathNoSet
18895 00002BCB 750B             JNC     short Del_found
18896 00002BCD 08C9             JNZ     short _bad_path
18897 00002BCF 7407             OR      CL,CL
18898                          JZ      short _bad_path
18899
18899 00002BD1 B80200           No_file:
18900                          MOV     AX,error_file_not_found
18901 00002BD4 F9               ErrorReturn:
18902                          STC
18903                          ;call    LCritDisk
18904                          ;retn
18905 00002BD5 E937ED           ; 18/12/2022
18906                          jmp     LCritDisk
18907
18908 00002BD8 B80300           _bad_path:
18909 00002BDB EBF7             MOV     AX,error_path_not_found
18910                          JMP      short ErrorReturn
18911
18912 00002BDD 750C             Del_found:
18913 00002BDF 803E[7F05]00     JNZ     short NOT_DIR        ; Check for dir specified
18914 00002BE4 7405             CMP     byte [DELALL],0      ; DelAll = 0 allows delete of dir.
18915                          JZ      short NOT_DIR
18916 00002BE6 B80500           Del_access_err:
18917 00002BE9 EBE9             MOV     AX,error_access_denied
18918                          JMP      short ErrorReturn
18919
18920 00002BEB 08E4             NOT_DIR:
18921 00002BED 78F7             OR      AH,AH                ; Check if device name
18922                          JS      short Del_access_err ; Can't delete I/O devices
18923
18924 ; Main delete loop. CURBUF+2:BX points to a matching directory entry.
18925
18926 00002BEF 800E[6F05]01     DELFILE:
18927                          OR      byte [FOUNDDEL],FILEFOUND ; file found, not deleted yet
18928
18929 ; If we are deleting the Volume ID, then we set VOLUME_CHNG flag to make
18930 ; DOS issue a build BPB call the next time this drive is accessed.
18931
18931 00002BF4 1E               PUSH    DS
18932 00002BF5 8A26[7F05]       MOV     AH,[DELALL]
18933 00002BF9 C53E[E205]       LDS     DI,[CURBUF]
18934
18935 ;hkn; SS override
18936 00002BFD 36F606[6B05]01  TEST    byte [SS:ATTRIB],attr_read_only ; are we deleting RO files too?
18937 00002C03 7509             JNZ     short DoDelete       ; yes
18938
18939 00002C05 F6470B01         TEST    byte [BX+dir_entry.dir_attr],attr_read_only
18940 00002C09 7403             JZ      short DoDelete       ; not read only
18941
18942 ; 30/01/2024 (PCDOS 7.1 IBMDOS.COM)
18943
18944 00002C0B 1F               skip_it:
18945 00002C0C EB57             POP     DS
18946                          JMP      SHORT DELNXT          ; Skip it (Note ES:BP not set)
18947
18948 00002C0E E8AF00           DoDelete:
18949                          call    REN_DEL_Check          ; Sets ES:BP = [THISDPB]
18950                          ;JNC     short DEL_SHARE_OK
18951                          ;POP     DS
18952                          ;JMP     SHORT DELNXT          ; Skip it
18953 00002C11 72F8             ; 30/01/2024
18954                          jc      short skip_it
18955
18956 DEL_SHARE_OK:
18957 ; 17/05/2019 - Retro DOS v4.0
18958 ; MSDOS 6.0

```

```

18958             ;test    byte [di+5],40h
18959 00002C13 F6450540 TEST    byte [DI+BUFFINFO.buf_flags],buf_dirty
18960             ;LB. if already dirty ;AN000;
18961 00002C17 7507 JNZ     short yesdirty ;LB. don't increment dirty count ;AN000;
18962 00002C19 E8A93F call    INC_DIRTY_COUNT ;LB. ;AN000;
18963             ;or      byte [di+5],40h
18964 00002C1C 804D0540 OR      byte [DI+BUFFINFO.buf_flags],buf_dirty
18965 yesdirty:
18966 00002C20 8827 mov     [bx],ah
18967             ;MOV     [BX+dir_entry.dir_name],AH ; Put in E5H or 0
18968             ;;;
18969             ; 30/01/2024 - Retro DOS v5.0
18970             ; (PCDOS 7.1 IBMDOS.COM)
18971 00002C22 31DB xor     bx,bx ; 0
18972             ;cmp     [es:bp+0Fh],bx
18973 00002C24 26395E0F cmp     [es:bp+DPB.FAT_SIZE],bx ; 0
18974 00002C28 7503 jnz     short yesdirty_fc_1 ; not FAT32
18975 00002C2A 8B5CFA mov     bx,[si-6] ; high word of the first cluster (FAT32)
18976 yesdirty_fc_1:
18977             ;mov     [ss:CLUSTNUM_HW],bx
18978 00002C2D 89DA mov     dx,bx ; * ; 30/01/2024 - Retro DOS v5.0
18979             ;;;
18980             MOV     BX,[SI] ; Get firclus pointer
18981 00002C2F 8B1C POP     DS
18982 00002C31 1F             ;;;
18983             mov     [CLUSTNUM_HW],dx ; * ; 30/01/2024 - Retro DOS v5.0
18984 00002C32 8916[E80A]             ;;;
18985             OR      byte [FOUNDEL],FILEDELETED ; 10h ; Deleted file
18986 00002C36 800E[6F05]10
18987             ; 30/01/2024
18988             ;CMP     BX,2
18989             ;JB      short DELNXT ; File has invalid FIRCLUS (too small)
18990             ;;;
18991             ; 30/01/2024 - Retro DOS v5.0
18992             ; (PCDOS 7.1 IBMDOS.COM)
18993             ;cmp     word [CLUSTNUM_HW],0
18994             ;and     dx,dx ; 30/01/2024 - Retro DOS v5.0
18995 00002C3B 21D2 jnz     short yesdirty_fc_2
18996 00002C3D 7505 cmp     bx,2
18997 00002C3F 83FB02 jb      short DELNXT ; File has invalid FIRCLUS (too small)
18998 00002C42 7221 yesdirty_fc_2:
18999             ;cmp     word [es:bp+0Fh],0
19000             ;cmp     word [es:bp+DPB.FAT_SIZE],0
19001 00002C44 26837E0F00 jnz     short yesdirty_fc_3 ; not FAT32
19002 00002C49 750C ;push    bx
19003             ;mov     bx,[CLUSTNUM_HW]
19004             ;cmp     bx,[es:bp+2Fh]
19005             ;cmp     bx,[es:bp+DPB.LAST_CLUSTER+2]
19006             ;30/01/2024 - Retro DOS v5.0
19007             ;mov     dx,[CLUSTNUM_HW]
19008             ;dx = [CLUSTNUM_HW] ; *
19009             ;cmp     dx,[es:bp+DPB.LAST_CLUSTER+2]
19010 00002C4B 263B562F ;pop     bx
19011             ;jne     short yesdirty_fc_4
19012 00002C4F 750A ;cmp     bx,[es:bp+2Dh]
19013             ;cmp     bx,[es:bp+DPB.LAST_CLUSTER]
19014 00002C51 263B5E2D jmp     short yesdirty_fc_4
19015 00002C55 E804 yesdirty_fc_3:
19016             ;;;
19017             ;cmp     bx,[es:bp+0Dh]
19018             ;CMP     BX,[ES:BP+DPB.MAX_CLUSTER]
19019 00002C57 263B5E0D yesdirty_fc_4: ; 30/01/2024
19020             ;JA      short DELNXT ; File has invalid FIRCLUS (too big)
19021 00002C5B 7708             ;;;
19022             ;call    RELEASE ; Free file data
19023 00002C5D E86C2F             ;JC      short No_fileJ
19024 00002C60 7258
19025             ; DOS 3.3 FastOpen
19026             ;
19027             ;
19028 00002C62 E8D900 CALL    FastOpen_Delete ; delete the dir info in fastopen
19029             ;
19030             ; DOS 3.3 FastOpen
19031             ;
19032             ;
19033 00002C65 C42E[8A05] DELNXT:
19034 00002C69 E82F1C LES     BP,[THISDPB] ; Possible to get here without this set
19035 00002C6C 724C call    GETENTRY ; Registers need to be reset
19036 00002C6E E8261B JC      short No_fileJ
19037             ;call    NEXTENT
19038             ;JNC     short DELFILE
19039             ; 30/01/2024
19040 00002C71 7203 jc      short DELNXT2
19041 00002C73 E979FF jmp     DELFILE
19042 00002C76 C42E[8A05] DELNXT2:
19043             LES     BP,[THISDPB] ; NEXTENT sets ES=DOSGROUP
19044             ;;;
19045             ; 30/01/2024 - Retro DOS v5.0
19046             ; (PCDOS 7.1 IBMDOS.COM)
19047             ;
19048 00002C7A E8EB07 call    update_fat32_fsinfo
19049             ;;;
19050             ; 12/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
19051             ;MOV     AL,[ES:BP+DPB.DRIVE]
19052             ;mov     al,[es:bp+0]
19053             ; 15/12/2022
19054             MOV     AL,[ES:BP]
19055 00002C7D 268A4600 call    FLUSHBUF
19056 00002C81 E80F3E JC      short No_fileJ
19057 00002C84 7234
19058             ;
19059             ; Now we need to test FoundDel for our flags. The cases to consider are:
19060             ;
19061             ; not found not deleted file not found
19062             ; not found deleted *** impossible ***
19063             ; found not deleted access denied (read-only)
19064             ; found deleted no error
19065             ;
19066 00002C86 F606[6F05]10 TEST    byte [FOUNDEL],FILEDELETED ; did we delete a file?
19067 00002C8B 7426 JZ      short DelError ; no, figure out what's wrong.
19068             ;
19069             ; We set VOLCHNG_FLAG to indicate that we have changed the volume label
19070             ; and to force the DOS to issue a media check.
19071             ;
19072 00002C8D F606[6B05]08 TEST    byte [ATTRIB],attr_volume_id ; 8
19073 00002C92 741C jz      short No_Set_Flag
19074 00002C94 50 PUSH    AX
19075 00002C95 06 PUSH    ES
19076 00002C96 57 PUSH    DI
19077 00002C97 C43E[A205] LES     DI,[THISCDS]
19078 00002C9B 268A25 MOV     AH,[ES:DI] ; Get drive
19079 00002C9E 80EC41 SUB     AH,'A' ; Convert to 0-based
19080 00002CA1 8826[A10A] mov     [VOLCHNG_FLAG],AH
19081

```

```

19082          ; MSDOS 6.0
19083 00002CA5 30FF      XOR     BH,BH          ;>32mb delete volume id from boot record ;AN000;
19084 00002CA7 E8EB04    call    Set_Media_ID      ;>32mb set volume id to boot record ;AN000;
19085
19086 00002CAA E8A938     call    FATREAD_CDS          ; force media check
19087 00002CAD 5F        POP     DI
19088 00002CAE 07        POP     ES
19089 00002CAF 58        POP     AX
19090
19091 No_Set_Flag:         ;call    LCritDisk          ; carry is clear
19092                     ;retn
19093                     ; 18/12/2022
19094 00002CB0 E95CEC     jmp     LCritDisk
19095
19096 00002CB3 F606[6F05]01 DelError:         TEST     byte [FOUNDDEL],FILEFOUND ; not deleted. Did we find file?
19097 00002CB8 7503        JNZ     short Del_access_errJ ; yes. Access denied
19098
19099 00002CBA E914FF       JMP     No_file ; 10/08/2018          ; Nope
19100 Del_access_errJ:
19101 00002CBD E926FF       JMP     Del_access_err ; 10/08/2018
19102
19103 ; 08/08/2018 - Retro DOS v3.0
19104
19105 ;Break <REN_DEL_Check - check for access for rename and delete>
19106 -----
19107 ; Procedure Name : REN_DEL_Check
19108 ;
19109 ; Inputs:
19110 ; [THISDPB] set
19111 ; [CURBUF+2]:BX points to entry
19112 ; [CURBUF+2]:SI points to firlus field of entry
19113 ; [WFP_Start] points to name
19114 ; Function:
19115 ; Check for Exclusive access on given file.
19116 ; Used by RENAME, SET_FILE_INFO, and DELETE.
19117 ; Outputs:
19118 ; ES:BP = [THISDPB]
19119 ; NOTE: The WFP string pointed to by [WFP_Start] will be Modified. The
19120 ; last element will be loaded from the directory entry. This is
19121 ; so the name given to the sharer doesn't have any meta chars in
19122 ; it.
19123 ; Carry set if sharing violation, INT 24H generated
19124 ; NOTE THAT AX IS NOT error_sharing_violation.
19125 ; This is because input AX is preserved.
19126 ; Caller must set the error if needed.
19127 ; Carry clear
19128 ; OK
19129 ; AX,DS,BX,SI,DI preserved
19130 -----
19131
19132 ; 31/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
19133
19134 REN_DEL_Check:
19135
19136 00002CC0 1E          PUSH    DS
19137 00002CC1 57          PUSH    DI
19138 00002CC2 50          PUSH    AX
19139 00002CC3 53          PUSH    BX
19140 00002CC4 56          PUSH    SI          ; Save CURBUF pointers
19141
19142 00002CC5 16          push     ss
19143 00002CC6 07          pop      es
19144
19145 ;hkn; context ES will assume ES to DOSDATA
19146 ;hkn; ASSUME ES:DOSGROUP
19147
19148 ;hkn; SS override
19149 00002CC7 368B3E[B205] MOV     DI,[SS:WFP_START] ; ES:DI -> WFP
19150 00002CCC 89DE        MOV     SI,BX
19151
19152 ;hkn; SS override
19153 00002CCE 368E1E[E405] MOV     DS,[SS:CURBUF+2] ; DS:SI -> entry (FCB style name)
19154 00002CD3 89FB        MOV     BX,DI          ; Set backup limit for skipback
19155 ;ADD     BX,2          ; Skip over d: to point to leading '\'
19156 ; 31/01/2024
19157 00002CD5 43          inc     bx
19158 00002CD6 43          inc     bx
19159 00002CD7 E8EFEA     call    StrLen          ; CX is length of ES:DI including NUL
19160 00002CDA 49          DEC     CX          ; Don't include nul in count
19161 00002CDB 01CF        ADD     DI,CX          ; Point to NUL at end of string
19162 00002CDD E85A51     call    SkipBack        ; Back up one element
19163 00002CE0 47          INC     DI          ; Point to start of last element
19164
19165 ; 17/05/2019 - Retro DOS v4.0
19166 ;hkn; SS override
19167 ; MSDOS 6.0
19168 00002CE1 36893E[0106] MOV     [SS:SAVE_BX],DI ;IFS. save for DOS_RENAME ;AN000;
19169
19170 00002CE6 E8BDF9     call    PackName        ; Transfer name from entry to ASCIZ tail.
19171 00002CE9 5E          POP     SI          ; Get back entry pointers
19172 00002CEA 5B          POP     BX
19173 00002CEB 53          PUSH    BX
19174 00002CEC 56          PUSH    SI          ; Back on stack
19175
19176 00002CED 16          push     ss
19177 00002CEE 1F          pop      ds
19178
19179 ;hkn; context DS will assume ES to DOSDATA
19180 ;hkn; ASSUME DS:DOSGROUP
19181
19182 ; 07/07/2024
19183 ;push     es          ; (not necessary) ; 31/01/2024
19184 ;push     di
19185
19186 ; Close the file if possible by us.
19187 ;
19188 ;if installed
19189 00002CEF FF1E[C400]   call    far [Jshare+(13*4)] ; 13 = ShCloseFile
19190 ;else
19191 ; Call    ShCloseFile
19192 ;endif
19193 ;;;
19194 ; 31/01/2024 - Retro DOS v5.0
19195 ; PCDOS 7.1 IBMDOS.COM
19196 00002CF3 803E[0303]FF cmp     byte [fShare],0FFh
19197 00002CF8 750A        jne     short rdc_1
19198 00002CFA 8C06[A005]   mov     [THISSFT+2],es
19199 00002CFE 893E[9E05]   mov     [THISSFT],di
19200 00002D02 EB0A        jmp     short rdc_2
19201 rdc_1:
19202 ;;;
19203 00002D04 8C1E[A005]   MOV     [THISSFT+2],DS
19204
19205 ;hkn; AUXSTACK is in DOSDATA

```

```

19206 00002D08 C706[9E05][6507]      MOV     word [THISSFT],AUXSTACK-SF_ENTRY.size ; RENAMEDMA+(384-59)
19207                                ; Scratch space
19208                                ; 30/01/2024
19209                                ; 07/07/2024
19210                                ; pop     di
19211                                ; pop     es
19212                                ;
19213 00002D0E 30E4                    XOR     AH,AH          ; Indicate file to DOOPEN (high bit off)
19214 00002D10 E8EC28                  call    DOOPEN        ; Fill in SFT for share check
19215 00002D13 C43E[9E05]              LES     DI,[THISSFT]
19216                                ; mov     word [es:di+2],10h
19217 00002D17 26C745021000             MOV     word [ES:DI+SF_ENTRY.sf_mode],SHARING_DENY_BOTH ; 10h
19218                                ; requires exclusive access
19219                                ; MOV     word [ES:DI+SF_ENTRY.sf_ref_count],1 ; Pretend open
19220 00002D1D 26C7050100             mov     word [ES:DI],1
19221 00002D22 E85F57                    call    ShareEnter
19222 00002D25 720D                      jc      short CheckDone
19223 00002D27 C43E[9E05]              LES     DI,[THISSFT]
19224                                ; MOV     word [ES:DI+SF_ENTRY.sf_ref_count],0
19225 00002D2B 26C7050000             mov     word [ES:DI],0 ; Pretend closed and free
19226                                ;
19227 00002D30 E84C57                    call    ShareEnd      ; Tell sharer we're done with THISSFT
19228 00002D33 F8                      CLC
19229                                ;
19230 00002D34 C42E[8A05]              CheckDone:
19231 00002D38 5E                        LES     BP,[THISDPB]
19232 00002D39 5B                        POP     SI
19233 00002D3A 58                        POP     BX
19234 00002D3B 5F                        POP     AX
19235 00002D3C 1F                        POP     DI
19236 00002D3D C3                        POP     DS
19237                                ; retn
19238                                ;
19239                                ; Break <FastOpen_Delete - delete dir info in fastopen>
19240                                ; -----
19241                                ; Procedure Name : FastOpen_Delete
19242                                ; Inputs:
19243                                ; None
19244                                ; Function:
19245                                ; Call FastOpen to delete the dir info.
19246                                ; Outputs:
19247                                ; None
19248                                ; -----
19249                                ;
19250                                ; 31/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
19251                                ;
19252 00002D3E 9C                        FastOpen_Delete:
19253 00002D3F 56                        PUSHF          ; save flag
19254 00002D40 57                        PUSH     SI    ; save registers
19255 00002D41 53                        push     di ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19256 00002D42 50                        PUSH     BX
19257                                ; MOV     SI,[WFP_START] ; MSDOS 3.3
19258                                ; hkn; SS override
19259                                ; 17/05/2019 - Retro DOS v4.0
19260                                ; MSDOS 6.0
19261 00002D43 368B36[B205]             MOV     SI,[ss:WFP_START] ; ds:si points to path name
19262                                ;
19263 00002D48 B003                      MOV     AL,FONC_delete ; al = 3
19264                                ;
19265                                ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19266                                ; if 0
19267                                ; fastinvoke:
19268                                ; hkn; FastTable is in DOSDATA
19269                                ; MOV     BX,FastTable+2
19270                                ; CALL    far [BX] ; call fastopen
19271                                ; POP     AX ; restore registers
19272                                ; POP     BX
19273                                ; pop     di ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19274                                ; POP     SI
19275                                ; POPF          ; restore flag
19276                                ; retn
19277                                ; else
19278 00002D4A EB0F                      jmp     short fastinvoke ; 31/01/2024
19279                                ; endif
19280                                ;
19281                                ; 13/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
19282                                ; DOSCODE:65A0h (MSDOS 5.0 MSDOS.SYS)
19283                                ;
19284                                ; 31/01/2024 Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
19285                                ; DOSCODE:65B4h (MSDOS 6.22 MSDOS.SYS)
19286                                ; DOSCODE:6D07h (PCDOS 7.1 IBMDOS.COM)
19287                                ;
19288                                ; Break <FastOpen_Rename - Rename directory> ; PTR 5622
19289                                ; -----
19290                                ; PROCEDURE Name : FastOpen_Rename
19291                                ;
19292                                ; Inputs:
19293                                ; REN_WFP = Path Name
19294                                ; NAME1 = New Name
19295                                ; Function:
19296                                ; Call FastOpen to rename the dir entry in the cache
19297                                ; Outputs:
19298                                ; None
19299                                ; -----
19300                                ;
19301                                ; FastOpen_Rename:
19302                                ; 17/05/2019 - Retro DOS v4.0
19303                                ; 08/08/2018 - Retro DOS v3.0
19304                                ; MSDOS 6.0
19305 00002D4C 9C                        PUSHF          ;AN001 save flag
19306 00002D4D 56                        PUSH     SI    ;AN001 save registers
19307 00002D4E 57                        PUSH     DI    ;AN001
19308 00002D4F 53                        PUSH     BX    ;AN001
19309 00002D50 50                        PUSH     AX    ;AN001
19310                                ;
19311                                ; hkn; SS override
19312 00002D51 368B36[B405]             MOV     SI,[ss:REN_WFP] ;AN001 ;AN001 ds:si-->Path name addr
19313                                ;
19314                                ; hkn; NAME1 is in DOSDATA
19315 00002D56 BF[4B05]                 MOV     DI,NAME1 ;AN001 ds:di-->New name addr
19316                                ; mov     al,6
19317 00002D59 B006                      MOV     AL,FONC_Rename ;AN001 al = 6
19318                                ;
19319                                ; fastinvoke: ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19320                                ;
19321                                ; hkn; FastTable is in DOSDATA
19322 00002D5B BB[3E11]                 MOV     BX,FastTable+2
19323 00002D5E FFI1                     CALL    far [BX] ;AN001 call fastopen
19324                                ;
19325 00002D60 58                        POP     AX ; restore registers ;AN001
19326 00002D61 5B                        POP     BX ;AN001
19327 00002D62 5F                        POP     DI ;AN001
19328 00002D63 5E                        POP     SI ;AN001
19329 00002D64 9D                        POPF          ; restore flag ;AN001

```

```

19330 00002D65 C3      retn                                ;AN001
19331
19332 ;Break      <FastOpen_Update - update dir info in fastopen>
19333 ;-----
19334 ; Procedure Name : FastOpen_Update
19335 ;
19336 ; Inputs:
19337 ;     DL      drive number (A=0,B=1,,, )
19338 ;     CX      first cluster #
19339 ;     AH      0 updates dir entry
19340 ;             1 updates CLUSNUM , BP = new CLUSNUM
19341 ;     ES:DI    directory entry
19342 ; Function:
19343 ;     Call FastOpen to update the dir info.
19344 ; Outputs:
19345 ;     None
19346 ;-----
19347
19348 FastOpen_Update:
19349     PUSHF                    ; save flag
19350     PUSH     SI
19351     push     di ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19352     PUSH     BX              ; save regs
19353     PUSH     AX
19354     MOV      AL,FONC_update ; al = 4
19355     JMP      short fastinvoke
19356
19357     ; 17/05/2019
19358
19359     ; MSDOS 6.0
19360 ;entry Fast_Dispatch          ; future fastxxxx entry      ;AN000;
19361 ;-----
19362 Fast_Dispatch:
19363 ;hkn; FastTable is in DOSDATA
19364     MOV      SI,FastTable+2 ; index to the      ;AN000;
19365 ;hkn; use SS override
19366     CALL     far [SS:SI]    ; RMFD call fastopen
19367     retn
19368
19369 ;=====
19370 ; RENAME.ASM, MSDOS 6.0, 1991
19371 ;=====
19372 ; 08/08/2018 - Retro DOS v3.0
19373 ; 17/05/2019 - Retro DOS v4.0
19374
19375 ; TITLE  DOS_RENAME - Internal RENAME call for MS-DOS
19376 ; NAME   DOS_RENAME
19377
19378 ;** Low level routine for renaming files
19379 ;-----
19380 ; DOS_RENAME
19381 ;
19382 ; Modification history:
19383 ;
19384 ; Created: ARR 30 March 1983
19385 ;
19386 ;-----
19387
19388 ; Procedure Name : DOS_RENAME
19389 ;
19390 ; Inputs:
19391 ;     [WFP_START] Points to SOURCE WFP string ("d:/" must be first 3
19392 ;                 chars, NUL terminated)
19393 ;     [CURR_DIR_END] Points to end of current dir part of string [SOURCE]
19394 ;                 (= -1 if current dir not involved, else
19395 ;                 points to first char after last "/" of current dir part)
19396 ;     [REN_WFP] Points to DEST WFP string ("d:/" must be first 3
19397 ;                 chars, NUL terminated)
19398 ;     [THISCDS] Points to CDS being used
19399 ;                 (Low word = -1 if NUL CDS (Net direct request))
19400 ;     [SATTRIB] Is attribute of search, determines what files can be found
19401 ; Function:
19402 ;     Rename the specified file(s)
19403 ;     NOTE: This routine uses most of AUXSTACK as a temp buffer.
19404 ; Outputs:
19405 ;     CARRY CLEAR
19406 ;     OK
19407 ;     CARRY SET
19408 ;     AX is error code
19409 ;     error_file_not_found
19410 ;         No match for source, or dest path invalid
19411 ;     error_not_same_device
19412 ;         Source and dest are on different devices
19413 ;     error_access_denied
19414 ;         Directory specified (not simple rename),
19415 ;         Device name given, Destination exists.
19416 ;         NOTE: In third case some renames may have
19417 ;         been done if metas.
19418 ;     error_path_not_found
19419 ;         Bad path (not in curr dir part if present)
19420 ;         SOURCE ONLY
19421 ;     error_bad_curr_dir
19422 ;         Bad path in current directory part of path
19423 ;         SOURCE ONLY
19424 ;     error_sharing_violation
19425 ;         Deny both access required, generates an INT 24.
19426 ; DS preserved, others destroyed
19427 ;
19428 ;-----
19429
19430 ; 14/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
19431 ; 31/01/2024 Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
19432
19433 DOS_RENAME:
19434
19435 ;hkn; DOS_RENAME is called from file.asm and fcbio.asm. DS has been set up
19436 ;hkn; at this point to DOSDATA.
19437
19438     call     TestNet
19439     JNC      short LOCAL_RENAME
19440
19441 ;IF NOT Installed
19442 ; transfer NET_RENAME
19443 ;ELSE
19444 ;MOV      AX,(MuItNET SHL 8) OR 17
19445 ;INT      2FH
19446 ;return
19447
19448     mov      ax, 1111h
19449     int      2Fh ; Multiplex - NETWORK REDIRECTOR - RENAME REMOTE FILE
19450 ; SS = DS = DOS CS,
19451 ; SDA first filename pointer = offset of fully-qualified old name
19452 ; SDA CDS pointer -> current directory
19453 ; Return: CF set on error

```

```

19454 00002D80 C3          retn
19455                      ;ENDIF
19456
19457                      LOCAL_RENAME:
19458 00002D81 C606[2303]02    MOV     byte [EXTERR_LOCUS],errLOC_Disk ; 2
19459 00002D86 8B36[B205]    MOV     SI,[WFP_START]
19460 00002D8A 8B3E[B405]    MOV     DI,[REN_WFP]
19461 00002D8E 8A04          MOV     AL,[SI]
19462 00002D90 8A25          MOV     AH,[DI]
19463 00002D92 0D2020        OR      AX,2020H          ; Lower case
19464 00002D95 38E0          CMP     AL,AH
19465 00002D97 7405          JZ      short SAMEDRV
19466 00002D99 B81100        MOV     AX,error_not_same_device ; 11h
19467 00002D9C F9          STC
19468 00002D9D C3          retn
19469
19470                      SAMEDRV:
19471 00002D9E FF36[2E03]    PUSH    WORD [DMAADD+2]
19472 00002DA2 FF36[2C03]    PUSH    WORD [DMAADD]
19473 00002DA6 8C1E[2E03]    MOV     [DMAADD+2],DS
19474
19475                      ;hkn; RENAMEDMA is in DOSDATA
19476 00002DAA C706[2C03][2006] MOV     WORD [DMAADD],RENAMEDMA
19477 00002DB0 C606[7005]00    MOV     byte [FOUND_DEV],0      ; Rename fails on DEVS, assume not a dev
19478 00002DB5 E82AEB        call    ECritDisk
19479 00002DB8 E86507        call    DOS_SEARCH_FIRST      ; Sets [NoSetDir] to 1, [CURBUF+2]:BX
19480                                ; points to entry
19481 00002DBB 7314          JNC     short Check_Dev
19482 00002DBD 83F812        CMP     AX,error_no_more_files ; 12h
19483 00002DC0 7503          JNZ     short GOTERR
19484 00002DC2 B80200        MOV     AX,error_file_not_found ; 2
19485
19486 00002DC5 F9          GOTERR: STC
19487                      RENAME_POP:
19488 00002DC6 8F06[2C03]    POP     WORD [DMAADD]
19489 00002DCA 8F06[2E03]    POP     WORD [DMAADD+2]
19490                                ;call    LCritDisk
19491                                ;retn
19492                                ; 16/12/2022
19493 00002DCE E93EEB        jmp     LCritDisk
19494
19495                      Check_Dev:
19496                      ; 17/05/2019 - Retro DOS v4.0
19497                      ;mov     ax,5
19498 00002DD1 B80500        MOV     AX,error_access_denied; Assume error
19499
19500                      ; MSDOS 6.0
19501 00002DD4 1E          PUSH    DS                ;PTM.                ;AN000;
19502 00002DD5 C536[2C03]    LDS     SI,[DMAADD]        ;PTM.  check if source a dir ;AN000;
19503                                ;add     si,21
19504 00002DD9 83C615        ADD     SI,find_buf.attr    ;PTM.                ;AN000;
19505                                ;test    byte [si+11],10h
19506 00002DDC F6440B10    TEST    byte [SI+dir_entry.dir_attr],attr_directory ;PTM.                ;AN000;
19507 00002DE0 7407          JZ      short notdir        ;PTM.                ;AN000;
19508 00002DE2 8B36[B405]    MOV     SI,[REN_WFP]        ;PTM.  if yes, make sure path ;AN000;
19509 00002DE6 E86AFA        call    Check_PathLen2      ;PTM.  length < 67      ;AN000;
19510
19511 00002DE9 1F          notdir: POP     DS                ;PTM.                ;AN000;
19512 00002DEA 77D9          JA      short GOTERR        ;PTM.                ;AN000;
19513
19514                      ; MSDOS 3.3 & MSDOS 6.0
19515 00002DEC 803E[7005]00    CMP     byte [FOUND_DEV],0
19516 00002DF1 75D2          JNZ     short GOTERR
19517
19518                      ; At this point a source has been found. There is search continuation info (a
19519                      ; la DOS_SEARCH_NEXT) for the source at RENAMEDMA, together with the first
19520                      ; directory entry found.
19521                      ; [THISCDs], [THISDPB], and [THISDRV] are set and will remain correct
19522                      ; throughout the RENAME since it is known at this point that the source and
19523                      ; destination are both on the same device.
19524                      ; [SATTRIB] is also set.
19525
19526 00002DF3 89DE          MOV     SI,BX
19527                                ;add     si,26
19528 00002DF5 83C61A        ADD     SI,dir_entry.dir_first
19529 00002DF8 E8C5FE        call    REN_DEL_Check
19530 00002DFB 7305          JNC     short REN_OK1
19531 00002DFD B82000        MOV     AX,error_sharing_violation ; 20h
19532 00002E00 EBC4          JMP     short RENAME_POP
19533
19534                      ;-----
19535                      ; Check if the source is a file or directory. If file, delete the entry
19536                      ; from the Fastopen cache. If directory, rename it later
19537                      ;-----
19538
19539                      REN_OK1:
19540                      ; MSDOS 6.0
19541                      ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19542                      ;PUSH    SI
19543 00002E02 C536[2C03]    LDS     SI,[DMAADD]        ;BN00X; PTM.  check if source a dir ;AN000;
19544                                ;add     si,21
19545 00002E06 83C615        ADD     SI,find_buf.attr    ;;BN00XPTM.P5520          ;AN000;
19546                                ;test    byte [si+11],10h
19547 00002E09 F6440B10    TEST    byte [SI+dir_entry.dir_attr],attr_directory ;;BN00XPTM. ;AN000;
19548                                ;JZ      short NOT_DIR1      ;;BN00XPTM.          ;AN000;
19549 00002E0D 7503          jnz     short SWAP_SOURCE ; 31/01/2024
19550                                ;POP     SI                ;BN00X
19551                                ;JMP     SHORT SWAP_SOURCE    ;BN00X
19552                      ;NOT_DIR1:
19553                      ;POP     SI                ;;BN00X it is a file, delete the entry
19554
19555                      ; MSDOS 3.3 (& MSDOS 6.0)
19556 00002E0F E82CFF        call    FastOpen_Delete      ; delete dir info in fastopen DOS 3.3
19557
19558                      SWAP_SOURCE:
19559                      ; MSDOS 3.3
19560                      ;MOV     SI,[REN_WFP]
19561                      ;MOV     [WFP_START],SI
19562                      ; MSDOS 6.0
19563 00002E12 A1[B205]        MOV     AX,[WFP_START]      ; Swap source and destination
19564 00002E15 8B36[B405]    MOV     SI,[REN_WFP]      ; Swap source and destination
19565 00002E19 8936[B205]    MOV     [WFP_START],SI    ; WFP_START = Destination path
19566 00002E1D A3[B405]        MOV     [REN_WFP],AX       ; REN_WFP = Source path
19567                                ; MSDOS 3.3 (& MSDOS 6.0)
19568 00002E20 C706[B605]FFFF MOV     word [CURR_DIR_END],-1; No current dir on dest
19569 00002E26 C706[7E05]FFE5 MOV     word [CREATING],0E5FFh
19570                                MOV     WORD [CREATING],DIRFREE*256+0FFh ; Creating, not DEL *.*
19571                                ; A rename is like a CREATE_NEW as far
19572                                ; as the destination is concerned.
19573 00002E2C E8E61C        call    GetPathNoSet
19574
19575                      ; If this GETPATH fails due to file not found, we know all renames will work
19576                      ; since no files match the destination name. If it fails for any other
19577                      ; reason, the rename fails on a path not found, or whatever (also fails if
19578                      ; we find a device or directory). If the GETPATH succeeds, we aren't sure

```



```

19578 ; if the rename should fail because we haven't built an explicit name by
19579 ; substituting for the meta chars in it. In this case the destination file
19580 ; spec with metas is in [NAME1] and the explicit source name is at RENAMEDMA
19581 ; in the directory entry part.
19582
19583 00002E2F 722A JC short NODEST
19584
19585 ; MSDOS 6.0
19586 ;JZ short BAD_ACC ; Dest string is a directory ;AC000;
19587 ; !! MSDOS 3.3 !!
19588 ;JZ short BAD_ACC ; !! ; Dest string is a directory
19589
19590 00002E31 08E4 OR AH,AH ; Device?
19591 00002E33 7933 JNS short SAVEDEST ; No, continue
19592
19593 00002E35 B80500 BAD_ACC: MOV AX,error_access_denied
19594 00002E38 F9 STC
19595
19596 00002E39 9C RENAME_CLEAN: PUSHF ; Save carry state
19597 00002E3A 50 PUSH AX ; and error code (if carry set)
19598 ;;;
19599 ; 31/01/2024 - Retro DOS v5.0
19600 ; (PCDOS 7.1 IBMDOS.COM)
19601 00002E3B C42E[8A05] les bp,[THISDPB]
19602 00002E3F E82606 call update_fat32_fsinfo
19603 ;;;
19604 00002E42 A0[7605] MOV AL,[THISDRV]
19605 00002E45 E84B3C call FLUSHBUF
19606 00002E48 58 POP AX
19607 00002E49 803E[4A03]00 CMP byte [FAILERR],0
19608 00002E4E 7504 JNZ short BAD_ERR ; User FAILED to I 24
19609 00002E50 9D POPF
19610 00002E51 E972FF JMP RENAME_POP
19611
19612
19613 00002E54 58 BAD_ERR: POP AX ; saved flags
19614 ; 16/12/2022
19615 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
19616
19617 00002E55 B80300 BAD_PATH: ; *
19618 00002E58 E96AFF MOV AX,error_path_not_found
19619 JMP GOTERR
19620
19621 00002E5B 75F8 NODEST: JNZ short BAD_PATH
19622 00002E5D 803E[4A03]00 CMP byte [FAILERR],0
19623 00002E62 75F1 JNZ short BAD_PATH ; Search for dest failed
19624 ; because user FAILED on I 24
19625 ; 14/11/2022
19626 00002E64 08C9 OR CL,CL
19627 ;JNZ short SAVEDEST
19628 ; 17/05/2019
19629 00002E66 74ED jz short BAD_PATH ; *
19630 ;BAD_PATH: ; *
19631 ; MOV AX,error_path_not_found
19632 ; ;STC
19633 ; ;JMP RENAME_POP
19634 ; ; 17/05/2019
19635 ; jmp GOTERR
19636
19637 ; 16/12/2022
19638 %if 0
19639 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
19640 or cl,cl
19641 jnz short SAVEDEST
19642 ;jz short BAD_PATH ; *
19643
19644 BAD_PATH: ; *
19645 ;mov ax,3
19646 mov ax,error_path_not_found
19647 stc
19648 jmp RENAME_POP
19649 %endif
19650
19651 00002E68 16 SAVEDEST: push ss
19652 00002E69 07 pop es
19653
19654 ;hkn; NAME1 & NAME2 is in DOSDATA
19655 00002E6A BF[5705] MOV DI,NAME2
19656 00002E6D BE[4B05] MOV SI,NAME1
19657
19658 00002E70 B90B00 MOV CX,11
19659 00002E73 F3A4 REP MOVSB ; Save dest with metas at NAME2
19660 ;;;
19661 ; 31/01/2024
19662 ; (PCDOS 7.1 IBMDOS.COM)
19663 00002E75 A1[DC0A] mov ax,[DIRSTART_HW]
19664 00002E78 A3[E60A] mov [DESTSTART_HW],ax
19665 ;;;
19666 00002E7B A1[C205] MOV AX,[DIRSTART]
19667 00002E7E A3[6405] MOV [DESTSTART],AX
19668
19669 BUILDDEST: ; 31/01/2024
19670 ;push ss
19671 ;pop es ; needed due to JMP BUILDDEST below
19672
19673 ;hkn; RENAMEDMA, NAME1, NAME2 in DOSDATA
19674 00002E81 BB[3506] MOV BX,RENAMEDMA+21 ; Source of replace chars
19675 00002E84 BF[4B05] MOV DI,NAME1 ; Real dest name goes here
19676 00002E87 BE[5705] MOV SI,NAME2 ; Raw dest
19677
19678 00002E8A B90B00 MOV CX,11
19679
19680 ; 17/05/2019 - Retro DOS v4.0
19681
19682 ; MSDOS 6.0
19683 00002E8D E82601 CALL NEW_RENAME ;IFS. replace ? chars ;AN000;
19684
19685 ; MSDOS 3.3
19686
19687 ; 08/08/2018 - Retro DOS v3.0
19688 ; MSDOS 6.0
19689 -----
19690 ;Procedure: NEW_RENAME
19691 ;
19692 ;Input: DS:SI -> raw string with ?
19693 ; ES:DI -> destination string
19694 ; DS:BX -> source string
19695 ;Function: replace ? chars of raw string with chars in source string and
19696 ; put in destination string
19697 ;Output: ES:DI-> new string
19698 -----
19699 ;
19700 ;NEW_RENAME:
19701 ;NEWNAM:

```

```

19702 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 341Ah
19703 ; LODSB
19704 ; CMP AL,"?"
19705 ; JNZ short NOCHG
19706 ; MOV AL,[BX] ; Get replace char
19707 ;NOCHG:
19708 ; STOSB
19709 ; INC BX ; Next replace char
19710 ; LOOP NEWNAM
19711 ; ; MSDOS 6.0
19712 ; ;retn
19713
19714 ; MSDOS 3.3 & MSDOS 6.0
19715 ;mov byte [ATTRIB],16h
19716 00002E90 C606[6B05]16 MOV byte [ATTRIB],attr_all; Stop duplicates with any attributes
19717 00002E95 C606[7E05]FF MOV byte [CREATING],0FFh
19718 00002E9A E8331F call DEVNAME ; Check if we built a device name
19719 00002E9D 7396 JNC short BAD_ACC
19720 ;;;
19721 ; 31/01/2024
19722 ; (PCDOS 7.1 IBMDOS.COM)
19723 00002E9F 8B1E[E60A] mov bx,[DESTSTART_HW]
19724 00002EA3 891E[EE0A] mov [ROOTCLUST_HW],bx
19725 ;;;
19726 00002EA7 8B1E[6405] MOV BX,[DESTSTART]
19727 00002EAB C42E[8A05] LES BP,[THISDPB]
19728 00002EAF E8A91A call SETDIRSRCH ; Reset search to start of dir
19729 00002EB2 7281 JC short BAD_ACC ; Screw up
19730 00002EB4 E88618 call FINDENTRY ; See if new name already exists
19731 ;JNC short BAD_ACC ; Error if found
19732 ; 31/01/2024
19733 00002EB7 7346 jnc short BAD_ACCJ
19734 00002EB9 803E[4A03]00 CMP byte [FAILERR],0
19735 00002EBE 753F JNZ short BAD_ACCJ ; Find failed because user FAILED to I 24
19736 00002EC0 A1[6405] MOV AX,[DESTSTART] ; DIRSTART of dest
19737 ;;;
19738 ; 31/01/2024
19739 00002EC3 8B16[E60A] mov dx,[DESTSTART_HW]
19740 00002EC7 C42E[8A05] les bp,[THISDPB]
19741 00002ECB 26837E0F00 cmp word [es:bp+DPB.FAT_SIZE],0
19742 ;cmp word [es:bp+0Fh],0
19743 00002ED0 7506 jnz short builddst_1 ; not FAT32
19744 00002ED2 3B16[3106] cmp dx,[RENAMEDMA+17] ; DIRSTART_HW of source
19745 00002ED6 7506 jne short builddst_2
19746 builddst_1:
19747 ;;;
19748 00002ED8 3B06[2F06] CMP AX,[RENAMEDMA+15] ; DIRSTART of source
19749 00002EDC 7451 JZ short SIMPLE_RENAME ; If =, just give new name
19750 builddst_2: ; 31/01/2024
19751 ;mov al,[RENAMEDMA+32]
19752 00002EDE A0[4006] MOV AL,[RENAMEDMA+21+dir_entry.dir_attr]
19753 00002EE1 A810 TEST AL,attr_directory ; 10h
19754 00002EE3 751A JNZ short BAD_ACCJ ; Can only do a simple rename on dirs,
19755 ; otherwise the . and .. entries get
19756 ; wiped.
19757 00002EE5 A2[6B05] MOV [ATTRIB],AL
19758 00002EE8 8C1E[A005] MOV [THISSFT+2],DS
19759
19760 ;hkn; AUXSTACK is in DOSDATA
19761 ;mov si,RENAMEDMA+145h
19762 00002EEC BE[6507] MOV SI,AUXSTACK-SF_ENTRY.size ; RENAMEDMA+325
19763 00002EEF 8936[9E05] MOV [THISSFT],SI
19764 ;mov word [SI+2],2
19765 00002EF3 C744020200 MOV word [SI+SF_ENTRY.sf_mode],SHARING_COMPAT+open_for_both
19766 00002EF8 31C9 XOR CX,CX ; Set "device ID" for call into makenode
19767 00002EFA E87E25 call RENAME_MAKE ; This is in mknode
19768 00002EFD 7303 JNC short GOT_DEST
19769 BAD_ACCJ:
19770 00002EFF E933FF JMP BAD_ACC
19771
19772 GOT_DEST:
19773 00002F02 53 push bx
19774 00002F03 C43E[9E05] LES DI,[THISSFT] ; RENAME_MAKE entered this into sharing
19775 00002F07 E87555 call ShareEnd ; we need to remove it.
19776 00002F0A 5B pop bx
19777
19778 ; A zero length entry with the correct new name has now been made at
19779 ; [CURBUF+2]:BX.
19780
19781 00002F0B C43E[E205] LES DI,[CURBUF]
19782
19783 ; 31/01/2024 - Retro DOS v5.0
19784 %if 0
19785 ; MSDOS 6.0
19786 ;test byte [es:di+5],40h
19787 TEST byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
19788 ;LB. if already dirty ;AN000;
19789 JNZ short yesdirty1 ;LB. don't increment dirty count ;AN000;
19790 call INC_DIRTY_COUNT ;LB. ;AN000;
19791 ;or byte [es:di+5],40h
19792 OR byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
19793 yesdirty1:
19794 %else
19795 ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19796 00002F0F E8A73C call SET_BUF_DIRTY
19797 %endif
19798 00002F12 89DF MOV DI,BX
19799 ;add di,11
19800 00002F14 83C70B ADD DI,dir_entry.dir_attr ; Skip name
19801
19802 ;hkn; RENAMEDMA is in DOSDATA
19803 ;mov si,RENAMEDMA+32 ; 05/07/2024
19804 00002F17 BE[4006] MOV SI,RENAMEDMA+21+dir_entry.dir_attr
19805 ;mov cx,21
19806 00002F1A B91500 MOV CX,dir_entry.size-dir_entry.dir_attr
19807 00002F1D F3A4 REP MOVSB
19808 00002F1F E86E00 CALL GET_SOURCE
19809 00002F22 7269 JC short RENAME_OVER
19810 00002F24 89DF MOV DI,BX
19811 00002F26 8E06[E405] MOV ES,[CURBUF+2]
19812 00002F2A B0E5 MOV AL,DIRFREE ; 0E5h
19813 00002F2C AA STOSB ; "free" the source
19814 00002F2D EB13 JMP SHORT DIRTY_IT
19815
19816 SIMPLE_RENAME:
19817 00002F2F E85E00 CALL GET_SOURCE ; Get the source back
19818 00002F32 7259 JC short RENAME_OVER
19819 00002F34 89DF MOV DI,BX
19820 00002F36 8E06[E405] MOV ES,[CURBUF+2]
19821
19822 ;hkn; NAME1 is in DOSDATA
19823 00002F3A BE[4B05] MOV SI,NAME1 ; New Name
19824 00002F3D B90B00 MOV CX,11
19825 00002F40 F3A4 REP MOVSB

```



```

19826 DIRTY_IT:
19827     MOV     DI,[CURBUF]
19828
19829 ; 31/01/2024 - Retro DOS v5.0
19830 %if 0
19831 ; MSDOS 6.0
19832     TEST    byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
19833 ;LB. if already dirty ;AN000;
19834     JNZ     short yesdirty2 ;LB. don't increment dirty count ;AN000;
19835     call    INC_DIRTY_COUNT ;LB. ;AN000;
19836
19837     OR      byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
19838 %else
19839 ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19840     call    SET_BUF_DIRTY
19841 %endif
19842
19843 ;-----
19844 ; Check if the source is a directory of file. If directory rename it to the
19845 ; the new name in the Fastopen cache buffer. If file name it has been
19846 ; previously deleted.
19847 ;-----
19848
19849 ;yesdirty2: ; 31/01/2024
19850 ; MSDOS 6.0
19851     PUSH    SI
19852     LDS     SI,[DMAADD] ;;BN00XPTM. chek if source a dir ;AN000;
19853     ;add     si,21
19854     ADD     SI,find_buf.attr ;;BN00XPTM.P5520 ;AN000;
19855     ;test    byte [si+08h],10h
19856     TEST    byte [SI+dir_entry.dir_attr],attr_directory ;;BN00XPTM. ;AN000;
19857     JZ      short NOT_DIR2 ;;BN00XPTM. ;AN000;
19858     call    FastOpen_Rename ;;BN00X rename dir entry in fastopen
19859     ; 31/01/2024
19860     ;POP     SI
19861     ;JMP     SHORT NOT_DIRTY1
19862 NOT_DIR2: ;;BN00X it is a file, delete the entry
19863     POP     SI
19864 NOT_DIRTY1: ;;BN00X
19865 NEXT_SOURCE:
19866 ;hkn; RENAMEDMA is in DOSDATA
19867     MOV     SI,RENAMEDMA+1 ;Name
19868
19869 ; WARNING! Rename_Next leaves the disk critical section *ALWAYS*. We need
19870 ; to enter it before going to RENAME_Next.
19871
19872     call    ECritDisk
19873     MOV     byte [CREATING],0 ; Correct setting for search (we changed it
19874 ; to FF when we made the prev new file).
19875     call    RENAME_NEXT
19876
19877 ; Note, now, that we have exited the previous ENTER and so are back to where
19878 ; we were before.
19879
19880     JC      short RENAME_OVER
19881
19882     ;lea     si,[bx+26]
19883     LEA     SI,[BX+dir_entry.dir_first]
19884     call    REN_DEL_Check
19885     JNC     short REN_OK2
19886     MOV     AX,error_sharing_violation ; 20h
19887 jmp_to_rename_clean: ; 28/12/2022
19888     JMP     RENAME_CLEAN ; 10/08/2018
19889
19890 ;-----
19891 ; Check if file or directory. If file, delete file from the Fastopen cache,
19892 ; if directory, rename directory name in the Fastopen cache.
19893 ;-----
19894
19895 REN_OK2:
19896 ; MSDOS 6.0
19897 ;mov     al,[RENAMEDMA+32]
19898     MOV     AL,[RENAMEDMA+21+dir_entry.dir_attr] ; PTR P5622
19899     ;test    al,10h
19900     TEST    AL,attr_directory ;;BN00X directory
19901     JZ      short Ren_Directory ;;BN00X no - file, delete it
19902
19903 ; MSDOS 3.3 & MSDOS 6.0
19904     call    FastOpen_Delete ;;BN00X delete dir info in fastopen DOS 3.3
19905 jmp_to_builddest: ; 28/12/2022
19906 ; 31/01/2024
19907     push    ss
19908     pop     es
19909     JMP     BUILDDDEST ;;BN00X
19910
19911 ; MSDOS 6.0
19912 Ren_Directory:
19913     call    FastOpen_Rename ;;BN00X delete dir info in fastopen DOS 3.3
19914     ;JMP     BUILDDDEST
19915     ; 28/12/2022
19916     jmp     short jmp_to_builddest
19917
19918 RENAME_OVER:
19919     CLC
19920     ;JMP     RENAME_CLEAN ; 10/08/2018
19921     ; 28/12/2022
19922     jmp     short jmp_to_rename_clean
19923
19924 ;-----
19925 ; Procedure: GET_SOURCE
19926 ;
19927 ; Inputs:
19928 ; RENAMEDMA has source info
19929 ; Function:
19930 ; Re-find the source
19931 ; Output:
19932 ; [CURBUF] set
19933 ; [CURBUF+2]:BX points to entry
19934 ; Carry set if error (currently user FAILED to I 24)
19935 ; DS preserved, others destroyed
19936 ;-----
19937
19938 GET_SOURCE:
19939 ;;;
19940 ; 01/02/2024 - Retro DOS v5.0
19941 ; (PCDOS 7.1 IBMDOS.COM)
19942     les     bp,[THISDPB]
19943     xor     bx,bx ; 0
19944     ;cmp     [es:bp+0Fh],bx
19945     cmp     [es:bp+DPB.FAT_SIZE],bx ; > 0 ?
19946     jnz     short gs_cont ; yes, it is not FAT32
19947     mov     bx,[RENAMEDMA+17] ; DirStart+2
19948 gs_cont:
19949     mov     [ROOTCLUST_HW],bx ; hw of cluster number

```

```

19950      ;;;
19951 00002FA4 8B1E[2F06]    MOV     BX,[RENAMEDMA+15]    ; DirStart
19952      ; 01/02/2024
19953      ;LES     BP,[THISDPB]
19954 00002FA8 E8B019      call    SETDIRSRCH
19955 00002FAB 7214        JC      short gs_ret_label    ; retc
19956 00002FAD E8FF1D      call    STARTSRCH
19957 00002FB0 A1[2D06]    MOV     AX,[RENAMEDMA+13]    ; Lastent
19958      ;call    GETENT
19959      ; 18/12/2022
19960 00002FB3 E9E818      jmp     GETENT
19961 ;gs_ret_label:
19962 ;retn
19963
19964 ; MSDOS 6.0
19965 -----
19966 ;Procedure: NEW_RENAME
19967 ;
19968 ;Input: DS:SI -> raw string with ?
19969 ;      ES:DI -> destination string
19970 ;      DS:BX -> source string
19971 ;Function: replace ? chars of raw string with chars in source string and
19972 ;      put in destination string
19973 ;Output: ES:DI-> new string
19974 -----
19975
19976 NEW_RENAME:
19977 ; 17/05/2019 - Retro DOS v4.0
19978 NEWNAM:
19979 ; DOSCODE:680Eh (MSDOS 6.21, MSDOS.SYS)
19980 00002FB6 AC          LODSB
19981 00002FB7 3C3F        CMP     AL,"?" ; 3Fh
19982 00002FB9 7502        JNZ     short NOCHG
19983 00002FBB 8A07        MOV     AL,[BX]          ; Get replace char
19984
19985 00002FBD AA          NOCHG: STOSB
19986 00002FBE 43          INC     BX          ; Next replace char
19987 00002FBF E2F5        LOOP    NEWNAM
19988      ; MSDOS 6.0
19989 gs_ret_label:      ; 18/12/2022
19990 00002FC1 C3          retn
19991
19992 -----
19993 ; FINFO.ASM, MSDOS 6.0, 1991
19994 -----
19995 ; 08/08/2018 - Retro DOS v3.0
19996 ; 17/05/2019 - Retro DOS v4.0
19997
19998 ;** Low level routines for returning file information and setting file
19999 ; attributes
20000 ;
20001 ; GET_FILE_INFO
20002 ; SET_FILE_ATTRIBUTE
20003 ;
20004 ; Modification history:
20005 ;
20006 ; Created: ARR 30 March 1983
20007 ;
20008 ; M025: Return access_denied if attempting to set
20009 ; attribute of root directory.
20010 ;
20011 ;
20012 ;SUBTTL GET_FILE_INFO -- Get File Information
20013
20014 -----
20015 ; Procedure Name : GET_FILE_INFO
20016 -----
20017 ; Inputs:
20018 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20019 ; terminated)
20020 ; [CURR_DIR_END] Points to end of Current dir part of string
20021 ; (= -1 if current dir not involved, else
20022 ; points to first char after last "/" of current dir part)
20023 ; [THISCDS] Points to CDS being used
20024 ; (Low word = -1 if NUL CDS (Net direct request))
20025 ; [SATTRIB] Is attribute of search, determines what files can be found
20026 ; Function:
20027 ; Get Information about a file
20028 ; Returns:
20029 ; CARRY CLEAR
20030 ; AX = Attribute of file
20031 ; CX = Time stamp of file
20032 ; DX = Date stamp of file
20033 ; BX:DI = Size of file (32 bit)
20034 ; CARRY SET
20035 ; AX is error code
20036 ; error_file_not_found
20037 ; Last element of path not found
20038 ; error_path_not_found
20039 ; Bad path (not in curr dir part if present)
20040 ; error_bad_curr_dir
20041 ; Bad path in current directory part of path
20042 ; DS preserved, others destroyed
20043 -----
20044
20045 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
20046
20047 GET_FILE_INFO:
20048
20049 ;hkn; get_file_info is called from file.asm and fcbio.asm. DS has been set
20050 ;hkn; to DOSDATA at this point. So DOSassume is OK.
20051
20052 00002FC2 E864E8      call    TestNet
20053 00002FC5 7306        JNC     short LOCAL_INFO
20054
20055 ;IF NOT Installed
20056 ; transfer NET_GET_FILE_INFO
20057 ;ELSE
20058 ; MOV     AX,(MultNET SHL 8) OR 15
20059 ; INT     2FH
20060 ; return
20061
20062 00002FC7 B80F11      mov     ax, 110Fh
20063 00002FCA CD2F        int     2Fh    ; Multiplex - NETWORK REDIRECTOR - GET REMOTE FILE'S ATTRIBUTES
20064      ; SS = DOS CS, SDA first filename pointer -> fully-qualified name of file
20065      ; SDA CDS pointer -> current directory
20066      ; Return: CF set on error, AX = file attributes
20067 00002FCC C3          retn
20068 ;ENDIF
20069
20070 LOCAL_INFO:
20071 00002FCD E812E9      call    ECritDisk
20072 00002FD0 C606[4C03]01 MOV     byte [NoSetDir],1    ; if we find a dir, don't change to it
20073      ; MSDOS 3.3

```

```

20074         ;call GETPATH
20075         ; MSDOS 6.0
20076 00002FD5 E8C900 call GET_FAST_PATH
20077         ; MSDOS 3.3 & MSDOS 6.0
20078 00002FD8 7312 JNC short info_check_dev
20079 NO_PATH:
20080         JNZ short bad_path1
20081         OR CL,CL
20082 00002FDE 7407 JZ short bad_path1
20083 info_no_file:
20084 00002FE0 B80200 MOV AX,error_file_not_found
20085 BadRet:
20086 00002FE3 F9 STC
20087 JustRet:
20088         ;call LCritDisk
20089         ;retn
20090         ; 18/12/2022
20091 00002FE4 E928E9 jmp LCritDisk
20092
20093 bad_path1:
20094 00002FE7 B80300 MOV AX,error_path_not_found
20095 00002FEA EBF7 jmp short BadRet
20096
20097 info_check_dev:
20098         OR AH,AH
20099 00002FEE 78F0 JS short info_no_file ; device
20100
20101         ; MSDOS 6.0
20102 ;SR;
20103 ; If root dir then CurBuf == -1. Check for this case and return subdir attr
20104 ;for a root dir
20105
20106 00002FF0 833E[E205]FF cmp word [CURBUF],-1 ;is it a root dir?
20107 00002FF5 7506 jne short not_root ;no, CurBuf ptr is valid
20108
20109         xor ah,ah
20110 00002FF9 B010 mov al,attr_directory ; 10h
20111         ;clc
20112         ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
20113         ; (DOSCODE:683Eh)
20114         ; 16/12/2022
20115         ;clc
20116 00002FFB EBE7 jmp short JustRet
20117
20118 not_root:
20119         ; MSDOS 3.3 (& MSDOS 6.0)
20120 00002FFD 1E PUSH DS
20121 00002FFE 8E1E[E405] MOV DS,[CURBUF+2]
20122 00003002 89DE MOV SI,BX
20123 00003004 31DB XOR BX,BX ; Assume size=0 (dir)
20124 00003006 89DF MOV DI,BX
20125         ;mov
20126 00003008 8B4C16 CX,[si+16h]
20127         MOV CX,[SI+dir_entry.dir_time]
20128         ;mov
20129 0000300B 8B5418 DX,[si+18h]
20130         MOV DX,[SI+dir_entry.dir_date]
20131 0000300E 30E4 XOR AH,AH
20132         ;mov
20133 00003010 8A440B al,[si+0Bh]
20134         MOV AL,[SI+dir_entry.dir_attr]
20135         ;test
20136 00003013 A810 TEST AL,10h
20137 00003015 7506 JNZ short NO_SIZE
20138         ;mov
20139 00003017 8B7C1C di,[si+1Ch]
20140         MOV DI,[SI+dir_entry.dir_size_l]
20141         ;mov
20142 0000301A 8B5C1E bx,[si+1Eh]
20143         MOV BX,[SI+dir_entry.dir_size_h]
20144 NO_SIZE:
20145         POP DS
20146         ;CLC
20147         ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
20148         ; (DOSCODE:6864h)
20149         ; 16/12/2022
20150         ;clc
20151 0000301E EBC4 jmp short JustRet
20152
20153 ;Break <SET_FILE_ATTRIBUTE -- Set File Attribute>
20154 -----
20155 ; Procedure Name : SET_FILE_ATTRIBUTE
20156 ; Inputs:
20157 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20158 ; terminated)
20159 ; [CURR_DIR_END] Points to end of Current dir part of string
20160 ; (= -1 if current dir not involved, else
20161 ; Points to first char after last "/" of current dir part)
20162 ; [THISCDS] Points to CDS being used
20163 ; (Low word = -1 if NUL CDS (Net direct request))
20164 ; [SATTRIB] is attribute of search (determines what files may be found)
20165 ; AX is new attributes to give to file
20166 ; Function:
20167 ; Set File Attributes
20168 ; Returns:
20169 ; CARRY CLEAR
20170 ; No error
20171 ; CARRY SET
20172 ; AX is error code
20173 ; error_file_not_found
20174 ; Last element of path not found
20175 ; error_path_not_found
20176 ; Bad path (not in curr dir part if present)
20177 ; error_bad_curr_dir
20178 ; Bad path in current directory part of path
20179 ; error_access_denied
20180 ; Attempt to set an attribute which cannot be set
20181 ; (attr_directory, attr_volume_ID)
20182 ; error_sharing_violation
20183 ; Sharing mode of file did not allow the change
20184 ; (this request requires exclusive write/read access)
20185 ; (INT 24H generated)
20186 ; DS preserved, others destroyed
20187 -----
20188 ; 01/02/2024 - Retro DOS v5.0
20189
20190 SET_FILE_ATTRIBUTE:
20191 ;hkn; set_file_attr is called from file.asm. DS has been set
20192 ;hkn; to DOSDATA at this point. So DOSassume is OK.
20193
20194 00003020 A9D8FF TEST AX,~attr_changeable ; 0FFD8h
20195 00003023 7412 JZ short set_look
20196
20197 _BAD_ACC:
20198         MOV byte [EXTERR_LOCUS],errLOC_Unk ; 1
20199         ;;;
20200         ; 01/02/2024 - Retro DOS v5.0
20201         ; (PCDOS 7.1 IBMDOS.COM)

```

```

20198 00003025 E8B5E2      call    set_exerr_locus_unk
20199                      ;;;
20200 00003028 C606[2703]07    MOV     byte [EXTERR_CLASS],errCLASS_Apperr ; 7
20201 0000302D C606[2603]04    MOV     byte [EXTERR_ACTION],errACT_Abort ; 4
20202 00003032 B80500      MOV     AX,error_access_denied ; 5
20203 00003035 F9          STC
20204 00003036 C3          retn
20205
20206 set_look:
20207 00003037 E8FEF7      call    TestNet
20208 0000303A 7308      JNC     short LOCAL_SET
20209
20210 ;IF NOT Installed
20211 ; transfer NET_SEQ_SET_FILE_ATTRIBUTE
20212 ;ELSE
20213 0000303C 50          PUSH     AX
20214
20215 ;MOV     AX,(MultNET SHL 8) OR 14
20216 ;INT     2FH
20217
20218 0000303D B80E11      mov     ax, 110Eh
20219 00003040 CD2F      int     2Fh ; Multiplex - NETWORK REDIRECTOR - SET REMOTE FILE'S ATTRIBUTES
20220                      ; SS = DOS CS, SDA first filename pointer -> fully-qualified name of file
20221                      ; SDA CDS pointer -> current directory
20222                      ; STACK: WORD new file attributes
20223                      ; Return: CF set on error
20224
20225 00003042 5B          POP      BX ; clean stack
20226 00003043 C3          retn
20227 ;ENDIF
20228
20229 LOCAL_SET:
20230 00003044 E89BE8      call    ECritDisk
20231 00003047 50          PUSH     AX ; Save new attributes
20232 00003048 C606[4C03]01    MOV     byte [NoSetDir],1 ; if we find a dir, don't change to it
20233 0000304D E8BF1A      call    GETPATH ; get path through fastopen if there ;AC000;
20234 00003050 7308      JNC     short set_check_device
20235 00003052 5B          POP      BX ; Clean stack (don't zap AX)
20236 00003053 EB85      JMP     short NO_PATH
20237
20238 ; MSDOS 6.0
20239 cannot_set_root: ; M025:
20240 00003055 B80500      mov     ax,error_access_denied; M025: return error is attempting
20241                      ; stc ; M025: to set attr. of root
20242                      ; jmp short OK_BYE ; M025:
20243                      ; 01/02/2024
20244 00003058 EB89      jmp     short BadRet
20245
20246 set_check_device:
20247 0000305A 08E4      OR      AH,AH
20248 0000305C 7906      JNS     short set_check_share
20249 0000305E 58          POP      AX
20250 0000305F E8ADE8      call    LCritDisk
20251 00003062 EBC1      JMP     short _BAD_ACC ; device
20252
20253 set_check_share:
20254 00003064 58          POP      AX ; Get new attributes
20255
20256 ; MSDOS 6.0
20257 00003065 833E[E205]FF    cmp     word [CURBUF], -1 ; M025: Q: is this the root dir
20258 0000306A 74E9      je      short cannot_set_root ; M025: Y: return error
20259
20260 ; MSDOS 3.3 & MSDOS 6.0
20261 0000306C E851FC      call    REN_DEL_Check
20262 0000306F 7305      JNC     short set_do
20263 00003071 B82000      MOV     AX,error_sharing_violation ; 32
20264 00003074 EB28      jmp     short OK_BYE
20265
20266 set_do:
20267 ; MSDOS 3.3 & MSDOS 6.0
20268 00003076 C43E[E205]    LES     DI,[CURBUF]
20269 0000307A 2680670BD8    AND     BYTE [ES:BX+dir_entry.dir_attr],~attr_changeable ; 0D8h
20270 0000307F 2608470B      OR      BYTE [ES:BX+dir_entry.dir_attr],AL
20271
20272 ; 01/02/2024 - Retro DOS v5.0
20273 %if 0
20274 ; MSDOS 6.0
20275 TEST     byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
20276                      ;LB. if already dirty ;AN000;
20277 JNZ      short yesdirty3 ;LB. don't increment dirty count ;AN000;
20278 call     INC_DIRTY_COUNT ;LB. ;AN000;
20279
20280 OR      byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
20281 yesdirty3:
20282 %else
20283 ; 01/02/2024
20284 ; (PCDOS 7.1 IBMDOS.COM)
20285 00003083 E8333B      call    SET_BUF_DIRTY
20286 %endif
20287 00003086 A0[7605]    MOV     AL,[THISDRV]
20288 ;;; 10/1/86 F.C update fastopen cache
20289 00003089 52          PUSH     DX
20290 0000308A 57          PUSH     DI
20291 0000308B B400      MOV     AH,0 ; dir entry update
20292 0000308D 88C2      MOV     DL,AL ; drive number A=0,B=1,,
20293 0000308F 89DF      MOV     DI,BX ; ES:DI -> dir entry
20294 00003091 E8D2FC      call    FastOpen_Update
20295 00003094 5F          POP      DI
20296 00003095 5A          POP      DX
20297 ;;; 9/11/86 F.C update fastopen cache
20298 00003096 E8FA39      call    FLUSHBUF
20299 00003099 7303      JNC     short OK_BYE
20300 0000309B B80200      MOV     AX,error_file_not_found
20301 OK_BYE:
20302 ;call    LCritDisk
20303 ;retn
20304 ; 16/12/2022
20305 0000309E E96EE8      jmp     LCritDisk
20306
20307 ;-----
20308
20309 ; 17/05/2019 - Retro DOS v4.0
20310
20311 ; MSDOS 6.0
20312 GET_FAST_PATH:
20313 ;hkn; use SS override for FastOpenFlg
20314 000030A1 36800E[4611]01    OR      byte [ss:FastOpenFlg],FastOpen_Set
20315                      ;FO. trigger fastopen ;AN000;
20316 000030A7 E8651A      call    GETPATH
20317 000030AA 9C          PUSHF ;FO. ;AN000;
20318 000030AB 368026[4611]80    AND     byte [ss:FastOpenFlg],Fast_yes
20319                      ;FO. clear all fastopen flags ;AN000;
20320 000030B1 9D          POPF ;FO. ;AN000;
20321 000030B2 C3          retn

```

```

20322
20323
20324
20325
20326
20327
20328
20329
20330
20331
20332
20333
20334
20335
20336
20337
20338
20339
20340
20341
20342
20343
20344
20345
20346
20347
20348
20349
20350
20351
20352
20353
20354
20355
20356
20357
20358
20359
20360
20361
20362
20363 000030B3 2E8E06[0700]
20364 000030B8 26C43E[9E05]
20365
20366
20367
20368 000030BD E882E7
20369 000030C0 7503
20370 000030C2 E84B21
20371
20372
20373 000030C5 26FF05
20374
20375 000030C8 C3
20376
20377
20378
20379
20380
20381
20382
20383
20384
20385
20386
20387
20388
20389
20390
20391
20392
20393
20394
20395
20396
20397
20398
20399
20400
20401
20402
20403
20404
20405
20406
20407
20408
20409
20410
20411
20412
20413
20414
20415
20416
20417
20418
20419
20420
20421
20422
20423
20424
20425
20426
20427
20428
20429
20430
20431
20432
20433
20434
20435
20436
20437
20438
20439
20440
20441
20442
20443
20444
20445

```

```

=====
; DUP.ASM, MSDOS 6.0, 1991
=====
; 08/08/2018 - Retro DOS v3.0
; 17/05/2019 - Retro DOS v4.0

; ** Low level DUP routine for use by EXEC when creating a new process.
; Exports the DUP to the server machine and increments the SFT ref count
;
; DOS_DUP
;
; Modification history:
;
; Created: ARR 30 March 1983

;BREAK <DOS_DUP -- DUP SFT across network>
-----
; Procedure Name : DOS_DUP
;
; Inputs:
; [THISSFT] set to the SFT for the file being DUPed
; (a non net SFT is OK, in this case the ref
; count is simply incremented)
; Function:
; Signal to the devices that a logical open is occurring
; Returns:
; ES:DI point to SFT
; Carry clear
; SFT ref_count is incremented
; Registers modified: None.
; NOTE:
; This routine is called from $CREATE_PROCESS_DATA_BLOCK at DOSINIT
; time with SS NOT DOSGROUP. There will be no Network handles at
; that time.
-----

DOS_DUP:
;LES DI,[CS:THISSFT] ; MSDOS 3.3

; MSDOS 6.0
mov es,[cs:DosDSeg]
les di,[es:THISSFT]

;Entry Dos_Dup_Direct
DOS_Dup_Direct:
call IsSFTNet
JNZ short DO_INC
call DEV_OPEN_SFT
DO_INC:
;INC word [ES:DI+SF_ENTRY.sf_ref_count]
inc word [ES:DI] ; Clears carry (if this ever wraps
; we're in big trouble anyway)

ret

=====
; CREATE.ASM, MSDOS 6.0, 1991
=====
; 08/08/2018 - Retro DOS v3.0
; 18/05/2019 - Retro DOS v4.0

;TITLE DOS_CREATE/DOS_CREATE_NEW - Internal CREATE calls for MS-DOS
;NAME DOS_CREATE
-----
; ** Internal Create and Create new to create a local or NET file and SFT.
;
; DOS_CREATE
; DOS_CREATE_NEW
; SET_MKND_ERR
; SET_Media_ID
; SET_EXT_Mode
;
; Revision history:
;
; A000 version 4.00 Jan. 1988
; A001 D490 -- Change IOCTL subfunctios from 63h,43h to 66h, 46h

;Installed = TRUE

; i_need THISSFT,DWORD
; i_need THISCDS,DWORD
; I_need EXTERR,WORD
; I_Need ExtErr_locus,BYTE
; I_need JShare,DWORD
; I_need VOLCHNG_FLAG,BYTE
; I_need SATTRIB,BYTE
; I_need CALLVIDM,DWORD
; I_need EXTOPEN_ON,BYTE ;AN000; extended open
; I_need NAME1,BYTE ;AN000;
; I_need NO_NAME_ID,BYTE ;AN000;
; I_need Packet_Temp,WORD ;AN000;
; I_need DOS34_FLAG,WORD ;AN000;
; I_need SAVE_BX,WORD ;AN000;

;***DOS_CREATE - Create a File
-----
; DOS_Create is called to create the specified file, truncating
; the old one if it exists.
;
; ENTRY AX is Attribute to create
; (ds) = DOSDATA
; [WFP_START] Points to WFP string ("d:" must be first 3 chars, NUL
; terminated)
; [CURR_DIR_END] Points to end of Current dir part of string
; (= -1 if current dir not involved, else
; Points to first char after last "/" of current dir part)
; [THISCDS] Points to CDS being used
; (Low word = -1 if NUL CDS (Net direct request))
; [THISSFT] Points to SFT to fill in if file created
; (sf_mode field set so that FCB may be detected)
; [SATTRIB] Is attribute of search, determines what files can be found
;
; EXIT sf_ref_count is NOT altered
; CARRY CLEAR
; THISSFT filled in.
; sf_mode = unchanged for FCB, sharing_compat + open_for_both
; CARRY SET
; AX is error code
; error_path_not_found
; Bad path (not in curr dir part if present)
; error_bad_curr_dir
; Bad path in current directory part of path
; error_access_denied
; Attempt to re-create read only file , or

```

```

20446 ; create a second volume id or create a dir
20447 ; error_sharing_violation
20448 ; The sharing mode was correct but not allowed
20449 ; generates an INT 24
20450 ; USES all but DS
20451 ;-----
20452 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
20453 ; DOSCODE:6920h (MSDOS 5.0, MSDOS.SYS)
20454 ;
20455 ; 01/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
20456 ; DOSCODE:708Ah (PCDOS 7.1, IBMDOS.COM)
20457 ;
20458 DOS_CREATE:
20459 ; 18/05/2019 - Retro DOS v4.0
20460 ; DOSCODE:6934h (MSDOS 6.21, MSDOS.SYS)
20461 ;
20462 ;hkn; dispatched to from file.asm and fcbio.asm. DS set up to DOSDATA at
20463 ;hkn; this point.
20464
20465 XOR AH,AH ; Truncate is OK
20466
20467 ; Enter here from Dos_Create_New
20468 ;
20469 ;
20470 ; (ah) = 0 iff truncate OK
20471
20472 Create_inter:
20473 TEST AL, ~(attr_all+attr_ignore+attr_volume_id) ; 80h
20474 ; Mask out any meaningless bits
20475 JNZ short AttErr
20476 TEST AL, attr_volume_id
20477 JZ short NoReset
20478
20479 ; MSDOS 6.0
20480 ; 16/12/2022
20481 OR byte [DOS34_FLAG], DBCS_VOLID ; 80h ; AN000; FOR dbcs volid
20482 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
20483 ; or word [DOS34_FLAG], DBCS_VOLID ; 80h
20484
20485 MOV AL, attr_volume_id ; 8
20486 NoReset:
20487 OR AL, attr_archive ; File changed ; 20h
20488 TEST AL, attr_directory+attr_device ; 50h
20489 JZ short ATT_OK
20490 AttErr:
20491 MOV AX, 5 ; Attribute problem
20492 ; MOV byte [EXTERR_LOCUS], errLOC_Unk ; 1
20493 ; 01/02/2024
20494 call set_exerr_locus_unk
20495 JMP SHORT SET_MKND_ERR ; Gotta use MKDIR to make dirs, NEVER allow
20496 ; attr_device to be set.
20497 ATT_OK:
20498 LES DI, [THISFT]
20499 PUSH ES
20500 LES SI, [THISCDS]
20501 CMP SI, -1
20502 JNE short TEST_RE_NET
20503
20504 ; No CDS, it must be redirected.
20505
20506 POP ES
20507
20508 ; MSDOS 6.0
20509 ; Extended open hooks
20510 ; test byte [EXTOPEN_ON], 1
20511 TEST byte [EXTOPEN_ON], EXT_OPEN_ON ; AN000; EO. from extended open
20512 JZ short NOEXTOP ; AN000; EO. no, do normal
20513 IFS_extopen:
20514 PUSH AX ; AN000; EO.
20515 MOV AX, (MULTNET SHL 8) OR 46 ; AN000; EO. pass create attr
20516 mov ax, 112Eh ; AN000; EO. issue extended open verb
20517 NOEXTOP2: ; 01/02/2024 (PCDOS 7.1 IBMDOS.COM)
20518 INT 2FH ; AN000; EO.
20519 POP BX ; AN000; EO. trash bx
20520 MOV byte [EXTOPEN_ON], 0 ; AN000; EO.
20521 retn ; AN000; EO.
20522 NOEXTOP: ; AN000;
20523 ; Extended open hooks
20524
20525 ; IF NOT Installed
20526 ; transfer NET_SEQ_CREATE
20527 ; ELSE
20528 PUSH AX
20529
20530 ; MOV AX, (MULTNET SHL 8) OR 24
20531 ; INT 2FH
20532
20533 mov ax, 1118h
20534 ; 01/02/2024
20535 ; int 2Fh ; Multiplex - NETWORK REDIRECTOR - CREATE/TRUNCATE FILE
20536 ; ES:DI -> uninitialized SFT, SS = DOS CS
20537 ; SDA first filename pointer -> fully-qualified name of file
20538 ; STACK: WORD file creation mode???
20539
20540 ; POP BX ; BX is trashed anyway
20541 ; retn
20542 jmp short NOEXTOP2 ; 01/02/2024
20543 ; ENDIF
20544
20545 ; We have a CDS. See if it's network
20546
20547 TEST_RE_NET:
20548 ; test word [es:si+43h], 8000h
20549 TEST word [ES:SI+curdir.flags], curdir_isnet
20550 ; 07/12/2022
20551 ; test byte [es:si+44h], 80h
20552 ; 17/12/2022
20553 test byte [ES:SI+curdir.flags+1], curdir_isnet >> 8
20554 POP ES
20555 JZ short LOCAL_CREATE
20556
20557 ; MSDOS 6.0
20558 CALL Set_EXT_mode ; AN000; EO.
20559 JC SHORT dochk ; AN000; EO.
20560 ; or word [es:di+2], 2
20561 ; OR word [ES:DI+SF_ENTRY.sf_mode], SHARING_COMPAT+open_for_both ; IFS.
20562 ; 17/12/2022
20563 or byte [ES:DI+SF_ENTRY.sf_mode], SHARING_COMPAT+open_for_both ; IFS.
20564
20565 ; Extended open hooks
20566 dochk:
20567 TEST byte [EXTOPEN_ON], EXT_OPEN_ON ; AN000; EO. from extended open
20568 JNZ short IFS_extopen ; AN000; EO. yes, issue extended open
20569 ; Extended open hooks

```



```

20570
20571 ;IF NOT Installed
20572 ; transfer NET_CREATE
20573 ;ELSE
20574 0000312A 50      PUSH    AX
20575
20576 ;MOV    AX,(MULTNET SHL 8) OR 23
20577 ;INT    2FH
20578
20579 0000312B B81711  mov     ax,1117h
20580
20581 ; 01/02/2024
20582 ;int     2Fh ; Multiplex - NETWORK REDIRECTOR - CREATE/TRUNCATE REMOTE FILE
20583 ; ES:DI -> uninitialized SFT, SS = DOS CS
20584 ; SDA first filename pointer -> fully-qualified name of file to open
20585 ; SDA CDS pointer -> current directory
20586 ; Return: CF set on error
20587
20588 ;POP     BX ; BX is trashed anyway
20589 ;nomore:
20590 ;retn
20591 0000312E EBD2     jmp     short NOEXTOP2 ; 01/02/2024
20592 ;ENDIF
20593
20594 ;** It's a local create. We have a local CDS for it.
20595
20596 LOCAL_CREATE:
20597 ; MSDOS 6.0
20598 00003130 E8B300   CALL    Set_EXT_mode ;AN000;EO. set mode if from extended open
20599 00003133 7205     JC      short setdone ;AN000;EO.
20600
20601 ; MSDOS 3.3 & MSDOS 6.0
20602 ; 17/12/2022
20603 ;or      word [es:di+2],2
20604 ;OR      word [ES:DI+SF_ENTRY.sf_mode],SHARING_COMPAT+open_for_both
20605 ;or      byte [es:di+2],2
20606 00003135 26804D0202 or     byte [ES:DI+SF_ENTRY.sf_mode],SHARING_COMPAT+open_for_both
20607
20608 0000313A E8A5E7   call    ECritDisk
20609 0000313D E81723   call    MakeNode
20610 00003140 7317     JNC     short Create_ok
20611 00003142 C606[A10A]FF mov     byte [VOLCHNG_FLAG],-1; indicate no change in volume label
20612 00003147 E8C5E7   call    LCritDisk
20613
20614 ;entry SET_MKND_ERR
20615 SET_MKND_ERR:
20616
20617 ; Looks up MakeNode errors and converts them. AL is MakeNode
20618 ; error, SI is GETPATH bad spot return if path_not_found error.
20619
20620 ;hkn; CRTERRTAB is in TABLE seg (DOSCODE)
20621 0000314A BB[5131] MOV     BX,CRTERRTAB
20622 ;XLAT ; MSDOS 3.3
20623 ; 18/05/2019 - Retro DOS v4.0
20624 0000314D 2E      CS
20625 0000314E D7      XLAT
20626
20627 0000314F F9      CreatBadRet:
20628 00003150 C3      STC
20629 ;retn
20630
20631 ; 13/05/2019 - Retro DOS v4.0
20632 ; DOSCODE:69C4h (MSDOS 6.21, MSDOS.SYS)
20633 ; -----
20634
20635 ;** Internal Create and Create new to create a local or NET file and SFT.
20636
20637 ; 17/07/2018 - Retro DOS v3.0
20638 ; Offset 12B1h of IBMDOS.COM (MSDOS 3.3), 1987
20639
20640 ;CRTERRTAB: ; 19/07/2018 - MSDOS 3.3
20641 ; db      0,5,52h,50h,3,5,20h
20642
20643 ;CRTERRTAB: ; 18/05/2019 - MSDOS 6.0
20644 ; db      0,5,52h,50h,3,5,20h,2
20645
20646 ; 08/08/2018
20647
20648 CRTERRTAB: ;LABEL BYTE ; Lookup table for MakeNode returns
20649 DB 0 ; none
20650 DB error_access_denied ; MakeNode error 1
20651 DB error_cannot_make ; MakeNode error 2
20652 DB error_file_exists ; MakeNode error 3
20653 DB error_path_not_found ; MakeNode error 4
20654 DB error_access_denied ; MakeNode error 5
20655 DB error_sharing_violation ; MakeNode error 6
20656 ; MSDOS 6.0
20657 DB error_file_not_found ; MakeNode error 7
20658
20659 ; -----
20660
20661 ; we have just created a new file. This results in the truncation of old
20662 ; files. we must inform the sharer to slash all the open SFT's for this
20663 ; file to the current size.
20664
20665 ; If we created a volume id on the diskette, set the VOLCHNG_FLAG to logical
20666 ; drive number to force a Build BPB after Media check.
20667
20668 ;;; FASTOPEN 8/29/86
20669 Create_ok:
20670 call FastOpen_Delete
20671 ;;; FASTOPEN 8/29/86
20672 mov al,[SATTRIB]
20673 test al,attr_volume_id
20674 jz short NoVolLabel
20675 LES DI,[THISCDS]
20676 ;mov ah,[ES:DI+curdir.text]; get drive letter
20677 mov ah,[ES:DI] ; 09/08/2018
20678 sub ah,'A' ; 41h ; convert to drive number
20679 mov [VOLCHNG_FLAG],ah ;set flag to indicate valid change
20680
20681 ; 18/05/2019 - Retro DOS v4.0
20682
20683 ; MSDOS 6.0
20684 00003171 B701   MOV     BH,1 ;AN000;>32mb set volume id to boot record
20685 00003173 E81F00   CALL    Set_Media_ID ;AN000;>32mb
20686
20687 00003176 E869E7   call    ECritDisk
20688 00003179 E8DA33   call    FATREAD_CDS ; force a media check
20689 0000317C E890E7   call    LCritDisk
20690
20691 NoVolLabel:
20692 MOV     ax,2
20693 LES     DI,[THISSFT]
20694 ;if installed

```

```

20694      ;call  JShare + 14 * 4
20695 00003186 FF1E[C800]      call  far [JShare+(14*4)] ; 14 = shsu
20696      ;else
20697      ;call  shsu
20698      ;endif
20699 0000318A E882E7      call  LCritDisk
20700 0000318D E95601      jmp    SET_SFT_MODE
20701
20702      ;-----
20703      ; Procedure Name : Dos_Create_New
20704      ;
20705      ; Inputs:
20706      ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20707      ; terminated)
20708      ; [CURR_DIR_END] Points to end of Current dir part of string
20709      ; (= -1 if current dir not involved, else
20710      ; points to first char after last "/" of current dir part)
20711      ; [THISCDs] Points to CDS being used
20712      ; (Low word = -1 if NUL CDS (Net direct request))
20713      ; [THISSFT] Points to SFT to fill in if file created
20714      ; (sf_mode field set so that FCB may be detected)
20715      ; [SATTRIB] Is attribute of search, determines what files can be found
20716      ; AX is Attribute to create
20717      ; Function:
20718      ; Try to create the specified file truncating an old one that exists
20719      ; Outputs:
20720      ; sf_ref_count is NOT altered
20721      ; CARRY CLEAR
20722      ; THISSFT filled in.
20723      ; sf_mode = sharing_compat + open_for_both for Non-FCB SFT
20724      ; CARRY SET
20725      ; AX is error code
20726      ; error_path_not_found
20727      ; Bad path (not in curr dir part if present)
20728      ; error_bad_curr_dir
20729      ; Bad path in current directory part of path
20730      ; error_access_denied
20731      ; Create a second volume id or create a dir
20732      ; error_file_exists
20733      ; Already a file by this name
20734      ; DS preserved, others destroyed
20735      ;-----
20736
20737      DOS_Create_New:
20738 00003190 B401      MOV     AH,1      ; Truncate is NOT OK
20739 00003192 E936FF      JMP     Create_inter
20740
20741      ; MSDOS 6.0
20742      ;-----
20743      ; Procedure Name : Set_Media_ID
20744      ;
20745      ; Inputs:
20746      ; NAME1= Volume ID
20747      ; BH= 0, delete volume id
20748      ; 1, set new volume id
20749      ; DS= DOSGROUP
20750      ; Function:
20751      ; Set Volume ID to DOS 4.00 Boot record.
20752      ; Outputs:
20753      ; CARRY CLEAR
20754      ; volume id set
20755      ; CARRY SET
20756      ; AX is error code
20757      ;-----
20758
20759      ; 18/05/2019 - Retro DOS v4.0
20760      ; 01/02/2024 - Retro DOS v5.0
20761
20762 00003195 50      Set_Media_ID:
20763 00003196 06      PUSH    AX      ;AN000;>32mb
20764 00003197 57      PUSH    ES      ;AN000;>32mb
20765      PUSH    DI      ;AN000;>32mb
20766      INC     AH      ;AN000;>32mb bl=drive #
20767 0000319A 88E3      MOV     BL,AH      ;AN000;>32mb bl=drive # (A=1,B=2,,)
20768 0000319C B00D      MOV     AL,0DH      ;AN000;>32mb generic IOCTL
20769      MOV     CX,0866H ;AN001;>32mb get media id
20770      ; 01/02/2024
20771      ; (PCDOS 7.1 IBMDOS.COM)
20772 0000319E B96648      mov     cx,4866h      ; ch = FAT32 disk drive (CATEGORY CODE)
20773      ; cl = minor code,
20774      ; get volume serial number (and name)
20775
20776      ;hkn; PACKET_TEMP is in DOSDATA
20777 000031A1 BA[A00D]      MOV     DX,Packet_Temp ;AN000;>32mb
20778
20779      Set_Media_ID_1:
20780      ; 01/02/2024
20781 000031A4 51      push    cx
20782
20783 000031A5 53      PUSH    BX      ;AN000;>32mb
20784 000031A6 52      PUSH    DX      ;AN000;>32mb
20785 000031A7 30FF      XOR     BH,BH      ;AN000;>32mb
20786
20787      ;invoke $IOCTL      ;AN000;>32mb
20788 000031A9 E8D7F6      call    _$IOCTL
20789
20790 000031AC 5A      POP     DX      ;AN000;>32mb
20791 000031AD 5B      POP     BX      ;AN000;>32mb
20792
20793      ; 01/02/2024
20794 000031AE 59      pop     cx
20795      ;JC short geterr ;AN000;>32mb
20796 000031AF 730A      jnc     short Set_Media_ID_2
20797 000031B1 80FD48      cmp     ch,48h      ; is it FAT32 disk drive request?
20798 000031B4 F9      stc
20799 000031B5 7529      jne     short geterr ; (ch=8 request failed!)
20800 000031B7 B508      mov     ch,8      ; set category code for (old) FAT disk drive
20801      ; (except FAT32)
20802 000031B9 EBE9      jmp     short Set_Media_ID_1 ; and try again
20803
20804      Set_Media_ID_2:
20805 000031BB 08FF      OR     BH,BH      ;AN000;>32mb delete volume id
20806 000031BD 7405      JZ     short NoName ;AN000;>32mb yes
20807
20808      ;hkn; NAME1 is in DOSDATA
20809 000031BF BE[4B05]      MOV     SI,NAME1      ;AN000;>32mb
20810
20811 000031C2 EB03      JMP     SHORT doset      ;AN000;>32mb yes
20812      NoName:
20813      ;AN000;
20814
20815      ;hkn; NO_NAME_ID is in DOSDATA
20816 000031C4 BE[C10D]      MOV     SI,NO_NAME_ID ;AN000;>32mb
20817
20818      doset:
20819      ;AN000;

```



```

20818 000031C7 89D7      MOV     DI,DX          ;AN000;;>32mb
20819      ;add      di,6
20820 000031C9 83C706    ADD     DI,MEDIA_ID_INFO.MEDIA_Label ;AN000;;>32mb
20821
20822      ;hkn; ES & DS must point to SS
20823      ;hkn; PUSH    CS          ;AN000;;>32mb move new volume id to packet
20824 000031CC 16      PUSH    SS          ;AN000;;>32mb move new volume id to packet
20825
20826 000031CD 1F      POP     DS          ;AN000;;>32mb
20827
20828      ;hkn; PUSH    CS          ;AN000;;>32mb
20829 000031CE 16      PUSH    SS          ;AN000;;>32mb
20830
20831 000031CF 07      POP     ES          ;AN000;;>32mb
20832
20833      ; 01/02/2024
20834 000031D0 51      push    cx
20835
20836 000031D1 B90B00    MOV     CX,11          ;AN000;;>32mb
20837 000031D4 F3A4      REP     MOVSB          ;AN000;;>32mb
20838
20839      ;MOV     CX,0846H      ;AN001;;>32mb
20840      ; 01/02/2024
20841 000031D6 59      pop     cx
20842 000031D7 B146      mov     cl,46h          ; set volume serial number (and name)
20843
20844 000031D9 B00D      MOV     AL,0DH          ;AN000;;>32mb
20845 000031DB 30FF      XOR     BH,BH          ;AN000;;>32mb
20846      ;invoke $IOCTL      ;AN000;;>32mb set volume id
20847 000031DD E8A3F6    call    _$IOCTL
20848
20849      geterr:
20850 000031E0 16      ;hkn; PUSH    CS          ;AN000;;>32mb
20851      PUSH    SS          ;AN000;;>32mb
20852 000031E1 1F      POP     DS          ;AN000;;>32mb ds= dosgroup
20853
20854 000031E2 5F      POP     DI          ;AN000;;>32mb
20855 000031E3 07      POP     ES          ;AN000;;>32mb
20856 000031E4 58      POP     AX          ;AN000;;>32mb
20857 000031E5 C3      retn          ;AN000;;>32mb
20858
20859      ; MSDOS 6.0
20860
20861      ;-----
20862      ; Procedure Name : Set_EXT_mode
20863      ;
20864      ; Inputs:
20865      ; [EXTOPEN_ON]= flag for extended open
20866      ; SAVE_BX= mode specified in Extended Open
20867      ; Function:
20868      ; Set mode in ThisSFT
20869      ; Outputs:
20870      ; carry set,mode is set if from Extended Open
20871      ; carry clear, mode not set yet
20872      ;-----
20873
20874      ; 13/05/2019 - Retro DOS v4.0
20875
20876      Set_EXT_mode:
20877
20878      ;hkn; SS override
20879 000031E6 36F606[F605]01  TEST    byte [ss:EXTOPEN_ON],EXT_OPEN_ON ;AN000;EO. from extended open
20880 000031EC 740B      JZ      short NOTEX      ;AN000;EO. no, do normal
20881 000031EE 50      PUSH    AX          ;AN000;EO.
20882
20883      ;hkn; SS override
20884 000031EF 36A1[0106]  MOV     AX,[ss:SAVE_BX]          ;AN000;EO.
20885      ;or      [es:di+2],ax
20886 000031F3 26094502  OR     [ES:DI+SF_ENTRY.sf_mode],AX ;AN000;EO.
20887 000031F7 58      POP     AX          ;AN000;EO.
20888 000031F8 F9      STC          ;AN000;EO.
20889 000031F9 C3      NOTEX: retn          ;AN000;EO.
20890
20891      ;=====
20892      ; OPEN.ASM, MSDOS 6.0, 1991
20893      ;=====
20894      ; 08/08/2018 - Retro DOS v3.0
20895      ; 18/05/2019 - Retro DOS v4.0
20896
20897      ; TITLE DOS_OPEN - Internal OPEN call for MS-DOS
20898      ; NAME DOS_OPEN
20899
20900      ;** OPEN.ASM - File Open
20901      ;-----
20902      ; Low level routines for opening a file from a file spec.
20903      ; Also misc routines for sharing errors
20904      ;
20905      ; DOS_Open
20906      ; Check_Access_AX
20907      ; SHARE_ERROR
20908      ; SET_SFT_MODE
20909      ; Code_Page_Mismatched_Error ; DOS 4.00
20910
20911      ; Revision history:
20912      ;
20913      ; Created: ARR 30 March 1983
20914      ; A000 version 4.00 Jan. 1988
20915      ;
20916      ; M034 - The value in save_bx must be pushed on to the stack for
20917      ; remote extended opens and not save_cx.
20918      ;
20919      ; M035 - if open made from exec then we must set the appropriate bits
20920      ; on the stack before calling off to the redir.
20921      ; M042 - Bit 11 of DOS34_FLAG set indicates that the redir knows how
20922      ; to handle open from exec. In this case set the appropriate bit
20923      ; else do not.
20924      ;-----
20925
20926      ;Installed = TRUE
20927
20928      ; i_need NoSetDir,BYTE
20929      ; i_need THISSFT,DWORD
20930      ; i_need THISCDS,DWORD
20931      ; i_need CURBUF,DWORD
20932      ; i_need CurrentPDB,WORD
20933      ; i_need CURR_DIR_END,WORD
20934      ; I_need RetryCount,WORD
20935      ; I_need Open_Access,BYTE
20936      ; I_need fSharing,BYTE
20937      ; i_need JShare,DWORD
20938      ; I_need FastOpenFlg,byte
20939      ; I_need EXTOPEN_ON,BYTE ;AN000;; DOS 4.00
20940      ; I_need ALLOWED,BYTE ;AN000;; DOS 4.00
20941      ; I_need EXTERR,WORD ;AN000;; DOS 4.00

```

```

20942 ; I_need EXTERR_LOCUS,BYTE ;AN000;; DOS 4.00
20943 ; I_need EXTERR_ACTION,BYTE ;AN000;; DOS 4.00
20944 ; I_need EXTERR_CLASS,BYTE ;AN000;; DOS 4.00
20945 ; I_need CPSWFLAG,BYTE ;AN000;; DOS 4.00
20946 ; I_need EXITHOLD,DWORD ;AN000;; DOS 4.00
20947 ; I_need THISDPB,DWORD ;AN000;; DOS 4.00
20948 ; I_need SAVE_CX,WORD ;AN000;; DOS 4.00
20949 ; I_need SAVE_BX,WORD ;M034
20950 ;
20951 ; I_need DOS_FLAG,BYTE
20952 ; I_need DOS34_FLAG,WORD ;M042
20953 ;
20954 ;Break <DOS_Open - internal file access>
20955 ;-----
20956 ; Procedure Name : DOS_Open
20957 ;
20958 ; Inputs:
20959 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20960 ; terminated)
20961 ; [CURR_DIR_END] Points to end of Current dir part of string
20962 ; (= -1 if current dir not involved, else
20963 ; Points to first char after last "/" of current dir part)
20964 ; [THISCDS] Points to CDS being used
20965 ; (Low word = -1 if NUL CDS (Net direct request))
20966 ; [THISSFT] Points to SFT to fill in if file found
20967 ; (sf_mode field set so that FCB may be detected)
20968 ; [SATTRIB] Is attribute of search, determines what files can be found
20969 ; AX is Access and Sharing mode
20970 ; High NIBBLE of AL (Sharing Mode)
20971 ; sharing_compat file is opened in compatibility mode
20972 ; sharing_deny_none file is opened Multi reader, Multi writer
20973 ; sharing_deny_read file is opened Only reader, Multi writer
20974 ; sharing_deny_write file is opened Multi reader, Only writer
20975 ; sharing_deny_both file is opened Only reader, Only writer
20976 ; Low NIBBLE of AL (Access Mode)
20977 ; open_for_read file is opened for reading
20978 ; open_for_write file is opened for writing
20979 ; open_for_both file is opened for both reading and writing.
20980 ;
20981 ; For FCB SFTs AL should = sharing_compat + open_for_both
20982 ; (not checked)
20983 ; Function:
20984 ; Try to open the specified file
20985 ; Outputs:
20986 ; sf_ref_count is NOT altered
20987 ; CARRY CLEAR
20988 ; THISSFT filled in.
20989 ; CARRY SET
20990 ; AX is error code
20991 ; error_file_not_found
20992 ; Last element of path not found
20993 ; error_path_not_found
20994 ; Bad path (not in curr dir part if present)
20995 ; error_bad_curr_dir
20996 ; Bad path in current directory part of path
20997 ; error_invalid_access
20998 ; Bad sharing mode or bad access mode or bad combination
20999 ; error_access_denied
21000 ; Attempt to open read only file for writting, or
21001 ; open a directory
21002 ; error_sharing_violation
21003 ; The sharing mode was correct but not allowed
21004 ; generates an INT 24 on compatibility mode SFTs
21005 ; DS preserved, others destroyed
21006 ;-----
21007 ;
21008 ; 18/05/2019 - Retro DOS v4.0
21009 ; DOSCODE:6A60h (MSDOS 6.21, MSDOS.SYS)
21010 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
21011 ; DOSCODE:6A4Ch (MSDOS 5.0, MSDOS.SYS)
21012 ;
21013 ; 01/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
21014 ; DOSCODE:71BBh (PCDOS 7.1, IBMDOS.COM)
21015 ;
21016 DOS_OPEN:
21017 ; DS has been set up to DOSDATA in file.asm and fcbio2.asm.
21018 ;
21019 000031FA C606[4C03]00 MOV byte [NoSetDir],0
21020 000031FF E83301 CALL Check_Access_AX
21021 00003202 722B JC short do_ret_label ; retc
21022 ;
21023 00003204 C43E[9E05] LES DI,[THISSFT]
21024 00003208 30E4 XOR AH,AH
21025 ;
21026 ; slease! move only access/sharing mode in. Leave sf_isFCB unchanged
21027 ;
21028 0000320A 26884502 MOV [ES:DI+SF_ENTRY.sf_mode],AL ; For moment do this on FCBs too
21029 0000320E 06 PUSH ES
21030 0000320F C436[A205] LES SI,[THISCDS]
21031 ; 18/08/2018
21032 00003213 83FEFF CMP SI,-1 ; 0FFFFh
21033 00003216 7530 JNZ short TEST_RE_NET1
21034 00003218 07 POP ES
21035 ;
21036 ; MSDOS 6.0
21037 ;Extended open hooks
21038 00003219 F606[F605]01 TEST byte [EXTOPEN_ON],EXT_OPEN_ON ;FT. from extended open ;AN000;
21039 0000321E 7410 JZ short _NOEXTOP ;FT. no, do normal ;AN000;
21040 ;_IFS_extopen: ;AN000;
21041 00003220 A0[0106] MOV AL,[SAVE_BX] ; M034 - save_bx has original bx
21042 ; with which call was made. This
21043 ; has the open access bits.
21044 ; ;MOV AL,[SAVE_CX] ; M034 - FT. al= create attribute
21045 ;
21046 00003223 50 PUSH AX ;FT. pass create attr to IFS ;AN000;
21047 ;mov ax,112Eh
21048 ;MOV AX,(MULTNET SHL 8) OR 46 ;FT. issue extended open verb ;AN000;
21049 00003224 B82E11 mov ax,(MULTNET*256)+46
21050 00003227 CD2F INT 2FH ;FT. ;AN000;
21051 00003229 5B POP BX ;FT. trash bx ;AN000;
21052 0000322A C606[F605]00 MOV byte [EXTOPEN_ON],0 ;FT. ;AN000;
21053 ;
21054 do_ret_label:
21055 0000322F C3 retn ;FT. ;AN000;
21056 ;_NOEXTOP:
21057 ;Extended open hooks
21058 ;
21059 ;IF NOT Installed
21060 ;transfer NET_SEQ_OPEN
21061 ;ELSE
21062 ;
21063 do_net_int2f:
21064 00003230 F606[8600]01 test byte [DOS_FLAG],EXECOPEN ; Q: was this open call made from exec
21065 00003235 7409 jz short not_exec_open ; N: just do net open

```

```

21066                                     ; Y: check to see if redir is aware
21067                                     ;   of this
21068
21069                                     ; M042 - start
21070 ;test word [DOS34_FLAG],EXEC_AWARE_REDIR ; 800h
21071 00003237 F606[1206]08 test byte [DOS34_FLAG+1],(EXEC_AWARE_REDIR>>8)
21072                                     ; Q: does this redir know how to
21073                                     ;   this
21074 0000323C 7402 jz short not_exec_open ; N: just do net open
21075                                     ; Y: set bit 3 of access byte and
21076                                     ;   set sharing mode to DENY_WRITE
21077                                     ; M042 - end
21078
21079 ; NOTE: This specific mode has not been set for the code assembled
21080 ; under the "NOT Installed" conditional. Currently Installed is
21081 ; always one.
21082                                     ; M035 - set the bits on the stack
21083 ;mov al,23h
21084 0000323E B023 mov AL,SHARING_DENY_WRITE+EXEC_OPEN
21085
21086 not_exec_open:
21087 ; MSDOS 3.3 & MSDOS 6.0
21088 00003240 50 PUSH AX
21089
21090 ;MOV AX,(MultNET SHL 8) OR 22
21091 ;INT 2FH
21092
21093 00003241 B81611 mov ax,1116h
21094 00003244 CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - OPEN EXISTING REMOTE FILE
21095                                     ; ES:DI -> uninitialized SFT, SS = DOS CS
21096                                     ; SDA first filename pointer -> fully-qualified name of file to open
21097                                     ; STACK: WORD file open mode
21098                                     ; Return: CF set on error
21099
21100 00003246 5B POP BX ; clean stack
21101 ;do_ret_label: ; 09/08/2018
21102 00003247 C3 retn
21103 ;ENDIF
21104
21105 TEST_RE_NET1:
21106 ;TEST word [ES:SI+curdir.flags],curdir_isnet
21107 ; 17/12/2022
21108 00003248 26F6444480 test byte [ES:SI+curdir.flags+1],curdir_isnet>>8
21109 0000324D 07 POP ES
21110 ; 18/05/2019
21111 0000324E 7409 JZ short LOCAL_OPEN
21112
21113 ;Extended open hooks
21114 ; MSDOS 6.0
21115 00003250 F606[F605]01 TEST byte [EXTOPEN_ON],EXT_OPEN_ON ;FT. from extended open ;AN000;
21116 00003255 75C9 JNZ short _IFS_extopen ;FT. issue extended open ;AN000;
21117 ;Extended open hooks
21118
21119 ;IF NOT Installed
21120 ; transfer NET_OPEN
21121 ;ELSE
21122 00003257 EBD7 jmp short do_net_int2f
21123 ;ENDIF
21124
21125 LOCAL_OPEN:
21126 ; MSDOS 3.3 & MSDOS 6.0
21127 00003259 E886E6 call ECritDisk
21128
21129 ; DOS 3.3 FastOpen 6/16/86
21130
21131 ;or byte [FastOpenFlg],5
21132 0000325C 800E[4611]05 OR byte [FastOpenFlg],FastOpen_Set+Special_Fill_Set ; only open can
21133
21134 00003261 E8AB18 call GETPATH
21135
21136 ; DOS 3.3 FastOpen 6/16/86
21137
21138 JNC short Open_found
21139 JNZ short bad_path2
21140 00003268 08C9 OR CL,CL
21141 0000326A 740D JZ short bad_path2
21142
21143 0000326C B80200 OpenFNF: MOV AX,error_file_not_found ; 2
21144
21145 OpenBadRet:
21146 ;hkn; FastOpenFlg is in DOSDATA use SS override
21147 ; 12/08/2018
21148 ;mov byte [cs:FastOpenFlg],0 ; IBMDOS.COM (MSDOS 3.3) offset 36CAh
21149 0000326F 368026[4611]80 AND BYTE [SS:FastOpenFlg],Fast_yes ;; DOS 3.3
21150 00003275 F9 STC
21151 ;call LCritDisk
21152 ; 16/12/2022
21153 00003276 E996E6 jmp LCritDisk
21154 ;JMP Clear_FastOpen ; 10/08/2018
21155 ;retn ; 08/09/2018
21156 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
21157 ;jmp Clear_FastOpen
21158
21159 bad_path2:
21160 00003279 B80300 MOV AX,error_path_not_found ; 3
21161 0000327C EBF1 JMP short OpenBadRet
21162
21163 Open_Bad_Access:
21164 0000327E B80500 MOV AX,error_access_denied; 5
21165 00003281 EBEC JMP short OpenBadRet
21166
21167 Open_found:
21168 00003283 74F9 JZ short Open_Bad_Access ; test for directories
21169 00003285 08E4 OR AH,AH
21170 00003287 783E JS short open_ok ; Devices don't have attributes
21171 00003289 8E06[E405] MOV ES,[CURBUF+2] ; get buffer location
21172 ;mov al,[es:bx+0Bh]
21173 0000328D 268A470B MOV AL,[ES:BX+dir_entry.dir_attr]
21174 00003291 A808 TEST AL,attr_volume_id ; can't open volume ids
21175 00003293 75E9 JNZ short Open_Bad_Access
21176 00003295 A801 TEST AL,attr_read_only ; check write on read only
21177 00003297 742E JZ short open_ok
21178
21179 ; The file is marked READ-ONLY. We verify that the open mode allows access to
21180 ; the read-only file. Unfortunately, with FCB's and net-FCB's we cannot
21181 ; determine at the OPEN time if such access is allowed. Thus, we defer such
21182 ; processing until the actual write operation:
21183 ;
21184 ; If FCB, then we change the mode to be read_only.
21185 ; If net_FCB, then we change the mode to be read_only.
21186 ; If not open for read then error.
21187
21188 00003299 1E push ds
21189 0000329A 56 push si

```

```

21190 0000329B C536[9E05]      LDS     SI,[THISSFT]
21191                          ;mov     CX,[si+2]
21192 0000329F 8B4C02          MOV     CX,[SI+SF_ENTRY.sf_mode]
21193                          ; 17/12/2022
21194                          ;test    ch,80h
21195 000032A2 F6C580          test    ch,sf_isFCB>>8
21196                          ;TEST    CX,sf_isFCB ; 8000h ; is it FCB?
21197 000032A5 750A          JNZ     short ResetAccess ; yes, reset the access
21198 000032A7 88CA          MOV     DL,CL
21199 000032A9 80E2F0          AND     DL,SHARING_MASK ; 0F0h
21200 000032AC 80FA70          CMP     DL,SHARING_NET_FCB ; 70h ; is it net FCB?
21201 000032AF 7508          JNZ     short NormalOpen ; no
21202 ResetAccess:
21203                          ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
21204                          ;AND     CX,~access_mask ; 0FFF0h ; clear access
21205                          ; 16/12/2022
21206                          ;and     cl,0F0h ; 18/05/2019
21207                          ;;;
21208                          ; 01/02/2024 - Retro DOS v5.0
21209                          ; (PCDOS 7.1 IBMDOS.COM)
21210                          ;and     cx,0FFFCh
21211 000032B1 80E1FC          and     cl,0FCh ; ~3 ; ~open_mode_mask ; clear access
21212                          ;;;
21213                          ; OR     CX,open_for_read ; 0 ; stick in open_for_read
21214 000032B4 894C02          MOV     [SI+SF_ENTRY.sf_mode],CX
21215 000032B7 EB0C          JMP     SHORT FillSFT
21216
21217                          ; The SFT is normal. See if the requested access is open_for_read
21218
21219 NormalOpen:
21220                          ;AND     CL,access_mask ; 0Fh ; remove extras
21221                          ;;;
21222                          ; 01/02/2024 - Retro DOS v5.0
21223                          ; (PCDOS 7.1 IBMDOS.COM)
21224 000032B9 80E103          and     cl,3 ; it was 'and cl,0Fh' in MSDOS 6.22
21225                          ; (and cl,access_mask)
21226                          ; this may be open_mode_mask
21227                          ;;;
21228 000032BC 80F900          CMP     CL,open_for_read ; 0 ; is it open for read?
21229 000032BF 7404          JZ     short FillSFT ; yes
21230 000032C1 5E          pop     si
21231 000032C2 1F          pop     ds
21232 000032C3 EBB9          JMP     short Open_Bad_Access
21233
21234                          ; All done, restore registers and fill the SFT.
21235                          ;
21236 FillSFT:
21237 000032C5 5E          pop     si
21238 000032C6 1F          pop     ds
21239
21240 000032C7 E83523          open_ok: call    DOOPEN ; Fill in SFT
21241
21242 ;hkn; FastOpenFlg is in DOSDATA. use SS override
21243 ; 18/05/2019
21244 ;and     byte [ss:FastOpenFlg],80h
21245 000032CA 368026[4611]80      AND     BYTE [SS:FastOpenFlg],Fast_yes ; DOS 3.3
21246 ; 12/08/2018
21247 ;and     byte [FastOpenFlg],Fast_yes
21248
21249 ; MSDOS 6.0
21250 000032D0 E84300          CALL    DO_SHARE_CHECK
21251 000032D3 7303          JNC     short SHARE_OK
21252 ;call    LCritDisk
21253 ; 16/12/2022
21254 000032D5 E937E6          jmp     LCritDisk
21255 ;JMP     short Clear_FastOpen
21256 ;retn ; 18/05/2019
21257 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
21258 ;jmp     short Clear_FastOpen
21259
21260 ; MSDOS 3.3
21261 ;DO_SHARE_CHECK:
21262 ; MOV     CX,[RetryCount] ; Get # tries to do
21263 ;OpenShareRetry:
21264 ; push     CX ; Save number left to do
21265 ; call     SHARE_CHECK ; Final check
21266 ; pop      CX ; CX = # left
21267 ; JNC     short SHARE_OK ; No problem with access
21268 ; call     Idle
21269 ; LOOP    OpenShareRetry ; One more retry used up
21270 ;OpenShareFail:
21271 ; LES     DI,[THISSFT]
21272 ; call     SHARE_ERROR
21273 ; JNC     short DO_SHARE_CHECK ; User wants more retry
21274
21275 ;12/08/2018
21276 ;mov     byte [ss:FastOpenFlg],0
21277 ;08/09/2018
21278 ;mov     byte [FastOpenFlg],0
21279 ;call     LCritDisk
21280 ;JMP     short Clear_FastOpen
21281 ;retn
21282
21283 SHARE_OK:
21284 ; MSDOS 3.3 & MSDOS 6.0
21285 000032D8 B80300          MOV     AX,3
21286 000032DB C43E[9E05]      LES     DI,[THISSFT]
21287 ;if installed
21288 ;call     JShare + 14 * 4
21289 000032DF FF1E[C800]      call     far [Jshare+(14*4)] ; 14 = ShSU
21290 ;else
21291 ; Call     ShSU
21292 ;endif
21293 000032E3 E829E6          call     LCritDisk
21294
21295 ;FallThru Set_SFT_Mode
21296
21297 ;-----
21298 ; Procedure Name : SET_SFT_MODE
21299 ;
21300 ; Finish SFT initialization for new reference. Set the correct mode.
21301 ;
21302 ; Inputs:
21303 ; ThisSFT points to SFT
21304 ;
21305 ; Outputs:
21306 ; Carry clear
21307 ; Registers modified: AX.
21308 ;-----
21309
21310 ;hkn; called from create. DS already set up to DOSDATA.
21311
21312 SET_SFT_MODE:
21313 000032E6 C43E[9E05]      LES     DI,[THISSFT]

```

```

21314 000032EA E8231F      call    DEV_OPEN_SFT
21315                      ;test word [es:di+2],8000h
21316                      ; 17/12/2022
21317                      ;test byte [es:di+3],80h
21318 000032ED 26F6450380  test    byte [ES:DI+SF_ENTRY.sf_mode+1],sf_isFCB>>8
21319                      ;TEST word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB ; Clears carry
21320 000032F2 7407        jz      short Clear_FastOpen ; sf_mode correct (retz)
21321 000032F4 A1[3003]    mov     AX,[CurrentPDB]
21322                      ;mov [es:di+31h],ax
21323 000032F7 26894531    mov     [ES:DI+SF_ENTRY.sf_PID],AX ; For FCB sf_PID=PDB
21324
21325                      Clear_FastOpen:
21326 000032FB C3          retn                                ;;;; DOS 3.3
21327
21328                      ;-----
21329                      ; Procedure Name : SHARE_ERROR
21330                      ;
21331                      ; Called on sharing violations. ES:DI points to SFT. AX has error code
21332                      ; If SFT is FCB or compatibility mode gens INT 24 error.
21333                      ; Returns carry set AX=error_sharing_violation if user says ignore (can't
21334                      ; really ignore). Carry clear if user wants a retry. ES, DI, DS preserved
21335                      ;-----
21336                      SHARE_ERROR:
21337                      ; 17/12/2022
21338                      ;test byte [es:di+3],80h
21339                      test    byte [ES:DI+SF_ENTRY.sf_mode+1],sf_isFCB>>8 ; 80h
21340 000032FC 26F6450380  ;TEST word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB ; 8000h
21341                      jnz     short _HARD_ERR
21342 00003301 7509        mov     CL,[ES:DI+SF_ENTRY.sf_mode]
21343 00003303 268A4D02    and     CL,SHARING_MASK ; 0F0h
21344 00003307 80E1F0      cmp     CL,SHARING_COMPAT ; 0
21345                      ;JNE short _NO_HARD_ERR
21346                      ; 21/09/2023
21347                      jnz     short _NO_HARD_ERR
21348 0000330A 7505        _HARD_ERR:
21349                      call    SHARE_VIOLATION
21350 0000330C E83851      ;retnc ; User wants retry
21351                      jnc     short Clear_FastOpen
21352 0000330F 73EA        _NO_HARD_ERR:
21353                      mov     AX,error_sharing_violation ; 20h
21354 00003311 B82000      stc
21355 00003314 F9          retn
21356 00003315 C3
21357
21358                      ; MSDOS 6.0
21359                      ;-----
21360                      ; Procedure Name : DO_SHARE_CHECK
21361                      ;
21362                      ; Input: THISDPB, WFP_Start, THISSFT set
21363                      ; Functions: check file sharing mode is valid
21364                      ; Output: carry set, error
21365                      ; carry clear, share ok
21366                      ;-----
21367
21368                      ; 18/05/2019 - Retro DOS v4.0
21369                      DO_SHARE_CHECK:
21370 00003316 E8C9E5      call    ECritDisk ; enter critical section
21371                      OPN_RETRY:
21372 00003319 8B0E[1A00]  mov     CX,[RetryCount] ; Get # tries to do
21373                      OpenShareRetry:
21374 0000331D 51          push    CX ; Save number left to do
21375 0000331E E82151      call    SHARE_CHECK ; Final Check
21376 00003321 59          pop     CX ; CX = # left
21377 00003322 730E        jnc     short Share_Ok2 ; No problem with access
21378 00003324 E8BDE4      call    Idle
21379 00003327 E2F4        loop   OpenShareRetry ; One more retry used up
21380                      OpenShareFail:
21381 00003329 C43E[9E05]  les     DI,[THISSFT]
21382 0000332D E8CCFF      call    SHARE_ERROR
21383 00003330 73E7        jnc     short OPN_RETRY ; User wants more retry
21384                      Share_Ok2:
21385                      ;call LCritDisk ; leave critical section
21386                      ;retn
21387                      ; 18/12/2022
21388 00003332 E9DAE5      jmp     LCritDisk
21389
21390                      ;-----
21391                      ; Procedure Name : Check_Access
21392                      ;
21393                      ; Inputs:
21394                      ; AX is mode
21395                      ; High NIBBLE of AL (Sharing Mode)
21396                      ; sharing_compat file is opened in compatibility mode
21397                      ; sharing_deny_none file is opened Multi reader, Multi writer
21398                      ; sharing_deny_read file is opened Only reader, Multi writer
21399                      ; sharing_deny_write file is opened Multi reader, Only writer
21400                      ; sharing_deny_both file is opened Only reader, Only writer
21401                      ; Low NIBBLE of AL (Access Mode)
21402                      ; open_for_read file is opened for reading
21403                      ; open_for_write file is opened for writing
21404                      ; open_for_both file is opened for both reading and writing.
21405                      ; Function:
21406                      ; Check this access mode for correctness
21407                      ; Outputs:
21408                      ; [open_access] = AL input
21409                      ; Carry Clear
21410                      ; Mode is correct
21411                      ; AX unchanged
21412                      ; Carry Set
21413                      ; Mode is bad
21414                      ; AX = error_invalid_access
21415                      ; No other registers effected
21416                      ;-----
21417
21418                      ; 23/01/2024 - Retro DOS v5.0
21419                      Check_Access_AX:
21420 00003335 A2[6E05]    mov     [OPEN_ACCESS],AL
21421 00003338 53          push    BX
21422
21423                      ; If sharing, then test for special sharing mode for FCBs
21424
21425 00003339 88C3        mov     BL,AL
21426 0000333B 80E3F0      and     BL,SHARING_MASK ; 0F0h
21427
21428                      ;CMP byte [FSHARING],-1
21429                      ;JNZ short CheckShareMode ; not through server call, must be ok
21430                      ; 23/01/2024
21431                      ; PCDOS 7.1 IBMDOS.COM
21432 0000333E 803E[7205]00  cmp     byte [FSHARING],0
21433 00003343 7405        jz      short CheckShareMode ; not through server call, must be ok
21434
21435 00003345 80FB70      cmp     BL,SHARING_NET_FCB ; 70h
21436 00003348 7405        jz      short CheckAccessMode ; yes, we have an FCB
21437                      CheckShareMode:

```

```

21438 0000334A 80FB40      CMP     BL,40h          ; is this a good sharing mode?
21439 0000334D 770B      JA      short Make_Bad_Access
21440                                CheckAccessMode:
21441 0000334F 88C3      MOV     BL,AL
21442                                ; 23/01/2024
21443 00003351 80E303    and     b1,3 ; PCDOS 7.1 IBMDOS.COM
21444                                ;AND    BL,access_mask ; 0Fh
21445                                ; 04/02/2026
21446                                ;CMP    BL,2
21447                                ;JA     short Make_Bad_Access
21448                                ;POP    BX
21449                                ;CLC
21450                                ;retn
21451                                ; 04/02/2026
21452 00003354 80FB03    cmp     b1,3
21453 00003357 F5      cmc
21454                                ; if b1>2 -> cf=1
21455 00003358 7303      jnc     short CheckAccessMode_OK
21456
21457                                Make_Bad_Access:
21458 0000335A B80C00    MOV     AX,error_invalid_access ; 0Ch
21459                                ; 04/02/2026
21460                                CheckAccessMode_OK:
21461 0000335D 5B      POP     BX
21462                                ;STC
21463 0000335E C3      retn
21464
21465                                ;=====
21466                                ; DINFO.ASM, MSDOS 6.0, 1991
21467                                ;=====
21468                                ; 08/08/2018 - Retro DOS v3.0
21469                                ; 18/05/2019 - Retro DOS v4.0
21470                                ; 02/02/2024 - Retro DOS v5.0
21471
21472                                ;** Low level routine for returning disk drive information from a local
21473                                ; or NET device
21474                                ;
21475                                ; DISK_INFO
21476                                ;
21477                                ; Modification history:
21478                                ;
21479                                ; Created: ARR 30 March 1983
21480                                ;
21481                                ; Break <DISK_INFO -- Get Disk Drive Information>
21482                                ;-----
21483                                ; Procedure Name : DISK_INFO
21484                                ;
21485                                ; Inputs:
21486                                ; [THISCDs] Points to the Macro List Structure of interest
21487                                ; (It MAY NOT be NUL, error not detected)
21488                                ; Function:
21489                                ; Get Interesting Drive Information
21490                                ; Returns:
21491                                ; DX = Number of free allocation units
21492                                ; BX = Total Number of allocation units on disk
21493                                ; CX = Sector size
21494                                ; AL = Sectors per allocation unit
21495                                ; AH = FAT ID BYTE
21496                                ; Carry set if error (currently user FAILED to I 24)
21497                                ; Segs except ES preserved, others destroyed
21498                                ;-----
21499
21500                                ;hkn; called from getset.asm and misc.asm. DS has already been set up to
21501                                ;hkn; DOSDATA.
21502
21503                                ; 02/02/2024 - Retro DOS v5.0
21504                                ; PCDOS 7.1 IBMDOS.COM - DOSCODE:732Bh
21505
21506                                DISK_INFO:
21507                                ; 08/08/2018 - Retro DOS v3.0
21508                                ; IBM DOS.COM (MSDOS 3.3, 1987) - Offset 37C5h
21509
21510 0000335F E8C7E4    call    TestNet
21511 00003362 7318      JNC     short LOCAL_DSK_INFO
21512
21513                                ;;;
21514                                ; 02/02/2024 (PCDOS 7.1 IBMDOS.COM)
21515 00003364 31F6      xor     si,si ; free cluster count hw = 0
21516 00003366 31FF      xor     di,di ; number of clusters hw = 0
21517                                ;;;
21518
21519                                ;IF NOT Installed
21520                                ; transfer NET_DISK_INFO
21521                                ;ELSE
21522                                ;MOV     AX,(MULTNET SHL 8) OR 12
21523                                ;INT     2FH
21524                                ;return
21525
21526 00003368 B80C11    mov     ax,110Ch
21527 0000336B CD2F      int     2Fh ; Multiplex - NETWORK REDIRECTOR - GET DISK SPACE
21528                                ; ES:DI -> current directory
21529                                ; Return: AL = sectors per cluster, BX = total clusters
21530                                ; CX = bytes per sector, DX = number of available clusters
21531                                ;;;
21532                                ; 02/02/2024 (PCDOS 7.1 IBMDOS.COM)
21533 0000336D 83FBFF    cmp     bx,0FFFFh
21534 00003370 7502      jne     short dsk_info_1
21535 00003372 89DF      mov     di,bx
21536                                dsk_info_1:
21537 00003374 83FAFF    cmp     dx,0FFFFh
21538 00003377 7502      jne     short disk_info_retn
21539 00003379 89D6      mov     si,dx
21540                                disk_info_retn:
21541                                ;;;
21542 0000337B C3      retn
21543                                ;ENDIF
21544
21545                                LOCAL_DSK_INFO:
21546                                ;MOV     byte [EXERR_LOCUS],errLOC_Disk
21547                                ;;;
21548                                ; 02/02/2024 (PCDOS 7.1 IBMDOS.COM)
21549 0000337C E867DF    call    set_exerr_locus_disk
21550                                ;;;
21551 0000337F E860E5    call    ECritDisk
21552 00003382 E8D131    call    FATREAD_CDS ; perform media check.
21553                                ;JC     short CRIT_LEAVE
21554                                ;;; 02/02/2024
21555                                ;JC     short dsk_info_2
21556 00003387 31C0      xor     ax,ax
21557 00003389 A3E80A      mov     [CLUSTNUM_HW],ax ; 0 ; clear high word of cluster number
21558                                ;;;
21559 0000338C B80200      MOV     BX,2
21560 0000338F E84A2F      call    UNPACK ; Get first FAT sector into CURBUF
21561                                ;JC     short CRIT_LEAVE

```



```

21562             ;;;
21563             ; 02/02/2024 - Retro DOS v5.0
21564 00003392 7303      jnc     short dsk_info_3
21565 dsk_info_2:
21566             ; jmp     CRIT_LEAVE
21567 00003394 E978E5     jmp     LCritDisk
21568 dsk_info_3:
21569             ;;;
21570 00003397 C536[E205] LDS     SI,[CURBUF]
21571             ; 02/02/2024
21572             ; mov     ah,[si+20]
21573             ; mov     ah,[si+24] ; PCDOS 7.1 IBMDOS.COM
21574 0000339B 8A6418     MOV     AH,[SI+BUFINSZ] ; get FAT ID BYTE
21575
21576 ;hkn; SS is DOSDATA
21577 0000339E 16         push    ss
21578 0000339F 1F         pop     ds
21579
21580             ; 02/02/2024 - Retro DOS v5.0
21581 %if 0
21582             ; mov     cx,[es:bp+0Dh]
21583             MOV     CX,[ES:BP+DPB.MAX_CLUSTER]
21584
21585             ; Examine the current free count. If it indicates that we have an invalid
21586             ; count, do the expensive calculation.
21587
21588             ; mov     dx,[es:bp+1Fh]
21589             MOV     DX,[ES:BP+DPB.FREE_CNT] ; get free count
21590             CMP     DX,-1 ; is it valid?
21591             JZ      short DoScan
21592
21593             ; Check to see if it is in a reasonable range. If so, trust it and return.
21594             ; Otherwise, we need to blast out an internal error message and then recompute
21595             ; the count.
21596
21597             CMP     DX,CX ; is it in a reasonable range?
21598             JB      short GotVal ; yes, trust it.
21599 DoScan:
21600             XOR     DX,DX
21601             DEC     CX
21602 SCANFREE:
21603             call    UNPACK
21604             JC      short CRIT_LEAVE
21605             JNZ     short NOTFREECLUS
21606             INC     DX ; A free one
21607 NOTFREECLUS:
21608             INC     BX ; Next cluster
21609             LOOP    SCANFREE
21610             DEC     BX ; BX was next cluster. Convert to
21611 ReturnVals:
21612             DEC     BX ; count
21613             ; mov     al,[es:bp+4]
21614             MOV     AL,[ES:BP+DPB.CLUSTER_MASK]
21615             INC     AL ; Sectors/cluster
21616             ; mov     cx,[es:bp+2]
21617             MOV     CX,[ES:BP+DPB.SECTOR_SIZE] ; Bytes/sector
21618             ; mov     [es:bp+1Fh],dx
21619             MOV     [ES:BP+DPB.FREE_CNT],DX
21620             CLC
21621 CRIT_LEAVE:
21622             ; call    LCritDisk
21623             ; retn
21624             ; 17/12/2022
21625             jmp     LCritDisk
21626
21627             ; We have correctly computed everything previously. Load up registers for
21628             ; return.
21629
21630 GotVal:
21631             MOV     BX,CX ; get cluster count
21632             JMP     short ReturnVals
21633
21634 %else
21635             ; 02/02/2024 (PCDOS 7.1 IBMDOS.COM)
21636 000033A0 31F6      xor     si,si ; 0
21637 000033A2 89F7      mov     di,si ; 0
21638             ; mov     dx,[es:bp+1Fh]
21639 000033A4 268B561F   mov     dx,[es:bp+DPB.FREE_CNT] ; get free count
21640             ; cmp     [es:bp+0Fh],si
21641 000033A8 2639760F   cmp     [es:bp+DPB.FAT_SIZE],si ; FAT32 (16 bit FAT size = 0) ?
21642 000033AC 7406      jz      short dsk_info_4 ; yes
21643             ; mov     cx,[es:bp+0Dh]
21644 000033AE 268B4E0D   mov     cx,[es:bp+DPB.MAX_CLUSTER]
21645 000033B2 EB18      jmp     short dsk_info_5 ; zf=0, si=di=0
21646
21647 dsk_info_4:
21648             ; mov     di,[es:bp+2Fh]
21649 000033B4 268B7E2F   mov     di,[es:bp+DPB.LAST_CLUSTER+2]
21650             ; mov     cx,[es:bp+2Dh]
21651 000033B8 268B4E2D   mov     cx,[es:bp+DPB.LAST_CLUSTER]
21652             ; mov     si,[es:bp+21h]
21653 000033BC 268B7621   mov     si,[es:bp+DPB.FREE_CNT_HW] ; hw of free cluster count
21654 000033C0 39F2      cmp     dx,si ; same (zero) ?
21655 ;dsk_info_5:
21656 000033C2 7504      jnz     short dsk_info_6 ; not same (not zero)
21657 000033C4 42         inc     dx
21658 000033C5 740B      jz      short dsk_info_8 ; 0FFFFh -> 0 (free count is invalid/initial)
21659             ; free count calculation is needed
21660             dec     dx
21661 dsk_info_6:
21662 000033C8 39FE      cmp     si,di ; same hw ?
21663 000033CA 7502      jne     short dsk_info_7 ; no
21664 dsk_info_5: ; 02/02/2024 - Retro DOS v5.0
21665 000033CC 39CA      cmp     dx,cx ; same lw ?
21666 dsk_info_7:
21667 000033CE 7268      jb      short GotVal ; free cluster count < last cluster number
21668 000033D0 31D2      xor     dx,dx ; 0
21669 dsk_info_8:
21670 000033D2 31F6      xor     si,si ; 0
21671 000033D4 83E901   sub     cx,1 ; last cluster number - 1 = number of clusters
21672 000033D7 19F7      sbb     di,si
21673             ; or     byte [es:bp+18h],1
21674 000033D9 26804E1801 or     byte [es:bp+DPB.FIRST_ACCESS],1 ; set first access bit 0
21675             ; (Update flag for FSINFO sector)
21676 SCANFREE:
21677 000033DE 56         push    si
21678 000033DF FF36[EC0A] push    word [CCONTENT_HW]
21679 000033E3 57         push    di
21680 000033E4 E8F52E     call    UNPACK
21681 000033E7 5F         pop     di
21682 000033E8 8F06[EC0A] pop     word [CCONTENT_HW]
21683 000033EC 5E         pop     si
21684 000033ED 7246      jc      short CRIT_LEAVE
21685 000033EF 7504      jnz     short NOTFREECLUS

```

```

21686 000033F1 42          inc     dx          ; a free one
21687 000033F2 7501       jnz     short NOTFREECLUS
21688 000033F4 46          inc     si          ; increase hw of free cluster count
21689
21690 NOTFREECLUS:
21691 000033F5 43          inc     bx          ; next cluster
21692 000033F6 7504       jnz     short NOTFREECLUS2
21693 000033F8 FF06[E80A]   inc     word [CLUSTNUM_HW] ; increase hw of (next) cluster number
21694
21695 NOTFREECLUS2:
21696 000033FC 83E901     sub     cx,1          ; decrease remain cluster count for calculation
21697 000033FF 83DF00     sbb     di,0
21698 00003402 75DA       jnz     short SCANFREE
21699 00003404 E302       jcxz    NOTFREECLUS3   ; calculation completed
21700 00003406 EBD6       jmp     short SCANFREE
21701
21702 NOTFREECLUS3:
21703 00003408 8B3E[E80A]   mov     di,[CLUSTNUM_HW]
21704 0000340C 83EB01     sub     bx,1
21705 0000340F 83DF00     sbb     di,0          ; di:bx = last cluster number
21706
21707 ReturnVals:
21708 00003412 31C9       xor     cx,cx
21709 00003414 83EB01     sub     bx,1
21710 00003417 19CF       sbb     di,cx          ; di:bx = number of clusters
21711          ;mov     al,[es:bp+4]
21712 00003419 268A4604   mov     al,[es:bp+DPB.CLUSTER_MASK] ; spc - 1
21713 0000341D FEC0       inc     al          ; sectors per cluster
21714          ;mov     [es:bp+1Fh],dx
21715 0000341F 2689561F   mov     [es:bp+DPB.FREE_CNT],dx      ; free cluster count,1w
21716          ;cmp     [es:bp+0Fh],cx ; 0
21717 00003423 26394E0F   cmp     [es:bp+DPB.FAT_SIZE],cx ; 0 ; FAT32 (16 bit FAT size = 0) ?
21718 00003427 7507       jnz     short ReturnVals2          ; no
21719          ;mov     [es:bp+21h],si
21720 00003429 26897621   mov     [es:bp+DPB.FREE_CNT_HW],si ; hw of free cluster count
21721
21722 0000342D E83800     call    update_fat32_fsinfo
21723
21724 ReturnVals2:
21725          ;mov     cx,[es:bp+2]
21726 00003430 268B4E02   mov     cx,[es:bp+DPB.SECTOR_SIZE] ; bytes per sector
21727 00003434 F8          cld
21728
21729 CRIT_LEAVE:
21730          ;call    LCritDisk
21731          ;retn
21732 00003435 E9D7E4     jmp     LCritDisk
21733
21734 GotVal:
21735 00003438 89CB       mov     bx,cx
21736 0000343A EBD6       jmp     short ReturnVals
21737
21738 %endif
21739
21740 ; ===== S U B R O U T I N E =====
21741
21742 ; 03/02/2024 - Retro DOS v5.0
21743
21744 modify_spc:
21745 0000343C 50          push    ax          ; ax = sectors per cluster
21746 0000343D 52          push    dx
21747 0000343E F7E1       mul     cx          ; bytes per sector
21748          ;cmp     dx,0
21749 00003440 21D2       and     dx,dx
21750 00003442 7503       jnz     short mspc_1
21751 00003444 3D0040     cmp     ax,16384      ; 16 kilobytes (per cluster)
21752          ;***
21753          ;actual disk size limit
21754          ;without invalidating cluster counts is
21755          ;2 GB (512K clusters * 8 sectors per cluster)
21756          ;(128K clusters * 32 sectors per cluster)
21757
21758 mspc_1:
21759 00003447 5A          pop     dx
21760 00003448 58          pop     ax
21761 00003449 760E       jbe     short mspc_3   ; bytes per cluster <= 16 KB
21762          ;***
21763          ; bytes per cluster > 16 KB
21764 0000344B 31FF       xor     di,di          ; 0
21765 0000344D BBFEFF     mov     bx,0FFFEh      ; (invalidated)
21766 00003450 09F6       or      si,si          ; hw of free cluster count
21767 00003452 7404       jz      short mspc_2   ; si = 0
21768 00003454 89FE       mov     si,di          ; si = 0
21769 00003456 89DA       mov     dx,bx          ; dx = bx = 0FFFEh (invalidated)
21770
21771 mspc_2:
21772          retn
21773
21774 mspc_3:
21775 00003459 D1E0       shl     ax,1          ; sectors per clust = sectors per clust * 2
21776          ;(ax <= 32768) -modified spc limit-
21777 0000345B D1EF       shr     di,1          ; cluster count = cluster count / 2
21778          ; di:bx = modified value of total clusters
21779
21780 mspc_4:
21781 0000345D D1DB       rcr     bx,1
21782 0000345F D1EE       shr     si,1          ; free clusters = free clusters / 2
21783 00003461 D1DA       rcr     dx,1          ; si:dx = modified value of free clusters
21784
21785 ; -----
21786          ; 03/02/2024
21787          ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7431h
21788
21789 modify_cluster_count:
21790 00003463 09FF       or      di,di          ; hw of cluster count
21791 00003465 75D5       jnz     short modify_spc
21792 00003467 C3          retn
21793
21794 ; ===== S U B R O U T I N E =====
21795
21796 ; write FSINFO sector onto disk
21797
21798 ; 03/02/2024 - Retro DOS v5.0
21799 ; -----
21800 ; FAT32 FSInfo Sector Structure
21801 ; -----
21802 ; ref: Microsoft FAT32 File System Specification (2000)
21803
21804 struc FSINFO ; offset ;
21805 .LeadSig: resb 4 ; 0 ; Value 0x41615252. Lead Signature.
21806 .Reserved1: resb 480 ; 4 ; Reserved. Must be 0. Never be used.
21807 .StrucSig: resb 4 ; 484 ; Value 0x61417272. Fields Signature.
21808 .Free_Count: resb 4 ; 488 ; Last known free cluster count. (*)
21809 .Nxt_Free: resb 4 ; 492 ; Start clus for free clus srch. (**)
21810 .Reserved2: resb 12 ; 496 ; Reserved. Must be 0. Never be used.
21811 .Trailsig: resb 4 ; 508 ; Value 0xAA550000. Trail Signature.

```



```

21810 .size:
21811 endstruc
21812
21813 ; (*) If the value is 0xFFFFFFFF, then the free count is unknown
21814 ; and must be computed.
21815 ; (**) If the value is 0xFFFFFFFF, then free cluster search must be started
21816 ; from cluster 2.
21817 ; Lead Signature, Fields (Structure) Signature and Trail Signature
21818 ; are used to validate FSInfo sector.
21819
21820 ; -----
21821
21822 ; 03/02/2024 - Retro DOS v5.0
21823 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7436h
21824 ; (Windows ME IO.SYS - BIOSCODE:7227h)
21825
21826 update_fat32_fsinfo:
21827     push    cx
21828     push    dx
21829     xor     cx,cx
21830     ;mov     dx,[es:bp+25h]
21831     mov     dx,[es:bp+DPB.FSINFO_SECTOR]
21832     ;cmp     [es:bp+0Fh],cx
21833     cmp     [es:bp+DPB.FAT_SIZE],cx ; 0
21834     ; (16bit FAT size field = 0 for FAT32 fs)
21835     jne     short u_fat32_inf_1
21836     cmp     dx,0FFFFh ; -1
21837     jne     short u_fat32_inf_2
21838
21839 u_fat32_inf_1:
21840     pop     dx
21841     pop     cx
21842     ;and     byte [es:bp+18h],0F4h
21843     and     byte [es:bp+DPB.FIRST_ACCESS],0F4h ; clear bit 0,1 and 3
21844     ; bit 0 - FSINFO update (dirty) bit
21845     ; bit 1 - BPB_RootClus update bit
21846     ; bit 3 - BPB_ExtFlags update bit
21847     ret     3
21848
21849 u_fat32_inf_2:
21850     push    ax
21851     push    bx
21852     push    di
21853     push    si
21854     push    ds
21855     cmp     byte [ss:BuffInHMA],0 ; is buffers in HMA?
21856     jz      short u_fat32_inf_3 ; no
21857     lds     di,[ss:LoMemBuff] ; read it into scratch buffer
21858     ;sub     di,24
21859     sub     di,BUFINSIZ ; space for buffer header
21860     ; (buffer header size = 24)
21861     ;clc
21862     jmp     short u_fat32_inf_4
21863
21864 u_fat32_inf_3:
21865     push    es
21866     push    bp
21867     call    GETCURHEAD ; ds:di = first buffer in queue
21868     push    dx
21869     call    BUFWRITE ; Bufwrite writes a buffer to the disk,
21870     ; if it's dirty.
21871     pop     dx
21872     pop     bp
21873     pop     es
21874
21875 ;u_fat32_inf_4:
21876     jc      short u_fat32_inf_5
21877 u_fat32_inf_4:
21878     xor     cx,cx
21879     ;lea     bx,[di+24]
21880     lea     bx,[di+BUFINSIZ] ; buffer data address
21881     ;mov     byte [ss:ALLOWED],18h
21882     mov     byte [ss:ALLOWED],Allowed_FAIL+Allowed_RETRY
21883     mov     [ss:HIGH_SECTOR],cx ; 0
21884     inc     cx ; cx = sector count = 1
21885     ; es:bp = DPB
21886     ; ds:bx = buffer (data) address
21887     push    bx
21888     push    dx
21889     call    DREAD ; HIGH_SECTOR:dx = disk sector address
21890     ; read fs info sector
21891     pop     dx
21892     pop     bx
21893     jc      short u_fat32_inf_5
21894     ;cmp     word [bx+FSINFO.LeadSig],5252h
21895     cmp     word [bx],5252h ; 'RR'
21896     jne     short u_fat32_inf_5
21897     ;cmp     word [bx+2],4161h ; 'aA' ; (NASM syntax)
21898     cmp     word [bx+FSINFO.LeadSig+2],4161h
21899     jne     short u_fat32_inf_5
21900     ;cmp     word [bx+1E4h],7272h ; 'rr' at offset 484
21901     cmp     word [bx+FSINFO.StrucSig],7272h
21902     jne     short u_fat32_inf_5
21903     ;cmp     word [bx+1E6h],6141h ; 'Aa' at offset 486
21904     cmp     word [bx+FSINFO.StrucSig+2],6141h
21905     jne     short u_fat32_inf_5
21906     ;cmp     word [bx+1FEh],0AA55h ; boot signature at offset 510
21907     cmp     word [bx+FSINFO.TrailSig+2],0AA55h
21908     jne     short u_fat32_inf_5
21909
21910     ;mov     ax,[es:bp+1Fh]
21911     mov     ax,[es:bp+DPB.FREE_CNT]
21912     ;mov     [bx+1E8h],ax
21913     mov     [bx+FSINFO.Free_Count],ax ; at offset 488
21914     ;mov     ax,[es:bp+21h]
21915     mov     ax,[es:bp+DPB.FREE_CNT+2]
21916     ;mov     [bx+1EAh],ax
21917     mov     [bx+FSINFO.Free_Count+2],ax
21918
21919     ;mov     ax,[es:bp+39h]
21920     mov     ax,[es:bp+DPB.FAT32_NXTFREE]
21921     ;mov     [bx+1ECh],ax
21922     mov     [bx+FSINFO.Nxt_Free],ax ; at offset 492
21923     ;mov     ax,[es:bp+3Bh]
21924     mov     ax,[es:bp+DPB.FAT32_NXTFREE+2]
21925     ;mov     [bx+1EEh],ax
21926     mov     [bx+FSINFO.Nxt_Free+2],ax
21927
21928     xor     cx,cx
21929     mov     [ss:HIGH_SECTOR],cx ; 0
21930     inc     cx ; 1
21931     call    DWRITE
21932
21933

```

```

21934      u_fat32_inf_5:
21935      pop     ds
21936      pop     si
21937      pop     di
21938      pop     bx
21939      pop     ax
21940      jmp     u_fat32_inf_1
21941
21942      ;=====
21943      ; ISEARCH.ASM, MSDOS 6.0, 1991
21944      ;=====
21945      ; 22/07/2018 - Retro DOS v3.0
21946
21947      ; TITLE  DOS_SEARCH - Internal SEARCH calls for MS-DOS
21948      ; NAME    DOS_SEARCH
21949
21950      ;** Low level routines for doing local and NET directory searches
21951      ;
21952      ; DOS_SEARCH_FIRST
21953      ; DOS_SEARCH_NEXT
21954      ; RENAME_NEXT
21955      ;
21956      ; Revision history:
21957      ;
21958      ; Created: ARR 30 March 1983
21959      ; A000 version 4.00 Jan. 1988
21960      ; A001 PTM 3564 -- search for fastopen
21961
21962      ;Installed = TRUE
21963
21964      ;-----
21965      ; Procedure Name : DOS_SEARCH_FIRST
21966      ;
21967      ; Inputs:
21968      ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
21969      ; terminated)
21970      ; [CURR_DIR_END] Points to end of Current dir part of string
21971      ; (= -1 if current dir not involved, else
21972      ; Points to first char after last "/" of current dir part)
21973      ; [THISCDS] Points to CDS being used
21974      ; (Low word = -1 if NUL CDS (Net direct request))
21975      ; [SATTRIB] Is attribute of search, determines what files can be found
21976      ; [DMAADD] Points to 53 byte buffer
21977      ; Function:
21978      ; Initiate a search for the given file spec
21979      ; Outputs:
21980      ; CARRY CLEAR
21981      ; The 53 bytes of DMAADD are filled in as follows:
21982      ;
21983      ; LOCAL
21984      ; Drive Byte (A=1, B=2, ...) High bit clear
21985      ; NEVER STORE DRIVE BYTE AFTER found_it
21986      ; 11 byte search name with Meta chars in it
21987      ; Search Attribute Byte, attribute of search
21988      ; WORD LastEnt value
21989      ; WORD DirStart
21990      ; 4 byte pad
21991      ; 32 bytes of the directory entry found
21992      ;
21993      ; NET
21994      ; 21 bytes First byte has high bit set
21995      ; 32 bytes of the directory entry found
21996      ;
21997      ; CARRY SET
21998      ; AX = error code
21999      ; error_no_more_files
22000      ; No match for this file
22001      ; error_path_not_found
22002      ; Bad path (not in curr dir part if present)
22003      ; error_bad_curr_dir
22004      ; Bad path in current directory part of path
22005      ; DS preserved, others destroyed
22006      ;-----
22007
22008      ; 24/01/2024
22009      %if 1
22010      ; 17/05/2019 - Retro DOS v4.0
22011      GET_FAST_SEARCH:
22012      ; 22/07/2018
22013      ; MSDOS 6.0
22014      ; 17/12/2022
22015      OR     byte [ss:DOS34_FLAG+1],(SEARCH_FASTOPEN>>8) ; 04h
22016      ;OR     word [ss:DOS34_FLAG],SEARCH_FASTOPEN ; 400h
22017      ;FO.trigger fastopen ;AN000;
22018      ;call DOS_SEARCH_FIRST
22019      ;retn
22020      ; 24/01/2024
22021      ; 17/12/2022
22022      ;jmp DOS_SEARCH_FIRST
22023      %endif
22024
22025      ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
22026      ; DOSCODE:6C22h (MSDOS 5.0, MSDOS.SYS)
22027
22028      ; 03/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
22029      ; DOSCODE:74E9h (PCDOS 7.1, IBMDOS.COM)
22030
22031      DOS_SEARCH_FIRST:
22032      ; IBMDOS.COM (MSDOS 3.3 kernel) - Offset 3826h
22033
22034      LES     DI,[THISCDS]
22035      CMP     DI,-1
22036      JNZ     short TEST_RE_NET2
22037
22038      ;IF NOT Installed
22039      ; transfer NET_SEQ_SEARCH_FIRST
22040      ;ELSE
22041      ;mov     ax,1119h
22042      MOV     AX,(MultNET<<8)|25
22043      INT     2Fh
22044      retn
22045      ;ENDIF
22046
22047      TEST_RE_NET2:
22048      ;test     word [es:di+43h],8000h
22049      ; 17/12/2022
22050      ;test     byte [es:di+44h],80h
22051      ; 28/12/2022
22052      test     byte [ES:DI+curdir.flags+1],curdir_isnet>>8
22053      ;TEST     word [ES:DI+curdir.flags],curdir_isnet
22054      JZ       short LOCAL_SEARCH_FIRST
22055
22056      ;IF NOT Installed
22057      ; transfer NET_SEARCH_FIRST

```

```

22058
22059
22060 00003536 B81B11
22061 00003539 CD2F
22062 0000353B C3
22063
22064
22065
22066 0000353C E8A3E3
22067
22068
22069
22070
22071 0000353F F606[1206]04
22072
22073 00003544 7405
22074
22075 00003546 800E[4611]01
22076
22077 0000354B C606[4C03]01
22078
22079
22080
22081
22082
22083
22084
22085
22086 00003550 16
22087 00003551 1F
22088 00003552 8B36[B205]
22089
22090 00003556 AC
22091 00003557 08C0
22092 00003559 7409
22093 0000355B 3C3F
22094 0000355D 75F7
22095
22096
22097 0000355F 8026[4611]80
22098
22099
22100 00003564 E8A815
22101
22102
22103 00003567 7318
22104 00003569 7511
22105 0000356B 08C9
22106 0000356D 740D
22107
22108
22109 0000356F B81200
22110
22111
22112 00003572 368026[4611]80
22113
22114 00003578 F9
22115
22116
22117
22118 00003579 E993E3
22119
22120
22121
22122 0000357C B80300
22123 0000357F EBF1
22124
22125
22126 00003581 08E4
22127 00003583 790A
22128 00003585 C706[4803]FFFF
22129 0000358B FE06[7005]
22130
22131
22132
22133
22134
22135 0000358F C43E[2C03]
22136 00003593 8B36[B205]
22137 00003597 AC
22138 00003598 2C40
22139 0000359A AA
22140
22141 0000359B C43E[2C03]
22142 0000359F 47
22143
22144
22145 000035A0 1E
22146
22147 000035A1 F606[4611]10
22148 000035A6 7408
22149 000035A8 89DE
22150 000035AA 8E1E[E405]
22151 000035AE EB03
22152
22153
22154 000035B0 BE[4B05]
22155
22156
22157 000035B3 A4
22158 000035B4 26807DFF05
22159 000035B9 7505
22160 000035BB 26C645FFE5
22161
22162
22163
22164
22165 000035C0 B90500
22166 000035C3 F3A5
22167
22168
22169 000035C5 1F
22170
22171 000035C6 A0[6B05]
22172 000035C9 AA
22173 000035CA 50
22174 000035CB A1[4803]
22175 000035CE AB
22176 000035CF A1[C205]
22177 000035D2 AB
22178
22179
22180
22181

```

```

;ELSE
;mov ax,111bh
MOV AX,(MULTNET<<8)|27
INT 2FH
retn
;ENDIF
; 18/05/2019 - Retro DOS v4.0
LOCAL_SEARCH_FIRST:
call ECritDisk
; MSDOS 6.0
;;test word [DOS34_FLAG],400h
; 17/12/2022
;test byte [DOS34_FLAG+1],04h
test byte [DOS34_FLAG+1],(SEARCH_FASTOPEN>>8)
;TEST word [DOS34_FLAG],SEARCH_FASTOPEN ;AN000;
JZ short NOFN ;AN000;
;or byte [FastOpenFlg],1
OR byte [FastOpenFlg],FastOpen_Set ;AN000;
NOFN:
MOV byte [NoSetDir],1 ; if we find a dir, don't change to it
; 03/02/2024
%if 0
; MSDOS 6.0
CALL CHECK_QUESTION ;AN000;;FO. is '?' in path
JNC short norm_GETPATH ;AN000;;FO. no
%else
; 03/02/2024
push ss
pop ds ;AN000;;FO. ds:si -> final path
mov si,[WFP_START] ;AN000;;FO.
getnext:
lodsb ;AN000;;FO. get char
or al,al ;AN000;;FO. is it null
jz short NO_Question ;AN000;;FO. yes
cmp al,'?' ;AN000;;FO. is '?'
jne short getnext ;AN000;;FO. no
%endif
;and byte [FastOpenFlg],80h
AND byte [FastOpenFlg],Fast_yes ;AN000;;FO. reset fastopen
NO_Question:
; 03/02/2024
norm_GETPATH:
call GETPATH
; BX = offset NAME1
;_getdone:
JNC short find_check_dev
JNZ short bad_path3
OR CL,CL
JZ short bad_path3
find_no_more:
;mov ax,12h
MOV AX,error_no_more_files
BadBye:
; MSDOS 6.0
AND byte [SS:FastOpenFlg],Fast_yes ;AN000;;FO. reset fastopen
STC
;call LCritDisk
;retn
; 18/12/2022
jmp LCritDisk
bad_path3:
;mov ax,3
MOV AX,error_path_not_found
JMP short BadBye
find_check_dev:
OR AH,AH
JNS short found_entry
MOV word [LASTENT],-1 ; Cause DOS_SEARCH_NEXT to fail
INC byte [FOUND_DEV] ; Tell DOS_RENAME we found a device
found_entry:
; We set the physical drive byte here Instead of after found_it; Doing
; a search-next may not have wfp_start set correctly
LES DI,[DMAADD]
MOV SI,[WFP_START] ; get pointer to beginning
LODSB
SUB AL,'A'-1 ; logical drive
STOSB ; High bit not set (local)
found_it:
LES DI,[DMAADD]
INC DI
; MSDOS 6.0
PUSH DS ;FO.;AN001; save ds
;test byte [FastOpenFlg],10h
TEST byte [FastOpenFlg],Set_For_Search ;FO.;AN001; from fastopen
JZ short notfast ;FO.;AN001;
MOV SI,BX ;FO.;AN001;
MOV DS,[CURBUF+2] ;FO.;AN001;
JMP SHORT movmov ;FO.;AN001;
notfast:
MOV SI,NAME1 ; find_buf 2 = formatted name
movmov:
; Special E5 code
MOVSB
CMP BYTE [ES:DI-1],5
JNZ short NOTKANJB
MOV BYTE [ES:DI-1],0E5H
NOTKANJB:
;MOV CX,10
;REP MOVSB
; 03/02/2024
mov cx,5
rep movsw
; 08/09/2018
POP DS ;FO.;AN001; restore ds
MOV AL,[ATTRIB]
STOSB
PUSH AX ; Save AH device info
MOV AX,[LASTENT]
STOSW
MOV AX,[DIRSTART]
STOSW
; 03/02/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMDOS.COM
;;

```

```

22182 000035D3 A1[DC0A]      MOV     AX,[DIRSTART_HW]
22183 000035D6 AB            STOSW
22184 000035D7 83C702        add     di,2
22185                        ; 4 bytes of 21 byte cont structure left for NET stuff
22186                        ;ADD     DI,4
22187                        ;;;
22188
22189 000035DA 58              POP     AX                ; Recover AH device info
22190 000035DB 08E4            OR      AH,AH
22191 000035DD 781B            JS      short DOSREL        ; Device entry is DOSGROUP relative
22192 000035DF 833E[E205]FF    CMP     WORD [CURBUF],-1
22193 000035E4 7510            JNZ     short OKSTORE
22194
22195                        ; MSDOS 6.0
22196 000035E6 F606[4611]10    TEST    byte [FastOpenFlg],Set_For_Search
22197                        ;AN000;;FO. from fastopen and is good
22198 000035EB 7509            JNZ     short OKSTORE        ;AN000;;FO.
22199
22200                        ; The user has specified the root directory itself, rather than some
22201                        ; contents of it. We can't "find" that.
22202
22203 000035ED 26C745F8FFFF    MOV     WORD [ES:DI-8],-1    ; Cause DOS_SEARCH_NEXT to fail by
22204                        ; stuffing a -1 at Lastent
22205 000035F3 E979FF            JMP     find_no_more
22206
22207 OKSTORE:
22208 000035F6 8E1E[E405]      MOV     DS,[CURBUF+2]
22209 DOSREL:
22210                        ; BX = offset NAME1 (from GETPATH)
22211 000035FA 89DE            MOV     SI,BX                ; SI-> start of entry
22212
22213                        ; NOTE: DOS_RENAME depends on BX not being altered after this point
22214
22215                        ;;mov     cx,32
22216                        ;MOV     CX,dir_entry.size
22217                        ; 03/02/2024
22218 000035FC B91000        mov     cx,dir_entry.size>>1
22219                        ;;;; 7/29/86
22220 000035FF 89F8            MOV     AX,DI                ; save the 1st byte addr
22221                        ;REP     MOVSB
22222 00003601 F3A5            rep     movsw
22223                        ;
22224 00003603 89C7            MOV     DI,AX                ; restore 1st byte addr
22225 00003605 26803D05      CMP     BYTE [ES:DI],05H    ; special char check
22226 00003609 7504            JNZ     short NO05
22227 0000360B 26C605E5      MOV     BYTE [ES:DI],0E5H  ; convert it back to E5
22228 NO05:
22229
22230                        ;;;; 7/29/86
22231
22232 ;hkn; FastOpenFlg is in DOSDATA use SS
22233 ; 16/12/2022
22234 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
22235 ; MSDOS 6.0
22236 ;AND     byte [SS:FastOpenFlg],Fast_yes ;AN000;;FO. reset fastopen
22237 ; 18/05/2019 - Retro DOS v4.0
22238 0000360F 16            push    ss
22239 00003610 1F            pop     ds
22240 ; 16/12/2022
22241 00003611 8026[4611]80    AND     byte [FastOpenFlg],Fast_yes ; 80h
22242
22243 ;hkn; SS is DOSDATA
22244 ;push    ss
22245 ;pop     ds
22246
22247 ; 27/06/2024
22248 ; cf=0
22249 ;CLC
22250
22251 ;call    LCritDisk
22252 ;retn
22253 ; 16/12/2022
22254 00003616 E9F6E2        jmp     LCritDisk
22255
22256 ;BREAK <DOS_SEARCH_NEXT - scan for subsequent matches>
22257 -----
22258 ;
22259 ; Procedure Name : DOS_SEARCH_NEXT
22260 ;
22261 ; Inputs:
22262 ; [DMAADD] Points to 53 byte buffer returned by DOS_SEARCH_FIRST
22263 ; (only first 21 bytes must have valid information)
22264 ; Function:
22265 ; Look for subsequent matches
22266 ; Outputs:
22267 ; CARRY CLEAR
22268 ; The 53 bytes at DMAADD are updated for next call
22269 ; (see DOS_SEARCH_FIRST)
22270 ; CARRY SET
22271 ; AX = error code
22272 ; error_no_more_files
22273 ; No more files to find
22274 ; DS preserved, others destroyed
22275 -----
22276
22277 ;hkn; called from search.asm. DS already set up at this point.
22278
22279 ; 03/02/2024 - Retro DOS v5.0
22280 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:75ECh
22281
22282 DOS_SEARCH_NEXT:
22283 00003619 C43E[2C03]      LES     DI,[DMAADD]        ; 24/01/2024 (Retro DOS v5-v4)
22284 0000361D 268A05        MOV     AL,[ES:DI]
22285 00003620 A880            TEST    AL,80H            ; Test for NET
22286 00003622 7406            JZ      short LOCAL_SEARCH_NEXT
22287 ;IF NOT Installed
22288 ; transfer NET_SEARCH_NEXT
22289 ;ELSE
22290 ;mov     ax,111Ch
22291 00003624 B81C11        MOV     AX,(MultNET<<8)|28
22292 00003627 CD2F            INT     2FH              ; Multiplex - NETWORK REDIRECTOR - FINDNEXT
22293                        ; SS = DS = DOS CS, [DTA] = 21-byte findfirst search data
22294                        ; Return: CF set on error, AX = DOS error code
22295                        ; CF clear if successful
22296 00003629 C3            retn
22297 ;ENDIF
22298
22299 LOCAL_SEARCH_NEXT:
22300 ;AL is drive A=1
22301 ;mov     byte [EXTERR_LOCUS],2
22302 0000362A C606[2303]02    MOV     byte [EXTERR_LOCUS],errLOC_Disk
22303 0000362F E8B0E2        call    ECritDisk
22304
22305 ;hkn; DummyCDS is in DOSDATA

```

```

22306 00003632 C706[A205][F304]      MOV     word [THISCDS],DUMMYCDS
22307                                ;hkn; Segment address is DOSDATA - use ds
22308                                ;hkn; MOV     WORD [THISCDS+2],CS
22309 00003638 8C1E[A405]      mov     [THISCDS+2],DS
22310
22311 0000363C 0440      ADD     AL,'A'-1
22312 0000363E E88F44      call    InitCDS
22313
22314                                ; call    GETTHISDRV          ; Set CDS pointer
22315
22316 00003641 7253      JC      short No_files          ; Bogus drive letter
22317 00003643 C43E[A205]      LES     DI,[THISCDS]          ; Get CDS pointer
22318                                ;les     bp,[es:di+45h]
22319 00003647 26C46D45      LES     BP,[ES:DI+curdir.devptr] ; Get DPB pointer
22320 0000364B E817D0      call    GOTDPB          ; [THISDPB] = ES:BP
22321
22322                                ; 16/12/2022
22323 0000364E 268A4600      mov     al,[ES:BP]
22324                                ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
22325                                ;mov     AL,[ES:BP+DPB.DRIVE] ; mov al,[ES:BP+0]
22326 00003652 A2[7605]      mov     [THISDRV],AL
22327                                ;mov     word [CREATING],0E500h
22328 00003655 C706[7E05]00E5      MOV     WORD [CREATING],(DIRFREE*256)+0
22329 0000365B C606[4C03]01      MOV     byte [NoSetDir],1          ; if we find a dir, don't change to it
22330 00003660 C536[2C03]      LDS     SI,[DMAADD]
22331 00003664 AC      LODSB          ; Drive Byte
22332
22333                                ;entry RENAME_NEXT          ; Entry used by DOS_RENAME
22334 RENAME_NEXT:
22335                                ;context ES
22336 00003665 16      push     ss
22337 00003666 07      pop      es          ; THIS BLOWS ES:BP POINTER TO DPB
22338
22339                                ;hkn; NAME1 is in DOSDATA
22340 00003667 BF[4B05]      MOV     DI,NAME1
22341
22342 0000366A B90B00      MOV     CX,11
22343 0000366D F3A4      REP     MOVSB          ; Search name
22344 0000366F AC      LODSB          ; Attribute
22345
22346                                ;hkn; SS override
22347 00003670 36A2[6B05]      MOV     [SS:ATTRIB],AL
22348 00003674 AD      LODSW          ; LastEnt
22349 00003675 09C0      OR      AX,AX
22350                                ; 03/02/2024
22351                                ;JNS     short cont_load
22352 00003677 781D      js      short No_files
22353 No_files:
22354                                ;JMP     find_no_more
22355
22356 cont_load:
22357 00003679 50      PUSH     AX          ; Save LastEnt
22358 0000367A AD      LODSW          ; DirStart
22359 0000367B 89C3      MOV     BX,AX
22360
22361                                ;;
22362                                ; 03/02/2024 - Retro DOS v5.0
22363                                ; (PCDOS 7.1 IBMDOS.COM)
22364 0000367D AD      lodsw          ; DIRSTART_HW
22365                                ;;
22366
22367                                ;hkn; SS is DOSDATA
22368                                ;context DS
22369 0000367E 16      push     ss
22370 0000367F 1F      pop      ds
22371 00003680 C42E[8A05]      LES     BP,[THISDPB]          ; Recover ES:BP
22372
22373                                ;;
22374                                ; 03/02/2024 - Retro DOS v5.0
22375                                ; (PCDOS 7.1 IBMDOS.COM)
22376
22377                                ;cmp     word [es:bp+0Fh],0
22378 00003684 26837E0F00      cmp     word [es:bp+DPB.FAT_SIZE],0
22379 00003689 7402      jz      short cont_load2 ; FAT32 fs
22380 0000368B 31C0      xor     ax,ax ; 0
22381 cont_load2:
22382 0000368D A3[EE0A]      mov     [ROOTCLUST_HW],ax          ; 0 or DIRSTART_HW
22383                                ;;
22384
22385                                ;invoke SetDirSrch
22386 00003690 E8C812      call    SETDIRSRCH
22387 00003693 7304      JNC     short SEARCH_GOON
22388 00003695 58      POP     AX          ; Clean stack
22389                                ;JMP     short No_files
22390                                ; 03/02/2024
22391 No_files:
22392 00003696 E9D6FE      JMP     find_no_more
22393
22394 SEARCH_GOON:
22395 00003699 E81317      call    STARTSRCH
22396 0000369C 58      POP     AX          ; Restore LastEnt
22397 0000369D E8FE11      call    GETENT
22398 000036A0 72F4      JC      short No_files
22399 000036A2 E8F210      call    NEXTENT
22400 000036A5 72EF      JC      short No_files
22401 000036A7 30E4      XOR     AH,AH          ; If Search_Next, can't be a DEV
22402 000036A9 E9EFFE      JMP     found_it ; 10/08/2018
22403
22404                                ; MSDOS 6.0
22405                                ;-----
22406                                ;
22407                                ; Procedure Name : CHECK_QUESTION
22408                                ;
22409                                ; Input: [WFP_START]= pointer to final path
22410                                ; Function: check '?' char
22411                                ; Output: carry clear, if no '?'
22412                                ; carry set, if '?' exists
22413                                ;-----
22414
22415                                ; 03/02/2024
22416 %if 0
22417                                ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
22418 CHECK_QUESTION:
22419 ;hkn; wfp_start is in DOSDATA;hkn; MOV     WORD PTR ThisCDS+2,CS
22420 ;hkn; PUSH     CS          ;AN000;;FO.
22421                                push     ss
22422                                pop      ds          ;AN000;;FO. ds:si -> final path
22423                                ; 16/12/2022
22424                                ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
22425                                MOV     SI,[WFP_START]          ;AN000;;FO.
22426                                ;mov     si,[ss:WFP_START]
22427 getnext:
22428                                LODSB          ;AN000;
22429                                OR      AL,AL          ;AN000;;FO. get char
22430                                ;AN000;;FO. is it null

```

```

22430          JZ      short NO_Question      ;AN000;;FO. yes
22431          CMP     AL,'?'                  ;AN000;;FO. is '?'
22432          JNZ     short getnext           ;AN000;;FO. no
22433          STC                                     ;AN000;;FO.
22434          NO_Question:                      ;AN000;
22435          retn                                ;AN000;;FO.
22436      %endif
22437
22438      ;=====
22439      ; ABORT.ASM, MSDOS 6.0, 1991
22440      ;=====
22441      ; 23/07/2018 - Retro DOS v3.0
22442      ; 18/05/2019 - Retro DOS v4.0
22443
22444      **
22445      ;
22446      ; Internal Abort call closes all handles and FCBS associated with a process.
22447      ; If process has NET resources a close all is sent out over the net.
22448      ;
22449      ; DOS_ABORT
22450      ;
22451      ; Modification history:
22452      ;
22453      ; Created: ARR 30 March 1983
22454      ;
22455      ; M038 SR 10/16/90 Free SFT with the PSP of the process
22456      ; being terminated only if it is busy.
22457      ;
22458
22459      ;Break <DOS_ABORT -- CLOSE all files for process>
22460      ;-----
22461      ;
22462      ; Procedure Name : DOS_ABORT
22463      ;
22464      ; Inputs:
22465      ; [CurrentPDB] set to PID of process aborting
22466      ; Function:
22467      ; Close all files and free all SFTs for this PID
22468      ; Returns:
22469      ; None
22470      ; All destroyed except stack
22471      ;-----
22472
22473      ; 03/02/2024 - Retro DOS v5.0
22474      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7685h
22475      ; (Win ME IO.SYS - BIOSCODE:74F4h)
22476
22477      DOS_ABORT:
22478      MOV     ES,[SS:CurrentPDB] ; SS override
22479      MOV     CX,[ES:PDB.JFN_Length] ; Number of JFNs
22480      reset_free_jfn:
22481      MOV     BX,CX
22482      PUSH    CX
22483      DEC     BX ; get jfn (start with last one)
22484
22485      CALL    _$CLOSE
22486      POP     CX
22487      LOOP    reset_free_jfn ; and do 'em all
22488
22489      ; Note: We do need to explicitly close FCBS. Reasons are as follows: If we
22490      ; are running in the no-sharing no-network environment, we are simulating the
22491      ; 2.0 world and thus if the user doesn't close the file, that is his problem
22492      ; BUT... the cache remains in a state with garbage that may be reused by the
22493      ; next process. We scan the set and blast the ref counts of the FCBS we own.
22494      ;
22495      ; If sharing is loaded, then the following call to close process will
22496      ; correctly close all FCBS. We will then need to walk the list AFTER here.
22497      ;
22498      ; Finally, the following call to NET_Abort will cause an EOP to be sent to all
22499      ; known network resources. These resources are then responsible for cleaning
22500      ; up after this process.
22501      ;
22502      ; sleazy, eh?
22503
22504      ;context DS ; SS is DOSDATA
22505      push    ss
22506      pop     ds ; 09/09/2018
22507
22508      ;CallInstall Net_Abort, MultNET, 29
22509      mov     ax,111Dh
22510      int     2Fh ; Multiplex - NETWORK REDIRECTOR
22511      ; ; - CLOSE ALL REMOTE FILES FOR PROCESS
22512      ; ; DS???, SS = DOS CS
22513
22514      ;if installed
22515      call    far [Jshare+(4*4)] ; 4 = MFTCloseP
22516      ;else
22517      call    MFTCloseP
22518      ;endif
22519
22520      ; Scan the FCB cache for guys that belong to this process and zap their ref
22521      ; counts.
22522      les     di,[ss:SFTFCB] ; SS override
22523      ; grab the pointer to the table
22524      ;;;
22525      ; 03/02/2024 - Retro DOS v5.0
22526      ; PCDOS 7.1 IBMDOS.COM
22527      SFTFCB_check:
22528      mov     cx,es
22529      or      cx,di
22530      jcxz    FCBScanDone
22531      push    di
22532      SFTFCB_OK:
22533      ;;;
22534      ;mov     cx,[es:di+4]
22535      mov     cx,[es:di+SFT.SFCount]
22536      ;jcxz    FCBScanDone
22537      ; 27/06/2024
22538      ;;;
22539      jcxz    SFTFCB_check2
22540      ;;;
22541      lea     di,[di+6]
22542      LEA     DI,[DI+SFT.SFTable] ; point at table
22543      mov     ax,[SS:PROC_ID] ; SS override
22544      FCBTest:
22545      ;cmp     [es:di+31h],ax
22546      cmp     [es:di+SF_ENTRY.sf_PID],ax ; is this one of ours
22547      jnz     short FCBNext ; no, skip it
22548
22549      ; 03/02/2024 - Retro DOS v5.0
22550      %if 0
22551      mov     word [es:di],0
22552      ;mov     word [es:di+SF_ENTRY.sf_ref_count],0 ; yes, blast ref count
22553      %else

```



```

22554             ; 03/02/2024
22555             ; PCDOS 7.1 IBMDOS.COM
22556 000036EA E8FA13 call     SFT_FREE
22557 %endif
22558
22559 FCBNext:
22560 add     di,SF_ENTRY.size ; 59 (for MSDOS 6.0)
22561 000036F0 E2F2     loop    FCBTest
22562
22563             ; 27/06/2024 (PCDOS 7.1 IBMDOS.COM)
22564             ;;;
22565 SFTFCB_check2:
22566 000036F2 5F     pop     di
22567 000036F3 26C43D les     di,[es:di]
22568 000036F6 83FFFF cmp     di,0FFFFh
22569 000036F9 75D5     jne     short SFTFCB_check
22570             ;;;
22571
22572 FCBScanDone:
22573
22574 ; walk the SFT to eliminate all busy SFT's for this process.
22575
22576 000036FB 31DB     xor     bx,bx
22577 Scan:
22578 000036FD 53     push    bx
22579 000036FE E8E33F call    SFFromSFN
22580 00003701 5B     pop     bx
22581             ;jnc     short Scan1
22582             ;retn
22583
22584             ; 18/12/2022
22585             ;jc      short NO_Question ; retn
22586             ; 03/02/2024
22587 00003702 7232     jc      short RET2
22588
22589 ;M038
22590 ; Do what the comment above says, check for busy state
22591
22592 Scan1:
22593             ;cmp     word [es:di],0
22594             ;jz      short scan_next ; MSDOS 3.3
22595             ; MSDOS 6.0
22596 00003704 26833DFF cmp     word [es:di],sf_busy ; -1
22597             ;cmp     word [es:di+SF_ENTRY.sf_ref_count],sf_busy
22598             ; Is Sft busy? ;M038
22599             jnz     short scan_next ; no
22600
22601 ; we have a SFT that is busy. See if it is for the current process
22602
22603 0000370A 36A1[3C03] mov     ax,[SS:PROC_ID] ; SS override
22604             ;cmp     [es:di+31h],ax
22605 0000370E 26394531 cmp     [es:di+SF_ENTRY.sf_PID],ax
22606 00003712 750D     jnz     short scan_next
22607 00003714 36A1[3E03] mov     ax,[SS:USER_ID] ; SS override
22608             ;sub     ax,[es:di+2Fh] ; PCDOS 7.1 IBMDOS.COM ; 03/02/2024
22609             ;cmp     [es:di+2Fh],ax
22610 00003718 2639452F cmp     [es:di+SF_ENTRY.sf_UID],ax
22611 0000371C 7503     jnz     short scan_next
22612
22613 ; This SFT is labelled as ours.
22614
22615 ; 03/02/2024 - Retro DOS v5.0
22616 %if 0
22617     mov     word [es:di],0
22618     ;mov     word [es:di+SF_ENTRY.sf_ref_count],0
22619 %else
22620     ; 03/02/2024
22621     ; PCDOS 7.1 IBMDOS.COM
22622 0000371E E8C613 call    SFT_FREE
22623 %endif
22624
22625 scan_next:
22626 00003721 43     inc     bx
22627 00003722 EBD9     jmp     short Scan
22628
22629 ;=====
22630 ; CLOSE.ASM, MSDOS 6.0, 1991
22631 ;=====
22632 ; 23/07/2018 - Retro DOS v3.0
22633 ; 18/05/2019 - Retro DOS v4.0
22634
22635 ;** Internal Close and Commit calls to close a local or NET SFT.
22636 ;
22637 ; DOS_CLOSE
22638 ; DOS_COMMIT
22639 ; FREE_SFT
22640 ; SetSFTTimes
22641 ;
22642 ; Revision history:
22643 ;
22644 ; AN000 version 4.00Jan. 1988
22645 ; A005 PTM 3718 --- lost clusters when fastopen installed
22646 ; A011 PTM 4766 --- C2 fastopen problem
22647
22648 ;Installed = TRUE
22649
22650 ;Break <DOS_CLOSE -- CLOSE FILE from SFT>
22651 ;-----
22652 ;
22653 ; Procedure Name : DOS_CLOSE
22654 ;
22655 ; Inputs:
22656 ; [THISSFT] set to the SFT for the file being used
22657 ; Function:
22658 ; Close the indicated file via the SFT
22659 ; Returns:
22660 ; sf_ref_count decremented otherwise
22661 ; ES:DI point to SFT
22662 ; Carry set if error
22663 ; AX has error code
22664 ; DS preserved, others destroyed
22665 ;-----
22666
22667 ;hkn; DOS_CLOSE called from fcbio.asm and handle.asm. DS already set up.
22668
22669 ; 18/05/2019 - Retro DOS v4.0
22670 ; DOSCODE:6E2Eh (MSDOS 6.21, MSDOS.SYS)
22671
22672 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
22673 ; DOSCODE:6E1Ah (MSDOS 5.0, MSDOS.SYS)
22674
22675 ; 23/07/2018 - IBMDOS.COM (MSDOS 3.3), 1987 - Offset 39D0h
22676
22677 ; 03/02/2024 - Retro DOS v5.0

```

```

22678          ; PC DOS 7.1 IBMDOS.COM - DOSCODE:76FEh
22679          ; (Win ME IO.SYS - BIOSCODE:7579h)
22680
22681 DOS_CLOSE:
22682 00003724 C43E[9E05]      LES     DI,[THISSFT]
22683                          ;mov     bx,[ES:DI+5]
22684 00003728 268B5D05        MOV     BX,[ES:DI+SF_ENTRY.sf_flags]
22685
22686          ; Network closes are handled entirely by the net code.
22687
22688          ;;test bx,8000h
22689          ;TEST  BX,sf_isnet
22690          ; 17/12/2022
22691          ;test  bh,80h
22692 0000372C F6C780          test    bh,(sf_isnet>>8)
22693 0000372F 7406            JZ      short LocalClose
22694
22695          ;CallInstall Net_Close,MultNET,6
22696 00003731 B80611          mov     ax,1106h
22697 00003734 CD2F          int     2Fh          ; Multiplex - NETWORK REDIRECTOR - CLOSE REMOTE FILE
22698                          ; ES:DI -> SFT
22699                          ; SFT DPB field -> DPB of drive containing file
22700                          ; Return: CF set on error, AX = DOS error code
22701                          ; CF clear if successful
22702                          ; 03/02/2024
22703 00003736 C3            RET2:   retn
22704
22705          ; All closes release the sharing information.
22706          ; No commit releases sharing information
22707          ;
22708          ; All closes decrement the ref count.
22709          ; No commit decrements the ref count.
22710
22711 LocalClose:
22712 00003737 E8A8E1          call    ECritDisk
22713 0000373A E8C001          CALL    SetSFTtimes
22714 0000373D E84801          CALL    FREE_SFT          ; dec ref count or mark as busy
22715
22716          ;hkn; SS is DOSDATA
22717          ;Context DS
22718 00003740 16            push    ss
22719 00003741 1F            pop     ds
22720
22721          push    ax
22722 00003743 53            push    bx
22723 00003744 E8384D          call    ShareEnd
22724 00003747 5B            pop     bx
22725 00003748 58            pop     ax
22726
22727          ; Commit enters here. AX from commit MUST be <> 1, BX is flags word
22728
22729 CloseEntry:
22730 00003749 50            PUSH    AX
22731
22732          ; File clean or device does not get stamped nor disk looked at.
22733
22734          ;test  bx,0C0h
22735          ; 17/12/2022
22736 0000374A F6C3C0          test    bl,devid_file_clean+devid_device
22737          ;TEST  BX,devid_file_clean+devid_device
22738 0000374D 7403            JZ      short rdird
22739          ; 14/11/2022
22740 0000374F E9FD00          JMP     FREE_SFT_OK          ; either clean or device
22741          ;jnz    short FREE_SFT_OK ; 24/07/2019
22742
22743          ; Retrieve the directory entry for the file
22744
22745          rdird:
22746 00003752 E84001          CALL    DirFromSFT
22747          ;mov     al,5
22748 00003755 B005            MOV     AL,error_access_denied
22749
22750          ; 03/02/2024
22751          ;JNC    short clook
22752          ;; 14/11/2022
22753          ;JMP    CloseFinish          ; pretend the close worked.
22754          ;;jc    short CloseFinish ; 24/07/2019
22755
22756          ;;;
22757          ; 03/02/2024 - Retro DOS v5.0
22758          ; (PCDOS 7.1 IBMDOS.COM)
22759 00003757 723D          jc      short jmp_to_CloseFinish ; pretend the close worked.
22760          rdird2:
22761          ;test    word [si+2],4
22762 00003759 F6440204        test    byte [si+SF_ENTRY.sf_mode],4 ; devid_device_null
22763          ;                                ; bit 2 - null device
22764 0000375D 751C            jnz     short clook
22765
22766          push    ds
22767 00003760 53            push    bx
22768          ; 27/06/2024
22769          ;lds     bx,[si+7]
22770 00003761 C55C07          lds     bx,[si+SF_ENTRY.sf_devptr] ; pointer to DPB
22771
22772          mov     bl,[bx]          ; DPB.DRIVE
22773 00003766 30FF          xor     bh,bh          ; 0
22774 00003768 36F687[0813]04    test    byte [ss:bx+drive_flags],4
22775          ;                                ; bit 2 - last access date/time flag?
22776          ;                                ; or disk accessed flag !?
22777 0000376E 5B            pop     bx
22778 0000376F 7409          jz      short skip_upd_laccdt ; no support for last access date&time
22779 00003771 1F            pop     ds
22780 00003772 1E            push    ds
22781 00003773 E8EAD3          call    DATE16
22782          ;mov     [es:di+12h],ax
22783 00003776 26894512        mov     [es:di+dir_entry.dir_1staccdt],ax
22784          skip_upd_laccdt:
22785 0000377A 1F            pop     ds
22786          ;;;
22787
22788          clook:
22789
22790          ; ES:DI points to entry
22791          ; DS:SI points to SFT
22792          ; ES:BX points to buffer header
22793
22794          push    di
22795 0000377C 56            push    si
22796          ;lea     si,[si+20h]
22797 0000377D 8D7420          LEA     SI,[SI+SF_ENTRY.sf_name]
22798
22799          ; ES:DI point to directory entry
22800          ; DS:SI point to unpacked name
22801

```



```

22802 00003780 E85AE0      call    XCHGP
22803
22804      ; ES:DI point to unpacked name
22805      ; DS:SI point to directory entry
22806
22807 00003783 E88F10      call    MetaCompare
22808 00003786 E854E0      call    XCHGP
22809 00003789 5E          pop     si
22810 0000378A 5F          pop     di
22811 0000378B 740C          JZ      short CLOSE_GO      ; Name OK
22812
22813 0000378D 89F7      Bye:    MOV     DI,SI
22814 0000378F 1E          PUSH    DS
22815 00003790 07          POP     ES      ; ES:DI points to SFT
22816 00003791 16          PUSH    SS
22817 00003792 1F          POP     DS
22818 00003793 F9          STC
22819      ;mov    al,2
22820 00003794 B002      MOV     AL,error_file_not_found
22821
22822      jmp_to_CloseFinish: ; 03/02/2024
22823      ;JMP     CloseFinish ; 24/07/2019
22824      ; 03/02/2024
22825      ; (PCDOS 7.1 IBMDOS.COM)
22826 00003796 E9DE00      jmp     CloseFinish2
22827
22828      ; 18/05/2019 - Retro DOS v4.0
22829      CLOSE_GO:
22830      ; 03/02/2024
22831      ;mov     al,[si+4]
22832 00003799 8A4404      mov     al,[si+SF_ENTRY.sf_attr]
22833
22834      ; MSDOS 6.0
22835      ;test    word [si+2],8000h
22836      ;TEST    word [SI+SF_ENTRY.sf_mode],sf_isFCB ; FCB ?
22837      ; 17/12/2022
22838      ;test    byte [si+3],80h
22839 0000379C F6440380      test    byte [SI+SF_ENTRY.sf_mode+1],(sf_isFCB>>8) ; FCB ?
22840 000037A0 740A          JZ      short nofcb      ; no, set dir attr, sf_attr
22841      ; MSDOS 3.3 & MSDOS 6.0
22842      ;mov     ch,[es:di+0Bh]
22843 000037A2 268A6D0B      MOV     CH,[ES:DI+dir_entry.dir_attr]
22844
22845      ; 03/02/2024
22846      ;mov     al,[si+4]
22847      ;MOV     AL,[SI+SF_ENTRY.sf_attr]
22848
22849      ;hkn; SS override
22850 000037A6 36A2[6B05]      MOV     [SS:ATTRIB],AL
22851      ; MSDOS 3.3
22852      ;call    MatchAttributes
22853      ;JNZ     short Bye      ; attributes do not match
22854      ; 18/05/2019
22855 000037AA EB04          JMP     SHORT setattr      ;FT.
22856      nofcb:
22857      ; 03/02/2024
22858      ; MSDOS 6.0
22859      ;mov     al,[si+4]
22860      ;MOV     AL,[SI+SF_ENTRY.sf_attr] ;FT.      ;AN000;
22861
22862 000037AC 2688450B      MOV     [ES:DI+dir_entry.dir_attr],AL ;FT.      ;AN000;
22863      setattr:
22864      ; MSDOS 3.3 (& MSDOS 6.0)
22865      ;or      byte [es:di+0Bh],20h
22866 000037B0 26804D0B20      OR      BYTE [ES:DI+dir_entry.dir_attr],attr_archive ;Set archive
22867      ; MSDOS 6.0
22868      ;mov     ax,[es:di+1Ah]
22869 000037B5 268B451A      MOV     AX,[ES:DI+dir_entry.dir_first] ;AN011
22870      ;F.O. save old first cluster
22871      ;hkn; SS override
22872 000037B9 36A3[BD0E]      MOV     [SS:OLD_FIRSTCLUS],AX ;AN011;F.O. save old first cluster
22873
22874      ;;;
22875      ; 03/02/2024
22876      ; PCDOS 7.1 IBMDOS.COM
22877      ;mov     ax,[es:di+14h]
22878 000037BD 268B4514      mov     ax,[ES:DI+dir_entry.dir_fclus_hi] ; old first cluster, hw
22879      ; 27/06/2024
22880 000037C1 36A3[3A11]      MOV     [ss:OLD_FIRSTCLUS_HW],ax
22881      ;;;
22882
22883      ;mov     ax,[si+0Bh]
22884      ;MOV     AX,[SI+SF_ENTRY.sf_firclus]
22885      ;;;
22886      ; 03/02/2024
22887      ; PCDOS 7.1 IBMDOS.COM
22888 000037C5 8B442B      mov     ax,[si+2Bh] ; mov ax,[si+SF_ENTRY.sf_chain]
22889      ; first cluster (32 bit) low word !
22890
22891      ;;;
22892 000037C8 2689451A      MOV     [ES:DI+dir_entry.dir_first],AX ;Set firclus pointer
22893      ;;; 03/02/2024
22894 000037CC 8B442D      mov     ax,[si+2Dh] ; mov ax,[si+SF_ENTRY.sf_chain+2]
22895      ; first cluster (32 bit) high word !
22896
22897 000037CF 26894514      ;mov     [es:di+14h],ax
22898      ;mov     [es:di+dir_entry.dir_fclus_hi],ax
22899      ;;;
22900      ; 03/02/2024
22901      %if 0
22902      ;mov     ax,[si+11h]
22903      MOV     AX,[SI+SF_ENTRY.sf_size]
22904      ;mov     [es:di+1Ch],ax
22905      MOV     [ES:DI+dir_entry.dir_size_l],AX ;Set size
22906      ;mov     ax,[si+13h]
22907      MOV     AX,[SI+SF_ENTRY.sf_size+2]
22908      ;mov     [es:di+1Eh],ax
22909      MOV     [ES:DI+dir_entry.dir_size_h],AX
22910      ;mov     ax,[si+0Fh]
22911      MOV     AX,[SI+SF_ENTRY.sf_date]
22912      ;mov     [es:di+18h],ax
22913      MOV     [ES:DI+dir_entry.dir_date],AX ;Set date
22914      ;mov     ax,[si+0Dh]
22915      MOV     AX,[SI+SF_ENTRY.sf_time]
22916      ;mov     [es:di+16h],ax
22917      MOV     [ES:DI+dir_entry.dir_time],AX ;Set time
22918      %else
22919      ; 03/02/2024 - Retro DOS v5.0
22920      push    si
22921 000037D4 83C60D      add     si,0Dh
22922 000037D7 AD          lodsw    [si+SF_ENTRY.sf_time]
22923 000037D8 26894516      mov     [es:di+dir_entry.dir_time],ax ; Set time
22924 000037DC AD          lodsw    [si+SF_ENTRY.sf_date]
22925 000037DD 26894518      mov     [es:di+dir_entry.dir_date],ax ; Set date

```

```

22926 000037E1 AD lodsw ; [si+SF_ENTRY.sf_size]
22927 000037E2 2689451C mov [es:di+dir_entry.dir_size_l],ax ; Set size
22928 000037E6 AD lodsw ; [si+SF_ENTRY.sf_size+2]
22929 000037E7 2689451E mov [es:di+dir_entry.dir_size_h],ax
22930 000037EB 5E pop si
22931 %endif
22932
22933 ; MSDOS 6.0
22934 ;; File Tagging
22935 000037EC 26F6470540 TEST byte [ES:BX+BUFFINFO.buf_flags],buf_dirty
22936 ;LB. if already dirty ;AN000;
22937 000037F1 7508 JNZ short yesdirty4 ;LB. don't increment dirty count ;AN000;
22938 ; 02/06/2019
22939 000037F3 E8CF33 call INC_DIRTY_COUNT ;LB. ;AN000;
22940 ; MSDOS 3.3 (& MSDOS 6.0)
22941 ;or byte [es:bx+5],40h
22942 000037F6 26804F0540 OR byte [ES:BX+BUFFINFO.buf_flags],buf_dirty ;Buffer dirty
22943 yesdirty4:
22944 000037FB 1E push ds
22945 000037FC 56 push si
22946
22947 ; 03/02/2024
22948 %if 0
22949 ; MSDOS 6.0
22950 ;mov cx,[si+0Bh]
22951 ; 07/12/2022
22952 MOV CX,[SI+SF_ENTRY.sf_firclus] ; do this for Fastopen
22953 %else
22954 ; 03/02/2024
22955 ; PCDOS 7.1
22956 000037FD 8B4C2B mov cx,[si+2Bh] ; [es:di+SF_ENTRY.sf_chain]
22957 ; first cluster (32 bit) low word !?
22958 %endif
22959
22960 ;hkn; SS override
22961 00003800 36A0[7605] MOV AL,[SS:THISDRV]
22962 ; MSDOS 3.3
22963 ;push ss
22964 ;pop ds
22965 ;MOV AL,[THISDRV]
22966 ;;; 10/1/86 update fastopen cache
22967 ; MSDOS 3.3 & MSDOS 6.0
22968 00003804 52 PUSH DX
22969 00003805 B400 MOV AH,0 ; dir entry update
22970 00003807 88C2 MOV DL,AL ; drive number A=0, B=1,,,
22971 ;;;
22972 ; 03/02/2024 - Retro DOS v5.0
22973 ; PCDOS 7.1
22974 00003809 53 push bx ; *
22975 0000380A 8B5C2D mov bx,[si+2Dh] ; [es:di+SF_ENTRY.sf_chain+2]
22976 ; first cluster (32 bit) high word !?
22977 0000380D 09DB or bx,bx
22978 0000380F 7510 jnz short do_update2
22979 ;;;
22980 ; MSDOS 6.0
22981 00003811 09C9 OR CX,CX ;AN005; first cluster 0; may be truncated
22982 00003813 750C JNZ short do_update2 ;AN005; no, do update
22983 00003815 B403 MOV AH,3 ;AN005; do a delete cache entry
22984 ; 27/06/2024
22985 ;;;
22986 ;mov di,[si+1Bh]
22987 ;MOV DI,[SI+SF_ENTRY.sf_dirsec] ;AN005; cx:di = dir sector
22988 ;mov cx,[si+1Dh]
22989 ;MOV CX,[SI+SF_ENTRY.sf_dirsec+2] ;AN005;
22990 00003817 C57C1B lds di,[si+SF_ENTRY.sf_dirsec]
22991 0000381A 8CD9 mov cx,ds
22992 ;;;
22993 ;mov dh,[si+1Fh]
22994 0000381C 8A741F MOV DH,[SI+SF_ENTRY.sf_dirpos] ;AN005; dh = dir pos
22995 0000381F EB1A JMP SHORT do_update ;AN011;F.O.
22996
22997 do_update2: ;AN011;F.O.
22998 ;;;
22999 ; 03/02/2024
23000 ; PCDOS 7.1
23001 00003821 363B1E[3A11] cmp bx,[ss:OLD_FIRSTCLUS_HW] ; same as old first cluster?
23002 00003826 7507 jnz short do_update3 ; no
23003 ;;;
23004
23005 ;hkn; SS override fort OLD_FIRSTCLUS
23006 ;
23007 00003828 363B0E[BD0E] CMP CX,[SS:OLD_FIRSTCLUS] ;AN011;F.O. same as old first cluster?
23008 0000382D 740C JZ short do_update ;AN011;F.O. yes
23009 do_update3: ; 03/02/2024
23010 0000382F B402 MOV AH,2 ;AN011;F.O. delete the old entry
23011 00003831 368B0E[BD0E] MOV CX,[SS:OLD_FIRSTCLUS] ;AN011;F.O.
23012 ;;;
23013 ; 03/02/2024
23014 ; PCDOS 7.1
23015 00003836 368B1E[3A11] mov bx,[ss:OLD_FIRSTCLUS_HW]
23016 ;;;
23017 do_update: ;AN005;
23018 ;hkn; SS is DOSDATA
23019 ;Context DS
23020 0000383B 16 push ss
23021 0000383C 1F pop ds
23022 ;;;
23023 ; 03/02/2024 - Retro DOS v5.0
23024 0000383D 87DE xchg bx,si ; PCDOS 7.1 IBMDOS.COM
23025 ;;;
23026 ; MSDOS 3.3 & MSDOS 6.0
23027 0000383F E824F5 call FastOpen_Update ; invoke fastopen
23028 ;;;
23029 ; 03/02/2024
23030 00003842 87DE xchg bx,si ; PCDOS 7.1 IBMDOS.COM
23031 00003844 5B pop bx ; *
23032 ;;;
23033 00003845 5A POP DX
23034
23035 ;;; 10/1/86 update fastopen cache
23036 00003846 E84A32 call FLUSHBUF ; flush all relevant buffers
23037 00003849 5F pop di
23038 0000384A 07 pop es
23039 ;mov al,5
23040 0000384B B005 MOV AL,error_access_denied
23041 0000384D 7215 JC short CloseFinish
23042 FREE_SFT_OK:
23043 ; 03/02/2024
23044 ;CLC ; signal no error.
23045
23046 ;;;
23047 ; 03/02/2024 - Retro DOS v5.0
23048 ; PCDOS 7.1 IBMDOS.COM
23049 ;test word [es:di+5],8080h

```

```

23050 0000384F 26F745058080      test    word [es:di+SF_ENTRY.sf_flags],sf_isnet+devide_device
23051 00003855 7520          jnz     short CloseFinish2
23052 00003857 06          push    es
23053 00003858 55          push    bp
23054          ;les    bp,[es:di+7]
23055 00003859 26C46D07      les     bp,[es:di+SF_ENTRY.sf_devptr] ; DPB
23056 0000385D E808FC      call   update_fat32_fsinfo
23057 00003860 5D          pop     bp
23058 00003861 07          pop     es
23059 00003862 EB13          jmp     short CloseFinish2
23060
23061 CloseFinish:      ; 03/02/2024 - Retro DOS v5.0
23062 00003864 1E          push    ds
23063 00003865 53          push    bx
23064 00003866 26C55D07      lds     bx,[es:di+7]          ; [es:di+SF_ENTRY.sf_devptr] ; DPB
23065 0000386A 8A1F          mov     bl,[bx]              ; DPB.DRIVE
23066 0000386C 30FF          xor     bh,bh ; 0
23067 0000386E 3680A7[0813]FB and     byte [ss:bx+drive_flags],0FBh ; clear bit 2
23068          ; bit 2 - last access date/time flag?
23069          ; or disk accessed (successful) flag !?
23070          pop     bx
23071 00003875 1F          pop     ds
23072 00003876 F9          stc
23073          ;;;
23074
23075 CloseFinish2:      ; 03/02/2024 - Retro DOS v5.0
23076 ;Closefinish:
23077
23078 ; Indicate to the device that the SFT is being closed.
23079
23080 ;;;; 7/21/86
23081 00003877 9C          PUSHF
23082 00003878 E89D19      call   DEV_CLOSE_SFT
23083 0000387B 9D          POPF
23084 ;;;; 7/21/86
23085 ;
23086 ; See if the ref count indicates that we have busied the SFT. If so, mark the
23087 ; SFT as being free. Note that we do NOT need to be in critSFT as we are ONLY
23088 ; going to be moving from busy to free.
23089 ;
23090 0000387C 59          POP     CX
23091 0000387D 9C          PUSHF
23092          ; 03/02/2024
23093          ;DEC    CX          ; if cx != 1
23094          ;JNZ    short NoFree      ; then do NOT free SFT
23095 0000387E E203      loop   NoFree ; PCDOS 7.1 IBMDOS.COM
23096
23097 ; 03/02/2024 - Retro DOS v5.0
23098 %if 0
23099     mov     [es:di],cx
23100     ;MOV    [ES:DI+SF_ENTRY.sf_ref_Count],CX ; mov [es:di+0],cx
23101 %else
23102     ; 03/02/2024
23103     ; PCDOS 7.1 IBMDOS.COM
23104 00003880 E86412      call   SFT_FREE
23105 %endif
23106
23107 NoFree:
23108     call   LCritDisk
23109     POPF
23110     retn
23111
23112 ;-----
23113 ;
23114 ; Procedure Name : FREE_SFT
23115 ;
23116 ; ES:DI -> SFT. Decs sft_ref_count. If the count goes to 0, mark it as busy.
23117 ; Flags preserved. Return old ref count in AX
23118 ;
23119 ; Note that busy is indicated by the SFT ref count being -1.
23120 ;
23121 ;-----
23122
23123 FREE_SFT:
23124 00003888 9C          PUSHF
23125 00003889 268B05      mov     ax,[es:di]
23126          ;MOV    AX,[ES:DI+SF_ENTRY.sf_ref_count]
23127 0000388C 48          DEC     AX
23128 0000388D 7501      JNZ     short SetCount
23129 0000388F 48          DEC     AX
23130
23131 SetCount:
23132     xchg     ax,[es:di]
23133     ;XCHG    AX,[ES:DI+SF_ENTRY.sf_ref_count]
23134     POPF
23135     retn
23136
23137 ; 18/05/2019 - Retro DOS v4.0
23138
23139 ;-----
23140 ;
23141 ; Procedure Name : DirFromSFT
23142 ;
23143 ; DirFromSFT - locate a directory entry given an SFT.
23144 ;
23145 ; Inputs: ES:DI point to SFT
23146 ;         DS = DOSDATA
23147 ; Outputs:
23148 ;         EXTERR_LOCUS = errLOC_Disk
23149 ;         CurBuf points to buffer
23150 ;         Carry Clear -> operation OK
23151 ;         ES:DI point to entry
23152 ;         ES:BX point to buffer
23153 ;         DS:SI point to SFT
23154 ;         Carry SET -> operation failed
23155 ;         registers trashified
23156 ; Registers modified: ALL
23157 ;-----
23158
23159 ; 04/02/2024 - Retro DOS v5.0
23160 ; PCDOS 7.1 IBMDOS.COM
23161
23162 DirFromSFT:
23163     ;mov     byte [EXTERR_LOCUS],2
23164     ;MOV     byte [EXTERR_LOCUS],errLOC_Disk
23165 00003895 E84EDA      call   set_exerr_locus_disk
23166
23167     push     es
23168     push     di
23169     ; MSDOS 3.3
23170     ;mov     dx,[es:di+1Dh]
23171     ;MOV     dx,[ES:DI+SF_ENTRY.sf_dirsec]
23172     ; MSDOS 6.0
23173     ;mov     dx,[es:[di+1Dh]]

```

```

23174 0000389A 268B551D      MOV     DX,[ES:DI+SF_ENTRY.sf_dirsec+2] ;F.C. >32mb
23175 0000389E 8916[0706]      MOV     [HIGH_SECTOR],DX ;F.C. >32mb
23176                                ; 04/02/2024
23177 000038A2 52                push    dx
23178                                ;mov     dx,[es:di+1Bh]
23179 000038A3 268B551B      MOV     DX,[ES:DI+SF_ENTRY.sf_dirsec]
23180                                ; 04/02/2024
23181                                ; 19/05/2019
23182                                ;PUSH    word [HIGH_SECTOR] ;F.C. >32mb
23183                                ; MSDOS 3.3 & MSDOS 6.0
23184 000038A7 52                PUSH    DX
23185 000038A8 E89A2C      call    FATREAD_SFT ; ES:BP points to DPB, [THISDRV] set
23186                                ; [THISDPB] set
23187 000038AB 5A                POP     DX
23188 000038AC 8F06[0706]      POP     word [HIGH_SECTOR] ;F.C. >32mb
23189 000038B0 721E      JC      short PopDone
23190                                ; 22/09/2023
23191                                ;XOR     AL,AL ; * ; Pre read
23192                                ;;mov     byte [ALLOWED],18h
23193                                ;MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
23194                                ;call    GETBUFR
23195                                ; 22/09/2023
23196 000038B2 E86F30      call    GETBUFFER ; * ; Pre read
23197 000038B5 7219      JC      short PopDone
23198 000038B7 5E                pop     si
23199 000038B8 1F                pop     ds ; Get back SFT pointer
23200
23201                                ;hkn; SS override
23202 000038B9 36C43E[E205]      LES     DI,[SS:CURBUF]
23203                                ;or      byte [es:di+5],4
23204 000038BE 26804D0504      OR      byte [ES:DI+BUFFINFO.buf_flags],buf_isDIR
23205 000038C3 89FB      MOV     BX,DI ; ES:BX point to buffer header
23206                                ;;lea     di,[di+16] ; MSDOS 3.3
23207                                ;;lea     di,[di+20] ; MSDOS 6.0
23208                                ; 04/02/2024
23209                                ;lea     di,[di+24] ; MSDOS 7.1
23210 000038C5 8D7D18      LEA     DI,[DI+BUFINSZ] ; Point to buffer
23211                                ;mov     al,32
23212 000038C8 B020      MOV     AL,dir_entry.size
23213                                ;mul     byte [si+1Fh] ; MSDOS 6.0
23214 000038CA F6641F      MUL     byte [SI+SF_ENTRY.sf_dirpos]
23215 000038CD 01C7      ADD     DI,AX ; Point at the entry
23216 000038CF C3                retn ; carry is clear
23217
23218 000038D0 5F                PopDone: pop     di
23219 000038D1 07                pop     es
23220
23221 000038D2 C3                PopDone_retn: retn
23222
23223                                ;-----
23224                                ;
23225                                ;** DOS_Commit - Update Directory Entries
23226                                ;
23227                                ; ENTRY same as DOS_CLOSE (??? BUGBUG - update this jgl)
23228                                ; (DS) = DOSGROUP
23229                                ; EXIT Same as DOS_CLOSE except ref_count field is not altered
23230                                ; USES all but DS
23231                                ;-----
23232
23233                                ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
23234                                ; DOSCODE:6F72h (MSDOS 5.0, MSDOS.SYS)
23235
23236                                ; 04/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
23237                                ; DOSCODE:78B8h (PCDOS 7.1 IBMDOS.COM)
23238
23239                                DOS_COMMIT:
23240                                ;hkn; called from srvcall. DS already set up.
23241                                LES     DI,[THISST]
23242 000038D3 C43E[9E05]      ;mov     bx,[es:di+5]
23243                                MOV     BX,[ES:DI+SF_ENTRY.sf_flags]
23244 000038D7 268B5D05      ;test    bx,0C0h
23245                                ; 17/12/2022
23246                                test    bl,devid_file_clean+devid_device ;Clears carry
23247 000038DB F6C3C0      ;TEST    BX,devid_file_clean+devid_device ;Clears carry
23248                                jnz     short PopDone_retn
23249 000038DE 75F2      ;test    bx,8000h
23250                                ; 17/12/2022
23251                                ;test    bh,80h
23252                                test    bh,(sf_isnet>>8) ; 80h
23253 000038E0 F6C780      ;TEST    BX,sf_isnet ; 8000h
23254                                JZ      short LOCAL_COMMIT
23255 000038E3 7406      ;
23256
23257                                ;IF NOT Installed
23258                                ; transfer NET_COMMIT
23259                                ;ELSE
23260                                ;mov     ax,1107h
23261 000038E5 B80711      MOV     AX,(MultNET<<8)|7
23262 000038E8 CD2F      int     2Fh ; Multiplex - NETWORK REDIRECTOR - COMMIT REMOTE FILE
23263                                ; ES:DI -> SFT
23264                                ; SFT DPB field -> DPB of drive containing file
23265                                ; Return: CF set on error, AX = DOS error code
23266                                ; CF clear if successful
23267                                localcommit_retn: ; 18/12/2022
23268 000038EA C3                retn
23269                                ;ENDIF
23270
23271                                ; Perform local commit operation by doing a close but not releaseing the SFT.
23272                                ; There are three ways we can do this. One is to enter a critical section to
23273                                ; protect a potential free. The second is to increment the ref count to mask
23274                                ; the close decrementing.
23275                                ;
23276                                ; The proper way is to let the caller's of close decide if a decrement should
23277                                ; be done. we do this by providing another entry into close after the
23278                                ; decrement and after the share information release.
23279
23280                                ; DOSCODE:6FA0h (MSDOS 6.21, MSDOS.SYS)
23281                                ; DOSCODE:6F8Ch (MSDOS 5.0, MSDOS.SYS)
23282                                ; 06/08/2025
23283                                ; DOSCODE:78D0h (PCDOS 7.1, IBMDOS.COM)
23284
23285                                LOCAL_COMMIT:
23286                                ; 06/08/2025
23287                                ;call    ECritDisk
23288                                ; MSDOS 6.0
23289 000038EB E8F4DF      call    ECritDisk ;PTM.
23290 000038EE E80C00      call    SetSFTTimes
23291 000038F1 B8FFFF      MOV     AX,-1 ; 0FFFFh
23292 000038F4 E852FE      call    CloseEntry
23293                                ; MSDOS 6.0
23294 000038F7 9C                PUSHF ;PTM. ;AN000;
23295 000038F8 E81519      call    DEV_OPEN_SFT ;PTM. increment device count ;AN000;
23296                                ; 06/08/2025
23297                                ;POPF ;PTM. ;AN000;

```

```

23298             ;call LCritDisk          ;PTM.                      ;AN000;
23299             ; 18/12/2022
23300             ;jmp LCritDisk
23301             ; 06/08/2025
23302 000038FB EB86             jmp NoFree
23303             ;localcommit_retn:
23304             ; retn
23305
23306             ;Break <SetSFTTimes - signal a change in the times for an SFT>
23307             ;-----
23308             ;
23309             ; Procedure Name : SetSFTTimes
23310             ;
23311             ; SetSFTTimes - Examine the flags for a SFT and set the time appropriately.
23312             ; Reflect these times in other SFT's for the same file.
23313             ;
23314             ; Inputs: ES:DI point to SFT
23315             ; BX = sf_flags set appropriately
23316             ; Outputs: Set sft times to current time if File & dirty & !nodate
23317             ; Registers modified: All except ES:DI, BX, AX
23318             ;
23319             ;-----
23320
23321             ; 04/02/2024 - Retro DOS v5.0
23322             ; PCDOS 7.1 IBMDOS.COM
23323
23324 SetSFTTimes:
23325
23326             ; 04/02/2024
23327             %if 0
23328             ; File clean or device does not get stamped nor disk looked at.
23329
23330             ;test bx,0C0h
23331             ; 17/12/2022
23332             test bl,devid_file_clean+devid_device
23333             ;TEST BX,devid_file_clean+devid_device
23334             ;retnz ; clean or device => no timestamp
23335             jnz short localcommit_retn
23336
23337             ; file and dirty. See if date is good
23338
23339             ;test bx,4000h
23340             ; 17/12/2022
23341             ;test bh,40h
23342             test bh,(sf_close_nodate>>8)
23343             ;TEST BX,sf_close_nodate
23344             ;retnz ; nodate => no timestamp
23345             jnz short localcommit_retn
23346             %else
23347             ; 04/02/2024
23348             ; (PCDOS 7.1 IBMDOS.COM)
23349             ;test bx,40C0h
23350 000038FD F7C3C040             test bx,sf_close_nodate+devid_file_clean+devid_device
23351 00003901 75E7             jnz short localcommit_retn
23352             %endif
23353
23354             push ax
23355             push bx
23356 00003905 E858D2             call DATE16 ; Date/Time to AX/DX
23357             ;mov [es:di+0Fh],ax
23358 00003908 2689450F             MOV [ES:DI+SF_ENTRY.sf_date],AX
23359             ;mov [es:di+0Dh],dx
23360 0000390C 2689550D             MOV [ES:DI+SF_ENTRY.sf_time],DX
23361 00003910 31C0             XOR AX,AX
23362             ;if installed
23363             ;call JShare + 14 * 4
23364 00003912 FF1E[C800]             call far [JShare+(14*4)] ; 14 = ShSU
23365             ;else
23366             ; call ShSU
23367             ;endif
23368 00003916 5B             pop bx
23369 00003917 58             pop ax
23370 00003918 C3             retn
23371
23372             ;=====
23373             ; DIRCALL.ASM, MSDOS 6.0, 1991
23374             ;=====
23375             ; 23/07/2018 - Retro DOS v3.0
23376             ; 18/05/2019 - Retro DOS v4.0
23377
23378             ; DOSCODE:6FDAh (MSDOS 6.21, MSDOS.SYS)
23379
23380             ;TITLE DIRCALL - Directory manipulation internal calls
23381             ;NAME DIRCALL
23382
23383             ;** Low level directory manipulation routines for making removing and
23384             ; verifying local or NET directories
23385             ;
23386             ; DOS_MKDIR
23387             ; DOS_CHDIR
23388             ; DOS_RMDIR
23389             ;
23390             ; Modification history:
23391             ;
23392             ; Created: ARR 30 March 1983
23393
23394             ;BREAK <DOS_MkDir - Make a directory entry>
23395             ;-----
23396             ;
23397             ; Procedure Name : DOS_MkDir
23398             ;
23399             ; Inputs:
23400             ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
23401             ; terminated)
23402             ; [CURR_DIR_END] Points to end of Current dir part of string
23403             ; (= -1 if current dir not involved, else
23404             ; Points to first char after last "/" of current dir part)
23405             ; [THISCDS] Points to CDS being used
23406             ; (Low word = -1 if NUL CDS (Net direct request))
23407             ; Function:
23408             ; Make a new directory
23409             ; Returns:
23410             ; Carry Clear
23411             ; No error
23412             ; Carry Set
23413             ; AX is error code
23414             ; error_path_not_found
23415             ; Bad path (not in curr dir part if present)
23416             ; error_bad_curr_dir
23417             ; Bad path in current directory part of path
23418             ; error_access_denied
23419             ; Already exists, device name
23420             ; DS preserved, Others destroyed
23421             ;-----

```

```

23422
23423 ;hkn; called from path.asm. DS already set up.
23424
23425 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
23426 ; DOSCODE:6FC6h (MSDOS 5.0, MSDOS.SYS)
23427
23428 ; 04/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
23429 ; DOSCODE:78FFh (PCDOS 7.1, IBMDOS.COM)
23430
23431 DOS_MKDIR:
23432 call TestNet
23433 JNC short LOCAL_MKDIR
23434
23435 ;IF NOT Installed
23436 ; transfer NET_MKDIR
23437 ;ELSE
23438 ;mov ax,1103h
23439 MOV AX,(MultNet<<8)|3
23440 int 2Fh ; Multiplex - NETWORK REDIRECTOR - MAKE REMOTE DIRECTORY
23441 ; SS = DOS CS
23442 ; SDA first filename pointer -> fully-qualified directory name
23443 ; SDA CDS pointer -> current directory
23444 ; Return: CF set on error, AX = DOS error code
23445 ; CF clear if successful
23446 retn
23447 ;ENDIF
23448
23449 NODEACCERRJ:
23450 ;mov ax,5
23451 MOV AX,error_access_denied
23452 _BadRet:
23453 STC
23454 ;call LCritDisk
23455 ;retn
23456 ; 18/12/2022
23457 jmp LCritDisk
23458
23459 PATHNFJ:
23460 call LCritDisk
23461 jmp SET_MKND_ERR ; Map the MakeNode error and return
23462
23463 LOCAL_MKDIR:
23464 call ECritDisk
23465
23466 ; MakeNode requires an SFT to fiddle with. we use a temp spot (RENBUFF)
23467
23468 MOV [THISST+2],SS
23469
23470 ;hkn; DOSDATA
23471 MOV WORD [THISST],RENBUFF
23472
23473 ; NOTE: Need WORD PTR because MASM takes type of
23474 ; TempSFT (byte) instead of type of sf_mft (word).
23475
23476 ;mov word [RENBUFF+33h],0 ; MSDOS 6.0
23477 MOV WORD [RENBUFF+SF_ENTRY.sf_MFT],0
23478 ; make sure SHARER won't complain.
23479
23480 ;mov al,10h
23481 MOV AL,attr_directory
23482 call MakeNode
23483 JC short PATHNFJ
23484 CMP AX,3
23485 JZ short NODEACCERRJ ; Can't make a device into a directory
23486 LES BP,[THISDPB] ; Makenode zaps this
23487 LDS DI,[CURBUF]
23488 SUB SI,DI
23489 PUSH SI ; Pointer to dir_first
23490
23491 ; 04/02/2024
23492 %if 0
23493 ; MSDOS 6.0
23494 ;push word [DI+8]
23495 PUSH WORD [DI+BUFFINFO.buf_sector+2] ; F.C. >32mb
23496 ; MSDOS 3.3 & MSDOS 6.0
23497 ;push word [di+6]
23498 PUSH WORD [DI+BUFFINFO.buf_sector] ; Sector of new node
23499 %else
23500 ; 04/02/2024
23501 ; (PCDOS 7.1 IBMDOS.COM)
23502 lds ax,[di+BUFFINFO.buf_sector] ; Sector of new node
23503 push ds
23504 push ax
23505 %endif
23506
23507 push ss
23508 pop ds
23509
23510 ; 04/02/2024
23511 ;PUSH word [DIRSTART] ; Parent for .. entry
23512 XOR AX,AX
23513 ;MOV [DIRSTART],AX ; Null directory
23514 ;;;
23515 ; Retro DOS v5.0
23516 ; PCDOS 7.1 IBMDOS.COM
23517 mov dx,ax ; 0
23518 xchg dx,[DIRSTART_HW] ; Null directory
23519 xchg ax,[DIRSTART]
23520 ;
23521 ;cmp word [es:bp+0Fh],0
23522 cmp word [es:bp+DPB.FAT_SIZE],0
23523 jnz short LOCAL_MKDIR_cont ; not FAT32
23524 ;cmp [es:bp+35h],ax
23525 cmp [es:bp+DPB.ROOT_CLUSTER],ax
23526 jne short LOCAL_MKDIR_cont
23527 ;cmp [es:bp+37h],dx
23528 cmp [es:bp+DPB.ROOT_CLUSTER+2],dx
23529 jne short LOCAL_MKDIR_cont
23530 ;
23531 xor dx,dx
23532 xor ax,ax
23533 ; dx:ax = 0 for root directory
23534 LOCAL_MKDIR_cont:
23535 push dx
23536 ;;;
23537 push ax
23538 call NEWDIR
23539 JC short NODEEXISTSPOPDEL ; No room
23540 call GETENT ; First entry
23541 JC short NODEEXISTSPOPDEL ; Screw up
23542 DI,[CURBUF]
23543 LES
23544
23545 ; 04/02/2024
23546 %if 0

```



```

23546 ; MSDOS 6.0
23547 TEST byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
23548 ;LB. if already dirty ;AN000;
23549 JNZ short yesdirty5 ;LB. don't increment dirty count ;AN000;
23550 call INC_DIRTY_COUNT ;LB. ;AN000;
23551
23552 ; MSDOS 3.3 & MSDOS 6.0
23553 ;or byte [es:di+5],40h ; 07/12/2022
23554 OR byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
23555 %else
23556 ; 04/02/2024
23557 ; (PCDOS 7.1 IBMDOS.COM)
23558 00003995 E82132 call SET_BUF_DIRTY
23559 %endif
23560
23561 yesdirty5:
23562 ;;;add di,16 ; MSDOS 3.3
23563 ;;;add di,20 ; MSDOS 6.0
23564 ; 04/02/2024
23565 ;add di,24 ; PCDOS 7.1
23566 00003998 83C718 ADD DI,BUFINSIZ ; Point at buffer
23567 0000399B B82E20 MOV AX,202EH ; ". "
23568 ;;;
23569 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23570 0000399E 8B16[DC0A] mov dx,[DIRSTART_HW]
23571 000039A2 8916[F20A] mov [CLUSTERS_HW],dx
23572 ;;;
23573 000039A6 8B16[C205] MOV DX,[DIRSTART] ; Point at itself
23574 000039AA E8831A call SETDOTENT
23575 000039AD B82E2E MOV AX,2E2EH ; ". "
23576 000039B0 5A POP DX ; Parent
23577 ;;;
23578 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23579 000039B1 8F06[F20A] pop word [CLUSTERS_HW]
23580 ;;;
23581 000039B5 E8781A call SETDOTENT
23582 000039B8 C42E[8A05] LES BP,[THISDPB]
23583 ; 22/09/2023
23584 ;;;mov byte [ALLOWED],18h
23585 ;MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
23586 000039BC 5A POP DX ; Entry sector
23587 ; MSDOS 6.0
23588 000039BD 8F06[0706] POP word [HIGH_SECTOR] ;F.C. >32mb
23589
23590 ;XOR AL,AL ; * ; Pre read
23591 ;call GETBUFR
23592 ; 22/09/2023
23593 000039C1 E8602F call GETBUFFER ; * ; Pre read
23594 000039C4 7265 JC short NODEEXISTSP
23595 ;;;
23596 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23597 000039C6 A1[DC0A] mov ax,[DIRSTART_HW]
23598 ;;;
23599 000039C9 8B16[C205] MOV DX,[DIRSTART]
23600 000039CD C53E[E205] LDS DI,[CURBUF]
23601 ;or byte [di+5],4
23602 000039D1 804D0504 OR byte [DI+BUFFINFO.buf_flags],buf_isDIR
23603 000039D5 5E POP SI ; dir_first pointer
23604 000039D6 01FE ADD SI,DI
23605 000039D8 8914 MOV [SI],DX ; dir_entry.dir_first
23606 ;;;
23607 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23608 000039DA 8944FA mov [si-6],ax ; dir_entry.dir_fclus_hi
23609 ;;;
23610
23611 000039DD 31D2 XOR DX,DX
23612 000039DF 895402 MOV [SI+2],DX ; Zero size
23613 000039E2 895404 MOV [SI+4],DX
23614
23615 DIRUP:
23616 ; MSDOS 6.0
23617 TEST byte [DI+BUFFINFO.buf_flags],buf_dirty
23618 ;LB. if already dirty ;AN000;
23619 JNZ short yesdirty6 ;LB. don't increment dirty count ;AN000;
23620 call INC_DIRTY_COUNT ;LB. ;AN000;
23621
23622 ; MSDOS 3.3 & MSDOS 6.0
23623 ;or byte [di+5],40h
23624 OR byte [DI+BUFFINFO.buf_flags],buf_dirty ; Dirty buffer
23625 yesdirty6:
23626 push ss
23627 pop ds
23628 ;;;
23629 000039F4 E871FA call update_fat32_fsinfo
23630 ;;;
23631 000039F7 268A4600 mov al,[es:bp]
23632 ;MOV AL,[ES:BP+DPB.DRIVE] ; mov al,[es:bp+0]
23633 000039FB E89530 call FLUSHBUF
23634 ;mov ax,5
23635 000039FE B80500 MOV AX,error_access_denied
23636 ;call LCritDisk
23637 ;retn
23638 ; 18/12/2022
23639 00003A01 E90BDF jmp LCritDisk
23640
23641 NODEEXISTSPOPDEL:
23642 ;;;
23643 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23644 00003A04 5A pop dx ; Parent
23645 ;;;
23646 00003A05 5A POP DX ; Parent
23647 00003A06 5A POP DX ; Entry sector
23648 ; MSDOS 6.0
23649 00003A07 8F06[0706] POP word [HIGH_SECTOR] ; F.C. >32mb
23650 00003A0B C42E[8A05] LES BP,[THISDPB]
23651 ; 22/09/2023
23652 ;;;mov byte [ALLOWED],18h
23653 ;MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
23654 ;XOR AL,AL ; * ; Pre read
23655 ;call GETBUFR
23656 ; 22/09/2023
23657 00003A0F E8122F call GETBUFFER ; * ; Pre read
23658 00003A12 7217 JC short NODEEXISTSP
23659 00003A14 C53E[E205] LDS DI,[CURBUF]
23660 ;or byte [di+5],4
23661 00003A18 804D0504 OR byte [DI+BUFFINFO.buf_flags],buf_isDIR
23662 00003A1C 5E POP SI ; dir_first pointer
23663 00003A1D 01FE ADD SI,DI
23664 ;sub si,1Ah ; 26
23665 00003A1F 83EE1A SUB SI,dir_entry.dir_first;Point back to start of dir entry
23666 00003A22 C604E5 MOV BYTE [SI],0E5H ; Free the entry
23667 00003A25 E8BDFF CALL DIRUP ; Error doesn't matter since erroring anyway
23668
23669 00003A28 E9F9FE NODEEXISTS:
JMP NODEACCERRJ ; 10/08/2018

```

```

23670
23671
23672 00003A2B 5E
23673 00003A2C EBFA
23674
23675
23676
23677
23678
23679
23680
23681
23682
23683
23684
23685
23686
23687
23688
23689
23690
23691
23692
23693
23694
23695
23696
23697
23698
23699
23700
23701
23702
23703
23704
23705
23706
23707
23708
23709
23710
23711
23712
23713
23714
23715
23716
23717
23718
23719
23720
23721 00003A2E E8F8DD
23722 00003A31 7306
23723
23724
23725
23726
23727
23728 00003A33 B80511
23729 00003A36 CD2F
23730
23731
23732
23733
23734
23735 00003A38 C3
23736
23737
23738
23739 00003A39 E8A6DE
23740
23741
23742
23743
23744
23745 00003A3C 26F6454420
23746 00003A41 740C
23747
23748 00003A43 26C74549FFFF
23749
23750
23751
23752 00003A49 26C7454BFFFF
23753
23754
23755
23756 00003A4F C606[4C03]00
23757
23758 00003A54 C606[6D05]16
23759
23760
23761 00003A59 800E[4611]01
23762 00003A5E E8AE10
23763
23764
23765
23766 00003A61 9F
23767 00003A62 8026[4611]80
23768
23769 00003A67 9E
23770
23771
23772
23773
23774
23775
23776
23777 00003A68 B80300
23778 00003A6B 7206
23779 00003A6D 7556
23780 00003A6F 8B0E[C205]
23781
23782
23783
23784
23785
23786
23787 00003A73 E999DE
23788
23789
23790
23791
23792
23793

NODEEXISTSP:
    POP     SI                ; Clean stack
    JMP     short NODEEXISTSP

; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)

;BREAK <DOS_ChDir -- Verify a directory>
;-----
;
; Procedure Name : DOS_ChDir
;
; Inputs:
;   [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
;               terminated)
;   [CURR_DIR_END] Points to end of Current dir part of string
;               (= -1 if current dir not involved, else
;               Points to first char after last "/" of current dir part)
;   [THISCDS] Points to CDS being used May not be NUL
; Function:
;   Validate the path for potential new current directory
; Returns:
;   NOTE:
;   [SATTRIB] is modified by this call
;   Carry Clear
;   CX is cluster number of the DIR, LOCAL CDS ONLY
;   Caller must NOT set ID fields on a NET CDS.
;   Carry Set
;   AX is error code
;       error_path_not_found
;       Bad path
;       error_access_denied
;       device or file name
;   DS preserved, Others destroyed
;-----

;hkn; called from path.asm and dir2.asm. DS already set up.

; 18/05/2019 - Retro DOS v4.0
; DOSCODE:70DAh (MSDOS 6.21, MSDOS.SYS)

; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:70C6h (MSDOS 5.0, MSDOS.SYS)

; 04/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; PCDOS 7.1 IBMDOS.COM - DOSCODE:7A23h

; (Windows ME IO.SYS - BIOSCODE:7839h)
; (MSDOS 6.22 MSDOS.SYS - DOSCODE:70DAh)

DOS_CHDIR:
    call    TestNet
    JNC     short LOCAL_CHDIR

;IF NOT Installed
; transfer NET_CHDIR
;ELSE
    mov     ax,1105h
    MOV     AX,(MULTINET<<8)|5
    int     2Fh        ; Multiplex - NETWORK REDIRECTOR - CHDIR
                        ; SS = DOS CS
                        ; SDA first filename pointer -> fully-qualified directory name
                        ; SDA CDS pointer -> current directory
                        ; Return: CF set on error, AX = DOS error code
                        ; CF clear if successful
    retn
;ENDIF

LOCAL_CHDIR:
    call    ECritDisk
    ; MSDOS 6.0
    ; test word [es:di+43h],2000h
    ; TEST word [ES:DI+curdir.flags],curdir_splice ;PTM.
    ; 17/12/2022
    ; test byte [es:di+44h],20h
    test    byte [ES:DI+curdir.flags+1],(curdir_splice>>8) ;PTM.
    JZ      short nojoin ;PTM.
    mov     word [es:di+49h], 0FFFFh
    MOV     word [ES:DI+curdir.ID],0FFFFh ;PTM.
    ;;;
    ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
    mov     word [es:di+4Bh], 0FFFFh
    mov     word [es:di+curdir.ID+2],0FFFFh
    ;;;
nojoin:
    ; MSDOS 3.3 & MSDOS 6.0
    MOV     byte [NoSetDir],0 ; FALSE
    mov     byte [SATTRIB],16h
    MOV     byte [SATTRIB],attr_directory+attr_system+attr_hidden
                        ; Dir calls can find these
; DOS 3.3 6/24/86 FastOpen
    OR      byte [FastOpenFlg],FastOpen_Set ; set fastopen flag
    call    GETPATH

    ; 04/02/2024
    PUSHF                                ;AN000;
    lahf
    AND     byte [FastOpenFlg],Fast_yes ; clear it all ;AC000;
    POPF
    sahf                                ;AN000;

; DOS 3.3 6/24/86 FastOpen

    ; MSDOS 3.3
    mov     byte [FastOpenFlg],0

    mov     ax,3
    MOV     AX,error_path_not_found
    JC      short ChDirDone
    JNZ     short NOTDIRPATH ; Path not a DIR
    MOV     CX,[DIRSTART] ; Get cluster number
    ; 27/06/2024
    CLC
ChDirDone:
    call    LCritDisk
    retn
    ; 18/12/2022
    jmp     LCritDisk

;BREAK <DOS_RmDir -- Remove a directory>
;-----
;
; Procedure Name : DOS_RmDir
;

```



```

23794 ; Inputs:
23795 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
23796 ; terminated)
23797 ; [CURR_DIR_END] Points to end of Current dir part of string
23798 ; (= -1 if current dir not involved, else
23799 ; Points to first char after last "/" of current dir part)
23800 ; [THISCDS] Points to CDS being used
23801 ; (Low word = -1 if NUL CDS (Net direct request))
23802 ; Function:
23803 ; Remove a directory
23804 ; NOTE: Attempt to remove current directory must be detected by caller
23805 ; Returns:
23806 ; NOTE:
23807 ; [SATTRIB] is modified by this call
23808 ; Carry Clear
23809 ; No error
23810 ; Carry Set
23811 ; AX is error code
23812 ; error_path_not_found
23813 ; Bad path (not in curr dir part if present)
23814 ; error_bad_curr_dir
23815 ; Bad path in current directory part of path
23816 ; error_access_denied
23817 ; device or file name, root directory
23818 ; Bad directory ('.', '..', messed up)
23819 ; DS preserved, Others destroyed
23820 ;-----
23821 ;hkn; called from path.asm. DS already set up.
23822
23823 ; 18/05/2019 - Retro DOS v4.0
23824 ; DOSCODE:711Fh (MSDOS 6.21, MSDOS.SYS)
23825
23826 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
23827 ; DOSCODE:710Bh (MSDOS 5.0, MSDOS.SYS)
23828
23829 ; 06/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
23830 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7A6Bh
23831
23832 DOS_RMDIR:
23833 call TestNet
23834 00003A76 E8B0DD JNC short LOCAL_RMDIR
23835 00003A79 7306
23836
23837 ;IF NOT Installed
23838 ; transfer NET_RMDIR
23839 ;ELSE
23840 ;mov ax,1101h
23841 MOV AX,(MULTINET<<8)|1
23842 00003A7E CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - REMOVE REMOTE DIRECTORY
23843 ; SS = DOS CS
23844 ; SDA first filename pointer -> fully-qualified directory name
23845 ; SDA CDS pointer -> current directory
23846 ; Return: CF set on error, AX = DOS error code
23847 ; CF clear if successful
23848 00003A80 C3 retn
23849 ;ENDIF
23850
23851 LOCAL_RMDIR:
23852 call ECritDisk
23853 00003A84 C606[4C03]00 MOV byte [NoSetDir],0
23854 ;mov byte [SATTRIB],16h
23855 00003A89 C606[6D05]16 MOV byte [SATTRIB],attr_directory+attr_system+attr_hidden
23856 ; Dir calls can find these
23857 00003A8E E87E10 call GETPATH
23858 00003A91 7229 JC short NOPATH ; Path not found
23859 00003A93 7530 JNZ short NOTDIRPATH ; Path not a DIR
23860
23861 ; 06/04/2024
23862 %if 0
23863 MOV DI,[DIRSTART]
23864 OR DI,DI ; Root ?
23865 JNZ short rmdir_get_buf ; No
23866 JMP SHORT NOTDIRPATH
23867 %else
23868 ; 06/02/2024 - Retro DOS v5.0
23869 ; PCDOS 7.1 IBMDOS.COM
23870 00003A95 8B3E[C205] mov di,[DIRSTART]
23871 00003A99 09FF or di,di ; Root ?
23872 00003A9B 7506 jnz short LOCAL_RMDIR_cont; No
23873
23874 ;cmp word [DIRSTART_HW],0
23875 00003A9D 393E[DC0A] cmp [DIRSTART_HW],di ; 0
23876 00003AA1 7422 jz short NOTDIRPATH ; root directory
23877
23878 LOCAL_RMDIR_cont:
23879 ;cmp word [es:bp+0Fh],0
23880 00003AA3 26837E0F00 cmp word [es:bp+DPB.FAT_SIZE],0
23881 00003AA8 751E jnz short rmdir_get_buf ; not FAT32
23882 ;cmp di,[es:bp+35h]
23883 00003AAA 263B7E35 cmp di,[es:bp+DPB.ROOT_CLUSTER]
23884 00003AAE 7518 jne short rmdir_get_buf
23885 00003AB0 8B3E[DC0A] mov di,[DIRSTART_HW]
23886 ;cmp di,[es:bp+37h]
23887 00003AB4 263B7E37 cmp di,[es:bp+DPB.ROOT_CLUSTER+2]
23888 00003AB8 750E jne short rmdir_get_buf
23889 00003ABA EB09 jmp short NOTDIRPATH
23890 %endif
23891
23892 NOPATH:
23893 ;mov ax,3
23894 00003ABC B80300 MOV AX,error_path_not_found
23895 00003ABF E965FE JMP _BadRet
23896
23897 NOTDIRPATHPOP:
23898 POP AX ; MSDOS 6.0 ;F.C. >32mb
23899 POP AX
23900 NOTDIRPATHPOP2:
23901 POP AX
23902 NOTDIRPATH:
23903 JMP NODEACCERRJ
23904
23905 rmdir_get_buf:
23906 00003AC8 C53E[E205] LDS DI,[CURBUF]
23907 00003ACC 29FB SUB BX,DI ; Compute true offset
23908 00003ACE 53 PUSH BX ; Save entry pointer
23909
23910 ; 06/04/2024
23911 %if 0
23912 ; MSDOS 6.0
23913 ;push word [di+8]
23914 ;push word [DI+BUFFINFO.buf_sector+2] ;F.C. >32mb
23915
23916 ; MSDOS 3.3 (& MSDOS 6.0)
23917 ;push word [di+6]

```

```

23918      PUSH    WORD [DI+BUFFINFO.buf_sector] ; Save sector number
23919  %else
23920      ; 06/02/2024 - Retro DOS v5.0
23921      ; PCDOS 7.1 IBMDOS.COM
23922
23923      ;les    di,[di+6]
23924 00003ACF C47D06      les    di,[di+BUFFINFO.buf_sector]
23925 00003AD2 06          push    es
23926 00003AD3 57          push    di
23927  %endif
23928
23929 ;hkn; SS is DOSDATA
23930 ;context DS
23931      push    ss
23932 00003AD5 1F          pop     ds
23933      ;context ES
23934 00003AD6 16          push    ss
23935 00003AD7 07          pop     es
23936
23937 ;hkn; NAME1 is in DOSDATA
23938 00003AD8 BF[4B05]    MOV     DI,NAME1
23939      ;MOV     AL,'?' ; 3Fh
23940      ;MOV     CX,11
23941      ;REP     STOSB
23942      ;XOR     AL,AL
23943      ;STOSB                                     ; Nul terminate it
23944      ; 27/06/2024
23945 00003ADB B83F00      mov     ax,3Fh
23946 00003ADE B90A00      mov     cx,10
23947 00003AE1 F3AA      rep     stosb ; al = "?"
23948 00003AE3 AB          stosw   ; ah = 0
23949      ;
23950 00003AE4 E8C812      call    STARTSRCH ; Set search
23951 00003AE7 E8B10D      call    GETENTRY  ; Get start of directory
23952 00003AEA 72D6      JC      short NOTDIRPATHPOP ; Screw up
23953 00003AEC 8E1E[E405] MOV     DS,[CURBUF+2]
23954 00003AF0 89DE      MOV     SI,BX
23955 00003AF2 AD          LODSW
23956      ;CMP     AX,(' ' SHL 8) OR '.' ; First entry '.'?
23957 00003AF3 3D2E20      cmp     ax,202Eh ; "."
23958 00003AF6 75CA      JNZ     short NOTDIRPATHPOP ; Nope
23959      ;add     si,30
23960 00003AF8 83C61E      ADD     SI,dir_entry.size-2 ; Next entry
23961 00003AFB AD          LODSW
23962      ;CMP     AX,('.' SHL 8) OR '.' ; Second entry '..'?
23963      ;cmp     ax, '.'
23964 00003AFC 3D2E2E      cmp     ax,2E2Eh
23965 00003AFF 75C1      JNZ     short NOTDIRPATHPOP ; Nope
23966
23967 ;hkn; SS is DOSDATA
23968 ;context DS
23969 00003B01 16          push    ss
23970 00003B02 1F          pop     ds
23971 00003B03 C706[4803]0200 MOV     word [LASTENT],2 ; Skip . and ..
23972 00003B09 E88F0D      call    GETENTRY  ; Get next entry
23973 00003B0C 72B4      JC      short NOTDIRPATHPOP ; Screw up
23974      ;mov     byte [ATTRIB],16h
23975 00003B0E C606[6B05]16      MOV     byte [ATTRIB],attr_directory+attr_hidden+attr_system
23976 00003B13 E83B0C      call    SRCH      ; Do a search
23977 00003B16 73AA      JNC     short NOTDIRPATHPOP ; Found another entry!
23978 00003B18 803E[4A03]00 CMP     byte [FAILERR],0
23979 00003B1D 75A3      JNZ     short NOTDIRPATHPOP ; Failure of search due to I 24 FAIL
23980 00003B1F C42E[8A05]    LES     BP,[THISDPB]
23981      ;;;
23982      ; 06/02/2024 - Retro DOS v5.0
23983      ; PCDOS 7.1 IBMDOS.COM
23984 00003B23 8B1E[DC0A]    mov     bx,[DIRSTART_HW]
23985 00003B27 891E[E80A]    mov     [CLUSTNUM_HW],bx
23986      ;;;
23987 00003B2B 8B1E[C205]    MOV     BX,[DIRSTART]
23988 00003B2F E89A20      call    RELEASE   ; Release data in sub dir
23989 00003B32 728E      JC      short NOTDIRPATHPOP ; Screw up
23990      ;;;
23991      ; 06/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
23992 00003B34 E831F9      call    update_fat32_fsinfo
23993      ;;;
23994 00003B37 5A          POP     DX ; Sector # of entry
23995 00003B38 8F06[0706]    POP     word [HIGH_SECTOR] ; MSDOS 6.0 ; F.C. >32mb
23996      ; 22/09/2023
23997      ;mov     byte [ALLOWED],18h
23998      ;MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
23999      ;XOR     AL,AL ; * ; Pre read
24000      ;call    GETBUFFER ; * ; Get sector back
24001 00003B3C E8E52D      call    GETBUFFER ; * ; Pre Read
24002 00003B3F 7283      JC      short NOTDIRPATHPOP2 ; Screw up
24003
24004 rmdir_fde: ; 06/02/2024
24005 00003B41 C53E[E205]    LDS     DI,[CURBUF]
24006      ;or     byte [di+5],4
24007 00003B45 804D0504      OR     byte [DI+BUFFINFO.buf_flags],buf_isDIR
24008 00003B49 5B          POP     BX ; Pointer to start of entry
24009 00003B4A 01FB      ADD     BX,DI ; Corrected
24010 00003B4C C607E5      MOV     BYTE [BX],0E5H ; Free the entry
24011
24012 ;DOS 3.3 FastOpen 6/16/86 F.C.
24013 00003B4F 1E          PUSH    DS
24014
24015 ;hkn; SS is DOSDATA
24016 ;context DS
24017 00003B50 16          push    ss
24018 00003B51 1F          pop     ds
24019
24020      ; MSDOS 6.0
24021 00003B52 E8E9F1      call    FastOpen_Delete ; call fastopen to delete an entry
24022
24023 ; ; MSDOS 3.3
24024 ;_FastOpen_Delete:
24025      ; push    ax
24026      ; mov     si,[WFP_START]
24027      ; mov     bx,FastTable
24028      ; ;mov     al,3 ; FONC_delete
24029      ; mov     al,FONC_delete
24030      ; call    far [BX+2] ; FastTable+2
24031      ; pop     ax
24032
24033 00003B55 1F          POP     DS
24034 ;DOS 3.3 FastOpen 6/16/86 F.C.
24035
24036 00003B56 E98CFE      JMP     DIRUP ; In MKDIR, dirty buffer and flush
24037
24038 ;=====
24039 ; DISK.ASM, MSDOS 6.0, 1991
24040 ;=====
24041 ; 23/07/2018 - Retro DOS v3.0

```

```

24042 ; 04/05/2019 - Retro DOS v4.0
24043 ; 06/02/2024 - Retro DOS v5.0
24044
24045 ; TITLE DISK - Disk utility routines
24046 ; NAME Disk
24047
24048 ;** Low level Read and write routines for local SFT I/O on files and devs
24049 ;
24050 ; SWAPCON
24051 ; SWAPBACK
24052 ; DOS_READ
24053 ; DOS_WRITE
24054 ; get_io_sft
24055 ; DirRead
24056 ; FIRSTCLUSTER
24057 ; SET_BUF_AS_DIR
24058 ; FATSecRd
24059 ; DREAD
24060 ; CHECK_WRITE_LOCK
24061 ; CHECK_READ_LOCK
24062
24063 ; Revision history:
24064 ;
24065 ; A000 version 4.00 Jan. 1988
24066 ;
24067 -----
24068 ;
24069 ; M065 : B#5276. On raw read/write of a block of characters if a critical
24070 ; error happens, DOS retries the entire block assuming that
24071 ; zero characters were transferred. Modified the code to take
24072 ; into account the number of characters transfered before
24073 ; retrying the operation.
24074 ;
24075 ;-----
24076 ;
24077 ;Installed = TRUE
24078
24079 ;Break <SwapCon, Swap Back - old-style I/O to files>
24080
24081 ; **** Drivers for file input from devices ****
24082 ;-----
24083 ; Indicate that there is no more I/O occurring through another SFT outside
24084 ; of handles 0 and 1
24085 ;
24086 ; Inputs: DS is DOSDATA
24087 ; Outputs: CONSWAP is set to false.
24088 ; Registers modified: none
24089 ;-----
24090 ;
24091 ; IBMDOS.COM (MSDOS 3.3) - Offset 3CF8h
24092 ;
24093 ; DOSCODE:71E3h (MSDOS 6.21, MSDOS.SYS)
24094 ; 04/05/2019 - Retro DOS v4.0
24095 ;
24096 ; DOSCODE:71CFh (MSDOS 5.0, MSDOS.SYS)
24097 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
24098
24099
24100 SWAPBACK:
24101 00003B59 C606[5703]00 MOV BYTE [CONSWAP],0 ; signal no conswaps
24102 00003B5E C3 retn
24103
24104 ;-----
24105 ;
24106 ; Procedure Name : SWAPCON
24107 ;
24108 ; Copy ThisSFT to CONSFT for use by the 1-12 primitives.
24109 ;
24110 ; Inputs: ThisSFT as the sft of the desired file
24111 ; DS is DOSDATA
24112 ; Outputs: CONSWAP is set. CONSFT = ThisSFT.
24113 ; Registers modified: none
24114 ;-----
24115 ;
24116 SWAPCON:
24117 ; MSDOS 3.3
24118 ;push es
24119 ;push di
24120 ;mov byte [CONSWAP],1
24121 ;les di,[THISFT]
24122 ;mov word [CONSFT],di
24123 ;mov word [CONSFT+2],es
24124 ;pop di
24125 ;pop es
24126 ;retn
24127
24128 ; MSDOS 6.0
24129 00003B5F C606[5703]01 mov byte [CONSWAP],1 ; ConSwap = TRUE
24130 00003B64 50 push ax
24131 00003B65 A1[9E05] mov ax,[THISFT]
24132 00003B68 A3[E605] mov [CONSFT],ax
24133 00003B6B A1[A005] mov ax,[THISFT+2]
24134 00003B6E A3[E805] mov [CONSFT+2],ax
24135 00003B71 58 pop ax
24136 00003B72 C3 retn
24137
24138 ; DOSCODE:71FDh (MSDOS 6.21, MSDOS.SYS)
24139 ; 04/05/2019 - Retro DOS v4.0
24140
24141 ;Break <DOS_READ -- MAIN READ ROUTINE AND DEVICE IN ROUTINES>
24142 ;-----
24143 ;
24144 ; Inputs:
24145 ; ThisSFT set to the SFT for the file being used
24146 ; [DMAADD] contains transfer address
24147 ; CX = No. of bytes to read
24148 ; DS = DOSDATA
24149 ; Function:
24150 ; Perform read operation
24151 ; Outputs:
24152 ; Carry clear
24153 ; SFT Position and cluster pointers updated
24154 ; CX = No. of bytes read
24155 ; ES:DI point to SFT
24156 ; Carry set
24157 ; AX is error code
24158 ; CX = 0
24159 ; ES:DI point to SFT
24160 ; DS preserved, all other registers destroyed
24161 ;-----
24162 ;
24163 ;hkn; called from fcbio.asm, handle.asm and dev.asm. DS is be set up.
24164
24165

```

```

24166 ; DOSCODE:71E9h (MSDOS 5.0, MSDOS.SYS)
24167 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
24168
24169 ; 07/02/2024
24170 ; 06/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
24171 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:7B76h
24172
24173 ; (Windows ME IO.SYS - BIOSCODE:7997h)
24174 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:71FDh)
24175
24176 DOS_READ:
24177 LES DI,[THISSFT]
24178
24179 ; verify that the sft has been opened in a mode that allows reading.
24180
24181 ;mov al,[es:di+2]
24182 MOV AL,[ES:DI+SF_ENTRY.sf_mode]
24183 ;and al,0Fh ; (MSDOS 5.0 & 6.22)
24184 ;AND AL,access_mask
24185 ; 06/02/2024 - Retro DOS v5.0
24186 ; (PCDOS 7.1 & win Me)
24187 00003B7B 2403 and al,3 ; open_mode_mask ?
24188
24189 ;cmp al,1
24190 CMP AL,open_for_write
24191 00003B7F 7503 JNE short READ_NO_MODE ; Is read or both
24192 00003B81 E96406 jmp SET_ACC_ERR
24193
24194 READ_NO_MODE:
24195 call SETUP
24196 00003B87 E30B JCXZ NoIORet ; no bytes to read - fast return
24197 00003B89 E8B6DC call ISSFTNet
24198 00003B8C 7408 JZ short LOCAL_READ
24199
24200 ;IF NOT Installed
24201 ; transfer NET_READ
24202 ;ELSE
24203 ;mov ax,1108h
24204 00003B8E B80811 MOV AX,(MULTNET<<8)|8
24205 00003B91 CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - READ FROM REMOTE FILE
24206 ; ES:DI -> SFT
24207 ; SFT DPB field -> DPB of drive containing file
24208 ; CX = number of bytes, SS = DOS CS, SDA DTA field -> user buffer
24209 ; Return: CF set on error, CX = bytes read
24210 00003B93 C3 retn
24211 ;ENDIF
24212
24213 ; The user ended up requesting 0 bytes of input. We do nothing for this case
24214 ; except return immediately.
24215
24216 NoIORet:
24217 00003B94 F8 CLC
24218 00003B95 C3 retn
24219
24220 LOCAL_READ:
24221 ;test word [es:di+5],80h
24222 ;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device ; Check for named device I/O
24223 00003B96 26F6450580 test byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; 02/06/2019
24224 00003B9B 750E JNZ short READDEV
24225
24226 ;mov byte [EXTRERR_LOCUS],2
24227 00003B9D C606[2303]02 MOV byte [EXTRERR_LOCUS],errLOC_Disk
24228 00003BA2 E83DDD call ECritDisk
24229 00003BA5 E8F705 call DISKREAD
24230
24231 critexit:
24232 ;call LCritDisk
24233 ;retn
24234 ; 16/12/2022
24235 00003BA8 E964DD jmp LCritDisk
24236
24237 ; We are reading from a device. Examine the status of the device to see if we
24238 ; can short-circuit the I/O. If the device in the EOF state or if it is the
24239 ; null device, we can safely indicate no transfer.
24240
24241 READDEV:
24242 ;mov byte [EXTRERR_LOCUS],4
24243 00003BAB C606[2303]04 MOV byte [EXTRERR_LOCUS],errLOC_SerDev
24244 ;mov b1,[es:di+5]
24245 00003BB0 268A5D05 MOV BL,[ES:DI+SF_ENTRY.sf_flags]
24246 00003BB4 C43E[2C03] LES DI,[DMAADD]
24247 ;test b1,40h
24248 00003BB8 F6C340 test BL,devid_device_EOF ; End of file?
24249 00003BBB 7407 JZ short ENDRDDEVJ3
24250 ;test b1,4
24251 00003BBD F6C304 test BL,devid_device_null ; NUL device?
24252 00003BC0 7405 JZ short TESTRAW ; NO
24253 00003BC2 30C0 XOR AL,AL ; Indicate EOF by setting zero
24254 ENDRDDEVJ3:
24255 ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility!)
24256 ;JMP short ENDRDDEVJ2
24257 ; 16/12/2022
24258 00003BC4 E93F01 jmp ENDRDDEV ; 04/05/2019
24259
24260 ; We need to hit the device. Figure out if we do a raw read or we do the
24261 ; bizarre std_con_string_input.
24262
24263 TESTRAW:
24264 ;test b1,20h
24265 00003BC7 F6C320 test BL,devid_device_raw ; Raw mode?
24266 00003BCA 7508 JNZ short DVRDRAW ; Yes, let the device do all local editing
24267 ;test b1,1
24268 00003BCC F6C301 test BL,devid_device_con_in; Is it console device?
24269 00003BCF 7458 JZ short NOTRDCON
24270 00003BD1 E96701 JMP READCON
24271
24272 DVRDRAW:
24273 00003BD4 06 PUSH ES
24274 00003BD5 1F POP DS ; Xaddr to DS:DI
24275
24276 ; 04/05/2019 - Retro DOS v4.0
24277
24278 ; MSDOS 6.0
24279 ;SR;
24280 ;Check for win386 presence -- if present, do polled read of characters
24281
24282 00003BD6 36F606[5B0F]01 test byte [ss:Iswin386],1 ; 19/05/2019
24283 00003BDC 7408 jz short ReadRawRetry ;not present
24284 00003BDE F6C301 test b1,devid_device_con_in;is it console device?
24285 00003BE1 7403 jz short ReadRawRetry ;no, do normal read
24286 00003BE3 E9A800 jmp do_polling ;yes, do win386 polling loop
24287
24288 ReadRawRetry:
24289

```

```

24290 ; 07/02/2024
24291 %if 0
24292     MOV     BX,DI ; DS:BX transfer addr
24293     ; 06/02/2024 ; *
24294     ;XOR     AX,AX ; Media Byte, unit = 0
24295     ;MOV     DX,AX ; Start at 0
24296     ; 06/02/2024
24297     ;cld
24298     ;call    SETREAD
24299     ; 06/02/2024 ; *
24300     call    SETREAD_X
24301 %else
24302     call    SETREAD_XJ
24303 %endif
24304
24305     PUSH    DS ; Save Seg part of Xaddr
24306
24307 ;hkn; SS override
24308     LDS     SI,[SS:THISSFT]
24309     call    DEVIOCALL
24310     MOV     DX,DI ; DS:DX is preserved by INT 24
24311     MOV     AH,86H ; Read error
24312
24313 ;hkn; SS override
24314     MOV     DI,[SS:DEVCALL_REQSTAT]
24315     ; MSDOS 3.3
24316     ;test    di,8000h
24317     ;jz      short CRDRK
24318     ; MSDOS 6.0
24319     or       di,di
24320     jns     short CRDRK ; no errors
24321     ; MSDOS 3.3 (& MSDOS 6.0)
24322     call    CHARHARD
24323
24324 ; 06/02/2024 - Retro DOS v5.0
24325 %if 0
24326     MOV     DI,DX ; DS:DI is Xaddr
24327     ; 04/05/2019
24328     ; MSDOS 6.0
24329     add     di,[ss:CALLSCNT] ; update ptr and count to reflect the M065
24330     sub     cx,[ss:CALLSCNT] ; number of chars xferred M065
24331 %else
24332     mov     di,[ss:CALLSCNT]
24333     sub     cx,di ; update transfer count
24334     add     di,dx ; update pointer
24335 %endif
24336     ; MSDOS 3.3 (& MSDOS 6.0)
24337     OR      AL,AL
24338     JZ      short CRDRK ; Ignore
24339     CMP     AL,3
24340     JZ      short CRDFERR ; fail.
24341     POP     DS ; Recover saved seg part of Xaddr
24342     JMP     short ReadRawRetry ; Retry
24343
24344 ; We have encountered a device-driver error. We have informed the user of it
24345 ; and he has said for us to fail the system call.
24346
24347 CRDFERR:
24348     POP     DI ; Clean stack
24349 DEVIOFERR:
24350
24351 ;hkn; SS override
24352     LES     DI,[SS:THISSFT]
24353     jmp     SET_ACC_ERR_DS
24354
24355 CRDRK:
24356     POP     DI ; Chuck saved seg of Xaddr
24357     MOV     DI,DX
24358
24359 ;hkn; SS override
24360     ADD     DI,[ss:CALLSCNT] ; Amount transferred
24361     ;JMP     SHORT ENDRDDEVJ3
24362     ; 16/12/2022
24363     jmp     short ENDRDDEVJ2
24364
24365 ; We are going to do a cooked read on some character device. There is a
24366 ; problem here, what does the data look like? Is it a terminal device, line
24367 ; CR line CR line CR, or is it file data, line CR LF line CR LF? Does it have
24368 ; a ^Z at the end which is data, or is the ^Z not data? In any event we're
24369 ; going to do this: Read in pieces up to CR (CRs included in data) or ^Z (^Z
24370 ; included in data). this "simulates" the way con works in cooked mode
24371 ; reading one line at a time. With file data, however, the lines will look
24372 ; like, LF line CR. This is a little weird.
24373
24374 NOTRDCON:
24375     ;MOV     AX,ES
24376     ;MOV     DS,AX
24377     ; 07/02/2024
24378     push    es
24379     pop     ds
24380
24381 ; 07/02/2024
24382 %if 0
24383     MOV     BX,DI
24384     ; 06/02/2024 ; *
24385     ;XOR     DX,DX
24386     ;MOV     AX,DX
24387     ; 06/02/2024
24388     ;xor     ax,ax
24389     ;cld
24390     PUSH    CX
24391     MOV     CX,1
24392     ;call    SETREAD
24393     ; 06/02/2024 ; *
24394     call    SETREAD_X
24395     POP     CX
24396 %else
24397     push    cx
24398     mov     cx,1
24399     call    SETREAD_XJ
24400     pop     cx
24401 %endif
24402
24403 ;hkn; SS override
24404     LDS     SI,[SS:THISSFT]
24405     ;lds     si,[si+7]
24406     LDS     SI,[SI+SF_ENTRY.sf_devptr]
24407 DVRDLP:
24408     call    DSKSTATCHK
24409     call    DEVIOCALL2
24410     PUSH    DI ; Save "count" done
24411     MOV     AH,86H
24412
24413 ;hkn; SS override

```

```

24414 00003C44 368B3E[5D03]      MOV     DI,[SS:DEVCALL_REQSTAT]
24415
24416      ; MSDOS 3.3
24417      ;test  di,8000h
24418      ;jz    short CRDOK
24419      ; MSDOS 6.0
24420 00003C49 09FF      or     di,di
24421 00003C4B 7917      jns    short CRDOK
24422
24423 00003C4D E8CE23      call   CHARHARD
24424 00003C50 5F        POP     DI
24425
24426      ;hkn; SS override
24427 00003C51 36C706[6C03]0100  MOV     word [SS:CALLSCNT],1
24428 00003C58 3C01      CMP     AL,1
24429 00003C5A 74DF      JZ      short DVRDLP          ; Retry
24430 00003C5C 3C03      CMP     AL,3
24431 00003C5E 74B7      JZ      short DEVIOFERR          ; FAIL
24432 00003C60 30C0      XOR     AL,AL          ; Ignore, Pick some random character
24433 00003C62 EB12      JMP     SHORT DVRDIGN
24434
24435 CRDOK:
24436 00003C64 5F        POP     DI
24437
24438      ;hkn; SS override
24439 00003C65 36833E[6C03]01  CMP     word [SS:CALLSCNT],1
24440      ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility!)
24441 00003C6B 751F      JNZ     short ENDRDDEVJ2
24442      ; 16/12/2022
24443      ;jnz    short ENDRDDEV ; 24/07/2019
24444
24445 00003C6D 1E        PUSH    DS
24446
24447      ;hkn; SS override
24448 00003C6E 368E1E[6A03]  MOV     DS,[SS:CALLXAD+2]
24449 00003C73 8A05      MOV     AL,[DI]          ; Get the character we just read
24450 00003C75 1F        POP     DS
24451
24452 DVRDIGN:
24453
24454      ;hkn; SS override
24455 00003C76 36FF06[6803]  INC     WORD [SS:CALLXAD]          ; Next character
24456 00003C7B 36C706[5D03]0000  MOV     word [SS:DEVCALL_REQSTAT],0
24457 00003C82 47        INC     DI          ; Next character
24458 00003C83 3C1A      CMP     AL,1Ah          ; ^Z?
24459      ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility!)
24460      ;jz     short ENDRDDEVJ2          ; Yes, done zero set (EOF)
24461      ; 16/12/2022
24462      ;jz     short ENDRDDEV ; 24/07/2019
24463 00003C87 3C0D      CMP     AL,c_CR ; 0Dh          ; CR?
24464 00003C89 E0B0      LOOPNZ  DVRDLP          ; Loop if no, else done
24465 00003C8B 40        INC     AX          ; Resets zero flag so NOT EOF, unless
24466      ; AX=FFFF which is not likely
24467
24468 ENDRDDEVJ2:
24469      ; 16/12/2022
24470      ;JMP    short ENDRDDEV          ; changed short to long for win386
24471      ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility!)
24472      jmp    ENDRDDEV
24473
24474      ; 04/05/2019
24475
24476      ; MSDOS 6.0
24477
24478 ;SR:
24479 ;Polling code for raw read on CON when WIN386 is present
24480 ;
24481 ;At this point -- ds:di is transfer address
24482 ;
24483 ; cx is count
24484
24485 do_polling:
24486
24487 ; 07/02/2024
24488 %if 0
24489     mov     bx,di          ;ds:bx is xfer address
24490     ; 06/02/2024 ; *
24491     ;xor     ax,ax
24492     ;mov     dx,ax
24493     ; 06/02/2024
24494     ;cld
24495     ;call    SETREAD          ;prepare device packet
24496     ; 06/02/2024 ; *
24497     call    SETREAD_X
24498 %else
24499     call    SETREAD_XJ
24500 %endif
24501
24502 do_io:
24503 ;Change read to a NON-DESTRUCTIVE READ, NO WAIT
24504
24505     mov     byte [es:bx+2],DEVDRND ; 5 ;Change command code
24506     push    ds
24507     lds     si,[ss:THISSFT]          ;get device header
24508     call    DEVIOCALL          ;call device driver
24509     pop     ds
24510
24511     ;test    word [es:bx+3],8000h
24512     ; 16/12/2022
24513     ;test    byte [es:bx+4],80h
24514     test    byte [es:bx+SRHEAD.REQSTAT+1],STERR>>8
24515     ;test    word [es:bx+SRHEAD.REQSTAT],STERR ;check if error
24516     jz      short check_busy          ;no
24517
24518     push    ds
24519     mov     dx,di
24520
24521 invoke_charhard:      ; 07/02/2024
24522     ;invoke charhard          ;invoke int 24h handler
24523     call    CHARHARD
24524     mov     di,dx
24525     or     al,al
24526     jz      short pop_done_read      ;ignore by user, assume read done
24527     cmp     al,3
24528     jz      short devrderr          ;user asked to fail
24529     pop     ds
24530     jmp     short do_io          ;user asked to retry
24531
24532 check_busy:
24533     ;test    word [es:bx+3],200h
24534     ; 16/12/2022
24535     test    byte [es:bx+SRHEAD.REQSTAT+1],02h
24536     ;test    word [es:bx+SRHEAD.REQSTAT],0200h ;see if busy bit set
24537     jnz     short no_char          ;yes, no character available
24538
24539 ;Character is available. Read in 1 character at a time until all characters
24540 ;are read in or no character is available

```



```

24538 00003CC1 26C6470204      mov     byte [es:bx+2],DEV RD ; 4 ;command code is READ now
24539 00003CC6 26C747120100     mov     word [es:bx+18],1      ;change count to 1 character
24540 00003CCC 1E                push    ds
24541 00003CCD 36C536[9E05]      lds     si,[ss:THISSFT]
24542 00003CD2 E8B915          call    DEVIOCALL
24543
24544 00003CD5 89FA          mov     dx,di
24545 00003CD7 B486          mov     ah,86h
24546                      ;mov     di,[es:bx+3]
24547 00003CD9 268B7F03      mov     di,[es:bx+SRHEAD.REQSTAT] ;get returned status
24548 00003CDD F7C70080      test    di,STERR ; 8000h      ;was there an error during read?
24549                      ;jz      short next_char      ;no,read next character
24550                      ; 07/02/2024
24551 00003CE1 75C7          jnz     short invoke_charhard
24552
24553                      ; 07/02/2024
24554                      %if 0
24555                      ;invoke charhard      ;invoke int 24h handler
24556                      call    CHARHARD
24557                      mov     di,dx      ;restore di
24558                      or      al,al      ;
24559                      jz      short pop_done_read ;ignore by user,assume read is done
24560                      cmp     al,3
24561                      jz      short devrderr ;user issued a 'fail',indicate error
24562                      pop     ds
24563                      jmp     short do_io ;user issued a retry
24564                      %endif
24565
24566                      next_char:
24567                      pop     ds
24568                      mov     di,dx
24569                      dec     cx      ;decrement count
24570                      ;jcxz    done_read      ;all characters read in
24571                      ; 07/02/2024
24572                      jz      short done_read
24573                      inc     word [es:bx+14] ;update transfer address
24574                      jmp     short do_io ;read next character in
24575
24576                      devrderr:
24577                      pop     di      ;discard segment address
24578                      les     di,[ss:THISSFT]
24579                      ;transfer SET_ACC_ERR_DS ;indicate error
24580                      jmp     SET_ACC_ERR_DS
24581
24582                      no_char:
24583                      ;Since no character is available, we let win386 switch the VM out
24584
24585                      push    ax
24586                      mov     ah,84h ; Microsoft Networks - KEYBOARD BUSY LOOP
24587                      int     2Ah      ;indicate idle to WIN386
24588
24589                      ;when control returns from WIN386, we continue the raw read
24590
24591                      pop     ax
24592                      jmp     short do_io ; 27/06/2024
24593
24594                      pop_done_read:
24595                      pop     ds
24596                      done_read:
24597                      add     di,[ss:CALLSCNT] ; 19/05/2019
24598
24599                      ; 16/12/2022
24600
24601                      ;jmp     ENDRDDEVJ3 ;jump back to normal DOS raw read exit
24602                      ;jmp     ENDRDDEV ; 04/05/2019
24603
24604                      ; 04/05/2019 - Retro DOS v4.0
24605                      ENDRDDEV:
24606                      push    ss
24607                      pop     ds
24608                      jmp     short endrddev1
24609
24610                      ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24611                      ;jmp     ENDRDDEVJ3 ;jump back to normal DOS raw read exit
24612
24613                      TRANBUF:
24614                      LODSB
24615                      STOSB
24616                      CMP     AL,c_CR ; 0Dh ; Check for carriage return
24617                      JNZ     short NORMCH
24618                      MOV     BYTE [SI],c_LF ; 0Ah
24619                      NORMCH:
24620                      CMP     AL,c_LF ; 0Ah
24621                      LOOPNZ  TRANBUF
24622                      JNZ     short ENDRDCON
24623                      XOR     SI,SI ; Cause a new buffer to be read
24624                      call    OUTT ; Transmit linefeed
24625                      OR      AL,1 ; Clear zero flag--not end of file
24626                      ENDRDCON:
24627                      ;hkn; SS is DOSDATA
24628                      push    ss
24629                      pop     ds
24630                      CALL    SWAPBACK
24631                      MOV     [CONTPOS],SI
24632
24633                      ; 16/12/2022
24634                      ;ENDRDDEV:
24635                      ;;hkn; SS is DOSDATA
24636                      ; push    ss
24637                      ; pop     ds
24638                      ;
24639                      endrddev1: ; 04/05/2019
24640                      MOV     [NEXTADD],DI
24641                      JNZ     short SETSFTC ; Zero set if Ctrl-Z found in input
24642                      LES     DI,[THISSFT]
24643                      ;and     byte [es:di+5],0BFh
24644                      AND     BYTE [ES:DI+SF_ENTRY.sf_flags],~devid_device_EOF
24645                      ; Mark as no more data available
24646                      SETSFTC:
24647                      ; 31/07/2019
24648                      ;call    SETSFT
24649                      ;retn
24650                      jmp     SETSFT
24651
24652                      ; 16/12/2022
24653                      %if 0
24654                      ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24655                      ENDRDDEV:
24656                      ;hkn; SS is DOSDATA
24657                      push    ss
24658                      pop     ds
24659                      MOV     [NEXTADD],DI
24660                      JNZ     short SETSFTC ; Zero set if Ctrl-Z found in input
24661                      LES     DI,[THISSFT]
24662                      ;and     byte [es:di+5],0BFh

```

```

24662                AND     BYTE [ES:DI+SF_ENTRY.sf_flags],~devid_device_EOF
24663                ; Mark as no more data available
24664
24665                SETSFCT:
24666                ;call    SETSFCT
24667                ;retn
24668                jmp      SETSFCT
24669                %endif
24670
24671                READCON:
24672                CALL     SWAPCON
24673                MOV      SI,[CONTPOS]
24674                OR       SI,SI
24675                JNZ      short TRANBUF
24676                CMP      BYTE [CONBUF],128 ; 80h
24677                JZ       short GETBUF
24678                MOV      WORD [CONBUF],0FF80H ; Set up 128-byte buffer with no template
24679
24680                GETBUF:
24681                PUSH     CX
24682                PUSH     ES
24683                PUSH     DI
24684
24685                ;hkn; CONBUF is in DOSDATA
24686                MOV      DX,CONBUF
24687
24688                call     _$STD_CON_STRING_INPUT; Get input buffer
24689                POP      DI
24690                POP      ES
24691                POP      CX
24692
24693                ;hkn; CONBUF is in DOSDATA
24694                MOV      SI,CONBUF+2
24695
24696                CMP      BYTE [SI],1AH ; Check for Ctrl-Z in first character
24697                JNZ      short TRANBUF
24698                MOV      AL,1AH
24699                STOSB
24700                DEC      DI
24701                MOV      AL,C_LF
24702                call     OUTT ; Send linefeed
24703                XOR      SI,SI
24704                JMP      short ENDRDCON ; 04/05/2019
24705
24706                ; 24/07/2018 - Retro DOS v3.0
24707
24708                ;Break    <DOS_WRITE -- MAIN WRITE ROUTINE AND DEVICE OUT ROUTINES>
24709                ;-----
24710                ;
24711                ; Procedure Name : DOS_WRITE
24712                ;
24713                ; Inputs:
24714                ; ThisSFT set to the SFT for the file being used
24715                ; [DMAADD] contains transfer address
24716                ; CX = No. of bytes to write
24717                ; Function:
24718                ; Perform write operation
24719                ; NOTE: If CX = 0 on input, file is truncated or grown
24720                ; to current sf_position
24721                ; Outputs:
24722                ; Carry clear
24723                ; SFT Position and cluster pointers updated
24724                ; CX = No. of bytes written
24725                ; ES:DI point to SFT
24726                ; Carry set
24727                ; AX is error code
24728                ; CX = 0
24729                ; ES:DI point to SFT
24730                ; DS preserved, all other registers destroyed
24731                ;-----
24732
24733                ; 04/05/2019 - Retro DOS v4.0
24734                ; DOSCODE:742Ch (MSDOS 6.21, MSDOS.SYS)
24735
24736                ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
24737                ; DOSCODE:7418h (MSDOS 5.0, MSDOS.SYS)
24738
24739                ; 08/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
24740                ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7b8Fh
24741
24742                DOS_WRITE:
24743                LES      DI,[THISSFT]
24744                ;mov     al,[ES:DI+2]
24745                MOV      AL,[ES:DI+SF_ENTRY.sf_mode]
24746                ;;and    al,0Fh
24747                ;AND     AL,access_mask
24748                ; 07/02/2024 - Retro DOS v5.0
24749                ; (PCDOS 7.1 & Win Me)
24750                and     al,3 ; open_mode_mask ?
24751                ;cmp     al,0
24752                CMP      AL,open_for_read
24753                JNE      short Check_FCB_RO ;Is write or both
24754
24755                BadMode:
24756                jmp      SET_ACC_ERR
24757
24758                ; NOTE: The following check for writting to a Read Only File is performed
24759                ; ONLY on FCBs!!!!
24760                ; We ALLOW writes to Read only files via handles to allow a CREATE
24761                ; of a read only file which can then be written to.
24762                ; This is OK because we are NOT ALLOWED to OPEN a RO file via handles
24763                ; for writting, or RE-CREATE an EXISTING RO file via handles. Thus,
24764                ; CREATING a NEW RO file, or RE-CREATING an existing file which
24765                ; is NOT RO to be RO, via handles are the only times we can write
24766                ; to a read-only file.
24767
24768                Check_FCB_RO:
24769                ;;test   word [es:di+2],8000h
24770                ;TEST    word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB
24771                ;JZ      short WRITE_NO_MODE ; Not an FCB
24772
24773                ;test   byte [es:di+3],80h
24774                ;TEST    byte [ES:DI+SF_ENTRY.sf_mode+1],(sf_isFCB>>8)
24775                ;JZ      short WRITE_NO_MODE ; Not an FCB
24776
24777                ;test   byte [es:di+4],1
24778                ;TEST    byte [ES:DI+SF_ENTRY.sf_attr],attr_read_only
24779                ;JNZ     short BadMode ; Can't write to Read_Only files via FCB
24780
24781                WRITE_NO_MODE:
24782                call     SETUP
24783                call     IsSFTNet
24784                JZ       short LOCAL_WRITE
24785
24786                ;IF NOT Installed
24787                ; transfer NET_WRITE
24788                ;ELSE
24789                ;mov     ax,1109h

```



```

24786 00003D9B B80911      MOV     AX,(MULTNET<<8)|9
24787 00003D9E CD2F        int     2Fh      ; Multiplex - NETWORK REDIRECTOR - WRITE TO REMOTE FILE
24788                          ; ES:DI -> SFT
24789                          ; SFT DPB field -> DPB of drive containing file
24790                          ; CX = number of bytes, SS = DOS CS, SDA DTA field -> user buffer
24791                          ; Return: CF set on error, CX = bytes written
24792 00003DA0 C3          retn
24793 ;ENDIF
24794
24795 LOCAL_WRITE:
24796 ;;test word [es:di+5],80h
24797 ;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device
24798 ;jnz short WRTDEV
24799
24800 ;test byte [es:di+5],80h
24801 00003DA1 26F6450580    TEST    byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; Check for named device I/O
24802 00003DA6 756D        jnz     short WRTDEV
24803
24804 ;mov byte [EXTERR_LOCUS],2
24805 00003DA8 C606[2303]02  MOV     byte [EXTERR_LOCUS],errLOC_Disk
24806 00003DAD E832DB      call    ECritDisk
24807
24808 00003DB0 E8A805      call    DISKWRITE
24809
24810 ; 04/05/2019 - Retro DOS v4.0
24811
24812 ; MSDOS 6.0
24813 ; Extended Open
24814 00003DB3 7210        JC      short nocommit
24815
24816 00003DB5 C43E[9E05]    LES     DI,[THISSFT]
24817
24818 ;;test word [ES:DI+2],4000h
24819 ;TEST word [ES:DI+SF_ENTRY.sf_mode],AUTO_COMMIT_WRITE
24820 ;JZ short nocommit
24821
24822 ;test byte [ES:DI+3],40h
24823 00003DB9 26F6450340    TEST    byte [ES:DI+SF_ENTRY.sf_mode+1],(AUTO_COMMIT_WRITE>>8)
24824 00003DBE 7405        JZ      short nocommit
24825
24826 00003DC0 51          PUSH    CX
24827 00003DC1 E80FFB      call    DOS_COMMIT
24828 00003DC4 59          POP     CX
24829 nocommit:
24830 ; Extended Open
24831 ;call LCritDisk
24832 ;retn
24833 ; 18/12/2022
24834 00003DC5 E947DB      jmp     LCritDisk
24835
24836 DVWRTRAW:
24837 00003DC8 31C0        XOR     AX,AX          ; Media Byte, unit = 0
24838 00003DCA E87815      call    SETWRITE
24839 00003DCD 1E          PUSH    DS          ; Save seg of transfer
24840
24841 ;hkn; SS override
24842 00003DCE 36C536[9E05]    LDS     SI,[SS:THISSFT]
24843 00003DD3 E8B814      call    DEVIOCALL    ; DS:SI -> DEVICE
24844
24845 00003DD6 89FA        MOV     DX,DI          ; Offset part of xaddr saved in DX
24846 00003DD8 B487        MOV     AH,87H
24847
24848 ;hkn; SS override
24849 00003DDA 368B3E[5D03]    MOV     DI,[SS:DEVCALL_REQSTAT]
24850
24851 ; MSDOS 3.3
24852 ;test di,8000h
24853 ;jz short CWRTRK
24854
24855 ; MSDOS 6.0
24856 00003DDF 09FF      or      di,di
24857 00003DE1 791F      jns     short CWRTRK
24858
24859 ; MSDOS 3.3 (& MSDOS 6.0)
24860 00003DE3 E83822      call    CHARHARD
24861
24862 ; 04/05/2019 - Retro DOS v4.0
24863
24864 ; 08/02/2024
24865 00003DE6 368B3E[6C03]    mov     di,[ss:CALLSCNT]
24866 ; MSDOS 6.0
24867 ;sub cx,[ss:CALLSCNT] ; update ptr & count to reflect M065
24868 00003DEB 29F9      sub     cx,di
24869 00003DED 89D3      mov     bx,dx          ; number of chars xferred M065
24870 ;add bx,[ss:CALLSCNT] ; M065
24871 00003DEF 01FB      add     bx,di          ; M065
24872 00003DF1 89DF      mov     di,bx          ; M065
24873
24874 ; MSDOS 3.3
24875 ;MOV BX,DX ; Recall transfer addr M065
24876
24877 ; MSDOS 3.3 (& MSDOS 6.0)
24878 00003DF3 08C0      OR      AL,AL
24879 00003DF5 740B      JZ      short CWRTRK    ; Ignore
24880 00003DF7 3C03      CMP     AL,3
24881 00003DF9 7403      JZ      short CWRFERR
24882 00003DFB 1F        POP     DS          ; Recover saved seg of transfer
24883 00003DFC EBCA      JMP     short DVWRTRAW ; Try again
24884
24885 00003DFE 58        POP     AX          ; Chuck saved seg of transfer
24886 00003DFF E914FE      JMP     CRDFERR      ; Will pop one more stack element
24887
24888 00003E02 58        POP     AX          ; Chuck saved seg of transfer
24889 00003E03 1F        POP     DS
24890 00003E04 A1[6C03]    MOV     AX,[CALLSCNT] ; Get actual number of bytes transferred
24891
24892 00003E07 C43E[9E05]    LES     DI,[THISSFT]
24893 00003E0B 89C1      MOV     CX,AX
24894 ;call ADDREC
24895 ;retn
24896 ; 16/12/2022
24897 ; 10/06/2019
24898 00003E0D E9B404      jmp     ADDREC
24899 ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24900 ;call ADDREC
24901 ;retn
24902
24903 WRTNUL:
24904 00003E10 89CA      MOV     DX,CX          ; Entire transfer done
24905
24906 WRTCOOKJ:
24907 JMP     WRTCOOKDONE
24908
24909 WRTDEV:
24910 ;mov byte [EXTERR_LOCUS],4
24911 MOV     byte [EXTERR_LOCUS],errLOC_SerDev

```

```

24910 ;or byte [es:di+5],40h
24911 00003E1A 26804D0540 OR BYTE [ES:DI+SF_ENTRY.sf_flags],devid_device_EOF
24912 ; Reset EOF for input
24913 ;mov bl,[es:di+5]
24914 00003E1F 268A5D05 MOV BL,[ES:DI+SF_ENTRY.sf_flags]
24915 00003E23 31C0 XOR AX,AX
24916 00003E25 E3E0 JCXZ ENDWRDEV ; problem of creating on a device.
24917 00003E27 1E PUSH DS
24918 00003E28 88D8 MOV AL,BL
24919 00003E2A C51E[2C03] LDS BX,[DMAADD] ; Xaddr to DS:BX
24920 00003E2E 89DF MOV DI,BX ; Xaddr to DS:DI
24921 ; 27/06/2024 (PCDOS 7.1 IBMDOS.COM)
24922 ; 08/02/2024
24923 00003E30 99 cwd
24924 ;xor dx,dx ; Set starting point
24925 ;test al,20h
24926 00003E31 A820 test AL,devid_device_raw ; Raw?
24927 ;jz short TEST_DEV_CON
24928 ;jmp DVWRTRAW
24929 ; 16/12/2022
24930 00003E33 7593 jnz short DVWRTRAW
24931 ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24932 ;jz short TEST_DEV_CON
24933 ;jmp short DVWRTRAW
24934
24935 TEST_DEV_CON:
24936 ;test al,2
24937 00003E35 A802 test AL,devid_device_con_out ; Console output device?
24938 00003E37 756E jnz short WRITECON
24939 ;test al,4
24940 00003E39 A804 test AL,devid_device_null
24941 00003E3B 75D3 JNZ short WRTNUL
24942 00003E3D 89D0 MOV AX,DX
24943 00003E3F 803F1A CMP BYTE [BX],1Ah ; ^Z?
24944 00003E42 74CE JZ short WRTCOOKJ ; Yes, transfer nothing
24945 00003E44 51 PUSH CX
24946 00003E45 B90100 MOV CX,1
24947 00003E48 E8FA14 call SETWRITE
24948 00003E4B 59 POP CX
24949
24950 ;hkn; SS override
24951 00003E4C 36C536[9E05] LDS SI,[SS:THISSFT]
24952 ;
24953 ;SR; Removed x25 support from here
24954 ;
24955 ;lds si,[si+7]
24956 00003E51 C57407 LDS SI,[SI+SF_ENTRY.sf_devptr]
24957
24958 00003E54 E8A81F call DSKSTATCHK
24959 00003E57 E83714 call DEVIOCALL2
24960 00003E5A 57 PUSH DI
24961 00003E5B B487 MOV AH,87H
24962
24963 ;hkn; SS override
24964 00003E5D 368B3E[5D03] MOV DI,[SS:DEVCALL_REQSTAT]
24965
24966 ; MSDOS 3.3
24967 ;test di,8000h
24968 ;jz short CWROK
24969
24970 ; MSDOS 6.0
24971 00003E62 09FF or di,di
24972 00003E64 7916 jns short CWROK
24973
24974 ; MSDOS 3.3 (& MSDOS 6.0)
24975 00003E66 E8B521 call CHARHARD
24976 00003E69 5F POP DI
24977
24978 ;hkn; SS override
24979 00003E6A 36C706[6C03]0100 MOV word [SS:CALLSCNT],1
24980 00003E71 3C01 CMP AL,1
24981 00003E73 74DF JZ short DVWRTLP ; Retry
24982 00003E75 08C0 OR AL,AL
24983 00003E77 740C JZ short DVWRTIGN ; Ignore
24984 ; 10/08/2018
24985 00003E79 E99AFD JMP CRDFERR ; Fail, pops one stack element
24986
24987 00003E7C 5F CWROK: POP DI
24988
24989 ;hkn; SS override
24990 00003E7D 36833E[6C03]00 CMP word [SS:CALLSCNT],0
24991 00003E83 741C JZ short WRTCOOKDONE
24992
24993 00003E85 42 DVWRTIGN: INC DX
24994
24995 ;hkn; SS override for CALLXAD
24996 00003E86 36FF06[6803] INC WORD [SS:CALLXAD]
24997 00003E8B 47 INC DI
24998 00003E8C 1E PUSH DS
24999 00003E8D 368E1E[6A03] MOV DS,[SS:CALLXAD+2]
25000 00003E92 803D1A CMP BYTE [DI],1Ah ; ^Z?
25001 00003E95 1F POP DS
25002 00003E96 7409 JZ short WRTCOOKDONE
25003
25004 ;hkn; SS override
25005 00003E98 36C706[5D03]0000 MOV word [SS:DEVCALL_REQSTAT],0
25006 00003E9F E2B3 LOOP DVWRTLP
25007
25008 00003EA1 89D0 WRTCOOKDONE: MOV AX,DX
25009 00003EA3 1F POP DS
25010 00003EA4 E960FF JMP ENDWRDEV ; 10/08/2018
25011
25012 WRITECON:
25013 00003EA7 1E PUSH DS
25014
25015 ;hkn; SS is DOSDATA
25016 00003EA8 16 push ss
25017 00003EA9 1F pop ds
25018 00003EAA E8B2FC CALL SWAPCON
25019 00003EAD 1F POP DS
25020 00003EAE 89DE MOV SI,BX
25021 00003EB0 51 PUSH CX
25022
25023 00003EB1 AC WRCONLP: LODSB
25024 00003EB2 3C1A CMP AL,1Ah ; ^Z?
25025 00003EB4 7405 JZ short CONEOF
25026 00003EB6 E878DD call OUTT
25027 00003EB9 E2F6 LOOP WRCONLP
25028
25029 00003EBB 58 CONEOF: POP AX ; Count
25030 00003EBC 1F POP DS
25031 00003EBD 29C8 SUB AX,CX ; Amount actually written
25032 00003EBF E897FC CALL SWAPBACK
25033 00003EC2 E942FF JMP ENDWRDEV

```

```

25034
25035
25036
25037
25038
25039
25040
25041
25042
25043
25044
25045
25046
25047
25048
25049
25050
25051
25052 00003EC5 36803E[5703]00
25053 00003ECB 7512
25054
25055 00003ECD 16
25056 00003ECE 1F
25057 00003ECF 06
25058 00003ED0 57
25059 00003ED1 E8FA37
25060 00003ED4 7206
25061 00003ED6 8CC6
25062 00003ED8 8EDE
25063 00003EDA 89FE
25064
25065 00003EDC 5F
25066 00003EDD 07
25067 00003EDE C3
25068
25069 00003EDF 83FB01
25070 00003EE2 77E9
25071 00003EE4 36C536[E605]
25072 00003EE9 F8
25073
25074 00003EEA C3
25075
25076
25077
25078
25079
25080
25081
25082
25083
25084
25085
25086
25087
25088
25089
25090
25091
25092
25093
25094
25095
25096
25097
25098
25099
25100
25101
25102
25103
25104
25105
25106
25107
25108
25109 00003EEB 31D2
25110
25111 00003EED 3916[DC0A]
25112 00003EF1 7509
25113
25114
25115 00003EF3 3916[C205]
25116 00003EF7 7503
25117 00003EF9 92
25118 00003EFA EB0C
25119
25120
25121
25122
25123 00003EFC 88C2
25124
25125 00003EFE 26225604
25126
25127
25128
25129
25130 00003F02 268A4E05
25131 00003F06 D3E8
25132
25133
25134
25135
25136
25137 00003F08 8816[7305]
25138 00003F0C 89C1
25139 00003F0E 88D4
25140
25141
25142
25143
25144
25145
25146
25147
25148
25149
25150
25151
25152
25153
25154
25155
25156
25157

;-----
; Procedure Name : get_io_sft
;
; Convert JFN number in BX to sf_entry in DS:SI we get the normal SFT if
; CONSWAP is FALSE or if the handle desired is 2 or more. Otherwise, we
; retrieve the sft from ConsFT which is set by SwapCon.
;-----

; 04/05/2019 - Retro DOS v4.0
; DOSCODE:7583h (MSDOS 6.21, MSDOS.SYS)
; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:756Fh (MSDOS 5.0, MSDOS.SYS)

GET_IO_SFT:
    ;test byte [SS:CONSWAP],0FFh
    cmp byte [SS:CONSWAP],0 ;smr;SS Override
    jnz short GetRedir
GetNormal:
    push ss
    pop ds
    PUSH ES
    PUSH DI
    call SFFromHandle
    JC short RET44P
    MOV SI,ES
    MOV DS,SI
    MOV SI,DI
RET44P:
    POP DI
    POP ES
    retn
GetRedir:
    CMP BX,1
    JA short GetNormal
    LDS SI,[SS:CONSFT]
    CLC
get_io_sft_retn:
    retn

;Break <DIRREAD -- READ A DIRECTORY SECTOR>
;-----
; Procedure Name : DIRREAD
;
; Inputs:
; AX = Directory block number (relative to first block of directory)
; ES:BP = Base of drive parameters
; [DIRSEC] = First sector of first cluster of directory
; [CLUSNUM] = Next cluster
; [CLUSFAC] = Sectors/Cluster
; Function:
; Read the directory block into [CURBUF].
; Outputs:
; [NXTCLUSNUM] = Next cluster (after the one skipped to)
; [SECCLUSPOS] Set
; ES:BP unchanged
; [CURBUF] Points to Buffer with dir sector
; Carry set if error (user said FAIL to I 24)
; DS preserved, all other registers destroyed.
;-----

;hkn; called from dir.asm. DS already set up to DOSDATA.

; 08/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)

DIRREAD:
; Note that ClusFac is the sectors per cluster. This is NOT necessarily
; the same as what is in the DPB! In the case of the root directory, we have
; ClusFac = # sectors in the root directory. The root directory is detected
; by DIRStart = 0.

    XOR DX,DX
    ; 08/02/2024 (PCDOS 7.1)
    cmp [DIRSTART_HW],dx
    jnz short SubDir
    ;CMP word [DIRSTART],0
    ; 21/09/2023
    cmp [DIRSTART],dx ; 0
    jnz short SubDir
    XCHG AX,DX
    JMP short DoRead

; Convert the sector number in AX into cluster and sector-within-cluster pair

SubDir:
    MOV DL,AL
    and dl,[es:bp+4]
    AND DL,[ES:BP+DPB.CLUSTER_MASK]

; (DX) = sector-in-cluster

    ;mov cl,[es:bp+5]
    MOV CL,[ES:BP+DPB.CLUSTER_SHIFT]
    SHR AX,CL

; (DX) = position in cluster
; (AX) = number of clusters to skip

DoRead:
    MOV [SECCLUSPOS],DL
    MOV CX,AX
    MOV AH,DL

; (CX) = number of clusters to skip.
; (AH) = remainder

; 04/05/2019 - Retro DOS v4.0

; 08/02/2024
%if 0
; MSDOS 6.0
; MOV DX,[DIRSEC+2] ;>32mb
; MOV [HIGH_SECTOR],DX ;>32mb
; MOV DX,[DIRSEC]
; ADD DL,AH
; ADC DH,0
; ADC word [HIGH_SECTOR],0 ;>32mb
; 21/09/2023
xor bx,bx ; 0
mov dx,[DIRSEC]

```

```

25158      add     dl,ah
25159      adc     dh,b1 ; 0
25160      adc     bx,[DIRSEC+2]
25161      mov     [HIGH_SECTOR],bx
25162
25163      MOV     BX,[CLUSNUM]
25164      MOV     [NXTCLUSNUM],BX
25165      JCXZ    FIRSTCLUSTER
25166
25167      %else
25168      ; 08/02/2024 (PCDOS 7.1)
25169      mov     bx,[CLUSNUM_HW]
25170      mov     [NXTCLUSNUM_HW],bx
25171      mov     [CLUSTNUM_HW],bx
25172      mov     dx,[DIRSEC+2]
25173      mov     [HIGH_SECTOR],dx
25174      mov     dx,[DIRSEC]
25175      add     dl,ah
25176      adc     dh,0
25177      adc     word [HIGH_SECTOR],0
25178      mov     bx,[CLUSNUM]
25179      mov     [NXTCLUSNUM],bx
25180      jcxz    FIRSTCLUSTER
25181
25182      %endif
25183
25184      SKPCLLP:
25185      call    UNPACK
25186      jc      short get_io_sft_retn
25187      ;;;
25188      ; 08/02/2024 (PCDOS 7.1)
25189      push    word [CLUSTNUM_HW]
25190      pop     word [CLUSTERS_HW]
25191      push    word [CONTENT_HW]
25192      pop     word [CLUSTNUM_HW]
25193      ;;;
25194      XCHG    BX,DI
25195      call    ISEOF ; test for eof based on fat size
25196      JAE     short HAVESKIPPED
25197      LOOP    SKPCLLP
25198
25199      HAVESKIPPED:
25200      MOV     [NXTCLUSNUM],BX
25201      ;;;
25202      ; 08/02/2024 (PCDOS 7.1)
25203      mov     bx,[CLUSTNUM_HW]
25204      mov     [NXTCLUSNUM_HW],bx
25205      ;
25206      mov     bx,[CLUSTERS_HW]
25207      mov     [CLUSTNUM_HW],bx
25208      ;;;
25209      MOV     DX,DI
25210      MOV     BL,AH
25211      call    FIGREC
25212
25213      ;entry FIRSTCLUSTER
25214
25215      FIRSTCLUSTER:
25216      ; 22/09/2023
25217      ;;mov     byte [ALLOWED],18h
25218      ;MOV     byte [ALLOWED],Allowed_RETRY+Allowed_FAIL ; *
25219      ;XOR     AL,AL ; * ; Indicate pre-read
25220      ;call    GETBUFFER
25221      call    GETBUFFER ; * ; pre-read
25222      ;jc      short get_io_sft_retn
25223      ; 08/02/2024
25224      jc      short dirread_retn
25225
25226      ;entry SET_BUF_AS_DIR
25227
25228      SET_BUF_AS_DIR:
25229
25230      ; Set the type of CURBUF to be a directory sector.
25231      ; Only flags are modified.
25232
25233      PUSH     DS
25234      PUSH     SI
25235      LDS     SI,[CURBUF]
25236      or      byte [si+5],4
25237      OR      byte [SI+BUFFINFO.buf_flags],buf_isDIR ; clears carry
25238      POP     SI
25239      POP     DS
25240
25241      dirread_retn:
25242      retn
25243
25244      ;Break <FATSECRD -- READ A FAT SECTOR>
25245      -----
25246      ;
25247      ; Procedure Name : FATSECRD
25248      ; Inputs:
25249      ; Same as DREAD
25250      ; DS:BX = Transfer address
25251      ; CX = Number of sectors
25252      ; DX = Absolute record number
25253      ; ES:BP = Base of drive parameters
25254      ; Function:
25255      ; Calls BIOS to perform FAT read.
25256      ; Outputs:
25257      ; Same as DREAD
25258      -----
25259
25260      ; 04/05/2019 - Retro DOS v4.0
25261      ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25262
25263      ; 09/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
25264
25265      FATSECRD:
25266      ;hkn; SS override
25267      ;mov     byte [ss:ALLOWED],18h
25268      MOV     byte [SS:ALLOWED],Allowed_RETRY+Allowed_FAIL
25269      MOV     DI,CX
25270      ;mov     cl,[es:bp+8]
25271      MOV     CL,[ES:BP+DPB.FAT_COUNT]
25272      ; MSDOS 3.3
25273      ;;mov     al,[es:bp+0Fh]
25274      ;MOV     AL,[ES:BP+DPB.FAT_SIZE]
25275      ;XOR     AH,AH
25276      ; MSDOS 6.0
25277      ;mov     ax,[es:bp+0Fh]
25278      MOV     AX,[ES:BP+DPB.FAT_SIZE] ;>32mb
25279      XOR     CH,CH
25280      ;;;
25281      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
25282      or      ax,ax
25283      jnz     short FATSECRD_cont ; not FAT32
25284      ;test    byte [es:bp+23h],80h
25285      test    byte [es:bp+DPB.EXT_FLAGS],80h ; (FAT32 bs extd flags bit 7)

```

```

25282 00003FA2 7402          jz      short FATSECRD_cont
25283                      ;mov     cx,1                      ; only one FAT is active
25284 00003FA4 B101          mov     cx,1
25285                      FATSECRD_cont:
25286 00003FA6 36FF36[0706]   push    word [ss:HIGH_SECTOR]
25287                      ;;;
25288 00003FAB 52              PUSH     DX
25289                      NXTFAT:
25290                      ; MSDOS 6.0
25291                      ;hkn; SS override
25292                      ; 09/02/2024 (PCDOS 7.1)
25293                      ;MOV     word [SS:HIGH_SECTOR],0      ;>32mb FAT sectors cannot exceed
25294 00003FAC 51              PUSH     CX                      ;32mb
25295 00003FAD 50              PUSH     AX
25296                      ;;;
25297                      ; 08/02/2024 (PCDOS 7.1 IBMDOS.COM)
25298 00003FAE 36FF36[0706]   push    word [ss:HIGH_SECTOR]
25299 00003FB3 52              push    dx
25300                      ;;;
25301 00003FB4 89F9            MOV      CX,DI
25302 00003FB6 E88600          call    DSKREAD
25303                      ;;;
25304                      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
25305 00003FB9 5A              pop      dx
25306 00003FBA 368F06[0706]   pop      word [ss:HIGH_SECTOR]
25307                      ;;;
25308 00003FBF 58              POP      AX
25309 00003FC0 59              POP      CX
25310 00003FC1 7440            JZ       short RET41P          ; Carry clear
25311                      ;;;
25312                      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
25313 00003FC3 09C0            or       ax,ax
25314 00003FC5 7511            jnz      short NXTFAT2      ; not FAT32
25315                      ;add     dx,[es:bp+31h]
25316 00003FC7 26035631        add      dx,[es:bp+DPB.FAT32_SIZE]
25317 00003FCB 50              push    ax
25318                      ;mov     ax,[es:bp+33h]
25319 00003FCC 268B4633        mov      ax,[es:bp+DPB.FAT32_SIZE+2]
25320 00003FD0 361106[0706]   adc      [ss:HIGH_SECTOR],ax
25321 00003FD5 58              pop      ax
25322 00003FD6 EB08            jmp      short NXTFAT3
25323                      NXTFAT2:
25324                      ;;;
25325 00003FD8 01C2            ADD      DX,AX
25326                      ;;;
25327                      ; 09/02/2024 (PCDOS 7.1)
25328 00003FDA 368316[0706]00  adc      word [ss:HIGH_SECTOR],0
25329                      NXTFAT3:
25330                      ;;;
25331 00003FE0 E2CA            LOOP     NXTFAT
25332 00003FE2 5A              POP      DX
25333                      ;;;
25334                      ; 09/02/2024 (PCDOS 7.1)
25335 00003FE3 368F06[0706]   pop      word [ss:HIGH_SECTOR]
25336                      ;;;
25337 00003FE8 89F9            MOV      CX,DI
25338                      ;
25339                      ; NOTE FALL THROUGH
25340                      ;
25341                      ;Break      <DREAD -- DO A DISK READ>
25342                      ;-----
25343                      ;
25344                      ; Procedure Name : DREAD
25345                      ;
25346                      ; Inputs:
25347                      ; DS:BX = Transfer address
25348                      ; CX = Number of sectors
25349                      ; DX = Absolute record number (LOW)
25350                      ; [HIGH_SECTOR] = Absolute record number (HIGH)
25351                      ; ES:BP = Base of drive parameters
25352                      ; [ALLOWED] must be set in case call to HARDERR needed
25353                      ; Function:
25354                      ; Calls BIOS to perform disk read. If BIOS reports
25355                      ; errors, will call HARDERRRW for further action.
25356                      ; Outputs:
25357                      ; Carry set if error (currently user FAILED to INT 24)
25358                      ; DS,ES:BP preserved. All other registers destroyed.
25359                      ;-----
25360                      ;
25361                      ;entry DREAD
25362                      DREAD:
25363 00003FEA E85200          call    DSKREAD
25364 00003FED 7497            jz       short dirread_retn    ; Carry clear
25365                      ;hkn; SS override
25366 00003FEF 36C606[7505]00  MOV      BYTE [SS:READOP],0    ; Read
25367 00003FF5 E89A00          call    HARDERRRW
25368 00003FF8 3C01            CMP      AL,1                  ; Check for retry
25369 00003FFA 74EE            JZ       short DREAD
25370                      ;
25371                      fail_ignore:
25372 00003FFC 3C03            CMP      AL,3                  ; Check for FAIL
25373 00003FFE F8              CLC
25374 00003FFF 7501            JNZ      short NO_CAR          ; Ignore
25375 00004001 F9              STC
25376                      NO_CAR:
25377 00004002 C3              retn
25378                      ;
25379                      ; 09/02/2024 - Retro DOS v5.0
25380                      RET41P:
25381 00004003 5A              POP      DX
25382                      ;;;
25383                      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
25384 00004004 368F06[0706]   pop      word [ss:HIGH_SECTOR]
25385                      ;;;
25386 00004009 C3              retn
25387                      ;
25388                      ; 24/07/2018 - Retro DOS v3.0
25389                      ;
25390                      ;Break      <CHECK_WRITE_LOCK>
25391                      ;-----
25392                      ;
25393                      ; Procedure Name : CHECK_WRITE_LOCK
25394                      ;
25395                      ; Inputs:
25396                      ; output of SETUP
25397                      ; ES:DI -> SFT
25398                      ; Function:
25399                      ; check write lock
25400                      ; Outputs:
25401                      ; Carry set if error
25402                      ; Carry clear if ok
25403                      ;
25404                      ;-----
25405

```

```

25406 ; 04/05/2019 - Retro DOS v4.0
25407 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25408
25409 CHECK_WRITE_LOCK:
25410 ; MSDOS 6.0
25411 ; test byte [es:di+4],8
25412 0000400A 26F6450408 TEST byte [ES:DI+SF_ENTRY.sf_attr],attr_volume_id ;volume id
25413 ; JZ short write_cont ;no
25414 ; call SET_ACC_ERR_DS
25415 ; retn
25416 ; jnz SET_ACC_ERR_DS
25417 ; 19/08/2018
25418 ; jz short write_cont
25419 ; jmp SET_ACC_ERR_DS
25420 ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
25421 0000400F 7403 JZ short write_cont
25422 ; call SET_ACC_ERR_DS
25423 ; retn
25424 ; 16/12/2022
25425 00004011 E9D201 jmp SET_ACC_ERR_DS
25426
25427 write_cont: ;
25428 00004014 51 PUSH CX ;save reg
25429 00004015 09C9 OR CX,CX ;
25430 00004017 7501 JNZ short Not_Truncate ;
25431 00004019 49 dec cx ;(cx) = -1; check for lock on whole file
25432 Not_Truncate: ;
25433 0000401A B080 MOV AL,80H ;check write access
25434 0000401C E8C043 call LOCK_CHECK ;check lock
25435 0000401F 59 POP CX ;restore reg
25436 00004020 7305 JNC short WRITE_OK ;lock ok
25437 00004022 E87301 call WRITE_LOCK_VIOLATION ;issue I24
25438 00004025 73ED JNC short write_cont ;retry
25439 WRITE_OK: ;
25440 00004027 C3 retn ;
25441
25442 ;Break <CHECK_READ_LOCK>
25443 -----
25444 ;
25445 ; Procedure Name : CHECK_READ_LOC
25446 ;
25447 ; Inputs:
25448 ; ES:DI -> SFT
25449 ; output of SETUP
25450 ; Function:
25451 ; check read lock
25452 ; Outputs:
25453 ; Carry set if error
25454 ; Carry clear if ok
25455 ;-----
25456
25457 CHECK_READ_LOCK:
25458 ; MSDOS 6.0
25459 ; test byte [es:di+4],8
25460 00004028 26F6450408 TEST byte [ES:DI+SF_ENTRY.sf_attr],attr_volume_id ;volume id
25461 ; JZ short do_retry ; no
25462 ; call SET_ACC_ERR
25463 ; retn
25464 ; jnz SET_ACC_ERR
25465 ; 16/12/2022
25466 ; 28/07/2019
25467 0000402D 7403 jz short do_retry
25468 0000402F E9B601 jmp SET_ACC_ERR
25469 ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
25470 ; JZ short do_retry
25471 ; call SET_ACC_ERR
25472 ; retn
25473 do_retry: ;
25474 00004032 30C0 xor al,al ;check read access
25475 00004034 E8A843 call LOCK_CHECK ;check lock
25476 00004037 7305 JNC short READLOCK_OK ;lock ok
25477 00004039 E83C01 call READ_LOCK_VIOLATION ;issue I24
25478 0000403C 73F4 JNC short do_retry ;retry
25479 READLOCK_OK: ;
25480 dw_ret_label: ; 09/02/2024 ;
25481 0000403E C3 retn ;
25482
25483 ;=====
25484 ; DISK2.ASM, MSDOS 6.0, 1991
25485 ;=====
25486 ; 24/07/2018 - Retro DOS v3.0
25487 ; 04/05/2019 - Retro DOS v4.0
25488
25489 ; TITLE DISK2 - Disk utility routines
25490 ; NAME Disk2
25491
25492 ;** Low level Read and write routines for local SFT I/O on files and devs
25493 ;
25494 ; DskRead
25495 ; DWRITE
25496 ; DSKWRITE
25497 ; HarderrRW
25498 ; SETUP
25499 ; BREAKDOWN
25500 ; READ_LOCK_VIOLATION
25501 ; WRITE_LOCK_VIOLATION
25502 ; DISKREAD
25503 ; SET_ACC_ERR_DS
25504 ; SET_ACC_ERR
25505 ; SETSFT
25506 ; SETCLUS
25507 ; AddRec
25508 ;
25509 ; Revision history:
25510 ;
25511 ; AN000 version 4.00 Jan. 1988
25512 ; M039 DB 10/17/90 - Disk read/write optimization
25513 ;
25514 ; 04/05/2019 - Retro DOS v4.0
25515 ; DOSCODE:7699h (MSDOS 6.21, MSDOS.SYS)
25516 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25517 ; DOSCODE:7685h (MSDOS 5.0, MSDOS.SYS)
25518
25519 ;Break <DSKREAD -- PHYSICAL DISK READ>
25520 -----
25521 ;
25522 ; Procedure Name : DSKREAD
25523 ;
25524 ; Inputs:
25525 ; DS:BX = Transfer addr
25526 ; CX = Number of sectors
25527 ; [HIGH_SECTOR] = Absolute record number (HIGH)
25528 ; DX = Absolute record number (LOW)
25529 ; ES:BP = Base of drive parameters

```



```

25530 ; Function:
25531 ; Call BIOS to perform disk read
25532 ; Outputs:
25533 ; DI = CX on entry
25534 ; CX = Number of sectors unsuccessfully transfered
25535 ; AX = Status word as returned by BIOS (error code in AL if error)
25536 ; Zero set if OK (from BIOS) (carry clear)
25537 ; Zero clear if error (carry clear)
25538 ; SI Destroyed, others preserved
25539 ;-----
25540
25541 DSKREAD:
25542 0000403F 51      PUSH    CX
25543 ;mov    ah,[es:bp+17h] ; 04/05/2019
25544 00004040 268A6617 MOV     AH,[ES:BP+DPB.MEDIA]
25545 ;mov    al,[es:bp+1]
25546 00004044 268A4601 MOV     AL,[ES:BP+DPB.UNIT]
25547 00004048 53      PUSH    BX
25548 00004049 06      PUSH    ES
25549 0000404A E8C512 CALL    SETREAD
25550 0000404D EB22      JMP     short DODSKOP
25551
25552 ;Break    <DWRITE -- SEE ABOUT WRITING>
25553 ;-----
25554 ;
25555 ; Procedure Name : DWRITE
25556 ;
25557 ; Inputs:
25558 ; DS:BX = Transfer address
25559 ; CX = Number of sectors
25560 ; [HIGH_SECTOR] = Absolute record number (HIGH)
25561 ; DX = Absolute record number (LOW)
25562 ; ES:BP = Base of drive parameters
25563 ; [ALLOWED] must be set in case HARDERR called
25564 ; Function:
25565 ; Calls BIOS to perform disk write. If BIOS reports
25566 ; errors, will call HARDERRRW for further action.
25567 ; Output:
25568 ; Carry set if error (currently, user FAILED to I 24)
25569 ; BP preserved. All other registers destroyed.
25570 ;-----
25571 ;
25572 ;entry DWRITE
25573 DWRITE:
25574 0000404F E81100 CALL    DSKWRITE
25575 00004052 74EA      JZ     short dw_ret_label ; Carry clear (retz)
25576
25577 ;hkn; SS override
25578 00004054 36C606[7505]01 MOV     BYTE [SS:READOP],1 ; write
25579 0000405A E83500 CALL    HARDERRRW
25580 0000405D 3C01      CMP     AL,1 ; Check for retry
25581 0000405F 74EE      JZ     short DWRITE
25582
25583 ; 09/02/2024
25584 %if 0
25585 CMP     AL,3 ; Check for FAIL
25586 CLC
25587 JNZ     short NO_CAR2 ; Ignore
25588 STC
25589 NO_CAR2:
25590 dw_ret_label:
25591 retn
25592 %else
25593 ; 09/02/2024 - Retro DOS v5.0
25594 00004061 EB99      JMP     short fail_ignore
25595 %endif
25596
25597 ;Break    <DSKWRITE -- PHYSICAL DISK WRITE>
25598 ;-----
25599 ;
25600 ; Procedure Name : DSKWRITE
25601 ;
25602 ; Inputs:
25603 ; DS:BX = Transfer addr
25604 ; CX = Number of sectors
25605 ; DX = Absolute record number (LOW)
25606 ; [HIGH_SECTOR] = Absolute record number (HIGH)
25607 ; ES:BP = Base of drive parameters
25608 ; Function:
25609 ; Call BIOS to perform disk read
25610 ; Outputs:
25611 ; DI = CX on entry
25612 ; CX = Number of sectors unsuccessfully transfered
25613 ; AX = Status word as returned by BIOS (error code in AL if error)
25614 ; Zero set if OK (from BIOS) (carry clear)
25615 ; Zero clear if error (carry clear)
25616 ; SI Destroyed, others preserved
25617 ;-----
25618 ;
25619 ;
25620 ;entry DSKWRITE
25621 DSKWRITE:
25622 00004063 51      PUSH    CX
25623 ;mov    ah,[es:bp+17h] ; 04/05/2019
25624 00004064 268A6617 MOV     AH,[ES:BP+DPB.MEDIA]
25625 ;mov    al,[es:bp+1]
25626 00004068 268A4601 MOV     AL,[ES:BP+DPB.UNIT]
25627 0000406C 53      PUSH    BX
25628 0000406D 06      PUSH    ES
25629 0000406E E8D412 CALL    SETWRITE
25630
25631 DODSKOP:
25632 00004071 8CD9      MOV     CX,DS ; Save DS
25633 00004073 1F      POP     DS ; DS:BP points to DPB
25634 00004074 1E      PUSH    DS
25635 ;lds    si,[ds:bp+13h] ; 04/05/2019
25636 00004075 3EC57613 LDS     SI,[ds:BP+DPB.DRIVER_ADDR] ; 07/09/2018
25637 00004079 E81512 CALL    DEVIOCALL2
25638
25639 MOV     DS,CX ; Restore DS
25640 0000407E 07      POP     ES ; Restore ES
25641 0000407F 5B      POP     BX
25642
25643 ;hkn; SS override
25644 00004080 368B0E[6C03] MOV     CX,[SS:CALLSCNT] ; Number of sectors transferred
25645 00004085 5F      POP     DI
25646 00004086 29F9      SUB     CX,DI
25647 00004088 F7D9      NEG     CX ; Number of sectors not transferred
25648
25649 ;hkn; SS override
25650 0000408A 36A1[5D03] MOV     AX,[SS:DEVCALL_REQSTAT]
25651 ;test    ax,8000h
25652 ; 17/12/2022
25653 ;test    ah,80h

```

```

25654 0000408E F6C480      test    ah,(STERR>>8)
25655                      ;test    AX,STERR
25656 00004091 C3          retn
25657
25658                      ;Break    <HardErrRW - map extended errors and call harderr>
25659                      -----
25660                      ;
25661                      ; Procedure Name : HardErrRW
25662                      ;
25663                      ; Inputs:
25664                      ;   AX is error code from read or write
25665                      ;   Other registers set as per HARDERR
25666                      ; Function:
25667                      ;   Checks the error code for special extended
25668                      ;   errors and maps them if needed. Then invokes
25669                      ;   Harderr
25670                      ; Outputs:
25671                      ;   Of HARDERR
25672                      ;   AX may be modified prior to call to HARDERR.
25673                      ;   No other registers altered.
25674                      ;
25675                      -----
25676                      ;
25677                      ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25678                      HARDERRRW:
25679                      ;cmp     al,0Fh
25680 00004092 3C0F          CMP     AL,error_I24_wrong_disk
25681 00004094 7512          JNZ     short DO_ERR          ; Nothing to do
25682
25683                      ; MSDOS 3.3
25684                      ;push    ds
25685                      ;push    si
25686                      ;lds     si,[ss:CALLVIDRW]
25687                      ;mov     [ss:EXTERRPT+2],ds
25688                      ;mov     [ss:EXTERRPT],si
25689                      ;pop     si
25690                      ;pop     ds
25691
25692                      ; MSDOS 6.0
25693 00004096 50          push    ax
25694 00004097 36A1[7003]   mov     ax,[ss:CALLVIDRW]          ; get ptr lo ;smr;SS Override
25695 0000409B 36A3[2803]   mov     [ss:EXTERRPT],ax          ; set ext err ptr lo
25696 0000409F 36A1[7203]   mov     ax,[ss:CALLVIDRW+2]      ; get ptr hi from dev
25697 000040A3 36A3[2A03]   mov     [ss:EXTERRPT+2],ax      ; set ext err ptr hi
25698 000040A7 58          pop     ax
25699                      DO_ERR:
25700                      ;;call  HARDERR
25701                      ;;retn
25702                      ; 16/12/2022
25703                      ; 10/06/2019
25704 000040A8 E9A41F      jmp     HARDERR
25705                      ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
25706                      ;call  HARDERR
25707                      ;retn
25708
25709                      ; 24/07/2018 - Retro DOS v3.0
25710
25711                      ;Break    <SETUP -- SETUP A DISK READ OR WRITE FROM USER>
25712                      -----
25713                      ;
25714                      ; Procedure Name : SETUP
25715                      ;
25716                      ; Inputs:
25717                      ;   ES:DI point to SFT (value also in THISSFT)
25718                      ;   DMAAdd contains transfer address
25719                      ;   CX = Byte count
25720                      ;   DS = DOSDATA
25721                      ;   WARNING Stack must be clean, two ret addrs on stack, 1st of caller,
25722                      ;   2nd of caller of caller.
25723                      ; Outputs:
25724                      ;   CX = byte count
25725                      ;   [THISDPB] = Base of drive parameters if file
25726                      ;   = Pointer to device header if device or NET
25727                      ;   ES:DI Points to SFT
25728                      ;   [NEXTADD] = Displacement of disk transfer within segment
25729                      ;   [TRANS] = 0 (No transfers yet)
25730                      ;   BytPos = Byte position in file
25731                      ;
25732                      ; The following fields are relevant to local files (not devices) only:
25733                      ;
25734                      ;   SecPos = Position of first sector (local files only)
25735                      ;   [BYTSECPOS] = Byte position in first sector (local files only)
25736                      ;   [CLUSNUM] = First cluster (local files only)
25737                      ;   [SECCLUSPOS] = Sector within first cluster (local files only)
25738                      ;   [THISDRV] = Physical unit number (local files only)
25739                      ;
25740                      ; RETURNS ONE LEVEL UP WITH:
25741                      ;   CX = 0
25742                      ;   CARRY = Clear
25743                      ; IF AN ERROR IS DETECTED
25744                      ; All other registers destroyed
25745                      -----
25746
25747                      ;hkn; called from disk.asm. DS has been set up to DOSDATA.
25748
25749                      ; DOSCODE:770Bh (MSDOS 6.21, MSDOS.SYS)
25750
25751                      ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25752                      ; DOSCODE:76F7h (MSDOS 5.0, MSDOS.SYS)
25753
25754                      ; 09/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
25755                      ; DOSCODE:80D8h (PCDOS 7.1, IBMDOS.COM)
25756
25757                      SETUP:
25758                      ; IBMDOS.COM (MSDOS 3.3) - Offset 411Bh
25759
25760                      ;lds     si,[es:di+7]
25761 000040AB 26C57507      LDS     SI,[ES:DI+SF_ENTRY.sf_devptr]
25762
25763                      ;hkn; SS override
25764 000040AF 368C1E[8C05]   MOV     [ss:THISDPB+2],ds
25765
25766                      ;hkn; SS is DOSDATA
25767 000040B4 16          push    ss
25768 000040B5 1F          pop     ds
25769
25770 000040B6 8936[8A05]   MOV     [THISDPB],SI
25771
25772 000040BA 8B1E[2C03]   MOV     BX,[DMAADD]
25773 000040BE 891E[B805]   MOV     [NEXTADD],BX          ;Set NEXTADD to start of xaddr
25774 000040C2 C606[7405]00  MOV     BYTE [TRANS],0        ;No transfers
25775                      ;mov     ax,[es:di+15h]
25776 000040C7 268B4515      MOV     AX,[ES:DI+SF_ENTRY.sf_position]
25777                      ;mov     dx,[es:di+17h]

```



```

25778 000040CB 268B5517      MOV     DX,[ES:DI+SF_ENTRY.sf_position+2]
25779 000040CF 8916[D005]      MOV     [BYTPOS+2],DX      ;Set it
25780 000040D3 A3[CE05]        MOV     [BYTPOS],AX
25781                                ;test word [es:di+5],8080h
25782 000040D6 26F745058080    TEST    word [ES:DI+SF_ENTRY.sf_flags],sf_isnet+devid_device
25783 000040DC 7555          JNZ     short NOSETSTUFF    ;Following not done on devs or NET
25784 000040DE 06          PUSH    ES
25785 000040DF C42E[8A05]      LES     BP,[THISDPB]      ;Point at the DPB
25786
25787                                ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
25788                                ;;mov bl,[es:bp+0]
25789                                ;MOV    BL,[ES:BP+DPB.DRIVE]
25790                                ; 05/12/2022
25791 000040E3 268A5E00    mov     bl,[es:bp]
25792
25793 000040E7 881E[7605]    MOV     [THISDRV],BL      ;Set THISDRV
25794                                ;mov bx,[es:bp+2]
25795 000040EB 268B5E02    MOV     BX,[ES:BP+DPB.SECTOR_SIZE]
25796
25797                                ;; MSDOS 3.3
25798                                ;cmp dx,bx
25799                                ;jnb short EOFERR
25800                                ;div bx
25801                                ;mov [SECPPOS],ax
25802                                ;mov [BYTSECPPOS],dx
25803                                ;mov dx,ax
25804                                ;;and al,[es:bp+4]
25805                                ;AND    AL,[ES:BP+DPB.CLUSTER_MASK]
25806                                ;mov [SECCCLUSPOS],al
25807                                ;mov ax,cx
25808                                ;;mov cl,[es:bp+5]
25809                                ;MOV    CL,[ES:BP+DPB.CLUSTER_SHIFT]
25810                                ;shr dx,cl
25811                                ;mov [CLUSNUM],dx
25812                                ;pop es
25813                                ;mov cx,ax
25814
25815                                ; 04/05/2019 - Retro DOS v4.0
25816
25817                                ; MSDOS 6.0
25818                                ;M039: Optimized this section.
25819 000040EF 51          PUSH    CX      ;SHR32 and DIV32 use CX.
25820 000040F0 E81406      call    DIV32      ;DX:AX/BX = CX:AX + DX (rem)
25821 000040F3 8916[CC05]    MOV     [BYTSECPPOS],DX
25822 000040F7 A3[C405]      MOV     [SECPPOS],AX
25823 000040FA 890E[C605]    MOV     [SECPPOS+2],CX
25824 000040FE 89CA      MOV     DX,CX
25825
25826 00004100 89C3      MOV     BX,AX
25827                                ;and bl,[es:bp+4]
25828 00004102 26225E04    AND     BL,[ES:BP+DPB.CLUSTER_MASK]
25829 00004106 881E[7305]    MOV     [SECCCLUSPOS],BL
25830
25831 0000410A E82106      call    SHR32      ;(DX:AX SHR dpb_cluster_shift)
25832 0000410D 59          POP     CX      ;CX = byte count.
25833
25834                                ; 09/02/2024 (PCDOS7.1 IBMDOS.COM)
25835                                ;JNZ short EOFERR      ;cluster number above 64k
25836                                ;;;
25837                                ;cmp word [es:bp+0Fh],0
25838 0000410E 26837E0F00    cmp     word [es:bp+DPB.FAT_SIZE],0
25839 00004113 750C      jnz     short setup_1    ; not FAT32 fs
25840                                ;cmp dx,[es:bp+2Fh]
25841 00004115 263B562F    cmp     dx,[es:bp+DPB.LAST_CLUSTER+2]
25842 00004119 750E      jne     short setup_2
25843                                ;cmp ax,[es:bp+2Dh]
25844 0000411B 263B462D    cmp     ax,[es:bp+DPB.LAST_CLUSTER]
25845 0000411F EB08      jmp     short setup_2
25846                                ; FAT12 or FAT16 fs
25847 00004121 09D2      or      dx,dx
25848 00004123 7523      jnz     short EOFERR
25849                                ;cmp ax,[es:bp+0Dh]
25850 00004125 263B460D    cmp     ax,[es:bp+DPB.MAX_CLUSTER]
25851
25852 00004129 771D      ja      short EOFERR
25853 0000412B 8916[DE0A]    mov     [CLUSNUM_HW],dx
25854                                ;;;
25855                                ;;cmp ax,[es:bp+0Dh]
25856                                ;CMP    AX,[ES:BP+DPB.MAX_CLUSTER] ;>32mb if > disk size ;AN000;
25857                                ;JA      short EOFERR      ;>32mb then EOF ;AN000;
25858                                ;;
25859
25860 0000412F A3[BC05]      MOV     [CLUSNUM],AX
25861 00004132 07          POP     ES      ; ES:DI point to SFT
25862
25863                                ;M039
25864
25865                                NOSETSTUFF:
25866 00004133 89C8      MOV     AX,CX      ; AX = Byte count.
25867 00004135 0306[2C03]    ADD     AX,[DMAADD]    ; See if it will fit in one segment
25868 00004139 730C      JNC     short setup_OK ; Must be less than 64k
25869 0000413B A1[2C03]      MOV     AX,[DMAADD]
25870 0000413E F7D8      NEG     AX      ; Amount of room left in segment (know
25871                                ; less than 64K since max value of CX
25872                                ; is FFFF).
25873 00004140 7501      JNZ     short NoDec
25874 00004142 48          DEC     AX
25875                                NoDec:
25876 00004143 89C1      MOV     CX,AX      ; Can do this much
25877 00004145 E304      JCXZ    NOROOM      ; Silly user gave Xaddr of FFFF in segment
25878 00004147 C3          retn
25879
25880                                EOFERR:
25881 00004148 07          POP     ES      ; ES:DI point to SFT
25882 00004149 31C9      XOR     CX,CX      ; No bytes read
25883                                ;;;;
25884                                ; 7/18/86
25885                                ; MSDOS 3.3
25886                                ;MOV    BYTE [DISK_FULL],1 ; set disk full flag
25887                                ;;;;
25888 0000414B 5B          POP     BX      ; Kill return address
25889 0000414C F8          CLC
25890 0000414D C3          retn      ; RETURN TO CALLER OF CALLER
25891
25892                                ;Break <BREAKDOWN -- CUT A USER READ OR WRITE INTO PIECES>
25893                                ;-----
25894                                ;
25895                                ; Procedure Name : BREAKDOWN
25896                                ;
25897                                ; Inputs:
25898                                ; CX = Length of disk transfer in bytes
25899                                ; ES:BP = Base of drive parameters
25900                                ; [BYTSECPPOS] = Byte position within first sector
25901                                ; DS = DOSDATA

```

```

25902 ; Outputs:
25903 ; [BYTCNT1] = Bytes to transfer in first sector
25904 ; [SECCNT] = No. of whole sectors to transfer
25905 ; [BYTCNT2] = Bytes to transfer in last sector
25906 ; AX, BX, DX destroyed. No other registers affected.
25907 ;-----
25908
25909 BREAKDOWN:
25910 MOV AX,[BYTSECPOS]
25911 MOV BX,CX
25912 OR AX,AX
25913 JZ short SAVFIR ; Partial first sector?
25914 ;sub ax,[es:bp+2]
25915 SUB AX,[ES:BP+DPB.SECTOR_SIZE]
25916 NEG AX ; Max number of bytes left in first sector
25917 SUB BX,AX ; Subtract from total length
25918 JAE short SAVFIR
25919 ADD AX,BX ; Don't use all of the rest of the sector
25920 XOR BX,BX ; And no bytes are left
25921 SAVFIR:
25922 MOV [BYTCNT1],AX
25923 MOV AX,BX
25924 XOR DX,DX
25925 ;div word [ES:BP+2]
25926 DIV word [ES:BP+DPB.SECTOR_SIZE] ; How many whole sectors?
25927 MOV [SECCNT],AX
25928 MOV [BYTCNT2],DX ; Bytes remaining for last sector
25929 ; MSDOS 3.3
25930 ;OR DX,[BYTCNT1] ; SMR ONESECTORFIX BUGBUG
25931 ;retnz ; NOT (BYTCNT1 = BYTCNT2 = 0)
25932 ;CMP AX,1
25933 ;retnz
25934 ;MOV AX,[ES:BP+DPB.SECTOR_SIZE] ; Buffer EXACT one sector I/O
25935 ;MOV [BYTCNT2],AX
25936 ;MOV [SECCNT],DX ; DX = 0
25937 _RET45:
25938 retn
25939
25940 ; DOSCODE:77BFh (MSDOS 6.21, MSDOS.SYS)
25941
25942 ;-----
25943 ;
25944 ; Procedure Name : READ_LOCK_VIOLATION
25945 ;
25946 ; ES:DI points to SFT. This entry used by NET_READ
25947 ; Carry set if to return error (CX=0,AX=error_sharing_violation).
25948 ; Else do retrys.
25949 ; ES:DI,DS,CX preserved
25950 ;-----
25951
25952 READ_LOCK_VIOLATION:
25953 MOV byte [READOP],0
25954 ERR_ON_CHECK:
25955 ;;test word [es:di+2],8000h
25956 ;TEST word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB
25957 ;JNZ short HARD_ERR
25958
25959 ; 04/05/2019
25960 ;test byte [es:di+3],80h
25961 TEST byte [ES:DI+SF_ENTRY.sf_mode+1],(sf_isFCB>>8)
25962 JNZ short HARD_ERR
25963
25964 ;PUSH CX
25965 ;;mov cl,[es:di+2]
25966 ;MOV CL,[ES:DI+SF_ENTRY.sf_mode]
25967 ;;and cl,0F0h
25968 ;AND CL,SHARING_MASK
25969 ;;cmp cl,0
25970 ;CMP CL,SHARING_COMPAT
25971 ;POP CX
25972 ;JNE short NO_HARD_ERR
25973 ; 21/09/2023
25974 mov al,[ES:DI+SF_ENTRY.sf_mode]
25975 and al,SHARING_MASK
25976 ;cmp al,SHARING_COMPAT
25977 ;jne short NO_HARD_ERR
25978 jnz short NO_HARD_ERR
25979 HARD_ERR:
25980 call LOCK_VIOLATION
25981 jnc short _RET45 ; User wants Retrys
25982 NO_HARD_ERR:
25983 XOR CX,CX ;No bytes transferred
25984 ;mov ax,21h
25985 MOV AX,error_lock_violation
25986 STC
25987 RET3: ; 06/02/2024
25988 retn
25989
25990 ;-----
25991 ;
25992 ; Procedure Name : WRITE_LOCK_VIOLATION
25993 ;
25994 ; Same as READ_LOCK_VIOLATION except for READOP.
25995 ; This entry used by NET_WRITE
25996 ;-----
25997
26000 WRITE_LOCK_VIOLATION:
26001 MOV byte [READOP],1
26002 JMP short ERR_ON_CHECK
26003
26004 ; 04/05/2019 - Retro DOS v4.0
26005
26006 ; DOSCODE:77ECh (MSDOS 6.21, MSDOS.SYS)
26007
26008 ;Break <DISKREAD -- PERFORM USER DISK READ>
26009 ;-----
26010 ;
26011 ; Procedure Name : DISKREAD
26012 ;
26013 ; Inputs:
26014 ; Outputs of SETUP
26015 ; Function:
26016 ; Perform disk read
26017 ; Outputs:
26018 ; Carry clear
26019 ; CX = No. of bytes read
26020 ; ES:DI point to SFT
26021 ; SFT offset and cluster pointers updated
26022 ; Carry set
26023 ; CX = 0
26024 ; ES:DI point to SFT
26025 ; AX has error code

```

```

26026 ;-----
26027
26028 ;hkn; called from disk.asm. DS already set up.
26029
26030 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
26031 ; DOSCODE:77D8h (MSDOS 5.0, MSDOS.SYS)
26032
26033 ; 09/02/2024
26034 ; 06/02/2024 - Retro DOS v5.0
26035 ; PCDS 7.1 IBMDOS.COM - DOSCODE:81D5h
26036
26037 DISKREAD:
26038 ;mov ax,[es:di+11h]
26039 0000419F 268B4511 MOV AX,[ES:DI+SF_ENTRY.sf_size]
26040 ;mov bx,[es:di+13h]
26041 000041A3 268B5D13 MOV BX,[ES:DI+SF_ENTRY.sf_size+2]
26042 000041A7 2B06[CE05] SUB AX,[BYTPOS]
26043 000041AB 1B1E[D005] SBB BX,[BYTPOS+2]
26044 000041AF 7226 JB short RDERR ;Read starts past EOF
26045 000041B1 750A JNZ short ENUF ;More than 64k to EOF
26046 000041B3 09C0 OR AX,AX
26047 000041B5 7420 JZ short RDERR ;Read starts at EOF
26048 000041B7 39C8 CMP AX,CX
26049 000041B9 7302 JAE short ENUF ;I/O fits
26050 000041BB 89C1 MOV CX,AX ;Limit read to up til EOF
26051
26052 ENUF:
26053 ; MSDOS 3.3
26054 ;test byte [es:di+4],8
26055 ;TEST byte [ES:DI+SF_ENTRY.sf_attr],attr_volume_id
26056 ;jnz short SET_ACC_ERR
26057 ;call LOCK_CHECK
26058 ;jnb short _READ_OK
26059 ;call READ_LOCK_VIOLATION
26060 ;jnb short ENUF
26061 ;retn
26062
26063 000041BD E868FE ; MSDOS 6.0
26064 ;call CHECK_READ_LOCK ;IFS. check read lock ;AN000;
26065 ;JNC short _READ_OK ; There are no locks
26066 ;retn
26067 000041C0 72D5 ; 06/02/2024
26068 jc short RET3
26069
26070 000041C2 C42E[8A05] _READ_OK:
26071 000041C6 E885FF LES BP,[THISDPB]
26072 CALL BREAKDOWN
26073
26074 ; 10/02/2024
26075 %if 0
26076 MOV CX,[CLUSNUM] ; *
26077 ;;;
26078 ; 06/02/2023 (PCDS 7.1 IBMDOS.COM)
26079 mov dx,[CLUSNUM_HW] ; *
26080 ;;;
26081 call FNDCLUS
26082 ; MSDOS 6.0 ;M022 conditional removed here
26083 JC short SET_ACC_ERR_DS ; fix to take care of I24 fail
26084 ; migrated from 330a - HKN
26085
26086 %else
26087 ; 10/02/2024 - Retro DOS v5.0
26088 call FNDCLUS_X ; *
26089 jc short SET_ACC_ERR ; ds=ss
26090 %endif
26091 ;;;
26092 ; 09/02/2024 (PCDS 7.1 IBMDOS.COM)
26093 cmp word [CLSKIP_HW],0
26094 jnz short RDERR
26095 ;;;
26096 ;OR CX,CX
26097 ;JZ short SKIPERR
26098 ; 06/02/2024
26099 jcxz SKIPERR
26100
26101 RDERR:
26102 000041D7 B40E MOV AH,0EH ;MS. read/data/fail ;AN000;
26103 000041D9 E97202 jmp WRTERR22
26104
26105 ;RDLASTJ:
26106 ;JMP RDLAST ;M039
26107
26108 SETSFTJ2:
26109 JMP SETSFT
26110
26111 CANOT_READ:
26112 ; MSDOS 3.3
26113 ;POP CX ;M039.
26114 ; MSDOS 3.3 & MSDOS 6.0
26115 000041DF 59 POP CX ;Clean stack.
26116 000041E0 5B POP BX
26117 ;;;
26118 ; 09/02/2024 (PCDS 7.1 IBMDOS.COM)
26119 ;pop word [CCONTENT_HW] ; PCDS 7.1 BUG!?
26120 ; I think, this would be 'pop word [ss:CCONTENT_HW]'
26121 ; because ds<=>ss while jumping here.
26122 ; Erdogan Tan - 09/02/2024
26123 000041E1 368F06[EC0A] pop word [ss:CCONTENT_HW] ; *
26124 ;;;
26125 ;entry SET_ACC_ERR_DS
26126 SET_ACC_ERR_DS:
26127
26128 ;hkn; SS is DOSDATA
26129 ;Context DS
26130 000041E6 16 push ss
26131 000041E7 1F pop ds
26132
26133 ;entry SET_ACC_ERR
26134 SET_ACC_ERR:
26135 000041E8 31C9 XOR CX,CX
26136 ;mov ax,5
26137 000041EA B80500 MOV AX,error_access_denied
26138 000041ED F9 STC
26139 000041EE C3 retn
26140
26141 SKIPERR:
26142 000041EF 8916[BA05] MOV [LASTPOS],DX
26143 000041F3 891E[BC05] MOV [CLUSNUM],BX
26144 ;;;
26145 ; 09/02/2024 (PCDS 7.1 IBMDOS.COM)
26146 000041F7 8B1E[E80A] mov bx,[CLUSTNUM_HW]
26147 000041FB 891E[DE0A] mov [CLUSNUM_HW],bx
26148 ;;;
26149 000041FF 833E[D205]00 CMP word [BYTCNT1],0

```

```

26150 00004204 7405          JZ      short RDMID
26151
26152 00004206 E82016        call    BUFRD
26153          ;JC      short SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
26154          ; 10/02/2024
26155          ; ds=ss
26156 00004209 72DD          jc      short SET_ACC_ERR
26157
26158
26159 0000420B 833E[D605]00    RDMID:
26160          CMP      word [SECCNT],0
26161          ;JZ      RDLAST ; 10/08/2018
26162          JZ      short RDLAST
26163
26163 00004212 E8A816        call    NEXTSEC
26164 00004215 72C5          JC      short SETSFTJ2
26165
26166 00004217 C606[7405]01    MOV     BYTE [TRANS],1          ; A transfer is taking place
26167
26168 0000421C 8A16[7305]    ONSEC:
26169 00004220 8B0E[D605]    MOV     DL,[SECCUSPOS]          ; (dx/DL = Extent start) ((dh = ?))
26170          MOV     CX,[SECCNT]
26171          ;;;
26172          ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26173          mov     bx,[CLUSNUM_HW]
26174          mov     [CLUSTNUM_HW],bx
26175          ;;;
26176          MOV     BX,[CLUSNUM]
26177
26177 00004230 E8D116        RDLP:
26178          call    OPTIMIZE
26179          ;JC      short SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
26180          ; 10/02/2024
26181          ; ds=ss
26182          jc      short SET_ACC_ERR
26183          ;;;
26184          ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26185          push    word [CONTENT_HW]          ; (Next physical cluster, hw)
26186          ;;;
26187          PUSH    DI                      ;DI = Next physical cluster.
26188          PUSH    AX                      ;AX = # of sectors remaining.
26189          PUSH    BX                      ;[DMAADD+2]:BX = Transfer address.
26190          ;mov     byte [ALLOWED],38h
26191          MOV     MOV     byte [ALLOWED],Allowed_RETRY+Allowed_FAIL+Allowed_IGNORE
26192          MOV     DS,[DMAADD+2]
26193
26193 00004245 52              PUSH    DX                      ;[HIGH_SECTOR]:DX = phys. sector #.
26194 00004246 51              PUSH    CX                      ;CX = # of contiguous sectors to read.
26195
26196          ; 04/05/2019 - Retro DOS v4.0
26197
26198          ; 09/02/2024 - Retro DOS v5.0
26199          ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8284h
26200          ; Windows ME IO.SYS - BIOSCODE:0B69Eh
26201          ; MSDOS 6.22 IO.SYS - DOSCODE:787Bh
26202
26203          ; NOTE: Secondary Buffer Cache is not used in PCDOS 7.1 IBMDOS.COM.
26204
26205          ;call    nul_sub          ; PCDOS 7.1 (nul_sub: retn)
26206          ;          ; 'nul_sub:' at DOSCODE:AD57h
26207
26208          ; MSDOS 6.0 (& windows ME)
26209          ;call    SET_RQ_SC_PARAMS          ;LB. do this for SC ;AN000;
26210
26211          ; MSDOS 3.3 (& MSDOS 6.0)
26212 00004247 E8A0FD        call    DREAD
26213
26214          ; 10/02/2024
26215          ; ds<>ss
26216
26217          ; MSDOS 3.3
26218          ;pop     bx
26219          ;pop     dx
26220          ;jc      short CANOT_READ
26221          ;add     bx,dx          ; (bx = Extent end)
26222          ;mov     al,[es:bp] ; mov al,[es:bp+0]
26223          ;;mov     al,[ES:BP+DPB.DRIVE]
26224          ;call    SETVISIT
26225          ; ->***
26226
26227          ;M039
26228          ; MSDOS 6.0
26229          pop     cx
26230          pop     dx
26231          pop     WORD [ss:TEMP_VAR]
26232          jc      short CANOT_READ ; * 09/02/2024
26233
26233 00004253 368C1E[E06]    mov     [ss:TEMP_VAR2],ds
26234
26235          ;
26236          ; CX = # of contiguous sectors read. (These constitute a block of
26237          ; sectors, also termed an "Extent".)
26238          ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
26239          ; [TEMP_VAR2]:[TEMP_VAR] = Transfer address (destination data address).
26240          ; ES:BP -> Drive Parameter Block (DPB).
26241          ;
26242          ; The Buffer Queue must now be scanned: the contents of any dirty
26243          ; buffers must be "read" into the transfer memory block, so that the
26244          ; transfer memory reflects the most recent data.
26245
26245 00004258 E87600        call    DskRdBufScan
26246
26247          ;Context DS
26248          push    ss
26249          pop     ds
26250
26251          pop     cx
26252          pop     bx
26253
26254          ;
26255          ; CX = # of sector remaining.
26256          ; BX = Next physical cluster.
26257          ;;;
26258          ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26259 0000425F 8F06[E80A]    pop     word [CLUSTNUM_HW]          ; (Next physical cluster, hw)
26260          ;;;
26261
26262          ;M039
26263
26264          ;;;
26265          ; 25/07/2018 - Retro DOS v3.0
26266          ; ***->
26267          ; MSDOS 3.3
26268          ; IBMDOS.COM (1987) - Offset 42BDh
26269          ;bufq:
26270          ; DX = Extent start.
26271          ; BX = Extent end.
26272          ; AL = Drive #.
26273          ; DS:DI-> 1st buffer in queue.

```

```

26274 ;
26275 ; or byte [di+5],20h
26276 ; or byte [DI+BUFFINFO.buf_flags],buf_visit ; Bit 5 = reserved
26277 ; cmp al,[di+4]
26278 ; cmp al,[DI+BUFFINFO.buf_ID]
26279 ; jnz short bufq3
26280 ; cmp [di+6],dx
26281 ; cmp [DI+BUFFINFO.buf_sector],dx
26282 ; jb short bufq3 ; Jump if Extent start > buffer sector.
26283 ; cmp [di+6],bx
26284 ; cmp [DI+BUFFINFO.buf_sector],bx
26285 ; jnb short bufq3 ; Jump if Extent end >= buffer sector.
26286 ;
26287 ; Buffer sector is in the Extent (contiguous sectors to read)
26288 ;
26289 ; Buffer's sector is in Extent: if it is dirty, copy its contents to
26290 ; transfer memory; otherwise, just re-position it in the buffer queue
26291 ; as MRU (Most Recently Used).
26292 ;
26293 ; test byte [di+5],40h
26294 ; test byte [DI+BUFFINFO.buf_flags],buf_dirty ; Bit 6 = dirty flag
26295 ; jz short bufq2 ; clear buffer, check the next buff sec
26296 ; pop ax ; transfer address
26297 ; push ax
26298 ; push di
26299 ; push dx
26300 ; sub dx,[di+6]
26301 ; sub dx,[DI+BUFFINFO.buf_sector]
26302 ; neg dx
26303 ;
26304 ; DX = offset (in sectors) of buffer sector within Transfer memory
26305 ; block.
26306 ;
26307 ; mov si,di
26308 ; mov di,ax
26309 ; mov ax,dx
26310 ; mov cx,[es:bp+6]
26311 ; mov cx,[ES:BP+DPB.SECTOR_SIZE] ; CX = sector size (in bytes).
26312 ; mul cx
26313 ; add di,ax
26314 ;
26315 ; lea si,[si+16]
26316 ; lea si,[SI+BUFINISIZ] ;DS:SI -> buffer data.
26317 ; shr cx,1
26318 ; push es
26319 ; mov es,[SS:DMAADD+2]
26320 ;
26321 ; CX = sector size (in WORDS) ; CF=1 if odd # of bytes.
26322 ; DS:SI-> Buffer sector data.
26323 ; ES:DI-> Destination within Transfer memory block.
26324 ;
26325 ; rep movsw ;Copy buffer sector to Transfer memory
26326 ; adc cx,0 ;CX=1 if odd # of bytes, else CX=0.
26327 ; rep movsb ;Copy last byte.
26328 ; jnc short bufq1
26329 ; movsb
26330 ;bufq1:
26331 ; pop es
26332 ; pop dx
26333 ; pop di
26334 ; mov al,[es:bp] ; mov al,[es:bp+0]
26335 ; mov al,[ES:BP+DPB.DRIVE]
26336 ;bufq2:
26337 ; call SCANPLACE
26338 ;bufq3:
26339 ; call SKIPVISIT
26340 ; jnz short bufq
26341 ;
26342 ; push ss
26343 ; pop ds
26344 ; pop cx
26345 ; pop cx
26346 ; pop bx
26347 ;bufq4:
26348 ;;;;;;
26349 ;
26350 00004263 E313 JCXZ RDLAST
26351 ;
26352 00004265 E84B20 call ISEOF ; test for eof on fat size
26353 00004268 732B JAE short SETSFT
26354 ;
26355 0000426A B200 MOV DL,0
26356 ;INC word [LASTPOS] ; we'll be using next cluster
26357 ;;;
26358 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26359 ; add word [LASTPOS],1
26360 ; adc word [LASTPOS_HW],0
26361 ; 09/02/2024 - Retro DOS v5.0
26362 0000426C FF06[BA05] inc word [LASTPOS]
26363 00004270 75BE jnz short RDLP
26364 00004272 FF06[E20A] inc word [LASTPOS_HW]
26365 ;;;
26366 00004276 EBB8 JMP short RDLP ; 19/05/2019
26367 ;
26368 RDLAST:
26369 00004278 A1[D405] MOV AX,[BYTCNT2]
26370 0000427B 09C0 OR AX,AX
26371 0000427D 7416 JZ short SETSFT
26372 0000427F A3[D205] MOV [BYTCNT1],AX
26373 ;
26374 00004282 E83816 call NEXTSEC
26375 00004285 720E JC short SETSFT
26376 ;
26377 00004287 C706[CC05]0000 MOV word [BYTSECPOS],0
26378 0000428D E89915 call BUFRD
26379 ; 10/08/2018
26380 00004290 7303 JNC short SETSFT
26381 ;JMP SET_ACC_ERR_DS
26382 ; 10/02/2024
26383 ; ds=ss
26384 00004292 E953FF jmp SET_ACC_ERR
26385 ;
26386 ;-----
26387 ;
26388 ; Procedure Name : SETSFT
26389 ; Inputs:
26390 ; [NEXTADD],[CLUSNUM],[LASTPOS] set to determine transfer size
26391 ; and set cluster fields
26392 ; Function:
26393 ; Update [THISSFT] based on the transfer
26394 ; Outputs:
26395 ; sf_position, sf_lstclus, and sf_cluspos updated
26396 ; ES:DI points to [THISSFT]
26397 ; CX No. of bytes transferred

```

```

26398 ; Carry clear
26399 ;
26400 ;-----
26401 ;entry SETSFT
26402 ;
26403 ; 26/07/2018 - Retro DOS v3.0
26404 ;
26405 ; 09/02/2024 - Retro DOS v5.0
26406 ; (PCDOS 7.1 IBMDOS.COM)
26407
26408 SETSFT:
26409 LES DI,[THISFT]
26410 00004295 C43E[9E05]
26411 ; Same as SETSFT except ES:DI already points to SFT
26412 ;entry SETCLUS
26413 SETCLUS:
26414 MOV CX,[NEXTADD]
26415 SUB CX,[DMAADD] ; Number of bytes transfered
26416 0000429D 2B0E[2C03]
26417 ;;test word [es:di+5],80h
26418 ;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device
26419 ;JNZ short ADDREC ; don't set clusters if device
26420 ;
26421 ; 04/05/2019 - Retro DOS v4.0
26422 ;test byte [es:di+5],80h
26423 000042A1 26F6450580
26424 000042A6 751C
26425 TEST byte [ES:DI+SF_ENTRY.sf_flags],devid_device
26426 JNZ short ADDREC ; don't set clusters if device
26427 ;
26428 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26429 mov ax,[CLUSNUM_HW]
26430 mov [es:di+37h],ax
26431 ;mov [es:di+SF_ENTRY.sf_1stclus+2],ax
26432 mov ax,[LASTPOS_HW]
26433 mov [es:di+0Bh],ax
26434 ;mov [es:di+SF_ENTRY.sf_cluspos_hw],ax ; PCDOS 7.1 & Win ME
26435 ;;mov [es:di+SF_ENTRY.sf_firclus],ax ; MSDOS 5.0-6.22
26436 ;;;
26437 000042B6 A1[BC05]
26438 MOV AX,[CLUSNUM]
26439 ;;mov [es:di+18h],ax ; MSDOS 3.3
26440 ;mov [es:di+35h],ax ; MSDOS 6.0 (& MSDOS 6.21)
26441 000042B9 26894535
26442 000042BD A1[BA05]
26443 MOV [ES:DI+SF_ENTRY.sf_1stclus],AX
26444 MOV AX,[LASTPOS]
26445 ;mov [es:di+19h],ax
26446 MOV [ES:DI+SF_ENTRY.sf_cluspos],AX
26447 ;-----
26448 ; Procedure : AddRec
26449 ; Inputs:
26450 ; ES:DI points to SFT
26451 ; CX is No. Bytes transferred
26452 ; Function:
26453 ; Update the SFT offset based on the transfer
26454 ; Outputs:
26455 ; sf_position updated to point to first byte after transfer
26456 ; ES:DI points to SFT
26457 ; CX No. of bytes transferred
26458 ; Carry clear
26459 ;-----
26460 ;entry AddRec
26461 ADDREC:
26462 000042C4 E309
26463 JCXZ RET28 ; If no records read, don't change position
26464 ;add [es:di+15h],cx
26465 ADD [ES:DI+SF_ENTRY.sf_position],CX ; Update current position
26466 000042CA 2683551700
26467 ;adc word [es:di+17h], 0
26468 ADC WORD [ES:DI+SF_ENTRY.sf_position+2],0
26469 RET28:
26470 CLC
26471 retn
26472 ; 25/07/2018
26473 ; MSDOS 6.0
26474 ;Break <DskRdBufScan -- Disk Read Buffer Scan>
26475 ;-----
26476 ; Procedure Name : DskRdBufScan
26477 ;
26478 ; Inputs:
26479 ; CX = # of contiguous sectors read. (These constitute a block of
26480 ; sectors, also termed an "Extent".)
26481 ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
26482 ; [TEMP_VAR2]:[TEMP_VAR] = Transfer address (destination data address).
26483 ; ES:BP -> Drive Parameter Block (DPB).
26484 ;
26485 ; Function:
26486 ; The Buffer Queue is scanned: the contents of any dirty buffers are
26487 ; "read" into the transfer memory block, so that the transfer memory
26488 ; reflects the most recent data.
26489 ;
26490 ; Outputs:
26491 ; Transfer memory updated as required.
26492 ;
26493 ; Uses:
26494 ; DS,AX,BX,CX,SI,DI destroyed.
26495 ; SS override for all global variables.
26496 ;
26497 ; Notes:
26498 ; FIRST_BUFF_ADDR is set-up to contain the LAST buffer to check, rather
26499 ; than the FIRST.
26500 ;-----
26501 ;M039: Created
26502 ;
26503 ; 04/05/2019 - Retro DOS v4.0
26504 ; DOSCODE:78F0h (MSDOS 6.21, MSDOS.SYS)
26505 ;
26506 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
26507 ; DOSCODE:78DCh (MSDOS 5.0, MSDOS.SYS)
26508 ;
26509 ; 09/02/2024 - Retro DOS v5.0
26510 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8313h
26511 ;
26512 ;procedure DskRdBufScan,NEAR
26513 ;
26514 ;ASSUME DS:NOTHING
26515
26516 DskRdBufScan:
26517 000042D1 36833E[7100]00
26518 000042D7 743C
26519 cmp word [ss:DirtyBufferCount],0 ; Any dirty buffers?
26520 je short bufx ; -no, skip all work.
26521 mov bx,[ss:HIGH_SECTOR]
26522 mov si,bx

```



```

26522 000042E0 01D1      add     cx,dx
26523 000042E2 83D600      adc     si,0
26524
26525 000042E5 E8CC25      call    GETCURHEAD          ;DS:DI -> 1st buf in queue.
26526                ;mov     ax,[di+2]
26527 000042E8 8B4502      mov     ax,[di+BUFFINFO.buf_prev]
26528 000042EB 36A3[7E11]    mov     [ss:FIRST_BUFF_ADDR],ax
26529
26530                ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26531                ;;mov     al,[es:bp+0]
26532                ;mov     al,[es:bp+DPB.DRIVE]
26533                ; 15/12/2022
26534 000042EF 268A4600      mov     al,[es:bp]
26535
26536                ; BX:DX = Extent start.
26537                ; SI:CX = Extent end + 1.
26538                ; AL = Drive #.
26539                ; DS:DI-> 1st buffer in queue.
26540                ;[FIRST_BUFF_ADDR] = Address offset of last buffer in queue.
26541
26542 bufq:
26543                ;cmp     al,[di+4]
26544 000042F3 3A4504      cmp     al,[di+BUFFINFO.buf_ID] ;Same drive?
26545 000042F6 7514      jne     short bufq1          ; -no, jump.
26546
26547                ; Cmp32     bx,dx,<WORD PTR [di.buf_sector+2]>,<WORD PTR [di.buf_sector]>
26548                ; ja     short bufq1          ;Jump if Extent start > buffer sector.
26549
26550                ;cmp     bx,[di+8]
26551 000042F8 3B5D08      cmp     bx,[di+BUFFINFO.buf_sector+2]
26552 000042FB 7503      jne     short bufq01
26553                ;cmp     dx,[di+6]
26554 000042FD 3B5506      cmp     dx,[di+BUFFINFO.buf_sector]
26555
26556 bufq01:
26557                ja     short bufq1
26558
26559                ; Cmp32     si,cx,<WORD PTR [di.buf_sector+2]>,<WORD PTR [di.buf_sector]>
26560                ; ja     short bufq2          ;Jump if Extent end >= buffer sector.
26561
26562                ;cmp     si,[di+8]
26563 00004302 3B7508      cmp     si,[di+BUFFINFO.buf_sector+2]
26564                ;jne     short bufq02
26565 00004307 3B4D06      cmp     cx,[di+6]
26566                cmp     cx,[di+BUFFINFO.buf_sector]
26567
26568 bufq02:
26569                ja     short bufq2
26570 bufq1:
26571                cmp     di,[ss:FIRST_BUFF_ADDR]          ;Scanned entire buffer queue?
26572                mov     di,[di]
26573                ;mov     di,[di+BUFFINFO.buf_next] ; Set-up for next buffer.
26574 00004313 75DE      jne     short bufq          ; -no, do next buffer
26575
26576 bufx:
26577                retn          ;Exit.
26578
26579                ; Buffer's sector is in Extent: if it is dirty, copy its contents to
26580                ; transfer memory; otherwise, just re-position it in the buffer queue
26581                ; as MRU (Most Recently Used).
26582
26583 bufq2:
26584                push     ax
26585                ;test     byte [di+5],40h
26586                test     byte [di+BUFFINFO.buf_flags],buf_dirty ;Buffer dirty?
26587                jz     short bufq3          ; -no, jump.
26588
26589                ; SaveReg <cx,dx,si,di,es>
26590                push     cx
26591                push     dx
26592                push     si
26593                push     di
26594                push     es
26595
26596                mov     ax,dx
26597                ;sub     ax,[di+6]
26598                sub     ax,[di+BUFFINFO.buf_sector]
26599                neg     ax
26600
26601                ; AX = offset (in sectors) of buffer sector within Transfer memory
26602                ; block. (Note: the upper word of the sector # may be ignored
26603                ; since no more than 64k bytes will ever be read. This 64k limit
26604                ; is imposed by the input parameters of the disk read operation.)
26605
26606                ;lea     si,[di+20]
26607                lea     si,[di+BUFINSZ]          ;DS:SI -> buffer data.
26608                ;mov     cx,[es:bp+2]
26609                mov     cx,[es:bp+DPB.SECTOR_SIZE] ;CX = sector size (in bytes).
26610                mul     cx          ;AX = offset (in bytes) of buf. sector
26611                ;mov     di,[ss:TEMP_VAR]
26612                ; 09/02/2024
26613                les     di,[ss:TEMP_VAR]
26614                add     di,ax
26615                ;mov     es,[ss:TEMP_VAR2]
26616                shr     cx,1
26617
26618                ; CX = sector size (in WORDs) ; CF=1 if odd # of bytes.
26619                ; DS:SI-> Buffer sector data.
26620                ; ES:DI-> Destination within Transfer memory block.
26621
26622                ; 09/02/2024 - Retro DOS v5.0
26623                %if 0
26624                rep     movsw          ;Copy buffer sector to Transfer memory
26625                ;; 04/05/2019
26626                ;adc     cx,0          ;CX=1 if odd # of bytes, else CX=0.
26627                ;rep     movsb          ;Copy last byte.
26628                ;jnc     short bufq03
26629                ;movsb
26630                ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26631                ;adc     cx,0
26632                ;rep     movsb
26633                ; 22/09/2023
26634                jnc     short bufq03
26635                movsb
26636                %else
26637                ;;
26638                ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26639                cmp     byte [ss:DDMOVE],0
26640                ;jz     short bufq03+1 ; rep movsw (skip 32bit prefix)
26641                ; 06/04/2024
26642                jz     short bufq03
26643                shr     cx,1
26644                ;bufq03:
26645                ;rep     movsd
26646                ; 06/04/2024
26647                db     66h          ; 32bit prefix
26648                bufq03:

```

```

26646 00004346 F3A5      rep      movsw
26647                  ;;;
26648      %endif
26649
26650      ;bufq03:      ; 09/02/2024
26651      ;RestoreReg <es,di,si,dx,cx>
26652 00004348 07      pop      es
26653 00004349 5F      pop      di
26654 0000434A 5E      pop      si
26655 0000434B 5A      pop      dx
26656 0000434C 59      pop      cx
26657
26658      ; DS:DI -> current buffer.
26659      bufq3:
26660 0000434D 89F8      mov      ax,di      ;DS:AX -> Current buffer.
26661                  ;invoke SCANPLACE
26662      call     SCANPLACE
26663 00004352 363B06[7E11] cmp      ax,[ss:FIRST_BUFF_ADDR] ;Last buffer?
26664 00004357 58      pop      ax
26665      ;jne     short bufq      ; -no, jump.
26666      ;jmp     short bufx      ; -yes, exit.
26667      ; 12/06/2019
26668      ;retn
26669      ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compability)
26670 00004358 7599      jne     short bufq
26671      ;jmp     short bufx
26672      ; 09/02/2024
26673 0000435A C3      retn      ; Exit
26674
26675      ;EndProc DskRdBufScan
26676
26677      ;=====
26678      ; DISK3.ASM, MSDOS 6.0, 1991
26679      ;=====
26680      ; 04/05/2019 - Retro DOS v4.0
26681      ; 24/07/2018 - Retro DOS v3.0
26682
26683      ;Break <DISKWRITE -- PERFORM USER DISK WRITE>
26684      ;-----
26685      ;
26686      ; Procedure Name : DISKWRITE
26687      ;
26688      ; Inputs:
26689      ; Outputs of SETUP
26690      ; Function:
26691      ; Perform disk write
26692      ; Outputs:
26693      ; Carry clear
26694      ; CX = No. of bytes written
26695      ; ES:DI point to SFT
26696      ; SFT offset and cluster pointers updated
26697      ; Carry set
26698      ; CX = 0
26699      ; ES:DI point to SFT
26700      ; AX has error code
26701      ;-----
26702
26703      ; DOSCODE:797Ah (MSDOS 6.21, MSDOS.SYS)
26704
26705      ; 20/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
26706      ; DOSCODE:7966h (MSDOS 5.0, MSDOS.SYS)
26707
26708      ; 10/02/2024
26709      ; 09/02/2024 - Retro DOS v5.0
26710      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:83A3h
26711
26712      ; (Windows ME IO.SYS - BIOSCODE:0B7B2h)
26713      ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:797Ah)
26714
26715      DISKWRITE:
26716      ; MSDOS 3.3
26717      ; IBMDOS.COM - Offset 436Dh
26718      ; test byte [es:di+4],8
26719      ; TEST byte [ES:DI+SF_ENTRY.sf_attr],attr_volume_id
26720      ; jz short write_cont
26721      ; jmp SET_ACC_ERR_DS
26722      ;write_cont:
26723      ; push cx
26724      ; or cx,cx
26725      ; jnz short Not_Truncate
26726      ; mov cx,-1
26727      ; dec cx
26728      ;Not_Truncate:
26729      ; call LOCK_CHECK
26730      ; pop cx
26731      ; jnb short _WRITE_OK
26732      ; call WRITE_LOCK_VIOLATION
26733      ; jnb short DISKWRITE
26734      ; retn
26735
26736      ; MSDOS 6.0
26737 0000435B E8ACFC      call     CHECK_WRITE_LOCK      ;IFS. check write lock;AN000;
26738      ; 19/08/2018
26739 0000435E 7304      jnc     short _WRITE_OK      ;IFS. lock check ok ;AN000;
26740 00004360 C3      retn
26741
26742      WRTEOFJ:
26743 00004361 E95602      jmp     WRTEOF
26744
26745      _WRITE_OK:
26746      ; 27/07/2018
26747      ; IBMDOS.COM - Offset 438Eh
26748
26749      ; MSDOS 3.3 (& MSDOS 6.0)
26750      ; and word [es:di+5],0BFBFh
26751 00004364 26816505BFBF      and     word [ES:DI+SF_ENTRY.sf_flags],~(sf_close_nodate|devid_file_clean)
26752      ; Mark file as dirty, clear no date on close
26753      ; 10/02/2024
26754      %if 0
26755      ; 04/05/2019 - Retro DOS v4.0
26756
26757      ; MSDOS 6.0
26758      ; mov ax,[es:di+11h]
26759      MOV AX,[ES:DI+SF_ENTRY.sf_size]      ;M039
26760      MOV [TEMP_VAR],AX      ;M039
26761      ; mov ax,[es:di+13h]
26762      MOV AX,[ES:DI+SF_ENTRY.sf_size+2]      ;M039
26763      MOV [TEMP_VAR2],AX      ;M039
26764      %else
26765      ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
26766      ; les ax,[es:di+11h]
26767 0000436A 26C44511      les     ax,[es:di+SF_ENTRY.sf_size]
26768 0000436E 8C06[0E06]      mov     [TEMP_VAR2],es
26769 00004372 A3[0C06]      mov     [TEMP_VAR],ax

```



```

26770 %endif
26771
26772 ; TEMP_VAR2:TEMP_VAR = Current file size (sf_size);M039
26773
26774 ; MSDOS 3.3 (& MSDOS 6.0)
26775 00004375 C42E[8A05] LES BP,[THISDPB]
26776
26777 00004379 E8D2FD call BREAKDOWN
26778
26779 0000437C A1[CE05] MOV AX,[BYTPOS]
26780 0000437F 8B16[D005] MOV DX,[BYTPOS+2]
26781 00004383 E3DC JCXZ WRTEOFJ ;Make the file length = sf_position
26782 00004385 01C8 ADD AX,CX
26783 00004387 83D200 ADC DX,0 ;DX:AX = last byte to write + 1.
26784
26785 ;;;
26786 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26787 0000438A 7303 jnc short dskwrt_1
26788 ACC_ERRWJ2:
26789 ; jmp SET_ACC_ERRW
26790 ; jmp SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
26791 ; 10/02/2024
26792 ; ds=ss
26793 0000438C E959FE jmp SET_ACC_ERR
26794
26795 dskwrt_1:
26796 ; test byte [es:di+3],10h
26797 0000438F 26F6450310 test byte [es:di+SF_ENTRY.sf_mode+1],10h
26798 00004394 750B jnz short dskwrt_3 ; > 2GB file size (up to 4GB) allowed
26799 00004396 81FAFF7F cmp dx,7FFFh ; check for 2GB file size limit
26800 0000439A 7503 jne short dskwrt_2
26801 0000439C 83F8FF cmp ax,0FFFFh
26802 dskwrt_2:
26803 0000439F 77EB ja short ACC_ERRWJ2 ; error,
26804 ; file position/pointer overs 2GB limit!
26805 dskwrt_3:
26806 ;;;
26807
26808 ; mov bx,[es:bp+2]
26809 000043A1 268B5E02 MOV BX,[ES:BP+DPB.SECTOR_SIZE]
26810
26811 ; MSDOS 3.3
26812 ; cmp dx,bx
26813 ; jnb short WRTERR33
26814 ; div bx
26815 ; mov bx,ax
26816 ; OR DX,DX
26817 ; JNZ short CALCLUS
26818 ; dec ax
26819 ; CALCLUS:
26820 ; MSDOS 3.3
26821 ; mov cl,[es:bp+5]
26822 ; MOV CL,[ES:BP+DPB.CLUSTER_SHIFT]
26823 ; shr ax,cl
26824 ; push ax
26825 ; push dx
26826 ; push es
26827 ; les di,[THISFT]
26828 ; mov ax,[es:di+11h]
26829 ; mov dx,[es:di+13h]
26830 ; mov ax,[ES:DI+SF_ENTRY.sf_size]
26831 ; mov dx,[ES:DI+SF_ENTRY.sf_size+2]
26832 ; pop es
26833 ; DX:AX = current file size (in bytes).
26834 ; div word [es:bp+2]
26835 ; div word [ES:BP+DPB.SECTOR_SIZE]
26836 ; mov cx,ax
26837 ; or dx,dx
26838 ; jz short NORND
26839 ; inc ax
26840 ; NORND:
26841 ; MSDOS 6.0
26842 000043A5 E85F03 CALL DIV32 ;DX:AX/BX = CX:AX + DX (rem.).
26843 000043A8 89C6 MOV SI,AX
26844 000043AA 890E[0706] MOV [HIGH_SECTOR],CX
26845
26846 ; [HIGH_SECTOR]:SI = Last full sector to write.
26847
26848 000043AE 09D2 OR DX,DX
26849 000043B0 52 PUSH DX ;M039: Free DX for use by SHR32
26850 000043B1 89CA MOV DX,CX ;M039
26851 000043B3 7506 JNZ short CALCLUS
26852 000043B5 83E801 SUB AX,1 ;AX must be zero base indexed ;AC000;
26853 000043B8 83DA00 SBB DX,0 ;M039 ;F.C. >32mb ;AN000;
26854
26855 CALCLUS:
26856 ; MSDOS 6.0
26857 000043BB E87003 CALL SHR32 ;F.C. >32mb ;AN000;
26858 ; POP DX
26859 ;;;
26860 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26861 000043BE 59 pop cx
26862 000043BF 87CA xchg cx,dx
26863 000043C1 51 push cx ; **!
26864 ; cx:ax = Last cluster to write
26865 ;;;
26866
26867 ; AX = Last cluster to write.
26868 ; DX = # of bytes in last sector to write (the "tail").
26869 ; BX = [ES:BP+DPB.SECTOR_SIZE]
26870
26871 000043C2 50 PUSH AX ; *!
26872 000043C3 52 PUSH DX
26873 ;M039
26874 000043C4 8B16[0E06] mov dx,[TEMP_VAR2]
26875 000043C8 A1[0C06] mov ax,[TEMP_VAR] ;DX:AX = current file size (in bytes).
26876 000043CB E83903 call DIV32 ;DX:AX/BX = CX:AX + DX (rem.)
26877 000043CE 890E[0E06] mov [TEMP_VAR2],cx
26878 000043D2 890E[CA05] mov [VALSEC+2],cx
26879 000043D6 89C1 mov cx,ax
26880 000043D8 89F3 mov bx,si
26881
26882 ; [HIGH_SECTOR]:BX = Last full sector to write.
26883 ; [VALSEC+2]:CX = Last full sector of current file.
26884 ; [TEMP_VAR2]:CX = Last full sector of current file.
26885 ; DX = # of bytes in last sector of current file.
26886 ;M039
26887 000043DA 09D2 OR DX,DX
26888 000043DC 7407 JZ short NORND
26889 ; ADD AX,1 ;Round up if any remainder ;AC000;
26890 ; ADC word [VALSEC+2],0
26891 ; 22/09/2023
26892 000043DE 40 inc ax ; 0FFFFh -> 0
26893 000043DF 7504 jnz short NORND

```

```

26894 000043E1 FF06[CA05]      inc     word [VALSEC+2]
26895                          NORND:
26896                          ; MSDOS 3.3 & MSDOS 6.0
26897 000043E5 A3[C805]        MOV     [VALSEC],AX
26898
26899                          ; [VALSEC] = Last sector of current file.
26900
26901 000043E8 31C0              XOR     AX,AX
26902 000043EA A3[DE05]        MOV     [GROWCNT],AX
26903 000043ED A3[E005]        MOV     [GROWCNT+2],AX
26904 000043F0 58              POP     AX      ; # of bytes in last sector to write (the "tail").
26905
26906                          ; MSDOS 6.0
26907 000043F1 8B3E[0706]      MOV     DI,[HIGH_SECTOR]      ;F.C. >32mb      ;AN000;
26908 000043F5 3B3E[0E06]      CMP     DI,[TEMP_VAR2]      ;M039; F.C. >32mb      ;AN000;
26909 000043F9 726C              JB      short NOGROW      ;F.C. >32mb      ;AN000;
26910 000043FB 7408              JZ      short lowsec      ;F.C. >32mb      ;AN000;
26911 000043FD 29CB              SUB     BX,CX      ;F.C. >32mb      ;AN000;
26912 000043FF 1B3E[0E06]      SBB     DI,[TEMP_VAR2]      ;M039; F.C. >32mb di:bx no. of sectors ;AN000;
26913 00004403 EB08              JMP     short yesgrow      ;F.C. >32mb      ;AN000;
26914
26915 lowsec:
26916                          ;MOV     DI,0      ;F.C. >32mb
26917 00004405 31FF              xor     di,di
26918                          ; MSDOS 3.3 & MSDOS 6.0
26919 00004407 29CB              SUB     BX,CX      ; Number of full sectors
26920 00004409 725C              JB      short NOGROW
26921 0000440B 744D              JZ      short TESTTAIL
26922
26923 yesgrow:
26924 0000440D 89D1              ; MSDOS 3.3 (& MSDOS 6.0)
26925 0000440F 93              MOV     CX,DX
26926                          XCHG     AX,BX
26927 00004410 26F76602          mul     word [es:bp+2]
26928                          MUL     word [ES:BP+DPB.SECTOR_SIZE] ; Bytes of full sector growth
26929
26930 00004414 8916[0706]      ; MSDOS 6.0
26931 00004418 A3[0E06]        MOV     [HIGH_SECTOR],DX      ;F.C. >32mb save dx      ;AN000;
26932 0000441B 89F8              MOV     [TEMP_VAR2],AX      ;M039; F.C. >32mb save ax      ;AN000;
26933                          MOV     AX,DI      ;F.C. >32mb      ;AN000;
26934 0000441D 26F76602          mul     word [es:bp+2]
26935                          MUL     word [ES:BP+DPB.SECTOR_SIZE] ;F.C. >32mb do higher word multiply ;AN000;
26936 00004421 0306[0706]      ADD     AX,[HIGH_SECTOR]      ;F.C. >32mb add lower value ;AN000;
26937                          ;MOV     DX,AX      ;F.C. >32mb DX:AX is the result of ;AN000;
26938                          ; 09/02/2024
26939 00004425 92              xchg     ax,dx
26940 00004426 A1[0E06]        MOV     AX,[TEMP_VAR2]      ;M039; F.C. >32mb a 32 bit multiply ;AN000;
26941
26942                          ; MSDOS 3.3 (& MSDOS 6.0)
26943 00004429 29C8              SUB     AX,CX      ; Take off current "tail"
26944 0000442B 83DA00          SBB     DX,0      ; 32-bit extension
26945 0000442E 01D8              ADD     AX,BX      ; Add on new "tail"
26946 00004430 83D200          ADC     DX,0      ; ripple tim's head off
26947 00004433 EB2B              JMP     SHORT SETGRW
26948
26949 HAVSTART:
26950                          ;int 3
26951                          ;;;
26952                          ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26953 00004435 8936[F80A]      mov     [CLSKIP_HW],si
26954                          ;;;
26955 00004439 89C1              MOV     CX,AX
26956 0000443B E85A13          call    SKPCLP
26957                          ;;;
26958                          ; 09/02/2024
26959 0000443E A1[F80A]        mov     ax,[CLSKIP_HW]
26960 00004441 39C8              cmp     ax,cx
26961 00004443 7502              jne     short HAVSTART_cont
26962                          ;;;
26963                          ; 10/02/2024
26964 00004445 E311              JCXZ     DOWRTJ
26965                          ; 16/12/2022
26966                          ;jcxz     DOWRT
26967                          ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26968                          ;jcxz     DOWRTJ
26969                          ;;;
26970                          ; 09/02/2024
26971 HAVSTART_cont:
26972                          ;mov     [CCOUNT_HW],ax
26973                          ;call    ALLOCATE
26974                          ; 07/07/2024
26975 00004447 E88815          call    ALLOCATE_X
26976                          ; 10/02/2024
26977 0000444A 730C              JNC     short DOWRTJ
26978                          ; 16/12/2022
26979                          ;jnc     short DOWRT
26980                          ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26981                          ;jnc     short DOWRTJ
26982                          ;;;
26983                          ; 10/02/2024
26984                          ;entry    WRTERR
26985 WRTERR:
26986 0000444C B40F              MOV     AH,0FH      ;MS. write/data/fail/abort ;AN000;
26987
26988                          ;entry WRTERR22
26989 WRTERR22:
26990 0000444E A0[7605]        MOV     AL,[THISDRV]      ;MS. ;AN000;
26991
26992                          ; 27/07/2018
26993 WRTERR33:
26994                          ;MOV     CX,0      ;No bytes transferred
26995 00004451 31C9              XOR     CX,CX
26996
26997 00004453 C43E[9E05]      LES     DI,[THISSFT]
26998                          ;CLC ; 19/05/2019
26999                          ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27000                          ; 16/12/2022
27001                          ;clc
27002 00004457 C3              retn
27003
27004                          ; 10/02/2024
27005                          ; 16/12/2022
27006                          ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27007 DOWRTJ:
27008 00004458 EB79              JMP     short DOWRT
27009
27010 TESTTAIL:
27011 0000445A 29D0              SUB     AX,DX
27012 0000445C 7609              JBE     short NOGROW
27013 0000445E 31D2              XOR     DX,DX
27014 SETGRW:
27015 00004460 A3[DE05]        MOV     [GROWCNT],AX
27016 00004463 8916[E005]      MOV     [GROWCNT+2],DX
27017 NOGROW:

```

```

27018             ; 09/02/2024
27019             ; POP     AX ; *!
27020
27021             ; 10/02/2024
27022             %if 0
27023             MOV     CX,[CLUSNUM] ; *+ ; First cluster accessed
27024             ;;;
27025             ; 09/02/2024
27026             mov     dx,[CLUSNUM_HW] ; *+
27027             ;;;
27028             call    FNDCLUS
27029
27030             %else
27031             ; 10/02/2024 - Retro DOS v5.0
27032             call    FNDCLUS_X ; *+
27033             %endif
27034             ;;;
27035             ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27036             pop     ax ; *!
27037             pop     si ; **! ; si:ax = Last cluster
27038             ;;;
27039             JC      short ACC_ERRWJ ; ds=ss
27040             MOV     [CLUSNUM],BX
27041             MOV     [LASTPOS],DX
27042             SUB     AX,DX ; Last cluster minus current cluster
27043             ; 09/02/2024
27044             ; JZ      short DOWRT ; If we have last clus, we must have first
27045             ;;;
27046             ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27047             sbb     si,[LASTPOS_HW]
27048             mov     dx,[CLUSTNUM_HW]
27049             mov     [CLUSNUM_HW],dx
27050             jnz     short NOGROW2
27051             or      ax,ax
27052             jz      short DOWRT
27053             NOGROW2:
27054             cmp     cx,[CLSKIP_HW]
27055             jne     short dskwrt_4
27056             ;;;
27057             JCXZ     HAVSTART ; See if no more data
27058             dskwrt_4:
27059             ; 09/02/2024
27060             PUSH     CX ; No. of clusters short of first
27061             ;;;
27062             ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27063             push     word [CLSKIP_HW]
27064             mov     [CCOUNT_HW],si
27065             ;;;
27066             MOV     CX,AX
27067             call    ALLOCATE
27068             ;;;
27069             ; 09/02/2024
27070             pop     word [CLSKIP_HW]
27071             ;;;
27072             POP     CX
27073             JC      short WRTERR
27074             MOV     DX,[LASTPOS]
27075
27076             ; 09/02/2024
27077             %if 0
27078             INC     DX
27079             DEC     CX
27080             JZ      short NOSKIP
27081             %else
27082             ;;;
27083             ; 09/02/2024 Retro DOS v5.0
27084             ; (PCDOS 7.1 IBMDOS.COM)
27085             ; add     dx,1
27086             ; adc     word [LASTPOS_HW],0
27087             inc     dx
27088             jnz     short dskwrt_10
27089             inc     word [LASTPOS_HW]
27090             dskwrt_10:
27091             sub     cx,1
27092             sbb     word [CLSKIP_HW],0
27093             jnz     short dskwrt_5
27094             or      cx,cx
27095             jz      short NOSKIP
27096             dskwrt_5:
27097             ;;;
27098             %endif
27099             call    SKPCLP
27100             JC      short ACC_ERRWJ ; ds=ss ; 10/02/2024
27101
27102             NOSKIP:
27103             MOV     [CLUSNUM],BX
27104             ;;;
27105             ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27106             mov     ax,[CLUSTNUM_HW]
27107             mov     [CLUSNUM_HW],ax
27108             ;;;
27109             MOV     [LASTPOS],DX
27110             DOWRT:
27111             CMP     word [BYTCNT1],0
27112             JZ      short WRTMID
27113             ;;;
27114             ; 09/02/2024
27115             mov     bx,[CLUSNUM_HW]
27116             mov     [CLUSTNUM_HW],bx
27117             ;;;
27118             ; 09/02/2024
27119             ; MOV     BX,[CLUSNUM] ; (not used in 'BUFVRT') ; 09/02/2024
27120             call    BUFVRT
27121             JC      short ACC_ERRWJ
27122             ; 09/02/2024
27123             jnc     short WRTMID
27124
27125             ; 09/02/2024
27126             ACC_ERRWJ:
27127             ; 10/08/2018
27128             ; JMP     SET_ACC_ERRW
27129             ; 16/12/2022
27130             ; jmp     SET_ACC_ERR_DS ; ds<=>ss ; 10/02/2024
27131             ; 10/02/2024
27132             ; ds=ss
27133             jmp     SET_ACC_ERR
27134             ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27135             ; jmp     SET_ACC_ERRW
27136
27137             WRTMID:
27138             MOV     AX,[SECCNT]
27139             OR      AX,AX
27140             ; 20/11/2022
27141             ; JZ      short WRTLAST ; 24/07/2019 ; M039
27142             ; 10/02/2024

```

```

27142 000044EF 7503      jnz     short WRTMID2
27143 000044F1 E98600     jmp     WRTLAST
27144                                WRTMID2:
27145 000044F4 0106[C405]  ADD     [SECPOS],AX
27146                                ; 19/05/2019
27147                                ; MSDOS 6.0
27148 000044F8 8316[C605]00 ADC     WORD [SECPOS+2],0      ;F.C. >32mb      ;AN000;
27149 000044FD E8BD13     call    NEXTSEC
27150                                ; 16/12/2022
27151 00004500 72E5       JC      short ACC_ERRWJ
27152                                ;JC      short SET_ACC_ERRW      ;M039
27153 00004502 C606[7405]01 MOV     BYTE [TRANS],1        ; A transfer is taking place
27154 00004507 8A16[7305]  MOV     DL,[SECCLUSPOS]      ; (dx/DL = Extent start) ((dh = ?))
27155                                ;;;
27156                                ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27157 0000450B 8B1E[DE0A]  mov     bx,[CLUSNUM_HW]
27158 0000450F 891E[E80A]  mov     [CLUSTNUM_HW],bx
27159                                ;;;
27160 00004513 8B1E[BC05]  MOV     BX,[CLUSNUM]
27161 00004517 8B0E[D605]  MOV     CX,[SECCNT]
27162                                WRTLTP:
27163 0000451B E8E613     call    OPTIMIZE
27164                                ; 09/02/2024
27165                                ;JC      short SET_ACC_ERRW
27166                                ; 16/12/2022
27167 0000451E 72C7       JC      short ACC_ERRWJ ; ds=ss ; 10/02/2024
27168
27169                                ;M039
27170                                ;
27171                                ; DI = Next physical cluster.
27172                                ; AX = # sectors remaining.
27173                                ; [DMAADD+2]:BX = transfer address (source data address).
27174                                ; CX = # of contiguous sectors to write. (These constitute a block of
27175                                ; sectors, also termed an "Extent".)
27176                                ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
27177                                ; ES:BP -> Drive Parameter Block (DPB).
27178                                ;
27179                                ; Purge the Buffer Queue and the Secondary Cache of any buffers which
27180                                ; are in Extent; they are being over-written.
27181                                ;;;
27182                                ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27183 00004520 FF36[EC0A]  push    word [CCONTENT_HW]
27184                                ;;;
27185
27186 00004524 57          push    di      ; CCONTENT_HW:DI = Next physical cluster
27187 00004525 50          push    ax      ; AX = # sectors remaining
27188
27189                                ; MSDOS 3.3
27190                                ; IBMDOS.COM (1987) - Offset 4497h
27191                                ;push    dx
27192                                ;push    bx
27193                                ;mov     al,[es:bp]
27194                                ;;mov    AL,[ES:BP+DPB.DRIVE] ; mov al,[es:bp+0]
27195                                ;mov     bx,cx
27196                                ;add     bx,dx ; (bx = Extent end)
27197
27198                                ; DX = Extent start.
27199                                ; BX = Extent end.
27200                                ; AL = Drive #.
27201
27202                                ;call    SETVISIT
27203
27204                                ;wbufq1:
27205                                ;;or     byte [di+5],20h
27206                                ;;or     byte [DI+BUFFINFO.buf_flags],buf_visit ; Bit 5 = reserved
27207                                ;;cmp    al,[di+4]
27208                                ;;cmp    al,[DI+BUFFINFO.buf_ID]
27209                                ;;jnz    short wbufq2 ; Jump if Extent start > buffer sector.
27210                                ;;cmp    [di+6],dx
27211                                ;;cmp    [DI+BUFFINFO.buf_sector],dx
27212                                ;;jb     short wbufq2
27213                                ;;cmp    [di+6],bx
27214                                ;;cmp    [DI+BUFFINFO.buf_sector],bx
27215                                ;;jnb    short wbufq2 ; Jump if Extent end >= buffer sector.
27216
27217                                ;; Buffer sector is in the Extent
27218
27219                                ;;mov    word [di+4],20FFh
27220                                ;;mov    word [DI+BUFFINFO.buf_ID],20FFh
27221                                ;;                                ; .buf_ID, AL = FFh (Free buffer)
27222                                ;;                                ; .buf_flags, AH = 0, reset/clear
27223                                ;call    SCANPLACE
27224                                ;wbufq2:
27225                                ;call    SKIPVISIT
27226                                ;jnz     short wbufq1
27227                                ;pop     bx
27228                                ;pop     dx
27229
27230                                ; MSDOS 6.0
27231 00004526 E89101     call    DskwrtBufPurge ;DS trashed.
27232
27233                                ;ASSUME DS:NOTHING
27234                                ;M039
27235                                ; MSDOS 3.3 & MSDOS 6.0
27236                                ;hkn; SS override for DMAADD and ALLOWED
27237 00004529 368E1E[2E03]  MOV     DS,[SS:DMAADD+2]
27238                                ;mov     byte [ss:ALLOWED],38h
27239 0000452E 36C606[4B03]38  MOV     byte [SS:ALLOWED],Allowed_RETRY+Allowed_FAIL+Allowed_IGNORE
27240
27241                                ; put logic from DWRITE in-line here so we can modify it
27242                                ; for DISK FULL conditions.
27243
27244                                ; 20/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27245                                ; DOSCODE:7AD8h (MSDOS 5.0 MSDOS.SYS)
27246
27247                                ; 16/12/2022
27248                                ; MSDOS 3.3 (& MSDOS 5.0)
27249                                ;call    DWRITE
27250
27251                                ;DWRITE_OKAY:
27252                                ;
27253                                ; 16/12/2022
27254                                ; MSDOS 5.0 (& MSDOS 3.3)
27255                                ;pop     cx
27256                                ;pop     bx
27257                                ;push    ss
27258                                ;pop     ds
27259                                ;jc      short SET_ACC_ERRW
27260                                ;jcxz    WRTLAST
27261                                ;mov     dl,0
27262                                ;inc     word [LASTPOS]
27263                                ;jmp     short WRTLTP
27264
27265                                ; 16/12/2022

```

```

27266             ; 20/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27267 DWRITE_LUP:
27268             ; 23/07/2019 - Retro DOS v3.2
27269
27270             ; MSDOS 6.0
27271 call     DSKWRITE
27272 00004537 7417 jz      short DWRITE_OKAY ; ds<>ss
27273
27274 ;; int     3
27275
27276 cmp      al,error_handle_Disk_Full ; compressed volume full?
27277 0000453B 742D jz      short DWRITE_DISK_FULL
27278
27279             ; 16/12/2022
27280
27281 ;;hkn; SS override
27282 MOV      BYTE [SS:READOP],1
27283 call     HARDERRR
27284 00004546 3C01 CMP     AL,1 ; Check for retry
27285 00004548 74EA JZ      short DWRITE_LUP
27286
27287             ; 16/12/2022
27288             ; 23/07/2019
27289 ;POP      CX ; *4*
27290 ;POP      BX ; *5*
27291
27292 ;push     ss
27293 ;pop      ds
27294
27295
27296             ; 20/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27297
27298             ; 16/12/2022
27299 0000454A 3C03 CMP     AL,3 ; Check for FAIL
27300 0000454C F8 CLC
27301 0000454D 7501 JNZ     short DWRITE_OKAY ; Ignore
27302 0000454F F9 STC
27303
27304 DWRITE_OKAY:
27305             ; 16/12/2022
27306             ; 23/07/2019
27307             ; MSDOS 3.3 (& MSDOS 6.0)
27308 00004550 59 POP     CX ; *4*
27309 00004551 5B POP     BX ; *5*
27310
27311 ;         CX = # sectors remaining.
27312 ;         BX = Next physical cluster.
27313
27314 ;;hkn; SS override
27315 ;Context DS
27316             ; 16/12/2022
27317 ;push     ss
27318 ;pop      ds
27319
27320             ; 09/02/2024 - Retro DOS v5.0
27321             ; 16/12/2022
27322 ;jc       short SET_ACC_ERRW
27323
27324             ; 16/12/2022
27325 00004552 16 push    ss
27326 00004553 1F pop     ds
27327
27328 ;;;
27329             ; 09/02/2024 - Retro DOS v5.0
27330             ; (PCDOS 7.1 IBMDOS.COM)
27331 00004554 8F06[E80A] pop     word [CLUSTNUM_HW] ; CLUSTNUM_HW:BX = Next physical cluster
27332 00004558 725D jc      short SET_ACC_ERRW ; ds=ss
27333 ;;;
27334
27335 0000455A E31E JCXZ    WRTLAST
27336
27337             ; 10/02/2024
27338 MOV      DL,0
27339 ;xor      dl,dl ; 23/07/2019
27340 0000455E FF06[BA05] INC     word [LASTPOS]; We'll be using next cluster
27341 ;;;
27342             ; 09/02/2024 - Retro DOS v5.0
27343             ; (PCDOS 7.1 IBMDOS.COM)
27344 00004562 75B7 jnz     short WRTLP
27345 00004564 FF06[E20A] inc     word [LASTPOS_HW]
27346 ;;;
27347 00004568 EBB1 JMP     short WRTLP
27348
27349             ; 23/07/2019 - Retro DOS v3.2
27350             ; 09/08/2018
27351             ; MSDOS 6.0
27352 DWRITE_DISK_FULL:
27353 ;Context DS ;SQ 3-5-93 DS must be setup on return!
27354             ; 16/12/2022
27355 0000456A 16 push    ss
27356 0000456B 1F pop     ds
27357 0000456C 59 pop     cx ; unjunk stack
27358 0000456D 5B pop     bx
27359 ;;;
27360             ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27361 0000456E 8F06[E80A] pop     word [CLUSTNUM_HW]
27362 ;;;
27363 00004572 C606[0B06]01 mov     byte [DISK_FULL],1
27364 ;stc
27365 00004577 E9D2FE jmp     WRTERR ; 24/07/2019 ; go to disk full exit
27366
27367 WRTLAST:
27368 MOV      AX,[BYTCNT2]
27369 0000457D 09C0 OR      AX,AX
27370 0000457F 7413 JZ      short FINWRT
27371 00004581 A3[D205] MOV     [BYTCNT1],AX
27372 00004584 E83613 call    NEXTSEC
27373 00004587 722E JC      short SET_ACC_ERRW
27374 00004589 C706[CC05]0000 MOV     word [BYTSECPOS],0
27375 0000458F E8D012 call    BUFWRT
27376 00004592 7223 JC      short SET_ACC_ERRW
27377
27378 FINWRT:
27379 00004594 C43E[9E05] LES     DI,[THISST]
27380 00004598 A1[DE05] MOV     AX,[GROWCNT]
27381 0000459B 8B0E[E005] MOV     CX,[GROWCNT+2]
27382 0000459F 09C0 OR      AX,AX
27383 000045A1 7502 JNZ     short UPDATE_size
27384 000045A3 E30F JCXZ    SAMSIZ
27385
27386 UPDATE_size:
27387 000045A5 26014511 ;add [es:di+11h],ax
27388 ADD     [ES:DI+SF_ENTRY.sf_size],AX
27389 000045A9 26114D13 ;adc [es:di+13h],cx
27390 ADC     [ES:DI+SF_ENTRY.sf_size+2],CX

```

```

27390
27391
27392 ; Make sure that all other SFT's see this growth also.
27393 000045AD B80100      MOV     AX,1
27394 ;if installed
27395 ;call JShare + 14 * 4
27396 000045B0 FF1E[C800]  call    far [JShare+(14*4)] ; 14 = ShSU
27397 ;else
27398 ; Call shSU
27399 ;endif
27400
27401 SAMSIZ:
27402 000045B4 E9E2FC      jmp     SETCLUS ; ES:DI already points to SFT
27403
27404 ; 16/12/2022
27405 ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27406 SET_ACC_ERRW:
27407 ;jmp SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
27408 ; 10/02/2024
27409 ; ds=ss
27410 000045B7 E92EFC      jmp     SET_ACC_ERR
27411
27412 WRTEOF:
27413 000045BA 89C1        MOV     CX,AX
27414 000045BC 09D1        OR      CX,DX
27415 ;JZ short KILLFIL
27416 ; 10/02/2024
27417 000045BE 7503        jnz     short WRTEOF1
27418 000045C0 E9A600      jmp     KILLFIL
27419
27420 ;;;
27421 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27422 WRTEOF1:
27423 000045C3 81FAFF7F    cmp     dx,7FFFh ; 2GB file size limit
27424 000045C7 7503        jne     short WRTEOF2
27425 000045C9 83F8FF    cmp     ax,0FFFFh
27426 WRTEOF2:
27427 000045CC 760D        jna     short WRTEOF3
27428 ;
27429 000045CE 06          push    es
27430 ;les di,ds:THISSFT
27431 ; 27/06/2024
27432 000045CF C43E[9E05]    les     di,[THISSFT]
27433 ;test byte [es:di+3],10h
27434 000045D3 26F6450310    test   byte [es:di+SF_ENTRY.sf_mode+1],10h
27435 000045D8 07          pop     es
27436 000045D9 74DC        jz      short SET_ACC_ERRW ; > 2GB file size not allowed
27437 WRTEOF3:
27438 ;;;
27439
27440 000045DB 83E801    SUB     AX,1
27441 000045DE 83DA00    SBB     DX,0
27442
27443 ; MSDOS 3.3
27444 ;;;div word [es:bp+2]
27445 ;div word [ES:BP+DPB.SECTOR_SIZE]
27446 ;;mov c1,[es:bp+5]
27447 ;mov c1,[ES:BP+DPB.CLUSTER_SHIFT]
27448 ;shr ax,c1
27449
27450 ; MSDOS 6.0
27451 000045E1 53          PUSH    BX
27452 ;mov bx,[es:bp+2]
27453 000045E2 268B5E02    MOV     BX,[ES:BP+DPB.SECTOR_SIZE] ;F.C. >32mb ;AN000;
27454 000045E6 E81E01    CALL    DIV32 ;F.C. >32mb ;AN000;
27455 000045E9 5B          POP     BX ;F.C. >32mb ;AN000;
27456 000045EA 89CA        MOV     DX,CX ;M039
27457 000045EC 890E[0706]    MOV     [HIGH_SECTOR],CX ;M039: Probably extraneous, but not sure.
27458 000045F0 E83B01    CALL    SHR32 ;F.C. >32mb ;AN000;
27459
27460 000045F3 89C1        MOV     CX,AX
27461 000045F5 E84311    call    FNDCLUS
27462 SET_ACC_ERRWJ2:
27463 000045F8 72BD        JC      short SET_ACC_ERRW
27464 ;;;
27465 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27466 000045FA A1[F80A]    mov     ax,[CLSKIP_HW]
27467 000045FD 39C8        cmp     ax,cx
27468 000045FF 7502        jne     short dskwrt_6
27469 ;;;
27470
27471 00004601 E324        JCXZ    RELFILE
27472
27473 ;;;
27474 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27475 dskwrt_6:
27476 ;mov [CCOUNT_HW],ax
27477 ;;;
27478 ;
27479 ;call ALLOCATE
27480 ; 07/07/2024
27481 00004603 E8CC13    call    ALLOCATE_X
27482 ;JC short WRTERRJ ;;;;;;;;;; disk full
27483 ; 16/12/2022
27484 00004606 7303        jnc     short UPDATE
27485 00004608 E941FE      JMP     WRTERR
27486 UPDATE:
27487 0000460B C43E[9E05]    LES     DI,[THISSFT]
27488 0000460F A1[CE05]    MOV     AX,[BYTPOS]
27489 ;mov [es:di+11h],ax
27490 00004612 26894511    MOV     [ES:DI+SF_ENTRY.sf_size],AX
27491 00004616 A1[D005]    MOV     AX,[BYTPOS+2]
27492 ;mov [es:di+13h],ax
27493 00004619 26894513    MOV     [ES:DI+SF_ENTRY.sf_size+2],AX
27494 ;
27495 ; Make sure that all other SFT's see this growth also.
27496 ;
27497 0000461D B80200      MOV     AX,2
27498 ;if installed
27499 ;call JShare + 14 * 4
27500 00004620 FF1E[C800]  call    far [JShare+(14*4)] ; 14 = ShSU
27501 ;else
27502 ; Call shSU
27503 ;endif
27504 00004624 31C9      XOR     CX,CX ; 0
27505 ;jmp ADDREC
27506 ; 08/02/2024
27507 00004626 C3          retn
27508
27509 ; 16/12/2022
27510 ;WRTERRJ:
27511 ;JMP WRTERR
27512
27513 ;;;;;;;;;; 7/18/86

```



```

27514
27515
27516
27517
27518 00004627 06
27519 00004628 C43E[9E05]
27520
27521
27522 0000462C 50
27523 0000462D A1[E20A]
27524 00004630 263B450B
27525
27526 00004634 58
27527 00004635 7504
27528
27529
27530 00004637 263B5519
27531
27532 0000463B 731C
27533
27534
27535 0000463D 268B552B
27536
27537
27538
27539 00004641 26C745190000
27540
27541
27542
27543
27544
27545
27546
27547 00004647 26C7450B0000
27548
27549
27550
27551
27552 0000464D 26895535
27553
27554
27555 00004651 268B552D
27556
27557 00004655 26895537
27558
27559
27560
27561
27562 00004659 07
27563
27564
27565
27566
27567
27568
27569
27570 0000465A 31D2
27571 0000465C 4A
27572 0000465D 8916[EA0A]
27573
27574
27575 00004661 E86E15
27576
27577
27578
27579 00004664 73A5
27580
27581
27582
27583
27584
27585
27586
27587 00004666 E97FFB
27588
27589
27590
27591
27592
27593 00004669 31DB
27594 0000466B 06
27595 0000466C C43E[9E05]
27596
27597
27598
27599
27600
27601
27602
27603
27604
27605
27606
27607
27608 00004670 26875D2D
27609
27610 00004674 891E[E80A]
27611 00004678 31DB
27612 0000467A 26895D0B
27613
27614 0000467E 26895D37
27615
27616
27617 00004682 26895D19
27618
27619 00004686 26895D35
27620 0000468A 26875D2B
27621
27622
27623
27624
27625 0000468E 07
27626
27627
27628
27629 0000468F 3B1E[E80A]
27630 00004693 7507
27631
27632
27633 00004695 09DB
27634
27635
27636
27637

;;;;;;;;;;;;;;;;
RELFILE:
; MSDOS 6.0
PUSH ES ;AN002; BL Reset Lstclus and cluspos to
LES DI,[THISSFT] ;AN002; BL beginning of file if current
;;;
; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
push ax ; cluspos is past EOF.
mov ax,[LASTPOS_HW]
cmp ax,[es:di+0Bh] ; [ES:DI+SF_ENTRY.sf_firclus]
;cmp ax,[es:di+SF_ENTRY.sf_cluspos_hw]
pop ax
jne short dskwrt_7
;;;
;cmp dx,[es:di+19h]
CMP DX,[ES:DI+SF_ENTRY.sf_cluspos] ;AN002; BL cluspos is past EOF.
; 10/02/2024
dskwrt_7: JAE short SKIPRESET ;AN002; BL
;;;
; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
mov dx,[es:di+2Bh] ; [ES:DI+SF_ENTRY.sf_chain]
;mov dx,[es:di+SF_ENTRY.sf_fccluster] ; first cluster (32 bit) lw !?
;;;
;mov [es:di+19h],0
MOV word [ES:DI+SF_ENTRY.sf_cluspos],0 ;AN002; BL
; 10/02/2024
%if 0
;mov dx,[es:di+0Bh]
MOV DX,[ES:DI+SF_ENTRY.sf_firclus] ;AN002; BL
%else
;;;
; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
mov word [es:di+0Bh],0
;mov word [es:di+SF_ENTRY.sf_cluspos_hw],0
;;;
%endif
;mov [es:di+35h],dx
MOV [ES:DI+SF_ENTRY.sf_lstclus],DX ;AN002; BL
;;;
; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
mov dx,[es:di+2Dh] ; [ES:DI+SF_ENTRY.sf_chain]
;mov di,[es:di+SF_ENTRY.sf_fccluster+2] ; first clust (32 bit) hw !?
mov [es:di+37h],dx
;mov [es:di+SF_ENTRY.sf_lstclus+2],dx
;;;
SKIPRESET:
POP ES ;AN002; BL
;AN002; BL
;
; 10/02/2024
%if 0
MOV DX,0FFFFH
%else
;;;
; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
xor dx,dx
dec dx
mov [CLUSDATA_HW],dx ; 0FFFFh
;;;
%endif
call RELBLKS
; 16/12/2022
; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
dskwrt_8: ; 10/02/2024
jnc short UPDATE
SET_ACC_ERRWJ:
;JC short SET_ACC_ERRWJ2
;JMP SHORT UPDATE
; 16/12/2022
;jmp SET_ACC_ERR_DS ; ds<>ss
; 10/02/2024
; ds=ss
;jmp SET_ACC_ERR
; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;JC short SET_ACC_ERRWJ2
;JMP SHORT UPDATE
KILLFIL:
XOR BX,BX
PUSH ES
LES DI,[THISSFT]
; 10/02/2024
%if 0
;mov [es:di+19h],bx
MOV [ES:DI+SF_ENTRY.sf_cluspos],BX
;mov [es:di+35h],bx ; 04/05/2019
MOV [ES:DI+SF_ENTRY.sf_lstclus],BX
;xchg bx,[es:di+0Bh]
XCHG BX,[ES:DI+SF_ENTRY.sf_firclus]
%else
;;;
; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
xchg bx,[es:di+2Dh] ; [ES:DI+SF_ENTRY.sf_chain+2]
;xchg bx,[es:di+SF_ENTRY.sf_fccluster+2] ; first clust (32 bit) hw !?
mov [CLUSTNUM_HW],bx
xor bx,bx ; 0
mov [es:di+0Bh],bx ; [ES:DI+SF_ENTRY.sf_firclus]
;mov [es:di+SF_ENTRY.sf_cluspos_hw],bx
mov [es:di+37h],bx
;mov [es:di+SF_ENTRY.sf_lstclus+2],bx
;mov [es:di+19h],bx
mov [es:di+SF_ENTRY.sf_cluspos],bx
;mov [es:di+35h],bx
mov [es:di+SF_ENTRY.sf_lstclus],bx
xchg bx,[es:di+2Bh] ; [ES:DI+SF_ENTRY.sf_chain]
;xchg bx,[es:di+SF_ENTRY.sf_fccluster] ; first cluster (32 bit) lw !?
;;;
%endif
;mov [es:di+19h],bx
POP ES
;;;
; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
cmp bx,[CLUSTNUM_HW]
jnz short dskwrt_9
;;;
OR BX,BX
;JZ short UPDATEJ
; 16/12/2022
;jz short UPDATE
; 10/12/2024

```

```

27638 00004697 7503      jnz     short dskwrt_9
27639 00004699 E96FFF      jmp     UPDATE
27640
27641      ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27642      ;jz     short UPDATEJ
27643
27644 dskwrt_9:  ; 10/02/2024
27645 ;; 10/23/86 FastOpen update
27646      PUSH     ES      ; since first cluster # is 0
27647      PUSH     BP      ; we must delete the old cache entry
27648      PUSH     AX
27649      PUSH     CX
27650      PUSH     DX
27651      LES      BP,[THISDPB]      ; get current DPB
27652      ; 15/12/2022
27653      mov     dl,[ES:BP] ; mov dl,[es:bp+0]
27654      ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27655      ;MOV     DL,[ES:BP+DPB.DRIVE] ; get current drive
27656      MOV     CX,BX      ; first cluster #
27657      MOV     AH,2      ; delete cache entry by drive:firclus
27658      call    FastOpen_Update      ; call fastopen
27659      POP      DX
27660      POP      CX
27661      POP      AX
27662      POP      BP
27663      POP      ES
27664 ;; 10/23/86 FastOpen update
27665
27666      call    RELEASE
27667      ;JC     short SET_ACC_ERRWJ
27668 ;UPDATEJ:
27669      ; 20/11/2022
27670      ;JMP     short UPDATE ; 10/08/2018
27671      ; 10/02/2024
27672      jmp     short dskwrt_8
27673
27674 ;Break <DskWrtBufPurge -- Disk Write Buffer Purge>
27675 -----
27676 ;
27677 ; Procedure Name : DskWrtBufPurge
27678 ;
27679 ; Inputs:
27680 ;     CX = # of contiguous sectors to write. (These constitute a block of
27681 ;     sectors, also termed an "Extent".)
27682 ;     [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
27683 ;     ES:BP -> Drive Parameter Block (DPB).
27684 ;
27685 ; Function:
27686 ;     Purge the Buffer Queue and the Secondary Cache of any buffers which
27687 ;     are in Extent; they are being over-written.
27688 ;
27689 ; Outputs:
27690 ;     (Same as Input.)
27691 ;
27692 ; Uses:
27693 ;     All registers except DS,AX,SI,DI preserved.
27694 ;     SS override for all global variables.
27695 -----
27696 ;M039: Created
27697
27698 ;procedure DskWrtBufPurge,NEAR
27699 ;
27700 ;ASSUME DS:NOTHING
27701
27702 ; 04/05/2019 - Retro DOS v4.0
27703 ; DOSCODE:7C0Eh (MSDOS 6.21, MSDOS.SYS)
27704
27705 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27706 ; DOSCODE:7BD4h (MSDOS 5.0, MSDOS.SYS)
27707
27708 ; 10/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
27709 ; DOSCODE:8714h (PCDOS 7.1, IBMDOS.COM)
27710
27711 ; NOTE: Secondary (Buffer) Cache is not used in PCDOS 7.1 IBMDOS.COM
27712
27713 ; (Win ME IO.SYS - BIOSCODE:BB93h)
27714 ; (MSDOS 6.22 IO.SYS - DOSCODE:7C0Eh)
27715
27716 DskWrtBufPurge:
27717 ;SaveReg <bx,cx>
27718      push     bx
27719      push     cx
27720
27721      mov     bx,[ss:HIGH_SECTOR] ;BX:DX = Extent start (sector #).
27722      mov     si,bx
27723      add     cx,dx
27724      adc     si,0      ;SI:DX = Extent end + 1.
27725
27726      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27727      ;mov     al,[es:bp+0]
27728      ;mov     al,[es:bp+DPB.DRIVE]
27729      ; 15/12/2022
27730      mov     al,[es:bp]
27731
27732 ; 10/02/2024 - Retro DOS v5.0
27733 ; PCDOS 7.1 IBMDOS.COM
27734 %if 0
27735 ;
27736 ;     BX:DX = Extent start.
27737 ;     SI:DX = Extent end + 1.
27738 ;     AL = Drive #
27739 ;
27740      cmp     word [ss:SC_CACHE_COUNT],0 ;Secondary cache in-use?
27741      je      short nosc      ; -no, jump.
27742
27743 ; If any of the sectors to be written are in the secondary cache (SC),
27744 ; invalidate the entire SC. (This is an optimization; we really only
27745 ; need to invalidate those sectors which intersect, but that's slower.)
27746
27747      cmp     al,[ss:CurSC_DRIVE] ;Same drive?
27748      jne     short nosc      ; -no, jump.
27749
27750      push     ax
27751      mov     ax,[ss:CurSC_SECTOR]
27752      mov     di,[ss:CurSC_SECTOR+2] ;DI:AX = SC start.
27753
27754      ;Cmp32 si,cx,di,ax      ;Extent end < SC start?
27755      ;jbe     short sc5      ; -yes, jump.
27756
27757      cmp     si,di
27758      jne     short sc01
27759      cmp     cx,ax
27760 sc01:
27761      jbe     short sc5

```



```

27762      add     ax,[ss:SC_CACHE_COUNT]
27763      adc     di,0                      ;DI:AX = SC end + 1.
27764
27765      ;Cmp32  bx,dx,di,ax              ;Extent start > SC end?
27766      ;jae    short sc5                ; -yes, jump.
27767
27768      cmp     bx,di
27769      jne     short sc02
27770      cmp     dx,ax
27771      sc02:
27772      jnb     short sc5
27773
27774      mov     word [ss:SC_STATUS],0 ;Extent intersects SC: invalidate SC.
27775      sc5:
27776      pop     ax
27777
27778      %endif
27779
27780      ; Free any buffered sectors which are in Extent; they are being over-
27781      ; written.
27782
27783      nosc:
27784      call    GETCURHEAD                ;DS:DI -> first buffer in queue.
27785
27786      _bufq:
27787      ;cmpo    al,[di+4]
27788      cmp     al,[di+BUFFINFO.buf_ID] ;Same drive?
27789      jne     short bufq5                ; -no, jump.
27790
27791      ; Cmp32  bx,dx,<WORD PTR [di.buf_sector+2]>,<WORD PTR [di.buf_sector]>
27792      ; ja     short bufq5                ;Jump if Extent start > buffer sector.
27793
27794      ;cmp     bx,[di+8]
27795      cmp     bx,[di+BUFFINFO.buf_sector+2]
27796      jne     short bufq04
27797      ;cmp     dx,[di+6]
27798      cmp     dx,[di+BUFFINFO.buf_sector]
27799      bufq04:
27800      ja     short bufq5
27801
27802      ; Cmp32  si,cx,<WORD PTR [di.buf_sector+2]>,<WORD PTR [di.buf_sector]>
27803      ; jbe    short bufq5                ;Jump if Extent end < buffer sector.
27804
27805      ;cmp     si,[di+8]
27806      cmp     si,[di+BUFFINFO.buf_sector+2]
27807      jne     short bufq05
27808      ;cmp     cx,[di+6]
27809      cmp     cx,[di+BUFFINFO.buf_sector]
27810      bufq05:
27811      jbe     short bufq5
27812
27813      ; Buffer's sector is in Extent, so free it; it is being over-written.
27814
27815      ;test    byte [di+5],40h
27816      test    byte [di+BUFFINFO.buf_flags],buf_dirty ;Buffer dirty?
27817      jz      short bufq4                ; -no, jump.
27818      call    DEC_DIRTY_COUNT            ; -yes, decrement dirty count.
27819      bufq4:
27820      ;mov     word [di+4],20FFh
27821      mov     word [di+BUFFINFO.buf_ID],((buf_visit<<8)|0FFh)
27822
27823      call    SCANPLACE
27824      jmp     short bufq6
27825      bufq5:
27826      mov     di,[di]
27827      ;mov     di,[di+BUFFINFO.buf_next]
27828      bufq6:
27829      cmp     di,[ss:FIRST_BUFF_ADDR]    ;Scanned entire buffer queue?
27830      jne     short _bufq                ; --no, go do next buffer.
27831
27832      ;RestoreReg <cx,bx>
27833      pop     cx
27834      pop     bx
27835      retn
27836
27837      ;EndProc DskWrtBufPurge
27838
27839      ;Break   <DIV32 -- PERFORM 32 BIT DIVIDE>
27840      ;-----
27841      ;
27842      ; Procedure Name : DIV32
27843      ;
27844      ; Inputs:
27845      ; DX:AX = 32 bit dividend  BX= divisor
27846      ; Function:
27847      ; Perform 32 bit division: DX:AX/BX = CX:AX + DX (rem.)
27848      ; Outputs:
27849      ; CX:AX = quotient , DX= remainder
27850      ; Uses:
27851      ; All registers except AX,CX,DX preserved.
27852      ;-----
27853      ;M039: DIV32 optimized for divisor of 512 (common sector size).
27854
27855      ; 04/05/2019 - Retro DOS v4.0
27856      ; DOSCODE:7C94h (MSDOS 6.21, MSDOS.SYS)
27857
27858      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27859      ; DOSCODE:7C5Ah (MSDOS 5.0, MSDOS.SYS)
27860
27861      DIV32:
27862      cmp     bx,512
27863      jne     short div5
27864
27865      mov     cx,dx
27866      mov     dx,ax                      ; CX:AX = Dividend
27867      and     dx,(512-1)                 ; DX = Remainder
27868      mov     al,ah
27869      mov     ah,cl
27870      mov     cl,ch
27871      xor     ch,ch
27872      shr     cx,1
27873      rcr     ax,1
27874      retn
27875      div5:
27876      mov     cx,ax
27877      mov     ax,dx
27878      xor     dx,dx
27879      div     bx                          ; 0:AX/BX
27880      xchg    cx,ax
27881      div     bx                          ; DX:AX/BX
27882      retn
27883
27884      ;Break   <SHR32 -- PERFORM 32 BIT SHIFT RIGHT>
27885      ;-----

```

```

27886 ;
27887 ; Procedure Name : SHR32
27888 ;
27889 ; Inputs:
27890 ; DX:AX = 32 bit sector number
27891 ; Function:
27892 ; Perform 32 bit shift right
27893 ; Outputs:
27894 ; AX = cluster number
27895 ; ZF = 1 if no error
27896 ; = 0 if error (cluster number > 64k)
27897 ; Uses:
27898 ; DX,CX
27899 ;-----
27900 ; M017 - SHR32 rewritten for better performance
27901 ; M039 - Additional optimization
27902 ;
27903 ; 04/05/2019 - Retro DOS v4.0
27904 ; DOSCODE:7CBBh (MSDOS 6.21, MSDOS.SYS)
27905 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27906 ; DOSCODE:7C81h (MSDOS 5.0, MSDOS.SYS)
27907
27908 SHR32:
27909 ;mov c], [es:bp+5]
27910 mov c], [ES:BP+DPB.CLUSTER_SHIFT]
27911 xor ch, ch ;ZF=1
27912 jcxz norota
27913 rotashft2:
27914 shr dx, 1 ;ZF reflects state of DX.
27915 rcr ax, 1 ;ZF not affected.
27916 loop rotashft2
27917 norota:
27918 ret
27919
27920 ;-----
27921 ; DIR.ASM, MSDOS 6.0, 1991
27922 ;-----
27923 ; 27/07/2018 - Retro DOS v3.0
27924 ; 19/05/2019 - Retro DOS v4.0
27925
27926 ; TITLE DIR - Directory and path cracking
27927 ; NAME Dir
27928
27929 ;Break <FINDENTRY -- LOOK FOR AN ENTRY>
27930 ;-----
27931 ;
27932 ; Procedure Name : FINDENTRY, SEARCH
27933 ;
27934 ; Inputs:
27935 ; [THISDPB] set
27936 ; [SECCLUSPOS] = 0
27937 ; [DIRSEC] = Starting directory sector number
27938 ; [CLUSNUM] = Next cluster of directory
27939 ; [CLUSFAC] = Sectors/Cluster
27940 ; [NAME1] = Name to look for
27941 ; Function:
27942 ; Find file name in disk directory.
27943 ; "?" matches any character.
27944 ; Outputs:
27945 ; Carry set if name not found
27946 ; ELSE
27947 ; Zero set if attributes match (always except when creating)
27948 ; AH = Device ID (bit 7 set if not disk)
27949 ; [THISDPB] = Base of drive parameters
27950 ; DS = DOSGROUP
27951 ; ES = DOSGROUP
27952 ; [CURBUF+2]:BX = Pointer into directory buffer
27953 ; [CURBUF+2]:SI = Pointer to First Cluster field in directory entry
27954 ; [CURBUF] has directory record with match
27955 ; [NAME1] has file name
27956 ; [LASTENT] is entry number of the entry
27957 ; All other registers destroyed.
27958 ;-----
27959
27960 ;hkn; called from rename.asm and dir2.asm. DS must be already set up at
27961 ;hkn; this point.
27962
27963 SEARCH:
27964 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27965 ; DOSCODE:7C90h (MSDOS 5.0, MSDOS.SYS)
27966
27967 ; 19/05/2019 - Retro DOS v4.0
27968 ; DOSCODE:7CCAh (MSDOS 6.21, MSDOS.SYS)
27969
27970 ; 27/07/2018 - Retro DOS v3.0
27971 ; IBMDOS.COM (MSDOS 3.3) - Offset 45B3h
27972 ; 15/03/2018 - Retro DOS v2.0
27973
27974 ; 24/01/2024
27975
27976 ; 13/02/2024 - Retro DOS v5.0
27977 ; DOSCODE:8797h (PCDOS 7.1, IBMDOS.COM)
27978
27979 ;entry FindEntry
27980 FINDENTRY:
27981 call STARTSRCH
27982 mov AL, [ATTRIB]
27983 ;and al, 9Eh
27984 AND AL, ~attr_ignore ; Ignore useless bits
27985 ;cmp al, 8
27986 CMP AL, attr_volume_id ; Looking for vol ID only ?
27987 JNZ short NOTVOLSRCH ; No
27988 CALL SETROOTSRCH ; Yes force search of root
27989 NOTVOLSRCH:
27990 CALL GETENTRY
27991 ;JNC short SRCH
27992 ;JMP SETESRET
27993 ; 24/01/2024
27994 jnc short SETESRET
27995
27996 ;entry Srch
27997 SRCH:
27998 ;;
27999 ; 13/02/2024 (PCDOS 7.1 IBMDOS.COM)
28000 mov word [LNE_COUNT], 0 ; reset long name entry count
28001 SRCH2:
28002 ;;
28003
28004 PUSH DS
28005 MOV DS, [CURBUF+2]
28006
28007 ; (DS:BX) = directory entry address
28008
28009 mov ah, [BX]

```

```

28010          ;MOV    AH,[BX+dir_entry.dir_name] ; mov ah,[bx+0]
28011 0000475E 08E4      OR      AH,AH          ; End of directory?
28012 00004760 7461      JZ       short FREE
28013
28014          ;;;
28015          ; 13/02/2024 (PCDOS 7.1 IBMDOS.COM)
28016 00004762 50        push    ax
28017 00004763 8A470B    mov     al,[bx+0Bh]      ; [BX+dir_entry.dir_attr]
28018 00004766 E8A103    call    check_longname
28019 00004769 58        pop     ax
28020 0000476A 7447      jz       short NEXTENT2
28021          ;;;
28022
28023          ;hkn; SS override
28024 0000476C 363A26[7F05] CMP     AH,[SS:DELALL]      ; Free entry?
28025 00004771 7450      JZ       short FREE
28026          ;test  byte [bx+0Bh],8
28027 00004773 F6470B08    TEST    byte [BX+dir_entry.dir_attr],attr_volume_id
28028          ; Volume ID file?
28029 00004777 7405      JZ       short CHKFNAM      ; NO
28030
28031          ;hkn; SS override
28032 00004779 36FE06[7B05] INC     BYTE [SS:VOLID]
28033          CHKFNAM:
28034          ; Context ES
28035 0000477E 8CD6      MOV     SI,SS
28036 00004780 8EC6      MOV     ES,SI
28037 00004782 89DE      MOV     SI,BX
28038
28039          ;hkn; NAME1 is in DOSDATA
28040 00004784 BF[4B05]    MOV     DI,NAME1
28041          ;;;; 7/29/86
28042
28043          ;hkn; SS override for NAME1
28044          ;CMP     BYTE [SS:NAME1],0E5H ; special char check
28045          ;JNZ     short NO_E5
28046          ;MOV     BYTE [SS:NAME1],05H
28047          ; 22/09/2023
28048 00004787 26803DE5    cmp     byte [es:di],0E5h
28049 0000478B 7504      jnz     short NO_E5
28050 0000478D 26C60505    mov     byte [es:di],05h
28051          NO_E5:
28052          ;;;; 7/29/86
28053 00004791 E88100      CALL    MetaCompare
28054 00004794 7449      JZ       short FOUND
28055 00004796 1F        POP     DS
28056
28057          ;entry NEXTENT
28058          NEXTENT:
28059 00004797 C42E[8A05]    LES     BP,[THISDPB]
28060 0000479B E88600      CALL    NEXTENTRY
28061 0000479E 73B1      JNC     short SRCH
28062          ;JMP     SHORT SETESRET
28063          ; 24/01/2024
28064          SETESRET:
28065 000047A0 16        PUSH    SS
28066 000047A1 07        POP     ES
28067
28068          ;;;
28069          ; 13/02/2024 (PCDOS 7.1 IBMDOS.COM)
28070 000047A2 FF36[DA05]    push    word [ENTLAST]
28071 000047A6 8F06[B50A]    pop     word [ENTLAST_PREV] ; previous ENTLAST
28072 000047AA 7306      jnc     short SETESRETN
28073 000047AC C706[A50A]0000    mov     word [LNE_COUNT],0 ; reset long name entry count
28074          SETESRETN:
28075          ;;;
28076
28077 000047B2 C3        retn
28078
28079          ;;;
28080          ; 13/02/2024 (PCDOS 7.1 IBMDOS.COM)
28081          NEXTENT2:
28082 000047B3 1F        pop     ds
28083 000047B4 FF06[A50A]    inc     word [LNE_COUNT] ; Long Name entry count
28084          NEXTENT3:
28085 000047B8 C42E[8A05]    les     bp,[THISDPB]
28086 000047BC E86500      call    NEXTENTRY
28087 000047BF 7396      jnc     short SRCH2
28088 000047C1 EBDD      jmp     short SETESRET
28089          ;;;
28090
28091          FREE:
28092 000047C3 1F        POP     DS
28093 000047C4 8B0E[4803]    MOV     CX,[LASTENT]
28094 000047C8 3B0E[D805]    CMP     CX,[ENTFREE]
28095 000047CC 7304      JAE     short TSTALL
28096 000047CE 890E[D805]    MOV     [ENTFREE],CX
28097          TSTALL:
28098 000047D2 3A26[7F05]    CMP     AH,[DELALL]      ; At end of directory?
28099          NEXTENTJ:
28100          ;je     short NEXTENT      ; No - continue search
28101          ;;;
28102          ; 13/02/2024
28103 000047D6 74E0      je     short NEXTENT3      ; No - continue search
28104          ;;;
28105 000047D8 890E[DA05]    MOV     [ENTLAST],CX
28106 000047DC F9        STC
28107 000047DD EBC1      JMP     SHORT SETESRET
28108
28109          FOUND:
28110          ; We have a file with a matching name. We must now consider the attributes:
28111          ; ATTRIB Action
28112          ; -----
28113          ; Volume_ID Is Volume_ID in test?
28114          ; Otherwise If no create then Is ATTRIB+extra superset of test?
28115          ; If create then Is ATTRIB equal to test?
28116
28117 000047DF 8A2C      MOV     CH,[SI]          ; Attributes of file
28118 000047E1 1F        POP     DS
28119 000047E2 8A26[6B05]    MOV     AH,[ATTRIB]      ; Attributes of search
28120          ;and  ah,9Eh
28121 000047E6 80E4DE    AND     AH,~attr_ignore
28122          ;lea  si,[si+15]
28123 000047E9 8D740F    LEA     SI,[SI+dir_entry.dir_first-dir_entry.dir_attr]
28124          ; point to first cluster field
28125          ;test  ch,8
28126 000047EC F6C508    TEST    CH,attr_volume_id ; Volume ID file?
28127 000047EF 7409      JZ       short check_one_volume_id ; Nope check other attributes
28128          ;test  ah,8
28129 000047F1 F6C408    TEST    AH,attr_volume_id ; Can we find Volume ID?
28130          ;JZ     short NEXTENTJ      ; Nope, (not even $FCB_CREATE)
28131          ; 16/12/2022
28132 000047F4 74A1      JZ       short NEXTENT ; 19/05/2019
28133          ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)

```

```

28134      ;JZ      short NEXTENTJ
28135 000047F6 30E4      XOR      AH,AH      ; Set zero flag for $FCB_CREATE
28136 000047F8 EB11      JMP      SHORT RETFF      ; Found volume ID
28137      check_one_volume_id:
28138      ;CMP      ah,8
28139 000047FA 80FC08      CMP      AH,attr_volume_id      ; Looking only for volume ID?
28140      ;JZ      short NEXTENTJ      ; Yes, continue search
28141      ; 16/12/2022
28142 000047FD 7498      je      short NEXTENT ; 19/05/2019
28143      ;JZ      short NEXTENTJ
28144 000047FF E8C105      CALL     MatchAttributes
28145 00004802 7407      JZ      SHORT RETFF
28146 00004804 F606[7E05]FF      TEST     BYTE [CREATING],-1      ; Pass back mismatch if creating
28147      ; 16/12/2022
28148      ;JZ      short NEXTENTJ      ; otherwise continue searching
28149 00004809 748C      jz      short NEXTENT ; 19/05/2019
28150      RETFF:
28151 0000480B C42E[8A05]      LES      BP,[THISDPB]
28152      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28153      ;MOV      AH,[ES:BP+DPB.DRIVE]      ; mov ah,[es:bp+0]
28154      ; 15/12/2022
28155 0000480F 268A6600      MOV      AH,[ES:BP]
28156      ;SETESRET:
28157      ;PUSH     SS
28158      ;POP      ES
28159      ;retn
28160      ; 24/01/2024
28161 00004813 EB8B      jmp      short SETESRET
28162
28163      ;-----
28164      ;
28165      ; Procedure Name : MetaCompare
28166      ;
28167      ; Inputs:
28168      ; DS:SI -> 11 character FCB style name NO '?'
28169      ; Typically this is a directory entry. It MUST be in upper case
28170      ; ES:DI -> 11 character FCB style name with possible '?'
28171      ; Typically this is a FCB or SFT. It MUST be in upper case
28172      ; Function:
28173      ; Compare FCB style names allowing for ? match to any char
28174      ; Outputs:
28175      ; Zero if match else NZ
28176      ; Destroys CX,SI,DI all others preserved
28177      ;-----
28178
28179      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28180      ; DOSCODE:7D3Fh (MSDOS 5.0, MSDOS.SYS)
28181
28182      MetaCompare:
28183 00004815 B90B00      MOV      CX,11
28184      WILDCRD:
28185 00004818 F3A6      REPE     CMPSB
28186 0000481A 7407      JZ      short MetaRet      ; most of the time we will fail.
28187      CHECK_META:
28188 0000481C 26807DFF3F      CMP      BYTE [ES:DI-1],"?"
28189 00004821 74F5      JZ      short WILDCRD
28190      MetaRet:
28191 00004823 C3      retn      ; Zero set, Match
28192
28193      ;Break      <NEXTENTRY -- STEP THROUGH DIRECTORY>
28194      ;-----
28195      ;
28196      ; Procedure Name : NEXTENTRY
28197      ;
28198      ; Inputs:
28199      ; Same as outputs of GETENTRY, above
28200      ; Function:
28201      ; Update BX, and [LASTENT] for next directory entry.
28202      ; Carry set if no more.
28203      ;-----
28204
28205      NEXTENTRY:
28206      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28207      ; DOSCODE:7D4Eh (MSDOS 5.0, MSDOS.SYS)
28208
28209      ; 19/05/2019 - Retro DOS v4.0
28210      ; DOSCODE:7D88h (MSDOS 6.21, MSDOS.SYS)
28211
28212      ; 27/07/2018 - Retro DOS v3.0
28213      ; IBMDOS.COM (MSDOS 3.3) - Offset 4671h
28214      ; 15/03/2018 - Retro DOS v2.0
28215
28216      ; 14/02/2024 - Retro DOS v5.0
28217      ; DOSCODE:8884h (PCDOS 7.1, IBMDOS.COM)
28218
28219 00004824 A1[4803]      MOV      AX,[LASTENT]
28220 00004827 3B06[DA05]      CMP      AX,[ENTLAST]
28221 0000482B 741C      JZ      short NONE
28222 0000482D 40      INC      AX
28223      ;ADD      BX,32
28224 0000482E 8D5F20      LEA      BX,[BX+32]
28225 00004831 39D3      CMP      BX,DX
28226      ; 21/11/2022 - MSDOS 5.0 MSDOS.SYS (DOSCODE:7D5Dh)
28227      ;JB      short HAVIT ; MSDOS 6.0 src (dir.asm)
28228      ; 16/12/2022
28229 00004833 7540      jne      short HAVIT ; MSDOS 6.21 (DOSCODE:7D97h)
28230
28231      ;;;
28232      ; 14/02/2024 (PCDOS 7.1 IBMDOS.COM)
28233 00004835 833E[C205]00      cmp      word [DIRSTART],0
28234 0000483A 750F      jnz      short nextentry_cont
28235 0000483C 833E[DC0A]00      cmp      word [DIRSTART_HW],0
28236 00004841 7508      jnz      short nextentry_cont
28237      ;cmp      ax,[es:bp+9]
28238 00004843 263B4609      cmp      ax,[es:bp+DPB.ROOT_ENTRIES] ; Number of root dir entries
28239      ;jnb      short NONE
28240      ;jmp      short GETENT
28241 00004847 7255      jnb      short GETENT
28242      NONE:      ; 14/02/2024
28243 00004849 F9      stc
28244 0000484A C3      retn
28245
28246      nextentry_cont:
28247      ;;;
28248
28249 0000484B 8A1E[7305]      MOV      BL,[SECCLUSPOS]
28250 0000484F FEC3      INC      BL
28251 00004851 3A1E[7705]      CMP      BL,[CLUSFAC]
28252 00004855 7223      JB      short SAMECLUS
28253      ;;;
28254      ; 14/02/2024 (PCDOS 7.1 IBMDOS.COM)
28255 00004857 8B1E[E00A]      mov      bx,[NXTCLUSNUM_HW]
28256 0000485B 891E[E80A]      mov      [CLUSTNUM_HW],bx
28257      ;;;

```

```

28258 0000485F 8B1E[DC05]      MOV     BX,[NXTCLUSNUM]
28259 00004863 E84D1A      call    ISEOF
28260 00004866 73E1      JAE     short NONE
28261      ;;;
28262      ; 14/02/2024
28263 00004868 833E[E80A]00      cmp     word [CLUSTNUM_HW],0
28264 0000486D 752F      jnz     short GETENT
28265      ;;;
28266      ; 23/07/2019
28267 0000486F 83FB02      CMP     BX,2
28268      ;JB     short NONE
28269      ;JMP     short GETENT
28270      ; 16/12/2022
28271 00004872 732A      jnb     short GETENT
28272      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28273      ;JB     short NONE
28274      ;JMP     short GETENT
28275      ; 14/02/2024 - Retro DOS v5.0
28276      ;NONE:
28277      ;STC
28278 00004874 C3      retn
28279      HAVIT:
28280 00004875 A3[4803]      MOV     [LASTENT],AX
28281 00004878 F8      CLC
28282      nextentry_retn:
28283 00004879 C3      retn
28284
28285      SAMECLUS:
28286 0000487A 881E[7305]      MOV     [SECCLUSPOS],BL
28287 0000487E A3[4803]      MOV     [LASTENT],AX
28288 00004881 1E      PUSH    DS
28289 00004882 C53E[E205]      LDS     DI,[CURBUF]
28290      ; 19/05/2019
28291      ; MSDOS 6.0
28292      ;;mov dx,[di+8]
28293      ; 23/09/2023
28294      ;MOV     DX,[DI+BUFFINFO.buf_sector+2] ;AN000; >32mb
28295      ;hkn; SS override
28296      ;MOV     [SS:HIGH_SECTOR],DX ;AN000; >32mb
28297
28298      ; 14/02/2024
28299      %if 0
28300      ; 23/09/2023
28301      mov     si,[di+BUFFINFO.buf_sector+2]
28302
28303      ;mov dx,[di+6]
28304      MOV     DX,[DI+BUFFINFO.buf_sector] ;AN000; >32mb
28305
28306      ;inc dx ; MSDOS 3.3
28307      ; MSDOS 6.0
28308      ;ADD     DX,1
28309      ;ADC     word [SS:HIGH_SECTOR],0 ;AN000; >32mb
28310      ; 23/09/2023
28311      inc     dx
28312      jnz     short nextexntry_fc
28313      inc     si
28314      ;inc word [SS:HIGH_SECTOR]
28315      nextexntry_fc:
28316      ; 23/09/2023
28317      mov     [SS:HIGH_SECTOR],si
28318      ; MSDOS 3.3 & MSDOS 6.0
28319      POP     DS
28320      %else
28321      ; 14/02/2024 - Retro DOS v5.0
28322 00004886 C55506      lds     dx,[di+BUFFINFO.buf_sector]
28323 00004889 8CDE      mov     si,ds
28324 0000488B 1F      pop     ds
28325 0000488C 42      inc     dx
28326 0000488D 7501      jnz     short nextexntry_fc
28327 0000488F 46      inc     si
28328      nextexntry_fc:
28329 00004890 8936[0706]      mov     [HIGH_SECTOR],si
28330      %endif
28331
28332 00004894 E8DEF6      call    FIRSTCLUSTER
28333 00004897 31DB      XOR     BX,BX
28334 00004899 EB21      JMP     short SETENTRY
28335
28336      ;-----
28337      ;
28338      ; Procedure Name : GETENTRY
28339      ;
28340      ; Inputs:
28341      ; [LASTENT] has directory entry
28342      ; ES:BP points to drive parameters
28343      ; [DIRSEC],[CLUSNUM],[CLUSFAC],[ENTLAST] set for DIR involved
28344      ; Function:
28345      ; Locates directory entry in preparation for search
28346      ; GETENT provides entry for passing desired entry in AX
28347      ; Outputs:
28348      ; [CURBUF+2]:BX = Pointer to next directory entry in CURBUF
28349      ; [CURBUF+2]:DX = Pointer to first byte after end of CURBUF
28350      ; [LASTENT] = New directory entry number
28351      ; [NXTCLUSNUM],[SECCLUSPOS] set via DIRREAD
28352      ; Carry set if error (currently user FAILED to I 24)
28353      ;-----
28354
28355      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28356      GETENTRY:
28357      ; 27/07/2018 - Retro DOS v3.0
28358 0000489B A1[4803]      MOV     AX,[LASTENT]
28359
28360      ;entry GETENT
28361      GETENT:
28362 0000489E A3[4803]      MOV     [LASTENT],AX
28363
28364      ;
28365      ; Convert the entry number in AX into a byte offset from the beginning of the
28366      ; directory.
28367      ;
28367 000048A1 B105      mov     cl,5 ; shift left by 5 = mult by 32
28368 000048A3 D3C0      rol     ax,cl ; keep high order bits
28369 000048A5 89C2      mov     dx,ax
28370      ; 19/05/2019 - Retro DOS v4.0
28371      ;and ax,0FFE0h
28372      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28373      ;and ax,~(32-1) ; mask off high order bits
28374      ; 16/12/2022
28375      and     al,0E0h ; ~31
28376 000048A9 83E21F      and     dx,1Fh
28377      ;and dx,32-1 ; mask off low order bits
28378
28379      ;
28379      ; DX:AX contain the byte offset of the required directory entry from the
28380      ; beginning of the directory. Convert this to a sector number. Round the
28381      ; sector size down to a multiple of 32.

```

```

28382 ;
28383 ;mov bx,[es:bp+2]
28384 MOV BX,[ES:BP+DPB.SECTOR_SIZE]
28385 and bl,0E0h
28386 ;AND BL,255-31 ; Must be multiple of 32
28387 DIV BX
28388 ; 14/02/2024
28389 ;MOV BX,DX ; Position within sector
28390 ; NOTE: This BX value is not used in DIRREAD
28391 ; Erdogan Tan - 14/02/2024
28392 ;PUSH BX
28393 push dx
28394 ;
28395 call DIRREAD
28396 POP BX
28397 ;retc
28398 jc short nextentry_retn
28399 SETENTRY:
28400 MOV DX,[CURBUF]
28401 ;;;add dx,16 ; MSDOS 3.3
28402 ;;;add dx,20 ; MSDOS 6.0
28403 ;add dx,24 ; PCDOS 7.1 ; 14/02/2024
28404 ADD DX,BUFINSIZ
28405 ADD BX,DX
28406 ;add dx,[es:bp+2]
28407 ADD DX,[ES:BP+DPB.SECTOR_SIZE] ; Always clears carry
28408 ; 29/12/2022
28409 ; MSDOS 6.21 MSDOS.SYS contains a 'CLC' here, at DOSCODE:7E15h
28410 ; 27/06/2024
28411 ; PCDOS 7.1 IBMDOS.COM contains 'CLC' here, at DOSCODE:8931h
28412 clc
28413 retn
28414
28415 ;-----
28416 ; 23/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28417 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8933h
28418
28419 sft_fcb_table:
28420 times 20 db 0
28421 000048CB 00<rep 14h>
28422 sftfcb.cluster:
28423 dd 0
28424 sftfcb.direntry:
28425 dw 0
28426 sftfcb_entry_size equ $ - sftfcb.cluster ; = 6
28427
28428 000048E5 00<rep 72h>
28429 times 114 db 0 ; 6*20 = 120 ; entry size = 6 bytes
28430
28431 SRCH_CLUSTER:
28432 dw 0
28433 SRCH_CLUSTER_HW:
28434 dw 0
28435
28436 ;-----
28437 ;Break <SETDIRSRCH SETROOTSRCH -- Set Search environments>
28438 ;-----
28439 ;
28440 ; Procedure Name : SETDIRSRCH,SETROOTSRCH
28441 ;
28442 ; Inputs:
28443 ; BX cluster number of start of directory
28444 ; ES:BP Points to DPB
28445 ; DI next cluster number from fastopen extended info. DOS 3.3 only
28446 ; Function:
28447 ; Set up a directory search
28448 ; Outputs:
28449 ; [DIRSTART] = BX
28450 ; [CLUSFAC],[CLUNUM],[SECCLUSPOS],[DIRSEC] set
28451 ; Carry set if error (currently user FAILED to I 24)
28452 ; destroys AX,DX,BX
28453 ;-----
28454
28455 ; 23/01/2024 - Retro DOS v5.0
28456 %if 0
28457 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28458 SETDIRSRCH:
28459 OR BX,BX
28460 JZ short SETROOTSRCH
28461 MOV [DIRSTART],BX
28462 ;mov al,[es:bp+4]
28463 MOV AL,[ES:BP+DPB.CLUSTER_MASK]
28464 INC AL
28465 MOV [CLUSFAC],AL
28466
28467 ; DOS 3.3 for FastOpen F.C. 6/12/86
28468 ;SAVE <SI>
28469 push si
28470 ;test byte [FastOpenFlg],2
28471 TEST byte [FastOpenFlg],Lookup_Success
28472 JNZ short UNP_OK
28473
28474 ; DOS 3.3 for FastOpen F.C. 6/12/86
28475 ;invoke UNPACK
28476 call UNPACK
28477 JNC short UNP_OK
28478 ;RESTORE <SI>
28479 pop si
28480 ;return
28481 retn
28482
28483 UNP_OK:
28484 MOV [CLUNUM],DI
28485 MOV DX,BX
28486 XOR BL,BL
28487 MOV [SECCLUSPOS],BL
28488 ;invoke FIGREC
28489 call FIGREC
28490 ;RESTORE <SI>
28491 pop si
28492
28493 ; 19/05/2019 - Retro DOS v4.0
28494
28495 ; MSDOS 6.0
28496 ;PUSH DX ;AN000; >32mb
28497 ;MOV DX,[HIGH_SECTOR] ;AN000; >32mb
28498 ;MOV [DIRSEC+2],DX ;AN000; >32mb
28499 ;POP DX ;AN000; >32mb
28500
28501 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28502 ;push dx
28503 ;mov dx,[HIGH_SECTOR]
28504 ;mov [DIRSEC+2],dx
28505 ;pop dx

```



```

28506      ;MOV     [DIRSEC],dx
28507      ; 16/12/2022
28508      mov     ax,[HIGH_SECTOR]
28509      mov     [DIRSEC+2],ax
28510      MOV     [DIRSEC],dx
28511
28512      ; 16/12/2022
28513      ; cf=0 (at the return of FIGREC)
28514      ;CLC
28515      retn
28516
28517      ;entry SETROOTSRCH
28518      SETROOTSRCH:
28519      XOR     ax,ax
28520      MOV     [DIRSTART],ax
28521      ; 22/09/2023
28522      mov     [DIRSEC+2],ax ; 0
28523      MOV     [SECCLUSPOS],AL
28524      DEC     ax
28525      MOV     [CLUSNUM],ax
28526      ;mov     ax,[es:bp+0Bh]
28527      MOV     ax,[ES:BP+DPB.FIRST_SECTOR]
28528      ; 19/05/2019
28529      ;;mov     dx,[es:bp+10h] ; MSDOS 3.3
28530      ;;mov     dx,[es:bp+11h] ; MSDOS 6.0
28531      MOV     dx,[ES:BP+DPB.DIR_SECTOR]
28532      SUB     ax,dx
28533      MOV     [CLUSFAC],AL
28534      MOV     [DIRSEC],dx ;F.C. >32mb
28535      ; 22/09/2023
28536      ; MSDOS 6.0
28537      ;MOV     WORD [DIRSEC+2],0 ;F.C. >32mb
28538      CLC
28539      retn
28540      %else
28541      ;;;
28542      ; 23/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28543      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:89C3h
28544      ;;;
28545      SETDIRSRCH:
28546      mov     ax,[ROOTCLUST_HW]
28547      ;cmp     word [ROOTCLUST_HW],0
28548      and     ax,ax
28549      jnz     short SETDIRSRCH_FAT32
28550
28551      or     bx,bx
28552      ;jz     short SETROOTSRCH
28553      jz     short SETROOTSRCH2 ; ax = 0
28554      SETDIRSRCH_FAT32:
28555      ;mov     ax,[ROOTCLUST_HW]
28556      mov     [DIRSTART_HW],ax
28557      mov     [CLUSTNUM_HW],ax
28558
28559      mov     [DIRSTART],bx
28560      ;mov     al,[es:bp+4]
28561      mov     al,[es:bp+DPB.CLUSTER_MASK]
28562      ;inc     al
28563      inc     ax
28564      mov     [CLUSFAC],al
28565
28566      push     si
28567      ;test    byte [FastOpenFlg],2
28568      test    byte [FastOpenFlg],Lookup_Success
28569      jnz     short UNP_OK
28570
28571      call     UNPACK
28572      jnc     short UNP_OK
28573
28574      pop      si
28575      retn
28576
28577      SETROOTSRCH:
28578      xor     ax,ax ; 0
28579      SETROOTSRCH2:
28580      mov     [ROOTCLUST_HW],ax ;0
28581      ;cmp     word [es:bp+0Fh],0
28582      cmp     [es:bp+DPB.FAT_SIZE],ax ; 0
28583      jnz     short SETROOTSRCH_FAT ; not FAT32
28584      SETROOTSRCH_FAT32:
28585      ;;mov     bx,[es:bp+37h]
28586      mov     bx,[es:bp+DPB.ROOT_CLUSTER+2]
28587      ;mov     [ROOTCLUST_HW],bx
28588      ;;cmp     bx,[es:bp+2Fh]
28589      cmp     bx,[es:bp+DPB.LAST_CLUSTER+2]
28590      ; 27/06/2024 - Retro DOS v5.0
28591      mov     ax,[es:bp+DPB.ROOT_CLUSTER+2]
28592      mov     [ROOTCLUST_HW],ax
28593      cmp     ax,[es:bp+DPB.LAST_CLUSTER+2]
28594      ;mov     bx,[es:bp+35h]
28595      mov     bx,[es:bp+DPB.ROOT_CLUSTER]
28596      jne     short sdsrch_fat32_1
28597      ;cmp     bx,[es:bp+2Dh]
28598      cmp     bx,[es:bp+DPB.LAST_CLUSTER]
28599      sdsrch_fat32_1:
28600      ja     short sdsrch_fat32_3 ; **
28601      ; 27/06/2024
28602      and     ax,ax ; [ROOTCLUST_HW]
28603      ;cmp     word [ROOTCLUST_HW],0
28604      ;jnz     short sdsrch_fat32_2
28605      jnz     short SETDIRSRCH_FAT32
28606      cmp     bx,2
28607      ;jb     short sdsrch_fat32_3
28608      sdsrch_fat32_2:
28609      ;jmp     SETDIRSRCH_FAT32
28610      jnb     short SETDIRSRCH_FAT32
28611
28612      sdsrch_fat32_3:
28613      stc     ; **
28614      ;jmp     short setdirsrch_retn
28615      retn
28616
28617      UNP_OK:
28618      ; CCONTENT_HW:DI = Contents of FAT for given cluster
28619      ; (from UNPACK)
28620      mov     [CLUSNUM],di
28621      mov     dx,[CONTENT_HW]
28622      mov     [CLUSNUM_HW],dx
28623      mov     [cs:SRCH_CLUSTER],bx ; Directory start cluster number
28624      ; for searching/locating (directory entry)
28625      mov     dx,[CLUSTNUM_HW]
28626      mov     [cs:SRCH_CLUSTER_HW],dx
28627
28628      mov     dx,bx
28629      xor     bl,bl

```

```

28630 000049D2 881E[7305]      mov     [SECCLUSPOS],b1
28631
28632      ;CLUSTNUM_HW:DX = Physical cluster number (to FIGREC)
28633
28634 000049D6 E8BA0F      call    FIGREC
28635
28636 000049D9 8B36[0706]      mov     si,[HIGH_SECTOR]
28637 000049DD 8936[C005]      mov     [DIRSEC+2],si
28638 000049E1 5E      pop     si
28639 SETROOTSRCH3:
28640 000049E2 8916[BE05]      mov     [DIRSEC],dx ; *
28641
28642      ;pop     si
28643 000049E6 F8      cld     ; (this may not be needed,
28644      ; FIGREC clears cf except an abnormal sector calc overflow)
28645 000049E7 C3      retn
28646
28647 SETROOTSRCH_FAT:
28648      ;xor     ax,ax
28649 000049E8 A3[C005]      mov     [DIRSEC+2],ax ; 0
28650 000049EB A3[C205]      mov     [DIRSTART],ax ; 0
28651 000049EE A3[DC0A]      mov     [DIRSTART_HW],ax ; 0
28652 000049F1 2EA3[5949]      mov     [cs:SRCH_CLUSTER_HW],ax ; 0
28653 000049F5 40      inc     ax ; search start dir cluster num = 1
28654      ; (root directory)
28655 000049F6 2EA3[5749]      mov     [cs:SRCH_CLUSTER],ax ; 1 ; FAT root directory (<2)
28656 000049FA 48      dec     ax ; 0
28657 000049FB A2[7305]      mov     [SECCLUSPOS],al
28658 000049FE 48      dec     ax ; -1
28659 000049FF A3[BC05]      mov     [CLUSNUM],ax ; 0FFFFh
28660 00004A02 A3[DE0A]      mov     [CLUSNUM_HW],ax ; 0FFFFh
28661
28662      ;mov     ax,[es:bp+0Bh]
28663 00004A05 268B460B      mov     ax,[es:bp+DPB.FIRST_SECTOR]
28664      ;mov     dx,[es:bp+11h]
28665 00004A09 268B5611      mov     dx,[es:bp+DPB.DIR_SECTOR]
28666 00004A0D 29D0      sub     ax,dx
28667 00004A0F A2[7705]      mov     [CLUSFAC],al
28668      ;mov     [DIRSEC],dx ; *
28669      ;cld
28670
28671 ;setdirsrch_retn:
28672      ;retn
28673      jmp     short SETROOTSRCH3
28674      ;;;
28675 %endif
28676
28677 ;-----
28678 ; 23/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28679 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8A90h
28680
28681 ; set SFT number and entry in the new internal table (only for FCB calls)
28682
28683 set_sftfcb_entry:
28684 00004A14 50      push    ax
28685 00004A15 51      push    cx
28686 00004A16 52      push    dx
28687 00004A17 53      push    bx
28688
28689      ;push    bp
28690      ;push    si
28691      ;push    di ; ES:DI = SFT entry
28692
28693 00004A18 E84B00      call    find_sft_entry_number
28694      ; ax = SFT entry index number
28695      ; of the last SFT entry
28696 00004A1B 31DB      xor     bx,bx
28697 00004A1D B91400      mov     cx,20 ; find empty entry (slot) in the table
28698      ; Retro DOS v5.0
28699 00004A20 31D2      xor     dx,dx ; *
28700
28701 set_sftfcb_e_1:
28702      ;cmp     cs:(sftfcb_cluster+2)[bx],0
28703      ;cmp     word [cs:bx+sftfcb_cluster+2],0
28704      ; ; directory (search) starting cluster, hw
28705 00004A22 2E3997[E148]      cmp     [cs:bx+sftfcb_cluster+2],dx ; 0
28706      ;jnz     short set_sftfcb_e_2
28707      ; 27/06/2024
28708      ;jnz     short set_sftfcb_e_3
28709      ;cmp     cs:sftfcb_cluster[bx],
28710      ;cmp     word [cs:bx+sftfcb_cluster],0
28711      ; ; directory (search) starting cluster, lw
28712 00004A29 2E3997[DF48]      cmp     [cs:bx+sftfcb_cluster],dx ; 0
28713
28714 00004A2E 752C      jnz     short set_sftfcb_e_3
28715      ; not empty (sftcb table) entry
28716      ; Retro DOS v5.0
28717      ;push    cx
28718
28719 00004A30 2E8B0E[5749]      mov     cx,[cs:SRCH_CLUSTER]
28720      ; search start dir cluster number
28721      ;mov     cs:sftfcb_cluster[bx],cx
28722 00004A35 2E898F[DF48]      mov     [cs:bx+sftfcb_cluster],cx
28723      ; directory start cluster
28724 00004A3A 2E8B0E[5949]      mov     cx,[cs:SRCH_CLUSTER_HW]
28725
28726      ; PCDOS 7.1 BUG! (This would be '[cs:bx:sftfcb_cluster+2],cx')
28727      ; Erdogan Tan - 23/01/2024
28728      ;mov     cs:sftfcb_cluster[bx],cx
28729      ;mov     [cs:bx:sftfcb_cluster],cx ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8ABFh
28730      ; Retro DOS v5.0
28731 00004A3F 2E898F[E148]      mov     [cs:bx+sftfcb_cluster+2],cx
28732
28733      ;pop     cx
28734
28735      ;mov     dx,[ss:LASTENT] ; LAST (found) entry in the directory
28736      ;mov     cs:sftfcb_direntry[bx],dx
28737      ;mov     [cs:bx+sftfcb_direntry],dx
28738      ; directory entry number
28739 00004A44 368B0E[4803]      mov     cx,[ss:LASTENT]
28740 00004A49 2E898F[E348]      mov     [cs:bx+sftfcb_direntry],cx
28741
28742 00004A4E 93      xchg     ax,bx
28743      ;xor     dx,dx ; *
28744      ; dx = 0
28745 00004A4F B90600      mov     cx,6 ; sftfcb table has 6 byte entries
28746 00004A52 F7F1      div     cx
28747 00004A54 93      xchg     ax,bx
28748 00004A55 2E8887[CB48]      mov     [cs:bx+sft_fcb_table],al
28749      ; put SFT entry number in SFT-FCB
28750      ; table (offset is FCB index number 0 to 19)
28751 00004A5A EB05      jmp     short set_setfcb_e_4
28752
28753 set_sftfcb_e_3:
28754      ;add     bx,6

```



```

28754 00004A5C 83C306      add     bx,sftfcb_entry_size ; 6
28755 00004A5F E2C1        loop    set_sftfcb_e_1
28756
28757 set_setfcb_e_4:
28758     ;pop     di
28759     ;pop     si
28760     ;pop     bp
28761
28762     pop     bx
28763     pop     dx
28764     pop     cx
28765     pop     ax
28766     retn
28767
28768 ;-----
28769
28770 ; 24/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
28771 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:8AECh
28772
28773 ; 14/02/2024
28774 find_sft_entry_number:
28775     push    es                ; es:di = SFT entry
28776     xor     cx,cx
28777     mov     dx,es
28778     mov     es,[cs:DosDSeg]
28779     les     bx,[es:SFT_ADDR] ; address of the first SFT
28780 f_sfte_1:
28781     mov     ax,es
28782     cmp     ax,dx                ; same SFT segment ?
28783     jne     short f_sfte_2      ; no
28784     mov     ax,di
28785     sub     ax,bx                ; ax = entry offset
28786     ;sub     ax,6                ; ax = offset from start of the SFT table
28787     sub     ax,SFT.SFTable
28788     ;mov     bx,59
28789     mov     bx,SF_ENTRY.size ; (SFT entry size)
28790     xor     dx,dx
28791     div     bx                ; ax = SFT entry index in the table
28792     add     ax,cx                ; ax = SFT index number (for requested SFT entry)
28793     pop     es
28794     retn
28795 f_sfte_2:
28796     add     cx,[es:bx+4]        ; SFT.SFCount ; number of entries in the table
28797     les     bx,[es:bx]          ; SFT.SFLink
28798     cmp     bx,0FFFFh          ; -1 ; the last SFT
28799     jnz     short f_sfte_1
28800     stc                     ; (not found)
28801     pop     es
28802     retn
28803
28804 ;-----
28805
28806 ; 24/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
28807 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:8B22h
28808
28809 ; 14/02/2024
28810 int_2Fh_1230h:
28811     call    find_sft_entry_number
28812     jc     short find_sfte_i_error
28813     push    es                ; ax = SFT entry index number (of the requested SFT entry)
28814     push    di
28815     push    cs
28816     pop     es
28817     mov     cx,20                ; statically allocated with 20 entries,
28818     ; and used only for FCB calls
28819     mov     di,sft_fcb_table ; new file system (internal) table
28820     ; only for fcb calls
28821 scan_next_sftfcb:
28822     repne scasb
28823     stc
28824     jnz     short sfte_i_notfound ; not found (cf=1)
28825     lea     bx,[di-1]          ; offset of the entry in the table
28826     sub     bx,sft_fcb_table ; index into new file system table
28827     mov     dx,bx
28828     add     bx,bx                ; 2*bx
28829     add     bx,dx                ; 3*bx
28830     add     bx,bx                ; bx = 6*bx ; entry size = 6 bytes
28831     mov     si,[es:bx+sftfcb.cluster+2] ; dir start cluster number, hw
28832     mov     dx,[es:bx+sftfcb.direntry] ; directory entry number
28833     push    cx
28834     mov     cx,[es:bx+sftfcb.cluster] ; dir start cluster number, lw
28835     or     cx,cx
28836     jnz     short sfte_i_found
28837     or     si,si
28838     jnz     short sfte_i_found
28839     pop     cx
28840     or     cx,cx
28841     jnz     short scan_next_sftfcb
28842     stc                     ; not found (cf=1)
28843     jmp     short sfte_i_notfound
28844
28845 sfte_i_found:
28846     pop     ax                ; clear stack
28847     cld                     ; found (cf=0)
28848
28849 sfte_i_notfound:
28850     pop     di
28851     pop     es
28852 find_sfte_i_error:
28853     mov     ax,0
28854     retn
28855
28856 ;-----
28857
28858 ; 24/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
28859 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:8B6Dh
28860
28861 ; 14/02/2024
28862 SFT_FREE:
28863     push    ax
28864     push    cx
28865     push    dx
28866     push    bx
28867     ;push    bp                ; 14/02/2024 - Retro DOS v5.0
28868     push    si
28869     push    di
28870     mov     word [es:di],0 ; [ES:DI+SF_ENTRY.sf_ref_Count]
28871     call    int_2Fh_1230h
28872     jc     short sftf_1
28873     mov     word [cs:bx+sftfcb.cluster+2],0 ; clear directory start cluster
28874     ; in the new (sftfcb) table
28875     mov     word [cs:bx+sftfcb.cluster],0 ; ((empty entry))
28876 sftf_1:
28877     ;pop     di
28878     ;pop     si

```

```

28878             ;pop     bp
28879 00004B05 5B     pop     bx
28880 00004B06 5A     pop     dx
28881 00004B07 59     pop     cx
28882 00004B08 58     pop     ax
28883 00004B09 C3     retn
28884
28885             ; ===== S U B R O U T I N E =====
28886
28887             ; 13/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28888             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8B94h
28889
28890 check_longname:
28891 00004B0A 240F     and     al,0Fh
28892 00004B0C 3C0F     cmp     al,0Fh
28893 00004B0E C3     retn
28894
28895             ;-----
28896
28897             ;=====
28898             ; DIR2.ASM, MSDOS 6.0, 1991
28899             ;=====
28900             ; 27/07/2018 - Retro DOS v3.0
28901             ; 19/05/2019 - Retro DOS v4.0
28902             ; 14/02/2024 - Retro DOS v5.0
28903
28904             ; TITLE DIR2 - Directory and path cracking
28905             ; NAME Dir2
28906
28907 ;Break <GETPATH -- PARSE A WFP>
28908
28909             ;-----
28910
28911             ; Procedure Name : GETPATH
28912
28913             ; Inputs:
28914             ; [WFP_START] Points to WFP string ("d:\" must be first 3 chars, NUL
28915             ; terminated; d:/ (note forward slash) indicates a real device).
28916             ; [CURR_DIR_END] Points to end of Current dir part of string
28917             ; (= -1 if current dir not involved, else
28918             ; Points to first char after last "/" of current dir part)
28919             ; [THISCDs] Points to CDS being used
28920             ; [SATTRIB] Is attribute of search, determines what files can be found
28921             ; [NoSetDir] set
28922             ; [THISDPB] set to DPB if disk otherwise garbage.
28923             ; Function:
28924             ; Crack the path
28925             ; Outputs:
28926             ; Sets EXTERR_LOCUS = errLOC_Disk if disk file
28927             ; Sets EXTERR_LOCUS = errLOC_Unk if char device
28928             ; ID1 field of [THISCDs] updated appropriately
28929             ; [ATTRIB] = [SATTRIB]
28930             ; ES:BP Points to DPB
28931             ; Carry set if bad path
28932             ; SI Points to path element causing failure
28933             ; Zero set
28934             ; [DIRSTART],[DIRSEC],[CLUSNUM], and [CLUSFAC] are set up to
28935             ; start a search on the last directory
28936             ; CL is zero if there is a bad name in the path
28937             ; CL is non-zero if the name was simply not found
28938             ; [ENTFREE] may have free spot in directory
28939             ; [NAME1] is the name.
28940             ; CL = 81H if '*'s or '?' in NAME1, 80H otherwise
28941             ; Zero reset
28942             ; File in middle of path or bad name in path or attribute mismatch
28943             ; or path too long or malformed path
28944             ; ELSE
28945             ; [CurBuf] = -1 if root directory
28946             ; [CURBUF] contains directory record with match
28947             ; [CURBUF+2]:BX Points into [CURBUF] to start of entry
28948             ; [CURBUF+2]:SI Points into [CURBUF] to dir_first field for entry
28949             ; AH = device ID
28950             ; bit 7 of AH set if device SI and BX
28951             ; will point DOSGROUP relative The firclus
28952             ; field of the device entry contains the device pointer
28953             ; [NAME1] Has name looked for
28954             ; If last element is a directory zero is set and:
28955             ; [DIRSTART],[SECCLUSPOS],[DIRSEC],[CLUSNUM], and [CLUSFAC]
28956             ; are set up to start a search on it.
28957             ; unless [NoSetDir] is non zero in which case the return is
28958             ; like that for a file (except for zero flag)
28959             ; If last element is a file zero is reset
28960             ; [DIRSEC],[CLUSNUM],[CLUSFAC],[NXTCLUSNUM],[SECCLUSPOS],
28961             ; [LASTENT],[ENTLAST] are set to continue search of last
28962             ; directory for further matches on NAME1 via the NEXTENT
28963             ; entry point in FindEntry (or GETENT entry in GETENTRY in
28964             ; which case [NXTCLUSNUM] and [SECCLUSPOS] need not be valid)
28965             ; DS preserved, Others destroyed
28966             ;-----
28967
28968 ;hkn; called from delete.asm, finfo.asm, mknnode.asm and rename.asm.
28969 ;hkn; DS already set up at this point.
28970
28971             ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28972
28973             ; 15/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28974             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8B99h
28975
28976 GETPATH:
28977 ;mov     word [CREATING],0E500h
28978 MOV     WORD [CREATING],DIRFREE*256+0 ; Not Creating, not DEL *.*
28979
28980             ; Same as GetPath only CREATING and DELALL already set
28981
28982             ;entry GetPathNoSet
28983 GetPathNoSet:
28984 ;;mov     byte [EXTERR_LOCUS],2
28985 ;MOV     byte [EXTERR_LOCUS],errLOC_Disk
28986 ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
28987 call     set_exerr_locus_disk
28988 ;
28989 MOV     word [CURBUF],-1 ; initial setting
28990
28991             ; See if the input indicates a device that has already been detected. If so,
28992             ; go build the guy quickly. Otherwise, let findpath find the device.
28993 00004B1E 8B3E[B205] MOV     DI,[WFP_START] ; point to the beginning of the name
28994             ; cmp     word [DI+1],5C3Ah
28995             ; CMP     WORD [DI+1],'\ ' << 8 + ':'
28996 00004B22 817D013A5C cmp     word [DI+1],':\'
28997 00004B27 7435     JZ     short CrackIt
28998
28999             ; Let ChkDev find it in the device list
29000
29001 00004B29 83C703     ADD     DI,3

```

```

29002             ; 18/08/2018
29003             ;MOV     SI,DI             ; let CHKDEV see the original name
29004             ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
29005             ; 16/12/2022
29006             ;mov     si,di ; not required ! (it is written in CHKDEV proc already!)
29007 00004B2C E8B200 CALL     CHKDEV
29008 00004B2F 722B JC      short InternalError
29009
29010 Build_devJ:
29011 00004B31 A0[6D05] MOV     AL,[ATTRIB]
29012 00004B34 A2[6B05] MOV     [ATTRIB],AL
29013             ; 15/02/2024
29014             ;;mov     byte [EXTERR_LOCUS],1
29015             ;MOV     byte [EXTERR_LOCUS],errLOC_Unk ; In the particular case of
29016             ; "finding" a char device
29017             ; set LOCUS to Unknown. This makes
29018             ; certain idiotic problems reported
29019             ; by a certain 3 letter OEM go away.
29020             ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29021 00004B37 E8A3C7 call     set_exerr_locus_unk
29022
29023             ; Take name in name1 and pack it back into where wfp_start points. This
29024             ; guarantees wfp_start pointing to a canonical representation of a device.
29025             ; We are allowed to do this as GetPath is *ALWAYS* called before entering a
29026             ; wfp into the share set.
29027             ;
29028             ; We copy chars from name1 to wfp_start remembering the position of the last
29029             ; non-space seen +1. This position is kept in DX.
29030
29031             ;hkn; SS is DOSDATA
29032 00004B3A 16 push    ss
29033 00004B3B 07 pop     es
29034
29035             ;hkn; NAME1 is in DOSDATA
29036 00004B3C BE[4B05] mov     si,NAME1
29037 00004B3F 8B3E[B205] mov     di,[WFP_START]
29038 00004B43 89FA mov     dx,di
29039 00004B45 B90800 mov     cx,8             ; 8 chars in device name
29040
29041 00004B48 AC MoveLoop: lodsb
29042 00004B49 AA stosb
29043 00004B4A 3C20 cmp     al," "
29044 00004B4C 7402 jz      short NoSave
29045
29046 00004B4E 89FA mov     dx,di
29047 NoSave: loop MoveLoop
29048 00004B50 E2F6
29049
29050             ; DX is the position of the last seen non-space + 1. We terminate the name
29051             ; at this point.
29052
29053 00004B52 89D7 mov     di,dx
29054             ;mov     byte [di],0             ; end of string
29055             ; 15/02/2024
29056 00004B54 880D mov     [di],cl ; 0
29057 00004B56 E8D502 call    Build_device_ent ; Clears carry sets zero
29058 00004B59 FEC0 inc     al             ; reset zero
29059 00004B5B C3 retn
29060
29061 InternalError:
29062 InternalError_loop:
29063 00004B5C EBFE JMP     short InternalError_loop ; freeze
29064
29065             ; Start off at the correct spot. Optimize if the current dir part is valid.
29066
29067 CrackIt:
29068             ; 15/02/2024
29069 %if 0
29070 MOV     SI,[CURR_DIR_END] ; get current directory pointer
29071 CMP     SI,-1             ; valid?
29072 JNZ     short LOOK_SING   ; Yes, use it.
29073 LEA     SI,[DI+3]         ; skip D:\.
29074 LOOK_SING:
29075 %endif
29076             ;mov     byte [ATTRIB],16h
29077 00004B5E C606[6B05]16 MOV     byte [ATTRIB],attr_directory+attr_system+attr_hidden
29078                                     ; Attributes to search through Dirs
29079 00004B63 C43E[A205] LES     DI,[THISCDS]
29080 00004B67 B8FFFF MOV     AX,-1 ; 0FFFFh
29081             ;;;
29082             ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29083             ;mov     bx,[es:di+4Bh]
29084 00004B6A 268B5D4B mov     bx,[es:di+curdir.ID+2]
29085 00004B6E 891E[EE0A] mov     [ROOTCLUST_HW],bx
29086             ;;;
29087             ;mov     bx,[es:di+73]
29088 00004B72 268B5D49 MOV     BX,[ES:DI+curdir.ID]
29089 00004B76 8B36[B605] MOV     SI,[CURR_DIR_END]
29090
29091             ; AX = -1
29092             ; BX = cluster number of current directory. This number is -1 if the media
29093             ; has been uncertainly changed.
29094             ; SI = offset in DOSGroup into path to end of current directory text. This
29095             ; may be -1 if no current directory part has been used.
29096
29097 00004B7A 39C6 CMP     SI,AX             ; if Current directory is not part
29098 00004B7C 7449 JZ      short NO_CURR_D             ; then we must crack from root
29099             ;;;
29100             ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29101 00004B7E 3906[EE0A] cmp     [ROOTCLUST_HW],ax ; 0FFFFh
29102 00004B82 7504 jnz     short CrackIt2
29103             ;;;
29104 00004B84 39C3 CMP     BX,AX             ; is the current directory cluster valid
29105
29106             ; DOS 3.3 6/25/86
29107 00004B86 743F JZ      short NO_CURR_D             ; no, crack from the root
29108
29109 CrackIt2: ; 15/02/2024 - Retro DOS v5.0
29110
29111             ;test     byte [FastOpenFlg],1
29112 00004B88 F606[4611]01 TEST     byte [FastOpenFlg],FastOpen_Set ; for fastopen ?
29113 00004B8D 7445 JZ      short GOT_SEARCH_CLUSTER ; no
29114 00004B8F 06 PUSH     ES             ; save registers
29115 00004B90 57 PUSH     DI
29116 00004B91 51 PUSH     CX
29117 00004B92 FF74FF PUSH     word [SI-1] ; save \ and 1st char of next element
29118 00004B95 56 PUSH     SI
29119 00004B96 53 PUSH     BX
29120             ;;; 15/02/2024
29121 00004B97 FF36[EE0A] push    word [ROOTCLUST_HW]
29122             ;;;
29123 00004B9B C644FF00 MOV     BYTE [SI-1],0 ; call fastopen to look up cur dir info
29124 00004B9F 8B36[B205] MOV     SI,[WFP_START]
29125

```

```

29126 ;hkn; FastOpenTable, Dir_Info_Buff & FastOpen_Ext_Info are in DOSDATA
29127 00004BA3 BB[3C11]      MOV     BX, FastOpenTable
29128 00004BA6 BF[7C0D]      MOV     DI, Dir_Info_Buff
29129 00004BA9 B9[4711]      MOV     CX, FastOpen_Ext_Info
29130 ;mov     al, 1
29131 00004BAC B001          MOV     AL, FONC_Look_up
29132 00004BAE 1E            PUSH    DS
29133 00004BAF 07            POP     ES
29134 ;call    far [BX+2]
29135 00004BB0 FF5F02        CALL    far [BX+fastopen_entry.name_caching]
29136 00004BB3 7203          JC      short GO_Chk_end1 ;fastopen not installed, or wrong drive.
29137 ; Go to Got_Srch_cluster
29138 ; 29/12/2022
29139 ;CMP     BYTE [SI], 0 ;fastopen has current dir info?
29140 ;JE      short GO_Chk_end ;yes. Go to got_search_cluster
29141 ;stc
29142 ;jmp     short GO_Chk_end ;Go to No_Curr_D
29143
29144 00004BB5 803C01        cmp     byte [si], 1
29145 GO_Chk_end1: ; 29/12/2022
29146 00004BB8 F5            cmc
29147 ; [si] = 0 -> cf = 0
29148 ; [si] > 0 -> cf = 1
29149
29150 ;GO_Chk_end1:
29151 ; 29/12/2022
29152 ;clc
29153
29154 GO_Chk_end: ; restore registers
29155 ;;; 15/02/2024
29156 00004BB9 8F06[EE0A]    pop     word [ROOTCLUST_HW]
29157 ;;;
29158 00004BBD 5B            POP     BX
29159 00004BBE 5E            POP     SI
29160 00004BBF 8F44FF        POP     word [SI-1]
29161 00004BC2 59            POP     CX
29162 00004BC3 5F            POP     DI
29163 00004BC4 07            POP     ES
29164 00004BC5 730D          JNC     short GOT_SEARCH_CLUSTER ; crack based on cur dir
29165
29166 ; DOS 3.3 6/25/86
29167 ;
29168 ; We must crack the path beginning at the root. Advance pointer to beginning
29169 ; of path and go crack from root.
29170
29171 NO_CURR_D:
29172 00004BC7 8B36[B205]    MOV     SI, [WFP_START]
29173 ;LEA     SI, [SI+3] ; skip "d:/"
29174 ; 15/02/2024
29175 00004BCB 83C603        add     si, 3
29176 00004BCE C42E[8A05]    LES     BP, [THISDPB] ; Get ES:BP
29177 00004BD2 EB37          JMP     short ROOTPATH
29178
29179 ; We are able to crack from the current directory part. Go set up for search
29180 ; of specified cluster.
29181
29182 GOT_SEARCH_CLUSTER:
29183 00004BD4 C42E[8A05]    LES     BP, [THISDPB] ; Get ES:BP
29184 00004BD8 E880FD        call    SETDIRSRCH
29185 ;JC      short SETFERR
29186 ;JMP     short FINDPATH
29187 ; 16/12/2022
29188 00004BDB 733E          jnc     short FINDPATH ; 17/08/2018
29189 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
29190 ;JC      short SETFERR
29191 ;JMP     short FINDPATH
29192 SETFERR:
29193 00004BDD 30C9          XOR     CL, CL ; set zero
29194 00004BDF F9            STC
29195 00004BE0 C3            retn
29196
29197 ;-----
29198 ;
29199 ; Procedure Name : ChkDev
29200 ;
29201 ; Check to see if the name at DS:DI is a device. Returns carry set if not a
29202 ; device.
29203 ; Blasts CX, SI, DI, AX, BX
29204 ;-----
29205
29206 CHKDEV:
29207 00004BE1 89FE          MOV     SI, DI
29208 ;MOV     DI, SS
29209 ;MOV     ES, DI
29210 ; 27/06/2024
29211 00004BE3 16            push    ss
29212 00004BE4 07            pop     es
29213
29214 00004BE5 BF[4B05]        MOV     DI, NAME1
29215 00004BE8 B90900        MOV     CX, 9
29216 TESTLOOP:
29217 00004BEB E8A411        call    GETLET
29218
29219 00004BEE 3C2E          CMP     AL, '.'
29220 00004BF0 740E          JZ      short TESTDEVICE
29221 00004BF2 E8F111        call    PATHCHRCMP
29222 00004BF5 7407          JZ      short NOTDEV
29223 00004BF7 08C0          OR      AL, AL
29224 00004BF9 7405          JZ      short TESTDEVICE
29225
29226 00004BFB AA            STOSB
29227 00004BFC E2ED        LOOP   TESTLOOP
29228 NOTDEV:
29229 00004BFE F9            STC
29230 00004BFF C3            retn
29231
29232 TESTDEVICE:
29233 ;ADD     CX, 2
29234 ; 24/09/2023
29235 00004C00 41            inc     CX
29236 00004C01 41            inc     CX
29237 00004C02 B020          MOV     AL, ' '
29238 00004C04 F3AA          REP     STOSB
29239 ;MOV     AX, SS
29240 ;MOV     DS, AX
29241 ; 27/06/2024
29242 00004C06 16            push    ss
29243 00004C07 1F            pop     ds
29244 ;call    DEVNAME
29245 ;retn
29246 ; 18/12/2022
29247 00004C08 E9C501        jmp     DEVNAME
29248
29249 ;Break <ROOTPATH, FINDPATH -- PARSE A PATH>

```

```

29250
29251
29252
29253
29254
29255
29256
29257
29258
29259
29260
29261
29262
29263
29264
29265
29266
29267
29268
29269
29270
29271
29272
29273
29274
29275
29276 00004C0B E879FD
29277
29278 00004C0E 30E4
29279
29280 00004C10 3824
29281 00004C12 7507
29282
29283
29284
29285
29286
29287
29288
29289
29290 00004C14 A0[6D05]
29291 00004C17 A2[6B05]
29292
29293
29294
29295 00004C1A C3
29296
29297
29298
29299
29300
29301
29302
29303
29304
29305
29306
29307
29308
29309
29310
29311
29312
29313
29314
29315
29316
29317
29318
29319
29320
29321
29322
29323
29324
29325
29326
29327
29328
29329
29330
29331
29332
29333
29334
29335
29336
29337
29338
29339
29340
29341
29342
29343
29344
29345
29346
29347
29348
29349
29350
29351
29352
29353
29354
29355
29356
29357
29358
29359
29360
29361
29362 00004C1B 06
29363 00004C1C 56
29364 00004C1D 89F7
29365 00004C1F 8B0E[C205]
29366 00004C23 833E[B605]FF
29367 00004C28 7415
29368 00004C2A 3B3E[B605]
29369 00004C2E 750F
29370 00004C30 C43E[A205]
29371
29372
29373 00004C34 A1[DC0A]

-----
; Procedure Name : ROOTPATH,FINDPATH
;
; Inputs:
;   Same as FINDPATH but,
;   SI Points to asciz string of path which is assumed to start at
;   the root (no leading '/').
; Function:
;   Search from root for path
; Outputs:
;   Same as FINDPATH but:
;   If root directory specified, [CURBUF] and [NAME1] are NOT set, and
;   [NoSetDir] is ignored.
-----

; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:7F47h (MSDOS 5.0, MSDOS.SYS)

; 15/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; PCDOS 7.1 IBMDOS.COM - DOSCODE:8C9Dh

; (MSDOS 6.22 MSDOS.SYS - DOSCODE:7F82h)
; (Windows ME IO.SYS - BIOSCODE:829Eh)

ROOTPATH:
call    SETROOTSRCH
; 24/09/2023
xor     ah,ah
;CMP    BYTE [SI],0
cmp     [si],ah ; 0
jnz     short FINDPATH

;;;
; 15/02/2024
; windows ME IO.SYS - BIOSCODE:82A6h
;mov     word [CURBUF],0FFFFh
;;;

; Root dir specified
MOV     AL,[SATTRIB]
MOV     [ATTRIB],AL
; 24/09/2023
;XOR     AH,AH                ; Sets "device ID" byte, sets zero
;                               ; (dir), clears carry.

ret     n

; Inputs:
; [ATTRIB] Set to get through directories
; [SATTRIB] Set to find last element
; ES:BP Points to DPB
; SI Points to asciz string of path (no leading '/').
; [SECCLUSPOS] = 0
; [DIRSEC] = Phys sec # of first sector of directory
; [CLUSNUM] = Cluster # of next cluster
; [CLUSFAC] = Sectors per cluster
; [NoSetDir] set
; [CURR_DIR_END] Points to end of Current dir part of string
;               (= -1 if current dir not involved, else
;               Points to first char after last "/" of current dir part)
; [THISCDS] Points to CDS being used
; [CREATING] and [DELALL] set
; Function:
;   Parse path name
; Outputs:
;   ID1 field of [THISCDS] updated appropriately
;   [ATTRIB] = [SATTRIB]
;   ES:BP Points to DPB
;   [THISDPB] = ES:BP
;   Carry set if bad path
;   SI Points to path element causing failure
;   Zero set
;   [DIRSTART],[DIRSEC],[CLUSNUM], and [CLUSFAC] are set up to
;   start a search on the last directory
;   CL is zero if there is a bad name in the path
;   CL is non-zero if the name was simply not found
;   [ENTFREE] may have free spot in directory
;   [NAME1] is the name.
;   CL = 81H if '*'s or '?' in NAME1, 80H otherwise
;   Zero reset
;   File in middle of path or bad name in path
;   or path too long or malformed path
ELSE
; [CURBUF] contains directory record with match
; [CURBUF+2]:BX Points into [CURBUF] to start of entry
; [CURBUF+2]:SI Points to fcb_FIRCLUS field for entry
; [NAME1] Has name looked for
AH = device ID
; bit 7 of AH set if device SI and BX
; will point DOSGROUP relative The firclus
; field of the device entry contains the device pointer
; If last element is a directory zero is set and:
; [DIRSTART],[SECCLUSPOS],[DIRSEC],[CLUSNUM], and [CLUSFAC]
; are set up to start a search on it,
; unless [NoSetDir] is non zero in which case the return is
; like that for a file (except for zero flag)
; If last element is a file zero is reset
; [DIRSEC],[CLUSNUM],[CLUSFAC],[NXTCLUSNUM],[SECCLUSPOS],
; [LASTENT],[ENTLAST] are set to continue search of last
; directory for further matches on NAME1 via the NEXTENT
; entry point in FindEntry (or GETENT entry in GETENTRY in
; which case [NXTCLUSNUM] and [SECCLUSPOS] need not be valid)
; Destroys all other registers

; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:7F58h (MSDOS 5.0, MSDOS.SYS)

; 15/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; PCDOS 7.1 IBMDOS.COM - DOSCODE:8CAEh

;entry  FINDPATH
FINDPATH:
PUSH    ES                ; Save ES:BP
PUSH    SI
MOV     DI,SI
MOV     CX,[DIRSTART]     ; Get start clus of dir being searched
CMP     word [CURR_DIR_END],-1
JZ      short NOIDS       ; No current dir part
CMP     DI,[CURR_DIR_END]
JNZ     short NOIDS       ; Not to current dir end yet
LES     DI,[THISCDS]
;;;
; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
mov     ax,[DIRSTART_HW]

```

```

29374          ;mov     [es:di+48h],ax
29375 00004C37 2689454B      mov     [es:di+curdir.ID+2],ax
29376          ;;;
29377          ;mov     [es:di+73],cx
29378 00004C3B 26894D49      MOV     [ES:DI+curdir.ID],CX ; Set current directory cluster
29379 NOIDS:
29380
29381          ; Parse the name off of DS:SI into NAME1. AL = 1 if there was a meta
29382          ; character in the string. CX,DI may be destroyed.
29383          ;
29384          ; invoke NAMETRANS
29385          ; MOV     CL,AL
29386          ;
29387          ; The above is the slow method. The name has *already* been munged by
29388          ; TransPath so no special casing needs to be done. All we do is try to copy
29389          ; the name until ., \ or 0 is hit.
29390
29391          ;MOV     AX,SS
29392          ;MOV     ES,AX
29393          ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29394 00004C3F 16      push    ss
29395 00004C40 07      pop     es
29396
29397          ;hkn; Name1 is in DOSDATA
29398 00004C41 BF[4B05]      MOV     DI,NAME1
29399 00004C44 B82020      MOV     AX,' ' ; 2020h
29400 00004C47 AA      STOSB
29401 00004C48 AB      STOSW
29402 00004C49 AB      STOSW
29403 00004C4A AB      STOSW
29404 00004C4B AB      STOSW
29405 00004C4C AB      STOSW
29406
29407          ;hkn; Name1 is in DOSDATA
29408 00004C4D BF[4B05]      MOV     DI,NAME1
29409 00004C50 30E4      XOR     AH,AH ; bits for CL
29410          GetNam:
29411          ; 19/05/2019 - Retro DOS v4.0
29412          ; INC     CL ; ?*! ; MSDOS 6.0 ; AN000; KK increment valid count
29413
29414          ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
29415          ; 16/12/2022
29416          ; inc     cl ; not required !
29417
29418 00004C52 AC      LODSB
29419 00004C53 3C2E      CMP     AL,'.' ; 2Eh
29420 00004C55 7412      JZ      short _SetExt
29421 00004C57 08C0      OR      AL,AL
29422 00004C59 7424      JZ      short _GetDone
29423 00004C5B 3C5C      CMP     AL,'\' ; 5Ch
29424 00004C5D 7420      JZ      short _GetDone
29425 00004C5F 3C3F      CMP     AL,'?' ; 3Fh
29426 00004C61 7503      JNZ     short StoNam
29427 00004C63 80CC01      OR      AH,1
29428          StoNam:
29429 00004C66 AA      STOSB
29430 00004C67 EBE9      JMP     short GetNam
29431          _SetExt:
29432 00004C69 BF[5305]      MOV     DI,NAME1+8
29433          GetExt:
29434 00004C6C AC      LODSB
29435 00004C6D 08C0      OR      AL,AL
29436 00004C6F 740E      JZ      short _GetDone
29437 00004C71 3C5C      CMP     AL,'\'
29438 00004C73 740A      JZ      short _GetDone
29439 00004C75 3C3F      CMP     AL,'?'
29440 00004C77 7503      JNZ     short StoExt
29441 00004C79 80CC01      OR      AH,1
29442          StoExt:
29443 00004C7C AA      STOSB
29444 00004C7D EBED      JMP     short GetExt
29445          _GetDone:
29446          DEC     SI
29447 00004C80 88E1      MOV     CL,AH ; 0 or 1 ; 29/12/2022
29448 00004C82 80C980      OR      CL,80H
29449 00004C85 5F      POP     DI ; Start of this element
29450 00004C86 07      POP     ES ; Restore ES:BP
29451 00004C87 39FE      CMP     SI,DI
29452 00004C89 7503      JNZ     short check_device
29453 00004C8B E9ED00      JMP     _BADPATH ; NUL parse (two delims most likely)
29454          check_device:
29455 00004C8E 56      PUSH    SI ; Start of next element
29456          ;MOV     AL,[SI]
29457          ; 15/02/2024
29458 00004C8F 08C0      OR      AL,AL
29459          ; 23/09/2023
29460          ; cmp     byte [si],0
29461 00004C91 7508      JNZ     short NOT_LAST
29462
29463          ; for last element of the path switch to the correct search attributes
29464
29465 00004C93 8A3E[6D05]      MOV     BH,[SATTRIB]
29466 00004C97 883E[6B05]      MOV     [ATTRIB],BH
29467
29468          NOT_LAST:
29469
29470          ; check name1 to see if we have a device...
29471
29472 00004C9B 06      PUSH    ES ; Save ES:BP
29473
29474          ;hkn; SS is DOSDATA
29475          ;context ES
29476 00004C9C 16      push    ss
29477 00004C9D 07      pop     es
29478 00004C9E E82F01      call    DEVNAME ; blast BX
29479 00004CA1 07      POP     ES ; Restore ES:BP
29480 00004CA2 7208      JC      short FindFile ; Not a device
29481 00004CA4 08C0      OR      AL,AL ; Test next char again
29482          ;JZ      short GO_BDEV
29483          ;JMP     FILEINPATH ; Device name in middle of path
29484          ; 27/06/2024
29485 00004CA6 752D      jnz     short FILEINPATH_j
29486
29487          GO_BDEV:
29488 00004CA8 5E      POP     SI ; Points to NUL at end of path
29489 00004CA9 E985FE      JMP     Build_devj
29490
29491          FindFile:
29492          ;;; 7/28/86
29493 00004CAC 803E[4B05]E5      CMP     BYTE [NAME1],0E5H ; if 1st char = E5
29494 00004CB1 7505      JNZ     short NOE5 ; no
29495 00004CB3 C606[4B05]05      MOV     BYTE [NAME1],05H ; change it to 05
29496          NOE5:
29497          ;;; 7/28/86

```



```

29498 00004CB8 57          PUSH    DI                ; Start of this element
29499 00004CB9 06          PUSH    ES                ; Save ES:BP
29500 00004CBA 51          PUSH    CX                ; CL return from NameTrans
29501 ;DOS 3.3 FastOpen 6/12/86 F.C.
29502
29503 00004CBB E8B002        CALL    LookupPath          ; call fastopen to get dir entry
29504 00004CBE 7303        JNC     short DIR_FOUND    ; found dir entry
29505
29506 ;DOS 3.3 FastOpen 6/12/86 F.C.
29507 00004CC0 E87AFA        call    FINDENTRY
29508 DIR_FOUND:
29509 00004CC3 59          POP     CX
29510 00004CC4 07          POP     ES
29511 00004CC5 5F          POP     DI
29512 00004CC6 7303        JNC     short LOAD_BUF
29513 00004CC8 E9D500        JMP     BADPATHPOP
29514
29515 LOAD_BUF:
29516 00004CCB C53E[E205]    LDS     DI,[CURBUF]
29517 ;test byte [bx+0Bh],10h
29518 00004CCF F6470B10    TEST    BYTE [Bx+dir_entry.dir_attr],attr_directory
29519 00004CD3 7503        JNZ     short GO_NEXT      ; DOS 3.3
29520 FILEINPATH_j:
29521 00004CD5 E9A700        JMP     FILEINPATH         ; Error or end of path
29522
29523 ; if we are not setting the directory, then check for end of string
29524
29525 GO_NEXT:
29526 ;hkn; SS override
29527 00004CD8 36803E[4C03]00    CMP     BYTE [SS:NoSetDir],0
29528 00004CDE 7423        JZ      short SetDir
29529 00004CE0 89FA        MOV     DX,DI              ; Save pointer to entry
29530 00004CE2 8CD9        MOV     CX,DS
29531
29532 ;hkn; SS is DOSDATA
29533 ;context DS
29534 00004CE4 16          push    ss
29535 00004CE5 1F          pop     ds
29536 00004CE6 5F          POP     DI                ; Start of next element
29537 ; 19/05/2019 - Retro DOS v4.0
29538 ; MSDOS 6.0
29539 00004CE7 F606[4611]01    TEST    byte [FastOpenFlg],FastOpen_Set ;only DOSOPEN can take advantage of
29540 00004CEC 740B        JZ      short _nofast      ; the FastOpen
29541 00004CEE F606[4611]02    TEST    byte [FastOpenFlg],Lookup_Success ; Lookup just happened
29542 00004CF3 7404        JZ      short _nofast      ; no
29543 00004CF5 8B3E[9C0D]    MOV     DI,[Next_Element_Start] ; no need to insert it again
29544 _nofast:
29545 00004CF9 803D00        CMP     BYTE [DI],0
29546 ;JNZ short NEXT_ONE ; DOS 3.3
29547 ;JMP _SETRET ; retn ; Got it
29548 ;retn ; 05/09/2018
29549 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
29550 ;jmp _SETRET
29551 ; 16/12/2022
29552 00004CFC 7431        jz      short _SETRET
29553
29554 NEXT_ONE:
29555 00004CFE 57          PUSH    DI                ; Put start of next element back on stack
29556 00004CFF 89D7        MOV     DI,DX
29557 00004D01 8ED9        MOV     DS,CX              ; Get back pointer to entry
29558 SetDir:
29559 ;;;
29560 ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29561 00004D03 31D2        xor     dx,dx ; 0
29562 ;cmp [es:bp+0Fh],dx
29563 00004D05 2639560F    cmp     [es:bp+DPB.FAT_SIZE],dx ; 0
29564 00004D09 7503        jnz     short SetDir2 ; not FAT32
29565 00004D0B 8B54FA        mov     dx,[si-6] ; dir_entry.dir_fclus_hi
29566 SetDir2:
29567 00004D0E 368916[EE0A]    mov     [ss:ROOTCLUST_HW],dx ; 0
29568 ;;;
29569 00004D13 8B14        MOV     DX,[SI] ; dir_entry.dir_first
29570
29571 ;DOS 3.3 FastOpen 6/12/86 F.C.
29572 00004D15 1E          PUSH    DS                ; save [curbuf+2]
29573 ;hkn; SS is DOSDATA
29574 00004D16 16          push    ss
29575 00004D17 1F          pop     ds ; set DS Dosgroup
29576 ;test byte [FastOpenFlg],2
29577 00004D18 F606[4611]02    TEST    byte [FastOpenFlg],Lookup_Success
29578 00004D1D 7411        JZ      short DO_NORMAL    ; fastopen not in memory or path not
29579 00004D1F 89D3        MOV     BX,DX ; not found
29580 00004D21 8B3E[BC05]    MOV     DI,[CLUSNUM] ; clusnum was set in LookupPath
29581 00004D25 50          PUSH    AX ; save device id (AH)
29582 00004D26 E832FC        call    SETDIRSRCH
29583 00004D29 58          POP     AX ; restore device id (AH)
29584 00004D2A 83C402        ADD     SP,2 ; pop ds in stack
29585 00004D2D EB36        JMP     short FAST_OPEN_SKIP
29586
29587 ; 16/12/2022
29588 _SETRET:
29589 00004D2F C3          retn
29590
29591 DO_NORMAL:
29592 00004D30 1F          POP     DS ; DS = [curbuf + 2]
29593 ;DOS 3.3 FastOpen 6/12/86 F.C.
29594
29595 00004D31 29FB        SUB     BX,DI ; Offset into sector of start of entry
29596 00004D33 29FE        SUB     SI,DI ; Offset into sector of dir_first
29597 00004D35 53          PUSH    BX
29598 00004D36 50          PUSH    AX
29599 00004D37 56          PUSH    SI
29600 00004D38 51          PUSH    CX
29601
29602 ; 15/02/2024
29603 %if 0
29604 ;push word [di+6]
29605 PUSH    WORD [DI+BUFFINFO.buf_sector] ;AN000;>32mb
29606 ; 19/05/2019
29607 ; MSDOS 6.0
29608 ;push word [di+8]
29609 PUSH    WORD [DI+BUFFINFO.buf_sector+2] ;AN000;>32mb
29610 %else
29611 ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29612 ;lds bx,[di+6]
29613 00004D39 C55D06    lds     bx,[di+BUFFINFO.buf_sector]
29614 00004D3C 53          push    bx
29615 00004D3D 1E          push    ds
29616 %endif
29617
29618 00004D3E 89D3        MOV     BX,DX
29619
29620 ;hkn; SS is DOSDATA
29621 ;context DS

```

```

29622 00004D40 16      push    ss
29623 00004D41 1F      pop     ds
29624                ;invoke SETDIRSRCH          ; This uses UNPACK which might blow
29625 00004D42 E816FC    call    SETDIRSRCH          ; the entry sector buffer
29626                ; 19/05/2019
29627                ; MSDOS 6.0
29628 00004D45 8F06[0706] POP     word [HIGH_SECTOR]
29629 00004D49 5A      POP     DX
29630 00004D4A 7203    JC      short SKIP_GETB
29631                ; 22/09/2023
29632                ;;mov byte [ALLOWED],18h
29633                ;MOV byte [ALLOWED],Allowed_RETRY+Allowed_FAIL ; *
29634                ;XOR AL,AL ; *
29635                ;;invoke GETBUFR          ; Get the entry buffer back
29636                ;call GETBUFR
29637 00004D4C E8D51B    call    GETBUFR ; * ; pre-read
29638                call    GETBUFR ; * ; pre-read
29639 00004D4F 59      POP     CX
29640 00004D50 5E      POP     SI
29641 00004D51 58      POP     AX
29642 00004D52 5B      POP     BX
29643 00004D53 7305    JNC     short SET_THE_BUF
29644 00004D55 5F      POP     DI          ; Start of next element
29645 00004D56 89FE    MOV     SI,DI        ; Point with SI
29646 00004D58 EB21    JMP     SHORT _BADPATH
29647
29648                SET_THE_BUF:
29649 00004D5A E81DF2    call    SET_BUF_AS_DIR
29650 00004D5D 8B3E[E205] MOV     DI,[CURBUF]
29651 00004D61 01FE    ADD     SI,DI          ; Get the offsets back
29652 00004D63 01FB    ADD     BX,DI
29653                ; DOS 3.3 FastOpen 6/12/86 F.C.
29654                FAST_OPEN_SKIP:
29655 00004D65 5F      POP     DI          ; Start of next element
29656 00004D66 E8C202    CALL    InsertPath      ; insert dir entry info
29657                ; DOS 3.3 FastOpen 6/12/86 F.C.
29658 00004D69 8A05    MOV     AL,[DI]
29659 00004D6B 08C0    OR      AL,AL
29660 00004D6D 74C0    JZ      short _SETRET    ; At end
29661 00004D6F 47      INC     DI          ; Skip over "/"
29662 00004D70 89FE    MOV     SI,DI        ; Point with SI
29663 00004D72 E87110    call    PATHCHRCMP
29664 00004D75 7503    JNZ     short find_bad_name ; oops
29665 00004D77 E9A1FE    JMP     FINDPATH        ; Next element
29666
29667                find_bad_name:
29668 00004D7A 4E      DEC     SI          ; Undo above INC to get failure point
29669                _BADPATH:
29670 00004D7B 30C9    XOR     CL,CL        ; Set zero
29671 00004D7D EB28    JMP     SHORT BADPRET
29672
29673                FILEINPATH:
29674 00004D7F 5F      POP     DI          ; Start of next element
29675
29676                ;hkn; SS is DOSDATA
29677                ;context DS          ; Got to from one place with DS gone
29678 00004D80 16      push    ss
29679 00004D81 1F      pop     ds
29680
29681                ; DOS 3.3 FastOpen
29682                ;test byte [FastOpenFlg],1
29683 00004D82 F606[4611]01    TEST    byte [FastOpenFlg],FastOpen_Set ; do this here is we don't want to
29684 00004D87 740B    JZ      short NO_FAST    ; device info to fastopen
29685                ;test byte [FastOpenFlg],2
29686 00004D89 F606[4611]02    TEST    byte [FastOpenFlg],Lookup_Success
29687 00004D8E 7404    JZ      short NO_FAST
29688 00004D90 8B3E[9C0D] MOV     DI,[Next_Element_Start] ; This takes care of one time lookup
29689                ; success
29690                NO_FAST:
29691                ; DOS 3.3 FastOpen
29692 00004D94 8A05    MOV     AL,[DI]
29693 00004D96 08C0    OR      AL,AL
29694                ;JZ short INCRET
29695                ;MOV SI,DI          ; Path too long
29696                ;JMP SHORT BADPRET
29697                ; 27/06/2024
29698 00004D98 750B    jnz     short BADPRET_X
29699
29700                INCRET:
29701                ; DOS 3.3 FasOpen 6/12/86 F.C.
29702                CALL    InsertPath      ; insert dir entry info
29703 00004D9A E88E02    CALL    InsertPath      ; insert dir entry info
29704
29705                ; DOS 3.3 FasOpen 6/12/86 F.C.
29706 00004D9D FEC0    INC     AL          ; Reset zero
29707                ; 16/12/2022
29708                ;_SETRET:
29709 00004D9F C3      retn
29710
29711                BADPATHPOP:
29712 00004DA0 5E      POP     SI          ; Start of next element
29713 00004DA1 8A04    MOV     AL,[SI]
29714                ; 27/06/2024
29715                ;MOV SI,DI          ; Start of bad element
29716 00004DA3 08C0    OR      AL,AL        ; zero if bad element is last, non-zero if path too long
29717                ; 27/06/2024
29718 00004DA5 89FE    mov     si,di
29719                BADPRET:
29720 00004DA7 A0[6D05]    MOV     AL,[ATTRIB]
29721 00004DAA A2[6B05]    MOV     [ATTRIB],AL    ; Make sure return correct
29722 00004DAD F9      STC
29723 00004DAE C3      retn
29724
29725                ;Break <STARTSRCH -- INITIATE DIRECTORY SEARCH>
29726                ;-----
29727                ;
29728                ; Procedure Name : STARTSRCH
29729                ;
29730                ; Inputs:
29731                ; [THISDPB] Set
29732                ; Function:
29733                ; Set up a search for GETENTRY and NEXTENTRY
29734                ; Outputs:
29735                ; ES:BP = Drive parameters
29736                ; Sets up LASTENT, ENTFREE=ENTLAST=-1, VOLID=0
29737                ; Destroys ES,BP,AX
29738                ;-----
29739
29740                STARTSRCH:
29741 00004DAF C42E[8A05]    LES     BP,[THISDPB]
29742 00004DB3 31C0    XOR     AX,AX
29743 00004DB5 A3[4803]    MOV     [LASTENT],AX
29744 00004DB8 A2[7B05]    MOV     [VOLID],AL    ; No volume ID found
29745 00004DBB 48      DEC     AX

```



```

29746 00004DBC A3[D805]      MOV     [ENTFREE],AX
29747 00004DBF A3[DA05]      MOV     [ENTLAST],AX
29748 00004DC2 C3              retn
29749
29750 ;BREAK <MatchAttributes - the final check for attribute matching>
29751 ;-----
29752 ; Procedure Name : MatchAttributes
29753 ;
29754 ; Input:      [Attrib] = attribute to search for
29755 ;           CH = found attribute
29756 ; Output:    JZ <match>
29757 ;           JNZ <nomatch>
29758 ; Registers modified: noneski
29759 ;-----
29760
29761 MatchAttributes:
29762 00004DC3 50      PUSH     AX
29763
29764 ;hkn; SS override
29765 00004DC4 36A0[6B05] MOV     AL,[ss:ATTRIB]      ; AL <- SearchSet
29766 00004DC8 F6D0      NOT      AL                ; AL <- SearchSet'
29767 00004DCA 20E8      AND      AL,CH            ; AL <- SearchSet' and FoundSet
29768 ;and      al,16h
29769 00004DCC 2416      AND      AL,attr_all      ; AL <- SearchSet' and FoundSet and Important
29770 ;
29771 ; the result is non-zero if an attribute is not in the search set
29772 ; and in the found set and in the important set. This means that we do not
29773 ; have a match. Do a JNZ <nomatch> or JZ <match>
29774 ;
29775 00004DCE 58      POP      AX
29776 00004DCF C3              retn
29777
29778 ; 19/05/2019 - Retro DOS v4.0
29779 ; DOSCODE:8148h (MSDOS 6.21, MSDOS.SYS)
29780
29781 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
29782 ; DOSCODE:810Dh (MSDOS 5.0, MSDOS.SYS)
29783
29784 ; 16/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
29785 ; DOSCODE:8E74h (PCDOS 7.1, IBMDOS.COM)
29786
29787 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8148h)
29788 ; (Windows ME IO.SYS - BIOSCODE:843Ch)
29789
29790 ;Break <DevName - Look for name of device>
29791 ;-----
29792 ;
29793 ; Procedure Name : DevName
29794 ;
29795 ; Inputs:
29796 ;   DS,ES:DOSDATA
29797 ;   Filename in NAME1
29798 ;   ATTRIB set so that we can error out if looking for volume IDs
29799 ; Function:
29800 ;   Determine if file is in list of I/O drivers
29801 ; Outputs:
29802 ;   Carry set if not a device
29803 ;   ELSE
29804 ;   Zero flag set
29805 ;   BH = Bit 7,6 = 1, bit 5 = 0 (cooked mode)
29806 ;   bits 0-4 set from low byte of attribute word
29807 ;   DEVPT = DWORD pointer to Device header of device
29808 ; BX destroyed, others preserved
29809 ;-----
29810
29811 DEVNAME:
29812 ; 28/07/2018 - Retro DOS v3.0
29813 ; IBMDOS.COM (MSDOS 3.3) - Offset 49FBh
29814
29815 00004DD0 56      PUSH     SI
29816 00004DD1 57      PUSH     DI
29817 00004DD2 51      PUSH     CX
29818 00004DD3 50      PUSH     AX
29819
29820 ; E5 special code
29821 00004DD4 FF36[4B05] PUSH     WORD [NAME1]
29822 00004DD8 803E[4B05]05 CMP      byte [NAME1],5
29823 00004DDD 7505      JNZ      short NOKTR
29824 00004DDF C606[4B05]E5 MOV      byte [NAME1],0E5h
29825
29826 NOKTR:
29827 00004DE4 F606[6B05]08 ;test byte [ATTRIB],8
29828 ; TEST byte [ATTRIB],attr_volume_id
29829 ; If looking for VOL id don't find devs
29830 ; JNZ short RET31
29831
29832 ;hkn; NULDEV is in DOSDATA
29833 00004DEB BE[4800] MOV      SI,NULDEV
29834
29835 LOOKIO:
29836 ; 21/11/2022
29837 ;test byte [SI+SYSDEV.ATT+1],80h
29838 ; 17/12/2022
29839 ;test byte [si+5],80h
29840 test byte [SI+SYSDEV.ATT+1],(DEVTP>>8)
29841 ;test word [si+4],8000h
29842 ;TEST word [SI+SYSDEV.ATT],DEVTP
29843 JZ      short SKIPDEV      ; Skip block devices (NET and LOCAL)
29844 MOV     AX,SI
29845 ;add si,10
29846 ADD     SI,SYSDEV.NAME
29847
29848 ;hkn; NAME1 is in DOSDATA
29849 MOV     DI,NAME1
29850 MOV     CX,4
29851 ; All devices are 8 letters
29852 REPE    CMPSW
29853 ; Check for name in list
29854 ;MOV SI,AX
29855 ; 27/06/2024
29856 xchg    ax,si
29857 JZ      short IOCHK
29858 ; Found it?
29859
29860 SKIPDEV:
29861 LDS     SI,[SI]
29862 CMP     SI,-1
29863 ; Get address of next device
29864 ; At end of list?
29865 JNZ     short LOOKIO
29866
29867 RET31:
29868 STC
29869 ; Not found
29870
29871 RETNV:
29872 MOV     CX,SS
29873 MOV     DS,CX
29874
29875 POP     WORD [NAME1]
29876 POP     AX
29877 POP     CX
29878 POP     DI
29879 POP     SI
29880 RETN

```

```

29870
29871
29872 IOCHK:
29873 ;hkn; SS override for DEVPT
29874 MOV [SS:DEVPT+2],DS ; Save pointer to device
29875 ;mov bh,[si+4]
29876 MOV BH,[SI+SYSDEV.ATT]
29877 OR BH,0C0h
29878 and bh,0DFh
29879 ;AND BH,~(020h) ; Clears Carry
29880 MOV [SS:DEVPT],SI
29881 JMP short RETNV
29882
29883 ;BREAK <Build_device_ent - Make a Directory entry>
29884 -----
29885 ; Procedure Name : Build_device_ent
29886 ;
29887 ; Inputs:
29888 ; [NAME1] has name
29889 ; BH is attribute field (supplied by DEVNAME)
29890 ; [DEVPT] points to device header (supplied by DEVNAME)
29891 ; Function:
29892 ; Build a directory entry for a device at DEVFCB
29893 ; Outputs:
29894 ; BX points to DEVFCB
29895 ; SI points to dir_first field
29896 ; AH = input BH
29897 ; AL = 0
29898 ; dir_first = DEVPT
29899 ; Zero Set, Carry Clear
29900 ; DS,ES,BP preserved, others destroyed
29901 -----
29902 Build_device_ent:
29903 MOV AX," " ; 2020h
29904
29905 ;hkn; DEVFCB is in DOSDATA
29906 MOV DI,DEVFCB+8 ; Point to extent field
29907
29908 ; Fill dir_ext BUGBUG - use ERRNZs for this stuff!
29909
29910 STOSW
29911 STOSB ; Blank out extent field
29912 ;mov al,40h
29913 MOV AL,attr_device
29914
29915 ; Fill Dir_attr
29916
29917 STOSB ; Set attribute field
29918 XOR AX,AX
29919 MOV CX,10
29920
29921 ; Fill dir_pad
29922
29923 REP STOSW ; Fill rest with zeros
29924 call DATE16
29925
29926 ;hkn; DEVFCB is in DOSDATA
29927 MOV DI,DEVFCB+dir_entry.dir_time ; 09/08/2018
29928 XCHG AX,DX
29929
29930 ; Fill dir_time
29931
29932 STOSW
29933 XCHG AX,DX
29934
29935 ; Fill dir_date
29936
29937 STOSW
29938 MOV SI,DI ; SI points to dir_first field
29939 MOV AX,[DEVPT]
29940
29941 ; Fill dir_first
29942
29943 STOSW ; Dir_first points to device
29944 MOV AX,[DEVPT+2]
29945
29946 ; Fill dir_size_1
29947
29948 STOSW
29949 MOV AH,BH ; Put device atts in AH
29950
29951 ;hkn; DEVFCB is in DOSDATA
29952 MOV BX,DEVFCB
29953 XOR AL,AL ; Set zero, clear carry
29954 retn
29955
29956 ;Break <validateCDS - given a CDS, validate the media and the current directory>
29957 -----
29958 ;
29959 ; validateCDS - Get current CDS. Splice it. Call FatReadCDS to check
29960 ; media. If media has been changed, do DOS_Chdir to validate path.
29961 ; If invalid, reset original CDS to root.
29962 ;
29963 ; Inputs: ThisCDS points to CDS of interest
29964 ; SS:DI points to temp buffer
29965 ; Outputs: The current directory string is validated on the appropriate
29966 ; drive
29967 ; ThisDPB changed
29968 ; ES:DI point to CDS
29969 ; Carry set if error (currently user FAILED to I 24)
29970 ; Registers modified: all
29971 -----
29972
29973 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
29974 ; DOSCODE:819Bh (MSDOS 5.0, MSDOS.SYS)
29975
29976 ; 16/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
29977 ; DOSCODE:8F01h (PCDOS 7.1, IBMDOS.COM)
29978
29979 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:81D6h)
29980 ; (Windows ME IO.SYS - BIOSCODE:84CBh)
29981
29982 validateCDS:
29983 ; 19/05/2019 - Retro DOS v4.0
29984 ; 28/07/2018 - Retro DOS v3.0
29985
29986 %define Temp [bp-2] ; word
29987 %define SaveCDS [bp-6] ; dword
29988 %define SaveCDSL [bp-6] ; word
29989 %define SaveCDSH [bp-4] ; word
29990
29991 ;Enter
29992 push bp
29993 mov bp,sp

```

```

29994 00004E5F 83EC06      sub     sp,6
29995
29996 00004E62 897EFE      MOV     Temp,DI
29997
29998 ;hkn; SS override
29999 00004E65 36C536[A205]  LDS     SI,[SS:THISCDS]
30000 00004E6A 8976FA      MOV     SaveCDSL,SI
30001 00004E6D 8C5EFC      MOV     SaveCDSH,DS
30002 ;EnterCrit critDisk
30003 00004E70 E86FCA      call    ECritDisk
30004 ; 21/11/2022
30005 ;test byte [SI+curdir.flags+1],80h
30006 ;test word [si+67],8000h
30007 ; 17/12/2022
30008 ;test byte [SI+68],80h
30009 00004E73 F6444480     test    byte [SI+curdir.flags+1],(curdir_isnet>>8)
30010 ;TEST word [SI+curdir.flags],curdir_isnet ; Clears carry
30011 00004E77 7403      JZ      short _DoSplice
30012 00004E79 E9A300      JMP     FatFail
30013 _DoSplice:
30014 00004E7C 30D2      XOR     DL,DL
30015 00004E7E 368616[4C03] XCHG    DL,[SS:NoSetDir]
30016
30017 ;hkn; SS is DOSDATA
30018 ;Context ES
30019 00004E83 16      push    ss
30020 00004E84 07      pop     es
30021 ;Invoke FStrcpy
30022 00004E85 E838C9     call    FStrCpy
30023 00004E88 8B76FE     MOV     SI,Temp
30024
30025 ;hkn; SS is DOSDATA
30026 ;Context DS
30027 00004E8B 16      push    ss
30028 00004E8C 1F      pop     ds
30029 ;Invoke Splice
30030 00004E8D E84930     call    Splice
30031
30032 ;hkn; SS is DOSDATA
30033 ;Context DS
30034 00004E90 16      push    ss
30035 00004E91 1F      pop     ds
30036 00004E92 8816[4C03] MOV     [NoSetDir],DL
30037 00004E96 C43E[A205] LES     DI,[THISCDS]
30038 ;SAVE <BP>
30039 00004E9A 55      push    bp
30040 ;Invoke FATREAD_CDS
30041 00004E9B E8B816     call    FATREAD_CDS
30042 ;RESTORE <BP>
30043 00004E9E 5D      pop     bp
30044 00004E9F 727E     JC      short FatFail
30045
30046 00004EA1 C536[A205]  LDS     SI,[THISCDS] ; if (ThisCDS->ID == -1) {
30047
30048 ;;;
30049 ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30050 ;cmp word [si+4Bh],0FFFFh
30051 00004EA5 837C4BFF     cmp     word [si+curdir.ID+2],-1
30052 00004EA9 7566     jnz     short RestoreCDS
30053 ;;;
30054
30055 ;cmp word [si+73],-1
30056 00004EAB 837C49FF     cmp     word [SI+curdir.ID],-1
30057 00004EAF 7560     jnz     short RestoreCDS
30058
30059 ;hkn; SS is DOSDATA
30060 ;Context ES
30061 00004EB1 16      push    ss
30062 00004EB2 07      pop     es
30063
30064 ;hkn; SS override
30065 ;SAVE <wfp_start> ; t = wfp_start;
30066 00004EB3 36FF36[B205] push    word [SS:WFP_START]
30067 ;cmp si,[bp-6]
30068 00004EB8 3B76FA     cmp     SI,SaveCDSL ; if not spliced
30069 00004EBB 750B     JNZ     short DoChdir
30070 ;mov di,[bp-2]
30071 00004EBD 8B7EFE     MOV     DI,Temp
30072
30073 ;hkn; SS override
30074 00004EC0 36893E[B205] MOV     [SS:WFP_START],DI ; wfp_start = d;
30075 ;Invoke FStrCpy ; strcpy (d, ThisCDS->Text);
30076 00004EC5 E8F8C8     call    FStrCpy
30077 DoChdir:
30078 ;hkn; SS is DOSDATA
30079 ;Context DS
30080 00004EC8 16      push    ss
30081 00004EC9 1F      pop     ds
30082 ;SAVE <<WORD PTR SAttrib>,BP> ; c = DOSchDir ();
30083 00004ECA FF36[6D05] push    word [SATTRIB]
30084 00004ECE 55      push    bp
30085 ;Invoke DOS_ChDir
30086 00004ECF E85CEB     call    DOS_CHDIR
30087 ;RESTORE <BP,BX,wfp_start> ; wfp_start = t;
30088 00004ED2 5D      pop     bp
30089 00004ED3 5B      pop     bx
30090 00004ED4 8F06[B205] pop     word [WFP_START]
30091 00004ED8 881E[6D05] MOV     [SATTRIB],BL
30092 ;;;
30093 ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30094 00004EDC 8B3E[DC0A] mov     di,[DIRSTART_HW]
30095 ;;;
30096 00004EE0 C576FA     LDS     SI,SaveCDS
30097 00004EE3 7311     JNC     short SetCluster ; if (c == -1) {
30098
30099 ;hkn; SS override for THISCDS
30100 00004EE5 368936[A205] MOV     [SS:THISCDS],SI ; ThisCDS = TmpCDS;
30101 00004EEA 368C1E[A405] MOV     [SS:THISCDS+2],DS
30102 00004EEF 31C9     XOR     CX,CX ; TmpCDS->text[3] = c = 0;
30103 ;;;
30104 ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30105 00004EF1 31FF     xor     di,di
30106 ;;;
30107 00004EF3 884C03     MOV     [SI+3],CL ; }
30108 SetCluster:
30109 ; 16/02/2024
30110 ;mov word [si+73],0FFFFh
30111 ;MOV word [SI+curdir.ID],-1; TmpCDS->ID = -1;
30112 ;
30113 00004EF6 36C536[A205] LDS     SI,[SS:THISCDS] ; ThisCDS->ID = c;
30114 ; 21/11/2022
30115 ;test byte [si+curdir.flags+1],20h
30116 ; 19/05/2019
30117 ; MSDOS 6.0

```

```

30118 ; 17/12/2022
30119 ;test byte [si+68],20h
30120 00004EFB F6444420 test byte [SI+curdir.flags+1],(curdir_splce>>8)
30121 ;;test word [si+67],2000h
30122 ;TEST word [SI+curdir.flags],curdir_splce ;AN000;;MS. for Join and Subst
30123 00004EFF 7405 JZ short _setdirclus ;AN000;;MS.
30124 00004F01 B9FFFF MOV CX,-1 ; 0FFFFh ;AN000;;MS.
30125 ;;;
30126 ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30127 00004F04 89CF mov di,cx
30128 ;;;
30129 _setdirclus:
30130 ;;;
30131 ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30132 00004F06 36893E[DC0A] mov [ss:DIRSTART_HW],di
30133 ;mov [si+4Bh],di
30134 00004F0B 897C4B mov [si+curdir.ID+2],di
30135 ;;;
30136 ;mov [si+73],cx
30137 00004F0E 894C49 MOV [SI+curdir.ID],CX ; }
30138 RestoreCDS:
30139 00004F11 C47EFA LES DI,SaveCDS
30140 00004F14 36893E[A205] MOV [SS:THISCDS],DI
30141 00004F19 368C06[A405] MOV [SS:THISCDS+2],ES
30142 00004F1E F8 CLC
30143 FatFail:
30144 ;LeaveCrit_critDisk
30145 00004F1F E8EDC9 call LCritDisk
30146 ;
30147 ;les di,[bp-6]
30148 00004F22 C47EFA LES DI,SaveCDS
30149 ;Leave
30150 00004F25 89EC mov sp,bp
30151 00004F27 5D pop bp
30152 00004F28 C3 retn
30153
30154 ; 28/07/2018 - Retro DOS v3.0
30155 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 43BDh
30156
30157 ;Break <CheckThisDevice - Check for being a device>
30158 -----
30159 ;
30160 ; CheckThisDevice - Examine the area at DS:SI to see if there is a valid
30161 ; device specified. We will return carry if there is a device present.
30162 ; The forms of devices we will recognize are:
30163 ;
30164 ; [path]device
30165 ;
30166 ; Note that the drive letter has *already* been removed. All other forms
30167 ; are not considered to be devices. If such a device is found we change
30168 ; the source pointer to point to the device component.
30169 ;
30170 ; Inputs: ES is DOSDATA
30171 ; DS:SI contains name
30172 ; Outputs: ES is DOSDATA
30173 ; DS:SI point to name or device
30174 ; Carry flag set if device was found
30175 ; Carry flag reset otherwise
30176 ; Registers Modified: all except ES:DI, DS
30177 -----
30178
30179 CheckThisDevice:
30180 00004F29 57 push di
30181 00004F2A 56 push si
30182 00004F2B 89F7 MOV DI,SI
30183
30184 ; Check for presence of \dev\ (Dam multiplan!)
30185
30186 00004F2D 8A04 MOV AL,[SI]
30187 00004F2F E8B40E call PATHCHRCMP ; is it a path char?
30188 00004F32 7517 JNZ short ParseDev ; no, go attempt to parse device
30189 00004F34 46 INC SI ; simulate LODSB
30190
30191 ; We have the leading path separator. Look for DEV part.
30192
30193 00004F35 AD LODSW
30194 00004F36 0B2020 OR AX,2020h
30195 00004F39 3D6465 cmp ax,"de"
30196 ;CMP AX,"e"<< 8 + "d"
30197 00004F3C 752D JNZ short NotDevice ; not "de", assume not device
30198 00004F3E AC LODSB
30199 00004F3F 0C20 OR AL,20h
30200 00004F41 3C76 CMP AL,"v" ; Not "v", assume not device
30201 00004F43 7526 JNZ short NotDevice
30202 00004F45 AC LODSB
30203 00004F46 E89D0E call PATHCHRCMP ; do we have the last path separator?
30204 00004F49 7520 JNZ short NotDevice ; no. go for it.
30205
30206 ; DS:SI now points to a potential drive. Preserve them as NameTrans advances
30207 ; SI and DevName may destroy DS.
30208
30209 ParseDev:
30210 00004F4B 1E push ds
30211 00004F4C 56 push si ; preserve the source pointer
30212 00004F4D E8DA0D call NameTrans ; advance DS:SI
30213 00004F50 803C00 CMP BYTE [SI],0 ; parse entire string?
30214 00004F53 F9 STC ; simulate a Carry return from DevName
30215 00004F54 750B JNZ short SkipSearch ; no parse. simulate a file return.
30216
30217 ;hkn; SS is DOSDATA
30218 00004F56 16 push ss
30219 00004F57 1F pop ds
30220
30221 ; M026 - start - fix ported from ROMDOS2 for bug # 2849
30222 ;
30223 ; SR;
30224 ; We have to set Attrib before invoking DevName. Otherwise, the value from
30225 ; a previous DOS call is used and DevName thinks it is not a device if the
30226 ; old call set the volume attribute bit.
30227
30228 00004F58 A0[6D05] mov al,[SATTRIB]
30229 00004F5B A2[6B05] mov [ATTRIB],al ;set Attrib for DevName
30230
30231 ; M026 - end
30232
30233 00004F5E E86FFE call DEVNAME
30234
30235 SkipSearch:
30236 00004F61 5E pop si
30237 00004F62 1F pop ds
30238
30239 ; SI points to the beginning of the potential device. If we have a device
30240 ; then we do not change SI. If we have a file, then we reset SI back to the
30241 ; original value. At this point Carry set indicates FILE.

```

```

30242
30243
30244 00004F63 5F
30245 00004F64 7302
30246 00004F66 89FE
30247
30248 00004F68 5F
30249 00004F69 F5
30250 00004F6A C3
30251
30252 00004F6B F9
30253 00004F6C EBF5
30254
30255
30256
30257
30258
30259
30260
30261
30262
30263
30264
30265
30266
30267
30268
30269
30270
30271
30272
30273
30274
30275
30276
30277
30278
30279
30280
30281
30282 00004F6E 36F606[4611]01
30283 00004F74 7515
30284
30285 00004F76 EB11
30286
30287
30288
30289 00004F78 83F8FF
30290 00004F7B 7506
30291 00004F7D 36C606[4611]00
30292
30293
30294 00004F83 368026[4611]FB
30295
30296
30297 00004F89 F9
30298 00004F8A C3
30299
30300
30301
30302
30303 00004F8B 36F606[4611]08
30304 00004F91 75E3
30305
30306 00004F93 BB[3C11]
30307
30308
30309 00004F96 368B36[B205]
30310 00004F9B BF[7C0D]
30311 00004F9E B9[4711]
30312 00004FA1 B001
30313 00004FA3 1E
30314 00004FA4 07
30315
30316
30317
30318 00004FA5 FF5F02
30319 00004FA8 72CE
30320
30321 00004FAA 8D5CFE
30322
30323
30324 00004FAD 363B1E[B205]
30325 00004FB2 74C4
30326
30327
30328
30329
30330 00004FB4 803C00
30331 00004FB7 752D
30332 00004FB9 51
30333
30334
30335 00004FBA 368A0E[6B05]
30336 00004FBF 368A2E[6D05]
30337 00004FC4 36882E[6B05]
30338
30339 00004FC9 268A6D0B
30340 00004FCD E8F3FD
30341
30342 00004FD0 59
30343 00004FD1 75B6
30344
30345
30346
30347
30348 00004FD3 36803E[4C03]00
30349 00004FD9 740B
30350
30351
30352
30353
30354
30355 00004FDB 26F6450B10
30356 00004FE0 7404
30357 00004FE2 89CB
30358 00004FE4 EB17
30359
30360 00004FE6 89CB
30361
30362 00004FE8 8B470B
30363 00004FEB 36A3[C205]
30364
30365

CheckReturn:
    pop     di                ; get original SI
    JNC     short Check_Done  ; if device then do not reset pointer
    MOV     SI,DI
Check_Done:
    pop     di
    CMC
    retn                ; invert carry. Carry => device
NotDevice:
    STC
    JMP     short CheckReturn

;BREAK <LookupPath - call fastopen to get dir entry info>
;-----
;
; Procedure Name : LookupPath
;
; Output DS:SI -> path name,
;         ES:DI -> dir entry info buffer
;         ES:CX -> extended dir info buffer
;
;         carry flag clear : tables pointed by ES:DI and ES:CX are filled by
;                           FastOpen, DS:SI points to char just one after
;                           the last char of path name which is fully or
;                           partially found in FastOpen
;         carry flag set : FastOpen not in memory or path name not found
;-----

; 21/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
; PC DOS 7.1 IBMDOS.COM - DOSCODE:9015h

; (MSDOS 6.22 MSDOS.SYS - DOSCODE:82D9h)

LookupPath:
;   PUSH    AX

;hkn; SS override
;test     byte [ss:FastOpenFlg],1
;TEST     byte [ss:FastOpenFlg],FastOpen_Set ; flg is set in DOSOPEN
;JNZ      short FASTINST ; and this routine is
NOLOOK:
;JMP      NOLOOKUP ; executed once

; 21/02/2024
NOTFOUND:
;CMP      AX,-1 ; not in memory ?
;JNZ      short Partial_Success ; yes, in memory
;MOV      byte [SS:FastOpenFlg],0 ; no more fastopen
Partial_Success:
;and      byte [SS:FastOpenFlg],0FBh
;AND      byte [SS:FastOpenFlg],Special_Fill_Reset
NOLOOKUP:
;   POP     AX
;   STC
;   RETN

FASTINST:
;hkn; SS override
;test     byte [ss:FastOpenFlg],8
;TEST     byte [ss:FastOpenFlg],No_Lookup ; no more lookup?
;JNZ      short NOLOOK ; yes
;MOV      BX,FastOpenTable ; get fastopen related tab

;hkn; SS override
;MOV      SI,[SS:WFP_START] ; si points to path name
;MOV      DI,Dir_Info_Buff
;MOV      CX,FastOpen_Ext_Info
;MOV      AL,FONC_Look_up ; al = 1
;PUSH     DS
;POP      ES

;hkn; SS override
;call     far [bx+2]
;CALL     far [BX+fastopen_entry.name_caching] ;call fastopen
;JC       short NOTFOUND ; fastopen not in memory

;LEA      BX,[SI-2]

;hkn; SS override
;CMP      BX,[SS:WFP_START] ; path found ?
;JZ       short NOTFOUND ; no

; 19/05/2019 - Retro DOS v4.0

; MSDOS 6.0
;CMP      BYTE [SI],0 ; * ; 21/02/2024 ; AN000;FO.
;JNZ      short parfnd ; AN000;FO.; partiallyfound
;PUSH     CX ; AN000;FO.; is attribute matched ?

;hkn; SS override for attrib/sattrib
;MOV      CL,[ss:ATTRIB] ; AN000;FO.;
;MOV      CH,[ss:SATTRIB] ; AN000;FO.; attrib=sattrib
;MOV      [ss:ATTRIB],CH ; AN000;FO.;
;mov      ch,[es:di+0Bh]
;MOV      CH,[ES:DI+dir_entry.dir_attr] ; AN000;FO.;
;call     MatchAttributes ; AN000;FO.;
;MOV      [ss:ATTRIB],CL ; AN001;FO.; restore attrib
;POP      CX ; AN000;FO.;
;JNZ      short NOLOOKUP ; AN000;FO.; not matched

; 21/02/2024 - Retro DOS v5.0
; PC DOS 7.1 IBMDOS.COM
;
;cmp      byte [ss:NoSetDir],0
;jz       short parfnd
;
;cmp      byte [si],0 ; * ; byte [si] = 0
;jnz      short parfnd
;
;test     byte [es:di+0Bh],10h
;test     byte [es:di+dir_entry.dir_attr],attr_directory
;jz       short parfnd
;mov      bx,cx
;jmp      short parfnd2
parfnd:
;mov      bx,cx
;mov      ax,[bx+0Bh]
;mov      ax,[bx+FEI.dirstart]
;mov      [ss:DIRSTART],ax
; 27/06/2024
;mov      ax,[bx+7]

```

```

30366 00004FEF 8B4707      mov     ax,[bx+FEI.c\usnum+2]
30367 00004FF2 36A3[DE0A]      mov     [ss:CLUSNUM_HW],ax
30368                      ;mov     ax,[bx+5]
30369 00004FF6 8B4705      mov     ax,[bx+FEI.c\usnum]
30370 00004FF9 36A3[BC05]      mov     [ss:CLUSNUM],ax
30371                      parfnd2:
30372 00004FFD 368936[9C0D]      mov     [ss:Next_Element_Start],si
30373                      ;mov     ax,[bx+9]
30374 00005002 8B4709      mov     ax,[bx+FEI.lastent]
30375 00005005 36A3[4803]      mov     [ss:LASTENT],ax
30376                      ;;;
30377
30378                      ; 21/02/2024
30379                      %if 0
30380                      parfnd:
30381                      ;hkn; SS override
30382                      MOV     [ss:Next_Element_Start],SI          ; save si
30383                      MOV     BX,CX
30384                      ; MSDOS 6.0
30385                      ;mov     ax,[bx+7]
30386                      MOV     AX,[BX+FEI.lastent]                ;AN000;;FO. restore lastentry
30387                      ;hkn; SS override for LASTENT, DIRSTART, CLUSNUM
30388                      MOV     [ss:LASTENT],AX                    ;AN000;;FO.
30389                      MOV     AX,[BX+FEI.dirstart]                ;AN001;;FO. restore dirstart
30390                      MOV     [ss:DIRSTART],AX                    ;AN001;;FO.
30391                      ; MSDOS 3.3 (& MSDOS 6.0)
30392                      ;;mov     ax,[bx+3] ; MSDOS 3.3
30393                      ;mov     ax,[bx+5] ; MSDOS 6.0
30394                      MOV     AX,[BX+FEI.c\usnum]                ; restore next cluster num
30395                      MOV     [ss:CLUSNUM],AX                      ;
30396                      %endif
30397
30398 00005009 06                      PUSH     ES                      ; save ES
30399                      ;hkn; SS override
30400 0000500A 36C41E[8A05]      LES     BX,[ss:THISDPB]                ; put drive id
30401 0000500F 268A27      mov     ah,[es:bx] ; 15/08/2018
30402                      ;MOV     AH,[es:bx+DPB.DRIVE]                ; in AH for DOOPEN
30403 00005012 07                      POP      ES                      ; pop ES
30404                      ;SR;
30405                      ; we cannot have a root dir if we have come here. So, we zero out CurBuf to
30406                      ;indicate it is not a root dir
30407
30408 00005013 36C706[E205]0000      mov     word [ss:CURBUF],0                ; indicate not root dir
30409 0000501A 368C06[E405]      MOV     WORD [ss:CURBUF+2],ES                ; [curbuf+2].bx points to
30410 0000501F 89FB                      MOV     BX,DI                      ; start of entry
30411                      ;lea     si,[di+1Ah]
30412 00005021 8D751A      LEA     SI,[DI+dir_entry.dir_first]        ; [curbuf+2]:si points to
30413                      ;                      ; dir_first field in the
30414                      ;                      ; dir entry
30415                      ;hkn; SS override for FastOpenFlg
30416                      ;or     byte [ss:FastOpenFlg],12h ; 29/12/2022
30417 00005024 36800E[4611]12      OR      byte [ss:FastOpenFlg],Lookup_Success+Set_For_Search
30418                      ; POP     AX
30419 0000502A C3                      RETN
30420
30421                      ;BREAK <InsertPath - call fastopen to insert dir entry info>
30422                      ;-----
30423                      ;
30424                      ; Procedure Name : InsertPath
30425                      ; Input: FastOpen_Set flag set when from DOSOPEN otherwise 0
30426                      ;       Lookup_Success flag set when got dir entry info from FASTOPEN
30427                      ;       DS = DOSDATA
30428                      ; Output: FastOpen_Ext_Info is set and path dir info is inserted
30429                      ;
30430                      ;-----
30431
30432                      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
30433                      ; 21/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
30434
30435                      InsertPath:
30436 0000502B 9C                      PUSHF
30437                      ;hkn; SS override for FastOpenFlag
30438                      ;test    byte [ss:FastOpenFlg], 1
30439 0000502C 36F606[4611]01      TEST    byte [ss:FastOpenFlg],FastOpen_Set ;only DOSOPEN can take advantage of
30440 00005032 747C                      JZ      short GET_NEXT_ELEMENT ; the FastOpen
30441                      ;test    byte [ss:FastOpenFlg],2
30442 00005034 36F606[4611]02      TEST    byte [ss:FastOpenFlg],Lookup_Success ; Lookup just happened
30443 0000503A 740D                      JZ      short INSERT_DIR_INFO ; no
30444                      ;and     byte [ss:FastOpenFlg],0FDh
30445 0000503C 368026[4611]FD      AND     byte [ss:FastOpenFlg],Lookup_Reset ; we got dir info from fastopen so
30446 00005042 368B3E[9C0D]      MOV     DI,[ss:Next_Element_Start] ; no need to insert it again
30447 00005047 EB61                      JMP     short GET_NEXT2
30448
30449                      INSERT_DIR_INFO:                      ; save registers
30450 00005049 1E                      PUSH     DS
30451 0000504A 06                      PUSH     ES
30452 0000504B 53                      PUSH     BX
30453 0000504C 56                      PUSH     SI
30454 0000504D 57                      PUSH     DI
30455 0000504E 51                      PUSH     CX
30456 0000504F 50                      PUSH     AX
30457
30458                      ;hkn; SS override
30459 00005050 36C53E[E205]      LDS     DI,[ss:CURBUF]                ; DS:DI -> buffer header
30460 00005055 BE[4711]      MOV     SI,FastOpen_Ext_Info
30461
30462                      ; 21/02/2024
30463                      %if 0
30464                      ;mov     ax,[di+6]
30465                      MOV     AX,[DI+BUFFINFO.buf_sector] ; get directory sector
30466                      ; MSDOS 6.0
30467                      ;mov     [ss:si+1],ax
30468                      MOV     [ss:SI+FEI.dirsec],AX                ;AN000; >32mb save dir sector
30469                      ; 19/05/2019 - Retro DOS v4.0
30470                      MOV     AX,[DI+BUFFINFO.buf_sector+2] ;AN000; >32mb
30471
30472                      ;hkn; SS is DOSDATA
30473                      push     ss
30474                      pop      ds
30475                      ; MSDOS 3.3
30476                      ;;mov     [si+1],ax
30477                      ;MOV     [SI+FEI.dirsec],AX
30478                      ; MSDOS 6.0
30479                      ;mov     [si+3],ax
30480                      MOV     [SI+FEI.dirsec+2],AX                ;AN000;>32mb save high dir sector
30481                      %else
30482                      ;lds     ax,[di+6]
30483 00005058 C54506      lds     ax,[di+BUFFINFO.buf_sector] ; get directory sector
30484                      ;mov     [ss:si+1],ax
30485                      ; 27/06/2024
30486                      ;mov     [ss:si+FEI.dirsec],ax
30487                      ;
30488                      ;mov     [ss:si+3],ax
30489 0000505B 368C5C03      mov     [ss:si+FEI.dirsec+2],ds

```



```

30490 0000505F 16      push    ss
30491 00005060 1F      pop     ds
30492                ; 27/06/2024
30493 00005061 894401    mov     [si+FEI.dirsec],ax
30494                %endif
30495
30496                ; 21/02/2024 (PCDOS 7.1 IBMDOS.COM)
30497                ;;;
30498 00005064 A1[DE0A]    mov     ax,[CLUSNUM_HW]
30499                ;mov     [si+7],ax
30500 00005067 894407    mov     [si+FEI.clusnum+2],ax
30501                ;;;
30502
30503                ; MSDOS 3.3 (& MSDOS 6.0)
30504 0000506A A1[BC05]    MOV     AX,[CLUSNUM]      ; save next cluster number
30505                ;mov     [si+5],ax ; MSDOS 6.0
30506                ;;mov    [si+3],ax ; MSDOS 3.3
30507 0000506D 894405    MOV     [SI+FEI.clusnum],AX
30508                ; MSDOS 6.0
30509 00005070 A1[4803]    MOV     AX,[LASTENT]      ;AN000;FO. save lastentry for search first
30510                ;;mov    [si+7],ax
30511                ;;mov    [si+9],ax ; PCDOS 7.1 ; 21/02/2024
30512 00005073 894409    MOV     [SI+FEI.lastent],AX ;AN000;FO.
30513 00005076 A1[C205]    MOV     AX,[DIRSTART]    ;AN001;FO. save for search first
30514                ;;mov    [si+9],ax
30515                ;;mov    [si+11],ax ; PCDOS 7.1 ; 21/02/2024
30516 00005079 89440B    MOV     [SI+FEI.dirstart],AX ;AN001;FO.
30517
30518                ; MSDOS 3.3 (& MSDOS 6.0)
30519 0000507C 89D8        MOV     AX,BX
30520                ;;;add    di,16 ; MSDOS 3.3
30521                ;;;add    di,20 ; MSDOS 6.0
30522                ;;;add    di,24 ; PCDOS 7.1 ; 21/02/2024
30523 0000507E 83C718    ADD     DI,BUFINSIZ      ; DS:DI -> start of data in buffer
30524 00005081 29F8        SUB     AX,DI             ; AX=BX relative to start of sector
30525                ;mov     cl,32
30526 00005083 B120        MOV     CL,dir_entry.size
30527 00005085 F6F1        DIV     CL
30528                ;MOV     [SI+FEI.dirpos],AL ; save directory entry # in buffer
30529 00005087 8804        mov     [si],al
30530
30531 00005089 1E          PUSH    DS
30532 0000508A 07          POP     ES
30533
30534 0000508B 8E1E[E405]    MOV     DS,[CURBUF+2]
30535 0000508F 89DF        MOV     DI,BX           ; DS:DI -> dir entry info
30536                ;cmp     word [di+1Ah],0
30537 00005091 837D1A00    CMP     word [DI+dir_entry.dir_first],0
30538                ; never insert info when file is empty
30539 00005095 740C        JZ      short SKIP_INSERT ; e.g. newly created file
30540
30541                ; 21/02/2024 - Erdogan Tan
30542                ; Note: PCDOS 7.1 IBMDOS.COM code doesn't check high word
30543                ; of the 1st cluster here
30544                ;;;
30545                ; 21/02/2024 - Retro DOS v5.0
30546                ;jnz     short dont_skip_insert
30547                ;;cmp     word [di+14h],0
30548                ;;cmp     word [di+dir_entry.dir_fclus_hi],0
30549                ;jz      short SKIP_INSERT
30550                ;dont_skip_insert: ; Retro DOS v5.0
30551                ;;;
30552
30553 00005097 56          PUSH    SI
30554 00005098 5B          POP     BX           ; ES:BX -> extended info
30555
30556                ;mov     al,2
30557 00005099 B002        MOV     AL,FONC_insert ; call fastopen insert operation
30558 0000509B BE[3C11]    MOV     SI,FastOpenTable
30559                ;call    far [es:si+2] ; call dword ptr es:[si+2] ; 29/12/2022
30560                ; 07/12/2022
30561 0000509E 26FF5C02    CALL    far [ES:SI+fastopen_entry.name_caching]
30562
30563 000050A2 F8          CLC
30564                SKIP_INSERT:
30565 000050A3 58          POP     AX
30566 000050A4 59          POP     CX           ; restore registers
30567 000050A5 5F          POP     DI
30568 000050A6 5E          POP     SI
30569 000050A7 5B          POP     BX
30570 000050A8 07          POP     ES
30571 000050A9 1F          POP     DS
30572
30573                GET_NEXT2:
30574 000050AA 36800E[4611]08    ;or     [ss:FastOpenFlg],8
30575                OR      byte [SS:FastOpenFlg],No_Lookup
30576                ; we got dir info from fastopen so
30577 000050B0 9D          GET_NEXT_ELEMENT:
30578 000050B1 C3          POPF
30579                RETN
30580
30581                ;=====
30582                ; DEV.ASM (MSDOS 6.0, 1991)
30583                ;=====
30584                ; 17/07/2018 - Retro DOS v3.0
30585                ; 30/04/2019 - Retro DOS v4.0
30586                ; 21/02/2024 - Retro DOS v5.0
30587
30588                ;** Misc Routines to do 1-12 low level I/O and call devices
30589
30590                ; offset 12B8h of IBMDOS.COM (MSDOS 3.3), 1987
30591
30592                ;DOSCODE:8401h (MSDOS 6.21 & 6.22, MSDOS.SYS) ; 21/02/2024
30593                ;BIOSCODE:8608h (Windows ME, IO.SYS) ; 21/02/2024
30594
30595                ;DOSCODE:9163h (PCDOS 7.1, IBMDOS.COM) ; 21/02/2024
30596
30597                ;Public DEV001S, DEV001E ; Pathgen labels
30598                ;DEV001s:
30599                ; length of packets
30600                ;LenTab: DB DRDWRHL, DRDNDHL, DRDWRHL, DSTATHL, DFLSHL, DRDNDHL
30601                ; 21/02/2024
30602                ;;LenTab: db 22,14,22,13,15,14 ; MSDOS 5.0-6.22 & windows ME
30603                ;LenTab: db 22,14,22,13,13,14 ; PCDOS 7.1
30604
30605                ; Error Function
30606
30607                CmdTab:
30608                DB 86h, DEVRD ; 0 input
30609                DB 86h, DEVRDND ; 1 input status
30610                DB 87h, DEVRT ; 2 output
30611                DB 87h, DEVOST ; 3 output status
30612                DB 86h, DEVIFL ; 4 input flush
30613                DB 86h, DEVRDND ; 5 input status with system WAIT

```

```

30614 ; Offset 12BEh of IBMDOS.COM (MSDOS 3.3), 1987
30615
30616 ;CmdTab:
30617 ; db 86h, 4
30618 ; db 86h, 5
30619 ; db 87h, 8
30620 ; db 87h, 10
30621 ; db 86h, 7
30622 ; db 86h, 5
30623
30624 ;DEV001E:
30625
30626 ; 30/04/2019 - Retro DOS v4.0
30627 ; DOSCODE:8413h (MSDOS 6.21, MSDOS.SYS)
30628
30629 ;Break <IOFUNC -- DO FUNCTION 1-12 I/O>
30630 -----
30631 ;
30632 ; Procedure Name : IOFUNC
30633 ;
30634 ; Inputs:
30635 ; DS:SI Points to SFT
30636 ; AH is function code
30637 ; = 0 Input
30638 ; = 1 Input Status
30639 ; = 2 Output
30640 ; = 3 Output Status
30641 ; = 4 Flush
30642 ; = 5 Input Status - System WAIT invoked for K09 if no char
30643 ; present.
30644 ; AL = character if output
30645 ; Function:
30646 ; Perform indicated I/O to device or file
30647 ; Outputs:
30648 ; AL is character if input
30649 ; If a status call
30650 ; zero set if not ready
30651 ; zero reset if ready (character in AL for input status)
30652 ; For regular files:
30653 ; Input Status
30654 ; Gets character but restores position
30655 ; Zero set on EOF
30656 ; Input
30657 ; Gets character advances position
30658 ; Returns ^Z on EOF
30659 ; Output Status
30660 ; Always ready
30661 ; AX altered, all other registers preserved
30662 -----
30663
30664 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
30665 ; DOSCODE:83D8h (MSDOS 5.0, MSDOS.SYS)
30666
30667 ; 21/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
30668 ; DOSCODE:9175h (PCDOS 7.1, IBMDOS.COM)
30669
30670 ; DOSCODE:8413h (MSDOS 6.22, MSDOS.SYS)
30671 ; BIOSCODE:861Ah (Windows ME, IO.SYS)
30672
30673 IOFUNC:
30674 000050C4 368C16[8C03] MOV [SS:IOXAD+2],SS ; SS override for IOXAD, IOSCNT,
30675 ; DEVIOBUF
30676 000050C9 36C706[8A03][BC03] MOV WORD [SS:IOXAD],DEVIOBUF
30677 000050D0 36C706[8E03]0100 MOV WORD [SS:IOSCNT],1
30678 000050D7 36A3[BC03] MOV WORD [SS:DEVIOBUF],AX
30679 ;test byte [si+6],80h
30680 ;TEST word [SI+SF_ENTRY.sf_flags],sf_isnet ; 8000h
30681 000050DB F6440680 test byte [SI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
30682 000050DF 7403 JZ short IOTO22 ;AN000;
30683 000050E1 E9A300 JMP IOTOFILE ;AN000;
30684
30685 IOTO22:
30686 ;test word [si+5],80h
30687 ;TEST word [SI+SF_ENTRY.sf_flags],devid_device
30688 000050E4 F6440580 test byte [SI+SF_ENTRY.sf_flags],devid_device
30689 000050E8 7503 JNZ short IOTO33 ;AN000;
30690 000050EA E99A00 JMP IOTOFILE ;AN000;
30691
30692 IOTO33:
30693 push es ; * (MSDOS 6.21)
30694 call save_world
30695 MOV DX,DS
30696 MOV BX,SS
30697 MOV DS,BX
30698 MOV ES,BX
30699 XOR BX,BX
30700 cmp ah,5 ; system wait enabled?
30701 jnz short _no_sys_wait
30702 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
30703 ; 16/12/2022
30704 ;or bh,04h
30705 ;;or bx,0400H ; Set bit 10 in status word for driver
30706 ; ; It is up to device driver to carry out
30707 ; ; appropriate action.
30708 ; 04/07/2024
30709 mov bh,4
30710 _no_sys_wait:
30711 MOV [IOCALL_REQSTAT],BX
30712 XOR BX,BX
30713 MOV [IOMED],BL
30714
30715 MOV BL,AH ; get function
30716 MOV AH,[cs:BX+LenTab]
30717 SHL BX,1
30718 MOV CX,[cs:BX+CmdTab]
30719 MOV BX,IOCALL ; DOSDATA:037Ch
30720 MOV [IOCALL_REQLEN],AH
30721 MOV [IOCALL_REQFUNC],CH
30722
30723 MOV DS,DX
30724 CALL DEVIIOCALL
30725 DI,[SS:IOCALL_REQSTAT]; SS override
30726 and di,di
30727 js short DevErr
30728
30729 OKDevIO:
30730 MOV AX,SS
30731 MOV DS,AX
30732
30733 ;cmp ch,5
30734 CMP CH,DEVDRND
30735 JNZ short DNODRD
30736 MOV AL,[IORCHR]
30737 MOV [DEVIOBUF],AL
30738
30739 DNODRD:
30740 MOV AH,[IOCALL_REQSTAT+1]

```



```

30738 00005146 F6D4          NOT     AH          ; Zero = busy, not zero = ready
30739                      ;and     ah,2
30740 00005148 80E402        AND     AH,STBUI>>8
30741
30742 QuickReturn:              ;AN000; 2/13/KK
30743 0000514B E8F3B2        call    restore_world
30744 0000514E 07           pop     es ; * (MSDOS 6.21)
30745
30746                      ; SR;
30747                      ; We return ax = -1 if the user failed on I24. This is the case if
30748                      ; IoStatFail = -1 (set after return from the I24)
30749
30750                      ; MSDOS 6.0
30751 0000514F 9C           pushf
30752 00005150 36A0[8300]    mov     al,[ss:IoStatFail] ;assume fail error
30753 00005154 98           cbw     ;sign extend to word
30754
30755                      ; 21/02/2024
30756 %if 0
30757                      ; PCDOS 7.1 IBMDOS.COM
30758                      ; cbw
30759                      inc     ax          ; 21/02/2024 - Erdogan Tan
30760                      ; this may be a BUG
30761                      ; MSDOS 6.22 MSDOS.SYS & win ME IO.SYS code is
30762                      ; cbw
30763                      ; cmp     ax,-1
30764                      ; jne     short not_fail_ret
30765                      ; inc     byte [ss:IoStatFail]
30766                      ; popf
30767                      ; retn
30768                      jnz     short not_fail_ret
30769                      dec     ax
30770                      jnz     short not_fail_ret ; jns ?
30771 %else
30772                      ;;cbw
30773                      ;cmp     ax,-1          ; (MSDOS 6.22 & windows ME)
30774                      ;jne     short not_fail_ret
30775                      ; 27/06/2024
30776 00005155 3CFF        cmp     al,0FFh ; -1
30777 00005157 7507        jne     short not_fail_ret
30778 %endif
30779
30780 00005159 36FE06[8300]  inc     byte [ss:IoStatFail]
30781 0000515E 9D           popf
30782 0000515F C3           retn
30783
30784 not_fail_ret:
30785 00005160 36A1[BC03]    mov     ax,[ss:DEVIOBUF] ;ss override
30786 00005164 9D           popf
30787 00005165 C3           retn
30788
30789 DevErr:
30790 00005166 88CC        MOV     AH,CL
30791 00005168 E8B30E        call    CHARHARD
30792 0000516B 3C01        CMP     AL,1
30793 0000516D 7507        JNZ     short NO_RETRY
30794 0000516F E8CFB2        call    restore_world
30795                      ; 12/05/2019
30796 00005172 07           pop     es ; * (MSDOS 6.21)
30797 00005173 E94EFF        JMP     IOFUNC ; 10/08/2018
30798
30799 NO_RETRY:
30800                      ; Know user must have wanted Ignore OR Fail. Make sure device shows ready
30801                      ; ready so that DOS doesn't get caught in a status loop when user
30802                      ; simply wants to ignore the error.
30803                      ;
30804                      ; SR; If fail wanted by user set ax to special value (ax = -1). This
30805                      ; should be checked by the caller on return
30806
30807                      ; SS override
30808 00005176 368026[8003]FD and     byte [SS:IOCALL_REQSTAT+1],0FDh
30809                      ;AND     BYTE [SS:IOCALL_REQSTAT+1],~(STBUI>>8)
30810
30811                      ; SR;
30812                      ; Check if user failed
30813
30814                      ; MSDOS 6.0
30815 0000517C 3C03        cmp     al,3
30816 0000517E 7505        jnz     short not_fail
30817 00005180 36FE0E[8300]  dec     byte [ss:IoStatFail] ;set flag indicating fail on I24
30818 not_fail:
30819 00005185 EBAC        JMP     short OKDevIO
30820
30821 IOTOFILE:
30822 00005187 08E4        OR     AH,AH
30823 00005189 7421        JZ     short IOIN
30824 0000518B FECC        DEC     AH
30825 0000518D 7405        JZ     short IOIST
30826 0000518F FECC        DEC     AH
30827 00005191 7411        JZ     short IOUT
30828 IOUT_retn: ; 18/12/2022
30829 00005193 C3           retn          ; NON ZERO FLAG FOR OUTPUT STATUS
30830
30831 IOIST:
30832 00005194 FF7415        ;push word [si+15h]
30833                      PUSH    WORD [SI+SF_ENTRY.sf_position] ; Save position
30834                      ;push word [si+17h]
30835 0000519A E80F00        PUSH    WORD [SI+SF_ENTRY.sf_position+2]
30836                      CALL    IOIN
30837 0000519D 8F4417        ;pop word [si+17h]
30838                      POP     WORD [SI+SF_ENTRY.sf_position+2] ; Restore position
30839                      ;pop word [si+15h]
30840 000051A3 C3           POP     WORD [SI+SF_ENTRY.sf_position]
30841                      retn
30842 IOUT:
30843 000051A4 E82500        CALL    SETXADDR
30844 000051A7 E8CAEB        call    DOS_WRITE
30845                      ;CALL RESTXADDR ; If you change this into a jmp don't
30846                      ; 18/12/2022
30847 000051AA EB4F        jmp     RESTXADDR
30848 ;IOUT_retn:
30849                      ;retn          ; come crying to me when things don't
30850                      ; work ARR
30851
30852 IOIN:
30853 000051AC E81D00        CALL    SETXADDR
30854                      ; SS override for DOS34_FLAG
30855                      ;OR     word [SS:DOS34_FLAG],Disable_EOF_I24 ;AN000;
30856                      ;or     word [ss:DOS34_FLAG],40h
30857                      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
30858                      ; 16/12/2022
30859 000051AF 36800E[1106]40 or     byte [ss:DOS34_FLAG],40h
30860 000051B5 E8BBE9        CALL    DOS_READ
30861                      ;AND     word [SS:DOS34_FLAG],NO_Disable_EOF_I24 ;AN000;
30862                      ;and     word [SS:DOS34_FLAG],0FFBFh
30863                      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)

```

```

30862          ; 16/12/2022
30863 000051B8 368026[1106]BF and byte [SS:DOS34_FLAG],0BFh ; 07/12/2022
30864 000051BE 09C9 OR CX,CX ; Check EOF
30865 000051C0 E83800 CALL RESTXADDR
30866          ; SS override
30867 000051C3 36A0[BC03] MOV AL,[SS:DEVI0BUF] ; Get byte from trans addr
30868 000051C7 75CA jnz short IOUT_retn
30869 000051C9 B01A MOV AL,1AH ; ^Z if no bytes
30870 000051CB C3 retn
30871
30872 SETXADDR:
30873          ; SS override
30874 000051CC 368F06[6C03] POP WORD [SS:CALLSCNT] ; Return address
30875          ;
30876 000051D1 06 push es ; * (MSDOS 6.21)
30877          ;
30878 000051D2 E883B2 call save_world
30879          ; SS override for DMAADD and THISSFT
30880          ; 24/09/2023
30881          ; PUSH WORD [SS:DMAADD] ; Save Disk trans addr
30882          ; PUSH WORD [SS:DMAADD+2]
30883 000051D5 368C1E[A005] MOV [SS:THISSFT+2],DS
30884
30885          ; 22/02/2024
30886 %if 0
30887 push ss
30888 pop ds
30889
30890          ; 24/09/2023
30891 push word [DMAADD]
30892 push word [DMAADD+2]
30893
30894 MOV [THISSFT],SI ; Finish setting SFT pointer
30895 MOV CX,[IOXAD+2]
30896 MOV [DMAADD+2],CX
30897 MOV CX,[IOXAD]
30898 MOV [DMAADD],CX ; Set byte trans addr
30899
30900 %else
30901          ; 22/02/2024
30902          ; PCDOS 7.1 IBMDOS.COM
30903 000051DA 36C50E[2C03] lds cx,[ss:DMAADD] ; Save Disk transfer address
30904 000051DF 51 push cx
30905 000051E0 1E push ds
30906 000051E1 36C50E[8A03] lds cx,[ss:IOXAD] ; Set byte trans address
30907 000051E6 368C1E[2E03] mov [ss:DMAADD+2],ds
30908 000051EB 16 push ss
30909 000051EC 1F pop ds
30910 000051ED 890E[2C03] mov [DMAADD],cx
30911 000051F1 8936[9E05] mov [THISSFT],si
30912
30913 %endif
30914 000051F5 8B0E[8E03] MOV CX,[IOSCNT] ; ioscnt specifies length of buffer
30915 000051F9 EB10 JMP SHORT RESTRET ; RETURN ADDRESS
30916
30917 RESTXADDR:
30918 000051FB 8F06[6C03] POP WORD [CALLSCNT] ; Return address
30919 000051FF 8F06[2E03] POP WORD [DMAADD+2] ; Restore Disk trans addr
30920 00005203 8F06[2C03] POP WORD [DMAADD]
30921
30922 call restore_world
30923
30924 pop es ; * (MSDOS 6.21)
30925          ; SS override
30926 0000520B 36FF26[6C03] RESTRET:
30927 JMP WORD [SS:CALLSCNT] ; Return address
30928
30929 ; DOSCODE:8569h (MSDOS 6.21, MSDOS.SYS)
30930 ; 21/11/2022
30931 ; DOSCODE:852Eh (MSDOS 5.0, MSDOS.SYS)
30932
30933 ; 22/02/2024 - Retro DOS v5.0
30934 ; DOSCODE:92CAh (PCDOS 7.1, IBMDOS.COM)
30935
30936 ;Break <DEV_OPEN_SFT, DEV_CLOSE_SFT - OPEN or CLOSE A DEVICE>
30937
30938 ;-----
30939 ;** Dev_Open_SFT - Open the Device for an SFT
30940 ;
30941 ; Dev_Open_SFT issues an open call to the device associated with
30942 ; the SFT.
30943 ;
30944 ; ENTRY (ES:DI) = SFT
30945 ; EXIT none
30946 ; USES all
30947 ;-----
30948
30949 DEV_OPEN_SFT:
30950 push es ; * (MSDOS 6.21)
30951 call save_world
30952 ;mov al,0Dh
30953 MOV AL,DEVOPN
30954 JMP SHORT DO_OPCLS
30955
30956 ;-----
30957 ; Procedure Name : DEV_CLOSE_SFT
30958 ;
30959 ; Inputs:
30960 ; ES:DI Points to SFT
30961 ; Function:
30962 ; Issue a CLOSE call to the correct device
30963 ; Outputs:
30964 ; None
30965 ; ALL preserved
30966 ;-----
30967
30968 DEV_CLOSE_SFT:
30969 push es ; * (MSDOS 6.21)
30970 call save_world
30971 ;mov al,0Eh
30972 MOV AL,DEVCLS
30973
30974 ; Main entry for device open and close. AL contains the function
30975 ; requested. Subtlety: if Sharing is NOT loaded then we do NOT issue
30976 ; open/close to block devices. This allows networks to function but
30977 ; does NOT hang up with bogus change-line code.
30978
30979 ;entry DO_OPCLS
30980 DO_OPCLS:
30981 ; Is the SFT for the net? If so, no action necessary.
30982
30983 ; MSDOS 6.0
30984 ;test word [es:di+5],8000h
30985 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
30986 test byte [es:di+SF_ENTRY.sf_flags+1],(sf_isnet>>8)

```

```

30986 00005223 7564      jnz     short OPCLS_DONE      ; NOP on net SFTs
30987 00005225 30E4      XOR      AH,AH                ; Unit
30988                      ;test   byte [es:di+5],80h
30989 00005227 26F6450580 TEST     byte [ES:DI+SF_ENTRY.sf_flags],devid_device
30990                      ;les     di,[es:di+7]
30991 0000522C 26C47D07      LES      DI,[ES:DI+SF_ENTRY.sf_devptr] ; Get DPB or device
30992 00005230 7511      JNZ      short GOT_DEV_ADDR
30993
30994                      ; We are about to call device open/close on a block driver. If no
30995                      ; sharing then just short circuit to done.
30996
30997                      ; MSDOS 6.0
30998                      ; SS override
30999 00005232 36803E[0303]01 CMP      byte [ss:fshare],1      ;AN010; /NC or no SHARE
31000 00005238 764F      JBE      short OPCLS_DONE      ;AN010; yes
31001
31002                      ; 22/02/2024
31003 %if 0
31004                      ; MSDOS 3.3 (& MSDOS 6.0)
31005                      ;mov     ah,[es:di+1]
31006                      MOV      AH,[ES:DI+DPB.UNIT] ; (ah) = unit
31007                      mov     cl,[es:di]
31008                      ;MOV     CL,[ES:DI+DPB.DRIVE] ; (cl) = drive
31009 %else
31010                      ; 22/02/2024 - Retro DOS v5.0
31011                      ;mov     cx,[es:di+DPB.DRIVE]
31012 0000523A 268B0D      mov     cx,[es:di]
31013 0000523D 88EC      mov     ah,ch                ; AH = unit
31014                      ; CL = drive
31015 %endif
31016                      ;;les     di,[es:di+12h] ; MSDOS 3.3
31017                      ;les     di,[es:di+13h] ; MSDOS 6.0
31018 0000523F 26C47D13      LES      DI,[ES:DI+DPB.DRIVER_ADDR] ; Get device
31019 GOT_DEV_ADDR:          ; ES:DI -> device
31020                      ;test     word [es:di+4],800h
31021                      ;TEST     word [ES:DI+SYSDEV.ATT],DEVOPCL
31022 00005243 26F6450508 test     byte [ES:DI+SYSDEV.ATT+1],(DEVOPCL>>8)
31023 00005248 743F      JZ      short OPCLS_DONE      ; Device can't
31024 0000524A 06          PUSH     ES
31025 0000524B 1F          POP      DS
31026 0000524C 89FE      MOV      SI,DI                ; DS:SI -> device
31027
31028 OPCLS_RETRY:
31029                      ;Context ES
31030 0000524E 16          push     ss
31031 0000524F 07          pop      es
31032                      ; DEVCALL is in DOSDATA
31033 00005250 BF[5A03]      MOV      DI,DEVCALL
31034
31035 00005253 89FB      MOV      BX,DI
31036 00005255 50          PUSH     AX
31037                      ;mov     al,13
31038 00005256 B00D      MOV      AL,DOPCLHL
31039 00005258 AA          STOSB
31040 00005259 58          POP      AX                ; Length
31041
31042 0000525A 86E0      XCHG     AH,AL
31043                      ;STOSB
31044                      ; 22/02/2024 (PCDOS 7.1 IBMDOS.COM)
31045 0000525C AB          stosw
31046 0000525D 86E0      XCHG     AH,AL                ; Unit, Command
31047                      ;STOSB
31048                      ; Command
31049 0000525F 26C7050000 MOV      WORD [ES:DI],0        ; Status
31050 00005264 50          PUSH     AX                ; Save Unit,Command
31051                      ;invoke DEVIOCALL2
31052 00005265 E82900      call     DEVIOCALL2
31053
31054                      ;mov     di,[es:bx+3]
31055 00005268 268B7F03      MOV      DI,[ES:BX+SRHEAD.REQUEST]
31056                      ;test     di,8000h
31057                      ;jz      short OPCLS_DONEP
31058 0000526C 21FF      and     di,di
31059 0000526E 7918      jns     short OPCLS_DONEP      ; No error
31060                      ; 21/11/2022
31061                      ;test     word [si+4],8000h
31062                      ;TEST     word [SI+SYSDEV.ATT],DEVTYP
31063                      ;test     word [si+5],80h
31064 00005270 F6440580 test     byte [SI+SYSDEV.ATT+1],(DEVTYP>>8)
31065 00005274 7404      JZ      short BLKDEV
31066 00005276 B486      MOV      AH,86h
31067 00005278 EB04      JMP      SHORT HRDERR          ; Read error in data, Char dev
31068
31069 0000527A 88C8      MOV      AL,CL                ; Drive # in AL
31070 0000527C B406      MOV      AH,6                ; Read error in data, Blk dev
31071
31072 HRDERR:
31073 0000527E E89D0D      ;invoke CHARHARD
31074 00005281 3C01      call     CHARHARD
31075 00005283 7503      cmp     al,1
31076                      jne     short OPCLS_DONEP      ; IGNORE or FAIL
31077 00005285 58          POP      AX                ; Note that FAIL is essentially IGNORED
31078 00005286 EBC6      JMP      short OPCLS_RETRY      ; Get back Unit, Command
31079
31080 00005288 58          OPCLS_DONEP:
31081                      POP      AX                ; Clean stack
31082 00005289 E8B5B1      OPCLS_DONE:
31083 0000528C 07          call     restore_world
31084 0000528D C3          pop     es ; * (MSDOS 6.21)
31085                      retn
31086
31087                      ; 30/04/2019 - Retro DOS v4.0
31088                      ; DOSCODE:85EAh (MSDOS 6.21, MSDOS.SYS)
31089                      ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31090                      ; DOSCODE:85AFh (MSDOS 5.0, MSDOS.SYS)
31091
31092                      ; 22/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
31093                      ; DOSCODE:9348h (PCDOS 7.1, IBMDOS.COM)
31094
31095                      ;Break <DEVIOCALL, DEVIOCALL2 - CALL A DEVICE>
31096
31097                      ;-----
31098                      ;** DevIoCall - Call Device
31099                      ;
31100                      ; ENTRY DS:SI Points to device SFT
31101                      ; ES:BX Points to request data
31102                      ; EXIT DS:SI -> Device driver
31103                      ; USES DS:SI,AX
31104                      ;-----
31105                      ;** DevIoCall2 - Call Device
31106                      ;
31107                      ; ENTRY DS:SI Points to DPB
31108                      ; ES:BX Points to request data
31109                      ; EXIT DS:SI -> Device driver
31110                      ; USES DS:SI,AX

```

```

31110 ;-----
31111
31112 DEVIOCALL:
31113 ; SS override for CALLSSEC,
31114 ;lds si,[si+7] ; CALLNEWSC, HIGH_SECTOR & CALLDEVAD
31115 LDS SI,[SI+SF_ENTRY.sf_devptr]
31116
31117 ;entry DEVIOCALL2
31118 DEVIOCALL2:
31119 ;EnterCrit critDevice
31120 call ECritDevice
31121
31122 ; MSDOS 6.0
31123 ;TEST word [SI+SYSDEV.ATT],DEVTYP ;AN000; >32mb block device ?
31124 ;test byte [si+5],80h
31125 test byte [si+SYSDEV.ATT+1],(DEVTYP>>8)
31126 jnz short chardev2 ;AN000; >32mb no
31127
31128 ; 16/12/2022
31129 ; 22/11/2022
31130 mov al,[ES:BX+SRHEAD.REQFUNC] ; [es:bx+2]
31131 cmp al,DEVVRD ; 4
31132 je short chkext
31133 cmp al,DEVWRT ; 8
31134 je short chkext
31135 cmp al,DEVWRTV ; 9
31136 jne short chardev2
31137
31138 ; 16/12/2022
31139 ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
31140 ;cmp byte [es:bx+2],4
31141 ;CMP byte [ES:BX+SRHEAD.REQFUNC],DEVVRD ;AN000; >32mb read ?
31142 ;JZ short chkext ;AN000; >32mb yes
31143 ;cmp byte [es:bx+2],8
31144 ;CMP byte [ES:BX+SRHEAD.REQFUNC],DEVWRT ;AN000; >32mb write ?
31145 ;JZ short chkext ;AN000; >32mb yes
31146 ;cmp byte [es:bx+2],9
31147 ;CMP byte [ES:BX+SRHEAD.REQFUNC],DEVWRTV
31148 ; ;AN000; >32mb write/verify ?
31149 ;JNZ short chardev2 ;AN000; >32mb no
31150
31151 chkext:
31152 ; 22/02/2024 - Retro DOS v5.0 (PCDOS 7.1)
31153 %if 0
31154 CALL RW_SC ;AN000;LB. use secondary cache if there
31155 JC short dev_exit ;AN000;LB. done
31156 %endif
31157
31158 ;test byte [si+4],2
31159 TEST byte [SI+SYSDEV.ATT],EXTDRV ;AN000;>32mb extended driver?
31160 JZ short chksector ;AN000;>32mb no
31161 ADD BYTE [ES:BX],8 ;AN000;>32mb make length to 30
31162
31163 ;MOV AX,[SS:CALLSSEC] ;AN000;>32mb
31164 ;MOV word [SS:CALLSSEC],-1 ;AN000;>32mb old sector ==-1
31165 ; 22/02/2024
31166 mov ax,-1 ; 0FFFFh
31167 xchg ax,[ss:CALLSSEC]
31168
31169 MOV [SS:CALLNEWSC],AX ;AN000;>32mb new sector =
31170 MOV AX,[SS:HIGH_SECTOR] ;AN000; >32mb low sector,high sector
31171 MOV [SS:CALLNEWSC+2],AX ;AN000; >32mb
31172 JMP short chardev2 ;AN000; >32mb
31173
31174
31175 chksector:
31176 CMP word [SS:HIGH_SECTOR],0 ;AN000; >32mb
31177 JZ short chardev2 ;AN000; >32mb if >32mb
31178 ;mov word [es:bx+3],8107h
31179 MOV word [ES:BX+SRHEAD.REQSTAT],STERR+STDON+error_I24_not_DOS_disk
31180 ;AN000; >32mb
31181 JMP SHORT dev_exit ;AN000; >32mb
31182
31183 chardev2:
31184 ;AN000;
31185 ; As above only DS:SI points to device header on entry, and DS:SI is
31186 ; preserved
31187
31188 ;;;
31189 ; 22/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
31190 inc byte [ss:DEVIO_IN_PROGRESS] ; lock (deviocal1 in progress)
31191 ;;;
31192
31193 ;mov ax,[si+6]
31194 MOV AX,[SI+SYSDEV.STRAT]
31195 MOV [SS:CALLDEVAD],AX
31196 MOV [SS:CALLDEVAD+2],DS
31197 CALL far [SS:CALLDEVAD]
31198
31199 ;mov ax,[si+8]
31200 MOV AX,[SI+SYSDEV.INT]
31201 MOV [SS:CALLDEVAD],AX
31202 CALL far [SS:CALLDEVAD]
31203
31204 ;;;
31205 ; 22/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
31206 dec byte [ss:DEVIO_IN_PROGRESS] ; unlock (deviocal1 completed)
31207 ;;;
31208
31209 ; 22/02/2024 - Retro DOS v5.0 (PCDOS 7.1)
31210 %if 0
31211 ; MSDOS 6.0
31212 CALL VIRREAD ;AN000;LB. move data from SC to buffer
31213 JC short chardev2 ;AN000;LB. bad sector or exceeds max sec
31214 %endif
31215
31216 dev_exit:
31217 ;LeaveCrit critDevice
31218 ;call LCritDevice
31219 ;retn
31220 ; 18/12/2022
31221 jmp LCritDevice
31222
31223 ; DOSCODE:8669h (MSDOS 6.21, MSDOS.SYS)
31224 ; 22/11/2022
31225 ; DOSCODE:862Eh (MSDOS 5.0, MSDOS.SYS)
31226
31227 ;Break <SETREAD, SETWRITE -- SET UP HEADER BLOCK>
31228 ;-----
31229 ;
31230 ; Procedure Name : SETREAD, SETWRITE
31231 ;
31232 ; Inputs:
31233 ; DS:BX = Transfer Address
31234 ; CX = Record Count

```

```

31234 ; DX = Starting Record
31235 ; AH = Media Byte
31236 ; AL = Unit Code
31237 ; Function:
31238 ; Set up the device call header at DEVCALL
31239 ; Output:
31240 ; ES:BX Points to DEVCALL
31241 ; No other registers effected
31242 ;
31243 ;-----
31244
31245 SETREAD_XJ:
31246 ;;;
31247 ; 07/02/2024 - Retro DOS v5.0
31248 mov     bx,di
31249 jmp     short SETREAD_X
31250 ;;;
31251
31252 SETREAD_XT:
31253 ;;;
31254 ; 07/02/2024 - Retro DOS v5.0
31255 mov     bx,TIMEBUF
31256 push    bx
31257 SETREAD_XTC:
31258 mov     cx,6
31259 ;;;
31260 SETREAD_X:
31261 ;;;
31262 ; 06/02/2024 - Retro DOS v5.0
31263 xor     ax,ax
31264 ;mov     dx,ax ; 0
31265 cwd
31266 ;;;
31267
31268 ; -----
31269
31270 SETREAD:
31271 PUSH    DI
31272 PUSH    CX
31273 PUSH    AX
31274 MOV     CL,DEV RD ; mov cl,4
31275 SETCALLHEAD:
31276 MOV     AL,DRDWRHL ; mov al,16h
31277 PUSH    SS
31278 POP     ES
31279
31280 MOV     DI,DEVCALL ; DEVCALL is in DOSDATA
31281
31282 STOSB ; length
31283 POP     AX ;
31284 STOSB ; Unit
31285 PUSH    AX
31286 MOV     AL,CL
31287 STOSB ; Command code
31288 XOR     AX,AX
31289 STOSW ; Status
31290 ADD     DI,8 ; Skip link fields
31291 POP     AX
31292 XCHG    AH,AL
31293 STOSB ; Media byte
31294 XCHG    AL,AH
31295 PUSH    AX
31296 MOV     AX,BX
31297 STOSW
31298
31299 MOV     AX,DS
31300 STOSW ; Transfer addr
31301
31302 POP     CX ; Real AX
31303 POP     AX ; Real CX
31304 STOSW ; Count
31305
31306 XCHG    AX,DX ; AX=Real DX, DX=real CX, CX=real AX
31307 STOSW ; Start
31308 XCHG    AX,CX
31309 XCHG    DX,CX
31310 POP     DI
31311
31312 MOV     BX,DEVCALL ; DEVCALL is in DOSDATA
31313 retn
31314
31315 ;entry SETWRITE
31316 SETWRITE:
31317 ; Inputs:
31318 ; DS:BX = Transfer Address
31319 ; CX = Record Count
31320 ; DX = Starting Record
31321 ; AH = Media Byte
31322 ; AL = Unit Code
31323 ; Function:
31324 ; Set up the device call header at DEVCALL
31325 ; Output:
31326 ; ES:BX Points to DEVCALL
31327 ; No other registers effected
31328
31329 PUSH    DI
31330 PUSH    CX
31331 PUSH    AX
31332 MOV     CL,DEVWRT ; mov cl,8
31333 ADD     CL,[SS:VERFLG] ; SS override
31334 JMP     SHORT SETCALLHEAD
31335
31336 ; 22/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
31337 %if 0 ; SC
31338
31339 ; 30/04/2019 - Retro DOS v4.0
31340 ; DOSCODE:86A8h (MSDOS 6.21, MSDOS.SYS)
31341 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31342 ; DOSCODE:866Dh (MSDOS 5.0, MSDOS.SYS)
31343
31344 ;Break <RW_SC -- Read Write Secondary Cache>
31345 ;-----
31346 ;
31347 ;
31348 ; Procedure Name : RW_SC
31349 ;
31350 ; Inputs:
31351 ; [SC_CACHE_COUNT]= secondary cache count
31352 ; [SC_STATUS]= SC validity status
31353 ; [SEQ_SECTOR]= last sector read
31354 ; Function:
31355 ; Read from or write through secondary cache
31356 ; Output:
31357 ; ES:BX Points to DEVCALL

```

```

31358 ; carry clear, I/O is not done
31359 ; [SC_FLAG]=1 if continuous sectors will be read
31360 ; carry set, I/O is done
31361 ;
31362 ;-----
31363
31364 RW_SC:
31365 ; SS override for all variables used.
31366
31367 CMP word [ss:SC_CACHE_COUNT],0 ;AN000;LB. secondary cache exists?
31368 ;JZ short scexit4 ;AN000;LB. no, do nothing
31369 ; 04/02/2026
31370 JZ short scexit5
31371 CMP word [ss:CALLSCNT],1 ;AN000;LB. sector count = 1 (buffer I/O)
31372 JNZ short scexit4 ;AN000;LB. no, do nothing
31373 PUSH CX ;AN000;LB.
31374 PUSH DX ;AN000;LB. yes
31375 PUSH DS ;AN000;LB. save registers
31376 PUSH SI ;AN000;LB.
31377 PUSH ES ;AN000;LB.
31378 PUSH DI ;AN000;LB.
31379
31380 MOV DX,[ss:CALLSSEC] ;AN000;LB. starting sector
31381 CMP BYTE [ss:DEVCALL_REQFUNC],DEV RD ;AN000;LB. read ?
31382 JZ short doread ;AN000;LB. yes
31383 CALL INVALIDATE_SC ;AN000;LB. invalidate SC
31384 ;JMP scexit2 ;AN000;LB. back to normal
31385 ; 04/02/2026
31386 CLC
31387 JMP scexit
31388 scexit4: ;AN000;
31389 CLC ;AN000;LB. I/O not done yet
31390 scexit5: ; 04/02/2026
31391 RETN ;AN000;LB.
31392 doread: ;AN000;
31393 CALL SC2BUF ;AN000;LB. check if in SC
31394 JC short readSC ;AN000;LB.
31395 MOV word [ss:DEVCALL_REQSTAT],STDON ;AN000;LB. fake done and ok
31396 STC ;AN000;LB. set carry
31397 JMP short saveseq ;AN000;LB. save seq. sector #
31398 readSC: ;AN000;
31399 MOV AX,[ss:HIGH_SECTOR] ;AN000;LB. subtract sector num from
31400 MOV CX,[ss:CALLSSEC] ;AN000;LB. saved sequential sector
31401 SUB CX,[ss:SEQ_SECTOR] ;AN000;LB. number
31402 SBB AX,[ss:SEQ_SECTOR+2] ;AN000;LB.
31403 ; 24/09/2023
31404 ;CMP AX,0 ;AN000;LB. greater than 64K
31405 JNZ short saveseq2 ;AN000;LB. yes,save seq. sector #
31406 chklow:
31407 CMP CX,1 ;AN000;LB. <= 1
31408 ;JA short saveseq2 ;AN000;LB. no, not sequential
31409 ; 04/02/2026
31410 JA short saveseq
31411 MOV word [ss:SC_STATUS],-1 ;AN000;LB. presume all SC valid
31412 MOV AX,[ss:SC_CACHE_COUNT] ;AN000;LB. yes, sequential
31413 MOV [ss:CALLSCNT],AX ;AN000;LB. read continuous sectors
31414 readsr:
31415 MOV AX,[ss:CALLXAD+2] ;AN000;LB. save buffer addr
31416 MOV [ss:TEMP_VAR2],AX ;AN000;LB. in temp vars
31417 MOV AX,[ss:CALLXAD] ;AN000;LB.
31418 MOV [ss:TEMP_VAR],AX ;AN000;LB.
31419
31420 MOV AX,[ss:SC_CACHE_PTR] ;AN000;LB. use SC cache addr as
31421 MOV [ss:CALLXAD],AX ;AN000;LB. transfer addr
31422 MOV AX,[ss:SC_CACHE_PTR+2] ;AN000;LB.
31423 MOV [ss:CALLXAD+2],AX ;AN000;LB.
31424 MOV byte [ss:SC_FLAG],1 ;AN000;LB. flag it for later;
31425 MOV AL,[ss:SC_DRIVE] ;AN000;LB. current drive
31426 MOV [ss:CurSC_DRIVE],AL ;AN000;LB. set current drive
31427 MOV AX,[ss:CALLSSEC] ;AN000;LB. current sector
31428 MOV [ss:CurSC_SECTOR],AX ;AN000;LB. set current sector
31429 MOV AX,[ss:HIGH_SECTOR] ;AN000;LB.
31430 MOV [ss:CurSC_SECTOR+2],AX ;AN000;LB.
31431 saveseq2: ;AN000;
31432 CLC ;AN000;LB. clear carry
31433 saveseq: ;AN000;
31434 MOV AX,[ss:HIGH_SECTOR] ;AN000;LB. save current sector #
31435 MOV [ss:SEQ_SECTOR+2],AX ;AN000;LB. for access mode ref.
31436 MOV AX,[ss:CALLSSEC] ;AN000;LB.
31437 MOV [ss:SEQ_SECTOR],AX ;AN000;LB.
31438 ; 04/02/2026
31439 ;JMP short scexit ;AN000;LB.
31440 ;scexit2: ;AN000;LB.
31441 ;CLC ;AN000;LB. clear carry
31442 scexit: ;AN000;
31443 POP DI ;AN000;LB.
31444 POP ES ;AN000;LB. restore registers
31445 POP SI ;AN000;LB.
31446 POP DS ;AN000;LB.
31447 POP DX ;AN000;LB.
31448 POP CX ;AN000;LB.
31449 RETN ;AN000;LB.
31450
31451 ;Break <IN_SC -- check if in secondary cache>
31452 ;-----
31453 ;
31454 ; Procedure Name : IN_SC
31455 ;
31456 ; Inputs: [SC_DRIVE]= requesting drive
31457 ; [CURSC_DRIVE]= current SC drive
31458 ; [CURSC_SECTOR]= starting sector # of SC
31459 ; [SC_CACHE_COUNT]= SC count
31460 ; [HIGH_SECTOR]:DX= sector number
31461 ; Function:
31462 ; Check if the sector is in secondary cache
31463 ; Output:
31464 ; carry clear, in SC
31465 ; CX= the index in the secondary cache
31466 ; carry set, not in SC
31467 ;-----
31468
31469 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31470 IN_SC:
31471 ; SS override for all variables used
31472 MOV AL,[ss:SC_DRIVE] ;AN000;LB. current drive
31473 CMP AL,[ss:CurSC_DRIVE] ;AN000;LB. same as SC drive
31474 JNZ short outrange2 ;AN000;LB. no
31475 MOV AX,[ss:HIGH_SECTOR] ;AN000;LB. subtract sector num from
31476 MOV CX,DX ;AN000;LB. secondary starting sector
31477 SUB CX,[ss:CurSC_SECTOR] ;AN000;LB. number
31478 SBB AX,[ss:CurSC_SECTOR+2] ;AN000;LB.
31479 ; 24/09/2023
31480 ;CMP AX,0 ;AN000;LB. greater than 64K
31481

```



```

31482      JNZ      short outrange2                ;AN000;;LB. yes
31483      CMP      CX,[ss:SC_CACHE_COUNT]        ;AN000;;LB. greater than SC count
31484      JAE      short outrange2                ;AN000;;LB. yes
31485      CLC
31486      ;JMP      short inexit                    ;AN000;;LB. clear carry
31487      ; 16/12/2022
31488      retn    ; 30/04/2019
31489      ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
31490      ;jmp      short inexit
31491
31492      outrange2:                                ;AN000;;LB. set carry
31493      STC
31494      inexit:                                ;AN000;;LB.
31495      retn                                    ;AN000;;LB.
31496
31497      ;Break      <INVALIDATE_SC - invalide secondary cache>
31498      ;-----
31499      ;
31500      ; Procedure Name : Invalidate_Sc
31501      ;
31502      ; Inputs:  [SC_DRIVE]= requesting drive
31503      ;           [CURSC_DRIVE]= current SC drive
31504      ;           [CURSC_SECTOR]= starting scetor # of SC
31505      ;           [SC_CACHE_COUNT]= SC count
31506      ;           [SC_STATUS]= SC status word
31507      ;           [HIGH_SECTOR]:DX= sector number
31508      ;
31509      ; Function:
31510      ;   invalidate secondary cache if in there
31511      ; Output:
31512      ;   [SC_STATUS] is updated
31513      ;-----
31514      ;
31515      INVALIDATE_SC:
31516      ; SS override for all variables used
31517      ;
31518      CALL      IN_SC                            ;AN000;;LB. in secondary cache
31519      JC        short outrange                    ;AN000;;LB. no
31520      MOV      AX,1                              ;AN000;;LB. invalidate the sector
31521      SHL      AX,CL                             ;AN000;;LB. in the secondary cache
31522      NOT      AX
31523      AND      [ss:SC_STATUS],AX                 ;AN000;;LB. save the status
31524      outrange:
31525      retn                                    ;AN000;;LB.
31526
31527      ; DOSCODE:87A5h (MSDOS 6.21, MSDOS.SYS)
31528      ; 22/11/2022
31529      ; DOSCODE:876Ah (MSDOS 5.0, MSDOS.SYS)
31530
31531      ;Break      <VIRREAD- virtually read data into buffer>
31532      ;-----
31533      ;
31534      ; Procedure Name : SC_FLAG
31535      ;
31536      ; Inputs:  SC_FLAG = 0, no sectors were read into SC
31537      ;           1, continuous sectors were read into SC
31538      ;
31539      ; Function:
31540      ;   Move data from SC to buffer
31541      ; Output:
31542      ;   carry clear, data is moved to buffer
31543      ;   carry set, bad sector or exceeds maximum sector
31544      ;   SC_FLAG =0
31545      ;   CALLSCNT=1
31546      ;   SC_STATUS= -1 if succeeded
31547      ;
31548      ;           0 if failed
31549      ;-----
31550      ;
31551      VIRREAD:
31552      ; SS override for all variables used
31553      ;
31554      CMP      byte [ss:SC_FLAG],0                ;AN000;;LB. from SC fill
31555      JZ        short sc2end                      ;AN000;;LB. no
31556      ; 04/02/2026
31557      JZ        short sc2end2
31558      MOV      AX,[ss:TEMP_VAR2]                  ;AN000;;LB. restore buffer addr
31559      MOV      [ss:CALLXAD+2],AX                 ;AN000;;LB.
31560      MOV      AX,[ss:TEMP_VAR]                  ;AN000;;LB.
31561      MOV      [ss:CALLXAD],AX                   ;AN000;;LB.
31562      MOV      byte [ss:SC_FLAG],0                ;AN000;;LB. reset sc_flag
31563      MOV      word [ss:CALLSCNT],1              ;AN000;;LB. one sector transferred
31564
31565      ;TEST      word [ss:DEVCALL_REQSTAT],STERR ;AN000;;LB. error?
31566      test     byte [ss:DEVCALL_REQSTAT+1],(STERR>>8) ; 80h
31567      JNZ      short scerror                    ;AN000;;LB. yes
31568      PUSH     DS
31569      PUSH     SI
31570      PUSH     ES
31571      PUSH     DI
31572      PUSH     DX
31573      PUSH     CX
31574      XOR      CX,CX                             ;AN000;;LB. we want first sector in SC
31575      CALL     SC2BUF2                            ;AN000;;LB. move data from SC to buf
31576      POP      CX
31577      POP      DX
31578      POP      DI
31579      POP      ES
31580      POP      SI
31581      POP      DS
31582      ;JMP      SHORT sc2end                      ;AN000;;LB. return
31583      ; 04/02/2026
31584      sc2end:
31585      CLC
31586      sc2end2:
31587      retn                                    ;AN000;;LB.
31588
31589      scerror:
31590      MOV      word [ss:CALLSCNT],1              ;AN000;;LB. reset sector count to 1
31591      MOV      word [ss:SC_STATUS],0             ;AN000;;LB. invalidate all SC sectors
31592      MOV      byte [ss:CurSC_DRIVE],-1         ;AN000;;LB. invalidate drive
31593      STC
31594      retn                                    ;AN000;;LB.
31595
31596      ; 30/04/2019 - Retro DOS v4.0
31597      ; DOSCODE:87FDh (MSDOS 6.21, MSDOS.SYS)
31598      ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31599      ; DOSCODE:87C2h (MSDOS 5.0, MSDOS.SYS)
31600
31601      ;Break      <SC2BUF- move data from SC to buffer>
31602      ;-----
31603      ;
31604      ; Procedure Name : SC2BUF
31605      ;
31606      ; Inputs:  [SC_STATUS] = SC validity status

```

```

31606 ; [SC_SECTOR_SIZE] = request sector size
31607 ; [SC_CACHE_PTR] = pointer to SC
31608 ; Function:
31609 ; Move data from SC to buffer
31610 ; Output:
31611 ; carry clear, in SC and data is moved
31612 ; carry set, not in SC and data is not moved
31613 ;-----
31614
31615 SC2BUF:
31616 ; SS override for all variables used
31617 CALL IN_SC ;AN000;LB. in secondary cache
31618 ;JC short noSC ;AN000;LB. no
31619 ; 24/09/2023
31620 JC short sextit
31621 MOV AX,1 ;AN000;LB. check if valid sector
31622 SHL AX,CL ;AN000;LB. in the secondary cache
31623 TEST [ss:SC_STATUS],AX ;AN000;LB.
31624 JZ short noSC ;AN000;LB. invalid
31625 ;entry SC2BUF2
31626 SC2BUF2: ;AN000;
31627 ;MOV AX,CX ;AN000;LB. times index with
31628 ;MUL word [ss:SC_SECTOR_SIZE] ;AN000;LB. sector size
31629 ; 24/09/2023
31630 mov ax,[ss:SC_SECTOR_SIZE]
31631 xchg ax,cx ; cx = [ss:SC_SECTOR_SIZE]
31632 mul cx
31633 ADD AX,[ss:SC_CACHE_PTR] ;AN000;LB. add SC starting addr
31634 ADC DX,[ss:SC_CACHE_PTR+2] ;AN000;LB.
31635 MOV DS,DX ;AN000;LB. DS:SI-> SC sector addr
31636 MOV SI,AX ;AN000;LB.
31637 MOV ES,[ss:CALLXAD+2] ;AN000;LB. ES:DI-> buffer addr
31638 MOV DI,[ss:CALLXAD] ;AN000;LB.
31639 ; 24/09/2023
31640 ;MOV CX,[ss:SC_SECTOR_SIZE] ;AN000;LB. count= sector size
31641 SHR CX,1 ;AN000;LB. may use DWORD move for 386
31642 ;entry MOVWORDS
31643 MOVWORDS: ;AN000;
31644 CMP byte [ss:DDMOVE],0 ;AN000;LB. 386 ?
31645 JZ short nodd ;AN000;LB. no
31646 SHR CX,1 ;AN000;LB. words/2
31647 DB 66H ;AN000;LB. use double word move
31648 nodd:
31649 REP MOVSW ;AN000;LB. move to buffer
31650 CLC ;AN000;LB. clear carry
31651 retn ;AN000;LB. exit
31652 noSC: ;AN000;
31653 STC ;AN000;LB. set carry
31654 sextit: ;AN000;
31655 retn ;AN000;LB.
31656
31657 %endif ; SC
31658
31659 ;=====
31660 ; MKNODE.ASM, MSDOS 6.0, 1991
31661 ;=====
31662 ; 29/07/2018 - Retro DOS v3.0
31663 ; 19/05/2019 - Retro DOS v4.0
31664 ; 22/02/2024 - Retro DOS v5.0
31665
31666 ; TITLE MKNODE - Node maker
31667 ; NAME MKNODE
31668
31669 ;** MKNODE.ASM
31670 ;-----
31671 ; Low level routines for making a new local file system node
31672 ; and filling in an SFT from a directory entry
31673 ;
31674 ; BUILDDIR
31675 ; SETDOTENT
31676 ; MakeNode
31677 ; NEWENTRY
31678 ; FREEENT
31679 ; NEWDIR
31680 ; DOOPEN
31681 ; RENAME_MAKE
31682 ; CHECK_VIRT_OPEN
31683 ;
31684 ; Revision history:
31685 ;
31686 ; AN000 version 4.0 Jan. 1988
31687 ; A004 PTM 3680 --- Make SFT NAME field offset same as 3.30
31688
31689 ;Break <BUILDDIR,NEWDIR -- ALLOCATE DIRECTORIES>
31690 ;-----
31691 ;
31692 ; Procedure Name : BUILDDIR,NEWDIR
31693 ;
31694 ; Inputs:
31695 ; ES:BP Points to DPB
31696 ; [THISSFT] Set if using NEWDIR entry point
31697 ; (used by ALLOCATE)
31698 ; [LASTENT] current last valid entry number in directory if no free
31699 ; entries
31700 ; [DIRSTART] Points to first cluster of dir (0 means root)
31701 ; Function:
31702 ; Grow directory if no free entries and not root
31703 ; Outputs:
31704 ; CARRY SET IF FAILURE
31705 ; ELSE
31706 ; AX entry number of new entry
31707 ; If a new dir [DIRSTART],[CLUSFAC],[CLUSNUM],[DIRSEC] set
31708 ; AX = first entry of new dir
31709 ; GETENT should be called to set [LASTENT]
31710 ;
31711 ;-----
31712
31713 ; 19/05/2019 - Retro DOS v4.0
31714 ; DOSCODE:8845h (MSDOS 6.21, MSDOS.SYS)
31715 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31716 ; DOSCODE:880Ah (MSDOS 6.21, MSDOS.SYS)
31717
31718 ; 24/09/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 MSDOS.SYS)
31719 ; DOSCODE:8845h (MSDOS 6.22, MSDOS.SYS)
31720
31721 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
31722 ; DOSCODE:940Ah (PCDOS 7.1, IBMDOS.COM)
31723
31724 ; (Windows ME IO.SYS - BIOSCODE:890Ch)
31725
31726 BUILDDIR:
31727 ; 29/07/2018 - Retro DOS v3.0
31728 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 4E66h
31729

```



```

31730 00005351 A1[D805]      MOV     AX,[ENTFREE]
31731 00005354 83F8FF      CMP     AX,-1 ; 0FFFFh
31732                      ;JZ      short CHECK_IF_ROOT
31733                      ;CLC
31734                      ;retn
31735                      ; 24/09/2023
31736 00005357 750E      jne     short builddir_cmc_retn ; cf=1 (will be 0)
31737
31738                      CHECK_IF_ROOT:
31739                      ; 23/02/2024 (PCDOS 7.1 IBMDOS.COM)
31740                      ;;;
31741 00005359 833E[DC0A]00    cmp     word [DIRSTART_HW],0
31742 0000535E 7509      jnz     short NEWDIR
31743                      ;;;
31744 00005360 833E[C205]00    cmp     word [DIRSTART],0
31745 00005365 7502      jnz     short NEWDIR
31746                      ;STC                      ; Can't grow root
31747                      ; 24/09/2023
31748                      ; [DIRSTART]=0, cf=0, zf=1 (cf will be 1 after cmc instruction)
31749                      builddir_cmc_retn:
31750                      ; 24/09/2023
31751 00005367 F5      cmc     ; cf=1 <-> cf=0
31752                      builddir_retn:
31753 00005368 C3      retn
31754
31755                      ;entry     NEWDIR
31756                      NEWDIR:
31757                      ; 23/02/2024
31758                      ;;;
31759 00005369 8B1E[DC0A]    mov     bx,[DIRSTART_HW]
31760 0000536D 891E[E80A]    mov     [CLUSTNUM_HW],bx
31761 00005371 09DB      or      bx,bx
31762                      ;;;
31763 00005373 8B1E[C205]    MOV     BX,[DIRSTART]
31764                      ;;;
31765 00005377 7504      jnz     short NEWDIR2 ; 23/02/2024
31766                      ;;;
31767 00005379 09DB      OR      BX,BX
31768 0000537B 7405      JZ      short NULLDIR
31769                      NEWDIR2:
31770 0000537D E8B708      call    GETEOF
31771 00005380 72E6      jc      short builddir_retn ; Screw up
31772                      NULLDIR:
31773                      ;MOV     CX,1
31774                      ; 23/02/2024
31775                      ;;;
31776 00005382 31C9      xor     cx,cx
31777 00005384 890E[F00A]    mov     [CCOUNT_HW],cx ; 0
31778 00005388 41      inc     cx ; 1
31779                      ;;;
31780 00005389 E84906      call    ALLOCATE
31781 0000538C 72DA      jc      short builddir_retn
31782 0000538E 8B16[C205]    MOV     DX,[DIRSTART]
31783                      ; 23/02/2024
31784                      ;;;
31785 00005392 3B16[DC0A]    cmp     dx,[DIRSTART_HW]
31786 00005396 7519      jnz     short ADDINGDIR
31787                      ;;;
31788 00005398 09D2      OR      DX,DX
31789 0000539A 7515      JNZ     short ADDINGDIR
31790                      ; 23/02/2024
31791                      ;;;
31792 0000539C FF36[E80A]    push    word [CLUSTNUM_HW]
31793 000053A0 8F06[EE0A]    pop     word [ROOTCLUST_HW]
31794                      ;;;
31795 000053A4 E8B4F5      call    SETDIRSRCH
31796 000053A7 72BF      jc      short builddir_retn
31797 000053A9 C706[4803]FFFF MOV     word [LASTENT],-1
31798 000053AF EB3F      JMP     SHORT GOTDIRREC
31799                      ADDINGDIR:
31800 000053B1 53      PUSH     BX
31801                      ; 23/02/2024
31802                      ;;;
31803 000053B2 FF36[E80A]    push    word [CLUSTNUM_HW]
31804                      ;
31805 000053B6 8B1E[DE0A]    mov     bx,[CLUSNUM_HW]
31806 000053BA 891E[E80A]    mov     [CLUSTNUM_HW],bx
31807                      ;;;
31808 000053BE 8B1E[BC05]    MOV     BX,[CLUSNUM]
31809 000053C2 E8EE0E      call    ISEOF
31810                      ;;;
31811 000053C5 8F06[E80A]    pop     word [CLUSTNUM_HW] ; 23/02/2024
31812                      ;;;
31813 000053C9 5B      POP     BX
31814 000053CA 721D      JB      short NOTFIRSTGROW
31815                      ;;; 10/17/86 update CLUSNUM in the fastopen cache
31816 000053CC 891E[BC05]    MOV     [CLUSNUM],BX
31817                      ; 24/09/2023
31818                      ;PUSH CX ; (not necessary)
31819 000053D0 50      PUSH     AX
31820 000053D1 55      PUSH     BP
31821                      ; 23/02/2024
31822                      ;;;
31823 000053D2 A1[E80A]      mov     ax,[CLUSTNUM_HW]
31824 000053D5 A3[DE0A]      mov     [CLUSNUM_HW],ax
31825                      ;;;
31826 000053D8 B401      MOV     AH,1                      ; CLUSNUM update
31827                      ; 15/12/2022
31828 000053DA 268A5600     mov     dl,[ES:BP] ; 09/09/2018
31829                      ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
31830                      ;mov     dl,[es:bp+0]
31831                      ;MOV     DL,[ES:BP+DPB.DRIVE] ; drive #
31832 000053DE 8B0E[C205]    MOV     CX,[DIRSTART] ; first cluster #
31833 000053E2 89DD      MOV     BP,BX ; CLUSNUM
31834 000053E4 E87FD9      call    FastOpen_Update
31835 000053E7 5D      POP     BP
31836 000053E8 58      POP     AX
31837                      ; 24/09/2023
31838                      ;POP     CX
31839
31840                      ;;; 10/17/86 update CLUSNUM in the fastopen cache
31841                      NOTFIRSTGROW:
31842 000053E9 89DA      MOV     DX,BX
31843 000053EB 30DB      XOR     BL,BL
31844 000053ED E8A305      call    FIGREC
31845                      GOTDIRREC:
31846                      ;mov     cl,[es:bp+4]
31847 000053F0 268A4E04     MOV     CL,[ES:BP+DPB.CLUSTER_MASK]
31848                      ;INC     CL
31849                      ; 27/06/2024
31850 000053F4 41      inc     cx
31851 000053F5 30ED      XOR     CH,CH
31852                      ZERODIR:
31853 000053F7 51      PUSH     CX

```

```

31854 ; 22/09/2023
31855 ;;mov byte [ALLOWED],18h
31856 ;MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
31857 000053F8 B0FF MOV AL,0FFh
31858 ;call GETBUFFR
31859 000053FA E82915 call GETBUFFRD ; *
31860 000053FD 7302 JNC short GET_SSIZE
31861 000053FF 59 POP CX
31862 00005400 C3 retn
31863
31864 GET_SSIZE:
31865 ;mov CX,[es:bp+2]
31866 00005401 268B4E02 MOV CX,[ES:BP+DPB.SECTOR_SIZE]
31867 00005405 06 PUSH ES
31868 00005406 C43E[E205] LES DI,[CURBUF]
31869 ;or byte [es:di+5],4
31870 0000540A 26804D0504 OR byte [ES:DI+BUFFINFO.buf_flags],buf_isDIR
31871 0000540F 57 PUSH DI
31872 ;;;add di,16 ; MSDOS 3.3
31873 ;;;add di,20 ; MSDOS 6.0
31874 ;;;add di,24 ; PCDOS 7.1 ; 23/02/2024
31875 00005410 83C718 ADD DI,BUFINSIZ
31876 00005413 31C0 XOR AX,AX
31877 00005415 D1E9 SHR CX,1
31878 00005417 F3AB REP STOSW
31879 00005419 7301 JNC short EVENZ
31880 0000541B AA STOSB
31881 EVENZ:
31882 0000541C 5F POP DI
31883
31884 ; 23/02/2024
31885 %if 0
31886 ; MSDOS 6.0
31887 TEST byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
31888 ;LB. if already dirty ;AN000;
31889 JNZ short yesdirty7 ;LB. don't increment dirty count ;AN000;
31890 call INC_DIRTY_COUNT ;LB. ;AN000;
31891
31892 ;or byte [es:di+5],40h
31893 OR byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
31894 %else
31895 ; 23/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
31896 0000541D E89917 call SET_BUF_DIRTY
31897 %endif
31898
31899 yesdirty7:
31900 00005420 07 POP ES
31901 00005421 59 POP CX
31902
31903 ; 19/05/2019 - Retro DOS v4.0
31904
31905 ; MSDOS 3.3
31906 ;INC DX
31907
31908 ; MSDOS 6.0
31909 ; 24/09/2023
31910 ;add dx,1
31911 ;;adc word [HIGH_SECTOR],0
31912 ;; 24/09/2023
31913 ;; ax=0
31914 ;adc [HIGH_SECTOR],ax ; 0
31915 ; 24/09/2023
31916 00005422 42 inc dx
31917 00005423 7504 jnz short loop_zerodir
31918 00005425 FF06[0706] inc word [HIGH_SECTOR]
31919
31920 00005429 E2CC loop_zerodir:
31921 LOOP ZERODIR
31922 0000542B A1[4803] MOV AX,[LASTENT]
31923 0000542E 40 INC AX
31924 ; 24/09/2023
31925 ; cf=0
31926 ;CLC
31927 0000542F C3 retn
31928
31929 ;-----
31930 ;
31931 ; Procedure Name : SETDOTENT
31932 ;
31933 ; set up a . or .. directory entry for a directory.
31934 ;
31935 ; Inputs: ES:DI point to the beginning of a directory entry.
31936 ; AX contains ". " or ".. "
31937 ; DX contains first cluster of entry
31938 ;
31939 ;-----
31940
31941 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
31942
31943 SETDOTENT:
31944 ; Fill in name field
31945 00005430 AB STOSW
31946 00005431 B90400 MOV CX,4
31947 00005434 B82020 MOV AX," " ; 2020h
31948 00005437 F3AB REP STOSW
31949 00005439 AA STOSB
31950
31951 ; Set up attribute
31952 ;mov al, 10h
31953 0000543A B010 MOV AL,attr_directory
31954 0000543C AA STOSB
31955
31956 ; Initialize time and date of creation
31957 ;ADD DI,10
31958 ; 23/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
31959 ;;;
31960 0000543D 83C708 add di,8
31961 00005440 A1[F20A] mov ax,[CLUSTERS_HW]
31962 00005443 AB stosw ; Set up first cluster field (hw)
31963 ;;;
31964 00005444 8B36[9E05] MOV SI,[THISSFT]
31965 ;mov ax,[si+0Dh]
31966 00005448 8B440D MOV AX,[SI+SF_ENTRY.sf_time]
31967 0000544B AB STOSW
31968 ;mov ax,[si+0Fh]
31969 0000544C 8B440F MOV AX,[SI+SF_ENTRY.sf_date]
31970 0000544F AB STOSW
31971
31972 ; Set up first cluster field (lw)
31973 00005450 89D0 MOV AX,DX
31974 00005452 AB STOSW
31975
31976 ; 0 file size
31977 ;XOR AX,AX

```

```

31978 00005453 91      xchg    ax,cx ; 23/02/2024
31979 00005454 AB      STOSW
31980 00005455 AB      STOSW
31981 00005456 C3      retn
31982
31983 ;Break    <MAKENODE -- CREATE A NEW NODE>
31984 ;-----
31985 ;
31986 ; Procedure Name : MakeNode
31987 ;
31988 ; Inputs:
31989 ;     AL - attribute to create
31990 ;     AH = 0 if it is ok to truncate a file already by this name
31991 ;     AH != 0 if truncation not allowed (preexisting file is an error)
31992 ;           (AH ignored on dirs and devices)
31993 ;
31994 ;     NOTE: When making a DIR or volume ID, AH need not be set since
31995 ;           a name already existant is ALWAYS an error in these cases.
31996 ;
31997 ;     [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
31998 ;           terminated)
31999 ;     [CURR_DIR_END] Points to end of Current dir part of string
32000 ;           (= -1 if current dir not involved, else
32001 ;           Points to first char after last "/" of current dir part)
32002 ;     [THISCDs] Points to CDS being used
32003 ;     [THISSFT] Points to an empty SFT. EXCEPT sf_mode filled in.
32004 ;
32005 ; Function:
32006 ;     Make a new node
32007 ;
32008 ; Outputs:
32009 ;     Sets EXTERR_LOCUS = errLOC_Disk or errLOC_Unk via GetPathNoseT
32010 ;     CARRY SET IF ERROR
32011 ;     AX = 1 A node by this name exists and is a directory
32012 ;     AX = 2 A new node could not be created
32013 ;     AX = 3 A node by this name exists and is a disk file
32014 ;           (AH was NZ on input)
32015 ;     AX = 4 Bad Path
32016 ;     SI return from GetPath maintained
32017 ;     AX = 5 Attribute mismatch
32018 ;     AX = 6 Sharing Violation
32019 ;           (INT 24 generated ALWAYS since create is always compat mode)
32020 ;     AX = 7 file not found for Extended Open (not exists and fails)
32021 ;     ELSE
32022 ;     AX = 0 Disk Node
32023 ;     AX = 3 Device Node (error in some cases)
32024 ;     [DIRSTART],[DIRSEC],[CLUSFAC],[CLUSNUM] set to directory
32025 ;           containing new node.
32026 ;     [CURBUF+2]:BX Points to entry
32027 ;     [CURBUF+2]:SI Points to entry.dir_first
32028 ;     [THISSFT] is filled in
32029 ;     sf_mode = unchanged.
32030 ;     Attribute byte in entry is input AL
32031 ;     DS preserved, others destroyed
32032 ;-----
32033 ; 19/05/2019 - Retro DOS v4.0
32034 ; DOSCODE:8925h (MSDOS 6.21, MSDOS.SYS)
32035 ;
32036 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32037 ; DOSCODE:88EAh (MSDOS 5.0, MSDOS.SYS)
32038 ;
32039 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
32040 ; DOSCODE:951Ah (PCDOS 7.1, IBMDOS.COM)
32041 ;
32042 ; MakeNode:
32043 ;
32044 00005457 C706[7E05]FFE5    ;mov    word [CREATING],0E5FFh
32045 0000545D 50              MOV     WORD [CREATING],DIRFREE*256 + 0FFh ; Creating, not DEL *.*
32046 0000545E C606[4C03]00    PUSH    AX          ; Save AH value
32047 00005463 A2[6D05]       MOV     byte [NoSetDir],0
32048 00005466 E8ACF6         MOV     [SATTRIB],AL
32049 00005469 88CA          call    GetPathNoSet
32050                MOV     DL,CL          ; Save CL info
32051                ;MOV     CX,AX          ; Device ID to CH
32052                ; 23/02/2024
32053 0000546B 91              xchg    ax,cx
32054 0000546C 58              POP     AX          ; Get back AH
32055 0000546D 732D           JNC     short make_exists ; File existed
32056 00005471 80FA80         JNZ     short make_err_4 ; Path bad
32057 00005474 7405           CMP     DL,80h       ; Check "CL" return from GETPATH
32058                JZ      short make_type ; Name simply not found, and no metas
32059 00005476 B004           MOV     AL,4         ; case 1 bad path
32060 ; make_err_ret:
32061                ;XOR     AH,AH
32062                ; 23/02/2024
32063 00005478 98              cbw
32064 00005479 F9              STC
32065 ; make_retn: ; 22/11/2022
32066 0000547A C3              retn
32067
32068                ;entry RENAME_MAKE ; Used by DOS_RENAME to "copy" a node
32069 RENAME_MAKE:
32070 make_type:
32071 ;Extended Open hooks
32072 ; MSDOS 6.0
32073 ;TESTB EXTOPEN_ON,EXT_OPEN_ON;FT. from extended open ;AN000;
32074 0000547B F606[F605]01    test    byte [EXTOPEN_ON],EXT_OPEN_ON ; 1
32075 00005480 7411           JZ      short make_type2 ;FT. no ;AN000;
32076 00005482 800E[F605]04    OR      byte [EXTOPEN_ON],EXT_FILE_NOT_EXISTS ; 4
32077                ;FT. set for extended open ;AN000;
32078 ;TESTB EXTOPEN_FLAG,0F0h ;FT. not exists and fails ;AN000;
32079 00005487 F606[F405]F0    test    byte [EXTOPEN_FLAG],0F0h
32080 0000548C 7505           JNZ     short make_type2 ;FT. no ;AN000;
32081 0000548E F9              STC ;FT. set carry ;AN000;
32082 0000548F B80700         MOV     AX,7         ;FT. file not found ;AN000;
32083                ; 22/11/2022
32084 make_retn:
32085                ;return
32086 00005492 C3              retn ;FT. ;AN000;
32087
32088 ; Extended Open hooks
32089
32090 make_type2:
32091 00005493 C43E[9E05]    LES     DI,[THISSFT]
32092 00005497 31C0           XOR     AX,AX ; nothing exists Disk Node
32093 00005499 F9              STC ; Not found
32094 0000549A EB59           JMP     short make_new
32095
32096 ; The node exists. It may be either a device, directory or file:
32097 ; Zero set => directory
32098 ; High bit of CH on => device
32099 ; else => file
32100
32101 make_exists:

```

```

32102 0000549C 7447      JZ      short make_exists_dir
32103 0000549E B003      MOV     AL,3          ; file exists type 3 (error or device node)
32104                      ;test  byte [ATTRIB],18h
32105 000054A0 F606[6B05]18 TEST    byte [ATTRIB],attr_volume_id+attr_directory
32106 000054A5 753A      JNZ     short make_err_ret_5
32107                      ; Cannot already exist as Disk or Device Node
32108                      ; if making DIR or Volume ID
32109 000054A7 08ED      OR      CH,CH
32110 000054A9 781A      JS      short make_share ; No further checks on attributes if device
32111 000054AB 08E4      OR      AH,AH
32112 000054AD 75C9      JNZ     short make_err_ret ; truncating NOT OK (AL = 3)
32113 000054AF 51        PUSH    CX          ; Save device ID
32114 000054B0 8E06[E405] MOV     ES,[CURBUF+2]
32115                      ;mov  ch,[es:bx+0Bh]
32116 000054B4 268A6F0B MOV     CH,[ES:BX+dir_entry.dir_attr] ; Get file attributes
32117                      ;test  ch,1
32118 000054B8 F6C501    test    CH,attr_read_only
32119 000054BB 7523      JNZ     short make_err_ret_5P ; Cannot create on read only files
32120 000054BD E803F9    call    MatchAttributes
32121 000054C0 59        POP     CX          ; Devid back in CH
32122 000054C1 751E      JNZ     short make_err_ret_5 ; Attributes not ok
32123 000054C3 30C0      XOR     AL,AL        ; AL = 0, Disk Node
32124                      make_share:
32125                      ;XOR  AH,AH
32126                      ; 23/02/2024
32127 000054C5 98        cbw
32128 000054C6 50        PUSH    AX          ; Save Disk or Device node
32129 000054C7 51        PUSH    CX          ; Save Device ID
32130 000054C8 88EC      MOV     AH,CH        ; Device ID to AH
32131 000054CA E83201    CALL    DOOPEN       ; Fill in SFT for share check
32132 000054CD C43E[9E05]    LES     DI,[THISSFT]
32133 000054D1 56        push    si
32134 000054D2 53        push    bx          ; Save CURBUF pointers
32135 000054D3 E8AE2F    call    ShareEnter
32136 000054D6 734E      jnc     short MakeEndShare
32137
32138                      ; User failed request.
32139 000054D8 5B        pop     bx
32140 000054D9 5E        pop     si
32141 000054DA 59        pop     cx
32142 000054DB 58        pop     ax
32143
32144                      Make_Share_ret:
32145 000054DC B006      MOV     AL,6
32146 000054DE EB98      JMP     short make_err_ret
32147
32148                      make_err_ret_5P:
32149 000054E0 59        POP     CX          ; Get back device ID
32150                      make_err_ret_5:
32151 000054E1 B005      MOV     AL,5          ; Attribute mismatch
32152                      ; 22/11/2022
32153 000054E3 EB93      JMP     short make_err_ret
32154
32155                      make_exists_dir:
32156 000054E5 B001      MOV     AL,1          ; exists as directory, always an error
32157                      ; 22/11/2022
32158 000054E7 EB8F      JMP     short make_err_ret
32159
32160                      make_save:
32161 000054E9 50        PUSH    AX          ; Save whether Disk or File
32162 000054EA 89C8      MOV     AX,CX        ; Device ID to AH
32163 000054EC E86800    CALL    NEWENTRY
32164 000054EF 58        POP     AX          ; 0 if Disk, 3 if File
32165 000054F0 73A0      jnc     short make_retn
32166 000054F2 B002      MOV     AL,2          ; create failed case 2
32167                      make_save_retn:
32168 000054F4 C3        retn
32169
32170                      make_new:
32171 000054F5 E8F1FF    call    make_save
32172 000054F8 72FA      jc      short make_save_retn ; case 2 fail
32173                      ;test  byte [ATTRIB],10h
32174 000054FA F606[6B05]10 TEST    BYTE [ATTRIB],attr_directory
32175 000054FF 75F3      jnz     short make_save_retn ; Don't "open" directories,
32176                      ; so don't tell the sharer about them
32177 00005501 50        push    ax
32178 00005502 53        push    bx
32179 00005503 56        push    si
32180 00005504 E87D2F    call    ShareEnter
32181 00005507 5E        pop     si
32182 00005508 5B        pop     bx
32183 00005509 58        pop     ax
32184 0000550A 73E8      jnc     short make_save_retn
32185
32186                      ; We get here by having the user FAIL a share problem. Typically a failure of
32187                      ; this nature is an out-of-space or an internal error. We clean up as best as
32188                      ; possible: delete the newly created directory entry and return share_error.
32189
32190 0000550C 50        PUSH    AX
32191 0000550D C43E[E205]    LES     DI,[CURBUF]
32192                      ;mov  byte [es:bx],0E5h
32193 00005511 26C607E5 MOV     BYTE [ES:BX],DIRFREE ; nuke newly created entry.
32194
32195                      ; 23/02/2024
32196                      %if 0
32197                      ; MSDOS 6.0
32198                      ;test  byte [es:di+5],40h
32199                      TEST    byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
32200                      ;LB. if already dirty ;AN000;
32201                      JNZ     short yesdirty8 ;LB. don't increment dirty count ;AN000;
32202                      ; 22/11/2022
32203                      call    INC_DIRTY_COUNT ;LB. ;AN000;
32204                      ;or  byte [es:di+5],40h
32205                      OR      byte [ES:DI+BUFFINFO.buf_flags],buf_dirty ; flag buffer as dirty
32206                      yesdirty8:
32207                      %else
32208                      ; 23/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
32209 00005515 E8A116    call    SET_BUF_DIRTY
32210                      %endif
32211 00005518 C42E[8A05]    LES     BP,[THISDPB]
32212                      ; 15/12/2022
32213 0000551C 268A4600 mov     al,[ES:BP]
32214                      ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
32215                      ;;mov  al,[es:bp+0]
32216                      ;MOV  AL,[ES:BP+DPB.DRIVE] ; get drive for flush
32217 00005520 E87015    call    FLUSHBUF    ; write out buffer.
32218 00005523 58        POP     AX
32219 00005524 EBB6      jmp     short Make_Share_ret
32220
32221                      ; We have found an existing file. We have also entered it into the share set.
32222                      ; At this point we need to call newentry to correctly address the problem of
32223                      ; getting rid of old data (create an existing file) or creating a new
32224                      ; directory entry (create a new file). Unfortunately, this operation may
32225                      ; result in an INT 24 that the user doesn't return from, thus locking the file

```

```

32226 ; irretrievably into the share set. The correct solution is for us to LEAVE
32227 ; the share set now, do the operation and then reassert the share access.
32228 ;
32229 ; We are allowed to do this! There is no window! After all, we are in
32230 ; critDisk here and for someone else to get in, they must enter critDisk also.
32231 ;
32232 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32233 ; DOSCODE:89C8h (MSDOS 5.0, MSDOS.SYS)
32234
32235 MakeEndShare:
32236 00005526 C43E[9E05] LES DI,[THISSFT] ; grab SFT
32237 0000552A 31C0 XOR AX,AX
32238 0000552C E8B3C3 call ECritSFT
32239 0000552F 268705 xchg AX,[ES:DI]
32240 ;XCHG AX,[ES:DI+SF_ENTRY.sf_ref_count]
32241 00005532 50 push ax
32242 00005533 57 push di
32243 00005534 06 push es
32244 00005535 9C PUSHF
32245 00005536 E8462F call ShareEnd ; remove sharing
32246 00005539 9D POPF
32247 0000553A 07 pop es
32248 0000553B 5F pop di
32249 0000553C 268F05 pop word [ES:DI]
32250 ;pop word [ES:DI+SF_ENTRY.sf_ref_count]
32251 0000553F E8CDC3 call LCritSFT
32252 ; 22/11/2022
32253 ; DOSCODE:89E4h (MSDOS 5.0, MSDOS.SYS)
32254 00005542 5B pop bx
32255 00005543 5E pop si
32256 00005544 59 pop cx
32257 00005545 58 pop ax
32258 00005546 E8A0FF CALL make_save
32259
32260 ; If the user failed, we do not reenter into the sharing set.
32261
32262 00005549 72A9 jc short make_save_retn ; bye if error
32263 0000554B 50 push ax
32264 0000554C 53 push bx
32265 0000554D 56 push si
32266 0000554E 9C PUSHF
32267 0000554F E8322F call ShareEnter
32268 00005552 9D POPF
32269 00005553 5E pop si
32270 00005554 5B pop bx
32271 00005555 58 pop ax
32272
32273 ; If Share_check fails, then we have an internal ERROR!!!!
32274
32275 makeendshare_retn:
32276 00005556 C3 retn
32277
32278 ;-----
32279 ;
32280 ; Procedure Name : NEWENTRY
32281 ;
32282 ; Inputs:
32283 ; [THISSFT] set
32284 ; [THISDPB] set
32285 ; [LASTENT] current last valid entry number in directory if no free
32286 ; entries
32287 ; [VOLID] set if a volume ID was found during search
32288 ; Attrib Contains attributes for new file
32289 ; [DIRSTART] Points to first cluster of dir (0 means root)
32290 ; CARRY FLAG INDICATES STATUS OF SEARCH FOR FILE
32291 ; NC means file existed (device)
32292 ; C means file did not exist
32293 ; AH = Device ID byte
32294 ; If FILE
32295 ; [CURBUF+2]:BX points to start of directory entry
32296 ; [CURBUF+2]:SI points to dir_first of directory entry
32297 ; If device
32298 ; DS:BX points to start of "fake" directory entry
32299 ; DS:SI points to dir_first of "fake" directory entry
32300 ; (has DWORD pointer to device header)
32301 ; Function:
32302 ; Make a new directory entry
32303 ; If an old one existed it is truncated first
32304 ; Outputs:
32305 ; Carry set if error
32306 ; Can't grow dir, atts didn't match, attempt to make 2nd
32307 ; vol ID, user FAILED to I 24
32308 ; else
32309 ; outputs of DOOPEN
32310 ; DS, BX, SI preserved (meaning on SI BX, not value), others destroyed
32311 ;
32312 ;-----
32313
32314 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32315 ; DOSCODE:89F9h (MSDOS 5.0, MSDOS.SYS)
32316
32317 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
32318 ; DOSCODE:961Ah (PCDOS 7.1, IBMDOS.COM)
32319
32320 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8A34h)
32321 ; (Windows ME IO.SYS - BIOSCODE:8B7Dh)
32322
32323 NEWENTRY:
32324 00005557 C42E[8A05] LES BP,[THISDPB]
32325 0000555B 7315 JNC short EXISTENT
32326 0000555D 803E[4A03]00 CMP byte [FAILERR],0
32327 ;STC
32328 ;jnz short makeendshare_retn ; User FAILED, node might exist
32329 ; 24/09/2023
32330 00005562 750C jnz short ERRRET3
32331 00005564 E8EAFD CALL BUILDDIR ; Try to build dir
32332 00005567 72ED jc short makeendshare_retn ; Failed
32333 00005569 E832F3 call GETENT ; Point at that free entry
32334 0000556C 72E8 jc short makeendshare_retn ; Failed
32335 0000556E E80E JMP SHORT FREESPOT
32336
32337 ERRRET3:
32338 00005570 F9 STC
32339 newentry_retn:
32340 00005571 C3 retn
32341
32342 EXISTENT:
32343 00005572 08E4 OR AH,AH ; Check if file is I/O device
32344 00005574 7903 JNS short NOT_DEV1
32345 00005576 E98600 JMP DOOPEN ; If so, proceed with open
32346
32347 NOT_DEV1:
32348 00005579 E84D01 call FREEENT; Free cluster chain
32349 0000557C 72F3 jc short newentry_retn ; Failed

```

```

32350
32351
32352 0000557E F606[6B05]08
32353 00005583 7407
32354 00005585 803E[7B05]00
32355 0000558A 75E4
32356
32357 0000558C 8E06[E405]
32358 00005590 89DF
32359
32360 00005592 BE[4B05]
32361
32362 00005595 B90500
32363 00005598 F3A5
32364 0000559A A4
32365 0000559B A0[6B05]
32366 0000559E AA
32367
32368
32369 0000559F B105
32370
32371
32372 000055A1 31C0
32373 000055A3 F3AB
32374 000055A5 E8B8B5
32375 000055A8 92
32376 000055A9 AB
32377 000055AA 92
32378
32379
32380 000055AB 268945FA
32381
32382 000055AF AB
32383 000055B0 31C0
32384 000055B2 57
32385
32386 000055B3 AB
32387 000055B4 AB
32388 000055B5 AB
32389
32390 000055B6 8B36[E205]
32391
32392
32393
32394
32395 000055BA 26F6440540
32396
32397 000055BF 7508
32398 000055C1 E80116
32399
32400
32401 000055C4 26804C0540
32402
32403 000055C9 C42E[8A05]
32404
32405 000055CD 268A4600
32406
32407
32408
32409 000055D1 50
32410 000055D2 53
32411
32412
32413
32414
32415
32416
32417 000055D3 06
32418 000055D4 57
32419 000055D5 C43E[9E05]
32420
32421
32422 000055D9 26F6450580
32423 000055DE 7512
32424 000055E0 1E
32425 000055E1 53
32426 000055E2 C51E[8A05]
32427
32428 000055E6 26895D07
32429 000055EA 8CDB
32430
32431 000055EC 26895D09
32432 000055F0 5B
32433 000055F1 1F
32434
32435
32436
32437
32438
32439
32440
32441
32442 000055F2 5F
32443 000055F3 07
32444
32445 000055F4 E89C14
32446
32447
32448
32449
32450
32451 000055F7 5B
32452 000055F8 58
32453 000055F9 5E
32454 000055FA 88C4
32455 000055FC 7301
32456 000055FE C3
32457
32458
32459
32460
32461
32462
32463
32464
32465
32466
32467
32468
32469
32470
32471
32472
32473

```

```

FREESPT:
;test byte [ATTRIB],8
test BYTE [ATTRIB],attr_volume_id
JZ short NOTVOLID
CMP BYTE [VOLID],0
JNZ short ERRRET3 ; Can't create a second volume ID
NOTVOLID:
MOV ES,[CURBUF+2]
MOV DI,BX
MOV SI,NAME1
MOV CX,5
REP MOVSW
MOVSB ; Move name into dir entry
MOV AL,[ATTRIB]
STOSB ; Attributes
;; File Tagging for Create DOS 4.00
MOV CL,5 ;FT. assume normal FBUGBUG ;AN000;
;; File Tagging for Create DOS 4.00
XOR AX,AX
REP STOSW ; Zero pad
call DATE16
XCHG AX,DX
STOSW ; dir_time
XCHG AX,DX
; 23/02/2024 (PCDOS 7.1 IBMDOS.COM)
;;
mov [es:di-6],ax ; last access date
;;
STOSW ; dir_date
XOR AX,AX
PUSH DI ; Correct SI input value
; (recomputed for new buffer)
; Zero dir_first and size
STOSW
STOSW
STOSW
updnxt:
MOV SI,[CURBUF]
; 19/05/2019 - Retro DOS v4.0
; MSDOS 6.0
TEST byte [ES:SI+BUFFINFO.buf_flags],buf_dirty ;LB. if already dirty ;AN000;
JNZ short yesdirty9 ;LB. don't increment dirty count ;AN000;
call INC_DIRTY_COUNT ;LB. ;AN000;
;or byte [es:si+5],40h
OR byte [ES:SI+BUFFINFO.buf_flags],buf_dirty
yesdirty9:
LES BP,[THISDPB]
; 15/12/2022
MOV AL,[ES:BP]
; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;;mov al,[es:bp+0]
;MOV AL,[ES:BP+DPB.DRIVE] ; Sets AH value again (in AL)
PUSH AX
PUSH BX
; If we have a file, we need to increment the open ref. count so that
; we have some protection against invalid media changes if an Int 24
; error occurs.
; Do nothing for a device.
push es
push di
LES DI,[THISSFT]
;test word [es:di+5],80h
;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device
test byte [ES:DI+SF_ENTRY.sf_flags],devid_device
jnz short GotADevice
push ds
push bx
LDS BX,[THISDPB]
;mov [es:di+7],bx
MOV [ES:DI+SF_ENTRY.sf_devptr],BX
MOV BX,DS
;mov [es:di+9],bx
MOV [ES:DI+SF_ENTRY.sf_devptr+2],BX
pop bx
pop ds ; need to use DS for segment later on
; 23/02/2024 Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
%if 0
call DEV_OPEN_SFT ; increment ref. count
mov byte [VIRTUAL_OPEN],1; set flag
%endif
GotADevice:
pop di
pop es
call FLUSHBUF
; 23/02/2024 Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
%if 0
call CHECK_VIRT_OPEN ; decrement ref. count;AN000;
%endif
POP BX
POP AX
POP SI ; Get SI input back
MOV AH,AL ; Get I/O driver number back
jnc short DOOPEN
retn ; Failed
;NOTE FALL THROUGH
; DOSCODE:8AE4h (MSDOS 6.21, MSDOS.SYS)
; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:8AA9h (MSDOS 5.0, MSDOS.SYS)
; DOOPEN
;-----
;
; Inputs:
; [THISDPB] points to DPB if file
; [THISSFT] points to SFT being used
; AH = Device ID byte
; If FILE
; [CURBUF+2]:BX points to start of directory entry

```



```

32474 ; [CURBUF+2]:SI points to dir_first of directory entry
32475 ; If device
32476 ; DS:BX points to start of "fake" directory entry
32477 ; DS:SI points to dir_first of "fake" directory entry
32478 ; (has DWORD pointer to device header)
32479 ; Function:
32480 ; Fill in SFT from dir entry
32481 ; Outputs:
32482 ; CARRY CLEAR
32483 ; sf_ref_count and sf_mode fields not altered
32484 ; sf_flags high byte = 0
32485 ; sf_flags low byte = AH except
32486 ; sf_flags Bit 6 set (not dirty or not EOF)
32487 ; sf_attr sf_date sf_time sf_name set from entry
32488 ; sf_position = 0
32489 ; If device
32490 ; sf_devptr = dword at dir_first (pointer to device header)
32491 ; sf_size = 0
32492 ; If file
32493 ; sf_firclus sf_size set from entry
32494 ; sf_devptr = [THISDPB]
32495 ; sf_cluspos = 0
32496 ; sf_lstclus = sf_firclus
32497 ; sf_dirsec sf_dirpos set
32498 ; DS,SI,BX preserved, others destroyed
32499 ;
32500 ;-----
32501 ;
32502 ; 23/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
32503 ; DOSCODE:96C3h (PC DOS 7.1, IBMDOS.COM)
32504 ;
32505 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8AE4h)
32506 ; (Windows ME IO.SYS - BIOSCODE:8C4Ch)
32507 ;
32508 ;entry DOOPEN
32509 DOOPEN:
32510 ; Generate and store attribute
32511 ;
32512 000055FF 88E6 MOV DH,AH ; AH to different place
32513 ; 23/02/2024 (PC DOS 7.1 IBMDOS.COM)
32514 ;;;
32515 00005601 30D2 xor dl,dl ; 0
32516 00005603 08E4 or ah,ah
32517 00005605 780D js short DEV_SFT0
32518 00005607 C43E[8A05] les di,[THISDPB]
32519 ;cmp word [es:di+0Fh],0
32520 0000560B 26837D0F0F0 cmp word [es:di+DPB.FAT_SIZE],0
32521 00005610 7402 jz short DEV_SFT0 ; FAT32
32522 00005612 FEC2 inc dl ; 1
32523 DEV_SFT0:
32524 ;;;
32525 00005614 C43E[9E05] LES DI,[THISSFT]
32526 ;add di,4
32527 00005618 83C704 ADD DI,SF_ENTRY.sf_attr ; Skip ref_count and mode fields
32528 ; 24/09/2023
32529 0000561B 31C0 xor ax,ax
32530 ;XOR AL,AL ; Assume it's a device, devices have an
32531 ; attribute of 0 (for R/O testing etc).
32532 0000561D 08F6 OR DH,DH ; See if our assumption good.
32533 0000561F 7807 JS short DEV_SFT1 ; If device DS=DOSGROUP
32534 00005621 8E1E[E405] MOV DS,[CURBUF+2]
32535 ;mov al,[BX+0Bh]
32536 00005625 8A470B MOV AL,[BX+dir_entry.dir_attr]
32537 ; If file, get attrib from dir entry
32538 DEV_SFT1:
32539 00005628 AA STOSB ; sf_attr, ES:DI -> sf_flags
32540 ;
32541 ; Generate and store flags word
32542 ;
32543 ; 24/09/2023
32544 ;XOR AX,AX
32545 ; ah=0
32546 00005629 88F0 MOV AL,DH
32547 ;or al,40h
32548 0000562B 0C40 OR AL,devid_file_clean
32549 0000562D AB STOSW ; sf_flags, ES:DI -> sf_devptr
32550 ;
32551 ; Generate and store device pointer
32552 ;
32553 0000562E 1E PUSH DS
32554 ;lds ax,[bx+1Ah]
32555 0000562F C5471A LDS AX,[BX+dir_entry.dir_first] ; Assume device
32556 00005632 08F6 OR DH,DH
32557 00005634 7805 JS short DEV_SFT2
32558 ;hkn; SS override
32559 LDS AX,[SS:THISDPB] ; was file
32560 00005636 36C506[8A05] DEV_SFT2:
32561 STOSW ; store offset
32562 0000563B AB MOV AX,DS
32563 0000563C 8CD8 POP DS
32564 0000563E 1F STOSW ; store segment
32565 0000563F AB ; ES:DI -> sf_firclus
32566 ;
32567 ; Generate pointer to, generate and store first cluster
32568 ; (irrelevant for devices)
32569 ;
32570 ;
32571 00005640 56 PUSH SI ; Save pointer to dir_first
32572 ;
32573 ; 23/02/2024
32574 %if 0
32575 ;
32576 MOVSW ; dir_first -> sf_firclus
32577 ; DS:SI -> dir_size_l, ES:DI -> sf_time
32578 ;
32579 ; Copy time/date of last modification
32580 ;
32581 ;sub si,6
32582 SUB SI,dir_entry.dir_size_l - dir_entry.dir_time
32583 %else
32584 ; 23/02/2024 - Retro DOS v5.0
32585 ; (PC DOS 7.1 IBMDOS.COM)
32586 ;;;
32587 00005641 83C720 add di,32 ; SF_ENTRY.sf_chain ; first cluster (32 bit) !?
32588 00005644 A5 movsw ; first cluster, lw
32589 ;add si,0FFF8h
32590 00005645 83C6F8 add si,-8
32591 00005648 AD lodsw ; 27/06/2024 ; first cluster, hw
32592 ;
32593 00005649 08D2 or dl,dl
32594 0000564B 7402 jz short FILE_SFT0 ; FAT32
32595 0000564D 31C0 xor ax,ax ; 0 ; clear hw of first cluster (invalid)
32596 FILE_SFT0:
32597 0000564F AB stosw

```

```

32598             ;add    di,0FFDEh
32599 00005650 83C7DE add    di,-34          ; SF_ENTRY.sf_time
32600             ;;;
32601 %endif
32602             ; DS:SI->dir_time
32603 00005653 A5      MOVSW             ; dir_time -> sf_time
32604             ; DS:SI -> dir_date, ES:DI -> sf_date
32605 00005654 A5      MOVSW             ; dir_date -> sf_date
32606             ; DS:SI -> dir_first, ES:DI -> sf_size
32607
32608 ; Generate and store file size (0 for devices)
32609
32610 00005655 AD      LODSW             ; skip dir_first, DS:SI -> dir_size_l
32611 00005656 AD      LODSW             ; dir_size_l in AX, DS:SI -> dir_size_h
32612             ; MOV    CX,AX          ; dir_size_l in CX
32613             ; 23/02/2024
32614 00005657 91      xchg    ax,cx
32615 00005658 AD      LODSW             ; dir_size_h (size AX:CX), DS:SI -> ???
32616 00005659 08F6   OR     DH,DH
32617 0000565B 7904   JNS    short FILE_SFT1
32618 0000565D 31C0   XOR    AX,AX
32619 0000565F 89C1   MOV    CX,AX          ; Devices are open ended
32620 FILE_SFT1:
32621 00005661 91      XCHG    AX,CX
32622 00005662 AB      STOSW             ; Low word of sf_size
32623 00005663 91      XCHG    AX,CX
32624 00005664 AB      STOSW             ; High word of sf_size
32625             ; ES:DI -> sf_position
32626 ; Initialize position to 0
32627
32628 00005665 31C0   XOR    AX,AX
32629 00005667 AB      STOSW
32630 00005668 AB      STOSW             ; sf_position
32631             ; ES:DI -> sf_cluspos
32632
32633 ; Generate cluster optimizations for files
32634
32635 00005669 08F6   OR     DH,DH
32636 0000566B 784E   JS     short DEV_SFT3
32637 0000566D AB      STOSW             ; sf_cluspos ; 19h
32638
32639 ; 23/02/2024
32640 %if 0
32641             ; mov    ax,[bx+1Ah]
32642             MOV    AX,[BX+dir_entry.dir_first]
32643             ; 19/05/2019
32644             ; MSDOS 3.3
32645             ; STOSW             ; sf_lstclus ; 1Bh
32646             ; MSDOS 6.0
32647             PUSH    DI          ; AN004; save dirsec offset
32648             ; sub    di,1Bh
32649             SUB     DI,SF_ENTRY.sf_dirsec ; AN004; es:di -> SFT
32650             ; mov    [es:di+35h],ax
32651             MOV     [ES:DI+SF_ENTRY.sf_lstclus],AX ; AN004; save it
32652             POP     DI          ; AN004; restore dirsec offset
32653 %else
32654             ; 23/02/2024 - Retro DOS v5.0
32655             ; (PCDOS 7.1 IBMDOS.COM)
32656             ;;;
32657             ; add    di,0FFF0h
32658 0000566E 83C7F0 add    di,-16
32659 00005671 AB      stosw             ; sf_cluspos_h (sf_firclus in MSDOS 5.0-6.22)
32660             ; add    di,SF_ENTRY.sf_dirsec
32661 00005672 83C70E add    di,14          ; sf_dirsec ; 27
32662 00005675 268B4510 mov    ax,[es:di+10h] ; [ES:DI+SF_ENTRY.sf_chain] ; 43
32663             ; first cluster (32 bit)
32664 00005679 2689451A mov    [es:di+1Ah],ax
32665             ; mov    [es:di+SF_ENTRY.sf_lstclus-SF_ENTRY.sf_dirsec],ax ; 53
32666 0000567D 268B4512 mov    ax,[es:di+12h] ; [ES:DI+SF_ENTRY.sf_chain+2] ; 45
32667 00005681 2689451C mov    [es:di+1Ch],ax
32668             ; mov    [es:di+SF_ENTRY.sf_lstclus+2-SF_ENTRY.sf_dirsec],ax ; 55
32669             ;;;
32670 %endif
32671
32672 ; DOS 3.3 FastOpen 6/13/86
32673
32674 00005685 1E      PUSH    DS
32675
32676 ;hkn; SS is DOSDATA
32677 00005686 16      push    ss
32678 00005687 1F      pop     ds
32679             ; test    byte [FastOpenFlg],4
32680 00005688 F606[4611]04 TEST    byte [FastOpenFlg],Special_Fill_Set
32681 0000568D 7411      JZ     short Not_FastOpen
32682
32683 ;hkn; FastOpen_Ext_Info is in DOSDATA
32684 0000568F BE[4711] MOV     SI,FastOpen_Ext_Info
32685
32686             ; mov    ax,[si+1]
32687 00005692 8B4401 MOV    AX,[SI+FEI.dirsec]
32688 00005695 AB      STOSW             ; sf_dirsec
32689             ; MSDOS 6.0
32690             ; mov    ax,[si+3]
32691 00005696 8B4403 MOV    AX,[SI+FEI.dirsec+2]
32692             ;;; changed for >32mb
32693 00005699 AB      STOSW             ; sf_dirsec
32694             ; 19/08//2018
32695 0000569A 8A04   mov    al,[SI]
32696             ; MOV    AL,[SI+FEI.dirpos] ; mov al,[SI+0]
32697 0000569C AA      STOSB             ; sf_dirpos
32698 0000569D 1F      POP     DS
32699             ; JMP     short Next_Name
32700             ; 24/09/2023
32701 0000569E EB1E   jmp     short FILE_SFT2          ; cf=0 (after 'test' instruction)
32702
32703 ; DOS 3.3 FastOpen 6/13/86
32704
32705 Not_FastOpen:
32706             ; POP     DS          ; normal path
32707
32708 ;hkn; SS override
32709             ; MOV    SI,[SS:CURBUF] ; DS:SI->buffer header
32710             ; 16/12/2022
32711             ; 28/07/2019
32712 000056A0 8B36[E205] mov    si,[CURBUF]
32713 000056A4 1F      pop     ds
32714             ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
32715             ; pop     ds
32716             ; mov    si,[ss:CURBUF]
32717
32718             ; mov    ax,[si+6]
32719 000056A5 8B4406 MOV    AX,[SI+BUFFINFO.buf_sector] ; F.C. >32mb ; AN000;
32720 000056A8 AB      STOSW             ; sf_dirsec ; F.C. >32mb ; AN000;
32721             ; 19/05/2019

```



```

32722      ; MSDOS 6.0
32723      ;mov ax,[si+8]
32724 000056A9 8B4408      MOV AX,[SI+BUFFINFO.buf_sector+2] ;F.C. >32mb ;AN000;
32725 000056AC AB          STOSW      ; sf_dirsec ;F.C. >32mb ;AN000;
32726
32727 000056AD 89D8      MOV AX,BX
32728      ;;;add si,16 ; MSDOS 3.3
32729      ;;;add si,20 ; MSDOS 6.0
32730      ;;;add si,24 ; PC DOS 7.1 ; 23/02/2024
32731 000056AF 83C618      ADD SI,BUFINSIZ ; DS:SI-> start of data in buffer
32732 000056B2 29F0      SUB AX,SI ; AX = BX relative to start of sector
32733      ;mov cl,32
32734 000056B4 B120      MOV CL,dir_entry.size
32735 000056B6 F6F1      DIV CL
32736 000056B8 AA          STOSB      ; sf_dirpos
32737
32738 000056B9 EB03      Next_Name:
32739      JMP SHORT FILE_SFT2
32740
32741      ; 24/09/2023
32742      ; cf=0 (after 'or' instruction)
32743 DEV_SFT3:
32744      ;add di,7
32745 000056BB 83C707      ADD DI,SF_ENTRY.sf_name-SF_ENTRY.sf_cluspos
32746 FILE_SFT2:
32747      ; Copy in the object's name
32748
32749 000056BE 89DE      MOV SI,BX ; DS:SI points to dir_name
32750 000056C0 B90B00      MOV CX,11
32751 000056C3 F3A4      REP MOVSB ; sf_name
32752 000056C5 5E          POP SI ; recover DS:SI -> dir_first
32753
32754      ;hkn; SS is DOSDATA
32755 000056C6 16          push ss
32756 000056C7 1F          pop ds
32757      ; 24/09/2023
32758      ; cf=0
32759      ;CLC
32760 000056C8 C3          retn
32761
32762      ;-----
32763      ;
32764      ; Procedure Name : FREEENT
32765      ;
32766      ; Inputs:
32767      ; ES:BP -> DPB
32768      ; [CURBUF] Set
32769      ; [CURBUF+2]:BX points to directory entry
32770      ; [CURBUF+2]:SI points to above dir_first
32771      ; Function:
32772      ; Free the cluster chain for the entry if present
32773      ; Outputs:
32774      ; Carry set if error (currently user FAILED to I 24)
32775      ; (NOTE dir_firclus and dir_size_l/h are wrong)
32776      ; DS BX SI ES BP preserved (BX,SI in meaning, not value) others destroyed
32777      ;-----
32778
32779      ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32780      ; 24/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
32781
32782 FREEENT:
32783 000056C9 1E          PUSH DS
32784 000056CA C53E[E205]    LDS DI,[CURBUF]
32785 000056CE 8B0C          MOV CX,[SI] ; Get pointer to clusters
32786      ; 24/02/2023 (PC DOS 7.1 IBMDOS.COM)
32787      ;;;
32788 000056D0 8B44FA      mov ax,[si-6] ; hw of first cluster (dir_first)
32789      ;;;
32790      ; 19/05/2019 - Retro DOS v4.0
32791      ; MSDOS 6.0
32792      ;mov dx,[di+8] ; 24/02/2024
32793 000056D3 8B5508      MOV DX,[DI+BUFFINFO.buf_sector+2] ;F.C. >32mb ;AN000;
32794      ;hkn; SS override
32795 000056D6 368916[0706]    MOV [SS:HIGH_SECTOR],DX ;F.C. >32mb ;AN000;
32796      ;mov dx,[di+6] ; 24/02/2024
32797 000056DB 8B5506      MOV DX,[DI+BUFFINFO.buf_sector]
32798 000056DE 1F          POP DS
32799
32800      ; 24/02/2023 (PC DOS 7.1 IBMDOS.COM)
32801      ;;;
32802      ;cmp word [bp+0Fh],0
32803 000056DF 26837E0F00      cmp word [es:bp+DPB.FAT_SIZE],0
32804 000056E4 7515      jnz short freeent2 ; not FAT32
32805      ;cmp ax,0
32806 000056E6 09C0      or ax,ax
32807 000056E8 7505      jnz short freeent1
32808      ;
32809 000056EA 83F902      cmp cx,2
32810 000056ED 7242      jb short RET1 ; was 0 length file (or mucked Firclus if AX:CX=1)
32811
32812 freeent1:
32813 000056EF 263B462F      ;cmp ax,[es:bp+2Fh]
32814 000056F3 7511      cmp ax,[es:bp+DPB.LAST_CLUSTER+2]
32815      ;jne short freeent3
32816 000056F5 263B4E2D      ;cmp cx,[es:bp+2Dh]
32817 000056F9 EB0B      cmp cx,[es:bp+DPB.LAST_CLUSTER]
32818      ;jmp short freeent3
32819 000056FB 31C0      freeent2:
32820      xor ax,ax
32821      ;;;
32822 000056FD 83F902      CMP CX,2
32823 00005700 722F      JB short RET1 ; was 0 length file (or mucked Firclus if CX=1)
32824
32825      ;cmp cx,[es:bp+0Dh]
32826 00005702 263B4E0D      CMP CX,[ES:BP+DPB.MAX_CLUSTER]
32827      ; 24/02/2024
32828      ;JA short RET1 ; Treat like zero length file (firclus mucked)
32829 00005706 7718      ja short freeent_retn ; 24/02/2024
32830 00005708 29FB      SUB BX,DI
32831 0000570A 53          PUSH BX ; Save offset
32832 0000570B FF36[0706]    PUSH word [HIGH_SECTOR] ;F.C. >32mb ;AN000;
32833 0000570F 52          PUSH DX ; Save sector number
32834 00005710 89CB      MOV BX,CX
32835      ; 24/02/2023 (PC DOS 7.1 IBMDOS.COM)
32836      ;;;
32837 00005712 A3[E80A]      mov [CLUSTNUM_HW],ax
32838      ;;;
32839 00005715 E8B404      call RELEASE ; Free any data allocated
32840 00005718 5A          POP DX
32841 00005719 8F06[0706]    POP word [HIGH_SECTOR] ;F.C. >32mb ;AN000;
32842 0000571D 7302      JNC short GET_BUF_BACK
32843 0000571F 5B          POP BX
32844
32845 00005720 C3          freeent_retn:
32846      retn ; Screw up

```

```

32846
32847
32848 GET_BUF_BACK:
32849 ; 22/09/2023
32850 ; mov byte [ALLOWED],18h
32851 ; MOV byte [ALLOWED],Allowed_RETRY+Allowed_FAIL ; *
32852 ; XOR AL,AL ; *
32853 ; call GETBUFR ; Get sector back
32854 ; call GETBUFFER ; * ; pre read
32855 00005721 E80012
32856
32857 POP BX ; Get offset back
32858 jc short freent_retn
32859 call SET_BUF_AS_DIR
32860 ADD BX,[CURBUF] ; Correct it for new buffer
32861
32862 ; MOV SI,BX
32863 ; add si,1Ah
32864 ; ADD SI,dir_entry.dir_first; Get corrected SI
32865 ; 24/02/2024 (PCDOS 7.1 IBMDOS.COM)
32866 ; lea si,[bx+1Ah]
32867 0000572E 8D771A
32868 lea si,[bx+dir_entry.dir_first]
32869 RET1:
32870 CLC
32871 retn
32872
32873 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
32874 %if 0
32875
32876 ; -----
32877 ; Procedure Name : CHECK_VIRT_OPEN
32878 ;
32879 ; CHECK_VIRT_OPEN checks to see if we had performed a "virtual open" (by
32880 ; examining the flag [VIRTUAL_OPEN] to see if it is 1). If we did, then
32881 ; it calls Dev_Close_SFT to decrement the ref. count. It also resets the
32882 ; flag [VIRTUAL_OPEN].
32883 ; No registers affected (including flags).
32884 ; On input, [THISFT] points to current SFT.
32885 ; -----
32886
32887 CHECK_VIRT_OPEN:
32888 PUSH AX
32889 lahf ; preserve flags
32890 CMP byte [VIRTUAL_OPEN],0
32891 JZ short ALL_CLOSED
32892 mov byte [VIRTUAL_OPEN],0 ; reset flag
32893 push es
32894 push di
32895 LES DI,[THISFT]
32896 call DEV_CLOSE_SFT
32897 pop di
32898 pop es
32899
32900 ALL_CLOSED:
32901 sahf ; restore flags
32902 POP AX
32903 retn
32904
32905 %endif
32906
32907 ; =====
32908 ; ROM.ASM, MSDOS 6.0, 1991
32909 ; =====
32910 ; 29/07/2018 - Retro DOS v3.0
32911 ; 20/05/2019 - Retro DOS v4.0
32912 ; 10/02/2024 - Retro DOS v5.0
32913
32914 ; TITLE ROM - Miscellaneous routines
32915 ; NAME ROM
32916
32917 ;** Misc Low level routines for doing simple FCB computations, Cache
32918 ; reads and writes, I/O optimization, and FAT allocation/deallocation
32919 ;
32920 ; SKPCLP
32921 ; FNDCLUS
32922 ; BUFSEC
32923 ; BUFRD
32924 ; BUFWRT
32925 ; NEXTSEC
32926 ; OPTIMIZE
32927 ; FIGREC
32928 ; ALLOCATE
32929 ; RESTFATBYT
32930 ; RELEASE
32931 ; RELBLKS
32932 ; GETEOF
32933
32934 ; Modification history:
32935 ;
32936 ; Created: ARR 30 March 1983
32937 ; M039: DB 10/25/90 - Disk read/write optimization.
32938
32939 ;Break <FNDCLUS -- Skip over allocation units>
32940
32941 ; -----
32942 ; Procedure Name : FNDCLUS
32943 ;
32944 ; Inputs:
32945 ; CX = No. of clusters to skip
32946 ; ES:BP = Base of drive parameters
32947 ; [THISFT] point to SFT
32948 ; Outputs:
32949 ; BX = Last cluster skipped to
32950 ; CX = No. of clusters remaining (0 unless EOF)
32951 ; DX = Position of last cluster
32952 ; Carry set if error (currently user FAILED to I 24)
32953 ; DI destroyed. No other registers affected.
32954 ; -----
32955
32956 ; 10/02/2024 - Retro DOS v5.0
32957 ; (PCDOS 7.1 IBMDOS.COM)
32958 FNDCLUS_X:
32959 mov cx,[CLUSNUM]
32960 mov dx,[CLUSNUM_HW]
32961 ;;;
32962
32963 ; 20/05/2019 - Retro DOS v4.0
32964 ; DOSCODE:8BF2h (MSDOS 6.21, MSDOS.SYS)
32965 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32966 ; DOSCODE:8BB7h (MSDOS 5.0, MSDOS.SYS)
32967
32968 ; 25/02/2024
32969 ; 10/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)

```

```

32970 ; DOSCODE:9801h (PCDOS 7.1 IBMDOS.COM)
32971
32972 FNDCLUS:
32973 PUSH ES
32974 LES DI,[THISSFT] ; setup addressability to SFT
32975
32976 ;;;
32977 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
32978 mov [CLSKIP_HW],dx
32979 ;mov bx,[es:di+37h]
32980 mov bx,[es:di+SF_ENTRY.sf_lstclus+2] ; 24/02/2024
32981 mov [CLUSTNUM_HW],bx
32982 or bx,bx ; *
32983 mov dx,[es:di+0Bh] ; [ES:DI+SF_ENTRY.sf_fircclus] ; MSDOS 5.0-6.22
32984 ;mov dx,[es:di+SF_ENTRY.sf_cluspos_hw] ; PCDOS 7.1
32985 mov [LASTPOS_HW],dx
32986 ;;;
32987
32988 ;;mov bx,[es:di+1Bh] ; MSDOS 3.3
32989 ;mov bx,[es:di+35h] ; MSDOS 6.0
32990 MOV BX,[ES:DI+SF_ENTRY.sf_lstclus]
32991 ;mov dx,[es:di+19h]
32992 MOV DX,[ES:DI+SF_ENTRY.sf_cluspos]
32993 ; 10/02/2024
32994 ;OR BX,BX
32995 ;JZ short NOCLUS
32996 ;;;
32997 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
32998 jnz short fndclus_1 ; *
32999 or bx,bx
33000 jnz short fndclus_1
33001 jmp NOCLUS
33002
33003 fndclus_1:
33004 ;;;
33005
33006 ; 10/02/2024
33007 %if 0
33008 SUB CX,DX
33009 JNB short FINDIT
33010 ADD CX,DX
33011 XOR DX,DX
33012 ;mov bx,[es:di+0Bh]
33013 MOV BX,[ES:DI+SF_ENTRY.sf_fircclus]
33014 FINDIT:
33015 POP ES
33016 JCXZ RET9
33017 %else
33018 ;;;
33019 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
33020 sub cx,dx
33021 push ax
33022 mov ax,[LASTPOS_HW]
33023 sbb [CLSKIP_HW],ax
33024 pop ax
33025 jnb short FINDIT
33026 mov bx,[LASTPOS_HW]
33027 add cx,dx
33028 adc [CLSKIP_HW],bx
33029 xor dx,dx
33030 mov [LASTPOS_HW],dx ; 0
33031 mov bx,[es:di+2Dh] ; [ES:DI+SF_ENTRY.sf_chain+2] ; MSDOS 5.0-6.22
33032 ; [ES:DI+SF_ENTRY.sf_fcluster+2] ; PCDOS 7.1
33033 mov [CLUSTNUM_HW],bx
33034 mov bx,[es:di+2Bh] ; [ES:DI+SF_ENTRY.sf_chain] ; MSDOS 5.0-6.22
33035 ; [ES:DI+SF_ENTRY.sf_fcluster] ; PCDOS 7.1
33036 FINDIT:
33037 pop es
33038 cmp cx,[CLSKIP_HW]
33039 jnz short SKPCLP
33040 jcxz RET9 ; cf=0
33041 ;;;
33042 %endif
33043
33044 ;entry SKPCLP
33045 SKPCLP:
33046
33047 ; 10/02/2024
33048 %if 0
33049 call UNPACK
33050 jc short fndclus_retn ; retc
33051
33052 ; 09/09/2018
33053
33054 ; MSDOS 3.3
33055 ;push bx
33056 ;mov bx,di
33057 ;call ISEOF
33058 ;pop bx
33059 ;jae short RET9
33060
33061 ; 20/05/2019 - Retro DOS v4.0
33062
33063 ; MSDOS 6.0
33064 xchg bx,di
33065 call ISEOF
33066 xchg bx,di
33067 jae short RET9
33068
33069 XCHG BX,DI
33070 INC DX
33071
33072 LOOP SKPCLP ; RMFS
33073 %else
33074 ;;;
33075 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
33076
33077 ;push word [LASTPOS_HW]
33078 ;push dx
33079 ;push word [CLSKIP_HW]
33080 ;push cx
33081
33082 call UNPACK
33083
33084 ;pop cx
33085 ;pop word [CLSKIP_HW]
33086 ;pop dx
33087 ;pop word [LASTPOS_HW]
33088
33089 jc short fndclus_retn
33090
33091 push word [CLUSTNUM_HW]
33092 push bx
33093 mov bx,di

```

```

33094 000057A4 8B3E[EC0A]      mov     di,[CCONTENT_HW]
33095 000057A8 893E[E80A]      mov     [CLUSTNUM_HW],di
33096 000057AC E8040B      call    IsEOF
33097 000057AF 7206          jc      short SKPCLP2
33098 000057B1 5B          pop     bx
33099 000057B2 8F06[E80A]      pop     word [CLUSTNUM_HW]
33100                      ;jmp     short RET9
33101                      ; 25/02/2024
33102 000057B6 C3          retn
33103
33104 SKPCLP2:
33105 000057B7 83C404      add     sp,4
33106 000057BA 42          inc     dx
33107 000057BB 7504          jnz     short SKPCLP3
33108 000057BD FF06[E20A]      inc     word [LASTPOS_HW]
33109 SKPCLP3:
33110 000057C1 83E901      sub     cx,1
33111 000057C4 831E[F80A]00      sbb     word [CLSKIP_HW],0
33112 000057C9 75CD          jnz     short SKPCLP      ; loop
33113 000057CB 09C9          or      cx,cx
33114 000057CD 75C9          jnz     short SKPCLP      ; loop
33115                      ;;;
33116 %endif
33117
33118 RET9:
33119          ;CLC
33120          ; 25/02/2024
33121          ; cf=0
33122 000057CF C3          retn
33123
33124 NOCLUS:
33125 000057D0 07          POP     ES
33126
33127          ; 10/02/2024
33128 %if 0
33129          INC     CX
33130          DEC     DX
33131 %else
33132          ;;;
33133          ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
33134          ;push    di
33135          ;xor     di,di ; 0
33136          ;add     cx,1
33137          ;adc     [CLSKIP_HW],di ; CLSKIP_HW:BX = Last cluster skipped to
33138          ;sub     dx,1
33139          ;sbb     [LASTPOS_HW],di      ; LASTPOS_HW:DX = Position of last cluster
33140          ;pop     di
33141          ;;;
33142          ; 25/02/2024 - Retro DOS v5.0
33143 000057D1 4A          dec     dx
33144 000057D2 7904          jns     short noclus1
33145 000057D4 FF0E[E20A]      dec     word [LASTPOS_HW]
33146 noclus1:
33147 000057D8 41          inc     cx
33148 000057D9 7504          jnz     short fndclus_retn
33149 000057DB FF06[F80A]      inc     word [CLSKIP_HW]
33150 %endif
33151          ; 25/02/2024
33152          ; cf=0
33153          ;CLC
33154
33155 fndclus_retn:
33156 000057DF C3          retn
33157
33158 ;Break <BUFSEC -- BUFFER A SECTOR AND SET UP A TRANSFER>
33159 ;-----
33160 ;
33161 ; Procedure Name : BUFSEC
33162 ;
33163 ; Inputs:
33164 ; AH = priority of buffer
33165 ; AL = 0 if buffer must be read, 1 if no pre-read needed
33166 ; ES:BP = Base of drive parameters
33167 ; [CLUSNUM] = Physical cluster number
33168 ; [SECCLUSPOS] = Sector position of transfer within cluster
33169 ; [BYTCNT1] = Size of transfer
33170 ; Function:
33171 ; Insure specified sector is in buffer, flushing buffer before
33172 ; read if necessary.
33173 ; Outputs:
33174 ; ES:DI = Pointer to buffer
33175 ; SI = Pointer to transfer address
33176 ; CX = Number of bytes
33177 ; [NEXTADD] updated
33178 ; [TRANS] set to indicate a transfer will occur
33179 ; Carry set if error (user FAILED to I 24)
33180 ;-----
33181
33182          ; 25/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
33183          ; PCDOS 7.1 IBMDOS.COM - DOSCODE:98C2h
33184
33185          ; (MSDOS 5.0 MSDOS.SYS - DOSCODE:8BF1h) - Retro DOS v4.0 (& v4.1)
33186          ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8C2Ch) - Retro DOS v4.2
33187          ; (Windows ME IO.SYS - BIOSCODE:0BE33h)
33188
33189 BUFSEC:
33190          ; 25/02/2024
33191          ;;;
33192          ;push    word [CLUSTNUM_HW]
33193 000057E0 8B16[DE0A]      mov     dx,[CLUSNUM_HW]
33194          ;mov     [CLUSTNUM_HW],dx
33195 000057E4 8716[E80A]      xchg    dx,[CLUSTNUM_HW]
33196 000057E8 52          push    dx      ; [CLUSTNUM_HW]
33197          ;;;
33198 000057E9 8B16[BC05]      MOV     DX,[CLUSNUM]
33199 000057ED 8A1E[7305]      MOV     BL,[SECCLUSPOS]
33200          ;mov     byte [ALLOWED],38h
33201 000057F1 C606[4B03]38      MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY+Allowed_IGNORE
33202 000057F6 E89A01      CALL    FIGREC
33203 000057F9 E82F11      call    GETBUFFR
33204          ; 25/02/2024
33205          ;;;
33206 000057FC 8F06[E80A]      pop     word [CLUSTNUM_HW]
33207          ;;;
33208 00005800 72DD          jc      short fndclus_retn
33209
33210 00005802 C606[7405]01      MOV     BYTE [TRANS],1      ; A transfer is taking place
33211 00005807 8B36[B805]      MOV     SI,[NEXTADD]
33212 0000580B 89F7          MOV     DI,SI
33213 0000580D 8B0E[D205]      MOV     CX,[BYTCNT1]
33214 00005811 01CF          ADD     DI,CX
33215 00005813 893E[B805]      MOV     [NEXTADD],DI
33216 00005817 C43E[E205]      LES     DI,[CURBUF]
33217          ;or      byte [es:di+5],8

```

```

33218 0000581B 26804D0508      OR      byte [ES:DI+BUFFINFO.buf_flags],buf_isDATA
33219      ;;lea di,[di+16] ; MSDOS 3.3
33220      ;;lea di,[di+20] ; MSDOS 6.0
33221      ;lea      di,[di+24] ; PC DOS 7.1 ; 25/02/2024
33222 00005820 8D7D18      LEA      DI,[DI+BUFINSIZ] ; Point to buffer
33223 00005823 033E[CC05]      ADD      DI,[BYTSECPOS]
33224 00005827 F8      CLC ; (not necessary!?) ; 25/02/2024
33225 00005828 C3      retn
33226
33227      ;Break <BUFRD, BUFVRT -- PERFORM BUFFERED READ AND WRITE>
33228
33229      ;-----
33230      ; Procedure Name : BUFRD
33231      ;
33232      ; Do a partial sector read via one of the system buffers
33233      ; ES:BP Points to DPB
33234      ; Carry set if error (currently user FAILED to I 24)
33235      ;
33236      ; DS - set to DOSDATA
33237      ;
33238      ;-----
33239
33240      ; 25/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
33241      ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33242      ; 20/05/2019 - Retro DOS v4.0
33243
33244      BUFRD:
33245          PUSH      ES
33246          xor ax,ax      ; pre-read sector
33247          CALL      BUFSEC
33248          JNC short BUF_OK ; ds=ss
33249
33250      BUF_IO_FAIL:      ; this label used by BUFVRT also
33251          POP ES
33252          JMP      SHORT RBUFPLACED ; ds=ss
33253
33254      BUF_OK:
33255          MOV      BX,ES
33256          MOV      ES,[DMAADD+2]
33257          MOV      DS,BX
33258          XCHG     DI,SI
33259          SHR      CX,1
33260      ;M039
33261      ; MSDOS 3.3
33262      ;JNC short EVENRD
33263      ;MOVSB
33264      ;EVENRD:
33265      ;REP      MOVSW
33266
33267      ; CX = # of whole WORDs ; CF=1 if odd # of bytes.
33268      ; DS:SI-> Source within Buffer.
33269      ; ES:DI-> Destination within Transfer memory block.
33270
33271      ; MSDOS 6.0
33272      rep movsw      ;Copy Buffer to Transfer memory.
33273      ;adc      cx,0      ;CX=1 if odd # of bytes, else CX=0.
33274      ;rep movsb      ;Copy last byte.
33275      ; 16/12/2022
33276      jnc short EVENRD ; **** 20/05/2019
33277      movsb ; ****
33278      ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
33279      ;adc      cx,0
33280      ;rep movsb
33281      ;M039
33282      EVENRD: ; ****
33283          POP ES
33284      ;hkn; SS override
33285          LDS      DI,[SS:CURBUF]
33286      ;;lea bx,[di+16]
33287      ;;lea bx,[di+20] ; MSDOS 6.0
33288      ;lea bx,[di+24] ; PC DOS 7.1; 25/02/2024
33289      LEA      BX,[DI+BUFINSIZ]
33290          SUB      SI,BX      ; Position in buffer
33291          call     PLACEBUF
33292      ;cmp      si,[es:bp+2]
33293      CMP      SI,[ES:BP+DPB.SECTOR_SIZE] ; Read Last byte?
33294      JB short RBUFPLACEDC ; ds<=ss ; No, leave buf where it is
33295      ;M039
33296      ; MSDOS 3.3
33297      ;call     PLACEHEAD      ; Make it prime candidate for chucking
33298      ; even though it is MRU.
33299      ; MSDOS 6.0
33300      MOV      [ss:BufferQueue],DI ; Make it prime candidate for
33301      ;M039      ; chucking even though it is MRU.
33302
33303      RBUFPLACEDC:
33304          CLC
33305      ;RBUFPLACED:
33306          push     ss
33307          pop      ds
33308      RBUFPLACED: ; 25/02/2024 (ds=ss)
33309          retn
33310
33311      ;-----
33312      ; Procedure : BUFVRT
33313      ;
33314      ; Do a partial sector write via one of the system buffers
33315      ; ES:BP Points to DPB
33316      ; Carry set if error (currently user FAILED to I 24)
33317      ;
33318      ; DS - set to DOSDATA
33319      ;
33320      ;-----
33321
33322      ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33323      ; 20/05/2019 - Retro DOS v4.0
33324
33325      BUFVRT:
33326          ;MOV      AX,[SECPOS]
33327          ; MSDOS 6.0
33328          ;ADD      AX,1      ; Set for next sector
33329          ;MOV      [SECPOS],AX ; F.C. >32mb ; AN000;
33330          ;ADC      word [SECPOS+2],0 ; F.C. >32mb ; AN000;
33331          ; 24/09/2023
33332          inc      word [SECPOS]
33333          jnz      short bufw_secpos
33334          inc      word [SECPOS+2]
33335      bufw_secpos:
33336          MOV      AX,[SECPOS+2] ; F.C. >32mb ; AN000;
33337          CMP      AX,[VALSEC+2] ; F.C. >32mb ; AN000;
33338          MOV      AL,1 ; F.C. >32mb ; AN000;
33339          JA short NOREAD ; F.C. >32mb ; AN000;
33340          JB short _doread ; F.C. >32mb ; AN000;
33341          MOV      AX,[SECPOS] ; F.C. >32mb ; AN000;

```

```

33342
33343 ; MSDOS 3.3
33344 ;INC AX
33345 ;MOV [SECPOS],AX ; 09/09/2018
33346
33347 ; 20/05/2019
33348 ; MSDOS 3.3 & MSDOS 6.0
33349 0000587C 3B06[C805] CMP AX,[VALSEC] ; Has sector been written before?
33350 00005880 B001 MOV AL,1
33351 00005882 7702 JA short NOREAD ; Skip pre-read if SECPOS>VALSEC
33352
33353 00005884 30C0 _doread:
33354 XOR AL,AL
33355 00005886 06 NOREAD:
33356 00005887 E856FF PUSH ES
33357 0000588A 72A5 CALL BUFSEC
33358 0000588C 8E1E[2E03] JC short BUF_IO_FAIL
33359 00005890 D1E9 MOV DS,[DMAADD+2]
33360 SHR CX,1
33361 ;M039
33362 ; MSDOS 3.3
33363 ;JNC short EVENWRT ; 09/09/2018
33364 ;MOVSB
33365 ;EVENWRT:
33366 ;REP MOVSW
33367
33368 ; CX = # of whole WORDs; CF=1 if odd # of bytes.
33369 ; DS:SI-> Source within Transfer memory block.
33370 ; ES:DI-> Destination within Buffer.
33371
33372 00005892 F3A5 ; MSDOS 6.0
33373 rep movsw ;Copy Transfer memory to Buffer.
33374 ;adc cx,0 ;Cx=1 if odd # of bytes, else Cx=0.
33375 ;rep movsb ;Copy last byte.
33376 ; 16/12/2022
33377 00005894 7301 jnc short EVENWRT ; **** 20/05/2019
33378 00005896 A4 movsb ; ****
33379 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
33380 ;adc cx,0
33381 ;rep movsb
33382 ;M039
33383 00005897 07 EVENWRT: ; ****
33384 POP ES
33385
33386 00005898 36C51E[E205] ;hkn; SS override
33387 LDS BX,[SS:CURBUF]
33388
33389 0000589D F6470540 ; MSDOS 6.0
33390 TEST byte [BX+BUFFINFO.buf_flags],buf_dirty
33391 ;LB. if already dirty ;AN000;
33392 000058A1 7507 JNZ short yesdirty10 ;LB. don't increment dirty count ;AN000;
33393 000058A3 E81F13 call INC_DIRTY_COUNT ;LB. ;AN000;
33394
33395 000058A6 804F0540 ;or byte [bx+5],40h
33396 OR byte [BX+BUFFINFO.buf_flags],buf_dirty
33397 yesdirty10:
33398 ;;;lea si,[bx+16]
33399 ;;;lea si,[bx+20] ; MSDOS 6.0
33400 000058AA 8D7718 LEA SI,[bx+24] ; PCDOS 7.1; 25/02/2024
33401 000058AD 29F7 SUB DI,SI ; Position in buffer
33402 ;M039
33403 ; MSDOS 3.3
33404 ;MOV SI,DI
33405 ;MOV DI,BX
33406 ;call PLACEBUF
33407 ;cmp si,[es:bp+2]
33408 ;CMP SI,[ES:BP+DPB.SECTOR_SIZE] ; written last byte?
33409 ;JB short WBUFPLACED ; No, leave buf where it is
33410 ;call PLACEHEAD ; Make it prime candidate for chucking
33411 ; even though it is MRU.
33412 ; 10/02/2024
33413 000058AF 16 push ss
33414 000058B0 1F pop ds
33415
33416 ; MSDOS 6.0
33417 ;cmp di,[es:bp+2]
33418 000058B1 263B7E02 CMP di,[ES:BP+DPB.SECTOR_SIZE] ; written last byte?
33419 000058B5 7204 JB short WBUFPLACED ; No, leave buf where it is
33420
33421 ; 10/02/2024
33422 ;MOV [ss:BufferQueue],BX ; Make it prime candidate for
33423 ; chucking even though it is MRU.
33424 000058B7 891E[6D00] mov [BufferQueue],bx
33425 ;M039
33426
33427 WBUFPLACED:
33428 000058BB F8 CLC
33429 ; 10/02/2024
33430 ;push ss
33431 ;pop ds
33432 000058BC C3 retn
33433
33434 ;Break <NEXTSEC -- Compute next sector to read or write>
33435 ;-----
33436 ;
33437 ; Procedure Name : NEXTSEC
33438 ;
33439 ; Compute the next sector to read or write
33440 ; ES:BP Points to DPB
33441 ;-----
33442 ;
33443 ;
33444 ; 29/02/2024
33445 ; 26/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
33446
33447 NEXTSEC:
33448 000058BD F606[7405]FF test byte [TRANS],0FFh ; -1
33449 ;JZ short CLRET
33450 ; 29/02/2024
33451 000058C2 743D jz short CLRET2
33452
33453 000058C4 A0[7305] MOV AL,[SECCLUSPOS]
33454 000058C7 FEC0 INC AL
33455 ;cmp al,[es:bp+4]
33456 000058C9 263A4604 CMP AL,[ES:BP+DPB.CLUSTER_MASK]
33457 000058CD 762E JBE short SAVPOS
33458
33459 ; 26/02/2024 (PCDOS 7.1 IBMDOS.COM)
33460 ;;;
33461 000058CF 8B1E[DE0A] mov bx,[CLUSNUM_HW]
33462 000058D3 891E[E80A] mov [CLUSTNUM_HW],bx
33463 ;;;
33464
33465 000058D7 8B1E[BC05] MOV BX,[CLUSNUM]

```



```

33466 000058DB E8D509      call    IsEOF
33467 000058DE 7322      JAE     short NONEXT
33468
33469 000058E0 E8F909      call    UNPACK
33470                      ;JC     short NONEXT
33471                      ; 26/02/2024
33472 000058E3 721E      JC     short NONEXT2
33473 clusgot:
33474                      ; 26/02/2024 - Retro DOS v5.0
33475                      ;;;
33476                      ;push    di
33477                      ;mov     di,[CONTENT_HW]
33478                      ;mov     [CLUSNUM_HW],di
33479                      ;pop     di
33480 000058E5 FF36[EC0A]    push    word [CONTENT_HW]
33481 000058E9 8F06[DE0A]    pop     word [CLUSNUM_HW]
33482                      ;;;
33483 000058ED 893E[BC05]    MOV     [CLUSNUM],DI
33484 000058F1 FF06[BA05]    INC     word [LASTPOS]
33485                      ; 26/02/2024 - Retro DOS v5.0
33486                      ;;;
33487 000058F5 7504      JNZ     short clusgot1
33488 000058F7 FF06[E20A]    INC     word [LASTPOS_HW]
33489 clusgot1:
33490                      ;;;
33491 000058FB B000      MOV     AL,0
33492 SAVPOS:
33493 000058FD A2[7305]    MOV     [SECCLUSPOS],AL
33494 CLRET:
33495 00005900 F8      CLC
33496 CLRET2:                ; 29/02/2024
33497 00005901 C3      retn
33498 NONEXT:
33499 00005902 F9      STC
33500 NONEXT2:              ; 26/02/2024
33501 00005903 C3      retn
33502
33503 ;Break    <OPTIMIZE -- DO A USER DISK REQUEST WELL>
33504 ;-----
33505 ;
33506 ; Procedure Name : OPTIMIZE
33507 ;
33508 ; Inputs:
33509 ;     BX = Physical cluster
33510 ;     CX = No. of records
33511 ;     DL = sector within cluster
33512 ;     ES:BP = Base of drive parameters
33513 ;     [NEXTADD] = transfer address
33514 ; Outputs:
33515 ;     AX = No. of records remaining
33516 ;     BX = Transfer address
33517 ;     CX = No. or records to be transferred
33518 ;     DX = Physical sector address (LOW)
33519 ;     [HIGH_SECTOR] = Physical sector address (HIGH)
33520 ;     DI = Next cluster
33521 ;     [CLUSNUM] = Last cluster accessed
33522 ;     [NEXTADD] updated
33523 ;     Carry set if error (currently user FAILED to I 24)
33524 ; ES:BP unchanged. Note that segment of transfer not set.
33525 ;-----
33526 ;
33527 ;
33528 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33529 ; 27/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
33530
33531 OPTIMIZE:
33532 00005904 52      PUSH    DX
33533                      ; 27/02/2024 (PCDOS 7.1 IBMDOS.COM)
33534                      ;;;
33535 00005905 FF36[E80A]    push    word [CLUSTNUM_HW]
33536                      ;;;
33537 00005909 53      PUSH    BX
33538                      ;mov     al,[es:bp+4]
33539 0000590A 268A4604    MOV     AL,[ES:BP+DPB.CLUSTER_MASK]
33540 0000590E FEC0      INC     AL                ; Number of sectors per cluster
33541 00005910 88C4      MOV     AH,AL
33542 00005912 28D0      SUB     AL,DL                ; AL = Num of sectors left in first cluster
33543 00005914 89CA      MOV     DX,CX
33544                      ;MOV     CX,0
33545                      ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
33546                      ; 16/12/2022
33547 00005916 31C9      XOR     CX,CX                ; sub cx,cx
33548 OPTCLUS:
33549 ; AL has number of sectors available in current cluster
33550 ; AH has number of sectors available in next cluster
33551 ; BX has current physical cluster
33552 ; CX has number of sequential sectors found so far
33553 ; DX has number of sectors left to transfer
33554 ; ES:BP Points to DPB
33555 ; ES:SI has FAT pointer
33556
33557 do_norm3:
33558 00005918 E8C109      call    UNPACK
33559 0000591B 7263      JC     short OP_ERR        ; ds=ss ; 27/02/2024
33560 ;clusgot2:
33561 0000591D 00C1      ADD     CL,AL
33562 0000591F 80D500      ADC     CH,0
33563 00005922 39D1      CMP     CX,DX
33564 00005924 735F      JAE     short BLKDON
33565 00005926 88E0      MOV     AL,AH
33566                      ;INC     BX
33567                      ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
33568                      ;;;
33569                      ; 28/06/2024
33570 00005928 57      push    di
33571                      ;xor     di,di
33572                      ;add     bx,1
33573                      ;adc     [CLUSTNUM_HW],di
33574                      ;
33575 00005929 43      inc     bx
33576 0000592A 7504      JNZ     short clusgot2
33577 0000592C FF06[E80A]    INC     word [CLUSTNUM_HW]
33578 clusgot2:
33579 00005930 8B3E[EC0A]    mov     di,[CONTENT_HW]
33580 00005934 3B3E[E80A]    cmp     di,[CLUSTNUM_HW]
33581                      ; 28/06/2024
33582 00005938 5F      pop     di
33583 00005939 7504      JNE     short clusgot3
33584 0000593B 39DF      cmp     di,bx
33585 0000593D 74D9      JE     short OPTCLUS
33586 clusgot3:
33587                      ;;;
33588                      ;CMP     DI,BX
33589                      ;JZ     short OPTCLUS

```

```

33590 ;DEC BX
33591 ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
33592 ;;;
33593 ;sub bx,1
33594 ;sbb word [CLUSTNUM_HW],0
33595 ;
33596 0000593F 4B dec bx
33597 00005940 7904 jns short FINCLUS
33598 00005942 FF0E[E80A] dec word [CLUSTNUM_HW]
33599 ;;;
33600 FINCLUS:
33601 ;MOV [CLUSNUM],BX ; Last cluster accessed
33602 ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
33603 ;;;
33604 ;push bx
33605 ;add [LASTPOS],bx
33606 ;mov bx,[CLUSTNUM_HW]
33607 ;adc [LASTPOS_HW],bx
33608 ;mov [CLUSNUM_HW],bx
33609 ;pop word [CLUSNUM]
33610 ;
33611 00005946 891E[BC05] mov [CLUSNUM],bx
33612 0000594A 011E[BA05] add [LASTPOS],bx
33613 0000594E A1[E80A] mov ax,[CLUSTNUM_HW]
33614 00005951 1106[E20A] adc [LASTPOS_HW],ax
33615 00005955 A3[DE0A] mov [CLUSNUM_HW],ax
33616 ;;;
33617 00005958 29CA SUB DX,CX ; Number of sectors still needed
33618 0000595A 52 PUSH DX
33619 0000595B 89C8 MOV AX,CX
33620 ;mul word[ES:BP+2]
33621 0000595D 26F76602 MUL word [ES:BP+DPB.SECTOR_SIZE]
33622 ; Number of sectors times sector size
33623 00005961 8B36[B805] MOV SI,[NEXTADD]
33624 00005965 01F0 ADD AX,SI ; Adjust by size of transfer
33625 00005967 A3[B805] MOV [NEXTADD],AX
33626 0000596A 58 POP AX ; Number of sectors still needed
33627 0000596B 5A POP DX ; Starting cluster
33628 ; 27/02/2024 (PCDOS 7.1 IBMDOS.COM)
33629 ;SUB BX,DX ; Number of new clusters accessed
33630 ;ADD [LASTPOS],BX
33631 ;;;
33632 0000596C 5B pop bx ; Starting cluster, hw
33633 0000596D 891E[E80A] mov [CLUSTNUM_HW],bx
33634 00005971 2916[BA05] sub [LASTPOS],dx
33635 00005975 191E[E20A] sbb [LASTPOS_HW],bx
33636 ;;;
33637 00005979 5B POP BX ; BL = sector position within cluster
33638 0000597A E81600 call FIGREC
33639 0000597D 89F3 MOV BX,SI
33640 ; 24/09/2023
33641 ; cf=0 (at the return of FIGREC)
33642 ;CLC
33643 0000597F C3 retn
33644 OP_ERR:
33645 ;ADD SP,4
33646 ; 27/02/2024 (PCDOS 7.1 IBMDOS.COM)
33647 ;;;
33648 00005980 83C406 add sp,6
33649 ;;;
33650 00005983 F9 STC
33651 00005984 C3 retn
33652 BLKDON:
33653 00005985 29D1 SUB CX,DX ; Number of sectors in cluster we don't want
33654 00005987 28CC SUB AH,CL ; Number of sectors in cluster we accepted
33655 00005989 FECC DEC AH ; Adjust to mean position within cluster
33656 0000598B 8826[7305] MOV [SECCLUSPOS],AH
33657 0000598F 89D1 MOV CX,DX ; Anyway, make the total equal to the request
33658 00005991 EBB3 JMP SHORT FINCLUS
33659 ;
33660 ;Break <FIGREC -- Figure sector in allocation unit>
33661 ;-----
33662 ;
33663 ; Procedure Name : FIGREC
33664 ;
33665 ; Inputs:
33666 ; DX = Physical cluster number
33667 ; BL = Sector position within cluster
33668 ; ES:BP = Base of drive parameters
33669 ; Outputs:
33670 ; DX = physical sector number (LOW)
33671 ; [HIGH_SECTOR] Physical sector address (HIGH)
33672 ; No other registers affected.
33673 ;-----
33674 ;
33675 ;
33676 ; 10/06/2019
33677 ; 20/05/2019 - Retro DOS v4.0
33678 ; DOSCODE:8D96h (MSDOS 6.21, MSDOS.SYS)
33679 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33680 ; DOSCODE:8D5Bh (MSDOS 5.0, MSDOS.SYS)
33681 ;
33682 ; 27/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
33683 ; DOSCODE:9A89h (PCDOS 7.1, IBMDOS.COM)
33684 FIGREC:
33685 00005993 51 PUSH CX
33686 ; 27/02/2024
33687 ;;;
33688 00005994 50 push ax
33689 00005995 31C9 xor cx,cx
33690 ;;;
33691 ;mov cl,[es:bp+5]
33692 00005997 268A4E05 MOV CL,[ES:BP+DPB.CLUSTER_SHIFT]
33693 ;DEC DX
33694 ;DEC DX
33695 ; 27/02/2024
33696 ;;;
33697 ;mov ax,[ss:CLUSTNUM_HW]
33698 ; ds = ss ; Retro DOS v5.0
33699 0000599B A1[E80A] mov ax,[CLUSTNUM_HW]
33700 0000599E 83EA02 sub dx,2
33701 000059A1 83D800 sbb ax,0
33702 000059A4 E307 jcxz noshift
33703 ;;;
33704 ;
33705 ; 27/02/2024
33706 %if 0
33707 ; MSDOS 3.3
33708 ;SHL DX,CL
33709 ;
33710 ;hkn; SS override HIGH_SECTOR
33711 ; MSDOS 6.0
33712 MOV word [SS:HIGH_SECTOR],0 ; F.C. >32mb
33713 ; 24/09/2023

```



```

33715 xor     ch,ch ;F.C. >32mb
33716 OR      CL,CL ;F.C. >32mb
33717 JZ       short noshift ;F.C. >32mb
33718 ; 27/02/2024
33719 ;XOR     CH,CH ;F.C. >32mb
rotleft: ;F.C. >32mb
33720 RLC                     ;F.C. >32mb
33721         DX,1           ;F.C. >32mb
33722         10/06/2019
33723 RCL word [ss:HIGH_SECTOR],1 ;F.C. >32mb
33724 LOOP rotleft          ;F.C. >32mb
%else
    ; 27/02/2024 - Retro DOS v5.0
    ; (PCDOS 7.1 IBMDOS.COM)
rotleft:
    cll
    rcl dx,1
    rcl ax,1
    loop rotleft
%endif

noshift:
    ; MSDOS 3.3 & MSDOS 6.0
    OR DL,BL

    ; 27/02/2024 - Retro DOS v5.0
    ;;
    cmp word [es:bp+0fh],0
    cmc word [es:bp+DPB.FAT_SIZE],cx ; 0
    jnz short noshift1 ; not FAT32
    add dx,[es:bp+29h]
    add dx,[es:bp+DPB.FCLUS_FSECTOR]
    adc ax,[es:bp+2bh]
    adc ax,[es:bp+DPB.FCLUS_FSECTOR+2]
    jmp short noshift2
noshift1:
    ;;
    add dx,[es:bp+0bh]
ADD     DX,[ES:BP+DPB.FIRST_SECTOR]
    ; MSDOS 6.0
    ; 10/06/2019
    ADC word [ss:HIGH_SECTOR],0 ;F.C. >32mb
    ; 24/09/2023
    ; cx=0
    ADC word [ss:HIGH_SECTOR],cx ; 0
    ; 27/02/2024 - Retro DOS v5.0
    ;;
    adc ax,cx ; adc ax,0
noshift2:
    mov [ss:HIGH_SECTOR],ax
    mov [HIGH_SECTOR],ax
    pop ax
    ;;
    ; MSDOS 3.3 & MSDOS 6.0
POP CX
figrec_retn:
retn

; 20/05/2019 - Retro DOS v4.0
; DOSTCODE:8DC2h (MSDOS 6.21, MSDOS.SYS)

; 30/07/2018 - Retro DOS v3.0
; IBMDOS.COM (MSDOS 3.3, 1987) - Offset

;Break <ALLOCATE -- Assign disk space>
-----
; Procedure Name : ALLOCATE - Allocate Disk Space
;
;   ALLOCATE is called to allocate disk clusters. The new clusters are
;   FAT-chained onto the end of the existing file.
;
;   The DPB contains the cluster # of the last free cluster allocated
;   (dpb_next_free). We start at this cluster and scan towards higher
;   numbered clusters, looking for the necessary free blocks.
;
;   Once again, fancy terminology gets in the way of correct coding. When
;   using next_free, start scanning AT THAT POINT and not the one following it.
;   This fixes the boundary condition bug when only free = next_free = 2.
;
;   If we get to the end of the disk without satisfaction:
;
;       if (dpb_next_free == 2) then we've scanned the whole disk.
;       return (insufficient_disk_space)
;   ELSE
;       dpb_next_free = 2; start scan over from the beginning.
;
; Note that there is no multitasking interlock. There is no race when
; examining the entries in an in-core FAT block since there will be no
; context switch. When UNPACK context switches while waiting for a FAT read
; we are done with any in-core FAT blocks, so again there is no race. The
; only special concern is that V2 and V3 MSDOS left the last allocated
; cluster as "00"; marking it EOF only when the entire alloc request was
; satisfied. We can't allow another activation to think this cluster is
; free, so we give it a special temporary mark to show that it is, indeed,
; allocated.
;
; Note that when we run out of space this algorithm will scan from
; dpb_next_free to the end, then scan from cluster 2 through the end,
; redundantly scanning the later part of the disk. This only happens when
; we run out of space, so sue me.
; -----
C A V E A T P A T T E R N S O N
;
The use of FATBYT and RESTFATBYT is somewhat mysterious. Here is the
explanation:
;
In the NUL file case (sf_firclus currently 0) ALLOCATE is called with
entry BX = 0. What needs to be done in this case is to stuff the cluster
number of the first cluster allocated in sf_firclus when the ALLOCATE is
complete. THIS VALUE IS SAVED TEMPORARILY IN CLUSTER 0, HENCE THE CURRENT
VALUE IN CLUSTER 0 MUST BE SAVED AND RESTORED. This is a side effect of
the fact that PACK and UNPACK don't treat requests for clusters 0 and 1 as
errors. This "stuff" is done by the call to PACK which is right before
the
LOOP findfre ; alloc more if needed
instruction when the first cluster is allocated to the nul file. The
value is recalled from cluster 0 and stored at sf_firclus at ads4:
;
This method is obviously useless (because it is non-reentrant) for
```

```

33838 ; multitasking will have to be changed. Storing the required value on
33839 ; the stack is recommended. Setting sf_firclus at the PACK of cluster 0
33840 ; (instead of actually doing the PACK) is BAD because it doesn't handle
33841 ; problems with INT 24 well.
33842 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
33843 ; C A V E A T P A T T E R S O N ;
33844 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
33845 ;
33846 ENTRY BX = Last cluster of file (0 if null file)
33847 CX = No. of clusters to allocate
33848 ES:BP = Base of drive parameters
33849 [THISSFT] = Points to SFT
33850 ;
33851 EXIT 'C' set if insufficient space
33852 [FAILERR] can be tested to see the reason for failure
33853 CX = max. no. of clusters that could be added to file
33854 'C' clear if space allocated
33855 BX = First cluster allocated
33856 FAT is fully updated
33857 sf_FIRCLUS field of SFT set if file was null
33858 ;
33859 USES ALL but SI, BP
33860 ;
33861 ;callmagic proc near
33862 ; push ds ;push segment of routine
33863 ; push Offset MagicPatch ;push offset for routine
33864 ; retf ;simulate jmp far
33865 ; ;far return address is on
33866 ; ;stack, so far return from
33867 ; ;call will return this routine
33868 ;callmagic endp
33869 ;
33870 ; 27/02/2024 - Retro DOS v5.0
33871 ;PCDOS 7.1 IBMDOS.COM - DOSCODE:9AC5h
33872 ;
33873 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8DC2h)
33874 ; (Windows ME IO.SYS - BIOSCODE:0c078h)
33875 ;
33876 ; 25/09/2023
33877 %if 1 ; 27/02/2024
33878 callmagic:
33879 push ds
33880 push word [ss:OffsetMagicPatch]
33881 retf
33882 %endif
33883 ;
33884 ; 10/02/2024 - Retro DOS v5.0
33885 %if 1
33886 ALLOCATE_X:
33887 mov [CCOUNT_HW],ax
33888 ;jmp short ALLOCATE
33889 %endif
33890 ;
33891 ; 27/02/2024 - Retro DOS v5.0
33892 ; PCDO$ 7.1 IBMDOS.COM - DOSCODE:9ACFh
33893 ;
33894 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8DC9h)
33895 ; (Windows ME IO.SYS - BIOSCODE:0C07Fh)
33896 ;
33897 ALLOCATE:
33898 ; 10/09/2018
33899 ;BEGIN MAGICDRV MODIFICATIONS
33900 ;
33901 ;7/5/92 scottq
33902 ;
33903 ;This is the disk compression patch location which allows
33904 ;the disk compression software to fail allocations if the
33905 ;FAT would allow allocation, but the free space for compressed
33906 ;data would not.
33907 ;
33908 ;; call far ptr MAGICPATCH
33909 ;; We cannot do a far call since we cannot have fix-ups[romdos,hidos],
33910 ;; but we do know the segment and offset of the routine
33911 ;; so simulate a far call to dosdata:magicpatch
33912 ;; note dosassume above, so DS -> dosdata
33913 ;
33914 ; MSDOS 6.0
33915 ;clc ;clear carry so we fall through
33916 ; ;if no patch is present
33917 ;push cs ;push segment for far return
33918 ;call callmagic ;this is a near call
33919 ;jnc short Regular_Allocate_Path
33920 ;jmp Disk_Full_Return
33921 ;
33922 ; 27/02/2024 - Retro DOS v5.0
33923 ; DOSCODE:9ACFh (PCDOS 7.1, IBMDOS.COM)
33924 ;
33925 ; 25/09/2023
33926 %if 1 ; 27/02/2024 - Retro DOS v5.0
33927 clc ; COUNT_HW:CX = Number of clusters to allocate
33928 push cs
33929 call callmagic
33930 jnc short Regular_Allocate_Path
33931 jmp Disk_Full_Return
33932 Regular_Allocate_Path:
33933 %endif
33934 ;
33935 ; 20/05/2019 - Retro DOS v4.0
33936 ;END MAGICDRV MODIFICATIONS
33937 ;
33938 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33939 ; DOSCODE:8D87h (MSDOS 5.0, MSDOS.SYS)
33940 ;
33941 ; 27/02/2024 - Retro DOS v5.0 (Modified PCDO$ 7.1 IBMDOS.COM)
33942 ; DOSCODE:9AD6h (PCDOS 7.1, IBMDOS.COM)
33943 ;
33944 000059DF 53 PUSH BX ; save (bx)
33945 000059E0 31DB XOR BX,BX
33946 ;
33947 ; 27/02/2024
33948 ;;
33949 000059E2 FF36[E80A] push word [CLUSTNUM_HW]
33950 000059E6 891E[E80A] mov [CLUSTNUM_HW],bx ; cluster 0
33951 ;;
33952 ;
33953 000059EA E8EF08 call UNPACK
33954 000059ED 893E[9605] MOV [FATBYT],DI ; save correct cluster 0 value
33955 ;
33956 ; 27/02/2024
33957 ;;
33958 000059F1 8B1E[EC0A] mov bx,[CONTENT_HW]
33959 000059F5 891E[E40A] mov [FATBYT_HW],bx
33960 000059F9 8F06[E80A] pop word [CLUSTNUM_HW]
33961 ;;

```

```

33962
33963 000059FD 5B          POP     BX
33964 000059FE 72CA        jc     short figrec_retn      ; abort if error [INTERR?]
33965
33966 ; 27/02/2024
33967 %if 0
33968     PUSH     CX
33969     PUSH     BX
33970
33971     MOV      DX,BX
33972 ;;mov     bx,[es:bp+1Ch] ; MSDOS 3.3
33973 ;mov     bx,[es:bp+1Dh] ; MSDOS 6.0
33974     mov     bx,[ES:BP+DPB.NEXT_FREE]
33975     cmp     bx,2
33976     ja     short FINDFRE
33977
33978 ; couldn't find enough free space beyond dpb_next_free, or dpb_next_free is
33979 ; <2 or >dpb_max_clus. Reset it and restart the scan
33980
33981 ads1:
33982 ;;mov     word [es:bp+1Ch],2 ; MSDOS 3.3
33983 ;mov     word [es:bp+1Dh],2 ; MSDOS 6.0
33984     mov     word [ES:BP+DPB.NEXT_FREE],2
33985     mov     bx,1 ; Counter next instruction so first
33986 ; cluster examined is 2
33987
33988 %else
33989 ; 27/02/2024 - Retro DOS v5.0
33990 ; (PCDOS 7.1 IBMDOS.COM)
33991 ;;
33992 push     word [CCOUNT_HW]
33993 push     cx
33994 push     word [CLUSTNUM_HW]
33995 push     bx
33996     mov     dx,bx
33997
33998     xor     bx,bx ; 0
33999     ;cmp    [es:bp+0Fh],bx ; 0
34000     cmp     [es:bp+DPB.FAT_SIZE],bx ; 0
34001     xchg    bx,[CLUSTNUM_HW]
34002     mov     [CLUSDATA_HW],bx
34003     ;mov    bx,[es:bp+1Dh]
34004     mov     bx,[es:bp+DPB.NEXT_FREE]
34005     jnz     short ads1 ; not FAT32
34006
34007     ;mov    bx,[es:bp+3Bh]
34008     mov     bx,[es:bp+DPB.FAT32_NXTFREE+2]
34009     mov     [CLUSTNUM_HW],bx
34010     ;cmp    bx,0
34011     or      bx,bx
34012     ;mov    bx,[es:bp+39h]
34013     mov     bx,[es:bp+DPB.FAT32_NXTFREE]
34014     jnz     short FINDFRE
34015 ads1:
34016     cmp     bx,2
34017     ja     short FINDFRE
34018 ads2:
34019     xor     bx,bx ; 0
34020     ;cmp    [es:bp+0Fh],bx
34021     cmp     [es:bp+DPB.FAT_SIZE],bx ; 0
34022     jnz     short ads3 ; not FAT32
34023
34024     ;mov    word [es:bp+39h],2
34025     mov     word [es:bp+DPB.FAT32_NXTFREE],2
34026     ;mov    [es:bp+3Bh],bx
34027     mov     [es:bp+DPB.FAT32_NXTFREE+2],bx ; 0
34028 ads3:
34029     mov     [CLUSTNUM_HW],bx ; 0
34030     ;mov    word [es:bp+1Dh],2
34031     mov     word [es:bp+DPB.NEXT_FREE],2
34032     inc     bx ; 1
34033     ;or     [es:bp+18h],b1
34034     or      [es:bp+DPB.FIRST_ACCESS],b1 ; 1
34035     ;;
34036 %endif
34037
34038 ; Scanning both forwards and backwards for a free cluster
34039 ;
34040 ; (BX) = forwards scan pointer
34041 ; (CX) = clusters remaining to be allocated
34042 ; (DX) = current last cluster in file
34043 ; (TOS) = last cluster of file
34044
34045 FINDFRE:
34046 %if 0
34047     INC     BX
34048     ;cmp    bx,[es:bp+0Dh]
34049     CMP     BX,[ES:BP+DPB.MAX_CLUSTER]
34050     ja     short ads7 ; at end of disk
34051     call    UNPACK ; check out this cluster
34052     jc     short ads4 ; FAT error [INTERR?]
34053     jnz     short FINDFRE ; not free, keep on truckin
34054
34055 ; Have found a free cluster. Chain it to the file
34056 ;
34057 ; (BX) = found free cluster #
34058 ; (DX) = current last cluster in file
34059
34060 ;;mov     [es:bp+1Ch],bx
34061 ;mov     [es:bp+1Dh],bx ; MSDOS 6.0
34062     mov     [ES:BP+DPB.NEXT_FREE],bx ; next time start search here
34063     xchg    ax,dx ; save (dx) in ax
34064     mov     dx,1 ; mark this free guy as "1"
34065     call    PACK ; set special "temporary" mark
34066     jc     short ads4 ; FAT error [INTERR?]
34067     ;cmp    word [es:bp+1Eh],-1
34068     ;cmp    word [es:bp+1Fh],-1 ; MSDOS 6.0
34069     CMP     word [ES:BP+DPB.FREE_CNT],-1 ; Free count valid?
34070     JZ     short NO_ALLOC ; No
34071     ;dec    word [es:bp+1Eh]
34072     dec     word [es:bp+1Fh] ; MSDOS 6.0
34073     DEC     word [ES:BP+DPB.FREE_CNT] ; Reduce free count by 1
34074 NO_ALLOC:
34075     xchg    ax,dx ; (dx) = current last cluster in file
34076     XCHG    BX,DX
34077     MOV     AX,DX
34078     call    PACK ; link free cluster onto file
34079 ; CAVEAT.. On Nul file, first pass stuffs
34080 ; cluster 0 with FIRCLUS value.
34081     jc     short ads4 ; FAT error [INTERR?]
34082     xchg    BX,AX ; (BX) = last one we looked at
34083     mov     dx,bx ; (dx) = current end of file
34084     LOOP    FINDFRE ; alloc more if needed
34085 %else

```

```

34086      ; 27/02/2024 - Retro DOS v5.0
34087      ; (PCDOS 7.1 IBMDOS.COM)
34088      ;;;
34089      ;add    bx,1
34090      ;adc    word [CLUSTNUM_HW],0
34091      inc    bx
34092      jnz    short FINDFRE2
34093      inc    word [CLUSTNUM_HW]
34094      FINDFRE2:
34095      ;cmp    word [es:bp+0Fh],0
34096      cmp    word [es:bp+DPB.FAT_SIZE],0
34097      jnz    short ads4 ; not FAT32
34098      ; FAT32
34099      push   bx
34100      mov    bx,[CLUSTNUM_HW]
34101      ;cmp    bx,[es:bp+2Fh]
34102      cmp    bx,[es:bp+DPB.LAST_CLUSTER+2]
34103      pop    bx
34104      jne    short ads5
34105      ;cmp    bx,[es:bp+2Dh]
34106      cmp    bx,[es:bp+DPB.LAST_CLUSTER]
34107      jmp    short ads5
34108      ads4:
34109      ;cmp    bx,[es:bp+0Dh]
34110      cmp    bx,[es:bp+DPB.MAX_CLUSTER]
34111      ads5:
34112      jbe    short ads6 ; jna short ads6
34113      jmp    ads15
34114      ads6:
34115      ; 27/02/2024 - Retro DOS v5.0
34116      ;push   cx
34117      ;push   word [CCOUNT_HW]
34118      ;push   bx
34119      ;push   word [CLUSTNUM_HW]
34120      ;push   dx
34121      ;push   word [CLUSDATA_HW]
34122
34123      call    UNPACK
34124
34125      ; 27/02/2024 - Retro DOS v5.0
34126      ;pop    word [CLUSDATA_HW]
34127      ;pop    dx
34128      ;pop    word [CLUSTNUM_HW]
34129      ;pop    bx
34130      ;pop    word [CCOUNT_HW]
34131      ;pop    cx
34132
34133      jnc    short ads7
34134
34135      jmp    ads13
34136      ads7:
34137      jnz    short FINDFRE
34138
34139      ;mov    [es:bp+1Dh],bx
34140      mov    [es:bp+DPB.NEXT_FREE],bx
34141
34142      ;cmp    word [es:bp+0Fh],0
34143      cmp    word [es:bp+DPB.FAT_SIZE],0
34144      jnz    short ads8 ; not FAT32
34145      ; FAT32
34146      push   bx
34147      mov    bx,[CLUSTNUM_HW]
34148      ;mov    [es:bp+3Bh],bx
34149      mov    [es:bp+DPB.FAT32_NXTFREE+2],bx
34150      pop    bx
34151      ;mov    [es:bp+39h],bx
34152      mov    [es:bp+DPB.FAT32_NXTFREE],bx
34153      ads8:
34154      ;or     byte [es:bp+18h],1
34155      or     byte [es:bp+DPB.FIRST_ACCESS],1
34156
34157      ; 27/02/2024 - Retro DOS v5.0
34158      ;push   cx
34159      ;push   word [CCOUNT_HW]
34160
34161      push   bx
34162      push   word [CLUSTNUM_HW]
34163
34164      push   dx
34165      push   word [CLUSDATA_HW]
34166
34167      xor     dx,dx ; 0
34168      mov     [CLUSDATA_HW],dx ; 0
34169      inc     dx ; 1 ; mark this free guy as "1"
34170      call    PACK ; set special "temporary" mark
34171
34172      pop     word [CLUSTNUM_HW]
34173      pop     bx
34174      pop     word [CLUSDATA_HW]
34175      pop     dx
34176
34177      ; 27/02/2024 - Retro DOS v5.0
34178      ;pop    word [CCOUNT_HW]
34179      ;pop    cx
34180
34181      jc     short ads13
34182
34183      push   ax
34184      xor     ax,ax ; 0
34185      ;cmp    [es:bp+0Fh],ax ; 0
34186      cmp    [es:bp+DPB.FAT_SIZE],ax ; 0
34187      jnz    short ads10 ; not FAT32
34188      ; FAT32
34189      dec     ax ; 0FFFFh ; -1
34190      ;cmp    [es:bp+21h],ax
34191      cmp    [es:bp+DPB.FREE_CNT+2],ax ; 0FFFFh
34192      jnz    short ads9
34193      ;cmp    [es:bp+1Fh],ax
34194      cmp    [es:bp+DPB.FREE_CNT],ax ; 0FFFFh
34195      ;ads9:
34196      jz     short NO_ALLOC ; Free count not valid
34197      ads9:
34198      ; Reduce free count by 1
34199      ;add    [es:bp+1Fh],ax
34200      add    [es:bp+DPB.FREE_CNT],ax ; -1
34201      ;adc    [es:bp+21h],ax
34202      adc    [es:bp+DPB.FREE_CNT+2],ax ; -1
34203      jmp    short NO_ALLOC
34204      ads10:
34205      dec     ax ; 0FFFFh ; -1
34206      ;cmp    [es:bp+1Fh],ax
34207      cmp    [es:bp+DPB.FREE_CNT],ax ; 0FFFFh
34208      jz     short NO_ALLOC ; Free count not valid
34209      ;dec    word [es:bp+1Fh]

```

```

34210 00005AEE 26FF4E1F      dec     word [es:bp+DPB.FREE_CNT] ; Reduce free count by 1
34211 NO_ALLOC:
34212 00005AF2 58      pop     ax
34213
34214      ; 27/02/2024 - Retro DOS v5.0
34215      ;push    cx
34216      ;push    word [CCOUNT_HW]
34217
34218 00005AF3 52      push    dx
34219 00005AF4 FF36[EA0A]    push    word [CLUSDATA_HW]
34220
34221 00005AF8 E89008      call    PACK
34222
34223 00005AFB 8F06[E80A]    pop     word [CLUSTNUM_HW]
34224 00005AFF 5B      pop     bx
34225
34226      ; 27/02/2024 - Retro DOS v5.0
34227      ;pop     word [CCOUNT_HW]
34228      ;pop     cx
34229
34230 00005B00 7227      jc     short ads13
34231
34232 00005B02 8B16[E80A]    mov     dx,[CLUSTNUM_HW]
34233 00005B06 8916[EA0A]    mov     [CLUSDATA_HW],dx
34234 00005B0A 89DA      mov     dx,bx
34235
34236 00005B0C 83E901      sub     cx,1
34237 00005B0F 831E[F00A]00    sbb     word [CCOUNT_HW],0
34238
34239 00005B14 3B0E[F00A]    cmp     cx,[CCOUNT_HW]
34240 00005B18 7502      jne     short ads11
34241 00005B1A E303      jcxz    ads12
34242 ads11:
34243 00005B1C E937FF      jmp     FINDFRE
34244      ;;;
34245 %endif
34246
34247
34248 ; we've successfully extended the file. Clean up and exit
34249 ;
34250 ; (BX) = last cluster in file
34251
34252 ads12:
34253 00005B1F BAFFFF      ; 27/02/2024
MOV     DX,0FFFFH
34254      ;;;
34255      mov     [CLUSDATA_HW],dx
34256      ;;;
34257 00005B26 E86208      call    PACK ; mark last cluster EOF
34258
34259 ; Note that FAT errors jump here to clean the stack and exit. This saves us
34260 ; 2 whole bytes. Hope its worth it...
34261 ;
34262 ; 'C' set if error
34263 ; calling (BX) and (CX) pushed on stack
34264
34265 ; 27/02/2024
34266 %if 0
34267
34268 ads4:
34269 POP     BX
34270 POP     CX ; Don't need this stuff since we're successful
34271 jc     short figrec_retn
34272 call    UNPACK ; Get first cluster allocated for return
34273 ; CAVEAT... In nul file case, UNPACKs cluster 0.
34274
34275 jc     short figrec_retn
34276 call    RESTFATBYT ; Restore correct cluster 0 value
34277 jc     short figrec_retn
34278 XCHG    BX,DI ; (DI) = last cluster in file upon our entry
34279 OR      DI,DI ; clear 'C'
34280 jnz     short figrec_retn ; we were extending an existing file
34281 %else
34282 ; 27/02/2024 - Retro DOS v5.0
34283 ; (PCDOS 7.1 IBMDOS.COM)
34284      ;;;
34285 ads13:
34286 00005B29 5B      pop     bx
34287 00005B2A 8F06[E80A]    pop     word [CLUSTNUM_HW]
34288 00005B2E 59      pop     cx
34289 00005B2F 8F06[F00A]    pop     word [CCOUNT_HW]
34290 00005B33 7301      jnc     short ads14
34291 ads_ret:
34292 retn
34293 ads14:
34294 call    UNPACK ; Get first cluster allocated for return
34295 jc     short ads_ret
34296 call    RESTFATBYT ; Restore correct cluster 0 value
34297 jc     short ads_ret
34298 push    ax
34299 mov     ax,[CCONTENT_HW]
34300 xchg    ax,[CLUSTNUM_HW]
34301 xchg    bx,di ; CLUSTNUM_HW:DI = last cluster in file upon our entry
34302 or      ax,di
34303 pop     ax
34304 jnz     short ads_ret ; we were extending an existing file
34305      ;;;
34306 %endif
34307
34308 ; we were doing the first allocation for a new file. Update the SFT cluster
34309 ; info
34310 dofastk:
34311 ; 27/02/2024
34312 %if 0
34313 ; 20/05/2019
34314 ; MSDOS 6.0
34315 ;push    dx ; * MSDOS 6.0
34316 ;mov     dl,[es:bp+0]
34317 ;MOV     DL,[ES:BP+DPB.DRIVE] ; get drive #
34318 ;mov     dl,[es:bp]
34319
34320 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
34321 ; DOSCODE:8DF9h (MSDOS 5.0, MSDOS.SYS)
34322
34323 ; 16/12/2022
34324 ;push    dx ; *
34325 ;mov     dl,[ES:BP+DPB.DRIVE]
34326 ; 15/12/2022
34327 ;mov     dl,[es:bp]
34328
34329 ; MSDOS 3.3 & MSDOS 6.0
34330 PUSH     ES
34331 LES     DI,[THISSFT]
34332 ;mov     [es:di+0Bh],bx
34333 MOV     [ES:DI+SF_ENTRY.sf_firclus],BX
;mov     [es:di+1Bh],bx ; MSDOS 3.3

```

```

34334      ;mov     [es:di+35h],bx ; MSDOS 6.0
34335      MOV     [ES:DI+SF_ENTRY.sf_1stclus],BX
34336      POP     ES
34337      ;retn
34338
34339      ;pop     dx ; * MSDOS 6.0
34340
34341      ; 16/12/2022
34342      ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
34343      ;pop     dx ; *
34344
34345      %else
34346      ; 27/02/2024 - Retro DOS v5.0
34347      ; (PCDOS 7.1 IBMDOS.COM)
34348      ;;;
34349      push     dx
34350      push     es
34351      les     di,[THISST]
34352      mov     dx,[CLUSTNUM_HW]
34353      mov     [es:di+2Bh],bx ; [es:di+SF_ENTRYT.sf_chain]
34354      mov     [es:di+2Dh],dx ; 32 bit first cluster, 1w
34355      mov     [es:di+35h],bx ; [es:di+SF_ENTRYT.sf_chain+2]
34356      mov     [es:di+37h],dx ; 32 bit first cluster, hw
34357      mov     [es:di+35h],bx ; [es:di+SF_ENTRYT.sf_1stclus]
34358      mov     [es:di+37h],dx ; 32 bit last cluster, 1w
34359      mov     [es:di+37h],dx ; [es:di+SF_ENTRYT.sf_1stclus+2]
34360      pop     es ; 32 bit last cluster, hw
34361      pop     dx
34362      ;;;
34363      %endif
34364
34365      00005B6B C3
34366      retn
34367
34368      ;** we're at the end of the disk, and not satisfied. See if we've scanned ALL
34369      ; of the disk...
34370
34371      ; 27/02/2024
34372      %if 0
34373      ads7:
34374      ;cmp     word [es:bp+1Dh],2
34375      cmp     word [ES:BP+DPB.NEXT_FREE],2
34376      jnz     short ads1 ; start scan from front of disk
34377      %else
34378      ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
34379      ;;;
34380      ads15:
34381      ;cmp     word [es:bp+0Fh],0
34382      cmp     word [es:bp+DPB.FAT_SIZE],0
34383      jz      short ads16 ; FAT32
34384      ; FAT
34385      ;cmp     word [es:bp+1Dh],2
34386      cmp     word [es:bp+DPB.NEXT_FREE],2
34387      jmp     short ads17
34388      ads16:
34389      ;cmp     word [es:bp+3Bh],0
34390      cmp     word [es:bp+DPB.FAT32_NXTFREE+2],0
34391      jnz     short ads17
34392      jnz     short ads19 ; 27/02/2024
34393      ;cmp     word [es:bp+39h],2
34394      cmp     word [es:bp+DPB.FAT32_NXTFREE],2
34395      ads17:
34396      jz      short ads18
34397      ads19:
34398      jmp     ads2 ; start scan from front of disk
34399      ;;;
34400      %endif
34401
34402      ; Sorry, we've gone over the whole disk, with insufficient luck. Lets give
34403      ; the space back to the free list and tell the caller how much he could have
34404      ; had. We have to make sure we remove the "special mark" we put on the last
34405      ; cluster we were able to allocate, so it doesn't become orphaned.
34406      ;
34407      ; (CX) = clusters remaining to be allocated
34408      ; (TOS) = last cluster of file (before call to ALLOCATE)
34409      ; (TOS+1) = # of clusters wanted to allocate
34410
34411      ads18:
34412      ; 27/02/2024
34413      POP     BX ; (BX) = last cluster of file
34414      MOV     DX,0FFFFH
34415      ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
34416      ;;;
34417      mov     [CLUSDATA_HW],dx
34418      pop     word [CLUSTNUM_HW]
34419      ;;;
34420      call    RELBLKS ; give back any clusters just allocated
34421      POP     AX
34422      ; No. of clusters requested
34423      ; Don't "retc". We are setting Carry anyway,
34424      ; Alloc failed, so proceed with return CX
34425      ; setup.
34426      ;;;
34427      ; 27/02/2024
34428      pop     word [CLUSTERS_HW]
34429      ;;;
34430      SUB     AX,CX ; AX=No. of clusters allocated
34431      ;;;
34432      ; 27/02/2024
34433      sbb     word [CLUSTERS_HW],0
34434      ;;;
34435      call    RESTFATBYT ; Don't "retc". We are setting Carry anyway,
34436      ; Alloc failed.
34437      Disk_Full_Return: ; label added for magic patch 8-6-92 scottq
34438      ; MSDOS 6.0
34439      MOV     byte [DISK_FULL],1 ;MS. indicating disk full
34440      STC
34441      retn
34442
34443      ;-----
34444      ;
34445      ; Procedure Name : RESTFATBYT
34446      ;
34447      ; SEE ALLOCATE CAVEAT
34448      ; Carry set if error (currently user FAILED to I 24)
34449      ;-----
34450      ; 27/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
34451
34452      RESTFATBYT:
34453      PUSH     BX
34454      PUSH     DX
34455      PUSH     DI
34456
34457      XOR     BX,BX

```



```

34458
34459 ; 27/02/2024 - Retro DOS v5.0
34460 ; (PCDOS 7.1 IBMDOS.COM)
34461 ;;;
34462 ;push word [CLUSTNUM_HW]
34463 ;push word [CLUSDATA_HW]
34464 ;push word [CCONTENT_HW] ; (*) (not necessary ?)
34465 00005BB5 891E[E80A] mov [CLUSTNUM_HW],bx ; 0
34466 00005BB9 8B16[E40A] mov dx,[FATBYT_HW]
34467 00005BBD 8916[EA0A] mov [CLUSDATA_HW],dx
34468 ;;;
34469
34470 00005BC1 8B16[9605] MOV DX,[FATBYT]
34471
34472 00005BC5 E8C307 call PACK
34473
34474 ; 27/02/2024 - Retro DOS v5.0
34475 ; (PCDOS 7.1 IBMDOS.COM)
34476 ;;;
34477 ; 28/06/2024 ; (*)
34478 ;pop word [CCONTENT_HW]
34479 ;pop word [CLUSDATA_HW]
34480 ;pop word [CLUSTNUM_HW]
34481 ;;;
34482
34483 00005BC8 5F POP DI
34484 00005BC9 5A POP DX
34485 00005BCA 5B POP BX
34486
34487 ; 16/12/2022
34488 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
34489 ;RELEASE_flush:
34490 00005BCB C3 retn
34491
34492 ;Break <RELEASE -- DEASSIGN DISK SPACE>
34493 -----
34494 ;
34495 ; Procedure Name : RELEASE
34496 ;
34497 ; Inputs:
34498 ; BX = Cluster in file
34499 ; ES:BP = Base of drive parameters
34500 ; Function:
34501 ; Frees cluster chain starting with [BX]
34502 ; Carry set if error (currently user FAILED to I 24)
34503 ; AX,BX,DX,DI all destroyed. Other registers unchanged.
34504 ;
34505 -----
34506
34507 ; 28/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
34508 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9D13h
34509
34510 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8E86h)
34511 ; (Windows ME IO.SYS - BIOSCODE:0C27Fh)
34512
34513 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
34514 ; 20/05/2019 - Retro DOS v4.0
34515 RELEASE:
34516 00005BCC 31D2 XOR DX,DX
34517
34518 ; 28/02/2024 - Retro DOS v5.0
34519 ; (PCDOS 7.1 IBMDOS.COM)
34520 ;;;
34521 00005BCE 8916[EA0A] mov [CLUSDATA_HW],dx ; (dx = 0, release)
34522 ;;;
34523
34524 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9D19h
34525 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8E88h)
34526 ; (Windows ME IO.SYS - BIOSCODE:0C285h)
34527
34528 ;entry RELBLKS
34529 RELBLKS:
34530
34531 ; Enter here with DX=0FFFFH to put an end-of-file mark in the first cluster
34532 ; and free the rest in the chain.
34533
34534 00005BD2 E80707 call UNPACK
34535 ;jc short RELEASE_flush
34536 ;jz short RELEASE_flush
34537 ; 28/12/2024 (PCDOS 7.1 IBMDOS.COM)
34538 00005BD5 7652 jbe short RET12 ; jna short RET12
34539 ; CCONTENT_HW:DI= Content of FAT
34540 ; for given cluster (next cluster)
34541 00005BD7 89F8 MOV AX,DI
34542 00005BD9 52 PUSH DX
34543
34544 ;;; 28/12/2024 - Retro DOS v5.0
34545 ;push ax
34546 ;;;
34547
34548 00005BDA E8AE07 call PACK
34549
34550 ;;; 28/12/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
34551 ;pop ax
34552 00005BDD 8B16[EC0A] mov dx,[CCONTENT_HW]
34553 00005BE1 8916[E80A] mov [CLUSTNUM_HW],dx ; next cluster (hw) to be released
34554 00005BE5 89C3 mov bx,ax ; next cluster (lw) to be released
34555 ;;;
34556
34557 00005BE7 5A POP DX
34558 ;jc short RELEASE_flush
34559 ; 28/12/2024 (PCDOS 7.1 IBMDOS.COM)
34560 00005BE8 723F jc short RET12
34561
34562 00005BEA 09D2 OR DX,DX
34563 00005BEC 751F JNZ short NO_DEALLOC ; Was putting EOF mark
34564
34565 ; 28/02/2024
34566 %if 0
34567 ;cmp word [es:bp+1Eh],-1 ; MSDOS 3.3
34568 ;cmp word [es:bp+1Fh],-1 ; MSDOS 6.0
34569 CMP word [ES:BP+DPB.FREE_CNT],-1 ; Free count valid?
34570 JZ short NO_DEALLOC ; No
34571 INC word [ES:BP+DPB.FREE_CNT] ; Increase free count by 1
34572 NO_DEALLOC:
34573 MOV BX,AX
34574 dec ax ; check for "1"
34575 jz short RELEASE_flush ; is last cluster of incomplete chain
34576 %else
34577 ; 28/02/2024 - Retro DOS v5.0
34578 ; (PCDOS 7.1 IBMDOS.COM)
34579 ;;;
34580 00005BEE 3B16[EA0A] cmp dx,[CLUSDATA_HW] ; > 0 ? (delete, release)
34581 00005BF2 7519 jnz short NO_DEALLOC ; no, EOF (allocate, -1), (dx = 0)

```

```

34582      ;cmp word [es:bp+1Fh],0FFFFh
34583 00005BF4 26837E1FFF      cmp word [es:bp+DPB.FREE_CNT],0FFFFh ; -1
34584      ; Free count valid?
34585 00005BF9 7412      jz short NO_DEALLOC ; No (dx = 0)
34586      relblks_ifc:
34587      ;inc word ptr es:[bp+1Fh]
34588 00005BFB 26FF461F      inc word [ES:BP+DPB.FREE_CNT] ; Increase free count by 1
34589 00005BFF 7519      jnz short NO_DEALLOC2
34590
34591      ;cmp [es:bp+0Fh],dx
34592 00005C01 2639560F      cmp [es:bp+DPB.FAT_SIZE],dx
34593 00005C05 7513      jnz short NO_DEALLOC2 ; not FAT32
34594
34595      ;inc word [es:bp+21h]
34596 00005C07 26FF4621      inc word [es:bp+DPB.FREE_CNT+2]
34597 00005C0B EB0D      jmp short NO_DEALLOC2
34598      NO_DEALLOC:
34599      ;cmp [es:bp+0Fh],dx
34600 00005C0D 2639560F      cmp [es:bp+DPB.FAT_SIZE],dx ; dx is -1 or 0
34601      ; (valid 16 bit fat size is another number)
34602 00005C11 7507      jnz short NO_DEALLOC2 ; FAT (FAT16 or FAT12)
34603      ; dx = 0, FAT32
34604      ;cmp word [es:bp+21h],0FFFFh
34605 00005C13 26837E21FF      cmp word [es:bp+DPB.FREE_CNT+2],0FFFFh
34606 00005C18 75E1      jnz short relblks_ifc
34607      NO_DEALLOC2:
34608 00005C1A 833E[E80A]00      cmp word [CLUSTNUM_HW],0
34609 00005C1F 7503      jnz short NO_DEALLOC3
34610 00005C21 48      dec ax ; check for "1"
34611 00005C22 7405      jz short RET12 ; is last cluster of incomplete chain
34612      NO_DEALLOC3:
34613      ;;;
34614      %endif
34615
34616 00005C24 E88C06      call ISEOF
34617 00005C27 72A3      JB short RELEASE ; Carry clear if JMP not taken
34618
34619      ; 16/12/2022
34620      ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
34621      ; 28/02/2024 (PCDOS 7.1 IBMDOS.COM)
34622      %if 0
34623      RELEASE_flush:
34624      ; MSDOS 6.0
34625      mov al,[es:bp]
34626      ;MOV AL,[ES:BP+DPB.DRIVE]
34627      push si ; FLUSHBUF may trash these and we guarantee
34628      push cx ; them to be preserved.
34629      push es
34630      push bp
34631      call FLUSHBUF ; commit buffers for this drive
34632      pop bp
34633      pop es
34634      pop cx
34635      pop si
34636      %endif ; 28/02/2024
34637
34638      RET12:
34639 00005C29 C3      retn
34640
34641      ;Break <GETEOF -- Find the end of a file>
34642      ;-----
34643      ;
34644      ; Procedure Name : GETEOF
34645      ;
34646      ; Inputs:
34647      ; ES:BP Points to DPB
34648      ; BX = Cluster in a file
34649      ; DS = CS
34650      ; Outputs:
34651      ; BX = Last cluster in the file
34652      ; Carry set if error (currently user FAILED to I 24)
34653      ; DI destroyed. No other registers affected.
34654      ;-----
34655      ;
34656      ; 28/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
34657      %if 0
34658      GETEOF:
34659      call UNPACK
34660      jc short RET12
34661      PUSH BX
34662      MOV BX,DI
34663      call ISEOF
34664      POP BX
34665      JAE short RET12
34666      MOV BX,DI
34667      JMP short GETEOF
34668      %else
34669      ; 28/02/2024 - Retro DOS v5.0
34670      ;;;
34671      GETEOF4: ; **
34672      pop word [CLUSTNUM_HW] ; * ; 28/02/2024
34673 00005C2A 8F06[E80A]      GETEOF1:
34674      mov di,ax
34675 00005C2E 89C7      mov [CLUSTERS_HW],dx ; CLUSTERS_HW:DI = Cluster Count
34676 00005C30 8916[F20A]      pop ax
34677 00005C34 58      pop dx
34678 00005C35 5A
34679 00005C36 C3      retn
34680
34681      ; PC DOS 7.1 IBMDOS.COM - DOSCODE:9D7Ch
34682      GETEOF:
34683      push dx
34684      push ax
34685 00005C38 50      xor dx,dx ; 0 ; cluster count = 0
34686 00005C3B 31C0      xor ax,ax ; 0
34687      GETEOF2:
34688      call UNPACK
34689 00005C3D E89C06      jc short GETEOF1
34690      ; CCONTENT_HW:DI = next cluster (cluster content)
34691      inc ax
34692 00005C42 40      jnz short GETEOF3
34693 00005C43 7501      inc dx
34694      GETEOF3:
34695 00005C46 FF36[E80A]      push word [CLUSTNUM_HW] ; * ; 28/02/2024
34696 00005C4A 53      push bx
34697      ;push word [CLUSTNUM_HW]
34698 00005C4B 8B1E[EC0A]      mov bx,[CCONTENT_HW]
34699 00005C4F 891E[E80A]      mov [CLUSTNUM_HW],bx
34700 00005C53 89FB      mov bx,di
34701 00005C55 E85B06      call ISEOF
34702      ;pop word [CLUSTNUM_HW] ; * ; 28/02/2024
34703      pop bx
34704      ;jae short GETEOF1 ; EOF
34705 00005C59 73CF      jae short GETEOF4 ; ** ; 28/02/2024

```



```

34706             ;mov     bx,[CONTENT_HW] ; not EOF
34707             ;mov     [CLUSTNUM_HW],bx
34708 00005C5B 5B      pop     bx      ; * ; 28/02/2024
34709 00005C5C 89FB    mov     bx,di
34710 00005C5E EBDD    jmp     short GETEOF2 ; get next cluster
34711             ;
34712             ;
%endif

;=====
; FCB.ASM, MSDOS 6.0, 1991
;=====
; 30/07/2018 - Retro DOS v3.0
; 20/05/2019 - Retro DOS v4.0
; 29/02/2024 - Retro DOS v5.0

; TITLE FCB - FCB parse calls for MSDOS
; NAME FCB

;** FCB.ASM - Low level routines for parsing names into FCBs and analyzing
; filename characters
;
; MakeFcb
; NameTrans
; PATHCHRCMP
; GetLet
; UCase
; GetLet3
; GetCharType
; TESTKANJ
; NORMSCAN
; DELIM
;
; Revision history:
;
; A000 version 4.00 Jan. 1988
;
; M048 - access FILE_UCASE_TAB using DS rather than SS.

TableLook EQU -1

SCANSEPARATOR EQU 1
DRVBIT EQU 2
NAMBIT EQU 4
EXTBIT EQU 8

;-----
; Procedure : MakeFcb
;-----

; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:8E77h (MSDOS 5.0, MSDOS.SYS)

; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; PCDOS 7.1 IBMDOS.COM - DOSCODE:9DB0h

; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8ED3h)
; (Windows ME IO.SYS - BIOSCODE:8D92h)

MAKEFCB:
;hkn; SS override
;MOV     BYTE [SS:SpaceFlag],0
;XOR     DL,DL ; Flag--not ambiguous file name
; 29/02/2024
;mov     [ss:SpaceFlag],dl ; 0
;test    al,2
;test    AL,DRVBIT ; Use current drive field if default?
;JNZ     short DEFDRV
;MOV     BYTE [ES:DI],0 ; No - use default drive
; 29/02/2024
;mov     [es:di],dl ; 0
DEFDRV:
;INC     DI
;MOV     CX,8
;test    al,4
;test    AL,NAMBIT ; Use current name fields as default?
;XCHG    AX,BX ; Save bits in BX
;MOV     AL," "
;JZ      short FILLB ; If not, go fill with blanks
;ADD     DI,CX
;XOR     CX,CX ; Don't fill any
FILLB:
;REP     STOSB
;MOV     CL,3
;test    BL,EXTBIT ; Use current extension as default
;JZ      short FILLB2
;ADD     DI,CX
;XOR     CX,CX
FILLB2:
;REP     STOSB
;XCHG    AX,CX ; Put zero in AX
;STOSW
;STOSW ; Initialize two words after to zero
;SUB     DI,16 ; Point back at start
;test    bl,1
;test    BL,SCANSEPARATOR; Scan off separators if not zero
;JZ      short SKSPSPC
;CALL     SCANB ; Peel off blanks and tabs
;CALL     DELIM ; Is it a one-time-only delimiter?
;JNZ     short NOSCAN
;INC     SI ; Skip over the delimiter
SKSPSPC:
;CALL     SCANB ; Always kill preceding blanks and tabs
NOSCAN:
;CALL     GETLET
;JBE     short NODRV ; Quit if termination character
;CMP     BYTE [SI],":" ; Check for potential drive specifier
;JNZ     short NODRV
;INC     SI ; Skip over colon
;SUB     AL,"@" ; Convert drive letter to drive number (A=1)
;JBE     short BADDRV ; Drive letter out of range
;PUSH     AX
;call     GetVisDrv
;POP      AX
;JNC     short HAVDRV

; 20/05/2019 - Retro DOS v4.0
; MSDOS 6.0
;hkn; SS override
;CMP     byte [SS:DrvErr],error_not_DOS_disk ; 1ah
; ; if not FAT drive ;AN000;
;JZ      short HAVDRV ; assume ok ;AN000;

```

```

34830 BADDRV:
34831 00005CC1 B2FF      MOV     DL,-1
34832 HAVDRV:
34833 00005CC3 AA        STOSB           ; Put drive specifier in first byte
34834 00005CC4 46        INC     SI
34835 00005CC5 4F        DEC     DI           ; Counteract next two instructions
34836 NODRV:
34837 00005CC6 4E        DEC     SI           ; Back up
34838 00005CC7 47        INC     DI           ; Skip drive byte
34839
34840 ;entry NORMSCAN
34841 NORMSCAN:
34842 00005CC8 B90800     MOV     CX,8
34843 00005CCB E82200     CALL    GETWORD       ; Get 8-letter file name
34844 00005CCE 803C2E     CMP     BYTE [SI], "."
34845 00005CD1 7510        JNZ     short NODOT
34846 00005CD3 46        INC     SI           ; skip over dot if present
34847
34848 ; 24/09/2023
34849 ;mov     CX,3
34850 00005CD4 B103      mov     cl,3 ; ch=0
34851
34852 ; MSDOS 6.0
34853 ;hkn; SS override
34854 ;TEST    word [SS:DOS34_FLAG],DBCS_VOLID2 ; 100h ;AN000;
34855 ; 10/06/2019
34856 00005CD6 36F606[1206]01 test    byte [SS:DOS34_FLAG+1],(DBCS_VOLID2>>8) ; 1
34857 00005CDC 7402        JZ      short VOLOK       ;AN000;
34858 00005CDE A4        MOVSB           ; 2nd byte of DBCS ;AN000;
34859 ; 24/09/2023
34860 ;MOV     CX,2           ;AN000;
34861 00005CDF 49        dec     cx ; cx=2
34862 ;JMP     SHORT contvol ;AN000;
34863 VOLOK:
34864 ;MOV     CX,3           ; Get 3-letter extension
34865 contvol:
34866 00005CE0 E81300     CALL    MUSTGETWORD
34867 NODOT:
34868 00005CE3 88D0      MOV     AL,DL
34869
34870 ; MSDOS 6.0
34871 ;and     word [ss:DOS34_FLAG],0FEFFh
34872 ; 18/12/2022
34873 00005CE5 368026[1206]FE and     byte [ss:DOS34_FLAG+1],0FEh ; (~DBCS_VOLID2)>>8
34874 ;and     word [ss:DOS34_FLAG],~DBCS_VOLID2 ; ### BUG FIX ###
34875
34876 00005CEB C3        retn
34877
34878 NONAM:
34879 00005CEC 01CF      ADD     DI,CX
34880 00005CEE 4E        DEC     SI
34881 00005CEF C3        retn
34882
34883 GETWORD:
34884 00005CF0 E89F00     CALL    GETLET
34885 00005CF3 76F7      JBE     short NONAM ; Exit if invalid character
34886 00005CF5 4E        DEC     SI
34887
34888 ; UGH!!! Horrible bug here that should be fixed at some point:
34889 ; If the name we are scanning is longer than CX, we keep on reading!
34890
34891 MUSTGETWORD:
34892 00005CF6 E89900     CALL    GETLET
34893
34894 ; If spaceFlag is set then we allow spaces in a pathname
34895
34896 ;IF NOT TABLELOOK
34897 ; JB      short FILLNAM ; MSDOS 3.3
34898 ;ENDIF
34899 00005CF9 750C      JNZ     short MustCheckCX
34900
34901 ;hkn; SS override
34902 00005CFB 36F606[4E03]FF test    BYTE [SS:SpaceFlag],0FFh
34903 00005D01 7419      JZ      short FILLNAM
34904 00005D03 3C20      CMP     AL," "
34905 00005D05 7515      JNZ     short FILLNAM
34906
34907 MustCheckCX:
34908 00005D07 E3ED      JCXZ    MUSTGETWORD
34909 00005D09 49        DEC     CX
34910 00005D0A 3C2A      CMP     AL,"*" ; check for ambiguous file specifier
34911 00005D0C 7504      JNZ     short NOSTAR
34912 00005D0E B03F      MOV     AL,"?"
34913 00005D10 F3AA      REP     STOSB
34914
34915 NOSTAR:
34916 00005D12 AA        STOSB
34917 00005D13 3C3F      CMP     AL,"?"
34918 00005D15 75DF      JNZ     short MUSTGETWORD
34919 00005D17 80CA01     OR      DL,1 ; Flag ambiguous file name
34920 00005D1A EBDA      JMP     short MUSTGETWORD
34921
34922 FILLNAM:
34923 00005D1C B020      MOV     AL," "
34924 00005D1E F3AA      REP     STOSB
34925 00005D20 4E        DEC     SI
34926 00005D21 C3        retn
34927
34928 SCANB:
34929 00005D22 AC        LODSB
34930 00005D23 E89D00     CALL    SPCHK
34931 00005D26 74FA      JZ      short SCANB
34932 00005D28 4E        DEC     SI
34933 scanb_retn:
34934 retn
34935
34936 ;-----
34937 ; Procedure Name : NameTrans
34938 ;
34939 ; NameTrans is used by FindPath to scan off an element of a path. We must
34940 ; allow spaces in pathnames
34941 ;
34942 ; Inputs: DS:SI points to start of path element
34943 ; Outputs: Name1 has unpacked name, uppercased
34944 ;         ES = DOSGroup
34945 ;         DS:SI advanced after name
34946 ; Registers modified: DI,AX,DX,CX
34947 ;-----
34948
34949 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
34950 ; 20/05/2019 - Retro DOS v4.0
34951
34952 ; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
34953 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9E7Ch

```

```

34954
34955
34956 NameTrans:
34957 ;hkn; SS override
34958 MOV     BYTE [SS:SpaceFlag],1
34959 push    ss
34960 pop     es
34961
34962 ;hkn; NAME1 is in DOSDATA
34963 MOV     DI,NAME1
34964 PUSH    DI
34965
34966 ; 29/02/2024
34967 %if 0
34968 MOV     AX,' ' ; 2020h
34969 MOV     CX,5
34970 STOSB
34971 REP     STOSW ; Fill "FCB" at NAME1 with spaces
34972 XOR     AL,AL ; Set stuff for NORMSCAN
34973 MOV     DL,AL
34974 %else
34975 ; 29/02/2024
34976 ; (PCDOS 7.1 IBMDOS.COM)
34977 ;;;
34978 mov     al, 20h ; ' '
34979 mov     cx, 11
34980 rep stosb ; Fill "FCB" at NAME1 with spaces
34981 xchg    ax, cx
34982 cwd
34983 ;;;
34984 %endif
34985 STOSB
34986 POP     DI
34987
34988 CALL    NORMSCAN
34989
34990 ;hkn; SS override for NAME1
34991 CMP     byte [SS:NAME1],0E5H
34992 jnz     short scanb_retn
34993 MOV     byte [SS:NAME1],5 ; Magic name translation
34994 retn
34995
34996 ;Break <GETLET, DELIM -- CHECK CHARACTERS AND CONVERT>
34997 ;=====
34998
34999 ; 20/05/2019 - Retro DOS v4.0
35000 ; DOSCODE:8FD2h (MSDOS 6.21, MSDOS.SYS)
35001
35002 ;If TableLook
35003
35004 ;hkn; Table SEGMENT
35005 ; PUBLIC CharType
35006 ;-----
35007
35008 ; Character type table for file name scanning
35009 ; Table provides a mapping of characters to validity bits.
35010 ; Four bits are provided for each character. Values 7Dh and above
35011 ; have all bits set, so that part of the table is chopped off, and
35012 ; the translation routine is responsible for screening these values.
35013 ; The bit values are defined in DOSSYM.INC
35014
35015 ; ; ^A and NUL
35016 ;CharType:
35017 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35018 ; ; ^C and ^B
35019 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35020 ; ; ^E and ^D
35021 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35022 ; ; ^G and ^F
35023 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35024 ; ; TAB and BS
35025 ; db LOW ((NOT FFCB+FCHK+FDELIM+FSPCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35026 ; ; ^K and ^J
35027 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35028 ; ; ^M and ^L
35029 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35030 ; ; ^O and ^N
35031 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35032 ; ; ^Q and ^P
35033 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35034 ; ; ^S and ^R
35035 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35036 ; ; ^U and ^T
35037 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35038 ; ; ^W and ^V
35039 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35040 ; ; ^Y and ^X
35041 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35042 ; ; ESC and ^Z
35043 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35044 ; ; ^] and ^[ db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35045 ; ; ^_ and ^^
35046 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35047 ; ; ! and SPACE
35048 ; db LOW (NOT FCHK+FDELIM+FSPCHK)
35049 ; ; # and "
35050 ; db LOW (NOT FFCB+FCHK)
35051 ; ; $ - )
35052 ; db 3 dup (0Fh)
35053 ; ; + and *
35054 ; db LOW ((NOT FFCB+FCHK+FDELIM) SHL 4) OR 0Fh
35055 ; ; - and '
35056 ; db NOT (FFCB+FCHK+FDELIM)
35057 ; ; / and .
35058 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FCHK) AND 0Fh
35059 ; ; 0 - 9
35060 ; db 5 dup (0Fh)
35061 ; ; ; and :
35062 ; db LOW ((NOT FFCB+FCHK+FDELIM) SHL 4) OR LOW (NOT FFCB+FCHK+FDELIM) AND 0Fh
35063 ; ; = and <
35064 ; db LOW ((NOT FFCB+FCHK+FDELIM) SHL 4) OR LOW (NOT FFCB+FCHK+FDELIM) AND 0Fh
35065 ; ; ? and >
35066 ; db NOT FFCB+FCHK+FDELIM
35067 ; ; A - Z
35068 ; db 13 dup (0Fh)
35069 ; ; \ and [
35070 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR 0Fh
35071 ; ; ^ and ]
35072 ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35073 ; ; _ - {
35074 ; db 15 dup (0Fh)
35075 ; ; } and |
35076 ; db NOT FFCB+FCHK+FDELIM
35077 ;
35078

```

```

35079 ;CharType_last equ ($ - CharType) * 2 ; This is the value of the last
35080 ; ; character in the table
35081
35082 ;FCHK equ 1 ; normal name char, no chks needed
35083 ;FDELIM equ 2 ; is a delimiter
35084 ;FSPCHK equ 4 ; set if character is not a space or equivalent
35085 ;FFCB equ 8 ; is valid in an FCB
35086
35087 ; DOSCODE:8FD2h (MSDOS 6.21, MSDOS.SYS)
35088 ;-----
35089 ; DOSCODE:8F76h (MSDOS 5.0, MSDOS.SYS)
35090
35091 CharType: ; 63 bytes
35092 00005D53 6666666606666666 db 66h, 66h, 66h, 66h, 06h, 66h, 66h, 66h ; 0-7
35093 00005D5B 6666666666666666 db 66h, 66h, 66h, 66h, 66h, 66h, 66h, 66h ; 8-15
35094 00005D63 F8F6FFFFFF4FF46E db 0F8h,0F6h,0FFh,0FFh,0FFh, 4Fh,0F4h, 6Eh ; 16-23
35095 00005D6B FFFFFFFF4444F4 db 0FFh,0FFh,0FFh,0FFh,0FFh, 44h, 44h,0F4h ; 24-31
35096 00005D73 FFFFFFFF6666FF db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh ; 32-39
35097 00005D7B FFFFFFFF6666FF db 0FFh,0FFh,0FFh,0FFh,0FFh, 6Fh, 66h,0FFh ; 40-47
35098 00005D83 FFFFFFFF6666FF db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh ; 48-55
35099 00005D8B FFFFFFFF6666FF db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0F4h ; 56-62
35100
35101 CharType_last equ ($ - CharType) * 2
35102
35103 ; Offset 12CAh of IBMDOS.COM (MSDOS 3.3), 1987
35104 ;-----
35105 ;CharType:
35106 ; db 0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h
35107 ; db 0F6h,0F0h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h
35108 ; db 0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h
35109 ; db 0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h
35110 ; db 0F8h,0FFh,0F6h,0FFh,0FFh,0FFh,0FFh,0FFh
35111 ; db 0FFh,0FFh,0FFh,0F4h,0F4h,0FFh,0FEh,0F6h
35112 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35113 ; db 0FFh,0FFh,0F4h,0F4h,0F4h,0F4h,0F4h,0FFh
35114 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35115 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35116 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35117 ; db 0FFh,0FFh,0FFh,0F6h,0F6h,0F6h,0FFh,0FFh
35118 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35119 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35120 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35121 ; db 0FFh,0FFh,0FFh,0FFh,0F4h,0FFh,0FFh,0FFh
35122 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35123 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35124 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35125 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35126 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35127 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35128 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35129 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35130 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35131 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35132 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35133 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35134 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35135 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35136 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35137 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35138
35139 ;hkn; Table ENDS
35140
35141 ;ENDIF
35142
35143 ; 20/05/2019 - Retro DOS v4.0
35144 ; DOSCODE:9011h (MSDOS 6.21, MSDOS.SYS)
35145
35146 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35147 ; DOSCODE:8FB5h (MSDOS 5.0, MSDOS.SYS)
35148
35149 ;-----
35150 ;
35151 ; Procedure Names : GetLet, UCase, GetLet3
35152 ;
35153 ; These routines take a character, convert it to upper case, and check
35154 ; for delimiters. Three different entry points:
35155 ; GetLet - DS:[SI] = character to convert
35156 ; UCase - AL = character to convert
35157 ; GetLet3 - AL = character
35158 ; [BX] = translation table to use
35159 ;
35160 ; Exit (in all cases) : AL = upper case character
35161 ; CY set if char is control char other than TAB
35162 ; ZF set if char is a delimiter
35163 ;
35164 ; Uses : AX, flags
35165 ;
35166 ; NOTE: This routine exists in a fast table lookup version, and a slow
35167 ; inline version. Return with carry set is only possible in the inline
35168 ; version. The table lookup version is the one in use.
35169 ;-----
35170
35171 ; This entry point has character at [SI]
35172
35173 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5517h
35174 GETLET:
35175 00005D92 AC LODSB
35176
35177 ; This entry point has character in AL
35178
35179 ;entry UCase
35180 UCase:
35181 ; 09/08/2018
35182 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5518h
35183 _UCase:
35184 00005D93 53 PUSH BX
35185 00005D94 BB[7E0B] MOV BX,FILE_UCASE_TAB+2
35186
35187 ; Convert the character in AL to upper case
35188
35189 gl_0:
35190 00005D97 3C61 CMP AL,"a"
35191 00005D99 7214 JB short gl_2 ; Already upper case, go check type
35192 00005D9B 3C7A CMP AL,"z"
35193 00005D9D 7702 JA short gl_1
35194 00005D9F 2C20 SUB AL,20H ; Convert to upper case
35195
35196 ; Map European character to upper case
35197
35198 gl_1:
35199 00005DA1 3C80 CMP AL,80H
35200 00005DA3 720A JB short gl_2 ; Not EuroChar, go check type
35201 00005DA5 2C80 SUB AL,80H ; translate to upper case with this index
35202

```

```

35203 ; M048 - Start
35204 ; Lantastic call Ucase thru int 2f without setting SS to DOSDATA.
35205 ; So we shall set up DS and to access FILE_UCASE_TAB in BX and also
35206 ; preserve it.
35207
35208 ; 09/08/2018 - Retro DOS v3.0
35209 ; MSDOS 3.3
35210 ;;XLAT BYTE [CS:BX] ; ds as file_ucase_tab is in DOSDATA
35211 ;CS XLAT
35212
35213 ; 20/05/2019 - Retro DOS v4.0
35214
35215 ; MSDOS 6.0
35216 00005DA7 1E push ds
35217 ;getdseg <ds>
35218 00005DA8 2E8E1E[0700] mov ds,[cs:DosDSeg]
35219 00005DAD D7 XLAT ; ds as file_ucase_tab is in DOSDATA
35220 00005DAE 1F pop ds
35221
35222 ; M048 - End
35223
35224 ; Now check the type
35225
35226 ;If TableLook
35227 gl_2:
35228 ; 20/05/2019 - Retro DOS v4.0
35229 00005DAF 50 PUSH AX
35230
35231 ; MSDOS 3.3
35232 ;mov bx,CharType
35233 ;; 09/08/2018
35234 ;;xlat byte [cs:bx]
35235 ;cs xlat
35236
35237 ; MSDOS 6.0
35238 00005DB0 E81800 CALL GetCharType ; returns type flags in AL
35239
35240 ;test al,1
35241 00005DB3 A801 TEST AL,FCHK ; test for normal character
35242 00005DB5 58 POP AX
35243
35244 00005DB6 5B POP BX
35245 00005DB7 C3 RETN
35246
35247 ; This entry has character in AL and lookup table in BX
35248
35249 ; MSDOS 6.0
35250 ;entry GetLet3
35251 GETLET3: ; 10/08/2018
35252 00005DB8 53 PUSH BX
35253 00005DB9 EBDC JMP short gl_0
35254 ;ELSE
35255 ;
35256 ;gl_2:
35257 ; POP BX
35258 ; CMP AL,"."
35259 ; retz
35260 ; CMP AL,"'"
35261 ; retz
35262 ; CALL PATHCHRCMP
35263 ; retz
35264 ; CMP AL,"["
35265 ; retz
35266 ; CMP AL,"]"
35267 ; retz
35268 ;ENDIF
35269
35270 ;-----
35271 ;
35272 ; DELIM - check if character is a delimiter
35273 ; Entry : AX = character to check
35274 ; Exit : ZF set if character is not a delimiter
35275 ; Uses : Flags
35276 ;
35277 ;-----
35278
35279 ;entry DELIM
35280 DELIM:
35281 ;If TableLook
35282 ; 20/05/2019 - Retro DOS v4.0
35283 00005DBB 50 PUSH AX
35284
35285 ; MSDOS 3.3
35286 ;push bx
35287 ;mov bx,CharType
35288 ;;09/08/2018
35289 ;;xlat byte [cs:bx]
35290 ;cs xlat
35291 ;pop bx
35292
35293 ; MSDOS 6.0
35294 00005DBC E80C00 CALL GetCharType
35295
35296 ;test al,2
35297 00005DBF A802 TEST AL,FDELIM
35298 00005DC1 58 POP AX
35299 00005DC2 C3 RETN
35300 ;ELSE
35301 ; CMP AL,":"
35302 ; retz
35303 ;
35304 ; CMP AL,"<"
35305 ; retz
35306 ; CMP AL,"|"
35307 ; retz
35308 ; CMP AL,">"
35309 ; retz
35310 ;
35311 ; CMP AL,"+"
35312 ; retz
35313 ; CMP AL,"="
35314 ; retz
35315 ; CMP AL,";"
35316 ; retz
35317 ; CMP AL","
35318 ; retz
35319 ;ENDIF
35320
35321 ;-----
35322 ;
35323 ; SPCHK - checks to see if a character is a space or equivalent
35324 ; Entry : AL = character to check
35325 ; Exit : ZF set if character is a space
35326 ; Uses : flags

```

```

35327 ;
35328 ;-----
35329 ;
35330 ;entry SPCHK
35331 SPCHK:
35332 ;IF TableLook
35333 ; 20/05/2019 - Retro DOS v4.0
35334 00005DC3 50 PUSH AX
35335 ;
35336 ; MSDOS 3.3
35337 ;push bx
35338 ;mov bx,CharType
35339 ; 09/08/2018
35340 ;xlat byte [cs:bx]
35341 ;cs xlat
35342 ;pop bx
35343 ;
35344 ; MSDOS 6.0
35345 00005DC4 E80400 CALL GetCharType
35346 ;
35347 ;test al,4
35348 00005DC7 A804 TEST AL,FSPCHK
35349 00005DC9 58 POP AX
35350 00005DCA C3 RETN
35351 ;ELSE
35352 ; CMP AL,9 ; Filter out tabs too
35353 ; retz
35354 ; WARNING! " " MUST be the last compare
35355 ; CMP AL," "
35356 ; return
35357 ;ENDIF
35358 ;
35359 ;-----
35360 ;
35361 ; GetCharType - return flag bits indicating character type
35362 ; Bits are defined in DOSSYM.INC. Uses lookup table
35363 ; defined above at label CharType.
35364 ;
35365 ; Entry : AL = character to return type flags for
35366 ; Exit : AL = type flags
35367 ; Uses : AL, flags
35368 ;
35369 ;-----
35370 ;
35371 ; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
35372 ;
35373 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35374 ;
35375 ; 20/05/2019 - Retro DOS v4.0
35376 ; MSDOS 6.0
35377 ;
35378 GetCharType:
35379 ;cmp al,7Eh
35380 00005DCB 3C7E cmp al,CharType_last ; beyond end of table?
35381 00005DCD 7314 jae short gct_90 ; return standard value
35382 ;
35383 00005DCF 53 push bx
35384 00005DD0 8B[535D] mov bx,CharType ; load lookup table
35385 00005DD3 D0E8 shr al,1 ; adjust for half-byte table entry size
35386 ;xlat cs:[bx] ; get flags
35387 00005DD5 2ED7 cs xlat
35388 00005DD7 5B pop bx
35389 ;
35390 ; carry clear from previous shift means we want the low nibble. Otherwise
35391 ; we have to shift the flags down to the low nibble
35392 ;
35393 00005DD8 7306 jnc short gct_80 ; carry clear, no shift needed
35394 ;
35395 ; 29/02/2024
35396 %if 0
35397 shr al,1 ; we want high nibble, shift it down
35398 shr al,1
35399 shr al,1
35400 shr al,1
35401 %else
35402 ; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
35403 ;;;
35404 00005DDA 51 push cx
35405 00005ddb B104 mov cl,4 ; we want high nibble, shift it down
35406 00005ddd D2E8 shr al,cl
35407 00005ddf 59 pop cx
35408 ;;;
35409 %endif
35410 ;
35411 gct_80:
35412 00005DE0 240F and al,0Fh ; clear the unused nibble
35413 00005DE2 C3 retn
35414 gct_90:
35415 00005DE3 B00F mov al,0Fh ; set all flags
35416 00005DE5 C3 retn
35417 ;
35418 ;-----
35419 ;
35420 ; Procedure : PATHCHRCMP
35421 ;
35422 ;-----
35423 ;
35424 PATHCHRCMP:
35425 00005DE6 3C2F CMP AL,'/'
35426 00005DE8 7606 JBE short PathRet
35427 00005DEA 3C5C CMP AL,'\'
35428 00005DEC C3 retn
35429 GotFor:
35430 00005DED B05C MOV AL,'\'
35431 00005DEF C3 retn
35432 PathRet:
35433 00005DF0 74FB JZ short GotFor
35434 00005DF2 C3 retn
35435 ;
35436 ;=====
35437 ; MSCRTL.C.ASM, MSDOS 6.0, 1991
35438 ;=====
35439 ; 30/07/2018 - Retro DOS v3.0
35440 ; 29/04/2019 - Retro DOS v4.0
35441 ; 29/02/2024 - Retro DOS v5.0
35442 ;
35443 ; 15/03/2018 - Retro DOS v2.0 (MSDOS 2.11, CTRL.C.ASM, 1983)
35444 ;
35445 ;** MSCTRL.C.ASM - ^C and error handler for MSDOS
35446 ;
35447 ; TITLE Control C detection, Hard error and EXIT routines
35448 ; NAME IBMCTRLC
35449 ;
35450 ;** Low level routines for detecting special characters on CON input,

```



```

35451 ; the ^C exit/int code, the Hard error INT 24 code, the
35452 ; process termination code, and the INT 0 divide overflow handler.
35453 ;
35454 ; FATAL
35455 ; FATAL1
35456 ; reset_environment
35457 ; DSKSTATCHK
35458 ; SPOOLINT
35459 ; STATCHK
35460 ; CNTCHAND
35461 ; DIVOV
35462 ; CHARHARD
35463 ; HardErr
35464 ;
35465 ; Revision history:
35466 ;
35467 ; AN000 version 4.0 Jan 1988
35468 ; A002 PTM -- dir >\pt3 hangs
35469 ; A003 PTM 3957- fake version for IBMCAHE.COM
35470 ;
35471 ; M011: NEC's 8086 clone chip uses Intel's undocumented bit number in
35472 ; flags register. In order to return to user normally DOS used to
35473 ; move F202 into flags, which sets bit number 1 in flags.uncondit-
35474 ; ionally. Now it is modified to maintain the state of bit 1.
35475 ;
35476 ; M024: suppressed fail and ignore options if not in the middle of int
35477 ; 24 and if Ctrl P or ctrl printscrn is pressed in routine
35478 ; charhard.
35479 ;
35480 ; 29/04/2019 - Retro DOS v4.0
35481 ; MSDOS 6.0
35482 ; public LowInt23Addr
35483 LowInt23Addr: ; LABEL DWORD
35484 DW LowInt23, 0
35485 ;
35486 ; public LowInt24Addr
35487 LowInt24Addr: ; LABEL DWORD
35488 DW LowInt24, 0
35489 ;
35490 ; public LowInt28Addr
35491 LowInt28Addr: ; LABEL DWORD
35492 DW LowInt28, 0
35493 ;
35494 ;Break <Checks for ^C in CON I/O>
35495 ;
35496 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35497 ; 05/05/2019 - Retro DOS v4.0
35498 ;
35499 ; -----
35500 ;
35501 ; Procedure Name : DSKSTATCHK
35502 ;
35503 ; Check for ^C if only one level in
35504 ;
35505 ; -----
35506 ;
35507 ; ;procedure DSKSTATCHK,NEAR ; Check for ^C if only one level in
35508 ;
35509 ; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
35510 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9F51h
35511 ;
35512 DSKSTATCHK:
35513 cmp byte [ss:INDOS], 1
35514 jz short dskstatchk1
35515 retn
35516
35517 dskstatchk1:
35518 push cx
35519 push es
35520 push bx
35521 push ds
35522 push si
35523 mov bx, ss
35524 mov es, bx
35525 mov ds, bx
35526 mov byte [DSKSTCOM], 5 ; DEVRDND
35527 mov byte [DSKSTCALL], 14 ; DRDNDHL
35528 mov byte [DSKSTST], 0
35529 mov bx, DSKSTCALL
35530 lds si, [BCON]
35531 call DEVIOCALL2
35532 push ds
35533 push ss
35534 pop ds
35535 test byte [DSKSTST+1], 2
35536 jz short _GotCh
35537 xor al, al
35538
35539 RET36:
35540 pop si
35541 pop si
35542 pop ds
35543 pop bx
35544 pop es
35545 pop cx
35546 retn
35547
35548 _GotCh:
35549 mov al, [DSKCHRET]
35550 cmp al, 3 ; "C"-"@"
35551 jnz short RET36
35552 mov byte [DSKSTCOM], 4 ; DEVRD
35553 mov byte [DSKSTCALL], 16h ; DRDWRHL
35554 mov [DSKCHRET], cl
35555 mov word [DSKSTST], 0
35556 mov word [DSKSTCNT], 1
35557 pop ds
35558 call DEVIOCALL2 ; Eat the ^C
35559 pop si
35560 pop ds
35561 pop bx
35562 pop es
35563 pop cx
35564 jmp CNTCHAND
35565
35566 ; 05/05/2019
35567 NOSTOP:
35568 ; MSDOS 6.0
35569 CMP AL,"P"-"@"
35570 JNZ short check_next
35571 ; SS override
35572 CMP BYTE [SS:SCAN_FLAG],0 ; ALT_Q ?
35573 JZ short INCHKJ ; no
35574 check_end: ; 24/09/2023
35575 retn
35576 check_next:

```

```

35575             ;IF      NOT TOGLPRN
35576             ;CMP     AL,"N"-"@"
35577             ;JZ      short INCHKJ
35578             ;ENDIF
35579
35580 00005E79 3C03      CMP     AL,"C"-"@"
35581             ; 24/09/2023
35582             ;JZ      short INCHKJ
35583 ;check_end:
35584             ;retn
35585 00005E7B 75FB      jnz     short check_end
35586
35587             ; 24/09/2023
35588             ; 08/09/2018
35589 INCHKJ:           ; 10/08/2018
35590 00005E7D E9A500    jmp     INCHK
35591
35592             ; MSDOS 3.3
35593             ;CMP     AL,"P"-"@" ; cmp al,16
35594             ;JZ      short INCHKJ
35595
35596             ; 15/04/2018
35597             ;IF      NOT TOGLPRN
35598             ;CMP     AL,"N"-"@"
35599             ;JZ      SHORT INCHKJ
35600             ;;ENDIF
35601
35602             ;CMP     AL,"C"-"@" ; cmp al,3
35603             ;JZ      short INCHKJ
35604             ;RETN
35605
35606             ; 08/09/2018
35607 ;INCHKJ:; 10/08/2018
35608             ; JMP     INCHK
35609
35610 ;-----
35611 ;
35612 ; Procedure Name : SpoolInt
35613 ;
35614 ; SpoolInt - signal processes that the DOS is truly idle. We are allowed to
35615 ; do this ONLY if we are working on a 1-12 system call AND if we are not in
35616 ; the middle of an INT 24.
35617 ;-----
35618 ;
35619 ;
35620 SPOOLINT:
35621 00005E80 9C          PUSHF
35622             ; 15/03/2018
35623 00005E81 36803E[5803]00 CMP     BYTE [SS:IDLEINT],0 ; SS override
35624 00005E87 7423      JZ      SHORT POPFRET
35625 00005E89 36803E[2003]00 CMP     BYTE [SS:ERRORMODE],0
35626 00005E8F 751B      JNZ     SHORT POPFRET ; No spool ints in error mode
35627
35628             ; 30/07/2018
35629
35630             ; Note that we are going to allow an external program to issue system
35631             ; calls at this time. We MUST preserve IdleInt across this.
35632
35633 00005E91 36FF36[5803] PUSH    WORD [SS:IDLEINT]
35634
35635             ; 05/05/2019 - Retro DOS v4.0
35636
35637             ; MSDOS 6.0
35638 00005E96 36803E[660D]00 cmp     byte [SS:DosHashMA],0 ; Q: is dos running in HMA (M021)
35639 00005E9C 7504      jne     short do_low_int28 ; Y: the int must be done from low mem
35640 00005E9E CD28      INT     int_spooler ; int 28h; N: Execute user int 28 handler
35641 00005EA0 EB05      jmp     short spool_ret_addr
35642
35643 do_low_int28:
35644             ;call far [ss:LowInt28Addr]
35645 00005EA2 2EFF1E[FB5D] call    far [cs:LowInt28Addr] ; 05/05/2019
35646
35647 spool_ret_addr:
35648             ;INT     int_spooler ; INT 28h
35649
35650 00005EA7 368F06[5803] POP     WORD [SS:IDLEINT]
35651 POPFRET:
35652 POPF
35653 _RET18:
35654 00005EAD C3          RETN
35655
35656             ; 05/05/2019 - Retro DOS v4.0
35657             ; DOSCODE:9137h (MSDOS 6.21, MSDOS.SYS)
35658             ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35659             ; DOSCODE:90DBh (MSDOS 5.0, MSDOS.SYS)
35660
35661 ;-----
35662 ;
35663 ; Procedure Name : STATCHK
35664 ;
35665 ;-----
35666 STATCHK:
35667 00005EAE E84EFF      CALL     DSKSTATCHK ; Allows ^C to be detected under
35668                                     ; input redirection
35669
35670 00005EB1 53          PUSH    BX
35671 00005EB2 31DB      XOR     BX,BX
35672 00005EB4 E80EE0      CALL     GET_IO_SFT
35673 00005EB7 5B          POP     BX
35674 00005EB8 72F3      JC      SHORT _RET18
35675
35676 00005EBA B401      MOV     AH,1
35677 00005EBC E805F2      CALL     IOFUNC
35678 00005EBF 74BF      JZ      SHORT SPOOLINT
35679 00005EC1 3C13      CMP     AL,'S'-'@'
35680 00005EC3 75A7      JNZ     SHORT NOSTOP
35681
35682             ; 05/05/2019
35683             ; MSDOS 6.0 ; SS override
35684 00005EC5 36803E[BC0D]00 CMP     BYTE [SS:SCAN_FLAG],0 ; AN000; ALT_R ?
35685 00005ECB 75AB      JNZ     short check_end ; AN000; yes
35686
35687 00005ECD 30E4      XOR     AH,AH
35688 00005ECF E8F2F1      CALL     IOFUNC ; Eat Cntrl-S
35689 00005ED2 EB4A      JMP     SHORT PAUSOSTRT
35690 PRINTOFF:
35691 PRINTON:
35692 00005ED4 36F616[FE02] NOT     BYTE [SS:PFLAG] ; 14/03/2018
35693
35694             ; 30/07/2018 - Retro DOS v3.0
35695 00005ED9 53          PUSH    BX
35696 00005EDA B80400      MOV     BX,4
35697 00005EDD E8E5DF      call    GET_IO_SFT
35698 00005EE0 5B          POP     BX

```



```

35699 00005EE1 72CA      jc      short _RET18
35700 00005EE3 06      PUSH    ES
35701 00005EE4 57      PUSH    DI
35702 00005EE5 1E      PUSH    DS
35703 00005EE6 07      POP     ES
35704 00005EE7 89F7     MOV     DI,SI      ; ES:DI -> SFT
35705                      ;test word [es:di+5],800h
35706                      ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_net_spool
35707                      ; 05/05/2019
35708 00005EE9 26F6450608 test    byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_net_spool>>8)
35709 00005EEE 7418     JZ      short NORM_PR      ; Not redirected, echo is OK
35710
35711                      ;Callinstall NetSpoolEchoCheck,MultNet,38,<AX>,<AX>
35712                      ; See if allowed
35713 00005EF0 50      push    ax
35714 00005EF1 B82611     mov     ax,1126h
35715 00005EF4 CD2F     int     2Fh      ; Multiplex - NETWORK REDIRECTOR - ???
35716                      ; Return: CF set on error, AX = error code
35717                      ; STACK unchanged
35718 00005EF6 58      pop     ax
35719
35720 00005EF7 730F     JNC     short NORM_PR      ; Echo is OK
35721
35722                      ; SS override
35723 00005EF9 36C606[FE02]00 MOV     BYTE [SS:PFLAG],0      ; If not allowed, disable echo
35724
35725                      ;Callinstall NetSpoolClose,MultNet,36,<AX>,<AX> ; and close
35726
35727 00005EFF 50      push    ax
35728 00005F00 B82411     mov     ax,1124h
35729 00005F03 CD2F     int     2Fh      ; Multiplex - NETWORK REDIRECTOR - ???
35730                      ; ES:DI -> SFT, SS = DOS CS
35731 00005F05 58      pop     ax
35732
35733 00005F06 EB10     JMP     SHORT RETP6
35734
35735 00005F08 36803E[FE02]00 NORM_PR:
35736 00005F0E 7505     CMP     BYTE [SS:PFLAG],0      ; SS override
35737 00005F10 E805F3     JNZ     short PRNOPN
35738 00005F13 EB03     call    DEV_CLOSE_SFT
35739                      JMP     SHORT RETP6
35740 00005F15 E8F8F2     PRNOPN:
35741                      call    DEV_OPEN_SFT
35742 00005F18 5F      RETP6:
35743 00005F19 07      POP     DI
35744                      POP     ES
35745 00005F1A C3      STATCHK_RETN:
35746                      RETN
35747 00005F1B E862FF     PAUSOLP:
35748                      CALL     SPOOLINT
35749 00005F1E B401     PAUSOSTRT:
35750 00005F20 E8A1F1     MOV     AH,1
35751 00005F23 74F6     CALL     IOFUNC
35752                      JZ      SHORT PAUSOLP
35753 00005F25 53      INCHK:
35754 00005F26 31DB     PUSH    BX
35755 00005F28 E89ADF     XOR     BX,BX
35756 00005F2B 58      CALL     GET_IO_SFT
35757 00005F2C 72EC     POP     BX
35758 00005F2E 30E4     JC      SHORT STATCHK_RETN ; 30/07/2018
35759 00005F30 E891F1     XOR     AH,AH
35760                      CALL     IOFUNC
35761                      ; 30/07/2018
35762                      ; MSDOS 3.3
35763                      ;CMP     AL,'P'-'@' ;cmp al,16
35764                      ;JNZ     SHORT NOPRINT
35765                      ;cmp     byte [SS:SCAN_FLAG],0
35766                      ;JZ      SHORT PRINTON
35767                      ;mov     byte [ss:SCAN_FLAG],0
35768
35769                      ; 05/05/2019
35770                      ; MSDOS 6.0
35771 00005F33 3C10     CMP     AL,"P"-"@"
35772                      ;;; 7/14/86 ALT_Q key fix
35773 00005F35 749D     JZ      short PRINTON      ; no! must be CTRL_P
35774
35775 ;NOPRINT:
35776 ;IF     NOT TOGLPRN
35777 ;CMP     AL,"N"-"@"
35778 ;JZ      short PRINTOFF
35779 ;ENDIF
35780 CMP     AL,"C"-"@" ; cmp al,3
35781 ;retnz
35782 jnz     short STATCHK_RETN
35783
35784 ; !! NOTE: FALL THROUGH !!
35785
35786 ;-----
35787 ;
35788 ; Procedure Name : CNTHAND ( CTRL_C HANDLER )
35789 ;
35790 ; "AC" and CR/LF is printed. Then the user registers are restored and the
35791 ; user CTRL-C handler is executed. At this point the top of the stack has 1)
35792 ; the interrupt return address should the user CTRL-C handler wish to allow
35793 ; processing to continue; 2) the original interrupt return address to the code
35794 ; that performed the function call in the first place. If the user CTRL-C
35795 ; handler wishes to continue, it must leave all registers unchanged and RET
35796 ; (not IRET) with carry CLEAR. If carry is SET then an terminate system call
35797 ; is simulated.
35798 ;-----
35799
35800 ; 29/02/2024 - Retro DOS v5.0
35801
35802 CNTCHAND:
35803 ; MSDOS 6.0
35804                      ; SS override
35805                      ; AN002; from RAWOUT
35806 ;TEST    word [SS:DOS34_FLAG],CTRL_BREAK_FLAG
35807 ;JNZ     short around_deadlock ; AN002;
35808
35809 ; 05/05/2019 - Retro DOS v4.0
35810 ; (MSDOS 6.21 MSDOS.SYS DOSCODE:91C4h, 29/12/2022)
35811 00005F3B 36F606[1206]02 TEST    byte [SS:DOS34_FLAG+1],(CTRL_BREAK_FLAG>>8) ; 2
35812 00005F41 7508     JNZ     short around_deadlock ; AN002;
35813
35814 00005F43 B003     MOV     AL,3      ; Display "AC"
35815 00005F45 E87DBD     CALL    BUFOUT
35816 00005F48 E812BC     CALL    CRLF
35817 00005F4B 16      around_deadlock:
35818 00005F4C 1F      PUSH    SS
35819 00005F4D 803E[5703]00 POP     DS
35820 00005F52 7403     CMP     BYTE [CONSWAP],0
35821 00005F54 E802DC     JZ      SHORT NOSWAP
35822                      CALL    SWAPBACK
35823 NOSWAP:

```

```

35823 00005F57 FA
35824 00005F58 8E16[8605]
35825 00005F5C 8B26[8405]
35826 00005F60 E8DEA4
35827
35828
35829
35830
35831
35832
35833
35834
35835
35836
35837
35838
35839
35840
35841
35842
35843
35844
35845
35846
35847 00005F63 07
35848
35849 00005F64 1E
35850
35851 00005F65 2E8E1E[0700]
35852 00005F6A C606[2103]00
35853
35854
35855 00005F6F C606[B812]00
35856
35857 00005F74 C606[2003]00
35858 00005F79 8926[3203]
35859
35860 00005F7D 8306[3203]02
35861
35862 00005F82 803E[660D]00
35863 00005F87 1F
35864 00005F88 7504
35865
35866
35867 00005F8A CD23
35868 00005F8C EB06
35869
35870
35871
35872 00005F8E F8
35873 00005F8F 2EFF1E[F35D]
35874
35875
35876
35877
35878
35879
35880
35881
35882
35883
35884
35885
35886
35887
35888
35889
35890 00005F94 FA
35891
35892
35893
35894
35895
35896
35897
35898
35899
35900
35901
35902 00005F95 50
35903 00005F96 8CD8
35904
35905 00005F98 2E8E1E[0700]
35906 00005F9D A3[630D]
35907 00005FA0 58
35908 00005FA1 A3[3A03]
35909 00005FA4 9C
35910 00005FA5 58
35911
35912
35913
35914
35915
35916
35917
35918
35919 00005FA6 3B26[3203]
35920 00005FAA 750A
35921
35922
35923
35924
35925
35926
35927
35928
35929 00005FAC A1[3A03]
35930 00005FAF 8E1E[630D]
35931
35932
35933
35934 00005FB3 E93BA3
35935
35936
35937
35938
35939
35940
35941
35942
35943
35944
35945 00005FB6 A801
35946

CLI
MOV SS,[USER_SS]
MOV SP,[USER_SP]
CALL restore_world
; 30/07/2018 - Retro DOS v3.0
; MSDOS 3.3 (IBMDOS.COM - Offset 56ACh)
; 14/03/2018 - Retro DOS v2.0
;MOV BYTE [CS:INDOS],0
;MOV BYTE [CS:ERRORMODE],0
;MOV [CS:ConC_Spsave],SP
;clc ;30/07/2018
;INT int_ctrl_c ; 23h ; Execute user Ctrl-C handler
;;int 23h ; DOS - CONTROL "C" EXIT ADDRESS
; Return: return via RETF 2 with CF set
; DOS will abort program with errorlevel 0
; else
; interrupted DOS call continues

; 05/05/2019 - Retro DOS v4.0
; MSDOS 6.0 (MSDOS 6.21, MSDOS.SYS,91ECh)

; CS was used to address these variables. We have to use DOSDATA
pop es ; * ; MSDOS 6.21 (MSDOS.SYS, DOSCODE:91ECh)
; (pop es, after 'call restore_world')
push ds
;getdseg <ds> ; ds -> dosdata
mov ds,[cs:DosDSeg]
mov byte [INDOS],0 ; Go to known state
; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
;;
mov byte [INDOS_FLAG],0 ; 2nd 'in dos' flag (what for?)
;;
mov byte [ERRORMODE],0
mov [ConC_Spsave],SP ; save his SP
; User SP has changed because of push. Adjust for it
add word [ConC_Spsave],2

cmp byte [DosHashMA],0 ; Q: is dos running in HMA (M021)
pop ds ; restore ds
jne short do_low_int23 ; Y: the int must be done from low mem
; 04/02/2026
;CLC
INT int_ctrl_c ; int 23h ; N: Execute user Ctrl-C handler
jmp short ctrlc_ret_addr

; 05/05/2019
do_low_int23:
clc
call far [cs:LowInt23Addr]

; 30/07/2018
; MSDOS 3.3 (IBMDOS.COM - Offset 56C0h)

; The user has returned to us. The circumstances we allow are:
;
; IRET We retry the operation by redispaching the system call
; CLC/RETF POP the stack and retry
; ... Exit the current process with ^C exit
;
; User's may RETURN to us and leave interrupts on.
; Turn 'em off just to be sure

ctrlc_ret_addr: ; 05/05/2019
CLI

; MSDOS 3.3
;MOV [CS:USER_IN_AX],ax ; save the AX
;PUSHF ; and the flags (maybe new call)
;POP AX

; 05/05/2019
; MSDOS 6.0

; we have to use DOSDATA for these variables. Previously CS was used
push ax
mov ax,ds
;getdseg <ds> ; ds -> dosdata
mov ds,[cs:DosDSeg]
mov [TEMPSEG],ax
pop ax
MOV [USER_IN_AX],ax ; save the AX
pushf ; and the flags (maybe new call)
pop ax

; See if the input stack is identical to the output stack

; MSDOS 3.3
;CMP SP,[CS:ConC_Spsave]
;JNZ SHORT ctrlc_try_new ; current SP not the same as saved SP

; MSDOS 6.0
CMP SP,[ConC_Spsave]
JNZ SHORT ctrlc_try_new ; current SP not the same as saved SP

; Repeat the operation by redispaching the system call.

ctrlc_repeat:
; MSDOS 3.3
;MOV AX,[CS:USER_IN_AX]
; 05/05/2019
; MSDOS 6.0
mov ax,[USER_IN_AX]
mov ds,[TEMPSEG] ; restore ds and original sp
; MSDOS 3.3 & MSDOS 6.0
;transfer COMMAND
COMMANDJ:
JMP COMMAND

; The current SP is NOT the same as the input SP. Presume that he
; RETF'd leaving some flags on the stack and examine the input

ctrlc_try_new:
; 29/02/2024
;ADD SP,2 ; pop those flags
;
;
;test ax,1
;TEST AX,f_Carry ; did he return with carry?
test al,f_Carry ; test al,1
;

```

```

35947             ; 29/02/2024
35948 00005FB8 58      pop     ax ; (PCDOS 7.1 IBMDOS.COM)
35949             ;
35950 00005FB9 74F1     jz      short ctrlc_repeat ; no carry set, just retry
35951             ;
35952             ; MSDOS 6.0
35953 00005FBB 8E1E[630D] mov     ds,[TEMPSEG] ; restore ds
35954             ;
35955             ; well... time to abort the user.
35956             ; Signal a ^C exit and use the EXIT system call..
35957
35958 ctrlc_abort:
35959             ; MSDOS 3.3
35960             ; ;MOV     AX,(EXIT SHL 8) + 0
35961             ; ;MOV     AX,(EXIT*256) + 0 ; 4C00h
35962             ; mov     byte [CS:DidCTRLC],0FFh ; 14/03/2018
35963             ; ;transfer COMMAND ; give up by faking $EXIT
35964             ; ;JMP     SHORT COMMANDJ
35965             ; ;JMP     COMMAND
35966             ;
35967             ; 05/05/2019 - Retro DOS v4.0
35968             ; MSDOS 6.0
35969 00005FBF B8004C     mov     ax,(EXIT<<8)+0 ; 4C00h
35970 00005FC2 1E        push    ds
35971             ; ;getdseg <ds> ; ds -> dosdata
35972 00005FC3 2E8E1E[0700] mov     ds,[cs:DosDSeg]
35973 00005FC8 C606[4D03]FF mov     byte [DidCTRLC],-1 ; 0FFh
35974 00005FCD 1F        pop     ds
35975             ; ;transfer COMMAND ; give up by faking $EXIT
35976 00005FCE EBE3      jmp     short COMMANDJ
35977             ; ;JMP     COMMAND
35978
35979 ;Break <DIVISION OVERFLOW INTERRUPT>
35980 ;-----
35981 ;
35982 ; Procedure Name : DIVOV
35983 ;
35984 ; Default handler for division overflow trap
35985 ;
35986 ;-----
35987
35988             ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35989 DIVOV:
35990             ; 05/05/2019 - Retro DOS v4.0
35991             ; 30/07/2018
35992             ; 07/07/2018 - Retro DOS v3.0
35993 00005FD0 BE[1B0A]   mov     si,DIVMES
35994 00005FD3 2E8B1E[2E0A] mov     bx,[cs:DivMesLen]
35995             ; mov     ax,cs
35996             ; mov     ss,ax
35997             ; 05/05/2019
35998             ; ;getdseg <ss> ; we are in an ISR, flag is CLI
35999 00005FD8 2E8E16[0700] mov     ss,[cs:DosDSeg]
36000 00005FDD BC[A007]   mov     sp,AUXSTACK
36001             ; call    RealDivov ; MSDOS 3.3
36002 00005FE0 E80200     call    _OUTMES ; MSDOS 6.0
36003 00005FE3 EBDA      jmp     short ctrlc_abort ; Use Ctrl-C abort on divide overflow
36004
36005             ; 30/07/2018
36006
36007             ; MSDOS 6.0
36008 ;-----
36009 ;
36010 ; Procedure Name : OutMes
36011 ;
36012 ;
36013 ; OutMes: perform message output
36014 ; Inputs: SS:SI points to message
36015 ; BX has message length
36016 ; Outputs: message to BCON
36017 ;
36018 ; Actually, cs:si points to the message now. The segment address is filled in
36019 ; at init. time ([diskchret+2]). This will be temporarily changed to DOSCODE.
36020 ; NB. This procedure is called only from DIVOV. -SR
36021 ;
36022 ;-----
36023
36024             ; MSDOS 3.3
36025 ;-----
36026 ;
36027 ; RealDivov: perform actual divide overflow stuff.
36028 ; Inputs: none
36029 ; Outputs: message to BCON
36030 ;-----
36031
36032             ; 05/05/2019 - Retro DOS v4.0
36033             ; DOSCODE:926Ch (MSDOS 6.21, MSDOS.SYS)
36034
36035             ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
36036             ; DOSCODE:9210h (MSDOS 5.0, MSDOS.SYS)
36037 ;-----
36038 ;
36039 ; Procedure Name : OutMes
36040 ;
36041 ;
36042 ; OutMes: perform message output
36043 ; Inputs: SS:SI points to message
36044 ; BX has message length
36045 ; Outputs: message to BCON
36046 ;
36047 ; Actually, cs:si points to the message now. The segment address is filled in
36048 ; at init. time ([diskchret+2]). This will be temporarily changed to DOSCODE.
36049 ; NB. This procedure is called only from DIVOV. -SR
36050 ;
36051 ;-----
36052
36053             ; 30/07/2018
36054             ; MSDOS 6.0
36055 _OUTMES:
36056             ; MSDOS 3.3
36057 ;RealDivov:
36058             ; 07/07/2018 - Retro DOS v3.0
36059             ; Context ES
36060 00005FE5 16        push    ss ; 05/05/2019
36061 00005FE6 07        ; PUSH CS ; 30/07/2018 ; get ES addressability
36062             ; POP     ES
36063             ; Context DS
36064 00005FE7 16        push    ss ; 05/05/2019
36065             ; PUSH CS ; 30/07/2018 ; get DS addressability
36066             ; POP     DS
36067 00005FE8 1F        mov     byte [DSKSTCOM],DEVWRT
36068 00005FE9 C606[9403]08 mov     byte [DSKSTCALL],DRDWRHL
36069 00005FEE C606[9203]16 mov     word [DSKSTST],0
36070 00005FF3 C706[9503]0000 ; BX = [DivMesLen] = 19
36071             ; MOV     [DSKSTCNT],BX

```

```

36071 00005FFD BB[9203]          MOV     BX,DSKSTCALL
36072 00006000 8936[A003]        MOV     [DSKCHRET+1],SI          ; transfer address (need an EQU)
36073                                     ; 08/09/2018
36074                                     ;mov     [DEVIOPTR],si
36075                                     ; MSDOS 6.0
36076                                     ; CS is used for string, fill in
36077                                     ; segment address
36078                                     ; 29/02/2024
36079 00006004 8C0E[A203]        MOV     [DOSSEG_INIT],cs ;
36080                                     MOV     [DSKCHRET+3],CS
36081 00006008 C536[3200]        LDS     SI,[BCON]
36082 0000600C E882F2            CALL    DEVIOPTR
36083
36084                                     ; 14/03/2018
36085                                     ; ;MOV     WORD [CS:DSKCHRET+1],DEVIOPTR
36086                                     ; ; 08/09/2018
36087                                     ;mov     word [CS:DEVIOPTR],DEVIOPTR
36088                                     ; ;MOV     WORD [CS:DSKSTCNT],1
36089
36090                                     ; 05/05/2019 - Retro DOS v4.0 (MSDOS 6.0, MSDOS 6.21)
36091
36092                                     ; ES still points to DOSDATA. ES is
36093                                     ; not destroyed by devioctl2. So use
36094                                     ; ES override.
36095
36096 0000600F 26C706[A003][BC03] MOV     WORD [ES:DSKCHRET+1],DEVIOPTR
36097 00006016 26C706[A403]0100 MOV     WORD [ES:DSKSTCNT],1
36098
36099 0000601D C3                  RETN
36100
36101 ;Break      <CHARHRD,HARDERR,ERROR -- HANDLE DISK ERRORS AND RETURN TO USER>
36102 ;-----
36103 ;
36104 ; Procedure Name : CHARHARD
36105 ;
36106 ;
36107 ; Character device error handler
36108 ; Same function as HARDERR
36109 ;-----
36110
36111                                     ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
36112 CHARHARD:
36113                                     ; 05/05/2019 - Retro DOS v4.0
36114                                     ; 30/07/2018
36115                                     ; 08/07/2018 - Retro DOS v3.0
36116
36117                                     ; MSDOS 6.0
36118
36119                                     ; M024 - start
36120 0000601E 36803E[2003]00      cmp     byte [SS:ERRORMODE], 0; Q: are we in the middle of int 24
36121                                     ;jne     short @f          ; Y: allow fail
36122 00006024 750B                jne     short chard1
36123
36124 00006026 80CC10              OR      AH,Allowed_RETRY ; 10h; assume ctrl p
36125
36126 00006029 36F606[FE02]FF      test    byte [ss:PFLAG],-1      ; Q: has ctrl p been pressed
36127 0000602F 7503                jnz     short ctrlp          ; Y:
36128 ;@@:
36129 chard1:                      ; M024 - end
36130                                     ; MSDOS 6.0 & MSDOS 3.3
36131
36132 ; Character device error handler
36133 ; Same function as HARDERR
36134
36135 ;or      ah,38h
36136 00006031 80CC38              or      ah,Allowed_IGNORE+Allowed_RETRY+Allowed_FAIL
36137 ctrlp:                      ; SS override for Allowed and EXITHOLD
36138 00006034 368826[4B03]        mov     [SS:ALLOWED],ah
36139
36140                                     ; 15/03/2018
36141 00006039 368C06[8205]        MOV     [SS:EXITHOLD+2],ES
36142 0000603E 36892E[8005]        MOV     [SS:EXITHOLD],BP
36143 00006043 56                  PUSH    SI
36144                                     ;and     di,0FFh
36145 00006044 81E7FF00            AND     DI,STECODE
36146 00006048 8CDD                MOV     BP,DS          ;Device pointer is BP:SI
36147 0000604A E89D00            CALL    FATALC
36148 0000604D 5E                  POP     SI
36149                                     ;return
36150 0000604E C3                  RETN
36151
36152 ;-----
36153 ;
36154 ; Procedure Name : HardErr
36155 ;
36156 ; Hard disk error handler. Entry conditions:
36157 ; DS:BX = Original disk transfer address
36158 ; DX = Original logical sector number
36159 ; CX = Number of sectors to go (first one gave the error)
36160 ; AX = Hardware error code
36161 ; DI = Original sector transfer count
36162 ; ES:BP = Base of drive parameters
36163 ; [READOP] = 0 for read, 1 for write
36164 ; Allowed Set with allowed responses to this error (other bits MUST BE 0)
36165 ; Output:
36166 ; [FAILERR] will be set if user responded FAIL
36167 ;-----
36168
36169                                     ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
36170                                     ; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
36171 HARDERR:
36172                                     ; 05/05/2019 - Retro DOS v4.0
36173                                     ; 30/07/2018
36174                                     ; 08/07/2018 - Retro DOS v3.0
36175
36176 0000604F 97                  XCHG     AX,DI          ; Error code in DI, count in AX
36177                                     ;and     di,0FFh
36178 00006050 81E7FF00            AND     DI,STECODE          ; And off status bits
36179                                     ;CMP     DI,WRECODE          ; Write Protect Error?
36180                                     ;cmp     di,0
36181 00006054 83FF00            cmp     DI,error_I24_write_protect ; Write Protect Error?
36182 00006057 750A                JNZ     short NOSETWRPERR
36183 00006059 50                  PUSH    AX
36184                                     ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
36185                                     ;MOV     AL,[ES:BP+DPB.DRIVE]
36186                                     ; ;MOV     AL,[ES:BP+0]
36187                                     ; 15/12/2022
36188 0000605A 268A4600            mov     al,[ES:BP]
36189                                     ; 15/03/2018
36190 0000605E 36A2[2203]        MOV     [SS:WPERR],AL          ; Flag drive with WP error
36191 00006062 58                  POP     AX
36192 NOSETWRPERR:
36193 00006063 29C8                SUB     AX,CX          ; Number of sectors successfully transferred
36194 00006065 01C2                ADD     DX,AX          ; First sector number to retry

```

```

36195 ; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
36196 ;;;
36197 00006067 368316[0706]00 adc word [ss:HIGH_SECTOR],0
36198 ;;;
36199 0000606D 52 PUSH DX
36200 ; 08/07/2018
36201 ;MUL word [ES:BP+2] ; Number of bytes transferred
36202 0000606E 26F76602 MUL word [ES:BP+DPB.SECTOR_SIZE]
36203 00006072 5A POP DX
36204 00006073 01C3 ADD BX,AX ; First address for retry
36205 ;XOR AH,AH ; * ; Flag disk section in error
36206 ; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
36207 ;;;
36208 00006075 31C0 xor ax,ax ; * ; 29/02/2024 - Retro DOS v5.0
36209 ;cmp word [ss:HIGH_SECTOR],0
36210 00006077 363906[0706] cmp [ss:HIGH_SECTOR],ax ; 0 ; *
36211 0000607C 7506 jnz short TESTDIR
36212 ;;;
36213 ;CMP DX,[ES:BP+6] ; In reserved area?
36214 0000607E 263B5606 CMP DX,[ES:BP+DPB.FIRST_FAT]
36215 00006082 7246 JB SHORT ERRINT
36216
36217 TESTDIR: ; 29/02/2024
36218 00006084 FEC4 INC AH ; Flag for FAT
36219
36220 ; 29/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
36221 ;;;
36222 ;cmp word [es:bp+0Fh],0
36223 00006086 26837E0F00 cmp word [es:bp+DPB.FAT_SIZE],0
36224 0000608B 7525 jnz short TESTDIR3 ; not FAT32
36225 0000608D 52 push dx
36226 0000608E 368B16[0706] mov dx,[ss:HIGH_SECTOR]
36227 ;cmp dx,[es:bp+2Bh]
36228 00006093 263B562B cmp dx,[es:bp+DPB.FCLUS_FSECTOR+2]
36229 00006097 5A pop dx
36230 00006098 7504 jnz short TESTDIR1 ; Err in FAT must force recomp of freespace
36231 ;cmp dx,[es:bp+29h]
36232 0000609A 263B5629 cmp dx,[es:bp+DPB.FCLUS_FSECTOR]
36233
36234 TESTDIR1:
36235 jnb short TESTDIR2
36236 000060A0 26C7461FFFFF ;mov word [es:bp+1Fh],0FFFFh ; -1
36237 ;mov word [es:bp+DPB.FREE_CNT],0FFFFh
36238 000060A6 26C74621FFFF ;mov word [es:bp+21h],0FFFFh ; -1
36239 000060AC EB1C ;mov word [es:bp+DPB.FREE_CNT+2],0FFFFh
36240 jmp short ERRINT
36241
36242 TESTDIR2:
36243 inc ah
36244 ;inc ah
36245 000060B0 EB16 jmp short ERRINT2
36246 ; 29/02/2024 - Retro DOS v5.0
36247 TESTDIR3:
36248 ;;;
36249 ;CMP DX,[ES:BP+10h] ; MSDOS 3.3
36250 ;cmp dx,[ES:BP+11h] ; MSDOS 6.0 - 05/05/2019
36251 000060B2 263B5611 CMP DX,[ES:BP+DPB.DIR_SECTOR] ; In FAT?
36252 ;JAE short TESTDIR ; No
36253 000060B6 7308 jae short TESTDIR4 ; 29/02/2024
36254
36255 ; Err in FAT must force recomp of freespace
36256 ;mov word [ES:BP+1Eh],-1 ; MSDOS 3.3
36257 ;mov word [ES:BP+1Fh],-1 ; MSDOS 6.0 - 05/05/2019
36258 000060B8 26C7461FFFFF MOV word [ES:BP+DPB.FREE_CNT],-1
36259
36260 JMP SHORT ERRINT
36261
36262 ;TESTDIR:
36263 TESTDIR4: ; 29/02/2024
36264 INC AH
36265 000060C0 FEC4 ;CMP DX,[ES:BP+0BH] ; In directory?
36266 000060C2 263B560B CMP DX,[ES:BP+DPB.FIRST_SECTOR]
36267 000060C6 7202 JB SHORT ERRINT
36268 ERRINT2: ; 29/02/2024
36269 INC AH ; Must be in data area
36270 ERRINT:
36271 000060CA D0E4 SHL AH,1 ; Make room for read/write bit
36272 000060CC 360A26[7505] OR AH,[SS:READOP] ; 15/03/2018
36273
36274 ; 15/08/2018
36275 000060D1 360A26[4B03] OR AH,[SS:ALLOWED] ; SS override for allowed and EXITHOLD
36276 ; Set the allowed_ bits
36277 ;entry FATAL
36278 FATAL:
36279 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
36280 ;MOV AL,[ES:BP+DPB.DRIVE]
36281 ;;MOV AL,[ES:BP+0] ; Get drive number
36282 ; 15/12/2022
36283 000060D6 268A4600 MOV AL,[ES:BP]
36284
36285 ;entry FATAL1
36286 FATAL1:
36287 ; 15/03/2018
36288 000060DA 368C06[8205] MOV [SS:EXITHOLD+2],ES
36289 000060DF 36892E[8005] MOV [SS:EXITHOLD],BP ; The only things we preserve
36290 ;LES SI,[ES:BP+12h] ; MSDOS 3.3
36291 ;LES SI,[ES:BP+13h] ; MSDOS 6.0 - 05/05/2019
36292 000060E4 26C47613 LES SI,[ES:BP+DPB.DRIVER_ADDR]
36293 000060E8 8CC5 MOV BP,ES ; BP:SI points to the device involved
36294
36295 ; DI has the INT-24-style extended error. We now map the error code
36296 ; for this into the normalized get extended error set by using the
36297 ; ErrMap24 table as a translate table. Note that we translate ONLY
36298 ; the device returned codes and leave all others beyond the look up
36299 ; table alone.
36300
36301 ; 08/07/2018 - Retro DOS v3.0
36302 FATALC:
36303 000060EA E89F01 call SET_I24_EXTENDED_ERROR
36304 ;cmp di,0Ch
36305 000060ED 83FF0C CMP DI,error_I24_gen_failure
36306 000060F0 7603 JBE short GOT_RIGHT_CODE ; Error codes above gen_failure get
36307 000060F2 BF0C00 MOV DI,error_I24_gen_failure; mapped to gen_failure. Real codes
36308 ; Only come via GetExtendedError
36309
36310 ;** -----
36311 ;
36312 ; Entry point used by REDIRECTOR on Network I 24 errors.
36313 ;
36314 ; ASSUME DS:NOTHING,ES:NOTHING,SS:DOSDATA
36315 ;
36316 ; ALL I 24 regs set up. ALL Extended error info SET. ALLOWED Set.
36317 ; EXITHOLD set for restore of ES:BP.
36318 ; -----

```

```

36319 ;entry NET_I24_ENTRY
36320 NET_I24_ENTRY:
36321 GOT_RIGHT_CODE:
36322 000060F5 36803E[2003]00 CMP BYTE [SS:ERRORMODE],0 ; No INT 24s if already INT 24
36323 000060FB 7404 JZ SHORT NoSetFail
36324 000060FD 8003 MOV AL,3
36325 000060FF EB74 JMP short FailRet
36326 NoSetFail:
36327 00006101 368926[8805] MOV [SS:CONTSTK],SP ; SS override
36328 00006106 16 PUSH SS
36329 00006107 07 POP ES
36330
36331 ; wango!!! We may need to free some user state info... In
36332 ; particular, we may have locked down a JFN for a user and he may
36333 ; NEVER return to us. Thus, we need to free it here and then
36334 ; reallocate it when we come back.
36335
36336 00006108 36833E[AA05]FF CMP word [SS:SFN],-1 ; 0FFFFh
36337 0000610E 740C JZ short _NoFree
36338 00006110 1E push ds
36339 00006111 56 push si
36340 00006112 36C536[AE05] LDS SI,[SS:PJFN]
36341 00006117 C604FF MOV BYTE [SI],0FFh
36342 0000611A 5E pop si
36343 0000611B 1F pop ds
36344
36345 _NoFree:
36346 0000611C FA CLI
36347
36348 0000611D 36FE06[2003] INC BYTE [SS:ERRORMODE] ; Prepare to play with stack
36349 00006122 36FE0E[2103] DEC BYTE [SS:INDOS] ; Flag INT 24 in progress
36350 ; 29/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
36351 ;;;
36352 dec byte [ss:INDOS_FLAG] ; 2nd 'in dos' flag (what for?)
36353 00006127 36FE0E[B812] ;;;
36354
36355 ; 05/05/2019 - Retro DOS v4.0 (MSDOS 6.0, MSDOS 6.21)
36356
36357 ;;; Extended Open hooks
36358
36359 ; AN000;IFS.I24 error disabled
36360 ;test byte [ss:EXTOPEN_ON],2
36361 0000612C 36F606[F605]02 TEST byte [ss:EXTOPEN_ON],EXT_OPEN_I24_OFF
36362 00006132 7404 JZ short i24yes ; AN000;IFS.no
36363 faili24: ; AN000;
36364 00006134 8003 MOV AL,3 ; AN000;IFS.fake fail
36365 00006136 EB27 JMP short passi24 ; AN000;IFS.exit
36366 i24yes: ; AN000;
36367 ;;; Extended Open hooks
36368
36369 00006138 368E16[8605] MOV SS,[SS:USER_SS]
36370 0000613D 268B26[8405] MOV SP,[ES:USER_SP] ; User stack pointer restored
36371
36372 ;;;int 24h
36373 ;IN int_fatal_abort ; Fatal error interrupt vector,
36374 ; must preserve ES
36375
36376 00006142 26803E[660D]00 cmp byte [es:DoshASHMA],0 ; Q: is dos running in HMA (M021)
36377 00006148 7504 jne short do_low_int24 ; Y: the int must be done from low mem
36378 0000614A CD24 INT int_fatal_abort ; Fatal error interrupt vector,
36379 ; must preserve ES
36380 0000614C EB05 jmp short criterr_ret_addr
36381
36382 do_low_int24:
36383 ; 05/05/2019
36384 ; MSDOS 6.0
36385 0000614E 2EFF1E[F75D] call far [cs:LowInt24Addr]
36386 criterr_ret_addr:
36387 00006153 268926[8405] MOV [ES:USER_SP],SP ; restore our stack
36388 00006158 268C16[8605] MOV [ES:USER_SS],SS
36389 ;MOV BP,ES
36390 ;MOV SS,BP
36391 ; 30/06/2024
36392 0000615D 06 push es
36393 0000615E 17 pop ss
36394 passi24:
36395 0000615F 368B26[8805] MOV SP,[SS:CONTSTK]
36396 00006164 36FE06[2103] INC BYTE [SS:INDOS] ; Back in the DOS
36397
36398 ; 29/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
36399 ;;;
36400 00006169 36FE06[B812] inc byte [ss:INDOS_FLAG]
36401 ;;;
36402
36403 0000616E 36C606[2003]00 MOV BYTE [SS:ERRORMODE],0 ; Back from INT 24
36404 00006174 FB STI
36405 FailRet:
36406 00006175 36C42E[8005] LES BP,[SS:EXITHOLD]
36407
36408 ; 08/07/2018
36409
36410 ; Triage the user's reply.
36411
36412 0000617A 3C01 CMP AL,1
36413 0000617C 723D JB short CheckIgnore ; 0 => ignore
36414 0000617E 7445 JZ short CheckRetry ; 1 => retry
36415 00006180 3C03 CMP AL,3 ; 3 => fail
36416 00006182 7549 JNZ short DoAbort ; 2, invalid => abort
36417
36418 ; The reply was fail. See if we are allowed to fail.
36419
36420 ; SS override for ALLOWED, EXTOPEN_ON,
36421 ; ALLOWED, FAILERR, WPERR, SFN, PJFN
36422
36423 00006184 36F606[4B03]08 test byte [ss:ALLOWED],8
36424 0000618A 7441 test byte [ss:ALLOWED],Allowed_FAIL ; Can we?
36425 jz short DoAbort ; No, do abort
36426 DoFail:
36427 0000618C B003 MOV AL,3 ; just in case...
36428 ; AN000;EO. I24 error disabled
36429 ; 05/05/2019
36430 ;(MSDOS 6.0, MSCTRLC.ASM, 1991)
36431 00006194 7505 test byte [ss:EXTOPEN_ON],EXT_OPEN_I24_OFF ; 2
36432 jnz short Cleanup ; AN000;EO. no
36433 inc byte [SS:FAILERR] ; Tell everybody
36434 Cleanup:
36435 0000619B 36C606[2203]FF MOV byte [SS:WPERR],-1
36436 000061A1 36833E[AA05]FF CMP word [SS:SFN],-1
36437 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
36438 ;jnz short Cleanup2
36439 ;retn
36440 ; 17/12/2022
36441 000061A7 7411 jz short Cleanup_retn ; 08/07/2018 - Retro DOS v3.0
36442 Cleanup2:

```



```

36443 000061A9 1E          push    ds
36444 000061AA 56          push    si
36445 000061AB 50          push    ax
36446 000061AC 36A1[AA05]    MOV     AX,[ss:SFN]
36447 000061B0 36C536[AE05]  LDS     SI,[ss:PJFN]
36448 000061B5 8804          MOV     [SI],AL
36449 000061B7 58          pop     ax
36450 000061B8 5E          pop     si
36451 000061B9 1F          pop     ds
36452 Cleanup_retn:
36453 000061BA C3          retn
36454
36455          ; The reply was IGNORE. See if we are allowed to ignore.
36456
36457 CheckIgnore:
36458          ;test  byte [ss:ALLOWED],20h
36459 000061BB 36F606[4B03]20 test    byte [ss:ALLOWED],Allowed_IGNORE ; Can we?
36460          ; 29/02/2024
36461 000061C1 74C9    CheckRI:  jz     short DoFail          ; No, do fail
36462 000061C3 EBD6          jmp     short Cleanup
36463
36464          ; The reply was RETRY. See if we are allowed to retry.
36465
36466 CheckRetry:
36467          ;test  byte [ss:ALLOWED],10h
36468 000061C5 36F606[4B03]10 test    byte [ss:ALLOWED],Allowed_RETRY ; Can we?
36469          ;jz     short DoFail          ; No, do fail
36470          ;JMP     short Cleanup
36471          ; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
36472 000061CB EBF4          jmp     short CheckRI
36473
36474          ; The reply was ABORT.
36475 DoAbort:
36476 000061CD 16          push    ss
36477 000061CE 1F          pop     ds
36478
36479 000061CF 803E[5703]00    CMP     byte [CONSWAP],0
36480 000061D4 7403          JZ      short NOSWAP2
36481 000061D6 E880D9    call    SWAPBACK
36482 NOSWAP2:
36483          ; See if we are to truly abort. If we are in the process of aborting,
36484          ; turn this abort into a fail.
36485
36486          ;test  [fAborting],0FFh
36487          ;jnz   short DoFail
36488
36489 000061D9 803E[5903]00    cmp     byte [fAborting],0
36490 000061DE 75AC          JNZ     short DoFail
36491
36492          ; Set return code
36493
36494 000061E0 C606[7C05]02    MOV     BYTE [EXIT_TYPE],EXIT_HARD_ERROR ; 2
36495 000061E5 30C0          XOR     AL,AL
36496
36497          ; we are truly aborting the process. Go restore information from
36498          ; the PDB as necessary.
36499
36500 000061E7 E97510    jmp     exit_inner
36501
36502          ;** -----
36503          ;
36504          ; reset_environment checks the DS value against the CurrentPDB. If they are
36505          ; different, then an old-style return is performed. If they are the same,
36506          ; then we release jfns and restore to parent. We still use the PDB at DS:0 as
36507          ; the source of the terminate addresses.
36508          ;
36509          ; Some subtlety: We are about to issue a bunch of calls that *may* generate
36510          ; INT 24s. We *cannot* allow the user to restart the abort process; we may
36511          ; end up aborting the wrong process or turn a terminate/stay/resident into a
36512          ; normal abort and leave interrupt handlers around. What we do is to set a
36513          ; flag that will indicate that if any abort code is seen, we just continue the
36514          ; operation. In essence, we dis-allow the abort response.
36515          ;
36516          ; output:  none.
36517          ; -----
36518
36519          ;entry reset_environment
36520
36521 reset_environment:
36522          ; 30/07/2018 - Retro DOS v3.0
36523          ; IBMDOS.COM (MSDOS 3.3) - Offset 588Ah
36524
36525 ;***invoke Reset_Version          ; AN007 ;MS. reset version number
36526
36527 000061EA 1E          PUSH     DS          ; save PDB of process
36528
36529          ; There are no critical sections in force. Although we may enter
36530          ; here with critical sections locked down, they are no longer
36531          ; relevant. We may safely free all allocated resources.
36532
36533 000061EB B482          MOV     AH,82h
36534          ; Microsoft Networks - END DOS CRITICAL SECTIONS 0 THROUGH 7
36535          ;int  2Ah
36536 000061ED CD2A          INT     int_IBM
36537
36538          ; SS override
36539 000061EF 36C606[5903]FF    MOV     byte [SS:fAborting],-1; signal abort in progress
36540
36541          ; DOS 4.00 doesn't need it
36542          ;CallInstall NetResetEnvironment, MultNET, 34
36543          ; Allow REDIR to clear some stuff
36544          ; On process exit.
36545 000061F5 B82211    mov     ax, 1122h
36546 000061F8 CD2F          int     2Fh          ; Multiplex - NETWORK REDIRECTOR - PROCESS TERMINATION HOOK
36547          ; SS = DOS CS
36548          ;mov  al,22h
36549 000061FA B022          MOV     AL,int_terminate
36550 000061FC E86AAD    call    _$GET_INTERRUPT_VECTOR; and who to go to
36551
36552 000061FF 59          POP     CX          ; get ThisPDB
36553 00006200 06          push    es
36554 00006201 53          push    bx          ; save return address
36555
36556 00006202 368B1E[3003]    MOV     BX,[SS:CurrentPDB] ; get currentPDB
36557 00006207 8EDB          MOV     DS,BX
36558 00006209 A11600    MOV     AX,[PDB.PARENT_PID] ; get parentPDB
36559
36560          ; AX = parentPDB, BX = CurrentPDB, CX = ThisPDB
36561          ; Only free handles if AX <> BX and BX = CX and [exit_code].upper
36562          ; is not Exit_keep_process
36563
36564 0000620C 39D8          CMP     AX,BX
36565 0000620E 7418          JZ      short reset_return ; parentPDB = CurrentPDB
36566 00006210 39CB          CMP     BX,CX

```

```

36567 00006212 7514      JNZ     short reset_return    ; CurrentPDB <> ThisPDB
36568 00006214 50        PUSH    AX                      ; save parent
36569                                     ; SS override
36570                                     ;
36571 ;cmp     byte [SS:EXIT_TYPE],3
36572 00006215 36803E[7C05]03  CMP     BYTE [SS:EXIT_TYPE],EXIT_KEEP_PROCESS ; 15/08/2018
36573 0000621B 7406      JZ      short reset_to_parent ; keeping this process
36574
36575 ; we are truly removing a process. Free all allocation blocks
36576 ; belonging to this PDB
36577
36578 ;invoke arena_free_process
36579 0000621D E87E10      call    arena_free_process
36580
36581 ; kill off remainder of this process. Close file handles and signal
36582 ; to relevant network folks that this process is dead. Remember that
36583 ; CurrentPDB is STILL the current process!
36584
36585 ;invoke DOS_ABORT
36586 00006220 E889D4      call    DOS_ABORT
36587
36588 reset_to_parent:
36589                                     ; SS override
36590 00006223 368F06[3003]  POP     word [SS:CurrentPDB] ; set up process as parent
36591
36592 reset_return:
36593                                     ; come here for normal return
36594 00006228 16          ;Context DS                      ; DS is used to refer to DOSDATA
36595 00006229 1F          push    ss
36596                                     pop     ds
36597 0000622A B0FF        MOV     AL,-1
36598
36599 ; make sure that everything is clean In this case ignore any errors,
36600 ; we cannot "FAIL" the abort, the program being aborted is dead.
36601
36602 ;EnterCrit critDisk
36603 0000622C E8B3B6      call    ECritDisk
36604 ;invoke FLUSHBUF
36605 0000622F E86108      call    FLUSHBUF
36606 ;LeaveCrit critDisk
36607 00006232 E8DAB6      call    LCritDisk
36608
36609 ; 29/02/2024 - Retrto DOS v5.0
36610 ; PCDOS 7.1 IBMDOS.COM
36611 %if 0
36612 ; Decrement open ref. count if we had done a virtual open earlier.
36613
36614 call    CHECK_VIRT_OPEN
36615 %endif
36616
36617 00006235 FA          CLI
36618 00006236 C606[2103]00  MOV     BYTE [INDOS],0      ; Go to known state
36619
36620 ; 29/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
36621 ;;;
36622 0000623B C606[B812]00  mov     byte [INDOS_FLAG],0
36623 ;;;
36624
36625 00006240 C606[2203]FF  MOV     BYTE [WPERR],-1     ; Forget about WP error
36626 00006245 C606[5903]00  MOV     byte [fAborting],0  ; let aborts occur
36627 0000624A 8F06[8005]   POP     WORD [EXITHOLD]
36628 0000624E 8F06[8205]   POP     WORD [EXITHOLD+2]
36629
36630 ; Snake into multitasking... Get stack from CurrentPDB person
36631
36632 00006252 8E1E[3003]   MOV     DS,[CurrentPDB]
36633 00006256 8E163000     MOV     SS,[PDB.USER_STACK+2]
36634 0000625A 8B262E00     MOV     SP,[PDB.USER_STACK]
36635
36636 0000625E E8E0A1      call    restore_world
36637
36638 ; 05/05/2019
36639 00006261 07          pop     es ; * ; MSDOS 6.21 (DOSCODE:94A8h, MSDOS.SYS)
36640
36641 ; MSDOS 6.0
36642 00006262 50          push    ax
36643 00006263 8CD8        mov     ax,ds                ; set up ds, but save ds in TEMPSEG
36644 ;getdseg <ds>          ; and not on stack.
36645 00006265 2E8E1E[0700]  mov     ds,[cs:DosDseg]     ; ds -> dosdata
36646 0000626A A3[630D]     mov     [TEMPSEG],ax
36647 0000626D 58          pop     ax
36648 ; set up ds to DOSDATA
36649 ;MOV     [CS:USER_SP],AX ; MSDOS 3.3
36650 0000626E A3[8405]     mov     [USER_SP],ax
36651
36652 00006271 58          POP     AX
36653 00006272 58          POP     AX
36654 00006273 58          POP     AX
36655
36656 ; M011 : BEGIN
36657
36658 ; MSDOS 3.3
36659 ; MOV     AX,0F202h      ; STI
36660
36661 ; MSDOS 6.0
36662 00006274 9F          LAHF
36663 00006275 86E0        XCHG    AH,AL
36664 00006277 2402        AND     AL,2
36665 00006279 B4F2        MOV     AH,0F2h
36666
36667 ; M011 : END
36668
36669 ; MSDOS 3.3 (& MSDOS 6.0)
36670 0000627B 50          PUSH    AX
36671
36672 ;PUSH    word [CS:EXITHOLD+2]
36673 ;PUSH    word [CS:EXITHOLD]
36674
36675 ; MSDOS 6.0
36676 0000627C FF36[8205]   PUSH    word [EXITHOLD+2]
36677 00006280 FF36[8005]   PUSH    word [EXITHOLD]
36678
36679 ;MOV     AX,[CS:USER_SP]
36680
36681 ; MSDOS 6.0
36682 00006284 A1[8405]     MOV     AX,[USER_SP]
36683 00006287 8E1E[630D]     mov     ds,[TEMPSEG] ; restore ds
36684
36685 0000628B CF          IRET
36686 ; Long return back to user terminate address
36687 ;-----
36688 ;
36689 ; Procedure Name : SET_I24_EXTENDED_ERROR
36690 ;

```



```

36691 ; This routine handles extended error codes.
36692 ; Input : DI = error code from device
36693 ; Output: All EXTERR fields are set
36694 ;
36695 ;-----
36696
36697 SET_I24_EXTENDED_ERROR:
36698     PUSH    AX
36699             ; ErrMap24End is in DOSDATA
36700     MOV     AX,ErrMap24End
36701     SUB     AX,ErrMap24
36702             ; Change to dosdata to access
36703             ; ErrMap24 and EXTERR -SR
36704             ; 05/05/2019 - Retro DOS v4.0
36705
36706     ; MSDOS 6.0
36707     push    ds
36708     ;getdseg <ds>
36709     mov     ds,[cs:DosDSeg] ; ds ->dosdata
36710
36711     ; AX is the index of the first unavailable error. Do not translate
36712     ; if greater or equal to AX.
36713
36714     CMP     DI,AX
36715     MOV     AX,DI
36716     JAE     short NoTrans
36717
36718     ;MOV     AL,[CS:DI+ErrMap24] ; MSDOS 3.3
36719     mov     al,[ErrMap24+di] ; MSDOS 6.0
36720     XOR     AH,AH
36721 NoTrans:
36722     ;MOV     [CS:EXTERR],AX
36723     mov     [EXTERR],AX
36724     pop     ds
36725     ;assume ds:nothing
36726     POP     AX
36727
36728     ; Now Extended error is set correctly. Translate it to get correct
36729     ; error locus class and recommended action.
36730
36731     PUSH    SI
36732             ; ERR_TABLE_24 is in DOSCODE
36733     MOV     SI,ERR_TABLE_24
36734     call    CAL_LK
36735     POP     SI
36736     retn
36737
36738 ;=====
36739 ; FAT.ASM, MSDOS 6.0, 1991
36740 ;=====
36741 ; 30/07/2018 - Retro DOS v3.0
36742 ; 20/05/2019 - Retro DOS v4.0
36743 ; 02/03/2024 - Retro DOS v5.0
36744
36745 ; TITLE FAT - FAT maintenance routines
36746 ; NAME FAT
36747
36748 ;** FAT.ASM
36749 ;-----
36750 ; Low level local device routines for performing disk change sequence,
36751 ; setting cluster validity, and manipulating the FAT
36752 ;
36753 ; ISEof
36754 ; UNPACK
36755 ; PACK
36756 ; MAPCLUSTER
36757 ; FATREAD_SFT
36758 ; FATREAD_CDS
36759 ; FAT_operation
36760 ;
36761 ; Revision history:
36762 ;
36763 ; AN000 version Jan. 1988
36764 ; A001 PTM -- disk changed for look ahead buffers
36765 ;
36766 ; M014 - if a request for pack\unpack cluster 0 is made we write\read
36767 ; from CL0FATENTRY rather than disk.
36768 ;
36769 ; DOSCODE:94FAh (MSDOS 6.21, MSDOS.SYS)
36770
36771 ; 02/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
36772 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:A40Ch
36773
36774 ;Break <ISEOF - check the quantity in BX for EOF>
36775 ;-----
36776 ;
36777 ; Procedure Name : ISEOF
36778 ;
36779 ; ISEOF - check the fat value in BX for eof.
36780 ;
36781 ; Inputs: ES:BP point to DPB
36782 ; BX has fat value
36783 ; Outputs: JAE eof
36784 ; Registers modified: none
36785 ;
36786 ;-----
36787
36788 ISEOF:
36789 ; 02/03/2024 - Retro DOS v5.0
36790 ; PCDOS 7.1 IBMDOS.COM
36791 ;;;
36792 call IsFAT32 ; is it FAT32 drive ?
36793 jnc short ISEOF_FAT ; no, it has 12 or 16 bit FAT
36794
36795 ; FAT32 fs
36796 cmp word [ss:CLUSTNUM_HW],0FFFh
36797 jnz short ISEOF_other1 ; not EOF
36798 cmp bx,0FFF8h ; 32 bit compare
36799 ISEOF_other1:
36800 retn ; cf=0 -> EOF, cf=1 -> not EOF
36801
36802 ISEOF_FAT:
36803 ;;;
36804
36805 ;cmp word [es:bp+0Dh],0FF6h
36806 cmp word [ES:BP+DPB.MAX_CLUSTER],4096-10 ; is this 16 bit fat?
36807 jae short EOF16 ; yes, check for eof there
36808
36809 ;J.K. 8/27/86
36810 ;Modified to accept 0FF0h as an eof. This is to handle the diskfull case
36811 ;of any media that has "F0"(other) as a MediaByte.
36812 ;Hopefully, this does not create any side effect for those who may use any value
36813 ;other than "FF8-FFF" as an EOF for their own file.
36814

```

```

36815 000062CD 81FBF00F      cmp     bx,0FF0h
36816 000062D1 7404        je      short IsEOF_other
36817
36818 000062D3 81FBF80F      cmp     bx,0FF8h          ; do the 12 bit compare
36819 IsEOF_other:
36820 000062D7 C3        retn
36821 EOF16:
36822 000062D8 83FBF8      cmp     bx,0FFF8h          ; 16 bit compare
36823 000062DB C3        retn          ; cf=0 -> EOF, cf=1 -> not EOF
36824
36825 ; DOSCODE:9511h (MSDOS 6.21, MSDOS.SYS)
36826
36827 ;Break      <UNPACK -- UNPACK FAT ENTRIES>
36828 -----
36829 ;
36830 ; Procedure Name : UNPACK
36831 ;
36832 ; Inputs:
36833 ; BX = Cluster number (may be full 16-bit quantity)
36834 ; ES:BP = Base of drive parameters
36835 ; Outputs:
36836 ; DI = Contents of FAT for given cluster (may be full 16-bit quantity)
36837 ; Zero set means DI=0 (free cluster)
36838 ; Carry set means error (currently user FAILED to I 24)
36839 ; SI Destroyed, No other registers affected. Fatal error if cluster too big.
36840 ;
36841 ; NOTE: if BX = 0 then DI = contents of CL0FATENTRY
36842 ;
36843 -----
36844 ;
36845 ; 02/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
36846 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A44Eh
36847 ;
36848 ; (MSDOS 6.22 MSDOS .SYS - DOSCODE:9511h)
36849 ; (Windows ME IO.SYS - BIOSCODE:0C368h)
36850 ;
36851 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
36852 ; DOSCODE:94B5h (MSDOS 5.0, MSDOS.SYS)
36853 ;
36854 ; 20/05/2019 - Retro DOS v4.0
36855 UNPACK:
36856 ; MSDOS 6.0          ; M014 - Start
36857 000062DC 09DB      or     bx,bx          ; Q: are we unpacking cluster 0
36858 000062DE 7519      jnz     short up_cont    ; N: proceed with normal unpack
36859
36860 ; 02/03/2024
36861 %if 0
36862 mov     di,[CL0FATENTRY]    ; Y: return value in CL0FATENTRY
36863 or      di,di              ; return z if di=0
36864 retn                    ; done
36865 %else
36866 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:A435h
36867 ;;;
36868 up_1:
36869 000062E0 391E[E80A]  cmp     [CLUSTNUM_HW],bx ; 0
36870 000062E4 7513      jne     short up_cont
36871 000062E6 8B3E[3B0F]  mov     di,[CL0FATENTRY_HW]
36872 000062EA 09FF      or      di,di
36873 000062EC 893E[EC0A]  mov     [CCONTENT_HW],di
36874 000062F0 8B3E[8100]  mov     di,[CL0FATENTRY]
36875 000062F4 7502      jnz     short unpack_retn
36876 000062F6 09FF      or      di,di
36877 unpack_retn:
36878 000062F8 C3        retn     ; [CCONTENT_HW]:DI = contents of CL0FATENTRY
36879 ;;;
36880
36881 %endif
36882
36883 up_cont:
36884 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
36885 ;;;
36886 ; cmp     word [es:bp+0Fh],0
36887 000062F9 26837E0F00  cmp     word [es:bp+DPB.FAT_SIZE],0
36888 000062FE 7510      jnz     short up_fat      ; not FAT32
36889 ; FAT32
36890 ; mov     si,[es:bp+2Fh]
36891 00006300 268B762F  mov     si,[es:bp+DPB.LAST_CLUSTER+2]
36892 00006304 3936[E80A]  cmp     [CLUSTNUM_HW],si
36893 00006308 750A      jne     short up_2
36894 ; cmp     bx,[es:bp+2Dh]
36895 0000630A 263B5E2D  cmp     bx,[es:bp+DPB.LAST_CLUSTER]
36896 0000630E EB04      jmp     short up_2
36897 up_fat:
36898 ;;;
36899
36900 ; MSDOS 3.3 & MSDOS 6.0
36901 ; cmp     bx,[es:bp+0Dh]
36902 00006310 263B5E0D  cmp     BX,[es:bp+DPB.MAX_CLUSTER]
36903 up_2:
36904 00006314 7748      ja      short HURTFAT
36905 00006316 E84A01  call    MAPCLUSTER
36906 00006319 7240      jc      short _DoContext
36907
36908 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
36909 ;;;
36910 0000631B FF7502  push    word [di+2]      ; offset CLUSSAVE+2
36911 0000631E 368F06[EC0A]  pop     word [ss:CCONTENT_HW] ; high word of cluster number
36912 ;;;
36913
36914 00006323 8B3D      mov     DI,[DI]
36915 00006325 7521      jnz     short High12      ; MZ if high 12 bits, go get 'em
36916
36917 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
36918 ;;;
36919 00006327 E82401  call    IsFAT32
36920 0000632A 7211      jc      short up_fat32    ; FAT32 volume (drive)
36921 0000632C 36C706[EC0A]0000  mov     word [ss:CCONTENT_HW],0
36922 ;;;
36923
36924 00006333 268B760D  mov     SI,[ES:BP+DPB.MAX_CLUSTER] ; MZ is this 16-bit fat?
36925 00006337 81FEF60F  cmp     SI,4096-10 ; 0FF6h
36926 0000633B 721A      jb      short Unpack12    ; MZ No, go 'AND' off bits
36927
36928 ; 02/03/2024
36929 %if 0
36930 OR      DI,DI          ; MZ set zero condition code, clears carry
36931 JMP     SHORT _DoContext ; MZ go do context
36932 High12:
36933 SHR     DI,1
36934 SHR     DI,1
36935 SHR     DI,1
36936 SHR     DI,1
36937 %else
36938 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)

```

```

36939      ;;
36940      up_fat32:
36941      0000633D 09FF      or     di,di          ; set zero condition code, clears carry
36942      ;;mov     si,ss
36943      ;;mov     ds,si
36944      0000633F 16      push    ss
36945      00006340 1F      pop     ds
36946      00006341 7504      jnz     short up_retn
36947      00006343 093E[EC0A] or     [CCONTENT_HW],di      ; [CCONTENT_HW]:DI = 0 -> zf = 1
36948      up_retn:
36949      00006347 C3      retn
36950
36951      High12:
36952      00006348 36C706[EC0A]0000      mov     word [ss:CCONTENT_HW],0
36953      0000634F D1EF      shr     di,1
36954      00006351 D1EF      shr     di,1
36955      00006353 D1EF      shr     di,1
36956      00006355 D1EF      shr     di,1
36957      ;;
36958      %endif
36959
36960      Unpack12:
36961      00006357 81E7FF0F      AND     DI,0FFFh      ; Clears carry
36962      _DoContext:
36963      0000635B 16      PUSH    SS
36964      0000635C 1F      POP     DS
36965      0000635D C3      retn
36966
36967      HURTFAT:
36968
36969      ; 02/03/2024
36970      %if 0
36971      ;;mov     word [es:bp+1Eh],0FFFFh
36972      ;;mov     word [es:bp+1Fh],0FFFFh ; MSDOS 6.0
36973      MOV     word [ES:BP+DPB.FREE_CNT],-1 ; Err in FAT must force recomp of freespace
36974      %else
36975      ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
36976      ;;
36977      0000635E E84C02      call    chk_set_first_access
36978      ;;
36979      %endif
36980
36981      PUSH    AX
36982      00006362 B488      MOV     AH,Allowed_FAIL+80h ; 88h
36983
36984      ; 02/03/2024
36985      %if 0
36986
36987      ;hkn; SS override
36988      MOV     byte [SS:ALLOWED],Allowed_FAIL ; 8
36989      ;
36990      ; Signal Bad FAT to INT int_fatal_abort handler. We have an invalid cluster.
36991      ;
36992      MOV     DI,0FFFh      ; In case INT int_fatal_abort returns (it shouldn't)
36993      call    FATAL
36994      %else
36995      ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
36996      ;;
36997      ;;mov     byte [ss:ALLOWED],8
36998      00006364 36C606[4B03]08      mov     byte [ss:ALLOWED],Allowed_FAIL
36999      ;
37000      ; 02/03/2024 - Retro DOS v5.0
37001      0000636A 31FF      xor     di,di
37002      0000636C 4F      dec     di ; 0FFFh
37003      0000636D 57      push    di
37004      ; NOTE: DI would be 0FFFh here (or 0FFFh as in MSDOS 6.?)
37005      ; because, in SET_I24_EXTENDED_ERROR in FATAL,
37006      ; DI is compared with error code
37007      ;
37008      ; xor di,di
37009      ; dec di ; 0FFFh
37010      ; push di
37011      ; call FATAL
37012      ; pop di
37013      ; mov word [ss:CCONTENT_HW], di
37014      ;
37015      ; Erdogan Tan - 02/03/2024 - 01/07/2024
37016      ;
37017      0000636E 36FF36[E80A]      push    word [ss:CLUSTNUM_HW] ; (necessary!?)
37018      00006373 53      push    bx ; (necessary!?)
37019      00006374 E85FFD      call    FATAL
37020      00006377 5B      pop     bx
37021      00006378 368F06[E80A]      pop     word [ss:CLUSTNUM_HW]
37022      ;xor     di,di
37023      ;dec     di ; 0FFFh
37024      ;mov     word [ss:CCONTENT_HW],di
37025      ; 02/03/2024 - Retro DOS v5.0
37026      ;pop     word [ss:CCONTENT_HW]
37027      ; 01/07/2024
37028      0000637D 5F      pop     di
37029      0000637E 36893E[EC0A]      mov     [ss:CCONTENT_HW], di
37030      ;;
37031      %endif
37032      00006383 3C03      CMP     AL,3
37033      00006385 F8      CLC
37034      00006386 7501      JNZ     short OKU_RET      ; Try to ignore bad FAT
37035      00006388 F9      STC      ; User said FAIL
37036      OKU_RET:
37037      00006389 58      POP     AX
37038      hurtfat_retn:
37039      0000638A C3      retn
37040
37041      ; DOSCODE:9565h (MSDOS 6.21, MSDOS.SYS)
37042
37043      ;Break      <PACK -- PACK FAT ENTRIES>
37044      ;-----
37045      ;
37046      ; Procedure Name : PACK
37047      ;
37048      ; Inputs:
37049      ; BX = Cluster number
37050      ; DX = Data
37051      ; ES:BP = Pointer to drive DPB
37052      ; Outputs:
37053      ; The data is stored in the FAT at the given cluster.
37054      ; SI,DX,DI all destroyed
37055      ; Carry set means error (currently user FAILED to I 24)
37056      ; No other registers affected
37057      ;
37058      ; NOTE: if BX = 0 then data in DX is stored in CLOFATENTRY.
37059      ;-----
37060
37061      ; 02/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37062

```

```

37063
37064 ; INPUT:
37065 ; [CLUSDATA_HW]:DX = cluster data/content
37066 ; [CLUSTNUM_HW]:BX = cluster number (to be updated)
37067 ; ES:BP = pointer to DPB
37068
37069 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37070 ; 20/05/2019 - Retro DOS v4.0
37071
37072 PACK:
37073 ; MSDOS 6.0 ; M014 - start
37074 or bx,bx ; Q: are we packing cluster 0
37075 jnz short p_cont ; N: proceed with normal pack
37076
37077 ; 02/03/2024
37078 %if 0
37079 mov [CLOFATENTRY],dx ; Y: place value in CLOFATENTRY
37080 retn ; done
37081 %else
37082 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37083 ;;;
37084 cmp [CLUSTNUM_HW],bx ; 0
37085 jnz short p_cont
37086
37087 ;push ax
37088 ;mov ax,[CLUSDATA_HW] ; Cluster Data/Content (High word)
37089 ;mov [CLOFATENTRY_HW],ax
37090 ;pop ax
37091 push word [CLUSDATA_HW]
37092 pop word [CLOFATENTRY_HW]
37093
37094 mov [CLOFATENTRY],dx
37095 retn
37096 ;;;
37097 %endif
37098
37099 p_cont: ; M014 - end
37100 ; MSDOS 3.3 & MSDOS 6.0
37101 CALL MAPCLUSTER
37102 JC short _DoContext
37103 MOV SI,[DI]
37104 JZ short ALIGNED ; byte (not nibble) aligned
37105 PUSH CX ; move data to upper 12 bits
37106 MOV CL,4
37107 SHL DX,CL
37108 POP CX
37109 AND SI,0FH ; leave in original low 4 bits
37110 JMP SHORT PACKIN
37111
37112 ALIGNED:
37113 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37114 ;;;
37115 call IsFAT32
37116 jnb short p_fat
37117
37118 mov si,[ss:CLUSDATA_HW]
37119 xor si,[di+2] ; hw of cluster number
37120 ; offset CLUSSAVE+2
37121 and si,0FFFh
37122 xor [di+2],si
37123 mov [di],dx
37124 jmp short PACKIN2
37125
37126 p_fat:
37127 ;;;
37128 ;cmp word [es:bp+0dh],0FF6h
37129 CMP word [ES:BP+DPB.MAX_CLUSTER],4096-10 ; MZ 16 bit fats?
37130 JAE short Pack16 ; MZ yes, go clobber original data
37131 AND SI,0F000h ; MZ leave in upper 4 bits of original
37132 ;AND DX,0FFFh ; MZ store only 12 bits
37133 ; 01/07/2024
37134 and dh,0Fh
37135 JMP SHORT PACKIN ; MZ go store
37136
37137 Pack16:
37138 XOR SI,SI ; MZ no original data
37139
37140 PACKIN:
37141 OR SI,DX
37142 MOV [DI],SI
37143
37144 PACKIN2: ; 02/03/2024
37145
37146 ;hkn; SS override
37147 LDS SI,[SS:CURBUF]
37148 ; MSDOS 6.0
37149 TEST byte [SI+BUFFINFO.buf_flags],buf_dirty
37150 ;LB. if already dirty ;AN000;
37151 JNZ short yesdirty11 ;LB. don't increment dirty count ;AN000;
37152 ; 10/06/2019
37153 call INC_DIRTY_COUNT ;LB. ;AN000;
37154
37155 ;or byte [si+5],40h
37156 OR byte [SI+BUFFINFO.buf_flags],buf_dirty
37157 yesdirty11: ;LB. ;AN000;
37158 ;hkn; SS override
37159 CMP BYTE [SS:CLUSSPLIT],0 ; 15/08/2018
37160 ;hkn; SS is DOSDATA
37161 push ss
37162 pop ds
37163 jz short hurtfat_retn ; Carry clear
37164 PUSH AX
37165 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37166 ;;;
37167 push word [CLUSTNUM_HW]
37168 ;;;
37169 PUSH BX
37170 PUSH CX
37171 MOV AX,[CLUSSAVE]
37172 MOV DS,[CURBUF+2]
37173 ;;;add si,16 ; MSDOS 3.3
37174 ;add si,20 ; MSDOS 6.0
37175 ; 02/01/2024
37176 ;add si,24 ; PCDOS 7.1
37177 ADD SI,BUFINSIZ
37178 MOV [SI],AH
37179 ;hkn; SS is DOSDATA
37180 ;Context DS
37181 push ss
37182 pop ds
37183
37184 PUSH AX
37185
37186 ; MSDOS 6.0
37187 MOV DX,[CLUSSEC+2] ;F.C. >32mb ;AN000;
37188 MOV [HIGH_SECTOR],DX ;F.C. >32mb ;AN000;

```

```

37187
37188 ; MSDOS 3.3 & MSDOS 6.0
37189 0000641F 8B16[9005] MOV DX,[CLUSSEC]
37190
37191 ;MOV SI,1 ; *
37192 ;XOR AL,AL ; *
37193 ;call GETBUFFRB ; *
37194 ; 22/09/2023
37195 00006423 E8F704 call GETBUFFRA ; *
37196
37197 00006426 58 POP AX
37198 00006427 721B JC short POPP_RET
37199 00006429 C53E[E205] LDS DI,[CURBUF]
37200
37201 ; MSDOS 6.0
37202 0000642D F6450540 TEST byte [DI+BUFFINFO.buf_flags],buf_dirty
37203 ;LB. if already dirty ;AN000;
37204 00006431 7507 JNZ short yesdirty12 ;LB. don't increment dirty count ;AN000;
37205 00006433 E88F07 call INC_DIRTY_COUNT ;LB. ;AN000;
37206
37207 ;or byte [di+5],40h
37208 00006436 804D0540 OR byte [DI+BUFFINFO.buf_flags],buf_dirty
37209 yesdirty12:
37210 ;;;add di,16
37211 ;;;add di,20 ; MSDOS 6.0
37212 ;ADD DI,BUFINSIZ
37213 ;DEC DI
37214 ; 02/01/2024 - Retro DOS v5.0
37215 ;add di,23 ; PCDOS 7.1
37216 0000643A 83C717 add di,BUFINSIZ-1 ; 23 ; PCDOS 7.1 IBMDOS.COM
37217
37218 ;add di,[es:bp+2]
37219 0000643D 26037E02 ADD DI,[ES:BP+DPB.SECTOR_SIZE]
37220 00006441 8805 MOV [DI],AL
37221 00006443 F8 CLC
37222 POPP_RET:
37223 ; 02/03/2024
37224 00006444 16 PUSH SS
37225 00006445 1F POP DS
37226 00006446 59 POP CX
37227 00006447 5B POP BX
37228 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37229 ;;;
37230 ;pop word [ss:CLUSTNUM_HW]
37231 ; 01/07/2024
37232 ; ds = ss
37233 00006448 8F06[E80A] pop word [CLUSTNUM_HW]
37234 ;;;
37235 0000644C 58 POP AX
37236 0000644D C3 retn
37237
37238 ; ===== S U B R O U T I N E =====
37239
37240 ; 02/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37241 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A5B9h
37242 IsFAT32:
37243 ;cmp word [es:bp+0Fh],1
37244 0000644E 26837E0F01 cmp word [es:bp+DPB.FAT_SIZE],1
37245 00006453 730D jnb short isfat32eof_2 ; not FAT32
37246
37247 ;cmp word [es:bp+2Fh],0
37248 00006455 26837E2F00 cmp word [es:bp+DPB.LAST_CLUSTER+2],0
37249 0000645A 7505 jnz short isfat32eof_1 ; FAT32
37250
37251 ;cmp word [es:bp+2Dh],0FFF6h
37252 0000645C 26837E2DF6 cmp word [es:bp+DPB.LAST_CLUSTER],0FFF6h
37253 isfat32eof_1:
37254 00006461 F5 cmc ; cf=1 -> FAT32
37255 isfat32eof_2:
37256 00006462 C3 retn
37257
37258 ;-----
37259
37260 ; 31/07/2018 - Retro DOS v3.0
37261
37262 ;Break <MAPCLUSTER - BUFFER A FAT SECTOR>
37263 ;
37264 ;-----
37265 ; Procedure Name : MAPCLUSTER
37266 ;
37267 ; Inputs:
37268 ; ES:BP Points to DPB
37269 ; BX Is cluster number
37270 ; Function:
37271 ; Get a pointer to the cluster
37272 ; Outputs:
37273 ; DS:DI Points to contents of FAT for given cluster
37274 ; DS:SI Points to start of buffer
37275 ; Zero Not set if cluster data is in high 12 bits of word
37276 ; Zero set if cluster data is in low 12 or 16 bits
37277 ; Carry set if failed.
37278 ; SI is destroyed.
37279 ;
37280 ;-----
37281
37282 ; 20/05/2019 - Retro DOS v4.0
37283 ; DOSCODE:9601h (MSDOS 6.21, MSDOS.SYS)
37284 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37285 ; DOSCODE:95A5h (MSDOS 5.0, MSDOS.SYS)
37286
37287 ; 02/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37288 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A5D7h
37289
37290 MAPCLUSTER:
37291 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5A15h
37292 00006463 C606[7805]00 MOV BYTE [CLUSSPLIT],0
37293 ;SAVE <AX,BX,CX,DX>
37294 00006468 50 push ax
37295 00006469 53 push bx
37296 0000646A 51 push cx
37297 0000646B 52 push dx
37298 ; 02/03/2024
37299 ;MOV AX,BX ; AX = BX
37300
37301 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37302 ;;;
37303 ;push word [CLUSTNUM_HW]
37304 0000646C 89D8 mov ax,bx
37305 0000646E 8B16[E80A] mov dx,[CLUSTNUM_HW]
37306 ;
37307 00006472 52 push dx ; 02/03/2024 - Retro DOS v5.0
37308 ;
37309 00006473 31FF xor di,di
37310 00006475 E8D6FF call IsFAT32

```

```

37311 00006478 730E      jnc      short mapc11 ; not FAT32
37312                      ; FAT32
37313 0000647A D1E0      shl      ax,1
37314 0000647C D1D2      rcl      dx,1
37315 0000647E D1D7      rcl      di,1
37316 00006480 D1E0      shl      ax,1
37317 00006482 D1D2      rcl      dx,1
37318 00006484 D1D7      rcl      di,1
37319                      ; DI:DX:AX = 4*(DX:AX)
37320 00006486 EB14      jmp      short Map16          ; 32 bit FAT
37321 mapc11:
37322                      ;;;
37323 00006488 26817E0DF60F CMP      word [ES:BP+DPB.MAX_CLUSTER],4096-10 ; MZ 16 bit fat?
37324                      ; 02/03/2024 - Retro DOS v5.0
37325                      ;JAE      short Map16          ; MZ yes, do 16 bit algorithm
37326                      ;SHR      AX,1                ; AX = BX/2
37327                      ;
37328                      ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37329                      ;;;
37330 0000648E 7304      jae      short mapc12          ; 16 bit FAT
37331                      ; 12 bit FAT
37332 00006490 D1EA      shr      dx,1
37333 00006492 D1D8      rcr      ax,1
37334 mapc12:
37335 00006494 01D8      add      ax,bx
37336 00006496 1316[E80A] adc      dx,[CLUSTNUM_HW]
37337 0000649A 11FF      adc      di,di
37338                      ;;;
37339 Map16:
37340                      ;
37341                      ; 02/04/2024 - Retro DOS v5.0
37342                      ; (PCDOS 7.1 IBMDOS.COM)
37343 %if 0
37344                      ; MSDOS 6.0
37345 XOR      DI,DI ; *          ; MZ skip prev => AX=2*BX
37346                      ; >32mb fat ;AN000;
37347                      ;
37348                      ; MSDOS 3.3 (& MSDOS 6.0)
37349 ADD      AX,BX          ; AX = 1.5*fat = byte offset in fat
37350 ADC      DI,DI ; * MSDOS 6.0 ; >32mb fat ;DI is zero before op;AN000;
37351 %endif
37352 0000649C 268B4E02      MOV      CX,[ES:BP+DPB.SECTOR_SIZE]
37353
37354 ;IF FastDiv
37355 ;
37356 ; Gross hack: 99% of all disks have 512 bytes per sector. We test for this
37357 ; case and apply a really fast algorithm to get the desired results
37358 ;
37359 ; Divide method takes 157+4*4=173 (MOV and DIV)
37360 ; Fast method takes 39+20*4=119
37361 ;
37362 ; This saves a bunch.
37363
37364 000064A0 81F90002      CMP      CX,512          ; 4 Is this 512 byte sector?
37365 000064A4 751C      jne      short _DoDiv          ; 4 for no jump
37366
37367 ; 02/04/2024
37368 %if 0
37369 MOV      DX,AX          ; 2 get set for remainder
37370 AND      DX,512-1        ; 4 Form remainder
37371 MOV      AL,AH          ; 2 Quotient in formation in AL
37372                      ; MSDOS 3.3
37373 ;shr      al,1
37374                      ; MSDOS 6.0
37375 shr      di,1          ; 2
37376 rcr      al,1          ; 2
37377                      ; MSDOS 3.3 (& MSDOS 6.0)
37378 xor      ah,ah          ; 3
37379 %else
37380                      ; 02/04/2024 - Retro DOS v5.0
37381                      ; (PCDOS 7.1 IBMDOS.COM)
37382                      ;;;
37383 000064A6 50          push     ax
37384 000064A7 D1EF      shr      di,1
37385 000064A9 D1DA      rcr      dx,1
37386 000064AB D1D8      rcr      ax,1
37387 000064AD 88E0      mov      al,ah          ; 9 bit shift to right
37388 000064AF 88D4      mov      ah,dl
37389 000064B1 88F2      mov      dl,dh
37390 000064B3 30F6      xor      dh,dh
37391 000064B5 D1EF      shr      di,1
37392 000064B7 10F6      adc      dh,dh
37393 000064B9 87D7      xchg     dx,di          ; DI:AX = sector contains requested cluster data
37394 000064BB 5A          pop      dx
37395 000064BC 81E2FF01      and      dx,1FFh        ; offset in sector
37396                      ;;;
37397 %endif
37398 000064C0 EB07      jmp      short DivDone          ; 16
37399
37400 _DoDiv:
37401 ;ENDIF
37402
37403 ; 02/04/2024
37404 %if 0
37405                      ; MSDOS 3.3
37406 ;xor      dx,dx
37407                      ; MSDOS 6.0
37408 mov      dx,di          ; 2
37409                      ; MSDOS 3.3 (& MSDOS 6.0)
37410 DIV      CX              ; 155 AX is FAT sector # DX is sector index
37411 %else
37412                      ; 02/04/2024 - Retro DOS v5.0
37413                      ; (PCDOS 7.1 IBMDOS.COM)
37414                      ;;;
37415 000064C2 97          xchg     ax,di
37416 000064C3 92          xchg     ax,dx
37417 000064C4 F7F1      div      cx
37418 000064C6 97          xchg     ax,di
37419 000064C7 F7F1      div      cx
37420                      ;jmp      short DivDone
37421                      ;;;
37422 %endif
37423
37424 ;IF FastDiv
37425 DivDone:
37426 ;ENDIF
37427                      ;add      ax,[es:bp+6]
37428 000064C9 26034606      ADD      AX,[ES:BP+DPB.FIRST_FAT]
37429
37430                      ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37431                      ;;;
37432 000064CD 83D700      adc      di,0
37433                      ;;;
37434

```



```

37435 000064D0 49      DEC     CX                ; CX is sector size - 1
37436
37437                ;SAVE  <AX,DX,CX>
37438
37439                ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37440                ;;;
37441 000064D1 57      push    di ; 4*
37442                ;;;
37443 000064D2 50      push    ax ; 3*
37444 000064D3 52      push    dx ; 2*
37445 000064D4 51      push    cx ; 1*
37446
37447 000064D5 89C2     MOV     DX,AX
37448
37449                ; 02/03/2024 - Retro DOS v5.0
37450                ; mov  [HIGH_SECTOR],di ; *!* ; PCDOS 7.1 IBMDOS.COM
37451 000064D7 89FB     mov     bx,di
37452                ;
37453
37454                ; MSDOS 6.0
37455                ; 22/09/2023
37456                ;MOV   word [HIGH_SECTOR],0 ; *! ;F.C. >32mb low sector #
37457                ;
37458                ; MDOS 3.3 (& MSDOS 6.0)
37459                ;XOR   AL,AL          ; *
37460                ;MOV   SI,1          ; *
37461                ;;invoke GETBUFFRB ; *
37462                ;call  GETBUFFRB ; *
37463                ; 22/09/2023
37464 000064D9 E83D04   call    GETBUFFRC ; *! ; *!* ; 02/03/2024 - Retro DOS v5.0
37465
37466                ;RESTORE <CX,AX,DX>          ; CX is sec siz-1, AX is offset in sec
37467 000064DC 59      pop     cx ; 1*
37468 000064DD 58      pop     ax ; 2*
37469 000064DE 5A      pop     dx ; 3*
37470                ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37471                ;;;
37472 000064DF 5B      pop     bx ; 4*
37473                ;;;
37474
37475 000064E0 725A     JC      short MAP_POP
37476
37477 000064E2 C536[E205] LDS     SI,[CURBUF]
37478                ;;;lea di,[si+16]
37479                ;;;lea di,[si+20] ; MSDOS 6.0
37480                ;lea  di,[si+24] ; PCDOS 7.1; 02/03/2024
37481 000064E6 8D7C18   LEA     DI,[SI+BUFINSIZ]
37482 000064E9 01C7     ADD     DI,AX
37483 000064EB 39C8     CMP     AX,CX
37484 000064ED 7530     JNZ     short MAPRET ; ds<>ss
37485 000064EF 8A05     MOV     AL,[DI]
37486                ;Context DS                ;hkn; SS is DOSDATA
37487 000064F1 16      push    ss
37488 000064F2 1F      pop     ds
37489 000064F3 FE06[7805] INC     BYTE [CLUSSPLIT]
37490 000064F7 A2[8E05] MOV     [CLUSSAVE],AL
37491 000064FA 8916[9005] MOV     [CLUSSEC],DX
37492
37493                ; MSDOS 6.0
37494                ;MOV   WORD [CLUSSEC+2],0      ;F.C. >32mb ;AN000;
37495                ;INC   DX
37496                ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37497                ;;;
37498                ;xor   ax,ax ; 0
37499 00006500 83C201   add     dx,1
37500 00006503 891E[9205] mov     word [CLUSSEC+2],bx
37501 00006507 11C3     adc     bx,ax
37502                ;mov  [HIGH_SECTOR],bx ; *!*
37503                ;;;
37504
37505                ; 22/09/2023
37506                ;MOV   word [HIGH_SECTOR],0 ; *! ;F.C. >32mb FAT sector <32mb ;AN000;
37507                ;
37508                ; MDOS 3.3 (& MSDOS 6.0)
37509                ;XOR   AL,AL          ; *
37510                ;MOV   SI,1          ; *
37511                ;;invoke GETBUFFRB ; *
37512                ;call  GETBUFFRB ; *
37513                ; 22/09/2023
37514 00006509 E80D04   call    GETBUFFRC ; *! ; *!* ; 02/03/2024 - Retro DOS v5.0
37515 0000650C 722E     JC      short MAP_POP
37516
37517 0000650E C536[E205] LDS     SI,[CURBUF]
37518 00006512 8D7C18   LEA     DI,[SI+BUFINSIZ]
37519 00006515 8A05     MOV     AL,[DI]
37520                ;Context DS                ;hkn; SS is DOSDATA
37521 00006517 16      push    ss
37522 00006518 1F      pop     ds
37523 00006519 A2[8F05] MOV     [CLUSSAVE+1],AL
37524
37525                ;hkn; CLUSSAVE is in DOSDATA
37526 0000651C BF[8E05] MOV     DI,CLUSSAVE
37527 MAPRET:
37528                ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37529                ;;;
37530 0000651F 368F06[E80A] pop     word [ss:CLUSTNUM_HW] ; (ds<>ss)
37531                ;;;
37532                ;RESTORE <DX,CX,BX>
37533 00006524 5A      pop     dx
37534 00006525 59      pop     cx
37535 00006526 5B      pop     bx
37536
37537 00006527 31C0     XOR     AX,AX                ; MZ allow shift to clear carry
37538
37539                ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37540                ;;;
37541 00006529 E822FF     call    IsFAT32
37542 0000652C 720A     JC      short MapSet ; FAT32 fs
37543                ;;;
37544
37545 0000652E 26817E0DF60F CMP     word [ES:BP+DPB.MAX_CLUSTER],4096-10 ; MZ is this 16-bit fat?
37546 00006534 7302     JAE     short MapSet ; MZ no, set flags
37547 00006536 89D8     MOV     AX,BX
37548 MapSet:
37549 00006538 A801     TEST    AL,1                ; set zero flag if not on boundary
37550                ;RESTORE <AX>
37551 0000653A 58      pop     ax
37552 0000653B C3      retn
37553
37554 MAP_POP:
37555                ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37556                ;;;
37557                ;pop  word [ss:CLUSTNUM_HW]
37558                ; 02/03/2024 - Retro DOS v5.0

```

```

37559 0000653C 8F06[E80A]      pop     word [CLUSTNUM_HW] ; (ds=ss)
37560                          ;;;
37561                          ;RESTORE <DX,CX,BX,AX>
37562 00006540 5A              pop     dx
37563 00006541 59              pop     cx
37564 00006542 5B              pop     bx
37565 00006543 58              pop     ax
37566 fatread_sft_retn: ; 17/12/2022
37567 00006544 C3              retn
37568
37569                          ; 20/05/2019 - Retro DOS v4.0
37570                          ; DOSCODE:96B3h (MSDOS 6.21, MSDOS.SYS)
37571                          ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37572                          ; DOSCODE:9657h (MSDOS 5.0, MSDOS.SYS)
37573
37574                          ; 03/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37575                          ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A6C6h
37576
37577 ;Break      <FATREAD_SFT/FATREAD_CDS -- CHECK DRIVE GET FAT>
37578 -----
37579
37580 ; Procedure Name : FATREAD_SFT
37581 ;
37582 ; Inputs:
37583 ;   ES:DI points to an SFT for the drive of interest (local only,
37584 ;   giving a NET SFT will produce system crashing results).
37585 ;   DS DOSDATA
37586 ; Function:
37587 ;   Can be used by an SFT routine (like CLOSE) to invalidate buffers
37588 ;   if disk changed.
37589 ;   In other respects, same as FATREAD_CDS.
37590 ;   (note ES:DI destroyed!)
37591 ; Outputs:
37592 ;   Carry set if error (currently user FAILED to I 24)
37593 ; NOTE: This routine may cause FATREAD_CDS to "miss" a disk change
37594 ; as far as invalidating curdir_ID is concerned.
37595 ; Since getting a true disk changed on this call is a screw up
37596 ; anyway, that's the way it goes.
37597 ;
37598 -----
37599
37600 FATREAD_SFT:
37601 00006545 26C46D07      LES     BP,[ES:DI+SF_ENTRY.sf_devptr]
37602
37603 fatread_gotdpb:      ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37604                  ; 27/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
37605                  ;MOV     AL,[ES:BP+DPB.DRIVE] ; [es:bp+0]
37606                  ; 15/12/2022
37607 mov     AL,[ES:BP]
37608 MOV     [THISDRV],AL
37609 call    GOTDPB          ; Set THISDPB
37610 ;CALL    FAT_GOT_DPB
37611 ; 17/12/2022
37612 jmp     FAT_GOT_DPB
37613 ;fatread_sft_retn:
37614 ;retn
37615
37616 -----
37617
37618 ; Procedure Name : FATREAD_CDS
37619 ;
37620 ; Inputs:
37621 ;   DS:DOSDATA
37622 ;   ES:DI points to an CDS for the drive of interest (local only,
37623 ;   giving a NET or NUL CDS will produce system crashing results).
37624 ; Function:
37625 ;   If disk may have been changed, media is determined and buffers are
37626 ;   flagged invalid. If not, no action is taken.
37627 ; Outputs:
37628 ;   ES:BP = Drive parameter block
37629 ;   THISDPB = ES:BP
37630 ;   THISDRV set
37631 ;   Carry set if error (currently user FAILED to I 24)
37632 ;   DS preserved , all other registers destroyed
37633 ;
37634 -----
37635
37636                          ; 20/05/2019 - Retro DOS v4.0
37637                          ; DOSCODE:96C5h (MSDOS 6.21, MSDOS.SYS)
37638                          ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37639                          ; DOSCODE:9669h (MSDOS 5.0, MSDOS.SYS)
37640
37641                          ; 03/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37642                          ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A6D8h
37643                          ; (Windows ME IO.SYS - BIOSCODE:0C644h)
37644
37645 FATREAD_CDS:
37646 00006556 06              PUSH    ES
37647 00006557 57              PUSH    DI
37648
37649                          ;les     bp,[es:di+45h]
37650 LES     BP,[ES:DI+curdir.devptr]
37651
37652 ; 03/03/2024
37653 %if 0
37654 ; 27/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
37655 ;MOV     AL,[ES:BP+DPB.DRIVE] ; [es:bp+0]
37656 ; 15/12/2022
37657 mov     AL,[ES:BP]
37658 MOV     [THISDRV],AL
37659 call    GOTDPB          ;Set THISDPB
37660 CALL    FAT_GOT_DPB
37661 %else
37662 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37663 ;;;
37664 0000655C E8EAF7      call    fatread_gotdpb
37665 ;;;
37666 %endif
37667 0000655F 5F              POP     DI          ;Get back CDS pointer
37668 00006560 07              POP     ES
37669 00006561 72E1      jc     short fatread_sft_retn
37670 00006563 7542      jnz     short NO_CHANGE          ;Media NOT changed
37671
37672 ; Media changed. We now need to find all CDS structures which use this
37673 ; DPB and invalidate their ID pointers.
37674
37675 MED_CHANGE:
37676 ;XOR     AX,AX
37677 ;DEC     AX          ; AX = -1
37678 ; 03/03/2024
37679 00006565 B8FFFF      mov     ax,0FFFFh ; -1
37680
37681 00006568 1E              PUSH    DS
37682 00006569 8A0E[4700]     MOV     CL,[CDSCOUNT]

```



```

37683 0000656D 30ED          XOR    CH,CH          ; CX is number of structures
37684                      ;lds    si,[es:di+45h]
37685 0000656F 26C57545      LDS     SI,[ES:DI+curdir.devptr] ; Find all CDS with this devptr
37686
37687                      ;hkn; ss override
37688
37689                      ; Find all CDSs with this DevPtr
37690                      ;
37691                      ; (ax) = -1
37692                      ; (ds:si) = DevPtr
37693
37694 00006573 36C43E[3C00]    LES     DI,[SS:CDSADDR]          ; (es:di) = CDS pointer
37695 frcd20:
37696                      ;;test word [es:di+43h],8000h
37697                      ;TEST word [ES:DI+curdir.flags],curdir_isnet
37698 00006578 26F6454480      TEST    byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
37699 0000657D 7522          JNZ     short frcd25          ; Leave NET guys alone!!
37700
37701                      ; MSDOS 3.3
37702                      ;push    es
37703                      ;push    di
37704                      ;les     di,[es:di+45h]
37705                      ;;les    di,[ES:DI+curdir.devptr]
37706                      ;call    POINTCOMP
37707                      ;pop     di
37708                      ;pop     es
37709                      ;jnz     short frcd25
37710
37711                      ; MSDOS 6.0
37712 0000657F 263B7545      cmp     si,[ES:DI+curdir.devptr]
37713 00006583 751C          jne     short frcd25          ; no match
37714 00006585 8CDB          mov     bx,ds
37715 00006587 263B5D47      cmp     bx,[ES:DI+curdir.devptr+2]
37716 0000658B 7514          jne     short frcd25          ; CDS not for this drive
37717
37718                      ; MSDOS 3.3 (& MSDOS 6.0)
37719                      ;test    [es:di+49h],ax ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A70Fh
37720 0000658D 26854549      test    [ES:DI+curdir.ID],AX
37721                      ;JZ     short frcd25 ; = 0 ; If root (0), leave root
37722                      ; !!! test es:[di+49h],ax
37723                      ; !!! jz short frcd25
37724                      ; PCDOS 7.1 IBMDOS.COM bug (!?) ; Erdogan Tan - 03/03/2024
37725                      ; test dword ptr es:[di+49h],-1 ; win ME - BIOSCODE:0C694h
37726                      ; jz short frcd25
37727                      ;
37728                      ; 03/03/2024 - Retro DOS v5.0
37729 00006591 7506          jnz     short frcd24
37730
37731                      ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37732                      ;;;
37733                      ;test    [es:di+48h],ax
37734 00006593 2685454B      test    [es:di+curdir.ID+2],AX
37735 00006597 7408          jz      short frcd25 ; = 0
37736 frcd24:                      ; 03/03/2024
37737                      ;;;
37738                      ;mov     [es:di+49h],ax
37739 00006599 26894549      MOV     [ES:DI+curdir.ID],AX ; else invalid
37740                      ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37741                      ;;;
37742                      ;mov     [es:di+48h],ax
37743 0000659D 2689454B      mov     [es:di+curdir.ID+2],ax ; -1
37744                      ;;;
37745 frcd25:
37746                      ;add     di,81 ; MSDOS 3.3
37747                      ;add     di,88 ; MSDOS 6.0
37748 000065A1 83C758      ADD     DI,curdir.size          ; Point to next CDS
37749 000065A4 E2D2          LOOP   frcd20
37750 000065A6 1F          POP     DS
37751
37752 000065A7 C42E[8A05]    NO_CHANGE:
37753 000065AB F8          LES     BP,[THISDPB]
37754 000065AC C3          CLC
37755                      retn
37756
37757                      ; ===== S U B R O U T I N E =====
37758                      ; 05/01/2024 - Retro DOS 5.0
37759                      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A7Fh
37760                      ;;;
37761 chk_set_first_access:
37762 000065AD 26837E0F00      ;cmp     word [es:bp+0Fh],0
37763 000065B2 750D          cmp     word [es:bp+DPB.FAT_SIZE],0
37764                      jnz     short chk_set_fa_1 ; FAT (FAT12 or FAT16)
37765                      ; FAT32
37766 000065B4 26837E21FF      ;cmp     word [es:bp+21h],0FFFFh
37767                      cmp     word [es:bp+DPB.FREE_CNT_HW],0FFFFh ; -1
37768                      ;mov     word [es:bp+21h],0FFFFh ; High word of free cluster count
37769 000065B8 26C74621FFFF      mov     word [es:bp+DPB.FREE_CNT_HW],0FFFFh ; -1
37770 000065BF 7505          jne     short chk_set_fa_2
37771 chk_set_fa_1:
37772 000065C1 26837E1FFF      ;cmp     word [es:bp+1Fh],0FFFFh
37773                      cmp     word [es:bp+DPB.FREE_CNT],0FFFFh ; -1
37774 chk_set_fa_2:
37775                      ;mov     word [es:bp+1Fh],0FFFFh ; Count of free clusters, -1 if unknown
37776 000065C6 26C7461FFFFF      mov     word [es:bp+DPB.FREE_CNT],0FFFFh ; -1
37777 000065CC 7405          je      short chk_set_fa_3
37778                      ;or     byte [es:bp+18h],1
37779 000065CE 26804E1801      or     byte [es:bp+DPB.FIRST_ACCESS],1
37780 000065D3 C3          chk_set_fa_3:
37781                      retn
37782                      ;;;
37783                      ;-----
37784
37785                      ;Break <Fat_Operation - miscellaneous fat stuff>
37786                      ;-----
37787                      ;
37788                      ; Procedure Name : FAT_operation
37789                      ;-----
37790                      ;
37791                      ;
37792                      ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37793                      ;
37794                      ; 04/03/2024
37795                      ; 03/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37796                      ;
37797 FAT_operation:
37798                      ; 31/07/2018 - Retro DOS v3.0
37799 FATERR:
37800                      ;
37801                      ; 03/03/2024
37802 %if 0
37803                      ;mov     word [es:bp+1Eh],-1
37804                      ;mov     word [es:bp+1Fh],-1 ; MSDOS 6.0
37805                      MOV     word [ES:BP+DPB.FREE_CNT],-1
37806                      ; Err in FAT must force recomp of freespace

```

```

37807 %else
37808 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37809 ;;;
37810 000065D4 E8D6FF call    chk_set_first_access
37811 ;;;
37812 %endif
37813 ;and    di,0FFh
37814 000065D7 81E7FF00 AND     DI,STECODE ; Put error code in DI
37815 ;mov     byte [ALLOWED],18h
37816 000065DB C606[4B03]18 MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
37817 ;mov     ah,1Ah
37818 000065E0 B41A MOV     AH,2+Allowed_FAIL+Allowed_RETRY ; while trying to read FAT
37819 000065E2 A0[7605] MOV     AL,[THISDRV] ; Tell which drive
37820 000065E5 E8F2FA call    FATAL1
37821 000065E8 C42E[8A05] LES     BP,[THISDPB]
37822 000065EC 3C03 CMP     AL,3
37823 000065EE 7502 JNZ     short FAT_GOT_DPB ; User said retry
37824
37825 FATERR_fail: ; 03/03/2024
37826 000065F0 F9 STC ; User said FAIL
37827 000065F1 C3 retn
37828
37829 FAT_GOT_DPB:
37830 ;Context DS ;hkn; SS is DOSDATA
37831 000065F2 16 push    ss
37832 000065F3 1F pop     ds
37833
37834 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37835 ;;;
37836 000065F4 8CC0 mov     ax,es
37837 000065F6 09E8 or      ax,bp
37838 000065F8 74F6 jz      short FATERR_fail
37839 ;;;
37840
37841 ;;mov     al,0Fh
37842 ;MOV     AL,DMEDHL
37843 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37844 ;;;
37845 000065FA B013 mov     al,19
37846 ;;;
37847
37848 ;mov     ah,[es:bp+1]
37849 000065FC 268A6601 MOV     AH,[ES:BP+DPB.UNIT]
37850 00006600 A3[5A03] MOV     [DEVCALL_REQLEN],AX ; 09/09/2018
37851 00006603 C606[5C03]01 MOV     BYTE [DEVCALL_REQFUNC],DEVMDCH
37852 00006608 C706[5D03]0000 MOV     word [DEVCALL_REQSTAT],0
37853 ;;mov     al,[es:bp+16h]
37854 ;;mov     al,[es:bp+17h] ; MSDOS 6.0
37855 0000660E 268A4617 MOV     AL,[ES:BP+DPB.MEDIA]
37856 00006612 A2[6703] MOV     [CALLMED],AL
37857 00006615 06 PUSH    ES
37858 00006616 1E PUSH    DS
37859
37860 ;hkn; DEVCALL is in DOSDATA
37861 00006617 BB[5A03] MOV     BX,DEVCALL
37862 ;;lds     si,[es:bp+12h]
37863 ;;lds     si,[es:bp+13h] ; MSDOS 6.0
37864 0000661A 26C57613 LDS     SI,[ES:BP+DPB.DRIVER_ADDR] ; DS:SI Points to device header
37865 0000661E 07 POP     ES ; ES:BX Points to call header
37866 0000661F E86FEC call    DEVIOCALL2
37867 ;Context DS ;hkn; SS is DOSDATA
37868 00006622 16 push    ss
37869 00006623 1F pop     ds
37870 00006624 07 POP     ES ; Restore ES:BP
37871 00006625 8B3E[5D03] MOV     DI,[DEVCALL_REQSTAT]
37872 ;test     di,8000h
37873 ;jnz     short FATERR
37874 00006629 09FF or      di,di
37875 0000662B 78A7 js      short FATERR ; have error
37876
37877 ; 03/03/2024
37878 %if 0
37879 XOR     AH,AH
37880 ;xchg     ah,[es:bp+17h] ; MSDOS 3.3
37881 ;xchg     ah,[es:bp+18h] ; MSDOS 6.0
37882 XCHG     AH,[ES:BP+DPB.FIRST_ACCESS] ; Reset dpb_first_access
37883 %else
37884 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37885 ;;;
37886 ;mov     ah,[es:bp+18h]
37887 0000662D 268A6618 mov     ah,[es:bp+DPB.FIRST_ACCESS]
37888 00006631 80E480 and     ah,80h ; isolate (FAT) first access bit
37889 ;and     byte [es:bp+18h],7Fh
37890 00006634 268066187F and     byte [es:bp+DPB.FIRST_ACCESS],7Fh
37891 ; ; clear first access (FAT) bit 7
37892 ;;;
37893 %endif
37894
37895 00006639 A0[7605] MOV     AL,[THISDRV] ; Use physical unit number
37896 ; See if we had changed volume id by creating one on the diskette
37897 0000663C 3806[A10A] cmp     [VOLCHNG_FLAG],AL
37898 00006640 7508 jnz     short CHECK_BYT
37899 00006642 C606[A10A]FF mov     byte [VOLCHNG_FLAG],-1 ; 0FFh
37900 00006647 E9B000 jmp     GOGETBPB ; Need to get device driver to read in
37901 ; ; new volume label.
37902
37903 0000664A 0A26[6803] CHECK_BYT:
37904 OR      AH,[CALLRBYT]
37905 ;JNS     short CHECK_ZR ; ns = 0 or 1
37906 ;JMP     short NEWDSK
37907 ; 17/12/2022
37908 js      short NEWDSK
37909 ; 27/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
37910 ;JNS     short CHECK_ZR ; ns = 0 or 1
37911 ;JMP     short NEWDSK
37912
37913 00006650 743B CHECK_ZR:
37914 JZ      short CHKBUFFDIRT ; jump if I don't know
37915 ; 24/09/2023
37916 ; cf=0 (after 'or' instruction)
37917 00006652 C3 CLC
37918 retn ; If Media not changed (NZ)
37919
37920 00006653 06 DISK_CHNG_ERR:
37921 00006654 55 PUSH    ES
37922 ;;les     bp,[es:bp+12h]
37923 ;;les     bp,[es:bp+13h] ; MSDOS 6.0
37924 00006655 26C46E13 LES     BP,[ES:BP+DPB.DRIVER_ADDR] ; Get device pointer
37925 ;;test     word [es:bp+4],800h
37926 ;TEST     word [ES:BP+SYSDEV.ATT],DEVOPCL ; Did it set vol id?
37927 00006659 26F6460508 test     byte [es:bp+SYSDEV.ATT+1],(DEVOPCL>>8)
37928 0000665E 5D POP     BP
37929 0000665F 07 POP     ES
37930 ;JZ      short FAIL_OPJ2 ; Nope, FAIL

```

```

37931          ; 03/03/2024
37932 00006660 7477      jz     short FAIL_OP
37933 00006662 1E          PUSH    DS          ; Save buffer pointer for ignore
37934 00006663 57          PUSH    DI
37935 00006664 16          push    ss          ;hkn; SS is DOSDATA
37936 00006665 1F          pop     ds
37937          ;mov     byte [ALLOWED],18h
37938 00006666 C606[4B03]18 MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
37939 0000666B 06          PUSH    ES
37940 0000666C C43E[6903]  LES     DI,[CALLVIDM]          ; Get volume ID pointer
37941 00006670 8C06[2A03]  MOV     [EXERRPT+2],ES
37942 00006674 07          POP     ES
37943 00006675 893E[2803]  MOV     [EXERRPT],DI
37944          ;mov     ax,0Fh
37945 00006679 B80F00    MOV     AX,error_I24_wrong_disk
37946 0000667C C606[7505]01 MOV     byte [READOP],1          ; Write
37947          ;invoke HARDERR
37948 00006681 E8CBF9    call    HARDERR
37949 00006684 5F          POP     DI          ; Get back buffer for ignore
37950 00006685 1F          POP     DS
37951 00006686 3C03      CMP     AL,3
37952 FAIL_OPJ2:
37953 00006688 744F      JZ     short FAIL_OP
37954 0000668A E965FF    JMP     FAT_GOT_DPB          ; Retry
37955
37956 CHKBUFFDIRT:
37957          ; 20/05/2019 - Retro DOS v4.0
37958
37959          ; MSDOS 3.3
37960          ;lds     di,[BUFFHEAD]
37961
37962          ; MSDOS 6.0
37963          ;cmp     word [ss:DirtyBufferCount],0 ; any dirty buffers ? ;hkn;
37964          ; 03/03/2024 - Retro DOS v5.0
37965          ; ds=ss
37966          ;;;
37967 0000668D 833E[7100]00  cmp     word [DirtyBufferCount],0 ; (win ME IO.SYS - BIOSCODE:0C7A7h)
37968          ;;;
37969 00006692 741A      je     short NEWDSK          ; no, skip the check
37970 00006694 E81D02    call    GETCURHEAD          ; get pointer to first buffer
37971 nbuffer:
37972          ;cmp     al,[di+4]
37973 00006697 384504    cmp     [di+BUFFINFO.buf_ID],al          ; Unit OK ?
37974 0000669A 7509      jne     short lfnxt          ; no, go for next buffer
37975          ;test     byte [di+5],40h
37976 0000669C F6450540  TEST    byte [di+BUFFINFO.buf_flags],buf_dirty          ; is the buffer dirty ?
37977 000066A0 7403      jz     short lfnxt          ; no, go for next buffer
37978
37979 FAIL_OP2: ; 03/03/2024
37980          ;Context DS
37981 000066A2 16          push    ss
37982 000066A3 1F          pop     ds
37983          ; 24/09/2023
37984          ; cf=0 (after 'test' instruction)
37985          ;clc
37986 000066A4 C3          retn
37987
37988 lfnxt:
37989          ; 15/08/2018 - Retro DOS v3.0
37990          ; MSDOS 3.3
37991          ;lds     di,[di]
37992
37993          ; 20/05/2019 - Retro DOS v4.0
37994 000066A5 8B3D      mov     di,[di]
37995          ;mov     di,[di+BUFFINFO.buf_next]          ; get next buffer
37996
37997          ; MSDOS 3.3
37998          ;cmp     di,-1
37999          ;jne     short nbuffer
38000
38001          ; MSDOS 6.0
38002 000066A7 36393E[7E11]  cmp     [ss:FIRST_BUFF_ADDR],di          ; is this where we started ?;hkn;
38003 000066AC 75E9      jne     short nbuffer          ; no, check this guy also
38004
38005          ; If no dirty buffers, assume Media changed
38006 NEWDSK:
38007          ;mov     word [es:bp+1Eh],0FFFFh ; MSDOS 3.3
38008          ;mov     word [es:bp+1Fh],0FFFFh ; MSDOS 6.0
38009 000066AE 26C7461FFFFFFF  mov     word [ES:BP+DPB.FREE_CNT],-1 ; Media changed, must
38010          ; recompute
38011          ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
38012          ;;;
38013          ;cmp     word [es:bp+0Fh],0
38014 000066B4 26837E0F00  cmp     word [es:bp+DPB.FAT_SIZE],0 ; = 0 for FAT32 fs
38015 000066B9 7506      jnz     short newdsk2          ; not FAT32
38016          ;mov     word [es:bp+21h],0FFFFh
38017 000066BB 26C74621FFFF  mov     word [ES:BP+DPB.FREE_CNT_HW],0FFFFh ; -1
38018 newdsk2:
38019          ;;;
38020
38021          ; MSDOS 3.3
38022          ;call    SETVISIT
38023          ; MSDOS 6.0
38024 000066C1 E8F001    call    GETCURHEAD
38025 nxbuffer:
38026          ; MSDOS 3.3
38027          ;or     byte [di+5],20h
38028          ; MSDOS 3.3 & MSDOS 6.0
38029          ;cmp     [di+4],al
38030 000066C4 384504    cmp     [DI+BUFFINFO.buf_ID],al          ; This drive ?
38031 000066C7 7513      jne     short lfnxt2
38032          ;test     byte [di+5],40h
38033 000066C9 F6450540  TEST    byte [DI+BUFFINFO.buf_flags],buf_dirty
38034 000066CD 7584      jnz     short DISK_CHNG_ERR
38035          ;mov     word [di+4],20FFh
38036 000066CF C74504FF20  mov     word [DI+BUFFINFO.buf_ID],(buf_visit*256)+0FFh ; free up
38037 000066D4 E8EF01    call    SCANPLACE
38038          ; MSDOS 6.0
38039 000066D7 EB05      jmp     short skpbuff
38040
38041          ; 03/03/2024 - Retro DOS v5.0
38042 FAIL_OP:          ; This label & code is here
38043          ;Context DS          ; for reachability
38044          ;push    ss
38045          ;pop     ds
38046 000066D9 F9          STC
38047          ; 03/03/2024
38048          ;retn
38049 FAIL_OP2J: ; 03/03/2024
38050 000066DA EBC6      jmp     short FAIL_OP2 ; cf=1
38051
38052 lfnxt2:
38053 000066DC 8B3D      mov     di,[di]
38054          ;mov     di,[di+BUFFINFO.buf_next]

```

```

38055 skpbuff:
38056 ; MSDOS 6.0
38057 cmp di,[ss:FIRST_BUFF_ADDR] ;hkn;
38058 jne short nxbuffer
38059
38060 cmp word [ss:SC_CACHE_COUNT],0 ;LB. look ahead buffers ? ;AN001;
38061 jz short GOETBPB ;LB. no ;AN001;
38062 cmp al,[ss:CurSC_DRIVE] ;LB. same as changed drive ;AN001;
38063 jnz short GOETBPB ;LB. no ;AN001;
38064 mov byte [ss:CurSC_DRIVE],-1 ;LB. invalidate look ahead buffers ;AN000;
38065 ;lfnext2:
38066 ; MSDOS 3.3
38067 call SKIPVISIT
38068 jnz short nxbuffer
38069 GOETBPB:
38070 ; MSDOS 3.3 & MSDOS 6.0
38071 ;lds di,[es:bp+12h]
38072 ;lds di,[es:bp+13h] ; MSDOS 6.0
38073 lds di,[es:BP+DPB.DRIVER_ADDR]
38074 ; 20/05/2019
38075 ;test word [di+4],2000h
38076 ;TEST word [DI+SYSDEV.ATT],ISFATBYDEV
38077 000066FE F6450520 TEST byte [DI+SYSDEV.ATT+1],(ISFATBYDEV>>8)
38078 00006702 7533 JNZ short GETFREEBUF
38079 ;context DS ;hkn; SS is DOSDATA
38080 push ss
38081 00006705 1F pop ds
38082
38083 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
38084 ;;;
38085 ;mov word [es:bp+2],200h
38086 00006706 26C746020002 mov word [es:bp+DPB.SECTOR_SIZE],512 ; bytes per sector
38087 ;mov word [es:bp+6],1
38088 0000670C 26C746060100 mov word [es:bp+DPB.FIRST_FAT],1 ; starting sector of FATS
38089 ;mov byte [es:bp+8],1
38090 00006712 26C6460801 mov byte [es:bp+DPB.FAT_COUNT],1 ; number of FATS
38091 ;mov word [es:bp+0Dh],3
38092 00006717 26C7460D0300 mov word [es:bp+DPB.MAX_CLUSTER],3 ; cluster count + 1
38093 ;mov word [es:bp+0Fh],1
38094 0000671D 26C7460F0100 mov word [es:bp+DPB.FAT_SIZE],1 ; FAT sectors (16 bit)
38095 00006723 C706[E80A]0000 mov word [CLUSTNUM_HW],0 ; high word of cluster number (for UNPACK)
38096 ;;;
38097
38098 MOV BX,2
38099 0000672C E8ADFB CALL UNPACK ; Read the first FAT sector into CURBUF
38100 FAIL_OPJ:
38101 ;JC short FAIL_OP
38102 ; 03/03/2024
38103 0000672F 72A9 JC short FAIL_OP2J ; cf=1
38104 00006731 C53E[E205] LDS DI,[CURBUF]
38105 00006735 EB22 JMP SHORT GOTGETBUF
38106
38107 GETFREEBUF:
38108 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
38109 ;;;
38110 xor dx,dx ; 0 ; *
38111 00006739 36C53E[7A00] lds di,[ss:LowMemBuff]
38112 ;sub di,24
38113 0000673E 83EF18 sub di,BUFINSIZ
38114 00006741 363816[7900] cmp [ss:BuffInHMA],di ; 0
38115 00006746 7511 jnz short GOTGETBUF ; buffer is in HMA
38116 ; use [LowMemBuff] to transfer at first
38117 ;;;
38118
38119 PUSH ES ; Get a free buffer for BIOS to use
38120 00006749 55 PUSH BP
38121 ; MSDOS 3.3
38122 ;LDS DI,[ss:BUFFHEAD] ; 15/08/2018
38123 ; 03/03/2024 ; *
38124 ; MSDOS 6.0
38125 ;XOR DX,DX ; * ;LB. fake to get 1st ;AN000;
38126 ;hkn; SS override
38127 0000674A 368916[0706] MOV [ss:HIGH_SECTOR],DX ; 0 ;LB. buffer addr ;AN000;
38128 0000674F E86201 call GETCURHEAD ;LB. ;AN000;
38129 ; MSDOS 3.3 & MSDOS 6.0
38130 00006752 E8C203 call BUFWRITE
38131 00006755 5D POP BP
38132 00006756 07 POP ES
38133 ;JC short FAIL_OPJ
38134 ;JC short FAIL_OP
38135 ; 03/03/2024 - Retro DOS v5.0
38136 00006757 7281 JC short FAIL_OP2J ; cf=1
38137
38138 GOTGETBUF:
38139 ;;;add di,16
38140 ;;;add di,20 ; MSDOS 6.0
38141 ;add di,24 ; PC DOS 7.1 ; 03/03/2024
38142 00006759 83C718 ADD DI,BUFINSIZ
38143
38144 ;hkn; SS override
38145 0000675C 368C1E[6A03] MOV [ss:CALLXAD+2],DS
38146 ;Context DS ;hkn; SS is DOSDATA
38147 00006761 16 push ss
38148 00006762 1F pop ds
38149 00006763 893E[6803] MOV [CALLXAD],DI
38150 ;mov al,16h
38151 00006767 B016 MOV AL,DBPBHL ; 22
38152 ;mov ah,[es:bp+1]
38153 00006769 268A6601 MOV AH,[es:BP+DPB.UNIT]
38154 0000676D A3[5A03] MOV [DEVCALL_REQLEN],AX ; 09/09/2018
38155 00006770 C606[5C03]02 MOV BYTE [DEVCALL_REQFUNC],DEVBPB ; 2
38156 00006775 C706[5D03]0000 MOV word [DEVCALL_REQSTAT],0
38157 ;mov al,[es:bp+16h]
38158 ;mov al,[es:bp+17h]
38159 0000677B 268A4617 MOV AL,[es:BP+DPB.MEDIA]
38160 0000677F A2[6703] MOV [CALLMED],AL
38161 00006782 06 PUSH ES
38162 00006783 1E PUSH DS
38163
38164 ; 04/03/2024
38165 %if 0
38166 ;push word [es:bp+14h]
38167 ;push word [es:bp+15h] ; MSDOS 6.0
38168 PUSH WORD [es:BP+DPB.DRIVER_ADDR+2]
38169 ;push word [es:bp+12h]
38170 ;push word [es:bp+13h] ; MSDOS 6.0
38171 PUSH WORD [es:BP+DPB.DRIVER_ADDR]
38172
38173 ;hkn; DEVCALL is in DOSDATA
38174 MOV BX,DEVCALL
38175 POP SI
38176 POP DS ; DS:SI Points to device header
38177 %else
38178 ; 04/03/2024 - Retro DOS v5.0

```

```

38179          ; PCDOS 7.1 IBMDOS.COM
38180 00006784 BB[5A03]      mov     bx,DEVCALL
38181          ;lds     si,[es:bp+13h]
38182 00006787 26C57613      lds     si,[es:bp+DPB.DRIVER_ADDR] ; DS:SI Points to device header
38183          %endif
38184
38185 0000678B 07             POP     ES                ; ES:BX Points to call header
38186          ;invoke DEVIOCALL2
38187 0000678C E802EB        call    DEVIOCALL2
38188 0000678F 07             POP     ES                ; Restore ES:BP
38189          ;Context DS
38190 00006790 16             push    ss                ;hkn; SS is DOSDATA
38191 00006791 1F             pop     ds
38192 00006792 8B3E[5D03]     MOV     DI,[DEVCALL_REQSTAT]
38193
38194          ; 04/03/2024
38195          %if 0
38196          ; MSDOS 3.3
38197          ;test     di,8000h
38198          ;jnz     short FATERRJ
38199          ; MSDOS 6.0
38200          or      di,di
38201          js      short FATERRJ          ; have error
38202
38203          ; 04/03/2024
38204          ;;mov     al,[es:bp+16h]
38205          ;;mov     al,[es:bp+17h] ; MSDOS 6.0
38206          ;MOV     AL,[ES:BP+DPB.MEDIA]
38207
38208          LDS     SI,[CALLBPB]
38209          ;;mov     word [es:bp+1ch],0
38210          ;mov     word [es:bp+1dh],0 ; MSDOS 6.0
38211          MOV     word [ES:BP+DPB.NEXT_FREE],0 ; recycle scanning pointer
38212
38213          ;invoke $SETDPB
38214          call    _$SETDPB
38215
38216          ;hkn; SS override
38217          LDS     DI,[SS:CALLXAD]          ; Get back buffer pointer
38218
38219          ;mov     al,[es:bp+8]
38220          MOV     AL,[ES:BP+DPB.FAT_COUNT]
38221
38222          ; MSDOS 3.3
38223          ;;mov     ah,[es:bp+0Fh]
38224          ;MOV     AH,[ES:BP+DPB.FAT_SIZE]
38225          ;;mov     [DI-8],ax
38226          ;MOV     [DI+BUFFINFO.buf_wrtcnt-BUFINSIZ],AX
38227
38228          ; MSDOS 6.0
38229          ;mov     [di-0Ah],al
38230          MOV     [DI+BUFFINFO.buf_wrtcnt-BUFINSIZ],AL
38231                                     ;>32mb          ;AN000;
38232          ;mov     ax,[es:bp+0Fh]
38233          MOV     AX,[ES:BP+DPB.FAT_SIZE]          ;>32mb
38234          ;mov     [di-09h],ax          ;AC000;
38235          MOV     [DI+BUFFINFO.buf_wrtcntinc-BUFINSIZ],AX
38236                                     ;>32mb Correct buffer info ;AC000;
38237          ;Context DS          ;hkn; SS is DOSDATA
38238          push    ss
38239          pop     ds
38240          XOR     AL,AL          ;Media changed (Z), Carry clear
38241          retn
38242
38243          FATERRJ:
38244          JMP     FATERR
38245
38246          %else
38247          ; 04/03/2024 - Retro DOS v5.0
38248          ; PCDOS 7.1 IBMDOS.COM
38249          ;;;
38250          or      di,di
38251          jns     short gotgetbuf2
38252          ; error
38253          lds     di,[CALLXAD] ; Buffer (data) address
38254          ;mov     word [di-20],0FFh
38255          mov     word [di+BUFFINFO.buf_ID-BUFINSIZ],0FFh
38256                                     ; byte BUFFINFO.buf_ID = 0FFh ; FREE
38257                                     ; byte BUFFINFO.buf_flags = 0
38258          jmp     FATERR
38259
38260          gotgetbuf2:
38261          lds     si,[CALLBPB] ; Address of the BPB (DEVCALL offset 18)
38262
38263          xor     cx,cx ; 0
38264          ;mov     [es:bp+1dh],cx ; [ES:BP+DPB.NEXT_FREE] = 0
38265          ; 01/07/2024
38266          mov     [es:bp+DPB.NEXT_FREE],cx ; 0
38267                                     ; recycle scanning pointer
38268
38269          mov     dx,4152h          ; 'RA' ; FAT32 extended BPB/DPB signature
38270
38271          ;cmp     [si+0Bh],cx ; BPB.fatsecs ; 16 bit FAT size = 0 for FAT32 fs
38272          cmp     [si+A_BPB.SECTORSPERFAT],cx ; 0
38273          jnz     short gotgetbuf3 ; not FAT32
38274
38275          ;mov     [es:bp+39h],cx
38276          mov     [es:bp+DPB.FAT32_NXTFREE],cx ; 0
38277          ;mov     [es:bp+3Bh],cx
38278          mov     [es:bp+DPB.FAT32_NXTFREE+2],cx ; 0
38279          dec     cx ; -1
38280          ;mov     [es:bp+1Fh],cx ; (DPB.FREE_CNT) set free count to -1 (unknown)
38281          mov     [es:bp+DPB.FREE_CNT],cx ; 0FFFFh
38282          ;mov     [es:bp+21h],cx
38283          mov     [es:bp+DPB.FREE_CNT_HW],cx ; 0FFFFh
38284          ;
38285          mov     cx,4558h          ; 'XE' ; FAT32 extended BPB/DPB signature
38286          gotgetbuf3:
38287
38288          ;invoke $SETDPB
38289          call    _$SETDPB
38290
38291          ;hkn; SS override
38292          LDS     DI,[SS:CALLXAD]          ; Get back buffer pointer
38293
38294          ;mov     dx,[es:bp+25h]
38295          mov     dx,[es:bp+DPB.FSINFO_SECTOR]
38296          xor     cx,cx ; 0
38297          ;cmp     [es:bp+0Fh],cx
38298          cmp     [es:bp+DPB.FAT_SIZE],cx          ; 16 bit FAT size field
38299          jz      short gotgetbuf4 ; FAT32 fs
38300          jmp     gotgetbuf12 ; FAT fs
38301
38302          gotgetbuf4:

```

```

38303 000067E3 83FAFF      cmp     dx,0FFFFh          ; invalid ?
38304 000067E6 7503        jnz     short gotgetbuf5    ; no
38305 000067E8 E98C00      jmp     gotgetbuf12         ; skip reading FSINFO sector
38306
38307
38308 000067EB 36890E[0706]    gotgetbuf5:
38309 000067F0 89FB          mov     [ss:HIGH_SECTOR],cx ; 0
38310                                mov     bx,di
38311                                ;cmp     byte [ss:BuffInHMA],0
38312 000067F2 36380E[7900]    cmp     [ss:BuffInHMA],cl ; 0
38313 000067F7 7405        jz      short gotgetbuf6    ; buffer is in conventional (<=640KB) memory
38314 000067F9 36C51E[7A00]    lds     bx,[ss:LoMemBuff]   ; use a buffer in conventional memory
38315                                ;mov     di,bx
38316
38317 000067FE 36C606[4B03]18    gotgetbuf6:
38318 00006804 41          mov     byte [ss:ALLOWED],Allowed_FAIL+Allowed_RETRY ; 18h
38319                                inc     cx
38320 00006805 53          ;push  di
38321 00006806 E8E1D7    push    bx
38322 00006809 5B          call    DREAD
38323                                pop     bx
38324 0000680A 89DF          ;pop  di
38325 0000680C 725F          mov     di,bx ; Retro DOS v5.0
38326                                jc      short gotgetbuf11    ; ds:di = (FSINFO sector) buffer
38327                                ; FSI_HeadSig = 41615252h
38328
38329 0000680E 813D5252      cmp     word [di],5252h      ; 'RR' ; check if it is a valid FSINFO sector
38330                                ;cmp     word [di+FSINFO.LeadSig],5252
38331 00006812 7559        jnz     short gotgetbuf11    ; not valid
38332                                ;cmp     word [di+2],4161h      ; 'AA' ; (NASM syntax)
38333 00006814 817D026141    cmp     word [di+FSINFO.LeadSig+2],4161h
38334 00006819 7552        jnz     short gotgetbuf11    ; not valid
38335                                ;cmp     word [di+484],7272h      ; 'rr' ; FSI_StrucSig = 61417272h
38336 0000681B 81BDE4017272    cmp     word [di+FSINFO.StrucSig],7272h
38337 00006821 754A        jnz     short gotgetbuf11    ; not valid
38338                                ;cmp     word [di+486],6141h      ; 'Aa'
38339 00006823 81BDE6014161    cmp     word [di+FSINFO.StrucSig+2],6141h
38340 00006829 7542        jnz     short gotgetbuf11    ; not valid
38341                                ; valid
38342                                push    dx
38343 0000682B 52          push    bx
38344 0000682C 53          push    cx
38345 0000682D 51
38346
38347                                ;mov     bx,[es:bp+2Fh]
38348 0000682E 268B5E2F    mov     bx,[es:bp+DPB.LAST_CLUSTER+2]
38349                                ;mov     cx,[es:bp+2Dh]
38350 00006832 268B4E2D    mov     cx,[es:bp+DPB.LAST_CLUSTER]
38351
38352                                ;mov     ax,[di+488]      ; FSI_FreeCount ; bx:cx = number of clusters + 1
38353 00006836 8B85E801    mov     ax,[di+FSINFO.Free_Count]
38354                                ;mov     dx,[di+490]      ; FSI_FreeCount+2
38355 0000683A 8B95EA01    mov     dx,[di+FSINFO.Free_Count+2]
38356
38357 0000683E 39DA        cmp     dx,bx              ; is Free Count >= (Number of Clusters + 1) ?
38358 00006840 7502        jne     short gotgetbuf7    ; if yes, it is invalid value (must be 0FFFFFFFh)
38359 00006842 39C8        cmp     ax,cx
38360
38361 00006844 7308        gotgetbuf7:
38362                                jnb     short gotgetbuf8      ; yes, invalid value (must be 0FFFFFFFh)
38363                                ;mov     [es:bp+1Fh],ax      ; no,valid free count
38364 00006846 2689461F    mov     [es:bp+DPB.FREE_CNT],ax ; save free count into [es:bp+DPB.FREE_CNT]
38365                                ;mov     [es:bp+21h],dx
38366 0000684A 26895621    mov     [es:bp+DPB.FREE_CNT+2],dx
38367
38368
38369
38370 0000684E 8B85EC01    gotgetbuf8:
38371                                ;mov     ax,[di+492]      ; FSI_Nxt_Free
38372 00006852 8B95EE01    mov     ax,[di+FSINFO.Nxt_Free]
38373                                ;mov     dx,[di+494]
38374                                mov     dx,[di+FSINFO.Nxt_Free+2]
38375
38376 00006856 39DA        cmp     dx,bx              ; is the next free clust num >= (num of clusters + 1) ?
38377 00006858 7502        jne     short gotgetbuf9    ; invalid (if dx > bx)
38378 0000685A 39C8        cmp     ax,cx
38379
38380 0000685C 730C        gotgetbuf9:
38381                                jnb     short gotgetbuf10    ; invalid
38382                                ;mov     [es:bp+39h],ax      ; save next free (search) cluster number
38383 0000685E 26894639    mov     [es:bp+DPB.FAT32_NXTFREE],ax ; into [es:bp+DPB.FAT32_NXTFREE]
38384                                ;mov     [es:bp+3Bh],dx
38385 00006862 2689563B    mov     [es:bp+DPB.FAT32_NXTFREE+2],dx
38386                                ;mov     [es:bp+1Dh],ax      ; and into [es:bp+DPB.NEXT_FREE] ; low word
38387 00006866 2689461D    mov     [es:bp+DPB.NEXT_FREE],ax
38388
38389 0000686A 59          gotgetbuf10:
38390 0000686B 5B          pop     cx
38391 0000686C 5A          pop     bx
38392                                pop     dx
38393
38394 0000686D 36803E[7900]00    gotgetbuf11:
38395                                cmp     byte [ss:BuffInHMA],0 ; is buffer in HMA ?
38396                                ;jnz     short gotgetbuf12    ; yes
38397 00006873 753A        jnz     short gotgetbuf16    ; 04/03/2024
38398                                ;mov     word [di-20],0FFh      ; invalidate buffer (set it as free buffer)
38399 00006875 EB0F          ; 04/03/2024 - Retro DOS v5.0
38400                                jmp     short gotgetbuf17
38401
38402 00006877 36803E[7900]00    gotgetbuf12:
38403 0000687D 7530        cmp     byte [ss:BuffInHMA],0
38404                                jnz     short gotgetbuf16
38405                                ;cmp     word [es:bp+6],1
38406 0000687F 26837E0601    cmp     word [es:bp+DPB.FIRST_FAT],1 ; 1st FAT start sector
38407 00006884 7405        jz      short gotgetbuf13
38408
38409
38410 00006886 C745ECFF00    gotgetbuf17:
38411                                ;mov     word [di-20],0FFh      ; invalidate buffer (set it as free buffer)
38412                                ; di = buffer header + 4 (BUFINFO.buf_ID)
38413                                mov     word [di+BUFINFO.buf_ID-BUFINSIZ],0FFh
38414
38415 00006888 B845F2        gotgetbuf13:
38416                                ;mov     al,[es:bp+8]
38417 0000688B 268A4608    mov     al,[es:bp+DPB.FAT_COUNT]
38418                                ;mov     [di-14],al      ; buffer header address + 10
38419 0000688F 8845F2        mov     [di+BUFINFO.buf_wrtcnt-BUFINSIZ],al
38420                                ;mov     ax,[es:bp+0Fh]
38421 00006892 268B460F    mov     ax,[es:bp+DPB.FAT_SIZE]      ; 16 bit FAT size field
38422 00006896 09C0          or      ax,ax
38423 00006898 750D        jnz     short gotgetbuf14    ; FAT (FAT12 or FAT16) fs
38424
38425                                ; FAT32 fs
38426                                ;mov     ax,[es:bp+31h]      ; FAT sectors (per one FAT)

```



```

38427 0000689A 268B4631      mov     ax,[es:bp+DPB.FAT32_SIZE]
38428                        ;mov     [di-13],ax      ; BUFFINFO.buf_wrtcntinc ; # sectors between each write
38429 0000689E 8945F3      mov     [di+BUFFINFO.buf_wrtcntinc-BUFINSIZ],ax
38430                        ;mov     ax,[es:bp+33h]
38431 000068A1 268B4633      mov     ax,[es:bp+DPB.FAT32_SIZE+2]
38432 000068A5 EB05        jmp     short gotgetbuf15
38433
38434 gotgetbuf14:
38435                        ;mov     [di-13],ax      ; BUFFINFO.buf_wrtcntinc
38436 000068A7 8945F3      mov     [di+BUFFINFO.buf_wrtcntinc-BUFINSIZ],ax
38437
38438                        ;xor     ax,ax      ; 0
38439 000068AA 29C0        sub     ax,ax ; 0
38440
38441 gotgetbuf15:
38442                        ;;mov     [di-15],ax      ; BUFFINFO.buf_wrtcntinc+2 ; hw of sectors per FAT
38443                        ; PC DOS 7.1 BUG !? This would be 'mov [di-11],ax'
38444                        ; Erdogan Tan - 03/03/2024
38445                        ;mov     [di-11],ax
38446 000068AC 8945F5      mov     [di+BUFFINFO.buf_wrtcntinc+2-BUFINSIZ],ax
38447
38448 gotgetbuf16:
38449                        ;mov     ax,ss      ; SS is DOSDATA
38450                        ;mov     ds,ax
38451 000068AF 16        push    ss
38452 000068B0 1F        pop     ds
38453                        ;xor     al,al      ; Media changed (Z),Carry clear
38454 000068B1 31C0      xor     ax,ax
38455 000068B3 C3        retn
38456
38457                        ;;
38458 %endif
38459
38460 ;=====
38461 ; STDBUF.ASM
38462 ;=====
38463 ; Retro DOS v2.0 - 12/03/2018
38464
38465 ;
38466 ; Standard buffer management for MSDOS
38467 ;
38468 ;
38469 ;.xlist
38470 ;.xcref
38471 ;INCLUDE STDSW.ASM
38472 ;.cref
38473 ;.list
38474
38475 ;TITLE      STDBUF - MSDOS buffer management
38476 ;NAME       STDBUF
38477
38478 ;INCLUDE BUF.ASM
38479
38480 ;=====
38481 ; BUF.ASM
38482 ;=====
38483 ; 31/07/2018 - Retro DOS v3.0
38484 ; Retro DOS v2.0 - 12/03/2018
38485 ;
38486 ; buffer management for MSDOS
38487 ;
38488 ;CODE      SEGMENT BYTE PUBLIC 'CODE'
38489 ;          ASSUME SS:DOSGROUP,CS:DOSGROUP
38490 ;
38491 ;SUBTTL SETVISIT,SKIPVISIT -- MANAGE BUFFER SCANS
38492 ;
38493 ;SETVISIT:
38494 ; ; 31/07/2018 - Retro DOS v3.0
38495 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5CAfh
38496 ; Inputs:
38497 ;     None
38498 ; Function:
38499 ;     Set up a scan of I/O buffers
38500 ; Outputs:
38501 ;     All visit flags = 0
38502 ;     NOTE: This pre-scan is needed because a hard disk error
38503 ;           may cause a scan to stop in the middle leaving some
38504 ;           visit flags set, and some not set.
38505 ;     DS:DI Points to [BUFFHEAD]
38506 ; No other registers altered
38507 ;
38508 ;     LDS     DI,[SS:BUFFHEAD] ; 15/03/2018
38509 ;     PUSH    AX
38510 ;     ;;XOR     AX,AX      ;; MSDOS 2.11
38511 ;     ;mov     al,0DFh
38512 ;     mov     al,~buf_visit
38513 ;SETLOOP:
38514 ;     ;;MOV     [DI+7],AL ;; MSDOS 2.11
38515 ;     ;and     [DI+5],al
38516 ;     AND     [DI+BUFFINFO.buf_flags],AL
38517 ;     LDS     DI,[DI]
38518 ;     CMP     DI,-1
38519 ;     JNZ     SHORT SETLOOP
38520 ;     POP     AX ; 09/09/2018
38521 ;     LDS     DI,[SS:BUFFHEAD] ; 15/03/2018
38522 ;SVISIT_RETN:
38523 ;     RETN
38524 ;
38525 ;SKIPVISIT:
38526 ; ; 31/07/2018 - Retro DOS v3.0
38527 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5CC8h
38528 ;
38529 ; Inputs:
38530 ;     DS:DI Points to a buffer
38531 ; Function:
38532 ;     Skip visited buffers
38533 ; Outputs:
38534 ;     DS:DI Points to next unvisited buffer
38535 ;     Zero is set if skip to LAST buffer
38536 ; No other registers altered
38537 ;
38538 ;     CMP     DI,-1
38539 ;     ;retz
38540 ;     JZ     SHORT SVISIT_RETN
38541 ;
38542 ;     ;;CMP     BYTE [DI+7],1 ;; MSDOS 2.11
38543 ;     ;;;retnz
38544 ;     ;JNZ     SHORT SVISIT_RETN
38545 ;
38546 ;test     byte [di+5],20h
38547 ;TEST     byte [DI+BUFFINFO.buf_flags],buf_visit
38548 ;JNZ     short SKIPLLOOP
38549 ;
38550 ;     push    ax

```

```

38551 ; or al,1
38552 ; pop ax
38553 ; retn
38554 ;
38555 ;SKIPLoop:
38556 ; LDS DI,[DI]
38557 ; JMP SHORT SKIPVISIT
38558 ;
38559 ;=====
38560 ; BUF.ASM, MSDOS 6.0, 1991
38561 ;=====
38562 ; 31/07/2018 - Retro DOS v3.0
38563 ; 04/05/2019 - Retro DOS v4.0
38564 ; 06/03/2024 - Retro DOS v5.0
38565 ;
38566 ; TITLE BUF - MSDOS buffer management
38567 ; NAME BUF
38568 ;
38569 ;** BUF.ASM - Low level routines for buffer cache management
38570 ;
38571 ; GETCURHEAD
38572 ; ScanPlace
38573 ; PLACEBUF
38574 ; PLACEHEAD
38575 ; PointComp
38576 ; GETBUFFER
38577 ; GETBUFFRB
38578 ; FlushBuf
38579 ; BufWrite
38580 ; SET_RQ_SC_PARMS
38581 ;
38582 ; Revision history:
38583 ;
38584 ; AN000 version 4.00 Jan. 1988
38585 ; A004 PTM 3765 -- Disk reset failed
38586 ; M039 DB 10/17/90 - Disk write optimization
38587 ; I001 5.0 PTR 722211 - Preserve CY when in buffer in HMA
38588 ;
38589 ;Break <GETCURHEAD -- Get current buffer header>
38590 ;-----
38591 ; Procedure Name : GetCurHead
38592 ; Inputs:
38593 ; No Inputs
38594 ; Function:
38595 ; Returns the pointer to the first buffer in Queue
38596 ; and updates FIRST_BUFF_ADDR
38597 ; and invalidates LASTBUFFER (recency pointer)
38598 ; Outputs:
38599 ; DS:DI = pointer to the first buffer in Queue
38600 ; FIRST_BUFF_ADDR = offset ( DI ) of First buffer in Queue
38601 ; LASTBUFFER = -1
38602 ; No other registers altered
38603 ;-----
38604 ;
38605 ; 04/05/2019 - Retro DOS v4.0
38606 ; DOSCODE:98D2h (MSDOS 6.21, MSDOS.SYS)
38607 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
38608 ; DOSCODE:9876h (MSDOS 5.0, MSDOS.SYS)
38609 ;
38610 ; 06/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
38611 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0AA4Bh
38612 ;
38613 GETCURHEAD:
38614 000068B4 36C53E[6D00] lds di,[ss:BufferQueue] ; Pointer to the first buffer
38615 000068B9 36C706[1E00]FFFF mov word [ss:LastBuffer],-1 ; invalidate last buffer
38616 000068C0 36893E[7E11] mov [ss:FIRST_BUFF_ADDR],di ; save first buffer address
38617 000068C5 C3 retn
38618 ;
38619 ;Break <SCANPLACE, PLACEBUF -- PUT A BUFFER BACK IN THE POOL>
38620 ;-----
38621 ; Procedure Name : ScanPlace
38622 ; Inputs:
38623 ; Same as PLACEBUF
38624 ; Function:
38625 ; Save scan location and call PLACEBUF
38626 ; Outputs:
38627 ; DS:DI Points to saved scan location
38628 ; All registers, except DS:DI, preserved.
38629 ;-----
38630 ;M039: Rewritten to preserve registers.
38631 ;
38632 ;SCANPLACE:
38633 ; ; 31/07/2018 - Retro DOS v3.0
38634 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5DDCh
38635 ; push es
38636 ; les si,[di]
38637 ; les si,[DI+BUFFINFO.buf_link]
38638 ; call PLACEBUF
38639 ; push es
38640 ; pop ds
38641 ; mov di,si
38642 ; pop es
38643 ;scanplace_retn:
38644 ; retn
38645 ;
38646 ; MSDOS 6.0
38647 SCANPLACE:
38648 000068C6 FF35 push word [di]
38649 ;push word [di+BUFFINFO.buf_next] ;Save scan location
38650 000068C8 E80200 call PLACEBUF
38651 000068CB 5F pop di
38652 000068CC C3 retn
38653 ;-----
38654 ; Procedure Name : PlaceBuf
38655 ; Input:
38656 ; DS:DI points to buffer (DS->BUFFINFO array, DI=offset in array)
38657 ; Function:
38658 ; Remove buffer from queue and re-insert it in proper place.
38659 ; NO registers altered
38660 ;-----
38661 ;
38662 ;procedure PLACEBUF,NEAR
38663 ;
38664 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
38665 ; 20/05/2019 - Retro DOS v4.0
38666 PLACEBUF:
38667 ; 31/07/2018 - Retro DOS v3.0
38668 ;
38669 ; MSDOS 6.0
38670 000068CD 50 push AX ;Save only regs we modify ;AN000;
38671 000068CE 53 push BX ;AN000;
38672 ; 23/09/2023
38673 ;push SI ;AN000;
38674 ;

```



```

38675
38676 000068CF 8B05      mov     ax,[di]
38677      ;mov     ax,[di+BUFFINFO.buf_next]
38678 000068D1 368B1E[6D00]  mov     bx,[ss:BufferQueue] ; bx = offset of head of list;smr;SS override
38679
38680      cmp     ax,bx                ;Buf = last? ;AN000;
38681 000068D8 7422      je      short nret                ;Yes, special case ;AN000;
38682 000068DA 39DF      cmp     di,bx                ;Buf = first? ;AN000;
38683 000068DC 7506      jne     short not_first         ;Yes, special case ;AN000;
38684 000068DE 36A3[6D00]  mov     [ss:BufferQueue],ax     ;smr;SS override
38685 000068E2 EB18      jmp     short nret              ;Continue with repositioning;AN000;
38686
38687 not_first:
38688      ; 23/09/2023
38689      push    si
38690 000068E4 56      ;mov     si,[di+2]
38691 000068E5 8B7502  mov     SI,[DI+BUFFINFO.buf_prev] ;No, SI = prior Buf ;AN000;
38692 000068E8 8904      mov     [si],ax
38693 000068EA 96      ;mov     [SI+BUFFINFO.buf_next],AX ; ax has di->buf_next ;AN000;
38694      xchg    si,ax
38695 000068EB 894402  ;mov     [SI+BUFFINFO.buf_prev],AX ; ;AN000;
38696
38697 000068EE 8B7702  mov     SI,[BX+BUFFINFO.buf_prev] ;SI-> last buffer ;AN000;
38698 000068F1 893C      mov     [si],di
38699      ;mov     [SI+BUFFINFO.buf_next],DI ;Add Buf to end of list ;AN000;
38700 000068F3 897F02  mov     [BX+BUFFINFO.buf_prev],DI ;AN000;
38701 000068F6 897502  mov     [DI+BUFFINFO.buf_prev],SI ;Update link in Buf too ;AN000;
38702 000068F9 891D      mov     [di],bx
38703      ;mov     [DI+BUFFINFO.buf_next],BX ;AN000;
38704      ; 23/09/2023
38705 000068FB 5E      pop     si
38706
38707 nret:
38708      ; 23/09/2023 ;AN000;
38709      ;pop     SI ;AN000;
38710 000068FC 5B      pop     BX ;AN000;
38711 000068FD 58      pop     AX ;AN000;
38712      ;cmp     byte [di+4],0FFh
38713 000068FE 807D04FF  cmp     byte [di+BUFFINFO.buf_ID],-1 ; Buffer FREE? ;AN000;
38714 00006902 7505      jne     short pbx                ; M039: -no, jump.
38715 00006904 36893E[6D00]  mov     [ss:BufferQueue],di     ; M039: -yes, make it LRU.
38716
38717 pbx:
38718      retn ;AN000;
38719
38720      ; 31/07/2018 - Retro DOS v3.0
38721
38722      ; MSDOS 3.3
38723      ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5DDCh
38724
38725 ;PLACEBUF:
38726      ; 15/03/2018 - Retro DOS v2.0 (MSDOS 2.11)
38727
38728      CALL     save_world
38729      LES     CX,[DI]
38730      CMP     CX,-1                ; Buf is LAST?
38731      JZ      SHORT NRET           ; Buffer already last
38732      MOV     BP,ES                ; Pointsave = Buf.nextbuf
38733      PUSH    DS
38734      POP     ES                    ; Buf is ES:DI
38735      ; 15/03/2018
38736      LDS     SI,[SS:BUFFHEAD]    ; Curbuf = HEAD
38737      CALL    POINTCOMP           ; Buf == HEAD?
38738      JNZ     SHORT BUFLOOP
38739      MOV     [SS:BUFFHEAD],CX
38740      MOV     [SS:BUFFHEAD+2],BP   ; HEAD = Pointsave
38741      JMP     SHORT LOOKEND
38742
38743 ;BUFLOOP:
38744      ; 31/07/2018
38745      mov     ax,ds
38746      mov     bx,si
38747      ;lds     si,[SI+BUFFINFO.buf_link]
38748      LDS     SI,[SI]
38749      CALL    POINTCOMP
38750      jnz     short BUFLOOP
38751
38752      mov     ds,ax
38753      mov     si,bx
38754      mov     [SI],cx
38755      ;mov     [SI+BUFFINFO.buf_link],cx ; If Curbuf.nextbuf == buf
38756      mov     [SI+2],bp
38757      ;mov     [BX+BUFFINFO.buf_link+2],bp ; Curbuf.nextbuf = Pointsave
38758
38759 ;LOOKEND:
38760      mov     ax,ds
38761      mov     bx,si
38762      LDS     SI,[SI]
38763      CMP     SI,-1
38764      jnz     short LOOKEND
38765
38766 ;GOTHEEND:
38767      mov     ds,ax
38768      mov     [BX],di
38769      MOV     [BX+2],ES            ; Curbuf.nextbuf = Buf
38770      MOV     WORD [ES:DI],-1
38771      ;mov     word [ES:DI+BUFFINFO.buf_link],-1
38772      MOV     WORD [ES:DI+2],-1    ; Buf is LAST
38773      ;mov     word [ES:DI+BUFFINFO.buf_link+2],-1
38774
38775 ;NRET:
38776      CALL    restore_world
38777
38778      ;cmp     byte [di+4],-1
38779      cmp     byte [DI+BUFFINFO.buf_ID],-1 ; Free buffer ?
38780      jnz     short scanplace_retn
38781      call    PLACEHEAD
38782      retn
38783
38784 ;EndProc PLACEBUF
38785
38786 ;M039 - Removed PLACEHEAD.
38787
38788 -----
38789 ; places buffer at head
38790 ; NOTE::::: ASSUMES THAT BUFFER IS CURRENTLY THE LAST
38791 ; ONE IN THE LIST!!!!!!
38792 ; BUGBUG ---- this routine can be removed because it has only
38793 ; BUGBUG ---- one instruction. This routine is called from
38794 ; BUGBUG ---- 3 places. ( Size = 3*3+6 = 15 bytes )
38795 ; BUGBUG ---- if coded in line = 3 * 5 = 15 bytes
38796 ; BUGBUG ---- But kept as it is for modularity
38797 -----
38798 ;procedure PLACEHEAD,NEAR
38799 ; mov     word ptr [BufferQueue], di
38800 ; ret
38801 ;EndProc PLACEHEAD
38802 ;M039
38803
38804 -----

```

```

38799 ; Procedure Name : PLACEHEAD
38800 ;
38801 ; SAME AS PLACEBUF except places buffer at head
38802 ;-----
38803 ;
38804 ; MSDOS 3.3 (Retro DOS v3.0)
38805 ; 05/09/2018
38806 ; MSDOS 2.11 (Retro DOS v2.0)
38807 ; PLACEHEAD:
38808 ; 31/07/2018 - Retro DOS v3.0
38809 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5D4Ah
38810 ;
38811 ; CALL save_world
38812 ; PUSH DS
38813 ; POP ES
38814 ; 15/03/2018 - Retro DOS v2.0 (MSDOS 2.11)
38815 ; LDS SI,[SS:BUFFHEAD]
38816 ; 31/07/2018 - Retro DOS v3.0 (MSDOS 3.3)
38817 ; CALL POINTCOMP
38818 ; JZ SHORT GOTHEEND2
38819 ; MOV [ES:DI],SI
38820 ; mov [ES:DI+BUFFINFO.buf_link],si
38821 ; MOV [ES:DI+2],DS
38822 ; mov [ES:DI+BUFFINFO.buf_link+2],ds
38823 ; MOV [SS:BUFFHEAD],DI
38824 ; MOV [SS:BUFFHEAD+2],ES
38825 ; LOOKEND2:
38826 ; mov ax,ds
38827 ; mov bx,si
38828 ; lds si,[SI+BUFFINFO.buf_link]
38829 ; LDS SI,[SI]
38830 ; CALL POINTCOMP
38831 ; JNZ SHORT LOOKEND2 ; 05/09/2018
38832 ; mov ds,ax
38833 ; mov word [bx],-1
38834 ; mov word [BX+BUFFINFO.buf_link],-1
38835 ; mov word [bx+2],-1
38836 ; mov word [BX+BUFFINFO.buf_link+2],-1
38837 ; GOTHEEND2:
38838 ; call restore_world
38839 ; placehead_retn:
38840 ; ret
38841 ;
38842 ; 20/05/2019 - Retro DOS v4.0
38843 ; DOSCODE:9928h (MSDOS 6.21, MSDOS.SYS)
38844 ;
38845 ; Break <POINTCOMP -- 20 BIT POINTER COMPARE>
38846 ;-----
38847 ;
38848 ; Procedure Name : PointComp
38849 ; Inputs:
38850 ; DS:SI & ES:DI
38851 ; Function:
38852 ; Checks for ((SI==DI) && (ES==DS))
38853 ; Assumes that pointers are normalized for the
38854 ; same segment
38855 ;
38856 ; Compare DS:SI to ES:DI (or DS:DI to ES:SI) for equality
38857 ; DO NOT USE FOR < or >
38858 ; No Registers altered
38859 ;
38860 ;-----
38861 ;
38862 ; POINTCOMP:
38863 ; 31/07/2018 - Retro DOS v3.0
38864 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5D84h
38865 0000690A 39FE CMP SI,DI
38866 0000690C 750A jnz short _ret_label ; return if nz
38867 ; jnz short placehead_retn
38868 0000690E 51 PUSH CX
38869 0000690F 52 PUSH DX
38870 00006910 8CD9 MOV CX,DS
38871 00006912 8CC2 MOV DX,ES
38872 00006914 39D1 CMP CX,DX
38873 00006916 5A POP DX
38874 00006917 59 POP CX
38875 _ret_label:
38876 00006918 C3 ret
38877 ;
38878 ; 01/08/2018 - Retro DOS v3.0
38879 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5D93h
38880 ;
38881 ; Break <GETBUFFR, GETBUFFRB -- GET A SECTOR INTO A BUFFER>
38882 ;-----
38883 ;
38884 ; ** GetBuffr - Get a non-FAT Sector into a Buffer
38885 ;-----
38886 ; GetBuffr does normal ( non-FAT ) sector buffering
38887 ; It gets the specified local sector into one of the I/O buffers
38888 ; and shuffles the queue
38889 ;
38890 ; ENTRY (AL) = 0 means sector must be pre-read
38891 ; ELSE no pre-read
38892 ; (DX) = Desired physical sector number (LOW)
38893 ; HIGH_SECTOR = Desired physical sector number (HIGH)
38894 ; (ES:BP) = Pointer to drive parameters
38895 ; ALLOWED set in case of INT 24
38896 ; EXIT 'C' set if error (user FAIL response to INT24)
38897 ; 'C' clear if OK
38898 ; CURBUF Points to the Buffer for the sector
38899 ; the buffer type bits OF buf_flags = 0, caller must set it
38900 ; USES AX, BX, CX, SI, DI, Flags
38901 ;-----
38902 ;
38903 ; ** GetBuffrb - Get a FAT Sector into a Buffer
38904 ;-----
38905 ; GetBuffrb reads a sector from the FAT file system's FAT table.
38906 ; It gets the specified sector into one of the I/O buffers
38907 ; and shuffles the queue. We need a special entry point so that
38908 ; we can read the alternate FAT sector if the first read fails, also
38909 ; so we can mark the buffer as a FAT sector.
38910 ;
38911 ; ENTRY (AL) = 0 means sector must be pre-read
38912 ; ELSE no pre-read
38913 ; (DX) = Desired physical sector number (LOW)
38914 ; (SI) != 0
38915 ; HIGH_SECTOR = Desired physical sector number (HIGH)
38916 ; (ES:BP) = Pointer to drive parameters
38917 ; ALLOWED set in case of INT 24
38918 ; EXIT 'C' set if error (user FAIL response to INT24)
38919 ; 'C' clear if OK
38920 ; CUR ddBUF Points to the Buffer for the sector
38921 ; the buffer type bits OF buf_flags = 0, caller must set it
38922 ; USES AX, BX, CX, SI, DI, Flags

```

```

38923
38924
38925
38926
38927
38928 00006919 891E[0706]
38929
38930 0000691D 30C0
38931 0000691F BE0100
38932 00006922 EB09
38933
38934
38935
38936 00006924 30C0
38937
38938
38939 00006926 C606[4B03]18
38940
38941
38942
38943
38944
38945
38946
38947
38948
38949
38950
38951
38952
38953
38954 0000692B 31F6
38955
38956
38957
38958
38959 0000692D A3[9405]
38960
38961
38962
38963 00006930 09F6
38964 00006932 7429
38965
38966
38967 00006934 26837E0F00
38968 00006939 7522
38969
38970
38971 0000693B 268B4623
38972 0000693F A880
38973 00006941 741A
38974 00006943 83E00F
38975 00006946 7415
38976
38977 00006948 52
38978 00006949 89C1
38979
38980
38981 0000694B 26F76633
38982 0000694F 91
38983
38984 00006950 26F76631
38985 00006954 01D1
38986 00006956 5A
38987 00006957 01C2
38988 00006959 110E[0706]
38989
38990
38991
38992
38993 0000695D 268A4600
38994
38995
38996 00006961 C53E[1E00]
38997
38998
38999
39000 00006965 BBFFFF
39001
39002
39003
39004
39005 00006968 368B0E[0706]
39006
39007
39008
39009
39010
39011
39012 0000696D 39DF
39013 0000696F 7412
39014
39015
39016 00006971 3B5506
39017 00006974 750D
39018
39019
39020
39021 00006976 3B4D08
39022 00006979 7508
39023
39024
39025 0000697B 3A4504
39026
39027 0000697E 7503
39028
39029
39030
39031 00006980 E90301
39032
39033
39034
39035
39036
39037
39038
39039
39040
39041
39042
39043
39044
39045
39046

; 22/09/2023 - RetroDOS v4.2 MSDOS.SYS (optimization)
GETBUFFRC:
;mov word [HIGH_SECTOR],0
; 02/03/2024 - Retro DOS v5.0
mov [HIGH_SECTOR],bx
GETBUFFRA:
xor al,al
mov si,1
jmp short GETBUFFRFB

; 22/09/2023
GETBUFFER:
xor al,al
GETBUFFRD:
;mov byte [ALLOWED],18h
mov byte [ALLOWED],Allowed_FAIL+Allowed_RETRY

; 20/05/2019 - Retro DOS v4.0
; DOSCODE:9937h (MSDOS 6.21, MSDOS.SYS)
; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:98DBh (MSDOS 5.0, MSDOS.SYS)

; 07/07/2024
; 06/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; PCDOS 7.1 IBMDOS.COM - DOSCODE:0AAB0h

; (windows Me IO.SYS - BIOSCODE:0CA3Dh)
; (MSDOS 6.22 MSDOS.SYS - DOSCODE:9937h)

GETBUFFR:
XOR SI,SI

; This entry point is called for FAT buffering with SI != 0

GETBUFFRFB:
MOV [PREREAD],AX ; save pre-read flag

; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
; ; ;
or si,si
jz short getb1

;cmp word [es:bp+0Fh],0
;cmp word [es:bp+DPB.FAT_SIZE],0
;jnz short getb1 ; not FAT32

;mov ax,[es:bp+23h]
;mov ax,[es:bp+DPB.EXT_FLAGS]
;test al,80h ; bit 7 -- 1 means only one FAT is active
;jz short getb1
;and ax,0Fh ; Active FAT is the one referenced in bits 0-3
;jz short getb1

push dx
mov cx,ax ; Zero based number of active FAT.
; (Only valid if mirroring is disabled.)

;mul word [es:bp+33h]
;mul word [es:bp+DPB.FAT32_SIZE+2]
;xchg ax,cx
;mul word [es:bp+31h]
;mul word [es:bp+DPB.FAT32_SIZE]
;add cx,dx
;pop dx
;add dx,ax
;adc [HIGH_SECTOR],cx
getb1:
; ; ;

; 15/12/2022
mov al,[ES:BP]
; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
;MOV AL,[ES:BP+DPB.DRIVE] ; mov al,[es:bp+0]
LDS DI,[LastBuffer] ; Get the recency pointer

; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
; ; ;
mov bx,-1 ; 0FFFFh
; ; ;

; MSDOS 6.0
;hkn; SS override
MOV CX,[SS:HIGH_SECTOR] ; F.C. >32mb ;AN000;

; See if this is the buffer that was most recently returned.
; A big performance win if it is.

;CMP DI,-1 ; Recency pointer valid?
; 06/03/2024 - Retro DOS v5.0
cmp di,bx ; -1
je short getb5 ; No

;cmp dx,[di+6]
;CMP DX,[DI+BUFFINFO.buf_sector]
;JNZ short getb5 ; wrong sector

; MSDOS 6.0
;cmp cx,[di+8]
;CMP CX,[DI+BUFFINFO.buf_sector+2] ; F.C. >32mb ;AN000;
;JNZ short getb5 ; F.C. >32mb ;AN000;

;cmp al,[di+4]
;CMP AL,[DI+BUFFINFO.buf_ID]
;JZ getb35 ; Just asked for same buffer
;jnz short getb5
;jmp getb35

; 17/12/2022
; 28/07/2019
;jmp getb35x
; 07/12/2022 MSDOS 5.0 MSDOS.SYS compatibility)
;jmp getb35

; It's not the buffer most recently returned. See if it's in the
; cache.
; ;
; (cx:dx) = sector #
; (al) = drive #
; (si) = 0 iff non fat sector, != 0 if FAT sector read
; ??? list may be incomplete ???

getb5:
; MSDOS 3.3
;lds di,[SS:BUFFHEAD]
; MSDOS 6.0

```

```

39047 00006983 E82EFF      CALL     GETCURHEAD          ; get Q Head
39048
39049 getb10:
39050      ;cmp     dx,[di+6]
39051      CMP     DX,[DI+BUFFINFO.buf_sector]
39052      ;jne     short getb12          ; wrong sector lo
39053      ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39054      ;;;
39055      ;jne     short getb11
39056      ;;;
39057      ; MSDOS 6.0
39058      ;cmp     cx,[di+8]
39059      CMP     CX,[DI+BUFFINFO.buf_sector+2]
39060      ;jne     short getb12          ; wrong sector hi
39061      ; 06/03/2024
39062      ;;;
39063      ;jne     short getb11
39064      ;;;
39065
39066      ;cmp     al,[di+4]
39067      CMP     AL,[DI+BUFFINFO.buf_ID]
39068      ;je      short getb25 ; 05/09/2018      ; Found the requested sector
39069      ;jne     short getb12
39070      ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39071      ;;;
39072      ;jne     short getb11
39073      ;;;
39074      ;jmp     getb25
39075
39076 getb11:
39077      ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39078      ;;;
39079      ;cmp     byte [di+4],0FFh
39080      CMP     byte [di+BUFFINFO.buf_ID],0FFh      ; Free buffer ?
39081      ;jne     short getb12          ; no
39082      MOV     bx,di          ; save buffer (offset) addr
39083      ;;;
39084
39085 getb12:
39086      ; MSDOS 3.3
39087      ;mov     di,[DI]
39088      ;mov     di,[DI+BUFFINFO.buf_link]
39089      ;
39090      ; 15/08/2018
39091      ;lds     di,[di]
39092
39093      ;cmp     di,-1 ; 0FFFFh
39094      ;jne     short getb10
39095      ;lds     di,[SS:BUFFHEAD]
39096
39097      ; MSDOS 6.0
39098      MOV     di,[di]
39099      ;mov     DI,[DI+BUFFINFO.BUF_NEXT]
39100      CMP     DI,[SS:FIRST_BUFF_ADDR]
39101      ;jne     short getb10          ; back at the front again?
39102      ;no, continue looking
39103
39104      ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39105      ;;;
39106      ;cmp     bx,0FFFFh      ; -1 ; invalid (not a free buffer address)
39107      ;je      short getb12x
39108      MOV     di,bx          ; restore free buffer (header offset) address
39109      ;jmp     short getb13
39110
39111 getb12x:
39112      ;;;
39113      ; The requested sector is not available in the buffers. DS:DI now points
39114      ; to the first buffer in the Queue. Flush the first buffer & read in the
39115      ; new sector into it.
39116      ;
39117      ; BUGBUG - what goes on here? Isn't the first guy the most recently
39118      ; used guy? Shuld be for fast lookup. If he is, we shouldn't take
39119      ; him, we should take LRU. And the above lookup shouldn't be
39120      ; down a chain, but should be hashed.
39121      ;
39122      ; (DS:DI) = first buffer in the queue
39123      ; (CX:DX) = sector # we want
39124      ; (SI) = 0 iff non fat sector, != 0 if FAT sector read
39125
39126      ; MSDOS 3.3 & MSDOS 6.0
39127      ;hkn; SS override
39128      PUSH     CX      ; MSDOS 6.0
39129      push     si
39130      push     dx
39131      push     bp
39132      push     es
39133      CALL     BUFWRITE          ; write out the dirty buffer
39134      pop      es
39135      pop      bp
39136      pop      dx
39137      pop      si
39138      POP     word [SS:HIGH_SECTOR] ; MSDOS 6.0
39139      ;jc      short getbx          ; if got hard error
39140      ;jnc     short getb13
39141      ;jmp     getbx
39142
39143 getb13:
39144      ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39145      %if 0
39146      ; MSDOS 6.0
39147      CALL     SET_RQ_SC_PARMS    ; set parms for secondary cache
39148      %endif
39149
39150      ; We're ready to read in the buffer, if need be. If the caller
39151      ; wanted to just *write* the buffer then we'll skip reading it in.
39152
39153      XOR     AH,AH          ; initial flags
39154      ;hkn; SS override
39155      ;test     byte [ss:PREREAD],0FFh
39156      ;jnz     short getb20
39157      CMP     [SS:PREREAD],ah ; 0          ; am to Read in the new sector?
39158      ;jnz     short getb20          ; no, we're done
39159      ;;;lea     bx,[di+16] ; MSDOS 3.3
39160      ;;;lea     bx,[di+20] ; MSDOS 6.0
39161      ;lea     bx,[di+24] ; PCDOS 7.1; 06/03/2024
39162      LEA     BX,[DI+BUFINSIZ]      ; (ds:bx) = data address
39163      ;MOV     CX,1
39164      ; 22/09/2023
39165      sub     cx,cx ; 0
39166      push     si
39167      push     di
39168      push     dx
39169      ; MSDOS 6.0
39170      push     es ; ***

```

```

39171
39172 ; Note: As far as I can tell, all disk reads into buffers go through
39173 ; this point. -mrw 10/88
39174
39175 ; cmp byte [ss:BuffInHMA],0 ; is buffers in HMA?
39176 ; 22/09/2023
39177 000069DA 36380E[7900] cmp [ss:BuffInHMA],cl ; 0
39178 000069DF 7407 jz short getb14
39179 000069E1 1E push ds ; **
39180 000069E2 53 push bx ; *
39181 000069E3 36C51E[7A00] lds bx,[ss:LOMemBuff] ; Then let's read it into scratch buff
39182 getb14:
39183 ;M039: Eliminated redundant HMA code.
39184
39185 ; 22/09/2023
39186 000069E8 41 inc cx ; cx = 1
39187
39188 ; MSDOS 3.3 (& MSDOS 6.0)
39189 000069E9 09F6 OR SI,SI ; FAT sector ?
39190 000069EB 7407 JZ short getb15 ; no
39191
39192 000069ED E897D5 call FATSECRD
39193 ;mov ah,2
39194 000069F0 B402 MOV AH,buf_isFAT ; Set buf_flags
39195
39196 000069F2 EB05 JMP SHORT getb17 ; Buffer is marked free if read barfs
39197
39198 getb15:
39199 000069F4 E8F3D5 call DREAD ; Buffer is marked free if read barfs
39200 000069F7 B400 MOV AH,0 ; Set buf_flags to no type, DO NOT XOR!
39201 getb17:
39202
39203 ; 06/03/2024 - Retro DOS v5.0
39204 %if 0
39205 ; 17/12/2022
39206 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39207 ;;if 0
39208 ; MSDOS 6.0 ;I001
39209 pushf ;I001
39210 cmp byte [ss:BuffInHMA],0 ; did we read into scratch buff ? ;I001
39211 jz short not_in_hma ; no ;I001
39212 ;mov cx,[es:bp+2]
39213 mov cx,[es:BP+DPB.SECTOR_SIZE] ;I001
39214 shr cx,1 ;I001
39215 popf ; Retrieve possible CY from DREAD ;I001
39216 mov si,bx ;I001
39217 pop di ; * ;I001
39218 pop es ; ** ;I001
39219 cld ;I001
39220 pushf ; Preserve possible CY from DREAD ;I001
39221 rep movsw ; move the contents of scratch buff ;I001
39222 push es ;I001
39223 pop ds ;I001
39224 ;;endif
39225
39226 ; 17/12/2022
39227 %if 0
39228 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39229 ; MSDOS 5.0
39230 ; pushf
39231 ; cmp byte [ss:BuffInHMA],0 ; did we read into scratch buff ?
39232 ; jz short not_in_hma ; no
39233 ; popf
39234 ; mov cx,[es:BP+DPB.SECTOR_SIZE]
39235 ; shr cx,1
39236 ; mov si,bx
39237 ; pop di ; *
39238 ; pop es ; **
39239 ; cld
39240 ; rep movsw
39241 ; push es
39242 ; pop ds
39243 ; jmp short getb19 ; 27/11/2022
39244 %endif
39245
39246 not_in_hma: ;I001
39247 popf ;I001
39248
39249 %else
39250 ; 06/03/2024
39251 ; PC DOS 7.1 IBMDOS.COM
39252 ;;;
39253 000069F9 B500 mov ch,0
39254 000069FB 368A0E[7900] mov cl,[ss:BuffInHMA] ; did we read into scratch buff ?
39255 00006A00 E31C jcxz getb19 ; no
39256 ;mov cx,[es:bp+2]
39257 00006A02 268B4E02 mov cx,[es:bp+DPB.SECTOR_SIZE]
39258 00006A06 89DE mov si,bx
39259 00006A08 5F pop di
39260 00006A09 07 pop es
39261 00006A0A 9C pushf
39262 00006A0B D1E9 shr cx,1
39263 00006A0D FC cld
39264 00006A0E 36803E[6A00]00 cmp byte [ss:DDMOVE],0
39265 ;jz short getb18+1 ; rep movsw (skip 32bit prefix)
39266 00006A14 7403 jz short getb18
39267 00006A16 D1E9 shr cx,1
39268 ;getb18:
39269 ;rep movsd ; move the contents of scratch buffer
39270 00006A18 66 db 66h ; 32bit prefix
39271 getb18:
39272 rep movsw
39273
39274 ;mov dx,es
39275 ;mov ds,dx
39276 00006A1B 06 push es
39277 00006A1C 1F pop ds
39278 00006A1D 9D popf ; Retrieve possible CY from DREAD
39279 ;;;
39280 %endif
39281
39282 getb19:
39283 00006A1E 07 pop es ; ***
39284 00006A1F 5A pop dx
39285 00006A20 5F pop di
39286 00006A21 5E pop si
39287 00006A22 726C JC short getbx
39288
39289 ; The buffer has the data setup in it (if we were to read)
39290 ; Setup the various buffer fields
39291 ;
39292 ; (ds:di) = buffer address
39293 ; (es:bp) = DPB address
39294 ; (HIGH_SECTOR:DX) = sector #

```

```

39295 ; (ah) = BUF_FLAGS value
39296 ; (si) = 0 if non fat sector, != 0 if FAT sector read
39297
39298 ;hkn; SS override
39299 getb20: ; MSDOS 6.0
39300 MOV CX,[SS:HIGH_SECTOR]
39301 ;mov [di+8],cx
39302 MOV [DI+BUFFINFO.buf_sector+2],CX
39303 ; MSDOS 3.3 (& MSDOS 6.0)
39304 ;mov [di+6],dx
39305 MOV [DI+BUFFINFO.buf_sector],DX
39306 ;;;mov [di+0Ah],bp ; MSDOS 3.3
39307 ;;;mov [di+0Dh],bp ; MSDOS 6.0
39308 ;mov [di+0Fh],bp ; PCDOS 7.1 ; 06/03/2024
39309 MOV [DI+BUFFINFO.buf_DPB],BP
39310 ;;;mov [di+0Ch],es
39311 ;;;mov [di+0Fh],es ; MSDOS 6.0
39312 ;mov [di+11h],es ; PCDOS 7.1 ; 06/03/2024
39313 MOV [DI+BUFFINFO.buf_DPB+2],ES
39314 ; 15/12/2022
39315 mov al,[es:bp]
39316 ;mov al,[es:bp+0]
39317 ; 27/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
39318 ;MOV AL,[ES:BP+DPB.DRIVE]
39319 ;mov [di+4],ax
39320 MOV [DI+BUFFINFO.buf_ID],AX ; Set ID and Flags
39321 getb25:
39322 ; MSDOS 3.3
39323 ;mov ax,1
39324
39325 ; MSDOS 6.0
39326 ;mov byte [di+0Ah],1
39327 MOV byte [DI+BUFFINFO.buf_wrtcnt],1 ; Default to not a FAT sector ;AC000;
39328 XOR AX,AX
39329
39330 ; 07/07/2024 (PCDOS 7.1)
39331 ;mov [di+0Dh],ax ; 0
39332 mov [di+BUFFINFO.buf_wrtcntinc+2],ax ; 0
39333
39334 ; MSDOS 3.3 (& MSDOS 6.0)
39335 OR SI,SI ; FAT sector ?
39336 JZ short getb30 ; no
39337
39338 ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39339 ;;;
39340 ;;cmp word [es:bp+0Fh],0
39341 ;cmp word [es:bp+DPB.FAT_SIZE],0
39342 cmp [es:bp+DPB.FAT_SIZE],ax ; 0
39343 jnz short getb27 ; not FAT32
39344
39345 ; FAT32
39346 ;;test word [es:bp+23h],80h
39347 ;test byte [es:bp+23h],80h
39348 test byte [es:bp+DPB.EXT_FLAGS],80h
39349 jnz short getb26 ; bit 7 -- 1 means only one FAT is active
39350 ;mov al,[es:bp+8]
39351 mov al,[es:bp+DPB.FAT_COUNT]
39352 ;mov [di+0Ah],al
39353 mov [di+BUFFINFO.buf_wrtcnt],al
39354 getb26:
39355 ;mov ax,[es:bp+33h]
39356 mov ax,[es:bp+DPB.FAT32_SIZE+2]
39357 ;mov [di+0Dh],ax
39358 mov [di+BUFFINFO.buf_wrtcntinc+2],ax
39359 ;mov ax,[es:bp+31h]
39360 mov ax,[es:bp+DPB.FAT32_SIZE]
39361 jmp short getb30
39362 getb27:
39363 ;;;
39364
39365 ;mov al,[es:bp+8]
39366 MOV AL,[ES:BP+DPB.FAT_COUNT] ; update number of copies of
39367
39368 ; MSDOS 6.0
39369 MOV [DI+BUFFINFO.buf_wrtcnt],AL ; this sector present on disk
39370 ;mov ax,[es:bp+0Fh]
39371 MOV AX,[ES:BP+DPB.FAT_SIZE] ; offset of identical FAT
39372 ; sectors
39373
39374 ; MSDOS 3.3
39375 ;mov ah,[es:bp+0Fh]
39376 ;MOV AH,[ES:BP+DPB.FAT_SIZE]
39377
39378 ; BUGBUG - dos 6 can clean this up by not setting wrtcntinc unless wrtcnt
39379 ; is set
39380
39381 getb30:
39382 ; MSDOS 6.0
39383 ;mov [di+0Bh],ax
39384 MOV [DI+BUFFINFO.buf_wrtcntinc],AX
39385
39386 ; MSDOS 3.3
39387 ;;mov [di+8],ax ; 15/08/2018
39388 ;MOV [DI+BUFFINFO.buf_wrtcnt],AX
39389 CALL PLACEBUF
39390
39391 ;hkn; SS override for next 4
39392 getb35:
39393 ; 17/12/2022
39394 ; 07/12/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39395 ; MSDOS 3.3 & MSDOS 5.0 & MSDOS 6.0
39396 ;MOV [SS:CURBUF+2],DS
39397 ;MOV [SS:LastBuffer+2],DS
39398 ;MOV [SS:CURBUF],DI
39399 ;MOV [SS:LastBuffer],DI
39400 ;CLC
39401
39402 ; 17/12/2022
39403 ; 07/12/2022
39404 ; Retro DOS v4.0
39405 mov [ss:LastBuffer+2],ds
39406 mov [ss:LastBuffer],di
39407 cll
39408 getb35x: ; 28/07/2019
39409 MOV [ss:CURBUF+2],ds
39410 MOV [ss:CURBUF],di
39411
39412 ; Return with 'C' set appropriately
39413 ; (dx) = caller's original value
39414
39415 getbx:
39416 push ss
39417 pop ds
39418 ;retn

```



```

39419             ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39420 getbuffrb_retn:
39421 ;flushbuf_retn:      ; 17/12/2022
39422 00006A92 C3      retn
39423
39424 ;Break      <FLUSHBUF -- WRITE OUT DIRTY BUFFERS>
39425 -----
39426 ; Input:
39427 ;   DS = DOSGROUP
39428 ;   AL = Physical unit number local buffers only
39429 ;   = -1 for all units and all remote buffers
39430 ; Function:
39431 ;   Write out all dirty buffers for unit, and flag them as clean
39432 ;   Carry set if error (user FAILED to I 24)
39433 ;   Flush operation completed.
39434 ;   DS Preserved, all others destroyed (ES too)
39435 -----
39436
39437             ; 20/05/2019 - Retro DOS v4.0
39438             ; DOSCODE:9A35h (MSDOS 6.21, MSDOS.SYS)
39439
39440             ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
39441             ; DOSCODE:99DAh (MSDOS 5.0, MSDOS.SYS)
39442
39443             ; 06/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
39444             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0AC2Bh
39445
39446 FLUSHBUF:
39447 ; MSDOS 3.3
39448 ;;mov ah,-1 ; 01/08/2018 - Retro DOS v3.0
39449 ;lds di,[BUFFHEAD]
39450
39451             ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39452             ;;;
39453 00006A93 3CFF      cmp     al,0FFh          ; -1
39454 00006A95 750C      jne     short flshbuf_2
39455 00006A97 BB1A00    mov     bx,26
39456
39457 flshbuf_1:
39458 00006A9A 3680A7[0713]F7 and     ss:drv_flags_1[bx],0F7h
39459 00006AA0 4B        and     byte [ss:bx+drive_flags-1],0F7h ; clear bit 3
39460 00006AA1 75F7      dec     bx          ; backward
39461                        jnz     short flshbuf_1
39462 flshbuf_2:
39463             ;;;
39464
39465 00006AA3 E80EFE      call    GETCURHEAD
39466 ;TEST word [ss:DOS34_FLAG],FROM_DISK_RESET ; from disk reset ? ;hkn;
39467 00006AA6 36F606[1106]04 TEST    byte [ss:DOS34_FLAG],FROM_DISK_RESET ; 4
39468 00006AAC 7508      jnz     short scan_buf_queue
39469 00006AAE 36833E[7100]00 cmp     word [ss:DirtyBufferCount],0
39470 00006AB4 7423      je      short end_scan
39471
39472 scan_buf_queue:
39473 00006AB6 E82900      call    CHECKFLUSH
39474 ;push ax ; MSDOS 3.3
39475 ; MSDOS 6.0
39476 ;mov ah,[di+4]
39477 00006AB9 8A6504      mov     ah,[DI+BUFFINFO.buf_ID]
39478 00006ABC 363826[2203] cmp     [SS:WPERR],ah          ;hkn;
39479 00006AC1 7408      je      short free_the_buf
39480 ;TEST word [ss:DOS34_FLAG],FROM_DISK_RESET ; from disk reset ? ;hkn;
39481 00006AC3 36F606[1106]04 TEST    byte [ss:DOS34_FLAG],FROM_DISK_RESET ; 4
39482 00006AC9 7405      jz      short dont_free_the_buf
39483 ; MSDOS 3.3
39484 ;;mov al,[di+4]
39485 ;mov al,[DI+BUFFINFO.buf_ID]
39486 ;cmp [SS:WPERR],al          ;hkn;
39487 ; 15/08/2018
39488 ;jne short dont_free_the_buf
39489 free_the_buf:
39490 ; MSDOS 6.0 (& MSDOS 3.3)
39491 00006ACB C74504FF00 mov     word [DI+BUFFINFO.buf_ID],00FFh
39492 dont_free_the_buf:
39493 ;pop ax ; MSDOS 3.3
39494
39495 ; MSDOS 3.3
39496 ;mov di,[DI]
39497 ;;mov di,[DI+BUFFINFO.buf_link] ; .buf_next
39498 ;
39499 ; 15/08/2018
39500 ;lds di,[di]
39501 ;
39502 ;cmp di,-1 ; 0FFFFh
39503 ;jnz short scan_buf_queue
39504
39505 ; MSDOS 6.0
39506 00006AD0 8B3D      mov     di,[di]
39507 ;mov di,[DI+BUFFINFO.buf_next] ; .buf_link
39508 00006AD2 363B3E[7E11] cmp     di,[SS:FIRST_BUFF_ADDR]          ;hkn;
39509 00006AD7 75DD      jne     short scan_buf_queue
39510
39511 end_scan:
39512 00006AD9 16        push    ss
39513 00006ADA 1F        pop     ds
39514 ; 01/08/2018 - Retro DOS v3.0
39515 ;cmp byte [FAILERR],0
39516 ;jne short bad_flush
39517 ;retn
39518 ;bad_flush:
39519 ;stc
39520 ;retn
39521
39522 ; 17/12/2022
39523 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39524 ; 01/08/2018 - Retro DOS v3.0
39525 00006ADB 803E[4A03]01 cmp     byte [FAILERR],1
39526 00006AE0 F5        cmc
39527 flushbuf_retn:
39528 00006AE1 C3      retn
39529
39530 ; 17/12/2022
39531 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39532 ;cmp byte [FAILERR],0
39533 ;jne short bad_flush
39534 ;retn
39535 ;bad_flush:
39536 ;stc
39537 ;retn
39538
39539 -----
39540 ;
39541 ; Procedure Name : CHECKFLUSH
39542 ;

```

```

39543 ; Inputs : AL - Drive number, -1 means do not check for drive
39544 ;         DS:DI - pointer to buffer
39545 ;
39546 ; Function : write out a buffer if it is dirty
39547 ;
39548 ; Carry set if problem (currently user FAILED to I 24)
39549 ;
39550 ;-----
39551 ;
39552 ; 07/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
39553 ;
39554 CHECKFLUSH:
39555 ; MSDOS 6.0
39556 mov     ah,-1 ; 01/08/2018 Retro DOS v3.0
39557 ;cmp     [di+4],ah
39558 cmp     [DI+BUFFINFO.buf_ID],AH
39559 jz      short flushbuf_retn ; Skip free buffer, carry clear
39560 cmp     AH,AL
39561 jz      short DOBUFFER ; do this buffer
39562 ;cmp     al,[di+4]
39563 cmp     AL,[DI+BUFFINFO.buf_ID]
39564 clc
39565 jnz     short flushbuf_retn ; Buffer not for this unit or SFT
39566 ;
39567 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39568 ;;;
39569 xor     bx,bx
39570 mov     bl,al
39571 ;test    ss:drive_flags[bx],8
39572 test    byte [ss:bx+drive_flags],8 ; bit 3
39573 jnz     short flushbuf_retn
39574 ;;;
39575 DOBUFFER:
39576 ;test    byte [di+5],40h
39577 TEST     byte [DI+BUFFINFO.buf_flags],buf_dirty
39578 jz      short flushbuf_retn ; Buffer not dirty, carry clear by TEST
39579 PUSH     AX
39580 ;push    word [di+4]
39581 PUSH     WORD [DI+BUFFINFO.buf_ID]
39582 CALL     BUFWRITE
39583 POP      AX
39584 JC       short LEAVE_BUF ; Leave buffer marked free (lost).
39585 ;and     ah,0BFh
39586 AND      AH,~buf_dirty ; Buffer is clean, clears carry
39587 ;mov     [di+4],ax
39588 MOV     [DI+BUFFINFO.buf_ID],AX
39589 LEAVE_BUF:
39590 POP      AX ; Search info
39591 checkflush_retn:
39592 retn
39593 ;Break <BUFWRITE -- WRITE OUT A BUFFER IF DIRTY>
39594 ;-----
39595 ;
39596 ; Bufwrite writes a buffer to the disk, if it's dirty.
39597 ;
39598 ; ENTRY DS:DI Points to the buffer
39599 ;
39600 ; EXIT Buffer marked free
39601 ; Carry set if error (currently user FAILED to I 24)
39602 ;
39603 ; USES All buf DS:DI
39604 ; HIGH_SECTOR
39605 ;-----
39606 ;
39607 ; 20/05/2019 - Retro DOS v4.0
39608 ; DOSCODE:9AA0h (MSDOS 6.21, MSDOS.SYS)
39609 ;
39610 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
39611 ; DOSCODE:9A45h (MSDOS 5.0, MSDOS.SYS)
39612 ;
39613 ; 07/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
39614 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0ACB1h
39615 ;
39616 BUFWRITE:
39617 ; 10/09/2018
39618 ; 01/08/2018 - Retro DOS v3.0
39619 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5E94h
39620 MOV     AX,0FFh
39621 ;xchg    ax,[di+4]
39622 XCHG     AX,[DI+BUFFINFO.buf_ID] ; Free, in case write barfs
39623 CMP     AL,0FFh
39624 jz      short checkflush_retn; Buffer is free, carry clear.
39625 ;test    ah,40h
39626 test    AH,buf_dirty
39627 jz      short checkflush_retn; Buffer is clean, carry clear.
39628 ; MSDOS 6.0
39629 call    DEC_DIRTY_COUNT ; LB. decrement dirty count
39630 ;hkn; SS override
39631 CMP     AL,[SS:WPERR]
39632 jz      short checkflush_retn; If in WP error zap buffer
39633 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39634 %if 0
39635 ;hkn; SS override
39636 ; MSDOS 6.0
39637 MOV     [SS:SC_DRIVE],AL ;LB. set it for invalidation ;AN000;
39638 %endif
39639 ; 07/03/2024
39640 ;;;les    bp,[di+10] ; MSDOS 3.3
39641 ;;;les    bp,[di+13] ; MSDOS 6.0
39642 ;;;les    bp,[di+15] ; PCDOS 7.1 ; 07/03/2024
39643 ;LES     BP,[DI+BUFFINFO.buf_DPB]
39644 ;;;lea    bx,[di+16]
39645 ;;;lea    bx,[di+20] ; MSDOS 6.0
39646 ;;;lea    bx,[di+24] ; PCDOS 7.1 ; 07/03/2024
39647 LEA     BX,[DI+BUFINSIZ] ; Point at buffer
39648 ; 07/03/2024
39649 %if 0
39650 ;mov     dx,[di+6]
39651 MOV     DX,[DI+BUFFINFO.buf_sector] ;F.C. >32mb ;AN000;
39652 ; MSDOS 6.0
39653 ;mov     cx,[di+8]
39654 MOV     CX,[DI+BUFFINFO.buf_sector+2] ;F.C. >32mb ;AN000;
39655 ;hkn; SS override
39656 MOV     [SS:HIGH_SECTOR],CX ;F.C. >32mb ;AN000;
39657 %else
39658 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)

```



```

39667      ;;;
39668      ;les    dx,[di+6]
39669      ;les    dx,[di+BUFFINFO.buf_sector]
39670      ;mov    [ss:HIGH_SECTOR],es
39671
39672      ;;;les    bp,[di+10] ; MSDOS 3.3
39673      ;;;les    bp,[di+13] ; MSDOS 6.0
39674      ;;;les    bp,[di+15] ; PCDOS 7.1 ; 07/03/2024
39675      ;les    bp,[di+BUFFINFO.buf_DPB]
39676      ;;;
39677      %endif
39678      ;mov    cl,[di+10] ; MSDOS 6.0 & PCDOS 7.1
39679      ;MOV    CL,[DI+BUFFINFO.buf_wrtcnt] ;>32mb ;AC000;
39680      ; MSDOS 3.3
39681      ;;; mov    cx,[DI+8]
39682      ;mov    cx,[DI+BUFFINFO.buf_wrtcnt]
39683      ;MOV    AL,CH ; [DI+BUFFINFO.buf_wrtcntinc]
39684      ;XOR    CH,CH
39685      ;;;mov    ah,ch ; MSDOS 3.3
39686
39687      ;hkn; SS override for ALLOWED
39688      ;mov    byte [SS:ALLOWED],18h
39689      ;MOV    byte [SS:ALLOWED],Allowed_RETRY+Allowed_FAIL
39690      ;test    byte [di+5],8
39691      ; MSDOS 6.0 (& Retro DOS 3.0)
39692      ;test    ah,8
39693      ;test    AH,buf_isDATA
39694      ;JZ     short NO_IGNORE
39695      ;or      byte [SS:ALLOWED],20h
39696      ;OR     byte [SS:ALLOWED],Allowed_IGNORE
39697      NO_IGNORE:
39698      ;xor     ah,ah ; 10/09/2018 (MSDOS 3.3, Retro DOS v3.0)
39699      ; MSDOS 6.0
39700      ;mov     ax,[di+11]
39701      ;MOV     AX,[DI+BUFFINFO.buf_wrtcntinc] ;>32mb ;AC000;
39702
39703      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39704      ;;;
39705      ;mov     si,[di+13]
39706      ;mov     si,[di+BUFFINFO.buf_wrtcntinc+2]
39707      ;;;
39708
39709      ;PUSH    DI ; Save buffer pointer
39710      ;XOR     DI,DI ; Indicate failure
39711
39712      ;push    ds ; *
39713      ;push    bx ; **
39714      ;WRTAGAIN:
39715      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39716      ;;;
39717      ;push    si ; SI:AX = FAT size (in sectors)
39718      ;push    word [ss:HIGH_SECTOR]
39719      ;;;
39720      ;push    di ; ***
39721      ;push    cx ; ****
39722      ;push    ax ; *****
39723      ;MOV     CX,1
39724      ; 17/12/2022
39725      ; ch = 0
39726      ;mov     cl,1 ; 24/07/2019
39727      ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39728      ;mov     cx,1
39729      ;push    bx ; *****
39730      ;push    dx ; *****
39731      ;push    ds ; *****
39732
39733      ; Note: As far as I can tell, all disk reads into buffers go through this point. -mrw 10/88
39734
39735      ; MSDOS 6.0
39736      ;cmp     byte [ss:BuffInHMA],0 ; 10/06/2019
39737      ; 22/09/2023
39738      ;cmp     [ss:BuffInHMA],ch ; 0
39739      ;jz     short NBUFFINHMA
39740      ;push    cx
39741      ;push    es
39742      ;mov     si,bx
39743      ;mov     cx,[es:bp+DPB.SECTOR_SIZE]
39744      ;shr     cx,1
39745      ;les     di,[ss:LoMemBuff] ; 10/06/2019
39746      ;mov     bx,di
39747      ;cld
39748      ;rep     movsw
39749      ; 07/03/2024 - Retro DOS v5.0
39750      ; (PCDOS 7.1 IBMDOS.COM)
39751      ;;;
39752      ;cmp     byte [ss:DDMOVE],0
39753      ;jz     short bufwr_movsw ; skip 32bit prefix
39754      ;shr     cx,1
39755      ;db      66h ; 32bit op prefix (rep movsd)
39756      ;bufwr_movsw:
39757      ;rep     movsw
39758      ;;;
39759      ;push    es
39760      ;pop     ds
39761      ;pop     es
39762      ;pop     cx
39763      ;NBUFFINHMA:
39764      ;call    DWRITE ; write out the dirty buffer
39765      ;pop     ds ; *****
39766      ;pop     dx ; *****
39767      ;pop     bx ; *****
39768      ;pop     ax ; *****
39769      ;pop     cx ; *****
39770      ;pop     di ; ***
39771      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39772      ;;;
39773      ;pop     word [ss:HIGH_SECTOR]
39774      ;pop     si
39775      ;;;
39776      ;JC     short NOSET
39777      ; 05/07/2024 (PCDOS 7.1)
39778      ;mov     di,1
39779      ;INC     DI ; If at least ONE write succeeds,
39780      ; NOSET: ; the operation succeeds.
39781      ;ADD     DX,AX
39782      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39783      ;;;
39784      ;adc     [ss:HIGH_SECTOR],si
39785      ;;;
39786      ;LOOP    WRTAGAIN
39787      ;pop     bx ; **
39788      ;pop     ds ; *
39789      ;OR     DI,DI ; Clears carry
39790      ;JNZ     short BWROK ; At least one write worked

```

```

39791             ;STC             ; DI never got INCed, all writes failed.
39792             ; 05/07/2024 (PCDOS 7.1)
39793             ;sub     di,1
39794             ; 22/09/2023
39795 00006BB4 83FF01      cmp     di,1
39796             BWR0K:
39797 00006BB7 5F          POP     DI
39798 00006BB8 C3          retn
39799
39800             ; 07/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
39801 %if 0
39802             ; NOTE: Secondary Buffer Cache is not used in PCDOS 7.1 IBMDOS.COM.
39803
39804             ;** Set_RQ_SC_Parms - Set Secondary Cache Parameters
39805             ;-----
39806             ; Set_RQ_SC_Parms sets the sector size and drive number value
39807             ; for the secondary cache. This updates SC_SECTOR_SIZE &
39808             ; SC_DRIVE even if SC is disabled to save the testing
39809             ; code and time
39810             ;
39811             ; ENTRY ES:BP = drive parameter block
39812             ;
39813             ; EXIT [SC_SECTOR_SIZE]= drive sector size
39814             ; [SC_DRIVE]= drive #
39815             ;
39816             ; USES Flags
39817             ;-----
39818
39819             ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
39820             ; 04/05/2019 - Retro DOS v4.0
39821
39822 SET_RQ_SC_PARMS:
39823 ;hkn; SS override for all variables used in this procedure.
39824     push     ax
39825     ;mov     ax,[es:bp+2]
39826     MOV     ax,[ES:BP+DPB.SECTOR_SIZE] ; save sector size
39827     MOV     [ss:SC_SECTOR_SIZE],ax
39828     ;mov     al,[es:bp+0]
39829     ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39830     ;MOV     al,[ES:BP+DPB.DRIVE] ; save drive #
39831     ; 15/12/2022
39832     mov     al,[ES:BP]
39833     MOV     [ss:SC_DRIVE],al
39834     pop     ax
39835 srspx:
39836     retn
39837             ;LB. return
39838
39839 %endif
39840
39841             ; 07/03/2024 -Retro DOS v5.0
39842             ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:0AD57h)
39843 %if 0
39844 null_sub:
39845 SET_RQ_SC_PARMS:
39846     retn
39847 %endif
39848
39849             ; 01/02/2024 - Retro DOS v5.0
39850             ;-----
39851             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:AD58h
39852
39853 SET_BUF_DIRTY:
39854             ; test byte [es:di+5],40h ; ...
39855 00006BB9 26F6450540     test     byte [es:di+BUFFINFO.buf_flags],buf_dirty
39856 00006BBE 750A          jnz     short yesdirty2
39857 00006BC0 26804D0540     or      byte [es:di+5],40h
39858             or      byte [es:di+BUFFINFO.buf_flags],buf_dirty
39859
39860 ;INC_DIRTY_COUNT:
39861 ; inc word [ss:DirtyBufferCount]
39862 yesdirty2:
39863     retn
39864
39865 ;Break <INC_DIRTY_COUNT-increment dirty count>
39866             ;-----
39867             ; Input:
39868             ; none
39869             ; Function:
39870             ; increment dirty buffers count
39871             ; Output:
39872             ; dirty buffers count is incremented
39873             ;
39874             ; All registers preserved
39875             ;-----
39876
39877 INC_DIRTY_COUNT:
39878 ;; BUGBUG ---- remove this routine
39879 ;; BUGBUG ---- only one instruction is needed (speed win, space loose)
39880 00006BC5 36FF06[7100]   inc     word [ss:DirtyBufferCount] ;hkn;
39881 00006BCA C3          yesdirty2: ; 01/02/2024
39882             retn
39883
39884 ;Break <DEC_DIRTY_COUNT-decrement dirty count>
39885             ;-----
39886             ; Input:
39887             ; none
39888             ; Function:
39889             ; decrement dirty buffers count
39890             ; Output:
39891             ; dirty buffers count is decremented
39892             ;
39893             ; All registers preserved
39894             ;-----
39895
39896 DEC_DIRTY_COUNT:
39897 00006BCB 36833E[7100]00     cmp     word [ss:DirtyBufferCount],0 ;hkn;
39898 00006BD1 7405          jz      short ddcx ; BUGBUG - shouldn't it be an
39899 00006BD3 36FF0E[7100]   dec     word [ss:DirtyBufferCount]
39900             ; error condition to underflow here? ;hkn;
39901 ddcx:
39902     retn
39903
39904             ;=====
39905             ; MSPROC.ASM, MSDOS 6.0, 1992
39906             ;=====
39907             ; 02/08/2018 - Retro DOS v3.0
39908             ; 29/04/2019 - Retro DOS v4.0
39909             ; 07/03/2024 - Retro DOS v5.0
39910
39911             ; (15/04/2018 - Retro DOS v2.0, MSDOS 2.11 - PROC.ASM - 1983)
39912
39913             ; Pseudo EXEC system call for DOS
39914
39915             ; TITLE MSPROC - process maintenance

```

```

39915 ; NAME MSPROC
39916 ;
39917 ; =====
39918 ; ** Process related system calls and low level routines for DOS 2.x.
39919 ; I/O specs are defined in DISPATCH.
39920 ;
39921 ; $WAIT
39922 ; $EXEC
39923 ; $Keep_process
39924 ; Stay_resident
39925 ; $EXIT
39926 ; $ABORT
39927 ; abort_inner
39928 ;
39929 ; Modification history:
39930 ;
39931 ; Created: ARR 30 March 1983
39932 ; AN000 version 4.0 jan. 1988
39933 ; A007 PTM 3957 - fake vesrion for IBMCACHE.COM
39934 ; A008 PTM 4070 - fake version for MS WINDOWS
39935 ;
39936 ; M000 added support for loading programs into UMBs 7/9/90
39937 ;
39938 ; M004 - MS PASCAL 3.2 support. Please see under tag M003 in
39939 ; dossym.inc. 7/30/90
39940 ; M005 - Support for EXE programs with out STACK segment and
39941 ; with resident size < 64K - 256 bytes. A 256 byte
39942 ; stack is provided at the end of the program. Note that
39943 ; only SP is changed.
39944 ; M020 - Fix for Rational bug for details see exepatch.asm
39945 ;
39946 ; M028 - 4b04 implementation
39947 ;
39948 ; M029 - Support for EXEs without stack rewritten. If EXE is
39949 ; in memory block >= 64K, sp = 0. If memory block
39950 ; obtained is <64K, point sp at the end of the memory
39951 ; block. For EXEs smaller than 64K, 256 bytes are still
39952 ; added for a stack segment which may be needed if it
39953 ; is loaded in low memory situations.
39954 ;
39955 ; M030 - Fixing bug in EXEPACPATCH & changing 4b04 to 4b05
39956 ;
39957 ; M040 - Bug #3052. The environment sizing code would flag a
39958 ; a bad environment if it reached 32767 bytes. Changed
39959 ; to allow 32768 bytes of environment.
39960 ;
39961 ; M047 - Release the allocated UMB when we failed to load a
39962 ; COM file high. Also ensure that if the biggest block
39963 ; into which we load the com file is less than 64K then
39964 ; we provide atleast 256 bytes of stack to the user.
39965 ;
39966 ; M050 - Made Lie table search CASE insensitive
39967 ;
39968 ; M060 - Removed special version table from the kernal and
39969 ; put it in a device drive which puts the address
39970 ; in the DOS DATA area location UU_IFS_DOS_CALL
39971 ; as a DWORD.
39972 ;
39973 ; M063 - Modified UMB support. If the HIGH_ONLY bit is set on
39974 ; entry do not try to load low if there is no space in
39975 ; UMBs.
39976 ;
39977 ; M068 - Support for copy protect apps. Call ChkCopyProt to
39978 ; set a20off_count. Set bit EXECA20BIT in DOS_FLAG. Also
39979 ; change return address to LeaveDos if AL=5.
39980 ;
39981 ; 20-Jul-1992 bens Added ifdef RESTRICTED_BUILD code that
39982 ; controls building a version of MSDOS.SYS that only
39983 ; runs programs from a fixed list (defined in the
39984 ; file RESTRICT.INC). Search for "RESTRICTED_BUILD"
39985 ; for details. This feature is used to build a
39986 ; "special" version of DOS that can be handed out to
39987 ; OEM/ISV customers as part of a "service" disk.
39988 ;
39989 ; =====
39990 ;
39991 ; SAVEXIT EQU 10
39992 ;
39993 ; BREAK <$WAIT - return previous process error code>
39994 ;
39995 ; =====
39996 ; $WAIT - Return previous process error code.
39997 ;
39998 ; Assembler usage:
39999 ;
40000 ; MOV AH, WaitProcess
40001 ; INT int_command
40002 ;
40003 ; ENTRY none
40004 ; EXIT (ax) = exit code
40005 ; USES all
40006 ; =====
40007 ;
40008 ; 20/05/2019 - Retro DOS v4.0
40009 ; DOSCODE:9B55h (MSDOS 6.21, MSDOS.SYS)
40010 ;
40011 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
40012 ; DOSCODE:9A5Ah (MSDOS 5.0, MSDOS.SYS)
40013 ;
40014 ; 07/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
40015 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0AD78h
40016 ;
40017 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:9B55h)
40018 ; (Windows ME IO.SYS - BIOSCODE:944Ch)
40019 ;
40020 ; _$WAIT:
40021 ; 02/08/2018 - Retro DOS v3.0
40022 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5E1h
40023 00006BD9 31C0 xor AX,AX
40024 00006BDB 368706[3403] xchg AX,[ss:exit_code]
40025 00006BE0 E98B9A jmp SYS_RET_OK
40026 ;
40027 ; =====
40028 ; BREAK <$exec - load/go a program>
40029 ; EXEC.ASM - EXEC System Call
40030 ;
40031 ;
40032 ; Assembler usage:
40033 ; lds DX, Name
40034 ; les BX, Blk
40035 ; mov AH, Exec
40036 ; mov AL, FUNC
40037 ; int INT_COMMAND
40038 ;

```

```

40039 ; AL Function
40040 ; -----
40041 ; 0 Load and execute the program.
40042 ; 1 Load, create the program header but do not
40043 ; begin execution.
40044 ; 3 Load overlay. No header created.
40045 ;
40046 ; AL = 0 -> load/execute program
40047 ;
40048 ; +-----+
40049 ; | WORD segment address of |
40050 ; | environment. |
40051 ; +-----+
40052 ; | DWORD pointer to ASCIZ |
40053 ; | command line at 80h |
40054 ; +-----+
40055 ; | DWORD pointer to default |
40056 ; | FCB to be passed at 5Ch |
40057 ; +-----+
40058 ; | DWORD pointer to default |
40059 ; | FCB to be passed at 6Ch |
40060 ; +-----+
40061 ;
40062 ; AL = 1 -> load program
40063 ;
40064 ; +-----+
40065 ; | WORD segment address of |
40066 ; | environment. |
40067 ; +-----+
40068 ; | DWORD pointer to ASCIZ |
40069 ; | command line at 80h |
40070 ; +-----+
40071 ; | DWORD pointer to default |
40072 ; | FCB to be passed at 5Ch |
40073 ; +-----+
40074 ; | DWORD pointer to default |
40075 ; | FCB to be passed at 6Ch |
40076 ; +-----+
40077 ; | DWORD returned value of |
40078 ; | CS:IP |
40079 ; +-----+
40080 ; | DWORD returned value of |
40081 ; | SS:IP |
40082 ; +-----+
40083 ;
40084 ; AL = 3 -> load overlay
40085 ;
40086 ; +-----+
40087 ; | WORD segment address where |
40088 ; | file will be loaded. |
40089 ; +-----+
40090 ; | WORD relocation factor to |
40091 ; | be applied to the image. |
40092 ; +-----+
40093 ;
40094 ; Returns:
40095 ; AX = error_invalid_function
40096 ; = error_bad_format
40097 ; = error_bad_environment
40098 ; = error_not_enough_memory
40099 ; = error_file_not_found
40100 ; =====
40101 ;
40102 ; Revision history:
40103 ;
40104 ; A000 version 4.00 Jan. 1988
40105 ; =====
40106 ;
40107 ; Exec_Internal_Buffer EQU OPENBUF
40108 ; Exec_Internal_Buffer_Size EQU (128+128+53+curdirLen)
40109 ;
40110 ; =====
40111 ;
40112 ; ; warning message on buffers
40113 ; ; Please make sure that the following are contiguous and of the
40114 ; ; following sizes:
40115 ; ;
40116 ; ; OpenBuf 128
40117 ; ; RenBuf 128
40118 ; ; SearchBuf 53
40119 ; ; DummyCDS curdirLen
40120 ; ;
40121 ; ENDIF
40122 ;
40123 ; =====
40124 ;
40125 ; =====
40126 ;
40127 ; =====
40128 ;
40129 ; ; 20/05/2019 - Retro DOS v4.0
40130 ; ; DOSCODE:9B5Fh (MSDOS 6.21, MSDOS.SYS)
40131 ;
40132 ; ; 30/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
40133 ; ; DOSCODE:9B04h (MSDOS 5.0, MSDOS.SYS)
40134 ;
40135 ; ; 08/03/2024
40136 ; ; 07/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
40137 ; ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0AD82h
40138 ;
40139 ; ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:9B5Fh)
40140 ; ; (Windows ME IO.SYS - BIOSCODE:946Fh)
40141 ;
40142 ; _$EXEC:
40143 ; ; 02/08/2018 - Retro DOS v3.0
40144 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5EF1h
40145 ;
40146 ; EXEC001S:
40147 ; ; LocalVar Exec_Blk ,DWORD
40148 ; ; LocalVar Exec_Func ,BYTE
40149 ; ; LocalVar Exec_Load_High ,BYTE
40150 ; ; LocalVar Exec_FH ,WORD
40151 ; ; LocalVar Exec_Rel_Fac ,WORD
40152 ; ; LocalVar Exec_Res_Len_Para ,WORD
40153 ; ; LocalVar Exec_Environ ,WORD
40154 ; ; LocalVar Exec_Size ,WORD
40155 ; ; LocalVar Exec_Load_Block ,WORD
40156 ; ; LocalVar Exec_DMA ,WORD
40157 ; ; LocalVar ExecNameLen ,WORD
40158 ; ; LocalVar ExecName ,DWORD
40159 ; ;
40160 ; ; LocalVar Exec_DMA_Save ,WORD
40161 ; ; LocalVar Exec_NoStack ,BYTE
40162 ;

```

```

40163 ; MSDOS 3.3 (& MSDOS 6.0)
40164 ;%define Exec_Blk dword [bp-4]
40165 %define Exec_Blk [bp-4] ; 09/08/2018
40166 %define Exec_BlkL word [bp-4]
40167 %define Exec_BlkH word [bp-2]
40168 %define Exec_Func byte [bp-5]
40169 %define Exec_Load_High byte [bp-6]
40170 %define Exec_FH word [bp-8]
40171 %define Exec_Rel_Fac word [bp-10]
40172 %define Exec_Res_Len_Para word [bp-12]
40173 %define Exec_Environ word [bp-14]
40174 %define Exec_Size word [bp-16]
40175 %define Exec_Load_Block word [bp-18]
40176 %define Exec_DMA word [bp-20]
40177 %define ExecNameLen word [bp-22]
40178 ;%define ExecName dword [bp-26]
40179 %define ExecName [bp-26] ; 09/08/2018
40180 %define ExecNameL word [bp-26]
40181 %define ExecNameH word [bp-24]
40182 ; MSDOS 6.0
40183 %define Exec_DMA_Save word [bp-28]
40184 %define Exec_NoStack byte [bp-29]
40185
40186 ; =====
40187 ; validate function
40188 ; =====
40189
40190 ; M068 - Start
40191 ;
40192 ; Reset the A20OFF_COUNT to 0. This is done as there is a
40193 ; possibility that the count may not be decremented all the way to
40194 ; 0. A typical case is if the program for which we intended to keep
40195 ; the A20 off for a sufficiently long time (A20OFF_COUNT int 21
40196 ; calls), exits pre-maturely due to error conditions.
40197
40198 ; MSDOS 6.0
40199 00006BE3 36C606[8500]00 mov byte [SS:A20OFF_COUNT], 0
40200
40201 ; If al=5 (ExecReady) we'll change the return address on the stack
40202 ; to be LeaveDos in msdisp.asm. This ensures that the EXECA200FF
40203 ; bit set in DOS_FLAG by ExecReady is not cleared in msdisp.asm
40204
40205 00006BE9 3C05 cmp al,5 ; Q: is this ExecReady call
40206 ;jne short @f ; N: continue
40207 00006BEB 7505 jne short Exec_@f ; Y: change ret addr. to LeaveDos.
40208 ; Note CX is not input to ExecReady
40209 00006BED 59 pop cx
40210 00006BEE B9[F603] mov cx,LeaveDOS
40211 00006BF1 51 push cx
40212 ;@@:
40213 Exec_@f:
40214 ; M068 - End
40215
40216 ;Enter
40217
40218 00006BF2 55 push bp
40219 00006BF3 89E5 mov bp,sp
40220 ;sub sp,26 ; MSDOS 3.3
40221 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40222 ;sub sp,29 ; MSDOS 6.0 & MSDOS 6.22 & PC DOS 7.1 ; 07/03/2024
40223 ; 17/12/2022
40224 ; 20/05/2019
40225 00006BF5 83EC1E sub sp,30 ; Retro DOS v4.0
40226
40227 ; MSDOS 6.0
40228 00006BF8 3C05 cmp AL,5 ; only 0, 1, 3 or 5 are allowed ;M028
40229 ; M030
40230 00006BFA 7611 jna short Exec_Check_2
40231
40232 ; MSDOS 3.3
40233 ;cmp AL,3
40234 ;jna short Exec_Check_2
40235
40236 Exec_Bad_Fun:
40237 ;mov byte [ss:EXTERR_LOCUS],errLOC_unk ; 1
40238 ; Extended Error Locus;smr;SS Override
40239
40240 ; 01/07/2024
40241 00006BFC E8DEA6 call set_exerr_locus_unk
40242
40243 00006BFF B001 ;mov al,1
40244 mov al,error_invalid_function
40245
40246 Exec_Ret_Err:
40247 ;Leave
40248 00006C01 89EC mov sp,bp
40249 00006C03 5D pop bp
40250 00006C04 E9719A ;transfer SYS_RET_ERR
40251 jmp SYS_RET_ERR
40252
40253 ; MSDOS 6.0
40254 00006C07 E89C18 ExecReadyJ: call ExecReady ; M028
40255 00006C0A E91204 jmp norm_ovl ; do a Leave & xfer sysret_OK ; M028
40256
40257 Exec_Check_2:
40258 00006C0D 3C02 cmp AL,2
40259 00006C0F 74EB je short Exec_Bad_Fun
40260
40261 ; MSDOS 6.0
40262 00006C11 3C04 cmp al,4 ; 2 & 4 are not allowed
40263 00006C13 74E7 je short Exec_Bad_Fun
40264
40265 00006C15 3C05 cmp al,5 ; M028 ; M030
40266 00006C17 74EE je short ExecReadyJ ; M028
40267
40268 ;mov [bp-4],bx
40269 00006C19 895EFC mov Exec_BlkL,BX ; stash args
40270 ;mov [bp-2],es
40271 00006C1C 8C46FE mov Exec_BlkH,ES
40272 ;mov [bp-5],al
40273 00006C1F 8846FB mov Exec_Func,AL
40274 ;mov byte [bp-6],0
40275 00006C22 C646FA00 mov Exec_Load_High,0
40276
40277 ;mov [bp-26],dx
40278 00006C26 8956E6 mov ExecNameL,dx ; set up length of exec name
40279 ;mov [bp-24],ds
40280 00006C29 8C5EE8 mov ExecNameH,DS
40281 00006C2C 89D6 mov SI,dx ; move pointer to convenient place
40282 ;invoke DStrLen
40283 00006C2E E8A6AB call DStrLen
40284 ;mov [bp-22],cx
40285 00006C31 894EEA mov ExecNameLen,CX ; save length
40286

```

```

40287 ; MSDOS 6.0
40288 00006C34 36A0[0203] mov     al,[ss:AllocMethod] ; M063: save alloc method in
40289 00006C38 36A2[8400] mov     [ss:ALLOCMSAVE],al ; M063: AllocMsave
40290
40291 ;xor     AL,AL ; open for reading
40292 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
40293 ;;;
40294 00006C3C B0A0 mov     al,0A0h ; Access mode bits: (0 to 2)
40295 ;;; ; 000 read access
40296
40297 ; Sharing mode bits: (4 to 6)
40298 ; 010 deny others write access
40299 ;;;
40300 ; bit 7 - private
40301 ;;;
40302
40303 00006C3E 55 push     BP
40304
40305 ; MSDOS 6.0
40306 ;or      byte [ss:DOS_FLAG],1
40307 00006C3F 36800E[8600]01 or      byte [ss:DOS_FLAG],EXECOPEN ; this flag is set to indicate to
40308 ; the redir that this open call is
40309 ; due to an exec.
40310
40311 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
40312 ;;;
40313 00006C45 1E push     ds
40314 00006C46 52 push     dx
40315 ;;;
40316
40317 ;invoke $OPEN ; is the file there?
40318 00006C47 E87A13 call     _$OPEN
40319
40320 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
40321 ;;;
40322 00006C4A 5A pop      dx
40323 00006C4B 1F pop      ds
40324 00006C4C 7305 jnc      short Exec_Open_Ok
40325
40326 00006C4E 30C0 xor      al,al ; open for reading
40327 00006C50 E87113 call     _$OPEN
40328
40329 Exec_Open_Ok:
40330 ;;;
40331
40332 ; MSDOS 6.0
40333 00006C53 9C pushf
40334 ; 02/06/2019
40335 ;and     byte [ss:DOS_FLAG],0FEh
40336 00006C54 368026[8600]FE and     byte [ss:DOS_FLAG],~EXECOPEN ; reset flag
40337 00006C5A 9D popf
40338
40339 00006C5B 5D pop      BP
40340
40341 ; MSDOS 3.3 & MSDOS 6.0
40342 00006C5C 72A3 jc      short Exec_Ret_Err
40343
40344 ;mov     [bp-8],ax
40345 00006C5E 8946F8 mov     Exec_FH,AX
40346 00006C61 89C3 mov     BX,AX
40347 00006C63 30C0 xor     AL,AL
40348 ;invoke $Ioctl
40349 00006C65 E81BBC call     _$_IOCTL
40350 00006C68 7207 jc      short Exec_BombJ
40351
40352 ;test    dl,80h
40353 00006C6A F6C280 test    DL,devid_ISDEV
40354 00006C6D 740A jz      short Exec_Check_Environ
40355
40356 ;mov     al,2
40357 00006C6F B002 mov     AL,error_file_not_found
40358
40359 00006C71 E9C800 Exec_BombJ: jmp     Exec_Bomb
40360
40361 BadEnv:
40362 ;mov     al,0Ah
40363 00006C74 B00A mov     AL,error_bad_environment
40364 00006C76 E9C300 jmp     Exec_Bomb
40365
40366 Exec_Check_Environ:
40367 ;mov     word [bp-18],0
40368 00006C79 C746EE0000 mov     Exec_Load_Block,0
40369 ;mov     word [bp-14],0
40370 00006C7E C746F20000 mov     Exec_Environ,0
40371 ; overloads... no environment
40372
40373 00006C83 F646FB02 ;test    byte [bp-5],2
40374 00006C87 7552 test    Exec_Func,exec_func_overlay
40375 ;jnz     short Exec_Read_Header
40376
40377 00006C89 C576FC ;lds     si,[bp-4]
40378 00006C8C 8B04 lds     SI,Exec_Blkl ; get block
40379 ;mov     ax,[SI]
40380 00006C8E 09C0 mov     AX,[SI+EXEC1.ENVIRON] ; address of environ
40381 00006C90 750C or      AX,AX
40382 ;jnz     short Exec_Scan_Env
40383 00006C92 368E1E[3003] mov     DS,[ss:CurrentPDB] ;smr;SS Override
40384 ;mov     ax,[44]
40385 00006C97 A12C00 mov     AX,[PDB.ENVIRON]
40386
40387 ; MSDOS 6.0
40388 ;-----BUG 92 4/30/90-----
40389 ;
40390 ; Exec_environ is being correctly initialized after the environment has been
40391 ; allocated and copied form the parent's env. It must not be initialized here.
40392 ; Because if the call to $alloc below fails Exec_dealloc will deallocate the
40393 ; parent's environment.
40394 ; mov     Exec_Environ,AX
40395 ;
40396 ;-----
40397
40398 ;mov     [bp-14],ax
40399 ;mov     Exec_Environ,ax
40400
40401 00006C9A 09C0 or      AX,AX
40402 00006C9C 743D jz      short Exec_Read_Header
40403
40404 Exec_Scan_Env:
40405 00006C9E 8EC0 mov     ES,AX
40406 00006CA0 31FF xor     DI,DI
40407 ;mov     CX,7FFFh ; MSDOS 3.3
40408 00006CA2 B90080 mov     CX,8000h ; MSDOS 6.0 ; at most 32k of environment ;M040
40409 00006CA5 30C0 xor     AL,AL
40410

```



```

40411 Exec_Get_Environ_Len:
40412 repnz scasb ; find that nul byte
40413 jnz short BadEnv
40414
40415 dec CX ; Dec CX for the next nul byte test
40416 js short BadEnv ; gone beyond the end of the environment
40417
40418 scasb ; is there another nul byte?
40419 jnz short Exec_Get_Environ_Len ; no, scan some more
40420
40421 push DI
40422 ;lea bx,[DI+11h]
40423 lea BX,[DI+0Fh+2]
40424 ;add bx,[bp-22]
40425 add BX,ExecNameLen ; BX <- length of environment
40426 ; remember argv[0] length
40427 ; round up and remember argc
40428 mov CL,4
40429 shr BX,CL ; number of paragraphs needed
40430 push ES
40431 ;invoke $Alloc ; can we get the space?
40432 call _$ALLOC
40433 pop DS
40434 pop CX
40435
40436 ;jnc short Exec_Save_Environ
40437 ;jmp SHORT Exec_No_Mem ; nope... cry and sob
40438 ; 17/12/2022
40439 jnc short Exec_No_Mem ; 02/06/2019
40440 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40441 ;jnc short Exec_Save_Environ
40442 ;jmp SHORT Exec_No_Mem
40443
40444 Exec_Save_Environ:
40445 mov ES,AX
40446 ;mov [bp-14],ax
40447 mov Exec_Environ,AX ; save him for a rainy day
40448 xor SI,SI
40449 mov DI,SI
40450 rep movsb ; copy the environment
40451 mov AX,1
40452 stosw
40453 ;lds si,[bp-26]
40454 lds SI,ExecName
40455 ;mov cx,[bp-22]
40456 mov CX,ExecNameLen
40457 rep movsb
40458
40459 Exec_Read_Header:
40460 ; we read in the program header into the above data area and
40461 ; determine where in this memory the image will be located.
40462
40463 ;Context DS
40464 push ss
40465 pop ds
40466 ;mov cx,26
40467 mov CX,exec_header_len ; header size
40468 mov DX,exec_signature
40469 push ES
40470 push DS
40471 call ExecRead
40472 pop DS
40473 pop ES
40474 jc short Exec_Bad_File
40475
40476 or AX,AX
40477 jz short Exec_Bad_File
40478 ;cmp ax,26
40479 cmp AX,exec_header_len ; did we read the right number?
40480 jnz short Exec_Com_Filej ; yep... continue
40481
40482 test word [exec_max_BSS],-1 ; indicate load high?
40483 jnz short Exec_Check_Sig
40484
40485 ;mov byte [bp-6],0FFh
40486 mov Exec_Load_High,-1
40487
40488 Exec_Check_Sig:
40489 mov AX,[exec_signature] ; rms;NSS
40490 ;cmp ax,5A4Dh ; 'MZ'
40491 cmp AX,exe_valid_signature; zibo arises!
40492 jz short Exec_Save_Start ; assume com file if no signature
40493
40494 ;cmp ax,4D5Ah ; 'ZM'
40495 cmp AX,exe_valid_old_signature ; zibo arises!
40496 jz short Exec_Save_Start ; assume com file if no signature
40497
40498 Exec_Com_Filej:
40499 jmp Exec_Com_File
40500
40501 ; we have the program header... determine memory requirements
40502
40503 Exec_Save_Start:
40504 mov AX,[exec_pages] ; get 512-byte pages ;rms;NSS
40505 mov CL,5 ; convert to paragraphs
40506 shl AX,CL
40507 sub AX,[exec_par_dir] ; AX = size in paragraphs ;rms;NSS
40508 ;mov [bp-12],ax
40509 mov Exec_Res_Len_Para,AX
40510
40511 ; Do we need to allocate memory?
40512 ; Yes if function is not load-overlay
40513
40514 ;test byte [bp-5],2
40515 test Exec_Func,exec_func_overlay
40516 jz short Exec_Allocate ; allocation of space
40517
40518 ; get load address from block
40519
40520 ;les di,[bp-4]
40521 les DI,Exec_Blk
40522
40523 ; 07/03/2024
40524 %if 0
40525 mov ax,[es:di]
40526 ;mov AX,[ES:DI+EXEC3.load_addr]
40527 ;mov [bp-20],ax
40528 mov Exec_DMA,AX
40529
40530 ; 17/12/2022
40531 ;;mov ax,[es:di+2]
40532 ;mov AX,[ES:DI+EXEC3.reloc_fac]
40533 ;mov [bp-10],ax
40534 ;mov Exec_Rel_Fac,AX

```

```

40535
40536 ; 17/12/2022
40537 ; 30/11/2022 (!most proper code!)
40538 ;mov dx,[es:di+2]
40539 mov dx,[ES:DI+EXEC3.reloc_fac]
40540 ;mov [bp-10],dx
40541 mov Exec_Rel_Fac,dx
40542 %else
40543 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
40544 ;;;
40545 00006D28 06 push es
40546 00006D29 26C405 les ax,[es:di]
40547 ;les ax,[ES:DI+EXEC3.load_addr]
40548 ;mov [bp-20],ax
40549 00006D2C 8946EC mov Exec_DMA,ax
40550 ;mov [bp-10],es
40551 00006D2F 8C46F6 mov Exec_Rel_Fac,es
40552 00006D32 07 pop es
40553 ;;;
40554 %endif
40555 ; ax = Exec_DMA
40556 00006D33 E9DE00 jmp Exec_Find_Res
40557
40558 ; 17/12/2022
40559 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40560 ; 27/09/2023
40561 %if 0
40562 ; 02/06/2019 - Retro DOS v4.0
40563 ;mov ax,[bp-20] ; **
40564 mov AX,Exec_DMA ; **
40565 ; 10/08/2018
40566 jmp Exec_Find_Res ; M000
40567 %endif
40568
40569 Exec_No_Mem:
40570 ;mov al,8
40571 00006D36 B008 mov AL,error_not_enough_memory
40572 00006D38 EB02 jmp short Exec_Bomb
40573
40574 Exec_Bad_File:
40575 ;mov al,0Bh
40576 00006D3A B00B mov AL,error_bad_format
40577
40578 Exec_Bomb:
40579 ;mov bx,[bp-8]
40580 00006D3C 8B5EF8 mov BX,Exec_FH
40581 00006D3F E84104 call Exec_Dealloc
40582 ;LeaveCrit CritMem
40583 00006D42 E8CAAB call LCritMEM
40584 ;save <AX,BP>
40585 00006D45 50 push ax
40586 00006D46 55 push bp
40587 ;invoke $CLOSE
40588 00006D47 E81E0A call _$CLOSE
40589 ;restore <BP,AX>
40590 00006D4A 5D pop bp
40591 00006D4B 58 pop ax
40592 00006D4C E9B2FE jmp Exec_Ret_Err
40593
40594 Exec_Chk_Mem:
40595 ; 24/09/2023
40596 ; ds = DOSDATA
40597 ; 17/12/2022
40598 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40599 %if 0
40600 ; MSDOS 6.0 ; M063 - Start
40601 ;mov al,[ss:AllocMethod] ; save current alloc method in ax
40602 ; 10/06/2019
40603 00006D4F A0[0203] mov al,[AllocMethod]
40604 ;mov b1,[ss:ALLOCMSAVE]
40605 00006D52 8A1E[8400] mov b1,[ALLOCMSAVE]
40606 ;mov [ss:AllocMethod],b1 ; restore original allocmethod
40607 00006D56 881E[0203] mov [AllocMethod],b1
40608
40609 00006D5A F6C340 test b1,HIGH_ONLY ; 40h ; Q: was the HIGH_ONLY bit already set
40610 00006D5D 75D7 jnz short Exec_No_Mem ; Y: no space in UMBS. Quit
40611 ; ; N: continue
40612 ;
40613 00006D5F A840 test al,HIGH_ONLY ; Q: did we set the HIGH_ONLY bit
40614 00006D61 74D3 jz short Exec_No_Mem ; N: no memory
40615 ; 02/06/2019
40616 ;mov ax,[ss:SAVE_AX] ; Y: restore ax and
40617 00006D63 A1[8A00] mov ax,[SAVE_AX]
40618 ;jmp short Exec_Norm_Alloc ; Try again
40619 ; M063 - End
40620 00006D66 EB2B jmp short Exec_Norm_Alloc1
40621 %endif
40622
40623 ; 17/12/2022
40624 %if 0
40625 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40626 ; MSDOS 6.0 ; M063 - Start
40627 mov al,[ss:AllocMethod] ; save current alloc method in ax
40628 b1,[ss:ALLOCMSAVE]
40629 [ss:AllocMethod],b1 ; restore original allocmethod
40630
40631 test b1,HIGH_ONLY ; 40h ; Q: was the HIGH_ONLY bit already set
40632 jnz short Exec_No_Mem ; Y: no space in UMBS. Quit
40633 ; ; N: continue
40634 ;
40635 test al,HIGH_ONLY ; Q: did we set the HIGH_ONLY bit
40636 jz short Exec_No_Mem ; N: no memory
40637 ;
40638 mov ax,[ss:SAVE_AX] ; Y: restore ax and
40639 jmp short Exec_Norm_Alloc ; Try again
40640 ; M063 - End
40641 %endif
40642
40643 Exec_Allocate:
40644 ; 09/09/2018
40645
40646 ; M005 - START
40647 ; If there is no STACK segment for this exe file and if this
40648 ; not an overlay and the resident size is less than 64K -
40649 ; 256 bytes we shall add 256 bytes to the programs
40650 ; resident memory requirement and set Exec_SP to this value.
40651
40652 ; 17/12/2022
40653 00006D68 29DB sub bx,bx ; 0
40654
40655 ; MSDOS 6.0
40656 ;mov byte [bp-29],0
40657 mov Exec_NoStack,0
40658 ; 17/12/2022

```



```

40659 00006D6A 885EE3      mov     Exec_NoStack,b1 ; 0
40660 00006D6D 391E[D50E]      cmp     [exec_SS],bx ; 0
40661                      ;cmp     word [exec_SS],0      ; Q: is there a stack seg
40662 00006D71 7511      jne     short ea1      ; Y: continue normal processing
40663 00006D73 391E[D70E]      cmp     [exec_SP],bx ; 0
40664                      ;cmp     word [exec_SP],0      ; Q: is there a stack ptr
40665 00006D77 750B      jne     short ea1      ; Y: continue normal processing
40666
40667                      ;inc     byte [bp-29]
40668 00006D79 FE46E3      inc     Exec_NoStack
40669 00006D7C 3DF00F      cmp     ax,1000h-10h ; 0FF0h ; Q: is this >= 64K-256 bytes
40670 00006D7F 7303      jae     short ea1      ; Y: don't set Exec_SP
40671
40672 00006D81 83C010      add     ax,10h          ; add 10h paras to mem requirement
40673 ea1:
40674                      ; M005 - END
40675
40676                      ; MSDOS 6.0          ; M000 - start
40677                      ; 20/05/2019
40678                      ; (ds = ss = DOSDATA)
40679 00006D84 F606[0203]80      test    byte [AllocMethod],HIGH_FIRST ; 80h
40680                      ; Q: is the alloc strat high_first
40681 00006D89 7405      jz      short Exec_Norm_Alloc ; N: normal allocate
40682                      ; Y: set high_only bit
40683 00006D8B 800E[0203]40      or      byte [AllocMethod],HIGH_ONLY ; 40h
40684                      ; M000 - end
40685 Exec_Norm_Alloc:
40686 00006D90 A3[8A00]      mov     [SAVE_AX],ax    ; M000: save ax for possible 2nd
40687 Exec_Norm_Alloc1: ; 02/06/2019
40688                      ; M000: attempt at allocating memory
40689                      ; MSDOS 3.3
40690                      ;push     ax          ; M000
40691
40692 00006D93 BBFFFF      mov     BX,0FFFFh      ; see how much room in arena
40693 00006D96 1E      push    DS
40694                      ;invoke $Alloc      ; should have carry set and BX has max
40695 00006D97 E86F05      call    _$ALLOC
40696 00006D9A 1F      pop     DS
40697
40698                      ; MSDOS 6.0
40699 00006D9B A1[8A00]      mov     AX,[SAVE_AX]    ; M000
40700                      ; MSDOS 3.3
40701                      ;pop      ax          ; M000
40702
40703 00006D9E 83C010      add     AX,10h          ; room for header
40704 00006DA1 83FB11      cmp     BX,11h          ; enough room for a header
40705                      ; MSDOS 6.0
40706 00006DA4 72A9      jnb     short Exec_Chk_Mem ; M000
40707                      ; MSDOS 3.3
40708                      ;jnb     short Exec_No_Mem
40709
40710 00006DA6 39D8      cmp     AX,BX           ; is there enough for bare image?
40711                      ; MSDOS 6.0
40712 00006DA8 77A5      ja      short Exec_Chk_Mem ; M000
40713                      ; MSDOS 3.3
40714                      ;ja      short Exec_No_Mem
40715
40716                      ;test     byte [bp-6],0FFh
40717 00006DAA F646FAFF      test    Exec_Load_High,-1 ; if load high, use max
40718 00006DAE 7518      jnz     short Exec_BX_Max ; use max
40719
40720                      ; 09/09/2018
40721
40722 00006DB0 0306[D10E]      add     AX,[exec_min_BSS] ; go for min allocation;rms;NSS
40723                      ; MSDOS 6.0
40724 00006DB4 7299      jc      short Exec_Chk_Mem ; M000
40725                      ; MSDOS 3.3
40726                      ;jc      short Exec_No_Mem
40727
40728 00006DB6 39D8      cmp     AX,BX           ; enough space?
40729                      ; MSDOS 6.0
40730 00006DB8 7795      ja      short Exec_Chk_Mem ; M000: nope...
40731                      ; MSDOS 3.3
40732                      ;ja      short Exec_No_Mem
40733
40734 00006DBA 2B06[D10E]      sub     AX,[exec_min_BSS] ; rms;NSS
40735 00006DBE 0306[D30E]      add     AX,[exec_max_BSS] ; go for the MAX
40736 00006DC2 7204      jc      short Exec_BX_Max
40737
40738 00006DC4 39D8      cmp     AX,BX
40739 00006DC6 7602      jbe     short Exec_Got_Block
40740
40741 Exec_BX_Max:
40742 00006DC8 89D8      mov     AX,BX
40743
40744 Exec_Got_Block:
40745                      ; 03/08/2018 - Retro DOS v3.0
40746
40747 00006DCA 1E      push    DS
40748 00006DCB 89C3      mov     BX,AX
40749                      ;mov     [bp-16],bx
40750 00006DCD 895EF0      mov     Exec_Size,BX
40751                      ;invoke $Alloc      ; get the space
40752 00006DD0 E83605      call    _$ALLOC
40753 00006DD3 1F      pop     DS
40754                      ; MSDOS 6.0
40755                      ;jc      short Exec_Chk_Mem ; M000
40756                      ; MSDOS 3.3
40757                      ;jc      short Exec_No_Mem
40758                      ; 20/05/2019
40759 00006DD4 7303      jnc     short ea0
40760 00006DD6 E976FF      jmp     Exec_Chk_Mem
40761 ea0:
40762                      ; MSDOS 6.0
40763 00006DD9 8A0E[8400]      mov     cl,[ALLOCMSAVE] ; M063:
40764 00006DDD 880E[0203]      mov     [AllocMethod],cl ; M063: restore allocmethod
40765
40766                      ;M029; Begin changes
40767                      ; This code does special handling for programs with no stack segment. If so,
40768                      ; check if the current block is larger than 64K. If so, we do not modify
40769                      ; Exec_SP. If smaller than 64K, we make Exec_SP = top of block. In either
40770                      ; case Exec_SS is not changed.
40771
40772                      ; MSDOS 6.0
40773                      ;cmp     byte [bp-29],0
40774 00006DE1 807EE300      cmp     Exec_NoStack,0
40775                      ;je      @f
40776 00006DE5 7412      je      short ea2
40777
40778 00006DE7 81FB0010      cmp     bx,1000h        ; Q: >= 64K memory block
40779                      ;jae     @f          ; Y: Exec_SP = 0
40780 00006DEB 730C      jae     short ea2
40781
40782                      ;Make Exec_SP point at the top of the memory block

```

```

40783
40784 00006DED B104      mov     cl,4
40785 00006DEF D3E3      shl     bx,cl          ; get byte offset
40786 00006DF1 81EB0001  sub     bx,100h        ; take care of PSP
40787 00006DF5 891E[D70E] mov     [exec_SP],bx    ; Exec_SP = top of block
40788
40789 ea2:
40790 ;@@:
40791 ;M029; end changes
40792
40793      ;mov     [bp-18],ax
40794 00006DF9 8946EE      mov     Exec_Load_Block,AX
40795 00006DFC 83C010      add     AX,10h
40796      ;test    byte [bp-6],0FFh
40797 00006DFF F646FAFF      test    Exec_Load_High,-1
40798      jz      short Exec_Use_AX    ; use ax for load info
40799
40800      ;add     ax,[bp-16]
40801 00006E05 0346F0      add     AX,Exec_Size    ; go to end
40802      ;sub     ax,[bp-12]
40803 00006E08 2B46F4      sub     AX,Exec_Res_Len_Para ; drop off header
40804 00006E0B 83E810      sub     AX,10h          ; drop off pdb
40805
40806 Exec_Use_AX:
40807      ;mov     [bp-10],ax
40808 00006E0E 8946F6      mov     Exec_Rel_Fac,AX  ; new segment
40809 00006E11 8946EC      ;mov     [bp-20],ax
40810      mov     Exec_DMA,AX ; *+*    ; beginning of dma
40811
40812      ; Determine the location in the file of the beginning of
40813      ; the resident
40814
40815 ; 17/12/2022
40816 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40817 ;%if 0
40818
40819 Exec_Find_Res:
40820      ; MSDOS 6.0
40821      ;mov     dx,[bp-20]
40822      ;mov     DX,Exec_DMA ; *+*
40823      ;mov     [bp-28],dx
40824      ;mov     Exec_DMA_Save,DX
40825
40826      ; 17/12/2022
40827      ; AX = Exec_DMA
40828
40829      ; 02/06/2019 - Retro DOS v4.0
40830 00006E14 8946E4      ;mov     [bp-28],ax ; *+*
40831      mov     Exec_DMA_Save,AX ; *+*
40832
40833 ;%endif
40834
40835 ; 17/12/2022
40836 ;%if 0
40837      ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40838 Exec_Find_Res:
40839      ;mov     dx,[bp-20]
40840      mov     DX,Exec_DMA ; *+*
40841      ;mov     [bp-28],dx
40842      mov     Exec_DMA_Save,DX
40843 ;%endif
40844
40845      ; MSDOS 3.3 (& MSDOS 6.0)
40846 00006E17 8B16[CF0E]      mov     DX,[exec_par_dir]
40847 00006E1B 52          push    DX
40848 00006E1C B104      mov     cl,4
40849 00006E1E D3E2      shl     DX,CL          ; low word of location
40850 00006E20 58          pop     AX
40851 00006E21 B10C      mov     CL,12
40852 00006E23 D3E8      shr     AX,CL          ; high word of location
40853 00006E25 89C1      mov     CX,AX          ; CX <- high
40854
40855      ; Read in the resident image (first, seek to it)
40856 00006E27 8B5EF8      ;mov     bx,[bp-8]
40857 00006E2A 1E          mov     BX,Exec_FH
40858 00006E2B 30C0      push    DS
40859      xor     AL,AL
40860 00006E2D E8A10A      ;invoke $Lseek          ; Seek to resident
40861 00006E30 1F          call    _LSEEK
40862 00006E31 7303      pop     DS
40863      jnc     short Exec_Big_Read
40864 00006E33 E906FF      jmp     Exec_Bomb
40865
40866 Exec_Big_Read:          ; Read resident into memory
40867      ;mov     bx,[bp-12]
40868 00006E36 8B5EF4      mov     BX,Exec_Res_Len_Para
40869 00006E39 81FB0010      cmp     BX,1000h        ; Too many bytes to read?
40870 00006E3D 7203      jnb     short Exec_Read_OK
40871
40872 00006E3F BBE00F      mov     BX,0FE0h        ; Max in one chunk FE00 bytes
40873
40874 Exec_Read_OK:
40875      ;sub     [bp-12],bx
40876 00006E42 295EF4      sub     Exec_Res_Len_Para,BX ; We read (soon) this many
40877 00006E45 53          push    BX
40878 00006E46 B104      mov     CL,4
40879 00006E48 D3E3      shl     BX,CL          ; Get count in bytes from paras
40880 00006E4A 89D9      mov     CX,BX          ; Count in correct register
40881 00006E4C 1E          push    DS
40882      ;mov     ds,[bp-20]
40883 00006E4D 8E5EEC      ;mov     DS,Exec_DMA    ; Set up read buffer
40884
40885 00006E50 31D2      xor     DX,DX
40886 00006E52 51          push    CX
40887 00006E53 E81403      call    ExecRead        ; Save our count
40888 00006E56 59          pop     CX              ; Get old count to verify
40889 00006E57 1F          pop     DS
40890 00006E58 7248      jc      short Exec_Bad_FileJ
40891
40892      cmp     CX,AX        ; Did we read enough?
40893 00006E5C 5B          pop     BX              ; Get paragraph count back
40894 00006E5D 7408      jz      short ExecCheckEnd ; and do reloc if no more to read
40895
40896      ; The read did not match the request. If we are off by 512
40897      ; bytes or more then the header lied and we have an error.
40898
40899 00006E5F 29C1      sub     CX,AX
40900 00006E61 81F90002      cmp     CX,512
40901 00006E65 733B      jae     short Exec_Bad_FileJ
40902
40903      ; we've read in CX bytes... bump DTA location
40904
40905 ExecCheckEnd:
40906      ;add     [bp-20],bx

```

```

40907 00006E67 015EEC      add     Exec_DMA,BX          ; Bump dma address
40908                      ;test    word [bp-12],0FFFFh
40909 00006E6A F746F4FFFF    test    Exec_Res_Len_Para,-1
40910 00006E6F 75C5          jnz     short Exec_Big_Read
40911
40912                      ; The image has now been read in. We must perform relocation
40913                      ; to the current location.
40914
40915      exec_do_reloc:
40916                      ;mov     cx,[bp-10]
40917 00006E71 8B4EF6      mov     CX,Exec_Rel_Fac
40918 00006E74 A1[D50E]      mov     AX,[exec_SS]          ; get initial SS ;rms;NSS
40919 00006E77 01C8          add     AX,CX                ; and relocate him
40920 00006E79 A3[C10E]      mov     [exec_init_SS],AX     ; rms;NSS
40921
40922 00006E7C A1[D70E]      mov     AX,[exec_SP]          ; initial SP ;rms;NSS
40923 00006E7F A3[BF0E]      mov     [exec_init_SP],AX     ; rms;NSS
40924
40925 00006E82 C406[DB0E]    les     AX,[exec_IP]          ; rms;NSS
40926 00006E86 A3[C30E]      mov     [exec_init_IP],AX     ; rms;NSS
40927 00006E89 8CC0          mov     AX,ES                ; rms;NSS
40928 00006E8B 01C8          add     AX,CX                ; relocated...
40929 00006E8D A3[C50E]      mov     [exec_init_CS],AX     ; rms;NSS
40930
40931 00006E90 31C9          xor     CX,CX
40932 00006E92 8B16[DF0E]    mov     DX,[exec_rle_table]   ; rms;NSS
40933                      ;mov     bx,[bp-8]
40934 00006E96 8B5EF8      mov     BX,Exec_FH
40935 00006E99 1E          push    DS
40936 00006E9A 31C0          xor     AX,AX
40937                      ;invoke $Lseek
40938 00006E9C E8320A      call    _$LSEEK
40939 00006E9F 1F          pop     DS
40940 00006EA0 7303          jnc     short exec_get_entries
40941
40942      Exec_Bad_FileJ:
40943 00006EA2 E995FE      jmp     Exec_Bad_File
40944
40945      exec_get_entries:
40946 00006EA5 8B16[CD0E]    mov     DX,[exec_rle_count]   ; Number of entries left ;rms;NSS
40947
40948      exec_read_reloc:
40949 00006EA9 52          push    DX
40950                      ;mov     dx,OPENBUF
40951 00006EAA BA[BE03]    mov     DX,Exec_Internal_Buffer
40952                      ;;mov     cx,388 ; MSDOS 3.3 ; (390>>2)<<2
40953                      ;mov     cx,396 ; MSDOS 6.0 ; PCDOS 7.1 (08/03/2024)
40954 00006EAD B98C01    mov     CX,((Exec_Internal_Buffer_Size)/4)*4 ; (397>>2)<<2
40955 00006EB0 1E          push    DS
40956 00006EB1 E8B602      call    ExecRead
40957 00006EB4 07          pop     ES
40958 00006EB5 5A          pop     DX
40959 00006EB6 72EA          jc      short Exec_Bad_FileJ
40960
40961                      ;;mov     cx,97 ; MSDOS 3.3 ; (390>>2)
40962                      ;mov     cx,99 ; MSDOS 6.0 ; PCDOS 7.1 (08/03/2024)
40963 00006EB8 B96300    mov     CX,(Exec_Internal_Buffer_Size)/4 ; (397>>2)
40964                      ; Pointer to byte location in header
40965                      ;mov     di,OPENBUF
40966 00006EBB BF[BE03]    mov     DI,Exec_Internal_Buffer
40967                      ;mov     si,[bp-10]
40968 00006EBE 8B76F6      mov     SI,Exec_Rel_Fac      ; Relocate a single address
40969
40970      exec_reloc_one:
40971 00006EC1 09D2      or      DX,DX                ; Any more entries?
40972 00006EC3 7416          jz      short Exec_Set_PDBJ
40973
40974      exec_get_addr:
40975 00006EC5 26C51D      lds     BX,[ES:DI]           ; Get ra/sa of entry
40976 00006EC8 8CD8          mov     AX,DS                ; Relocate address of item
40977
40978                      ; MSDOS 6.0
40979                      ;;; add     AX,SI ; MSDOS 3.3
40980                      ;add     ax,[bp-28]
40981 00006ECA 0346E4      add     AX,Exec_DMA_Save
40982
40983 00006ECD 8ED8          mov     DS,AX
40984 00006ECF 0137          add     [BX],SI
40985 00006ED1 83C704      add     DI,4
40986 00006ED4 4A          dec     DX
40987 00006ED5 E2EA          loop   exec_reloc_one       ; End of internal buffer?
40988
40989                      ; We've exhausted a single buffer's worth. Read in the next
40990                      ; piece of the relocation table.
40991
40992 00006ED7 06          push    ES
40993 00006ED8 1F          pop     DS
40994 00006ED9 EBCE          jmp     short exec_read_reloc
40995
40996      Exec_Set_PDBJ:
40997                      ; MSDOS 6.0
40998
40999                      ; We now determine if this is a buggy exe packed file and if
41000                      ; so we patch in the right code. Note that fixexepatch will
41001                      ; point to a ret if dos loads low. The load segment as
41002                      ; determined above will be in exec_dma_save
41003
41004 00006EDB 06          push    es
41005 00006EDC 50          push    ax                  ; M030
41006 00006EDD 51          push    cx                  ; M030
41007                      ;mov     es,[bp-28]
41008 00006EDE 8E46E4      mov     es,Exec_DMA_Save
41009 00006EE1 36A1[C50E]    mov     ax,[ss:exec_init_CS] ; M030
41010 00006EE5 368B0E[C30E]  mov     cx,[ss:exec_init_IP] ; M030
41011 00006EEA 36FF16[670D]  call    word [ss:FixExePatch]
41012
41013                      ; 30/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
41014                      ; (MSDOS 5.0 MSDOS.SYS does not contain 'Rational386Patch')
41015                      ;call    word [ss:Rational386PatchPtr]
41016
41017                      ; 08/03/2024 - Retro DOS v5.0
41018                      ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:9E6Fh)
41019                      ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:0B096h)
41020 00006EEF 36FF16[3E13]  call    word [ss:Rational386PatchPtr]
41021
41022                      ; 08/03/2024
41023                      ; (Windows ME IO.SYS - BIOSCODE:976Fh)
41024                      ;call    Rational386Patch
41025
41026 00006EF4 59          pop     cx                  ; M030
41027 00006EF5 58          pop     ax                  ; M030
41028 00006EF6 07          pop     es
41029
41030 00006EF7 E9DD00      jmp     Exec_Set_PDB

```

```

41031
41032
41033 00006EFA E939FE      Exec_No_Memj:
41034                      jmp      Exec_No_Mem
41035
41036                      ; we have a .COM file. First, determine if we are merely
41037                      ; loading an overlay.
41038
41039      Exec_Com_File:
41040 00006EFD F646FB02      ;test    byte [bp-5],2
41041 00006F01 742D          test    Exec_Func,exec_func_overlay
41042                      jz      short Exec_Alloc_Com_File
41043 00006F03 C576FC          ;lds     si,[bp-4]
41044 00006F06 AD            lds     SI,Exec_Blk          ; get arg block
41045                      lodsw                     ; get load address
41046 00006F07 8946EC          ;mov     [bp-20],ax
41047 00006F0A B8FFFF          mov     Exec_DMA,AX
41048 00006F0D EB63          mov     AX,0FFFFh
41049                      jmp     short Exec_Read_Block ; read it all!
41050
41051      Exec_Chk_Com_Mem:
41052 00006F0F 36A0[0203]      ; MSDOS 6.0          ; M063 - Start
41053 00006F13 368A1E[8400]    mov     al,[ss:AllocMethod] ; save current alloc method in ax
41054 00006F18 36881E[0203]    mov     b1,[ss:ALLOCMSAVE]
41055 00006F1D F6C340          mov     [ss:AllocMethod],b1 ; restore original allocmethod
41056 00006F20 75D8          test    b1,HIGH_ONLY ; 40h ; Q: was the HIGH_ONLY bit already set
41057                      jnz     short Exec_No_Memj ; Y: no space in UMBS. Quit
41058                      ; N: continue
41059 00006F22 A840          test    al,HIGH_ONLY ; Q: did we set the HIGH_ONLY bit
41060 00006F24 74D4          jz      short Exec_No_Memj ; N: no memory
41061
41062                      ;mov     ax,[bp-18]
41063 00006F26 8B46EE          mov     ax,Exec_Load_Block ; M047: ax = block we just allocated
41064 00006F29 31DB          xor     bx,bx ; M047: bx => free arena
41065 00006F2B E87102          call    ChangeOwner ; M047: free this block
41066
41067 00006F2E EB0E          jmp     short Exec_Norm_Com_Alloc
41068                      ; M063 - End
41069
41070                      ; we must allocate the max possible size block (ick!)
41071                      ; and set up CS=DS=ES=SS=PDB pointer, IP=100, SP=max
41072                      ; size of block.
41073
41074      Exec_Alloc_Com_File:
41075 00006F30 36F606[0203]80    ; MSDOS 6.0          ; M000 -start
41076                      test    byte [ss:AllocMethod],HIGH_FIRST ; 80h
41077                      ; Q: is the alloc strat high_first
41078 00006F36 7406          jz      short Exec_Norm_Com_Alloc ; N: normal allocate
41079                      ; Y: set high_only bit
41080 00006F38 36800E[0203]40    or      byte [ss:AllocMethod],HIGH_ONLY ; 40h
41081                      ; M000 - end
41082                      ; M000
41083
41084 00006F3E BBFFFF          ; MSDOS 3.3 (& MSDOS 6.0)
41085                      mov     BX,0FFFFh
41086 00006F41 E8C503          ;invoke $Alloc ; largest piece available as error
41087 00006F44 09DB          call    _$ALLOC
41088                      or      BX,BX
41089 00006F46 74C7          ; MSDOS 6.0
41090                      jz      short Exec_Chk_Com_Mem; M000
41091                      ; MSDOS 3.3
41092                      jz      short Exec_No_Memj
41093
41094 00006F48 895EF0          ;mov     [bp-16],bx
41095 00006F4B 53            mov     Exec_Size,BX ; save size of allocation block
41096                      push    BX
41097 00006F4C E8BA03          ;invoke $ALLOC ; largest piece available
41098 00006F4F 5B            call    _$ALLOC
41099                      pop     BX ; get size of block...
41100 00006F50 8946EE          mov     [bp-18],ax
41101                      mov     Exec_Load_Block,AX
41102 00006F53 83C010          add     AX,10h ; increment for header
41103                      ;mov     [bp-20],ax
41104 00006F56 8946EC          mov     Exec_DMA,AX
41105
41106 00006F59 31C0          xor     AX,AX ; presume 64k read...
41107 00006F5B 81FB0010        cmp     BX,1000h ; 64k or more in block?
41108 00006F5F 730E          jae     short Exec_Read_Com ; yes, read only 64k
41109
41110 00006F61 89D8          mov     AX,BX ; convert size to bytes
41111 00006F63 B104          mov     CL,4
41112 00006F65 D3E0          shl     AX,CL
41113                      ; 17/12/2022
41114                      ; 30/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
41115                      ; (MSDOS 5.0, MSDOS.SYS compatibility)
41116                      ; MSDOS 5.0
41117                      ;cmp     AX,100h
41118                      ; 02/06/2019 - Retro DOS v4.0
41119                      ; MSDOS 6.0
41120                      ; 17/12/2022
41121 00006F67 3D0002          cmp     AX,200h ; enough memory for PSP and stack?
41122 00006F6A 76A3          jbe     short Exec_Chk_Com_Mem; M000: jump if not
41123                      ;jbe     short Exec_No_Memj ; M000: jump if not
41124                      ;; Retro DOS v3.0 modification (on MSDOS 6.0 code) -03/08/2018-
41125                      ;;jbe     short Exec_Chk_Com_Mem; M000: jump if not
41126                      ;jbe     short Exec_No_Memj ; M000: jump if not
41127
41128                      ; M047: size of the block is < 64K
41129 00006F6C 2D0001          sub     ax,100h ; M047: reserve 256 bytes for stack
41130
41131      Exec_Read_Com:
41132 00006F6F 2D0001          ; MSDOS 3.3 (& MSDOS 6.0)
41133                      sub     AX,100h ; remember size of psp
41134
41135      Exec_Read_Block:
41136 00006F72 50            push    AX ; save number to read
41137 00006F73 8B5EF8          ;mov     bx,[bp-8]
41138 00006F76 31C9          mov     BX,Exec_FH ; of com file
41139 00006F78 31C0          xor     CX,CX ; but seek to 0:0
41140                      xor     AX,AX ; seek relative to beginning
41141                      ;mov     DX,CX
41142 00006F7A 99            ; 08/03/2024
41143                      cwd
41144 00006F7B E85309          ;invoke $Lseek ; back to beginning of file
41145 00006F7E 59            call    _$LSEEK
41146                      pop     CX ; number to read
41147 00006F7F 8E5EEC          ;mov     ds,[bp-20]
41148 00006F82 31D2          mov     DS,Exec_DMA
41149 00006F84 51            xor     DX,DX
41150 00006F85 E8E201          push    CX
41151 00006F88 5E            call    ExecRead
41152 00006F89 7303          pop     SI ; get number of bytes to read
41153 00006F8B E9ACFD          jnc     short OkRead
41154                      jmp     Exec_Bad_File

```

```

41155 ; 10/09/2018
41156 okRead:
41157 00006F8E 39F0      cmp     AX,SI          ; did we read them all?
41158 ; MSDOS 6.0
41159 ;jz     short Exec_Chk_Com_Mem; M00: exactly the wrong number...no
41160 ; MSDOS 3.3
41161 ;jz     short Exec_No_Memj      ; M00: exactly the wrong number...
41162 00006F90 7503      jne     short OkRead2
41163 00006F92 E97AFF      jmp     Exec_Chk_Com_Mem
41164
41165 okRead2:
41166 ; MSDOS 6.0
41167 00006F95 368A1E[8400] mov     b1,[ss:ALLOCMSAVE] ; M063
41168 00006F9A 36881E[0203] mov     [ss:AllocMethod],b1 ; M063: restore alloc method
41169
41170 ; MSDOS 3.3 (& MSDOS 6.0)
41171 ;test   byte [bp-5],2
41172 00006F9F F646FB02 test    Exec_Func,exec_func_overlay
41173 00006FA3 7532      jnz     short Exec_Set_PDB ; no starto, chumo!
41174
41175 ;mov     ax,[bp-20]
41176 00006FA5 8B46EC      mov     AX,Exec_DMA
41177 00006FA8 83E810      sub     AX,10h
41178 00006FAB 36A3[C50E]     mov     [SS:exec_init_CS],AX
41179 00006FAF 36C706[C30E]0001 mov     word [SS:exec_init_IP],100h ; initial IP is 100h
41180
41181 ; SI is AT MOST FF00h. Add FE to account for PSP - word
41182 ; of 0 on stack.
41183
41184 00006FB6 81C6FE00      add     SI,0FEh          ; make room for stack
41185
41186 ; MSDOS 6.0
41187 00006FBA 83FEFE      cmp     si,0FFFEh        ; M047: Q: was there >= 64K available
41188 00006FBD 7404      je      short Exec_St_Ok  ; M047: Y: stack is fine
41189 00006FBF 81C60001      add     si,100h          ; M047: N: add the xtra 100h for stack
41190
41191 Exec_St_Ok:
41192 ; MSDOS 3.3 (& MSDOS 6.0)
41193 00006FC3 368936[BFOE]     mov     [SS:exec_init_SP],SI ; max value for read is also SP!;smr;SS Override
41194 00006FC8 36A3[C10E]     mov     [SS:exec_init_SS],AX ;smr;SS Override
41195 00006FCC 8ED8      mov     DS,AX
41196 00006FCE C7040000      mov     WORD [SI],0        ; 0 for return
41197
41198 ; MSDOS 6.0
41199
41200 ; M068
41201 ;
41202 ; We now determine if this is a Copy Protected App. If so the
41203 ; A200FF_COUNT is set to 6. Note that ChkCopyProt will point to
41204 ; a ret if DOS is loaded low. Also DS contains the load segment.
41205
41206 00006FD2 36FF16[6100]     call    word [ss:ChkCopyProt]
41207
41208 Exec_Set_PDB:
41209 ; MSDOS 3.3 (& MSDOS 6.0)
41210 ;mov     bx,[bp-8]
41211 00006FD7 8B5EF8      mov     BX,Exec_FH        ; we are finished with the file.
41212 00006FDA E8A601      call    Exec_Dealloc
41213 00006FDD 55      push    BP
41214 ;invoke $Close
41215 00006FDE E88707      call    _$CLOSE           ; release the jfn
41216 00006FE1 5D      pop     BP
41217 00006FE2 E89001      call    Exec_Alloc
41218 ;test   byte [bp-5],2
41219 00006FE5 F646FB02 test    Exec_Func,exec_func_overlay
41220 00006FE9 743A      jz      short Exec_Build_Header
41221
41222 ; MSDOS 6.0
41223 00006FEB E8BF01      call    Scan_Execname
41224 00006FEE E8D301      call    Scan_Special_Entries
41225
41226 ;SR;
41227 ;The current lie strategy uses the PSP to store the lie version. However,
41228 ;device drivers are loaded as overlays and have no PSP. To handle them, we
41229 ;use the Sysinit flag provided by the BIOS as part of a structure pointed at
41230 ;by BiosDataPtr. If this flag is set, the overlay call has been issued from
41231 ;Sysinit and therefore must be a device driver load. We then get the lie
41232 ;version for this driver and put it into the Sysinit PSP. When the driver
41233 ;issues the version check, it gets the lie version until the next overlay
41234 ;call is issued.
41235 00006FF1 36803E[2213]00      cmp     byte [ss:DriverLoad],0;was Sysinit processing done?
41236 00006FF7 7426      je      short norm_ovl    ;yes, no special handling
41237 00006FF9 56      push    si
41238 00006FFA 06      push    es
41239 00006FFB 36C436[2313]     les     si,[ss:BiosDataPtr] ;get ptr to BIOS data block
41240
41241 ; (es:si points to 'SysinitPresent' address/flag in retrodos4.s)
41242 00007000 26803C00      cmp     byte [es:si],0    ;in Sysinit?
41243 00007004 7411      je      short sysinit_done ;no, Sysinit is finished
41244
41245 00007006 368E06[3003]     mov     es,[ss:CurrentPDB] ;es = current PSP (Sysinit PSP)
41246 0000700B 36FF36[BB0E]     push    word [ss:SPECIAL_VERSION]
41247 ;pop     word [es:40h] ; 08/03/2024
41248 00007010 268F064000      pop     word [es:PDB.Version] ;store lie version in Sysinit PSP
41249 ;;; PDB.VERSION
41250 00007015 EB06      jmp     short setver_done
41251 sysinit_done:
41252 00007017 36C606[2213]00      mov     byte [ss:DriverLoad],0;Sysinit done,special handling off
41253
41254 0000701D 07      setver_done:
41255 0000701E 5E      pop     es
41256 ;pop     si
41257 norm_ovl:
41258 ;leave
41259 0000701F 89EC      mov     sp,bp
41260 00007021 5D      pop     bp
41261
41262 00007022 E94996      ;transfer SYS_RET_OK
41263 jmp     SYS_RET_OK ; overlay load -> done
41264
41265 Exec_Build_Header:
41266 ;mov     dx,[bp-18]
41267 mov     DX,Exec_Load_Block
41268 ; assign the space to the process
41269 ;mov     si,1
41270 mov     SI,ARENA.OWNER ; pointer to owner field
41271 ;mov     ax,[bp-14]
41272 mov     AX,Exec_Environ ; get environ pointer
41273 or      AX,AX
41274 jz      short No_Owner ; no environment
41275
41276 dec     AX ; point to header
41277 mov     DS,AX
41278 mov     [SI],DX ; assign ownership
No_Owner:

```

```

41279      ;mov     ax,[bp-18]
41280      ;mov     AX,Exec_Load_Block      ; get load block pointer
41281      ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
41282      ; 17/12/2022
41283 00007037 89D0      mov     ax,dx ; 06/06/2019
41284      ;mov     ax,Exec_Load_Block      ; get load block pointer
41285
41286      dec     AX
41287      mov     DS,AX      ; point to header
41288      mov     [SI],DX     ; assign ownership
41289
41290      ; MSDOS 6.0
41291 0000703E 1E      push    DS      ;AN000;MS. make ES=DS
41292 0000703F 07      pop     ES      ;AN000;MS.
41293      ;mov     di,8
41294 00007040 BF0800    mov     DI,ARENA.NAME ;AN000;MS. ES:DI points to destination
41295 00007043 E86701    call    Scan_Execname ;AN007;MS. parse execname
41296      ;                                ; ds:si->name, cx=name length
41297 00007046 51      push    CX      ;AN007;MS. save for fake version
41298 00007047 56      push    SI      ;AN007;MS. save for fake version
41299
41300      MoveName:      ;AN000;
41301 00007048 AC      lodsb         ;AN000;MS. get char
41302 00007049 3C2E      cmp     AL,'.'   ;AN000;MS. is '.', may be name.exe
41303 0000704B 7408      jz      short Mem_Done ;AN000;MS. no, move to header
41304      ;AN000;
41305 0000704D AA      stosb         ;AN000;MS. move char
41306      ; MSKK bug fix - limit length copied
41307 0000704E 83FF10    cmp     di,16 ; ARENAHEADERSIZE ; end of memory arena block?
41308 00007051 7302      jae     short Mem_Done ; jump if so
41309      ;
41310 00007053 E2F3      loop    MoveName ;AN000;MS. continue
41311      Mem_Done:      ;AN000;
41312 00007055 30C0      xor     AL,AL     ;AN000;MS. make ASCIIZ
41313      ;cmp     di,16
41314 00007057 83FF10    cmp     DI,ARENAHEADERSIZE ; 16 ;AN000;MS. if not all filled
41315 0000705A 7301      jae     short Fill8 ;AN000;MS.
41316
41317 0000705C AA      stosb         ;AN000;MS.
41318
41319      Fill8:          ;AN000;
41320 0000705D 5E      pop     SI      ;AN007;MS. ds:si -> file name
41321 0000705E 59      pop     CX      ;AN007;MS.
41322
41323 0000705F E86201    call    Scan_Special_Entries ;AN007;MS.
41324
41325      ; MSDOS 3.3 (& MSDOS 6.0)
41326 00007062 52      push    DX
41327      ;mov     si,[bp-16]
41328 00007063 8B76F0    mov     SI,Exec_Size
41329 00007066 01D6      add     SI,DX
41330      ;Invoke $Dup_PDB      ; ES is now PDB
41331 00007068 E8D3A5    call    _$DUP_PDB
41332 0000706B 5A      pop     DX
41333
41334      ;push    word [bp-14]
41335      push    Exec_Environ
41336      ;pop     word [ES:2Ch]
41337 0000706F 268F062C00 pop     word [ES:PDB.ENVIRON]
41338
41339      ; MSDOS 6.0      ; *** Added for DOS 5.00
41340      ;                                ; version number in PSP
41341 00007074 36FF36[BB0E] push    word [ss:SPECIAL_VERSION] ; Set the DOS version number to
41342 00007079 268F064000 pop     word [ES:PDB.Version] ; to be used for this application
41343      ; PDB.VERSION
41344
41345      ; MSDOS 3.3 (& MSDOS 6.0)      ; set up proper command line stuff
41346      ;lds     si,[bp-4]
41347 0000707E C576FC    lds     SI,Exec_Blk      ; get the block
41348 00007081 1E      push    DS      ; save its location
41349 00007082 56      push    SI
41350      ;lds     si,[si+6]
41351 00007083 C57406    lds     SI,[SI+EXEC0.5C_FCB] ; get the 5c fcb
41352
41353      ; DS points to user space 5C FCB
41354
41355 00007086 B90C00    mov     CX,12      ; copy drive, name and ext
41356 00007089 51      push    CX
41357 0000708A BF5C00    mov     DI,5Ch
41358 0000708D 8A1C      mov     BL,[SI]
41359 0000708F F3A4      rep     movsb
41360
41361      ; DI = 5Ch + 12 = 5Ch + 0Ch = 68h
41362
41363      ;xor     AX,AX      ; zero extent, etc for CPM
41364 00007091 91      xchg    ax,cx      ; 08/03/2024
41365 00007092 AB      stosw
41366 00007093 AB      stosw
41367
41368      ; DI = 5Ch + 12 + 4 = 5Ch + 10h = 6Ch
41369
41370 00007094 59      pop     CX
41371 00007095 5E      pop     SI      ; get block
41372 00007096 1F      pop     DS
41373 00007097 1E      push    DS      ; save (again)
41374 00007098 56      push    SI
41375      ;lds     si,[si+0Ah]
41376 00007099 C5740A    lds     SI,[SI+EXEC0.6C_FCB] ; get 6C FCB
41377
41378      ; DS points to user space 6C FCB
41379
41380 0000709C 8A3C      mov     BH,[SI]      ; do same as above
41381 0000709E F3A4      rep     movsb
41382 000070A0 AB      stosw
41383 000070A1 AB      stosw
41384 000070A2 5E      pop     SI      ; get block (last time)
41385 000070A3 1F      pop     DS
41386      ;ld     si,[si+2]
41387 000070A4 C57402    lds     SI,[SI+EXEC0.COM_LINE]; command line
41388
41389      ; DS points to user space 80 command line
41390
41391 000070A7 80C980    or      CL,80h
41392 000070AA 89CF      mov     DI,CX
41393 000070AC F3A4      rep     movsb      ; wham!
41394
41395      ; Process BX into default AX (validity of drive specs on args).
41396      ; We no longer care about DS:SI.
41397
41398 000070AE FEC9      dec     CL      ; get 0FFh in CL
41399 000070B0 88F8      mov     AL,BH
41400 000070B2 30FF      xor     BH,BH
41401      ;invoke GetVisDrv
41402 000070B4 E8B50A    call    GetVisDrv

```



```

41403 000070B7 7302          jnc     short Exec_BL
41404
41405 000070B9 88CF          mov     BH,CL
41406
41407 Exec_BL:
41408 000070BB 88D8          mov     AL,BL
41409 000070BD 30DB          xor     BL,BL
41410          ; invoke GetVisDrv
41411 000070BF E8AA0A        call    GetVisDrv
41412 000070C2 7302          jnc     short Exec_Set_Return
41413
41414 000070C4 88CB          mov     BL,CL
41415
41416 Exec_Set_Return:
41417          ; invoke Get_User_Stack          ; get his return address
41418 000070C6 E8AE93        call    Get_User_Stack
41419
41420          ; 08/03/2024
41421 %if 0
41422          ; push word [si+14h]
41423          push word [SI+user_env.user_CS] ; suck out the CS and IP
41424          ; push word [si+12h]
41425          push word [SI+user_env.user_IP]
41426          ; push word [si+14h]
41427          push word [SI+user_env.user_CS] ; suck out the CS and IP
41428          ; push word [si+12h]
41429          push word [SI+user_env.user_IP]
41430          ; pop word [ES:0Ah]
41431          pop word [ES:PDB.EXIT]
41432          ; pop word [ES:0Ch]
41433          pop word [ES:PDB.EXIT+2]
41434 %else
41435          ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
41436          ;;;
41437          ; lds ax,[si+12h]
41438 000070C9 C54412        lds     ax,[SI+user_env.user_IP] ; suck out the CS and IP
41439 000070CC 1E          push    ds
41440 000070CD 50          push    ax
41441          ; mov [es:0Ah],ax
41442 000070CE 26A30A00        mov     [ES:PDB.EXIT],ax
41443          ; mov [es:0Ch],ds
41444 000070D2 268C1E0C00    mov     [ES:PDB.EXIT+2],ds
41445          ;;;
41446 %endif
41447
41448 000070D7 31C0          xor     AX,AX
41449 000070D9 8ED8          mov     DS,AX
41450          ; save them where we can get them
41451          ; later when the child exits.
41452
41453 000070DB 8F068800        ; pop word [addr_int_terminate] ; 22h*4
41454          ; pop word [90h]
41455 000070DF 8F068A00        ; pop word [addr_int_terminate+2] ; (22h*4)+2
41456
41457 000070E3 36C706[2C03]8000    mov     WORD [SS:DMAADD],80h ; SS Override
41458 000070EA 368E1E[3003]        mov     DS,[SS:CurrentPDB] ; SS Override
41459 000070EF 368C1E[2E03]        mov     [SS:DMAADD+2],DS ; SS Override
41460
41461          ; test byte [bp-5],1
41462 000070F4 F646FB01        test    Exec_Func,exec_func_no_execute
41463 000070F8 7427          jz      short exec_go
41464
41465 000070FA 36C536[BFOE]        lds     SI,[SS:exec_init_SP] ; get stack SS Override
41466          ; les di,[bp-4]
41467 000070FF C47EFC          les     DI,Exec_Blk ; and block for return
41468          ; mov [es:di+10h],ds
41469 00007102 268C5D10        mov     [ES:DI+EXEC1.SS],DS ; return SS
41470
41471 00007106 4E          dec     SI ; 'push' default AX
41472 00007107 4E          dec     SI
41473 00007108 891C          mov     [SI],BX ; save default AX reg
41474          ; mov [es:di+0Eh], si
41475 0000710A 2689750E        mov     [ES:DI+EXEC1.SP],SI ; return 'SP'
41476
41477 0000710E 36C506[C30E]        lds     AX,[SS:exec_init_IP] ; SS Override
41478          ; mov [es:di+14h],ds
41479 00007113 268C5D14        mov     [ES:DI+EXEC1.CS],DS ; initial entry stuff
41480          ; mov [es:di+12h],ax
41481 00007117 26894512        mov     [ES:DI+EXEC1.IP],AX
41482
41483          ; leave
41484 0000711B 89EC          mov     sp,bp
41485 0000711D 5D          pop     bp
41486
41487          ; transfer SYS_RET_OK
41488 0000711E E94D95        jmp     SYS_RET_OK
41489
41490 exec_go:
41491 00007121 36C536[C30E]        lds     SI,[SS:exec_init_IP] ; get entry point SS Override
41492 00007126 36C43E[BFOE]        les     DI,[SS:exec_init_SP] ; new stack SS Override
41493 0000712B 8CC0          mov     AX,ES
41494
41495          ; MSDOS 6.0
41496 0000712D 36803E[660D]00    cmp     byte [SS:DosHashMA],0 ; Q: is dos in HMA (M021)
41497 00007133 741A          je      short Xfer_To_User ; N: transfer control to user
41498
41499 00007135 1E          push    ds ; Y: control must go to low mem stub
41500
41501 00007136 2E8E1E[0700]        mov     ds,[cs:DosDSeg] ; where we disable a20 and xfer
41502          ; control to user
41503 0000713B 800E[8600]04    or      byte [DOS_FLAG],EXECA200FF ; M068:
41504          ; M004: Set bit to signal int 21
41505          ; ah = 25 & ah= 49. See dossym.inc
41506          ; under TAG M003 & M009 for
41507          ; explanation
41508 00007140 8916[6300]        mov     [A200FF_PSP],dx ; M068: set the PSP for which A20 is
41509          ; M068: going to be turned OFF.
41510
41511 00007144 8CD8          mov     ax,ds ; ax = segment of low mem stub
41512 00007146 1F          pop     ds
41513
41514 00007147 50          push    ax ; ret far into the low mem stub
41515 00007148 B8[2410]        mov     ax,disa20_xfer
41516 0000714B 50          push    ax
41517 0000714C 8CC0          mov     AX,ES ; restore ax
41518 0000714E CB          retf
41519
41520 Xfer_To_User:
41521          ; DS:SI points to entry point
41522          ; AX:DI points to initial stack
41523          ; DX has PDB pointer
41524          ; BX has initial AX value
41525
41526 0000714F FA          cli

```

```

41527      ; 15/08/2018
41528 00007150 36C606[2103]00      mov     BYTE [SS:INDOS],0      ; SS Override
41529      ; 01/07/2024
41530 00007156 36C606[B812]00      mov     byte [ss:INDOS_FLAG],0
41531
41532 0000715C 8ED0                mov     SS,AX                    ; set up user's stack
41533 0000715E 89FC                mov     SP,DI                    ; and SP
41534 00007160 FB
41535
41536 00007161 1E                push    DS                        ; fake long call to entry
41537 00007162 56                push    SI
41538 00007163 8EC2                mov     ES,DX                    ; set up proper seg registers
41539 00007165 8EDA                mov     DS,DX
41540 00007167 89D8                mov     AX,BX                    ; set up proper AX
41541
41542 00007169 CB                retf
41543
41544      ; 04/08/2018 - Retro DOS v3.0
41545
41546      ;-----
41547      ;
41548      ;-----
41549
41550      ExecRead:
41551 0000716A E81600              CALL     Exec_Dealloc
41552      ;mov     bx,[bp-8]
41553 0000716D 8B5EF8              MOV      bx,Exec_FH
41554
41555 00007170 55                PUSH     BP
41556 00007171 E8FB06              call     _$READ
41557 00007174 5D                POP      BP
41558
41559      ;CALL     Exec_Alloc
41560      ;retn
41561      ; 18/12/2022
41562      ;jmp     short Exec_Alloc
41563
41564      ; 18/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
41565
41566      ;-----
41567      ;
41568      ;-----
41569
41570      Exec_Alloc:
41571 00007175 53                push     BX
41572      ;mov     BX,[CS:CurrentPDB] ; MSDOS 3.3
41573      ; 20/05/2019 - Retro DOS v4.0
41574      ; MSDOS 6.0
41575 00007176 368B1E[3003]          mov     bx,[SS:CurrentPDB] ; SS Override
41576 0000717B E81000              call     ChangeOwners
41577 0000717E E88EA7              call     LCritMEM
41578 00007181 5B                pop      BX
41579 00007182 C3                retn
41580
41581      ;-----
41582      ;
41583      ;-----
41584
41585      Exec_Dealloc:
41586 00007183 53                push     BX
41587      ;mov     bx,0
41588 00007184 29DB                sub      BX,BX                ; (bx) = ARENA_OWNER_SYSTEM
41589 00007186 E859A7              call     ECritMEM
41590 00007189 E80200              call     ChangeOwners
41591 0000718C 5B                pop      BX
41592 0000718D C3                retn
41593
41594      ; 18/12/2022
41595      %if 0
41596      ;-----
41597      ;
41598      ;-----
41599
41600      Exec_Alloc:
41601      push     BX
41602      ;mov     BX,[CS:CurrentPDB] ; MSDOS 3.3
41603      ; 20/05/2019 - Retro DOS v4.0
41604      ; MSDOS 6.0
41605      mov     bx,[SS:CurrentPDB] ; SS Override
41606      call     ChangeOwners
41607      call     LCritMEM
41608      pop      BX
41609      retn
41610
41611      %endif
41612
41613      ;-----
41614      ;
41615      ;-----
41616
41617      ChangeOwners:
41618 0000718E 9C                pushf
41619 0000718F 50                push     AX
41620      ;mov     ax,[bp-14]
41621 00007190 8B46F2              mov     AX,Exec_Environ
41622 00007193 E80900              call     ChangeOwner
41623      ;mov     ax,[bp-18]
41624 00007196 8B46EE              mov     AX,Exec_Load_Block
41625 00007199 E80300              call     ChangeOwner
41626 0000719C 58                pop      AX
41627 0000719D 9D                popf
41628      chgown_retn:
41629 0000719E C3                retn
41630
41631      ;-----
41632      ;
41633      ;-----
41634
41635      ChangeOwner:
41636 0000719F 09C0              or       AX,AX                ; is area allocated?
41637 000071A1 74FB              jz       short chgown_retn    ; no, do nothing
41638 000071A3 48                dec      AX
41639 000071A4 1E                push     DS
41640 000071A5 8ED8                mov     DS,AX
41641 000071A7 891E0100          mov     [ARENA.OWNER],BX
41642 000071AB 1F                pop      DS
41643 000071AC C3                retn
41644
41645      ;-----
41646      ;
41647      ;-----
41648
41649      ; 20/05/2019 - Retro DOS v4.0
41650

```



```

41651             ; MSDOS 6.0
41652 Scan_Execname:
41653             ;lds si,[bp-26] ; 08/03/2024
41654 000071AD C576E6             lds SI,ExecName ; DS:SI points to name
41655             ; M028
41656 Scan_Execname1:
41657 000071B0 89F1             Save_Begin:
41658             mov CX,SI ; CX= starting addr
41659 000071B2 AC             Scan0:
41660             lodsb ; get char
41661 000071B3 3C3A             cmp AL,':' ; is ':', may be A:name
41662 000071B5 74F9             jz short Save_Begin ; yes, save si
41663 000071B7 3C5C             cmp AL,'\' ; is '\', may be A:\name
41664 000071B9 74F5             jz short Save_Begin ; yes, save si
41665 000071BB 3C00             cmp AL,0 ; is end of name
41666 000071BD 75F3             jnz short Scan0 ; no, continue scanning
41667 000071BF 29CE             sub SI,CX ; get name's length
41668 000071C1 87F1             xchg SI,CX ; cx= length, si= starting addr
41669
41670 000071C3 C3             retn
41671
41672 ;-----
41673 ;
41674 ;-----
41675
41676 ; 20/05/2019 - Retro DOS v4.0
41677
41678 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
41679 ; DOSCODE:A0EDh (MSDOS 5.0, MSDOS.SYS)
41680
41681 ; 09/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
41682 ; DOSCODE:B370h (PCDOS 7.1, IBMDOS.COM)
41683
41684             ; MSDOS 6.0
41685
41686 Scan_Special_Entries:
41687
41688 000071C4 49             dec CX ; cx= name length
41689 ;M060 mov DI,[Special_Entries] ; es:di -> addr of special entries
41690             ;reset to current version
41691             ;mov word [ss:SPECIAL_VERSION],1406h
41692             ; (MSDOS 6.21, MSDOS.SYS, DOSCODE:A14Eh)
41693             ;mov word [ss:SPECIAL_VERSION],5
41694             ; (MSDOS 5.0, MSDOS.SYS, DOSCODE:A0EEh)
41695
41696             ; 5 for Retro DOS 4.0 (01/12/2022, MSDOS 5.0)
41697 000071C5 36C706[BB0E]070A mov word [ss:SPECIAL_VERSION],(MINOR_VERSION<<8)+MAJOR_VERSION
41698             ; 0005h for Retro DOS v4.1 (MSDOS 5.0)
41699             ; 24/09/2023
41700             ; 1606h for Retro DOS v4.2 (MSDOS 6.22)
41701
41702             ; 09/03/2024
41703             ; 0A07h for Retro DOS v5.0 (PCDOS 7.1)
41704             ; (0008h for Windows ME)
41705
41706 ;***call Reset_Version
41707
41708 ;M060 push SS
41709 ;M060 pop ES
41710
41711 000071CC 36C43E[5D00] les DI,[SS:UU_IFS_DOS_CALL] ;M060; ES:DI --> Table in SETVER.SYS
41712 000071D1 8CC0 mov AX,ES ;M060; First do a NULL ptr check to
41713 000071D3 09F8 or AX,DI ;M060; be sure the table exists
41714 000071D5 7427 jz short End_List ;M060; if ZR then no table
41715
41716 GetEntries:
41717 000071D7 268A05 mov AL,[ES:DI] ; end of list
41718 000071DA 08C0 or AL,AL
41719 000071DC 7420 jz short End_List ; yes
41720
41721 000071DE 36893E[0E06] mov [ss:TEMP_VAR2],DI ; save di
41722 000071E3 38C8 cmp AL,CL ; same length ?
41723 000071E5 751B jnz short SkipOne ; no
41724
41725 000071E7 47 inc DI ; es:di -> special name
41726 000071E8 51 push CX ; save length and name addr
41727 000071E9 56 push SI
41728
41729 ; M050 - BEGIN
41730
41731 000071EA 50 push ax ; save len
41732
41733 sse_next_char:
41734 000071EB AC lodsb
41735 000071EC E8A4EB call UCASE
41736 000071F0 750D jne short Not_Matched
41737 000071F2 E2F7 loop sse_next_char
41738
41739 ; repz cmpsb ; same name ?
41740 ; jnz short Not_Matched ; no
41741
41742 ; 09/03/2024 - PCDOS 7.1 IBMDOS.COM
41743 ;pop ax ; take len off the stack
41744
41745 ; M050 - END
41746
41747 000071F4 268B05 mov AX,[ES:DI] ; get special version
41748 000071F7 36A3[BB0E] mov [ss:SPECIAL_VERSION],AX ; save it
41749
41750 ;***mov AL,[ES:DI+2] ; get fake count
41751 ;***mov [ss:FAKE_COUNT],AL ; save it
41752
41753 ; 09/03/2024 - PCDOS 7.1 IBMDOS.COM
41754 000071FB 58 pop ax ; take len off the stack
41755
41756 000071FC 5E pop SI
41757 000071FD 59 pop CX
41758 ; 18/12/2022
41759 ;jmp SHORT End_List
41760
41761 ; 18/12/2022
41762 End_List:
41763 000071FE C3 retn
41764
41765 Not_Matched:
41766 000071FF 58 pop ax ; get len from stack ; M050
41767 00007200 5E pop SI ; restore si,cx
41768 00007201 59 pop CX
41769
41770 SkipOne:
41771 00007202 368B3E[0E06] mov DI,[ss:TEMP_VAR2] ; restore old di use ss override
41772 00007207 30E4 xor AH,AH ; position to next entry
41773 00007209 01C7 add DI,AX
41774

```

```

41775 0000720B 83C703      add     DI,3           ; DI -> next entry length
41776                    ;***add    DI,4           ; DI -> next entry length
41777
41778 0000720E EBC7        jmp     short GetEntries
41779
41780                    ; 18/12/2022
41781 ;End_List:
41782 ;retn
41783
41784 ; 04/08/2018 - Retro DOS v3.0
41785 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 633Dh
41786
41787 ;-----
41788 ;SUBTTL Terminate and stay resident handler
41789
41790 ; Input:   DX is an offset from CurrentPDB at which to
41791 ;          truncate the current block.
41792 ;
41793 ; output:  The current block is truncated (expanded) to be [DX+15]/16
41794 ;          paragraphs long. An exit is simulated via resetting CurrentPDB
41795 ;          and restoring the vectors.
41796 ;-----
41797
41798 ; 20/05/2019 - Retro DOS v4.0
41799 ; DOSCODE:A19Bh (MSDOS 6.21, MSDOS.SYS)
41800
41801 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
41802 ; DOSCODE:A13Bh (MSDOS 5.0, MSDOS.SYS)
41803
41804 ; 09/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
41805 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B3BCh
41806
41807 _$KEEP_PROCESS:
41808     push    AX           ; keep exit code around
41809     ;mov     byte [SS:EXIT_TYPE],3
41810     mov     BYTE [SS:EXIT_TYPE],EXIT_KEEP_PROCESS
41811     mov     ES,[SS:CurrentPDB]
41812     cmp     DX,6h        ; keep enough space around for system
41813     jae     short Keep_Shrink ; info
41814
41815     mov     DX,6h
41816
41817 Keep_Shrink:
41818     mov     BX,DX
41819     push    BX
41820     push    ES
41821     call    _$SETBLOCK    ; ignore return codes.
41822     pop     DS
41823     pop     BX
41824     jc      short Keep_Done ; failed on modification
41825
41826     mov     AX,DS
41827     add     AX,BX
41828     ;mov     [2],ax
41829     mov     [PDB.BLOCK_LEN],AX ;PBUGBUG
41830
41831 Keep_Done:
41832     pop     AX
41833     jmp     SHORT exit_inner ; and let abort take care of the rest
41834
41835 ;-----
41836 ;
41837 ;-----
41838
41839 STAY_RESIDENT:
41840     ;mov     ax,3100h
41841     mov     AX,(KEEP_PROCESS<<8)+0 ; Lower part is return code;PBUGBUG
41842     add     DX,15
41843     rcr     DX,1
41844     mov     CL,3
41845     shr     DX,CL
41846
41847     jmp     COMMAND
41848
41849 ;-----
41850 ;SUBTTL $EXIT - return to parent process
41851 ; Assembler usage:
41852 ;     MOV     AL, code
41853 ;     MOV     AH, Exit
41854 ;     INT     int_command
41855 ; Error return:
41856 ;     None.
41857 ;-----
41858
41859 ; 20/05/2019 - Retro DOS v4.0
41860 ; DOSCODE:A1D3h (MSDOS 6.21, MSDOS.SYS)
41861
41862 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
41863 ; DOSCODE:A173h (MSDOS 5.0, MSDOS.SYS)
41864
41865 ; 09/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
41866 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B3F4h
41867
41868 _$EXIT:
41869     ; 04/08/2018 - Retro DOS v3.0
41870     ; IBMDOSDOS.COM (MSDOS 3.3, 1987) - offset 6375h
41871
41872     xor     AH,AH
41873     xchg    AH,[SS:DidCTRLC]
41874     or      AH,AH
41875     ;mov     BYTE [SS:EXIT_TYPE],0
41876     mov     BYTE [SS:EXIT_TYPE],EXIT_TERMINATE
41877     jz      short exit_inner
41878     ;mov     BYTE [SS:EXIT_TYPE],1
41879     mov     BYTE [SS:EXIT_TYPE],EXIT_CTRL_C
41880
41881     ;entry   Exit_inner
41882 exit_inner:
41883     call    Get_User_Stack ;PBUGBUG
41884
41885     push    word [ss:CurrentPDB]
41886     ;pop     word [si+14h]
41887     pop     word [SI+user_env.user_CS] ;PBUGBUG
41888     jmp     short abort_inner
41889
41890 ;BREAK <$ABORT -- Terminate a process>
41891 ;-----
41892 ; Inputs:
41893 ; user_CS:00 must point to valid program header block
41894 ; Function:
41895 ; Restore terminate and Cntrl-C addresses, flush buffers and transfer
41896 ; to the terminate address
41897 ; Returns:
41898 ; TO THE TERMINATE ADDRESS

```

```

41899 ;-----
41900
41901 _$ABORT:
41902 0000726C 30C0      xor     AL,AL
41903                  ;mov     byte [SS:EXIT_TYPE],0
41904                  ;mov     byte [SS:EXIT_TYPE],AL ; = 0
41905 0000726E 36C606[7C05]00 mov     byte [SS:EXIT_TYPE],EXIT_ABORT
41906
41907                  ; abort_inner must have AL set as the exit code! The exit type
41908                  ; is retrieved from exit_type. Also, the PDB at user_CS needs
41909                  ; to be correct as the one that is terminating.
41910
41911 abort_inner:
41912 00007274 368A26[7C05]    mov     AH,[SS:EXIT_TYPE]
41913 00007279 36A3[3403]    mov     [SS:exit_code],AX
41914 0000727D E8F791      call    Get_User_Stack
41915
41916                  ;mov     ds,[si+14h]
41917 00007280 8E5C14    mov     DS,[SI+user_env.user_CS] ; set up old interrupts ;PBUGBUG
41918 00007283 31C0      xor     AX,AX
41919 00007285 8EC0      mov     ES,AX
41920                  ;mov     si,10
41921 00007287 BE0A00    mov     SI,SAVEEXIT
41922                  ;mov     di,88h
41923 0000728A BF8800    mov     DI,addr_int_terminate
41924 0000728D A5      movsw
41925 0000728E A5      movsw
41926 0000728F A5      movsw
41927 00007290 A5      movsw
41928 00007291 A5      movsw
41929 00007292 A5      movsw
41930 00007293 E954EF    jmp     reset_environment
41931
41932 ;-----
41933 ;
41934 ; fixexepatch will point to this is DOS loads low.
41935 ;
41936 ;-----
41937 ; MSDOS 6.0
41938
41939 ; 29/04/2019 - Retro DOS v4.0
41940 ; DOSCODE:A221h (MSDOS 6.21, MSDOS.SYS)
41941
41942 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
41943 ; DOSCODE:A1C1h (MSDOS 5.0, MSDOS.SYS)
41944
41945 ; 09/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
41946 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B442h
41947
41948 RetExePatch: ; proc near
41949
41950 00007296 C3      retn
41951
41952 ;=====
41953 ; ALLOC.ASM, MSDOS 6.0, 1991
41954 ;=====
41955 ; 04/08/2018 - Retro DOS v3.0
41956 ; 14/05/2019 - Retro DOS v4.0
41957 ; 10/03/2024 - Retro DOS v5.0
41958
41959 ; TITLE ALLOC.ASM - memory arena manager NAME Alloc
41960
41961 ;**
41962 ; Microsoft Confidential
41963 ; Copyright (C) Microsoft Corporation 1991
41964 ; All Rights Reserved.
41965 ;
41966 ; Memory related system calls and low level routines for MSDOS 2.X.
41967 ; I/O specs are defined in DISPATCH.
41968 ;
41969 ; $ALLOC
41970 ; $SETBLOCK
41971 ; $DEALLOC
41972 ; $AllocOper
41973 ; arena_free_process
41974 ; arena_next
41975 ; check_signature
41976 ; Coalesce
41977 ;
41978 ; Modification history:
41979 ;
41980 ; Created: ARR 30 March 1983
41981 ;
41982 ; Revision: M000 - added support for allocating UMBs. 7/9/90
41983 ; M003 - added support for link/unlink UMBs from
41984 ; DOS arena chain. 7/18/90
41985 ; M009 - Added error returns invalid function and
41986 ; arena trashed in set link state call.
41987 ; M010 - Release UMB arenas allocated to current PDB
41988 ; if UMB_HEAD is initialized.
41989 ;
41990 ; M016 - MACE utilities mkeyrate.com version 1.0
41991 ; support. Please see under M009 in
41992 ; ..\inc\dossym.inc. 8/31/90.
41993 ;
41994 ; M061 - In GetLastArena, if linking in UMBs check to make
41995 ; sure that umb_head arena is valid and also make
41996 ; sure that the previous arena is pointing to
41997 ; umb_head.
41998 ;
41999 ; M064 - allow HIGH_ONLY bit to be set by a call to
42000 ; set allloc strategy.
42001 ; use STRAT_MASK to mask out bits 6 & 7 of
42002 ; bx in AllocSetStrat.
42003 ;
42004 ; M068 - use a count value (A20OFF_COUNT) rather than
42005 ; a bit to indicate to dos dispatcher to turn
42006 ; a20 off before iret. See M016.
42007 ;
42008 ;
42009 ; BREAK <memory allocation utility routines>
42010
42011 ; 15/04/2018 - Retro DOS v2.0
42012 ;-----
42013 ; xenix memory calls for MSDOS
42014 ;
42015 ; CAUTION: The following routines rely on the fact that arena_signature and
42016 ; arena_owner_system are all equal to zero and are contained in DI.
42017 ;
42018 ; INCLUDE DOSSEG.ASM
42019
42020 ;CODE SEGMENT BYTE PUBLIC 'CODE'
42021 ; ASSUME SS:DOSGROUP,CS:DOSGROUP
42022

```

```

42023 ;.xlist
42024 ;.xcref
42025 ;INCLUDE DOSSYM.ASM
42026 ;INCLUDE DEVSYM.ASM
42027 ;.cref
42028 ;.list
42029
42030 ;TITLE ALLOC.ASM - memory arena manager
42031 ;NAME Alloc
42032
42033 ;SUBTTL memory allocation utility routines
42034 ;PAGE
42035
42036 ;
42037 ; arena data
42038 ;
42039 ; i_need arena_head,WORD ; seg address of start of arena
42040 ; i_need CurrentPDB,WORD ; current process data block addr
42041 ; i_need FirstArena,WORD ; first free block found
42042 ; i_need BestArena,WORD ; best free block found
42043 ; i_need LastArena,WORD ; last free block found
42044 ; i_need AllocMethod,BYTE ; how to alloc first(best)last
42045
42046 ;-----
42047
42048 ; 10/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
42049 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0B443h
42050 ;;;
42051 test_umb_flag:
42052 test byte [ss:UMBFLAG],LINKSTATE ; 1 ; Q: are umb's linked
42053 retn ; ZF=1 -> N: scan from arena_head
42054 ; ZF=0 -> Y: start_arena = umb_head
42055 ;;;
42056
42057 ;** Arena_Free_Process
42058 ;-----
42059 ; Free all arena blocks allocated to a process
42060 ;
42061 ; ENTRY (bx) = PID of process
42062 ; EXIT none
42063 ; USES ?????? BUGBUG
42064 ;-----
42065
42066 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
42067 ; DOSCODE:A1C2h (MSDOS 5.0, MSDOS.SYS)
42068
42069 ; 10/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
42070 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0B44Ah
42071
42072 arena_free_process:
42073 ; 14/05/2019 - Retro DOS v4.0
42074 ; 04/08/2018 - Retro DOS v3.0
42075 MOV AX,[ss:arena_head]
42076 arena_free_process_start:
42077 MOV DI,ARENA.SIGNATURE ; 0
42078 ;MOV AX,[ss:arena_head] ; 15/04/2018
42079 CALL check_signature ; ES <- AX, check for valid block
42080
42081 arena_free_process_loop:
42082 ;retc
42083 JC SHORT AFP_RETN ; Retro DOS v2.0 - 05/03/2018
42084 PUSH ES
42085 POP DS
42086 ;cmp [1],bx
42087 CMP [ARENA.OWNER],BX ; is block owned by pid?
42088 JNZ SHORT arena_free_next ; no, skip to next
42089 ;mov [1],di
42090 MOV [ARENA.OWNER],DI ; yes... free him
42091
42092 arena_free_next:
42093 ;cmp byte [di],5Ah ; 'z'
42094 CMP BYTE [DI],arena_signature_end
42095 ; ; end of road, Jack?
42096 ;retz ; never come back no more
42097 ;JZ SHORT AFP_RETN ; MSDOS 3.3 (& MSDOS 2.11)
42098 ; 14/05/2019
42099 ; MSDOS 6.0
42100 jz short arena_chk_umbs
42101
42102 CALL arena_next ; next item in ES/AX carry set if trash
42103 JMP SHORT arena_free_process_loop
42104
42105 ; MSDOS 6.0
42106 arena_chk_umbs: ; M010 - Start
42107 ; 20/05/2019
42108 mov ax,[ss:UMB_HEAD] ; ax = umb_head
42109 cmp ax,0FFFFh ; Q: is umb_head initialized
42110 je short ret_label ; N: we're done
42111
42112 mov di,ds ; di = last arena
42113 cmp di,ax ; Q: is last arena above umb_head
42114 ;jae short ret_label ; Y: we've scanned umbs also. done.
42115 ;jmp short arena_free_process_start
42116 ; ; M010 - End
42117 ; 10/03/2024 (PC DOS 7.1 IBMDOS.COM)
42118 jnb short arena_free_process_start
42119
42120 ; 10/03/2024
42121 AFP_RETN:
42122 RETN
42123
42124 ; BREAK <Arena Helper Routines>
42125
42126 ;** Arena_Next - Find Next item in Arena
42127 ;-----
42128 ; ENTRY DS - pointer to block head
42129 ; (di) = 0
42130 ; EXIT AX,ES - pointers to next head
42131 ; 'C' set iff arena damaged
42132 ;-----
42133
42134 arena_next:
42135 MOV AX,DS ; AX <- current block
42136 ADD AX,[ARENA.SIZE] ; AX <- AX + current block length
42137 INC AX ; remember that header!
42138
42139 ; fall into check_signature and return
42140 ;
42141 ; CALL check_signature ; ES <- AX, carry set if error
42142 ; RETN
42143
42144 ;** Check_Signature - Check Memory Block Signature
42145 ;-----
42146 ; ENTRY (AX) = address of block header

```

```

42147 ; (di) = 0
42148 ; EXIT ES = AX
42149 ; 'C' clear if signature good
42150 ; 'C' set if signature bad
42151 ; USES ES, Flags
42152 ;-----
42153
42154 check_signature:
42155
42156 MOV ES,AX ; ES <- AX
42157 ;cmp byte [es:di],4Dh ; 'M'
42158 CMP BYTE [ES:DI],arena_signature_normal
42159 ; IF next signature = not_end THEN
42160 JZ SHORT check_signature_ok ; GOTO ok
42161 ;cmp byte [es:di],5Ah ; 'Z'
42162 CMP BYTE [ES:DI],arena_signature_end
42163 ; IF next signature = end then
42164 JZ SHORT check_signature_ok ; GOTO ok
42165 STC ; set error
42166
42167 ret_label: ; MSDOS 6.0
42168 ;AFP_RETN: ; 10/03/2024
42169 ; Retro DOS v2.0 - 05/03/2018
42170 check_signature_ok:
42171 COALESCE_RETN:
42172 RETN
42173
42174 ;** Coalesce - Combine free blocks ahead with current block
42175 ;-----
42176 ; Coalesce adds the block following the argument to the argument block,
42177 ; iff it's free. Coalesce is usually used to join free blocks, but
42178 ; some callers (such as $setblock) use it to join a free block to it's
42179 ; preceeding allocated block.
42180 ; ENTRY (ds) = pointer to the head of a free block
42181 ; (di) = 0
42182 ; EXIT 'C' clear if OK
42183 ; (ds) unchanged, this block updated
42184 ; (ax) = address of next block, IFF not at end
42185 ; 'C' set if arena trashed
42186 ; USES (cx)
42187 ;-----
42188
42189 coalesce:
42190 ;cmp byte [di],5Ah ; 'Z'
42191 CMP BYTE [DI],arena_signature_end
42192 ; IF current signature = END THEN
42193 ;retz ; GOTO ok
42194 ;jz short COALESCE_RETN
42195 CALL arena_next ; ES, AX <- next block, Carry set if error
42196 ;retc ; IF no error THEN GOTO check
42197 jc short COALESCE_RETN
42198
42199 coalesce_check:
42200 ;cmp [es:1],di
42201 CMP [ES:ARENA.OWNER],DI
42202 ;retnz ; IF next block isnt free THEN return
42203 JNZ SHORT COALESCE_RETN
42204 ;mov cx,[es:3]
42205 MOV CX,[ES:ARENA.SIZE] ; CX <- next block size
42206 INC CX ; CX <- CX + 1 (for header size)
42207 ;ADD [3],CX
42208 ADD [ARENA.SIZE],CX ; current size <- current size + CX
42209 MOV CL,[ES:DI] ; move up signature
42210 MOV [DI],CL
42211 JMP SHORT Coalesce ; try again
42212
42213 ; 04/08/2018 - Retro DOS v3.0
42214 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 641Fh
42215
42216 ; BREAK <$Alloc - allocate space in memory>
42217
42218 ; MSDOS 6.0
42219 ;-----
42220 ;** $Alloc - Allocate Memory Space
42221 ;-----
42222 ; $Alloc services the INT21 that allocates memory space to a program.
42223 ; Alloc returns a pointer to a free block of memory that
42224 ; has the requested size in paragraphs.
42225 ;
42226 ; If the allocation strategy is HIGH_FIRST or HIGH_ONLY memory is
42227 ; scanned from umb_head if not from arena_head. If the strategy is
42228 ; HIGH_FIRST the scan is continued from arena_head if a block of
42229 ; appropriate size is not found in the UMBS. If the strategy is
42230 ; HIGH_FIRST+HIGH_ONLY only the UMBS are scanned for memory.
42231 ;
42232 ; In either case if bit 0 of UmbFlag is not initialized then the scan
42233 ; starts from arena_head.
42234 ;
42235 ; Assembler usage:
42236 ; MOV BX,size
42237 ; MOV AH,Alloc
42238 ; INT 21h
42239 ;
42240 ; BUGBUG - a lot can be done to improve performance. We can set marks
42241 ; so that we start searching the arena at it's first non-trivial free
42242 ; block, we can peephole the code, etc. (We can move some subr calls
42243 ; inline, etc.) I assume that this is called rarely and that the arena
42244 ; doesn't have too many memory objects in it beyond the first free one.
42245 ; verify that this is true; if so, this can stay as is
42246 ;
42247 ; ENTRY (bx) = requested size, in bytes
42248 ; (DS) = (ES) = DOSGROUP
42249 ; EXIT 'C' clear if memory allocated
42250 ; (ax:0) = address of requested memory
42251 ; 'C' set if request failed
42252 ; (AX) = error_not_enough_memory
42253 ; (bx) = max size we could have allocated
42254 ; (ax) = error_arena_trashed
42255 ; USES All
42256 ;-----
42257
42258 ; MSDOS 2.11 (& MSDOS 3.3)
42259 ;-----
42260 SUBTTL $Alloc - allocate space in memory
42261 ;-----
42262 ; Assembler usage:
42263 ; MOV BX,size
42264 ; MOV AH,Alloc
42265 ; INT 21h
42266 ; AX:0 is pointer to allocated memory
42267 ; BX is max size if not enough memory
42268 ;
42269 ; Description:
42270 ; Alloc returns a pointer to a free block of

```

```

42271 ; memory that has the requested size in paragraphs.
42272 ;
42273 ; Error return:
42274 ; AX = error_not_enough_memory
42275 ; = error_arena_trashed
42276 ;-----
42277
42278 ; DOSCODE:A28Eh (MSDOS 6.21, MSDOS.SYS)
42279
42280 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
42281 ; DOSCODE:A22Eh (MSDOS 5.0, MSDOS.SYS)
42282
42283 ; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
42284 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B4B5h
42285
42286 _$ALLOC:
42287 ; 25/05/2019 (Procedure has been checked and confirmed)
42288 ; 14/05/2019 - Retro DOS v4.0
42289 ; 04/08/2018 - Retro DOS v3.0
42290 ; EnterCrit critMem
42291 00007309 E8D6A5 call ECritMEM ; MSDOS 3.3 & MSDOS 6.0
42292
42293 ; 17/12/2022
42294 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42295 ;%if 0
42296 ; 14/05/2019
42297 0000730C 16 push ss
42298 0000730D 1F pop ds
42299
42300 ; MSDOS 6.0
42301 ; mov ax,[ss:arena_head]
42302 ; mov [ss:START_ARENA],ax ; assume LOW_FIRST
42303
42304 0000730E A1[2400] mov ax,[arena_head]
42305 00007311 A3[8E00] mov [START_ARENA],ax
42306
42307 ; test byte [ss:AllocMethod],HIGH_FIRST+HIGH_ONLY
42308 00007314 F606[0203]C0 test byte [AllocMethod],HIGH_FIRST+HIGH_ONLY
42309 ; Q: should we start scanning from
42310 ; UMB's
42311 00007319 740B jz short norm_alloc ; N: scan from arena_head
42312
42313 ; 10/03/2024
42314 ;%if 0
42315 ; cmp word [ss:UMB_HEAD],-1 ; Q: Has umb_head been initialized
42316 ; cmp word [UMB_HEAD],-1
42317 ; je short norm_alloc ; N: scan from arena_head
42318
42319 ; test byte [ss:UMBFLAG],LINKSTATE ; Q: are umb's linked
42320 test byte [UMBFLAG],LINKSTATE ; 1
42321 jz short norm_alloc ; N: scan from arena_head
42322 ;%else
42323 ; 10/03/2024 (PCDOS 7.1 IBMDOS.COM)
42324 ;
42325 0000731B E879FF call test_umb_flag
42326 0000731E 7406 jz short norm_alloc
42327 ;
42328 ;%endif
42329
42330 ; mov ax,[ss:UMB_HEAD]
42331 ; mov [ss:START_ARENA],ax ; start_arena = umb_head
42332 00007320 A1[8C00] mov ax,[UMB_HEAD]
42333 00007323 A3[8E00] mov [START_ARENA],ax
42334 ; M000 - end
42335
42336 norm_alloc:
42337 XOR AX,AX
42338 MOV DI,AX
42339 ; 15/03/2018
42340 ; MOV [SS:FirstArena],AX ; init the options
42341 ; MOV [SS:BestArena],AX
42342 ; MOV [SS>LastArena],AX
42343 ; 14/05/2019
42344 0000732A A3[4003] MOV [FirstArena],AX ; init the options
42345 0000732D A3[4203] MOV [BestArena],AX
42346 00007330 A3[4403] MOV [LastArena],AX
42347 00007333 50 PUSH AX ; alloc_max <- 0
42348
42349 ; 04/08/2018
42350 start_scan:
42351 ; MOV AX,[ss:arena_head] ; AX <- beginning of arena
42352 ; MOV AX,[arena_head]
42353
42354 ; 14/05/2019
42355 ; MSDOS 6.0
42356 00007334 A1[8E00] ; mov ax,[ss:START_ARENA] ; M000: AX <- beginning of arena
42357 mov ax,[START_ARENA]
42358 ; 27/09/2023 (BugFix) (*)
42359 ; ( jump from 'alloc_chk' (ds<>ss, ax = [ss:START_ARENA]))
42360 start_scan_x:
42361 00007337 E89DFF CALL check_signature ; ES <- AX, carry set if error
42362 0000733A 7233 JC SHORT alloc_err ; IF error THEN GOTO err
42363
42364 ;%endif
42365
42366 ; 17/12/2022
42367 ;%if 0
42368 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42369
42370 ; MSDOS 6.0
42371 mov ax,[ss:arena_head]
42372 mov [ss:START_ARENA],ax ; assume LOW_FIRST
42373
42374 test byte [ss:AllocMethod],HIGH_FIRST+HIGH_ONLY
42375 ; Q: should we start scanning from
42376 ; UMB's
42377 jz short norm_alloc ; N: scan from arena_head
42378
42379 ; cmp word [ss:UMB_HEAD],-1 ; Q: Has umb_head been initialized
42380 ; je short norm_alloc ; N: scan from arena_head
42381
42382 test byte [ss:UMBFLAG],LINKSTATE ; Q: are umb's linked
42383 jz short norm_alloc ; N: scan from arena_head
42384
42385 mov ax,[ss:UMB_HEAD]
42386 mov [ss:START_ARENA],ax ; start_arena = umb_head
42387 ; M000 - end
42388
42389 norm_alloc:
42390 XOR AX,AX
42391 MOV DI,AX
42392 ; 15/03/2018
42393 MOV [SS:FirstArena],AX ; init the options
42394 MOV [SS:BestArena],AX
42395 MOV [SS>LastArena],AX

```



```

42395             PUSH     AX                     ; alloc_max <- 0
42396             ; 04/08/2018
42397 start_scan:
42398             ; MOV     AX,[SS:arena_head]      ; AX <- beginning of arena
42399             ; 14/05/2019
42400             ; MSDOS 6.0
42401             mov     ax,[SS:START_ARENA]      ; M000: AX <- beginning of arena
42402             CALL    check_signature          ; ES <- AX, carry set if error
42403             JC      SHORT alloc_err          ; IF error THEN GOTO err
42404 %endif
42405
42406 alloc_scan:
42407             PUSH     ES
42408             POP      DS                      ; DS <- ES
42409             CMP      [ARENA.OWNER],DI ; 0
42410             JZ       SHORT alloc_free        ; IF current block is free THEN examine
42411
42412 alloc_next:
42413
42414             ; 10/03/2024
42415 %if 0
42416             ; MSDOS 6.0                      ; M000 - start
42417             test    byte [ss:UMBFLAG],LINKSTATE ; Q: are umb's linked
42418             jz      short norm_strat         ; N: see if we reached last arena
42419 %else
42420             ; 10/03/2024 (PCDOS 7.1 IBMDOS.COM)
42421             ;;;
42422             call    test_umb_flag
42423             jz      short norm_strat
42424             ;;;
42425 %endif
42426             test    byte [ss:AllocMethod],HIGH_FIRST
42427             ; Q: is alloc strategy high_first
42428             jz      short norm_strat         ; N: see if we reached last arena
42429             mov     ax,[ss:START_ARENA]
42430             cmp     ax,[ss:arena_head]      ; Q: did we start scan from
42431             ; arena_head
42432             jne     short norm_strat         ; N: see if we reached last arena
42433             mov     ax,ds                    ; ax = current block
42434             cmp     ax,[ss:UMB_HEAD]        ; Q: check against umb_head
42435             jmp     short alloc_chk_end
42436
42437 norm_strat:
42438             ; cmp     byte [di],5Ah ; 'Z'
42439             CMP      BYTE [DI],arena_signature_end
42440             ; IF current block is last THEN
42441
42442 alloc_chk_end:
42443             JZ       SHORT alloc_end          ; GOTO end
42444             CALL    arena_next                ; AX, ES <- next block, Carry set if error
42445             JNC      SHORT alloc_scan         ; IF no error THEN GOTO scan
42446
42447 alloc_err:
42448             POP      AX
42449
42450 alloc_trashed:
42451             ; LeaveCrit critMem
42452             call    LCritMEM ; MSDOS 3.3 & MSDOS 6.0
42453
42454 alloc_trashed2: ; 10/03/2024 - Retro DOS v5.0 (PCDOS 7.1) -jmp from GetLastArena-
42455             ; error error_arena_trashed
42456             mov     al,7
42457             MOV      AL,error_arena_trashed
42458 alloc_errj:
42459             JMP      SYS_RET_ERR
42460
42461 alloc_end:
42462             ; 18/05/2019
42463             CMP     WORD [SS:FirstArena],0
42464             jz      short alloc_chk
42465             jmp     alloc_do_split
42466
42467 alloc_chk:
42468             ; MSDOS 6.0
42469             mov     ax,[ss:arena_head]
42470             cmp     ax,[ss:START_ARENA]      ; Q: started scanning from arena_head
42471             je      short alloc_fail          ; Y: not enough memory
42472             ; N:
42473             ; Q: is the alloc strat HIGH_ONLY
42474             test    byte [ss:AllocMethod],HIGH_ONLY
42475             jnz     short alloc_fail          ; Y: return size of largest UMB
42476             mov     [ss:START_ARENA],ax     ; N: start scanning from arena_head
42477             ; 27/09/2023 (*)
42478             jmp     short start_scan_x ; (*) ; (BugFix)
42479             ; jmp     short start_scan
42480             ; M000 - end
42481
42482 alloc_fail:
42483             ; invoke Get_User_Stack
42484             CALL    Get_User_Stack
42485             POP      BX
42486             ; MOV     [SI].user_BX,BX
42487             ; MOV     [SI+2],BX
42488             mov     [SI+user_env.user_BX],bx
42489             ; LeaveCrit critMem
42490             call    LCritMEM ; MSDOS 3.3 & MSDOS 6.0
42491             ; error error_not_enough_memory
42492             mov     al,8
42493             MOV      AL,error_not_enough_memory
42494             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42495             jmp     short alloc_errj
42496             ; JMP     SYS_RET_ERR
42497
42498 alloc_free:
42499             CALL    Coalesce                  ; add following free block to current
42500             JC      SHORT alloc_err          ; IF error THEN GOTO err
42501             MOV     CX,[ARENA.SIZE]
42502             POP      DX                      ; check for max found size
42503             CMP     CX,DX
42504             JNA     SHORT alloc_test
42505             MOV     DX,CX
42506
42507 alloc_test:
42508             PUSH     DX
42509             CMP     BX,CX                    ; IF BX > size of current block THEN
42510             JA      SHORT alloc_next         ; GOTO next
42511
42512             ; 15/03/2018
42513             CMP     WORD [SS:FirstArena],0
42514             JNZ     SHORT alloc_best
42515             MOV     [SS:FirstArena],DS      ; save first one found
42516 alloc_best:
42517             CMP     WORD [SS:BestArena],0
42518             JZ      SHORT alloc_make_best   ; initial best
42519             PUSH     ES

```

```

42519 000073D5 368E06[4203]      MOV     ES,[SS:BestArena]
42520 000073DA 26390E0300      CMP     [ES:ARENA.SIZE],CX      ; is size of best larger than found?
42521 000073DF 07      POP     ES
42522 000073E0 7605      JBE     SHORT alloc_last
42523      alloc_make_best:
42524 000073E2 368C1E[4203]      MOV     [SS:BestArena],DS ; assign best
42525      alloc_last:
42526 000073E7 368C1E[4403]      MOV     [SS:LastArena],DS      ; assign last
42527 000073EC E955FF      JMP     alloc_next
42528      ;
42529      ; split the block high
42530      ;
42531      alloc_do_split_high:
42532 000073EF 368E1E[4403]      MOV     DS,[SS:LastArena]
42533 000073F4 8B0E0300      MOV     CX,[ARENA.SIZE]
42534 000073F8 29D9      SUB     CX,BX
42535 000073FA 8CDA      MOV     DX,DS
42536 000073FC 7449      JE      SHORT alloc_set_owner      ; sizes are equal, no split
42537 000073FE 01CA      ADD     DX,CX      ; point to next block
42538 00007400 8EC2      MOV     ES,DX      ; no decrement!
42539 00007402 49      DEC     CX
42540 00007403 87D9      XCHG    BX,CX      ; bx has size of lower block
42541 00007405 EB2B      JMP     SHORT alloc_set_sizes      ; cx has upper (requested) size
42542      ;
42543      ; we have scanned memory and have found all appropriate blocks
42544      ; check for the type of allocation desired; first and best are identical
42545      ; last must be split high
42546      ;
42547      alloc_do_split:
42548      ;
42549      ; 17/12/2022
42550      ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42551      ;%if 0
42552      ; 14/05/2019
42553      ; MSDOS 6.0      ; M000 - start
42554      ;xor     CX,CX
42555 00007407 368A0E[0203]      mov     cl,[ss:AllocMethod]
42556      ;and     CX,STRAT_MASK ; 0FF3Fh; mask off bit 7
42557 0000740C 80E13F      and     cl,3Fh
42558      ;cmp     CX,BEST_FIT ; 1      ; Q; is the alloc strategy best_fit
42559 0000740F 80F901      cmp     cl,BEST_FIT
42560 00007412 77DB      ja      short alloc_do_split_high
42561      ;%endif
42562      ;
42563      ; 17/12/2022
42564      ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42565      ; MSDOS 6.0 & MSDOS 5.0
42566      ;xor     CX,CX
42567      ;mov     cl,[ss:AllocMethod]
42568      ;and     CX,STRAT_MASK ; 0FF3Fh; mask off bit 7
42569      ;cmp     CX,BEST_FIT ; 1      ; Q; is the alloc strategy best_fit
42570      ;ja      short alloc_do_split_high
42571      ;
42572      ; 15/03/2018
42573      ;;CMP     BYTE [SS:AllocMethod], 1
42574      ; 04/08/2018
42575      ;CMP     BYTE [SS:AllocMethod],BEST_FIT
42576      ;JA SHORT alloc_do_split_high
42577      ;
42578 00007414 368E1E[4003]      MOV     DS,[SS:FirstArena]
42579 00007419 7205      JB      SHORT alloc_get_size
42580 0000741B 368E1E[4203]      MOV     DS,[SS:BestArena]
42581      ;
42582      alloc_get_size:
42583 00007420 8B0E0300      MOV     CX,[ARENA.SIZE]
42584 00007424 29D9      SUB     CX,BX      ; get room left over
42585 00007426 8CD8      MOV     AX,DS
42586 00007428 89C2      MOV     DX,AX      ; save for owner setting
42587 0000742A 741B      JE      SHORT alloc_set_owner      ; IF BX = size THEN (don't split)
42588 0000742C 01D8      ADD     AX,BX
42589 0000742E 40      INC     AX      ; remember the header
42590 0000742F 8EC0      MOV     ES,AX      ; ES <- DS + BX (new header location)
42591 00007431 49      DEC     CX      ; CX <- size of split block
42592      alloc_set_sizes:
42593 00007432 891E0300      MOV     [ARENA.SIZE],BX      ; current size <- BX
42594 00007436 26890E0300      MOV     [ES:ARENA.SIZE],CX      ; split size <- CX
42595      ;mov     b1,4Dh ; 'M'
42596 0000743B 834D      MOV     BL,arena_signature_normal
42597 0000743D 861D      XCHG    BL,[DI]      ; current signature <- 4D
42598 0000743F 26881D      MOV     [ES:DI],BL      ; new block sig <- old block sig
42599 00007442 26893E0100      MOV     [ES:ARENA.OWNER],DI
42600      ;
42601      alloc_set_owner:
42602 00007447 8EDA      MOV     DS,DX
42603 00007449 36A1[3003]      MOV     AX,[SS:CurrentPDB] ; 15/03/2018
42604 0000744D A30100      MOV     [ARENA.OWNER],AX
42605 00007450 8CD8      MOV     AX,DS
42606 00007452 40      INC     AX
42607 00007453 5B      POP     BX
42608      ;LeaveCrit critMem
42609 00007454 E8B8A4      call    LCritMEM ; MSDOS 3.3 & MSDOS 6.0
42610      ;
42611      ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42612      alloc_ok:
42613      ;transfer SYS_RET_OK
42614 00007457 E91492      JMP     SYS_RET_OK
42615      ;
42616      ; BREAK $SETBLOCK - change size of an allocated block (if possible)
42617      ;
42618      ; MSDOS 6.0
42619      ;-----
42620      ;** $SETBLOCK - Change size of an Allocated Block
42621      ;
42622      ; Setblock changes the size of an allocated block. First, we coalesce
42623      ; any following free space onto this block; then we try to trim the
42624      ; block down to the size requested.
42625      ;
42626      ; Note that if the guy wants to grow the block but that growth fails,
42627      ; we still go ahead and coalesce any trailing free blocks onto it.
42628      ; Thus the maximum-size-possible value that we return has already
42629      ; been allocated! This is a bug, dare we fix it? BUGBUG
42630      ;
42631      ; NOTE - $SETBLOCK is in bed with $ALLOC and jumps into $ALLOC to
42632      ; finish it's work. For this reason we build the allocsf
42633      ; structure on the frame, to make us compatible with $ALLOCS
42634      ; code.
42635      ;
42636      ; ENTRY (es) = segment of old block
42637      ; (bx) = newsize
42638      ; (ah) = SETBLOCK
42639      ;
42640      ; EXIT 'C' clear if OK
42641      ; 'C' set if error
42642      ; (ax) = error_invalid_block

```



```

42643 ;           = error_arena_trashed
42644 ;           = error_not_enough_memory
42645 ;           = error_invalid_function
42646 ;           (bx) = maximum size possible, iff (ax) = error_not_enough_memory
42647 ; USES      ??? BUGBUG
42648 ;-----
42649 ; MSDOS 2.11 (& MSDOS 3.3)
42650 ;-----
42651 ; SUBTTL $SETBLOCK - change size of an allocated block (if possible)
42652 ;-----
42653 ; Assembler usage:
42654 ;     MOV     ES,block
42655 ;     MOV     BX,newsize
42656 ;     MOV     AH,setblock
42657 ;     INT     21h
42658 ;     if setblock fails for growing, BX will have the maximum
42659 ;     size possible
42660 ; Error return:
42661 ;     AX = error_invalid_block
42662 ;     = error_arena_trashed
42663 ;     = error_not_enough_memory
42664 ;     = error_invalid_function
42665 ;-----
42666 ;
42667 ;
42668 ;_SETBLOCK:
42669 ; 04/08/2018 - Retro DOS v3.0
42670 ;EnterCrit critMem
42671 0000745A E885A4 call ECritMEM ; MSDOS 3.3 & MSDOS 6.0
42672 ;
42673 0000745D BF0000 MOV DI,ARENA.SIGNATURE
42674 00007460 8CC0 MOV AX,ES
42675 00007462 48 DEC AX
42676 00007463 E871FE CALL check_signature
42677 00007466 7303 JNC SHORT setblock_grab
42678 ;
42679 setblock_bad:
42680 00007468 E905FF JMP alloc_trashed
42681 ;
42682 setblock_grab:
42683 0000746B 8ED8 MOV DS,AX
42684 0000746D E877FE CALL Coalesce
42685 00007470 72F6 JC SHORT setblock_bad
42686 00007472 8B0E0300 MOV CX,[ARENA.SIZE]
42687 00007476 51 PUSH CX
42688 00007477 39CB CMP BX,CX
42689 00007479 76A5 JBE SHORT alloc_get_size
42690 0000747B E91EFF JMP alloc_fail
42691 ;
42692 ; BREAK $DEALLOC - free previously allocated piece of memory
42693 ;
42694 ; MSDOS 6.0
42695 ;-----
42696 ; ** $DEALLOC - Free Heap Memory
42697 ;
42698 ; ENTRY (es) = address of item
42699 ;
42700 ; EXIT 'C' clear of OK
42701 ; 'C' set if error
42702 ; (AX) = error_invalid_block
42703 ; USES ??? BUGBUG
42704 ;
42705 ; MSDOS 2.11 (& MSDOS 3.3)
42706 ;-----
42707 ; SUBTTL $DEALLOC - free previously allocated piece of memory
42708 ;-----
42709 ; Assembler usage:
42710 ;     MOV     ES,block
42711 ;     MOV     AH,dealloc
42712 ;     INT     21h
42713 ;
42714 ; Error return:
42715 ;     AX = error_invalid_block
42716 ;     = error_arena_trashed
42717 ;-----
42718 ;
42719 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
42720 ; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
42721 ;_DEALLOC:
42722 ; 14/05/2019 - Retro DOS v4.0
42723 ; 04/08/2018 - Retro DOS v3.0
42724 ;EnterCrit critMem
42725 0000747E E861A4 call ECritMEM ; MSDOS 3.3 & MSDOS 6.0
42726 ;
42727 ; MSDOS 6.0 ; M016, M068 - Start
42728 00007481 36F606[8600]04 test byte [ss:DOS_FLAG],EXECA200FF
42729 ; ; Q: was the previous call an int 21
42730 ; ; exec call
42731 00007487 740D jz short deallocate ; N: continue
42732 00007489 36803E[8500]00 cmp byte [ss:A200FF_COUNT], 0 ; Q: is count 0
42733 0000748F 7505 jne short deallocate ; N: continue
42734 ;mov byte [ss:A200FF_COUNT], 1 ; Y: set count to 1
42735 ; 25/09/2023
42736 00007491 36FE06[8500] inc byte [ss:A200FF_COUNT]
42737 ;
42738 00007496 BF0000 deallocate: ; M016, M068 - End
42739 00007499 8CC0 MOV DI,ARENA.SIGNATURE ; = 0
42740 0000749B 48 DEC AX
42741 0000749C E838FE CALL check_signature
42742 0000749F 720A JC SHORT dealloc_err
42743 000074A1 26893E0100 MOV [ES:ARENA.OWNER],DI
42744 ;LeaveCrit critMem
42745 000074A6 E866A4 call LCritMEM ; MSDOS 3.3 & MSDOS 6.0
42746 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42747 ;transfer SYS_RET_OK
42748 ;
42749 000074A9 EBAC dealloc_ok:
42750 jmp short alloc_ok
42751 ;JMP SYS_RET_OK
42752 ;
42753 dealloc_err:
42754 000074AB E861A4 ;LeaveCrit critMem
42755 call LCritMEM ; MSDOS 3.3 & MSDOS 6.0
42756 ;error error_invalid_block
42757 000074AE B009 mov al,9
42758 MOV AL,error_invalid_block
42759 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42760 dealloc_errj:
42761 000074B0 E9C591 AllocOperErrj: ; 17/12/2022
42762 JMP SYS_RET_ERR
42763 ;
42764 ; BREAK $AllocOper - get/set allocation mechanism
42765 ;
42766 ; MSDOS 6.0
42767 ;-----

```

```

42767 ;** $AllocOper - Get/Set Allocation Mechanism
42768 ;
42769 ; Assembler usage:
42770 ;     MOV     AH,AllocOper
42771 ;     MOV     BX,method
42772 ;     MOV     AL,func
42773 ;     INT     21h
42774 ;
42775 ; ENTRY
42776 ;     (al) = 0
42777 ;     Get allocation Strategy in (ax)
42778 ;
42779 ;     (al) = 1, (bx) = method = zw0000xy
42780 ;     Set allocation strategy.
42781 ;     w = 1 => HIGH_ONLY
42782 ;     z = 1 => HIGH_FIRST
42783 ;     xy = 00 => FIRST_FIT
42784 ;     = 01 => BEST_FIT
42785 ;     = 10 => LAST_FIT
42786 ;
42787 ;     (al) = 2
42788 ;     Get UMB link state in (al)
42789 ;
42790 ;     (al) = 3
42791 ;     Set UMB link state
42792 ;     (bx) = 0 => Unlink UMBs
42793 ;     (bx) = 1 => Link UMBs
42794 ;
42795 ; EXIT 'C' clear if OK
42796 ;
42797 ;     if (al) = 0
42798 ;     (ax) = existing method
42799 ;     if (al) = 1
42800 ;     Sets allocation strategy
42801 ;     if (al) = 2
42802 ;     (al) = 0 => UMBs not linked
42803 ;     (al) = 1 => UMBs linked in
42804 ;     if (al) = 3
42805 ;     Links/unlinks the UMBs into DOS chain
42806 ;
42807 ;     'C' set if error
42808 ;     AX = error_invalid_function
42809 ;
42810 ;
42811 ; Rev. M000 - added support for HIGH_FIRST in (al) = 1. 7/9/90
42812 ; Rev. M003 - added functions (al) = 2 and (al) = 3. 7/18/90
42813 ; Rev. M009 - (al) = 3 will return 'invalid function' in ax if
42814 ; umbhead has'nt been initialized by sysinit and 'trashed
42815 ; arena' if an arena sig is damaged.
42816 ;-----
42817 ; MSDOS 2.11 (& MSDOS 3.3)
42818 ;-----
42819 ; SUBTTL $AllocOper - get/set allocation mechanism
42820 ;
42821 ; Assembler usage:
42822 ;     MOV     AH,AllocOper
42823 ;     MOV     BX,method
42824 ;     MOV     AL,func
42825 ;     INT     21h
42826 ;
42827 ; Error return:
42828 ;     AX = error_invalid_function
42829 ;-----
42830 ;
42831 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
42832 ; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
42833 ;-----
42834 ;_ $ALLOCOPER:
42835 ; 14/05/2019 - Retro DOS v4.0
42836 ; MSDOS 6.0
42837 000074B3 08C0 or al,al ; 0
42838 000074B5 7411 jz short AllocGetStrat
42839 ; 17/12/2022
42840 ; cmp al,1
42841 ; jz short AllocSetStrat
42842 ;
42843 ; 01/12/2022
42844 ; cmp al,2
42845 ; jb short AllocSetStrat
42846 ; ja short AllocSetLink
42847 ; jmp short AllocGetLink
42848 ; AllocGetLink:
42849 ; MSDOS 6.0
42850 ; mov al,[ss:UMBFLAG] ; return link state in al
42851 ; and al,LINKSTATE
42852 ; transfer SYS_RET_OK
42853 ; jmp SYS_RET_OK
42854 ;
42855 000074B7 3C02 cmp al,2
42856 ; 17/12/2022
42857 000074B9 7216 jb short AllocSetStrat ; al = 1
42858 000074BB 7425 je short AllocGetLink
42859 ;
42860 ; cmp al,2
42861 ; jz short AllocGetLink
42862 000074BD 3C03 cmp al,3
42863 000074BF 7429 jz short AllocSetLink
42864 ;
42865 ; 15/04/2018
42866 ; CMP AL,1
42867 ; JB SHORT AllocOperGet
42868 ; JZ SHORT AllocOperSet
42869 ;
42870 ; AllocOperError:
42871 ;
42872 ; 10/03/2024
42873 %if 0
42874 ; 04/08/2018 - Retro DOS v3.0
42875 ; MSDOS 3.3 (& MSDOS 6.0) ; Extended Error Locus
42876 ; mov byte [ss:EXTERR_LOCUS],5
42877 ; MOV byte [SS:EXTERR_LOCUS],errLOC_Mem
42878 ; error error_invalid_function
42879 %else
42880 ; 10/03/2024 - Retro DOS v5.0
42881 ; (PCDOS 7.1 IBMDOS.COM)
42882 ;;;
42883 000074C1 E82C9E call set_exerr_locus_mem
42884 ;;;
42885 %endif
42886 ; mov al,1
42887 000074C4 B001 MOV AL,error_invalid_function
42888 ; 17/12/2022
42889 ; AllocOperErrj:
42890 ; JMP SYS_RET_ERR

```

```

42891             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42892             ; jmp short dealloc_errj
42893             ; 17/12/2022
42894 000074C6 EBE8             jmp short AllocOperErrj
42895
42896             ; 10/03/2024 - Retro DOS v5.0
42897             ; (PCDOS 7.1 IBMDOS.COM)
42898 %if 0
42899 AllocArenaError:
42900             ; MSDOS 6.0
42901             MOV     byte [SS:EXTERR_LOCUS],errLOC_Mem
42902             ; M009: Extended Error Locus
42903             ; error error_arena_trashed ; M009:
42904             ; mov     al,7
42905             MOV     AL,error_arena_trashed
42906             ; JMP     SYS_RET_ERR
42907             jmp     short AllocOperErrj ; 17/12/2022
42908 %endif
42909
42910 AllocGetStrat:
42911             ; MSDOS 6.0
42912 AllocOperGet:
42913             MOV     AL,[SS:AllocMethod]
42914             XOR     AH,AH
42915             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42916             ; transfer SYS_RET_OK
42917 AllocOperOk:
42918             ; 17/12/2022
42919             ; jmp     short dealloc_ok
42920             JMP     SYS_RET_OK
42921
42922 AllocSetStrat:
42923             ; 14/05/2019
42924             ; MSDOS 6.0
42925             push    bx ; M000 - start
42926             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42927             ; and     bx,STRAT_MASK ; 0FF3Fh; M064: mask off bit 6 & 7
42928             ; 17/12/2022
42929             and     bl,3Fh
42930             cmp     bx,2 ; BX must be 0-2
42931             ; cmp     bl,2
42932             pop     bx ; M000 - end
42933             ja      short AllocOperError
42934
42935 AllocOperSet:
42936             MOV     [SS:AllocMethod],BL
42937             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42938             ; transfer SYS_RET_OK
42939 AllocOperOkj:
42940             jmp     short AllocOperOk
42941             ; JMP     SYS_RET_OK
42942
42943 AllocGetLink:
42944             ; MSDOS 6.0
42945             mov     al,[ss:UMBFLAG] ; return link state in al
42946             ; and     al,1
42947             and     al,LINKSTATE
42948             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42949             ; transfer SYS_RET_OK
42950 AllocOperOkj2:
42951             ; 17/12/2022
42952             jmp     short AllocOperOk
42953             ; jmp     short AllocOperOkj
42954             ; ; JMP     SYS_RET_OK
42955
42956 AllocSetLink:
42957             ; MSDOS 6.0 ; M009 - start
42958             mov     cx,[ss:UMB_HEAD] ; cx = umb_head
42959
42960             ; 10/03/2024
42961 %if 0
42962             cmp     cx,0FFFFh ; Q: has umb_head been initialized
42963             je      short AllocOperError ; N: error
42964             ; Y: continue
42965             ; M009 - end
42966             cmp     bx,1
42967             jb      short UnlinkUmbs ; 0
42968             jz      short LinkUmbs ; 1
42969 %else
42970             ; 10/03/2024 - Retro DOS v5.0
42971             ; (PCDOS 7.1 IBMDOS.COM)
42972             ; ;
42973             inc     cx ; Q: has umb_head been initialized
42974             jz      short AllocOperError ; -1 ; N: error
42975             dec     cx
42976             dec     bx
42977             jz      short LinkUmbs ; 1
42978             inc     bx
42979             jz      short UnlinkUmbs ; 0
42980             ; ;
42981 %endif
42982             ; jmp     short AllocOperError
42983             ; 10/03/2024
42984             jnz     short AllocOperError ; > 1
42985
42986 UnlinkUmbs:
42987
42988             ; 10/03/2024
42989 %if 0
42990             ; test     byte [ss:UMBFLAG],1
42991             test     byte [ss:UMBFLAG],LINKSTATE ; Q: umbs unlinked?
42992             jz      short unlinked ; Y: return
42993
42994             call     GetLastArena ; get arena before umb_head in DS
42995             jc      short AllocArenaError ; M009: arena trashed
42996 %else
42997             ; 10/03/2024 - Retro DOS v5.0
42998             ; (PCDOS 7.1 IBMDOS.COM)
42999             ; ;
43000             call     test_umb_flag ; test byte [ss:UMBFLAG],LINKSTATE
43001             ; ; Q: umbs unlinked?
43002             jz      short unlinked ; Y: return
43003             call     GetLastArena ; get arena before umb_head in DS
43004             ; ;
43005 %endif
43006             ; make it last
43007             mov     byte [0],arena_signature_end
43008
43009             ; and     byte [ss:UMBFLAG],0FEh
43010             and     byte [ss:UMBFLAG],~LINKSTATE ; indicate unlink'd state in umbflag
43011
43012 unlinked:
43013             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43014             ; transfer SYS_RET_OK

```

```

43015             ; 17/12/2022
43016 0000750C EBC0      jmp     short AllocOperOk
43017             ; jmp     short AllocOperOkj2
43018             ;; JMP     SYS_RET_OK
43019
43020 LinkUmb:
43021
43022             ; 10/03/2024
43023 %if 0
43024             test    byte [ss:UMBFLAG],LINKSTATE ; Q: umbs linked?
43025             jnz     short linked                ; Y: return
43026
43027             call    GetLastArena                ; get arena before umb_head
43028             jc      short AllocArenaError      ; M009: arena trashed
43029
43030             ; make it normal. M061: ds points to
43031             ; arena before umb_head
43032 %else
43033             ; 10/03/2024 - Retro DOS v5.0
43034             ; (PCDOS 7.1 IBMDOS.COM)
43035             ;;;
43036             call    test_umb_flag ; Q: umbs linked?
43037             jnz     short linked ; Y: return
43038             call    GetLastArena ; get arena before umb_head
43039             ;;;
43040 %endif
43041             mov     byte [0],arena_signature_normal
43042             or      byte [ss:UMBFLAG],LINKSTATE ; indicate link'd state in umbflag
43043 linked:
43044             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43045             ; transfer SYS_RET_OK
43046             ; 17/12/2022
43047             jmp     short AllocOperOk
43048             ; jmp     short unlinked
43049             ;; JMP     SYS_RET_OK
43050
43051             ; MSDOS 6.0
43052 -----
43053             ; Procedure Name : GetLastArena - M003
43054             ;
43055             ; Inputs      : cx = umb_head
43056             ;
43057             ;
43058             ; Outputs     : If UMBs are linked
43059             ;                 ES = umb_head
43060             ;                 DS = arena before umb_head
43061             ;             else
43062             ;                 DS = last arena
43063             ;                 ES = next arena. will be umb_head if NC.
43064             ;
43065             ;             CY if error
43066             ;
43067             ; Uses        : DS, ES, DI, BX
43068             -----
43069
43070             ; 14/05/2019 - Retro DOS v4.0
43071             ; DOSCODE:A4D6h (MSDOS 6.21, MSDOS.SYS)
43072
43073             ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43074             ; DOSCODE:A476h (MSDOS 5.0, MSDOS.SYS)
43075
43076             ; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43077             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B6D9h
43078
43079             ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A4D6h)
43080             ; (Windows ME IO.SYS - BIOSCODE:9F25h)
43081
43082 GetLastArena:
43083 00007523 50          push    ax                ; save ax
43084
43085             mov     ax,[ss:arena_head]
43086             mov     es,ax                    ; es = arena_head
43087             xor     di,di
43088
43089             cmp     byte [es:di],arena_signature_end
43090             ; Q: is this the last arena
43091             je      short GLA_done           ; Y: return last arena in ES
43092
43093 GLA_next:
43094             mov     ds,ax
43095             call    arena_next              ; ax, es -> next arena
43096             ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43097             ; jc      short GLA_err
43098             ; 17/12/2022
43099             jc      short GLA_err2
43100
43101             ; 10/03/2024
43102 %if 0
43103             test    byte [ss:UMBFLAG],LINKSTATE ; Q: are UMBs linked
43104             jnz     short GLA_chkumb        ; Y: terminating condition is
43105             ; umb_head
43106             ; N: terminating condition is 05Ah
43107 %else
43108             ; 10/03/2024 (PCDOS 7.1 IBMDOS.COM)
43109             ;;;
43110             call    test_umb_flag
43111             jnz     short GLA_chkumb
43112             ;;;
43113 %endif
43114             cmp     byte [es:di],arena_signature_end
43115             ; Q: is this the last arena
43116             jmp     short GLA_@f
43117 GLA_chkumb:
43118             cmp     ax,cx                    ; Q: is this umb_head
43119 GLA_@f:
43120             jne     short GLA_next          ; N: get next arena
43121
43122 GLA_done:
43123
43124             ; 10/03/2024
43125 %if 0
43126             ; M061 - Start
43127             test    byte [ss:UMBFLAG],LINKSTATE ; Q: are UMBs linked
43128             jnz     short GLA_ret           ; Y: we're done
43129             ; N: let us confirm that the next
43130             ; arena is umb_head
43131 %else
43132             ; 10/03/2024 (PCDOS 7.1 IBMDOS.COM)
43133             ;;;
43134             call    test_umb_flag
43135             jnz     short GLA_ret ; cf=0
43136             ;;;
43137 %endif
43138             mov     ds,ax

```

```

43139 0000754F E87EFD      call    arena_next      ; ax, es -> next arena
43140                      ;jc     short GLA_err
43141 00007552 7207      jc      short GLA_err2
43142 00007554 39C8      cmp     ax,cx           ; Q: is this umb_head
43143 00007556 7502      jne     short GLA_err  ; N: error
43144                      ; M061 - End
43145
43146                      GLA_ret:
43147                      ; 17/12/2022
43148                      ;clic
43149                      ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43150 00007558 58      ;clic
43151 00007559 C3      pop     ax             ; M061
43152                      retn      ; M061
43153
43154 0000755A F9      GLA_err:
43155                      stc             ; M061
43156 0000755B 58      GLA_err2:
43157                      pop     ax
43158                      ; 10/03/2024 - Retro DOS v5.0
43159                      ; (PCDOS 7.1 IBMDOS.COM)
43160                      ;;;
43161 0000755C 58      pop     ax      ; return address to _$ALLOCOPER (the caller)
43162 0000755D E8909D  call    set_exerr_locus_mem
43163 00007560 E910FE  jmp     alloc_trashed2 ; (error return from _$ALLOCOPER)
43164                      ;;;
43165                      ; 10/03/2024
43166                      ;retn
43167
43168
43169                      ;=====
43170                      ; SRVCALL.ASM, MSDOS 6.0, 1991
43171                      ;=====
43172                      ; 04/08/2018 - Retro DOS v3.0
43173
43174                      ; TITLE SRVCALL - Server DOS call
43175                      ; NAME SRVCALL
43176
43177                      ;** SRVCALL.ASM - Server DOS call functions
43178                      ;
43179                      ;
43180                      ; $ServerCall
43181                      ;
43182                      ; Modification history:
43183                      ;
43184                      ; Created: ARR 08 August 1983
43185
43186                      ;AsmVars <Installed>
43187
43188                      ;include dpl.asm
43189
43190                      ;Installed = TRUE
43191
43192                      ; 29/04/2019 - Retro DOS v4.0 (MSDOS 6.0, MSDOS 6.21)
43193                      ; -----
43194                      ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43195
43196                      ;BREAK <ServerCall -- Server DOS call>
43197
43198                      ; DOSCODE:A517h (MSDOS 6.21, MSDOS.SYS)
43199                      ; DOSCODE:A4B7h (MSDOS 5.0, MSDOS.SYS)
43200
43201                      ; 10/03/2024
43202                      ; DOSCODE:B71Ah (PCDOS 7.1, IBMDOS.COM) ; Retro DOS v5.0
43203                      ; DOSCODE:A517h (MSDOS 6.22, MSDOS.SYS) ; Retro DOS v4.2
43204                      ; BIOSCODE:9F66h (Windows ME, IO.SYS)
43205
43206                      ;hkn; TABLE SEGMENT
43207                      ;Public SRVC001S,SRVC001E
43208                      ;SRVC001S label byte
43209
43210                      SRVC001S:
43211
43212 00007563 [6775]      SERVERTAB: dw      SERVER_DISP
43213 00007565 [B675]      SERVERLEAVE: dw      SERVERRETURN
43214 00007567 0B      SERVER_DISP: db      (SERVER_DISP_END-SERVER_DISP-1)/2 ; = 11
43215 00007568 [1C76]      dw      SRV_CALL      ; 0
43216 0000756A [B775]      dw      COMMIT_ALL    ; 1
43217 0000756C [ED75]      dw      CLOSE_NAME     ; 2
43218 0000756E [F675]      dw      CLOSE_UID      ; 3
43219 00007570 [FD75]      dw      CLOSE_UID_PID   ; 4
43220 00007572 [0476]      dw      GET_LIST       ; 5
43221 00007574 [5D76]      dw      GET_DOS_DATA    ; 6
43222 00007576 [8176]      dw      SPOOL_OPER     ; 7
43223 00007578 [8176]      dw      SPOOL_OPER     ; 8
43224 0000757A [8176]      dw      SPOOL_OPER     ; 9
43225 0000757C [8D76]      dw      _$SetExtendedError ; 10
43226
43227                      SERVER_DISP_END: ; LABEL BYTE
43228
43229                      ;SRVC001E label byte
43230
43231                      SRVC001E:
43232
43233                      ;hkn; TABLE ENDS
43234
43235                      ;-----
43236                      ;
43237                      ; Procedure Name : $ServerCall
43238                      ;
43239                      ; Inputs:
43240                      ; DS:DX -> DPL (except calls 7,8,9)
43241                      ; Function:
43242                      ; AL=0 Server DOS call
43243                      ; AL=1 Commit All files
43244                      ; AL=2 Close file by name (SHARING LOADED ONLY) DS:DX in DPL -> name
43245                      ; AL=3 Close all files for DPL_UID
43246                      ; AL=4 Close all files for DPL_UID/PID_PID
43247                      ; AL=5 Get open file list entry
43248                      ; IN: BX File Index
43249                      ; CX User Index
43250                      ; OUT:ES:DI -> Name
43251                      ; BX = UID
43252                      ; CX = # locked blocks held by this UID
43253                      ; AL=6 Get DOS data area
43254                      ; OUT: DS:SI -> Start
43255                      ; CX size in bytes of swap if indos
43256                      ; DX size in bytes of swap always
43257                      ; AL=7 Get truncate flag
43258                      ; AL=8 Set truncate flag
43259                      ; AL=9 Close all spool files
43260                      ; AL=10 SetExtendedError
43261
43262                      ;-----

```

```

43263
43264 ; 10/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
43265 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0B735h
43266
43267 _$ServerCall:
43268 ; 13/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43269 ; DOSCODE:A4D2h (MSDOS 5.0 MSDOS.SYS)
43270 ; 10/06/2019
43271 ; 29/04/2019 - Retro DOS v4.0
43272 ; DOSCODE:A532h (MSDOS 6.21 MSDOS.SYS)
43273
43274 ; 05/08/2018 - Retro DOS v3.0
43275 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 657Bh
43276 0000757E 3C07 CMP AL,7
43277 00007580 7204 JB short SET_STUFF
43278 00007582 3C09 CMP AL,9
43279 00007584 761A JBE short NO_SET_ID ; No DPL on calls 7,8,9
43280 SET_STUFF:
43281 00007586 89D6 MOV SI,DX ; Point to DPL with DS:SI
43282 ;mov bx,[si+12h]
43283 00007588 8B5C12 MOV BX,[SI+DPL.UID]
43284
43285 ; MSDOS 6.0
43286 ;SR;
43287 ; WIN386 updates the USER_ID itself. If WIN386 is present we skip the updating
43288 ; of USER_ID
43289
43290 0000758B 36F606[5B0F]01 test byte [SS:Iswin386],1
43291 00007591 7505 jnz short skip_win386
43292
43293 ;hkn; SS override for user_id and proc_id
43294 ; 15/08/2018
43295 00007593 36891E[3E03] MOV [SS:USER_ID],BX ; Set UID
43296
43297 skip_win386:
43298 00007598 8B5C14 MOV BX,[SI+DPL.PID]
43299 0000759B 36891E[3C03] MOV [SS:PROC_ID],BX ; Set process ID
43300 NO_SET_ID:
43301 ; 10/06/2019 - Retro DOS v4.0
43302 000075A0 2EFF36[6575] PUSH word [cs:SERVERLEAVE] ; push return address
43303 000075A5 2EFF36[6375] PUSH word [cs:SERVERTAB] ; push table address
43304 000075AA 50 PUSH AX
43305 000075AB E850A2 call TableDispatch
43306
43307 ;hkn; SS override
43308 ;;mov byte [SS:EXETERR_LOCUS],1
43309 ;MOV byte [SS:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
43310 ; 01/07/2024
43311 000075AE E82C9D call set_exerr_locus_unk
43312
43313 ;error error_invalid_function
43314 ;mov al,1
43315 000075B1 B001 MOV AL,error_invalid_function
43316 servercall_error:
43317 000075B3 E9C290 JMP SYS_RET_ERR
43318
43319 SERVERRETURN:
43320 000075B6 C3 retn
43321
43322 ; Commit - iterate through the open file list and make sure that the
43323 ; directory entries are correctly updated.
43324
43325 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43326 COMMIT_ALL:
43327 000075B7 31DB XOR BX,BX ; for (i=0; ThisSFT=getSFT(i); i++)
43328 000075B9 16 push ss
43329 000075BA 1F pop ds
43330 000075BB E824A3 call ECritSFT ; Gonna scan SFT cache, lock it down
43331 CommitLoop:
43332 000075BE 53 push bx
43333 000075BF E82201 call SFFFromSFN
43334 000075C2 7222 JC short CommitDone
43335 000075C4 26833D00 cmp word [es:di],0
43336 ;CMP word [ES:DI+SF_ENTRY.sf_Ref_Count],0
43337 ; if (ThisSFT->refcount != 0)
43338 000075C8 7418 JZ short CommitNext
43339 ;cmp word [es:di],0FFFFh ; -1
43340 000075CA 26833DFF cmp word [ES:DI],sf_busy
43341 ;CMP word [ES:DI+SF_ENTRY.sf_Ref_Count],sf_busy
43342 ; BUSY SFTs have god knows what
43343 000075CE 7412 JZ short CommitNext ; in them.
43344 ; 17/12/2022
43345 000075D0 26F6450680 test byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8) ; 80h
43346 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet ; 8000h
43347 000075D5 750B JNZ short CommitNext ; Skip Network SFTs so the SERVER
43348 ; doesn't deadlock
43349 000075D7 893E[9E05] MOV [THISST],DI
43350 000075DB 8C06[A005] MOV [THISST+2],ES
43351 000075DF E8F1C2 call DOS_COMMIT ; DOSCommit ();
43352 CommitNext:
43353 000075E2 5B pop bx
43354 000075E3 43 INC BX
43355 000075E4 EBD8 JMP short CommitLoop
43356 CommitDone:
43357 000075E6 E826A3 call LCritSFT
43358 000075E9 5B pop bx
43359 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43360 Commit_Ok:
43361 000075EA E98190 jmp SYS_RET_OK
43362
43363 CLOSE_NAME:
43364
43365 ;if installed
43366
43367 ;hkn; SS override
43368 ;call far [ss:MFTclon]
43369 000075ED 36FF1E[A400] Call far [SS:JShare+(5*4)] ; 5 = MFTclon
43370 ;else
43371 ; Call MFTclon
43372 ;endif
43373
43374 CheckReturns:
43375
43376 ; 10/03/2024
43377 %if 0
43378 JC short func_err
43379 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43380 ;transfer SYS_RET_OK
43381 Commit_Okj:
43382 jmp short Commit_Ok
43383 ;jmp SYS_RET_OK
43384 %else
43385 000075F2 73F6 jnc short Commit_Ok
43386 %endif

```



```

43387
43388
43389
43390
43391 000075F4 EBB0
43392
43393
43394
43395
43396
43397
43398 000075F6 36FF1E[9C00]
43399
43400
43401
43402 000075FB EBF5
43403
43404
43405
43406
43407
43408
43409 000075FD 36FF1E[A000]
43410
43411
43412
43413 00007602 EBEE
43414
43415
43416
43417
43418
43419
43420 00007604 36FF1E[B400]
43421
43422
43423
43424 00007609 72E9
43425 0000760B E8698E
43426
43427 0000760E 895C02
43428
43429 00007611 897C0A
43430
43431 00007614 8C4410
43432
43433
43434 00007617 894C04
43435
43436
43437
43438
43439 0000761A EBCE
43440
43441
43442
43443
43444 0000761C 58
43445 0000761D 1E
43446 0000761E 56
43447 0000761F E8558E
43448 00007622 5F
43449 00007623 07
43450
43451
43452
43453
43454 00007624 E8B6A1
43455
43456
43457
43458
43459
43460
43461 00007627 56
43462 00007628 B90600
43463 0000762B F3A5
43464 0000762D 47
43465 0000762E 47
43466 0000762F A5
43467 00007630 A5
43468 00007631 5E
43469 00007632 8B04
43470
43471
43472 00007634 8B5C02
43473
43474 00007637 8B4C04
43475
43476 0000763A 8B5406
43477
43478 0000763D 8B7C0A
43479
43480 00007640 8E440E
43481
43482 00007643 FF7408
43483
43484 00007646 8E5C0C
43485 00007649 5E
43486
43487
43488 0000764A 368C1E[EC05]
43489 0000764F 36891E[EA05]
43490 00007654 36C606[7205]FF
43491 0000765A E9188D
43492
43493
43494 0000765D 16
43495 0000765E 07
43496 0000765F BF[2003]
43497 00007662 B9[FA0A]
43498 00007665 BA[3A03]
43499 00007668 29F9
43500 0000766A 29FA
43501 0000766C D1E9
43502 0000766E 83D100
43503 00007671 D1E1
43504 00007673 E8018E
43505
43506 00007676 8C440E
43507
43508 00007679 897C08
43509
43510 0000767C 895406

func_err:
;transfer SYS_RET_ERR
;jmp SYS_RET_ERR
jmp short servercall_error

CLOSE_UID:
;if installed
;hkn; SS override
;call far [ss:MFTCtlu]
Call far [SS:JShare+(3*4)] ; 3 = MFTCtlu
;else
; Call MFTCtlu
;endif
JMP short CheckReturns

CLOSE_UID_PID:
;if installed
;hkn; SS override
;call far [ss:MFTCcloseP]
Call far [SS:JShare+(4*4)] ; 4 = MFTCcloseP
;else
; Call MFTCcloseP
;endif
JMP short CheckReturns

GET_LIST:
;if installed
;hkn; SS override
;call far [ss:MFT_get]
Call far [SS:JShare+(9*4)] ; 9 = MFT_get
;else
; Call MFT_get
;endif
JC short func_err
call Get_User_Stack
;mov [si+2],bx
MOV [SI+user_env.user_BX],BX
;mov [si+10],di
MOV [SI+user_env.user_DI],DI
;mov [si+16],es
MOV [SI+user_env.user_ES],ES
SetCXOK:
;mov [si+4],cx
MOV [SI+user_env.user_CX],CX
; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;transfer SYS_RET_OK
Commit_Ok2:
; 17/12/2022
jmp short Commit_Ok
;jmp short Commit_Ok2
;;jmp SYS_RET_OK

SRV_CALL:
POP AX ; get rid of call to $srvcall
push ds
push si
call Get_User_Stack
pop di
pop es

; DS:SI point to stack
; ES:DI point to DPL

call XCHGP

; DS:SI point to DPL
; ES:DI point to stack
;
;
; we now copy the registers from DPL to save stack

push si
MOV CX,6
REP MOVSW ; Put in AX,BX,CX,DX,SI,DI
INC DI
INC DI ; Skip user_BP
MOVSW ; DS
MOVSW ; ES
pop si ; DS:SI -> DPL
mov ax,[SI]
;MOV AX,[SI+DPL.AX]
;mov bx,[si+2]
MOV BX,[SI+DPL.BX]
;mov cx,[si+4]
MOV CX,[SI+DPL.CX]
;mov dx,[si+6]
MOV DX,[SI+DPL.DX]
;mov di,[si+10]
MOV DI,[SI+DPL.DI]
;mov es,[si+14]
MOV ES,[SI+DPL.ES]
;push word [si+8]
PUSH word [SI+DPL.SI]
;mov ds,[si+12]
MOV DS,[SI+DPL.DS]
POP SI

;hkn; SS override for next 3
MOV [SS:SAVEDS],DS
MOV [SS:SAVEBX],BX
MOV byte [SS:FSHARING],-1 ; set no redirect flag
jmp REDISP

GET_DOS_DATA:
push ss
pop es
MOV DI,SWAP_START
MOV CX,SWAP_END
MOV DX,SWAP_ALWAYS
SUB CX,DI
SUB DX,DI
SHR CX,1 ; div by 2, remainder in carry
ADC CX,0 ; div by 2 + round up
SHL CX,1 ; round up to 2 boundary.
call Get_User_Stack
;mov [si+14],es
MOV [SI+user_env.user_DS],ES
;mov [si+8],di
MOV [SI+user_env.user_SI],DI
;mov [si+6],dx
MOV [SI+user_env.user_DX],DX

```

```

43511 0000767F EB96          JMP      short SetCXOK
43512
43513 SPOOL_OPER:
43514      ;CallInstall NETSpoolOper,MuItNET,37,AX,BX
43515
43516 00007681 50          push     ax
43517 00007682 B82511      mov      ax,1125h
43518 00007685 CD2F      int      2Fh      ; Multiplex - NETWORK REDIRECTOR - REDIRECTED PRINTER MODE
43519                      ; STACK: WORD subfunction
43520                      ; Return: CF set on error, AX = error code
43521                      ; STACK unchanged
43522 00007687 5B          pop      bx
43523                      ; 17/12/2022
43524                      ;JC      short func_err2
43525 00007688 7390      jnc      short Commit_okj2
43526                      ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43527                      ;;jmp     SYS_RET_OK
43528                      ;jmp     short Commit_okj2
43529
43530 func_err2:
43531 0000768A E9EB8F      jmp      SYS_RET_ERR
43532
43533      ;Break      <$SetExtendedError - set extended error for later retrieval>
43534      ;-----
43535      ;
43536      ; Procedure Name : $SetExtendedError
43537      ;
43538      ; $SetExtendedError takes extended error information and loads it up for the
43539      ; next extended error call. This is used by interrupt-level processors to
43540      ; mask their actions.
43541      ;
43542      ; Inputs: DS:SI points to DPL which contains all registers
43543      ; Outputs: none
43544      ;
43545      ;-----
43546
43547      _$SetExtendedError:
43548
43549      ;hkn; SS override for all variables used
43550
43551 0000768D 8B04      mov      ax,[si]
43552                      ;MOV     AX,[SI+DPL.AX]
43553 0000768F 36A3[2403]  MOV     [SS:EXTERR],AX
43554                      ;mov     ax,[si+10]
43555 00007693 8B440A      MOV     AX,[SI+DPL.DI]
43556 00007696 36A3[2803]  MOV     [SS:EXTERRPT],AX
43557                      ;mov     ax,[si+14]
43558 0000769A 8B440E      MOV     AX,[SI+DPL.ES]
43559 0000769D 36A3[2A03]  MOV     [SS:EXTERRPT+2],AX
43560                      ;mov     ax,[si+2]
43561 000076A1 8B4402      MOV     AX,[SI+DPL.BX]
43562 000076A4 36A3[2603]  MOV     [SS:EXTERR_ACTION],AX
43563                      ;mov     ax,[si+4]
43564 000076A8 8B4404      MOV     AX,[SI+DPL.CX]
43565 000076AB 368826[2303]  MOV     [SS:EXTERR_LOCUS],AH
43566 000076B0 C3          retn
43567
43568      ;=====
43569      ; UTIL.ASM, MSDOS 6.0, 1991
43570      ;=====
43571      ; 05/08/2018 - Retro DOS v3.0
43572      ; 05/05/2019 - Retro DOS v4.0
43573      ; 11/03/2024 - Retro DOS v5.0
43574
43575      ;** Handle related utilities for MSDOS 2.X.
43576      ;-----
43577      ; pJFNFromHandle written
43578      ; SFFromHandle written
43579      ; SFFromSFN written
43580      ; JFNFree written
43581      ; SFNFree written
43582      ;
43583      ; Modification history:
43584      ;
43585      ; Created: MZ 1 April 1983
43586      ;-----
43587
43588      ; BREAK <pJFNFromHandle - return pointer to JFN table entry>
43589
43590      ;** pJFNFromHandle - Translate Handle to Pointer to JFN
43591      ;-----
43592      ; pJFNFromHandle takes a file handle and turns that into a pointer to
43593      ; the JFN entry (i.e., to a byte holding the internal file handle #)
43594      ;
43595      ; NOTE:
43596      ; This routine is called from $CREATE_PROCESS_DATA_BLOCK which is called
43597      ; at DOSINIT time with SS NOT DOSGROUP
43598      ;
43599      ; ENTRY (bx) = handle
43600      ; EXIT 'C' clear if ok
43601      ; (es:di) = address of JFN value
43602      ; 'C' set if error
43603      ; (ax) = error code
43604      ; USES AX, DI, ES, Flags
43605      ;-----
43606
43607      ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43608      ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43609
43610 pJFNFromHandle:
43611      ; 05/05/2019 - Retro DOS v4.0
43612      ;getdseg <es> ; es -> dosdata
43613 000076B1 2E8E06[0700]  mov     es,[cs:DosDSeg]
43614
43615      ;MOV     ES,[cs:CurrentPDB] ; get user process data block
43616 000076B6 268E06[3003]  mov     es,[es:CurrentPDB]
43617
43618      ;cmp     bx,[ES:32h]
43619 000076BB 263B1E3200      CMP     BX,[ES:PDB.JFN_Length]; is handle greater than allocated
43620 000076C0 7204          JB      short pjfn10 ; no, get offset
43621      ReturnCarry_inv_hndl: ; 05/08/2018 - Retro DOS v3.0
43622      ;mov     al,6
43623 000076C2 B006      MOV     AL,error_invalid_handle ; appropriate error
43624      ReturnCarry:
43625      STC                      ; signal error
43626 000076C5 C3          retn ; go back
43627
43628      pjfn10:
43629      ;les     di,[es:34h]
43630 000076C6 26C43E3400      LES     DI,[ES:PDB.JFN_Pointer] ; get pointer to beginning of table
43631 000076CB 01DF      ADD     DI,BX ; add in offset, clear 'C'
43632      ;clc
43633 000076CD C3          retn
43634

```



```

43635 ;BREAK <SFFromHandle - return pointer (or error) to SF entry from handle>
43636 ;-----
43637 ;
43638 ; Procedure Name : SFFromHandle
43639 ;
43640 ; SFFromHandle - Given a handle, get JFN and then index into SF table
43641 ;
43642 ; Input:      BX has handle
43643 ; Output:     Carry Set
43644 ;             AX has error code
43645 ;             Carry Reset
43646 ;             ES:DI has pointer to SF entry
43647 ; Registers modified: If error, AX,ES, else ES:DI
43648 ; NOTE:
43649 ; This routine is called from $CREATE_PROCESS_DATA_BLOCK which is called
43650 ; at DOSINIT time with SS NOT DOSGROUP
43651 ;-----
43652 ;
43653 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43654 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43655
43656 SFFromHandle:
43657     CALL    pJFNFromHandle    ; get jfn pointer
43658     ;retc                    ; return if error
43659     jc      short pJFNFromHandle_error
43660     CMP     BYTE [ES:DI],-1    ; unused handle
43661     ;JNZ     short GetSF      ; nope, suck out SF
43662     ;mov     al,6
43663     MOV     AL,error_invalid_handle ; appropriate error
43664     ;jmp     short ReturnCarry ; signal it
43665     ; 17/12/2022
43666     ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43667     jz      short ReturnCarry_inv_hndl ; Retro DOS v3.0 modification
43668     ;JNZ     short GetSF      ; nope, suck out SF
43669     ;mov     al,6
43670     MOV     AL,error_invalid_handle ; appropriate error
43671     ;jmp     short ReturnCarry ; signal it
43672
43673 GetSF:
43674     push    bx                ; save handle
43675     MOV     BL,[ES:DI]        ; get SFN
43676     XOR     BH,BH             ; ignore upper half
43677     CALL    SFFFromSFN       ; get real sf spot
43678     pop     bx                ; restore
43679     retn                    ; say goodbye
43680
43681 ;BREAK <SFFromSFN - index into SF table for SFN>
43682 ;
43683 ;** SFFromSFN - Get an SF Table entry from an SFN
43684 ;-----
43685 ; SFFromSfn uses an SFN to index an entry into the SF table. This
43686 ; is more than just a simple index instruction because the SF table
43687 ; can be made up of multiple pieces chained together. We follow the
43688 ; chain to the right piece and then do the index operation.
43689 ;
43690 ; NOTE:
43691 ; This routine is called from SFFromHandle which is called
43692 ; at DOSINIT time with SS NOT DOSGROUP
43693 ;
43694 ; ENTRY    BX has SF index
43695 ; EXIT     'C' clear if OK
43696 ;          ES:DI points to SF entry
43697 ;          'C' set if index too large
43698 ; USES     BX, DI, ES
43699 ;-----
43700 ;
43701 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43702 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43703
43704 SFFromSFN:
43705     ; 05/05/2019 - Retro DOS v4.0
43706     ;getdseg <es>            ; es -> dosdata
43707     mov     es,[cs:DosDSeg]
43708
43709     ;LES     DI,[CS:SFT_ADDR]    ; (es:di) = start of SFT table
43710     les     di,[es:SFT_ADDR]
43711
43712 sfsfn5:
43713     ;cmp     bx,[es:di+4]
43714     CMP     BX,[ES:DI+SFT.SFCount] ; is handle in this table?
43715     JB      short sfsfn7        ; yes, go grab it
43716     ;sub     bx,[es:di+4]
43717     SUB     BX,[ES:DI+SFT.SFCount]
43718     les     di,[es:di] ; 14/08/2018
43719     ;LES     DI,[ES:DI+SFT.SFLink] ; get next table segment
43720     CMP     DI,-1                ; end of tables?
43721     JNZ     short sfsfn5        ; no, try again
43722     STC
43723     retn                    ; return with error, not found
43724
43725 sfsfn7:
43726     push    ax
43727     ;mov     ax,53 ; MSDOS 3.3
43728     ;mov     ax,59 ; MSDOS 5.0 to PCDOS 7.1 (to win ME) ; 11/03/2024
43729     MOV     AX,SF_ENTRY.size    ; put it in a nice place
43730
43731     ; 17/12/2022
43732     mov     al,SF_ENTRY.size ; 28/05/2019
43733     ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43734     ;mov     ax,SF_ENTRY.size ; 59
43735
43736     MUL     BL                ; (ax) = offset into this SF block
43737     ADD     DI,AX             ; add base of SF block
43738     pop     ax
43739     ;add     di,6
43740     ADD     DI,SFT.SFTable    ; offset into structure, 'C' cleared
43741     retn                    ; return with 'C' clear
43742
43743 ; BREAK <JFNFree - return a jfn pointer if one is free>
43744 ;
43745 ;** JFNFree - Find a Free JFN Slot
43746 ;-----
43747 ; JFNFree scansthrough the JFN table and returns a pointer to a free slot
43748 ;
43749 ; ENTRY    (ss) = DOSDATA
43750 ; EXIT     'C' clear if OK
43751 ;          (bx) = new handle
43752 ;          (es:di) = pointer to JFN slot
43753 ;          'C' set if error
43754 ;          (al) = error code
43755 ; USES     bx, di, es, flags
43756 ;-----
43757 JFNFree:
43758     XOR     BX,BX            ; (bx) = initial JFN to try
43759     jfnf1:

```

```

43759 00007710 E89EFF      CALL    pJFNFromHandle    ; get the appropriate handle
43760 00007713 7209        JC      short jfnf5      ; no more handles
43761 00007715 26803DFF    CMP     BYTE [ES:DI],-1    ; free?
43762 00007719 7405        JE      short jfnfx      ; yes, carry is clear
43763 0000771B 43          INC     BX              ; no, next handle
43764 0000771C EBF2        JMP     short jfnf1      ; and try again
43765
43766 ; Error. 'C' set
43767 jfnf5:
43768 ;mov    al,4
43769 0000771E B004        MOV     AL,error_too_many_open_files
43770 jfnfx:
43771 00007720 C3          retn              ; bye
43772
43773 ; BREAK <SFNFree - Allocate a free SFN>
43774
43775 ;** SFNFree - Allocate a Free SFN/SFT
43776 ;-----
43777 ; SFNFree scans through the sf table looking for a free entry
43778 ; If it finds one it partially allocates it by setting SFT_REF_COUNT = -1
43779 ;
43780 ; The problem is that we want to mark the SFT busy so that other threads
43781 ; can't allocate the SFT before we're finished marking it up. However,
43782 ; we can't just mark it busy because we may get blown out of our open
43783 ; by INT24 and leave the thing orphaned. To solve this we mark it
43784 ; "allocation in progress" by setting SFT_REF_COUNT = -1. If we see
43785 ; an SFT with this value we look to see if it belongs to this user
43786 ; and process. If it does belong to us then it must be an orphan
43787 ; and we reclaim it.
43788 ;
43789 ; BUGBUG - improve the performance. I guess it's smaller to call SFFromSFN
43790 ; over and over, but we could at least set a high water mark...
43791 ; cause an N^2 loop calling slow SFFromSFN is real slow, too slow
43792 ; even though this is not a frequently called routine - jgl
43793 ;
43794 ; ENTRY (ss) = DOSDATA
43795 ; EXIT 'C' clear if no error
43796 ; (bx) = SFN
43797 ; (es:di) = pointer to SFT
43798 ; es:[di].SFT_REF_COUNT = -1
43799 ; 'C' set if error
43800 ; (al) = error code
43801 ; USES bx, di, es, Flags
43802 ;-----
43803
43804 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43805 ; DOSCODE:A682h (MSDOS 5.0 MSDOS.SYS)
43806
43807 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43808 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B8DEh
43809
43810 SFNFree:
43811 ; 12/08/2018
43812 ; 05/08/2018 - Retro DOS v3.0
43813 ;
43814 ; MSDOS 6.0
43815 00007721 50          push    ax
43816 00007722 31DB      xor     bx,bx          ; (bx) = SFN to consider
43817 sfnf5:
43818 00007724 53          push    bx
43819 00007725 E8BCFF    call    SFFromSFN        ; get the potential handle
43820 00007728 5B          pop     bx
43821 00007729 723A      JC      short sfnf95      ; no more free SFNs
43822 0000772B 26833D00    cmp     word [ES:DI],0
43823 0000772C 00          ;cmp    word [ES:DI+SF_ENTRY.sf_ref_count],0 ; free?
43824 0000772F 741D      JE      short sfnf20      ; yep, got one
43825
43826 ;cmp     word [es:di],0FFFFh ; -1
43827 00007731 26833DFF    cmp     word [ES:DI],sf_busy
43828 ;cmp     word [ES:DI+SF_ENTRY.sf_ref_count],sf_busy
43829 00007735 7403      JE      short sfnf10      ; special busy mark
43830 sfnf7:
43831 00007737 43          inc     bx              ; try the next one
43832 00007738 EBEA      jmp     short sfnf5
43833
43834 ; The SFT has the special "busy" mark; if it belongs to us then
43835 ; it was abandoned during a earlier call and we can use it.
43836 ;
43837 ; (bx) = SFN
43838 ; (es:di) = pointer to SFT
43839 ; (TOS) = caller's (ax)
43840
43841 sfnf10:
43842 0000773A 36A1[3E03]    mov     ax,[SS:USER_ID]
43843 ;cmp     [es:di+2Fh],ax
43844 0000773E 2639452F    cmp     [ES:DI+SF_ENTRY.sf_UID],ax
43845 00007742 75F3      JNZ     short sfnf7       ; not ours
43846 00007744 36A1[3C03]    mov     ax,[SS:PROC_ID]
43847 ;cmp     [es:di+31h],ax
43848 00007748 26394531    cmp     [ES:DI+SF_ENTRY.sf_PID],ax
43849 0000774C 75E9      JNZ     short sfnf7       ; can't use this one, try the next
43850
43851 ; We have an SFT to allocate
43852 ;
43853 ; (bx) = SFN
43854 ; (es:di) = pointer to SFT
43855 ; (TOS) = caller's (ax)
43856
43857 sfnf20:
43858 ; cf = 0 ;; Retro DOS v3.0
43859
43860 ;mov     word [es:di],0FFFFh
43861 0000774E 26C705FFFF    mov     word [ES:DI],sf_busy
43862 ; make sure that this is allocated
43863 ;mov     word [ES:DI+SF_ENTRY.sf_ref_count],sf_busy
43864
43865 00007753 36A1[3E03]    mov     ax,[SS:USER_ID]
43866 ;mov     [es:di+2Fh],ax
43867 00007757 2689452F    mov     [ES:DI+SF_ENTRY.sf_UID],ax
43868 0000775B 36A1[3C03]    mov     ax,[SS:PROC_ID]
43869 ;mov     [es:di+31h],ax
43870 0000775F 26894531    mov     [ES:DI+SF_ENTRY.sf_PID],ax
43871 sfnf21: ;; Retro DOS v3.0
43872 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43873 ;pop     ax
43874 ;;clc
43875 ;retn
43876 ; 17/12/2022
43877 00007763 58          pop     ax
43878 ;;clc
43879 00007764 C3          retn
43880
43881 ;** Error - no more free SFNs
43882 ;

```

```

43883 ; 'c' set
43884 ; (TOS) = saved ax
43885
43886 sfnf95:
43887 00007765 58      pop     ax
43888
43889 ; 11/03/2024
43890 %if 0
43891 ;mov     al,4
43892 mov     al,error_too_many_open_files
43893 retn                                ; return with 'c' and error
43894 %else
43895 ; 11/03/2024 (PCDOS 7.1 IBMDOS.COM)
43896 ;;;
43897 00007766 EBB6     jmp     short jfnf5
43898 ;;;
43899 %endif
43900
43901 ;=====
43902 ; HANDLE.ASM, MSDOS 6.0, 1991
43903 ;=====
43904 ; 13/07/2018 - Retro DOS v3.0
43905 ; 20/05/2019 - Retro DOS v4.0
43906 ; 11/03/2024 - Retro DOS v5.0
43907
43908 ; DOSCODE:A72Bh (MSDOS 6.21, MSDOS.SYS)
43909
43910 ; BREAK <$Close - return a handle to the system>
43911 ;-----
43912 ;
43913 ;** $Close - Close a file Handle
43914 ;
43915 ; BUGBUG - close gets called a LOT with invalid handles - sizzle that
43916 ; path
43917 ;
43918 ; Assembler usage:
43919 ;     MOV     BX, handle
43920 ;     MOV     AH, Close
43921 ;     INT     int_command
43922 ;
43923 ; ENTRY (bx) = handle
43924 ; EXIT  <normal INT21 return convention>
43925 ; USES   all
43926 ;-----
43927 ;
43928 ;
43929 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43930 ; DOSCODE:A6CBh (MSDOS 5.0 MSDOS.SYS)
43931
43932 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43933 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B926h
43934
43935 ; (Windows ME IO.SYS - BIOSCODE:0A145h)
43936 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A72Bh)
43937
43938 _$CLOSE:
43939 ; Grab the SFT pointer from the JFN.
43940
43941 00007768 E8F602     call    CheckOwner      ; get system file entry
43942 0000776B 722B     jc      short CloseError    ; error return
43943 0000776D 16       push    ss
43944 0000776E 1F       pop     ds
43945 0000776F 893E[9E05]  mov     [THISFT],DI      ; For DOS_CLOSE
43946 00007773 8C06[A005]  mov     [THISFT+2],ES    ; save offset of pointer
43947 ; save segment value
43948
43949 ; DS:SI point to JFN table entry.
43950 ; ES:DI point to SFT
43951 ;
43952 ; We now examine the user's JFN entry; If the file was a 70-mode file (network
43953 ; FCB, we examine the ref count on the SFT; if it was 1, we free the JFN.
43954 ; If the file was not a net FCB, we free the JFN too.
43955
43956 00007777 26833D01  cmp     word [ES:DI+SF_ENTRY.sf_ref_count],1
43957 0000777B 740A     jz      short FreeJFN      ; will the SFT become free?
43958 ;mov     al,[ES:DI+2]
43959 0000777D 268A4502  mov     AL,[ES:DI+SF_ENTRY.sf_mode]
43960 ;and     al,0F0h
43961 00007781 24F0     and     AL,SHARING_MASK
43962 ;cmp     al,70h
43963 00007783 3C70     cmp     AL,SHARING_NET_FCB
43964 00007785 7407     jz      short PostFree    ; 70-mode and big ref count => free it
43965
43966 ; The JFN must be freed. Get the pointer to it and replace the contents with
43967 ; -1.
43968
43969 FreeJFN:
43970 00007787 E827FF     call    pJFNFromHandle    ; d = pJFN (handle);
43971 0000778A 26C605FF  mov     BYTE [ES:DI],0FFh ; release the JFN
43972
43973 PostFree:
43974
43975 ; ThisSFT is correctly set, we have DS = DOSDATA. Looks OK for a DOS_CLOSE!
43976
43977 call     DOS_CLOSE
43978
43979 ; DOS_Close may return an error. If we see such an error, we report it but
43980 ; the JFN stays closed because DOS_Close always frees the SFT!
43981
43982 00007791 7205     jc      short CloseError
43983 ;mov     ah,3Eh
43984 00007793 B43E     mov     AH,CLOSE        ; MZ Bogus multiplan fix
43985 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43986 00007795 E9D68E     jmp     SYS_RET_OK
43987
43988 CloseError:
43989 00007798 E9DD8E     jmp     SYS_RET_ERR
43990
43991 ; BREAK <$Commit - commit the file>
43992 ;-----
43993 ;
43994 ;** $Commit - Commit a File
43995 ;
43996 ; $Commit "commits" a file to disk - all of it's buffers are
43997 ; flushed out. BUGBUG - I'm pretty sure that $Commit doesn't update
43998 ; the directory entry, etc., so this commit is pretty useless. check
43999 ; and fix this!! jgl
44000 ;
44001 ; Assembler usage:
44002 ;     MOV     BX, handle
44003 ;     MOV     AH, Commit
44004 ;     INT     int_command
44005 ;
44006 ; ENTRY (bx) = handle

```

```

44007 ; EXIT none
44008 ; USES all
44009 ;-----
44010
44011 _$COMMIT:
44012 ; Grab the SFT pointer from the JFN.
44013
44014 0000779B E8C302 call CheckOwner ; get system file entry
44015 ;JC short CommitError ; error return
44016 ; 11/03/2024
44017 0000779E 72F8 jc short CommitError
44018
44019 000077A0 16 push ss
44020 000077A1 1F pop ds ; For DOS_COMMIT
44021 000077A2 893E[9E05] MOV [THISST],DI ; save offset of pointer
44022 000077A6 8C06[A005] MOV [THISST+2],ES ; save segment value
44023
44024 ; ThisST is correctly set, we have DS = DOSDATA. Looks OK for a DOS_COMMIT
44025 ;
44026 ; ES:DI point to SFT
44027
44028 000077AA E826C1 call DOS_COMMIT
44029 000077AD 72E9 JC short CommitError
44030 ; 07/12/2022
44031 ;jc short CloseError
44032 ;mov ah,68h
44033 000077AF B468 MOV AH,COMMIT
44034 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44035 ;jmp SYS_RET_OK
44036 CommitOk:
44037 000077B1 EBE2 jmp short CloseOk
44038
44039 ; 11/03/2024
44040 ;CommitError:
44041 ; ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44042 ; ;jmp SYS_RET_ERR
44043 ; jmp short CloseError
44044
44045 ; BREAK <$ExtHandle - extend handle count>
44046
44047 ;** $ExtHandle - Extend Handle Count
44048 ;-----
44049 ; Assembler usage:
44050 ; MOV BX, Number of Opens Allowed (MAX=65534;66535 is
44051 ; MOV AX, 6700H reserved to mark SFT
44052 ; INT int_command busy )
44053 ;
44054 ; ENTRY (bx) = new number of handles
44055 ; EXIT 'C' clear if OK
44056 ; 'C' set iff err
44057 ; (ax) = error code
44058 ; AX = error_not_enough_memory
44059 ; error_too_many_open_files
44060 ;
44061 ; USES all
44062 ;-----
44063 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44064
44065 _$ExtHandle:
44066 000077B3 31ED XOR BP,BP ; 0: enlarge 1: shrink 2:psp
44067 ;cmp bx,20
44068 000077B5 83FB14 CMP BX,FILPERPROC
44069 000077B8 7303 JAE short exth2 ; Don't set less than FilPerProc no
44070 000077BA BB1400 MOV BX,FILPERPROC
44071 exth2:
44072 000077BD 368E06[3003] MOV ES,[ss:CurrentPDB] ; get user process data block;smr;SS Override
44073 ;mov cx,[ES:32h]
44074 000077C2 268B0E3200 MOV CX,[ES:PDB.JFN_Length] ; get number of handle allowed
44075 000077C7 39CB CMP BX,CX ; the requested == current
44076 ;JE short ok_done ; yes and exit
44077 ; 11/03/2024
44078 000077C9 74CA je short CloseOk
44079 000077CB 771E JA short larger ; go allocate new table
44080
44081 ; We're going to shrink the # of handles available
44082 ;
44083 ;MOV BP,1 ; shrink
44084 ; 11/03/2024
44085 000077CD 45 inc bp
44086
44087 ;mov ds,[ES:36h]
44088 000077CE 268E1E3600 MOV DS,[ES:PDB.JFN_Pointer+2] ;
44089 000077D3 89DE MOV SI,BX ;
44090 000077D5 29D9 SUB CX,BX ; get difference
44091
44092 ; BUGBUG - code a SCASB here, should be a bit smaller
44093 chck_handles:
44094 000077D7 803CFF CMP BYTE [SI],-1 ; scan through handles to ensure close
44095 000077DA 7539 JNZ short too_many_files ; status
44096 000077DC 46 INC SI
44097 000077DD E2F8 LOOP chck_handles
44098
44099 000077DF 83FB14 CMP BX,FILPERPROC ; = 20
44100 000077E2 7707 JA short larger ; no
44101
44102 ;MOV BP,2 ; psp
44103 ; 11/03/2024
44104 000077E4 45 inc bp
44105 ;mov di,24
44106 000077E5 BF1800 MOV DI,PDB.JFN_TABLE ; es:di -> jfn table in psp
44107 000077E8 53 PUSH BX
44108 000077E9 EB1B JMP short movhandl
44109
44110 larger:
44111 ;CMP BX,-1 ; 65535 is not allowed
44112 ;JZ short invalid_func ; 10/08/2018
44113 ; 11/03/2024 (PCDOS 7.1 IBMDOS.COM)
44114 ;;;
44115 000077EB 43 inc bx
44116 000077EC 747D jz short invalid_func
44117 000077EE 4B dec bx
44118 ;;;
44119 000077EF F8 CLC
44120 000077F0 53 PUSH BX ; save requested number
44121 000077F1 83C30F ADD BX,0FH ; adjust to paragraph boundary
44122 000077F4 B104 MOV CL,4
44123 ;ror bx,cl ; MSDOS 3.3
44124 000077F6 D3DB RCR BX,CL ; DOS 4.00 fix ;AC000;
44125 ;AND BX,1FFFH ; clear most 3 bits
44126 ; 01/07/2024
44127 000077F8 80E71F and bh,1Fh
44128
44129 000077FB 55 PUSH BP
44130 000077FC E80AFB call _$ALLOC ; allocate memory

```

```

44131 000077FF 5D          POP     BP
44132 00007800 7264        JC      short no_memory          ; not enough memory
44133
44134 00007802 8EC0        MOV     ES,AX          ; es:di points to new table memory
44135 00007804 31FF        XOR     DI,DI
44136 movhand1:
44137 00007806 368E1E[3003] MOV     DS,[ss:CurrentPDB]      ; get user PDB address;smr;SS Override
44138
44139 0000780B F7C50300      test    BP,3              ; enlarge ?
44140 0000780F 7409        JZ      short enlarge          ; yes
44141 00007811 59          POP     CX                ; cx = the amount you shrink
44142 00007812 51          PUSH    CX
44143 00007813 EB09        JMP     short copy_hand
44144
44145 ; Done. 'c' clear
44146
44147 ; 17/12/2022
44148 ;ok_done:
44149 ; ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44150 ; ; jmp short CommitOk
44151 ; ; 17/12/2022
44152 ; jmp SYS_RET_OK
44153
44154 too_many_files:
44155 ;mov al,4
44156 00007815 B004        MOV     AL,error_too_many_open_files
44157 ; ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44158 ; jmp SYS_RET_ERR
44159 CommitErrorj:
44160 ; jmp short CommitError
44161 ; ; 17/12/2022
44162 00007817 E95E8E      jmp     SYS_RET_ERR
44163
44164 ; 11/03/2024
44165 ; 17/12/2022
44166 ;ok_done:
44167 ; ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44168 ; jmp short CommitOk
44169 ; ; 17/12/2022
44170 ; jmp SYS_RET_OK
44171
44172 enlarge:
44173 ;mov cx,[32h]
44174 0000781A 8B0E3200      MOV     CX,[PDB.JFN_Length]      ; get number of old handles
44175 copy_hand:
44176 MOV     DX,CX
44177 ;lds si,[34h]
44178 00007820 C5363400      LDS     SI,[PDB.JFN_Pointer]      ; get old table pointer
44179 00007824 F3A4        REP     MOVSB          ; copy information to new table
44180 00007826 59          POP     CX                ; get new number of handles
44181 00007827 51          PUSH    CX                ; save it again
44182 00007828 29D1        SUB     CX,DX              ; get the difference
44183 0000782A B0FF        MOV     AL,-1              ; set availability to handles
44184 0000782C F3AA        REP     STOSB
44185 0000782E 368E1E[3003] MOV     DS,[ss:CurrentPDB]      ; get user process data block;smr;SS Override
44186 ;cmp word [34h],0
44187 00007833 833E340000    CMP     WORD [PDB.JFN_Pointer],0 ; check if original table pointer
44188 00007838 750D        JNZ     short update_info      ; yes, go update PDB entries
44189 0000783A 55          PUSH    BP
44190 0000783B 1E          PUSH    DS                ; save old table segment
44191 0000783C 06          PUSH    ES                ; save new table segment
44192 0000783D 8E063600      MOV     ES,[PDB.JFN_Pointer+2] ; get old table segment
44193 00007841 E83AFC        call    _$DEALLOC          ; deallocate old table memory
44194 00007844 07          POP     ES                ; restore new table segment
44195 00007845 1F          POP     DS                ; restore old table segment
44196 00007846 5D          POP     BP
44197
44198 update_info:
44199 00007847 F7C50200      test    BP,2              ; psp?
44200 0000784B 7408        JZ      short non_psp          ; no
44201 ;mov word [34h],18h ; 24
44202 0000784D C70634001800 MOV     WORD [PDB.JFN_Pointer],PDB.JFN_TABLE ; restore
44203 00007853 EB06        JMP     short final
44204 non_psp:
44205 ;mov word [34h],0
44206 00007855 C70634000000 MOV     WORD [PDB.JFN_Pointer],0 ; new table pointer offset always 0
44207 final:
44208 ;mov [36h],es
44209 0000785B 8C063600      MOV     [PDB.JFN_Pointer+2],ES ; update table pointer segment
44210 ;pop word [32h]
44211 0000785F 8F063200      POP     word [PDB.JFN_Length] ; restore new number of handles
44212 ; 11/03/2024
44213 ok_done:
44214 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44215 00007863 E9088E      jmp     SYS_RET_OK
44216 ;ok_done_j:
44217 ; jmp short ok_done
44218
44219 no_memory:
44220 00007866 5B          POP     BX                ; clean stack
44221 ;mov al,8
44222 00007867 B008        MOV     AL,error_not_enough_memory
44223 ; ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44224 ; jmp SYS_RET_ERR
44225 CommitErrorj2:
44226 00007869 EBAC        jmp     short CommitErrorj
44227
44228 invalid_func:
44229 ;mov al,1
44230 0000786B B001        MOV     AL,error_invalid_function
44231 ; ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44232 ; jmp SYS_RET_ERR
44233 CommitErrorj3:
44234 ; jmp short CommitErrorj2
44235 ; ; 17/12/2022
44236 0000786D EBA8        jmp     short CommitErrorj
44237
44238 ; 20/05/2019 - Retro DOS v4.0
44239 ; DOSCODE:A83Ah (MSDOS 6.21, MSDOS.SYS)
44240
44241 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44242 ; DOSCODE:A7DAh (MSDOS 5.0 MSDOS.SYS)
44243
44244 ; BREAK <$READ - Read from a file handle>
44245 ;-----
44246 ;
44247 ;** $Read - Read from a File Handle
44248 ;
44249 ; Assembler usage:
44250 ;
44251 ; LDS     DX, buf
44252 ; MOV     CX, count
44253 ; MOV     BX, handle
44254 ; MOV     AH, Read

```

```

44255 ; INT int_command
44256 ; AX has number of bytes read
44257 ;
44258 ; ENTRY (bx) = file handle
44259 ; (cx) = byte count
44260 ; (ds:dx) = buffer address
44261 ; EXIT Through system call return so that to user:
44262 ; 'C' clear if OK
44263 ; (ax) = bytes read
44264 ; 'C' set if error
44265 ; (ax) = error code
44266 ;
44267 ;-----
44268 ;
44269 ; 12/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
44270 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0BA2Eh
44271 ;
44272 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A83Ah)
44273 ; (Windows ME IO.SYS - BIOSCODE:0A256h)
44274 ;
44275 _$READ:
44276 0000786F BE[733B] MOV SI,DOS_READ
44277 ReadDo:
44278 00007872 E83CFE call pJFNFromHandle
44279 00007875 7208 JC short ReadError
44280 ;
44281 00007877 268A05 MOV AL,[ES:DI]
44282 0000787A E8E401 call CheckOwner ; get the handle
44283 0000787D 7303 JNC short ReadSetup ; no errors do the operation
44284 ;
44285 ; Have an error. 'C' set
44286 ;
44287 ReadError:
44288 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44289 ;;; jmp SYS_RET_ERR ; go to error traps
44290 ; jmp short CommitErrorj3
44291 ; 17/12/2022
44292 0000787F E9F68D jmp SYS_RET_ERR
44293 ;
44294 ReadSetup:
44295 00007882 36893E[9E05] MOV [ss:THISSFT],DI ; save offset of pointer;smr;SS Override
44296 00007887 368C06[A005] MOV [ss:THISSFT+2],ES ; save segment value ;smr;SS Override
44297 ; 20/05/2019 - Retro DOS v4.0
44298 ; MSDOS 6.0
44299 ;;; Extended Open
44300 ; test byte [es:di+3],20h
44301 0000788C 26F6450320 test byte [ES:DI+SF_ENTRY.sf_mode+1],(INT_24_ERROR>>8)
44302 ; ;AN000;;EO. need i24
44303 00007891 7406 JZ short needi24 ;AN000;;EO. yes
44304 00007893 36800E[F605]02 OR byte [ss:EXTOPEN_ON],EXT_OPEN_I24_OFF ; 2
44305 ; ;AN000;;EO. set it off;smr;SS Override
44306 needi24: ;AN000;
44307 ;
44308 ; 12/03/2024
44309 %if 0
44310 ;;; Extended Open
44311 push word [ss:DMAADD]
44312 push word [ss:DMAADD+2] ;smr;SS Override
44313 ;;;
44314 ; BAD SPOT FOR 286!!! SEGMENT ARITHMETIC!!!
44315 ;
44316 ; 26/07/2019
44317 ;
44318 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44319 ;
44320 ; (It is not necessary to call 'Align_Buffer' proc here/below because
44321 ; there is not another caller; it is better to put the code in this proc
44322 ; here instead of calling it as a subroutine; but I have modified code
44323 ; here for MSDOS 5.0 MSDOS.SYS address compatibility)
44324 ;
44325 ; MSDOS 6.0
44326 CALL Align_Buffer ;AN000;MS. align user's buffer
44327 ;
44328 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44329 ; MSDOS 3.3
44330 ; MOV BX,DX ; copy offset
44331 ; push CX ; don't stomp on count
44332 ; MOV CL,4 ; bits to shift bytes->para
44333 ; SHR BX,CL ; get number of paragraphs
44334 ; pop CX ; get count back
44335 ; MOV AX,DS ; get original segment
44336 ; ADD AX,BX ; get new segment
44337 ; MOV DS,AX ; in seg register
44338 ; AND DX,0Fh ; normalize offset
44339 ; MOV [ss:DMAADD],DX ; use user DX as offset ;smr;SS Override
44340 ; MOV [ss:DMAADD+2],DS ; use user DS as segment for DMA
44341 ; ;smr;SS Override
44342 %else
44343 ; 12/03/2024 (PC DOS 7.1 IBMDOS.COM)
44344 ;;;
44345 mov ax,ds ; original segment
44346 00007899 8CD8 lds bx,[ss:DMAADD]
44347 0000789B 36C51E[2C03] push bx
44348 000078A0 53 push ds
44349 000078A1 1E mov bx,dx
44350 000078A2 89D3 shr bx,1
44351 000078A4 D1EB shr bx,1
44352 000078A6 D1EB shr bx,1
44353 000078A8 D1EB shr bx,1
44354 000078AA D1EB add ax,bx ; new segment
44355 000078AC 01D8 and dx,0Fh ; normalize offset
44356 000078AE 83E20F mov [ss:DMAADD],dx ; use user DX as offset
44357 ; 23/03/2024
44358 000078B1 36A3[2E03] mov [ss:DMAADD+2],ax ; use user DS as segment for DMA
44359 ;;;
44360 %endif
44361 ;;;
44362 ; END BAD SPOT FOR 286!!! SEGMENT ARITHMETIC!!!
44363 ;;;
44364 push ss ; go for DOS addressability
44365 000078B5 16 pop ds
44366 000078B6 1F
44367 ;
44368 ; 12/03/2024 - Retro DOS v5.0
44369 ;;;
44370 mov [DMAADD],dx
44371 000078B7 8916[2C03] ;;;
44372 ;
44373 CALL SI ; DOS_READ ; indirect call to operation
44374 000078BB FFD6
44375 ;
44376 000078BD 8F06[2E03] pop word [DMAADD+2]
44377 000078C1 8F06[2C03] pop word [DMAADD]
44378 ; JNC short READ_OK ;AN002;

```



```

44379             ;JMP     short ReadError             ;AN002; if error, say bye bye
44380             ; 17/12/2022
44381 000078C5 72B8             ;JC     short ReadError
44382             ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44383             ;JNC     short READ_OK             ;AN002;
44384             ;JMP     short ReadError
44385
44386 READ_OK:
44387 000078C7 89C8             MOV     AX,CX             ; get correct return in correct reg
44388 Read_Ok_j:
44389             ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44390             ;JMP     SYS_RET_OK             ; successful return
44391             ;JMP     short ok_done_j
44392             ; 17/12/2022
44393 000078C9 E9A28D             JMP     SYS_RET_OK
44394
44395 ; 13/07/2018 - Retro DOS v3.0
44396
44397 ;-----
44398 ;
44399 ; Input: DS:DX points to user's buffer addr
44400 ; Function: rearrange segment and offset for READ/WRITE buffer
44401 ; Output: [DMAADD] set
44402
44403 ; 12/03/2024
44404 %if 0
44405
44406 ; 20/05/2019 - Retro DOS v4.0
44407 ; 26/07/2019
44408 ; ; MSDOS 6.0
44409 ;Align_Buffer:
44410 ; MOV     BX,DX             ; copy offset
44411 ; push    CX             ; don't stomp on count
44412 ; MOV     CL,4             ; bits to shift bytes->para
44413 ; SHR     BX,CL             ; get number of paragraphs
44414 ; pop     CX             ; get count back
44415 ; MOV     AX,DS             ; get original segment
44416 ; ADD     AX,BX             ; get new segment
44417 ; MOV     DS,AX             ; in seg register
44418 ; AND     DX,0Fh             ; normalize offset
44419 ; MOV     [ss:DMAADD],DX     ; use user DX as offset ;smr;SS Override
44420 ; MOV     [ss:DMAADD+2],DS   ; use user DS as segment for DMA
44421 ;                                     ;smr;SS Override
44422 ;
44423 ; retn
44424
44425 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44426 ;Align_Buffer:
44427 ; MOV     BX,DX             ; copy offset
44428 ; push    CX             ; don't stomp on count
44429 ; MOV     CL,4             ; bits to shift bytes->para
44430 ; SHR     BX,CL             ; get number of paragraphs
44431 ; pop     CX             ; get count back
44432 ; MOV     AX,DS             ; get original segment
44433 ; ADD     AX,BX             ; get new segment
44434 ; MOV     DS,AX             ; in seg register
44435 ; AND     DX,0Fh             ; normalize offset
44436 ; MOV     [ss:DMAADD],DX     ; use user DX as offset ;smr;SS Override
44437 ; MOV     [ss:DMAADD+2],DS   ; use user DS as segment for DMA
44438 ;                                     ;smr;SS Override
44439 ;
44440 ; retn
44441
44442 %endif
44443
44444 ; 20/05/2019 - Retro DOS v4.0
44445 ; DOSCODE:A8A0h (MSDOS 6.21, MSDOS.SYS)
44446
44447 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44448 ; DOSCODE:A840h (MSDOS 5.0 MSDOS.SYS)
44449
44450 ; 12/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44451 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BA8Ch)
44452
44453 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A8A0h)
44454 ; (Windows ME IO.SYS - BIOSCODE:0A2B9h)
44455
44456 ;BREAK <$WRITE - write to a file handle>
44457 ;-----
44458 ;
44459 ; Assembler usage:
44460 ; LDS     DX, buf
44461 ; MOV     CX, count
44462 ; MOV     BX, handle
44463 ; MOV     AH, write
44464 ; INT     int_command
44465 ; AX has number of bytes written
44466 ; Errors:
44467 ; AX = write_invalid_handle
44468 ;       = write_access_denied
44469 ;
44470 ; Returns in register AX
44471 ;
44472 ;-----
44473
44474 000078CC BE[743D]             _$WRITE:
44475 000078CF EBA1             MOV     SI,DOS_WRITE
44476             JMP     short ReadDo
44477
44478 ;BREAK <$LSEEK - move r/w pointer>
44479 ;-----
44480 ;
44481 ; Assembler usage:
44482 ; MOV     DX, offsetlow
44483 ; MOV     CX, offsethigh
44484 ; MOV     BX, handle
44485 ; MOV     AL, method
44486 ; MOV     AH, LSeek
44487 ; INT     int_command
44488 ; DX:AX has the new location of the pointer
44489 ; Error returns:
44490 ; AX = error_invalid_handle
44491 ;       = error_invalid_function
44492 ; Returns in registers DX:AX
44493 ;-----
44494
44495 ; 21/05/2019 - Retro DOS v4.0
44496 ; DOSCODE:A8A5h (MSDOS 6.21, MSDOS.SYS)
44497
44498 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44499 ; DOSCODE:A845h (MSDOS 5.0 MSDOS.SYS)
44500
44501 ; 12/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44502 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BA91h

```

```

44503
44504
44505 000078D1 E88D01    _LSEEK:
44506                      call    CheckOwner          ; get system file entry
44507                      ; 17/12/2022
44508 ;LSeekError:
44509                      ;JNC     short CHKOWN_OK          ;AN002;
44510                      ;JMP     short ReadError          ;AN002; error return
44511                      ; 17/12/2022
44512                      ; 02/06/2019
44513 000078D4 72A9      jc      short ReadError
44514                      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44515                      ;JNC     short CHKOWN_OK          ;AN002;
44516                      ;JMP     short ReadError          ;AN002; error return
44517
44518
44519                      CHKOWN_OK:
44520                      CMP      AL,2                    ;AN002;
44521 000078D6 3C02      jbe      short LSeekDisp          ; is the seek value correct?
44522                      ; yes, go dispatch
44523                      ; 12/03/2024
44524 %if 0
44525                      ;mov     byte [ss:EXTERR_LOCUS],1
44526                      MOV      byte [ss:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
44527                      ;smr;SS Override
44528 %else
44529                      ; 12/03/2024 (PCDOS 7.1 IBMDOS.COM)
44530                      ;;;
44531 000078DA E8009A      call    set_exerr_locus_unk ; Extended Error Locus
44532                      ;;;
44533 %endif
44534                      ;mov     al,1
44535 000078DD B001      mov     al,error_invalid_function ; invalid method
44536                      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44537 ;LSeekError2:
44538 000078DF EB9E      jmp     short ReadError
44539
44540
44541 000078E1 3C01      LSeekDisp:
44542 000078E3 720A      CMP      AL,1                    ; best way to dispatch; check middle
44543 000078E5 771B      JB      short LSeekStore          ; just store CX:DX
44544                      JA      short LSeekEOF          ; seek from end of file
44545                      ;add     dx,[es:di+21]
44546                      ADD      DX,[ES:DI+SF_ENTRY.sf_position]
44547 000078EB 26035515  ;adc     cx,[es:di+23]
44548                      ADC      CX,[ES:DI+SF_ENTRY.sf_position+2]
44549 000078EF 89C8      LSeekStore:
44550 000078F1 92      MOV      AX,CX                    ; AX:DX
44551                      XCHG     AX,DX                    ; DX:AX is the correct value
44552 ;LSeekSetpos:
44553                      ;mov     [es:di+21],ax
44554                      MOV      [ES:DI+SF_ENTRY.sf_position],AX
44555                      ;mov     [es:di+23],dx
44556                      MOV      [ES:DI+SF_ENTRY.sf_position+2],DX
44557                      call    Get_User_Stack
44558                      ;mov     [si+6],dx
44559                      MOV      [SI+user_env.user_DX],DX ; return DX:AX
44560                      ;jmp     SYS_RET_OK                ; successful return
44561                      ; 25/06/2019
44562                      ;jmp     SYS_RET_OK_c1c
44563                      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44564                      ;jmp     SYS_RET_OK_c1c
44565 00007900 EBC7      LSeekOK:
44566                      jmp     short Read_Ok
44567
44568
44569                      LSeekEOF:
44570                      ;;test word [es:di+5],8000h
44571                      ;test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
44572                      ; 21/05/2019 - Retro DOS v4.0
44573                      test     byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
44574                      JNZ      short Check_LSeek_Mode; Is Net
44575                      LOCAL_LSeek:
44576                      ;add     dx,[es:di+17]
44577                      ADD      DX,[ES:DI+SF_ENTRY.sf_size]
44578                      ;adc     cx,[es:di+19]
44579                      ADC      CX,[ES:DI+SF_ENTRY.sf_size+2]
44580                      JMP      short LSeekStore          ; go and set the position
44581
44582
44583                      Check_LSeek_Mode:
44584                      ;;test word [es:di+2],8000h
44585                      ;test word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB
44586                      ; 21/05/2019
44587                      test     byte [ES:DI+SF_ENTRY.sf_mode+1],(sf_isFCB>>8)
44588                      JNZ      short LOCAL_LSeek          ; FCB treated like local file
44589                      ;mov     ax,[es:di+2]
44590                      MOV      AX,[ES:DI+SF_ENTRY.sf_mode]
44591                      ;and     ax,0F0h
44592                      AND      AX,SHARING_MASK
44593                      ;cmp     ax,40h
44594                      CMP      AX,SHARING_DENY_NONE
44595                      JZ      short NET_LSEEK              ; LSEEK exported in this mode
44596                      ;cmp     ax,30h
44597                      CMP      AX,SHARING_DENY_READ
44598                      JNZ      short LOCAL_LSeek          ; Treated like local Lseek
44599                      NET_LSEEK:
44600                      ; JMP      short LOCAL_LSeek
44601                      ; REMOVE ABOVE INSTRUCTION TO ENABLE DCR 142
44602                      ;CallInstall Net_Lseek,MultNET,33
44603                      ;JNC     short LSeekSetPos
44604                      mov     ax,1121h
44605                      int      2Fh ; Multiplex - NETWORK REDIRECTOR - SEEK FROM END OF REMOTE FILE
44606                      ; CX:DX = offset (in bytes) from end
44607                      ; ES:DI -> SFT, SFT DPB field -> DPB of drive with file
44608                      ; SS = DOS CS
44609                      ; Return: CF set on error
44610                      ; CF clear if successful, DX:AX = new file position
44611                      jnb     short LSeekSetpos
44612                      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44613                      ;jmp     SYS_RET_ERR
44614 ;LSeekError3:
44615                      ; 17/12/2022
44616                      ;jmp     short LSeekError2
44617 00007932 E9438D      DupErr: ; 17/12/2022
44618                      jmp     SYS_RET_ERR
44619
44620                      ; 21/05/2019 - Retro DOS v4.0
44621                      ; DOSCODE:A95Bh (MSDOS 6.21, MSDOS.SYS)
44622                      ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44623                      ; DOSCODE:A8FBh (MSDOS 5.0 MSDOS.SYS)
44624
44625                      ;BREAK <$DUP - duplicate a jfn>
44626                      ;-----

```



```

44627 ;
44628 ; Assembler usage:
44629 ;     MOV     BX, fh
44630 ;     MOV     AH, Dup
44631 ;     INT     int_command
44632 ;     AX has the returned handle
44633 ; Errors:
44634 ;     AX = dup_invalid_handle
44635 ;     = dup_too_many_open_files
44636 ;
44637 ;-----
44638 ;
44639 ; 12/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44640 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BBEBh
44641 ;
44642 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A95Bh)
44643 ; (Windows ME IO.SYS - BIOSCODE:0A3EFh)
44644 ;
44645 ;
44646 ;_DUP:
44647 ;     MOV     AX,BX           ; save away old handle in AX
44648 ;     call    JFNFree        ; free handle? into ES:DI, new in BX
44649 ; DupErrorCheck:
44650 ;     JC      short DupErr    ; nope, bye
44651 ;     push    es
44652 ;     push    di             ; save away SFT
44653 ;     pop     si             ; into convenient place DS:SI
44654 ;     pop     ds
44655 ;     XCHG    AX,BX          ; get back old handle
44656 ;     call    CheckOwner     ; get sft in ES:DI
44657 ;     JC      short DupErr    ; errors go home
44658 ;     call    DOS_Dup_Direct
44659 ;     call    pJFNFromHandle  ; get pointer
44660 ;     MOV     BL,[ES:DI]     ; get SFT number
44661 ;     MOV     [SI],BL        ; stuff in new SFT
44662 ;     ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44663 ;     jmp     SYS_RET_OK     ; and go home
44664 ;     jmp     short ok_ret
44665 ;
44666 ; 17/12/2022
44667 ; DupErr:
44668 ;     ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44669 ;     jmp     SYS_RET_ERR
44670 ;     jmp     short ft_error
44671 ;
44672 ; BREAK <$DUP2 - force a dup on a particular jfn>
44673 ;-----
44674 ;
44675 ; Assembler usage:
44676 ;     MOV     BX, fh
44677 ;     MOV     CX, newfh
44678 ;     MOV     AH, Dup2
44679 ;     INT     int_command
44680 ; Error returns:
44681 ;     AX = error_invalid_handle
44682 ;
44683 ;-----
44684 ;
44685 ;_DUP2:
44686 ;     push    bx
44687 ;     push    cx             ; save source
44688 ;     MOV     BX,CX          ; get one to close
44689 ;     call    _SCLOSE        ; close destination handle
44690 ;     pop     bx
44691 ;     pop     ax             ; old in AX, new in BX
44692 ;     call    pJFNFromHandle  ; get pointer
44693 ;     JMP     short DupErrorCheck ; check error and do dup
44694 ;
44695 ; BREAK <FileTimes - modify write times on a handle>
44696 ;-----
44697 ;
44698 ; Assembler usage:
44699 ;     MOV     AH, FileTimes (57H)
44700 ;     MOV     AL, func
44701 ;     MOV     BX, handle
44702 ;     ; if AL = 1 then then next two are mandatory
44703 ;     MOV     CX, time
44704 ;     MOV     DX, date
44705 ;     INT     21h
44706 ;     ; if AL = 0 then CX/DX has the last write time/date
44707 ;     ; for the handle.
44708 ;
44709 ; AL=02      get extended attributes
44710 ;     BX=handle
44711 ;     CX=size of buffer (0, return max size )
44712 ;     DS:SI query list (si=-1, selects all EA)
44713 ;     ES:DI buffer to hold EA list
44714 ;
44715 ; AL=03      get EA name list
44716 ;     BX=handle
44717 ;     CX=size of buffer (0, return max size )
44718 ;     ES:DI buffer to hold name list
44719 ;
44720 ; AL=04      set extended attributes
44721 ;     BX=handle
44722 ;     ES:DI buffer of EA list
44723 ;
44724 ; Error returns:
44725 ;     AX = error_invalid_function
44726 ;     = error_invalid_handle
44727 ;
44728 ;-----
44729 ;
44730 ; 21/05/2019 - Retro DOS v4.0
44731 ; DOSCODE:A90Dh (MSDOS 6.21, MSDOS.SYS)
44732 ;
44733 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44734 ; DOSCODE:A8ADh (MSDOS 5.0 MSDOS.SYS)
44735 ;
44736 ; 12/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44737 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BAF5h
44738 ;
44739 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A90Dh)
44740 ; (Windows ME IO.SYS - BIOSCODE:0A327h)
44741 ;
44742 ;_FILE_TIMES:
44743 ;
44744 ; 12/03/2024
44745 ;%if 0
44746 ; 13/07/2018 - Retro DOS v3.0
44747 ;
44748 ; MSDOS 3.3
44749 ; cmp     al,2             ; correct subfunction ?
44750 ; jnb     short ft1

```

```

44751
44752 ;mov byte [ss:EXTERR_LOCUS], 1
44753 ;mov byte [ss:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
44754 ;SS Overr
44755 ;mov al,1
44756 ;mov al,error_invalid_function ; give bad return
44757 ;jmp SYS_RET_ERR
44758
44759 ; MSDOS 6.0
44760 cmp al,2 ; correct subfunction ?
44761 jae short inval_func
44762 ;ft1:
44763 call CheckOwner ; get sft
44764 ; 17/12/2022
44765 jc short LSeekError ; bad handle
44766
44767 or al,al ; get time/date ?
44768 jnz short ft_set_time
44769
44770 ;----- here we get the time & date from the sft for the user
44771
44772 cli ; is this cli/sti reqd ? BUGBUG
44773 ;mov cx,[es:di+13]
44774 mov cx,[es:di+SF_ENTRY.sf_time] ; get the time
44775 ;mov dx,[es:di+15]
44776 mov dx,[es:di+SF_ENTRY.sf_date] ; & date
44777 sti
44778 call Get_User_Stack
44779 ;mov [si+4],cx
44780 mov [si+user_env.user_CX],cx
44781 ;mov [si+6],dx
44782 mov [si+user_env.user_DX],dx
44783 jmp short ok_ret
44784
44785 ;----- here we set the time in sft
44786
44787 ft_set_time:
44788 call ECritSFT
44789 ;mov [es:di+13],cx
44790 mov [es:di+SF_ENTRY.sf_time],cx ; drop in new time
44791 ;mov [es:di+15],dx
44792 mov [es:di+SF_ENTRY.sf_date],dx ; and date
44793
44794 xor ax, ax
44795 call far [ss:JShare+(14*4)] ; 14 = shsu ; SS Override
44796
44797 ;----- set the flags in SFT entry
44798 ;and word [es:di+5],0FFBFh
44799 ; 18/12/2022
44800 ;and byte [es:di+5],0BFh
44801 and byte [es:di+SF_ENTRY.sf_flags],~devid_file_clean
44802 and word [es:di+SF_ENTRY.sf_flags],~devid_file_clean
44803 ; mark file as dirty
44804 ;or word [es:di+5],4000h
44805 ; 17/12/2022
44806 ;or byte [es:di+6],40h
44807 or byte [es:di+SF_ENTRY.sf_flags+1],(sf_close_nodate>>8)
44808 ;or word [es:di+SF_ENTRY.sf_flags],sf_close_nodate
44809 ; ask close not to
44810 ; bother about date
44811 ; and time
44812 call LCritSFT
44813
44814 ok_ret:
44815 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44816 ; 17/12/2022
44817 jmp SYS_RET_OK
44818 ;jmp short LSeekOk
44819
44820 inval_func:
44821 ;mov byte [ss:EXTERR_LOCUS],1
44822 mov byte [ss:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
44823 ;SS Overr
44824 ;mov al,1
44825 mov al,error_invalid_function ; give bad return
44826 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44827 ft_error:
44828 ;jmp SYS_RET_ERR
44829 ;jmp short LSeekError3
44830 ; 17/12/2022
44831 jmp short LSeekError
44832
44833 %else
44834 ; 12/03/2024 - Retro DOS v5.0
44835 ; PC DOS 7.1 IBMDOS.COM
44836 ;;; ; ((Ref: Ralf Brown's Interrupt List))
44837 00007961 3C07 cmp al,7 ; SET CREATION DATE AND TIME
44838 ; 6 = GET CREATION DATE AND TIME
44839 ; 5 = SET LAST ACCESS DATE AND TIME
44840 ;jnb short LSeekError1
44841 ; 12/03/2024
44842 00007963 737B jnb short inval_func
44843 00007965 3C04 cmp al,4 ; GET LAST ACCESS DATE AND TIME
44844 00007967 7304 jnb short ft1
44845 00007969 3C02 cmp al,2 ; 0 = GET FILE'S LAST-WRITTEN DATE AND TIME
44846 ; 1 = SET FILE'S LAST-WRITTEN DATE AND TIME
44847 ;jnb short LSeekError1 ; 2,3 -> invalid
44848 ; 12/03/2024
44849 0000796B 7373 jnb short inval_func
44850
44851 0000796D E8F100 ft1: call CheckOwner ; get sft
44852 00007970 72C0 jc short LSeekError ; bad handle
44853
44854 00007972 3C01 cmp al,1
44855 00007974 773A ja short ft2 ; PC DOS 7.1 (MSDOS 7) sub function
44856
44857 00007976 08C0 or al,al ; get time/date ?
44858 00007978 7514 jnz short ft_set_time ; no,set
44859
44860 ;lds cx,[es:di+0Dh]
44861 0000797A 26C54D0D lds cx,[es:di+SF_ENTRY.sf_time]
44862 ; get the time
44863 0000797E 8CDA mov dx,ds ; & date ; [es:di+SF_ENTRY.sf_date]
44864 00007980 E8F48A call Get_User_Stack
44865 ;mov [si+4],cx
44866 00007983 894C04 mov [si+user_env.user_CX],cx
44867 00007986 895406 mov [si+6],dx
44868 00007989 895406 mov [si+user_env.user_DX],dx
44869
44870 0000798C EB1F jmp short ft_ok
44871
44872 ft_set_time:
44873 call ECritSFT
44874

```

```

44875             ;mov     [es:di+0Dh],cx
44876 00007991 26894D0D      mov     [es:di+SF_ENTRY.sf_time],cx ; drop in new time
44877             ;mov     [es:di+0Fh],dx
44878 00007995 2689550F      mov     [es:di+SF_ENTRY.sf_date],dx ; and date
44879
44880             xor      ax,ax ; 0
44881             ;call     ss:ShSU
44882 0000799B 36FF1E[C800]    call     far [ss:JShare+(14*4)] ; 14 = ShSU
44883
44884             ;;and     word [es:di+5],0FFBFh
44885             ;and     word [es:di+SF_ENTRY.sf_flags],~devid_file_clean
44886 000079A0 26806505BF      and     byte [es:di+SF_ENTRY.sf_flags],~devid_file_clean ; 0BFh
44887
44888             ;;or      word [es:di+5],4000h
44889             ;or      word [es:di+SF_ENTRY.sf_flags],sf_close_nodate
44890 000079A5 26804D0640      or      byte [es:di+SF_ENTRY.sf_flags+1],(sf_close_nodate>>8) ; 40h
44891
44892             call     LCritSFT
44893 ft_ok:
44894             ok_ret:      ; 12/03/2024
44895 000079AD E9BE8C          jmp      SYS_RET_OK
44896
44897 ft2:
44898             ;test     byte [es:di+5],80h
44899 000079B0 26F6450580      test     byte [es:di+SF_ENTRY.sf_flags],devid_device
44900 000079B5 7505          jnz      short ft3 ; device
44901
44902             call     IsSFTNet
44903 ;ft3:
44904             jz        short ft5 ; local file
44905             ; 12/03/2024
44906 000079BC A801          test     al,1 ; 0 = GET, 1 = SET
44907 000079BE 750F          jnz      short ft_okj ; SET
44908
44909             ;mov     dx,[es:di+0Fh]
44910 000079C0 268B550F      mov     dx,[es:di+SF_ENTRY.sf_date]
44911             ;;;;
44912 ft7:
44913             ; 12/03/2024
44914 000079C4 29C9          sub      cx,cx ; 0 ; Retro DOS v5.0
44915             ; (Windows ME IO.SYS BIOSCODE:A348h)
44916             ;;;;
44917 ft4:
44918             call     Get_User_Stack
44919             ;mov     [si+4],cx
44920 000079C9 894C04      mov     [si+user_env.user_CX],cx ; time
44921             ;mov     [si+6],dx
44922 000079CC 895406      mov     [si+user_env.user_DX],dx ; date
44923 ft_okj:
44924 000079CF EBDC          jmp      short ft_ok
44925
44926 ft5:
44927 000079D1 16          push     ss ; Retrieve the directory entry for the file
44928 000079D2 1F          pop      ds
44929 000079D3 50          push     ax
44930 000079D4 52          push     dx
44931 000079D5 51          push     cx ; ES:DI point to SFT
44932 000079D6 E8BCBE      call     DirFromSFT ; locate a directory entry given an SFT
44933 000079D9 59          pop      cx
44934 000079DA 5A          pop      dx
44935 000079DB 730B          jnc      short ft6 ; ES:DI point to entry
44936             ; (DS:SI point to SFT)
44937             ; (ES:BX point to buffer header)
44938 000079DD 59          pop      cx
44939 ;ft_error:
44940             ;jmp      short LSeekError2
44941             ; 12/03/2024
44942             ;jmp      short LseekError
44943 000079DE EB05          jmp      short ft_error
44944
44945             ;;;;
44946             ; 12/03/2024 - Retro DOS v5.0
44947 inval_func:
44948 000079E0 E8FA98      call     set_exerr_locus_unk ; Extended Error Locus
44949
44950             ;mov     al,1
44951 000079E3 B001      mov     al,error_invalid_function ; give bad return
44952 ft_error:
44953             ;jmp      short LSeekError
44954             ; 12/03/2024
44955 000079E5 E9908C      jmp      SYS_RET_ERR
44956             ;;;;
44957 ft6:
44958 000079E8 58          pop      ax
44959 000079E9 A801      test     al,1 ; SET ?
44960 000079EB 7512          jnz      short ft8 ; yes
44961
44962             ; 12/03/2024 (ft7:, cx=0)
44963             ;xor      cx,cx ; 0 ; (always) last access time = 0
44964
44965             ;mov     dx,[es:di+12h]
44966 000079ED 268B5512      mov     dx,[es:di+dir_entry.dir_lstaccddate]
44967
44968             cmp      al,6 ; GET CREATION DATE AND TIME ?
44969 000079F3 75CF          jne      short ft7 ; no,GET LAST ACCESS DATE AND TIME
44970
44971             ;mov     cx,[es:di+0Eh]
44972 000079F5 268B4D0E      mov     cx,[es:di+dir_entry.dir_crttime]
44973             ;mov     dx,[es:di+10h]
44974 000079F9 268B5510      mov     dx,[es:di+dir_entry.dir_crtdate]
44975 ;ft7:
44976 000079FD EBC7          jmp      short ft4
44977
44978 ft8:
44979             ; check date (set) values
44980 000079FF F6C21F      test     dl,1Fh ; DAY (1 to 31) > 0 ?
44981             jnz      short ft9 ; yes,valid
44982
44983 ft_err_invd:
44984             ;mov     al,0Dh
44985             mov     al,error_invalid_data
44986 00007A06 EBDD          jmp      short ft_error
44987
44988 ft9:
44989             test     dx,1E0h ; MONTH (1 to 12) > 0 ?
44990 00007A0C 74F6          jz       short ft_err_invd ; no,invalid
44991 00007A0E 52          push     dx
44992 00007A0F 81E2E001      and     dx,1E0h ; isolate MONTH
44993 00007A13 81FA8001      cmp     dx,180h ; > 12 ?
44994 00007A17 5A          pop      dx
44995 00007A18 77EA          ja       short ft_err_invd ; yes,invalid
44996 00007A1A 3C05          cmp     al,5 ; SET LAST ACCESS DATE AND TIME ?
44997 00007A1C 7506          jnz      short ft10 ; no,SET CREATION DATE AND TIME
44998

```

```

44999          ;mov     [es:di+12h],dx
45000 00007A1E 26895512      mov     [es:di+dir_entry.dir_1staccdte],dx
45001 00007A22 EB20          jmp     short ft11
45002
45003          ; check time (set) values
45004 00007A24 89C8          mov     ax,cx
45005 00007A26 251FF8          and     ax,0F81Fh          ; isolate seconds/2 and hour
45006 00007A29 80FCB8          cmp     ah,0B8h          ; HOUR > 23 ?
45007 00007A2C 77D6          ja      short ft_err_invd ; yes,invalid
45008 00007A2E 3C1D          cmp     al,1Dh          ; SECONDS/2 > 29 (count of 2 seconds)
45009 00007A30 77D2          ja      short ft_err_invd ; yes,invalid
45010 00007A32 89C8          mov     ax,cx
45011 00007A34 25E007          and     ax,7E0h          ; isolate MINUTE
45012 00007A37 3D6007          cmp     ax,760h          ; MINUTE > 59 ?
45013 00007A3A 77C8          ja      short ft_err_invd ; yes,invalid
45014
45015          ;mov     [es:di+10h],dx
45016 00007A3C 26895510      mov     [es:di+dir_entry.dir_crtdate],dx
45017          ;mov     [es:di+0Eh],cx
45018 00007A40 26894D0E      mov     [es:di+dir_entry.dir_crttime],cx
45019
45020          ;test    byte [es:bx+5],40h
45021 00007A44 26F6470540      test    byte [es:bx+BUFFINFO.buf_flags],buf_dirty
45022 00007A49 7508          jnz     short ft12
45023
45024 00007A4B E877F1          call    INC_DIRTY_COUNT
45025
45026          ;or      byte [es:bx+5],40h
45027 00007A4E 26804F0540      or      byte [es:bx+BUFFINFO.buf_flags],buf_dirty
45028
45029 00007A53 06          ft12:   push    es
45030 00007A54 1F          pop     ds
45031 00007A55 89DF          mov     di,bx          ; DS:DI - pointer to buffer
45032 00007A57 B0FF          mov     al,0FFh        ; Drive number
45033                                     ; -1 means do not check for drive
45034 00007A59 E886F0          call    CHECKFLUSH
45035 00007A5C 7287          jc      short ft_error
45036          ;ok_ret:   ; 12/03/2024
45037 00007A5E E90D8C          jmp     SYS_RET_OK
45038          ;;;
45039          %endif
45040
45041          ;Break    <CheckOwner - verify ownership of handles from server>
45042          ;-----
45043          ; CheckOwner - Due to the ability of the server to close file handles for a
45044          ; process without the process knowing it (delete/rename of open files, for
45045          ; example), it is possible for the redirector to issue a call to a handle
45046          ; that it does not rightfully own. We check here to make sure that the
45047          ; issuing process is the owner of the SFT. At the same time, we do a
45048          ; SFFromHandle to really make sure that the SFT is good.
45049          ;
45050          ; ENTRY   BX has the handle
45051          ;          User_ID is the current user
45052          ; EXIT    Carry Clear => ES:DI points to SFT
45053          ;          Carry Set => AX has error code
45054          ; USES    none
45055          ;-----
45056
45057          ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45058          ; 21/05/2019 - Retro DOS v4.0
45059          CheckOwner:
45060          ; 13/07/2018 - Retro DOS v3.0
45061
45062 00007A61 E86AFC          call    SFFromHandle
45063 00007A64 721B          jc      short co_ret_label ; retc
45064
45065 00007A66 50          push    ax
45066
45067          ; MSDOS 6.0
45068
45069          ;SR; WIN386 patch - Do not check for USER_ID for using handles since these
45070          ;SR; are shared across multiple VMs in win386.
45071
45072 00007A67 36F606[5B0F]01      test    byte [ss:Iswin386],1 ; 02/06/2019
45073 00007A6D 7404          jz      short no_win386 ;win386 is not present
45074 00007A6F 31C0          xor     ax,ax          ;set the zero flag
45075 00007A71 EB08          jmp     short _skip_win386
45076
45077          no_win386:
45078 00007A73 36A1[3E03]          mov     ax,[SS:USER_ID]          ;smr;SS override
45079          ;cmp     ax,[es:di+47]
45080 00007A77 263B452F          cmp     ax,[es:di+SF_ENTRY.sf_UID]
45081
45082          _skip_win386:
45083 00007A7B 58          pop     ax
45084
45085          ; 17/12/2022
45086 00007A7C 7403          jz      short co_ret_label
45087          ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45088          ;jnz     short CheckOwner_err
45089          ;retn
45090
45091          CheckOwner_err:
45092          ;mov     al,6
45093 00007A7E B006          mov     al,error_invalid_handle
45094 00007A80 F9          stc
45095
45096          co_ret_label:
45097 00007A81 C3          retn
45098
45099          ;=====
45100          ; MACRO.ASM, MSDOS 6.0, 1991
45101          ;=====
45102          ; Retro   DOS v3.0 - 11/07/2018
45103          ; 21/05/2019 - Retro DOS v4.0
45104          ; 13/03/2024 - Retro DOS v5.0
45105
45106          ; TITLE   MACRO - Pathname and macro related internal routines
45107          ; NAME     MACRO
45108
45109          ; Microsoft Confidential
45110          ; Copyright (C) Microsoft Corporation 1991
45111          ; All Rights Reserved.
45112
45113          ;**  MACRO.ASM
45114          ;
45115          ; $AssignOper
45116          ; FIND_DPB
45117          ; InitCDS
45118          ; $UserOper
45119          ; GetVisDrv
45120          ; GetThisDrv
45121          ; GetCDSFromDrv
45122          ;

```

```

45123 ; Revision history:
45124 ;
45125 ; Created: MZ 4 April 1983
45126 ;         MZ 18 April 1983   Make TransFCB handle extended FCBs
45127 ;         AR 2 June 1983     Define/Delete macro for NET redir.
45128 ;         MZ 3 Nov 83        Fix InitCDS to reset length to 2
45129 ;         MZ 4 Nov 83        Fix NetAssign to use STRLEN only
45130 ;         MZ 18 Nov 83       Rewrite string processing for subtree
45131 ;                             aliasing.
45132 ;
45133 ; MSDOS performs several types of name translation. First, we maintain for
45134 ; each valid drive letter the text of the current directory on that drive.
45135 ; For invalid drive letters, there is no current directory so we pretend to
45136 ; be at the root. A current directory is either the raw local directory
45137 ; (consisting of drive:\path) or a local network directory (consisting of
45138 ; \\machine\path. There is a limit on the point to which a .. is allowed.
45139 ;
45140 ; Given a path, MSDOS will transform this into a real from-the-root path
45141 ; without . or .. entries. Any component that is > 8.3 is truncated to
45142 ; this and all * are expanded into ?'s.
45143 ;
45144 ; The second part of name translation involves subtree aliasing. A list of
45145 ; subtree pairs is maintained by the external utility SUBST. The results of
45146 ; the previous 'canonicalization' are then examined to see if any of the
45147 ; subtree pairs is a prefix of the user path. If so, then this prefix is
45148 ; replaced with the other subtree in the pair.
45149 ;
45150 ; A third part involves mapping this "real" path into a "physical" path. A
45151 ; list of drive/subtree pairs are maintained by the external utility JOIN.
45152 ; The output of the previous translation is examined to see if any of the
45153 ; subtrees in this list are a prefix of the string. If so, then the prefix
45154 ; is replaced by the appropriate drive letter. In this manner, we can
45155 ; 'mount' one device under another.
45156 ;
45157 ; The final form of name translation involves the mapping of a user's
45158 ; logical drive number into the internal physical drive. This is
45159 ; accomplished by converting the drive number into letter:CON, performing
45160 ; the above translation and then converting the character back into a drive
45161 ; number.
45162 ;
45163 ; There are two main entry points: TransPath and TransFCB. TransPath will
45164 ; take a path and form the real text of the pathname with all . and ..
45165 ; removed. TransFCB will translate an FCB into a path and then invoke
45166 ; TransPath.
45167 ;
45168 ; A000   version 4.00   Jan. 1988
45169 ;
45170 ;Installed = TRUE
45171 ;
45172 ;   I_need ThisCDS,DWORD      ; pointer to CDS used
45173 ;   I_need CDSAddr,DWORD      ; pointer to CDS table
45174 ;   I_need CDSCount,BYTE       ; number of CDS entries
45175 ;   I_need CurDrv,BYTE         ; current macro assignment (old
45176 ;                               ; current drive)
45177 ;   I_need NUMIO,BYTE          ; Number of physical drives
45178 ;   I_need fSharing,BYTE       ; TRUE => no redirection allowed
45179 ;   I_need DummyCDS,80h        ; buffer for dummy cds
45180 ;   I_need DIFFNAM,BYTE        ; flag for MyName being set
45181 ;   I_need MYNAME,16           ; machine name
45182 ;   I_need MYNUM,WORD           ; machine number
45183 ;   I_need DPBHEAD,DWORD        ; beginning of DPB chain
45184 ;   I_need EXTERR_LOCUS,BYTE    ; Extended Error Locus
45185 ;   I_need DrvErr,BYTE         ; drive error
45186 ;
45187 ;BREAK <$AssignOper -- Set up a Macro>
45188 ;-----
45189 ; Inputs:
45190 ; AL = 00 get assign mode          (ReturnMode)
45191 ; AL = 01 set assign mode          (SetMode)
45192 ; AL = 02 get attach list entry    (GetAsgList)
45193 ; AL = 03 Define Macro (attch start)
45194 ;     BL = Macro type
45195 ;     = 0 alias
45196 ;     = 1 file/device
45197 ;     = 2 drive
45198 ;     = 3 char device -> network
45199 ;     = 4 File device -> network
45200 ;     DS:SI -> ASCIZ source name
45201 ;     ES:DI -> ASCIZ destination name
45202 ; AL = 04 Cancel Macro
45203 ;     DS:SI -> ASCIZ source name
45204 ; AL = 05 Modified get attach list entry
45205 ; AL = 06 Get ifsfunc item
45206 ; AL = 07 set in_use of a drive's CDS
45207 ;     DL = drive number, 0=default 0=A,,
45208 ; AL = 08 reset in_use of a drive's CDS
45209 ;     DL = drive number, 0=A, 1=B,,
45210 ; Function:
45211 ; Do macro stuff
45212 ; Returns:
45213 ; Std Xenix style error return
45214 ;-----
45215 ;
45216 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45217 ; 21/05/2019 - Retro DOS v4.0
45218 ;
45219 ;_AssignOper:
45220 ; MSDOS 6.0
45221 00007A82 3C07      CMP     AL,7          ; set in_use ?          ;AN000;
45222 00007A84 7525      JNZ     short chk08      ; no                    ;AN000;
45223 ; srinuse:
45224 00007A86 50        PUSH     AX          ; save al              ;AN000;
45225 00007A87 88D0      MOV     AL,DL          ; AL= drive id         ;AN000;
45226 00007A89 E84B01    CALL    GetCDSFromDrv      ; ds:si -> cds         ;AN000;
45227 00007A8C 58        POP     AX          ;                      ;AN000;
45228 00007A8D 7216      JC      short baddrv      ; bad drive            ;AN000;
45229 ; cmp word [si+45h],0
45230 00007A8F 837C4500   CMP     WORD [SI+curdir.devptr],0 ; dpb ptr =0 ?        ;AN000;
45231 00007A93 7410      JZ      short baddrv      ; no                    ;AN000;
45232 00007A95 3C07      CMP     AL,7          ; set ?                ;AN000;
45233 00007A97 7506      JNZ     short resetdrv      ; no                    ;AN000;
45234 ; or word [si+43h],4000h
45235 ; 17/12/2022
45236 ; or byte [si+44h],40h
45237 00007A99 804C4440   OR     byte [SI+curdir.flags+1],(curdir_inuse>>8)
45238 ; OR word [SI+curdir.flags],curdir_inuse ; set in_use      ;AN000;
45239 00007A9D EB19      JMP     SHORT okdone        ;                      ;AN000;
45240 ; resetdrv:
45241 ; and word [si+43h],0BFFFh
45242 ; 18/12/2022
45243 00007A9F 806444BF   AND     byte [SI+curdir.flags+1],0BFh ; (~curdir_inuse)>>8
45244 ; AND word [SI+curdir.flags],~curdir_inuse ; reset in_use ;AN000;
45245 00007AA3 EB13      JMP     SHORT okdone        ;                      ;AN000;
45246 ;

```

```

45247 ; 17/12/2022
45248 baddrv:
45249 00007AA5 B80F00 MOV AX,error_invalid_drive ; error ;AN000;
45250 ;AN000;
45251 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45252 ;JMP SHORT ASS_ERR ; ;AN000;
45253 ; 17/12/2022
45254 ; 21/05/2019
45255 ASS_ERR:
45256 00007AA8 E9CD8B jmp SYS_RET_ERR
45257
45258 chk08:
45259 00007AAB 3C08 CMP AL,8 ; reset inuse ? ;AN000;
45260 00007AAD 74D7 JZ short srinuse ; yes ;AN000;
45261
45262 ;IF NOT INSTALLED
45263 ;transfer NET_ASSOPER
45264 ;ELSE
45265 ; MSDOS 3.3 (& MSDOS 6.0)
45266 00007AAF 50 PUSH AX
45267 ;mov ax,111eh
45268 ;MOV AX,(MultNET SHL 8) OR 30
45269 00007AB0 B81E11 mov ax,(MultNET*256)+30
45270 00007AB3 CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - DO REDIRECTION
45271 ; SS = DOS CS
45272 ; STACK: WORD function to execute
45273 ; Return: CF set on error, AX = error code
45274 ; STACK unchanged
45275 00007AB5 5B POP BX ; Don't zap error code in AX
45276 00007AB6 72F0 JC short ASS_ERR
45277 okdone:
45278 00007AB8 E9B38B jmp SYS_RET_OK
45279
45280 ; 17/12/2022
45281 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45282 ;ASS_ERR:
45283 ;jmp SYS_RET_ERR
45284
45285 ;ENDIF
45286
45287 ;Break <FIND_DPB - Find a DPB from a drive number>
45288 -----
45289 ** FIND_DPB - Find a DPB from a Drive #
45290 ;
45291 ; ENTRY AL has drive number A = 0
45292 ; EXIT 'C' set
45293 ; No DPB for this drive number
45294 ; 'C' clear
45295 ; DS:SI points to DPB for drive
45296 ; USES SI, DS, Flags
45297 -----
45298
45299 ; 21/05/2019 - Retro DOS v4.0
45300 FIND_DPB:
45301 00007ABB 36C536[2600] LDS SI,[SS:DPBHEAD] ;smr;SS override
45302 fdpb5:
45303 CMP SI,-1
45304 00007AC3 7409 JZ short fdpb10
45305 00007AC5 3A04 cmp al,[si]
45306 ;CMP AL,[SI+DPB.DRIVE]
45307 00007AC7 7406 jz short ret_label15 ; Carry clear (retz)
45308 ;lds si,[si+18h] ; MSDOS 3.3
45309 ;lds si,[si+19h] ; MSDOS 6.0
45310 00007AC9 C57419 LDS SI,[SI+DPB.NEXT_DPB]
45311 00007ACC EBF2 JMP short fdpb5
45312 fdpb10:
45313 00007ACE F9 STC
45314 ret_label15:
45315 00007ACF C3 retn
45316
45317 ; Break <InitCDS - set up an empty CDS>
45318 -----
45319 ** InitCDS - Setup an Empty CDS
45320 ;
45321 ; ENTRY ThisCDS points to CDS
45322 ; AL has uppercase drive letter
45323 ; EXIT ThisCDS is now empty
45324 ; (ES:DI) = CDS
45325 ; 'C' set if no DPB associated with drive
45326 ; USES AH,ES,DI, Flags
45327 -----
45328
45329 ; 21/05/2019 - Retro DOS v4.0
45330 ; DOSCODE:A9FDh (MSDOS 6.21, MSDOS.SYS)
45331
45332 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45333 ; DOSCODE:A99Dh (MSDOS 5.0, MSDOS.SYS)
45334
45335 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
45336 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BC91h
45337
45338 InitCDS:
45339 ; 19/08/2018
45340 ; 05/08/2018 - Retro DOS v3.0
45341 ; MSDOS 6.0
45342 00007AD0 50 push ax ; save (AL) for caller
45343 00007AD1 36C43E[A205] LES DI,[SS:THISCDS] ; (es:di) = CDS address
45344 ;mov word [es:di+67],0
45345 00007AD6 26C745430000 MOV word [ES:DI+curdir.flags],0 ; "free" CDS
45346 00007ADC 2C40 SUB AL,"A"-1 ; A = 1
45347 00007ADE 363806[4600] CMP [SS:NUMIO],AL ;smr;SS override
45348 00007AE3 7236 JC short icdsx ; Drive does not map a physical drive
45349 00007AE5 48 dec ax ; (AL) = 0 if A, 1 if B, etc.
45350 00007AE6 50 PUSH AX ; save drive number for later
45351 00007AE7 0441 add al,"A"
45352 00007AE9 B43A MOV AH,':'
45353 00007AEB 268905 mov [ES:DI],ax
45354 ;MOV [ES:DI+curdir.text],AX ; set "x:"
45355 ;mov ax,""
45356 ;mov [es:di+2],ax
45357 ;MOV word [ES:DI+curdir.text+2],"" ; NUL terminate
45358 00007AEE 26C745025C00 mov word [ES:DI+curdir.text+2],005ch ; 19/08/2018
45359 ;or word [es:di+67],4000h
45360 ;or byte [es:di+68],40h
45361 00007AF4 26804D4440 OR byte [ES:DI+curdir.flags+1],(curdir_inuse>>8)
45362 00007AF9 29C0 sub ax,ax
45363 ;MOV [es:di+73],ax ; 0
45364 00007AFB 26894549 MOV [ES:DI+curdir.ID],ax
45365 ;mov [es:di+75],ax ; 0
45366 00007AFF 2689454B MOV [ES:DI+curdir.ID+2],ax
45367 00007B03 B002 mov al,2
45368 ;mov [es:di+79],ax ; 2
45369 00007B05 2689454F MOV [ES:DI+curdir.end],ax
45370 00007B09 58 POP AX ; (al) = drive number

```



```

45371 00007B0A 1E      push    ds
45372 00007B0B 56      push    si
45373 00007B0C E8ACFF  call    FIND_DPB
45374 00007B0F 7208      JC      short icds5          ; 0000PPPPSSSS!!!!
45375      ;mov    [es:di+69],si
45376 00007B11 26897545  MOV     [ES:DI+curdir.devptr],SI
45377      ;mov    [es:di+71],ds
45378 00007B15 268C5D47  MOV     [ES:DI+curdir.devptr+2],DS
45379      icds5:
45380 00007B19 5E      pop     si
45381 00007B1A 1F      pop     ds
45382      icdsx:
45383 00007B1B 58      pop     ax
45384      RET45:
45385 00007B1C C3      retn
45386
45387      ;Break <$UserOper - get/set current user ID (for net)>
45388      -----
45389      $UserOper - retrieve or initiate a user id string. MSDOS will only
45390      maintain this string and do no verifications.
45391      ;
45392      ; Inputs: AL has function type (0-get 1-set 2-printer-set 3-printer-get
45393      ;                  4-printer-set-flags,5-printer-get-flags)
45394      ;                  DS:DX is user string pointer (calls 1,2)
45395      ;                  ES:DI is user buffer (call 3)
45396      ;                  BX is assign index (calls 2,3,4,5)
45397      ;                  CX is user number (call 1)
45398      ;                  DX is flag word (call 4)
45399      ; Outputs:      If AL = 0 then the current user string is written to DS:DX
45400      ;                  and user CX is set to the user number
45401      ;                  If AL = 3 then CX bytes have been put at input ES:DI
45402      ;                  If AL = 5 then DX is flag word
45403      ;
45404      -----
45405      ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
45406      ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45407      ; 21/05/2019 - Retro DOS v4.0
45408      _$UserOper:
45409      ; 05/08/2018 - Retro DOS v3.0
45410      ; MSDOS 6.0 (& MSDOS 3.3)
45411      ; PUSH    AX
45412      ; SUB     AL,1          ; quick dispatch on 0,1
45413      ; POP     AX
45414      ; 01/07/2024
45415 00007B1D 3C01      cmp     al,1
45416 00007B1F 720E      JB      short UserGet      ; return to user the string
45417 00007B21 742B      JZ      short UserSet      ; set the current user
45418 00007B23 3C05      CMP     AL,5              ; test for 2,3,4 or 5
45419 00007B25 763A      JBE     short UserPrint    ; yep
45420
45421      ; 13/03/2024
45422      %if 0
45423      ;mov     byte [ss:EXTERR_LOCUS],1
45424      MOV     byte [SS:EXTERR_LOCUS],errLOC_Unk ;smr;SS Override
45425      ; Extended Error Locus
45426      %else
45427      ; 13/03/2024 (PCDOS 7.1 IBMDOS.COM)
45428      ;;;
45429 00007B27 E8B397  call    set_exerr_locus_unk ; Extended Error Locus
45430      ;;;
45431      %endif
45432      ;error error_invalid_function; not 0,1,2,3
45433      ;mov     al,1
45434 00007B2A B001      MOV     AL,error_invalid_function
45435      useroper_error:
45436      ; 17/12/2022
45437      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45438 00007B2C E9498B  JMP     SYS_RET_ERR
45439      ;jmp     short ASS_ERR
45440
45441      UserGet:
45442      ; Transfer MYNAME to DS:DX
45443      ; Set Return CX to MYNUM
45444 00007B2F 1E      PUSH    DS          ; switch registers
45445 00007B30 07      POP     ES
45446 00007B31 89D7      MOV     DI,DX        ; destination
45447 00007B33 368B0E[0E00] MOV     CX,[SS:MYNUM] ; Get number ;smr;SS Override
45448 00007B38 E83C89  call    Get_User_Stack
45449      ;mov     [si+4],cx
45450 00007B3B 894C04  MOV     [SI+user_env.user_CX],CX ; Set number return
45451 00007B3E 16      push    ss          ; point to DOSDATA
45452 00007B3F 1F      pop     ds
45453 00007B40 BE[0503] MOV     SI,MYNAME     ; point source to user string
45454      UserMove:
45455 00007B43 B90F00  MOV     CX,15
45456 00007B46 F3A4      REP     MOVSB        ; blam.
45457 00007B48 31C0      XOR     AX,AX        ; 16th byte is 0
45458 00007B4A AA      STOSB
45459      UserBye:
45460 00007B4B E9208B  jmp     SYS_RET_OK    ; no errors here
45461
45462      UserSet:
45463      ; Transfer DS:DX to MYNAME
45464      ; CX to MYNUM
45465 00007B4E 36890E[0E00] MOV     [SS:MYNUM],CX ;smr;SS Override
45466 00007B53 89D6      MOV     SI,DX        ; user space has source
45467 00007B55 16      push    ss
45468 00007B56 07      pop     es
45469 00007B57 BF[0503] MOV     DI,MYNAME     ; point dest to user string
45470 00007B5A 36FE06[0403] INC     byte [SS:DIFFNAM] ; signal change ;smr;SS Override
45471 00007B5F EBE2      JMP     short UserMove
45472
45473      UserPrint:
45474
45475      ;IF NOT Installed
45476      ; transfer PRINTER_GETSET_STRING
45477      ;ELSE
45478 00007B61 50      PUSH    AX
45479      ;mov     ax,111Fh
45480      ;MOV     AX,(MultNET SHL 8) OR 31
45481 00007B62 B81F11  mov     ax,(MultNET<<8)|31
45482 00007B65 CD2F      int     2Fh          ; Multiplex - NETWORK REDIRECTOR - PRINTER SETUP
45483      ; STACK: WORD function
45484      ; Return: CF set on error, AX = error code
45485      ; STACK unchanged
45486 00007B67 5A      POP     DX          ; Clean stack
45487      ;JNC     short OKPA
45488 00007B68 73E1      jnc     short UserBye ; 21/05/2019
45489      ; 17/12/2022
45490 00007B6A EBC0      jmp     short useroper_error
45491      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45492      ;jnb     short OKPA
45493      ;jmp     short useroper_error
45494

```

```

45495 ; 17/12/2022
45496 ;OKPA:
45497 ; jmp short UserBye
45498
45499 ;ENDIF
45500
45501 ;Break <GetVisDrv - return visible drive>
45502 -----
45503 ; GetVisDrv - correctly map non-spliced inuse drives
45504 ;
45505 ; Inputs: AL has drive identifier (0=default)
45506 ; Outputs: Carry Set - invalid drive/macro
45507 ; Carry Clear - AL has physical drive (0=A)
45508 ; ThisCDS points to CDS
45509 ; Registers modified: AL
45510 -----
45511
45512 ; 21/05/2019 - Retro DOS v4.0
45513 ; DOSCODE:AA9Fh (MSDOS 6.21, MSDOS.SYS)
45514
45515 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45516 ; DOSCODE:AA3Fh (MSDOS 5.0, MSDOS.SYS)
45517
45518 GetVisDrv:
45519 ; 05/08/2018 - Retro DOS v3.0
45520 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 6839h
45521 CALL GETTHISDRV ; get inuse drive
45522 jc short RET45
45523 push ds
45524 push si
45525 LDS SI,[SS:THISCDS] ;smr;SS Override
45526 ;test word [si+67],2000h
45527 ; 17/12/2022
45528 ;test byte [si+68],20h
45529 test byte [SI+curdir.flags+1],(curdir_splice>>8)
45530 ;TEST word [SI+curdir.flags],curdir_splice
45531 pop si
45532 pop ds
45533 jz short RET45 ; if not spliced, return OK
45534 ; MSDOS 6.0
45535 ;mov byte [ss:DrvErr],0Fh
45536 MOV byte [SS:DrvErr],error_invalid_drive ;IFS. ;AN000;smr;SS Override
45537 STC ; signal error
45538 retn
45539
45540 ;Break <Getthisdrv - map a drive designator (0=def, 1=A...)>
45541 -----
45542 ; GetThisDrv - look through a set of macros and return the current drive and
45543 ; macro pointer
45544 ;
45545 ; Inputs: AL has drive identifier (1=A, 0=default)
45546 ; Outputs:
45547 ; Carry Set - invalid drive/macro
45548 ; Carry Clear - AL has physical drive (0=A)
45549 ; ThisCDS points to macro
45550 ; Registers modified: AL
45551 -----
45552
45553 ; 21/05/2019 - Retro DOS v4.0
45554 ; DOSCODE:AABCh (MSDOS 6.21, MSDOS.SYS)
45555
45556 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45557 ; DOSCODE:AA5Ch (MSDOS 5.0, MSDOS.SYS)
45558
45559 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
45560 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BD48h
45561
45562 GETTHISDRV:
45563 ; 05/08/2018
45564 ; 12/07/2018 - Retro DOS v3.0
45565 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 6850h
45566 ; MSDOS 3.3 (& MSDOS 6.0)
45567 OR AL,AL ; are we using default drive?
45568 JNZ SHORT GTD10 ; no, go get the CDS pointers
45569 MOV AL,[SS:CURDRV] ; get the current drive
45570 ;INC ax ; Counteract next instruction
45571 ; 04/09/2018
45572 ;inc al
45573 ; 07/12/2022
45574 inc ax
45575
45576 GTD10:
45577 ;DEC AX
45578 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45579 dec ax ; 0 = A
45580 ;dec al
45581 PUSH DS ; save world
45582 PUSH SI
45583
45584 ; 13/03/2024
45585 %if 0
45586 ;mov byte [ss:EXTERR_LOCUS],2
45587 MOV BYTE [SS:EXTERR_LOCUS],errLOC_Disk ;smr;SS Override
45588 TEST BYTE [SS:FSHARING],-1 ; Logical or Physical?;smr;SS Override
45589 JZ SHORT GTD20 ; Logical
45590
45591 %else
45592 ; 13/03/2024 (PCDOS 7.1 IBMDOS.COM)
45593 ;;;
45594 call set_exerr_locus_disk
45595 cmp byte [ss:FSHARING],0 ; Logical or Physical?
45596 jz short GTD20 ; Logical
45597 ;;;
45598 %endif
45599 PUSH AX
46000 PUSH ES
46001 PUSH DI
46002 MOV WORD [SS:THISCDS],DUMMYCDS ;smr;SS Override
46003 ;mov [SS:THISCDS+2],CS ; MSDOS 3.3
46004 MOV [SS:THISCDS+2],SS ; MSDOS 6.0 ;ThisCDS = &DummyCDS;smr;
46005 ADD AL,'A'
46006 CALL InitCDS ; InitCDS(c);
46007 ;test word [es:di+67],4000h
46008 ; 17/12/2022
46009 ;test byte [es:di+68],40h
46010 test byte [ES:DI+curdir.flags+1],(curdir_inuse>>8)
46011 ;TEST WORD [ES:DI+curdir.flags],curdir_inuse ; clears carry
46012 POP DI
46013 POP ES
46014 POP AX
46015 JZ SHORT GTD30 ; Not a physical drive.
46016 JMP SHORT GTDX ; carry clear
46017
46018 GTD20:
46019 CALL GetCDSFromDrv
46020 JC SHORT GTD30 ; Unassigned CDS -> return error already set
46021 ;test word [si+43h],4000h

```



```

45619             ; 17/12/2022
45620             ; test byte [si+44h],40h
45621 00007BC4 F6444440 test byte [SI+curdir.flags+1],(curdir_inuse>>8)
45622             ; TEST WORD [SI+curdir.flags],curdir_inuse ; Clears Carry
45623 00007BC8 750A JNZ SHORT GTDX ; carry clear
45624 GTD30:
45625             ; 21/05/2019
45626             ; MSDOS 6.0
45627 00007BCA B00F MOV AL,error_invalid_drive; invalid FAT drive
45628 00007BCC 36A2[1006] MOV BYTE [ss:DrvErr],AL ; save this for IOCTL
45629
45630             ; 13/03/2024
45631 %if 0
45632             ; MSDOS 3.3 (& MSDOS 6.0)
45633 MOV BYTE [ss:EXTERR_LOCUS],errLOC_Unk
45634 %else
45635             ; 13/03/2024 (PCDOS 7.1 IBMDOS.COM)
45636             ;;;
45637 00007BD0 E80A97 call set_exerr_locus_unk
45638             ;;;
45639 %endif
45640 00007BD3 F9 STC
45641 GTDX:
45642 00007BD4 5E POP SI ; restore world
45643 00007BD5 1F POP DS
45644 00007BD6 C3 RETN
45645
45646 ;Break <GetCDSFromDrv - convert a drive number to a CDS pointer>
45647 -----
45648 ; GetCDSFromDrv - given a physical drive number, convert it to a CDS
45649 ; pointer, returning an error if the drive number is greater than the
45650 ; number of CDS's
45651 ;
45652 ; Inputs: AL is physical unit # A=0...
45653 ; Outputs: Carry Set if Bad Drive
45654 ; Carry Clear
45655 ; DS:SI -> CDS
45656 ; [THISCDS] = DS:SI
45657 ; Registers modified: DS,SI
45658 ; -----
45659
45660             ; 21/05/2019 - Retro DOS v4.0
45661 GetCDSFromDrv:
45662 00007BD7 363A06[4700] CMP AL,[SS:CDSCOUNT] ; is this a valid designator;smr;SS Override
45663             ; JB SHORT GetCDS ; cf=1 ; yes, go get the macro
45664             ; STC ; signal error
45665             ; RETN ; bye
45666             ; 23/09/2023
45667 00007BDC F5 cmc ; cf=1 <-> cf=0
45668 00007BDD 7217 jc short GetCDS_retn
45669 GetCDS:
45670             ; 23/09/2023
45671             ; PUSH BX
45672 00007BDF 50 PUSH AX
45673 00007BE0 36C536[3C00] LDS SI,[SS:CDSADDR] ; get pointer to table;smr;SS Override
45674             ; mov bl,81 ; MSDOS 3.3
45675             ; mov bl,88 ; MSDOS 6.0
45676             ; 23/09/2023
45677             ; MOV BL,curdir.size ; size in convenient spot
45678             ; MUL BL ; get net offset
45679 00007BE5 B458 mov ah,curdir.size
45680 00007BE7 F6E4 mul ah
45681 00007BE9 01C6 ADD SI,AX ; * ; convert to true pointer
45682 00007BEB 368936[A205] MOV [SS:THISCDS],SI ; store convenient offset;smr;SS Override
45683 00007BF0 368C1E[A405] MOV [SS:THISCDS+2],DS ; store convenient segment;smr;SS Override
45684 00007BF5 58 POP AX
45685             ; 23/09/2023
45686             ; POP BX
45687             ; (cf must be 0 here) ; *
45688             ; CLC ; no error
45689 GetCDS_retn:
45690 00007BF6 C3 RETN ; bye!
45691
45692 ;=====
45693 ; MACRO2.ASM, MSDOS 6.0, 1991
45694 ;=====
45695 ; Retro DOS v3.0 - 12/07/2018
45696 ; 22/05/2019 - Retro DOS v4.0
45697 ; 13/03/2024 - Retro DOS v5.0
45698
45699 ;BREAK <TransFCB - convert an FCB into a path, doing substitution>
45700 -----
45701 ; TransFCB - Copy an FCB from DS:DX into a reserved area doing all of the
45702 ; gritty substitution.
45703 ;
45704 ; Inputs: DS:DX - pointer to FCB
45705 ; ES:DI - point to destination
45706 ; Outputs: Carry Set - invalid path in final map
45707 ; Carry Clear - FCB has been mapped into ES:DI
45708 ; Sattrib is set from possibly extended FCB
45709 ; ExtFCB set if extended FCB found
45710 ; Registers modified: most
45711 ; -----
45712
45713             ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45714 TransFCB:
45715             ; 22/05/2019 - Retro DOS v4.0
45716             ; 12/07/2018 - Retro DOS v3.0
45717             ; LocalVar FCBtmp,16
45718             ; ENTER
45719 00007BF7 55 push bp
45720 00007BF8 89E5 mov bp,sp
45721             ; sub sp,15 ; MSDOS 3.3
45722 00007BFA 83EC10 sub sp,16 ; MSDOS 6.0
45723 00007BFD 16 push ss
45724 00007BFE 07 pop es
45725 00007BFF 06 push es
45726 00007C00 57 push di
45727             ; lea di,[bp-15] ; MSDOS 3.3
45728             ; LEA DI,FCBtmp
45729 00007C01 8D7EF0 lea di,[bp-16] ; point to FCB temp area
45730 00007C04 36C606[6C05]00 mov byte [SS:EXTFCB],0 ; no extended FCB found;smr;SS Override
45731 00007C0A 36C606[6D05]00 mov byte [SS:SATTRIB],0 ; default search attributes;smr;SS Override
45732 00007C10 E836A6 call GetExtended ; get FCB, extended or not
45733             ; 06/12/2022
45734 00007C13 740D jz short GetDrive ; not an extended FCB, get drive
45735 00007C15 8A44FF mov AL,[SI-1] ; get attributes
45736 00007C18 36A2[6D05] mov [SS:SATTRIB],AL ; store search attributes;smr;SS Override
45737 00007C1C 36C606[6C05]FF mov byte [SS:EXTFCB],-1 ; signal extended FCB ;smr;SS Override
45738 GetDrive:
45739 00007C22 AC lodsb ; get drive byte
45740 00007C23 E862FF call GETTHISDRV
45741 00007C26 722A jc short BadPack
45742 00007C28 E86F03 call TextFromDrive ; convert 0-based drive to text

```

```

45743
45744
45745
45746
45747 00007C2B B90B00
45748 00007C2E 56
45749
45750 00007C2F AC
45751
45752
45753
45754
45755
45756
45757 00007C30 E898E1
45758
45759
45760 00007C33 A808
45761 00007C35 741B
45762
45763 00007C37 E2F6
45764 00007C39 5E
45765 00007C3A 89FB
45766 00007C3C E867AA
45767 00007C3F 5F
45768 00007C40 07
45769 00007C41 16
45770 00007C42 1F
45771
45772
45773 00007C43 8D76F0
45774 00007C46 803F00
45775 00007C49 7407
45776 00007C4B 55
45777 00007C4C E80E00
45778 00007C4F 5D
45779 00007C50 7303
45780
45781 00007C52 F9
45782
45783 00007C53 B003
45784
45785
45786 00007C55 89EC
45787 00007C57 5D
45788
45789 00007C58 C3
45790
45791
45792
45793
45794
45795
45796
45797
45798
45799
45800
45801
45802
45803
45804
45805
45806
45807
45808
45809
45810
45811
45812
45813
45814
45815
45816
45817
45818
45819
45820
45821
45822
45823
45824
45825
45826
45827
45828
45829
45830
45831
45832
45833
45834
45835
45836
45837
45838
45839
45840
45841
45842
45843
45844
45845
45846
45847
45848
45849
45850
45851
45852
45853
45854
45855
45856
45857
45858
45859
45860
45861
45862
45863
45864
45865
45866

; Scan the source to see if there are any illegal chars
;mov     bx,CharType          ; load lookup table
;mov     cx,11
;push    si                    ; back over name, ext
FCBScan:
;lodsb                      ; get a byte

; 09/08/2018
;;xlat    byte [es:bx]
;es      xlat

; 22/05/2019 - Retro DOS v4.0
call     GetCharType          ; get flags

;test     al,8
test     al,FFCB
jz        short BadPack
NextCh:
loop     FCBScan
pop      si
mov      bx,di
call     PackName              ; crunch the path
pop      di                    ; get original destination
pop      es
push     ss                    ; get DS addressability
pop      ds
;lea      si,[bp-15] ; MSDOS 3.3
;LEA      SI,FCBtmp            ; point at new pathname
lea      si,[bp-16]
cmp      byte [bx],0
jz        short BadPack
push     bp
call     TransPathSet          ; convert the path
pop      bp
jnc      short FCBRet          ; bye with transPath error code
BadPack:
STC
;mov      al,3
MOV      AL,error_path_not_found
FCBRet:
;LEAVE
mov      sp,bp
pop      bp
TransPath_retn:
retn

; 12/07/2018 - Retro DOS v3.0

;BREAK <TransPath - copy a path, do string sub and put in current dir>
;-----
;
; TransPath - copy a path from DS:SI to ES:DI, performing component string
; substitution, insertion of current directory and fixing . and ..
; entries. Perform splicing. Allow input string to match splice
; exactly.
;
; TransPathSet - Same as above except No splicing is performed if input path
; matches splice.
;
; TransPathNoSet - No splicing/local using is performed at all.
;
; The following anomalous behaviour is required:
;
; Drive letters on devices are ignored. (set up DummyCDS)
; Paths on devices are ignored. (truncate to 0-length)
; Raw net I/O sets ThisCDS => NULL.
; fSharing => dummyCDS and no subst/splice. Only canonicalize.
;
; Other behaviour:
;
; ThisCDS set up.
; FatRead done on local CDS.
; ValidateCDS done on local CDS.
;
; Brief flowchart:
;
; if fSharing then
;   set up DummyCDS (ThisCDS)
;   canonicalize (sets cMeta)
;   splice
;   fatRead
;   return
; if \\ or d:\ lead then
;   set up null CDS (ThisCDS)
;   canonicalize (sets cMeta)
;   return
; if device then
;   set up dummyCDS (ThisCDS)
;   canonicalize (sets cMeta)
;   return
; if file then
;   getCDS (sets (ThisCDS) from name)
;   validateCDS (may reset current dir)
;   Copy current dir
;   canonicalize (set cMeta)
;   splice
;   generate correct CDS (ThisCDS)
;   if local then
;     fatread
;   return
;
; Inputs:      DS:SI - point to ASCIZ string path
;              DI - point to buffer in DOSDATA
; Outputs:     Carry Set - invalid path specification: too many .., bad
;              syntax, etc. or user FAILED to I 24.
;              WFP_Start - points to beginning of buffer
;              Curr_Dir_End - points to end of current dir in path
;              DS - DOSDATA
; Registers modified: most
;-----

; 22/05/2019
; 13/05/2019 - Retro DOS v4.0
; DOSCODE:AB99h (MSDOS 6.21, MSDOS.SYS)

; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:AB39h (MSDOS 5.0, MSDOS.SYS)

; 13/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
; PC DOS 7.1 IBMDOS.COM - DOSCODE:0BE1Dh

```

```

45867 TransPath:
45868 XOR AL,AL
45869 JMP SHORT SetSplice
45870 TransPathSet:
45871 MOV AL,-1
45872 SetSplice:
45873 MOV [SS:NoSetDir],AL ; NoSetDir = !fExact; ;smr;SS Override
45874 MOV AL,-1
45875 TransPathNoSet:
45876 MOV [SS:FSPLICE],AL ; fSplice = TRUE; ;smr;SS Override
45877 MOV byte [ss:CMETA],-1 ;smr;SS Override
45878 MOV [SS:WFP_START],DI ;smr;SS Override
45879 MOV word [SS:Curr_Dir_End],-1 ; crack from start;smr;SS Override
45880 push ss
45881 pop es
45882 ;lea bp,[di+134]
45883 LEA BP,[DI+TEMPLEN] ; end of buffer
45884 ;
45885 ; if this is through the server dos call, fsharing is set. We set up a
45886 ; dummy cds and let the operation go.
45887 ;
45888 ;TEST byte [SS:FSHARING],-1 ; if no sharing ;smr;SS Override
45889 ;JZ short CheckUNC ; skip to UNC check
45890 ; 13/03/2024 (PCDOS 7.1 IBMDOS.COM)
45891 ;;;
45892 cmp byte [ss:FSHARING],0
45893 jz short CheckUNC
45894 ;;;
45895 ;
45896 ; ES:DI point to buffer
45897 ;
45898 CALL DriveFromText ; get drive and advance DS:SI
45899 call GETTHISDRV ; Set ThisCDS and convert to 0-based
45900 jc short NoPath
45901 CALL TextFromDrive ; drop in new
45902 LEA BX,[DI+1] ; backup limit
45903 CALL Canonicalize ; copy and canonicalize
45904 jc short TransPath_retn ; errors
45905 ;
45906 ; Perform splices for net guys.
45907 ;
45908 push ss
45909 pop ds
45910 MOV SI,[WFP_START] ; point to name
45911 TEST byte [FSPLICE],-1
45912 JZ short NoServerSplice
45913 CALL Splice
45914 NoServerSplice:
45915 push ss
45916 pop ds ; for FATREAD
45917 LES DI,[THISCDS] ; for fatread
45918 call ECritDisk
45919 call FATREAD_CDS
45920 call LCritDisk
45921 NoPath:
45922 ;mov al,3
45923 MOV AL,error_path_not_found ; Set up for possible bad path error
45924 retn ; any errors are in Carry flag
45925 ;
45926 ; Let the network decide if the name is for a spooled device. It will map
45927 ; the name if so.
45928 ;
45929 CheckUNC:
45930 MOV WORD [SS:THISCDS],-1 ; NULL thisCDS ;smr;SS Override
45931 ;call Install NetSpoolCheck,MultNET,35
45932 mov ax,1123h
45933 int 2Fh ; Multiplex - NETWORK REDIRECTOR - QUALIFY REMOTE FILENAME
45934 ; DS:SI -> ASCII filename to canonicalize
45935 ; ES:DI -> 128-byte buffer for qualified name
45936 ; Return: CF set if not resolved
45937 JNC short UNCDone
45938 ;
45939 ; At this point the name is either a UNC-style name (prefixed with two leading
45940 ; \s) or is a local file/device. Remember that if a net-spoiled device was
45941 ; input, then the name has been changed to the remote spooler by the above net
45942 ; call. Also, there may be a drive in front of the \\.
45943 ;
45944 NO_CHECK:
45945 CALL DriveFromText ; eat drive letter
45946 PUSH AX ; save it
45947 MOV AX,[SI] ; get first two bytes of path
45948 call PATHCHRCMP ; convert to normal form
45949 XCHG AH,AL ; swap for second byte
45950 call PATHCHRCMP ; convert to normal form
45951 JNZ short CheckDevice ; not a path char
45952 CMP AH,AL ; are they same?
45953 JNZ short CheckDevice ; nope
45954 ;
45955 ; We have a UNC request. We must copy the string up to the beginning of the
45956 ; local machine root path
45957 ;
45958 POP AX
45959 MOVSW ; get the lead \\.
45960 UNCCpy:
45961 LODSB ; get a byte
45962 call UCASE ; AN000; convert the char
45963 OR AL,AL
45964 JZ short UNCTerm ; end of string. All done.
45965 call PATHCHRCMP ; is it a path char?
45966 MOV BX,DI ; backup position
45967 STOSB
45968 JNZ short UNCCpy ; no, go copy
45969 CALL Canonicalize ; wham (and set cMeta)
45970 UNCDone:
45971 push ss
45972 pop ds
45973 retn ; return error code
45974 UNCTerm:
45975 STOSB ; AN000;
45976 JMP short UNCDone ; AN000;
45977 ;
45978 CheckDevice:
45979 ; Check DS:SI for device. First eat any path stuff
45980 ;
45981 POP AX ; retrieve drive info
45982 CMP BYTE [SI],0 ; check for null file
45983 JNZ short CheckPath
45984 ;mov al,2
45985 MOV AL,error_file_not_found ; bad file error
45986 STC ; signal error on null input
45987 RETN ; bye!
45988 CheckPath:
45989 push ax
45990

```

```

45991 00007D06 55          push    bp          ; save drive number
45992
45993 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45994 %if 0
45995 ; MSDOS 6.0
45996 ;;;BUGBUG BUG 10-26-1992 scottq
45997 ;;;This is a hack for the CDROM extensions (2.1) who scan looking
45998 ;;;for the following POP BP == 5Dh (restore <bp,ax>).
45999 ;;;The problem is that a direct call to CheckThisDevice can (and did)
46000 ;;;end up having a 5D in the opcode's displacement field. The
46001 ;;;scanning code would choke on this thinking it was a POP BP instruction.
46002 ;;;
46003 ;;;what we do here is do a call to a function that is less than 5Dh
46004 ;;;bytes away (and assert its not exactly 5D away) that jmps (transfers)
46005 ;;;to the correct function. This cannot accidentally insert a 5Dh.
46006 ;;;
46007 ;;;More info:
46008 ;;; This particular scan is begun at the UNCDone label for 32 bytes
46009 ;;;looking for pop BP, so you cannot put a 5D between here and there.
46010 ;;;
46011 ; call    no5Dshere
46012 start5Dhack:
46013 ;following is replaced with 5Dhack code--Invoke CheckThisDevice
46014 backfrom5Dhack:
46015
46016 %endif
46017
46018 ; 13/03/2024
46019 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:BECBh
46020 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:AC47h)
46021 ; ((Windows ME IO.SYS - BIOSCODE:A6C2h))
46022 %if 0
46023 ; call    no5Dshere
46024 %else
46025 ; 13/03/2024 - Retro DOS v5.0
46026
46027 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
46028 ; Note: 'call no5Dshere' is not required for MSDOS 5.0 MSDOS.SYS
46029 00007D07 E81FD2      call    CheckThisDevice    ; E8h,6Fh,0D6h
46030 %endif
46031 00007D0A 5D          pop     bp
46032 00007D0B 58          pop     ax          ; get drive letter back
46033 00007D0C 731C      JNC     short DoFile    ; yes we have a file.
46034
46035 ; We have a device. AX has drive letter. At this point we may fake a CDS ala
46036 ; sharing DOS call. We know by getting here that we are NOT in a sharing DOS
46037 ; call.
46038
46039 00007D0E 36C606[7205]FF  MOV     byte [SS:FSHARING],-1 ; simulate sharing dos call;smr;SS Override
46040 00007D14 E871FE      call    GETTHISDRV        ; set ThisCDS and init DUMMYCDS
46041 00007D17 36C606[7205]00  MOV     byte [SS:FSHARING],0 ; ;smr;SS Override
46042
46043 ; Now that we have noted that we have a device, we put it into a form that
46044 ; getpath can understand. Normally getpath requires d:\ to begin the input
46045 ; string. We relax this to state that if the d:\ is present then the path
46046 ; may be a file. If D:/ (note the forward slash) is present then we have
46047 ; a device.
46048
46049 00007D1D E87A02      CALL    TextFromDrive
46050 00007D20 B02F      MOV     AL,'/'          ; path sep.
46051 00007D22 AA          STOSB
46052 00007D23 E88B9A      call    StrCpy          ; move remainder of string
46053
46054 00007D26 F8          CLC                    ; everything OK.
46055 00007D27 16          push    ss
46056 00007D28 1F          pop     ds          ; remainder of OK stuff
46057 DoFile_retn:
46058 00007D29 C3          retn
46059
46060 ; 13/03/2024 - Retro DOS v5.0
46061 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:BEEeh
46062 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:AC6Ah)
46063 ; ((Windows ME IO.SYS - BIOSCODE:A6F2h))
46064 %if 1
46065
46066 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
46067 %if 0
46068
46069 no5Dshere:
46070 ; 10/08/2018
46071 jmp     CheckThisDevice    ; snoop for device
46072 %endif
46073
46074 ;.erre (no5Dshere - start5Dhack - 5D)
46075
46076 ; We have a file. Get the raw CDS.
46077
46078 DoFile:
46079 ; MSDOS 3.3 (& MSDOS 6.0)
46080
46081 00007D2A E83FFE      call    GetVisDrv        ; get proper CDS
46082 ;mov     al,3
46083 00007D2D B003      MOV     AL,error_path_not_found ; Set up for possible bad file error
46084 00007D2F 72F8      jc     short DoFile_retn ; CARRY set -> bogus drive/spliced
46085
46086 ; ThisCDS has correct CDS. DS:SI advanced to point to beginning of path/file.
46087 ; Make sure that CDS has valid directory; ValidateCDS requires a temp buffer
46088 ; Use the one that we are going to use (ES:DI).
46089
46090 ;SAVE     <DS,SI,ES,DI>          ; save all string pointers.
46091 00007D31 1E          push    ds
46092 00007D32 56          push    si
46093 00007D33 06          push    es
46094 00007D34 57          push    di
46095 00007D35 E824D1      call    ValidateCDS        ; poke CDS and make everything OK
46096 ;RESTORE <DI,ES,SI,DS>          ; get back pointers
46097 00007D38 5F          pop     di
46098 00007D39 07          pop     es
46099 00007D3A 5E          pop     si
46100 00007D3B 1F          pop     ds
46101 ;mov     al,3
46102 00007D3C B003      MOV     AL,error_path_not_found ; Set up for possible bad path error
46103 ;retc
46104 00007D3E 72E9      jc     short DoFile_retn
46105
46106 ; ThisCDS points to correct CDS. It contains the correct text of the
46107 ; current directory. Copy it in.
46108
46109 00007D40 1E          push    ds
46110 00007D41 56          push    si
46111 00007D42 36C536[A205] LDS     SI,[SS:THISCDS]        ; point to CDS ;smr;SS Override
46112 00007D47 89FB      MOV     BX,DI          ; point to destination
46113 ;add     bx,[si+79] ; MSDOS 6.0
46114 00007D49 035C4F      ADD     BX,[SI+curdir.end]    ; point to backup limit

```

```

46115             ;lea    bp,[di+134]
46116 00007D4C 8DAD8600    LEA     BP,[DI+TEMPLEN]           ; regenerate end of buffer
46117                                     ;AN000;
46118 00007D50 E86D9A      call    FStrCpy           ; copy string. ES:DI point to end
46119 00007D53 4F          DEC     DI             ; point to NUL byte
46120
46121             ; Make sure that there is a path char at end.
46122
46123 00007D54 B05C          MOV     AL,'\'
46124 00007D56 263845FF    CMP     [ES:DI-1],AL
46125 00007D5A 7401          JZ      short GetOrig
46126 00007D5C AA          STOSB
46127
46128             ; Now get original string.
46129
46130 GetOrig:
46131 00007D5D 4F          DEC     DI             ; point to path char
46132 00007D5E 5E          pop     si
46133 00007D5F 1F          pop     ds
46134
46135             ; BX points to the end of the root part of the CDS (at where a path char
46136             ; should be). Now, we decide whether we use this root or extend it with the
46137             ; current directory. See if the input string begins with a leading
46138             ; character.
46139 00007D60 E8CE00      CALL    PathSep           ; is DS:SI a path sep?
46140 00007D63 7511          JNZ     short PathAssure ; no, DI is correct. Assure a path char
46141 00007D65 08C0          OR      AL,AL           ; end of string?
46142 00007D67 7410          JZ      short DoCanon   ; yes, skip.
46143
46144             ; The string does begin with a \. Reset the beginning of the canonicalization
46145             ; to this root. Make sure that there is a path char there and advance the
46146             ; source string over all leading \s.
46147
46148 00007D69 89DF          MOV     DI,BX           ; back up to root point.
46149 SkipPath:
46150             LODSB
46151             call    PATHCHRCMP
46152             JZ      short SkipPath
46153             DEC     SI
46154             OR      AL,AL
46155             JZ      short DoCanon
46156
46157             ; DS:SI start at some file name. ES:DI points at some path char. Drop one in
46158             ; for yucks.
46159
46160 PathAssure:
46161 00007D76 B05C          MOV     AL,'\' ; 5Ch
46162 00007D78 AA          STOSB
46163
46164             ; ES:DI point to the correct spot for canonicalization to begin.
46165             ; BP is the max extent to advance DI
46166             ; BX is the backup limit for ..
46167
46168 DoCanon:
46169 00007D79 E85200      CALL    Canonicalize     ; wham.
46170             ;retc          ; badly formatted path.
46171 00007D7C 72AB          jc      short DoFile_retn
46172
46173             ; The string has been moved to ES:DI. Reset world to DOS context, pointers
46174             ; to wfp_start and do string substitution. BP is still the max position in
46175             ; buffer.
46176
46177             push    ss
46178             pop     ds
46179 00007D80 8B3E[B205]    MOV     DI,[WFP_START]   ; DS:SI point to string
46180 00007D84 C536[A205]    LDS     SI,[THISCDS]     ; point to CDS
46181 00007D88 E81702      CALL    PathPref         ; is there a prefix?
46182 00007D8B 7514          JNZ     short DoSplice   ; no, do splice
46183
46184             ; We have a match. Check to see if we ended in a path char.
46185
46186 00007D8D 8A44FF      MOV     AL,[SI-1]        ; last char to match
46187 00007D90 E853E0      call    PATHCHRCMP       ; did we end on a path char? (root)
46188 00007D93 740C          JZ      short DoSplice   ; yes, no current dir here.
46189
46190 Pathline:
46191 00007D95 26803D00    CMP     BYTE [ES:DI],0    ; end at NUL?
46192 00007D99 7406          JZ      short DoSplice
46193 00007D9B 47          INC     DI             ; point to after current path char
46194 00007D9C 36893E[B605]    MOV     [SS:Curr_dir_end],DI ; point to correct spot ;smr;SS override
46195
46196             ; Splice the result.
46197
46198 DoSplice:
46199             push    ss
46200             pop     ds
46201             MOV     SI,[WFP_START] ; point to beginning of string
46202             XOR     CX,CX
46203             TEST    byte [FSPLICE],-1
46204             JZ      short SkipSplice
46205             CALL    Splice           ; replaces in place.
46206 SkipSplice:
46207
46208             ; The final thing is to assure ourselves that a FATREAD is done on the local
46209             ; device.
46210
46211             push    ss
46212             pop     ds
46213             LES     DI,[THISCDS]     ; point to correct drive
46214             ;test word [es:di+67],8000h
46215             ; 17/12/2022
46216             ;test byte [es:di+68],80h
46217             test    byte [ES:DI+curdir.flags+1],curdir_isnet>>8 ; 04/12/2022
46218             ;TEST word [ES:DI+curdir.flags],curdir_isnet ; 8000h
46219             JNZ     short Done       ; net, no fatread necessary (retnz)
46220             JCXZ    Done
46221             call    ECritDisk
46222             call    FATREAD_CDS
46223             call    LCritDisk
46224             ;mov al, 3
46225             MOV     AL,error_path_not_found ; Set up for possible bad path error
46226 Done:
46227             retn           ; any errors in carry flag.
46228
46229             ; 13/07/2018
46230
46231             ;BREAK <Canonicalize - copy a path and remove . and .. entries>
46232             ;-----
46233             ; Canonicalize - copy path removing . and .. entries.
46234             ;
46235             ; Inputs:      DS:SI - point to ASCIZ string path
46236             ;              ES:DI - point to buffer
46237             ;              BX - backup limit (offset from ES) points to slash
46238             ;              BP - end of buffer
46239             ; Outputs:    Carry Set - invalid path specification: too many .., bad
46240             ;              syntax, etc.

```

```

46240 ; Carry Clear -
46241 ; DS:DI - advanced to end of string
46242 ; ES:DI - advanced to end of canonicalized form after nul
46243 ; Registers modified: AX CX DX (in addition to those above)
46244 ;-----
46245 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46246 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
46247
46248 Canonicalize:
46249 ; we copy all leading path separators.
46250
46251 LODSB ; while (PathChr (*s))
46252 call PATHCHRCMP
46253 JNZ short CanonDec
46254 CMP DI,BP ; if (d > dlim)
46255 JAE short CanonBad ; goto error;
46256 STOSB
46257 JMP short Canonicalize ; *d++ = *s++;
46258 CanonDec:
46259 DEC SI
46260
46261 ; Main canonicalization loop. We come here with DS:SI pointing to a textual
46262 ; component (no leading path separators) and ES:DI being the destination
46263 ; buffer.
46264
46265 CanonLoop:
46266 ; If we are at the end of the source string, then we need to check to see that
46267 ; a potential drive specifier is correctly terminated with a path sep char.
46268 ; Otherwise, do nothing
46269
46270 XOR AX,AX
46271 CMP [SI],AL ; if (*s == 0) {
46272 JNZ short DoComponent
46273 CMP BYTE [ES:DI-1],':' ; if (d[-1] == ':')
46274 JNZ short DoTerminate
46275 MOV AL,'\' ; *d++ = '\';
46276 STOSB
46277 MOV AL,AH
46278 DoTerminate:
46279 STOSB ; *d++ = 0;
46280 CLC ; return (0);
46281 retn
46282
46283 CanonBad:
46284 CALL ScanPathChar ; check for path chars in rest of string
46285 ;mov al,3
46286 MOV AL,error_path_not_found ; Set up for bad path error
46287 JZ short PathEnc ; path character encountered in string
46288 ;mov al,2
46289 MOV AL,error_file_not_found ; Set bad file error
46290 PathEnc:
46291 STC
46292 CanonBad_retn:
46293 retn
46294
46295 ; We have a textual component that we must copy. We uppercase it and truncate
46296 ; it to 8.3
46297
46298 DoComponent:
46299 ; if (!CopyComponent (s, d))
46300 CALL CopyComponent ; if (!CopyComponent (s, d))
46301 jc short CanonBad_retn ; return (-1);
46302
46303 ; We special case the . and .. cases. These will be backed up.
46304
46305 ;CMP WORD PTR ES:[DI], '.' + (0 SHL 8)
46306 CMP WORD [ES:DI],002Eh
46307 JZ short Skip1
46308 ;CMP WORD PTR ES:[DI], '..'
46309 CMP WORD [ES:DI],2E2Eh
46310 JNZ short CanonNormal
46311 DEC DI ; d--;
46312 Skip1:
46313 CALL SkipBack ; SkipBack ();
46314 ;mov al,3
46315 ; 07/07/2024 (*)
46316 ;MOV AL,error_path_not_found ; Set up for possible bad path error
46317 jc short CanonBad_retn ; AL=3 (*)
46318 JMP short CanonPath ; }
46319
46320 ; We have a normal path. Advance destination pointer over it.
46321
46322 CanonNormal:
46323 ; ; else
46324 ADD DI,CX ; d += ct;
46325
46326 ; We have successfully copied a component. We are now pointing at a path
46327 ; sep char or are pointing at a nul or are pointing at something else.
46328 ; If we point at something else, then we have an error.
46329
46330 CanonPath:
46331 CALL PathSep
46332 JNZ short CanonBad ; something else...
46333
46334 ; Copy the first path char we see.
46335
46336 LODSB ; get the char
46337 call PATHCHRCMP ; is it path char?
46338 JNZ short CanonDec ; no, go test for nul
46339 CMP DI,BP ; beyond buffer end?
46340 JAE short CanonBad ; yep, error.
46341 STOSB ; copy the one byte
46342
46343 ; Skip all remaining path chars
46344
46345 CanonPathLoop:
46346 LODSB ; get next byte
46347 call PATHCHRCMP ; path char again?
46348 JZ short CanonPathLoop ; yep, grab another
46349 DEC SI ; back up
46350 JMP short CanonLoop ; go copy component
46351
46352 ;BREAK <PathSep - determine if char is a path separator>
46353 ;-----
46354 ; PathSep - look at DS:SI and see if char is / \ or NUL
46355 ; Inputs: DS:SI - point to a char
46356 ; Outputs: AL has char from DS:SI (/ => \)
46357 ; Zero set if AL is / \ or NUL
46358 ; Zero reset otherwise
46359 ; Registers modified: AL
46360 ;-----
46361
46362
46363

```



```

46364
46365 00007E31 8A04
46366
46367 00007E33 08C0
46368 00007E35 74C4
46369
46370
46371
46372 00007E37 E9ACDF
46373
46374
46375
46376
46377
46378
46379
46380
46381
46382
46383
46384
46385
46386
46387
46388 00007E3A 39DF
46389 00007E3C 720A
46390 00007E3E 4F
46391 00007E3F 268A05
46392 00007E42 E8A1DF
46393 00007E45 75F3
46394
46395
46396
46397 00007E47 C3
46398
46399
46400 00007E48 B003
46401
46402
46403
46404 00007E4A C3
46405
46406
46407
46408
46409
46410
46411
46412
46413
46414
46415
46416
46417
46418
46419
46420
46421
46422
46423
46424
46425
46426
46427
46428
46429
46430
46431
46432 00007E4B 83EC0E
46433 00007E4E 1E
46434 00007E4F 56
46435 00007E50 06
46436 00007E51 57
46437 00007E52 55
46438 00007E53 89E5
46439 00007E55 B42E
46440 00007E57 AC
46441 00007E58 AA
46442 00007E59 38E0
46443 00007E5B 7518
46444 00007E5D E8D1FF
46445 00007E60 740B
46446
46447 00007E62 AC
46448 00007E63 AA
46449 00007E64 38E0
46450 00007E66 7557
46451 00007E68 E8C6FF
46452 00007E6B 7552
46453
46454 00007E6D 30C0
46455 00007E6F AA
46456 00007E70 897606
46457 00007E73 EB47
46458
46459 00007E75 8B7606
46460 00007E78 E8AFDE
46461 00007E7B 3B7606
46462 00007E7E 743F
46463 00007E80 36F606[7205]FF
46464 00007E86 7510
46465 00007E88 80E201
46466 00007E8B 360016[7A05]
46467 00007E90 7F2D
46468 00007E92 7504
46469 00007E94 08D2
46470 00007E96 742F
46471
46472 00007E98 897606
46473 00007E9B 16
46474 00007E9C 1F
46475 00007E9D BE[4B05]
46476 00007EA0 8D7E0A
46477 00007EA3 57
46478 00007EA4 E8FFA7
46479 00007EA7 5F
46480 00007EA8 E81E99
46481 00007EAB 49
46482 00007EAC 034E02
46483
46484
46485
46486
46487 00007EAF 3B4E00
46488 00007EB2 730B

PathSep:
    MOV     AL,[SI]                ; get the character
PathSepGotch:
    OR      AL,AL                  ; already have character
    OR      AL,AL                  ; test for zero
    JZ      short CanonBad_retn    ; return if equal to zero (NUL)
    ;call   PATHCHRCMP             ; check for path character
    ;retn                                ; and return HIS determination
    ; 18/12/2022
    jmp     PATHCHRCMP

;BREAK <SkipBack - move backwards to a path separator>
;-----
; SkipBack - look at ES:DI and backup until it points to a / ; Inputs: ES:DI - point to a char
; BX has current directory back up limit (point to a / \)
; Outputs: ES:DI backed up to point to a path char
; AL has char from output ES:DI (path sep if carry clear)
; Carry set if illegal backup
; Carry Clear if ok
; Registers modified: DI,AL
;-----

SkipBack:
    CMP     DI,BX                  ; while (TRUE) {
    JB      short SkipBad          ; if (d < dlim)
    DEC     DI                     ; goto err;
    MOV     AL,[ES:DI]              ; if (pathchr (*--d))
    call    PATHCHRCMP             ; break;
    JNZ     short SkipBack         ; }
    ;CLC
    ; 01/07/2024
    ; cf=0
    retn
SkipBad:
    ;err:
    mov     al,3
    MOV     AL,error_path_not_found ; bad path error
    ;STC
    ; 01/07/2024
    ; cf=1
    retn

;Break <CopyComponent - copy out a file path component>
;-----
; CopyComponent - copy a file component from a path string (DS:SI) into ES:DI
; Inputs: DS:SI - source path
; ES:DI - destination
; ES:BP - end of buffer
; Outputs: Carry Set - too long
; Carry Clear - DS:SI moved past component
; CX has length of destination
; Registers modified: AX,CX,DX
;-----

; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)

CopyComponent:
%define CopyBP [BP] ; word
%define CopyD [BP+2] ; dword
%define CopyDoff [BP+2] ; word
%define CopyS [BP+6] ; dword
%define CopySoff [BP+6] ; word
%define CopyTemp [BP+10] ; byte

SUB     SP,14 ; room for temp buffer
push    ds
push    si
push    es
push    di
push    bp
MOV     BP,SP
MOV     AH,'.'
LODSB
STOSB
CMP     AL,AH ; if ((*d++=*s++) == '.') {
JNZ     short NormalComp ; if (!pathsep(*s))
CALL    PathSep ;
JZ      short NulTerm

TryTwoDot:
LODSB ; if ((*d++=*s++) != '.')
STOSB
CMP     AL,AH
JNZ     short CopyBad
CALL    PathSep
JNZ     short CopyBad ; || !pathsep(*s))
NulTerm:
XOR     AL,AL ; ; *d++ = 0;
STOSB
MOV     CopySoff,SI
JMP     SHORT _Goodret ; }
NormalComp:
MOV     SI,CopySoff ; [bp+6] ; else {
call    NameTrans ; s = NameTrans (s, Name1);
CMP     SI,CopySoff ; if (s == CopySoff)
JZ      short CopyBad ; return (-1);
TEST    byte [SS:FSHARING],-1 ; if (!fsHaring) {;smr;SS override
JNZ     short DoPack
AND     DL,1 ; cMeta += fMeta;
ADD     [ss:CMETA],DL ; if (cMeta > 0);smr;SS override
JG      short CopyBad ; return (-1);
JNZ     short DoPack ; else
OR      DL,DL ; if (cMeta == 0 && fMeta == 0)
JZ      short CopyBadPath ; return (-1);
DoPack:
MOV     CopySoff,SI ; [bp+6] ;
push    ss
pop     ds
MOV     SI,Name1
LEA     DI,CopyTemp ; [bp+10]
push    di
call    PackName ; PackName (Name1, temp);
pop     di
call    StrLen ; if (strlen(temp)+d > bp)
DEC     CX
ADD     CX,CopyDoff ; [bp+2]
; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
; cmp     CX,[bp+0]
; 15/12/2022
; cmp     cx,[bp]
CMP     CX,CopyBP ; [bp+0]
JAE     short CopyBad ; return (-1);

```

```

46489 00007EB4 89FE      MOV     SI,DI      ;      strcpy (d, temp);
46490 00007EB6 C47E02    LES     DI,CopyD   ; [bp+2]
46491 00007EB9 E80499    call    FStrCpy
46492 _GoodRet:      ;      }
46493 00007EBC F8      CLC
46494 00007EBD E80B      JMP     SHORT CopyEnd      ;      return 0;
46495 CopyBad:
46496 00007EBF F9      STC
46497 00007EC0 E8F800    CALL    ScanPathChar      ; check for path chars in rest of string
46498 ;mov     al,2
46499 00007EC3 B002      MOV     AL,error_file_not_found ; Set up for bad file error
46500 00007EC5 7503      JNZ     short CopyEnd
46501 CopyBadPath:
46502 00007EC7 F9      STC
46503 ;mov     al,3
46504 00007EC8 B003      MOV     AL,error_path_not_found ; Set bad path error
46505 CopyEnd:
46506 00007ECA 5D      pop     bp
46507 00007ECB 5F      pop     di
46508 00007ECC 07      pop     es
46509 00007ECD 5E      pop     si
46510 00007ECE 1F      pop     ds
46511 00007ECF 9F      LAHF
46512 00007ED0 83C40E    ADD     SP,14      ; reclaim temp buffer
46513 00007ED3 E8F398    call    StrLen
46514 00007ED6 49      DEC     CX
46515 00007ED7 9E      SAHF
46516 00007ED8 C3      retn
46517
46518 ; 14/05/2019 - Retro DOS v4.0
46519 ; DOSCODE:AE22h (MSDOS 6.21, MSDOS.SYS)
46520
46521 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46522 ; DOSCODE:ADBFh (MSDOS 5.0, MSDOS.SYS)
46523
46524 ;Break <Splice - pseudo mount by string substitution>
46525 -----
46526 ; Splice - take a string and substitute a prefix if one exists. Change
46527 ; ThisCDS to point to physical drive CDS.
46528 ; Inputs: DS:SI point to string
46529 ; NoSetDir = TRUE => exact matches with splice fail
46530 ; Outputs: DS:SI points to thisCDS
46531 ; ES:DI points to DPB
46532 ; String at DS:SI may be reduced in length by removing prefix
46533 ; and substituting drive letter.
46534 ; CX = 0 If no splice done
46535 ; CX <> 0 otherwise
46536 ; ThisCDS points to proper CDS if spliced, otherwise it is
46537 ; left alone
46538 ; ThisDPB points to proper DPB
46539 ; Registers modified: DS:SI, ES:DI, BX,AX,CX
46540 -----
46541
46542 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
46543 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C0A6h
46544
46545 Splice:
46546 00007ED9 36F606[5A00]FF TEST     byte [SS:SPLICES],-1 ;smr;SS Override
46547 00007EDF 7469      JZ      short AllDone
46548 00007EE1 36FF36[A205] push     word [SS:THISCDS]
46549 00007EE6 36FF36[A405] push     word [SS:THISCDS+2] ; TmpCDS = ThisCDS;smr;SS Override
46550 00007EEB 1E      push     ds
46551 00007EEC 56      push     si
46552 00007EED 5F      pop      di
46553 00007EEE 07      pop      es
46554 00007EEF 31C0      XOR     AX,AX      ; for (i=1; s = GetCDSFromDrv (i); i++)
46555
46556 00007EF1 E8E3FC    call    GetCDSFromDrv
46557 00007EF4 724A      JC      short SpliceDone
46558 00007EF6 FEC0      INC     AL
46559 ; 17/12/2022
46560 ;test     byte [si+68],20h
46561 00007EF8 F6444420 test     byte [si+curdir.flags+1],curdir_splice>>8 ; 04/12/2022
46562 ;;test     word [si+67],2000h
46563 ;TEST     word [SI+curdir.flags],curdir_splice
46564 00007EFC 74F3      JZ      short SpliceScan ; if ( Spliced (i) ) {
46565 00007EFE 57      push     di
46566 00007EFF E8A000    CALL    PathPref      ; if (!PathPref (s, d))
46567 00007F02 7403      JZ      short SpliceFound ;
46568 SpliceSkip:
46569 00007F04 5F      pop      di
46570 00007F05 EBEA      JMP     short SpliceScan ; continue;
46571 SpliceFound:
46572 00007F07 26803D00 CMP     BYTE [ES:DI],0 ; if (*s || NoSetDir) {
46573 00007F0B 7508      JNZ     short SpliceDo
46574 00007F0D 36F606[4C03]FF TEST     byte [ss:NoSetDir],-1 ;smr;SS Override
46575 00007F13 75EF      JNZ     short SpliceSkip
46576 SpliceDo:
46577 00007F15 89FE      MOV     SI,DI      ; p = src + strlen (p);
46578 00007F17 06      push     es
46579 00007F18 1F      pop      ds
46580 00007F19 5F      pop      di
46581 00007F1A E87F00    CALL    TextFromDrive1 ; src = TextFromDrive1(src,i);
46582 00007F1D 36A1[B605] MOV     AX,[SS:CURR_DIR_END] ;smr;SS Override
46583 00007F21 09C0      OR      AX,AX
46584 00007F23 7808      JS      short NoPoke
46585 00007F25 01F8      ADD     AX,DI      ; curdirend += src-p;
46586 00007F27 29F0      SUB     AX,SI
46587 00007F29 36A3[B605] MOV     [SS:CURR_DIR_END],AX ;smr;SS Override
46588 NoPoke:
46589 00007F2D 803C00    CMP     BYTE [SI],0 ; if (*p)
46590 00007F30 7503      JNZ     short SpliceCopy ; *src++ = '\\';
46591 00007F32 B05C      MOV     AL,"\"
46592 00007F34 AA      STOSB
46593 SpliceCopy:      ; strcpy (src, p);
46594 00007F35 E88898    call    FStrCpy
46595 00007F38 83C404    ADD     SP,4      ; throw away saved stuff
46596 00007F3B 80C901    OR      CL,1      ; signal splice done.
46597 00007F3E E80C      JMP     SHORT DoSet ; return;
46598 SpliceDone:      ;
46599 00007F40 368F06[A405] pop     word [SS:THISCDS+2] ; ThisCDS = TmpCDS;
46600 00007F45 368F06[A205] pop     word [SS:THISCDS] ;smr;SS Override
46601 AllDone:
46602 00007F4A 31C9      XOR     CX,CX
46603 DoSet:
46604 00007F4C 36C536[A205] LDS     SI,[SS:THISCDS] ; ThisDPB = ThisCDS->devptr;smr;SS Override
46605 ;les     di,[si+69]
46606 00007F51 C47C45    LES     DI,[SI+curdir.devptr]
46607 00007F54 36893E[8A05] MOV     [SS:THISDPB],DI ;smr;SS Override
46608 00007F59 368C06[8C05] MOV     [SS:THISDPB+2],ES ;smr;SS Override
46609 Splice_retn:
46610 00007F5E C3      retn
46611
46612 ; 15/05/2019 - Retro DOS v4.0

```



```

46613 ; DOSCODE:AEA9h (MSDOS 6.21, MSDOS.SYS)
46614 ;
46615 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46616 ; DOSCODE:AE46h (MSDOS 5.0, MSDOS.SYS)
46617 ;
46618 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
46619 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C12Dh
46620 ;
46621 ;Break <$NameTrans - partially process a name>
46622 ;-----
46623 ; $NameTrans - allow users to see what names get mapped to. This call
46624 ; performs only string substitution and canonicalization, not splicing. Due
46625 ; to Transpath playing games with devices, we need to insure that the output
46626 ; has drive letter and : in it.
46627 ;
46628 ; Inputs:      DS:SI - source string for translation
46629 ;             ES:DI - pointer to buffer
46630 ;
46631 ; Outputs:
46632 ;   Carry Clear
46633 ;   Buffer at ES:DI is filled in with data
46634 ;   ES:DI point byte after nul byte at end of dest string in buffer
46635 ;   Carry Set
46636 ;   AX = error_path_not_found
46637 ;   Registers modified: all
46638 ;-----
46639 _$NameTrans:
46640     push    ds
46641     push    si
46642     push    es
46643     push    di
46644     push    cx ; MSDOS 6.0
46645 ;
46646 ; MSDOS 6.0
46647 ; M027 - Start
46648 ;
46649 ; Sattrib must be set up with default values here. Otherwise, the value from
46650 ; a previous DOS call is used for attrib and DevName thinks it is not a
46651 ; device if the old call set the volume attribute bit. Note that devname in
46652 ; dir2.asm gets ultimately called by Transpath. See also M026. Also save
46653 ; and restore CX.
46654 ;
46655 ;mov     ch,16h
46656     mov     ch,attr_hidden+attr_system+attr_directory
46657     call    SetAttrib
46658 ;
46659 ; M027 - End
46660 ;
46661 ; MSDOS 3.3 (& MSDOS 6.0)
46662     MOV     DI,OPENBUF
46663     CALL    TransPath ; to translation (everything)
46664     pop     cx ; MSDOS 6.0
46665     pop     di
46666     pop     es
46667     pop     si
46668     pop     ds
46669     JNC     short TransOK
46670     jmp     SYS_RET_ERR
46671 TransOK:
46672     MOV     SI,OPENBUF
46673     push    ss
46674     pop     ds
46675 ;GotText:
46676     call    FStrCpy
46677     jmp     SYS_RET_OK
46678 ;
46679 ;Break <DriveFromText - return drive number from a text string>
46680 ;-----
46681 ; DriveFromText - examine DS:SI and remove a drive letter, advancing the
46682 ; pointer.
46683 ;
46684 ; Inputs:      DS:SI point to a text string
46685 ; Outputs:     AL has drive number
46686 ;             DS:SI advanced
46687 ; Registers modified: AX,SI.
46688 ;-----
46689 ;
46690 DriveFromText:
46691     XOR     AL,AL ; drive = 0;
46692     ;CMP     BYTE [SI],0 ; if (*s &&
46693     ; 23/09/2023
46694     cmp     [si],al ; 0
46695     jz     short Splice_retn
46696     CMP     BYTE [SI+1],':' ; s[1] == ':' {
46697     jnz     short Splice_retn
46698     LODSW
46699     OR      AL,20h ; (convert to lowercase)
46700     ;sub     al,60h
46701     SUB     AL,'a'-1 ; s += 2;
46702     jnz     short Splice_retn
46703     MOV     AL,-1 ; nuke AL...
46704     ; 23/09/2023
46705     ;dec     al ; -1
46706     retn
46707 ;
46708 ;Break <TextFromDrive - convert a drive number to a text string>
46709 ;-----
46710 ; TextFromDrive - turn AL into a drive letter: and put it at es:di with
46711 ; trailing :. TextFromDrive1 takes a 1-based number.
46712 ;
46713 ; Inputs:      AL has 0-based drive number
46714 ; Outputs:     ES:DI advanced
46715 ; Registers modified: AX
46716 ;-----
46717 ;
46718 TextFromDrive:
46719     INC     AL
46720 TextFromDrive1:
46721     ;add     al,40h
46722     ADD     AL,'A'-1 ; *d++ = drive-1+'A';
46723     MOV     AH,": " ; 3Ah ; strcat (d, ":");
46724     STOSW
46725 PathPref_retn:
46726     retn
46727 ;
46728 ;Break <PathPref - see if one path is a prefix of another>
46729 ;-----
46730 ; PathPref - compare DS:SI with ES:DI to see if one is the prefix of the
46731 ; other. Remember that only at a pathchar break are we allowed to have a
46732 ; prefix: A:\ and A:\FOO
46733 ;
46734 ; Inputs:      DS:SI potential prefix
46735 ;             ES:DI string
46736 ; Outputs:     Zero set => prefix found

```

```

46737 ; DI/SI advanced past matching part
46738 ; Zero reset => no prefix, DS/SI garbage
46739 ; Registers modified: CX
46740 ; -----
46741
46742 PathPref:
46743     call    DStrLen          ; get length
46744     dec     CX              ; do not include nul byte
46745     repz    cmpsb           ; compare
46746     jnz     short PathPref_retn ; if NZ then return NZ
46747     push    ax              ; save char register
46748     mov     AL,[SI-1]       ; get last byte to match
46749     call    PATHCHRCMP      ; is it a path char (Root!)
46750     jz      short Prefix    ; yes, match root (I hope)
46751
46752 NotSep:
46753     mov     AL,[ES:DI]       ; get next char to match
46754     call    PathSepGotCh     ; was it a pathchar?
46755
46756 Prefix:
46757     pop     ax              ; get back original
46758     retn
46759
46760 ;Break <ScanPathChar - see if there is a path character in a string>
46761 ; -----
46762 ; ScanPathChar - search through the string (pointed to by DS:SI) for
46763 ; a path separator.
46764 ;
46765 ; Input: DS:SI target string (null terminated)
46766 ; Output: Zero set => path separator encountered in string
46767 ; Zero clear => null encountered
46768 ; Registers modified: SI
46769 ; -----
46770 ScanPathChar:
46771     lodsb                ; fetch a character
46772     call    PathSepGotCh
46773     jnz     short ScanPathChar ; not \, / or NUL => go back for more
46774     ; call    PATHCHRCMP      ; path separator?
46775     ; retn
46776     ; 18/12/2022
46777     jmp     PATHCHRCMP
46778
46779 ;=====
46780 ; FILE.ASM, MSDOS 6.0, 1991
46781 ;=====
46782 ; 14/07/2018 - Retro DOS v3.0
46783
46784 ; 13/05/2019 - Retro DOS v4.0
46785 ; DOSCODE:AF10h (MSDOS 6.21, MSDOS.SYS)
46786
46787 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46788 ; DOSCODE:AEADh (MSDOS 5.0, MSDOS.SYS)
46789
46790 ; 14/03/2024 - Retro DOS v5.0
46791
46792 ; MSDOS 2.11
46793 ;BREAK <$Open - open a file handle>
46794 ; -----
46795 ; Assembler usage:
46796 ;     LDS     DX, Name
46797 ;     MOV     AH, Open
46798 ;     MOV     AL, access
46799 ;     INT     int_command
46800 ;
46801 ; ACCESS      Function
46802 ; -----
46803 ; open_for_read file is opened for reading
46804 ; open_for_write file is opened for writing
46805 ; open_for_both file is opened for both reading and writing.
46806 ;
46807 ; Error returns:
46808 ;     AX = error_invalid_access
46809 ;         = error_file_not_found
46810 ;         = error_access_denied
46811 ;         = error_too_many_open_files
46812 ; -----
46813 ; MSDOS 6.0
46814 ; BREAK <$Open - open a file from a path string>
46815 ; -----
46816
46817 ;** $Open - Open a File
46818 ;
46819 ; given a path name in DS:DX and an open mode in AL, $Open opens the
46820 ; file and returns a handle
46821 ;
46822 ; ENTRY (DS:DX) = pointer to asciz name
46823 ; (AL) = open mode
46824 ; EXIT 'C' clear if OK
46825 ; (ax) = file handle
46826 ; 'C' set if error
46827 ; (ax) = error code
46828 ;
46829 ; USES all
46830 ; -----
46831
46832 ; 13/05/2019 - Retro DOS v4.0
46833 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46834
46835 ; 14/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
46836 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C194h
46837 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0AF10h)
46838 ; (Windows ME IO.SYS - BIOSCODE:0A9A7h)
46839
46840 _$OPEN:
46841     xor     ah,ah ; MSDOS 6.0
46842 _$Open2:
46843     ;mov     ch,16h
46844     mov     ch,attr_hidden+attr_system+attr_directory
46845     call    SetAttrib
46846     mov     cx,DOS_OPEN
46847
46848     ;xor     ah,ah ; MSDOS 3.3
46849
46850     push    ax
46851
46852 ;* General file open/create code. The $CREATE call and the various
46853 ; $OPEN calls all come here.
46854 ;
46855 ; We'll share a lot of the standard stuff of allocating SFTs, cracking
46856 ; path names, etc., and then dispatch to our individual handlers.
46857 ; WARNING - this info and list is just a guess, not definitive - jgl
46858 ;
46859 ; (TOS) = create mode
46860 ; (CX) = address of routine to call to do actual function

```

```

46861 ; (DS:DX) = ASCIZ name
46862 ; SAttrib = Attribute mask
46863
46864 ; Get a free SFT and mark it "being allocated"
46865
46866 AccessFile:
46867 call ECritSFT
46868 call SFNFree ; get a free sfn
46869 call LCritSFT
46870 ;jc short OpenFail ; oops, no free sft's
46871 ; 14/03/2024
46872 jc short OpenFail
46873
46874 MOV [SS:SFN],BX ; save the SFN for later;smr;SS Override
46875 MOV [SS:THISSFT],DI ; save the SF offset ;smr;SS Override
46876 MOV [SS:THISSFT+2],ES ; save the SF segment ;smr;SS Override
46877
46878 ; Find a free area in the user's JFN table.
46879
46880 call JFNFree ; get a free jfn
46881 ;jnc short SaveJFN
46882 ; 14/03/2024
46883 jc short OpenFail
46884 ;
46885 ;OpenFail:
46886 ;JMP OpenFail ; there were free JFNs... try SFN
46887
46888 SaveJFN:
46889 mov [ss:PJFN],DI ; save the jfn offset ;smr;SS Override
46890 MOV [ss:PJFN+2],ES ; save the jfn segment;smr;SS Override
46891 MOV [ss:JFN],BX ; save the jfn itself ;smr;SS Override
46892
46893 ; We have been given an JFN. We lock it down to prevent other tasks from
46894 ; reusing the same JFN.
46895
46896 MOV BX,[ss:SFN] ;smr;SS Override
46897 MOV [ES:DI],BL ; assign the JFN
46898 MOV SI,DX ; get name in appropriate place
46899 MOV DI,OPENBUF ; appropriate buffer
46900 push CX ; save routine to call
46901 call TransPath ; convert the path
46902 pop bx ; (bx) = routine to call
46903
46904 LDS SI,[ss:THISSFT] ;smr;SS Override
46905 ;JC short OpenClean ; no error, go and open file
46906 ; 14/03/2024
46907 jc short OpenClean
46908
46909 CMP byte [ss:CMETA],-1 ;smr;SS Override
46910 JZ short SetSearch
46911 ;mov al,2
46912 MOV AL,error_file_not_found ; no meta chars allowed
46913
46914 OpenClean:
46915 JMP short OpenClean
46916 ; 14/03/2024 (PCDOS 7.1 IBMDOS.COM)
46917 ;;;
46918
46919 OpenFail:
46920 STI
46921 pop cx ; clean stack
46922 ;
46923 jmp short OpenCritLeave
46924 ;;;
46925
46926 SetSearch:
46927 pop ax ; Mode (Open), Attributes (Create)
46928
46929 ; We need to get the new inheritance bits.
46930
46931 xor cx,cx
46932 ; MSDOS 6.0
46933 ;mov [si+2],cx ; 0
46934 MOV [SI+SF_ENTRY.sf_mode],cx ; initialize mode field to 0
46935 ;mov [si+51],cx ; 0
46936 MOV [SI+SF_ENTRY.sf_MFT],cx ; clean out sharing info
46937 ;
46938 CMP BX,DOS_OPEN
46939 JNZ short _DoOper
46940 ;test al,80h
46941 test AL,SHARING_NO_INHERIT ; look for no inher
46942 JZ short _DoOper ; 10/08/2018
46943 AND AL,7Fh ; mask off inherit bit
46944 ;mov cx,1000h
46945 MOV CX,sf_no_inherit
46946
46947 _DoOper:
46948 ;; MSDOS 3.3
46949 ;;mov word [si+2], 0
46950 ;;mov word [si+33h], 0
46951 ;MOV word [SI+SF_ENTRY.sf_mode],0
46952 ;MOV word [SI+SF_ENTRY.sf_MFT],0
46953 ; MSDOS 6.0
46954 ;** Check if this is an extended open. If so you must set the
46955 ; modes in sf_mode. Call Set_EXT_mode to do all this. See
46956 ; Set_EXT_mode in creat.asm
46957 ; MSDOS 6.0
46958 ;SAVE <di, es> ;M022 conditional removed here
46959 push di
46960 push es
46961 push ds
46962 pop es
46963 push si
46964 pop di ; (es:di) = SFT address
46965 call Set_EXT_mode
46966 ;RESTORE <es, di>
46967 pop es
46968 pop di
46969
46970 ;Context DS
46971 push ss
46972 pop ds
46973
46974 push cx
46975 CALL BX ; blam!
46976 pop cx
46977 LDS SI,[THISSFT]
46978 JC short OpenE2 ;AN000;FT. chek extended open hooks first
46979 ;jc short OpenE ; MSDOS 3.3
46980
46981 ; The SFT was successfully opened. Remove busy mark.
46982
46983 OpenOK:
46984 ;MOV word [SI+SF_ENTRY.sf_ref_count],1

```

```

46985 00008055 C7040100      mov     word [SI],1
46986                        ;or     [SI+5],CX
46987 00008059 094C05      OR      [SI+SF_ENTRY.sf_flags],CX ; set no inherit bit if necessary
46988
46989                        ; If the open mode is 70, we scan the system for other SFT's with the same
46990                        ; contents. If we find one, then we can 'collapse' this sft onto the already
46991                        ; opened one. Otherwise we use this new one. We compare uid/pid/mode/mft
46992                        ;
46993                        ; Since this is only relevant on sharer systems, we stick this code into the
46994                        ; sharer.
46995
46996                        ; 01/07/2024
46997                        ;;;
46998 0000805C 1E          push     ds
46999 0000805D 07          pop      es
47000 0000805E 89F7        mov      di,si
47001 00008060 E8B1C9      call     set_sftfcb_entry      ; set SFT-FCB entry
47002                        ; in the internal (SFT_FCB) table
47003                        ; (used for FCB calls only!)
47004                        ;;;
47005
47006 00008063 36A1[AC05]      MOV      AX,[ss:JFN]          ;smr;SS Override
47007 00008067 36FF1E[C000]    Call     far [ss:Jshare+(12*4)]; 12 = ShCol ;smr;SS Override
47008
47009 0000806C 36C706[AA05]FFFF    MOV      word [ss:SFN],-1      ; clear out sfm pointer      ;smr;SS Override
47010                        ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47011 OpenOkj:
47012 00008073 E9F885      jmp      SYS_RET_OK          ; bye with no errors
47013
47014                        ; Extended Open hooks check
47015                        ;
47016                        ; AL has error code. Stack has argument to dos_open/dos_create.
47017
47018 OpenClean:
47019 00008076 5B          pop      bx          ; clean off stack
47020
47021 OpenE:
47022 00008077 C7040000      ;MOV     word [SI+SF_ENTRY.sf_ref_count],0 ; release SFT
47023 0000807B 36C536[AE05]    MOV      word [SI],0
47024 00008080 C604FF      LDS      SI,[ss:PJFN]        ;smr;SS Override
47025                        MOV      BYTE [SI],0FFh      ; free the SFN...
47026
47027                        ; 14/03/2024
47028                        ;JMP     SHORT OpenCritLeave
47029
47030 OpenFail:
47031                        ;STI
47032                        ;pop     cx          ; Clean stack
47033
47034 00008083 36C706[AA05]FFFF    OpenCritLeave:
47035                        MOV      word [SS:SFN],-1      ; remove mark.
47036
47037                        ; MSDOS 6.0
47038 0000808A 36833E[2403]25    ; File Tagging DOS 4.00
47039                        CMP      word [SS:EXTERR],error_Code_Page_Mismatched
47040                        JNZ      short NORERR          ;AN000;;FT. code page mismatch
47041 00008092 E9EB85      jmp      From_GetSet        ;AN000;;FT. no
47042                        ;AN000;;FT. yes
47043
47044                        ; 14/03/2024
47045                        ; MSDOS 6.0
47046 Extended Open hooks check
47047 00008095 83F857      OpenE2:
47048 00008098 75DD          CMP      AX,error_invalid_parameter ;AN000;;EO. IFS extended open ?
47049 0000809A EBE7          JNZ      short OpenE        ;AN000;;EO. no.
47050                        JMP      short OpenCritLeave    ;AN000;;EO. keep handle
47051
47052                        ; 15/03/2024
47053 %if 0
47054 NORERR:
47055                        ; File Tagging DOS 4.00
47056                        jmp      SYS_RET_ERR          ; no free, return error
47057 %endif
47058
47059                        ; MSDOS 2.11
47060                        ;BREAK <$CREAT - create a new file and open him for input>
47061
47062                        -----
47063                        ; Assembler usage:
47064                        ;     LDS     DX, name
47065                        ;     MOV     AH, Creat
47066                        ;     MOV     CX, access
47067                        ;     INT     21h
47068                        ; AX now has the handle
47069
47070                        ; Error returns:
47071                        ;     AX = error_access_denied
47072                        ;     = error_path_not_found
47073                        ;     = error_too_many_open_files
47074                        -----
47075
47076                        ; MSDOS 6.0
47077                        ; BREAK <$Creat - create a brand-new file>
47078                        -----
47079                        ;** $Creat - Create a File
47080
47081                        ; $Creat creates the directory entry specified in DS:DX and gives it the
47082                        ; initial attributes contained in CX
47083
47084                        ; ENTRY (DS:DX) = ASCIZ path name
47085                        ; (CX) = initial attributes
47086                        ; 'C' set if error
47087                        ; (ax) = error code
47088                        ; 'c' clear if OK
47089                        ; (ax) = file handle
47090                        ; USES all
47091                        -----
47092
47093 _$CREAT:
47094 0000809C 51          push     cx          ; Save attributes on stack
47095 0000809D B9[C930]      mov      CX,DOS_CREATE      ; routine to call
47096
47097 AccessSet:
47098                        ;mov     byte [ss:SATTRIB],6
47099 000080A0 36C606[6D05]06    mov      byte [ss:SATTRIB],attr_hidden+attr_system ;smr;SS Override
47100                        ; 10/08/2018
47101 000080A6 E926FF      JMP      AccessFile        ; use good ol' open
47102
47103
47104                        ; MSDOS 6.0 (MSDOS 3.3)
47105                        ; BREAK <$CHMOD - change file attributes>
47106                        -----
47107
47108                        ;** $CHMOD - Change File Attributes

```

```

47109 ;
47110 ; Assembler usage:
47111 ;     LDS     DX, name
47112 ;     MOV     CX, attributes
47113 ;     MOV     AL,func (0=get, 1=set)
47114 ;     INT     21h
47115 ; Error returns:
47116 ;     AX = error_path_not_found
47117 ;     AX = error_access_denied
47118 ;
47119 ;-----
47120 ;
47121 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47122 ;
47123 ; 14/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
47124 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C27Ch
47125 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0AFF4h)
47126 ; (Windows ME IO.SYS - BIOSCODE:0AAB2h)
47127
47128 _$CHMOD:
47129 ; 14/03/2024 (PCDOS 7.1 IBMDOS.COM)
47130 ;;;
47131 000080A9 3CFF      cmp     al,0FFh      ; MS-DOS 7.20 (win98) - EXTENDED-LENGTH FILENAME OPERATIONS
47132 ;;;
47133 ; AX = 43FFh
47134 ; BP = 5053h ('PS')
47135 ; CL = function
47136 ; 39h "mkdir" create directory
47137 ; DS:DX -> ASCIZ pathname
47138 ; 56h rename file
47139 ; DS:DX -> ASCIZ filename of existing file (no wildcards)
47140 ; ES:DI -> ASCIZ new filename (no wildcards)
47141 ;;;
47142 ; ref: Ralf Brown's Interrupt List
47143 000080AB 7520      jnz     short std_chmod
47144 000080AD 81FD5350      cmp     bp,5053h
47145 000080B1 7515      jnz     short chmod_x_2
47146 000080B3 88CC      mov     ah,cl
47147 000080B5 36C606[5411]80      mov     word [ss:PATHNAMELEN],128
47148 000080BB 80F939      mov     byte [ss:PATHNAMELEN],128 ; Retro DOS v5.0
47149 000080BE 7503      cmp     cl,39h
47150 000080C0 E95EA7      jne     short chmod_x_1
47151 ; jmp     mkdir_x
47152 000080C3 80F956      cmp     cl,56h
47153 000080C6 7472      je      short rename_x
47154 000080C8 B001      chmod_x_2:
47155 ; mov     al,1
47156 ; jmp     short NORERR
47157 ; 15/03/2024
47158 NORERR:
47159 000080CA E9AB85      jmp     SYS_RET_ERR
47160
47161 std_chmod:
47162 ;;;
47163 ;
47164 ; 05/08/2018 - Retro DOS v3.0
47165 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 6FCCh
47166 000080CD BF[BE03]      MOV     DI,OPENBUF      ; appropriate buffer
47167 000080D0 50      push    ax
47168 000080D1 51      push    cx
47169 000080D2 89D6      MOV     SI,DX
47170 000080D4 E886FB      call    TransPathSet      ; get correct path
47171 000080D7 59      pop     cx
47172 000080D8 58      pop     ax
47173 000080D9 7255      JC      short ChModErr      ; and get function and attrs back
47174 000080DB 16      JC      short ChModErr      ; errors get mapped to path not found
47175 000080DC 1F      push    ss
47176 000080DD 803E[7A05]FF      pop     ds
47177 000080E2 754C      CMP     byte [CMETA],-1
47178 ; JNZ     short ChModErr
47179 ; mov     byte [SATTRIB],16h
47180 000080E4 C606[6D05]16      MOV     byte [SATTRIB],attr_hidden+attr_system+attr_directory
47181 000080E9 2C01      SUB     AL,1
47182 000080EB 7209      JB      short ChModGet      ; fast way to discriminate
47183 000080ED 7415      JZ      short ChModSet      ; 0 -> go get value
47184 ; 1 -> go set value
47185
47186 ; 14/04/2024
47187 %if 0
47188 ; mov     byte [EXTERR_LOCUS],1
47189 MOV     byte [EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
47190 %else
47191 ; 14/03/2024 (PCDOS 7.1 IBMDOS.COM)
47192 ;;;
47193 ; call     set_exerr_locus_unk ; Extended Error Locus
47194 ;;;
47195 %endif
47196 ; mov     al,1
47197 ; mov     al,error_invalid_function ; bad value
47198 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47199 chmod_errj:
47200 ; jmp     SYS_RET_ERR
47201 ; jmp     short ChModE
47202 000080F4 EBD4      jmp     short NORERR ; 06/12/2022
47203 ChModGet:
47204 ; call     GET_FILE_INFO      ; suck out the o1' info
47205 ; JC      short ChModE      ; error codes are already set for ret
47206 ; call     get_user_stack      ; point to user saved variables
47207 ; mov     [SI+4],ax
47208 MOV     [SI+user_env.user_CX],AX ; return the attributes
47209 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47210 OpenOkj2:
47211 ; 17/12/2022
47212 ; jmp     SYS_RET_OK
47213 ; jmp     short OpenOkj
47214 ; 25/06/2019
47215 jmp     SYS_RET_OK_c1c
47216 ChModSet:
47217 MOV     AX,CX
47218 call     SET_FILE_ATTRIBUTE      ; get attrs in position
47219 ; JC      short ChModE      ; go set
47220 ; JC      short ChModE      ; errors are set
47221 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47222 ; jmp     SYS_RET_OK
47223 OpenOkj3:
47224 ; jmp     short OpenOkj2
47225 ; 17/12/2022
47226 jmp     SYS_RET_OK
47227
47228 ; 17/12/2022
47229 %if 0
47230 ChModErr:
47231 NotFound: ; 17/12/2022
47232 ; mov     al,3
47233 ; mov     al,error_path_not_found
47234 ChModE:

```

```

47233 UnlinkE: ; 17/12/2022
47234 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47235 ;;;jmp SYS_RET_ERR
47236 ;jmp short chmod_errj
47237 ; 17/12/2022
47238 jmp short NORERR
47239 %endif
47240
47241 ; 22/05/2019 - Retro DOS v4.0
47242 ; DOSCODE:B039h (MSDOS 6.21, MSDOS.SYS)
47243
47244 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47245 ; DOSCODE:AFD6h (MSDOS 5.0, MSDOS.SYS)
47246
47247 ; BREAK <$UNLINK - delete a file entry>
47248 -----
47249 ;
47250 ** $UNLINK - Delete a File
47251 ;
47252 ;
47253 ; Assembler usage:
47254 ; LDS DX, name
47255 ; IF VIA SERVER DOS CALL
47256 ; MOV CX,SEARCH_ATTRIB
47257 ; MOV AH,Unlink
47258 ; INT 21h
47259 ;
47260 ; ENTRY (ds:dx) = path name
47261 ; (cx) = search_attribute, if via server_dos
47262 ; EXIT 'C' clear if no error
47263 ; 'C' set if error
47264 ; (ax) = error code
47265 ; = error_file_not_found
47266 ; = error_access_denied
47267 -----
47268 ;
47269 ;
47270 ; 15/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
47271 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C2E4h
47272
47273 _$UNLINK:
47274 push cx ; Save possible CX input parm
47275 MOV SI,DX ; Point at input string
47276 MOV DI,OPENBUF ; temp spot for path
47277 call TransPathSet ; go get normalized path
47278 pop cx
47279 JC short ChModErr ; badly formed path
47280 CMP byte [ss:CMETA],-1 ; meta chars? ;smr;SS override
47281 JNZ short NotFound
47282 push ss
47283 pop ds
47284 ;mov ch,6
47285 mov ch,attr_hidden+attr_system ; unlink appropriate files
47286 call SetAttrib
47287 call DOS_DELETE ; remove that file
47288 ;JC short UnlinkE ; error is there
47289 ; 17/12/2022
47290 jc short NORERR
47291
47292 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47293 Unlinkok:
47294 ;jmp SYS_RET_OK ; okay doksy
47295 jmp short OpenOkj3
47296
47297 ; 17/12/2022
47298 ChModErr: ; 17/12/2022
47299 NotFound:
47300 ;mov al,3
47301 MOV AL,error_path_not_found
47302 ChModE: ; 17/12/2022
47303 UnlinkE:
47304 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47305 ;;;jmp SYS_RET_ERR ; bye
47306 ;jmp short ChModE
47307 ; 17/12/2022
47308 jmp short NORERR
47309
47310 ;BREAK <$RENAME - move directory entries around>
47311 -----
47312 ;
47313 ; Assembler usage:
47314 ; LDS DX, source
47315 ; LES DI, dest
47316 ; IF VIA SERVER DOS CALL
47317 ; MOV CX,SEARCH_ATTRIB
47318 ; MOV AH,Rename
47319 ; INT 21h
47320 ;
47321 ; Error returns:
47322 ; AX = error_file_not_found
47323 ; = error_not_same_device
47324 ; = error_access_denied
47325 -----
47326 ;
47327 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47328 ; 15/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
47329
47330 _$RENAME:
47331 ; 15/03/2024 (PCDOS 7.1 IBMDOS.COM)
47332 ;;;
47333 ;mov word [ss:PATHNAMELEN],67
47334 ; 15/03/2024 - Retro DOS v5.0
47335 mov byte [ss:PATHNAMELEN],67 ; DIRSTRLEN = 67
47336 00008134 36C606[5411]43
47337 rename_x:
47338 ;;;
47339 ;
47340 ; MSDOS 3.3 (& MSDOS 6.0)
47341 push cx
47342 push ds
47343 push dx ; save source and possible CX arg
47344 PUSH ES
47345 POP DS ; move dest to source
47346 MOV SI,DI ; save for offsets
47347 MOV DI,RENBUF
47348 call TransPathSet ; munge the paths
47349 PUSH word [ss:WFP_START] ; get pointer ;smr;SS override
47350 POP word [ss:REN_WFP] ; stash it ;smr;SS override
47351 pop si
47352 pop ds
47353 pop cx ; get back source and possible CX arg
47354 epjc2:
47355 JC short ChModErr ; get old error
47356 CMP byte [ss:CMETA],-1 ;smr;SS override

```



```

47357 0000815C 75D2      JNZ     short NotFound
47358 0000815E 51        push    cx                ; Save possible CX arg
47359 0000815F BF[BE03]    MOV     DI,OPENBUF        ; appropriate buffer
47360 00008162 E8F8FA    call    TransPathSet      ; wham
47361 00008165 59        pop     cx
47362                ;JC     short epjc2
47363                ; 15/03/2024
47364 00008166 72C8      jc      short ChModErr
47365
47366 00008168 16        push    ss
47367 00008169 1F        pop     ds
47368 0000816A 803E[7A05]FF    CMP     byte [CMETA],-1
47369 0000816F 72BF      JB      short NotFound
47370
47371                ; MSDOS 6.0
47372                ;PUSH   WORD [THISCDS]                ;AN000;;MS.save thiscds
47373                ;PUSH   WORD [THISCDS+2]              ;AN000;;MS.
47374                ; 15/03/2024 (PCDOS 7.1 IBMDOS.COM)
47375                ;;;
47376 00008171 C43E[A205]    les     di,[THISCDS]
47377 00008175 57        push    di
47378 00008176 06        push    es
47379                ;;;
47380
47381 00008177 BF[BE03]    MOV     DI,OPENBUF        ;AN000;;MS.
47382 0000817A 16        PUSH    SS                ;AN000;;MS.
47383 0000817B 07        POP     ES                ;AN000;;MS.es:di-> source
47384 0000817C 30C0      XOR     AL,AL              ;AN000;;MS.scan all CDS
47385                ;AN000;;
47386 0000817E E856FA    call    GetCDSFromDrv     ;AN000;;MS.
47387 00008181 720F      JC      short dorn        ;AN000;;MS. end of CDS
47388 00008183 E80996    call    StrCmp            ;AN000;;MS. current dir ?
47389 00008186 7404      JZ      short rnerr       ;AN000;;MS. yes
47390 00008188 FEC0      INC     AL                ;AN000;;MS. next
47391 0000818A EBF2      JMP     short rnloop      ;AN000;;MS.
47392                ;AN000;;
47393                ;ADD     SP,4                ;AN000;;MS. pop thiscds
47394                ; 15/03/2024 (PCDOS 7.1 IBMDOS.COM)
47395 0000818C 58        pop     ax
47396 0000818D 58        pop     ax
47397
47398                ;error error_current_directory ;AN000;;MS.
47399 0000818E B010      mov     al,error_current_directory
47400                ;jmp     SYS_RET_ERR
47401                ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47402 00008190 EBA0      jmp     short Unlinke
47403
47404                ; 15/03/2024
47405                ;if 0
47406                ;AN000;;
47407                ;POP     WORD [SS:THISCDS+2]          ;AN000;;MS.;PBUGBUG;SS REQD??
47408                ;POP     WORD [SS:THISCDS]            ;AN000;;MS.;PBUGBUG;SS REQD??
47409                ;endif
47410                ;push    ss
47411                ;pop     ds
47412
47413                ; 15/03/2024
47414                ;if 1
47415                ;pop     word [THISCDS+2]
47416                ;pop     word [THISCDS]
47417                ;endif
47418                ; MSDOS 3.3 (& MSDOS 6.0)
47419                ;mov     ch,16h
47420 0000819C B516      mov     ch,attr_directory+attr_hidden+attr_system
47421                ; rename appropriate files
47422                ;call    SetAttrib
47423 000081A1 E8D2AB    call    DOS_RENAME        ; do the deed
47424 000081A4 728C      JC      short Unlinke    ; errors
47425
47426                ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47427                ;jmp     SYS_RET_OK
47428 000081A6 EB86      jmp     short Unlinkok
47429
47430                ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47431
47432                ; 14/07/2018 - Retro DOS v3.0
47433                ; MSDOS 3.3 (& MSDOS 6.0)
47434
47435                ;Break <$CreateNewFile - Create a new directory entry>
47436                ;-----
47437                ; CreateNew - Create a new directory entry. Return a file handle if there
47438                ; was no previous directory entry, and fail if a directory entry with
47439                ; the same name existed previously.
47440                ;
47441                ; Inputs: DS:DX point to an ASCIZ file name
47442                ; CX contains default file attributes
47443                ; Outputs: Carry Clear:
47444                ; AX has file handle opened for read/write
47445                ; Carry Set:
47446                ; AX has error code
47447                ; Registers modified: All
47448                ;-----
47449
47450                ;_$CreateNewFile:
47451 000081A8 51        push    cx                ; Save attributes on stack
47452 000081A9 B9[9031]    MOV     CX,DOS_Create_New ; routine to call
47453 000081AC E9F1FE    JMP     AccessSet         ; use good ol' open
47454
47455                ;** BinToAscii - convert a number to a string.
47456                ;-----
47457                ; BinToAscii converts a 16 bit number into a 4 ascii characters.
47458                ; This routine is used to generate temp file names so we don't spend
47459                ; the time and code needed for a true hex number, we just use
47460                ; A thorough 0.
47461                ;
47462                ; ENTRY (ax) = value
47463                ; (es:di) = destination
47464                ; EXIT (es:di) updated by 4
47465                ; USES cx, di, flags
47466                ;-----
47467
47468                ; MSDOS 3.3
47469                ;BinToAscii:
47470                ; mov     cx,4
47471                ;bta5:
47472                ; push    cx
47473                ; mov     cl,4
47474                ; rol     ax,cl
47475                ; push    ax
47476                ; and     al,0Fh
47477                ; add     al,'0'
47478                ; cmp     al,'9'
47479                ; jbe     short bta6
47480                ; add     al,7

```

```

47481 ;bta6:
47482 ; stosb
47483 ; pop ax
47484 ; pop cx
47485 ; loop bta5
47486 ; ret
47487
47488 ; 15/03/2024
47489 ; MSDOS 5.0-6.22 & windows ME
47490 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0B0D9h)
47491 ; (Windows ME IO.SYS - BIOSCODE:0ABA4h)
47492 %if 0
47493
47494 ; MSDOS 6.0
47495 BinToAscii:
47496 mov cx,404h ; (ch) = digit counter, (cl) = shift cnt
47497 bta5:
47498 rol ax,cl ; move leftmost nibble into rightmost
47499 push ax ; preserve remainder of digits
47500 and al,0Fh ; grab low nibble
47501 add al,'A' ; turn into ascii
47502 stosb ; drop in the character
47503 pop ax ; (ax) = shifted number
47504 dec ch
47505 jnz short bta5 ; process 4 digits
47506 ret
47507 %else
47508 ; 15/03/2024
47509 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0C385h
47510
47511 BinToAscii:
47512 push ax ; convert a number to a string ; ax = value
47513 xchg ah,al
47514 ;db 0D4h,10h
47515 aam 10h ; AH = AL / 16 and AL = remainder
47516 add ax,4141h ; 'AA'
47517 stosw
47518 pop ax
47519 ;db 0D4h,10h
47520 aam 10h
47521 add ax,4141h ; add ax,'AA'
47522 stosw
47523 ret
47524 %endif
47525
47526 ;Break <$CreateTempFile - create a unique name>
47527 -----
47528 ; $CreateTemp - given a directory, create a unique name in that directory.
47529 ; Method used is to get the current time, convert to a name and attempt
47530 ; a create new. Repeat until create new succeeds.
47531 ;
47532 ; Inputs: DS:DX point to a null terminated directory name.
47533 ; CX contains default attributes
47534 ; Outputs: Unique name is appended to DS:DX directory.
47535 ; AX has handle
47536 ; Registers modified: all
47537 -----
47538
47539 ; 10/07/2024
47540 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47541 ; 15/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
47542
47543 _$CreateTempFile:
47544 ;Enter
47545 push bp
47546 mov bp,sp
47547
47548 ;LocalVar EndPtr,DWORD
47549 ;LocalVar FilPtr,DWORD
47550 ;LocalVar Attr,WORD
47551
47552 sub sp,10
47553
47554 ;test cx,0FFD8h
47555 test CX,~attr_changeable
47556 jz short OKatts ; Ok if no non-changeable bits set
47557
47558 ; We need this "hook" here to detect these cases (like user sets one both of
47559 ; vol_id and dir bits) because of the structure of the or $CreateNewFile loop
47560 ; below. The code loops on error_access_denied, but if one of the non
47561 ; changeable attributes is specified, the loop COULD be infinite or WILL be
47562 ; infinite because CreateNewFile will fail with access_denied always. Thus we
47563 ; need to detect these cases before getting to the loop.
47564
47565 ;mov ax, 5
47566 MOV AX,error_access_denied
47567 JMP SHORT SETTMPERR
47568
47569 OKatts:
47570 ;MOV attr,CX ; save attribute
47571 mov [bp-10],cx
47572 ;MOV FilPtrL,DX ; pointer to file
47573 mov [bp-8],dx
47574 ;MOV FilPtrH,DS
47575 mov [bp-6],ds
47576 ;MOV EndPtrH,DS ; seg pointer to end of dir
47577 mov [bp-2],ds
47578 PUSH DS
47579 POP ES ; destination for nul search
47580 MOV DI,DX
47581 MOV CX,DI
47582 NEG CX ; number of bytes remaining in segment
47583 ; MSDOS 6.0
47584 OR CX,CX ;AN000;MS. cx=0 ? ds:dx on segment boundary
47585 JNZ short okok ;AN000;MS. no
47586 ;MOV CX,-1 ;AN000;MS.
47587 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47588 ; 17/12/2022
47589 dec cx ; mov cx,-1
47590 ;mov cx,-1 ; 0FFFh
47591 okok:
47592 XOR AX,AX ;AN000;
47593 REPZ SCASB ;AN000;
47594 ;AN000;
47595 DEC DI ; point back to the null
47596 MOV AL,[ES:DI-1] ; Get char before the NUL
47597 call PATHCHRCMP ; Is it a path separator?
47598 JZ short SETENDPTR ; Yes
47599 STOREPTH:
47600 MOV AL,'\'
47601 STOSB ; Add a path separator (and INC DI)
47602 SETENDPTR:
47603 ; 10/07/2024 - Retro DOS v5.0
47604 ;MOV EndPtrL,DI ; pointer to the tail

```



```

47605 ; 09/07/2024 (Retro DOS v4 BugFix - Erdogan Tan - Istanbul)
47606 ; (Note: I find this Retro DOS v4 Kernel bug while searching the reason
47607 ; of the AutoCAD R12 running/startup problem. Now it is solved here.)
47608 ;mov [bp-4],di ; (Retro DOS v4 !Bug!)
47609 000081FB 897EFC mov [bp-4],di ; !Fix!
47610 CreateLoop:
47611 000081FE 16 push ss ; let ReadTime see variables
47612 000081FF 1F pop ds
47613 00008200 55 push bp
47614 00008201 E88689 call READTIME ; go get time
47615 00008204 5D pop bp
47616 ;
47617 ; Time is in CX:DX. Go drop it into the string.
47618 ;
47619 ;les di,EndPtr ; point to the string
47620 00008205 C47EFC les di,[BP-4]
47621 00008208 89C8 mov ax,cx
47622 0000820A E8A2FF call BinToAscii ; store upper word
47623 0000820D 89D0 mov ax,dx
47624 0000820F E89DFF call BinToAscii ; store lower word
47625 00008212 30C0 xor al,al
47626 00008214 AA STOSB ; nul terminate
47627 ;LDS DX,FilePtr ; get name
47628 00008215 C556F8 lds dx,[bp-8]
47629 ;MOV CX,Attr ; get attr
47630 00008218 8B4EF6 mov cx,[bp-10]
47631 0000821B 55 push bp
47632 0000821C E889FF CALL _$CreateNewFile ; try to create a new file
47633 0000821F 5D pop bp
47634 00008220 732A JNC short CreateDone ; failed, go try again
47635 ;
47636 ; The operation failed and the error has been mapped in AX. Grab the extended
47637 ; error and figure out what to do.
47638 ;
47639 ;;; MSDOS 3.3 ; M049 - start
47640 ; mov ax,[ss:EXTERR] ;smr;SS Override
47641 ; cmp al,error_file_exists
47642 ; jz short CreateLoop ; file existed => try with new name
47643 ; cmp al,error_access_denied
47644 ; jz short CreateLoop ; access denied (attr mismatch)
47645 ;
47646 ; MSDOS 6.0
47647 ; cmp al,50h
47648 00008222 3C50 CMP AL,error_file_exists ; Q: did file already exist
47649 00008224 74D8 JZ short CreateLoop ; Y: try again
47650 ; cmp al,5
47651 00008226 3C05 CMP AL,error_access_denied; Q: was it access denied
47652 00008228 7521 JNZ short SETTMPERR ; N: Error out
47653 ; Y: Check to see if we got this due
47654 ; to the network drive. Note that
47655 ; the redir will set the exterr
47656 ; to error_cannot_make if this is
47657 ; so.
47658 ;
47659 ; 15/03/2024
47660 %if 0
47661 CMP byte [SS:EXTERR],error_net_access_denied ; M069
47662 ; See if it's REALLY an att mismatch
47663 je short SETTMPERR ; no, network error, stop
47664 ;M070
47665 ; If the user failed on an I24, we do not want to try again
47666 ;
47667 cmp byte [SS:EXTERR],error_FAIL_I24 ;User failed on I24? ;M070
47668 ;je short SETTMPERR ;yes, do not try again ;M070
47669 ;jmp short CreateLoop ;attr mismatch, try again ;M070
47670 ; 17/12/2022
47671 jne short CreateLoop ; 10/06/2019
47672 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47673 ;jz short SETTMPERR
47674 ;jmp short CreateLoop
47675 %else
47676 ; 15/03/2024 (PCDOS 7.1 IBMDOS.COM)
47677 ;;;
47678 0000822A 36A0[2403] mov al,[ss:EXTERR]
47679 0000822E 3C41 cmp al,41h ; error_net_access_denied
47680 00008230 7419 je short SETTMPERR
47681 00008232 3C53 cmp al,53h ; error_FAIL_I24
47682 00008234 7415 je short SETTMPERR
47683 00008236 3C50 cmp al,50h ; error_file_exists
47684 00008238 74C4 je short CreateLoop
47685 0000823A 36C43E[A205] les di,[ss:THISCDS]
47686 0000823F 83FFFF cmp di,0FFFFh ; -1
47687 00008242 74BA je short CreateLoop ; No CDS
47688 ;;;test word [es:di+43h],8000h
47689 ;test word [es:di+curdir.flags],curdir_isnet
47690 00008244 26F6454480 test byte [es:di+curdir.flags+1],(curdir_isnet>>8)
47691 00008249 75B3 jnz short CreateLoop
47692 ;;;
47693 %endif
47694 ;
47695 ;;; MOV AL,error_access_denied; Return this "extended" error
47696 ; M049 - end
47697 SETTMPERR:
47698 STC
47699 CreateDone:
47700 ;Leave
47701 0000824C 89EC mov sp,bp
47702 0000824E 5D pop bp
47703 0000824F 7203 JC short CreateFail
47704 00008251 E91A84 jmp SYS_RET_OK ; success!
47705 CreateFail:
47706 00008254 E92184 jmp SYS_RET_ERR
47707 ;
47708 ; SetAttrib will set the search attribute (SAttrib) either to the normal
47709 ; (CH) or to the value in CL if the current system call is through
47710 ; serverdoscall.
47711 ;
47712 ; Inputs: fSharing == FALSE => set sattrib to CH
47713 ; fSharing == TRUE => set sattrib to CL
47714 ; Outputs: none
47715 ; Registers changed: CX
47716 ;
47717 SetAttrib:
47718 ;test byte [SS:FSHARING],-1 ;smr;SS Override
47719 ;jnz short Set
47720 ; 15/03/2024
47721 00008257 36803E[7205]00 cmp byte [ss:FSHARING],0
47722 0000825D 7502 jnz short Set
47723 ;
47724 0000825F 88E9 mov cl,ch
47725 Set:
47726 00008261 36880E[6D05] mov byte [ss:SATTRIB],cl ;smr;SS Override
47727 00008266 C3 retn
47728 ;

```

```

47729 ;-----
47730 ; 16/03/2024 - Retro DOS v5.0
47731 ext_inval2:
47732 ;mov al,1
47733 00008267 B001 mov al,error_invalid_function
47734 eo_err:
47735 ;jmp SYS_RET_ERR
47736 00008269 EBE9 jmp short CreateFail
47737
47738 ; 14/07/2018 - Retro DOS v3.0
47739 ; MSDOS 6.0
47740
47741 ; 29/04/2019 - Retro DOS v4.0
47742
47743 ;Break <Extended_Open- Extended open the file>
47744 ;-----
47745 ; Input: AL= 0 reserved AH=6CH
47746 ; BX= mode
47747 ; CL= create attribute CH=search attribute (from server)
47748 ; DX= flag
47749 ; DS:SI = file name
47750 ; ES:DI = parm list
47751 ;
47752 ; DD SET EA list (-1) null
47753 ; DW n parameters
47754 ; DB type (TTTTTLL)
47755 ; DW IOMODE
47756 ; Function: Extended Open
47757 ; Output: carry clear
47758 ; AX= handle
47759 ; CX=1 file opened
47760 ; 2 file created/opened
47761 ; 3 file replaced/opened
47762 ; carry set: AX has error code
47763 ;-----
47764 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47765 ; 16/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
47766
47767 _$Extended_Open: ;AN000;
47768 ;ASSUME CS:DOSCODE,SS:DOSDATA ;AN000;
47769 0000826B 368916[F405] MOV [SS:EXTOPEN_FLAG],DX ;AN000;EO. save ext. open flag;smr;SS Override
47770 00008270 36C706[F705]0000 MOV word [SS:EXTOPEN_IO_MODE],0 ;AN000;EO. initialize IO mode;smr;SS Override
47771 ; 17/12/2022
47772 00008277 F6C6FE test dh,0FEh ; 04/12/2022
47773 ;;test dx,0FE00h
47774 ;;TEST DX,RESERVED_BITS_MASK ;AN000;EO. reserved bits 0 ?
47775 0000827A 75EB JNZ short ext_inval2 ;AN000;EO. no
47776 0000827C 88D4 MOV AH,DL ;AN000;EO. make sure flag is right
47777 0000827E 80FA00 CMP DL,0 ;AN000;EO. all fail ?
47778 00008281 74E4 JZ short ext_inval2 ;AN000;EO. yes, error
47779 ;and dl,0Fh
47780 00008283 80E20F AND DL,EXISTS_MASK ;AN000;EO. get exists action byte
47781 00008286 80FA02 CMP DL,2 ;AN000;EO. > 2
47782 00008289 77DC JA short ext_inval2 ;AN000;EO. yes, error
47783 ;and ah,0F0h
47784 0000828B 80E4F0 AND AH,NOT_EXISTS_MASK ;AN000;EO. get no exists action byte
47785 0000828E 80FC10 CMP AH,10H ;AN000;EO. > 10
47786 00008291 77D4 JA short ext_inval2 ;AN000;EO. yes, error
47787
47788 00008293 368C06[FB05] MOV [SS:SAVE_ES],ES ;AN000;EO. save API parms;smr;SS Override
47789 00008298 36893E[F905] MOV [SS:SAVE_DI],DI ;AN000;EO.;smr;SS Override
47790 0000829D 36FF36[F405] PUSH word [SS:EXTOPEN_FLAG] ;AN000;EO.;smr;SS Override
47791 000082A2 368F06[FD05] POP word [SS:SAVE_DX] ;AN000;EO.;smr;SS Override
47792 000082A7 36890E[FF05] MOV [SS:SAVE_CX],CX ;AN000;EO.;smr;SS Override
47793 000082AC 36891E[0106] MOV [SS:SAVE_BX],BX ;AN000;EO.;smr;SS Override
47794 000082B1 368C1E[0506] MOV [SS:SAVE_DS],DS ;AN000;EO.;smr;SS Override
47795 000082B6 368936[0306] MOV [SS:SAVE_SI],SI ;AN000;EO.;smr;SS Override
47796 000082BB 89F2 MOV DX,SI ;AN000;EO. ds:dx points to file name
47797 000082BD 89D8 MOV AX,BX ;AN000;EO. ax= mode
47798
47799 ; 16/03/2024
47800 %if 0
47801 JMP SHORT goopen2 ;AN000;;EO. do normal
47802 ext_inval2: ;AN000;;EO.
47803 ;mov al,1
47804 mov al,error_invalid_function ;AN000;EO.. invalid function
47805 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47806 eo_err:
47807 ;jmp SYS_RET_ERR
47808 jmp short CreateFail
47809 %endif
47810
47811 ; 16/03/2024
47812 %if 0
47813 ext_inval_parm: ;AN000;EO..
47814 POP CX ;AN000;EO.. pop up satck
47815 POP SI ;AN000;EO..
47816 ;error error_invalid_data ;AN000;EO.. invalid parms
47817 ;mov al,13
47818 mov al,error_invalid_data
47819 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47820 ;;jmp SYS_RET_ERR
47821 ;jmp short eo_err
47822 ; 17/12/2022
47823 jmp short CreateFail
47824 %endif
47825
47826 ; 17/12/2022
47827 ;error_return: ;AN000;EO.
47828 ; retn ;AN000;EO.. return with error
47829
47830 goopen2: ;AN000;
47831 ; 17/12/2022
47832 000082BF F6C720 test bh,20h
47833 test bh,INT_24_ERROR>>8 ; 04/12/2022
47834 ;;test bx,2000h
47835 000082C2 7406 TEST BX,INT_24_ERROR ;AN000;EO.. disable INT 24 error ?
47836 JZ short goopen ;AN000;EO.. no
47837 000082C4 36800E[F605]02 or byte [SS:EXTOPEN_ON],2
47838 OR byte [SS:EXTOPEN_ON],EXT_OPEN_I24_OFF ;AN000;EO.. set bit to disable;smr;SS Override
47839 goopen: ;AN000;
47840 ;or byte [SS:EXTOPEN_ON],1
47841 OR byte [SS:EXTOPEN_ON],EXT_OPEN_ON ;AN000;EO.. set Extended Open active;smr;SS Override
47842 ;AND word [SS:EXTOPEN_FLAG],0FFh ;AN000;EO.create new ?;smr;SS Override
47843 ; 18/12/2022
47844 000082D0 36C606[F505]00 mov byte [SS:EXTOPEN_FLAG+1],0 ; AND word [SS:EXTOPEN_FLAG],0FFh
47845 000082D6 36833E[F405]10 cmp word [SS:EXTOPEN_FLAG],10h
47846 000082DC 7516 CMP word [SS:EXTOPEN_FLAG],EXT_EXISTS_FAIL+EXT_NEXISTS_CREATE ;AN000;FT.;smr;SS Override
47847 000082DE E8C7FE JNZ short chknex ;AN000;;EO. no
47848 000082E1 723F call _$CreateNewFile ;AN000;;EO. yes
47849 JC short error_return ;AN000;;EO. error
47850 000082E3 36803E[F605]00 CMP byte [SS:EXTOPEN_ON],0 ;AN000;;EO. IFS does it;smr;SS Override
47851 000082E9 7438 JZ short ok_return2 ;AN000;;EO. yes
47852 ;mov word [SS:EXTOPEN_FLAG],2

```

```

47853 000082EB 36C706[F405]0200      MOV     word [SS:EXTOPEN_FLAG],ACTION_CREATED_OPENED ;AN000;EO. created/opened;smr;SS Override
47854 000082F2 EB7F                    JMP     short setXAttr ; 16/03/2024 ;AN000;;EO. set XAS
47855
47856      ; 17/12/2022
47857 ;ok_return2:
47858 ;      jmp     SYS_RET_OK                ;AN000;;EO.
47859
47860 chknxt:
47861      ; 17/12/2022
47862 000082F4 36F606[F405]01          test    byte [SS:EXTOPEN_FLAG],EXT_EXISTS_OPEN ; 1
47863      ;;test word [SS:EXTOPEN_FLAG],1
47864      ;;TEST word [SS:EXTOPEN_FLAG],EXT_EXISTS_OPEN ;AN000;;EO. exists open;smr;SS Override
47865 000082FA 752A                    JNZ     short exist_open                ;AN000;;EO. yes
47866 000082FC E89DFD                  call    _$CREAT                        ;AN000;;EO. must be replace open
47867 000082FF 7221                    JC      short error_return              ;AN000;;EO. return with error
47868 00008301 36803E[F605]00                CMP     byte [SS:EXTOPEN_ON],0          ;AN000;;EO. IFS does it;smr;SS Override
47869 00008307 741A                    JZ      short ok_return2                ;AN000;;EO. yes
47870 00008309 36C706[F405]0200                MOV     word [SS:EXTOPEN_FLAG],ACTION_CREATED_OPENED ;AN000;EO. presume create/open;smr;SS Override
47871 00008310 36F606[F605]04                TEST    byte [SS:EXTOPEN_ON],EXT_FILE_NOT_EXISTS ;AN000;;EO. file not exists ?;smr;SS Override
47872 00008316 755B                    JNZ     short setXAttr                  ;AN000;;EO. no
47873 00008318 36C706[F405]0300                MOV     word [SS:EXTOPEN_FLAG],ACTION_REPLACED_OPENED ;AN000;;EO. replaced/opened;smr;SS Override
47874 0000831F EB52                    JMP     SHORT setXAttr                  ;AN000;;EO. set XAS
47875
47876 00008321 F9                        STC                                     ; Set Carry again to flag error ;AN001;
47877 error_return:      ; 17/12/2022
47878 00008322 C3                        retn                                    ;AN000;;EO. return with error
47879
47880      ; 17/12/2022
47881 ok_return:
47882 ok_return2:
47883 00008323 E94883                    jmp     SYS_RET_OK
47884
47885 exist_open:
47886      ;test byte [SS:FSHARING],-1      ;AN000;;EO. server doscall?;smr;SS Override
47887      ;jz short noserver                ;AN000;;EO. no
47888      ; 16/03/2024
47889      ;;
47890 00008326 36803E[F7205]00                cmp     byte [ss:FSHARING],0          ; server doscall?
47891 0000832C 7402                    JZ      short noserver                ; no
47892      ;;
47893 0000832E 88E9                    MOV     CL,CH                        ;AN000;;EO. cl=search attribute
47894
47895 00008330 E893FC                    call    _$open2                      ;AN000;;EO. do open
47896 00008333 732F                    JNC     short ext_ok                  ;AN000;;EO.
47897 00008335 36803E[F605]00                CMP     byte [SS:EXTOPEN_ON],0          ;AN000;;EO. error and IFS call;smr;SS Override
47898 0000833B 74E4                    JZ      short error_return2            ;AN000;;EO. return with error
47899
47900 local_extopen:
47901      ;cmp ax,2
47902 0000833D 83F802                CMP     AX,error_file_not_found        ;AN000;;EO. file not found error
47903 00008340 75DF                    JNZ     short error_return2            ;AN000;;EO. no,
47904      ;;test word [SS:EXTOPEN_FLAG],10h
47905      ; 17/12/2022
47906 00008342 36F606[F405]10                test    byte [SS:EXTOPEN_FLAG],EXT_NEXISTS_CREATE ; 10h
47907      ;TEST word [SS:EXTOPEN_FLAG],EXT_NEXISTS_CREATE ;AN000;;EO. want to fail;smr;SS Override
47908      ;JNZ short do_creat                ;AN000;;EO. yes
47909      ;JMP short extexit                  ;AN000;;EO. yes
47910      ; 17/12/2022
47911 00008348 7446                    JZ      short extexit ; 10/06/2019
47912      ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47913      ;jnz short do_creat
47914      ;jmp short extexit
47915 do_creat:
47916 0000834A 368B0E[F605]                MOV     CX,[SS:SAVE_CX]                ;AN000;;EO. get ds:dx for file name;smr;SS Override
47917 0000834F 36C536[F0306]                LDS     SI,[SS:SAVE_SI]                ;AN000;;EO. cx = attribute;smr;SS Override
47918 00008354 89F2                    MOV     DX,SI                        ;AN000;;EO.
47919 00008356 E843FD                    call    _$CREAT                        ;AN000;;EO. do create
47920 00008359 7235                    JC      short extexit                  ;AN000;;EO. error
47921      ;mov word [SS:EXTOPEN_FLAG],2
47922 0000835B 36C706[F405]0200                MOV     word [SS:EXTOPEN_FLAG],ACTION_CREATED_OPENED
47923 00008362 EB0F                    JMP     SHORT setXAttr                  ;AN000;;EO. set XAS
47924
47925 ext_ok:
47926 00008364 36803E[F605]00                CMP     byte [SS:EXTOPEN_ON],0          ;AN000;;EO. IFS call ?;smr;SS Override
47927 0000836A 74B7                    JZ      short ok_return                ;AN000;;EO. yes
47928      ;mov word [SS:EXTOPEN_FLAG],1
47929 0000836C 36C706[F405]0100                MOV     word [SS:EXTOPEN_FLAG],ACTION_OPENED ;AN000;;EO. opened;smr;SS Override
47930
47931 setXAttr:
47932      ; 29/04/2019
47933 00008373 50                        push    ax
47934 00008374 E80081                    call    Get_User_Stack                ;AN000;;EO.
47935 00008377 36A1[F405]                MOV     AX,[SS:EXTOPEN_FLAG]          ;AN000;;EO.;smr;SS Override
47936      ;mov [si+4],ax
47937 0000837B 894404                MOV     [SI+user_env.user_CX],AX      ;AN000;;EO. set action code for cx
47938 0000837E 58                        pop     ax                            ;AN000;;EO.
47939 0000837F 8904                    mov     [si],ax                       ;AN000;;EO.
47940      ;MOV [SI+user_env.user_AX],AX      ;AN000;;EO. set handle for ax
47941      ; 17/12/2022
47942 00008381 EBA0                    jmp     short ok_return
47943 ;ok_return:
47944      ;jmp     SYS_RET_OK                ;AN000;;EO.
47945
47946      ; 16/03/2024
47947 %if 0
47948 extexit2:
47949      ;BX
47950      ;AX
47951      ;cmp word [SS:EXTOPEN_FLAG],2
47952      ;CMP word [SS:EXTOPEN_FLAG],ACTION_CREATED_OPENED
47953      ;AN000;EO. from create;smr;SS Override
47954      ;JNZ short justopen                ;AN000;EO.
47955      ;LDS SI,[SS:SAVE_SI]                ;AN000;EO. cx = attribute;smr;SS Override
47956      ;LDS DX,[SI]                        ;AN000;EO.
47957      ;call _$UNLINK                      ;AN000;EO. delete the file
47958      ;JMP SHORT reserror                ;AN000;EO.
47959
47960 justopen:
47961      ;call _$CLOSE                      ;AN000;EO. pretend never happend
47962      ;reserror:
47963      ;POP AX                            ;AN000;EO. restore error code from set XA
47964      ;JMP SHORT extexit                  ;AN000;EO.
47965
47966 ext_file_unfound:
47967      ;AN000;
47968      ;mov ax,2
47969      ;MOV AX,error_file_not_found        ;AN000;EO.
47970      ;JMP SHORT extexit                  ;AN000;EO.
47971 ext_inval:
47972      ;AN000;
47973      ;mov ax,1
47974      ;MOV AX,error_invalid_function ;AN000;EO.
47975
47976 lockoperr: ; 17/12/2022
47977 extexit:
47978      ;jmp     SYS_RET_ERR                ;AN000;EO.

```

```

47977
47978
47979
47980
47981
47982
47983
47984
47985
47986
47987
47988
47989
47990
47991
47992
47993
47994
47995
47996
47997
47998
47999
48000
48001
48002
48003
48004
48005
48006
48007
48008
48009
48010
48011
48012
48013
48014
48015
48016
48017
48018
48019
48020
48021
48022
48023
48024
48025
48026
48027
48028
48029
48030
48031
48032 00008383 3C01
48033 00008385 770C
48034
48035 00008387 57
48036 00008388 E843F3
48037 0000838B 731B
48038 0000838D 5F
48039
48040 0000838E B006
48041
48042
48043
48044
48045
48046 00008390 E9E582
48047
48048
48049
48050
48051
48052
48053
48054
48055
48056
48057
48058
48059 00008393 E8478F
48060
48061
48062
48063 00008396 B001
48064
48065
48066
48067 00008398 EBF6
48068
48069
48070
48071
48072
48073
48074
48075
48076
48077
48078
48079
48080
48081
48082
48083
48084
48085
48086
48087
48088
48089
48090
48091
48092
48093
48094
48095
48096
48097
48098
48099
48100

%endif
;=====
; LOCK.ASM, MSDOS 6.0, 1991
;=====
; 14/07/2018 - Retro DOS v3.0
; 22/05/2019 - Retro DOS v4.0
;BREAK <$LockOper - Lock Calls>
;-----
;
; Assembler usage:
;     MOV     BX, Handle      (DOS 3.3)
;     MOV     CX, OffsetHigh
;     MOV     DX, OffsetLow
;     MOV     SI, LengthHigh
;     MOV     DI, LengthLow
;     MOV     AH, LockOper
;     MOV     AL, Request
;     INT     21h
;
; Error returns:
;     AX = error_invalid_handle
;         = error_invalid_function
;         = error_lock_violation
;
; Assembler usage:
;     MOV     AX, 5C??      (DOS 4.00)
;
;     0? lock all
;     8? lock write
;     ?2 lock multiple
;     ?3 unlock multiple
;     ?4 lock/read
;     ?5 write/unlock
;     ?6 add (lseek EOF/lock/write/unlock)
;
;     MOV     BX, Handle
;     MOV     CX, count or size
;     LDS     DX, buffer
;     INT     21h
;
; Error returns:
;     AX = error_invalid_handle
;         = error_invalid_function
;         = error_lock_violation
;-----
;
; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
;
; 17/03/2024
; 16/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
;
_$LockOper:
    CMP     AL,1
    JA      short lock_bad_func
;
    PUSH    DI
    call    SFFromHandle
; Save LengthLow
; ES:DI -> SFT
    JNC     short lock_do
; have valid handle
    POP     DI
; Clean stack
;mov     al,6
;mov     al,error_invalid_handle
;
; 16/03/2024
;
; extexit:
; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;
; lockoperr:
; jmp     SYS_RET_ERR
; 17/12/2022
; jmp     short lockoperr ; jmp SYS_RET_ERR
;
; lock_bad_func:
;
; 16/03/2024
;
; if 0
; mov     byte [ss:EXTERR_LOCUS],1
; MOV     byte [SS:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus;smr;SS Override
;
; else
; 16/03/2024 (PCDOS 7.1 IBMDOS.COM)
;
;
; call     set_exerr_locus_unk ; Extended Error Locus
;
; endif
;
; mov     al,1
; mov     al,error_invalid_function
; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;
; lockoperrj:
; jmp     SYS_RET_ERR
; jmp     short lockoperr
;
; 22/05/2019 - Retro DOS v4.0
;
; MSDOS 6.0
; Align_buffer call has been deleted, since it corrupts the DTA (6/5/88) P5013
; Dead code deleted, MD, 23 Mar 90
;
; lock_do:
;     ; MSDOS 3.3
;     or     al,al
;     pop     ax
;     jz     short DOS_Lock
;
; DOS_Unlock:
;     ; test word [es:di+5],8000h
;     test   word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
;     jz     short LOCAL_UNLOCK
;     push    ax
;     mov     ax,110Bh
;     int     2Fh ; Multiplex - NETWORK REDIRECTOR - UNLOCK REGION OF FILE
;               ; BX = file handle, CX:DX = starting offset, SI = high word of size
;               ; STACK: WORD low word of size, ES:DI -> SFT for file
;               ; SFT DPB field -> DPB of drive containing file
;               ; Return: CF set error
;     pop     bx
;     jmp     short Valchk
;
; LOCAL_UNLOCK:
;     call    far [ss:JShare+(7*4)] ; 7 = clr_block ;smr;SS Override
;
; Valchk:
;     JNC     short Lock_OK
;
; lockerror:
;     jmp     SYS_RET_ERR
;
; Lock_OK:

```

```

48101 ; ;MOV AX,[SS:Temp_VAR] ;AN000;;MS. AX= number of bytes ;smr;SS Override
48102 ; jmp SYS_RET_OK
48103 ;DOS_Lock:
48104 ; ;test word [es:di+5],8000h
48105 ; test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
48106 ; JZ short LOCAL_LOCK
48107 ; ;CallInstall NET_XLock,MultNET,10
48108 ; mov ax, 110Ah
48109 ; int 2Fh ; Multiplex - NETWORK REDIRECTOR - LOCK REGION OF FILE
48110 ; ; BX = file handle, CX:DX = starting offset, SI = high word of size
48111 ; ; STACK: WORD low word of size, ES:DI -> SFT
48112 ; ; SFT DPB field -> DPB of drive containing file, SS = DOS CS
48113 ; ; Return: CF set error
48114 ; JMP short ValChk
48115 ;
48116 ;LOCAL_LOCK:
48117 ; Call far [ss:JShare+(6*4)] ; 6 = Set_Block ;smr;SS Override
48118 ; JMP short ValChk
48119 ;
48120 ; 17/12/2022
48121 LOCAL_UNLOCK:
48122 ; MSDOS 3.3
48123 ; Call far [ss:JShare+(7*4)] ; 7 = clr_block ;smr;SS Override
48124 ; MSDOS 6.0
48125 0000839A FF1E[AC00] Call far [JShare+(7*4)] ; 7 = clr_block ;smr;SS Override
48126 ValChk:
48127 0000839E 7302 JNC short Lock_OK
48128 lockerror:
48129 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48130 ; jmp SYS_RET_ERR
48131 ; jmp short lockoperrj
48132 ; 17/12/2022
48133 000083A0 EBEE jmp short lockoperr ; jmp SYS_RET_ERR
48134 Lock_OK:
48135 ;MOV AX,[SS:TEMP_VAR] ;AN000;;MS. AX= number of bytes ;smr;SS Override
48136 ; 10/06/2019
48137 000083A2 A1[0C06] mov ax,[TEMP_VAR]
48138 000083A5 E9C682 jmp SYS_RET_OK
48139 ;
48140 ; 22/05/2019
48141 lock_do:
48142 ; MSDOS 6.0
48143 000083A8 89C3 MOV BX,AX ; save AX
48144 000083AA BD[A903] MOV BP,Lock_Buffer ; get DOS LOCK buffer
48145 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48146 ; ;mov [bp+0],dx
48147 ;MOV [BP+LockBuf.Lock_position],DX ; set low offset
48148 ; 15/12/2022
48149 000083AD 895600 mov [bp],dx
48150 ;mov [bp+2],cx
48151 000083B0 894E02 MOV [BP+LockBuf.Lock_position+2],CX; set high offset
48152 ;
48153 ; 16/03/2024
48154 ; POP CX ; get low length
48155 ; ;mov [bp+4],cx
48156 ;MOV [BP+LockBuf.Lock_length],CX ; set low length
48157 000083B3 8F4604 pop word [bp+LockBuf.Lock_length]
48158 ;
48159 ;mov [bp+6],si
48160 000083B6 897606 MOV [BP+LockBuf.Lock_length+2],SI ; set high length
48161 000083B9 B90100 MOV CX,1 ; one range
48162 ;
48163 ; PUSH CS ;
48164 ; POP DS ; DS:DX points to
48165 ;
48166 000083BC 16 push ss
48167 000083BD 1F pop ds
48168 ;
48169 000083BE 89EA MOV DX,BP ; Lock_Buffer
48170 ;test al,1
48171 000083C0 A801 TEST AL,UNLOCK_ALL ; function 1
48172 ;JNZ short DOS_Unlock ; yes
48173 ;JMP short DOS_Lock ; function 0
48174 ; 17/12/2022
48175 ; 10/06/2019
48176 000083C2 740E JZ short DOS_Lock
48177 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48178 ;JNZ short DOS_Unlock
48179 ;JMP short DOS_Lock
48180 ;
48181 DOS_Unlock:
48182 ; ;test word [es:di+5],8000h
48183 ;test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
48184 000083C4 26F6450680 test byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
48185 000083C9 74CF JZ short LOCAL_UNLOCK
48186 ;
48187 ; 17/03/2024
48188 ;lock_unlock: ; 22/05/2019
48189 ;
48190 ;CallInstall Net_Xlock,MultNET,10
48191 ;
48192 ;
48193 ; MSDOS 3.3
48194 ; mov ax,110Bh
48195 ; int 2Fh ; Multiplex - NETWORK REDIRECTOR - UNLOCK REGION OF FILE
48196 ; ; BX = file handle, CX:DX = starting offset, SI = high word of size
48197 ; ; STACK: WORD low word of size, ES:DI -> SFT for file
48198 ; ; SFT DPB field -> DPB of drive containing file
48199 ; ; Return: CF set error
48200 ; 17/03/2024 - Retro DOS v5.0
48201 lock_unlock:
48202 ;
48203 ; MSDOS 6.0
48204 000083CB B80A11 mov ax,110Ah
48205 000083CE CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - LOCK REGION OF FILE
48206 ; ; BX = file handle, CX:DX = starting offset, SI = high word of size
48207 ; ; STACK: WORD low word of size, ES:DI -> SFT
48208 ; ; SFT DPB field -> DPB of drive containing file, SS = DOS CS
48209 ; ; Return: CF set error
48210 ;
48211 000083D0 EBCC JMP SHORT ValChk
48212 ;
48213 ; 17/12/2022
48214 %if 0
48215 LOCAL_UNLOCK:
48216 ; MSDOS 3.3
48217 ; Call far [ss:JShare+(7*4)] ; 7 = clr_block ;smr;SS Override
48218 ; MSDOS 6.0
48219 Call far [JShare+(7*4)] ; 7 = clr_block ;smr;SS Override
48220 ValChk:
48221 JNC short Lock_OK
48222 lockerror:
48223 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48224 ; jmp SYS_RET_ERR

```

```

48225      jmp      short lockoperrj
48226 Lock_OK:
48227      ;MOV     AX,[SS:TEMP_VAR] ;AN000;;MS. AX= number of bytes ;smr;SS Override
48228      ; 10/06/2019
48229      mov      ax,[TEMP_VAR]
48230      jmp      SYS_RET_OK
48231 %endif
48232
48233 DOS_Lock:
48234      ;;test   word [es:di+5],8000h
48235      ;test    word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
48236 000083D2 26F6450680      test    byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
48237      ;JZ      short LOCAL_LOCK
48238      ; 17/03/2024
48239 000083D7 75F2      jnz     short lock_unlock
48240
48241      ; 17/03/2024
48242 %if 0
48243      ;CallInstall NET_XLock,MultNET,10
48244
48245      mov      ax,110Ah
48246      int      2Fh      ; Multiplex - NETWORK REDIRECTOR - LOCK REGION OF FILE
48247      ; BX = file handle, CX:DX = starting offset, SI = high word of size
48248      ; STACK: WORD low word of size, ES:DI -> SFT
48249      ; SFT DPB field -> DPB of drive containing file, SS = DOS CS
48250      ; Return: CF set error
48251
48252      JMP      short valchk
48253 %endif
48254
48255 LOCAL_LOCK:
48256      ; MSDOS 3.3
48257      ;Call    far [ss:JShare+(6*4)] ; 6 = Set_Block ;smr;SS Override
48258      ; MSDOS 6.0
48259 000083D9 FF1E[A800]      call    far [JShare+(6*4)] ; 6 = Set_Block ;smr;SS Override
48260
48261 000083DD EBBF      jmp     short valchk
48262
48263      ; 14/07/2018 - Retro DOS v3.0
48264      ; LOCK_CHECK
48265      ;MSDOS 6.0 (& MSDOS 3.3)
48266
48267      ;-----
48268      ; Inputs:
48269      ; Outputs of SETUP
48270      ; [USER_ID] Set
48271      ; [PROC_ID] Set
48272      ; Function:
48273      ; Check for lock violations on local I/O
48274      ; Retries are attempted with sleeps in between
48275      ; Outputs:
48276      ; Carry clear
48277      ; Operation is OK
48278      ; Carry set
48279      ; A lock violation detected
48280      ; Outputs of SETUP preserved
48281      ;-----
48282
48283      ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
48284      ; 22/05/2019 - Retro DOS v4.0
48285
48286 000083DF 8B1E[1A00]      LOCK_CHECK:
48287      MOV      BX,[RetryCount] ; Number retries
48288 LockRetry:
48289      push     bx ; save regs
48290      push     ax ; MSDOS 6.0
48291
48292      ;MSDOS 3.3
48293      ;Call    far [ss:JShare+(8*4)] ; 8 = chk_block
48294      ;MSDOS 6.0
48295      Call     far [JShare+(8*4)] ; 8 = chk_block
48296
48297      pop      ax ; MSDOS 6.0
48298      pop      bx ; restore regs
48299      jnc     short lc_ret_label ; There are no locks (retnc)
48300 LockN:
48301      call     idle ; wait a while
48302      DEC      BX ; remember a retry
48303      JNZ     short LockRetry ; more retries left...
48304      STC
48305 lc_ret_label:
48306      retn
48307
48308      ; 14/07/2018 - Retro DOS v3.0
48309      ; LOCK_VIOLATION
48310      ;MSDOS 6.0 (& MSDOS 3.3)
48311
48312      ;-----
48313      ; Inputs:
48314      ; [THISDPB] set
48315      ; [READOP] indicates whether error on read or write
48316      ; Function:
48317      ; Handle Lock violation on compatibility (FCB) mode SFTs
48318      ; Outputs:
48319      ; Carry set if user says FAIL, causes error_lock_violation
48320      ; Carry clear if user wants a retry
48321      ;
48322      ; DS, ES, DI, CX preserved, others destroyed
48323      ;-----
48324
48325      ; 19/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48326
48327 000083F5 1E      LOCK_VIOLATION:
48328 000083F6 06      PUSH     DS
48329 000083F7 57      PUSH     ES
48330 000083F8 51      PUSH     DI
48331      PUSH     CX
48332 000083F9 B82100      ;mov      ax,21h
48333      MOV      AX,error_lock_violation
48334      ;mov     byte [ALLOWED],18h
48335 00008401 C42E[8A05]      MOV      MOV      byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
48336 00008405 BF0100      LES      BP,[THISDPB]
48337 00008408 89F9      MOV      DI,1 ; Fake some registers
48338      MOV      CX,DI
48339
48340      ; 19/03/2024 (PCDOS 7.1 IBMDOS.COM)
48341      ;;;
48342 0000840A 31D2      xor      dx,dx ; 0
48343      ;cmp     [es:bp+0Fh],dx
48344 0000840C 2639560F      cmp     [es:bp+DPB.FAT_SIZE],dx ; 0 ?
48345      jnz     short lockv_1 ; not FAT32
48346      ; FAT32
48347      ;mov     dx,[es:bp+2Bh]
48348 00008412 268B562B      mov     dx,[es:bp+DPB.FCLUS_FSECTOR+2]
48349 00008416 8916[0706]      mov     [HIGH_SECTOR],dx

```



```

48349          ;mov     dx,[es:bp+29h]
48350 0000841A 268B5629      mov     dx,[es:bp+DPB.FCLUS_FSECTOR]
48351 0000841E EB08          jmp     short lockv_2
48352          lockv_1:
48353 00008420 8916[0706]      mov     [HIGH_SECTOR],dx ; 0
48354          ;;;
48355
48356          ;mov     dx,[es:bp+11]
48357 00008424 268B560B      MOV     DX,[ES:BP+DPB.FIRST_SECTOR]
48358          ; 19/03/2024
48359 00008428 E824DC          call    HARDERR
48360 0000842B 59            POP     CX
48361          sharev_3:      ; 01/07/2024
48362 0000842C 5F            POP     DI
48363 0000842D 07            POP     ES
48364 0000842E 1F            POP     DS
48365 0000842F 3C01          CMP     AL,1
48366 00008431 74C1          jz      short lc_ret_label ; 1 = retry, carry clear
48367 00008433 F9            STC
48368 00008434 C3            retn
48369
48370          ; 14/07/2018 - Retro DOS v3.0
48371
48372          ;-----
48373
48374          ; do a retz to return error
48375
48376          ; 22/05/2019 - Retro DOS v4.0
48377 checkShare:
48378          ; MSDOS 3.3
48379          ;cmp     byte [cs:fShare],0
48380          ;retn
48381
48382          ; MSDOS 6.0
48383 00008435 1E            push    ds
48384          ;getdseg <ds>          ;smr;
48385 00008436 2E8E1E[0700]      mov     ds,[cs:DosDSeg]
48386 0000843B 803E[0303]00      cmp     byte [fShare],0
48387 00008440 1F            pop     ds
48388 00008441 C3            retn
48389          ;smr;
48390
48391          ;=====
48392          ; SHARE.ASM, MSDOS 6.0, 1991
48393          ;=====
48394          ; 14/07/2018 - Retro DOS v3.0
48395          ; 22/05/2019 - Retro DOS v4.0
48396          ; 19/03/2024 - Retro DOS v5.0
48397
48398          ; SHARE_CHECK
48399          ;-----
48400          Inputs:
48401          ; [THISSFT] Points to filled in local file/device SFT for new
48402          ; instance of file sf_mode ALWAYS has mode (even on FCB SFTs)
48403          ; [WFP_START] has full path of name
48404          ; [USER_ID] Set
48405          ; [PROC_ID] Set
48406          Function:
48407          ; Check for sharing violations on local file/device access
48408          Outputs:
48409          ; Carry clear
48410          ; Sharing approved
48411          ; Carry set
48412          ; A sharing violation detected
48413          ; AX is error code
48414          ; USES ALL but DS
48415          ;-----
48416
48417          ; 22/05/2019 - Retro DOS v4.0
48418          SHARE_CHECK:
48419 00008442 FF1E[9400]      call    far [JShare+(1*4)] ; 1 = MFT_Enter
48420          shchk_retn:
48421 00008446 C3            retn
48422
48423          ; SHARE_VIOLATION
48424          ;-----
48425          Inputs:
48426          ; [THISDPB] Set
48427          ; AX has error code
48428          Function:
48429          ; Handle sharing errors
48430          Outputs:
48431          ; Carry set if user says FAIL, causes error_sharing_violation
48432          ; Carry clear if user wants a retry
48433
48434          ; DS, ES, DI preserved, others destroyed
48435          ;-----
48436
48437          ; 19/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48438
48439          SHARE_VIOLATION:
48440          PUSH     DS
48441          PUSH     ES
48442          PUSH     DI
48443          ; 19/03/2024
48444 0000844A 31D2          xor     dx,dx ; Retro DOS v5.0
48445          ;
48446          ;MOV     byte [READOP],0 ; All share errors are reading
48447 0000844C 8816[7505]      mov     [READOP],dl ; 0
48448          ;
48449          ;mov     byte [ALLOWED],18h
48450 00008450 C606[4B03]18      MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
48451 00008455 C42E[8A05]      LES     BP,[THISDPB]
48452 00008459 BF0100          MOV     DI,1 ; Fake some registers
48453 0000845C 89F9          MOV     CX,DI
48454
48455          ; 19/03/2024
48456          ;mov     dx,[es:bp+17]
48457          ;MOV     DX,[ES:BP+DPB.DIR_SECTOR]
48458
48459          ; 19/03/2024 - Retro DOS v5.0
48460          ; PCDOS 7.1 IBMDOS.COM
48461          ;;;
48462          ;cmp     word [es:bp+0Fh],0
48463 0000845E 2639560F      cmp     word [es:bp+DPB.FAT_SIZE],dx ; 0
48464 00008462 740A          jz      short sharev_1 ; FAT32
48465          ; FAT16 or FAT12
48466          ;mov     word [HIGH_SECTOR],0
48467 00008464 8916[0706]      mov     [HIGH_SECTOR],dx ; 0
48468          ;mov     dx,[es:bp+11h]
48469 00008468 268B5611      mov     dx,[es:bp+DPB.DIR_SECTOR]
48470 0000846C EB0C          jmp     short sharev_2
48471          sharev_1:
48472          ;mov     dx,[es:bp+2Bh]

```

```

48473 0000846E 268B562B      mov     dx,[es:bp+DPB.FCLUS_FSECTOR+2]
48474 00008472 8916[0706]      mov     [HIGH_SECTOR],dx
48475                      ;mov     dx,[es:bp+29h]
48476 00008476 268B5629      mov     dx,[es:bp+DPB.FCLUS_FSECTOR]
48477 sharev_2:
48478                      ;;;
48479 0000847A E8D2DB      call    HARDERR
48480                      ; 01/07/2024
48481 %if 0
48482                      POP     DI
48483                      POP     ES
48484                      POP     DS
48485                      CMP     AL,1
48486                      jz      short shchk_retn      ; 1 = retry, carry clear
48487                      STC
48488                      retn
48489 %else
48490 0000847D EBAD      jmp     short sharev_3
48491 %endif
48492
48493 ;-----
48494 ;   ShareEnd - terminate sharing info on a particular SFT/UID/PID. This does
48495 ;   NOT perform a close, it merely asserts that the sharing information
48496 ;   for the SFT/UID/PID may be safely released.
48497 ;
48498 ;   Inputs:      ES:DI points to an SFT
48499 ;   Outputs:     None
48500 ;   Registers modified: all except DS,ES,DI
48501 ;-----
48502
48503 ShareEnd:
48504 ; 26/07/2019
48505 0000847F FF1E[9800]  call    far [Jshare+(2*4)]      ; 2 = MFTClose
48506 00008483 C3      retn
48507
48508 ;Break <ShareEnter - attempt to enter a node into the sharing set>
48509 ;-----
48510 ;   ShareEnter - perform a retried entry of a node into the sharing set. If
48511 ;   the max number of retries is exceeded, we notify the user via int 24.
48512 ;
48513 ;   Inputs:      ThisSFT points to the SFT
48514 ;               WFP_Start points to the WFP
48515 ;   Outputs:     Carry clear => successful entry
48516 ;               Carry set => failed system call
48517 ;   Registers modified: all
48518 ;-----
48519
48520 ShareEnter:
48521 00008484 51      push    cx
48522 retry:
48523 00008485 8B0E[1A00]  mov     cx,[RetryCount]
48524 attempt:
48525 00008489 C43E[9E05]  les     di,[THISFT]      ; grab sft
48526 0000848D 31C0      xor     ax,ax
48527                      ;mov     [es:di+51],ax
48528 0000848F 26894533  mov     [ES:DI+SF_ENTRY.sf_MFT],AX ; indicate free SFT
48529 00008493 51      push    cx
48530 00008494 E8ABFF      call    SHARE_CHECK      ; attempt to enter into the sharing set
48531 00008497 59      pop     cx
48532 00008498 730A      jnc     short done      ; success, let the user see this
48533 0000849A E84793      call    Idle            ; wait a while
48534 0000849D E2EA      loop   attempt         ; go back for another attempt
48535 0000849F E8A5FF      call    SHARE_VIOLATION ; signal the problem to the user
48536 000084A2 73E1      jnc     short retry     ; user said to retry, go do it
48537 done:
48538 000084A4 59      pop     cx
48539 000084A5 C3      retn
48540
48541 ;=====
48542 ; EXEPATCH.ASM (MSDOS 6.0, 1991)
48543 ;=====
48544 ; 29/04/2019 - Retro DOS 4.0
48545 ; 20/03/2024 - Retro DOS 5.0
48546
48547 ;** EXEPATCH.ASM
48548 ;-----
48549 ;   Contains the foll:
48550 ;
48551 ;       - code to find and overlay buggy unpack code
48552 ;       - new code to be overlayed on buggy unpack code
48553 ;       - old code sequence to identify buggy unpack code
48554 ;
48555 ;   Revision history:
48556 ;
48557 ;       Created: 5/14/90
48558 ;-----
48559
48560 ;-----
48561 ;
48562 ; M020 : Fix for rational bug - for details see routine header
48563 ; M028 : 4b04 implementation
48564 ; M030 : Fixing bug in EXEPACKPATCH (EXEC_CS is an un-relocated value)
48565 ; M032 : set turnoff bit only if DOS in HMA.
48566 ; M033 : if IP < 2 then not exepacked.
48567 ; M046 : support for a 4th version of exepacked files.
48568 ; M068 : support for copy protected apps.
48569 ; M071 : use A200FF_COUNT of 10.
48570 ;
48571 ;-----
48572
48573 PATCH1_COM_OFFSET EQU 06CH
48574 PATCH1_OFFSET EQU 028H
48575 PATCH1_CHKSUM EQU 0EF4EH
48576 CHKSUM1_LEN EQU 11CH/2 ; 142
48577
48578 PATCH2_COM_OFFSET EQU 076H
48579 PATCH2_OFFSET EQU 032H
48580
48581 ; The strings that start at offset 076h have two possible
48582 ; check sums that are defined as PATCH2_CHKSUM PATCH2A_CHKSUM
48583
48584 PATCH2_CHKSUM EQU 78B2H
48585 CHKSUM2_LEN EQU 119H/2
48586 PATCH2A_CHKSUM EQU 1C47H ; M046
48587 CHKSUM2A_LEN EQU 103H/2 ; M046
48588
48589 PATCH3_COM_OFFSET EQU 074H
48590 PATCH3_OFFSET EQU 032H
48591 PATCH3_CHKSUM EQU 4EDEH
48592 CHKSUM3_LEN EQU 117H/2
48593
48594 ;** Data structure passed for ExecReady call
48595 ;
48596 ;struc ERStruc

```



```

48597 ; .ER_Reserved:      resw    1      ; reserved, should be zero
48598 ; .ER_Flags:         resw    1
48599 ; .ER_ProgName:      resd    1      ; ptr to ASCIIIZ str of prog name
48600 ; .ER_PSP:           resw    1      ; PSP of the program
48601 ; .ER_StartAddr:     resd    1      ; Start CS:IP of the program
48602 ; .ER_ProgSize:      resd    1      ; Program size including PSP
48603 ; .Size:
48604 ;endstruc
48605 ;DOSCODE SEGMENT
48606
48607 ; 22/05/2019 - Retro DOS v4.0
48608 ; DOSCODE:B3DDh (MSDOS 6.21, MSDOS.SYS)
48609
48610 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
48611 ; DOSCODE:B37Ah (MSDOS 5.0, MSDOS.SYS)
48612
48613 ; 20/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48614 ; DOSCODE:C697h (PCDOS 7.1, IBMDOS.COM)
48615
48616 ; M028 - BEGIN
48617
48618 ;-----
48619 ;
48620 ; Procedure Name      : ExecReady
48621 ;
48622 ; Input               : DS:DX -> ERStruc (see exe.inc)
48623 ;-----
48624
48625 ExecReady:
48626     mov     si, dx                ; move the pointer into a friendly one
48627     ;;test  word [si+2],1
48628     ; 17/12/2022
48629     test    byte [si+ERStruc.ER_Flags], ER_EXE ; 1
48630     ;test   word [si+ERStruc.ER_Flags], ER_EXE ; COM or EXE ?
48631     jz      short er_setver      ; only setver for .COM files
48632
48633     ;mov     ax, [si+8]
48634     mov     ax, [si+ERStruc.ER_PSP]
48635     add     ax, 10h
48636     mov     es, ax
48637
48638     ;mov     cx, [si+10]
48639     mov     cx, [si+ERStruc.ER_StartAddr] ; M030
48640     ;mov     ax, [si+12] ; 11/04/2024
48641     mov     ax, [si+ERStruc.ER_StartAddr+2] ; M030
48642
48643     ;call    [ss:FixExePatch]
48644     call    word [ss:FixExePatch] ; 28/12/2022
48645
48646     ; 20/03/2024
48647     ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48648     call    word [ss:Rational386PatchPtr]
48649
48650 er_setver:
48651     ;;test  word [si+2],2        ; Q: is this an overlay
48652     ; 17/12/2022
48653     test    byte [si+ERStruc.ER_Flags], ER_OVERLAY ; 2
48654     ;test   word [si+ERStruc.ER_Flags], ER_OVERLAY
48655     jnz     short er_chkdoshi    ; Y: set A20OFF_COUNT if DOS high
48656     ; N: set up lie version first
48657
48658     push    ds
48659     push    si
48660     ;lds     si, [si+4]
48661     lds     si, [si+ERStruc.ER_ProgName]
48662     call    Scan_Execname1
48663     call    Scan_Special_Entries
48664     pop     si
48665     pop     ds
48666     ;mov     es, [si+8]
48667     mov     es, [si+ERStruc.ER_PSP]
48668     mov     ax, [ss:SPECIAL_VERSION]
48669     ;mov     [es:40h], ax ; 11/04/2024
48670     mov     [es:PDB.Version], ax
48671
48672 er_chkdoshi:
48673     cmp     byte [ss:DosHashMA], 0 ; M032: Q: is dos in HMA (M021)
48674     je      short er_done        ; M032: N: done
48675
48676     ; M068 - Start
48677
48678     ;mov     ax, [si+8]
48679     mov     ax, [si+ERStruc.ER_PSP]; ax = PSP
48680
48681     ;or      byte [ss:DOS_FLAG], 4
48682     or      byte [ss:DOS_FLAG], EXECA200FF ; Set bit to signal int 21
48683     ; ah = 25 & ah= 49. See dossym.inc
48684     ; under TAG M003 & M009 for
48685     ; explanation
48686
48687     ;;test  word [si+2],1
48688     ; 17/12/2022
48689     test    byte [si+ERStruc.ER_Flags], ER_EXE ; 1
48690     ;test   word [si+ERStruc.ER_Flags], ER_EXE ; Q: COM file
48691     jnz     short er_setA20      ; N: inc a20off_count, set
48692     ; a20off_psp and ret
48693
48694     push    ds
48695     mov     ds, ax                ; DS = load segment of com file.
48696     call    IsCopyProt           ; check if copy protected
48697     pop     ds
48698
48699 er_setA20:
48700     ; We need to inc the A20OFF_COUNT here. Note that if the count
48701     ; is non-zero at this point it indicates that the A20 is to be
48702     ; turned off for that many int 21 calls made by the app. In
48703     ; addition the A20 has to be turned off when we exit from this
48704     ; call. Hence the inc.
48705
48706     inc     byte [ss:A20OFF_COUNT]
48707     mov     [ss:A20OFF_PSP], ax   ; set the PSP for which A20 is to be
48708     ; turned OFF.
48709     ; M068 - End
48710
48711 er_done:
48712     xor     ax, ax
48713     retn
48714
48715 ; M028 - END
48716
48717 ; 20/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48718 ; %if 1
48719 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
48720 ; %if 0
48721
48722 ;-----
48723 ;
48724 ; procedure : Rational386Patch

```

```

48721 ;
48722 ; Older versions of the Rational DOS Extender have several bugs which trash
48723 ; 386 registers (usually just the high word of 32 bit registers) during
48724 ; interrupt processing. Lotus 123 3.1+ is a popular application that uses a
48725 ; version of the Rational extender with the 32 bit register trashing bugs.
48726 ;
48727 ; This routine applies patches to the Rational DOS Extender to work around
48728 ; most of the register trashing bugs.
48729 ;
48730 ; Note that there are additional register trashing bugs not fixed by these
48731 ; patches. In particular, the high word of ESP and the FS and GS registers
48732 ; may be modified on interrupts.
48733 ;
48734 ; There are two different Rational DOS Extender patches in this module.
48735 ; Rational386Patch is to correct 386 register trashing bugs on 386 or later
48736 ; processors. This patch code is executed when MS-DOS is running on a 386
48737 ; or later processor, regardless of whether MS-DOS is running in the HMA
48738 ; or not.
48739 ;
48740 ; The other Rational patch routine (RationalPatch, below) fixes a register
48741 ; trashing problem on 286 processors, and is only executed if MS-DOS is
48742 ; running in the HMA.
48743 ;
48744 ; This patch detection and replacement is based on an example supplied by
48745 ; Ben Williams at Rational.
48746 ;
48747 ;-----
48748 ;
48749 ; 22/05/2019 - Retro DOS v4.0
48750 ; DOSCODE:B448h (MSDOS 6.21, MSDOS.SYS)
48751 ;-----
48752 ;
48753 ; INPUT : ES = segment where program got loaded
48754 ;
48755 ;-----
48756
48757 rpFind1:
48758     db      0FAh, 0E4h, 21h, 60h, 33h, 0C0h, 0E6h, 43h, 8Bh, 16h
48759 0000850E FAE4216033C0E6438B-
48759 00008517 16
48760
48761 rpFind1Len equ    $ - rpFind1
48762
48763 ;     cli
48764 ;     in     al, 21h
48765 ;     pusha
48766 ;     xor     ax, ax
48767 ;     out     43h, al
48768 ;     mov     dx, ...
48769
48770 rpFind1a:
48771     db      0B0h, 0Eh, 0E6h, 37h, 33h, 0C0h, 0E6h, 0F2h
48772
48773 rpFind1aLen equ    $ - rpFind1a
48774
48775 ;     mov     al, 0Eh
48776 ;     out     37h, al
48777 ;     xor     ax, ax
48778 ;     out     0F2h, al
48779
48780 ; bug # 1 -- loss of high EAX on 386+ if not VCPI or DPMI
48781
48782 rpFind2:
48783     db      0Fh, 20h, 0C0h
48784
48785 rpFind2Len equ    $ - rpFind2
48786
48787 ;     mov     eax, cr0           ;may be preceeded by PUSH CX (51h)
48788
48789 rpFind3:
48790     db      0Fh, 22h, 0C0h, 0EAh
48791 00008523 0F22C0EA
48792
48793 rpFind3Len equ    $ - rpFind3
48794
48795 ;     mov     cr0, eax           ;may be preceeded by POP CX (59h)
48796 ;     jmp     far ptr xxx        ;change far ptr to go to replace3
48797 ;     mov     ss, bx             ;8E D3 ... and come back at or after this
48798
48799 ; note, there is no rpRep11 string
48800
48801 rpRep12:
48802     db      66h, 50h, 51h, 0Fh, 20h, 0C0h
48803 00008527 6650510F20C0
48804
48805 rpRep12Len equ    $ - rpRep12
48806
48807 ;     push     eax
48808 ;     push     cx
48809 ;     mov     eax, cr0
48810
48811 rpRep13:
48812     db      8Eh, 0D3h, 59h, 66h, 58h
48813 0000852D 8ED3596658
48814
48815 rpRep13Len equ    $ - rpRep13
48816
48817 ;     mov     ss, bx
48818 ;     pop      cx
48819 ;     pop      eax
48820
48821 ; bug # 2 -- loss of high EAX and ESI on 386+ only if VCPI
48822
48823 rpFind4:
48824     db      93h, 58h, 8Bh, 0Cch
48825 00008532 93588BCC
48826
48827 rpFind4Len equ    $ - rpFind4
48828
48829 ;     xchg     bx, ax
48830 ;     pop      ax
48831 ;     mov     cx, sp
48832
48833 rpFind5:
48834     db      0B8h, 0Ch, 0DEh, 0CDh, 67h, 8Bh, 0E1h, 0FFh, 0E3h
48835 00008536 B80CDECD678BE1FFE3
48836
48837 rpFind5Len equ    $ - rpFind5
48838
48839 ;     mov     ax, DE0Ch
48840 ;     int      67h
48841 ;     mov     sp, cx
48842 ;     jmp     bx
48843
48844 rpRep14:
48845     db      93h, 58h, 8Bh, 0Cch
48846     db      02Eh, 066h, 0A3h
48847 0000853F 93588BCC
48848 00008543 2E66A3
48849
48850 rpRep14o1Len equ    $ - rpRep14

```

```

48844
48845 00008546 0000          db      00h, 00h
48846 00008548 2E668936      db      02Eh, 066h, 89h, 36h
48847
48848 rpRep14o2Len equ $ - rpRep14
48849
48850 0000854C 0000          db      00h, 00h
48851
48852 rpRep14Len equ      $ - rpRep14
48853
48854 ;   xchg    bx, ax
48855 ;   pop     ax
48856 ;   mov     cx, sp
48857 ;   mov     dword ptr cs:[xxxx], eax
48858 ;   mov     dword ptr cs:[xxxx], esi
48859
48860 rpRep15:
48861 0000854E 8BE1          db      8Bh, 0E1h
48862 00008550 2E66A1      db      2Eh, 66h, 0A1h
48863
48864 rpRep15o1Len equ $ - rpRep15
48865
48866 00008553 0000          db      00h, 00h
48867 00008555 2E668B36      db      2Eh, 66h, 8Bh, 36h
48868
48869 rpRep15o2Len equ $ - rpRep15
48870
48871 00008559 0000          db      00h, 00h
48872 0000855B FFE3          db      0FFh, 0E3h
48873
48874 rpRep15Len equ      $ - rpRep15
48875
48876 ;   mov     sp, cx
48877 ;   mov     eax, dword ptr cs:[xxxx]
48878 ;   mov     esi, dword ptr cs:[xxxx]
48879 ;   jmp     bx
48880
48881 ; bug # 3 -- loss of high EAX, EBX, ECX, EDX on 386+ only if VCPI
48882
48883 rpFind6:
48884 0000855D FA5251      db      0FAh, 52h, 51h
48885
48886 rpFind6Len equ      $ - rpFind6
48887
48888 ;   cli
48889 ;   push    dx
48890 ;   push    cx
48891
48892 rpFind7a:
48893 00008560 B80CDE6626FF1E db      0B8h, 0Ch, 0DEh, 66h, 26h, 0FFh, 1Eh
48894
48895 rpFind7aLen equ      $ - rpFind7a
48896
48897 ;   mov     ax, 0DE0Ch
48898 ;   call    fword ptr es:[xxxx]
48899
48900 rpFind7b:
48901 00008567 595A5B      db      59h, 5Ah, 5Bh
48902
48903 rpFind7bLen equ      $ - rpFind7b
48904
48905 ;   pop     cx
48906 ;   pop     dx
48907 ;   pop     bx
48908
48909 rpRep16:
48910 0000856A FA6650665366516652 db      0FAh, 66h, 50h, 66h, 53h, 66h, 51h, 66h, 52h
48911
48912 rpRep16Len equ      $ - rpRep16
48913
48914 ;   cli
48915 ;   push    eax
48916 ;   push    ebx
48917 ;   push    ecx
48918 ;   push    edx
48919
48920 rpRep17:
48921 00008573 665A6659665B66585B db      66h, 5Ah, 66h, 59h, 66h, 5Bh, 66h, 58h, 5Bh
48922
48923 rpRep17Len equ      $ - rpRep17
48924
48925 ;   pop     edx
48926 ;   pop     ecx
48927 ;   pop     ebx
48928 ;   pop     eax
48929 ;   pop     bx
48930
48931 ; bug # 4 -- loss of high EAX and EBX on 386+ only if VCPI
48932
48933 rpFind8:
48934 0000857C 60061EB800008ED8 db      60h, 06h, 1Eh, 0B8h, 00h, 00h, 8Eh, 0D8h
48935
48936 rpFind8Len equ      $ - rpFind8
48937
48938 ;   pusha
48939 ;   push    es
48940 ;   push    ds
48941 ;   mov     ax, dgroup      ;jump back to here from replace8
48942 ;   mov     ds, ax
48943
48944 rpFind9 :
48945 00008584 1F0761      db      1Fh, 07h, 61h
48946
48947 rpFind9Len equ      $ - rpFind9
48948
48949 ;   pop     ds
48950 ;   pop     es
48951 ;   popa
48952
48953 rpRep18:
48954 00008587 6660061E      db      66h, 60h, 06h, 1Eh
48955
48956 rpRep18Len equ      $ - rpRep18
48957
48958 ;   pushad
48959 ;   push    es
48960 ;   push    ds
48961
48962 rpRep19:
48963 0000858B 1F076661C3      db      1Fh, 07h, 66h, 61h, 0C3h
48964
48965 rpRep19Len equ      $ - rpRep19
48966
48967 ;   pop     ds

```

```

48968 ; pop es
48969 ; popad
48970 ; retn ;no need to jmp back to main-line
48971 ;-----
48972
48973
48974 struct SearchPair
48975 .sp_off1: resw 1 ; offset of 1st search string
48976 .sp_len1: resw 1 ; length of 1st search string
48977 .sp_off2: resw 1 ; 2nd string
48978 .sp_len2: resw 1 ; 2nd string
48979 .sp_diff: resw 1 ; max difference between offsets
48980 .size:
48981 endstruc
48982
48983 ;rpBug1Strs SearchPair <offset rpFind2, rpFind2Len, offset rpFind3, rpFind3Len, 20h>
48984
48985 rpBug1Strs:
48986 dw rpFind2
48987 dw rpFind2Len ; 3
48988 dw rpFind3
48989 dw rpFind3Len ; 4
48990 dw 20h
48991
48992 ;rpBug2Strs SearchPair <offset rpFind4, rpFind4Len, offset rpFind5, rpFind5Len, 80h>
48993
48994 rpBug2Strs:
48995 dw rpFind4
48996 dw rpFind4Len ; 4
48997 dw rpFind5
48998 dw rpFind5Len ; 9
48999 dw 80h
49000
49001 ;rpBug3Strs SearchPair <offset rpFind6, rpFind6Len, offset rpFind7a, rpFind7aLen, 80h>
49002
49003 rpBug3Strs:
49004 dw rpFind6
49005 dw rpFind6Len ; 3
49006 dw rpFind7a
49007 dw rpFind7aLen ; 7
49008 dw 80h
49009
49010 ;rpBug4Strs SearchPair <offset rpFind8, 4, offset rpFind9, rpFind9Len, 80h>
49011
49012 rpBug4Strs:
49013 dw rpFind8
49014 dw rpRep18Len ; 4 ; 20/03/2024
49015 dw rpFind9
49016 dw rpFind9Len ; 3
49017 dw 80h
49018
49019 ;-----
49020
49021 struct StackVars
49022 .sv_wVersion: resw 1 ; Rational extender version #
49023 .sv_cbCodeSeg: resw 1 ; code seg size to scan
49024 .sv_pPatch: resw 1 ; offset of next avail patch byte
49025 .size:
49026 endstruc
49027
49028 ;-----
49029
49030 ; 22/05/2019 - Retro DOS v4.0
49031
49032 Rational386Patch:
49033 ; Do a few quick checks to see if this looks like a Rational
49034 ; Extended application. Hopefully this will quickly weed out
49035 ; most non Rational apps.
49036
49037 cmp word [es:0],395 ; version number goes here - versions
49038 jae short rp3QuickOut ; 3.95+ don't need patching
49039
49040 cmp word [es:0Ch],20h ; always has this value here
49041 jne short rp3QuickOut
49042
49043 push ax ; *
49044
49045 mov ax,18h ; extender has 18h at
49046 cmp [es:24],ax ; offsets 24, 28, & 36
49047 jne short rp3Q0_ax
49048 cmp [es:28],ax
49049 jne short rp3Q0_ax
49050 cmp [es:36],ax
49051 je short rp3Maybe
49052
49053 rp3Q0_ax:
49054 pop ax
49055 rp3QuickOut:
49056 retn
49057
49058 ; It might be the rational extender, do more extensive checking
49059
49060 rp3Maybe:
49061 cld
49062
49063 ; 20/03/2024
49064 %if 0
49065 push bx ; note ax pushed above
49066 push cx
49067 push dx
49068 push si
49069 push di
49070 push es
49071 push ds ; we use all of them
49072 push bp
49073
49074 %else
49075 ; 20/03/2024 (PCDOS 7.1 IBMDOS.COM)
49076 ;;;
49077 pusha
49078 push es
49079 push ds
49080 ;;;
49081 %endif
49082 sub sp,StackVars.size ; 6; make space for stack variables
49083 mov bp,sp
49084
49085 push cs
49086 pop ds
49087
49088 mov ax,[es:0] ; save version #
49089 ;mov [bp+StackVars.sv_wVersion],ax
49090 mov [bp],ax
49091
49092 ; check that binary version # matches
49093 ; ascii string
49094 call VerifyVersion
49095 jne short rp3Exit_j

```

```

49092
49093 ; Looks like this is it, find where to put the patch code. The
49094 ; patch will be located on top of Rational code specific to 80286
49095 ; processors, so these patches MUST NOT be applied if running on
49096 ; an 80286 system.
49097
49098 ; Rational says the code to patch will never be beyond offset 46xxh
49099
49100 mov     cx,4500h          ; force search len to 4700h (searches
49101 ;mov     [bp+2],cx
49102 mov     [bp+StackVars.sv_cbCodeSeg],cx      ; start at offset 200h)
49103
49104 mov     es,[es:20h]      ; es=code segment
49105
49106 mov     si,rpFind1      ; string to find
49107 mov     dx,rpFind1Len ; 10 ; length to match
49108 call    ScanCodeSeq    ; look for code seq
49109 jz      short rpGotPatch
49110
49111 ; According to Rational, some very old versions of the extender may not
49112 ; have the find1 code sequence. If the find1 code wasn't found above,
49113 ; try an alternative patch area which is on top of NEC 98xx switching code.
49114
49115 mov     si,rpFind1a
49116 mov     dx,rpFind1aLen ; 8
49117 call    ScanCodeSeq
49118 jz      short rpGotPatch
49119
49120 rp3Exit_j:
49121 jmp     rp3Exit
49122
49123 ; Found the location to write patch code! DI = offset in code seg.
49124
49125 rpGotPatch:
49126 ;mov     [bp+4],di
49127 mov     [bp+StackVars.sv_pPatch],di ; save patch pointer
49128
49129 ;-----
49130 ; Bug # 1 -- loss of high EAX on 386+ if not VCPI or DPMI
49131
49132 ;cmp     word [bp+0],381
49133 ;cmp     word [bp+StackVars.sv_wVersion],381 ; only need bug 1 if version
49134 cmp     word [bp],381
49135 jae     short rpBug2          ; < 3.81
49136
49137 mov     bx,rpBug1Strs        ; locate find2 & find3 code
49138 call    FindBadCode
49139 jc      short rpBug2
49140
49141 ; si = rpFind2 offset, di = rpFind3 offset
49142
49143 push    di
49144 mov     di,si                ; rpFind2 offset
49145 mov     dx,rpFind2Len ; 3
49146
49147 cmp     byte [es:di-1],51h    ; find2 preceeded by push cx?
49148 jne     short rp_no_cx
49149
49150 dec     di                   ; yes, gobble up push cx too
49151 inc     dx
49152 rp_no_cx:
49153 mov     si,rpRep12          ; patch out find2 sequence
49154 mov     cx,rpRep12Len ; 6
49155 call    GenPatch
49156
49157 pop     di                   ; rpFind3 offset
49158 cmp     byte [es:di-1],59h    ; find3 preceeded by pop cx?
49159 jne     short rp_no_cx2
49160
49161 mov     byte [es:di-1],90h    ; yes, no-op it
49162 rp_no_cx2:
49163 ;mov     ax,[bp+4]
49164 mov     ax,[bp+StackVars.sv_pPatch] ; change offset of far jmp
49165 ;mov     [es:di+4],ax
49166 mov     [es:di+rpFind3Len],ax ; to go to patch code
49167
49168 push    di                   ; save find3 offset
49169 mov     si,rpRep13          ; copy repl3 to patch area
49170 mov     cx,rpRep13Len ; 5
49171 call    CopyPatch
49172
49173 pop     bx                   ; find3 offset
49174 add     bx,rpFind3Len+4      ; 8 ; skip over find3 and far jmp
49175 call    GenJump             ; jmp back from patch area
49176 ;mov     [bp+4],di
49177 mov     [bp+StackVars.sv_pPatch],di ; to main-line, update patch
49178 ; area pointer
49179
49180 ;-----
49181 ; Bug # 2 -- loss of high regs on 386+ under VCPI only
49182
49183 rpBug2:
49184 mov     bx,rpBug2Strs        ; locate find4 & find5 code
49185 call    FindBadCode
49186 jc      short rpBug3
49187
49188 ; si = rpFind4 offset, di = rpFind5 offset
49189
49190 ;push    word [bp+4]
49191 push    word [bp+StackVars.sv_pPatch] ; save current patch pointer
49192 ; (where repl4 goes)
49193 push    di                   ; save find5 offset
49194
49195 mov     di,si
49196 mov     dx,rpFind4Len ; 4
49197 mov     si,rpRep14
49198 mov     cx,rpRep14Len ; 15
49199 call    GenPatch             ; patch out find4 code
49200
49201 pop     di                   ; find5 offset
49202 add     di,5                 ; keep 5 bytes of find5 code
49203 ;mov     bx,[bp+4]
49204 mov     bx,[bp+StackVars.sv_pPatch] ; jump to patch area
49205 push    bx                   ; save repl5 location
49206 call    GenJump
49207
49208 mov     si,rpRep15          ; copy repl5 code to patch
49209 mov     cx,rpRep15Len ; 15 ; area -- it has a jmp bx
49210 call    CopyPatch           ; so no need to jmp back to
49211 ; main-line code
49212
49213 ; patches have been made, now update the patch code to store/load dwords just
49214 ; after the code in the patch area
49215

```

```

49216 0000869F 5F      pop     di                ; repl5 location
49217 000086A0 5E      pop     si                ; repl4 location
49218
49219      ;mov     ax,[bp+4]
49220 000086A1 8B4604      mov     ax,[bp+StackVars.sv_pPatch] ; (where dwords go)
49221
49222      ;mov     [es:si+7],ax
49223 000086A4 26894407      mov     [es:si+rpRep14o1Len],ax      ; offset for EAX
49224      ;mov     [es:di+5],ax
49225 000086A8 26894505      mov     [es:di+rpRep15o1Len],ax
49226 000086AC 83C004      add     ax,4
49227      ;mov     [es:si+0Dh],ax
49228 000086AF 2689440D      mov     [es:si+rpRep14o2Len],ax      ; offset for ESI
49229      ;mov     [es:di+0Bh],ax
49230 000086B3 2689450B      mov     [es:di+rpRep15o2Len],ax
49231
49232      ;add     word [bp+4],8
49233 000086B7 83460408      add     word [bp+StackVars.sv_pPatch],8 ; reserve space for 2 dwords in
49234      ; patch area
49235
49236      ;-----
49237      ; Bug # 3 -- loss of high regs on 386+ under VCPI only
49238
49239      rpBug3:
49240 000086BB BB[A485]      mov     bx,rpBug3Strs      ; locate find6 & find7a code
49241 000086BE E86400      call    FindBadCode
49242 000086C1 722C      jc      short rpBug4
49243
49244      ;add     di,9
49245 000086C3 83C709      add     di,rpFind7aLen + 2      ; skip over offset in find7a
49246 000086C6 56      push    si                ; code and locate find7b
49247 000086C7 BE[6785]      mov     si,rpFind7b      ; sequence
49248 000086CA BA0300      mov     dx,rpFind7bLen ; 3
49249 000086CD E8AD00      call    ScanCodeSeq_di
49250 000086D0 5E      pop     si
49251 000086D1 751C      jnz     short rpBug4
49252
49253 000086D3 57      push    di                ; save find7b code offset
49254
49255 000086D4 89F7      mov     di,si
49256 000086D6 BA0300      mov     dx,rpFind6Len ; 3
49257 000086D9 BE[6A85]      mov     si,rpRep16
49258 000086DC B90900      mov     cx,rpRep16Len ; 9
49259 000086DF E86C00      call    GenPatch      ; patch out find6 code
49260
49261 000086E2 5F      pop     di
49262 000086E3 BA0300      mov     dx,rpFind7bLen ; 3
49263 000086E6 BE[7385]      mov     si,rpRep17
49264 000086E9 B90900      mov     cx,rpRep17Len ; 9
49265 000086EC E85F00      call    GenPatch      ; patch out find7b code
49266
49267      ;-----
49268      ; Bug # 4 -- loss of high regs on 386+ under VCPI only
49269
49270      rpBug4:
49271      ;cmp     word [bp+0],360
49272      ;cmp     word [bp+StackVars.sv_wVersion],360 ; only applies if
49273 000086EF 817E006801      cmp     word [bp],360
49274 000086F4 7627      jbe     short rp3Exit      ; version > 3.60 and < 3.95
49275
49276 000086F6 BB[AE85]      mov     bx,rpBug4Strs      ; locate find8 & find9 code
49277 000086F9 E82900      call    FindBadCode
49278 000086FC 721F      jc      short rp3Exit
49279
49280 000086FE 57      push    di                ; save find9 code offset
49281
49282 000086FF 89F7      mov     di,si
49283 00008701 BA0300      mov     dx,3
49284 00008704 BE[8785]      mov     si,rpRep18
49285 00008707 B90400      mov     cx,rpRep18Len ; 4
49286 0000870A E84100      call    GenPatch      ; patch out find8 code
49287
49288 0000870D 5F      pop     di                ; find9 offset
49289      ;mov     bx,[bp+4]
49290 0000870E 8B5E04      mov     bx,[bp+StackVars.sv_pPatch] ; patch find9 to jmp to
49291 00008711 E85A00      call    GenJump      ; patch area
49292
49293 00008714 BE[8885]      mov     si,rpRep19      ; copy replacement code to
49294 00008717 B90500      mov     cx,rpRep19Len ; 5      ; patch area--it does a RET
49295 0000871A E84500      call    CopyPatch      ; so no jmp back to main-line
49296
49297      rp3Exit:
49298 0000871D 83C406      add     sp,StackVars.size
49299
49300      ; 20/03/2024
49301      %if 0
49302      pop     bp
49303      pop     ds
49304      pop     es
49305      pop     di
49306      pop     si
49307      pop     dx
49308      pop     cx
49309      pop     bx
49310      %else
49311      ; 20/03/2024 (PCDOS 7.1 IBMDOS.COM)
49312      ;;;
49313 00008720 1F      pop     ds
49314 00008721 07      pop     es
49315 00008722 61      popa
49316      ;;;
49317      %endif
49318 00008723 58      pop     ax ; *
49319 00008724 C3      retn
49320
49321      ;-----
49322      ;
49323      ; FindBadCode
49324      ;
49325      ; Searches Rational code segment looking for a pair of find strings (all
49326      ; patches have at least two find strings).
49327      ;
49328      ; Entry:
49329      ; ES = code segment to search
49330      ; DS:BX = search pair structure for this search
49331      ; [bp].sv_cbCodeSeg = length of code seg to search
49332      ;
49333      ; Exit:
49334      ; CY flag clear if both strings found, and
49335      ; SI = offset in ES of 1st string
49336      ; DI = offset in ES of 2nd string
49337      ; CY set if either string not found, or strings too far apart
49338      ;
49339      ; Used:

```



```

49340 ; CX
49341 ;
49342 ;-----
49343 ;
49344 ;struc SearchPair
49345 ; .sp_off1: resw 1 ; offset of 1st search string
49346 ; .sp_len1: resw 1 ; length of 1st search string
49347 ; .sp_off2: resw 1 ; 2nd string
49348 ; .sp_len2: resw 1 ; 2nd string
49349 ; .sp_diff: resw 1 ; max difference between offsets
49350 ; .size:
49351 ;endstruc
49352
49353 FindBadCode:
49354 ;mov cx,[bp+2]
49355 00008725 8B4E02 mov cx,[bp+StackVars.sv_cbCodeSeg] ; search length
49356
49357 00008728 8B37 mov si,[bx] ; mov si,[bx+0]
49358 ;mov si,[bx+SearchPair.sp_off1] ; ds:si -> search string
49359
49360 ;mov dx,[bx+2]
49361 0000872A 8B5702 mov dx,[bx+SearchPair.sp_len1] ; dx = search len
49362 0000872D E84A00 call ScanCodeSeq
49363 00008730 751A jnz short fbc_error ; done if 1st not found
49364
49365 00008732 57 push di ; save 1st string offset
49366
49367 ;mov si,[bx+4]
49368 00008733 8B7704 mov si,[bx+SearchPair.sp_off2]
49369 ;mov dx,[bx+6]
49370 00008736 8B5706 mov dx,[bx+SearchPair.sp_len2]
49371 00008739 E84100 call ScanCodeSeq_di ; don't change flags after this!
49372
49373 0000873C 5E pop si ; restore 1st string offset
49374 0000873D 750D jnz short fbc_error
49375
49376 0000873F 89F8 mov ax,di ; sanity check that
49377 00008741 29F0 sub ax,si ; si < di && di - si <= allowed diff
49378 ;jc short fbc_error
49379 ; 04/02/2026
49380 00008743 7208 jc short fbc_err
49381 ;cmp ax,[bx+8]
49382 00008745 3B4708 cmp ax,[bx+SearchPair.sp_diff]
49383 00008748 7702 ja short fbc_error
49384
49385 0000874A F8 cld
49386 0000874B C3 retn
49387
49388 fbc_error:
49389 0000874C F9 stc
49390 fbc_err:
49391 0000874D C3 retn
49392
49393 ;-----
49394 ;
49395 ; GenPatch
49396 ;
49397 ; Generate a patch sequence. 1) insert a jump at the buggy code location
49398 ; (jumps to the patch code area), 2) copy the selected patch code to the
49399 ; patch area, 3) insert a jump from the patch area back to the main-line
49400 ; code.
49401 ;
49402 ; Entry:
49403 ; ES:DI = start of buggy code to be patched
49404 ; DX = length of buggy code to be patched
49405 ; DS:SI = replacement patch code
49406 ; CX = length of replacement patch code
49407 ; [bp].sv_pPatch = offset in ES of where to copy patch code
49408 ;
49409 ; Exit:
49410 ; DI, [bp].sv_pPatch = byte after generated patch code
49411 ;
49412 ; Used:
49413 ; AX, BX, SI, Flags
49414 ;-----
49415 ;
49416 ;
49417 GenPatch:
49418 0000874E 57 push di ;save offset of buggy code
49419
49420 ;mov bx,[bp+4]
49421 0000874F 8B5E04 mov bx,[bp+StackVars.sv_pPatch]
49422 ;jump from buggy code to patch area
49423 00008752 E81900 call GenJump
49424
49425 00008755 E80A00 call CopyPatch ;copy replacement code to patch area
49426
49427 00008758 5B pop bx ;offset of buggy code + buggy code
49428 00008759 01D3 add bx,dx ; length = return from patch offset
49429
49430 0000875B E81000 call GenJump ;jump from patch area back to main-
49431 ;mov [bp+4],di
49432 0000875E 897E04 mov [bp+StackVars.sv_pPatch],di
49433 ; line code, update patch pointer
49434 00008761 C3 retn
49435
49436 ;-----
49437 ;
49438 ; CopyPatch
49439 ;
49440 ; Copies patch code to patch location.
49441 ;
49442 ; Entry:
49443 ; DS:SI = patch code to be copied
49444 ; ES = segment of code to patch
49445 ; CX = length of code to copy
49446 ; [bp].sv_pPatch = offset in ES of where to copy patch code
49447 ;
49448 ; Exit:
49449 ; DI, [bp].sv_pPatch = byte after copied patch code
49450 ;
49451 ; Used:
49452 ; SI, Flags
49453 ;-----
49454 ;
49455 ;
49456 CopyPatch:
49457 00008762 51 push cx
49458 ;mov di,[bp+4]
49459 00008763 8B7E04 mov di,[bp+StackVars.sv_pPatch] ;patch pointer is the dest offset
49460 00008766 FC cld
49461 00008767 F3A4 rep movsb
49462 00008769 59 pop cx
49463 ;mov [bp+4],di

```

```

49464 0000876A 897E04      mov     [bp+StackVars.sv_pPatch],di ;update net pointer location
49465 0000876D C3          retn
49466
49467
49468
49469
49470
49471
49472
49473
49474
49475
49476
49477
49478
49479
49480
49481
49482
49483
49484
49485
49486 0000876E B0E9      GenJump:
49487 00008770 AA          mov     al,0E9h          ; jmp rel16 opcode
49488
49489 00008771 89D8          stosb
49490 00008773 29F8          mov     ax,bx          ; calc offset to 'to' location
49491 00008775 83E802      sub     ax,di
49492
49493 00008778 AB          sub     ax,2
49494
49495 00008779 C3          stosw          ; output offset
49496
49497
49498
49499
49500
49501
49502
49503
49504
49505
49506
49507
49508 0000877A BF0002      ScanCodeSeq:
49509
49510 0000877D 51          mov     di,200h
49511 0000877E 29D1      ScanCodeSeq_di:
49512 00008780 41          push    cx
49513
49514 00008781 56          sub     cx,dx
49515 00008782 57          inc     cx
49516 00008783 51          scsagain:
49517 00008784 89D1      push    si
49518 00008786 F3A6      push    di
49519 00008788 59          push    cx
49520 00008789 5F          mov     cx,dx
49521 0000878A 5E          rep     cmpsb
49522 0000878B 7403      pop     cx
49523 0000878D 47          pop     di
49524 0000878E E2F1      pop     si
49525
49526 00008790 59          je      short scsfound
49527
49528 00008791 C3          inc     di
49529
49530
49531
49532
49533
49534
49535
49536
49537
49538
49539
49540
49541
49542
49543 00008792 268B362A00      loop    scsagain
49544
49545 00008797 B30A      scsfound:
49546 00008799 83C603      pop     cx
49547
49548 0000879C E80E00      vvexit:
49549 0000879F 75F0          pop     cx
49550 000087A1 E80900      ; 18/12/2022
49551 000087A4 75EB          ;
49552 000087A6 26803C2E      ;
49553 000087AA 75E5          ;
49554 000087AC 4E          ;
49555
49556
49557
49558
49559
49560
49561 000087AD F6F3          ;
49562 000087AF 80C430      ;
49563 000087B2 4E          ;
49564 000087B3 26386401      ;
49565 000087B7 B400          ;
49566 000087B9 C3          ;
49567
49568
49569
49570
49571
49572
49573
49574
49575
49576
49577
49578
49579
49580
49581
49582
49583
49584 000087BA 06          ;
49585 000087BB 8CD8          ;
49586
49587

```



```

49588
49589 000087BD 2BC2          db 2Bh, 0C2h          ;sub ax,dx
49590
49591 first: ; label byte
49592
49593          db 8Eh,0D8h          ;mov ds,ax
49594          db 8Eh,0C0h          ;mov es,ax
49595          db 0BFh,0Fh,00h      ;mov di,000FH
49596          db 57h              ;push di
49597          db 0B9h,10h,00h      ;mov cx,0010H
49598          db 0B0h,0FFh        ;mov al,0FFH
49599          db 0F3h,0AEh        ;repz scasb
49600          db 47h              ;inc di
49601          db 8Bh,0F7h        ;mov si,di
49602          db 5Fh              ;pop di
49603          db 58h              ;pop ax
49604
49605 second_stop equ $-first
49606
49607 000087D3 2BC2          db 2Bh,0C2h          ;sub ax,dx
49608
49609 second: ; label byte
49610
49611          db 8Eh,0C0h          ;mov es,ax
49612          ;NextRec:
49613          db 0B9h,04h,02h      ;mov cx,0204h
49614          ;norm_agn:
49615          db 8Bh,0C6h          ;mov ax,si
49616          db 0F7h,0D0h          ;not ax
49617          db 0D3h,0E8h          ;shr ax,cl
49618          db 74h,13h          ;jz short SI_ok
49619          db 8Ch,0DAh          ;mov dx,ds
49620          db 83h,0CEh,0F0h     ;or si,0FFF0H
49621          db 2Bh,0D0h          ;sub dx,ax
49622          db 73h,08h          ;jnc short SItoDS
49623          db 0F7h,0DAh          ;neg dx
49624          db 0D3h,0E2h          ;shl dx,cl
49625          db 2Bh,0F2h          ;sub si,dx
49626          db 33h,0D2h          ;xor dx,dx
49627          ;SItoDS:
49628          db 8Eh,0DAh          ;mov ds,dx
49629          ;SI_ok:
49630          db 87h,0F7h          ;xchg si,di
49631          db 1Eh              ;push ds
49632          db 06h              ;push es
49633          db 1Fh              ;pop ds
49634          db 07h              ;pop es
49635          db 0FEh,0CDh          ;dec ch
49636          db 75h,0DBh          ;jnz short norm_agn
49637          db 0ACh              ;lodsb
49638          db 92h              ;xchg dx,ax
49639          db 4Eh              ;dec si
49640          db 0ADh              ;lodsw
49641          db 8Bh,0C8h          ;mov cx,ax
49642          db 46h              ;inc si
49643          db 8Ah,0C2h          ;mov al,dl
49644          db 24h,0FEh          ;and al,0FEH
49645          db 3Ch,0B0h          ;cmp al,RPTREC
49646          db 75h,05h          ;jne short TryEnum
49647          db 0ACh              ;lodsb
49648          db 0F3h,0AAh          ;rep stosb
49649
49650 ; db 0EBh,07h,90h          ;jmp short TryNext
49651          db 0EBh,06h          ;jmp short TryNext
49652
49653          ;TryEnum:
49654          db 3Ch,0B2h          ;cmp al,ENMREC
49655          db 75h,6Ch          ;jne short CorruptExe
49656          db 0F3h,0A4h          ;rep movsb
49657          ;TryNext:
49658
49659          db 92h              ;xchg dx,ax
49660          ; db 8Ah,0C2h          ;mov al,dl
49661
49662          db 0A8h,01h          ;test al,1
49663          db 74h,0B9h          ;jz short NextRec
49664          db 90h,90h          ;nop,nop
49665
49666 last_stop equ $-second
49667 size_str1 equ $-str1
49668
49669 ; The following is the code that we need to look for in the exe
49670 ; file.
49671
49672 scan_patch1: ; label byte
49673
49674          db 8Ch,0C3h          ;mov bx,es
49675          db 8Ch,0D8h          ;mov ax,ds
49676          db 2Bh,0C2h          ;sub ax,dx
49677          db 8Eh,0D8h          ;mov ds,ax
49678          db 8Eh,0C0h          ;mov es,ax
49679          db 0BFh,0Fh,00h      ;mov di,000FH
49680          db 0B9h,10h,00h      ;mov cx,0010H
49681          db 0B0h,0FFh        ;mov al,0FFH
49682          db 0F3h,0AEh        ;repz scasb
49683          db 47h              ;inc di
49684          db 8Bh,0F7h          ;mov si,di
49685          db 8Bh,0C3h          ;mov ax,bx
49686          db 2Bh,0C2h          ;sub ax,dx
49687          db 8Eh,0C0h          ;mov es,ax
49688          db 0BFh,0Fh,00h      ;mov di,000FH
49689          ;NextRec:
49690          db 0B1h,04h          ;mov cl,4
49691          db 8Bh,0C6h          ;mov ax,si
49692          db 0F7h,0D0h          ;not ax
49693          db 0D3h,0E8h          ;shr ax,cl
49694          db 74h,09h          ;jz short SI_ok
49695          db 8Ch,0DAh          ;mov dx,ds
49696          db 2Bh,0D0h          ;sub dx,ax
49697          db 8Eh,0DAh          ;mov ds,dx
49698          db 83h,0CEh,0F0h     ;or si,0FFF0H
49699          ;SI_ok:
49700          db 8Bh,0C7h          ;mov ax,di
49701          db 0F7h,0D0h          ;not ax
49702          db 0D3h,0E8h          ;shr ax,cl
49703          db 74h,09h          ;jz short DI_ok
49704          db 8Ch,0C2h          ;mov dx,es
49705          db 2Bh,0D0h          ;sub dx,ax
49706          db 8Eh,0C2h          ;mov es,dx
49707          db 83h,0CFh,0F0h     ;or di,0FFF0H
49708          ;DI_ok:
49709
49710 size_scan_patch1 equ $-scan_patch1
49711

```

```

49712 scan_patch2: ; label byte
49713
49714 db 8Ch,0C3h ;mov bx,es
49715 db 8Ch,0D8h ;mov ax,ds
49716 db 48h ;dec ax
49717 db 8Eh,0D8h ;mov ds,ax
49718 db 8Eh,0C0h ;mov es,ax
49719 db 0BFh,0Fh,00h ;mov di,000FH
49720 db 0B9h,10h,00h ;mov cx,0010H
49721 db 0B0h,0FFh ;mov al,0FFH
49722 db 0F3h,0AEh ;repz scasb
49723 db 47h ;inc di
49724 db 8Bh,0F7h ;mov si,di
49725 db 8Bh,0C3h ;mov ax,bx
49726 db 48h ;dec ax
49727 db 8Eh,0C0h ;mov es,ax
49728 db 0BFh,0Fh,00h ;mov di,000FH
49729 ;NextRec:
49730 db 0B1h,04h ;mov cl,4
49731 db 8Bh,0C6h ;mov ax,si
49732 db 0F7h,0D0h ;not ax
49733 db 0D3h,0E8h ;shr ax,cl
49734 db 74h,0Ah ;jz short SI_ok
49735 db 8Ch,0DAh ;mov dx,ds
49736 db 2Bh,0D0h ;sub dx,ax
49737 db 8Eh,0DAh ;mov ds,dx
49738 db 81h,0CEh,0F0h,0FFh ;or si,0FFF0H
49739 ;SI_ok:
49740
49741 db 8Bh,0C7h ;mov ax,di
49742 db 0F7h,0D0h ;not ax
49743 db 0D3h,0E8h ;shr ax,cl
49744 db 74h,0Ah ;jz short DI_ok
49745 db 8Ch,0C2h ;mov dx,es
49746 db 2Bh,0D0h ;sub dx,ax
49747 db 8Eh,0C2h ;mov es,dx
49748 db 81h,0CFh,0F0h,0FFh ;or di,0FFF0H
49749 ;DI_ok:
49750
49751
49752 size_scan_patch2 equ $-scan_patch2
49753
49754 scan_patch3: ; label byte
49755
49756 db 8Ch,0C3h ;mov bx,es
49757 db 8Ch,0D8h ;mov ax,ds
49758 db 48h ;dec ax
49759 db 8Eh,0D8h ;mov ds,ax
49760 db 8Eh,0C0h ;mov es,ax
49761 db 0BFh,0Fh,00h ;mov di,000FH
49762 db 0B9h,10h,00h ;mov cx,0010H
49763 db 0B0h,0FFh ;mov al,0FFH
49764 db 0F3h,0AEh ;repz scasb
49765 db 47h ;inc di
49766 db 8Bh,0F7h ;mov si,di
49767 db 8Bh,0C3h ;mov ax,bx
49768 db 48h ;dec ax
49769 db 8Eh,0C0h ;mov es,ax
49770 db 0BFh,0Fh,00h ;mov di,000FH
49771 ;NextRec:
49772 db 0B1h,04h ;mov cl,4
49773 db 8Bh,0C6h ;mov ax,si
49774 db 0F7h,0D0h ;not ax
49775 db 0D3h,0E8h ;shr ax,cl
49776 db 74h,09h ;jz short SI_ok
49777 db 8Ch,0DAh ;mov dx,ds
49778 db 2Bh,0D0h ;sub dx,ax
49779 db 8Eh,0DAh ;mov ds,dx
49780 db 83h,0CEh,0F0h ;or si,0FFF0H
49781 ;SI_ok:
49782 db 8Bh,0C7h ;mov ax,di
49783 db 0F7h,0D0h ;not ax
49784 db 0D3h,0E8h ;shr ax,cl
49785 db 74h,09h ;jz short DI_ok
49786 db 8Ch,0C2h ;mov dx,es
49787 db 2Bh,0D0h ;sub dx,ax
49788 db 8Eh,0C2h ;mov es,dx
49789 db 83h,0CFh,0F0h ;or di,0FFF0H
49790 ;DI_ok:
49791
49792
49793 size_scan_patch3 equ $-scan_patch3
49794
49795 scan_com: ; label byte
49796
49796 db 0ACh ;lods b
49797 db 8Ah,0D0h ;mov dl,al
49798 db 4Eh ;dec si
49799 db 0ADh ;lodsw
49800 db 8Bh,0C8h ;mov cx,ax
49801 db 46h ;inc si
49802 db 8Ah,0C2h ;mov al,dl
49803 db 24h,0FEh ;and al,0FEh
49804 db 3Ch,0B0h ;cmp al,RPTREC
49805 db 75h,06h ;jne short TryEnum
49806 db 0ACh ;lods b
49807 db 0F3h,0AAh ;rep stosb
49808 db 0EBh,07h,90h ;jmp short TryNext
49809 ;TryEnum:
49810 db 3Ch,0B2h ;cmp al,ENMREC
49811 db 75h,6Bh ;jne short CorruptExe
49812 db 0F3h,0A4h ;rep movsb
49813 ;TryNext:
49814 db 8Ah,0C2h ;mov al,dl
49815 db 0A8h,01h ;test al,1
49816 ; db 74h,0BAh ;jz short NextRec
49817
49818 size_scan_com equ $-scan_com
49819
49820 ;-----
49821
49822 ; 23/05/2019 - Retro DOS v4.0
49823 ; DOSCODE:B852h (MSDOS 6.21, MSDOS.SYS)
49824
49825 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
49826 ; DOSCODE:B530h (MSDOS 5.0, MSDOS.SYS)
49827
49828 ; 21/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
49829 ; DOSCODE:CB02h (PCDOS 7.1 IBMDOS.COM)
49830
49831 ExePatch:
49832 ; 21/03/2024 - Retro DOS v5.0
49833 ; ;28/12/2022 - Retro DOS v4.1
49834 call ExePackPatch
49835 call word [ss:RationalPatchPtr]

```

```

49836 00008912 C3      retn
49837                  ; 28/12/2022
49838                  ; jmp short ExePackPatch
49839
49840
49841
49842
49843
49844
49845
49846
49847
49848
49849
49850
49851
49852
49853
49854
49855
49856
49857
49858
49859
49860
49861
49862
49863
49864
49865
49866
49867
49868
49869 00008913 53      ; 21/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
49870 00008914 8CC3    ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
49871 00008916 81FBFF0F ; 23/05/2019 - Retro DOS v4.0
49872 0000891A 7602    ExePackPatch:
49873 0000891C 5B      push bx
49874 0000891D C3      mov bx,es ; bx has load segment
49875
49876 0000891E 1E      cmp bx,0FFFh ; Q: is the load segment > 64K
49877 0000891F 06      jbe short ep_cont ; N:
49878 00008920 50      pop bx ; Y: no need to patch
49879 00008921 51      retn
49880 00008922 56      ep_cont:
49881 00008923 57      push ds
49882
49883
49884
49885
49886 00008924 83E902   sub cx,2 ; M033 - start
49887 00008927 7303     jnb short epp_1 ; exepacked programs have an IP of 12h (>=2)
49888 00008929 E9B500   jmp ep_notpacked ; Q: is IP >=2
49889
49890
49891
49892
49893 0000892C 89CF     ; N: exit
49894 0000892E 8EC0     ; ax:cx now points to location of
49895 00008930 36893E[8700] ; 'RB' if this is an exepacked file.
49896
49897
49898
49899
49900
49901
49902
49903 0000893F 0E      ; M033 - end
49904 00008940 1F      epp_1:
49905
49906
49907 00008941 83C76C   mov di,cx
49908
49909
49910
49911 00008944 E8A200   mov es,ax
49912
49913
49914
49915
49916
49917
49918
49919
49920
49921
49922 00008951 83C728   mov [ss:UNPACK_OFFSET],di ; save pointer to 'RB' in
49923
49924
49925
49926
49927
49928
49929
49930
49931 00008956 BB8E00   cmp word [es:di],'RB' ; 4252h
49932
49933
49934
49935
49936
49937
49938
49939
49940
49941
49942 00008964 B166   ;ljne ep_notpacked
49943
49944
49945
49946
49947
49948
49949
49950 0000896A BF7600   je short epp_2
49951
49952
49953
49954
49955
49956
49957
49958
49959
49960
49961
49962
49963
49964
49965
49966
49967
49968
49969
49970
49971
49972
49973
49974
49975
49976
49977
49978
49979
49980
49981
49982
49983
49984
49985
49986
49987
49988
49989
49990
49991
49992
49993
49994
49995
49996
49997
49998
49999

```

```

49960          ;mov     di,32h
49961 00008975 BF3200 mov     di,PATCH2_OFFSET      ; es:di -> points to place in packed
49962          ;          ;          ;          file where we hope to find
49963          ;mov     cx,68          ;          ;          scan string.
49964          ;mov     cx,size_scan_patch2
49965          ; 21/03/2024
49966 00008978 B144   mov     cl,size_scan_patch2 ; 68
49967          ;mov     bx,140
49968 0000897A BB8C00 mov     bx,CHKSUM2_LEN
49969          ;mov     ax,78B2h
49970 0000897D B8B278 mov     ax,PATCH2_CHKSUM
49971 00008980 E87A00 call    chk_patchsum      ; check if patch and chk sum compare
49972
49973          ; M046 - Start
49974          ; Q: did we pass the test
49975 00008983 7310   jnc     short ep_patchcode2 ; Y: overlay code with new
49976          ; N: try with a different checksum
49977
49978 00008985 BE[6488] mov     si,scan_patch2
49979          ; ds:si -> scan string
49980          ;mov     cx,68
49981          ;mov     cx,size_scan_patch2
49982          ; 21/03/2024
49983 00008988 B144   mov     cl,size_scan_patch2 ; 68
49984          ;mov     bx,129
49985 0000898A BB8100 mov     bx,CHKSUM2A_LEN
49986          ;mov     ax,1C47h
49987 0000898D B8471C mov     ax,PATCH2A_CHKSUM
49988 00008990 E86A00 call    chk_patchsum      ; check if patch and chk sum compare
49989          ; Q: did we pass the test
49990 00008993 724C   jc      short ep_notpacked ; N: try with a different checksum
49991          ; Y: overlay code with new
49992
49993          ep_patchcode2:          ; M046 - End
49994 00008995 BE[BA87] mov     si,str1
49995          ;mov     cx,3
49996          ;mov     cx,first_stop
49997          ; 21/03/2024
49998 00008998 B103   mov     cl,first_stop ; 3
49999 0000899A F3A4   rep     movsb
50000 0000899C B89048 mov     ax,4890h          ; ax = opcodes for dec ax, nop
50001 0000899F AB     stosw
50002          ;add     si,2
50003          ; 21/03/2024
50004 000089A0 46     inc     si
50005 000089A1 46     inc     si
50006          ;mov     cx,20
50007          ;mov     cx,second_stop
50008          ; 21/03/2024
50009 000089A2 B114   mov     cl,second_stop ; 20
50010 000089A4 F3A4   rep     movsb
50011 000089A6 AB     stosw          ; put in dec ax and nop
50012          ;add     si,2
50013          ; 21/03/2024
50014 000089A7 46     inc     si
50015 000089A8 46     inc     si
50016          ;mov     cx,75
50017          ;mov     cx,last_stop
50018          ; 21/03/2024
50019 000089A9 B14B   mov     cl,last_stop ; 75
50020 000089AB F3A4   rep     movsb
50021 000089AD EB32   jmp     short ep_done
50022
50023          ep_chkpatch3:
50024          ;mov     di,74h
50025 000089AF BF7400 mov     di,PATCH3_COM_OFFSET ; es:di -> possible location of patch
50026          ;          ;          ; in another version of unpack
50027 000089B2 E83400 call    chk_common_str    ; check for match
50028
50029 000089B5 752A   jnz     short ep_notpacked ; Q: does the patch match
50030          ; N: exit
50031          ; Y: check for rest of patch string
50032 000089B7 BE[A888] mov     si,scan_patch3
50033          ; ds:si -> scan string
50034          ;mov     di,32h
50035 000089BA BF3200 mov     di,PATCH3_OFFSET      ; es:di -> points to place in packed
50036          ;          ;          ; file where we hope to find
50037          ;          ;          ; scan string.
50038          ;mov     cx,66
50039          ;mov     cx,size_scan_patch3
50040          ; 21/03/2024
50041 000089BD B142   mov     cl,size_scan_patch3 ; 66
50042          ;mov     bx,139
50043 000089BF BB8B00 mov     bx,CHKSUM3_LEN
50044          ;mov     ax,4EDEh
50045 000089C2 B8DE4E mov     ax,PATCH3_CHKSUM
50046 000089C5 E83500 call    chk_patchsum      ; check if patch and chk sum compare
50047 000089C8 7217   jc      short ep_notpacked ; Q: did we pass the test
50048          ; N: exit
50049          ; Y: overlay code with new
50050 000089CA BE[BA87] mov     si,str1
50051          ;mov     cx,3
50052          ;mov     cx,first_stop
50053          ; 21/03/2024
50054 000089CD B103   mov     cl,first_stop ; 3
50055 000089CF F3A4   rep     movsb
50056 000089D1 B048   mov     al,48h          ; al = opcode for dec ax
50057 000089D3 AA     stosb
50058          ;add     si,2
50059          ; 21/03/2024
50060 000089D4 46     inc     si
50061 000089D5 46     inc     si
50062          ;mov     cx,20
50063          ;mov     cx,second_stop
50064          ; 21/03/2024
50065 000089D6 B114   mov     cl,second_stop ; 20
50066 000089D8 F3A4   rep     movsb
50067 000089DA AA     stosb          ; put in dec ax
50068          ;add     si,2
50069          ; 21/03/2024
50070 000089DB 46     inc     si
50071 000089DC 46     inc     si
50072          ;mov     cx,75
50073          ;mov     cx,last_stop
50074          ; 21/03/2024
50075 000089DD B14B   mov     cl,last_stop ; 75
50076 000089DF F3A4   rep     movsb
50077
50078          ep_notpacked:
50079          ;stc
50080          ep_done:
50081 000089E1 5F     pop     di
50082 000089E2 5E     pop     si
50083 000089E3 59     pop     cx

```

```

50084 000089E4 58      pop     ax
50085 000089E5 07      pop     es
50086 000089E6 1F      pop     ds
50087 000089E7 5B      pop     bx
50088 000089E8 C3      retn
50089
50090
50091
50092
50093
50094
50095
50096
50097
50098
50099
50100
50101
50102
50103 000089E9 BE[EA88]
50104
50105
50106 000089EC B92000
50107
50108 000089EF F3A6
50109
50110
50111
50112
50113
50114
50115
50116
50117
50118
50119 000089F1 7409
50120 000089F3 26807DFF56
50121 000089F8 7502
50122
50123 000089FA F3A6
50124
50125 000089FC C3
50126
50127
50128
50129
50130
50131
50132
50133
50134
50135
50136
50137
50138
50139
50140
50141
50142
50143
50144
50145
50146
50147
50148 000089FD 57
50149
50150 000089FE F3A6
50151
50152 00008A00 7517
50153
50154
50155
50156
50157
50158
50159
50160 00008A02 368B3E[8700]
50161 00008A07 89D9
50162
50163 00008A09 89C3
50164 00008A0B 31C0
50165
50166 00008A0D 260305
50167
50168
50169 00008A10 47
50170 00008A11 47
50171 00008A12 E2F9
50172
50173 00008A14 5F
50174
50175 00008A15 39D8
50176
50177
50178
50179
50180
50181 00008A17 74E3
50182
50183
50184 00008A19 F9
50185 00008A1A C3
50186
50187
50188
50189
50190
50191
50192
50193
50194
50195
50196
50197
50198
50199
50200
50201
50202
50203
50204
50205
50206
50207

; 23/05/2019 - Retro DOS v4.0
chk_common_str:
mov     si,scan_com          ; ds:si -> scan string
;mov     cx,32
mov     cx,size_scan_com
repe     cmpsb
; M046 - start
; a fourth possible version of these exepacked programs have a
; 056h instead of 06Bh. See scan_com above
;
; db 75h, 6Bh                ;jne CorruptExe
;
; If the mismatch at this point is due to a 56h instead of 6Bh
; we shall try to match the rest of the string
;
jz      short ccs_done
cmp     byte [es:di-1],56h
jnz     short ccs_done
repe     cmpsb
ccs_done:
retn                                ; M046 - end

; 23/05/2019 - Retro DOS v4.0
chk_patchsum:
push     di
repe     cmpsb
jnz     short cp_fail          ; Q: does the patch match
; N: exit
; Y:
; we do a check sum starting from the location of the
; exepack signature 'RB' up to 11c/2 bytes, the end of the
; unpacking code.
mov     di,[ss:UNPACK_OFFSET] ; di -> start of unpack code
mov     cx,bx                  ; cx = length of check sum
mov     bx,ax                  ; save check sum passed to us in bx
xor     ax,ax
ep_chksum:
add     ax,[es:di]
;add     di,2
; 01/07/2024 (PCDOS 7.1 IBMDOS.COM)
inc     di
inc     di
loop    ep_chksum
pop     di                      ; restore di
cmp     ax,bx                  ; Q: does the check sum match
;jne     short cp_fail          ; N: exit
; Y:
; 25/09/2023
;clc
;retn
je      short ccs_done ; cf=0
cp_fail:
stc
retn

; 21/03/2024 - Retro DOS v5.0
; %if 1
; 28/12/2022 - Retro DOS v4.1
; %if 0
;
; M020 : BEGIN
;
;
; procedure : RationalPatch
;
; A routine (in Ration DOS extender) which is invoked at hardware interrupts
; clobbers CX register on 286 machines. (123 release 3 uses Rational DOS
; extender). This routine identifies Buggy Rational EXEs and fixes the bug.
;
; THE BUG is in the following code sequence:
;
;8b 0e 10 00      mov     cx, ds:[10h]          ; delay count
;90              even                          ; word align

```

```

50208 ;e2 fe          loop    $                ; wait          CLOBBERS CX
50209 ;e8 xx xx  call   set_A20                ; enable A20
50210 ;
50211 ; This patch routine replaces the mov & the loop with a far call into a
50212 ; routine in DOS data segment which is in low memory (because A20 line
50213 ; is off). The routine (RatBugCode) in DOS data saves & restores CX around
50214 ; a mov & loop.
50215 ;
50216 ; Identification of Buggy Rational EXE
50217 ; =====
50218 ;
50219 ; (ALL OFFSETS ARE IN THE PROGRAM SECTION - EXCLUDING THE EXE HEADER)
50220 ;
50221 ; OFFSET                Contains
50222 ; -----
50223 ; 0000h                100 times version number in binary
50224 ;                      bug exists in version 3.48 thru 3.83 (both inclusive)
50225 ;
50226 ; 000ah                the WORDS : 0000h, 0020h, 0000h, 0040h, 0001h
50227 ;
50228 ; 002ah                offset where version number is stored in ASCII
50229 ;                      e.g. '3.48A'
50230 ;
50231 ; 0030h                offset of copyright string. Copyright strings either
50232 ;                      start with "DOS/16M Copyright...." or
50233 ;                      "Copyright....". The string contains
50234 ;                      "Rational Systems, Inc."
50235 ;
50236 ; 0020h                word : Paragraph offset of the buggy code segment
50237 ;                      from the program image
50238 ; 0016h                word : size of buggy code segment
50239 ;
50240 ; Buggy code is definite to start after offset 200h in its segment
50241 ;
50242 ; -----
50243 ;
50244 ; 23/05/2019 - Retro DOS v4.0
50245 ; DOSCODE:B976h (MSDOS 6.21, MSDOS.SYS)
50246 ;
50247 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50248 ; DOSCODE:B654h (MSDOS 5.0, MSDOS.SYS)
50249 ;
50250 RScanPattern1:
50251 00008A1B 000020000000400001-
50251 00008A24 00
50252 ;
50253 RLen1 equ $ - RScanPattern1
50254 ;
50255 RScanPattern2:
50256 00008A25 8B0E100090E2FEE8
50257 ;
50258 RLen2 equ $ - RScanPattern2
50259 ;
50260 RScanPattern3:
50261 00008A2D 8B0E1000E2FEE8
50262 ;
50263 RLen3 equ $ - RScanPattern2
50264 ;
50265 ; DOSCODE:B98Fh (MSDOS 6.21, MSDOS.SYS)
50266 ; DOSCODE:B66Dh (MSDOS 5.0, MSDOS.SYS)
50267 ;
50268 ; -----
50269 ;
50270 ; INPUT : ES = segment where program got loaded
50271 ;
50272 ; -----
50273 ;
50274 ; 21/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
50275 ; DOSCODE:CC2Fh (PC DOS 7.1 IBMDOS.COM)
50276 ;
50277 RationalPatch:
50278 00008A34 FC
50279 cld
50280 ;
50281 ; 21/03/2024
50282 %if 0
50283     push    ax
50284     push    bx
50285     push    cx
50286     push    dx
50287     push    si
50288     push    di
50289 %else
50290     ; 21/03/2024 (PC DOS 7.1 IBMDOS.COM)
50291     ;;;
50292     pusha
50293     ;;;
50294 %endif
50295     push    es
50296     push    ds
50297     mov     di,0Ah
50298     ; we use all of them
50299     ; look for pat1 at offset 0Ah
50300     push    cs
50301     pop     ds
50302     mov     si,RScanPattern1
50303     ;mov     cx,10
50304     mov     cx,RLen1
50305     rep     cmpsb
50306     ; do we have the pattern ?
50307     jne     short rpexit
50308     mov     ax,[es:0]
50309     cmp     ax,348
50310     ; is it a buggy version ?
50311     jnb     short rpexit
50312     cmp     ax,383
50313     ; is it a buggy version ?
50314     jnb     short rpexit
50315     call    VerifyVersion
50316     jne     short rpexit
50317     mov     cx,[es:16h]
50318     ; Length of buggy code seg
50319     sub     cx,200h
50320     ; Length we search (we start
50321     ; at offset 200h)
50322     mov     es,[es:20h]
50323     ; es=buggy code segment
50324     mov     si,RScanPattern2
50325     ;mov     dx,8
50326     mov     dx,RLen2
50327     call    ScanCodeSeq
50328     ; look for code seq with nop
50329     jz     short rpfound
50330     mov     si,RScanPattern3
50331     ;mov     dx,15
50332     mov     dx,RLen3
50333     call    ScanCodeSeq
50334     ; look for code seq w/o nop
50335     jnz     short rpexit
50336 rpfound:

```



```

50331
50332 ; we set up a far call into DOS data
50333 ; dx has the length of the code seq we were searching for
50334
50335 00008A7E B09A      mov     al,9Ah          ; far call opcode
50336 00008A80 AA      stosb
50337 00008A81 B8[2611] mov     ax,RatBugCode
50338 00008A84 AB      stosw
50339 00008A85 8CD0     mov     ax,ss
50340 00008A87 AB      stosw
50341 00008A88 89D1     mov     cx,dx
50342 00008A8A 83E906   sub     cx,6          ; filler (with NOPs)
50343 00008A8D B090     mov     al,90h
50344 00008A8F F3AA     rep     stosb
50345 rpexit:
50346 00008A91 1F      pop     ds
50347 00008A92 07      pop     es
50348
50349 ; 21/03/2024
50350 %if 0
50351     pop     di
50352     pop     si
50353     pop     dx
50354     pop     cx
50355     pop     bx
50356     pop     ax
50357 %else
50358     ; 21/03/2024 (PCDOS 7.1 IBMDOS.COM)
50359     ;;;
50360 00008A93 61      popa
50361     ;;;
50362 %endif
50363 00008A94 C3      retn
50364
50365 ; M020 END
50366
50367 -----
50368 ; 21/03/2024
50369 ;%endif ; 28/12/2022
50370
50371 -----
50372 ;
50373 ; M068
50374 ;
50375 ; Procedure Name : IsCopyProt
50376 ;
50377 ; Inputs          : DS:100 -> start of com file just read in
50378 ;
50379 ; Outputs         : sets the A20OFF_COUNT variable to 10 if
50380 ;                  the program loaded in DS:100 uses a MICROSOFT
50381 ;                  copy protect scheme that relies on the A20 line
50382 ;                  being turned off for it's scheme to work.
50383 ;
50384 ;                  Note: The int 21 function dispatcher will turn
50385 ;                  a20 off, if the A20OFF_COUNT is non-zero
50386 ;                  and dec the A20OFF_COUNT before iretting
50387 ;                  to the user.
50388 ;
50389 ; Uses            : ES, DI, SI, CX
50390 ;
50391 -----
50392
50393 ; 23/05/2019 - Retro DOS v4.0
50394
50395 CPStartOffset      EQU      0175h
50396 CPID1offset EQU      0118h
50397 CPID2offset EQU      0173h
50398 CPID3offset EQU      0146h
50399 CPID4offset EQU      0124h
50400 ID1      EQU      05343h
50401 ID2      EQU      05044h
50402 ID3      EQU      0F413h
50403 ID4      EQU      08000h
50404
50405 ; DOSCODE:B9FAh (MSDOS 6.21, MSDOS.SYS)
50406
50407 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50408 ; DOSCODE:B71Ch (MSDOS 5.0, MSDOS.SYS)
50409
50410 ; 21/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
50411 ; DOSCODE:CC90h (PCDOS 7.1 IBMDOS.COM)
50412
50413 CPScanPattern:
50414 00008A95 89264801   db      89h,26h,48h,01h      ; mov [148],sp
50415 00008A99 8C0E4C01   db      8Ch,0Eh,4Ch,01h      ; mov [14C],cs
50416 00008A9D C7064A010001 db      0C7h,06h,4Ah,01h,00h,01h ; mov [14A],100h
50417 00008AA3 8C0E1301   db      8Ch,0Eh,13h,01h      ; mov [113],cs
50418 00008AA7 B82001   db      0B8h,20h,01h          ; mov ax,120h
50419 00008AAA BE0001   db      0BEh,00h,01h          ; mov si,100h
50420
50421 CPSPlen      EQU $ - CPScanPattern
50422
50423 ; DOSCODE:BA12h (MSDOS 6.21, MSDOS.SYS)
50424 ; DOSCODE:B734h (MSDOS 5.0, MSDOS.SYS)
50425 ; 21/03/2024
50426 ; DOSCODE:CCA8h (PCDOS 7.1 IBMDOS.COM)
50427
50428 IsCopyProt:
50429 00008AAD 813E1B014353 cmp     word [CPID1offset],ID1
50430 00008AB3 752D      jne     short CP_done
50431
50432 00008AB5 813E73014450 cmp     word [CPID2offset],ID2
50433 00008ABB 7525      jne     short CP_done
50434
50435 00008ABD 813E460113F4 cmp     word [CPID3offset],ID3
50436 00008AC3 751D      jne     short CP_done
50437
50438 00008AC5 813E24010080 cmp     word [CPID4offset],ID4
50439 00008ACB 7515      jne     short CP_done
50440
50441 00008ACD 0E      push    cs
50442 00008ACE 07      pop     es
50443 00008ACF BF[958A]   mov     di,CPScanPattern      ; es:di -> Pattern to find
50444
50445 00008AD2 BE7501   mov     si,CPStartOffset      ; ds:si -> possible location
50446                                ; of pattern
50447
50448 00008AD5 B91800   mov     cx,CPSPlen ; 24          ; cx = length of pattern
50449 00008AD8 F3A6     repe    cmpsb
50450 00008ADA 7506     jnz     short CP_done
50451
50452 00008ADC 36C606[8500]0A mov     byte [ss:A20OFF_COUNT],0Ah ; M071
50453 CP_done:
50454 00008AE2 C3      retn

```

```

50455
50456 ;DOSCODE ENDS
50457
50458 ;END
50459
50460 ;-----
50461
50462 ;align 2 ; 05/09/2018 (Error!)
50463
50464 ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50465 ;align 16 ; 08/09/2018 (OK.)
50466 00008AE3 90 align 2
50467
50468 ; 06/08/2018 - Retro DOS v3.0
50469 ;=====
50470 ; MSINIT.ASM
50471 ;=====
50472 ; 22/04/2019 - Retro DOS v4.0 (MSINIT.ASM, MSDOS 6.0, 1991)
50473
50474 ; MAIN ENTRY FOR DOS INITIALIZATION
50475 ;
50476 ; 15/07/2018 - Retro DOS v3.0
50477 ; (MSDOS 3.3, IBMDOS.COM, 1987)
50478
50479 ; temp iret instruction
50480
50481 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50482 ; DOSCODE:B76Ah (MSDOS 5.0, MSDOS.SYS)
50483
50484 initiret: ; MSDOS 6.0
50485 SYSBUF:
50486 ;IRETT: ; 06/05/2019
50487 00008AE4 CF iret
50488
50489 ; 22/04/2019 - Retro DOS v4.0
50490
50491 ; pointer to the BIOS data segment that will be available just to the
50492 ; initialization code
50493
50494 00008AE5 7000 InitBioDataSeg: dw 70h ; KERNEL_SEGMENT = 0070h
50495
50496 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50497 ; DOSCODE:B76Dh (MSDOS 5.0, MSDOS.SYS)
50498
50499 ; 22/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
50500 ; DOSCODE:CCE1h (PCDOS 7.1 IBMDOS.COM)
50501
50502 ; Convert AX from a number of bytes to a number of paragraphs (round up).
50503
50504 ParaRound:
50505 00008AE7 83C0F add ax, 15
50506 00008AEA D1D8 rcr ax, 1
50507 00008AEC D1E8 shr ax, 1
50508 00008AEE D1E8 shr ax, 1
50509 00008AF0 D1E8 shr ax, 1
50510 00008AF2 C3 retn
50511
50512 ; -----
50513
50514 ; 22/03/2024 - Retro DOS v5.0
50515 %if 1
50516 ; 28/12/2022 - Retro DOS v4.1
50517 %if 0
50518
50519 ;-----
50520 ;
50521 ; Procedure Name : WhatCPUType
50522 ;
50523 ; Inputs : none
50524 ;
50525 ; Outputs : AL = 0 if CPU < 286
50526 ; = 1 if CPU == 286
50527 ; = 2 if CPU >= 386
50528 ;
50529 ; Regs. Mod. : AX
50530 ;
50531 ;-----
50532
50533 WhatCPUType:
50534 ; 25/04/2019 - Retro DOS v4.0
50535 ;get_cpu_type ; done with a MACRO which can't be generated > once
50536
50537 ;CPU_TYPE.INC (MSDOS 6.0, 1991)
50538
50539 ; Note: this must be a macro, and not a subroutine in the BIOS since
50540 ; it is called from both CODE and SYSINITSEG.
50541 ;
50542 ;-----GET_CPU_TYPE-----May, 88 by M.Williamson
50543 ; Returns: AX = 0 if 8086 or 8088
50544 ; = 1 if 80286
50545 ; = 2 if 80386
50546
50547 ; 04/11/2022
50548 ; MSDOS 5.0 MSDOS.SYS - DOSCODE:0BB03h
50549
50550 ; 22/03/2024
50551 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0CCEDh
50552
50553 Get_CPU_Type: ;macro
50554 00008AF3 9C pushf
50555 00008AF4 53 push bx ; preserve bx
50556 00008AF5 31DB xor bx,bx ; init bx to zero
50557
50558 00008AF7 31C0 xor ax,ax ; 0000 into AX
50559 00008AF9 50 push ax ; put it on the stack...
50560 00008AFA 9D popf ; ...then shove it into the flags
50561 00008AFB 9C pushf ; get it back out of the flags...
50562 00008AFC 58 pop ax ; ...and into ax
50563 00008AFD 2500F0 and ax,0F000h ; mask off high four bits
50564 00008B00 3D00F0 cmp ax,0F000h ; was it all 1's?
50565 00008B03 740E je short cpu_8086 ; aye; it's an 8086 or 8088
50566
50567 00008B05 B800F0 mov ax,0F000h ; now try to set the high four bits..
50568 00008B08 50 push ax
50569 00008B09 9D popf
50570 00008B0A 9C pushf
50571 00008B0B 58 pop ax ; ...and see what happens
50572 00008B0C 2500F0 and ax,0F000h ; any high bits set ?
50573 00008B0F 7401 jz short cpu_286 ; nay; it's an 80286
50574
50575 00008B11 43 cpu_386: inc bx ; bx starts as zero
50576 ; inc twice if 386
50577 00008B12 43 cpu_286: inc bx ; just inc once if 286
50578 ; don't inc at all if 086

```



```

50579 00008B13 89D8      mov     ax,bx          ; put CPU type value in ax
50580 00008B15 5B        pop     bx            ; restore original bx
50581 00008B16 9D        popf
50582
50583                  ;endm
50584
50585                  ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS)
50586 00008B17 C3        retn     ; 19/09/2023
50587
50588                  ; -----
50589                  %endif      ; 28/12/2022
50590
50591                  ; =====
50592
50593                  ; MAIN ENTRY FOR DOS INITIALIZATION
50594
50595                  ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50596                  ; DOSCODE:B779h (MSDOS 5.0 MSDOS.SYS)
50597
50598                  ; 23/03/2024
50599                  ; 22/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
50600                  ; DOSCODE:CCEDh (PCDOS 7.1 IBMDOS.COM)
50601
50602                  ; DOSCODE:BA57h (MSDOS 6.22 MSDOS.SYS)
50603                  ; BIOSCODE:CF81h (Windows ME IO.SYS)
50604
50605                  ; 30/05/2019
50606                  ; 22/04/2019 - Retro DOS v4.0
50607                  ; 07/07/2018 - Retro DOS v3.0
50608                  ; Retro DOS v2.0 - 03/03/2018
50609                  ; 03/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50610                  ; MSDOS 5.0 - MSDOS.SYS, offset 79A9h
50611 DOSINIT:
50612                  ; MSDOS 6.21 - MSDOS.SYS, offset 7C77h
50613
50614                  ; Far call from SYSINIT
50615                  ; DX = Memory size in paragraphs
50616                  ; DS:SI = [DEVICE_LIST] (SYSINIT.S)
50617                  ; (Retro DOS v2.0, 16/03/2018)
50618
50619                  ; ES:DI = ptr to BIOS communication block (sysinit3.s)
50620                  ; (Retro DOS v4.0, 20/04/2019)
50621
50622 00008B18 FA          CLI
50623 00008B19 FC          CLD
50624
50625                  ; 03/11/2022
50626                  ; push dx ; 30/05/2019          ; save parameters from BIOS
50627
50628                  ; 17/12/2022
50629                  ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50630                  ; push dx ; *=          ; save parameters from BIOS
50631
50632 00008B1A 56          push    si
50633 00008B1B 1E          push    ds
50634 00008B1C 57          push    di          ; save di (ptr to BiosComBlock)
50635
50636 00008B1D 8C3        mov     bx,es          ; bx:di = ptr to BiosComBlock
50637
50638                  ; First, move the DOS data segment to its final location in low memory
50639
50640                  ;; mov ax,0BF69h ; MSDOS 6.21 MSDOS.SYS, file offset 7C7Fh
50641                  ;; mov ax,0BC77h ; MSDOS 5.0 MSDOS.SYS, file offset 79B1h
50642                  ; 22/03/2024
50643                  ; mov ax,0D20Fh ; PCDOS 7.1 IBMDOS.COM, file offset 92FFh
50644 00008B1F B8[DB8F]   mov     ax,MEMSTRT      ; get offset of end of init code
50645
50646                  ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50647 00008B22 83C00F      add     ax,15          ; round to nearest paragraph
50648                  ; and ax,~15 ; 0FFF0h          ; boundary
50649                  ; 12/04/2024
50650 00008B25 24F0        and     al,0F0h
50651
50652 00008B27 89C6        mov     si,ax          ; si = offset of DOSDATA in current
50653                  ; code segment
50654
50655                  ; 05/12/2022
50656                  ; 30/04/2019 - Retro DOS v4.0
50657                  ; xor si,si
50658
50659                  ; mov ax,cs
50660                  ; mov ds,ax          ; ds = current code segment
50661                  ; DS:SI now points to dosdata
50662 00008B29 0E          push    cs
50663 00008B2A 1F          pop     ds
50664
50665                  ; mov es,[cs:0BA49h] ; MSDOS 6.21 IO.SYS, offset 7C8Eh
50666                  ; mov es,[cs:InitBioDataSeg] ; First access to DosDataSg in
50667                  ; BData segment. Cannot use
50668                  ; getdseg macro here!!!
50669
50670 00008B2B 8E06[E58A]   mov     es,[InitBioDataSeg]
50671                  ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50672                  ; mov es,[cs:InitBioDataSeg] ; ds = cs !
50673
50674                  ; mov es,[es:3]
50675 00008B2F 268E060300   mov     es,[es:0300]      ; Get free location in low memory
50676
50677 00008B34 31FF        xor     di,di          ; ES:DI now points to RAM data
50678
50679                  ; mov cx,4970 ; Offset 0BA78h in MSDOS 6.21 MSDOS.SYS)
50680                  ; mov cx,4976 ; 25/05/2019
50681                  ; 22/03/2024
50682                  ; mov cx,4934 ; PCDOS 7.1 IBMDOS.COM
50683                  ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50684                  ; mov cx,4962
50685                  ; mov cx,MSDAT001E ; get end of dosdata = size of dosdata
50686 00008B36 B94613      mov     cx,DOSDATASIZE ; = 4962 for MSDOS 5.0 MSDOS.SYS
50687                  ; = 4934 for PCDOS 7.1 IBMDOS.COM ; 01/07/2024
50688 00008B39 F3A4        rep     movsb          ; move data to final location
50689
50690 00008B3B 5F          pop     di          ; restore ptr to BiosComBlock
50691 00008B3C 1F          pop     ds          ; restore parms from BIOS
50692 00008B3D 5E          pop     si
50693
50694                  ; 17/12/2022
50695                  ; pop dx ; 30/05/2019
50696                  ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50697                  ; pop dx ; *=
50698 00008B3E 06          push    es
50699 00008B3F 1E          push    ds
50700 00008B40 07          pop     es          ; es:si -> device chain
50701 00008B41 1F          pop     ds          ; ds points to dosdata
50702

```

```

50703 ;SR;
50704 ;we get a ptr to the BIOS exchange data block. This has been setup right
50705 ;now so that the EXEC call knows when SysInit is present to do the special
50706 ;lie table handling for device drivers. This can be expanded later on to
50707 ;establish a communication block from the BIOS to the DOS.
50708
50709 ;mov [1040h],di ; Offset 0BA87h in MSDOS 6.21 MSDOS.SYS)
50710 ;mov [1042h],bx
50711 00008B42 893E[2313] mov [BiosDataPtr],di
50712 00008B46 891E[2513] mov [BiosDataPtr+2],bx ; save ptr to BiosComBlock
50713
50714 00008B4A 2E8C1E[0700] mov [cs:DosDSeg],ds ; set pointer to dosdata in code seg
50715
50716 ; Set the segment of LowInt23/24/28Addr in msctrlc.asm to dosdata
50717
50718 00008B4F 2E8C1E[F55D] mov [cs:LowInt23Addr+2],ds; set pointers in code seg
50719 00008B54 2E8C1E[F95D] mov [cs:LowInt24Addr+2],ds
50720 00008B59 2E8C1E[FD5D] mov [cs:LowInt28Addr+2],ds
50721
50722 ;mov [346h],dx ; MSDOS 6.21 DOSDATA addresses
50723 ;mov [584h],sp
50724 ;mov [586h],ss
50725 00008B5E 8916[4603] mov [ENDMEM],dx ; *=
50726 00008B62 8926[8405] mov [USER_SP],sp
50727 00008B66 8C16[8605] mov [USER_SS],ss
50728
50729 ;mov ax,ds ; set up ss:sp to dosdata:dskstack
50730 ;mov ss,ax
50731 ; 01/07/2024
50732 00008B6A 1E push ds
50733 00008B6B 17 pop ss
50734
50735 ;mov sp,920h ; MSDOS 6.21 DOSDATA address
50736 ;mov sp,offset dosdata:dskstack
50737 00008B6C BC[2009] mov sp,DSKSTACK ; 920h ; PC DOS 7.1 IBMDOS.COM ; 22/03/2024
50738
50739 ;M023
50740 ; Init patch ptrs to default values
50741
50742 ; 22/03/2024
50743 %if 0
50744 ;mov word [1212h],RetExePatch
50745 ;mov word [1214h],RetExePatch
50746 ;mov word [61h],RetExePatch
50747 ;mov word [FixExePatch],RetExePatch ; M023
50748 ; 28/12/2022 - Retro DOS v4.1
50749 ;mov word [RationalPatchPtr],RetExePatch ; M023
50750 ;mov word [ChkCopyProt],RetExePatch ; M068
50751 %else
50752 ; 22/03/2024 (PCDOS 7.1 IBMDOS.COM)
50753 ;;;
50754 00008B6F B8[9672] mov ax,RetExePatch
50755 00008B72 A3[670D] mov [FixExePatch],ax
50756 00008B75 A3[690D] mov [RationalPatchPtr],ax ; 25/03/2024
50757 00008B78 A3[6100] mov [ChkCopyProt],ax
50758 ;;;
50759 %endif
50760
50761 ; 22/03/2024
50762 %if 1
50763 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50764 ;%if 0 ; 19/09/2023
50765
50766 ; Setup to call 386 Rational DOS Extender patch routine if running on
50767 ; a 386 or later. Unlike other patches, this is not dependent on MS-DOS
50768 ; running in the HMA.
50769
50770 00008B7B E875FF call whatCPUtype ; get cpu type (0 < 286,1==286,2 >= 386)
50771 00008B7E 3C02 cmp al,2 ; 386 or later?
50772 00008B80 B8[B885] mov ax,Rational386Patch
50773 00008B83 7303 jae short di_set_patch
50774 00008B85 B8[9672] mov ax,RetExePatch ; < 386, don't need this patch
50775 di_set_patch:
50776 00008B88 A3[3E13] mov [Rational386PatchPtr],ax ; patch routine or RET instr.
50777
50778 %endif
50779 ; Set up the variable temp_dosloc to point to the dos code segment
50780
50781 00008B8B 8CC8 mov ax,cs ; ax = current segment of DOS code
50782
50783 ; ax now holds segment of DOS code
50784 00008B8D A3[A30A] mov [TEMP_DOSLOC],ax ; store temp location of DOS
50785
50786 00008B90 8C06[4A00] mov word [NULDEV+2],es ; nuldev -> points to device chain
50787 00008B94 8936[4800] mov word [NULDEV],si
50788 ;SR;
50789 ; There are some locations in the win386 instance data structures
50790 ; which need to be set up with the DOS data segment. First, initialize
50791 ; the segment part of the instance table pointer in the SIS.
50792
50793 ;;mov [0FF2h],ds ; [win386_Info+14+2]
50794 ;;mov [0EF1h],ds ; PC DOS 7.1 IBMDOS.COM ; 22/03/2024
50795 00008B98 8C1E[F10E] mov [win386_Info+win386_SIS.Instance_Data_Ptr+2],ds
50796
50797 ; Now initialize the segment part of the pointer to the data in each
50798 ; instance table entry.
50799
50800 00008B9C 56 push si ; preserve pointer to device chain
50801 ; 18/12/2022
50802 ; cx = 0
50803 00008B9D B107 mov cl,7
50804 ;mov cx,7 ; There are 7 entries in the instance table
50805 ; M019
50806 ;;mov si,0FF6h ; offset (dosdata:Instance_Table+2)
50807 ;;mov si,0EF9h ; PC DOS 7.1 IBMDOS.COM ; 22/03/2024
50808 00008B9F BE[F90E] mov si,Instance_Table+2 ; point si to segment field
50809 Instance_init_loop:
50810 00008BA2 8C1C mov [si],ds ; set offset in instance entry
50811 ;add si,6
50812 00008BA4 83C606 add si,size_of_win386_IIS ; move on to next entry
50813 00008BA7 E2F9 loop Instance_init_loop
50814
50815 ;Initialize the WIN386 2.xx instance table with the DOS data segment value
50816
50817 ; 18/12/2022
50818 00008BA9 B105 mov cl,5
50819 ;mov cx,5 ; There are five entries in the instance table
50820
50821 ;mov si,(offset dosdata:OldInstanceJunk) + 6
50822 ;;mov si,11EDh ; point si to segment field
50823 ;;mov si,1102h ; PC DOS 7.1 IBMDOS.COM ; 22/03/2024
50824 00008BAB BE[0211] mov si,OldInstanceJunk+6
50825 OldInstance_init_loop:
50826 00008BAE 8C1C mov [si],ds ; set offset in instance entry

```

```

50827 00008BB0 83C606      add     si,6           ; move on to next entry
50828 00008BB3 E2F9      loop    OldInstance_init_loop
50829 00008BB5 5E        pop     si           ; restore pointer to device chain
50830
50831                      ; End of WIN386 2.xx compatibility bullshit
50832
50833                      ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50834 %if 0
50835                      ; 30/04/2019
50836                      ; push es
50837                      ; pop ds
50838                      ; ds:si points to console device
50839
50840                      ; 24/04/2019 - Retro DOS v4.0
50841
50842                      ; 15/07/2018
50843                      ; MSDOS 3.3 (IBMDOS.COM, 1987)
50844                      ; (Set INT 2Ah handler address to an 'IRET')
50845
50846                      ; need crit vector init'd to use devioctl
50847                      ; push ds           ; preserve segment of device chain
50848                      ; push es ; 30/04/2019
50849
50850 %endif
50851                      ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50852 00008BB6 06        push    es
50853                      ; 17/12/2022
50854                      ; pop ds
50855                      ; push ds
50856
50857 00008BB7 31C0      xor     ax,ax
50858 00008BB9 8ED8      mov     ds,ax           ; point DS to int vector table
50859 00008BBB B8[E48A]  mov     ax,initiret
50860                      ; mov [0A8h],ax ; [2Ah*4]
50861 00008BBE A3A800    mov     [addr_int_ibm],ax
50862 00008BC1 8CC8      mov     ax,cs
50863                      ; mov [0AAh],ax ; [(2Ah*4)+2]
50864 00008BC3 A3AA00    mov     [addr_int_ibm+2],ax
50865 00008BC6 1F        pop     ds           ; restore segment of device chain
50866
50867 00008BC7 E84702    call    CHARINIT        ; initialize console driver
50868 00008BCA 56        push    si           ; save pointer to header
50869
50870 00008BCB 16        push    ss           ; move pointer to dos data...
50871 00008BCC 07        pop     es           ; ...into ES
50872
50873                      ; initialize sft for file 0 (CON)
50874
50875                      ; 07/07/2018 - Retro DOS v3.0
50876                      ; 24/04/2019 - Retro DOS v4.0
50877                      ; mov di,SFTABL+6 ; SFT0_SFTable ; 22/03/2024
50878 00008BCD BFD200    MOV     DI,SFTABL+SFT.SFTable ; Point to sft 0
50879 00008BD0 B80300    MOV     AX,3
50880 00008BD3 AB        STOSW        ; Refcount
50881                      ; DEC AL
50882                      ; 22/03/2024 - Retro DOS v5.0
50883 00008BD4 48        dec     ax
50884 00008BD5 AB        STOSW        ; Access rd/wr, compatibility
50885 00008BD6 30C0    XOR     AL,AL
50886 00008BD8 AA        STOSB        ; attribute
50887                      ; mov al,0C3h
50888 00008BD9 B0C3      mov     al,devid_device_EOF|devid_device|ISCIN|ISCOUT
50889 00008BDB AB        STOSW        ; flags
50890 00008BDC 89F0      mov     ax,si
50891 00008BDE AB        stosw        ; device pointer in devptr
50892 00008BDF 8CD8      mov     ax,ds
50893 00008BE1 AB        stosw
50894 00008BE2 31C0      xor     ax,ax ; 0
50895
50896                      ; 22/03/2024
50897 %if 0
50898                      ; stosw ; firlus
50899                      ; stosw ; time
50900                      ; stosw ; date
50901                      dec     ax ; -1
50902                      ; stosw ; size
50903                      ; stosw
50904                      inc     ax ; 0
50905                      ; stosw ; position
50906                      ; stosw
50907 %else
50908                      ; 22/03/2024 (PCDOS 7.1 IBMDOS.COM)
50909                      ;;;
50910 00008BE4 83C720    add     di,32           ; SFTABL+SFT.SFTable + 43
50911 00008BE7 AB        stosw        ; SFT0_SFTable + 43 ; .sf_fclus32
50912 00008BE8 AB        stosw        ; SF_ENTRY.sf_fclus32+2
50913 00008BE9 83C7DE    add     di,-34          ; 0FFDEh ; SFTABL+SFT.SFTable + 13
50914 00008BEC AB        stosw        ; SFT0_SFTable + 13 ; .sf_time
50915 00008BED AB        stosw        ; SF_ENTRY.sf_date
50916 00008BEE 48        dec     ax ; -1
50917 00008BEF AB        stosw        ; SFT0_SFTable + 17 ; .sf_size
50918 00008BF0 AB        stosw        ; SF_ENTRY.sf_size + 2
50919 00008BF1 40        inc     ax ; 0
50920 00008BF2 AB        stosw        ; SFT0_SFTable + 21 ; .sf_position
50921 00008BF3 AB        stosw        ; SF_ENTRY.sf_position + 2
50922                      ;;;
50923 %endif
50924
50925                      ; add di,7 ; SFT0_SFTable + 32 ; 22/03/2024
50926 00008BF4 83C707    add     di,SF_ENTRY.sf_name-SF_ENTRY.sf_cluspos
50927                      ; point at name
50928
50929 00008BF7 83C60A    add     si,10
50930                      add     si,SYSDEV.NAME ; sdevname
50931                      ; point to name
50932 00008BFA B90400    mov     cx,4
50933 00008BFD F3A5      rep     movsw          ; name
50934 00008BFF B103      mov     cl,3
50935 00008C01 B020      mov     al," "
50936 00008C03 F3AA      rep     stosb          ; extension
50937 00008C05 5E        pop     si           ; get back pointer to header
50938
50939                      ; mark device as CON I/O
50940
50941                      ; 15/07/2018
50942 00008C06 804C0403  OR     BYTE [SI+4],ISCIN|ISCOUT ; or byte [si+4],3
50943                      ; 12/03/2018
50944                      ; mov [ss:32h],si
50945 00008C0A 368936[3200] MOV     [SS:BCON],SI
50946                      ; mov [ss:34h],ds
50947 00008C0F 368C1E[3400] MOV     [SS:BCON+2],DS
50948
50949                      ; initialize each device until the clock device is found
50950

```

```

50951 CHAR_INIT_LOOP:
50952     LDS     SI,[SI]                ; AUX device
50953     call    CHARINIT
50954     ;15/07/2018
50955     ;test    byte [SI+4],8
50956     TEST    BYTE [SI+SYSDEV.ATT],ISCLOCK
50957     JZ      SHORT CHAR_INIT_LOOP
50958     ; 12/03/2018
50959     ;mov     [ss:2Eh],si
50960     MOV     [SS:BCLOCK],SI
50961     ;mov     [ss:30h],ds
50962     MOV     [SS:BCLOCK+2],DS
50963     ;MOV     BP,MEMSTRT ; Retro DOS 3.0 ; ES:BP points to DPB
50964
50965     ;mov     bp,4970                ; bp = pointer to free mem
50966     ;mov     bp,4976 ; 25/05/2019 - Retro DOS v4.0
50967     ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0, MSDOS.SYS)
50968     ;mov     bp,4962 ; (MSDOS 5.0 MSDOS.SYS)
50969     ; 22/03/2024 - Retro DOS v5.0
50970     ;mov     bp,4934 ; (PCDOS 7.1 IBMDOS.COM)
50971     mov     bp,MSDAT001E          ; es:bp points to dpb area
50972
50973     mov     [ss:DPBHEAD],bp        ; set offset of pointer to DPB's
50974     mov     [ss:DPBHEAD+2],es      ; set segment of pointer to DPB's
50975
50976 PERDRV:
50977     ;lds     si,[SI+SYSDEV.NEXT] ; 15/07/2018
50978     LDS     SI,[SI]                ; Next device
50979     CMP     SI,-1 ; 0FFFFh
50980     JZ      SHORT CONTINIT
50981     ; 23/03/2024
50982     ;;;
50983     jnz     short PERDRV2
50984     jmp     CONTINIT
50985
50986 PERDRV2:
50987     ;;;
50988     call    CHARINIT
50989
50990     ; Retro DOS v2.0 - 16/03/2018 (NOTE for 'CHARINIT' return):
50991     ; [CALLUNIT] = Number of drives for (Disk) Block Dev Driver ([DRVMAX])
50992     ; (..when the command is 'DSK$INIT', as in 'CHARINIT')
50993     ; [CALLBPB] = [DEVCALL.COUNT] = Address of the BPB (DEVCALL offset 18)
50994     ; (REF: MSDOS 3.3 MSBIO2.ASM, MSDATA.INC, MSDISK.ASM, MSBIO1.ASM)
50995     ; (.. !DSK$IN' in MSBIO1.ASM)
50996     ; DEVCALL.MEDIA = CALLUNIT (DEVCALL offset 13)
50997
50998     ; 15/07/2018
50999     ;test    word [SI+4],8000h      ; DEVTYP
51000     ; 17/12/2022
51001     ;test    byte [SI+5],80h
51002     test    byte [SI+SYSDEV.ATT+1],(DEVTYP>>8) ; 80h
51003     ;TEST    word [SI+SYSDEV.ATT],DEVTYP ; 8000h
51004     JNZ     SHORT PERDRV          ; Skip any other character devs
51005
51006     MOV     CL,[SS:CALLUNIT] ; 12/03/2018
51007     XOR     CH,CH
51008     ; 07/07/2018
51009     ;MOV     [SI+10],CL            ; Number of units in name field
51010     mov     [si+SYSDEV.NAME],cl    ; sdevname
51011     MOV     DL,[SS:NUMIO] ; 15/03/2018
51012     XOR     DH,DH
51013     ADD     [SS:NUMIO],CL ; 12/03/2018
51014     PUSH    DS
51015     PUSH    SI
51016     LDS     BX,[SS:CALLBPB]      ; 12/03/2018
51017
51018 PERUNIT:
51019     MOV     SI,[BX]                ; DS:SI Points to BPB
51020     INC     BX
51021     INC     BX                    ; On to next BPB
51022     ; 15/12/2022
51023     ; 07/07/2018
51024     ;mov     [ES:BP+DPB.DRIVE],DL
51025     MOV     [ES:BP],DL
51026     ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51027     ;mov     [ES:BP+0],DL
51028     ;mov     [ES:BP+DPB.DRIVE],DL
51029
51030     ;MOV     [ES:BP+1],DH
51031     MOV     [ES:BP+DPB.UNIT],DH
51032     PUSH    BX
51033     PUSH    CX
51034     PUSH    DX
51035
51036     ; 23/03/2024 (PCDOS 7.1 IBMDOS.COM)
51037     ;;;
51038     mov     dx,4152h ; 'RA'        ; signature ('AR') for FAT32 extended BPB/DPB
51039     xor     cx,cx ; 0
51040     ;mov     [es:bp+29],cx
51041     mov     [es:bp+DPB.NEXT_FREE],cx
51042     cmp     [si+A_BPB.SECTORSPERFAT],cx ; 16 bit FAT size = 0 ?
51043     ;cmp     [si+11],cx ; BPB_FATSz16
51044     jnz     short PERUNIT2 ; FAT (FAT12 or FAT16) -old DPB-
51045     ;mov     [es:bp+57],cx ; FAT32 -new- DPB
51046     mov     [es:bp+DPB.FAT32_NXTFREE],cx
51047     ;mov     [es:bp+59],cx
51048     mov     [es:bp+DPB.FAT32_NXTFREE+2],cx
51049     dec     cx ; 0FFFFh ; -1
51050     ;mov     [es:bp+31],cx
51051     mov     [es:bp+DPB.FREE_CNT],cx
51052     ;mov     [es:bp+33],cx
51053     mov     [es:bp+DPB.FREE_CNT_HW],cx
51054     mov     cx,4558h ; 'XE'        ; signature ('EX') for FAT32 extended BPB/DPB
51055
51056 PERUNIT2:
51057     ;;;
51058     ;invoke   $SETDPB
51059     CALL     _SETDPB                ; build DPB!
51060
51061     ; 07/07/2018
51062     ;MOV     AX,[ES:BP+2]
51063     mov     ax,[ES:BP+DPB.SECTOR_SIZE]
51064     ; 12/03/2018
51065     CMP     AX,[SS:MAXSEC]          ; Q:is this the largest sector so far
51066     JBE     SHORT NOTMAX            ; N:
51067     MOV     [SS:MAXSEC],AX          ; Y: save it in maxsec
51068
51069 NOTMAX:
51070     ; set the next dpb field in the currently built bpb
51071     ; and mark as never accessed
51072
51073     ; 24/04/2019
51074     mov     ax,bp                  ; get pointer to DPB
51075     ;add     ax,33

```

```

51075 ;add ax,61 ; next DPB (PCDOS 7.1 DPB size = 61) ; 23/03/2024
51076 00008CAB 83C03D add ax,DPBSIZ ; advance pointer to next DPB
51077 ; set seg & offset of next DPB
51078 ;mov [es:bp+25],ax
51079 00008CAE 26894619 mov [es:bp+DPB.NEXT_DPB],ax
51080 ;mov [es:bp+27],es
51081 00008CB2 268C461B mov [es:bp+DPB.NEXT_DPB+2],es
51082 ; mark as never accessed
51083 ;mov byte [es:bp+24],0FFh
51084 00008CB6 26C64618FF mov byte [es:bp+DPB.FIRST_ACCESS],-1
51085
51086 00008CBB 5A POP DX
51087 00008CBC 59 POP CX
51088 00008CBD 5B POP BX
51089 00008CBE 8CD8 MOV AX,DS ; save segment of bpb array
51090 00008CC0 5E POP SI
51091 00008CC1 1F POP DS
51092 ; ds:si -> device header
51093 ; store it in the corresponding dpb
51094 ; 07/07/2018
51095 ;MOV [ES:BP+19],SI ; 24/04/2019
51096 00008CC2 26897613 mov [ES:BP+DPB.DRIVER_ADDR],si
51097 ;MOV [ES:BP+21],DS ; 24/04/2019
51098 00008CC6 268C5E15 mov [ES:BP+DPB.DRIVER_ADDR+2],ds
51099
51100 00008CCA 1E PUSH DS ; save pointer to device header
51101 00008CCB 56 PUSH SI
51102 00008CCC FEC6 INC DH ; inc unit #
51103 00008CCE FEC2 INC DL ; inc drive #
51104 00008CD0 8ED8 MOV DS,AX ; restore segment of BPB array
51105 ;;add bp,33 ; 24/04/2019
51106 ;;add bp,61 ; 23/03/2024 (PCDOS 7.1)
51107 00008CD2 83C53D ADD BP,DPBSIZ ; advance pointer to next dpb
51108 00008CD5 E28F LOOP PERUNIT ; process all units in each driver
51109
51110 00008CD7 5E POP SI ; restore pointer to device header
51111 00008CD8 1F POP DS
51112 00008CD9 E95AFF JMP PERDRV ; process all drivers in chain
51113
51114 CONTINIT:
51115 ; 24/04/2019
51116 ;;sub bp,33 ; set link in last DPB to -1
51117 ;;sub bp,61 ; 23/03/2024 (PCDOS 7.1)
51118 00008CDC 83ED3D sub bp,DPBSIZ ; back up to last dpb
51119 ; set last link offset & segment
51120 ; 23/03/2024 - Retro DOS v5.0
51121 %if 0
51122 ;mov word [bp+25],0FFFFh
51123 mov word [bp+DPB.NEXT_DPB],-1
51124 ;mov word [bp+27],0FFFFh
51125 mov word [bp+DPB.NEXT_DPB+2],-1
51126 %else
51127 ; 23/03/2024 (PCDOS 7.1 IBMDOS.COM)
51128 ;;;
51129 00008CDF B8FFFF mov ax,0FFFFh ; -1
51130 ;mov word [bp+25],ax
51131 00008CE2 894619 mov word [bp+DPB.NEXT_DPB],ax ; -1
51132 ;mov word [bp+27],ax
51133 00008CE5 89461B mov word [bp+DPB.NEXT_DPB+2],ax ; -1
51134 ;;;
51135 %endif
51136 ;;add bp,33
51137 ;;add bp,61 ; 23/03/2024 (PCDOS 7.1)
51138 00008CE8 83C53D add BP,DPBSIZ ; advance to free memory again
51139 ; the DPB chain is done.
51140 00008CEB 16 push ss
51141 00008CEC 1F pop ds
51142
51143 00008CED 89E8 mov ax,bp
51144 00008CEF E8F5FD call ParaRound ; round up to segment
51145
51146 00008CF2 8CDA mov dx,ds ; dx = dosdata segment
51147 00008CF4 01C2 add dx,ax ; dx = ds+ax first free segment
51148
51149 00008CF6 BB0F00 mov bx,0Fh
51150
51151 ; 24/05/2019
51152 ;mov cx,[ENDMEM]
51153 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51154 ; 17/12/2022
51155 ;mov cx,[ENDMEM]
51156 ; set seg inpacketto dosdata
51157 00008CF9 8C1E[A203] mov [DSKCHRET+3],ds ; mov [DOSSEG_INIT],ds
51158
51159 ; Patch in the segments of the interrupt vectors with current code segment.
51160 ; Also patch in the segment of the pointers in the dosdata area.
51161 ;
51162 ; Note: Formerly, temp_dosloc was initialized to -1 until after these
51163 ; calls were done. The procedure patch_misc_segments is called multiple
51164 ; times, and relies on temp_dosloc being initialized to -1 as a flag
51165 ; for the first invocation. Thus, we must set it to -1 for this call.
51166
51167 00008CFD 52 push dx ; preserve first free segment
51168
51169 00008CFE A1[A30A] mov ax,[TEMP_DOSLOC] ; ax = segment to patch in
51170 00008D01 8EC0 mov es,ax ; es = segment of DOS
51171 00008D03 C706[A30A]FFFF mov word [TEMP_DOSLOC],-1 ; -1 means first call to patch_misc_segments
51172
51173 00008D09 E8BC01 call patch_vec_segments ; uses AX as doscode segment
51174 00008D0C E8F101 call patch_misc_segments ; patch in segments for sharer and
51175 ; other tables with seg in ES.
51176 ; 17/12/2022
51177 ; cx = 0
51178 00008D0F 8C06[A30A] mov [TEMP_DOSLOC],es ; put back segment of dos code
51179
51180 00008D13 5A pop dx ; restore first free segment
51181
51182 ; We shall now proceed to set the offsets of the interrupt vectors handled
51183 ; by DOS to their appropriate values in DOSCODE. In case the DOS loads in
51184 ; HIMEM the offsets also will be patched to their appropriate values in the
51185 ; low_mem_stub by seg_reinit.
51186
51187 ;xor ax,ax ; 0
51188 ;mov ds,ax
51189 ;mov es,ax
51190 ; 17/12/2022
51191 ; cx = 0
51192 ;xor cx,cx ; 0
51193 00008D14 8ED9 mov ds,cx
51194 00008D16 8EC1 mov es,cx
51195
51196 ; set the segment of int 24 vector that was
51197 ; left out by patch_vec_segments above.
51198

```

```

51199 ; 17/12/2022
51200 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51201 ;%if 0
51202 ; 24/05/2019
51203 ;mov di,90h
51204 ;mov di,4*int_fatal_abort
51205 ;mov di,addr_int_fatal_abort
51206 00008D18 BF9200 mov di,addr_int_fatal_abort+2 ; 24/05/2019
51207
51208 00008D1B 36A1[A30A] mov ax,[ss:TEMP_DOSLOC]
51209 ;mov [di+2],ax ; int 24h segment
51210 00008D1F 8905 mov [di],ax ; 24/05/2019
51211
51212 ;mov di,82h
51213 ;mov di,INTBASE+2
51214
51215 ;%endif
51216 ; 17/12/2022
51217 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51218 ;mov di,90h
51219 ;mov di,4*int_fatal_abort
51220 ;mov di,addr_int_fatal_abort
51221 ;mov ax,[ss:TEMP_DOSLOC]
51222 ;mov [di+2],ax ; int 24h segment
51223 ;mov di,82h
51224 ;mov di,INTBASE+2
51225
51226 ; set default divide trap offset
51227
51228 ;mov word ptr ds:[0],offset doscode:divov
51229 00008D21 C7060000[D05F] mov word [0],DIVOV
51230
51231 ; set vectors 20-28 and 2a-3f to point to iret.
51232
51233 ;mov di,80h
51234 00008D27 BF8000 mov di,INTBASE
51235 ;mov ax,offset doscode:irett
51236 00008D2A B8[CB02] mov ax,IRETT
51237
51238 ; 17/12/2022
51239 ; cx = 0
51240 00008D2D B109 mov cx,9
51241 ;mov cx,9 ; set 9 offsets (skip 2 between each)
51242 ; sets offsets for ints 20h-28h
51243
51244 00008D2F AB iset1:
51245 stosw
51246 ;add di,2
51247 ; 20/09/2023
51248 00008D30 47 inc di
51249 00008D31 47 inc di
51250 00008D32 E2FB loop iset1
51251 00008D34 83C704 add di,4 ; skip vector 29h
51252
51253 ; mov cx,6 ; set 6 offsets (skip 2 between each)
51254 ; sets offsets for ints 2Ah-2Fh
51255 ;
51256 ; iset2:
51257 ; stosw
51258 ; add di,2
51259 ; loop iset2
51260
51261 ; 30h & 31h is the CPM call entry point whose segment address is set up by
51262 ; patch_vec_segments above. So skip it.
51263
51264 ; add di,8 ; skip vector 30h & 31h
51265
51266 ;;;
51267 ; 06/05/2019 - Retro DOS v4.0
51268 ;mov cx,5 ; set offsets for int 2Ah-2Eh
51269 00008D37 B105 mov cx,5 ; 28/06/2019
51270 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51271 ;mov cx,6
51272
51273 00008D39 AB iset2:
51274 stosw
51275 ;add di,2
51276 ; 20/09/2023
51277 00008D3A 47 inc di
51278 00008D3B 47 inc di
51279 00008D3C E2FB loop iset2
51280
51281 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51282 00008D3E 83C70C ; 17/12/2022
51283 add di,12 ; skip vectors 2Fh, 30h & 31h
51284 ;add di,8
51285 ;;;
51286 ; 17/12/2022
51287 00008D41 B10E mov cx,14
51288 ;mov cx,14 ; set 14 offsets (skip 2 between each)
51289 ; sets offsets for ints 32h-3Fh
51290
51291 00008D43 AB iset3:
51292 stosw
51293 ;add di,2
51294 ; 20/09/2023
51295 00008D44 47 inc di
51296 00008D45 47 inc di
51297 00008D46 E2FB loop iset3
51298
51299 ;if installed
51300 ; set the offset of int2f handler
51301 00008D48 C706BC00[3A07] mov word [0BCh],INT2F
51302 mov word [02Fh*4],INT2F
51303 ; set segment to doscode as we have to do int 2f to check for XMS
51304 mov ax,[ss:TEMP_DOSLOC] ; get segment of doscode
51305 00008D52 A3BE00 mov [0BEh],ax
51306 mov [(02Fh*4)+2],ax
51307 ;endif
51308 ; set up entry point call at vectors 30-31h. Note the segment of the
51309 ; long jump will be patched in by seg_reinit
51310
51311 00008D55 C606C000EA ;mov byte [C0h],0EAh
51312 ;mov byte [ENTRYPOINT],mi_long_jump
51313 00008D5A C706C100[CC02] mov word [ENTRYPOINT+1],CALL_ENTRY
51314
51315 00008D60 C7068000[C502] mov word [addr_int_abort],QUIT ; INT 20h
51316 00008D66 C7068400[F102] mov word [addr_int_command],COMMAND ; INT 21h
51317 00008D6C C706880000001 mov word [addr_int_terminate],100h ; INT 22h
51318 00008D72 89168A00 mov word [addr_int_terminate+2],dx
51319 00008D76 C7069400[2D05] mov word [addr_int_disk_read],ABSDRD ; INT 25h
51320 00008D7C C7069800[DF05] mov word [addr_int_disk_write],ABSDWRT ; INT 26h
51321 00008D82 C7069C00[3972] mov word [addr_int_keep_process],STAY_RESIDENT ; INT 27h
51322

```



```

51323 00008D88 16      push    ss
51324 00008D89 1F      pop     ds
51325
51326                ; 24/05/2019
51327                ;push    ss
51328                ;pop     es
51329                ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51330                ; 17/12/2022
51331                ;push    ss
51332                ;pop     es
51333
51334 00008D8A 52      push    dx                ; remember address of arena
51335
51336 00008D8B 42      inc     dx                ; leave room for arena header
51337                ;mov     [330h],dx
51338 00008D8C 8916[3003] mov     [CurrentPDB],dx        ; set current pdb
51339
51340 00008D90 31FF      xor     di,di                ; point es:di at end of memory
51341 00008D92 8EC2      mov     es,dx                ; ...where psp will be
51342 00008D94 31C0      xor     ax,ax
51343                ;mov     cx,80h                ; psp is 128 words
51344                ; 17/12/2022
51345 00008D96 B180      mov     cx,128 ; 28/06/2019
51346                ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51347                ;mov     cx,128
51348
51349 00008D98 F3AB      rep     stosw                ; zero out psp area
51350 00008D9A A1[4603]    mov     ax,[ENDMEM]
51351
51352                ; 17/12/2022
51353                ; cx = 0
51354 00008D9D E82589    call    SETMEM                ; build psp at dx; ax is memory size
51355
51356                ; ds, es now point to PSP
51357
51358 00008DA0 16      push    ss
51359 00008DA1 1F      pop     ds
51360
51361                ;mov     di,24
51362 00008DA2 BF1800    mov     di,PDB.JFN_TABLE        ; es:di -> pdb_jfn_table in psp
51363 00008DA5 31C0      xor     ax,ax
51364 00008DA7 AB      stosw
51365 00008DA8 AA      stosb                ; 0,1 and 2 are con device
51366 00008DA9 B0FF      mov     al,0FFh
51367                ;mov     cx,FILPERPROC-3 ; 17
51368                ; 17/12/2022
51369                ; cx = 4
51370 00008DAB B111      mov     cx,FILPERPROC-3 ; 17
51371 00008DAD F3AA      rep     stosb                ; rest are unused
51372
51373 00008DAF 16      push    ss
51374 00008DB0 07      pop     es
51375                ; must be set to print messages
51376 00008DB1 8C1E[2C00] mov     [SFT_ADDR+2],ds
51377
51378                ; after this point the char device functions for con will work for
51379                ; printing messages
51380
51381                ; 24/04/2019 - Retro DOS v4.0
51382
51383                ; 12/05/2019
51384                ;
51385                ;write_version_msg:
51386                ;
51387                ; if (not ibm)
51388                ; mov     si,offset doscode:header
51389                ; mov     si,HEADER
51390                ;outmes:
51391                ; lodsb    cs:byte ptr [si]
51392                ; cs
51393                ; lodsb
51394                ; cmp     al,"$"
51395                ; je      short outdone
51396                ; call    OUTT
51397                ; jmp     short outmes
51398                ;outdone:
51399                ; push    ss                ; out stomps on segments
51400                ; pop     ds
51401                ; push    ss
51402                ; pop     es
51403                ; endif
51404
51405                ; at this point es is dosdata
51406
51407                ; Fill in the segment addresses of sysinitvar and country_cdpq
51408                ; in sysinittable (ms_data.asm)
51409
51410                ;mov     si,0D28h
51411 00008DB5 BE[580D]    mov     si,SysInitTable
51412
51413                ; 17/12/2022
51414                ; ds = es = ss
51415
51416                ; 23/03/2024
51417                ;;;
51418                ; 17/12/2022
51419                ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51420
51421                ;mov     [es:si+6],es
51422                ;mov     [es:si+SYSI_EXT.Country_Tab+2],es
51423                ;mov     [es:si+2],es
51424                ;mov     [es:si+SYSI_EXT.SysInitVars+2],es
51425
51426 00008DB8 8C4406    mov     [si+SYSI_EXT.Country_Tab+2],es
51427 00008DBB 8C4402    mov     [si+SYSI_EXT.SysInitVars+2],es
51428
51429                ; buffhead -> dosdata:hashinitvar
51430
51431                ;mov     [es:BUFFHEAD+2],es        ; BUGBUG - unused, remove this
51432 00008DBE 8C06[3A00]    mov     [BUFFHEAD+2],es
51433                ;mov     si,offset dosdata:hashinitvar ; and all other references
51434                ;mov     si,6Dh
51435 00008DC2 BE[6D00]    mov     si,HASHINITVAR
51436                ;mov     [es:BUFFHEAD],si
51437 00008DC5 8936[3800]    mov     [BUFFHEAD],si
51438
51439 00008DC9 5A      pop     dx                ; restore address of arena
51440
51441                ;mov     [032Ch+2],dx
51442 00008DCA 8916[2E03]    mov     [DMAADD+2],dx
51443
51444                ;mov     [es:arena_head],dx
51445 00008DCE 8916[2400]    mov     [arena_head],dx
51446                ;;;

```

```

51447
51448 00008DD2 8EDA          mov     ds,dx
51449
51450          ;mov     byte [0], 'z'
51451 00008DD4 C60600005A      mov     byte [ARENA.SIGNATURE],arena_signature_end
51452          ;mov     word [1],0
51453 00008DD9 C70601000000    mov     word [ARENA.OWNER],arena_owner_system
51454
51455 00008DDF 36A1[4603]      mov     ax,[ss:ENDMEM]
51456 00008DE3 29D0          sub     ax,dx
51457 00008DE5 48              dec     ax
51458          ;mov     [3],ax ; 23/03/2024
51459 00008DE6 A30300          mov     [ARENA.SIZE],ax
51460
51461          ; point to sft 0
51462
51463          ;mov     di,offset dosdata:sftabl + sftable
51464          ;mov     di,SFTABL+6
51465 00008DE9 BF[D200]      mov     di,SFTABL+SFT.SFTable
51466 00008DEC B80300          mov     ax,3
51467 00008DEF AB              stosw             ; adjust refcount
51468
51469          ; es:di is shared data area i.e., es:di -> dosdata:sysinttable
51470
51471          ;mov     di,offset dosdata:sysinttable
51472          ;;mov     di,0D28h
51473          ;mov     di,0D58h ; 23/03/2024 (PCDOS 7.1)
51474 00008DF0 BF[580D]      mov     di,SysInitTable
51475
51476 00008DF3 42              inc     dx             ; advance dx from arena to psp
51477 00008DF4 8EDA          mov     ds,dx             ; point ds to psp
51478
51479          ; pass the address os seg_reinit
51480          ; in dx
51481 00008DF6 BA[648E]      mov     dx,seg_reinit
51482 00008DF9 B9[BA87]      mov     cx,exepatch_start
51483 00008DFC 81E9[0000]    sub     cx,_$STARTCODE    ; cx = (doscode - exepatch) - dosinit
51484
51485 00008E00 B8[E48A]      mov     ax,SYSBUF
51486 00008E03 2D[0000]      sub     ax,_$STARTCODE    ; ax = size of doscode - dosinit
51487
51488 00008E06 368B26[8405]    mov     sp,[ss:USER_SP]    ; use ss override for next 2
51489 00008E0B 368E16[8605]    mov     ss,[ss:USER_SS]
51490
51491 00008E10 CB              retf
51492
51493          ;
51494          ; END OF DOSINIT
51495          ;
51496          ;-----
51497
51498          CHARINIT:
51499          ; 11/04/2024
51500          ; 23/03/2024 - Retro DOS v5.0
51501          ; 24/04/2019 - Retro DOS v4.0
51502          ; 07/07/2018 - Retro DOS v3.0
51503          ;mov     byte [ss:035Ah],26 ; 1Ah ; MSDOS 6.22
51504 00008E11 36C606[5A03]19  MOV BYTE [SS:DEVCALL_REQLEN],DINITHL ; 25 ; PCDOS 7.1
51505          ;mov     byte [ss:035Bh],0
51506 00008E17 36C606[5B03]00  MOV BYTE [SS:DEVCALL_REQUNIT],0
51507          ;mov     byte [ss:035Ch],0
51508 00008E1D 36C606[5C03]00  MOV BYTE [SS:DEVCALL_REQFUNC],DEVINIT
51509          ;mov     word [ss:035BD],0
51510 00008E23 36C706[5D03]0000  MOV WORD [SS:DEVCALL_REQSTAT],0
51511 00008E2A 06              PUSH     ES
51512 00008E2B 53              PUSH     BX
51513 00008E2C 50              PUSH     AX
51514 00008E2D BB[5A03]      MOV BX,DEVCALL
51515          ;PUSH     CS
51516 00008E30 16              PUSH     SS ; 30/04/2019
51517 00008E31 07              POP     ES
51518 00008E32 E85CC4      CALL     DEVIOCALL2
51519 00008E35 58              POP     AX
51520 00008E36 5B              POP     BX
51521 00008E37 07              POP     ES
51522 00008E38 C3              RETN
51523
51524          ; 25/04/2019 - Retro DOS v4.0
51525
51526          ;-----
51527          ;
51528          ; check_XMM: routine to check presence of XMM driver
51529          ;
51530          ; Exit:  Sets up the XMM entry point in XMMcontrol in DOSDATA
51531          ;
51532          ; USED:  none
51533          ;-----
51534
51535          check_XMM: ; proc near
51536          ;
51537          ; determine whether or not an XMM driver is installed
51538          ;
51539          ;
51540 00008E39 50              push     ax
51541          ;mov     ax,(XMM_MULTIPLEX<<8)+XMM_INSTALL_CHECK
51542 00008E3A B80043      mov     ax,4300h
51543 00008E3D CD2F          int     2Fh
51544          ; - Multiplex - XMS - INSTALLATION CHECK
51545          ; Return: AL = 80h XMS driver installed
51546          ; AL <> 80h no driver
51547 00008E3F 3C80          cmp     al,80h           ; Q: installed
51548 00008E41 751D          jne     short cXMM_no_driver ; N: set error, quit
51549
51550          ; get the XMM control functions entry point, save it, we
51551          ; need to call it later.
51552          ;
51553          ;
51554          ;
51555          ;
51556          ;
51557          ;
51558 00008E47 B81043      mov     ax,(XMM_MULTIPLEX<<8)+XMM_FUNCTION_ADDR
51559 00008E4A CD2F          mov     ax,4310h
51560          int     2Fh
51561          ; - Multiplex - XMS - GET DRIVER ADDRESS
51562          ; Return: ES:BX -> driver entry point
51563 00008E4C 2E8E1E[0700]    mov     ds,[cs:DosDSeg]
51564
51565 00008E51 891E[7B10]      mov     [XMMcontrol],bx
51566 00008E55 8C06[7D10]      mov     [XMMcontrol+2],es
51567          cXMMExit:
51568 00008E59 F8              clc
51569 00008E5A 07              pop     es
51570 00008E5B 1F              pop     ds

```



```

51571 00008E5C 5A      pop     dx
51572 00008E5D 5B      pop     bx
51573 00008E5E 58      pop     ax
51574 00008E5F C3      retn                ; done
51575
51576                ; set carry if XMM driver not present
51577
51578 cXMM_no_driver:
51579 00008E60 F9      stc
51580 00008E61 58      pop     ax
51581 00008E62 C3      retn
51582
51583 -----
51584
51585 Procedure Name : seg_reinit
51586
51587 Inputs      : ES has final dos code location
51588              AX = 0 / 1
51589
51590 Outputs     : Patch in the sharer and other tables with seg in ES
51591              if AX = 0
51592                  if first entry
51593                      patch segment & offset of vectors with stub
51594                      and stub with segment in ES
51595                  else
51596                      patch stub with segment in ES
51597
51598              else if AX = 1
51599                  patch segment of vectors with segment in ES
51600
51601 NOTE        : This routine can be called at most twice!
51602
51603 Regs Mod.   : es, ax, di, cx, bx
51604 -----
51605
51606 00008E63 00      num_entry: db      0                ; keeps track of the # of times this routine
51607                                     ; has been called. (0 or 1)
51608
51609                ; 04/11/2022 - Retro DOS v4.0 (ref: MSDOS 5.0)
51610                ; MSDOS 5.0 MSDOS.SYS - DOSCODE:0BAB7h
51611                ; 25/05/2019 - Retro DOS v4.0 (ref: MSDOS 6.21)
51612                ; MSDOS 6.21 MSDOS.SYS - DOSCODE:0BDA5h
51613
51614                ; 23/03/2024 - Retro DOS v5.0 (ref: PCDOS 7.1)
51615                ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0D07Eh
51616
51617 seg_reinit: proc far
51618 00008E64 1E      push     ds
51619
51620 00008E65 2E8E1E[0700] mov     ds,[cs:DosDSeg]
51621
51622 00008E6A E89300   call     patch_misc_segments ; patch in segments for sharer and
51623                                     ; other tables with seg in ES.
51624
51625                ; 17/12/2022
51626                ; cx = 0
51627 00008E6D 39C8     cmp     ax,cx ; 0
51628 00008E6F 754A     jne     short patch_vec_seg ; patch vectors with segment in es
51629                ; 23/03/2024
51630                ; or ax, ax
51631                ; jnz short patch_vec_seg
51632
51633                ; 23/03/2024
51634                ; cmp [cs:num_entry],al ; 0
51635                ; 17/12/2022
51636 00008E71 2E380E[638E] cmp     [cs:num_entry],cl ; 0
51637                ; cmp byte [cs:num_entry],0 ; Q: is it the first call to this
51638 00008E76 7508     jne     short second_entry ; N: just patch the stub with
51639                                     ; segment in ES
51640                                     ; Y: patch the vectors with stub
51641
51642 00008E78 8CD8     mov     ax,ds
51643 00008E7A E84B00   call     patch_vec_segments ; patch the segment of vectors
51644 00008E7D E8CA00   call     patch_offset      ; patch the offsets of vectors
51645                                     ; with those in the stub.
51646                ; 17/12/2022
51647                ; cx = 0
51648 00008E80 8CC0     second_entry:
51649                mov     ax,es                ; patch the stub with segment in es
51650
51651                ; mov di,OFFSET DOSDATA:DOSINTTABLE
51652                ; mov di,1062h ; (same table addr for MSDOS 5.0 and MSDOS 6.21)
51653                ; mov di,0F7Ah ; 23/03/2024 ; PCDOS 7.1
51654 00008E82 BF[7A0F]   mov     di,DOSINTTABLE
51655
51656                ; 17/12/2022
51657                ; cx = 0
51658                ; mov cx,9
51659                ; mov cl,9
51660                ; 23/03/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
51661                ;
51662 00008E85 B108     mov     cx,8
51663                mov     cl,8
51664                ;
51665 00008E87 1E      push     ds
51666 00008E88 07      pop     es                ; es:di -> DOSINTTABLE
51667
51668 dosinttabloop:
51669                add     di,2
51670                ; 19/06/2023
51671 00008E89 47      inc     di
51672 00008E8A 47      inc     di
51673 00008E8B AB      stosw
51674 00008E8C E2FB     loop    dosinttabloop
51675
51676                ; For ROMDOS, this routine will only be called when the DOS wants to
51677                ; use the HMA, so we don't want to check CS
51678
51679 00008E8E 3D00F0   cmp     ax,0F000h          ; Q: is the DOS running in the HMA
51680 00008E91 722D     jb     short sr_done      ; N: done
51681
51682 00008E93 E8A3FF   call     check_XMM        ; Y: set up the XMS entry point
51683 00008E96 7228     jc     short sr_done      ; failed to set up XMS do not do
51684                                     ; A20 toggling in the stub.
51685                ; 17/12/2022
51686                ; cx = 0
51687 00008E98 E82A01   call     patch_in_nops    ; enable the stub to check A20 state
51688
51689 ; M021-
51690                ; mov byte [1211h],1
51691 00008E9B C606[660D]01 mov     byte [0D66h],1 ; 23/03/2024 ; PCDOS 7.1
51692                mov     byte [DosHashMA],1 ; set flag telling DOS control of HMA
51693
51694                ; set pointer to the routine that
51695                ; patches buggy exepacked code.

```

```

51695 ;mov [FixExePatch],offset DOSCODE:ExePatch
51696 00008EA0 C706[670D][0A89] mov word [FixExePatch],ExePatch
51697 ; M068: set pointer to the routine
51698 ; M068: that detects copy protected
51699 ; M068: apps
51700 ;mov [ChkCopyProt],offset DOSCODE:IsCopyProt
51701 00008EA6 C706[6100][AD8A] mov word [ChkCopyProt],IsCopyProt
51702 ; 23/03/2024 - PC DOS 7.1 IBMDOS.COM - DOSCODE:0D0C7h
51703 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0BDF1h)
51704 ;
51705 00008EAC E844FC call whatCPUType
51706 00008EAF 3C01 cmp al,1
51707 00008EB1 750D jne short sr_done ; we need Rational Patch only on 286 systems
51708 00008EB3 C706[690D][348A] mov word [RationalPatchPtr],RationalPatch
51709 ; 19/09/2023
51710 jmp short sr_done
51711 00008EB9 EB05
51712 ; 23/03/2024
51713 patch_vec_seg: ; patch vectors with segment in es
51714 mov ax,es
51715 00008EBB 8CC0 call patch_vec_segments ; patch in DOSCODE for the segments
51716 00008EBD E80800 ; NOTE we don't have to patch the
51717 ; offsets as they have been already
51718 ; set to the doscode offsets at
51719 ; DOSINIT.
51720 sr_done:
51721 mov byte [cs:num_entry],1
51722 00008EC0 2EC606[638E]01 pop ds
51723 00008EC6 1F retf ; ! far return !
51724 00008EC7 CB
51725
51726 ;-----
51727 ; Procedure Name : patch_vec_segments
51728 ;
51729 ; Inputs : ax -> has segment address to patch in
51730 ; ds -> DOSDATA
51731 ;
51732 ; Outputs : Patches in AX as the segment for the following vectors:
51733 ;
51734 ; 0,20-28,3a-3f
51735 ;
51736 ; Regs. Mod. : DI,CX,DX,AX
51737 ;-----
51738
51739 patch_vec_segments:
51740 push es
51741 xor cx,cx ; 0
51742 mov es,cx
51743
51744 ;mov di,82h
51745 mov di,INTBASE+2 ; di -> segment of int 20h vector
51746
51747 mov [es:2],ax ; segment of default divide trap handler
51748 ; set vectors 20h & 21h
51749 ; 04/11/2022
51750 ;mov cx,2
51751 ; 17/12/2022
51752 ;mov cl,2
51753 ps_set1:
51754 stosw ; int 20h segment
51755 ;add di,2
51756 ; 17/12/2022
51757 inc di
51758 inc di
51759 ;loop ps_set1
51760 ; 17/12/2022
51761 stosw ; int 21h segment
51762 ;inc di
51763 ;inc di
51764 ;add di,4
51765 add di,6 ; * ; skip int 22h vector
51766
51767 stosw ; set int 23h
51768 add di,6 ; skip int 24h
51769 ; set vectors 25h-28h and 2Ah-3Fh
51770 ; 04/11/2022
51771 ;mov cx,4
51772 ; 17/12/2022
51773 mov cl,4
51774 ps_set2:
51775 stosw
51776 ;add di,2
51777 ; 17/12/2022
51778 inc di
51779 inc di
51780 loop ps_set2
51781 add di,4
51782 ; skip int 29h vector (fast con) as it may
51783 ; already be set.
51784 ; 04/11/2022
51785 ;mov cx,6
51786 ; 17/12/2022
51787 mov cl,6
51788 ps_set3:
51789 stosw
51790 ;add di,2
51791 ; 17/12/2022
51792 inc di
51793 inc di
51794 loop ps_set3
51795 ; 30h & 31h is the CPM call entry point whose segment address is set up by
51796 ; below. So skip it.
51797 add di,8 ; skip vector 30h & 31h
51798 ; 04/11/2022
51799 ;mov cx,14
51800 ; 17/12/2022
51801 mov cl,14
51802 ps_set4:
51803 stosw
51804 ;add di,2
51805 ; 17/12/2022
51806 inc di
51807 inc di
51808

```

```

51819 00008EF8 E2FB      loop    ps_set4
51820
51821      ; set offset of int2f
51822
51823      ;if installed
51824      ; mov    word ptr es:[02fh * 4],offset doscode:int2f
51825      ;endif
51826      ;mov     [es:0C3h],ax
51827 00008EFA 26A3C300    mov     [es:ENTRYPOINT+3],ax
51828      ; 17/12/2022
51829      ; cx = 0
51830 00008EFE 07      pop     es
51831 00008EFF C3      retn
51832
51833      ;-----
51834      ;
51835      ; Procedure Name : patch_misc_segments
51836      ;
51837      ; Inputs      : es = segment to patch in
51838      ;              ds = dosdata
51839      ;
51840      ; outputs     : patches in the sharer and other tables in the dos
51841      ;              with right dos code segment in es
51842      ;
51843      ; Regs Mod    : DI,SI,CX
51844      ;
51845      ;-----
51846
51847      patch_misc_segments:
51848
51849      push    bx
51850      push    es
51851      push    ax
51852
51853      mov     ax,es          ; ax -> DOS segment
51854
51855      push    ds
51856      pop     es            ; es -> DOSDATA
51857
51858      ; initialize the jump table for the sharer...
51859
51860      ;mov     di,offset dosdata:jshare
51861      ;mov     di,90h
51862      mov     di,jshare
51863      ;mov     bx,[0AAAh]
51864      ;mov     bx,[0AA3h] ; 23/03/2024 ; PC DOS 7.1
51865 00008F0A 8B1E[A30A]    mov     bx,[TEMP_DOSLOC] ; bx = location to which the share
51866      ;              ; table was patched during the first
51867      ;              ; call to this routine
51868      mov     cx,15
51869      jumptabloop:
51870      ;add     di,2          ; skip offset
51871      ; 17/12/2022
51872      inc     di
51873      inc     di
51874      cmp     bx,-1 ; 0FFFFh ; Q: is this called for the 1st time
51875      jne     short share_patch ; Y: patch in sharer table
51876      ; N:
51877      cmp     bx,[es:di] ; Q: has share been installed
51878      jne     short no_share_patch ; Y: don't patch in sharer table
51879      share_patch:
51880      stosw
51881      no_share_patch:
51882      loop    jumptabloop
51883      ; BUGBUG patching the country info
51884      ; with dosdata can be done inline
51885      ; in dosinit.
51886      ; for dos 3.3 country info
51887      ; table address
51888
51889      ;mov     si,offset dosdata:country_cdpdg
51890      ;mov     si,122Ah
51891 00008F20 BE[2A12]    mov     si,COUNTRY_CDPG
51892
51893      ; initialize double word
51894      ; pointers with dosdata in ds
51895
51896      ;mov     [si+4Fh],ds
51897      ;mov     [si+54h],ds
51898      ;mov     [si+59h],ds
51899      ;mov     [si+5Eh],ds
51900      ;mov     [si+80h],ds
51901      ;mov     [si+63h],ds
51902      mov     [si+DOS_CCDPG.ccUcase_ptr+2],ds
51903      mov     [si+DOS_CCDPG.ccFileUcase_ptr+2],ds
51904      mov     [si+DOS_CCDPG.ccFileChar_ptr+2],ds
51905      mov     [si+DOS_CCDPG.ccCollate_ptr+2],ds
51906      mov     [si+DOS_CCDPG.ccMono_ptr+2],ds
51907      mov     [si+DOS_CCDPG.ccBCS_ptr+2],ds
51908
51909      ; fastopen routines are in doscode
51910      ; so patch with doscode seg in ax
51911
51912      ;mov     si,offset dosdata:fastopentable
51913      ;mov     si,0D30h
51914      mov     si,FastOpenTable
51915
51916      ; 17/12/2022
51917      ; bx = [TEMP_DOSLOC]
51918      cmp     bx,-1
51919      cmp     word [TEMP_DOSLOC],-1 ; Q: first time
51920      je      short fast_patch ; Y: patch segment
51921      ;mov     cx,[TEMP_DOSLOC]
51922      ; Q: has fastopen patched in it's
51923      ; segment
51924
51925      ; 17/12/2022
51926      cmp     bx,[si+fastopen_entry.name_caching+2]
51927      ;cmp     cx,[si+4]
51928      ;cmp     cx,[si+fastopen_entry.name_caching+2]
51929      jne     short no_fast_patch ; Y: don't patch in doscode seg
51930      fast_patch:
51931      ;mov     [si+4],ax
51932      mov     [si+fastopen_entry.name_caching+2],ax
51933      no_fast_patch:
51934      ; 17/12/2022
51935      ; cx = 0
51936      pop     ax
51937      pop     es
51938      pop     bx
51939      retn
51940
51941      ;-----
51942      ;
51943      ; Procedure Name : patch_offset

```

```

51943 ;
51944 ; Inputs : NONE
51945 ;
51946 ; Outputs : Patches in the offsets in the low_mem_stub for vectors
51947 ; 0,20-28,3a-3f, and 30,31
51948 ;
51949 ;
51950 ; Regs. Mod : AX,DI,CX
51951 ;-----
51952
51953 patch_offset:
51954 00008F4A 06 push es ; preserve es
51955
51956 00008F4B 31C0 xor ax,ax
51957 00008F4D 8EC0 mov es,ax
51958
51959 ;mov word ptr es:[0],offset dosdata:ldivov ; set default divide trap address
51960 ;mov word [es:0],108Ah
51961 ;mov word [es:0],0F9Eh ; 23/03/2024 ; PC DOS 7.1
51962 00008F4F 26C7060000[9E0F] mov word [es:0],ldivov
51963
51964 ;mov di,80h
51965 00008F56 BF8000 mov di,INTBASE ; di-> offset of int 20 handler
51966 ;mov ax,offset dosdata:lirett
51967 ;mov ax,10DAh
51968 00008F59 B8[6E10] mov ax,lirett
51969
51970 ; set vectors 20h & 21h to point to iret.
51971 ; 17/12/2022
51972 ; cx = 0
51973
51974 ;mov cx,2 ; set 2 offsets (skip 2 between each)
51975 po_iset1:
51976 00008F5C AB stosw ; int 20h offset
51977 ;add di,2 ; *
51978 ;loop po_iset1
51979 ; 17/12/2022
51980 00008F5D 47 inc di
51981 00008F5E 47 inc di
51982 00008F5F AB stosw ; int 21h offset
51983
51984 ;add di,4 ; skip vector 22h
51985 00008F60 83C706 ; 17/12/2022
51986 add di,6 ; *
51987 00008F63 AB stosw ; set offset of 23h
51988 ;add di,6 ; skip 24h
51989 ; 19/09/2023
51990 00008F64 83C712 add di,18 ; skip 23h segment and int 24-25-26-27h
51991
51992 ; set vectors 25-28 and 2a-3f to iret.
51993 ; 04/11/2022
51994 ;mov cx,4 ; set 4 offsets (skip 2 between each)
51995 ; 19/09/2023
51996 ; 17/12/2022
51997 ;mov cl,4 ; sets offsets for ints 25h-28h
51998 po_iset2:
51999 00008F67 AB stosw ; set offset for int 28h ; 19/09/2023
52000 ;add di,2
52001 ; 19/09/2023
52002 ; 17/12/2022
52003 ;inc di
52004 ;inc di
52005 ; 19/09/2023
52006 ;loop po_iset2
52007
52008 ;add di,4 ; skip vector 29h
52009 ; 19/09/2023
52010 00008F68 83C706 add di,6 ; skip int 28h segment and int 29h ; 19/09/2023
52011
52012 ; 04/11/2022
52013 ;mov cx,6 ; set 6 offsets (skip 2 between each)
52014 ; 17/12/2022
52015 ;mov cl,6 ; sets offsets for ints 2Ah-2Fh
52016 00008F6B B105 mov cl,5 ; sets offsets for ints 2Ah-2Eh
52017 po_iset3:
52018 00008F6D AB stosw
52019 ;add di,2
52020 ; 17/12/2022
52021 00008F6E 47 inc di
52022 00008F6F 47 inc di
52023 00008F70 E2FB loop po_iset3
52024
52025 ; 30h & 31h is the CPM call entry point whose offset address is set up by
52026 ; below. So skip it.
52027
52028 ;add di,8 ; skip vector 30h & 31h
52029 ; 17/12/2022
52030 00008F72 83C70C add di,12 ; skip vector 2Fh, 30h & 31h
52031
52032 ; 04/11/2022
52033 ;mov cx,14 ; set 14 offsets (skip 2 between each)
52034 ; sets offsets for ints 32h-3Fh
52035 ; 17/12/2022
52036 00008F75 B10E mov cl,14 ; 26/06/2019
52037 po_iset4:
52038 00008F77 AB stosw
52039 ;add di,2
52040 ; 17/12/2022
52041 00008F78 47 inc di
52042 00008F79 47 inc di
52043 00008F7A E2FB loop po_iset4
52044
52045 ;if installed
52046 ;mov word ptr es:[02fh * 4],offset dosdata:lint2f
52047 ;mov word [es:0BCh],10C6h ; (MSDOS 5.0 & 6.21)
52048 ;mov word [es:0BCh],0FDAh ; PC DOS 7.1 ; 23/03/2024
52049 00008F7C 26C706BC00[DA0F] mov word [es:(2Fh*4)],lint2f
52050 ;endif
52051
52052 ; set up entry point call at vectors 30-31h
52053 ;mov byte [es:0C0h],0EAh
52054 00008F83 26C606C000EA mov byte [es:ENTRYPOINT],mi_long_jump
52055 ;mov word [es:0C1h],10B0h ; 0FE4h ; 23/03/2024 (PC DOS 7.1)
52056
52057 00008F89 26C706C100[E40F] mov word [es:ENTRYPOINT+1],lcall_entry
52058
52059 ; 19/09/2023
52060 ;mov word [es:80h],1094h ; 0FA8h ; 23/03/2024
52061 00008F90 26C7068000[A80F] mov word [es:addr_int_abort],lquit ; int 20h
52062 ;mov word [es:84h],109Eh ; 0FB2h
52063 00008F97 26C7068400[B20F] mov word [es:addr_int_command],lcommand ; int 21h
52064 ;mov word [es:94h],10A8h ; 0FBCh
52065 00008F9E 26C7069400[BC0F] mov word [es:addr_int_disk_read],labsdrd ; int 25h
52066 ;mov word [es:98h],10B2h ; 0FC6h

```

```

52067 00008FA5 26C7069800[C60F]      mov     word [es:addr_int_disk_write],labsdwrt      ; int 26h
52068                                     ;mov     word [es:9Ch],10BCh ;; 0Fb0h
52069 00008FAC 26C7069C00[D00F]      mov     word [es:addr_int_keep_process],lstay_resident ; int 27h
52070                                     ; 17/12/2022
52071                                     ; CX = 0
52072                                     pop     es              ; restore es
52073 00008FB3 07                      retn
52074 00008FB4 C3
52075
52076
52077
52078
52079
52080
52081
52082
52083
52084
52085
52086
52087
52088
52089
52090
52091
52092
52093
52094
52095
52096
52097
52098
52099
52100
52101
52102
52103
52104
52105
52106
52107
52108 00008FB5 [9E0F]
52109 00008FB7 [A80F]
52110 00008FB9 [B20F]
52111 00008FBB [BC0F]
52112 00008FBD [C60F]
52113 00008FBF [D00F]
52114 00008FC1 [DA0F]
52115 00008FC3 [E40F]
52116
52117
52118
52119
52120 00008FC5 50
52121 00008FC6 56
52122 00008FC7 BE[B58F]
52123 00008FCA B89090
52124
52125
52126
52127
52128 00008FCD B108
52129
52130 00008FCF 2E8B3C
52131 00008FD2 AB
52132
52133
52134 00008FD3 46
52135 00008FD4 46
52136 00008FD5 E2F8
52137 00008FD7 5E
52138 00008FD8 58
52139 00008FD9 C3
52140
52141
52142
52143
52144
52145
52146
52147
52148
52149
52150
52151
52152
52153
52154
52155
52156
52157
52158
52159
52160
52161
52162
52163
52164
52165
52166
52167 00008FDA 00
52168
52169
52170
52171
52172
52173
52174
52175
52176
52177
52178
52179
52180
52181
52182
52183
52184
52185
52186
52187
52188
52189
52190

```

```

; 17/12/2022
; CX = 0
pop     es              ; restore es
retn

-----
;
; Procedure Name :      patch_in_nops
;
; Entry          :      ES -> DOSDATA
;
; Regs Mod       :      cx, di
;
; Description:
; This routine patches in 2 nops at the offsets specified in
; patch_table. This basically enables the low mem stub to start
; making XMS calls.
;
-----

; 04/11/2022
; MSDOS 5.0 MSDOS.SYS - DOSCODE:0BC50h

; 23/03/2024
; MSDOS 6.22 MSDOS.SYS - DOSCODE:0BF42h
; 23/03/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMDOS.COM - DOSCODE:0D1E9h

patch_table:      ; label byte
;dw     offset dosdata:i0patch
;dw     offset dosdata:i20patch
;dw     offset dosdata:i21patch
;dw     offset dosdata:i25patch
;dw     offset dosdata:i26patch
;dw     offset dosdata:i27patch
;dw     offset dosdata:i2fpatch
;dw     offset dosdata:cmpmpatch
dw        i0patch
dw        i20patch
dw        i21patch
dw        i25patch
dw        i26patch
dw        i27patch
dw        i2fpatch
dw        cmpmpatch

patch_table_size equ ($-patch_table)/2

patch_in_nops:
push     ax
push     si
mov      si,patch_table
mov      ax,9090h ; nop, nop
; 17/12/2022
; CX = 0
;mov    cx,8
;mov    cx,patch_table_size ; 8
mov      cl,patch_table_size ; 8
pin_loop:
mov      di,[cs:si]
stosw
add      si,2
; 17/12/2022
inc      si
inc      si
loop    pin_loop
pop      si
pop      ax
retn

; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;
; MSDOS 5.0 - MSDOS.SYS offset BC77h, file offset 7EA7h
;
-----
; 23/03/2024
; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0BF69h)
; PCDOS 7.1 IBMDOS.COM - DOSCODE:0D20Fh

; 05/12/2022 - temporary ; (paragraph alinment)
DOSCODE_END:
; 23/03/2024
;;times 9 db 0 ; db 9 dup(0)
;; 18/12/2022
;dw     0 ; times 2 db 0

; 23/03/2024
;times 7 db 0 ; MSDOS 6.22 MSDOS.SYS

; 23/03/2024 - Retro DOS v5.0
;db     0 ; PCDOS 7.1 IBMDOS.COM
; 01/07/2024
;times 12 db 0

; 01/07/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMDOS.COM - DOSCODE:0D0Fh
db      0

;align 16
; DOSCODE:BC80h (MSDOS 5.0 MSDOS.SYS file offset 7EB0h)
; MSDOS.SYS file offset: 32432 (start of DOSDATA)

; 23/03/2024
; PCDOS 7.1 IBMDOS.COM - DOSCODE:0D210h
; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0BF70h)

; -----

;memstrt label word
;
; MSDOS 6.21 - MSDOS.SYS offset BF69h, file offset 8189h
;
-----

MEMSTRT: ; 25/04/2019 - Retro DOS v4.0

; if not ROMDOS, then we close the dos code segment, otherwise we close
; the dos initialization segment

;ifndef ROMDOS

```

```

52191 ;doscode ends
52192
52193 ;else
52194
52195 ;;dosinitseg ends
52196
52197 ;endif ; ROMDOS
52198
52199 ;=====
52200
52201 ; DPUBLIC <ParaRound, cXMM_no_driver, cXMMexit, char_init_loop, charinit>
52202 ; DPUBLIC <check_XMM, continit, dosinttabloop, endlst>
52203 ; DPUBLIC <initiret, iset1, iset2, jumptabloop, nxtentry>
52204 ; DPUBLIC <notmax, patch_offset, perdrv>
52205 ; DPUBLIC <perunit, po_iset1, po_iset2, po_iset3>
52206 ; DPUBLIC <ps_set1, ps_set2, ps_set3, seg_reinit>
52207 ; DPUBLIC <sr_done, version_fake_table, xxx>
52208
52209 ;; burası doscode sonu
52210
52211 ;=====
52212 ; DOSDATA
52213 ;=====
52214 ; 29/04/2019 - Retro DOS 4.0
52215 ; 24/03/2024 - Retro DOS 5.0
52216
52217 ;[BITS 16]
52218
52219 ;[ORG 0]
52220
52221 ; 25/04/2019 - Retro DOS v4.0
52222
52223 ;=====
52224 ; DOSDATA - MSDOS 6.21 - MSDOS.SYS Offset 0BF70h, file offset 8190h
52225 ;=====
52226
52227 ;align 16
52228 ; ; DOSDATA (MSDOS.SYS kernel DATA) segment starts here...
52229 ; ; (4970 bytes for MSDOS 6.21)
52230 ; ; (4976 bytes for Retro DOS v4.0, 25/05/2019 modification.)
52231
52232 ;=====
52233 ; MSCONST.ASM (MSDOS 6.0, 1991)
52234 ;=====
52235 ; 03/11/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 MSDOS.SYS)
52236 ; 25/04/2019 - Retro DOS 4.0 (MSDOS 6.21)
52237 ; 16/07/2018 - Retro DOS 3.0
52238
52239 ;Break <Initialized data and data used at DOS initialization>
52240 ;-----
52241
52242 ; We need to identify the parts of the data area that are relevant to tasks
52243 ; and those that are relevant to the system as a whole. Under 3.0, the system
52244 ; data will be gathered with the system code. The process data under 2.x will
52245 ; be available for swapping and under 3.0 it will be allocated per-process.
52246 ;
52247 ; The data that is system data will be identified by [SYSTEM] in the comments
52248 ; describing that data item.
52249
52250 ;DOSDATA SEGMENT
52251
52252 ; 04/11/2022
52253 ;[ORG 0]
52254
52255 ; -----
52256 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
52257 ; -----
52258 ; DOSDATA segment start offset from beginning of MSDOS.SYS file: 32432 (7EB0h)
52259 ; (3DD0h+7EB0h = 0BC80h) - for MSDOS 5.0 kernel file -
52260 ; -----
52261
52262 ; 04/11/2022
52263
52264 ;DOSDATA:0000h
52265
52266 00008FDB 90<rep 5h> align 16
52267
52268 ; -----
52269 ; 06/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
52270 ; -----
52271
52272 segment .data vstart=0 ; 06/12/2022
52273
52274 ; =====
52275
52276 ; 06/12/2022
52277 ;DOSDATASTART equ $
52278 DOSDATASTART:
52279
52280 ;hkn; add 4 bytes to get correct offsets since jmp has been removed in START
52281
52282 ; ; 03/11/2022
52283 ; jmp DOSINIT ; MSDOS 5.0 - MSDOS.SYS (DOSDATA:0000h)
52284
52285 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
52286 ; db 4 dup (?)
52287 00000000 00<rep 4h> times 4 db 0
52288
52289 ; 29/04/2019 - Retro DOS v4.0 modification
52290 ; dw _$STARTCODE ; DOSCODE offset and/or size of DOSDATA
52291 ; dw 0
52292
52293 ;EVEN
52294
52295 ;align 2
52296
52297 ; WANGO!!! The following word is used by SHARE and REDIR to determin data
52298 ; area compatability. This location must be incremented EACH TIME the data
52299 ; area here gets mucked with.
52300 ;
52301 ; Also, do NOT change this position relative to DOSDATA:0.
52302
52303 MSCT001S: ; LABEL BYTE
52304
52305 Dataversion:
52306 00000004 0100 dw 1 ;AC000; [SYSTEM] version number for DOS DATA
52307
52308 ;hkn; add 8 bytes to get correct offsets since BugTyp, BugLev and "BUG " has
52309 ;hkn; been removed to DOSCODE above
52310
52311 ;M044
52312 ; First part of save area for saving last para of window memory
52313
52314 winoldPatch1: ; db 8 dup (?) ;M044

```



```

52315 00000006 00<rep 8h>          times 8 db 0
52316
52317          ; MSDOS 6.21 DOSDATA:000Eh
52318 MYNUM:          ; Offset 000Eh
52319 0000000E 0000          dw 0          ; [SYSTEM] A number that goes with MYNAME
52320 FCBLRU:          ; [SYSTEM] LRU count for FCB cache
52321 00000010 0000          dw 0
52322 OpenLRU:
52323 00000012 0000          dw 0          ; [SYSTEM] LRU count for FCB cache opens
52324 OEM_HANDLER:
52325 00000014 FFFFFFFF          dd -1          ; [SYSTEM] Pointer to OEM handler code
52326
52327          ; BUGBUG - who uses LeaveAddr? What if we want to rework the
52328          ; way that we leave DOS???? - jgl
52329
52330 LeaveAddr:
52331 00000018 [F603]          dw LeaveDOS ; <<OFFSET DOSCODE:LeaveDOS>> ; [SYSTEM]
52332 RetryCount:
52333 0000001A 0300          dw 3          ; [SYSTEM] Share retries
52334 RetryLoop:
52335 0000001C 0100          dw 1          ; [SYSTEM] Share retries
52336 LastBuffer:
52337 0000001E FFFFFFFF          dd -1          ; [SYSTEM] Buffer queue recency pointer
52338 CONTPOS:
52339 00000022 0000          dw 0          ; [SYSTEM] location in buffer of next read
52340 arena_head:
52341 00000024 0000          dw 0          ; [SYSTEM] Segment # of first arena in memory
52342
52343          ; 16/07/2018
52344          ; *****
52345          ; NOTE: INT 21H AH=52H ! (http://stanislavs.org/helppc/int_21-52.html)
52346          ; *****
52347          ; INT 21,52 - Get Pointer to DOS "INVARs" (Undocumented)
52348          ;
52349          ; AH = 52h
52350          ;
52351          ; on return:
52352          ; ES:BX = pointer to DOS "invars", a table of pointers used by DOS.
52353          ; Known "invars" fields follow (varies with DOS version):
52354          ;
52355          ; Offset Size Description
52356          ;
52357          ; -12 word sharing retry count (DOS 3.1-3.3)
52358          ; -10 word sharing retry delay (DOS 3.1-3.3)
52359          ; -8 dword pointer to current disk buffer (DOS 3.x)
52360          ; -4 word pointer in DOS code segment of unread CON input;
52361          ; 0 indicates no unread input (DOS 3.x)
52362          ; -2 word segment of first Memory Control Block (MCB)
52363          ; 00 dword pointer to first DRIVE PARAMETER TABLE (A:) in chain
52364          ; 04 dword pointer to DOS System File Table (SFT)
52365          ; 08 dword pointer to $CLOCK device driver
52366          ; 0C dword pointer to CON device driver
52367          ; 10 byte number of logical drives in system
52368          ; 11 word maximum bytes/block of any block device
52369          ; 13 dword pointer to DOS cache buffer header
52370          ; 17 18bytes NUL device header, first 4 bytes of device header
52371          ; point to the next device in device chain
52372          ;
52373          ; *****
52374
52375          ; The following block of data is used by SYSINIT.
52376          ; Do not change the order or size of this block
52377
52378          ;SYSINITVAR:
52379          ;-----
52380          SYSINITVARS:
52381          DPBHEAD:
52382 00000026 00000000          dd 0          ; [SYSTEM] Pointer to head of DPB-FAT list
52383 SFT_ADDR:
52384 0000002A [CC000000]          dd SFTABL ; [SYSTEM] Pointer to first SFT table
52385 BCLOCK:
52386 0000002E 00000000          dd 0          ; [SYSTEM] The CLOCK device
52387 BCON:
52388 00000032 00000000          dd 0          ; [SYSTEM] Console device entry points
52389 MAXSEC:
52390 00000036 8000          dw 128          ; [SYSTEM] Maximum allowed sector size
52391 BUFFHEAD:
52392 00000038 00000000          dd 0          ; [SYSTEM] Pointer to head of buffer queue
52393 CDSADDR:
52394 0000003C 00000000          dd 0          ; [SYSTEM] Pointer to curdir structure table
52395 SFTFCB:
52396 00000040 00000000          dd 0          ; [SYSTEM] pointer to FCB cache table
52397 KEEPCOUNT:
52398 00000044 0000          dw 0          ; [SYSTEM] count of FCB opens to keep
52399 NUMIO:
52400 00000046 00          db 0          ; [SYSTEM] Number of disk tables
52401 CDSCOUNT:
52402 00000047 00          db 0          ; [SYSTEM] Number of CDS structures in above
52403
52404          ; A fake header for the NUL device
52405 NULDEV:
52406 00000048 00000000          dd 0          ; [SYSTEM] Link to rest of device list
52407          ; dw 8004h
52408 0000004C 0480          dw DEVTYP|ISNULL ; [SYSTEM] Null device attributes
52409 0000004E [CD0D]          dw SNULDEV ; [SYSTEM] Strategy entry point
52410 00000050 [D30D]          dw INULDEV ; [SYSTEM] Interrupt entry point
52411 00000052 4E554C2020202020          db "NUL " ; [SYSTEM] Name of null device
52412 SPLICES:
52413 0000005A 00          db 0          ; [SYSTEM] TRUE => splices being done
52414
52415 Special_Entries:
52416 0000005B 0000          dw 0          ; [SYSTEM] address of special entries ;AN000;
52417 UU_IFS_DOS_CALL:
52418 0000005D 00000000          dd 0          ; [SYSTEM] entry for IFS DOS service ;AN000;
52419
52420          ; UU_IFS_HEADER:
52421          ; dd 0          ; [SYSTEM] IFS header chain ;AN000;
52422
52423 ChkCopyProt:
52424 00000061 0000          dw 0          ; M068
52425 A20OFF_PSP:
52426 00000063 0000          dw 0          ; M068
52427 BUFFERS_PARM1:
52428 00000065 0000          dw 0          ; [SYSTEM] value of BUFFERS= ,m ;AN000;
52429 BUFFERS_PARM2:
52430 00000067 0000          dw 0          ; [SYSTEM] value of BUFFERS= ,n ;AN000;
52431 BOOTDRIVE:
52432 00000069 00          db 0          ; [SYSTEM] the boot drive ;AN000;
52433 DDMOVE:
52434 0000006A 00          db 0          ; [SYSTEM] 1 if we need DWORD move ;AN000;
52435 EXT_MEM_SIZE:
52436 0000006B 0000          dw 0          ; [SYSTEM] extended memory size ;AN000;
52437
52438 HASHINITVAR: ; LABEL WORD ; AN000;

```

```

52439 ;
52440 ; Replaced by next two declarations
52441 ;
52442 ;UU_BUF_HASH_PTR:
52443 ; dd 0 ; [SYSTEM] buffer Hash table addr
52444 ;UU_BUF_HASH_COUNT:
52445 ; dw 1 ; [SYSTEM] number of Hash entries
52446
52447 BufferQueue:
52448 dd 0 ; [SYSTEM] Head of the buffer Queue
52449 DirtyBufferCount:
52450 dw 0 ; [SYSTEM] Count of Dirty buffers in the Que
52451 ; BUGBUG ---- change to byte
52452
52453 SC_CACHE_PTR:
52454 dd 0 ; [SYSTEM] secondary cache pointer
52455 SC_CACHE_COUNT:
52456 dw 0 ; [SYSTEM] secondary cache count
52457 BuffInHMA:
52458 db 0 ; Flag to indicate that buffs are in HMA
52459 LoMemBuff:
52460 dd 0 ; Ptr to intermediate buffer
52461 ; in Low mem when buffs are in HMA
52462 ;
52463 ; All variables which have UU_ as prefix can be reused for other
52464 ; purposes and can be renamed. All these variables were used for
52465 ; EMS support of Buffer Manager. Now they are useless for Buffer
52466 ; manager ---- MOHANS
52467 ;
52468 ;I_am UU_BUF_EMS_FIRST_PAGE,3,<0,0,0>
52469 UU_BUF_EMS_FIRST_PAGE:
52470 db 0,0,0 ; holds the first page above 640K
52471
52472 ;;;I_am UU_BUF_EMS_NPA640,WORD,<0> ; holds the number of pages
52473 ;UU_BUF_EMS_NPA640: ; above 640K
52474 ; dw 0
52475
52476 CL0FATENTRY:
52477 dw -1 ; M014: Holds the data that
52478 ; is used in pack/unpack rts.
52479 ; in fat.asm if cluster 0 is specified.
52480 ; SR;
52481 IoStatFail:
52482 db 0 ; IoStatFail has been added to
52483 ; record a fail on an I24
52484 ; issued from IOFUNC on a status call.
52485
52486 ;***I_am UU_BUF_EMS_MODE,BYTE,<-1> ; EMS mode ;AN000;
52487 ;***I_am UU_BUF_EMS_HANDLE,BYTE ; buffer EMS handle ;AN000;
52488 ;***I_am UU_BUF_EMS_PAGE_FRAME,WORD,<-1>; EMS page frame # ;AN000;
52489 ;***I_am UU_BUF_EMS_SEG_CNT,WORD,<1> ; EMS seg count ;AN000;
52490 ;***I_am UU_BUF_EMS_PFRAME,WORD ; EMS page frame seg address ;AN000;
52491 ;***I_am UU_BUF_EMS_RESERV,WORD,<0> ; reserved ;AN000;
52492 ;
52493 ;***I_am UU_BUF_EMS_MAP_BUFF,1,<0> ; this is not used to save the
52494 ; state of the buffers page.
52495 ; This one byte is retained to
52496 ; keep the size of this data
52497 ; block the same.;
52498
52499 ALLOCMSAVE:
52500 db 0 ; M063: temp var. used to
52501 ; M063: save alloc method in
52502 ; M063: msproc.asm
52503
52504 A20OFF_COUNT:
52505 db 0 ; M068: indiactes the # of
52506 ; M068: int 21 calls for
52507 ; M068: which A20 is off
52508
52509 DOS_FLAG:
52510 db 0 ; see DOSSYM.INC for Bit
52511 ; definitions
52512
52513 UNPACK_OFFSET:
52514 dw 0 ; saves pointer to the start
52515 ; of unpack code in exepatch.
52516 ; asm.
52517
52518 UMBFLAG:
52519 db 0 ; M003: bit 0 indicates the
52520 ; M003: link state of the UMBS
52521 ; M003: whether linked or not
52522 ; M003: to the DOS arena chain
52523
52524 SAVE_AX:
52525 dw 0 ; M000: temp varibale to store ax
52526 ; M000: in msproc.asm
52527
52528 UMB_HEAD:
52529 dw -1 ; M000: this is initialized to
52530 ; M000: the first umb arena by
52531 ; M000: BIOS sysinit.
52532
52533 START_ARENA:
52534 dw 1 ; M000: this is the first arena
52535 ; M000: from which DOS will
52536 ; M000: start its scan for alloc.
52537
52538 ; End of SYSINITVar block
52539 ;-----
52540
52541 ; 25/04/2019 - Retro DOS v4.0
52542
52543 ; 16/07/2018
52544 ; MSDOS 3.3 (& MDOS 6.0)
52545
52546 ;
52547 ; Sharer jump table
52548 ;
52549
52550 ;PUBLIC JShare
52551 ;EVEN
52552
52553 ;JShare LABEL DWORD
52554 ; DW OFFSET DOSCODE:BadCall, 0
52555 ; DW OFFSET DOSCODE:OKCall, 0 ; 1 MFT_enter
52556 ; DW OFFSET DOSCODE:OKCall, 0 ; 2 MFTClose
52557 ; DW OFFSET DOSCODE:BadCall, 0 ; 3 MFTclu
52558 ; DW OFFSET DOSCODE:BadCall, 0 ; 4 MFTCloseP
52559 ; DW OFFSET DOSCODE:BadCall, 0 ; 5 MFTclon
52560 ; DW OFFSET DOSCODE:BadCall, 0 ; 6 set_block
52561 ; DW OFFSET DOSCODE:BadCall, 0 ; 7 clr_block
52562 ; DW OFFSET DOSCODE:OKCall, 0 ; 8 chk_block
52563 ; DW OFFSET DOSCODE:BadCall, 0 ; 9 MFT_get
52564 ; DW OFFSET DOSCODE:BadCall, 0 ; 10 ShSave
52565 ; DW OFFSET DOSCODE:BadCall, 0 ; 11 ShChk
52566 ; DW OFFSET DOSCODE:OKCall, 0 ; 12 ShCol
52567 ; DW OFFSET DOSCODE:BadCall, 0 ; 13 ShCloseFile
52568 ; DW OFFSET DOSCODE:BadCall, 0 ; 14 ShSU
52569
52570 align 2

```



```

52563
52564 00000090 [3407]0000
52565 00000094 [3807]0000
52566 00000098 [3807]0000
52567 0000009C [3407]0000
52568 000000A0 [3407]0000
52569 000000A4 [3407]0000
52570 000000A8 [3407]0000
52571 000000AC [3407]0000
52572 000000B0 [3807]0000
52573 000000B4 [3407]0000
52574 000000B8 [3407]0000
52575 000000BC [3407]0000
52576 000000C0 [3807]0000
52577 000000C4 [3407]0000
52578 000000C8 [3407]0000
52579
52580
52581
52582
52583
52584
52585
52586
52587
52588
52589
52590
52591
52592
52593
52594
52595
52596
52597
52598
52599
52600
52601
52602
52603
52604
52605
52606
52607
52608
52609
52610
52611
52612
52613
52614
52615
52616
52617
52618
52619
52620
52621
52622
52623 000000CC FFFF
52624 000000CE FFFF
52625
52626 000000D0 0500
52627
52628 000000D2 00<rep 127h>
52629
52630
52631
52632
52633
52634 000001F9 00
52635 000001FA 00
52636
52637 000001FB 00<rep 80h>
52638 0000027B 00<rep 83h>
52639
52640 000002FE 00
52641 000002FF 00
52642 00000300 03
52643
52644 00000301 2F
52645 00000302 00
52646 00000303 00
52647 00000304 01
52648 00000305 20<rep 10h>
52649
52650
52651
52652
52653
52654
52655
52656
52657
52658
52659
52660
52661
52662
52663
52664
52665
52666
52667
52668
52669
52670
52671
52672
52673
52674
52675
52676
52677
52678
52679
52680 00000315 [650D]
52681 00000317 [650D]
52682 00000319 [650D]
52683 0000031B [650D]
52684
52685 0000031D 0000
52686

JShare:
        DW      BadCall,0
MFT_enter: DW      OKCall,0 ; 1  MFT_enter
MFTClose:  DW      OKCall,0 ; 2  MFTClose
MFTClu:    DW      BadCall,0 ; 3  MFTClu
MFTCloseP: DW      BadCall,0 ; 4  MFTCloseP
MFTClon:   DW      BadCall,0 ; 5  MFTClon
set_block: DW      BadCall,0 ; 6  set_block
clr_block: DW      BadCall,0 ; 7  clr_block
chk_block: DW      OKCall,0 ; 8  chk_block
MFT_get:   DW      BadCall,0 ; 9  MFT_get
ShSave:    DW      BadCall,0 ; 10 ShSave
ShChk:     DW      BadCall,0 ; 11 ShChk
ShCol:     DW      OKCall,0 ; 12 ShCol
ShCloseFile: DW      BadCall,0 ; 13 ShCloseFile
ShSu:      DW      BadCall,0 ; 14 ShSu

;=====
; CONST2.ASM (MSDOS 6.0, 1991)
;=====
; 25/04/2019 - Retro DOS 4.0
; 16/07/2018 - Retro DOS 3.0

;Break <Initialized data and data used at DOS initialization>
;-----

; We need to identify the parts of the data area that are relevant to tasks
; and those that are relevant to the system as a whole. Under 3.0, the system
; data will be gathered with the system code. The process data under 2.x will
; be available for swapping and under 3.0 it will be allocated per-process.
;
; The data that is system data will be identified by [SYSTEM] in the comments
; describing that data item.

;DOSDATA SEGMENT WORD PUBLIC 'DATA'

;
; Table of routines for assignable devices
;
; MSDOS allows assignment if the following standard devices:
;   stdin (usually CON input)
;   stdout (usually CON output)
;   auxin (usually AUX input)
;   auxout (usually AUX output)
;   stdlpt (usually PRN output)
;
; SPECIAL NOTE:
;   Status of a file is a strange idea. We choose to handle it in this manner:
;   If we're not at end-of-file, then we always say that we have a character.
;   Otherwise, we return ^Z as the character and set the ZERO flag. In this
;   manner we can support program written under the old DOS (they use ^Z as EOF
;   on devices) and programs written under the new DOS (they use the ZERO flag
;   as EOF).

; Default SFTs for boot up

        ;PUBLIC SFTABL

SFTABL:        ; LABEL    DWORD            ; [SYSTEM] file table
               DW -1                ; [SYSTEM] link to next table
               DW -1                ; [SYSTEM] link seg to next table
               ;dw 5 ; 24/03/2024
               DW sf_default_number ; [SYSTEM] Number of entries in table
               times 295 db 0 ; MSDOS 6.0 ; PCDOS 7.1 ; 24/03/2024
               times (sf_default_number*sf_entry_size) db 0

; the next two variables relate to the position of the logical stdout/stdin
; cursor. They are only meaningful when stdin/stdout are assigned to the
; console.
        ; DOSDATA:01F9h (MSDOS 6.21) ; PCDOS 7.1 ; 24/03/2024
CARPOS:        db 0                ; [SYSTEM] cursor position in stdin
STARTPOS:      db 0                ; [SYSTEM] position of cursor at beginning
               ; of buffered input call
INBUF:         times 128 db 0       ; [SYSTEM] general device input buffer
CONBUF:        times 131 db 0       ; [SYSTEM] The rest of INBUF and console buffer
               ; DOSDATA:02FEh (MSDOS 6.21) ; PCDOS 7.1
PFLAG:         db 0                ; [SYSTEM] printer echoing flag
VERFLG:        db 0                ; [SYSTEM] Initialize with verify off
CHARCO:        db 00000011b ; 3    ; [SYSTEM] Allows statchks every 4 chars...
switch_character:
chSwitch:      db '/'              ; UNUSED - obsolete datum, can be reused
AllocMethod:   db 0                ; [SYSTEM] how to alloc first(best)last
fShare:        db 0                ; [SYSTEM] TRUE => sharing installed
DIFFNAM:       db 1                ; [SYSTEM] Indicates when MYNAME has changed
MYNAME:        times 16 db 20h      ; [SYSTEM] My network name

; The following table is a list of addresses that the sharer patches to be
; PUSH AX to enable the critical sections

        ; DOSDATA:0315h (MSDOS 6.21) ; PCDOS 7.1 ; 24/03/2024

;PUBLIC      CritPatch

CritPatch:     ; LABEL WORD

;IRP sect,<critDisk,critDevice>

;IF (NOT REDIRECTOR) AND (NOT SHAREF)
;
;SR; Change code patch address to a variable in data segment
;
;
;       dw OFFSET DOSDATA: redir_patch
;       dw OFFSET DOSDATA: redir_patch
;
;
;hkhn         Short_Addr  E&sect
;hkhn         Short_Addr  L&sect
;
;ELSE
;       DW      0
;       DW      0
;ENDIF
;ENDM
;       DW      0

; 25/07/2019 - Retro DOS v4.0 (MSDOS 6.21)
dw redir_patch
dw redir_patch
dw redir_patch
dw redir_patch
dw 0

```

```

52687 ; WARNING!!! PRINT and PSPRINT *REQUIRE* ErrorMode to precede INDOS.
52688 ; Also, IBM server 1.0 requires this also.
52689
52690 ;db 90h ; 24/03/2024
52691
52692 ;EVEN ; Force swap area to start on word boundry
52693
52694 align 2
52695 ;PUBLIC SWAP_START
52696
52697 ; 10/03/2024 - Retro DOS v5.0
52698 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0320h
52699 SWAP_START: ; LABEL BYTE
52700 ERRORMODE: db 0 ; Flag for INT 24 processing
52701 INDOS: db 0 ; DOS status for interrupt processing
52702 WPERR: db -1 ; Write protect error flag
52703 EXTERR_LOCUS: db 0 ; Extended Error Locus
52704 EXTERR: dw 0 ; Extended Error code
52705
52706 ;WARNING Following two bytes Accessed as word in $GetExtendedError
52707 EXTERR_ACTION: db 0 ; Extended Error Action
52708 EXTERR_CLASS: db 0 ; Extended Error Class
52709 ; end warning
52710
52711 EXTERRPT: dd 0 ; Extended Error pointer
52712
52713 DMAADD: dd 80h ; User's disk transfer address (disp/seg)
52714 CurrentPDB: dw 0 ; Current process identifier
52715 ConC_Spsave: dw 0 ; saved SP before ^C
52716 exit_code: dw 0 ; exit code of last proc.
52717 CURDRV: db 0 ; Default drive (init A)
52718 CNTCFLAG: db 0 ; ^C check in dispatch disabled
52719 ; F.C. 2/17/86
52720 CPSWFLAG: db 0 ; Code Page Switching Flag DOS 4.00
52721 CPSWSAVE: db 0 ; copy of above in case of ABORT
52722 ;align 2
52723 ; 10/03/2024 - Retro DOS v5.0
52724 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:033Ah
52725 SWAP_ALWAYS: ; 05/08/2018
52726 USER_IN_AX: dw 0 ; User INPUT AX value (used for
52727 ; extended error type stuff.
52728 ; NOTE: does not have Correct value on
52729 ; 1-12, OEM, Get/Set CurrentPDB,
52730 ; GetExtendedError system calls)
52731 PROC_ID: dw 0 ; PID for sharing (0 = local)
52732 USER_ID: dw 0 ; Machine for sharing (0 = local)
52733 FirstArena: dw 0 ; first free block found
52734 BestArena: dw 0 ; best free block found
52735 LastArena: dw 0 ; last free block found
52736 ENDMEM: dw 0 ; End of memory used in DOSINIT
52737 LASTENT: dw 0 ; Last entry for directory search
52738 FAILERR: db 0 ; NZ if user did FAIL on I 24
52739 ALLOWED: db 0 ; Allowed I 24 answers (see allowed_)
52740 NoSetDir: db 0 ; true -> do not set directory
52741 DidCTRLC: db 0 ; true -> we did a ^C exit
52742 SpaceFlag: db 0 ; true -> embedded spaces are allowed in FC
52743
52744 ; Warning! The following items are accessed as a WORD in TIME.ASM
52745 ;EVEN
52746 align 2
52747 ; DOSDATA:0350h (MSDOS 6.21)
52748 DAY: db 0 ; Day of month
52749 MONTH: db 0 ; Month of year
52750 YEAR: dw 0 ; Year (with century)
52751 DAYCNT: dw -1 ; Day count from beginning of year
52752 WEEKDAY: db 0 ; Day of week
52753 ; end warning
52754
52755 CONSWAP: db 0 ; TRUE => console was swapped during device read
52756 IDLEINT: db 1 ; TRUE => idle int is allowed
52757 fAborting: db 0 ; TRUE => abort in progress
52758
52759 ; Combination of all device call parameters
52760 ;PUBLIC DEVCALL ;
52761 ;DEVCALL SRHEAD <> ; basic header for disk packet
52762 DEVCALL: ; 08/08/2018
52763 DEVCALL_REQLEN: db 0 ; Length in bytes of request block
52764 DEVCALL_REQUNIT: db 0 ; Device unit number
52765 DEVCALL_REQFUNC: db 0 ; Type of request
52766 DEVCALL_REQSTAT: dw 0 ; Status word
52767 times 8 db 0 ; Reserved for queue links
52768
52769 ;PUBLIC CALLUNIT
52770 CALLUNIT: ; LABEL BYTE ; unit number for disk
52771 CALLFLSH: ; LABEL WORD ;
52772 CALLMED: db 0 ; media byte
52773 CALLBR: ; LABEL DWORD ;
52774 ;PUBLIC CALLXAD
52775 CALLXAD: ; LABEL DWORD ;
52776 CALLRBYT: db 0 ;
52777 ;PUBLIC CALLVIDM
52778 CALLVIDM: ; LABEL DWORD ;
52779 times 3 db 0 ;
52780 ;PUBLIC CallBPB
52781 CALLBPB: ; LABEL DWORD ;
52782 CALLSCNT: ;
52783 dw 0 ;
52784 ;PUBLIC CALLSSEC
52785 CALLSSEC: ; LABEL WORD ;
52786 dw 0 ;
52787 CALLVIDRW: dd 0 ;
52788 ;MSDOS 6.0
52789 CALLNEWSC: dd 0 ; starting sector for >32mb
52790 CALLDEVAD: dd 0 ; stash for device entry point
52791
52792 ; Same as above for I/O calls ;
52793 ;
52794 ;PUBLIC IOCall
52795 ;IOCALL SRHEAD <> ;
52796 IOCALL: ; 07/08/2018
52797 IOCALL_REQLEN: db 0 ; Length in bytes of request block
52798 IOCALL_REQUNIT: db 0 ; Device unit number
52799 IOCALL_REQFUNC: db 0 ; Type of request
52800 IOCALL_REQSTAT: dw 0 ; Status word
52801 times 8 db 0 ; Reserved for queue links
52802 IOFLSH: ; LABEL WORD ;
52803 ;PUBLIC IORCHR
52804 IORCHR: ; LABEL BYTE ;
52805 IOMED: db 0 ;
52806 IOXAD: dd 0 ;
52807 IOSCNT: dw 0 ;
52808 IOSSEC: dw 0 ;
52809
52810 ; Call struct for DSKSTATCHK ;

```

```

52811 00000392 0E      DSKSTCALL: db DRDNDHL      ; = 14
52812 00000393 00      db 0
52813 00000394 05      DSKSTCOM:  db DEVRDND      ; = 5
52814 00000395 0000    DSKSTST:  dw 0
52815 00000397 00<rep 8h> times 8 db 0
52816 0000039F 00      DSKCHRET:  db 0
52817
52818 ;hkn; short_addr has been changed to provide offset in DOSCODE.
52819 ;hkn; deviobuf is in DATA seg (DOSDATA)
52820 ;hkn short_addr DEVIOPBUF
52821
52822 000003A0 [BC03]    DEVIOPBUF_PTR dw DEVIOPBUF
52823 000003A2 0000    DOSSEG_INIT dw 0 ; DOS segment set at Init
52824 000003A4 0100    DSKSTCNT:  dw 1
52825 000003A6 0000    dw 0
52826
52827 000003A8 00      CreatePDB: db 0 ; flag for creating a process
52828
52829 ;MSDOS 6.0
52830 Lock_Buffer: ; LABEL DWORD ;MS. DOS Lock Buffer for Ext Lock
52831 000003A9 00000000 dd 0 ;MS. position
52832 000003AD 00000000 dd 0 ;MS. length
52833
52834 ;hkn; the foll. was moved from dosmes.asm.
52835
52836 ; 29/01/2024 - Retro DOS v5.0 (PCDOS v7.1 IBMDOS.COM)
52837 ; DOSDATA:03B1h
52838 000003B1 90      IOCTL_drvnum: db 90h ; nop
52839
52840 ;EVEN
52841 align 2 ; needed to maintain offsets
52842
52843 ; DOSDATA:03B2h (MSDOS 6.21)
52844
52845 000003B2 0000    USERNUM: dw 0 ; 24 bit user number
52846 000003B4 00      db 0
52847
52848 ;IF IBM
52849 ;IF IBMCOPYRIGHT
52850 ; 24/03/2024 (PCDOS v7.1 IBMDOS.COM)
52851 000003B5 00      OEMNUM: db 0 ; 8 bit OEM number
52852 ;ELSE
52853 ;OEMNUM: db 0FFh ; 8 bit OEM number
52854 ;ENDIF
52855 ;ELSE
52856 ;OEMNUM: db 0FFh ; (MSDOS 6.22) ; 24/03/2024
52857 ;ENDIF
52858
52859 ;=====
52860 ; MS_DATA.ASM (MSDOS 6.0, 1991)
52861 ;=====
52862 ; 25/04/2019 - Retro DOS 4.0
52863
52864 ; Retro DOS v4.0 NOTE: (by Erdogan Tan, 25/04/2019)
52865 ;-----
52866 ; This data section which was named as uninitialized data
52867 ; (as overlayed by initialization code) but follows
52868 ; initialized data section from DOSDATA:03B6h address
52869 ; (in otherwords, the method is different than MSDOS 3.3,
52870 ; and there is not overlaying..)
52871 ; *****
52872 ; Reference: MSDOS 6.21 kernel DOSDATA section (4970 bytes)
52873 ; follows DOSCODE section in the kernel file (MSDOS.SYS)
52874 ; (it is located at offset 0BF70h, file offset 0BF70h-3DE0h)
52875 ; but starts from offset 0 (ORG 0) and ends at offset 1370h.
52876 ; TIMEBUF is at offset 03B6h.
52877 ; *****
52878
52879 ;Break <Uninitialized data overlayed by initialization code>
52880 ;-----
52881 ;DOSDATA SEGMENT WORD PUBLIC 'DATA'
52882 ; Init code overlaps with data area below
52883
52884 ; ORG 0
52885
52886 MSDAT001S: ; label byte
52887
52888 ; DOSDATA:03B6h ; MSDOS 6.21 (MSDOS.SYS, file offset 0BF70h-3DE0h+3B6h)
52889 000003B6 0000<rep 3h> TIMEBUF: ; times 6 db 0
52890 000003BC 0000 times 3 dw 0 ; Time read from clock device
52891 DEVIOPBUF: dw 0 ; Buffer for I/O under file assignment
52892
52893 ; The following areas are used as temp buffer in EXEC system call
52894
52895 ; DOSDATA:03BEh
52896 000003BE 00<rep 80h> OPENBUF: ;times 64 dw 0
52897 times 128 db 0 ; buffer for name operations
52898 0000043E 00<rep 80h> RENBUF: times 128 db 0 ; buffer for rename destination
52899
52900 ; Buffer for search calls
52901 SEARCHBUF:
52902 000004BE 00<rep 35h> times 53 db 0 ; internal search buffer
52903 DUMMYCDS: ;times 88 db 0
52904 000004F3 00<rep 58h> times curdirLen db 0
52905
52906 ; End of contiguous buffer
52907
52908 ; Temporary directory entry for use by many routines. Device directory
52909 ; entries (bogus) are built here.
52910
52911 ; DOSDATA:054Bh
52912
52913 DEVFCB: ; LABEL BYTE ; Uses NAME1, NAME2, combined
52914
52915 ; WARNING.. do not alter position of NAME1 relative to DEVFCB
52916 ; without first examining BUILD_DEVICE_ENT. Look carefully at DOS_RENAME
52917 ; as well as it is the only guy who uses NAME2 and DESTSTART.
52918
52919 NAME1:
52920 0000054B 00<rep Ch> times 12 db 0 ; File name buffer
52921
52922 00000557 00<rep Dh> NAME2: times 13 db 0 ;
52923 DESTSTART:
52924 00000564 0000 dw 0 ;
52925 ;DB ((SIZE DIR_ENTRY) - ($ - DEVFCB)) DUP (?)
52926 times 5 db 0
52927 00000566 00<rep 5h> times ((dir_entry.size)-($-DEVFCB)) db 0
52928
52929 ; End Temporary directory entry.
52930
52931 0000056B 00 ATTRIB: db 0 ; storage for file attributes
52932 EXTFCB:
52933 0000056C 00 db 0 ; TRUE => extended FCB in use
52934 SATTRIB:

```

```

52935 0000056D 00          db      0          ; Storage for search attributes
52936 OPEN_ACCESS:         db      0          ;
52937 0000056E 00          db      0          ; access of open system call
52938 FOUNDDEL:             db      0          ; true => file was deleted
52939 0000056F 00          db      0          ;
52940 FOUND_DEV:           db      0          ; true => search found a device
52941 00000570 00          db      0          ;
52942 FSPLICE:              db      0          ; true => do a splice in transpath
52943 00000571 00          db      0          ;
52944 FSHARING:            db      0          ; TRUE => no redirection
52945 00000572 00          db      0          ;
52946 SECCLUSPOS:          db      0          ; Position of first sector within cluster
52947 00000573 00          db      0          ;
52948 00000574 00          TRANS: db      0          ;
52949 00000575 00          READOP: db      0          ;
52950 THISDRV:             db      0          ;
52951 00000576 00          db      0          ;
52952 CLUSFAC:             db      0          ;
52953 00000577 00          db      0          ;
52954 CLUSSPLIT:           db      0          ;
52955 00000578 00          db      0          ;
52956 INSMODE:            db      0          ; true => insert mode in buffered read
52957 00000579 00          db      0          ;
52958 0000057A 00          CMETA:  db      0          ; count of meta'ed components found
52959 0000057B 00          VALID: db      0          ;
52960 EXIT_TYPE:          db      0          ; type of exit...
52961 0000057C 00          db      0          ;
52962          ; 24/03/2024
52963 0000057D 00          db      0          ; PCDOS 7.1 - DOSDATA:057Dh
52964          ;db     90h ; MSDOS 6.22 - DOSDATA:057Dh
52965
52966          ;EVEN
52967
52968 align 2
52969
52970          ; DOSDATA:057Eh
52971
52972          ; WARNING - the following two items are accessed as a word
52973
52974 CREATING:
52975 0000057E 00          db      0          ; true => creating a file
52976 0000057F 00          DELALL: db      0          ; = 0 iff BUGBUG
52977          ; = DIRFREE iff BUGBUG
52978 EXITHOLD:             dd      0          ; Temp location for proc terminate
52979 00000580 00000000     dd      0          ;
52980 USER_SP:             dw      0          ; User SP for system call
52981 00000584 0000      dw      0          ;
52982 USER_SS:             dw      0          ; User SS for system call
52983 00000586 0000      dw      0          ;
52984 CONTSTK:            dw      0          ;
52985 00000588 0000      dw      0          ;
52986 THISDPB:            dd      0          ;
52987 0000058A 00000000     dd      0          ;
52988 CLUSSAVE:           dw      0          ;
52989 0000058E 0000      dw      0          ;
52990 CLUSSEC:            dd      0          ;>32mb          AC0000
52991 00000590 00000000     dd      0          ;
52992 PREREAD:            dw      0          ; 0 means preread; 1 means optional
52993 00000594 0000      dw      0          ; Used by ALLOCATE
52994 00000596 0000      dw      0          ;
52995 FATBYT:             dw      0          ; Used by $SLEAZFUNC
52996 00000598 0000      dw      0          ;
52997          ; DOSDATA:059Ah
52998 0000059A 00000000     DEVPT:  dd      0          ;
52999 THISSFT:            dd      0          ;
53000 0000059E 00000000     dd      0          ; Address of user SFT
53001 THISCDS:           dd      0          ; Address of current CDS
53002 000005A2 00000000     dd      0          ;
53003 THISFCB:           dd      0          ; Address of user FCB
53004 000005A6 00000000     dd      0          ;
53005 000005AA FFFF      SFN: dw     -1          ; SystemFileNumber found for accessfile
53006 000005AC 0000      JFN: dw      0          ; JobFileNumber found for accessfile
53007 000005AE 00000000     PJFN:  dd      0          ; PointerJobFileNumber found for accessfile
53008 WFP_START:          dw      0          ;
53009 000005B2 0000      dw      0          ;
53010 REN_WFP:           dw      0          ;
53011 000005B4 0000      dw      0          ;
53012 CURR_DIR_END:      dw      0          ;
53013 000005B6 0000      dw      0          ;
53014 NEXTADD:           dw      0          ;
53015 000005B8 0000      dw      0          ;
53016 LASTPOS:           dw      0          ;
53017 000005BA 0000      dw      0          ;
53018 CLUSNUM:           dw      0          ;
53019 000005BC 0000      dw      0          ;
53020 000005BE 00000000     DIRSEC: dd      0          ;>32mb          AC0000
53021 DIRSTART:          dw      0          ;
53022 000005C2 0000      dw      0          ;
53023 000005C4 00000000     SECPOS: dd      0          ;>32mb Position of first sector accessed
53024 000005C8 00000000     VALSEC: dd      0          ;>32mb Number of valid (previously written)
53025          ; sectors
53026 BYTSECPOS:          dw      0          ; Position of first byte within sector
53027 000005CC 0000      dw      0          ;
53028 BYTPOS: ;times      4 db 0          ; Byte position in file of access
53029 000005CE 0000<rep 2h> times      2 dw 0
53030 BYTCNT1:           dw      0          ; No. of bytes in first sector
53031 000005D2 0000      dw      0          ;
53032 BYTCNT2:           dw      0          ; No. of bytes in last sector
53033 000005D4 0000      dw      0          ;
53034 000005D6 0000      SECCNT: dw      0          ; No. of whole sectors
53035          ; DOSDATA:05D8h
53036 ENTFREE:           dw      0          ;
53037 000005D8 0000      dw      0          ;
53038 ENTLAST:           dw      0          ;
53039 000005DA 0000      dw      0          ;
53040 NXTCLUSNUM:        dw      0          ;
53041 000005DC 0000      dw      0          ;
53042 GROWCNT:           dd      0          ;
53043 000005DE 00000000     dd      0          ;
53044 000005E2 00000000     CURBUF: dd      0          ;
53045 000005E6 00000000     CONSFT: dd      0          ; SFT of console swapped guy.
53046 000005EA 0000      SAVEBX: dw      0          ;
53047 000005EC 0000      SAVEDS: dw      0          ;
53048 RESTORE_TMP:       dw      0          ;
53049 000005EE 0000      dw      0          ; return address for restore world
53050 000005F0 0000      NSS: dw      0          ;
53051 000005F2 0000      NSP: dw      0          ;
53052          ; DOSDATA:05F4h
53053 EXTOPEN_FLAG:       dw      0          ;FT. extended open input flag ;AN000;
53054 000005F4 0000      dw      0          ;
53055 EXTOPEN_ON:         dw      0          ;FT. extended open conditional flag ;AN000;
53056 000005F6 00          db      0          ;
53057 EXTOPEN_IO_MODE:    dw      0          ;FT. extended open io mode ;AN000;
53058 000005F7 0000      dw      0          ;

```

```

53059 SAVE_DI:
53060 dw 0 ;FT. extended open saved DI ;AN000;
53061 SAVE_ES:
53062 dw 0 ;FT. extended open saved ES ;AN000;
53063 SAVE_DX:
53064 dw 0 ;FT. extended open saved DX ;AN000;
53065 SAVE_CX:
53066 dw 0 ;FT. extended open saved CX ;AN000;
53067 SAVE_BX:
53068 dw 0 ;FT. extended open saved BX ;AN000;
53069 SAVE_SI:
53070 dw 0 ;FT. extended open saved SI ;AN000;
53071 SAVE_DS:
53072 dw 0 ;FT. extended open saved DS ;AN000;
53073
53074 ; DOSDATA:0607h
53075
53076 ; HIGH_SECTOR is a hack to allow passing 32-bit sector numbers where
53077 ; we used to just pass 16 bits in a register. Now High_SECTOR holds
53078 ; the high 16, the low 16 are still in the register.
53079
53080 HIGH_SECTOR:
53081 dw 0 ;>32mb higher sector # ;AN000;
53082 ; 25/09/2023
53083 OffsetMagicPatch:
53084 ; 24/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
53085 dw MagicPatch ;scottg 8/6/92
53086 ; 06/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
53087 ;dw 0
53088 ;see dos\mpatch.asm
53089
53090 DISK_FULL:
53091 db 0 ;>32mb indicating disk full when 1 ;AN000;
53092 TEMP_VAR:
53093 dw 0 ; temporary variable for everyone ;AN000;
53094 TEMP_VAR2:
53095 dw 0 ; temporary variable 2 for everyone ;AN000;
53096 DrvErr: db 0 ; used to save drive error ;AN000;
53097 DOS34_FLAG:
53098 dw 0 ; common flag for DOS 3.4 ;AN000;
53099 NO_FILTER_PATH:
53100 dd 0 ; pointer to original path ;AN000;
53101 NO_FILTER_DPATH:
53102 dd 0 ; pointer to original path of destination ;AN000;
53103 ; M008
53104 AbsRdwr_SS:
53105 dw 0 ; INT 25/26 user stack segment
53106 AbsRdwr_SP:
53107 dw 0 ; INT 25/26 user stack offset
53108 ; I_am UU_Callback_flag,BYTE,<0> ; Unused
53109 ; M008
53110 ; 24/03/2024
53111 ; MSDOS 5.0 MSDOS.SYS - DOSDATA:061Fh
53112 ; MSDOS 6.22 MSDOS.SYS - DOSDATA:061Fh
53113 ; 01/01/2024
53114 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:061Fh
53115 db 0
53116
53117 ; make those pushes fast!!!
53118 ;EVEN
53119
53120 align 2
53121
53122 StackSize equ 180h ; 384 ; gross but effective
53123
53124 ;StackSize equ 300h ; 768 ; This is a "trial" change IBM hasn't
53125 ; ; made up their minds about
53126
53127 ; WARNING!!!! DskStack may grow into AUXSTACK due to interrupt service.
53128 ; This is NO problem as long as AUXSTACK comes immediately before DSKSTACK
53129
53130 RENAMEDMA: ; LABEL BYTE ; See DOS_RENAME
53131
53132 times StackSize db 0 ; db 384 dup(0) ; PC DOS 7.1
53133 AUXSTACK: ; LABEL BYTE
53134
53135 times StackSize db 0 ; db 384 dup(0) ; PC DOS 7.1
53136 DSKSTACK: ; LABEL BYTE
53137 ; 24/03/2024
53138 ; 01/01/2024 - Retro DOS v5.0 (PC DOS 7.1 IBMDOS.COM)
53139 ; (PC DOS 7.1 IBMDOS.COM - DOSDATA:0920h)
53140 db '@#IBM:12.01.2003.build_1.32#@ IBMDOS.COM(USA)',0
53141
53142 times StackSize-($-DSKSTACK) db 0 ; db 338 dup(0) ; PC DOS 7.1
53143 ;times StackSize db 0 ;
53144 IOSTACK: ; LABEL BYTE
53145
53146 ; DOSDATA:0AA0h
53147
53148 ; patch space for Boca folks.
53149 ; Say what????!!! This does NOT go into the swappable area!
53150 ; NOTE: we include the decl of ibmpatch in ms-dos even though it is not needed.
53151 ; This allows the REDIRECTOR to work on either IBM or MS-DOS.
53152
53153 IBMPATCH: ; label byte
53154 PRINTER_FLAG:
53155 db 0 ; [SYSTEM] status of PRINT utility
53156 VOLCHNG_FLAG:
53157 db 0 ; [SYSTEM] true if volume label created
53158 ; 24/03/2024 (PC DOS 7.1 IBMDOS.COM)
53159 db 0FFh ; -1
53160 VIRTUAL_OPEN:
53161 db 0 ; [SYSTEM] non-zero if we opened a virtual file
53162
53163 ; 24/03/2024 - Retro DOS v5.0
53164 ; PC DOS 7.1 IBMDOS.COM
53165 %if 0
53166
53167 ; Following 4 variables moved to MSDATA.asm from Mstable.asm (P4986)
53168
53169 ; DOSDATA:0AA3h ; MSDOS 6.22 ; MSDOS 5.0
53170 FSeek_drive:
53171 db 0 ;AN000; fastseek drive #
53172 FSeek_firclus:
53173 dw 0 ;AN000; fastseek first cluster #
53174 FSeek_logclus:
53175 dw 0 ;AN000; fastseek logical cluster #
53176 FSeek_logsave:
53177 dw 0 ;AN000; fastseek returned log clus #

```

```

53178 %endif
53179
53180 ; DOSDATA:0AAAh ; MSDOS 6.22 ; MSDOS 5.0
53181 ; DOSDATA:0AA3h ; PC DOS 7.1 ; 24/03/2024
53182
53183 TEMP_DOSLOC:
53184 00000AA3 FFFF dw -1 ;stores the temporary location of dos
53185 ;at SYSINIT time.
53186 ; 10/03/2024
53187 ;SWAP_END: ; LABEL BYTE
53188
53189 ; THE FOLLOWING BYTE MUST BE HERE, IMMEDIATELY FOLLOWING SWAP_END. IT CANNOT
53190 ; BE USED. If the size of the swap data area is ODD, it will be rounded up
53191 ; to include this byte.
53192
53193 ; 24/03/2024
53194 ; 05/01/2024 (PCDOS 7.1 IBMDOS.COM)
53195 ;db 0 ; DOSDATA:0AACh (MSDOS 5.0-6.22 MSDOS.SYS)
53196
53197 ; DOSDATA:0AADh
53198
53199 ;hkn; DB (512+80+32-(SWAP_END-ibmpatch)) DUP (?)
53200
53201 ; -----
53202 ; 05/01/2024 - Retro DOS v5.0
53203 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:0AA5h
53204
53205 ; 24/01/2024
53206 LNE_COUNT:
53207 00000AA5 0000 dw 0 ; long name entry count (for file)
53208
53209 00000AA7 00<rep Eh> times 14 db 0
53210
53211 ENTLAST_PREV:
53212 00000AB5 0000 dw 0 ; previous ENTLAST (for long name search !?)
53213
53214 00000AB7 00<rep 24h> times 36 db 0
53215
53216 absdrw_extd:
53217 00000ADB 00 db 0
53218 DIRSTART_HW:
53219 00000ADC 0000 dw 0
53220 CLUSNUM_HW:
53221 00000ADE 0000 dw 0
53222 NXTCLUSNUM_HW:
53223 00000AE0 0000 dw 0
53224 LASTPOS_HW:
53225 00000AE2 0000 dw 0
53226 FATBYT_HW:
53227 00000AE4 0000 dw 0
53228 DESTSTART_HW:
53229 00000AE6 0000 dw 0
53230 CLUSTNUM_HW:
53231 00000AE8 0000 dw 0
53232 CLUSDATA_HW: ; cluster data (0 = release, -1 = allocate) ; 30/01/2024
53233 00000AEA 0000 dw 0
53234 CCONTENT_HW: ; UNPACK output
53235 00000AEC 0000 dw 0
53236 ROOTCLUST_HW:
53237 00000AEE 0000 dw 0
53238 CCOUNT_HW: ; 09/02/2024 ; for ALLOCATE
53239 00000AF0 0000 dw 0
53240 CLUSTERS_HW:
53241 00000AF2 0000 dw 0
53242 00000AF4 0000 dw 0
53243 00000AF6 0000 dw 0
53244 CLSKIP_HW:
53245 00000AF8 0000 dw 0
53246
53247 ; 10/03/2024 - Retro DOS v5.0
53248 ; (PCDOS 7.1 IBMDOS.COM)
53249 SWAP_END: ; LABEL BYTE
53250
53251 ;DOSDATA ENDS
53252
53253 ;=====
53254 ; DOSTAB.ASM (MSDOS 6.0, 1991)
53255 ;=====
53256 ; 27/04/2019 - Retro DOS 4.0
53257 ; 16/07/2018 - Retro DOS 3.0
53258
53259 ;DOSDATA Segment
53260
53261 ; DOSDATA:0AADh (MSDOS 6.21, MSDOS.SYS)
53262
53263 ; DOSDATA:0AFAh (PCDOS 7.1, IBMDOS.COM) ; 05/01/2024
53264
53265 ;
53266 ; upper case table
53267 ; -----
53268 UCASE_TAB: ; label byte
53269 00000AFA 8000 dw 128
53270 00000AFC 809A45418E418F80 db 128,154,069,065,142,065,143,128
53271 00000B04 4545454949498E8F db 069,069,069,073,073,073,142,143
53272 00000B0C 9092924F994F5555 db 144,146,146,079,153,079,085,085
53273 00000B14 59999A9B9C9D9E9F db 089,153,154,155,156,157,158,159
53274 00000B1C 41494F55A5A5A6A7 db 065,073,079,085,165,165,166,167
53275 00000B24 A8A9AABACADAFAF db 168,169,170,171,172,173,174,175
53276 00000B2C B0B1B2B3B4B5B6B7 db 176,177,178,179,180,181,182,183
53277 00000B34 B8B9BABBBBCDBEBF db 184,185,186,187,188,189,190,191
53278 00000B3C C0C1C2C3C4C5C6C7 db 192,193,194,195,196,197,198,199
53279 00000B44 C8C9CACBCCDCECF db 200,201,202,203,204,205,206,207
53280 00000B4C D0D1D2D3D4D5D6D7 db 208,209,210,211,212,213,214,215
53281 00000B54 D8D9DADBDCDDDEDF db 216,217,218,219,220,221,222,223
53282 00000B5C E0E1E2E3E4E5E6E7 db 224,225,226,227,228,229,230,231
53283 00000B64 E8E9EAEBECEDEEFF db 232,233,234,235,236,237,238,239
53284 00000B6C F0F1F2F3F4F5F6F7 db 240,241,242,243,244,245,246,247
53285 00000B74 F8F9FABFBCFDFEFF db 248,249,250,251,252,253,254,255
53286
53287 ;
53288 ; file upper case table
53289 ; -----
53290 FILE_UCASE_TAB: ; label byte
53291 00000B7C 8000 dw 128
53292 00000B7E 809A45418E418F80 db 128,154,069,065,142,065,143,128
53293 00000B86 4545454949498E8F db 069,069,069,073,073,073,142,143
53294 00000B8E 9092924F994F5555 db 144,146,146,079,153,079,085,085
53295 00000B96 59999A9B9C9D9E9F db 089,153,154,155,156,157,158,159
53296 00000B9E 41494F55A5A5A6A7 db 065,073,079,085,165,165,166,167
53297 00000BA6 A8A9AABACADAFAF db 168,169,170,171,172,173,174,175
53298 00000BAE B0B1B2B3B4B5B6B7 db 176,177,178,179,180,181,182,183
53299 00000BB6 B8B9BABBBBCDBEBF db 184,185,186,187,188,189,190,191
53300 00000BBE C0C1C2C3C4C5C6C7 db 192,193,194,195,196,197,198,199
53301 00000BC6 C8C9CACBCCDCECF db 200,201,202,203,204,205,206,207
53302 00000BCE D0D1D2D3D4D5D6D7 db 208,209,210,211,212,213,214,215

```


53302	00000BD6	D8D9DADBDCDDDEDF	db	216, 217, 218, 219, 220, 221, 222, 223
53303	00000BDE	E0E1E2E3E4E5E6EF	db	224, 225, 226, 227, 228, 229, 230, 231
53304	00000BEE	E8E9EAEBCEDDEEFF	db	232, 233, 234, 235, 236, 237, 238, 239
53305	00000BEF	F0F1F2F3F4F5F6F7	db	240, 241, 242, 243, 244, 245, 246, 247
53306	00000BF6	F8F9FABFCFDFEFF	db	248, 249, 250, 251, 252, 253, 254, 255

```

; 24/03/2024 - Retro DOS v5.0
; PC DOS 7.1 IBMDOS.COM
%if 0
; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0BB1h

```

```
; file char list
```

```
FILE_CHAR_TAB:      ; label byte
dw      22          ; length
db      1,0,255     ; include all
db      0,0,20h     ; exclude 0 - 20h
db      2,14,'.'"/\[]:|<>+=;',' ; exclude 14 special
;db      24 dup (?) ; reserved
times 24 db 0
%endif
```

```
; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0BE1h
```

```
; 24/03/2024
; PCDOS 7.1 IBMDOS.COM - DOSDATA:0BFEh
```

```
; collate table
```

```
COLLATE_TAB:          ; label byte
```

[illegible]

```

; -----<MSKK01>

```

```
; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0CE3h
; 24/03/2024
; PC DOS 7.1 IBMDOS.COM - DOSDATA:0D00h
```

; 29/04/2019

```
; dBCS is not supported in DOS 3.3
;          DBCS_TAB          CC_D
```

; DBCS for DOS 4.00 2/12/KK

```
DBCS_TAB:    ; label byte                ;AN000; 2/12/KK
```

100-443888-100

```

;ifdef      DBCS
;ifndef     JAPAN
dw          6                      ; <MSKK01>
db          081h,09fh             ; <MSKK01>
db          0e0h,0fch             ; <MSKK01>
db          0,0                   ; <MSKK01>

db          0,0,0,0,0,0,0,0,0,0   ; <MSKK01>

```

```

; ifdef TAIWAN
; dw 4 ; <TAIWAN>
; db 081h,0FEh ; <TAIWAN>
; db 0,0 ; <TAIWAN>

```

```

; ... db 0,0,0,0,0,0,0,0,0,0,0,0,0,0

```

```

; ifdef KOREA ; Key1
; dw 4 ; <KOREA>
; db 0A1h,0FEh ; <KOREA>
; db 0,0 ; <KOREA>

```

```

; ... db 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0

```

```

;else
    dw    0          ;AN000; 2/12/KK    max number
    ;db    16 dup(0) ;AN000; 2/12/KK
    times 16 db 0

```

```

;      dw      6      ; 2/12/KK
;      db      081h,09Fh ; 2/12/KK
;      db      0E0h,0FCh ; 2/12/KK
;      db      0,0      ; 2/12/KK

```

```

;endif

```

```
; 24/03/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D12h
```

```

IBM DOSVERSION:
;db 7
;db 10 ; MSVERSION
db MAJOR_VERSION
db MINOR_VERSION

```

```

53426
53427 00000D14 C8A6          YRTAB:      db 200,166      ; Leap Year
53428 00000D16 C8A5C8A5C8A5    db 200,165,200,165,200,165
53429
53430 00000D1C 1F             MONTAB:      db 31
53431 february:
53432 00000D1D 1C1F1E1F1E1F1E1F- db 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
53433 00000D26 1E1F
53434
53435
53436 ; 24/03/2024 - Retro DOS v5.0
53437 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D28h
53438 %if 1
53439 ;
53440 ; file char list
53441 ; -----
53442 FILE_CHAR_TAB:      ; label byte
53443 dw 22                ; length
53444 db 1,0,255           ; include all
53445 db 0,0,20h           ; exclude 0 - 20h
53446 db 2,14,'.'/'\[]:|<>+;,,' ; exclude 14 special
53447 ;db 24 dup (?)        ; reserved
53448 times 24 db 0
53449 %endif
53450 ; 24/03/2024 - Retro DOS v5.0
53451 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D58h
53452 %if 1
53453 SysInitTable:
53454 dw DPBHEAD           ; SYSINITVARS
53455 dw 0
53456 dw COUNTRY_CDPG
53457 dw 0
53458 db 0,0,0
53459 TEMPSEG:
53460 dw 0
53461 redir_patch:
53462 db 0
53463 DosHashMA:
53464 db 0
53465 FixExePatch:
53466 dw 0
53467 RationalPatchPtr:
53468 dw 0
53469 %endif
53470
53471 ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0CF5h
53472 ; 24/03/2024
53473 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D6Bh
53474
53475 ; -----
53476 ;
53477 ;CASE MAPPER ROUTINE FOR 80H-FFH character range, DOS 3.3
53478 ; ENTRY: AL = Character to map
53479 ; EXIT: AL = The converted character
53480 ; Alters no registers except AL and flags.
53481 ; The routine should do nothing to chars below 80H.
53482 ; -----
53483 ; Example:
53484
53485 MAP_CASE:
53486 ;Procedure MAP_CASE,FAR
53487
53488 CMP AL,80h
53489 ;JAE short Map1 ;Map no chars below 80H ever
53490 ;RETF
53491 ; 24/03/2024 (PCDOS 7.1 IBMDOS.COM)
53492 ;;;
53493 jnb short L_RET ;Map no chars below 80H ever
53494 ;;;
53495 Map1:
53496 SUB AL,80h ;Turn into index value
53497 PUSH DS
53498 PUSH BX
53499 MOV BX,UCASE_TAB+2
53500 FINISH:
53501 PUSH CS ;Move to DS
53502 POP DS
53503 XLAT ;Get upper case character
53504 POP BX
53505 POP DS
53506 L_RET:
53507 RETF
53508
53509 ;EndProc MAP_CASE
53510
53511 ; -----
53512 ; 24/03/2024 - Retro DOS v5.0
53513 ; PCDOS 7.1 IBMDOS.COM
53514 %if 0
53515
53516 ; The variables for ECS version are moved here for the same data alignments
53517 ; as IBM-DOS and MS-DOS.
53518
53519 InterChar:
53520 db 0 ; Interim character flag ( 1= interim) ;AN000;
53521 ;----- NOTE: NEXT TWO BYTES SOMETIMES USED AS A WORD !! -----
53522 DUMMY: ; LABEL WORD
53523 InterCon:
53524 db 0 ; Console in Interim mode ( 1= interim) ;AN000;
53525 SaveCurFlg:
53526 db 0 ; Print, do not advance cursor flag ;AN000;
53527
53528 ; -----
53529
53530 ; 17/01/2024 - Retro DOS v5.0
53531 TEMPSEG: dw 0 ;hkn; used to store ds.
53532 ;redir_patch:
53533 ; db 0
53534
53535 ; DOSDATA:0D0Dh
53536
53537 Mark1: ; label byte
53538
53539 ;IF2
53540 ; IF ((OFFSET MARK1) GT (OFFSET MSVERSION) )
53541 ; %OUT !DATA CORRUPTION!MARK1 OFFSET TOO BIG. RE-ORGANIZE DATA.
53542 ;
53543 ; ENDF
53544 ;ENDIF
53545
53546 times 5 db 0
53547

```



```

53548 ; #####
53549 ;
53550 ; ** HACK FOR DOS 4.0 REDIR **
53551 ;
53552 ; The redir requires the following:
53553 ;
53554 ;     MSVERS offset D12H
53555 ;     YRTAB  offset D14H
53556 ;     MONTAB offset D1CH
53557 ;
53558 ; WARNING! WARNING!
53559 ;
53560 ; MARK1 SHOULD NOT BE >= 0D12H. IF SOME VARIABLE IS TO BE ADDED ABOVE DO SO
53561 ; WITHOUT VIOLATING THIS AND UPDATE THE FOLL. LINE
53562 ;
53563 ; CURRENTLY MARK1 = 0D0DH
53564 ;
53565 ; #####
53566 ;
53567 ; ORG 0D12h
53568 ;
53569 ; DOSDATA:0D12h (MSDOS 6.21, MSDOS.SYS)
53570 ;
53571 ; db 6
53572 ; db 20
53573 ;
53574 ; Offset 0C78h in IBMDOS.COM (MSDOS 3.3, 1987)
53575 MSVERSION: ; MS-DOS version in hex for $GET_VERSION
53576 MSMAJORV: DB MAJOR_VERSION ; DOS_MAJOR_VERSION
53577 MSMINORV: DB MINOR_VERSION ; DOS_MINOR_VERSION
53578 ;
53579 ; YRTAB & MONTAB moved from TABLE segment in ms_table.asm
53580 ;
53581 ; I_am YRTAB,8,<200,166,200,165,200,165,200,165>
53582 ; I_am MONTAB,12,<31,28,31,30,31,30,31,31,30,31,30,31>
53583 ;
53584 ; Days in year
53585 YRTAB:
53586 DB 200,166 ; Leap year
53587 DB 200,165
53588 DB 200,165
53589 DB 200,165
53590 ;
53591 ; Days of each month
53592 MONTAB:
53593 DB 31 ; January
53594 february:
53595 DB 28 ; February--reset each
53596 ; time year changes
53597 DB 31 ; March
53598 DB 30 ; April
53599 DB 31 ; May
53600 DB 30 ; June
53601 DB 31 ; July
53602 DB 31 ; August
53603 DB 30 ; September
53604 DB 31 ; October
53605 DB 30 ; November
53606 DB 31 ; December
53607 ;
53608 ; -----THE FOLL. BLOCK MOVED FROM TABLE SEG IN MS_TABLE.ASM-----
53609 ;
53610 ; SYS init extended table, DOS 3.3 F.C. 5/29/86
53611 SysInitTable:
53612 ; dw SYSINITVAR ; pointer to sysinit var
53613 dw SYSINITVARS ; segment
53614 dw 0 ; segment
53615 dw COUNTRY_CDPG ; pointer to country tabl
53616 dw 0 ; segment of pointer
53617 ;
53618 ;;;
53619 ; 17/01/2024 - Retro DOS v5.0
53620 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D60h
53621 ;
53622 times 3 db 0
53623 TEMPSEG:
53624 dw 0
53625 redir_patch:
53626 db 0
53627 ;
53628 ;%endif
53629 ;
53630 ; 17/01/2024
53631 ;%if 1
53632 ;
53633 ; M021-
53634 ;
53635 ; DosHashMA - This flag is set by seg_reinit when the DOS actually
53636 ; takes control of the HMA. When running, this word is a reliable
53637 ; indicator that the DOS is actually using HMA. You can't just use
53638 ; CS, because ROMDOS uses HMA with CS < F000.
53639 ;
53640 DosHashMA:
53641 db 0
53642 ;
53643 ; FixExePatch:
53644 dw 0 ; M012
53645 ;
53646 ;%endif
53647 ;
53648 ; UnknownPatch:
53649 dw 0
53650 ;;;
53651 ;
53652 ; 17/01/2024
53653 ;%if 0
53654 ;
53655 ; DOS 3.3 F.C. 6/12/86
53656 ; FASTOPEN communications area DOS 3.3 F.C. 5/29/86
53657 ;
53658 ; FastTable: ; a better name
53659 FastOpenTable:
53660 dw 2 ; number of entries
53661 dw FastRet ; pointer to ret instr.
53662 dw 0 ; and will be modified by
53663 dw FastRet ; FASTxxx when loaded in
53664 dw 0
53665 ;
53666 ; DOS 3.3 F.C. 6/12/86
53667 ;
53668 ; FastFlg: ; flags
53669 FastOpenFlg:
53670 ;
53671 ;

```

```

53672         db      0                ; don't change the foll: order
53673
53674     ;%endif
53675
53676     ; FastOpen_Ext_Info is used as a temporary storage for saving dirpos,dirsec
53677     ; and clusnum which are filled by DOS 3.nc when calling FastOpen Insert
53678     ; or filled by FastOpen when calling FastOpen Lookup
53679
53680     FastOpen_Ext_Info: ; label byte ;dirpos
53681         ;db      SIZE FASTOPEN_EXTENDED_INFO dup(0)
53682         ;times 11 db 0
53683         times FEI.size db 0
53684
53685     %endif      ; 24/03/2024
53686
53687     ; Dir_Info_Buff is a dir entry buffer which is filled by FastOpen
53688     ; when calling FastOpen Lookup
53689
53690     Dir_Info_Buff: ; label byte
53691         ;db      SIZE dir_entry dup (0)
53692         ;times 32 db 0
53693         times dir_entry.size db 0
53694
53695     Next_Element_Start:
53696         dw      0                ; save next element start offset
53697
53698     ; 24/03/2024
53699     %if 0
53700         ; MSDOS 6.22 MSDOS.SYS - DOSDATA:0D68h
53701
53702     Del_ExtCluster:
53703         dw      0                ; for dos_delete
53704     %endif
53705
53706     ; The following is a stack and its pointer for interrupt 2F which is used
53707     ; by NLSFUNC. There is no significant use of this stack, we are just trying
53708     ; not to destroy the INT 21 stack saved for the user.
53709
53710         ; MSDOS 6.22 MSDOS.SYS - DOSDATA:0D6Ah
53711         ; 24/03/2024
53712         ; PC DOS 7.1 IBMDOS.COM - DOSDATA:0D9Eh
53713
53714     USER_SP_2F: ; LABEL WORD
53715         dw      FAKE_STACK_2F
53716
53717     Packet_Temp: ; label word ; temporary packet used by readtime
53718     DOS_TEMP: ; label word ; temporary word
53719
53720     FAKE_STACK_2F:
53721         ; dw      14 dup (0) ; 12 register temporary storage
53722         times 14 dw 0
53723
53724     ; 24/03/2024 - Retro DOS v5.0
53725     ; PC DOS 7.1 IBMDOS.COM
53726     %if 0
53727     Hash_Temp: ; label word ; temporary word
53728         ;dw      4 dup (0) ; temporary hash table during config.sys
53729         times 4 dw 0
53730     %endif
53731
53732     SCAN_FLAG:
53733         db      0                ; flag to indicate key ALT_Q
53734
53735     DATE_FLAG:
53736         dw      0                ; flag to update the date
53737
53738     ; 24/03/2024
53739     %if 0
53740         ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0D93h
53741
53742     FETCHI_TAG: ; label word ; OBSOLETE - no longer used
53743         dw      0                ; formerly part of IBM's piracy protection
53744
53745     MSG_EXTERROR: ; label DWORD ; for system message addr
53746         dd      0                ; for extended error
53747         dd      0                ; for parser
53748         dd      0                ; for critical error
53749         dd      0                ; for IFS
53750         dd      0                ; for code reduction
53751
53752     SEQ_SECTOR: ; label DWORD ; last sector read
53753         dd      -1
53754
53755     SC_SECTOR_SIZE:
53756         dw      0                ; sector size for SC
53757
53758     SC_DRIVE:
53759         db      0                ; drive # for secondary cache
53760
53761     ; 17/01/2024
53762     CurSC_DRIVE:
53763         ; db      -1                ; current SC drive
53764
53765     CurSC_SECTOR:
53766         dd      0                ; current SC starting sector
53767
53768     SC_STATUS:
53769         dw      0                ; SC status word
53770
53771     SC_FLAG:
53772         db      0                ; SC flag
53773
53774     %endif
53775     ; 24/03/2024
53776     ; PC DOS 7.1 IBMDOS.COM - DOSDATA:0DBFh
53777     ; (MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0DB8h)
53778     AbsDskErr:
53779         dw      0                ; Storage for Abs disk read/write err
53780
53781     NO_NAME_ID: ; label byte,
53782         db      NO_NAME ; null media id
53783
53784     ;hkn; moved from TABLE segment in kstrin.asm
53785
53786     KISTR001S: ; label byte ; 2/17/KK
53787     LOOKSIZ: db 0 ; 0 if byte, NZ if word 2/17/KK
53788     KISTR001E: ; label byte ; 2/17/KK
53789
53790     ; the nul device driver used to be part of the code. However, since the
53791     ; header is in the data, and the entry points are only given as an offset,
53792     ; the strategy and interrupt entry points must also be in the data now.
53793
53794     ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0DC6h
53795     ; 24/03/2024
53796     ; PC DOS 7.1 IBMDOS.COM - DOSDATA:0DCDh
53797
53798     SNULDEV:
53799     ;procedure snuldev,far

```

```

53795         ;or      word [es:bx+3],100h
53796         ; 17/12/2022
53797         ;or      byte [es:bx+4],01h
53798         ; 05/01/2024 - Retro DOS v4.2 (*)
53799         ; (Original MSDOS and RetroDOS DATA address compatibility) (*)
53800         ;or      byte [es:bx+SRHEAD.REQSTAT+1],(STDON>>8)
53801         ; 24/03/2024 (PCDOS 7.1 IBMDOS.COM)
53802 00000DCD 26814F030001 or      word [es:bx+SRHEAD.REQSTAT],STDON ; set done bit
53803         ; 05/01/2024 - Retro DOS v5.0
53804         ;or      byte [es:bx+SRHEAD.REQSTAT+1],(STDON>>8)
53805 INULDEV:
53806 00000DD3 CB      retf                ; must not be a return!
53807 ;endproc snuldev
53808
53809 ;M044
53810 ; Second part of save area for saving last para of windows memory
53811
53812 ; 17/01/2024
53813 ;winoldPatch2:
53814 ;      ;db      8 dup (?)          ; M044
53815 ;      times 8 db 0
53816
53817         ; 24/03/2024 (PCDOS 7.1 IBMDOS.COM)
53818 00000DD4 00      db      0
53819
53820 UmbSave2:
53821         ;db      5 dup (?)          ; M062
53822 00000DD5 00<rep 5h> times 5 db 0
53823
53824 00000DDA 00      db      0          ; M062
53825
53826 ; DOSDATA:0DDbH
53827
53828 Mark2:      ; label byte
53829
53830 ;IF2
53831 ;      IF ((OFFSET MARK2) GT (OFFSET ERR_TABLE_21) )
53832 ;          %OUT !DATA CORRUPTION!MARK2 OFFSET TOO BIG. RE-ORGANIZE DATA.
53833 ;      ENDIF
53834 ;ENDIF
53835
53836 ;#####
53837 ;
53838 ; ** HACK FOR DOS 4.0 REDIR **
53839 ;
53840 ; The redir requires the following:
53841 ;
53842 ;      ERR_TABLE_21      offset DDBH
53843 ;      ERR_TABLE_24      offset E5BH
53844 ;      ErrMap24          offset EABH
53845 ;
53846 ; WARNING! WARNING!
53847 ;
53848 ; MARK2 SHOULD NOT BE >= 0DDbH. IF SOME VARIABLE IS TO BE ADDED ABOVE DO SO
53849 ; WITHOUT VIOLATING THIS AND UPDATE THE FOLL. LINE
53850 ;
53851 ; CURRENTLY MARK2 = 0DD0H
53852 ;
53853 ;#####
53854
53855         ;ORG      0DDbH
53856
53857 ; DOSDATA:0DDbH (MSDOS 6.21, MSDOS.SYS)
53858 ; 24/03/2024
53859 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0DDbH
53860
53861 ; -----
53862 ;
53863 ; The following table defines CLASS ACTION and LOCUS info for the INT 21H
53864 ; errors. Each entry is 4 bytes long:
53865 ;
53866 ;      Err#,Class,Action,Locus
53867 ;
53868 ; A value of 0FFh indicates a call specific value (ie. should already
53869 ; be set). AN ERROR CODE NOT IN THE TABLE FALLS THROUGH TO THE CATCH ALL AT
53870 ; THE END, IT IS ASSUMES THAT CLASS, ACTION, LOCUS IS ALREADY SET.
53871 ;
53872 ; -----
53873
53874 ;ErrTab Macro      err,class,action,locus
53875 ;ifidn <locus>,<0FFh>
53876 ;      DB      error_&err,errCLASS_&class,errACT_&action,0FFh
53877 ;ELSE
53878 ;      DB      error_&err,errCLASS_&class,errACT_&action,errLOC_&locus
53879 ;ENDIF
53880 ;ENDM
53881
53882 ERR_TABLE_21: ; LABEL      BYTE
53883 00000DDB 010704FF      DB      error_invalid_function,      errCLASS_Apperr,      errACT_Abort,      0FFh
53884 00000DDF 02080302      DB      error_file_not_found,      errCLASS_NotFnd,      errACT_User,      errLOC_Disk
53885 00000DE3 03080302      DB      error_path_not_found,      errCLASS_NotFnd,      errACT_User,      errLOC_Disk
53886 00000DE7 04010401      DB      error_too_many_open_files,      errCLASS_OutRes,      errACT_Abort,      errLOC_Unk
53887 00000DEB 050303FF      DB      error_access_denied,      errCLASS_Auth,      errACT_User,      0FFh
53888 00000DEF 06070401      DB      error_invalid_handle,      errCLASS_Apperr,      errACT_Abort,      errLOC_Unk
53889 00000DF3 07070505      DB      error_arena_trashed,      errCLASS_Apperr,      errACT_Panic,      errLOC_Mem
53890 00000DF7 08010405      DB      error_not_enough_memory,      errCLASS_OutRes,      errACT_Abort,      errLOC_Mem
53891 00000DFB 09070405      DB      error_invalid_block,      errCLASS_Apperr,      errACT_Abort,      errLOC_Mem
53892 00000DFE 0A070405      DB      error_bad_environment,      errCLASS_Apperr,      errACT_Abort,      errLOC_Mem
53893 00000E03 08090301      DB      error_bad_format,      errCLASS_BadFmt,      errACT_User,      errLOC_Unk
53894 00000E07 0C070401      DB      error_invalid_access,      errCLASS_Apperr,      errACT_Abort,      errLOC_Unk
53895 00000E0B 0D090401      DB      error_invalid_data,      errCLASS_BadFmt,      errACT_Abort,      errLOC_Unk
53896 00000E0F 0F080302      DB      error_invalid_drive,      errCLASS_NotFnd,      errACT_User,      errLOC_Disk
53897 00000E13 10030302      DB      error_current_directory,      errCLASS_Auth,      errACT_User,      errLOC_Disk
53898 00000E17 110D0302      DB      error_not_same_device,      errCLASS_Unk,      errACT_User,      errLOC_Disk
53899 00000E1B 12080302      DB      error_no_more_files,      errCLASS_NotFnd,      errACT_User,      errLOC_Disk
53900 00000E1F 500C0302      DB      error_file_exists,      errCLASS_Already,      errACT_User,      errLOC_Disk
53901 00000E23 200A0202      DB      error_sharing_violation,      errCLASS_Locked,      errACT_DlyRet,      errLOC_Disk
53902 00000E27 210A0202      DB      error_lock_violation,      errCLASS_Locked,      errACT_DlyRet,      errLOC_Disk
53903 00000E2B 540104FF      DB      error_out_of_structures,      errCLASS_OutRes,      errACT_Abort,      0FFh
53904 00000E2F 56030301      DB      error_invalid_password,      errCLASS_Auth,      errACT_User,      errLOC_Unk
53905 00000E33 52010402      DB      error_cannot_make,      errCLASS_OutRes,      errACT_Abort,      errLOC_Disk
53906 00000E37 32090303      DB      error_not_supported,      errCLASS_BadFmt,      errACT_User,      errLOC_Net
53907 00000E3B 550C0303      DB      error_already_assigned,      errCLASS_Already,      errACT_User,      errLOC_Net
53908 00000E3F 57090301      DB      error_invalid_parameter,      errCLASS_BadFmt,      errACT_User,      errLOC_Unk
53909 00000E43 530D0401      DB      error_FAIL_I24,      errCLASS_Unk,      errACT_Abort,      errLOC_Unk
53910 00000E47 24010405      DB      error_sharing_buffer_exceeded, errCLASS_OutRes,      errACT_Abort,      errLOC_Mem
53911 ; MSDOS 6.0
53912 00000E4B 26010401      DB      error_handle_EOF,      errCLASS_OutRes,      errACT_Abort,      errLOC_Unk ;AN000;
53913 00000E4F 27010401      DB      error_handle_Disk_Full,      errCLASS_OutRes,      errACT_Abort,      errLOC_Unk ;AN000;
53914 00000E53 5A0D0402      DB      error_sys_comp_not_loaded,      errCLASS_Unk,      errACT_Abort,      errLOC_Disk ;AN001;
53915 00000E57 FFFFFFFF      DB      0FFh,      0FFh,      0FFh,      0FFh
53916
53917 ; MSDOS 3.3 (IBMDOS.COM, 1987) - Offset 0D2Ah
53918 ;ERR_TABLE_21:      db 1,7,4,0FFh

```

```

53919 ; db 2,8,3,2
53920 ; db 3,8,3,2
53921 ; db 4,1,4,1
53922 ; db 5,3,3,0FFh
53923 ; db 6,7,4,1
53924 ; db 7,7,5,5
53925 ; db 8,1,4,5
53926 ; db 9,7,4,5
53927 ; db 0Ah,7,4,5
53928 ; db 0Bh,9,3,1
53929 ; db 0Ch,7,4,1
53930 ; db 0Dh,9,4,1
53931 ; db 0Fh,8,3,2
53932 ; db 10h,3,3,2
53933 ; db 11h,0Dh,3,2
53934 ; db 12h,8,3,2
53935 ; db 50h,0Ch,3,2
53936 ; db 20h,0Ah,2,2
53937 ; db 21h,0Ah,2,2
53938 ; db 54h,1,4,0FFh
53939 ; db 56h,3,3,1
53940 ; db 52h,1,4,2
53941 ; db 32h,9,3,3
53942 ; db 55h,0Ch,3,3
53943 ; db 57h,9,3,1
53944 ; db 53h,0Dh,4,1
53945 ; db 24h,1,4,5
53946 ; MSDOS 6.0 (MSDOS 6.21)
53947 ; db 26h,1,4,1
53948 ; db 27h,1,4,1
53949 ; db 5Ah,0Dh,4,2
53950 ; MSDOS 6.0 & MSDOS 3.3
53951 ; db 0FFh,0FFh,0FFh,0FFh
53952 ;
53953 ; DOSDATA:0E5Bh (MSDOS 6.21, MSDOS.SYS)
53954 ;
53955 ; -----
53956 ;
53957 ; The following table defines CLASS ACTION and LOCUS info for the INT 24H
53958 ; errors. Each entry is 4 bytes long:
53959 ;
53960 ; Err#,Class,Action,Locus
53961 ;
53962 ; A Locus value of 0FFh indicates a call specific value (ie. should already
53963 ; be set). AN ERROR CODE NOT IN THE TABLE FALLS THROUGH TO THE CATCH ALL AT
53964 ; THE END.
53965 ;
53966 ; -----
53967 ;
53968 ; ERR_TABLE_24: ; LABEL BYTE
53969 ; DB error_write_protect, errCLASS_Media, errACT_IntRet, errLOC_Disk
53970 ; DB error_bad_unit, errCLASS_Intrn, errACT_Panic, errLOC_Unk
53971 ; DB error_not_ready, errCLASS_HrdFail, errACT_IntRet, 0FFh
53972 ; DB error_bad_command, errCLASS_Intrn, errACT_Panic, errLOC_Unk
53973 ; DB error_CRC, errCLASS_Media, errACT_Abort, errLOC_Disk
53974 ; DB error_bad_length, errCLASS_Intrn, errACT_Panic, errLOC_Unk
53975 ; DB error_seek, errCLASS_HrdFail, errACT_Retry, errLOC_Disk
53976 ; DB error_not_DOS_disk, errCLASS_Media, errACT_IntRet, errLOC_Disk
53977 ; DB error_sector_not_found, errCLASS_Media, errACT_Abort, errLOC_Disk
53978 ; DB error_out_of_paper, errCLASS_TempSit, errACT_IntRet, errLOC_SerDev
53979 ; DB error_write_fault, errCLASS_HrdFail, errACT_Abort, 0FFh
53980 ; DB error_read_fault, errCLASS_HrdFail, errACT_Abort, 0FFh
53981 ; DB error_gen_failure, errCLASS_Unk, errACT_Abort, 0FFh
53982 ; DB error_sharing_violation, errCLASS_Locked, errACT_DlyRet, errLOC_Disk
53983 ; DB error_lock_violation, errCLASS_Locked, errACT_DlyRet, errLOC_Disk
53984 ; DB error_wrong_disk, errCLASS_Media, errACT_IntRet, errLOC_Disk
53985 ; DB error_not_supported, errCLASS_BadFmt, errACT_User, errLOC_Net
53986 ; DB error_FCB_unavailable, errCLASS_Apperr, errACT_Abort, errLOC_Unk
53987 ; DB error_sharing_buffer_exceeded, errCLASS_OutRes, errACT_Abort, errLOC_Mem
53988 ; DB 0FFh, errCLASS_Unk, errACT_Panic, 0FFh
53989 ;
53990 ; MSDOS 3.3 (IBMDOS.COM, 1987) - offset 0D9Eh
53991 ; ERR_TABLE_24: db 13h,0Bh,7,2
53992 ; db 14h,4,5,1
53993 ; db 15h,5,7,0FFh
53994 ; db 16h,4,5,1
53995 ; db 17h,0Bh,4,2
53996 ; db 18h,4,5,1
53997 ; db 19h,5,1,2
53998 ; db 1Ah,0Bh,7,2
53999 ; db 1Bh,0Bh,4,2
54000 ; db 1Ch,2,7,4
54001 ; db 1Dh,5,4,0FFh
54002 ; db 1Eh,5,4,0FFh
54003 ; db 1Fh,0Dh,4,0FFh
54004 ; db 20h,0Ah,2,2
54005 ; db 21h,0Ah,2,2
54006 ; db 22h,0Bh,7,2
54007 ; db 32h,9,3,3
54008 ; db 23h,7,4,1
54009 ; db 24h,1,4,5
54010 ; db 0FFh,0Dh,5,0FFh
54011 ;
54012 ; DOSDATA:0EABh (MSDOS 6.21, MSDOS.SYS)
54013 ;
54014 ; -----
54015 ;
54016 ; We need to map old int 24 errors and device driver errors into the new set
54017 ; of errors. The following table is indexed by the new errors
54018 ;
54019 ; -----
54020 ;
54021 ; Public ErrMap24
54022 ; ErrMap24: ; Label BYTE
54023 ; DB error_write_protect ; 0
54024 ; DB error_bad_unit ; 1
54025 ; DB error_not_ready ; 2
54026 ; DB error_bad_command ; 3
54027 ; DB error_CRC ; 4
54028 ; DB error_bad_length ; 5
54029 ; DB error_seek ; 6
54030 ; DB error_not_DOS_disk ; 7
54031 ; DB error_sector_not_found ; 8
54032 ; DB error_out_of_paper ; 9
54033 ; DB error_write_fault ; A
54034 ; DB error_read_fault ; B
54035 ; DB error_gen_failure ; C
54036 ; DB error_gen_failure ; D RESERVED
54037 ; DB error_gen_failure ; E RESERVED
54038 ; DB error_wrong_disk ; F
54039 ;
54040 ; ErrMap24: db 13h, 14h, 15h, 16h, 17h, 18h, 19h, 1Ah
54041 ; db 1Bh, 1Ch, 1Dh, 1Eh, 1Fh, 1Fh, 1Fh, 22h
54042 ;

```

```

54043 ErrMap24End: ; LABEL BYTE
54044
54045 ; DOSDATA:0EBBh (MSDOS 6.21, MSDOS.SYS)
54046
54047 ; 09/03/2024 - Retro DOS v5.0
54048 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0EBBh (SPECIAL_VERSION)
54049
54050 ; MSDOS 5.0 MSDOS.SYS - DOSDATA:0EBBh (FIRST_BUFF_ADDR)
54051 ; MSDOS 6.22 MSDOS.SYS - DOSDATA:0EBBh (FIRST_BUFF_ADDR)
54052 ; Windows ME IO.SYS - DOSDATA:0EBBh (SPECIAL_VERSION)
54053
54054 ; -----
54055
54056 ; 27/04/2019 - Retro DOS v4.0
54057
54058 ; 17/01/2024
54059 ; FIRST_BUFF_ADDR:
54060 ; dw 0 ; first buffer address
54061
54062 SPECIAL_VERSION:
54063 00000EBB 0000 dw 0 ; AN006; used by INT 2F 47H
54064
54065 ; 25/03/2024 - Retro DOS v5.0
54066 ; PCDOS 7.1 IBMDOS.COM
54067 %if 0
54068 FAKE_COUNT:
54069 times 255 db 0 ; AN008; fake version count
54070 %endif
54071
54072 OLD_FIRSTCLUS:
54073 00000EBD 0000 dw 0 ; AN011; save old first cluster for fastopen
54074
54075 ; -----
54076
54077 ;smr; moved from TABLE segment in exec.asm
54078
54079 00000EBF 0000 exec_init_SP: dw 0
54080 00000EC1 0000 exec_init_SS: dw 0
54081 00000EC3 0000 exec_init_IP: dw 0
54082 00000EC5 0000 exec_init_CS: dw 0
54083
54084 exec_signature:
54085 00000EC7 0000 dw 0 ; must contain 4D5A (yay zibo!)
54086 exec_len_mod_512:
54087 00000EC9 0000 dw 0 ; low 9 bits of length
54088 exec_pages:
54089 00000ECB 0000 dw 0 ; number of 512b pages in file
54090 exec_rle_count:
54091 00000ECD 0000 dw 0 ; count of reloc entries
54092 exec_par_dir:
54093 00000ECF 0000 dw 0 ; number of paragraphs before image
54094 exec_min_BSS:
54095 00000ED1 0000 dw 0 ; minimum number of para of BSS
54096 exec_max_BSS:
54097 00000ED3 0000 dw 0 ; max number of para of BSS
54098 exec_SS:
54099 00000ED5 0000 dw 0 ; stack of image
54100 exec_SP:
54101 00000ED7 0000 dw 0 ; SP of image
54102 exec_chksum:
54103 00000ED9 0000 dw 0 ; checksum of file (ignored)
54104 exec_IP:
54105 00000EDB 0000 dw 0 ; IP of entry
54106 exec_CS:
54107 00000EDD 0000 dw 0 ; CS of entry
54108 exec_rle_table:
54109 00000EDF 0000 dw 0 ; byte offset of reloc table
54110
54111 exec_header_len equ $-exec_signature ; PBUGBUG
54112
54113 ;smr; eom
54114
54115 ; -----
54116
54117 ;SR;
54118 ; WIN386 instance table for DOS
54119
54120 win386_Info:
54121 ;db 3,0
54122 ; 17/01/2024 - PCDOS 7.1 IBMDOS.COM - DOSDATA:0EE1h
54123 00000EE1 0400 db 4,0 ; WIN386_SIS version
54124 00000EE3 00000000 dd 0 ; .Next_Dev_Ptr
54125 win386_Inf_Virt_Dev_Ptr:
54126 00000EE7 00000000 dd 0 ; .Virt_Dev_File_Ptr
54127 00000EEB 00000000 dd 0 ; .Reference_Data
54128 Instance_Data_Ptr:
54129 00000EEF [F70E]0000 dw Instance_Table, 0
54130 ; 17/01/2024 - PCDOS 7.1 IBMDOS.COM
54131 ;;
54132 00000EF3 [3D0F]0000 dw Unknown_Table, 0 ; (what is this and what for ?)
54133 ; (win386_IIS.size)
54134 ;;
54135
54136 Instance_Table:
54137 00000EF7 [2200]00000200 dw CONTPOS,0,2
54138 00000EFD [3200]00000400 dw BCON,0,4
54139 00000F03 [F901]00000601 dw CARPOS,0,106h
54140 00000F09 [0003]00000100 dw CHARCO,0,1
54141 00000F0F [BF0E]00002200 dw exec_init_SP,0,34 ; M074
54142 00000F15 [8900]00000100 dw UMBFLAG,0,1 ; M019
54143 00000F1B [8C00]00000200 dw UMB_HEAD,0,2 ; M019
54144 ;;
54145 ; 17/01/2024 - PCDOS 7.1 IBMDOS.COM
54146 00000F21 [8600]C9000100 dw DOS_FLAG,0C9h,1
54147 00000F27 [B812]C9000100 dw INDOS_FLAG,0C9h,1 ; (what for a 2nd INDOS flag, windows?)
54148 00000F2D [B912]C9000100 dw DEVIO_IN_PROGRESS,0C9h,1
54149 ; "devio call in progress" status flag ptr
54150 ;;
54151 00000F33 00000000 dw 0, 0
54152
54153 ;;
54154 ; 17/01/2024 - PCDOS 7.1 IBMDOS.COM
54155 00000F37 FFFF dw 0FFFFh
54156 00000F39 FFFF dw 0FFFFh
54157 CLOFATENTRY_HW:
54158 00000F3B FFFF dw 0FFFFh
54159 Unknown_Table:
54160 00000F3D 0000 dw 0
54161 00000F3F C900 dw 0C9h
54162 00000F41 [CC00] dw SFTABL ; DOSDATA:00CCh
54163 00000F43 [F901] dw CARPOS
54164 00000F45 C900 dw 0C9h
54165 00000F47 [4D11] dw UNKNOWN1 ; ? (points to DOSDATA:114Dh)
54166 00000F49 0000 dw 0

```

```

54167 00000F4B 0000          dw      0
54168                      ;;;
54169
54170          ; M001; SR;
54171          ; M001; On DOSMGR call ( cx == 0 ), we need to return a table of offsets of
54172          ; M001; some DOS variables. Note that the only really important variable in
54173          ; M001; this is User_Id. The other variables are needed only to patch stuff
54174          ; M001; which does not need to be done in DOS 5.0.
54175
54176          ; 29/12/2022
54177          ; (MSDOS 6.21 MSDOS.SYS - DOSDATA:1022h)
54178          ; 25/03/2024
54179          ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0F4Dh
54180
54181          win386_DOSVars:
54182 00000F4D 05              db      5          ;Major version 5 ; M001
54183 00000F4E 00              db      0          ;Minor version 0 ; M001
54184 00000F4F [EC05]         dw      SAVEDS    ; M001
54185 00000F51 [EA05]         dw      SAVEBX    ; M001
54186 00000F53 [2103]         dw      INDOS     ; M001
54187 00000F55 [3E03]         dw      USER_ID   ; M001
54188 00000F57 [1503]         dw      CritPatch ; M001
54189 00000F59 [8C00]         dw      UMB_HEAD  ; M012
54190
54191          ;SR;
54192          ; Flag to indicate whether WIN386 is running or not
54193
54194          Iswin386:
54195 00000F5B 00              db      0
54196
54197          ; 09/03/2024 (PCDOS 7.1 IBMDOS.COM)
54198
54199 00000F5C 00              db      0
54200
54201          ; 09/03/2024 (PCDOS 7.1 IBMDOS.COM)
54202          %if 0
54203
54204          ;M018
54205          ; This variable contains the path to the vxD device needed for win386
54206
54207          ; 09/01/2024
54208          ;VxDpath: db      'c:\wina20.386',0      ;M018
54209
54210          ;End WIN386 support
54211
54212          ; -----
54213
54214          ;SR;
54215          ; These variables have been added for the special lie support for device
54216          ;drivers.
54217          ;
54218
54219          DriverLoad:
54220          db      1          ;initialized to do special handling
54221          BiosDataPtr:
54222          dd      0
54223
54224          %endif
54225
54226          ; 25/03/2024
54227          %if 1
54228          ; 29/12/2022 - Retro DOS v4.1
54229          %if 0
54230
54231          ; 27/04/2019 - Retro DOS v4.0
54232          ; 04/11/2022
54233          ; DOSDATA:1044h (MSDOS 6.21 & MSDOS 5.0, MSDOS.SYS)
54234
54235          ; -----
54236          ; Patch for Sidekick
54237          ;
54238          ; A documented method for finding the offset of the Errormode flag in the
54239          ; dos swappable data area if for the app to scan in the dos segment (data)
54240          ; for the following sequence of instructions.
54241          ;
54242          ; Ref: Part C, Article 11, pg 356 of MSDOS Encyclopedia
54243          ;
54244          ; The Offset of Errormode flag is 0320h
54245          ;
54246          ; -----
54247
54248 00000F5D 36F6062003FF     db      036h, 0F6h, 06h, 020h, 03h, 0FFh ; test ss:[errormode], -1
54249 00000F63 750C            db      075h, 0Ch          ; jnz NearLabel
54250 00000F65 36FF365803     db      036h, 0FFh, 036h, 058h, 03h ; push ss:[NearWord]
54251 00000F6A CD28            db      0CDh, 028h          ; int 28h
54252
54253          ; -----
54254          ; Patch for PortOfEntry - M036
54255          ;
54256          ; PortOfEntry by Sector Technology uses an un documented way of determining
54257          ; the offset of Errormode flag. The following patch is to support them in
54258          ; DOS 5.0. The corresponding code is actually in msdisp.asm
54259          ;
54260          ; -----
54261
54262 00000F6C 803E200300       db      080h, 03Eh, 020h, 03h, 00h ; cmp [errormode], 0
54263 00000F71 7537            db      075h, 037h          ; jnz NearLabel
54264 00000F73 BCA00A          db      0BCh, 0A0h, 0Ah          ; mov sp, dosdata:iostack
54265
54266          %endif ; 29/12/2022
54267
54268          ; DOSDATA:105Dh (MSDOS 6.21, MSDOS.SYS)
54269          ; 25/03/2024
54270          ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0F76h
54271
54272          ; -----
54273
54274          ;*** New FCB Implementation
54275          ; This variable is used as a cache in the new FCB implementation to remember
54276          ; the address of a local SFT that can be recycled for a regenerate operation
54277
54278 00000F76 00000000       LocalSFT: dd      0          ; 0 to indicate invalid pointer
54279
54280          ;DOSDATA ENDS
54281
54282          ; =====
54283          ; LMSTUB.ASM (MSDOS 6.0, 1991)
54284          ; =====
54285          ; 27/04/2019 - Retro DOS 4.0
54286          ; 25/03/2024 - Retro DOS 5.0
54287
54288          ;DOSDATA SEGMENT WORD PUBLIC 'DATA'
54289
54290          ; -----

```



```

54291 ; Low Memory Stub for DOS when DOS runs in HMA
54292 ;-----
54293 ;db 90h
54294
54295 ;EVEN
54296 align 2
54297
54298 ; DOSDATA:1062h (MSDOS 5.0-6.22, MSDOS.SYS)
54299 ; 25/03/2024
54300 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0F7Ah
54301
54302 DOSINTTABLE: ; LABEL DWORD
54303
54304 ;DW OFFSET DOSCODE:DIVOV , 0
54305 ;DW OFFSET DOSCODE:QUIT , 0
54306 ;DW OFFSET DOSCODE:COMMAND , 0
54307 ;DW OFFSET DOSCODE:ABSDRD , 0
54308 ;DW OFFSET DOSCODE:ABSDWRT , 0
54309 ;DW OFFSET DOSCODE:Stay_resident , 0
54310 ;DW OFFSET DOSCODE:INT2F , 0
54311 ;DW OFFSET DOSCODE:CALL_ENTRY , 0
54312 ;DW OFFSET DOSCODE:IRETT , 0
54313
54314 dw DIVOV , 0 ; DOSINTTABLE+0
54315 00000F7A [D05F]0000 dw QUIT , 0 ; DOSINTTABLE+4
54316 00000F7E [C502]0000 dw COMMAND , 0 ; DOSINTTABLE+8
54317 00000F82 [F102]0000 dw ABSDRD , 0 ; DOSINTTABLE+12
54318 00000F86 [2D05]0000 dw ABSDWRT , 0 ; DOSINTTABLE+16
54319 00000F8A [DF05]0000 dw STAY_RESIDENT , 0 ; DOSINTTABLE+20
54320 00000F8E [3972]0000 dw INT2F , 0 ; DOSINTTABLE+24
54321 00000F92 [3A07]0000 dw CALL_ENTRY , 0 ; DOSINTTABLE+28
54322 00000F96 [CC02]0000
54323
54324 ; 25/03/2024 - Retro DOS v5.0
54325 ; PCDOS 7.1 IBMDOS.COM
54326 %if 0
54327 dw IRETT , 0 ; DOSINTTABLE+32
54328 %endif
54329
54330 SS_Save: dw 0 ; save user's stack segment
54331 SP_Save: dw 0 ; save user's stack offset
54332
54333 ;-----
54334 ;
54335 ; LOW MEM STUB:
54336 ;
54337 ; The low mem stub contains the entry points into DOS for all interrupts
54338 ; handled by DOS. This stub is installed if the user specifies that the
54339 ; DOS load in HIMEM. Each entry point does this.
54340 ;
54341 ; 1. if jmp to 8 has been patched out
54342 ; 2. if A20 OFF
54343 ; 3. Enable A20
54344 ; 4. else
54345 ; 5. just go to dos entry
54346 ; 6. endif
54347 ; 7. else
54348 ; 8. just go to dos entry
54349 ; 9. endif
54350 ;
54351 ;-----
54352
54353 ; 27/04/2019 - Retro DOS v4.0
54354
54355 ; DOSDATA:108Ah (MSDOS 6.21, MSDOS.SYS)
54356
54357 ; 25/03/2024 - Retro DOS v5.0
54358 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0F9Eh
54359
54360 ;-----
54361 ;
54362 ; DIVIDE BY 0 handler
54363 ;
54364 ;-----
54365
54366 ldivov:
54367 ; The following jump, skipping the XMS calls will be patched to
54368 ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54369 ; needed because the stub is installed even before the XMS driver
54370 ; is loaded if the user specifies dos=high in the config.sys
54371 i0patch:
54372 00000F9E EB03 jmp short divov_cont
54373
54374 00000FA0 E8E200 call EnsureA20ON ; we must turn on A20 if OFF
54375 divov_cont:
54376 00000FA3 2EFF2E[7A0F] jmp far [cs:DOSINTTABLE] ; jmp to DOS
54377
54378 ;-----
54379 ;
54380 ; INT 20h Handler
54381 ;
54382 ; Here we do not have to set up the stack to return here as the abort call
54383 ; will return to the address after the int 21 ah=4b call. This would be the
54384 ; common exit point if A20 had been OFF (for TOGGLE DOS) and the A20 line
54385 ; will be restored then.
54386 ;
54387 ;-----
54388
54389 lquit:
54390 ; The following jump, skipping the XMS calls will be patched to
54391 ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54392 ; needed because the stub is installed even before the XMS driver
54393 ; is loaded if the user specifies dos=high in the config.sys
54394 i20patch:
54395 00000FA8 EB03 jmp short quit_cont
54396
54397 00000FAA E8D800 call EnsureA20ON ; we must turn on A20 if OFF
54398 quit_cont:
54399 00000FAD 2EFF2E[7E0F] jmp far [cs:DOSINTTABLE+4] ; jump to DOS
54400
54401 ;-----
54402 ;
54403 ; INT 21h Handler
54404 ;
54405 ;-----
54406
54407 lcommand:
54408 ; The following jump, skipping the XMS calls will be patched to
54409 ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54410 ; needed because the stub is installed even before the XMS driver
54411 ; is loaded if the user specifies dos=high in the config.sys
54412 i21patch:
54413 00000FB2 EB03 jmp short command_cont
54414

```

```

54415 00000FB4 E8CE00      call    EnsureA20ON      ; we must turn on A20 if OFF
54416 command_cont:
54417 00000FB7 2EFF2E[820F] jmp     far [cs:DOSINTTABLE+8]; jmp to DOS
54418
54419
54420
54421 ;
54422 ; INT 25h
54423 ;
54424 ;-----
54425
54426 labsdrd:
54427 ; The following jump, skipping the XMS calls will be patched to
54428 ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54429 ; needed because the stub is installed even before the XMS driver
54430 ; is loaded if the user specifies dos=high in the config.sys
54431 00000FBC EB03      jmp     short absdrd_cont
54432
54433 00000FBE E8C400      call    EnsureA20ON      ; we must turn on A20 if OFF
54434 absdrd_cont:
54435 00000FC1 2EFF2E[860F] jmp     far [cs:DOSINTTABLE+12] ; jmp to DOS
54436
54437
54438
54439 ;
54440 ; INT 26h
54441 ;
54442 ;-----
54443
54444 labsdwr:
54445 ; The following jump, skipping the XMS calls will be patched to
54446 ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54447 ; needed because the stub is installed even before the XMS driver
54448 ; is loaded if the user specifies dos=high in the config.sys
54449 00000FC6 EB03      jmp     short absdwr_cont
54450
54451 00000FC8 E8BA00      call    EnsureA20ON      ; we must turn on A20 if OFF
54452 absdwr_cont:
54453 00000FCB 2EFF2E[8A0F] jmp     far [cs:DOSINTTABLE+16] ; jmp to DOS
54454
54455
54456
54457 ;
54458 ; INT 27h
54459 ;
54460 ;-----
54461
54462 lstay_resident:
54463 ; The following jump, skipping the XMS calls will be patched to
54464 ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54465 ; needed because the stub is installed even before the XMS driver
54466 ; is loaded if the user specifies dos=high in the config.sys
54467 00000FD0 EB03      jmp     short sr_cont
54468
54469 00000FD2 E8B000      call    EnsureA20ON      ; we must turn on A20 if OFF
54470 sr_cont:
54471 00000FD5 2EFF2E[8E0F] jmp     far [cs:DOSINTTABLE+20] ; jmp to DOS
54472
54473
54474
54475 ;
54476 ; INT 2Fh
54477 ;
54478 ;-----
54479
54480 lint2f:
54481 ; The following jump, skipping the XMS calls will be patched to
54482 ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54483 ; needed because the stub is installed even before the XMS driver
54484 ; is loaded if the user specifies dos=high in the config.sys
54485 00000FDA EB03      jmp     short int2f_cont
54486
54487 00000FDC E8A600      call    EnsureA20ON      ; we must turn on A20 if OFF
54488 int2f_cont:
54489 00000FDF 2EFF2E[920F] jmp     far [cs:DOSINTTABLE+24] ; jmp to DOS
54490
54491
54492
54493 ;
54494 ; CPM entry
54495 ;
54496 ;-----
54497
54498 lcall_entry:
54499 ; The following jump, skipping the XMS calls will be patched to
54500 ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54501 ; needed because the stub is installed even before the XMS driver
54502 ; is loaded if the user specifies dos=high in the config.sys
54503 00000FE4 EB03      jmp     short callentry_cont
54504
54505 00000FE6 E89C00      call    EnsureA20ON      ; we must turn on A20 if OFF
54506 callentry_cont:
54507 00000FE9 2EFF2E[960F] jmp     far [cs:DOSINTTABLE+28] ; jmp to DOS
54508
54509
54510
54511 ; 25/03/2024 - Retro DOS v5.0
54512 ; PC DOS 7.1 IBMDOS.COM
54513 %if 0
54514
54515 lirett:
54516     iret
54517
54518 %endif
54519
54520 ;-----
54521 ;
54522 ; LowIntXX:
54523 ;
54524 ; Interrupts from DOS that pass control to a user program must be done from
54525 ; low memory, as the user program may change the state of the A20 line or
54526 ; they may require that the A20 line be OFF. The following piece of code is
54527 ; far call'd from the following places in DOS:
54528 ;
54529 ; 1. msctrlc.asm where dos issues an int 23h (ctrlc)
54530 ; 2. msctrlc.asm where dos issues an int 24h (critical error)
54531 ; 3. msctrlc.asm where dos issues an int 28h (idle int)
54532 ;
54533 ; The int 23 and int 24 handlers may decide to do a far return instead of an
54534 ; IRET and leave the flags on the stack. Therefore we save the return address
54535 ; before doing the ints and then do a far jump back into DOS.
54536 ;
54537 ;-----
54538

```



```

54539 ; 25/03/2024 - Retro DOS v5.0
54540 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:0FEEh
54541 ; (MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:10DBh)
54542
54543 00000FEE 00000000 DosRetAddr23: dd 0
54544 00000FF2 00000000 DosRetAddr24: dd 0
54545
54546 ; 25/03/2024 - Retro DOS v5.0
54547 ; PC DOS 7.1 IBMDOS.COM
54548 %if 0
54549 DosRetAddr28: dd 0
54550 %endif
54551
54552 ; Execute int 23h from low memory
54553 LowInt23:
54554 ; save the return address that is on
54555 ; the stack
54556 00000FF6 2E8F06[EE0F] pop word [cs:DosRetAddr23]
54557 00000FFB 2E8F06[F00F] pop word [cs:DosRetAddr23+2]
54558
54559 00001000 CD23 int 23h ; ctrl C
54560 ; turn on A20 it has been turned OFF
54561 ; by int 28/23/24 handler.
54562
54563 00001002 E88000 call EnsureA20ON ; M011: we must turn on A20 if OFF
54564
54565 00001005 2EFF2E[EE0F] jmp far [cs:DosRetAddr23] ; jump back to DOS
54566
54567 ; Execute int 24h from low memory
54568 LowInt24:
54569 ; save the return address that is on
54570 ; the stack
54571
54572 0000100A 2E8F06[F20F] pop word [cs:DosRetAddr24]
54573 0000100F 2E8F06[F40F] pop word [cs:DosRetAddr24+2]
54574
54575 00001014 CD24 int 24h ; crit error
54576 ; turn on A20 it has been turned OFF
54577 ; by int 28/23/24 handler.
54578
54579 00001016 E86C00 call EnsureA20ON ; M011: we must turn on A20 if OFF
54580
54581 00001019 2EFF2E[F20F] jmp far [cs:DosRetAddr24] ; jump back to DOS
54582
54583 ; Execute int 28h from low memory
54584 LowInt28:
54585 int 28h ; idle int
54586 ; turn on A20 it has been turned OFF
54587 ; by int 28/23/24 handler.
54588
54589 00001020 E86200 call EnsureA20ON ; M011: we must turn on A20 if OFF
54590
54591 00001023 CB retf
54592
54593 ; DOSDATA:1115h (MSDOS 6.21, MSDOS.SYS)
54594
54595 ; -----
54596 ;
54597 ; int 21 ah=4b (exec) call will jump to the following label before xferring
54598 ; control to the exec'd program. We turn off A20 inorder to allow programs
54599 ; that have been packed by the faulty exepack utility to unpack correctly.
54600 ; This is so because exepac'd programs rely on address wrap.
54601 ; -----
54602
54603 ; 25/03/2024 - Retro DOS v5.0
54604 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:1024h
54605
54606 disa20_xfer:
54607 call XMMDisableA20 ; disable A20
54608
54609 00001024 E84800 ; Look at msproc.asm at label exec_go for understanding the following:
54610
54611 ; DS:SI points to entry point
54612 ; AX:DI points to initial stack
54613 ; DX has PDB pointer
54614 ; BX has initial AX value
54615
54616 cli
54617 mov byte [cs:INDOS],0 ; SS Override
54618
54619 00001027 FA
54620 00001028 2EC606[2103]00
54621
54622 ; 25/03/2024 (PC DOS 7.1 IBMDOS.COM)
54623 ;;;
54624 mov byte [ss:INDOS_FLAG],0
54625 ;;;
54626 00001034 8ED0 mov SS,AX ; set up user's stack
54627 00001036 89FC mov SP,DI ; and SP
54628 00001038 FB sti
54629
54630 00001039 1E push DS ; fake long call to entry
54631 0000103A 56 push SI
54632 0000103B 8EC2 mov ES,DX ; set up proper seg registers
54633 0000103D 8EDA mov DS,DX
54634 0000103F 89D8 mov AX,BX ; set up proper AX
54635 00001041 CB retf
54636
54637 ; -----
54638 ;
54639 ; M003:
54640 ;
54641 ; If an int 21 ah=25 call is made immediately after an exec call, DOS will
54642 ; come here, turn A20 OFF restore user stack and registers before returning
54643 ; to user. This is done in dos\msdisp.asm. This has been done to support
54644 ; programs compiled with MS PASCAL 3.2. See under TAG M003 in DOSSYM.INC for
54645 ; more info.
54646 ;
54647 ; Also at this point DS is DOSDATA. So we can assume DS DOSDATA. Note that
54648 ; SS is also DOS stack. It is important that we do the XMS call on DOS's
54649 ; stack to avoid additional stack overhead for the user.
54650 ; -----
54651
54652 disa20_iret:
54653 call XMMDisableA20
54654 00001042 E82A00 dec byte [INDOS]
54655 00001045 FE0E[2103]
54656
54657 ; 25/03/2024 (PC DOS 7.1 IBMDOS.COM)
54658 ;;;
54659 00001049 FE0E[B812] dec byte [INDOS_FLAG]
54660 ;;;
54661
54662 0000104D 8E16[B605] mov SS,[USER_SS] ; restore user stack

```

```

54663 00001051 8B26[8405]      mov     SP,[USER_SP]
54664 00001055 89E5      mov     BP,SP
54665      ;mov     [BP+user_env.user_AX],AL
54666 00001057 884600    mov     [bp],al
54667
54668      ; 25/03/2024
54669      %if 0
54670      mov     AX,[NSP]
54671      mov     [USER_SP],AX
54672      mov     AX,[NSS]
54673      mov     [USER_SS],AX
54674      %else
54675      ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54676      ;;;
54677      ;les     ax, dword ptr ds:NSS
54678 0000105A C406[F005]    les     ax,[NSS]
54679 0000105E 8C06[8405]    mov     [USER_SP],es
54680 00001062 A3[8605]      mov     [USER_SS],ax
54681      ;;;
54682      %endif
54683 00001065 58      pop     AX                ; restore user regs
54684 00001066 5B      pop     BX
54685 00001067 59      pop     CX
54686 00001068 5A      pop     DX
54687 00001069 5E      pop     SI
54688 0000106A 5F      pop     DI
54689 0000106B 5D      pop     BP
54690 0000106C 1F      pop     DS
54691 0000106D 07      pop     ES
54692      liret:                ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54693 0000106E CF      iret
54694
54695      ;*****
54696      ;***XMMDisableA20 - switch 20th address line
54697      ;
54698      ; This routine is used to disable the 20th address line in
54699      ; the system using XMM calls.
54700      ;
54701      ; ENTRY   none                ;ds = _DATA
54702      ; EXIT    A20 line disabled
54703      ; USES    NOTHING
54704      ;
54705      ;*****
54706
54707      XMMDisableA20:
54708      push     bx
54709      push     ax
54710      ;mov     ah,XMM_LOCAL_DISABLE_A20
54711 00001071 B406      mov     ah,6
54712 00001073 2EFF1E[7B10]  call    far [cs:XMMcontrol]
54713 00001078 58      pop     ax
54714 00001079 5B      pop     bx
54715 0000107A C3      retn
54716
54717      ; The entry point in the BIOS XMS driver is defined here.
54718
54719      XMMcontrol:
54720      dd      0
54721
54722      ;-----
54723      ; 24/03/2024 - Retro DOS v5.0
54724      ; PCDOS 7.1 IBMDOS.COM - DOSDATA:107Fh
54725      ;;;
54726 0000107F 00000000    dd      0
54727      SC_STATUS:
54728 00001083 0000      dw      0
54729      ;;;
54730
54731      ;-----
54732      ;
54733      ;***EnsureA20ON - Ensures that A20 is ON
54734      ;
54735      ; This routine is used to query the A20 state in
54736      ; the system using XMM calls.
54737      ;
54738      ; ENTRY: none
54739      ;
54740      ; EXIT : A20 will be ON
54741      ;
54742      ; USES : NONE
54743      ;
54744      ;-----
54745
54746      ; 19/09/2023
54747      ;LowMemory: ; label dword                ; Set equal to 0000:0080
54748      ; dw      00080h
54749      ; dw      00000h
54750      ;
54751      ;HighMemory: ; label dword
54752      ; dw      00090h                ; Set equal to FFFF:0090
54753      ; dw      0FFFFh
54754      ;
54755      ; 25/03/2024 - Retro DOS v5.0
54756      ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1085h
54757      ; (MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:116Fh)
54758
54759      EnsureA20ON:
54760      pushf
54761      push     ds
54762      push     es
54763      push     cx
54764      push     si
54765      push     di
54766
54767      ; 19/09/2023
54768      ;lds     si,[cs:LowMemory]      ; Compare the 4 words at 0000:0080
54769      ;les     di,[cs:HighMemory]    ; with the 4 at FFFF:0090
54770
54771      ; 25/03/2024
54772      ; PCDOS 7.1 IBMDOS.COM
54773      ;;;
54774 0000108B 31F6      xor     si,si
54775 0000108D 8EDE      mov     ds,si
54776 0000108F 4E      dec     si
54777 00001090 BF9000    mov     di,90h ; 0FFFFh:0090h ; HighMemory
54778 00001093 8EC6      mov     es,si
54779 00001095 BE8000    mov     si,80h ; 0000h:0080h ; LowMemory
54780      ;;;
54781
54782 00001098 B90400    mov     cx,4
54783 0000109B FC      cld
54784 0000109C F3A7      repe    cmpsw
54785
54786 0000109E 7407      jz      short EA20_OFF

```

```

54787 EA20_RET:
54788 000010A0 5F      pop     di
54789 000010A1 5E      pop     si
54790 000010A2 59      pop     cx
54791 000010A3 07      pop     es
54792 000010A4 1F      pop     ds
54793 000010A5 9D      popf
54794 000010A6 C3      retn
54795
54796 EA20_OFF:
54797 ; We are going to do the XMS call on the DOS's AuxStack.
54798 ; NOTE: ints are disabled at this point.
54799
54800 000010A7 53      push    bx
54801 000010A8 50      push    ax
54802
54803 ; 25/03/2024
54804 %if 0
54805      mov     ax,ss                ; save user's stack pointer
54806      mov     [cs:SS_Save],ax
54807      mov     [cs:SP_Save],sp
54808      mov     ax,cs
54809      mov     ss,ax
54810 %else
54811 ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54812 ;;;
54813 000010A9 8CC8      mov     ax,cs
54814 000010AB 2E8C16[9A0F]  mov     [cs:SS_Save],ss
54815 000010B0 2E8926[9C0F]  mov     [cs:SP_Save],sp
54816 000010B5 8ED0      mov     ss,ax
54817 ;;;
54818 %endif
54819 000010B7 BC[A007]      mov     sp,AUXSTACK
54820 ; ss:sp -> DOSDATA:AuxStack
54821      mov     ah,XMM_LOCAL_ENABLE_A20
54822 000010BA B405      mov     ah,5
54823 000010BC 2EFF1E[7B10]  call    far [cs:XMMcontrol]
54824 000010C1 09C0      or      ax,ax
54825 000010C3 740E      jz      short XMMerror          ; AX = 0 fatal error
54826
54827      mov     ax,[cs:SS_Save]      ; restore user stack
54828      mov     ss,ax
54829 ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54830 ;;;
54831 000010C5 2E8E16[9A0F]  mov     ss,[cs:SS_Save]
54832 ;;;
54833 000010CA 2E8B26[9C0F]  mov     sp,[cs:SP_Save]
54834
54835 000010CF 58      pop     ax
54836 000010D0 5B      pop     bx
54837
54838 000010D1 EBCD      jmp     short EA20_RET
54839
54840 XMMerror:
54841 000010D3 B40F      mov     ah,0Fh                ; M006 - Start
54842 000010D5 CD10      int     10h                  ; get video mode
54843 000010D7 3C07      cmp     al,7
54844 000010D9 7405      je      short XMMcont        ; Q: are we an MDA
54845 ;xor     ah,ah ; 0          ; Y: do not change mode
54846 ;mov     al,02h            ; set video mode
54847 ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54848 ;;;
54849 000010DB B80200      mov     ax,2
54850 ;;;
54851 000010DE CD10      int     10h
54852
54853 XMMcont:
54854      mov     ah,05h            ; set display page
54855      xor     al,al            ; page 0
54856 ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54857 000010E0 B80005      mov     ax,500h
54858 ;;;
54859 000010E3 CD10      int     10h
54860
54861 000010E5 BE[1312]      mov     si,XMMERRMSG
54862 000010E8 0E      push    cs
54863 000010E9 1F      pop     ds
54864 000010EA FC      cld                ; clear direction flag
54865
54866 000010EB AC      lodsb
54867 000010EC 3C24      cmp     al,'$'
54868 000010EE 7409      jz      short XMMstall        ; indicates end of XMMERRMSG
54869 000010F0 B40E      mov     ah,0Eh
54870 000010F2 BB0700      mov     bx,7
54871 000010F5 CD10      int     10h
54872 000010F7 EBF2      jmp     short XMMprnt
54873
54874 XMMstall:
54875 000010F9 FB      sti
54876 000010FA EBF0      jmp     short XMMstall        ; allow the user to warm boot
54877 ; M006 - End
54878
54879 ;-----
54880 ; 27/04/2019 - Retro DOS v4.0
54881
54882 ; retrodos4.s ; offset 0ch in BIOS segment (0070h)
54883 ALTAH      equ 0Ch
54884
54885 ;This has been put in for WIN386 2.XX support. The format of the instance
54886 ;table was different for this. Segments will be patched in at init time.
54887
54888 ; 25/03/2024
54889 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:10FCh
54890 ; (MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:11E7h)
54891
54892 oldInstanceJunk:
54893 000010FC 7000      dw      70h                ;segment of BIOS
54894 000010FE 0000      dw      0                  ;indicate stacks in SYSINIT area
54895 00001100 0600      dw      6                  ;6 instance items
54896
54897      dw      0,offset dosdata:contpos, 2
54898      dw      0,offset dosdata:bcon, 4
54899      dw      0,offset dosdata:carpos,106h
54900      dw      0,offset dosdata:charco, 1
54901      dw      0,offset dosdata:exec_init_sp, 34                ;M032
54902      dw      070h,offset BData:altah, 1                ; altah byte in bios
54903
54904 00001102 0000[2200]0200      dw      0,CONTPOS,2
54905 00001108 0000[3200]0400      dw      0,BCON,4
54906 0000110E 0000[F901]0601      dw      0,CARPOS,106h
54907 00001114 0000[0003]0100      dw      0,CHARCO,1
54908 0000111A 0000[B0E]2200      dw      0,exec_init_SP,34
54909 00001120 70000C000100      dw      70h,ALTAH,1                ; altah byte in bios
54910

```

```

54911 ;-----
54912 ;
54913 ; 24/03/2024
54914 ; 17/01/2024
54915 %if 0
54916 ;
54917 ; M021-
54918 ;
54919 ; DosHashMA - This flag is set by seg_reinit when the DOS actually
54920 ; takes control of the HMA. When running, this word is a reliable
54921 ; indicator that the DOS is actually using HMA. You can't just use
54922 ; CS, because ROMDOS uses HMA with CS < F000.
54923 ;
54924 DosHashMA:
54925 db 0
54926 FixExePatch:
54927 dw 0 ; M012
54928 ;
54929 ; 21/03/2024 - Retro DOS v5.0
54930 ; 28/12/2022 - Retro DOS v4.1
54931 RationalPatchPtr:
54932 dw 0 ; M012
54933 ;
54934 %endif
54935 ;
54936 ; End M021
54937 ;-----
54938 ;
54939 ; 21/03/2024 - Retro DOS v5.0
54940 %if 1
54941 ; 28/12/2022 - Retro DOS v4.1
54942 ; %if 0
54943 ;
54944 ; M020 Begin
54945 ;
54946 RatBugCode: proc far
54947 push cx
54948 00001126 51 mov cx,[10h]
54949 00001127 8B0E1000 rbc_loop:
54950 ;loop $
54951 ;loop rbc_loop
54952 0000112B E2FE pop cx
54953 0000112D 59 retf
54954 0000112E CB
54955 ;
54956 ; M020 End
54957 ;
54958 %endif
54959 ;-----
54960 ;
54961 UMBsave1:
54962 ;db 11 dup (?) ; M023
54963 times 11 db 0
54964 0000112F 00<rep Bh>
54965 ;-----
54966 ;
54967 ; 16/01/2024 - RetroDOS v5.0
54968 ; PC DOS 7.1 IBMDOS.COM - DOSDATA: 113Ah
54969 ;
54970 OLD_FIRSTCLUS_HW:
54971 0000113A 0000 dw 0
54972 ;
54973 ; 17/01/2024
54974 %if 1
54975 ;
54976 ; DOS 3.3 F.C. 6/12/86
54977 ; FASTOPEN communications area DOS 3.3 F.C. 5/29/86
54978 ;
54979 FastTable: ; a better name
54980 FastOpenTable:
54981 dw 2 ; number of entries
54982 dw FastRet ; pointer to ret instr.
54983 dw 0 ; and will be modified by
54984 dw FastRet ; FASTxxx when loaded in
54985 dw 0
54986 ;
54987 ; DOS 3.3 F.C. 6/12/86
54988 ;
54989 FastFlg: ; flags
54990 FastOpenFlg:
54991 00001146 00 db 0 ; don't change the foll: order
54992 ;
54993 %endif
54994 ;
54995 ; 24/03/2024 - Retro DOS v5.0
54996 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:1147h
54997 %if 1
54998 ;
54999 FastOpen_Ext_Info:
55000 ;db 6 dup(0)
55001 00001147 00<rep 6h> times 6 db 0
55002 UNKNOWN1:
55003 0000114D 0000 dw 0
55004 ;db 5 dup(0)
55005 0000114F 00<rep 5h> times 5 db 0
55006 PATHNAMELEN:
55007 00001154 4300 dw 67
55008 ;
55009 ;db 31 dup(0)
55010 00001156 00<rep 1Fh> times 31 db 0
55011 %endif
55012 ;
55013 ; 17/01/2024
55014 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:1175h
55015 CurSC_DRIVE:
55016 00001175 FF db -1 ; 0FFh ; current SC drive
55017 ;
55018 ;M044
55019 ; Second part of save area for saving last para of windows memory
55020 ;
55021 ; 17/01/2024
55022 WinoldPatch2:
55023 ;db 8 dup (?) ; M044
55024 00001176 00<rep 8h> times 8 db 0
55025 ;
55026 FIRST_BUFF_ADDR:
55027 0000117E 0000 dw 0
55028 ;-----
55029 ;
55030 ; 17/01/2024 - RetroDOS v5.0
55031 ; PC DOS 7.1 IBMDOS.COM
55032 ; DOSDATA:1180h
55033 ;
55034 %if 1

```

```

55035 ;=====
55036 ; WPATCH.INC (MSDOS 6.0, 1991) ;;; windows 3.1 patches ;;;
55037 ;=====
55038 ; 27/04/2019 - Retro DOS 4.0
55039 ;=====
55040 ;
55041 ; DOSDATA:12CFh (MSDOS 6.21, MSDOS.SYS)
55042 ;
55043 ; Retro DOS v5.0
55044 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:1180h
55045 ;
55046 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55047 ; DOSDATA:12CFh (MSDOS 5.0, MSDOS.SYS)
55048 ;
55049 ; first and second DOS patches
55050 ; Non-console device read/write (system calls 3Fh and 40h)
55051 ;
55052 ; Code in disk.asm, 2 locations, one for read, one for write
55053 ; DVRDLP:
55054 ; DVWRTL:
55055 ;
55056 ;
55057 ; 036h lds si,SS:[????] ; ThisSFT
55058 ; lds si,si+7 ; sf_devptr
55059 ; 0E8h call ???? <- "simulate" int28 event ; DSKSTATCHK
55060 ;
55061 00001180 36C536 DOSP1_ID: db 036h,0C5h,036h
55062 00001183 3605C57407E8 DOSP1_THISSFT: db 036h,005h,0C5h,074h,007h,0E8h
55063 DOSP1_ID_LEN equ $-DOSP1_ID
55064 ;
55065 00001189 9090 db 90h, 90h
55066 ;
55067 0000118B 36C536 DOSP12_ID: db 036h,0C5h,036h
55068 0000118E 3605C57407E8 DOSP12_THISSFT: db 036h,005h,0C5h,074h,007h,0E8h
55069 DOSP12_ID_LEN equ $-DOSP1_ID
55070 ;
55071 ; DOSDATA:12E3h
55072 ;
55073 ; Third/Fourth DOS patch - System call 3Fh (Read) from console
55074 ;
55075 ; Code in disk.asm, 1 location
55076 ; GETBUF:
55077 ;
55078 ; 051h push cx <- begin special int28 mode
55079 ; push es
55080 ; push di
55081 ; mov dx,???? ; offset dosgroup:CONBUF
55082 ; call ???? ; $STD_CON_STRING_INPUT
55083 ; pop di
55084 ; pop es
55085 ; 059h pop cx <- end special int28 mode
55086 ;
55087 00001194 510657BA DOSP3_ID: db 051h,006h,057h,0BAh
55088 00001198 2902E8 DOSP3_CONBUF: db 029h,002h,0E8h
55089 DOSP3_ID_LEN equ $-DOSP3_ID
55090 0000119B 9AE35F07 db 09Ah,0E3h,05Fh,007h ; ????, pop di, pop es
55091 0000119F 59 DOSP4_ID: db 059h ; pop cx
55092 DOSP4_ID_OFF equ (DOSP4_ID - DOSP3_ID)
55093 ;
55094 ; DOSDATA:12EFh
55095 ;
55096 ; Fifth DOS patch - System call 40h (write) to console
55097 ;
55098 ; Code in disk.asm, 1 location
55099 ;
55100 ; push cx
55101 ; WRCONLP: lodsb
55102 ; cmp al,1Ah
55103 ; jz ????
55104 ; call ???? <- "simulate" int28 event
55105 ; loop WRCONLP
55106 ; CONEOF: pop ax
55107 ;
55108 000011A0 51 DOSP5_ID: db 051h ; push cx
55109 000011A1 AC3C1A7405 db 0ACh,03Ch,01Ah,074h,005h
55110 000011A6 E8 db 0E8h ; call
55111 DOSP5_ID_LEN equ $-DOSP5_ID
55112 ;
55113 ; DOSDATA:12F6h
55114 ;
55115 ; Seventh DOS patch - System call entry, patch USER_ID with VMid for share
55116 ;
55117 ; Code in disp.asm, 1 location
55118 ;
55119 ;
55120 ; mov [SaveDS],ds
55121 ; mov [SaveBX],bx
55122 ; mov bx,cs
55123 ; mov ds,bx
55124 ; inc [indos]
55125 ; xor ax,ax
55126 ; mov [USER_ID],AX <- Patch to set USER_ID to VMID
55127 ;
55128 000011A7 2E8C1E DOSP7_ID: db 02Eh,08Ch,01Eh
55129 000011AA 7E05 DOSP7_SAVEDS: db 07Eh,05h ; mov [SaveDS],ds
55130 000011AC 2E891E db 02Eh,089h,01Eh
55131 000011AF 7C05 DOSP7_SAVEBX: db 07Ch,05h ; mov [SaveBX],bx
55132 000011B1 8CCB db 08Ch,0CBh ; mov bx,cs
55133 000011B3 8EDB db 08Eh,0DBh ; mov ds,bx
55134 000011B5 FE06 db 0FEh,006h
55135 000011B7 CF02 DOSP7_INDOS: db 0CFh,002h ; inc [indos]
55136 000011B9 33C0 db 033h,0C0h ; xor ax,ax
55137 DOSP7_ID_LEN equ $-DOSP7_ID
55138 ;
55139 ; DOSDATA:130Ah
55140 ;
55141 ; Eighth DOS patch - OWNER check in handle calls. For share, need to NOP test
55142 ;
55143 ; Code in handle.asm, 1 location in routine CheckOwner
55144 ;
55145 ;
55146 ;
55147 ; push ax
55148 ; mov ax,ss:[USER_ID] <- patch to XOR AX,AX to set zero
55149 ; cmp ax,es:[di.sf_UID] <- NOP
55150 ; pop ax
55151 ; jz ????
55152 ;
55153 000011BB 50 DOSP8_ID: db 050h ; push ax
55154 000011BC 36A1 db 036h,0A1h
55155 000011BE EA02 DOSP8_USER_ID: db 0EAh,002h ; mov ax,ss:[USER_ID]
55156 000011C0 263B45 db 026h,03Bh,045h ; cmp ax,es:[di+2F]
55157 DOSP8_ID_LEN equ $-DOSP8_ID
55158 000011C3 2F58 db 02Fh,058h ; pop ax

```

```

55159
55160 ; DOSDATA:1314h
55161
55162 ; 10th, 11th, 12th DOS patch - System call 3Fh (Read) in raw mode
55163 ;
55164 ; Take RAW read to STDIN SFT and turn it into a polling loop doing
55165 ; a yeild when a character is not ready to be read.
55166
55167 ; Code in disk.asm, 3 locations
55168 ;
55169 ; DVRDRAW:
55170 ;     PUSH     ES
55171 ;     POP      DS
55172 ; ReadRawRetry:
55173 ;     MOV      BX,DI
55174 ;     XOR      AX,AX
55175 ;     MOV      DX,AX
55176 ;     call     SETREAD
55177 ;     PUSH     DS
55178 ;     LDS      SI,[THISST]
55179 ;     call     DEVIOCALL
55180 ;     MOV      DX,DI
55181 ;     MOV      AH,86H
55182 ;     MOV      DI,[DEVCALL.REQSTAT]
55183 ;     TEST     DI,STERR
55184 ;     JZ       CRDROK
55185 ;     call     CHARHARD
55186 ;     MOV      DI,DX
55187 ;     OR       AL,AL
55188 ;     JZ       CRDROK
55189 ;     CMP      AL,3
55190 ;     JZ       CRDFERR
55191 ;     POP      DS
55192 ;     JMP      ReadRawRetry
55193
55194 ; CRDFERR:
55195 ;     POP      DI
55196 ; DEVIOFERR:
55197 ;     LES      DI,[THISST]
55198 ;     jmp      SET_ACC_ERR_DS
55199 ;
55200 ; CRDROK:
55201 ;     POP      DI
55202 ;     MOV      DI,DX
55203 ;     ADD      DI,[CALLSCNT]
55204 ;     JMP      SHORT ENDRDDEVJ3
55205
55206 000011C5 061F DOSP10_ID: db 006H,01FH
55207 DOSP10_LOC_OFFSET equ $-DOSP10_ID
55208 000011C7 8BDF DOSP10_LOC: db 08BH,0DFH
55209 DOSP10_REENT2_OFFSET equ $-DOSP10_LOC
55210 000011C9 33C08BD0E8 DOSP10_ID_LEN db 033H,0C0H,08BH,0D0H,0E8H
55211 DOSP10_REENT1_OFFSET equ $-DOSP10_ID
55212 000011CE DF0E DOSP10_REENT1_OFFSET db 0DFH,00EH
55213 DOSP10_PACKVAL_OFFSET equ $-DOSP10_ID
55214 000011D0 1E36C5363605E8AF0E DOSP10_PACKVAL_OFFSET db 01EH,036H,0C5H,036H,036H,005H,0E8H,0AFH,00EH
55215 000011D9 8BD7B486368B3E DOSP10_PACKVAL_OFFSET db 08BH,0D7H,0B4H,086H,036H,08BH,03EH
55216 DOSP11_LOC_OFFSET db 009H,003H
55217 000011E0 0903 DOSP11_LOC_OFFSET equ $-DOSP10_ID
55218 000011E2 F7C700807419E84717 DOSP11_REENT_OFFSET db 0F7H,0C7H,000H,080H,074H,019H,0E8H,047H,017H
55219 000011EB 8BFA0AC074103C0374- DOSP11_REENT_OFFSET db 08BH,0FAH,00AH,0C0H,074H,010H,03CH,003H,074H,003H
55220 000011F4 03
55221 000011F5 1FEBCF DOSP12_LOC_OFFSET db 01FH,0EBH,0CFH
55222 000011F8 5F DOSP12_LOC_OFFSET equ $-DOSP10_ID
55223 000011F9 36C43E3605E9A104 DOSP12_LOC_OFFSET db 05FH,08BH,0FAH
55224
55225 ; DOSDATA:1353h
55226
55227 ; 13th DOS patch - Actually a SYSINIT patch. Patches the stack fault code
55228 ; which prints the fatal stack fault error on DOS >= 3.20.
55229 ;
55230 ; Sets focus to current VM so user can see fatal message.
55231 ;
55232 ;
55233 ; 10: lods b
55234 ;     cmp al, '$'
55235 ;     je 11
55236 ;     mov bl, 7
55237 ;     mov ah, 0Eh
55238 ;     int 10h
55239 ;     jmp 10
55240 ; 11: jmp $
55241
55242 DOSP13_ID: db 0ACh
55243 db 03Ch,024h
55244 db 074h,008h
55245 db 0B3h,007h
55246 db 0B4h,00Eh
55247 db 0CDh,010h
55248 db 0EBh,0F3h
55249 db 0EBh,0FEh
55250 DOSP13_ID_LEN equ $-DOSP13_ID
55251
55252 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55253 ; DOSDATA:1362h (MSDOS 5.0 MSDOS.SYS)
55254
55255 ; 05/01/2024
55256 %endif ; 05/11/2022
55257
55258 ;-----
55259
55260 ; DOSDATA:122Ah
55261
55262 Mark3: ; label byte
55263
55264 ;IF2
55265 ; IF ((OFFSET MARK3) GT (OFFSET COUNTRY_CDPG) )
55266 ; %OUT !DATA CORRUPTION!MARK3 OFFSET TOO BIG. RE-ORGANIZE DATA.
55267 ; ENDIF
55268 ;ENDIF
55269
55270 ;-----
55271
55272 ; 27/04/2019 - Retro DOS v4.0
55273 ; 05/01/2024 - Retro DOS v5.0
55274
55275 ;include msdos.c12 ; XMMERRMSG
55276
55277 ; DOSDATA:12B8h (MSDOS 6.22, MSDOS.SYS) ; 17/01/2024
55278 ; DOSDATA:1213h (PCDOS 7.1, IBMDOS.COM) ; 05/01/2024
55279
55280
55281

```



```

55282
55283
55284 00001213 0D0A
55285 00001215 413230204861726477-
55285 0000121E 617265204572726F72-
55285 00001227 0D0A24
55286
55287
55288
55289
55290
55291
55292
55293
55294
55295
55296
55297
55298
55299
55300
55301
55302
55303
55304
55305
55306
55307
55308
55309
55310
55311
55312
55313
55314
55315
55316
55317
55318
55319
55320
55321
55322
55323
55324
55325
55326
55327 0000122A 0000000000000000
55328 00001232 5C434F554E5452592E-
55328 0000123B 53595300
55329
55330 0000123F 00<rep 33h>
55331
55332
55333
55334
55335
55336
55337
55338
55339
55340
55341
55342
55343 00001272 B501
55344
55345
55346 00001274 0600
55347
55348 00001276 02
55349 00001277 [FA0A]
55350 00001279 0000
55351 0000127B 04
55352 0000127C [7C0B]
55353 0000127E 0000
55354 00001280 05
55355 00001281 [280D]
55356 00001283 0000
55357 00001285 06
55358 00001286 [FE0B]
55359 00001288 0000
55360 0000128A 07
55361 0000128B [000D]
55362 0000128D 0000
55363 0000128F 01
55364 00001290 2600
55365
55366
55367
55368
55369
55370
55371
55372 00001292 0100
55373 00001294 B501
55374
55375 00001296 0000
55376 00001298 2400000000
55377 0000129D 2C00
55378 0000129F 2E00
55379 000012A1 2D00
55380 000012A3 3A00
55381 000012A5 00
55382 000012A6 02
55383 000012A7 00
55384 000012A8 [680D]
55385 000012AA 0000
55386 000012AC 2C00
55387 000012AE 000000000000000000-
55387 000012B7 00
55388
55389
55390
55391
55392
55393
55394
55395
55396
55397 000012B8 00
55398
55399
55400
55401

XMMERRMSG:
db 0Dh,0Ah
db 'A20 Hardware Error',0Dh,0Ah,'$'

;-----
; 17/01/2024 - Note: COUNTRY_CDPG must be at DOSDATA:122Ah (to 12B8h) addr
; It is fixed at 122Ah in PC DOS 7.1 IBMDOS.COM and MSDOS 5.0-6.22 MSDOS.SYS
;-----
;#####
;
; ** HACK FOR DOS 4.0 REDIR **
;
; The dos 4.x redir requires that country_cdpG is at offset 0122ah. Any new
; data variable that is to be added to DOSDATA must go in between Mark3
; COUNTRY_CDPG if it can.
;
; MARK3 SHOULD NOT BE > 122AH
;
; As of 9/6/90, this area is FULL!
;#####
;
; ORG 0122Ah
;
; DOSDATA:122Ah (MSDOS 6.21, MSDOS.SYS)
;
; 09/01/2024 - Retro DOS v5.0
; PC DOS 7.1 IBMDOS.COM - DOSDATA:122Ah
;
; 17/01/2024
; MSDOS 5.0 MSDOS.SYS - DOSDATA:122Ah
; MSDOS 6.22 MSDOS.SYS - DOSDATA:122Ah
; 09/03/2024
; Windows ME IO.SYS - DOSDATA:122Ah
;
; The following table is used for DOS 3.3
; DOS country and code page information is defined here for DOS 3.3.
; The initial value for ccDosCountry is 1 (USA).
; The initial value for ccDosCodepage is 850.
;
; country and code page information
;-----
COUNTRY_CDPG: ; label byte
db 0,0,0,0,0,0,0,0 ; reserved words
db 'COUNTRY.SYS',0 ; path name of country.sys
;db 51 dup (?)
times 51 db 0
;-----<MSKK01>-----
;
; ifdef DBCS
; ifdef JAPAN
; dw 932 ; system code page id (JAPAN)
; endif
; ifdef TAIWAN
; dw 938 ; system code page id (TAIWAN)
; endif
; ifdef KOREA
; dw 934 ; system code page id (KOREA IBM)
; endif
; else
; dw 437 ; system code page id
; endif
;-----<MSKK01>-----
;
; dw 6 ; number of entries
COUNTRY_CDPG_76: ; COUNTRY_CDPG + 76
db SetUcase ; 2 ; Ucase type
dw UCASE_TAB ; pointer to upper case table
dw 0 ; segment of pointer
db SetUcaseFile ; 4 ; Ucase file char type
dw FILE_UCASE_TAB ; pointer to file upper case table
dw 0 ; segment of pointer
db SetFileList ; 5 ; valid file chars type
dw FILE_CHAR_TAB ; pointer to valid file char tab
dw 0 ; segment of pointer
db SetCollate ; 6 ; collate type
dw COLLATE_TAB ; pointer to collate table
dw 0 ; segment of pointer
db SetDBCS ; 7 ; AN000; DBCS Ev 2/12/KK
dw DBCS_TAB ; AN000; pointer to DBCS Ev table 2/12/KK
dw 0 ; AN000; segment of pointer 2/12/KK
db SetCountryInfo ; 1 ; country info type
dw NEW_COUNTRY_SIZE ; extended country info size
;-----<MSKK01>-----
;
; ifdef DBCS
; .....
; else
; 09/01/2024 - Retro DOS v5.0
; PC DOS 7.1 IBMDOS.COM - DOSDATA:1292h
_COUNTRY_ID:
dw 1 ; USA country id
dw 437 ; USA system code page id
COUNTRY_CDPG_108: ; COUNTRY_CDPG + 108
dw 0 ; date format
db '$',0,0,0,0 ; currency symbol
db ',',0 ; thousand separator
db '.',0 ; decimal separator
db '-',0 ; date separator
db ':',0 ; time separator
db 0 ; currency format flag
db 2 ; # of digits in currency
db 0 ; time format
dw MAP_CASE ; mono case routine entry point
dw 0 ; segment of entry point
db ',',0 ; data list separator
dw 0,0,0,0,0 ; reserved
;
; endif
;-----<MSKK01>-----
;-----
; 01/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
;
; PC DOS 7.1 IBMDOS.COM - DOSDATA:12B8h
;
INDOS_FLAG:
db 0 ; duplicated INDOS flag, what for ?
; (PC DOS 7.1 kernel CODE always updates it together
; with 'INDOS' flag !?)
;
; 09/01/2024
DEVIO_IN_PROGRESS:

```

```

55402 000012B9 00          db 0
55403
55404          ; 09/01/2024
55405          ; PCDOS 7.1 IBMDOS.COM - DOSDATA:12BAh
55406          ; -----
55407          ; INTERNATIONALIZATION INFORMATION
55408          ;   for Get/Set Extended Country Info functions
55409
55410 000012BA 454E5500      _ENU:      db 'ENU',0
55411 000012BE 55534100      _USA:      db 'USA',0
55412 000012C2 5553          _US: db 'US'
55413 000012C4 0100          dw 1
55414 000012C6 0200          dw 2
55415 000012C8 0000          dw 0
55416 000012CA 414D00        _AM: db 'AM',0
55417 000012CD 504D00        _PM: db 'PM',0
55418
55419 000012D0 4D2F642F7979202020- _MMDDYY:
55419 000012D9 2020646464642C4D4D- db 'M/d/yy      dddd,MMMdd,yyyy      '
55419 000012E2 4D4D64642C79797979-
55419 000012EB 2020202020202020
55420 000012F4 00          db 0
55421 000012F5 00          db 0
55422 000012F6 0000          dw 0
55423
55424          ; 09/01/2024
55425          ; PCDOS 7.1 IBMDOS.COM - DOSDATA:12F8h
55426
55427 VxDpath:
55428 000012F8 633A5C77696E613230- db 'c:\wina20.386',0
55428 00001301 2E33383600
55429 00001306 0000          dw 0
55430
55431          ; 09/01/2024
55432          ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1308h
55433
55434 drive_flags:
55435 00001308 00<rep 1Ah>    times 26 db 0
55436
55437 DriverLoad:
55438 00001322 01          db 1
55439 BiosDataPtr:      ; 08/03/2024
55440 00001323 0000<rep 2h>    times 2 dw 0
55441 00001327 00<rep 5h>    times 5 db 0
55442 0000132C 0400          dw 4
55443 0000132E [B812]        dw INDOS_FLAG
55444 00001330 [0813]        dw drive_flags
55445 00001332 [3613]        dw NLS_YES      ; "YN"
55446 00001334 [3A13]        dw unknown_zero_dd
55447
55448 00001336 59          NLS_YES: db 'Y'
55449 00001337 4E          NLS_NO:  db 'N'
55450
55451 00001338 79          NLS_yes2: db 'y'
55452
55453 00001339 6E          NLS_no2:  db 'n'
55454
55455 unknown_zero_dd:
55456 0000133A 00000000      dd 0
55457
55458          ; -----
55459
55460          ; 27/04/2019 - Retro DOS v4.0
55461
55462 ;include msdos.c12          ; XMMERRMSG
55463
55464          ; DOSDATA:12B8h (MSDOS 6.21, MSDOS.SYS)
55465
55466 ;XMMERRMSG:
55467          ; db      0Dh,0Ah
55468          ; db      'A20 Hardware Error',0Dh,0Ah,'$'
55469
55470          ; DOSDATA ends
55471
55472          ; 05/11/2022
55473          ; -----
55474          ; End of MSDOS 5.0 MSDOS.SYS /// Retro DOS v4.0 (2022) - 05/11/2022
55475          ; -----
55476
55477          ; 28/12/2022 - Retro DOS v4.1
55478          ; (windows 3.1 and Rational Extender patches are removed/disabled)
55479          ; (windows 3.1 does not use the patches below if DOS version is MSDOS 5.0)
55480          ; -----
55481 %if 0
55482
55483          ; -----
55484          ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55485
55486          ; =====
55487          ; WPATCH.INC (MSDOS 6.0, 1991)   ;; windows 3.1 patches ;;
55488          ; =====
55489          ; 27/04/2019 - Retro DOS 4.0
55490
55491 ;DOSDATA Segment
55492
55493          ; DOSDATA:12CFh (MSDOS 6.21, MSDOS.SYS)
55494
55495          ; Retro DOS v5.0
55496          ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1180h
55497
55498          ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55499          ; DOSDATA:12CFh (MSDOS 5.0, MSDOS.SYS)
55500
55501          ; first and second DOS patches
55502          ;   Non-console device read/write (system calls 3Fh and 40h)
55503          ;
55504          ; Code in disk.asm, 2 locations, one for read, one for write
55505          ;   DVRDLP:
55506          ;   DVWRTL:
55507          ;
55508          ;
55509          ; 036h      lds      si,SS:[????]          ; ThisSFT
55510          ;      lds      si,si+7          ; sf_devptr
55511          ; 0E8h      call    ????          <- "simulate" int28 event ; DSKSTATCHK
55512
55513 DOSP1_ID:  db      036h,0C5h,036h
55514 DOSP1_THISSFT: db      036h,005h,0C5h,074h,007h,0E8h
55515 DOSP1_ID_LEN equ     $-DOSP1_ID
55516
55517          db      90h, 90h
55518
55519 DOSP12_ID: db      036h,0C5h,036h
55520 DOSP12_THISSFT: db      036h,005h,0C5h,074h,007h,0E8h
55521 DOSP12_ID_LEN equ     $-DOSP1_ID

```



```

55522 ; DOSDATA:12E3h
55523 ;
55524 ; Third/Fourth DOS patch - System call 3Fh (Read) from console
55525 ;
55526 ; Code in disk.asm, 1 location
55527 ; GETBUF:
55528 ;
55529 ; 051h    push    cx        <- begin special int28 mode
55530 ; push    es
55531 ; push    di
55532 ; mov     dx,???? ; offset dosgroup:CONBUF
55533 ; call    ????    ; $STD_CON_STRING_INPUT
55534 ; pop     di
55535 ; pop     es
55536 ; 059h    pop     cx        <- end special int28 mode
55537 ;
55538 DOSP3_ID: db      051h,006h,057h,08Ah
55539 DOSP3_CONBUF: db    029h,002h,0E8h
55540 DOSP3_ID_LEN equ    $-DOSP3_ID
55541 db      09Ah,0E3h,05Fh,007h ; ???? , pop di, pop es
55542 DOSP4_ID: db      059h ; pop cx
55543 DOSP4_ID_OFF equ    (DOSP4_ID - DOSP3_ID)
55544 ;
55545 ; DOSDATA:12EFh
55546 ;
55547 ; Fifth DOS patch - System call 40h (Write) to console
55548 ;
55549 ; Code in disk.asm, 1 location
55550 ;
55551 ;
55552 ; push    cx
55553 ; WRCONLP: lodsb
55554 ; cmp     al,1Ah
55555 ; jz      ????
55556 ; call    ????    <- "simulate" int28 event
55557 ; loop    WRCONLP
55558 ; CONEOF: pop     ax
55559 ;
55560 DOSP5_ID: db      051h ; push cx
55561 db      0ACh,03Ch,01Ah,074h,005h
55562 db      0E8h ; call
55563 DOSP5_ID_LEN equ    $-DOSP5_ID
55564 ;
55565 ; DOSDATA:12F6h
55566 ;
55567 ; Seventh DOS patch - System call entry, patch USER_ID with VMid for share
55568 ;
55569 ; Code in disp.asm, 1 location
55570 ;
55571 ;
55572 ; mov     [SaveDS],ds
55573 ; mov     [SaveBX],bx
55574 ; mov     bx,cs
55575 ; mov     ds,bx
55576 ; inc     [indos]
55577 ; xor     ax,ax
55578 ; mov     [USER_ID],AX    <- Patch to set USER_ID to VMID
55579 ;
55580 DOSP7_ID: db      02Eh,08Ch,01Eh
55581 DOSP7_SAVEDS: db    07Eh,05h ; mov [SaveDS],ds
55582 db      02Eh,089h,01Eh
55583 DOSP7_SAVEBX: db    07Ch,05h ; mov [SaveBX],bx
55584 db      08Ch,0CBh ; mov bx,cs
55585 db      08Eh,0DBh ; mov ds,bx
55586 db      0FEh,006h
55587 DOSP7_INDOS: db    0CFh,002h ; inc [indos]
55588 db      033h,0C0h ; xor ax,ax
55589 DOSP7_ID_LEN equ    $-DOSP7_ID
55590 ;
55591 ; DOSDATA:130Ah
55592 ;
55593 ; Eighth DOS patch - OWNER check in handle calls. For share, need to NOP test
55594 ;
55595 ; Code in handle.asm, 1 location in routine CheckOwner
55596 ;
55597 ;
55598 ;
55599 ; push    ax
55600 ; mov     ax,ss:[USER_ID]    <- patch to XOR AX,AX to set zero
55601 ; cmp     ax,es:[di.sf_UID]  <- NOP
55602 ; pop     ax
55603 ; jz      ????
55604 ;
55605 DOSP8_ID: db      050h ; push ax
55606 db      036h,0A1h
55607 DOSP8_USER_ID: db    0EAh,002h ; mov ax,ss:[USER_ID]
55608 db      026h,03Bh,045h ; cmp ax,es:[di+2F]
55609 DOSP8_ID_LEN equ    $-DOSP8_ID
55610 db      02Fh,058h ; pop ax
55611 ;
55612 ; DOSDATA:1314h
55613 ;
55614 ; 10th, 11th, 12th DOS patch - System call 3Fh (Read) in raw mode
55615 ;
55616 ; Take RAW read to STDIN SFT and turn it into a polling loop doing
55617 ; a yeild when a character is not ready to be read.
55618 ;
55619 ; Code in disk.asm, 3 locations
55620 ;
55621 ; DVRDRAW:
55622 ; PUSH    ES
55623 ; POP     DS
55624 ; ReadRawRetry: <- Patch 10
55625 ; MOV     BX,DI
55626 ; XOR     AX,AX    <- Reenter #2
55627 ; MOV     DX,AX
55628 ; call    SETREAD
55629 ; PUSH    DS    <- Reenter #1
55630 ; LDS     SI,[THISFT]
55631 ; call    DEVIOCALL
55632 ; MOV     DX,DI
55633 ; MOV     AH,86h
55634 ; MOV     DI,[DEVCALL.REQSTAT]
55635 ; TEST    DI,STERR
55636 ; JZ      CRDROK
55637 ; call    CHARHARD
55638 ; MOV     DI,DX
55639 ; OR      AL,AL
55640 ; JZ      CRDROK
55641 ; CMP     AL,3
55642 ; JZ      CRDFERR
55643 ; POP     DS
55644 ; JMP     ReadRawRetry
55645 ;

```

```

55646 ; CRDFERR:
55647 ; POP DI <- Patch 11
55648 ; DEVIOFERR:
55649 ; LES DI,[THISSFT]
55650 ; jmp SET_ACC_ERR_DS
55651 ;
55652 ; CRDRK:
55653 ; POP DI <- Patch 12
55654 ; MOV DI,DX
55655 ; ADD DI,[CALLSCNT]
55656 ; JMP SHORT ENDRDDEVJ3
55657
55658 DOSP10_ID: db 006H,01FH
55659 DOSP10_LOC_OFFSET equ $-DOSP10_ID
55660 DOSP10_LOC: db 08BH,0DFH
55661 DOSP10_REENT2_OFFSET equ $-DOSP10_LOC
55662 db 033H,0C0H,08BH,0D0H,0E8H
55663 DOSP10_ID_LEN equ $-DOSP10_ID
55664 db 0DFH,00EH
55665 DOSP10_REENT1_OFFSET equ $-DOSP10_LOC
55666 db 01EH,036H,0C5H,036H,036H,005H,0E8H,0AFH,00EH
55667 db 08BH,0D7H,0B4H,086H,036H,08BH,03EH
55668 DOSP10_PACKVAL_OFFSET equ $-DOSP10_ID
55669 db 009H,003H
55670 db 0F7H,0C7H,000H,080H,074H,019H,0E8H,047H,017H
55671 db 08BH,0FAH,00AH,0C0H,074H,010H,03CH,003H,074H,003H
55672 db 01FH,0EBH,0CFH
55673 DOSP11_LOC_OFFSET equ $-DOSP10_ID
55674 db 05FH
55675 DOSP11_REENT_OFFSET equ $-DOSP10_LOC
55676 db 036H,0C4H,03EH,036H,005H,0E9H,0A1H,004H
55677
55678 DOSP12_LOC_OFFSET equ $-DOSP10_ID
55679 db 05FH,08BH,0FAH
55680 ; DOSDATA:1353h
55681
55682 ; 13th DOS patch - Actually a SYSINIT patch. Patches the stack fault code
55683 ; which prints the fatal stack fault error on DOS >= 3.20.
55684 ;
55685 ; Sets focus to current VM so user can see fatal message.
55686 ;
55687 ;
55688 ; 10: lods b <- Setfocus here
55689 ; cmp al, '$'
55690 ; je 11
55691 ; mov bl, 7
55692 ; mov ah, 0Eh
55693 ; int 10h
55694 ; jmp 10
55695 ; 11: jmp $
55696
55697 DOSP13_ID: db 0ACh ; 10: lods b
55698 db 03Ch,024h ; cmp al, '$'
55699 db 074h,008h ; je 11
55700 db 0B3h,007h ; mov bl, 7
55701 db 0B4h,00Eh ; mov ah, 0Eh
55702 db 0CDh,010h ; int 10h
55703 db 0EBh,0F3h ; jmp 10
55704 db 0EBh,0FEh ; 11: jmp $
55705 DOSP13_ID_LEN equ $-DOSP13_ID
55706
55707 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55708 ; DOSDATA:1362h (MSDOS 5.0 MSDOS.SYS)
55709
55710 ; 06/12/2022
55711 ; DOSDATASIZE equ $ - DOSDATASTART ; 4962 bytes (1362h)
55712
55713 ; DOSDATA ends
55714
55715 ; 05/01/2024
55716 %endif ; 05/11/2022
55717
55718 ;=====
55719 ; MPATCH.ASM (MSDOS 6.0, 1993)
55720 ;=====
55721 ; 27/04/2019 - Retro DOS 4.0
55722
55723 ; mpatch.asm -- holds data patch location for callouts
55724 ; -- allocate cluster in rom.asm
55725 ;
55726 ; This area is pointed to by OffsetMagicPatch[609h] in fixed DOS data.
55727 ; Currently, this location is used only by magicdrv.sys's patch to
55728 ; cluster allocation, however it can be expanded to be used by other
55729 ; patches. This is important since we have an easy-access pointer to
55730 ; this location in OffsetMagicPatch. Magicdrv.sys is guaranteed to
55731 ; only patch out a far call/retf, so any space after that could be
55732 ; used as a patch by using OffsetMagicPatch+6. See rom.asm on how
55733 ; to call out here.
55734 ;
55735 ; Currently, we allocate only the minimum space required for the 6
55736 ; byte magicdrv patch, so if you change the dos data, you may want
55737 ; to reserve space here if your new data will be position dependent
55738 ; and would prohibit growing of this table.
55739 ;
55740 ; history - created 8-7-92 by scottq
55741 ; - added Rational386PatchPtr 2-1-93 by jimmat
55742 ;
55743 ; Exported Functions
55744 ;=====
55745 ; MagicPatch - callout patched by magidrv.sys for cluster allocations
55746
55747 ; DosData Segment
55748
55749 ; DOSDATA:1362h (MSDOS 6.21, MSDOS.SYS)
55750
55751 ; -----
55752
55753 ; Rational386PatchPtr points to either a RET instruction (80286 or less) or
55754 ; a routine to fix buggy versions of the Rational DOS Extender (80386 or
55755 ; greater). Added to this file because it needed to be somewhere and is
55756 ; 'patch' related.
55757
55758 Rational386PatchPtr:
55759 dw 0 ; points to patch routine or RET instr.
55760 ; -----
55761
55762 ; 25/03/2024
55763 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1340h
55764 ; (Windows ME IO.SYS - DOSDATA:133Eh)
55765
55766 MagicPatch:
55767 ; MagicPatch proc far
55768 retf ; default is to just return to allocate
55769 nop ; however, this code will be patched

```

```

55770 00001342 90          nop          ;by magicdrv.sys to
55771 00001343 90          nop          ; call far ?:?
55772 00001344 90          nop          ; retf or perhaps just jmp far
55773 00001345 90          nop          ;retf/nop take one byte, so we need six instructions
55774                                     ;for 6 byte patch
55775 ;MagicPatch endp
55776
55777 ; -----
55778
55779 ;DosData Ends
55780
55781 ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:136Ah
55782 ;
55783 ; 25/03/2024 - Retro DOS v5.0
55784 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:1346h
55785
55786 ;; windows ME IO.SYS
55787 ;; db 12 dup(0)
55788
55789 ; (windows ME IO.SYS - DOSDATA:1344h)
55790
55791 ;-----
55792
55793 ;DOSDATA LAST SEGMENT
55794
55795 ; 29/04/2019 - Retro DOS v4.0
55796
55797 ;-----
55798 ; 25/05/2019 - Retro DOS v4.0 Modification (paragraph alignment)
55799
55800 ;db 0,1,12,64,19,0 ; ! Magic numbers !
55801
55802 ;align 16
55803
55804 ; !!! DOSDATA:1370h ; Retro DOS v4.0 only!
55805
55806 ;-----
55807
55808 ; 05/01/2024
55809 ;%endif ; 05/11/2022
55810
55811 ; 05/12/2022
55812 ;MSDAT001E: ; label byte
55813
55814 ; 22/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
55815 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55816 DOSDATAEND equ $
55817 DOSDATASIZE equ DOSDATAEND - DOSDATASTART ; = 4962 for MSDOS 5.0 MSDOS.SYS
55818 ; = 4970 for MSDOS 6.22 MSDOS.SYS
55819 MSDAT001E equ DOSDATAEND - DOSDATASTART ; = 4934 for PC DOS 7.1 IBMDOS.COM
55820 ; (= 4944 for windows ME IO.SYS)
55821 ;DOSDATA LAST ENDS
55822
55823 ; Retro DOS v4.0 by Erdogan Tan (Redevelopment of MSDOS 5.0 KERNEL via NASM)
55824 ; DECEMBER 2022, ISTANBUL - TURKIYE.
55825 ;=====
55826 ; END
55827 ;=====
55828 ; Retro DOS v4.0 by Erdogan Tan (Redevelopment of MSDOS 6.21 KERNEL via NASM)
55829 ;=====
55830 ; MAY 2019, ISTANBUL - TURKIYE.
55831 ;=====
55832 ; 25/03/2024 - RETRO DOS v5.0 KERNEL code is READY! (Start: 01/01/2024)

```